

1a

Begrüßung, Organisatorisches, Inhaltsübersicht und Grundlagen

Prof. Dr. Jasmin Blanchette

Lehr- und Forschungseinheit für
Theoretische Informatik und Theorembeweisen

Stand: 10. Mai 2024

Basiert auf Folien von PD Dr. David Sabel



Dozent

- ▶ Prof. Dr. Jasmin Blanchette
jasmin.blanchette@ifi.lmu.de

Wissenschaftliche Mitarbeiter

- ▶ Jannis Limperg
jannis.limperg@ifi.lmu.de
- ▶ Luca Maio
luca.maio@ifi.lmu.de

Tutorinnen und Tutoren

- ▶ Julia Doktor, Luis Gambarte,
Massin Guerdi, Paul Köhler,
Elisabeth Lempa, Jannis Limperg,
Luca Maio und Korbinian Meier

Korrektorinnen und Korrektoren

- ▶ Karl Deck, Julia Doktor,
Jakob Friedrich, Paul Köhler,
Matthias Königer,
Magdalena Mansfeld,
Korbinian Meier und Zhiyang Song

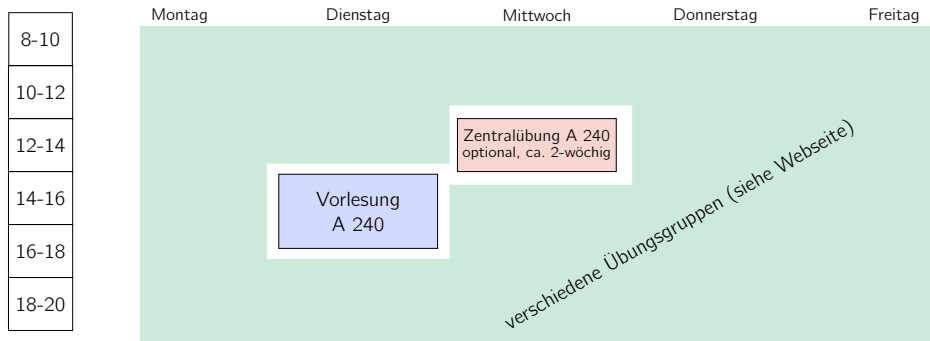
Formale Sprachen und Komplexität (FSK)

- ▶ Studierende der Informatik
- ▶ Studierende der Bioinformatik
- ▶ Studierende im Lehramt
- ▶ Studierende im Nebenfach Informatik

Theoretische Informatik für Studierende der Medieninformatik (TIMI)

- ▶ Studierende der Medieninformatik

Struktur der Veranstaltung



- ▶ **Vorlesung:** FSK 3V, TIMI 2V (integriert, Plan auf Webseite)
- ▶ **Zentralübung:** Zusatzangebot, Fragestunde und Beispiele (Plan auf Webseite)
- ▶ **Übungen:** in Präsenz; Besprechung der Hausaufgaben; FSK: 2Ü, TIMI: 1Ü

Webseiten

www.tcs.ifi.lmu.de/lehre/ss-2024/fsk_de.html (FSK)

www.tcs.ifi.lmu.de/lehre/ss-2024/timi_de.html (TIMI)

Moodle

moodle.lmu.de/course/view.php?id=32444 (FSK)

moodle.lmu.de/course/view.php?id=32603 (TIMI)

Anmeldung ist notwendig für Abgabe und Korrektur der [Hausaufgaben](#).

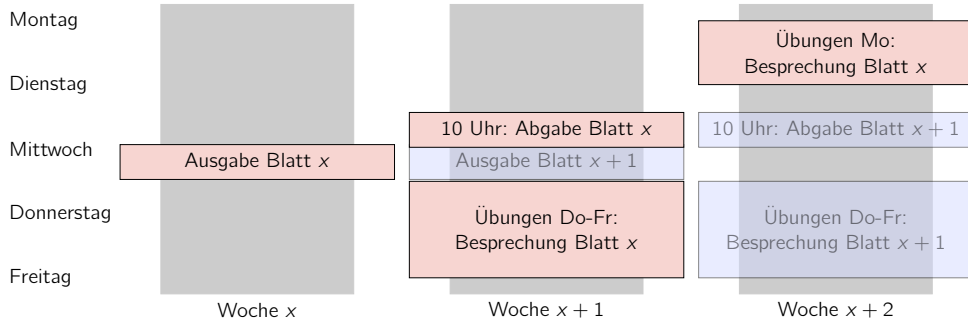
Zulip-Chat

Server-Adresse: chat.ifi.lmu.de

Stream: [TCS-24S-FSK-TIMI](#)

[Fragen und Kommentare](#) am besten dort stellen.

Hausaufgaben



- ▶ Die erste Übung ist zum Kennenlernen und zur Besprechung von Übungsblatt 0 (ohne Abgabe) da.
- ▶ Sie wählen Ihre Übungsgruppe selbst.
- ▶ Abgabe und Korrektur der Übungsblätter erfolgt elektronisch über Moodle.

- ▶ Ausgewählte Hausaufgaben werden bepunktet.
- ▶ Für jede Lösung zu einer bepunkteten Aufgabe gibt es 0 oder 1 oder 2 Punkte.
- ▶ Diese Punkte dienen als Feedback und zählen **nicht** zu einem Notenbonus.

- ▶ Die Bearbeitungszeit der Präsenzklausur beträgt 120 Minuten.
- ▶ Anmeldung zur Prüfung wird noch freigeschaltet.
- ▶ Teilnahme an der Nachholklausur ist auch ohne Teilnahme an der Erstklausur möglich.
- ▶ Die Erstklausur wird am 5. August 2024 ab 15:00 Uhr stattfinden.
- ▶ Das Datum der Nachholklausur ist noch nicht bekannt.

Auf der Webseite verfügbar

- ▶ Vorlesungsfolien
- ▶ Vorlesungsskript
 - ▶ Die nicht TIMI-relevanten Teile sind mit ★ markiert.
 - ▶ Kapitel 2 (Grundlagen) ist zum Teil dem Selbststudium überlassen.
- ▶ Übungsblätter

Wesentliche Quelle

- ▶ Uwe Schöning: Theoretische Informatik – kurz gefasst, 5. Auflage, Spektrum Akademischer Verlag, 2008
Zum Teil zu kurz gefasst

Weitere Literatur

- ▶ Alexander Asteroth und Christel Baier: Theoretische Informatik, Pearson, 2002.
Aufbau in anderer Reihenfolge, Zugriff über UB
- ▶ John E. Hopcroft, Rajeev Motwani und Jeffrey D. Ullman: Introduction to Automata Theory, Languages, and Computation, 3. Auflage, Pearson, 2006
Der Klassiker, umfangreich, Erstauflage 1979
- ▶ Ingo Wegener: Theoretische Informatik – eine algorithmenorientierte Einführung, 3. Auflage, Teubner Verlag, 2005.
Algorithmen stehen im Vordergrund, Zugriff über UB

Ziel der Veranstaltung

Ziel ist die Vermittlung von:

Theorie

- ▶ Die Theorie sagt uns, was **Computer (wie schnell) können** und was nicht.
- ▶ Viele Konzepte haben **praktische Anwendungen**.
- ▶ Die Theorie an sich ist zum Teil **sehr schön**.

Fähigkeiten

- ▶ Sie werden lernen, mit **abstrakten Konzepten** umzugehen.
- ▶ Sie werden lernen, **sorgfältig und präzise** zu arbeiten.
- ▶ Sie werden Fähigkeiten zur **Beweisführung** entwickeln.

Inhalte der Veranstaltung

Drei große Themen der Theoretischen Informatik:

1. Formale Sprachen und Automatentheorie

Wie stellt man Entscheidungsprobleme formal dar?

Insbesondere: Wie kann man Programmiersprachen u.Ä. erkennen?

2. Berechenbarkeitstheorie

Welche Probleme kann man algorithmisch

(bzw. mit dem Computer) überhaupt lösen?

3. Komplexitätstheorie

Welche Probleme kann man in annehmbarer Zeit lösen?

Thema: Formale Sprachen und Automatentheorie

1. Formale Sprachen und Automatentheorie

Wie stellt man Entscheidungsprobleme formal dar?

Insbesondere: Wie kann man Programmiersprachen u.Ä. erkennen?

Schlüsselkonzepte:

- ▶ Formale Sprachen und Entscheidungsprobleme
- ▶ Reguläre Ausdrücke (z.B. für Lexer)
- ▶ Grammatiken (z.B. für Parser)
- ▶ Automaten

2. Berechenbarkeitstheorie

*Welche Probleme kann man algorithmisch
(bzw. mit dem Computer) überhaupt lösen?*

Schlüsselkonzepte:

- ▶ Intuitive Berechenbarkeit
- ▶ Turingberechenbarkeit
- ▶ Imperative Programme (LOOP, WHILE, GOTO)
- ▶ Rekursive Funktionen
- ▶ Unentscheidbarkeit

3. Komplexitätstheorie

Welche Probleme kann man in annehmbarer Zeit lösen?

Schlüsselkonzepte:

- ▶ Die Klassen $\mathcal{P}$ und $\mathcal{NP}$
- ▶ $\mathcal{NP}$ -Schwere, $\mathcal{NP}$ -Vollständigkeit
- ▶ Konkrete $\mathcal{NP}$ -vollständige Probleme

Definition

Ein **Alphabet** Σ ist eine endliche nicht leere Menge von **Zeichen** (oder **Symbolen**).

Z.B. $\Sigma = \{a, b, c, d, e\}$.

Definition

Ein **Alphabet** Σ ist eine endliche nicht leere Menge von **Zeichen** (oder **Symbolen**).

Z.B. $\Sigma = \{a, b, c, d, e\}$.

Definition

Ein **Wort** w über Σ ist eine endliche Folge von Zeichen aus Σ .

Beispiele:

- ▶ *bade* ist ein Wort über $\{a, b, c, d, e\}$.
- ▶ *baden* ist **kein** Wort über $\{a, b, c, d, e\}$.

Weitere Notationen zu Wörtern

- ▶ Das leere Wort wird als ε notiert.
- ▶ Für $w = a_1 \cdots a_n$ ist $|w| = n$ die Länge des Wortes.
- ▶ Für $1 \leq i \leq |w|$ ist $w[i]$ das Zeichen an der i -ten Position in w .
- ▶ Für $a \in \Sigma$ und w ein Wort über Σ sei $\#_a(w) \in \mathbb{N}$ die Anzahl an Vorkommen des Zeichens a im Wort w .

Weitere Notationen zu Wörtern

- ▶ Das leere Wort wird als ε notiert.
- ▶ Für $w = a_1 \cdots a_n$ ist $|w| = n$ die Länge des Wortes.
- ▶ Für $1 \leq i \leq |w|$ ist $w[i]$ das Zeichen an der i -ten Position in w .
- ▶ Für $a \in \Sigma$ und w ein Wort über Σ sei $\#_a(w) \in \mathbb{N}$ die Anzahl an Vorkommen des Zeichens a im Wort w .

Beispiele:

- ▶ Es gilt $|\varepsilon| = 0$ und $\#_a(\varepsilon) = 0$ für alle $a \in \Sigma$.
- ▶ Für $\Sigma = \{a, b, c\}$ ist
 - ▶ $|abbccc| = 6$
 - ▶ $|aabbccc| = 8$
 - ▶ $\#_a(abbccc) = 1$
 - ▶ $\#_c(aabbccc) = 3$.
- ▶ Für $w = abbbcd$ ist $w[1] = a$, $w[5] = c$ und $w[7]$ undefiniert.

Konkatenation und Kleene-Stern

Definition

Das Wort $u \cdot v$ (alternativ uv) entsteht, indem Wort v hinten an Wort u angehängt wird.

Konkatenation und Kleene-Stern

Definition

Das Wort $u \cdot v$ (alternativ uv) entsteht, indem Wort v hinten an Wort u angehängt wird.

Die Konkatenation hilft folgende Mengen von Wörtern über Σ zu definieren:

Definition

Sei Σ ein Alphabet, dann definieren wir:

$$\begin{aligned}\Sigma^0 &:= \{\varepsilon\} \\ \Sigma^i &:= \{aw \mid a \in \Sigma, w \in \Sigma^{i-1}\} \text{ für } i > 0 \\ \Sigma^* &:= \bigcup_{i \in \mathbb{N}} \Sigma^i \\ \Sigma^+ &:= \bigcup_{i \in \mathbb{N}_{>0}} \Sigma^i\end{aligned}$$

Beachte: $\mathbb{N} = \{0, 1, 2, \dots\}$ und $\mathbb{N}_{>0} = \{1, 2, \dots\}$.

Beispiele für Konkatenation und Kleene-Stern

Sei $\Sigma = \{a, b\}$.

Dann ist

Beispiele für Konkatenation und Kleene-Stern

Sei $\Sigma = \{a, b\}$.

Dann ist

$$\Sigma^0 = \{\varepsilon\},$$

Beispiele für Konkatination und Kleene-Stern

Sei $\Sigma = \{a, b\}$.

Dann ist

$$\Sigma^0 = \{\varepsilon\},$$

$$\Sigma^1 = \Sigma = \{a, b\},$$

Beispiele für Konkatenation und Kleene-Stern

Sei $\Sigma = \{a, b\}$.

Dann ist

$$\Sigma^0 = \{\varepsilon\},$$

$$\Sigma^1 = \Sigma = \{a, b\},$$

$$\Sigma^2 = \{aa, ab, ba, bb\},$$

Beispiele für Konkatination und Kleene-Stern

Sei $\Sigma = \{a, b\}$.

Dann ist

$$\Sigma^0 = \{\varepsilon\},$$

$$\Sigma^1 = \Sigma = \{a, b\},$$

$$\Sigma^2 = \{aa, ab, ba, bb\},$$

$$\Sigma^3 = \{xw \mid x \in \{a, b\}, w \in \Sigma^2\} = \{aaa, aab, aba, abb, baa, bab, bba, bbb\}$$

Beispiele für Konkatination und Kleene-Stern

Sei $\Sigma = \{a, b\}$.

Dann ist

$$\Sigma^0 = \{\varepsilon\},$$

$$\Sigma^1 = \Sigma = \{a, b\},$$

$$\Sigma^2 = \{aa, ab, ba, bb\},$$

$$\Sigma^3 = \{xw \mid x \in \{a, b\}, w \in \Sigma^2\} = \{aaa, aab, aba, abb, baa, bab, bba, bbb\}$$

$\vdots$

und

$$\Sigma^* = \{\varepsilon, a, b, aa, ab, ba, bb, aaa, aab, aba, abb, baa, bab, bba, bbb, aaaa, \dots\}.$$

Weitere Notationen und Begriffe

Sei w ein Wort über Σ .

- ▶ w^m entsteht aus m -maligen Konkatenieren von w , d.h.

$$w^0 = \varepsilon \text{ und } w^m = ww^{m-1} \text{ für } m > 0$$

- ▶ $\overline{w}$ ist das rückwärts gelesene Wort w , d.h.

$$\overline{\varepsilon} = \varepsilon \text{ und für } w = a_1 \cdots a_n \text{ ist } \overline{w} = a_n \cdots a_1$$

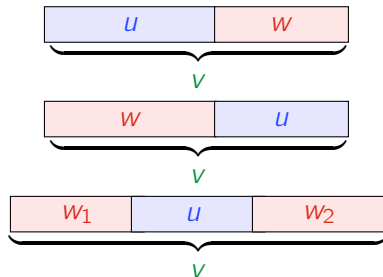
- ▶ w ist ein Palindrom g.d.w. $w = \overline{w}$.

Beispiele für Palindrome: anna, reliefpfeiler, lagerregal, annasusanna.

Sprechweisen: Präfix, Suffix, Teilwort

Seien u, v Wörter über einem Alphabet Σ .

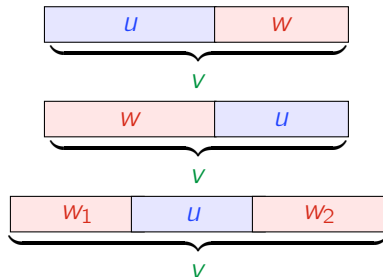
- ▶ u ist ein **Präfix** von v , wenn es ein Wort w gibt mit $uw = v$.
- ▶ u ist ein **Suffix** von v , wenn es ein Wort w gibt mit $wu = v$.
- ▶ u ist ein **Teilwort** von v , wenn es Wörter w_1, w_2 gibt mit $w_1uw_2 = v$.



Sprechweisen: Präfix, Suffix, Teilwort

Seien u, v Wörter über einem Alphabet Σ .

- ▶ u ist ein **Präfix** von v , wenn es ein Wort w gibt mit $uw = v$.
- ▶ u ist ein **Suffix** von v , wenn es ein Wort w gibt mit $wu = v$.
- ▶ u ist ein **Teilwort** von v , wenn es Wörter w_1, w_2 gibt mit $w_1uw_2 = v$.

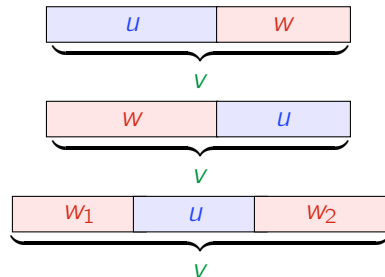


Beispiel: Sei $w = ababbaba$.

Sprechweisen: Präfix, Suffix, Teilwort

Seien u, v Wörter über einem Alphabet Σ .

- ▶ u ist ein **Präfix** von v , wenn es ein Wort w gibt mit $uw = v$.
- ▶ u ist ein **Suffix** von v , wenn es ein Wort w gibt mit $wu = v$.
- ▶ u ist ein **Teilwort** von v , wenn es Wörter w_1, w_2 gibt mit $w_1uw_2 = v$.



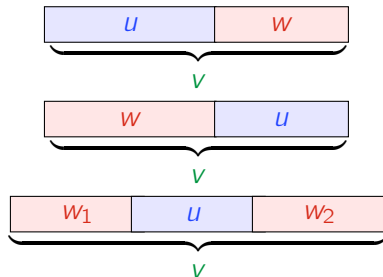
Beispiel: Sei $w = ababbaba$.

- ▶ w ist ein Präfix, Suffix und Teilwort von w .

Sprechweisen: Präfix, Suffix, Teilwort

Seien u, v Wörter über einem Alphabet Σ .

- ▶ u ist ein **Präfix** von v , wenn es ein Wort w gibt mit $uw = v$.
- ▶ u ist ein **Suffix** von v , wenn es ein Wort w gibt mit $wu = v$.
- ▶ u ist ein **Teilwort** von v , wenn es Wörter w_1, w_2 gibt mit $w_1uw_2 = v$.



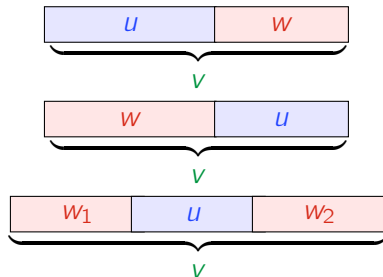
Beispiel: Sei $w = ababbaba$.

- ▶ w ist ein Präfix, Suffix und Teilwort von w .
- ▶ aba ist ein Präfix, Suffix und Teilwort von w .

Sprechweisen: Präfix, Suffix, Teilwort

Seien u, v Wörter über einem Alphabet Σ .

- ▶ u ist ein **Präfix** von v , wenn es ein Wort w gibt mit $uw = v$.
- ▶ u ist ein **Suffix** von v , wenn es ein Wort w gibt mit $wu = v$.
- ▶ u ist ein **Teilwort** von v , wenn es Wörter w_1, w_2 gibt mit $w_1uw_2 = v$.



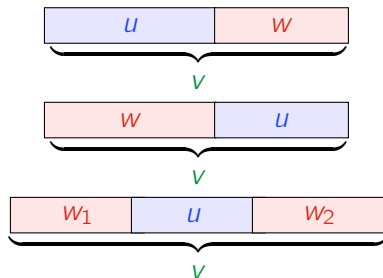
Beispiel: Sei $w = ababbaba$.

- ▶ w ist ein Präfix, Suffix und Teilwort von w .
- ▶ aba ist ein Präfix, Suffix und Teilwort von w .
- ▶ $ababb$ ist ein Präfix und Teilwort von w , aber kein Suffix von w .

Sprechweisen: Präfix, Suffix, Teilwort

Seien u, v Wörter über einem Alphabet Σ .

- ▶ u ist ein **Präfix** von v , wenn es ein Wort w gibt mit $uw = v$.
- ▶ u ist ein **Suffix** von v , wenn es ein Wort w gibt mit $wu = v$.
- ▶ u ist ein **Teilwort** von v , wenn es Wörter w_1, w_2 gibt mit $w_1uw_2 = v$.



Beispiel: Sei $w = ababbaba$.

- ▶ w ist ein Präfix, Suffix und Teilwort von w .
- ▶ aba ist ein Präfix, Suffix und Teilwort von w .
- ▶ $ababb$ ist ein Präfix und Teilwort von w , aber kein Suffix von w .
- ▶ bab ist ein Teilwort von w , aber weder ein Präfix noch ein Suffix.

Definition

Eine (formale) Sprache L über dem Alphabet Σ ist eine Teilmenge von Σ^* , d.h. $L \subseteq \Sigma^*$.

Beachte: L steht für „language“.

Definition

Eine (**formale**) **Sprache** L über dem Alphabet Σ ist eine Teilmenge von Σ^* , d.h. $L \subseteq \Sigma^*$.

Beachte: L steht für „language“.

Definition

Seien L, L_1, L_2 formale Sprachen über Σ .

- ▶ **Vereinigung**: $L_1 \cup L_2 := \{w \mid w \in L_1 \text{ oder } w \in L_2\}$
- ▶ **Schnitt**: $L_1 \cap L_2 := \{w \mid w \in L_1 \text{ und } w \in L_2\}$
- ▶ **Komplement** zu L : $\overline{L} := \Sigma^* \setminus L$
- ▶ **Produkt**: $L_1 \cdot L_2 = L_1 L_2 = \{uv \mid u \in L_1 \text{ und } v \in L_2\}$

Beispiele für Operationen auf formalen Sprachen

Seien $\Sigma = \{a, b\}$, $L_1 = \{a^i \mid i \in \mathbb{N}\}$ und $L_2 = \{b^i \mid i \in \mathbb{N}\}$.

$$L_1 \cup L_2 = ?$$

$$L_1 \cap L_2 = ?$$

$$\overline{L_1} = ?$$

$$L_1 L_2 = ?$$

$$L_2 L_1 = ?$$

$$L_1 L_1 = ?$$

Beispiele für Operationen auf formalen Sprachen

Seien $\Sigma = \{a, b\}$, $L_1 = \{a^i \mid i \in \mathbb{N}\}$ und $L_2 = \{b^i \mid i \in \mathbb{N}\}$.

$L_1 \cup L_2 =$ Sprache der Wörter, die nur aus a 's oder nur aus b 's bestehen

$L_1 \cap L_2 = ?$

$\overline{L_1} = ?$

$L_1 L_2 = ?$

$L_2 L_1 = ?$

$L_1 L_1 = ?$

Beispiele für Operationen auf formalen Sprachen

Seien $\Sigma = \{a, b\}$, $L_1 = \{a^i \mid i \in \mathbb{N}\}$ und $L_2 = \{b^i \mid i \in \mathbb{N}\}$.

$L_1 \cup L_2 =$ Sprache der Wörter, die nur aus a 's oder nur aus b 's bestehen

$L_1 \cap L_2 = \{\varepsilon\} \neq \emptyset$

$\overline{L_1} = ?$

$L_1 L_2 = ?$

$L_2 L_1 = ?$

$L_1 L_1 = ?$

Beispiele für Operationen auf formalen Sprachen

Seien $\Sigma = \{a, b\}$, $L_1 = \{a^i \mid i \in \mathbb{N}\}$ und $L_2 = \{b^i \mid i \in \mathbb{N}\}$.

$L_1 \cup L_2 =$ Sprache der Wörter, die nur aus a 's oder nur aus b 's bestehen

$L_1 \cap L_2 = \{\varepsilon\} \neq \emptyset$

$\overline{L_1} =$ Sprache der Wörter, die mindestens ein b enthalten

$L_1 L_2 = ?$

$L_2 L_1 = ?$

$L_1 L_1 = ?$

Beispiele für Operationen auf formalen Sprachen

Seien $\Sigma = \{a, b\}$, $L_1 = \{a^i \mid i \in \mathbb{N}\}$ und $L_2 = \{b^i \mid i \in \mathbb{N}\}$.

$L_1 \cup L_2 =$ Sprache der Wörter, die nur aus a 's oder nur aus b 's bestehen

$L_1 \cap L_2 = \{\varepsilon\} \neq \emptyset$

$\overline{L_1} =$ Sprache der Wörter, die mindestens ein b enthalten

$L_1 L_2 = \{a^i b^j \mid i, j \in \mathbb{N}\}$

$L_2 L_1 = ?$

$L_1 L_1 = ?$

Beispiele für Operationen auf formalen Sprachen

Seien $\Sigma = \{a, b\}$, $L_1 = \{a^i \mid i \in \mathbb{N}\}$ und $L_2 = \{b^i \mid i \in \mathbb{N}\}$.

$L_1 \cup L_2 =$ Sprache der Wörter, die nur aus a 's oder nur aus b 's bestehen

$L_1 \cap L_2 = \{\varepsilon\} \neq \emptyset$

$\overline{L_1} =$ Sprache der Wörter, die mindestens ein b enthalten

$L_1 L_2 = \{a^i b^j \mid i, j \in \mathbb{N}\}$

$L_2 L_1 = \{b^i a^j \mid i, j \in \mathbb{N}\}$

$L_1 L_1 = ?$

Beispiele für Operationen auf formalen Sprachen

Seien $\Sigma = \{a, b\}$, $L_1 = \{a^i \mid i \in \mathbb{N}\}$ und $L_2 = \{b^i \mid i \in \mathbb{N}\}$.

$L_1 \cup L_2 =$ Sprache der Wörter, die nur aus a 's oder nur aus b 's bestehen

$L_1 \cap L_2 = \{\varepsilon\} \neq \emptyset$

$\overline{L_1} =$ Sprache der Wörter, die mindestens ein b enthalten

$L_1 L_2 = \{a^i b^j \mid i, j \in \mathbb{N}\}$

$L_2 L_1 = \{b^i a^j \mid i, j \in \mathbb{N}\}$

$L_1 L_1 = L_1$

Kleenescher Abschluss

Sei L eine Sprache. Dann ist:

$$L^0 := \{\varepsilon\}$$

$$L^i := L \cdot L^{i-1} \text{ für } i > 0$$

$$L^* := \bigcup_{i \in \mathbb{N}} L^i$$

$$L^+ := \bigcup_{i \in \mathbb{N}_{>0}} L^i$$

Die Sprache L^* nennt man auch den **Kleeneschen Abschluss** von L , benannt nach Stephen Cole Kleene.

Kleenescher Abschluss

Sei L eine Sprache. Dann ist:

$$L^0 := \{\varepsilon\}$$

$$L^i := L \cdot L^{i-1} \text{ für } i > 0$$

$$L^* := \bigcup_{i \in \mathbb{N}} L^i$$

$$L^+ := \bigcup_{i \in \mathbb{N}_{>0}} L^i$$

Die Sprache L^* nennt man auch den **Kleeneschen Abschluss** von L , benannt nach Stephen Cole Kleene.

Beispiel: Sei $L = \{ab, ac\}$.

$$L^0 = ?$$

$$L^1 = ?$$

$$L^2 = ?$$

$$L^3 = ?$$

Kleenescher Abschluss

Sei L eine Sprache. Dann ist:

$$L^0 := \{\varepsilon\}$$

$$L^i := L \cdot L^{i-1} \text{ für } i > 0$$

$$L^* := \bigcup_{i \in \mathbb{N}} L^i$$

$$L^+ := \bigcup_{i \in \mathbb{N}_{>0}} L^i$$

Die Sprache L^* nennt man auch den **Kleeneschen Abschluss** von L , benannt nach Stephen Cole Kleene.

Beispiel: Sei $L = \{ab, ac\}$.

$$L^0 = \{\varepsilon\}$$

$$L^1 = ?$$

$$L^2 = ?$$

$$L^3 = ?$$

Kleenescher Abschluss

Sei L eine Sprache. Dann ist:

$$L^0 := \{\varepsilon\}$$

$$L^i := L \cdot L^{i-1} \text{ für } i > 0$$

$$L^* := \bigcup_{i \in \mathbb{N}} L^i$$

$$L^+ := \bigcup_{i \in \mathbb{N}_{>0}} L^i$$

Die Sprache L^* nennt man auch den **Kleeneschen Abschluss** von L , benannt nach Stephen Cole Kleene.

Beispiel: Sei $L = \{ab, ac\}$.

$$L^0 = \{\varepsilon\}$$

$$L^1 = L \cdot L^0 = L = \{ab, ac\}$$

$$L^2 = ?$$

$$L^3 = ?$$

Kleenescher Abschluss

Sei L eine Sprache. Dann ist:

$$L^0 := \{\varepsilon\}$$

$$L^i := L \cdot L^{i-1} \text{ für } i > 0$$

$$L^* := \bigcup_{i \in \mathbb{N}} L^i$$

$$L^+ := \bigcup_{i \in \mathbb{N}_{>0}} L^i$$

Die Sprache L^* nennt man auch den **Kleeneschen Abschluss** von L , benannt nach Stephen Cole Kleene.

Beispiel: Sei $L = \{ab, ac\}$.

$$L^0 = \{\varepsilon\}$$

$$L^1 = L \cdot L^0 = L = \{ab, ac\}$$

$$L^2 = L \cdot L^1 = \{abab, abac, acab, acac\}$$

$$L^3 = ?$$

Kleenescher Abschluss

Sei L eine Sprache. Dann ist:

$$L^0 := \{\varepsilon\}$$

$$L^i := L \cdot L^{i-1} \text{ für } i > 0$$

$$L^* := \bigcup_{i \in \mathbb{N}} L^i$$

$$L^+ := \bigcup_{i \in \mathbb{N}_{>0}} L^i$$

Die Sprache L^* nennt man auch den **Kleeneschen Abschluss** von L , benannt nach Stephen Cole Kleene.

Beispiel: Sei $L = \{ab, ac\}$.

$$L^0 = \{\varepsilon\}$$

$$L^1 = L \cdot L^0 = L = \{ab, ac\}$$

$$L^2 = L \cdot L^1 = \{abab, abac, acab, acac\}$$

$$L^3 = L \cdot L^2 = \{ababab, ababac, abacab, abacac, acabab, acabac, acacab, acacac\}$$

Weitere Beispiele für Operationen auf formalen Sprachen

$$((\{\epsilon, 1\} \cdot \{0, \dots, 9\}) \cup (\{2\} \cdot \{0, 1, 2, 3\})) \cdot \{:\} \cdot \{0, 1, 2, 3, 4, 5\} \cdot \{0, \dots, 9\}$$

Beschriebene Sprache = ?

$$\{0\} \cup (\{1, \dots, 9\} \cdot \{0, \dots, 9\}^*)$$

Beschriebene Sprache = ?

Weitere Beispiele für Operationen auf formalen Sprachen

$$((\{\epsilon, 1\} \cdot \{0, \dots, 9\}) \cup (\{2\} \cdot \{0, 1, 2, 3\})) \cdot \{:\} \cdot \{0, 1, 2, 3, 4, 5\} \cdot \{0, \dots, 9\}$$

Beschriebene Sprache = Sprache aller gültigen Uhrzeiten

$$\{0\} \cup (\{1, \dots, 9\} \cdot \{0, \dots, 9\}^*)$$

Beschriebene Sprache = ?

Weitere Beispiele für Operationen auf formalen Sprachen

$$((\{\epsilon, 1\} \cdot \{0, \dots, 9\}) \cup (\{2\} \cdot \{0, 1, 2, 3\})) \cdot \{:\} \cdot \{0, 1, 2, 3, 4, 5\} \cdot \{0, \dots, 9\}$$

Beschriebene Sprache = Sprache aller gültigen Uhrzeiten

$$\{0\} \cup (\{1, \dots, 9\} \cdot \{0, \dots, 9\}^*)$$

Beschriebene Sprache = Sprache aller natürlichen Zahlen

1b

Grammatiken und die Chomsky-Hierarchie

Prof. Dr. Jasmin Blanchette

Lehr- und Forschungseinheit für
Theoretische Informatik und Theorembeweisen

Stand: 10. Mai 2024

Basiert auf Folien von PD Dr. David Sabel und Dr. Jan Johannsen



Formale Sprachen darstellen

- ▶ Sei Σ ein Alphabet.
- ▶ Eine Sprache über Σ ist eine Teilmenge von Σ^* .

Formale Sprachen darstellen

- ▶ Sei Σ ein **Alphabet**.
- ▶ Eine **Sprache** über Σ ist eine Teilmenge von Σ^* .
- ▶ Für $\Sigma = \{ (,), +, -, *, /, a \}$ sei L_{ArEx} die Sprache aller korrekt geklammerten Ausdrücke.
Z.B. $((a + a) - a) * a \in L_{ArEx}$ aber $(a -) + a) \notin L_{ArEx}$.
- ▶ Unsere bisherigen Operationen auf Sprachen (Mengen) können das nicht darstellen.

Formale Sprachen darstellen

- ▶ Sei Σ ein Alphabet.
- ▶ Eine Sprache über Σ ist eine Teilmenge von Σ^* .
- ▶ Für $\Sigma = \{ (,), +, -, *, /, a \}$ sei L_{ArEx} die Sprache aller korrekt geklammerten Ausdrücke.
Z.B. $((a + a) - a) * a \in L_{ArEx}$ aber $(a -) + a \notin L_{ArEx}$.
- ▶ Unsere bisherigen Operationen auf Sprachen (Mengen) können das nicht darstellen.

Benötigt: Formalismus, um L_{ArEx} zu beschreiben

Formale Sprachen darstellen

Anforderungen:

- ▶ Beschreibung muss endlich sein.
- ▶ Sprache selbst muss aber auch unendlich viele Objekte erlauben.

Zwei wesentliche solchen Formalismen sind

- ▶ Grammatiken
- ▶ Automaten.

Grammatik für einen sehr kleinen Teil der deutschen Sprache:

<Satz> → <Subjekt><Prädikat><Objekt>

<Subjekt> → <Artikel><Attribut><Nomen>

<Objekt> → <Artikel><Attribut><Nomen>

<Artikel> → ϵ

<Artikel> → der

<Artikel> → das

<Attribut> → <Adjektiv>

<Attribut> → <Adjektiv><Attribut>

<Adjektiv> → kleine

<Adjektiv> → große

<Adjektiv> → nette

<Adjektiv> → blaue

<Nomen> → Frau

<Nomen> → Mann

<Nomen> → Auto

<Prädikat> → fährt

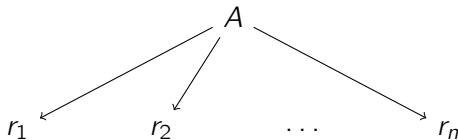
<Prädikat> → liebt

- ▶ **Grammatik** = endliche Menge von Regeln „linke Seite $\rightarrow$ rechte Seite“
- ▶ Symbole in spitzen Klammern wie $\langle \text{Artikel} \rangle$ sind **Variablen**, d.h. sie sind **Platzhalter**, die weiter **ersetzt** werden müssen.
- ▶ Z.B. kann

der kleine nette Mann fährt das große blaue Auto
durch die vorige Grammatik abgeleitet werden.

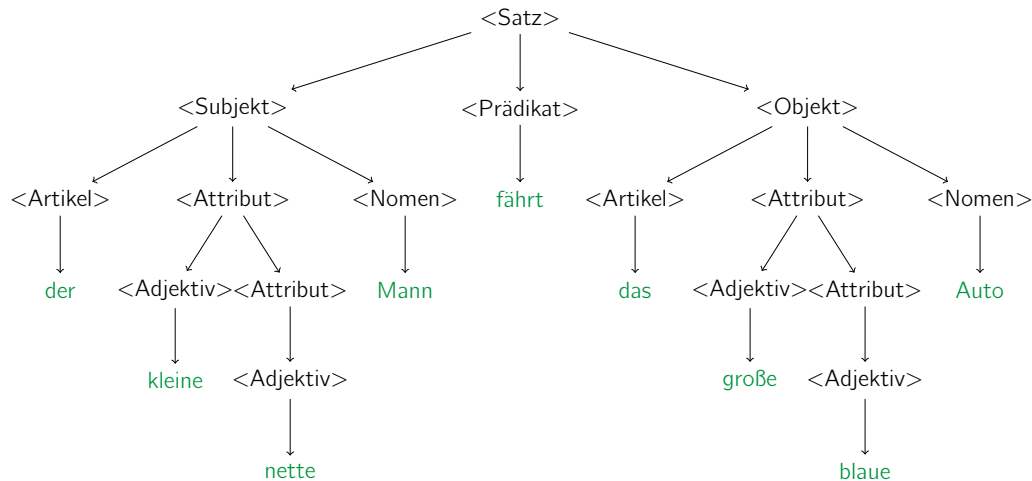
Syntaxbäume

- ▶ Ein **Syntaxbaum** stellt dar, wie ein Satz entsteht.
- ▶ Die Anwendung von $A \rightarrow r_1 r_2 \dots r_n$ wird durch den Teilbaum



dargestellt.

Syntaxbaum zum Beispiel



Definition einer Grammatik

Definition

Eine Grammatik ist ein 4-Tupel $G = (V, \Sigma, P, S)$ wobei:

- ▶ V ist eine endliche Menge von Variablen
- ▶ Σ (mit $V \cap \Sigma = \emptyset$) ist ein Alphabet von Zeichen
- ▶ P ist eine endliche Menge von Produktionen
von der Form $\ell \rightarrow r$ wobei $\ell \in (V \cup \Sigma)^+$ und $r \in (V \cup \Sigma)^*$
- ▶ $S \in V$ ist das Startsymbol.

Definition einer Grammatik

Definition

Eine Grammatik ist ein 4-Tupel $G = (V, \Sigma, P, S)$ wobei:

- ▶ V ist eine endliche Menge von Variablen (alternativ Nichtterminalen)
- ▶ Σ (mit $V \cap \Sigma = \emptyset$) ist ein Alphabet von Zeichen (alternativ Terminalen)
- ▶ P ist eine endliche Menge von Produktionen (alternativ Regeln) von der Form $\ell \rightarrow r$ wobei $\ell \in (V \cup \Sigma)^+$ und $r \in (V \cup \Sigma)^*$
- ▶ $S \in V$ ist das Startsymbol (alternativ Startvariable).

Definition einer Grammatik

Definition

Eine **Grammatik** ist ein 4-Tupel $G = (V, \Sigma, P, S)$ wobei:

- ▶ V ist eine endliche Menge von **Variablen** (alternativ **Nichtterminalen**)
- ▶ Σ (mit $V \cap \Sigma = \emptyset$) ist ein **Alphabet** von **Zeichen** (alternativ **Terminalen**)
- ▶ P ist eine endliche Menge von **Produktionen** (alternativ **Regeln**)
von der Form $\ell \rightarrow r$ wobei $\ell \in (V \cup \Sigma)^+$ und $r \in (V \cup \Sigma)^*$
- ▶ $S \in V$ ist das **Startsymbol** (alternativ **Startvariable**).

Produktionen mit ε auf der rechten Seite heißen **ε -Produktionen**.

Manchmal genügt es, die Produktionen P alleine zu notieren
(wenn klar ist, was V , Σ und S sind).

Beispiel für eine Grammatik

$G = (V, \Sigma, P, E)$ mit

$V = \{E, M, Z\},$

$\Sigma = \{+, *, 1, 2, (,)\}$ und

$P = \{E \rightarrow M,$

$E \rightarrow E + M,$

$M \rightarrow Z,$

$M \rightarrow M * Z,$

$Z \rightarrow 1,$

$Z \rightarrow 2,$

$Z \rightarrow (E)\}$

Definition

Sei $G = (V, \Sigma, P, S)$ eine Grammatik.

Eine **Satzform** ist ein Wort aus $(V \cup \Sigma)^*$.

Satzform u **geht unter** Grammatik G **unmittelbar in** Satzform v **über**, $u \Rightarrow_G v$, wenn

$$u = w_1 \ell w_2 \text{ und } v = w_1 r w_2 \text{ mit } \ell \rightarrow r \in P$$

- ▶ Wenn G klar ist, schreiben wir $u \Rightarrow v$ statt $u \Rightarrow_G v$.
- ▶ $\Rightarrow^*$ ist die reflexiv-transitive Hülle von $\Rightarrow$. Sie ist definiert durch folgende Regeln (und nur diese):
 - ▶ falls $u \Rightarrow v$, dann ist $u \Rightarrow^* v$
 - ▶ $u \Rightarrow^* u$
 - ▶ falls $u \Rightarrow^* v$ und $v \Rightarrow^* w$, dann ist $u \Rightarrow^* w$.

Definition

Sei $G = (V, \Sigma, P, S)$ eine Grammatik.

Eine Folge $(w_0, w_1, \dots, w_n)$ mit $w_0 = S$, $w_n \in \Sigma^*$ und $w_{i-1} \Rightarrow w_i$ für $i = 1, \dots, n$ heißt **Ableitung** von w_n .

Statt $(w_0, \dots, w_n)$ schreiben wir auch $w_0 \Rightarrow \dots \Rightarrow w_n$.

Beispiel für eine Ableitung

$G = (V, \Sigma, P, E)$ mit $V = \{E, M, Z\}$ und $\Sigma = \{+, *, 1, 2, (,)\}$ und
 $P = \{E \rightarrow M, E \rightarrow E + M, M \rightarrow Z, M \rightarrow M * Z, Z \rightarrow 1, Z \rightarrow 2, Z \rightarrow (E)\}$

Eine Ableitung von $(2 + 1) * (2 + 2)$:

$E \Rightarrow M$

Beispiel für eine Ableitung

$G = (V, \Sigma, P, E)$ mit $V = \{E, M, Z\}$ und $\Sigma = \{+, *, 1, 2, (,)\}$ und
 $P = \{E \rightarrow M, E \rightarrow E + M, M \rightarrow Z, M \rightarrow M * Z, Z \rightarrow 1, Z \rightarrow 2, Z \rightarrow (E)\}$

Eine Ableitung von $(2 + 1) * (2 + 2)$:

$E \Rightarrow M \Rightarrow M * Z$

Beispiel für eine Ableitung

$G = (V, \Sigma, P, E)$ mit $V = \{E, M, Z\}$ und $\Sigma = \{+, *, 1, 2, (,)\}$ und
 $P = \{E \rightarrow M, E \rightarrow E + M, M \rightarrow Z, M \rightarrow M * Z, Z \rightarrow 1, Z \rightarrow 2, Z \rightarrow (E)\}$

Eine Ableitung von $(2 + 1) * (2 + 2)$:

$E \Rightarrow M \Rightarrow M * Z \Rightarrow Z * Z$

Beispiel für eine Ableitung

$G = (V, \Sigma, P, E)$ mit $V = \{E, M, Z\}$ und $\Sigma = \{+, *, 1, 2, (,)\}$ und
 $P = \{E \rightarrow M, E \rightarrow E + M, M \rightarrow Z, M \rightarrow M * Z, Z \rightarrow 1, Z \rightarrow 2, \textcolor{red}{Z} \rightarrow (\textcolor{blue}{E})\}$

Eine Ableitung von $(2 + 1) * (2 + 2)$:

$E \Rightarrow M \Rightarrow M * Z \Rightarrow Z * \textcolor{red}{Z} \Rightarrow Z * (\textcolor{blue}{E})$

Beispiel für eine Ableitung

$G = (V, \Sigma, P, E)$ mit $V = \{E, M, Z\}$ und $\Sigma = \{+, *, 1, 2, (,)\}$ und
 $P = \{E \rightarrow M, \textcolor{red}{E} \rightarrow \textcolor{blue}{E} + \textcolor{blue}{M}, M \rightarrow Z, M \rightarrow M * Z, Z \rightarrow 1, Z \rightarrow 2, Z \rightarrow (E)\}$

Eine Ableitung von $(2 + 1) * (2 + 2)$:

$E \Rightarrow M \Rightarrow M * Z \Rightarrow Z * Z \Rightarrow Z * (\textcolor{red}{E}) \Rightarrow Z * (\textcolor{blue}{E} + \textcolor{blue}{M})$

Beispiel für eine Ableitung

$G = (V, \Sigma, P, E)$ mit $V = \{E, M, Z\}$ und $\Sigma = \{+, *, 1, 2, (,)\}$ und
 $P = \{E \rightarrow M, E \rightarrow E + M, M \rightarrow Z, M \rightarrow M * Z, Z \rightarrow 1, Z \rightarrow 2, \textcolor{red}{Z} \rightarrow (\textcolor{blue}{E})\}$

Eine Ableitung von $(2 + 1) * (2 + 2)$:

$E \Rightarrow M \Rightarrow M * Z \Rightarrow Z * Z \Rightarrow Z * (E) \Rightarrow \textcolor{red}{Z} * (E + M)$
 $\Rightarrow (\textcolor{blue}{E}) * (E + M)$

Beispiel für eine Ableitung

$G = (V, \Sigma, P, E)$ mit $V = \{E, M, Z\}$ und $\Sigma = \{+, *, 1, 2, (,)\}$ und
 $P = \{E \rightarrow M, E \rightarrow E + M, M \rightarrow Z, M \rightarrow M * Z, Z \rightarrow 1, Z \rightarrow 2, Z \rightarrow (E)\}$

Eine Ableitung von $(2 + 1) * (2 + 2)$:

$E \Rightarrow M \Rightarrow M * Z \Rightarrow Z * Z \Rightarrow Z * (E) \Rightarrow Z * (E + M)$
 $\Rightarrow (E) * (E + M) \Rightarrow (E) * (E + Z)$

Beispiel für eine Ableitung

$G = (V, \Sigma, P, E)$ mit $V = \{E, M, Z\}$ und $\Sigma = \{+, *, 1, 2, (,)\}$ und
 $P = \{E \rightarrow M, \textcolor{red}{E} \rightarrow \textcolor{blue}{E} + \textcolor{blue}{M}, M \rightarrow Z, M \rightarrow M * Z, Z \rightarrow 1, Z \rightarrow 2, Z \rightarrow (E)\}$

Eine Ableitung von $(2 + 1) * (2 + 2)$:

$E \Rightarrow M \Rightarrow M * Z \Rightarrow Z * Z \Rightarrow Z * (E) \Rightarrow Z * (E + M)$
 $\Rightarrow (E) * (E + M) \Rightarrow (\textcolor{red}{E}) * (E + Z) \Rightarrow (\textcolor{blue}{E} + \textcolor{blue}{M}) * (E + Z)$

Beispiel für eine Ableitung

$G = (V, \Sigma, P, E)$ mit $V = \{E, M, Z\}$ und $\Sigma = \{+, *, 1, 2, (,)\}$ und
 $P = \{\textcolor{red}{E} \rightarrow \textcolor{blue}{M}, E \rightarrow E + M, M \rightarrow Z, M \rightarrow M * Z, Z \rightarrow 1, Z \rightarrow 2, Z \rightarrow (E)\}$

Eine Ableitung von $(2 + 1) * (2 + 2)$:

$E \Rightarrow M \Rightarrow M * Z \Rightarrow Z * Z \Rightarrow Z * (E) \Rightarrow Z * (E + M)$
 $\Rightarrow (E) * (E + M) \Rightarrow (E) * (E + Z) \Rightarrow (\textcolor{red}{E} + M) * (E + Z)$
 $\Rightarrow (\textcolor{blue}{M} + M) * (E + Z)$

Beispiel für eine Ableitung

$G = (V, \Sigma, P, E)$ mit $V = \{E, M, Z\}$ und $\Sigma = \{+, *, 1, 2, (,)\}$ und
 $P = \{\textcolor{red}{E} \rightarrow \textcolor{blue}{M}, E \rightarrow E + M, M \rightarrow Z, M \rightarrow M * Z, Z \rightarrow 1, Z \rightarrow 2, Z \rightarrow (E)\}$

Eine Ableitung von $(2 + 1) * (2 + 2)$:

$$\begin{aligned} E &\Rightarrow M \Rightarrow M * Z \Rightarrow Z * Z \Rightarrow Z * (E) \Rightarrow Z * (E + M) \\ &\Rightarrow (E) * (E + M) \Rightarrow (E) * (E + Z) \Rightarrow (E + M) * (E + Z) \\ &\Rightarrow (M + M) * (\textcolor{red}{E} + Z) \Rightarrow (M + M) * (\textcolor{blue}{M} + Z) \end{aligned}$$

Beispiel für eine Ableitung

$G = (V, \Sigma, P, E)$ mit $V = \{E, M, Z\}$ und $\Sigma = \{+, *, 1, 2, (,)\}$ und
 $P = \{E \rightarrow M, E \rightarrow E + M, M \rightarrow Z, M \rightarrow M * Z, Z \rightarrow 1, Z \rightarrow 2, Z \rightarrow (E)\}$

Eine Ableitung von $(2 + 1) * (2 + 2)$:

$E \Rightarrow M \Rightarrow M * Z \Rightarrow Z * Z \Rightarrow Z * (E) \Rightarrow Z * (E + M)$
 $\Rightarrow (E) * (E + M) \Rightarrow (E) * (E + Z) \Rightarrow (E + M) * (E + Z)$
 $\Rightarrow (M + M) * (E + Z) \Rightarrow (M + M) * (M + Z)$
 $\Rightarrow (M + M) * (Z + Z)$

Beispiel für eine Ableitung

$G = (V, \Sigma, P, E)$ mit $V = \{E, M, Z\}$ und $\Sigma = \{+, *, 1, 2, (,)\}$ und
 $P = \{E \rightarrow M, E \rightarrow E + M, M \rightarrow Z, M \rightarrow M * Z, Z \rightarrow 1, \textcolor{red}{Z} \rightarrow \textcolor{blue}{2}, Z \rightarrow (E)\}$

Eine Ableitung von $(2 + 1) * (2 + 2)$:

$E \Rightarrow M \Rightarrow M * Z \Rightarrow Z * Z \Rightarrow Z * (E) \Rightarrow Z * (E + M)$
 $\Rightarrow (E) * (E + M) \Rightarrow (E) * (E + Z) \Rightarrow (E + M) * (E + Z)$
 $\Rightarrow (M + M) * (E + Z) \Rightarrow (M + M) * (M + Z)$
 $\Rightarrow (M + M) * (Z + \textcolor{red}{Z}) \Rightarrow (M + M) * (Z + \textcolor{blue}{2})$

Beispiel für eine Ableitung

$G = (V, \Sigma, P, E)$ mit $V = \{E, M, Z\}$ und $\Sigma = \{+, *, 1, 2, (,)\}$ und
 $P = \{E \rightarrow M, E \rightarrow E + M, M \rightarrow Z, M \rightarrow M * Z, Z \rightarrow 1, Z \rightarrow 2, Z \rightarrow (E)\}$

Eine Ableitung von $(2 + 1) * (2 + 2)$:

$E \Rightarrow M \Rightarrow M * Z \Rightarrow Z * Z \Rightarrow Z * (E) \Rightarrow Z * (E + M)$
 $\Rightarrow (E) * (E + M) \Rightarrow (E) * (E + Z) \Rightarrow (E + M) * (E + Z)$
 $\Rightarrow (M + M) * (E + Z) \Rightarrow (M + M) * (M + Z)$
 $\Rightarrow (M + M) * (Z + Z) \Rightarrow (M + M) * (Z + 2)$
 $\Rightarrow (M + Z) * (Z + 2)$

Beispiel für eine Ableitung

$G = (V, \Sigma, P, E)$ mit $V = \{E, M, Z\}$ und $\Sigma = \{+, *, 1, 2, (,)\}$ und
 $P = \{E \rightarrow M, E \rightarrow E + M, M \rightarrow Z, M \rightarrow M * Z, Z \rightarrow 1, \textcolor{red}{Z} \rightarrow \textcolor{blue}{2}, Z \rightarrow (E)\}$

Eine Ableitung von $(2 + 1) * (2 + 2)$:

$$\begin{aligned} E &\Rightarrow M \Rightarrow M * Z \Rightarrow Z * Z \Rightarrow Z * (E) \Rightarrow Z * (E + M) \\ &\Rightarrow (E) * (E + M) \Rightarrow (E) * (E + Z) \Rightarrow (E + M) * (E + Z) \\ &\Rightarrow (M + M) * (E + Z) \Rightarrow (M + M) * (M + Z) \\ &\Rightarrow (M + M) * (Z + Z) \Rightarrow (M + M) * (Z + 2) \\ &\Rightarrow (M + Z) * (\textcolor{red}{Z} + 2) \Rightarrow (M + Z) * (\textcolor{blue}{2} + 2) \end{aligned}$$

Beispiel für eine Ableitung

$G = (V, \Sigma, P, E)$ mit $V = \{E, M, Z\}$ und $\Sigma = \{+, *, 1, 2, (,)\}$ und
 $P = \{E \rightarrow M, E \rightarrow E + M, M \rightarrow Z, M \rightarrow M * Z, Z \rightarrow 1, Z \rightarrow 2, Z \rightarrow (E)\}$

Eine Ableitung von $(2 + 1) * (2 + 2)$:

$$\begin{aligned} E &\Rightarrow M \Rightarrow M * Z \Rightarrow Z * Z \Rightarrow Z * (E) \Rightarrow Z * (E + M) \\ &\Rightarrow (E) * (E + M) \Rightarrow (E) * (E + Z) \Rightarrow (E + M) * (E + Z) \\ &\Rightarrow (M + M) * (E + Z) \Rightarrow (M + M) * (M + Z) \\ &\Rightarrow (M + M) * (Z + Z) \Rightarrow (M + M) * (Z + 2) \\ &\Rightarrow (M + Z) * (Z + 2) \Rightarrow (M + Z) * (2 + 2) \\ &\Rightarrow (Z + Z) * (2 + 2) \end{aligned}$$

Beispiel für eine Ableitung

$G = (V, \Sigma, P, E)$ mit $V = \{E, M, Z\}$ und $\Sigma = \{+, *, 1, 2, (,)\}$ und
 $P = \{E \rightarrow M, E \rightarrow E + M, M \rightarrow Z, M \rightarrow M * Z, Z \rightarrow 1, \textcolor{red}{Z} \rightarrow \textcolor{blue}{2}, Z \rightarrow (E)\}$

Eine Ableitung von $(2 + 1) * (2 + 2)$:

$$\begin{aligned} E &\Rightarrow M \Rightarrow M * Z \Rightarrow Z * Z \Rightarrow Z * (E) \Rightarrow Z * (E + M) \\ &\Rightarrow (E) * (E + M) \Rightarrow (E) * (E + Z) \Rightarrow (E + M) * (E + Z) \\ &\Rightarrow (M + M) * (E + Z) \Rightarrow (M + M) * (M + Z) \\ &\Rightarrow (M + M) * (Z + Z) \Rightarrow (M + M) * (Z + 2) \\ &\Rightarrow (M + Z) * (Z + 2) \Rightarrow (M + Z) * (2 + 2) \\ &\Rightarrow (\textcolor{red}{Z} + Z) * (2 + 2) \Rightarrow (\textcolor{blue}{2} + Z) * (2 + 2) \end{aligned}$$

Beispiel für eine Ableitung

$G = (V, \Sigma, P, E)$ mit $V = \{E, M, Z\}$ und $\Sigma = \{+, *, 1, 2, (,)\}$ und
 $P = \{E \rightarrow M, E \rightarrow E + M, M \rightarrow Z, M \rightarrow M * Z, \textcolor{red}{Z} \rightarrow \textcolor{blue}{1}, Z \rightarrow 2, Z \rightarrow (E)\}$

Eine Ableitung von $(2 + 1) * (2 + 2)$:

$$\begin{aligned} E &\Rightarrow M \Rightarrow M * Z \Rightarrow Z * Z \Rightarrow Z * (E) \Rightarrow Z * (E + M) \\ &\Rightarrow (E) * (E + M) \Rightarrow (E) * (E + Z) \Rightarrow (E + M) * (E + Z) \\ &\Rightarrow (M + M) * (E + Z) \Rightarrow (M + M) * (M + Z) \\ &\Rightarrow (M + M) * (Z + Z) \Rightarrow (M + M) * (Z + 2) \\ &\Rightarrow (M + Z) * (Z + 2) \Rightarrow (M + Z) * (2 + 2) \\ &\Rightarrow (Z + Z) * (2 + 2) \Rightarrow (2 + \textcolor{red}{Z}) * (2 + 2) \Rightarrow (2 + \textcolor{blue}{1}) * (2 + 2) \end{aligned}$$

Ableitungen sind nicht eindeutig

Ableitung von letzter Folie:

$$\begin{aligned} E &\Rightarrow M \Rightarrow M * Z \Rightarrow Z * Z \Rightarrow Z * (E) \Rightarrow Z * (E + M) \\ &\Rightarrow (E) * (E + M) \Rightarrow (E) * (E + Z) \Rightarrow (E + M) * (E + Z) \\ &\Rightarrow (M + M) * (E + Z) \Rightarrow (M + M) * (M + Z) \\ &\Rightarrow (M + M) * (Z + Z) \Rightarrow (M + M) * (Z + 2) \\ &\Rightarrow (M + Z) * (Z + 2) \Rightarrow (M + Z) * (2 + 2) \\ &\Rightarrow (Z + Z) * (2 + 2) \Rightarrow (2 + Z) * (2 + 2) \Rightarrow (2 + 1) * (2 + 2) \end{aligned}$$

Ableitungen sind nicht eindeutig

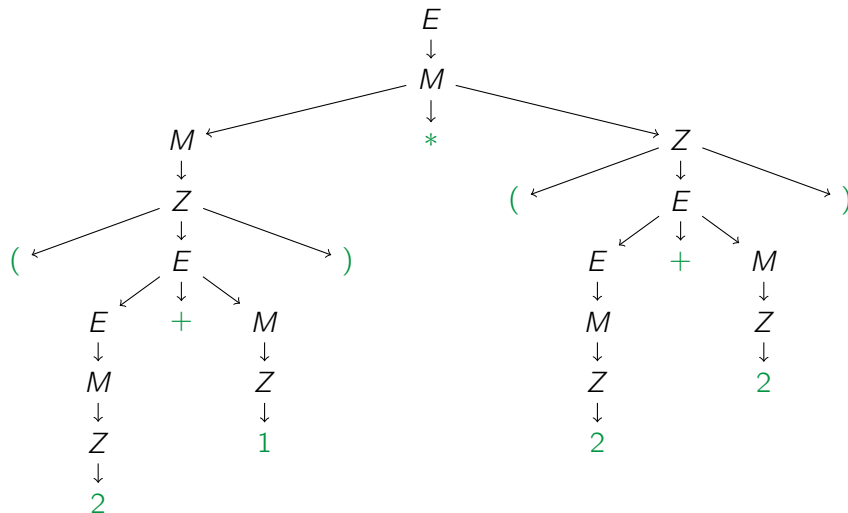
Ableitung von letzter Folie:

$$\begin{aligned} E &\Rightarrow M \Rightarrow M * Z \Rightarrow Z * Z \Rightarrow Z * (E) \Rightarrow Z * (E + M) \\ &\Rightarrow (E) * (E + M) \Rightarrow (E) * (E + Z) \Rightarrow (E + M) * (E + Z) \\ &\Rightarrow (M + M) * (E + Z) \Rightarrow (M + M) * (M + Z) \\ &\Rightarrow (M + M) * (Z + Z) \Rightarrow (M + M) * (Z + 2) \\ &\Rightarrow (M + Z) * (Z + 2) \Rightarrow (M + Z) * (2 + 2) \\ &\Rightarrow (Z + Z) * (2 + 2) \Rightarrow (2 + Z) * (2 + 2) \Rightarrow (2 + 1) * (2 + 2) \end{aligned}$$

Linksableitung (ersetzt immer die linkeste Variable):

$$\begin{aligned} E &\Rightarrow M \Rightarrow M * Z \Rightarrow Z * Z \Rightarrow (E) * Z \\ &\Rightarrow (E + M) * Z \Rightarrow (M + M) * Z \Rightarrow (Z + M) * Z \\ &\Rightarrow (2 + M) * Z \Rightarrow (2 + Z) * Z \Rightarrow (2 + 1) * Z \Rightarrow (2 + 1) * (E) \\ &\Rightarrow (2 + 1) * (E + M) \Rightarrow (2 + 1) * (M + M) \Rightarrow (2 + 1) * (Z + M) \\ &\Rightarrow (2 + 1) * (2 + M) \Rightarrow (2 + 1) * (2 + Z) \Rightarrow (2 + 1) * (2 + 2) \end{aligned}$$

Syntaxbaum zu beiden Ableitungen



Nichtdeterminismus beim Ableiten

Für eine Satzform u kann es verschiedene Satzformen v geben mit $u \Rightarrow_G v$.

Quellen des Nichtdeterminismus:

- ▶ Wähle **welche Produktion** $\ell \rightarrow r$ aus P angewendet wird.
- ▶ Wähle die **Position des Teilworts** ℓ in u , das durch r ersetzt wird.

Aber: Es gibt **nur endlich viele Satzformen** v für jeden Schritt.

Definition

Die von einer Grammatik $G = (V, \Sigma, P, S)$ erzeugte Sprache $L(G)$ ist

$$L(G) := \{w \in \Sigma^* \mid S \Rightarrow_G^* w\}$$

Beispiele für erzeugte Sprache

$$G_1 = (\{S\}, \{a\}, \{S \rightarrow aS\}, S)$$

$$L(G_1) = ?$$

$$G_2 = (\{S\}, \{a, b\}, \{S \rightarrow aS, S \rightarrow b\}, S)$$

$$L(G_2) = ?$$

Beispiele für erzeugte Sprache

$$G_1 = (\{S\}, \{a\}, \{S \rightarrow aS\}, S)$$

$$L(G_1) = ?$$

- ▶ $S \Rightarrow aS \Rightarrow aaS \Rightarrow \dots$ endet nie.
- ▶ Andere Ableitungen gibt es nicht.
- ▶ Daher sind keine Wörter aus $\{a\}^*$ ableitbar.

$$G_2 = (\{S\}, \{a, b\}, \{S \rightarrow aS, S \rightarrow b\}, S)$$

$$L(G_2) = ?$$

Beispiele für erzeugte Sprache

$$G_1 = (\{S\}, \{a\}, \{S \rightarrow aS\}, S)$$

$$L(G_1) = \emptyset$$

- ▶ $S \Rightarrow aS \Rightarrow aaS \Rightarrow \dots$ endet nie.
- ▶ Andere Ableitungen gibt es nicht.
- ▶ Daher sind keine Wörter aus $\{a\}^*$ ableitbar.

$$G_2 = (\{S\}, \{a, b\}, \{S \rightarrow aS, S \rightarrow b\}, S)$$

$$L(G_2) = ?$$

Beispiele für erzeugte Sprache

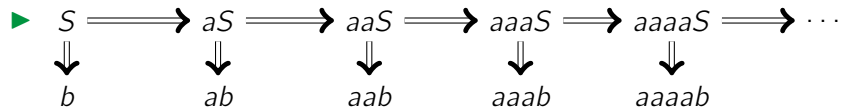
$$G_1 = (\{S\}, \{a\}, \{S \rightarrow aS\}, S)$$

$$L(G_1) = \emptyset$$

- ▶ $S \Rightarrow aS \Rightarrow aaS \Rightarrow \dots$ endet nie.
- ▶ Andere Ableitungen gibt es nicht.
- ▶ Daher sind keine Wörter aus $\{a\}^*$ ableitbar.

$$G_2 = (\{S\}, \{a, b\}, \{S \rightarrow aS, S \rightarrow b\}, S)$$

$$L(G_2) = ?$$



- ▶ Für alle $i \in \mathbb{N}$ gilt $S \Rightarrow^i a^i S \Rightarrow a^i b$.

Beispiele für erzeugte Sprache

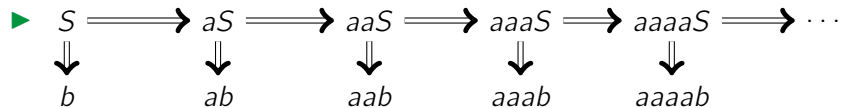
$$G_1 = (\{S\}, \{a\}, \{S \rightarrow aS\}, S)$$

$$L(G_1) = \emptyset$$

- ▶ $S \Rightarrow aS \Rightarrow aaS \Rightarrow \dots$ endet nie.
- ▶ Andere Ableitungen gibt es nicht.
- ▶ Daher sind keine Wörter aus $\{a\}^*$ ableitbar.

$$G_2 = (\{S\}, \{a, b\}, \{S \rightarrow aS, S \rightarrow b\}, S)$$

$$L(G_2) = \{a^n b \mid n \in \mathbb{N}\}$$



- ▶ Für alle $i \in \mathbb{N}$ gilt $S \Rightarrow^i a^i S \Rightarrow a^i b$.

Die Chomsky-Hierarchie

Noam Chomsky teilte die Grammatiken in Typen 0 bis 3 nach Art der erlaubten Regeln.

Definition

Sei $G = (V, \Sigma, P, S)$ eine Grammatik.

Die Chomsky-Hierarchie

Noam Chomsky teilte die Grammatiken in Typen 0 bis 3 nach Art der erlaubten Regeln.

Definition

Sei $G = (V, \Sigma, P, S)$ eine Grammatik.

- ▶ G ist automatisch vom Typ 0.

Die Chomsky-Hierarchie

Noam Chomsky teilte die Grammatiken in Typen 0 bis 3 nach Art der erlaubten Regeln.

Definition

Sei $G = (V, \Sigma, P, S)$ eine Grammatik.

- ▶ G ist automatisch vom Typ 0.
- ▶ G ist vom Typ 1 (alternativ kontextsensitiv), wenn:
für alle $\ell \rightarrow r \in P$ gilt $|\ell| \leq |r|$.

Die Chomsky-Hierarchie

Noam Chomsky teilte die Grammatiken in Typen 0 bis 3 nach Art der erlaubten Regeln.

Definition

Sei $G = (V, \Sigma, P, S)$ eine Grammatik.

- ▶ G ist automatisch vom Typ 0.
- ▶ G ist vom Typ 1 (alternativ kontextsensitiv), wenn:
für alle $\ell \rightarrow r \in P$ gilt $|\ell| \leq |r|$.
- ▶ G ist vom Typ 2 (alternativ kontextfrei), wenn:
 G ist vom Typ 1 und für alle $\ell \rightarrow r \in P$ gilt $\ell \in V$.

Die Chomsky-Hierarchie

Noam Chomsky teilte die Grammatiken in Typen 0 bis 3 nach Art der erlaubten Regeln.

Definition

Sei $G = (V, \Sigma, P, S)$ eine Grammatik.

- ▶ G ist automatisch vom Typ 0.
- ▶ G ist vom Typ 1 (alternativ kontextsensitiv), wenn:
für alle $\ell \rightarrow r \in P$ gilt $|\ell| \leq |r|$.
- ▶ G ist vom Typ 2 (alternativ kontextfrei), wenn:
 G ist vom Typ 1 und für alle $\ell \rightarrow r \in P$ gilt $\ell \in V$.
- ▶ G ist vom Typ 3 (alternativ regulär), wenn:
 G ist vom Typ 2 und für alle $A \rightarrow r \in P$ gilt $r = a$ oder $r = aA'$ für
 $a \in \Sigma, A' \in V$ (d.h. die rechten Seiten sind Satzformen aus $\Sigma \cup \Sigma V$).

Definition

Für $i \in \{0, 1, 2, 3\}$ nennt man eine formale Sprache $L \subseteq \Sigma^*$ vom Typ i , falls es eine Typ i -Grammatik G gibt, sodass $L(G) = L$ gilt.

Spricht man von dem Typ einer formalen Sprache, so ist meistens der größtmögliche Typ gemeint.

Beispiele für die Chomsky-Hierarchie

$G_1 = (\{S\}, \{a, b\}, \{S \rightarrow aS, S \rightarrow b\}, S)$ ist vom Typ 3 (regulär).

Beispiele für die Chomsky-Hierarchie

$G_1 = (\{S\}, \{a, b\}, \{S \rightarrow aS, S \rightarrow b\}, S)$ ist vom Typ 3 (regulär).

$G_2 = (\{E, M, Z\}, \{+, *, 1, 2, (,)\}, P, E)$ mit
 $P = \{E \rightarrow M, E \rightarrow E + M, M \rightarrow Z, M \rightarrow M * Z,$
 $Z \rightarrow 1, Z \rightarrow 2, Z \rightarrow (E)\}$ ist vom Typ 2 (kontextfrei).

Beispiele für die Chomsky-Hierarchie

$G_1 = (\{S\}, \{a, b\}, \{S \rightarrow aS, S \rightarrow b\}, S)$ ist vom Typ 3 (regulär).

$G_2 = (\{E, M, Z\}, \{+, *, 1, 2, (,)\}, P, E)$ mit
 $P = \{E \rightarrow M, E \rightarrow E + M, M \rightarrow Z, M \rightarrow M * Z,$
 $Z \rightarrow 1, Z \rightarrow 2, Z \rightarrow (E)\}$ ist vom Typ 2 (kontextfrei).

$G_3 = (\{S, B, C\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBC, S \rightarrow aBC, CB \rightarrow BC, aB \rightarrow ab,$
 $bB \rightarrow bb, bC \rightarrow bc, cC \rightarrow cc\}$ ist vom Typ 1 (kontextsensitiv).

Beispiele für die Chomsky-Hierarchie

$G_1 = (\{S\}, \{a, b\}, \{S \rightarrow aS, S \rightarrow b\}, S)$ ist vom Typ 3 (regulär).

$G_2 = (\{E, M, Z\}, \{+, *, 1, 2, (,)\}, P, E)$ mit
 $P = \{E \rightarrow M, E \rightarrow E + M, M \rightarrow Z, M \rightarrow M * Z,$
 $Z \rightarrow 1, Z \rightarrow 2, Z \rightarrow (E)\}$ ist vom Typ 2 (kontextfrei).

$G_3 = (\{S, B, C\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBC, S \rightarrow aBC, CB \rightarrow BC, aB \rightarrow ab,$
 $bB \rightarrow bb, bC \rightarrow bc, cC \rightarrow cc\}$ ist vom Typ 1 (kontextsensitiv).

$G_4 = (\{S, T, A, B, \$\}, \{a, b\}, P, S)$ mit
 $P = \{S \rightarrow \$T\$, T \rightarrow aAT, T \rightarrow bBT, T \rightarrow \epsilon, \$a \rightarrow a\$,$
 $\$b \rightarrow b\$, Aa \rightarrow aA, Ab \rightarrow bA, Ba \rightarrow aB, Bb \rightarrow bB,$
 $A\$ \rightarrow \$a, B\$ \rightarrow \$b, \$\$ \rightarrow \epsilon\}$ ist vom Typ 0.

Sonderregeln für ϵ -Produktionen

- Das leere Wort ϵ kann bisher nicht für Typ 1, 2, 3-Grammatiken erzeugt werden. Die Produktion $S \rightarrow \epsilon$ erfüllt die Typ 1-Bedingung $|S| \leq |\epsilon|$ nicht.

Sonderregeln für ϵ -Produktionen

- Das leere Wort ϵ kann bisher nicht für Typ 1, 2, 3-Grammatiken erzeugt werden. Die Produktion $S \rightarrow \epsilon$ erfüllt die Typ 1-Bedingung $|S| \leq |\epsilon|$ nicht.

Daher:

1. Sonderregel: ϵ -Produktion in Typ 1, 2, 3-Grammatiken

Eine Grammatik $G = (V, \Sigma, P, S)$ vom Typ 1, 2 oder 3 darf eine Produktion $S \rightarrow \epsilon \in P$ enthalten, vorausgesetzt, dass S auf keiner rechten Seite einer Produktion in P vorkommt.

Sonderregeln für ϵ -Produktionen

- Das leere Wort ϵ kann bisher nicht für Typ 1, 2, 3-Grammatiken erzeugt werden. Die Produktion $S \rightarrow \epsilon$ erfüllt die Typ 1-Bedingung $|S| \leq |\epsilon|$ nicht.

Daher:

1. Sonderregel: ϵ -Produktion in Typ 1, 2, 3-Grammatiken

Eine Grammatik $G = (V, \Sigma, P, S)$ vom Typ 1, 2 oder 3 darf eine Produktion $S \rightarrow \epsilon \in P$ enthalten, vorausgesetzt, dass S auf keiner rechten Seite einer Produktion in P vorkommt.

Zudem:

2. Sonderregel: ϵ -Produktionen in Typ 2, 3-Grammatiken

Eine Grammatik $G = (V, \Sigma, P, S)$ vom Typ 2 oder 3 darf Produktionen von der Form $A \rightarrow \epsilon \in P$ enthalten, wo $A \in V \setminus \{S\}$.

Begründung in der nächsten Vorlesung (nur FSK).

1c

Weitere Grammatikbegriffe sowie Eigenschaften von Sprachen

Prof. Dr. Jasmin Blanchette

Lehr- und Forschungseinheit für
Theoretische Informatik und Theorembeweisen

Stand: 10. Mai 2024

Basiert auf Folien von PD Dr. David Sabel und Dr. Jan Johannsen



Wiederholung: Definition einer Grammatik

Definition

Eine **Grammatik** ist ein 4-Tupel $G = (V, \Sigma, P, S)$ wobei:

- ▶ V ist eine endliche Menge von **Variablen** (alternativ **Nichtterminalen**)
- ▶ Σ (mit $V \cap \Sigma = \emptyset$) ist ein **Alphabet** von **Zeichen** (alternativ **Terminalen**)
- ▶ P ist eine endliche Menge von **Produktionen** (alternativ **Regeln**) von der Form $\ell \rightarrow r$ wobei $\ell \in (V \cup \Sigma)^+$ und $r \in (V \cup \Sigma)^*$
- ▶ $S \in V$ ist das **Startsymbol** (alternativ **Startvariable**).

Wiederholung: Die Chomsky-Hierarchie

Definition

Sei $G = (V, \Sigma, P, S)$ eine Grammatik.

- ▶ G ist automatisch vom Typ 0.
- ▶ G ist vom Typ 1 (alternativ kontextsensitiv), wenn:
für alle $\ell \rightarrow r \in P$ ist $|\ell| \leq |r|$.
- ▶ G ist vom Typ 2 (alternativ kontextfrei), wenn:
 G ist vom Typ 1 und für alle $\ell \rightarrow r \in P$ ist $\ell \in V$.
- ▶ G ist vom Typ 3 (alternativ regulär), wenn:
 G ist vom Typ 2 und für alle $A \rightarrow r \in P$ gilt $r = a$ oder $r = aA'$ für
 $a \in \Sigma, A' \in V$ (d.h. die rechten Seiten sind Wörter aus $\Sigma \cup \Sigma V$).

Typ 3 (regulär) vs. Typ 2 (kontextfrei)

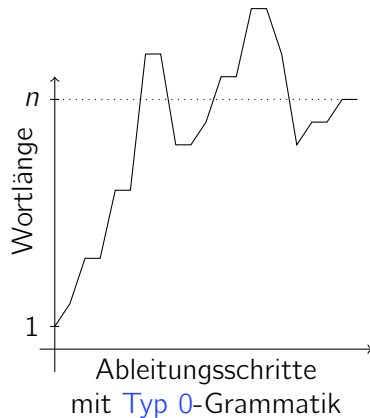
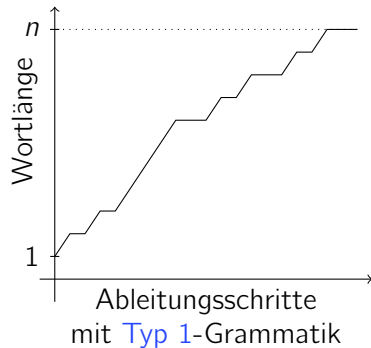
- ▶ Typ 3- und Typ 2-Produktionen sind immer von der Form $A \rightarrow r$, wobei $r \neq \varepsilon$.
- ▶ Bei Typ 3 muss zusätzlich r von der Form $a \in \Sigma$ oder $aA' \in \Sigma V$ sein.

Typ 2 (kontextfrei) vs. Typ 1 (kontextsensitiv)

- ▶ Typ 2-Produktionen $A \rightarrow r$ sind immer auf ein Vorkommen von A anwendbar.
- ▶ Typ 1-Produktionen können solche Ersetzungen auf einen Kontext einschränken. Sie erlauben Regeln von der Form $uAv \rightarrow urv$, die die Ersetzung von A durch r nur erlauben, wenn A durch u und v umrahmt ist.

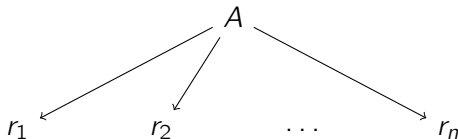
Typ 1 (kontextsensitiv) vs. Typ 0

Ableitung eines Wortes der Länge n



Wiederholung: Syntaxbäume

- ▶ Ein **Syntaxbaum** stellt dar, wie ein Wort entsteht.
- ▶ Die Anwendung von $A \rightarrow r_1 r_2 \dots r_n$ wird durch den Teilbaum



dargestellt.

Definition eines Syntaxbaums

Definition

Sei $G = (V, \Sigma, P, S)$ eine Typ 2-Grammatik und $S = w_0 \Rightarrow_G \cdots \Rightarrow_G w_n$ eine Ableitung von $w_n \in \Sigma^*$.

Der **Syntaxbaum** zur Ableitung wird wie folgt erstellt:

- ▶ Markiere die Wurzel des Baums mit S .
- ▶ Wenn $w_i \Rightarrow w_{i+1}$, $w_i = uAv$ und $w_{i+1} = urv$ (Produktion $A \rightarrow r$ verwendet), dann erzeuge im Syntaxbaum $|r|$ viele Knoten als Kinder des mit A markierten Knotens. Markiere die Kinder mit den Symbolen aus r (von links nach rechts).

Das Ergebnis des Syntaxbaums ist das Wort w_n an den Blättern, von links nach rechts gelesen.

Beispiel für ein Syntaxbaum

$G = (\{E, M, Z\}, \{+, *, 1, 2, (,)\}, P, E)$ mit

$P = \{E \rightarrow M, E \rightarrow E + M, M \rightarrow Z, M \rightarrow M * Z, Z \rightarrow 1, Z \rightarrow 2, Z \rightarrow (E)\}$

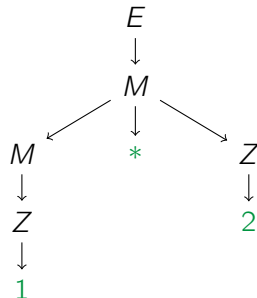
Beachte: Beide Ableitungen

$E \Rightarrow M \Rightarrow M * Z \Rightarrow Z * Z \Rightarrow 1 * Z \Rightarrow 1 * 2$

und

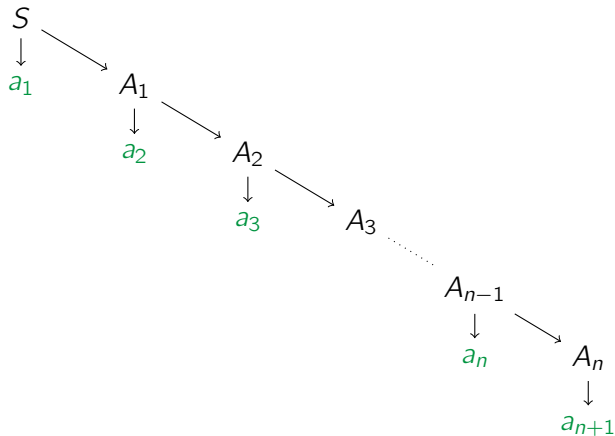
$E \Rightarrow M \Rightarrow M * Z \Rightarrow M * 2 \Rightarrow Z * 2 \Rightarrow 1 * 2$

haben denselben Syntaxbaum (rechts).



Syntaxbäume bei Typ 3-Grammatiken

Syntaxbäume bei Typ 3-Grammatiken sind immer listenartig:



Links- und Rechtsableitungen

- ▶ **Linksableitung:** Ersetze immer die linkeste Variable der Satzform.
- ▶ **Rechtsableitung:** Ersetze immer die rechteste Variable der Satzform.

Beispiele:

$$\begin{aligned} E &\Rightarrow E + M \\ &\Rightarrow M + M \\ &\Rightarrow M * Z + M \\ &\Rightarrow Z * Z + M \\ &\Rightarrow 1 * Z + M \\ &\Rightarrow 1 * 2 + M \\ &\Rightarrow 1 * 2 + Z \\ &\Rightarrow 1 * 2 + 3 \end{aligned}$$

$$\begin{aligned} E &\Rightarrow E + M \\ &\Rightarrow E + Z \\ &\Rightarrow E + 3 \\ &\Rightarrow M + 3 \\ &\Rightarrow M * Z + 3 \\ &\Rightarrow M * 2 + 3 \\ &\Rightarrow Z * 2 + 3 \\ &\Rightarrow 1 * 2 + 3 \end{aligned}$$

Satz

Sei G eine Typ 2-Grammatik und $w \in L(G)$.

Dann gibt es eine Linksableitung und eine Rechtsableitung von w .

Satz

Sei G eine Typ 2-Grammatik und $w \in L(G)$.

Dann gibt es eine Linksableitung und eine Rechtsableitung von w .

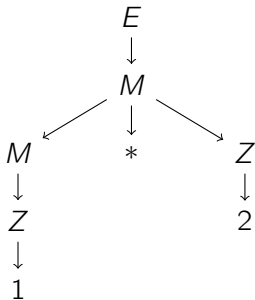
Beweis Da $w \in L(G)$, gibt es irgendeine Ableitung von w .

Konstruiere den Syntaxbaum zu dieser Ableitung.

Lies eine Links- bzw. Rechtsableitung am Syntaxbaum ab. □

Beispiel für das Ablesen einer Linksableitung

Syntaxbaum:

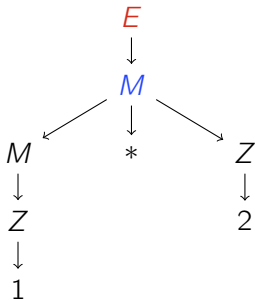


Linksableitung:

E

Beispiel für das Ablesen einer Linksableitung

Syntaxbaum:

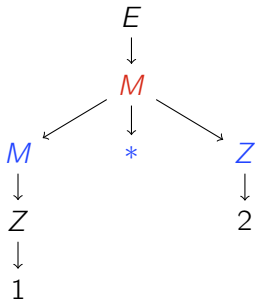


Linksableitung:

$E \Rightarrow M$

Beispiel für das Ablesen einer Linksableitung

Syntaxbaum:

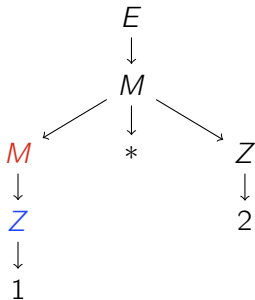


Linksableitung:

$E \Rightarrow M \Rightarrow M * Z$

Beispiel für das Ablesen einer Linksableitung

Syntaxbaum:

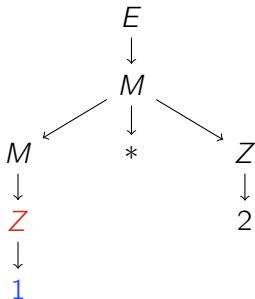


Linksableitung:

$$E \Rightarrow M \Rightarrow M * Z \Rightarrow Z * Z$$

Beispiel für das Ablesen einer Linksableitung

Syntaxbaum:

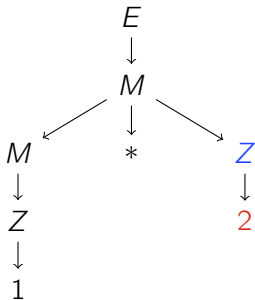


Linksableitung:

$E \Rightarrow M \Rightarrow M * Z \Rightarrow \textcolor{red}{Z} * Z \Rightarrow \textcolor{blue}{1} * Z$

Beispiel für das Ablesen einer Linksableitung

Syntaxbaum:



Linksableitung:

$$E \Rightarrow M \Rightarrow M * Z \Rightarrow Z * Z \Rightarrow 1 * Z \Rightarrow 1 * 2$$

Definition

Für Typ 2-Grammatiken erlauben wir abkürzende Schreibweise für die Menge der Produktionen P :

1. $A \rightarrow w_1 \mid w_2 \mid \cdots \mid w_n$ steht für $A \rightarrow w_1, A \rightarrow w_2, \dots, A \rightarrow w_n$.
2. $A \rightarrow u[v]w$ steht für die beiden Produktionen $A \rightarrow uvw$ und $A \rightarrow uw$ (d.h. $[v]$ meint, dass v optional ist).
3. $A \rightarrow u\{v\}w$ steht für $A \rightarrow uw$ oder $A \rightarrow uBw$ mit $B \rightarrow v \mid vB$ (d.h. $\{v\}$ meint, dass v beliebig oft wiederholt werden kann).

Grammatiken, die diese Notation verwenden, nennen wir Grammatiken in **erweiterter Backus-Naur-Form** (EBNF).

Anwendungen von kontextfreien Grammatiken

- ▶ Kontextfreie Grammatiken werden zur **syntaktischen Analyse** von Programmiersprachen und Domain Specific Languages verwendet.
- ▶ Tools wie **yacc** (für C/C++), **ANTLR** (für Java) und **PLY** (für Python) generieren syntaktische Analyser („Parser“) aus Grammatiken.
Beispiel für eine ANTLR-Grammatik (Quelle: www.antlr.org):

```
prog:    (expr NEWLINE)* ;
expr:    expr ('*' | '/') expr
        | expr ('+' | '-') expr
        | INT
        | '(' expr ')';
```

Anwendungen von kontextfreien Grammatiken

- ▶ Kontextfreie Grammatiken werden zur **syntaktischen Analyse** von Programmiersprachen und Domain Specific Languages verwendet.
- ▶ Tools wie **yacc** (für C/C++), **ANTLR** (für Java) und **PLY** (für Python) generieren syntaktische Analyser („Parser“) aus Grammatiken.
Beispiel für eine ANTLR-Grammatik (Quelle: www.antlr.org):

```
prog:    (expr NEWLINE)* ;
expr:    expr ('*' | '/') expr
        | expr ('+' | '-') expr
        | INT
        | '(' expr ')';
```

- ▶ Viele Fragestellungen sind jedoch kontextsensitiv oder Typ 0.
Praktisches Vorgehen: Nutze Typ 2-Sprache und Nebenbedingungen (z.B. Syntax als kontextfreie Grammatik und Nebenbedingungen, die prüfen, dass alle Variablen deklariert wurden).

Chomsky-Hierarchie: Teilmengenbeziehungen

Aus der Definition der Typ i -Sprachen folgt:

$$\text{Typ 3-Sprachen} \subseteq \text{Typ 2-Sprachen} \subseteq \text{Typ 1-Sprachen} \subseteq \text{Typ 0-Sprachen}$$

Chomsky-Hierarchie: Teilmengenbeziehungen

Aus der Definition der Typ i -Sprachen folgt:

$$\text{Typ 3-Sprachen} \subseteq \text{Typ 2-Sprachen} \subseteq \text{Typ 1-Sprachen} \subseteq \text{Typ 0-Sprachen}$$

Es gilt sogar:

$$\text{Typ 3-Sprachen} \subset \text{Typ 2-Sprachen} \subset \text{Typ 1-Sprachen} \subset \text{Typ 0-Sprachen}$$

Chomsky-Hierarchie: Teilmengenbeziehungen

Aus der Definition der Typ i -Sprachen folgt:

$$\text{Typ 3-Sprachen} \subseteq \text{Typ 2-Sprachen} \subseteq \text{Typ 1-Sprachen} \subseteq \text{Typ 0-Sprachen}$$

Es gilt sogar:

$$\text{Typ 3-Sprachen} \subset \text{Typ 2-Sprachen} \subset \text{Typ 1-Sprachen} \subset \text{Typ 0-Sprachen}$$

Trennende Beispiele sind:

- ▶ $\{a^n b^n \mid n \in \mathbb{N}_{>0}\}$ ist von Typ 2, aber nicht von Typ 3.
- ▶ $\{a^n b^n c^n \mid n \in \mathbb{N}_{>0}\}$ ist von Typ 1, aber nicht von Typ 2.
- ▶ Das sogenannte Halteproblem ist von Typ 0, aber nicht von Typ 1.

(Beweise folgen im Laufe der Veranstaltung.)

Beachte: Es gibt auch Sprachen, die nicht Typ 0 sind.

Das Komplement vom Halteproblem ist eine solche Sprache.

Abgeschlossenheit von Sprachen

Eine Klasse $\mathcal{K}$ von Sprachen (d.h. eine Menge von Mengen) ist abgeschlossen bezüglich

- ▶ Vereinigung g.d.w. aus $L_1, L_2 \in \mathcal{K}$ folgt stets $L_1 \cup L_2 \in \mathcal{K}$
- ▶ Schnittbildung g.d.w. aus $L_1, L_2 \in \mathcal{K}$ folgt stets $L_1 \cap L_2 \in \mathcal{K}$
- ▶ Komplementbildung g.d.w. aus $L \in \mathcal{K}$ folgt stets $\bar{L} \in \mathcal{K}$
- ▶ Produktbildung g.d.w. aus $L_1, L_2 \in \mathcal{K}$ folgt stets $L_1 L_2 \in \mathcal{K}$.

Wir werden im Laufe der Veranstaltung untersuchen, ob die Typ i -Sprachen abgeschlossen bezüglich obiger Operationen sind.

Abgeschlossenheit: Eigenschaften

Satz

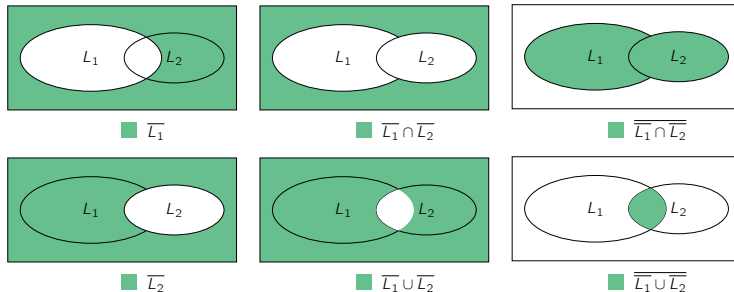
Sei die Klasse von Sprachen $\mathcal{K}$ abgeschlossen bezüglich Komplementbildung. Dann ist $\mathcal{K}$ abgeschlossen bezüglich Schnittbildung g.d.w. $\mathcal{K}$ abgeschlossen bezüglich Vereinigung ist.

Abgeschlossenheit: Eigenschaften

Satz

Sei die Klasse von Sprachen $\mathcal{K}$ abgeschlossen bezüglich Komplementbildung. Dann ist $\mathcal{K}$ abgeschlossen bezüglich Schnittbildung g.d.w. $\mathcal{K}$ abgeschlossen bezüglich Vereinigung ist.

Beweis Das gilt, da $L_1 \cup L_2 = \overline{\overline{L_1} \cap \overline{L_2}}$ und $L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}$. □



Definition

Eine Sprache L ist **entscheidbar**, wenn es einen Algorithmus gibt, der bei Eingabe eines Wortes w in endlicher Zeit feststellt, ob $w \in L$ gilt oder nicht.

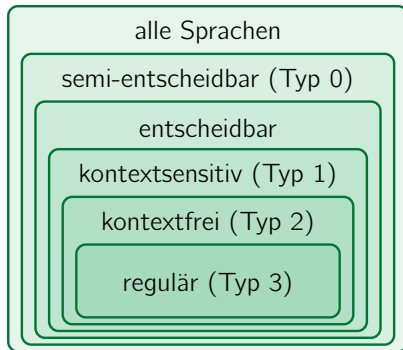
Eigenschaften der Typ i -Sprachen

- ▶ Alle Typ 1, 2, 3-Sprachen sind **entscheidbar**.
- ▶ Es gibt Typ 0-Sprachen, die **nicht entscheidbar** sind.
- ▶ Alle Typ 0-Sprachen sind **semi-entscheidbar** (alternativ **rekursiv aufzählbar**):
Für jede Typ 0-Sprache L gibt es einen Algorithmus, der bei Eingabe eines Wortes $w \in L$ in endlicher Zeit feststellt, dass $w \in L$ gilt, und bei einem Wort $w \notin L$ entweder feststellt, dass $w \notin L$ gilt, **oder nicht terminiert**.

Eigenschaften der Typ i -Sprachen

- ▶ Alle Typ 1, 2, 3-Sprachen sind **entscheidbar**.
- ▶ Es gibt Typ 0-Sprachen, die **nicht entscheidbar** sind.
- ▶ Alle Typ 0-Sprachen sind **semi-entscheidbar** (alternativ **rekursiv aufzählbar**):
Für jede Typ 0-Sprache L gibt es einen Algorithmus, der bei Eingabe eines Wortes $w \in L$ in endlicher Zeit feststellt, dass $w \in L$ gilt, und bei einem Wort $w \notin L$ entweder feststellt, dass $w \notin L$ gilt, **oder nicht terminiert**.
- ▶ Es gibt auch Sprachen, die **nicht semi-entscheidbar** sind:
 - ▶ Die Menge der Typ 0-Grammatiken ist abzählbar (jede Grammatik hat eine endliche Beschreibung, d.h. Grammatiken können der Größe nach aufgezählt werden).
 - ▶ Die Menge aller Sprachen $= \mathcal{P}(\Sigma^*)$ ist überabzählbar.
 - ▶ Wir betrachten mit den Typ i -Grammatiken einen sehr kleinen Teil aller Sprachen.

Übersicht über die Sprachen



Entscheidungsprobleme

- ▶ Das **Wortproblem** für Typ i -Grammatiken ist die Frage, ob für eine gegebene Typ i -Grammatik $G = (V, \Sigma, P, S)$ und ein Wort $w \in \Sigma^*$ $w \in L(G)$ gilt oder nicht.
- ▶ Das **Leerheitsproblem** für Typ i -Grammatiken ist die Frage, ob für eine gegebene Typ i -Grammatik G die Gleichheit $L(G) = \emptyset$ gilt.
- ▶ Das **Endlichkeitsproblem** für Typ i -Grammatiken ist die Frage, ob für eine gegebene Typ i -Grammatik G die Ungleichheit $|L(G)| < \infty$ gilt.
- ▶ Das **Schnittproblem** für Typ i -Grammatiken ist die Frage, ob für gegebene Typ i -Grammatiken G_1, G_2 gilt: $L(G_1) \cap L(G_2) = \emptyset$.
- ▶ Das **Äquivalenzproblem** für Typ i -Grammatiken ist, die Frage, ob für gegebene Typ i -Grammatiken G_1, G_2 gilt: $L(G_1) = L(G_2)$.

Probleme vs. Sprachen

- ▶ (Entscheidungs-)Problem: Funktion von Σ^* nach $\{\text{ja}, \text{nein}\}$
- ▶ (Formale) Sprache: Menge $\subseteq \Sigma^*$
- ▶ Die beiden Begriffe werden synonym verwendet.
 - ▶ ja (als Antwort auf ein Problem) = Element (einer Sprache)
 - ▶ nein (als Antwort auf ein Problem) = nicht Element (einer Sprache)

Probleme vs. Sprachen

- ▶ (Entscheidungs-)Problem: Funktion von Σ^* nach $\{\text{ja}, \text{nein}\}$
- ▶ (Formale) Sprache: Menge $\subseteq \Sigma^*$
- ▶ Die beiden Begriffe werden synonym verwendet.
 - ▶ ja (als Antwort auf ein Problem) = Element (einer Sprache)
 - ▶ nein (als Antwort auf ein Problem) = nicht Element (einer Sprache)
- ▶ Beispiele:
 - ▶ Das Primzahlproblem ist die Sprache L ,
die alle (in Dezimalnotation kodierten) Primzahlen enthält.
D.h. $L = \{2, 3, 5, 7, 11, \dots\}$.

Probleme vs. Sprachen

- ▶ (Entscheidungs-)Problem: Funktion von Σ^* nach $\{\text{ja}, \text{nein}\}$
- ▶ (Formale) Sprache: Menge $\subseteq \Sigma^*$
- ▶ Die beiden Begriffe werden synonym verwendet.
 - ▶ ja (als Antwort auf ein Problem) = Element (einer Sprache)
 - ▶ nein (als Antwort auf ein Problem) = nicht Element (einer Sprache)
- ▶ Beispiele:
 - ▶ Das Primzahlproblem ist die Sprache L , die alle (in Dezimalnotation kodierten) Primzahlen enthält. D.h. $L = \{2, 3, 5, 7, 11, \dots\}$.
 - ▶ Das Leerheitsproblem für Typ i -Grammatiken ist die Sprache, die alle (als Wörter kodierten) Typ i -Grammatiken G enthält, für die $L(G) = \emptyset$ gilt.

2a

**Grammatikbeispiele, Mehrdeutigkeit und
Entfernen von ϵ -Produktionen**

Prof. Dr. Jasmin Blanchette

Lehr- und Forschungseinheit für
Theoretische Informatik und Theorembeweisen

Stand: 10. Mai 2024

Basiert auf Folien von PD Dr. David Sabel



Beispiel für eine kontextsensitive Grammatik

$G = (\{S, B, C\}, \{a, b, c\}, P, S)$ mit

$P = \{S \rightarrow aSBC, S \rightarrow aBC, CB \rightarrow BC, aB \rightarrow ab, bB \rightarrow bb, bC \rightarrow bc, cC \rightarrow cc\}$

Beispiel für eine kontextsensitive Grammatik

$G = (\{S, B, C\}, \{a, b, c\}, P, S)$ mit

$P = \{S \rightarrow aSBC, S \rightarrow aBC, CB \rightarrow BC, aB \rightarrow ab, bB \rightarrow bb, bC \rightarrow bc, cC \rightarrow cc\}$

Ableitung:

$S \Rightarrow aSBC$

Beispiel für eine kontextsensitive Grammatik

$G = (\{S, B, C\}, \{a, b, c\}, P, S)$ mit

$P = \{S \rightarrow aSBC, S \rightarrow aBC, CB \rightarrow BC, aB \rightarrow ab, bB \rightarrow bb, bC \rightarrow bc, cC \rightarrow cc\}$

Ableitung:

$S \Rightarrow aSBC \Rightarrow aaSBCBC$

Beispiel für eine kontextsensitive Grammatik

$G = (\{S, B, C\}, \{a, b, c\}, P, S)$ mit

$P = \{\textcolor{red}{S} \rightarrow \textcolor{blue}{aSBC}, S \rightarrow aBC, CB \rightarrow BC, aB \rightarrow ab, bB \rightarrow bb, bC \rightarrow bc, cC \rightarrow cc\}$

Ableitung:

$S \Rightarrow aSBC \Rightarrow aa\textcolor{red}{S}BCBC \Rightarrow aa\textcolor{blue}{aSBC}BCBC$

Beispiel für eine kontextsensitive Grammatik

$G = (\{S, B, C\}, \{a, b, c\}, P, S)$ mit

$P = \{S \rightarrow aSBC, \textcolor{red}{S} \rightarrow \textcolor{blue}{a}BC, CB \rightarrow BC, aB \rightarrow ab, bB \rightarrow bb, bC \rightarrow bc, cC \rightarrow cc\}$

Ableitung:

$S \Rightarrow aSBC \Rightarrow aaSBCBC \Rightarrow aaa\textcolor{red}{S}BCBCBC \Rightarrow aaa\textcolor{blue}{a}BCBCBC$

Beispiel für eine kontextsensitive Grammatik

$G = (\{S, B, C\}, \{a, b, c\}, P, S)$ mit

$P = \{S \rightarrow aSBC, S \rightarrow aBC, CB \rightarrow BC, aB \rightarrow ab, bB \rightarrow bb, bC \rightarrow bc, cC \rightarrow cc\}$

Ableitung:

$S \Rightarrow aSBC \Rightarrow aaSBCBC \Rightarrow aaaSBCBCBC \Rightarrow aaa aB C B C B C B C$
 $\Rightarrow aaa ab C B C B C B C$

Beispiel für eine kontextsensitive Grammatik

$G = (\{S, B, C\}, \{a, b, c\}, P, S)$ mit

$P = \{S \rightarrow aSBC, S \rightarrow aBC, \textcolor{red}{CB} \rightarrow \textcolor{blue}{BC}, aB \rightarrow ab, bB \rightarrow bb, bC \rightarrow bc, cC \rightarrow cc\}$

Ableitung:

$S \Rightarrow aSBC \Rightarrow aaSBCBC \Rightarrow aaaSBCBCBC \Rightarrow aaaaBCBCBCBC$
 $\Rightarrow aaaab\textcolor{red}{CB}CBCBC \Rightarrow aaaab\textcolor{blue}{BC}CBCBC$

Beispiel für eine kontextsensitive Grammatik

$G = (\{S, B, C\}, \{a, b, c\}, P, S)$ mit

$P = \{S \rightarrow aSBC, S \rightarrow aBC, CB \rightarrow BC, aB \rightarrow ab, bB \rightarrow bb, bC \rightarrow bc, cC \rightarrow cc\}$

Ableitung:

$S \Rightarrow aSBC \Rightarrow aaSBCBC \Rightarrow aaaSBCBCBC \Rightarrow aaaaBCBCBCBC$
 $\Rightarrow aaaabCBCBCBC \Rightarrow aaaa**b**CCBCBC \Rightarrow aaaa**bb**CCBCBC$

Beispiel für eine kontextsensitive Grammatik

$G = (\{S, B, C\}, \{a, b, c\}, P, S)$ mit

$P = \{S \rightarrow aSBC, S \rightarrow aBC, \textcolor{red}{CB} \rightarrow \textcolor{blue}{BC}, aB \rightarrow ab, bB \rightarrow bb, bC \rightarrow bc, cC \rightarrow cc\}$

Ableitung:

$$\begin{aligned} S &\Rightarrow aSBC \Rightarrow aaSBCBC \Rightarrow aaaSBCBCBC \Rightarrow aaaaBCBCBCBC \\ &\Rightarrow aaaabCBCBCBC \Rightarrow aaaabBCCBCBC \Rightarrow aaaabbC\textcolor{red}{CB}CBC \\ &\Rightarrow aaaabbC\textcolor{blue}{BC}CBC \end{aligned}$$

Beispiel für eine kontextsensitive Grammatik

$G = (\{S, B, C\}, \{a, b, c\}, P, S)$ mit

$P = \{S \rightarrow aSBC, S \rightarrow aBC, \textcolor{red}{CB} \rightarrow \textcolor{blue}{BC}, aB \rightarrow ab, bB \rightarrow bb, bC \rightarrow bc, cC \rightarrow cc\}$

Ableitung:

$$\begin{aligned} S &\Rightarrow aSBC \Rightarrow aaSBCBC \Rightarrow aaaSBCBCBC \Rightarrow aaaaBCBCBCBC \\ &\Rightarrow aaaabCBCBCBC \Rightarrow aaaabBCCBCBC \Rightarrow aaaabbCCBCBC \\ &\Rightarrow aaaabb\textcolor{red}{CB}CCBC \Rightarrow aaaabb\textcolor{blue}{BC}CCBC \end{aligned}$$

Beispiel für eine kontextsensitive Grammatik

$G = (\{S, B, C\}, \{a, b, c\}, P, S)$ mit

$P = \{S \rightarrow aSBC, S \rightarrow aBC, \textcolor{red}{CB} \rightarrow \textcolor{blue}{BC}, aB \rightarrow ab, bB \rightarrow bb, bC \rightarrow bc, cC \rightarrow cc\}$

Ableitung:

$$\begin{aligned} S &\Rightarrow aSBC \Rightarrow aaSBCBC \Rightarrow aaaSBCBCBC \Rightarrow aaaaBCBCBCBC \\ &\Rightarrow aaaabCBCBCBC \Rightarrow aaaabBCCBCBC \Rightarrow aaaabbCCBCBC \\ &\Rightarrow aaaabbCBCCBC \Rightarrow aaaabbBCC\textcolor{red}{CB}C \Rightarrow aaaabbBCC\textcolor{blue}{BCC} \end{aligned}$$

Beispiel für eine kontextsensitive Grammatik

$G = (\{S, B, C\}, \{a, b, c\}, P, S)$ mit

$P = \{S \rightarrow aSBC, S \rightarrow aBC, \textcolor{red}{CB} \rightarrow \textcolor{blue}{BC}, aB \rightarrow ab, bB \rightarrow bb, bC \rightarrow bc, cC \rightarrow cc\}$

Ableitung:

$$\begin{aligned} S &\Rightarrow aSBC \Rightarrow aaSBCBC \Rightarrow aaaSBCBCBC \Rightarrow aaaaBCBCBCBC \\ &\Rightarrow aaaabCBCBCBC \Rightarrow aaaabBCCBCBC \Rightarrow aaaabbCCBCBC \\ &\Rightarrow aaaabbCBCCBC \Rightarrow aaaabbBCCCBC \Rightarrow aaaabbBC\textcolor{red}{B}CC \\ &\Rightarrow aaaabbBC\textcolor{blue}{BC}CC \end{aligned}$$

Beispiel für eine kontextsensitive Grammatik

$G = (\{S, B, C\}, \{a, b, c\}, P, S)$ mit

$P = \{S \rightarrow aSBC, S \rightarrow aBC, CB \rightarrow BC, aB \rightarrow ab, bB \rightarrow bb, bC \rightarrow bc, cC \rightarrow cc\}$

Ableitung:

$$\begin{aligned} S &\Rightarrow aSBC \Rightarrow aaSBCBC \Rightarrow aaaSBCBCBC \Rightarrow aaaaBCBCBCBC \\ &\Rightarrow aaaabCBCBCBC \Rightarrow aaaabBCCBCBC \Rightarrow aaaabbCCBCBC \\ &\Rightarrow aaaabbCBCCBC \Rightarrow aaaabbBCCCBC \Rightarrow aaaabbBCCBCC \\ &\Rightarrow aaaabbB\textcolor{red}{C}\textcolor{red}{B}CCC \Rightarrow aaaabbB\textcolor{blue}{C}\textcolor{blue}{C}CCC \end{aligned}$$

Beispiel für eine kontextsensitive Grammatik

$G = (\{S, B, C\}, \{a, b, c\}, P, S)$ mit

$P = \{S \rightarrow aSBC, S \rightarrow aBC, CB \rightarrow BC, aB \rightarrow ab, bB \rightarrow bb, bC \rightarrow bc, cC \rightarrow cc\}$

Ableitung:

$$\begin{aligned} S &\Rightarrow aSBC \Rightarrow aaSBCBC \Rightarrow aaaSBCBCBC \Rightarrow aaaaBCBCBCBC \\ &\Rightarrow aaaabCBCBCBC \Rightarrow aaaabBCCBCBC \Rightarrow aaaabbCCBCBC \\ &\Rightarrow aaaabbCBCCBC \Rightarrow aaaabbBCCCBBC \Rightarrow aaaabbBCCBCC \\ &\Rightarrow aaaabbBCBCCC \Rightarrow aaaab**b**BCCCC \Rightarrow aaaab**bb**BCCCC \end{aligned}$$

Beispiel für eine kontextsensitive Grammatik

$G = (\{S, B, C\}, \{a, b, c\}, P, S)$ mit

$P = \{S \rightarrow aSBC, S \rightarrow aBC, CB \rightarrow BC, aB \rightarrow ab, \textcolor{red}{bB} \rightarrow \textcolor{blue}{bb}, bC \rightarrow bc, cC \rightarrow cc\}$

Ableitung:

$$\begin{aligned} S &\Rightarrow aSBC \Rightarrow aaSBCBC \Rightarrow aaaSBCBCBC \Rightarrow aaaaBCBCBCBC \\ &\Rightarrow aaaabCBCBCBC \Rightarrow aaaabBCCBCBC \Rightarrow aaaabbCCBCBC \\ &\Rightarrow aaaabbCBCCBC \Rightarrow aaaabbBCCCBBC \Rightarrow aaaabbBCCBCC \\ &\Rightarrow aaaabbBCBCCC \Rightarrow aaaabbBBCCCC \Rightarrow aaaabb\textcolor{red}{bB}CCCC \\ &\Rightarrow aaaabb\textcolor{blue}{bb}CCCC \end{aligned}$$

Beispiel für eine kontextsensitive Grammatik

$G = (\{S, B, C\}, \{a, b, c\}, P, S)$ mit

$P = \{S \rightarrow aSBC, S \rightarrow aBC, CB \rightarrow BC, aB \rightarrow ab, bB \rightarrow bb, bC \rightarrow bc, cC \rightarrow cc\}$

Ableitung:

$$\begin{aligned} S &\Rightarrow aSBC \Rightarrow aaSBCBC \Rightarrow aaaSBCBCBC \Rightarrow aaaaBCBCBCBC \\ &\Rightarrow aaaabCBCBCBC \Rightarrow aaaabBCCBCBC \Rightarrow aaaabbCCBCBC \\ &\Rightarrow aaaabbCBCCBC \Rightarrow aaaabbBCCCBC \Rightarrow aaaabbBCCBCC \\ &\Rightarrow aaaabbBCBCCC \Rightarrow aaaabbBBCCCC \Rightarrow aaaabbbbBCCCC \\ &\Rightarrow aaaabbbb b C C C C \Rightarrow aaaabbbb bc C C C \end{aligned}$$

Beispiel für eine kontextsensitive Grammatik

$G = (\{S, B, C\}, \{a, b, c\}, P, S)$ mit

$P = \{S \rightarrow aSBC, S \rightarrow aBC, CB \rightarrow BC, aB \rightarrow ab, bB \rightarrow bb, bC \rightarrow bc, cC \rightarrow cc\}$

Ableitung:

$$\begin{aligned} S &\Rightarrow aSBC \Rightarrow aaSBCBC \Rightarrow aaaSBCBCBC \Rightarrow aaaaBCBCBCBC \\ &\Rightarrow aaaabCBCBCBC \Rightarrow aaaabBCCBCBC \Rightarrow aaaabbCCBCBC \\ &\Rightarrow aaaabbCBCCBC \Rightarrow aaaabbBCCCBBC \Rightarrow aaaabbBCCBCC \\ &\Rightarrow aaaabbBCBCCC \Rightarrow aaaabbBBCCCC \Rightarrow aaaabbbbBCCCC \\ &\Rightarrow aaaabbbbCCCC \Rightarrow aaaabbbb c C C C \Rightarrow aaaabbbb cc C C \end{aligned}$$

Beispiel für eine kontextsensitive Grammatik

$G = (\{S, B, C\}, \{a, b, c\}, P, S)$ mit

$P = \{S \rightarrow aSBC, S \rightarrow aBC, CB \rightarrow BC, aB \rightarrow ab, bB \rightarrow bb, bC \rightarrow bc, cC \rightarrow cc\}$

Ableitung:

$$\begin{aligned} S &\Rightarrow aSBC \Rightarrow aaSBCBC \Rightarrow aaaSBCBCBC \Rightarrow aaaaBCBCBCBC \\ &\Rightarrow aaaabCBCBCBC \Rightarrow aaaabBCCBCBC \Rightarrow aaaabbCCBCBC \\ &\Rightarrow aaaabbCBCCBC \Rightarrow aaaabbBCCCBBC \Rightarrow aaaabbBCCBCC \\ &\Rightarrow aaaabbBCBCCC \Rightarrow aaaabbBBCCCC \Rightarrow aaaabbbbBCCCC \\ &\Rightarrow aaaabbbbbCCCC \Rightarrow aaaabbbbcbCCC \Rightarrow aaaabbbbcbccC \\ &\Rightarrow aaaabbbbcbccC \end{aligned}$$

Beispiel für eine kontextsensitive Grammatik

$G = (\{S, B, C\}, \{a, b, c\}, P, S)$ mit

$P = \{S \rightarrow aSBC, S \rightarrow aBC, CB \rightarrow BC, aB \rightarrow ab, bB \rightarrow bb, bC \rightarrow bc, cC \rightarrow cc\}$

Ableitung:

$$\begin{aligned} S &\Rightarrow aSBC \Rightarrow aaSBCBC \Rightarrow aaaSBCBCBC \Rightarrow aaaaBCBCBCBC \\ &\Rightarrow aaaabCBCBCBC \Rightarrow aaaabBCCBCBC \Rightarrow aaaabbCCBCBC \\ &\Rightarrow aaaabbCBCCBC \Rightarrow aaaabbBCCCBBC \Rightarrow aaaabbBCCBCC \\ &\Rightarrow aaaabbBCBCCC \Rightarrow aaaabbBBCCCC \Rightarrow aaaabbbbBCCCC \\ &\Rightarrow aaaabbbbbCCCC \Rightarrow aaaabbbbcbCCC \Rightarrow aaaabbbbccCC \\ &\Rightarrow aaaabbbbcc cC \Rightarrow aaaabbbbcc cc \end{aligned}$$

Beispiel für eine kontextsensitive Grammatik

$G = (\{S, B, C\}, \{a, b, c\}, P, S)$ mit

$P = \{S \rightarrow aSBC, S \rightarrow aBC, CB \rightarrow BC, aB \rightarrow ab, bB \rightarrow bb, bC \rightarrow bc, cC \rightarrow cc\}$

Ableitung:

$$\begin{aligned} S &\Rightarrow aSBC \Rightarrow aaSBCBC \Rightarrow aaaSBCBCBC \Rightarrow aaaaBCBCBCBC \\ &\Rightarrow aaaabCBCBCBC \Rightarrow aaaabBCCBCBC \Rightarrow aaaabbCCBCBC \\ &\Rightarrow aaaabbCBCCBC \Rightarrow aaaabbBCCCBBC \Rightarrow aaaabbBCCBCC \\ &\Rightarrow aaaabbBCBCCC \Rightarrow aaaabbBBCCCC \Rightarrow aaaabbbbBCCCC \\ &\Rightarrow aaaabbbbbCCCC \Rightarrow aaaabbbbcbCCC \Rightarrow aaaabbbbccCC \\ &\Rightarrow aaaabbbbccccC \Rightarrow aaaabbbbcccccc \end{aligned}$$

Steckengebliebene Folge von Ableitungsschritten:

$S \Rightarrow aSBC \Rightarrow aaBCBC \Rightarrow aabCBC \Rightarrow aabcBC$

Beispiel für eine kontextsensitive Grammatik

$G = (\{S, B, C\}, \{a, b, c\}, P, S)$ mit

$P = \{S \rightarrow aSBC, S \rightarrow aBC, CB \rightarrow BC, aB \rightarrow ab, bB \rightarrow bb, bC \rightarrow bc, cC \rightarrow cc\}$

Ableitung:

$$\begin{aligned} S &\Rightarrow aSBC \Rightarrow aaSBCBC \Rightarrow aaaSBCBCBC \Rightarrow aaaaBCBCBCBC \\ &\Rightarrow aaaabCBCBCBC \Rightarrow aaaabBCCBCBC \Rightarrow aaaabbCCBCBC \\ &\Rightarrow aaaabbCBCCBC \Rightarrow aaaabbBCCCBBC \Rightarrow aaaabbBCCBCC \\ &\Rightarrow aaaabbBCBCCC \Rightarrow aaaabbBBCCCC \Rightarrow aaaabbbbBCCCC \\ &\Rightarrow aaaabbbbbCCCC \Rightarrow aaaabbbbcbCCC \Rightarrow aaaabbbbccCC \\ &\Rightarrow aaaabbbbccccC \Rightarrow aaaabbbbcccccc \end{aligned}$$

Steckengebliebene Folge von Ableitungsschritten:

$S \Rightarrow aSBC \Rightarrow aaBCBC \Rightarrow aabCBC \Rightarrow aabcBC$

$L(G) = ?$

Beispiel für eine kontextsensitive Grammatik

$G = (\{S, B, C\}, \{a, b, c\}, P, S)$ mit

$P = \{S \rightarrow aSBC, S \rightarrow aBC, CB \rightarrow BC, aB \rightarrow ab, bB \rightarrow bb, bC \rightarrow bc, cC \rightarrow cc\}$

Ableitung:

$$\begin{aligned} S &\Rightarrow aSBC \Rightarrow aaSBCBC \Rightarrow aaaSBCBCBC \Rightarrow aaaaBCBCBCBC \\ &\Rightarrow aaaabCBCBCBC \Rightarrow aaaabBCCBCBC \Rightarrow aaaabbCCBCBC \\ &\Rightarrow aaaabbCBCCBC \Rightarrow aaaabbBCCCBBC \Rightarrow aaaabbBCCBCC \\ &\Rightarrow aaaabbBCBCCC \Rightarrow aaaabbBBCCCC \Rightarrow aaaabbbbBCCCC \\ &\Rightarrow aaaabbbbbCCCC \Rightarrow aaaabbbbcbCCC \Rightarrow aaaabbbbccCC \\ &\Rightarrow aaaabbbbccccC \Rightarrow aaaabbbbcccccc \end{aligned}$$

Steckengebliebene Folge von Ableitungsschritten:

$S \Rightarrow aSBC \Rightarrow aaBCBC \Rightarrow aabCBC \Rightarrow aabcBC$

$L(G) = \{a^n b^n c^n \mid n \in \mathbb{N}_{>0}\}$

Beispiel für eine kontextsensitive Grammatik

Satz

Für $G = (\{S, B, C\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBC, S \rightarrow aBC, CB \rightarrow BC, aB \rightarrow ab, bB \rightarrow bb, bC \rightarrow bc, cC \rightarrow cc\}$
gilt $L(G) = \{a^n b^n c^n \mid n \in \mathbb{N}_{>0}\}$.

Beispiel für eine kontextsensitive Grammatik

Satz

Für $G = (\{S, B, C\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBC, S \rightarrow aBC, CB \rightarrow BC, aB \rightarrow ab, bB \rightarrow bb, bC \rightarrow bc, cC \rightarrow cc\}$
gilt $L(G) = \{a^n b^n c^n \mid n \in \mathbb{N}_{>0}\}$.

Beweis

⊇ Wir zeigen, dass $a^n b^n c^n \in L(G)$ für alle $n \in \mathbb{N}_{>0}$.

Beispiel für eine kontextsensitive Grammatik

Satz

Für $G = (\{S, B, C\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBC, S \rightarrow aBC, CB \rightarrow BC, aB \rightarrow ab, bB \rightarrow bb, bC \rightarrow bc, cC \rightarrow cc\}$
gilt $L(G) = \{a^n b^n c^n \mid n \in \mathbb{N}_{>0}\}$.

Beweis

- ⊇ Wir zeigen, dass $a^n b^n c^n \in L(G)$ für alle $n \in \mathbb{N}_{>0}$.
- ▶ Wende $n - 1$ mal $S \rightarrow aSBC$ und dann einmal $S \rightarrow aBC$ an:
 $S \Rightarrow^* a^{n-1} S (BC)^{n-1} \Rightarrow a^n (BC)^n$.

Beispiel für eine kontextsensitive Grammatik

Satz

Für $G = (\{S, B, C\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBC, S \rightarrow aBC, CB \rightarrow BC, aB \rightarrow ab, bB \rightarrow bb, bC \rightarrow bc, cC \rightarrow cc\}$
gilt $L(G) = \{a^n b^n c^n \mid n \in \mathbb{N}_{>0}\}$.

Beweis

- ⊇ Wir zeigen, dass $a^n b^n c^n \in L(G)$ für alle $n \in \mathbb{N}_{>0}$.
- ▶ Wende $n - 1$ mal $S \rightarrow aSBC$ und dann einmal $S \rightarrow aBC$ an:
 $S \Rightarrow^* a^{n-1} S (BC)^{n-1} \Rightarrow a^n (BC)^n$.
 - ▶ Wende $CB \rightarrow BC$ solange an, bis es kein Teilwort CB mehr gibt:
 $a^n (BC)^n \Rightarrow^* a^n B^n C^n$.

Beispiel für eine kontextsensitive Grammatik

Satz

Für $G = (\{S, B, C\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBC, S \rightarrow aBC, CB \rightarrow BC, aB \rightarrow ab, bB \rightarrow bb, bC \rightarrow bc, cC \rightarrow cc\}$
gilt $L(G) = \{a^n b^n c^n \mid n \in \mathbb{N}_{>0}\}$.

Beweis

- ⊇ Wir zeigen, dass $a^n b^n c^n \in L(G)$ für alle $n \in \mathbb{N}_{>0}$.
- ▶ Wende $n - 1$ mal $S \rightarrow aSBC$ und dann einmal $S \rightarrow aBC$ an:
 $S \Rightarrow^* a^{n-1} S (BC)^{n-1} \Rightarrow a^n (BC)^n$.
 - ▶ Wende $CB \rightarrow BC$ solange an, bis es kein Teilwort CB mehr gibt:
 $a^n (BC)^n \Rightarrow^* a^n B^n C^n$.
 - ▶ Wende $aB \rightarrow ab$ und anschließend $n - 1$ mal $bB \rightarrow bb$ an:
 $a^n B^n C^n \Rightarrow a^n b B^{n-1} C^n \Rightarrow^* a^n b^n C^n$.

Beispiel für eine kontextsensitive Grammatik

Satz

Für $G = (\{S, B, C\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBC, S \rightarrow aBC, CB \rightarrow BC, aB \rightarrow ab, bB \rightarrow bb, bC \rightarrow bc, cC \rightarrow cc\}$
gilt $L(G) = \{a^n b^n c^n \mid n \in \mathbb{N}_{>0}\}$.

Beweis

⊇ Wir zeigen, dass $a^n b^n c^n \in L(G)$ für alle $n \in \mathbb{N}_{>0}$.

- ▶ Wende $n - 1$ mal $S \rightarrow aSBC$ und dann einmal $S \rightarrow aBC$ an:
 $S \Rightarrow^* a^{n-1} S (BC)^{n-1} \Rightarrow a^n (BC)^n$.
- ▶ Wende $CB \rightarrow BC$ solange an, bis es kein Teilwort CB mehr gibt:
 $a^n (BC)^n \Rightarrow^* a^n B^n C^n$.
- ▶ Wende $aB \rightarrow ab$ und anschließend $n - 1$ mal $bB \rightarrow bb$ an:
 $a^n B^n C^n \Rightarrow a^n b B^{n-1} C^n \Rightarrow^* a^n b^n C^n$.
- ▶ Wende einmal $bC \rightarrow bc$ und anschließend $n - 1$ mal $cC \rightarrow cc$ an:
 $a^n b^n C^n \Rightarrow a^n b^n c C^{n-1} \Rightarrow^* a^n b^n c^n$.

Beispiel für eine kontextsensitive Grammatik

Satz

Für $G = (\{S, B, C\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBC, S \rightarrow aBC, CB \rightarrow BC, aB \rightarrow ab, bB \rightarrow bb, bC \rightarrow bc, cC \rightarrow cc\}$
gilt $L(G) = \{a^n b^n c^n \mid n \in \mathbb{N}_{>0}\}$.

Beweis

⊇ Wir zeigen, dass $a^n b^n c^n \in L(G)$ für alle $n \in \mathbb{N}_{>0}$.

- ▶ Wende $n - 1$ mal $S \rightarrow aSBC$ und dann einmal $S \rightarrow aBC$ an:
 $S \Rightarrow^* a^{n-1} S (BC)^{n-1} \Rightarrow a^n (BC)^n$.
- ▶ Wende $CB \rightarrow BC$ solange an, bis es kein Teilwort CB mehr gibt:
 $a^n (BC)^n \Rightarrow^* a^n B^n C^n$.
- ▶ Wende $aB \rightarrow ab$ und anschließend $n - 1$ mal $bB \rightarrow bb$ an:
 $a^n B^n C^n \Rightarrow a^n b B^{n-1} C^n \Rightarrow^* a^n b^n C^n$.
- ▶ Wende einmal $bC \rightarrow bc$ und anschließend $n - 1$ mal $cC \rightarrow cc$ an:
 $a^n b^n C^n \Rightarrow a^n b^n c C^{n-1} \Rightarrow^* a^n b^n c^n$.

Zusammensetzen aller Ableitungsschritte zeigt $S \Rightarrow^* a^n b^n c^n$.

Beispiel für eine kontextsensitive Grammatik

Satz

Für $G = (\{S, B, C\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBC, S \rightarrow aBC, CB \rightarrow BC, aB \rightarrow ab, bB \rightarrow bb, bC \rightarrow bc, cC \rightarrow cc\}$
gilt $L(G) = \{a^n b^n c^n \mid n \in \mathbb{N}_{>0}\}$.

Beweis (Fortsetzung)

$\subseteq$ Wir zeigen, dass alle Wörter in $L(G)$ von der Form $a^n b^n c^n$ sind.

Beispiel für eine kontextsensitive Grammatik

Satz

Für $G = (\{S, B, C\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBC, S \rightarrow aBC, CB \rightarrow BC, aB \rightarrow ab, bB \rightarrow bb, bC \rightarrow bc, cC \rightarrow cc\}$
gilt $L(G) = \{a^n b^n c^n \mid n \in \mathbb{N}_{>0}\}$.

Beweis (Fortsetzung)

⊆ Wir zeigen, dass alle Wörter in $L(G)$ von der Form $a^n b^n c^n$ sind.

Für $S \Rightarrow_G^* u$ mit u Satzform zeigen die Produktionen:

$$\#_a(u) = \#_b(u) + \#_B(u) = \#_c(u) + \#_C(u).$$

Beispiel für eine kontextsensitive Grammatik

Satz

Für $G = (\{S, B, C\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBC, S \rightarrow aBC, CB \rightarrow BC, aB \rightarrow ab, bB \rightarrow bb, bC \rightarrow bc, cC \rightarrow cc\}$
gilt $L(G) = \{a^n b^n c^n \mid n \in \mathbb{N}_{>0}\}$.

Beweis (Fortsetzung)

⊆ Wir zeigen, dass alle Wörter in $L(G)$ von der Form $a^n b^n c^n$ sind.

Für $S \Rightarrow_G^* u$ mit u Satzform zeigen die Produktionen:

$$\#_a(u) = \#_b(u) + \#_B(u) = \#_c(u) + \#_C(u).$$

Für $S \Rightarrow_G^* w$ mit $w \in \{a, b, c\}^*$ gilt: a 's werden ganz links erzeugt,
d.h. $w = a^n w'$ mit $w' \in \{b, c\}^*$ und $n = \#_b(w') = \#_c(w')$.

Beispiel für eine kontextsensitive Grammatik

Satz

Für $G = (\{S, B, C\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBC, S \rightarrow aBC, CB \rightarrow BC, aB \rightarrow ab, bB \rightarrow bb, bC \rightarrow bc, cC \rightarrow cc\}$
gilt $L(G) = \{a^n b^n c^n \mid n \in \mathbb{N}_{>0}\}$.

Beweis (Fortsetzung)

⊆ Wir zeigen, dass alle Wörter in $L(G)$ von der Form $a^n b^n c^n$ sind.

Für $S \Rightarrow_G^* u$ mit u Satzform zeigen die Produktionen:

$$\#_a(u) = \#_b(u) + \#_B(u) = \#_c(u) + \#_C(u).$$

Für $S \Rightarrow_G^* w$ mit $w \in \{a, b, c\}^*$ gilt: a 's werden ganz links erzeugt,

d.h. $w = a^n w'$ mit $w' \in \{b, c\}^*$ und $n = \#_b(w') = \#_c(w')$.

Es gilt $w' = bw''$, da jedes auf a folgende Symbol durch $aB \rightarrow ab$ erzeugt wird und die Produktionen keine Terminalsymbole vertauschen.

Beispiel für eine kontextsensitive Grammatik

Satz

Für $G = (\{S, B, C\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBC, S \rightarrow aBC, CB \rightarrow BC, aB \rightarrow ab, bB \rightarrow bb, bC \rightarrow bc, cC \rightarrow cc\}$
gilt $L(G) = \{a^n b^n c^n \mid n \in \mathbb{N}_{>0}\}$.

Beweis (Fortsetzung)

Ebenso können die Terminalsymbole des Wortes $w' \in \{b, c\}^*$ nur durch $bB \rightarrow bb$, $bC \rightarrow bc$ und $cC \rightarrow cc$ erzeugt worden sein.

Diese Produktionen erlauben nur **einen Wechsel von b zu c** und **keine Wechsel von c zu b** . Auch ein **Umordnen der Terminalsymbole** ist **nicht möglich**.

Beispiel für eine kontextsensitive Grammatik

Satz

Für $G = (\{S, B, C\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBC, S \rightarrow aBC, CB \rightarrow BC, aB \rightarrow ab, bB \rightarrow bb, bC \rightarrow bc, cC \rightarrow cc\}$
gilt $L(G) = \{a^n b^n c^n \mid n \in \mathbb{N}_{>0}\}$.

Beweis (Fortsetzung)

Ebenso können die Terminalsymbole des Wortes $w' \in \{b, c\}^*$ nur durch $bB \rightarrow bb$, $bC \rightarrow bc$ und $cC \rightarrow cc$ erzeugt worden sein.

Diese Produktionen erlauben nur **einen Wechsel von b zu c** und **keine Wechsel von c zu b** . Auch ein **Umordnen der Terminalsymbole** ist **nicht möglich**.

Daher gilt $w' = b^i c^j$ und mit $n = \#_b(w') = \#_c(w')$ sogar $w' = b^n c^n$.

Beispiel für eine kontextsensitive Grammatik

Satz

Für $G = (\{S, B, C\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBC, S \rightarrow aBC, CB \rightarrow BC, aB \rightarrow ab, bB \rightarrow bb, bC \rightarrow bc, cC \rightarrow cc\}$
gilt $L(G) = \{a^n b^n c^n \mid n \in \mathbb{N}_{>0}\}$.

Beweis (Fortsetzung)

Ebenso können die Terminalsymbole des Wortes $w' \in \{b, c\}^*$ nur durch $bB \rightarrow bb$, $bC \rightarrow bc$ und $cC \rightarrow cc$ erzeugt worden sein.

Diese Produktionen erlauben nur **einen Wechsel von b zu c** und **keine Wechsel von c zu b** . Auch ein **Umordnen der Terminalsymbole** ist **nicht möglich**.

Daher gilt $w' = b^i c^j$ und mit $n = \#_b(w') = \#_c(w')$ sogar $w' = b^n c^n$.

Also ist $w = a^n b^n c^n$.

Beispiel für eine kontextsensitive Grammatik

Satz

Für $G = (\{S, B, C\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBC, S \rightarrow aBC, CB \rightarrow BC, aB \rightarrow ab, bB \rightarrow bb, bC \rightarrow bc, cC \rightarrow cc\}$
gilt $L(G) = \{a^n b^n c^n \mid n \in \mathbb{N}_{>0}\}$.

Beweis (Fortsetzung)

Ebenso können die Terminalsymbole des Wortes $w' \in \{b, c\}^*$ nur durch $bB \rightarrow bb$, $bC \rightarrow bc$ und $cC \rightarrow cc$ erzeugt worden sein.

Diese Produktionen erlauben nur **einen Wechsel von b zu c** und **keine Wechsel von c zu b** . Auch ein **Umordnen der Terminalsymbole** ist **nicht möglich**.

Daher gilt $w' = b^i c^j$ und mit $n = \#_b(w') = \#_c(w')$ sogar $w' = b^n c^n$.

Also ist $w = a^n b^n c^n$.

Aber w war beliebig gewählt. Daher sind alle $w \in L(G)$ von der Form $a^n b^n c^n$.



Beispiel für eine Typ 0-Grammatik

$G = (\{S, T, A, B, \$\}, \{a, b\}, P, S)$ mit
 $P = \{S \rightarrow \$T\$, T \rightarrow aAT, T \rightarrow bBT, T \rightarrow \varepsilon, \$a \rightarrow a\$, \$b \rightarrow b\$, Aa \rightarrow aA,$
 $Ab \rightarrow bA, Ba \rightarrow aB, Bb \rightarrow bB, A\$ \rightarrow \$a, B\$ \rightarrow \$b, \$\$ \rightarrow \varepsilon\}$

Beispiel für eine Typ 0-Grammatik

$G = (\{S, T, A, B, \$\}, \{a, b\}, P, S)$ mit

$P = \{S \rightarrow \$T\$, T \rightarrow aAT, T \rightarrow bBT, T \rightarrow \varepsilon, \$a \rightarrow a\$, \$b \rightarrow b\$, Aa \rightarrow aA, \\ Ab \rightarrow bA, Ba \rightarrow aB, Bb \rightarrow bB, A\$ \rightarrow \$a, B\$ \rightarrow \$b, \$\$ \rightarrow \varepsilon\}$

Ableitung:

$S \Rightarrow \$T\$$

Beispiel für eine Typ 0-Grammatik

$G = (\{S, T, A, B, \$\}, \{a, b\}, P, S)$ mit

$P = \{S \rightarrow \$T\$, \textcolor{red}{T} \rightarrow \textcolor{blue}{a}AT, T \rightarrow bBT, T \rightarrow \varepsilon, \$a \rightarrow a\$, \$b \rightarrow b\$, Aa \rightarrow aA, \\ Ab \rightarrow bA, Ba \rightarrow aB, Bb \rightarrow bB, A\$ \rightarrow \$a, B\$ \rightarrow \$b, \$\$ \rightarrow \varepsilon\}$

Ableitung:

$S \Rightarrow \$\textcolor{red}{T}\$ \Rightarrow \$\textcolor{blue}{a}AT\$$

Beispiel für eine Typ 0-Grammatik

$G = (\{S, T, A, B, \$\}, \{a, b\}, P, S)$ mit

$P = \{S \rightarrow \$T\$, \textcolor{red}{T} \rightarrow \textcolor{blue}{a}A\textcolor{blue}{T}, T \rightarrow bBT, T \rightarrow \epsilon, \$a \rightarrow a\$, \$b \rightarrow b\$, Aa \rightarrow aA, \\ Ab \rightarrow bA, Ba \rightarrow aB, Bb \rightarrow bB, A\$ \rightarrow \$a, B\$ \rightarrow \$b, \$\$ \rightarrow \epsilon\}$

Ableitung:

$S \Rightarrow \$T\$ \Rightarrow \$aA\textcolor{red}{T}\$ \Rightarrow \$aAa\textcolor{blue}{AT}\$$

Beispiel für eine Typ 0-Grammatik

$G = (\{S, T, A, B, \$\}, \{a, b\}, P, S)$ mit

$P = \{S \rightarrow \$T\$, T \rightarrow aAT, \textcolor{red}{T} \rightarrow \textcolor{blue}{bBT}, T \rightarrow \epsilon, \$a \rightarrow a\$, \$b \rightarrow b\$, Aa \rightarrow aA, \\ Ab \rightarrow bA, Ba \rightarrow aB, Bb \rightarrow bB, A\$ \rightarrow \$a, B\$ \rightarrow \$b, \$\$ \rightarrow \epsilon\}$

Ableitung:

$S \Rightarrow \$T\$ \Rightarrow \$aAT\$ \Rightarrow \$aAaAT\$ \Rightarrow \$aAaAbBT\$$

Beispiel für eine Typ 0-Grammatik

$G = (\{S, T, A, B, \$\}, \{a, b\}, P, S)$ mit

$P = \{S \rightarrow \$T\$, T \rightarrow aAT, T \rightarrow bBT, \textcolor{red}{T} \rightarrow \epsilon, \$a \rightarrow a\$, \$b \rightarrow b\$, Aa \rightarrow aA, \\ Ab \rightarrow bA, Ba \rightarrow aB, Bb \rightarrow bB, A\$ \rightarrow \$a, B\$ \rightarrow \$b, \$\$ \rightarrow \epsilon\}$

Ableitung:

$S \Rightarrow \$T\$ \Rightarrow \$aAT\$ \Rightarrow \$aAaAT\$ \Rightarrow \$aAaAbB\textcolor{red}{T}\$ \\ \Rightarrow \$aAaAbB\$$

Beispiel für eine Typ 0-Grammatik

$G = (\{S, T, A, B, \$\}, \{a, b\}, P, S)$ mit

$P = \{S \rightarrow \$T\$, T \rightarrow aAT, T \rightarrow bBT, T \rightarrow \varepsilon, \$a \rightarrow a\$, \$b \rightarrow b\$, Aa \rightarrow aA, Ab \rightarrow bA, Ba \rightarrow aB, Bb \rightarrow bB, A\$ \rightarrow \$a, B\$ \rightarrow \$b, \$\$ \rightarrow \varepsilon\}$

Ableitung:

$S \Rightarrow \$T\$ \Rightarrow \$aAT\$ \Rightarrow \$aAaAT\$ \Rightarrow \$aAaAbBT\$$
 $\Rightarrow \$aAaAbB\$ \Rightarrow \$aAaAbB\$$

Beispiel für eine Typ 0-Grammatik

$G = (\{S, T, A, B, \$\}, \{a, b\}, P, S)$ mit
 $P = \{S \rightarrow \$T\$, T \rightarrow aAT, T \rightarrow bBT, T \rightarrow \epsilon, \$a \rightarrow a\$, \$b \rightarrow b\$, Aa \rightarrow aA,$
 $\textcolor{red}{A}b \rightarrow \textcolor{blue}{b}A, Ba \rightarrow aB, Bb \rightarrow bB, A\$ \rightarrow \$a, B\$ \rightarrow \$b, \$\$ \rightarrow \epsilon\}$

Ableitung:

$$\begin{aligned} S &\Rightarrow \$T\$ \Rightarrow \$aAT\$ \Rightarrow \$aAaAT\$ \Rightarrow \$aAaAbBT\$ \\ &\Rightarrow \$aAaAbB\$ \Rightarrow \$aaA\textcolor{red}{A}bB\$ \Rightarrow \$aaA\textcolor{blue}{b}AB\$ \end{aligned}$$

Beispiel für eine Typ 0-Grammatik

$G = (\{S, T, A, B, \$\}, \{a, b\}, P, S)$ mit

$P = \{S \rightarrow \$T\$, T \rightarrow aAT, T \rightarrow bBT, T \rightarrow \epsilon, \$a \rightarrow a\$, \$b \rightarrow b\$, Aa \rightarrow aA,$
 $\textcolor{red}{A}b \rightarrow \textcolor{blue}{b}A, Ba \rightarrow aB, Bb \rightarrow bB, A\$ \rightarrow \$a, B\$ \rightarrow \$b, \$\$ \rightarrow \epsilon\}$

Ableitung:

$S \Rightarrow \$T\$ \Rightarrow \$aAT\$ \Rightarrow \$aAaAT\$ \Rightarrow \$aAaAbBT\$$
 $\Rightarrow \$aAaAbB\$ \Rightarrow \$aaAAbB\$ \Rightarrow \$aa\textcolor{red}{A}bAB\$ \Rightarrow \$aab\textcolor{blue}{A}AB\$$

Beispiel für eine Typ 0-Grammatik

$G = (\{S, T, A, B, \$\}, \{a, b\}, P, S)$ mit

$P = \{S \rightarrow \$T\$, T \rightarrow aAT, T \rightarrow bBT, T \rightarrow \epsilon, \$a \rightarrow a\$, \$b \rightarrow b\$, Aa \rightarrow aA, Ab \rightarrow bA, Ba \rightarrow aB, Bb \rightarrow bB, A\$ \rightarrow \$a, B\$ \rightarrow \$b, \$\$ \rightarrow \epsilon\}$

Ableitung:

$$\begin{aligned} S &\Rightarrow \$T\$ \Rightarrow \$aAT\$ \Rightarrow \$aAaAT\$ \Rightarrow \$aAaAbBT\$ \\ &\Rightarrow \$aAaAbB\$ \Rightarrow \$aaAAbB\$ \Rightarrow \$aaAbAB\$ \Rightarrow \$aabAA\textcolor{red}{B}\$ \\ &\Rightarrow \$aabAA\textcolor{blue}{\$}b \end{aligned}$$

Beispiel für eine Typ 0-Grammatik

$G = (\{S, T, A, B, \$\}, \{a, b\}, P, S)$ mit

$P = \{S \rightarrow \$T\$, T \rightarrow aAT, T \rightarrow bBT, T \rightarrow \epsilon, \$a \rightarrow a\$, \$b \rightarrow b\$, Aa \rightarrow aA, \\ Ab \rightarrow bA, Ba \rightarrow aB, Bb \rightarrow bB, \textcolor{red}{A}\$ \rightarrow \textcolor{blue}{\$}a, B\$ \rightarrow \$b, \$\$ \rightarrow \epsilon\}$

Ableitung:

$S \Rightarrow \$T\$ \Rightarrow \$aAT\$ \Rightarrow \$aAaAT\$ \Rightarrow \$aAaAbBT\$ \\ \Rightarrow \$aAaAbB\$ \Rightarrow \$aaAAbB\$ \Rightarrow \$aaAbAB\$ \Rightarrow \$aabAAB\$ \\ \Rightarrow \$aabA\textcolor{red}{A}\$b \Rightarrow \$aabA\textcolor{blue}{\$}ab$

Beispiel für eine Typ 0-Grammatik

$G = (\{S, T, A, B, \$\}, \{a, b\}, P, S)$ mit

$P = \{S \rightarrow \$T\$, T \rightarrow aAT, T \rightarrow bBT, T \rightarrow \epsilon, \$a \rightarrow a\$, \$b \rightarrow b\$, Aa \rightarrow aA, \\ Ab \rightarrow bA, Ba \rightarrow aB, Bb \rightarrow bB, \textcolor{red}{A}\$ \rightarrow \textcolor{blue}{\$}a, B\$ \rightarrow \$b, \$\$ \rightarrow \epsilon\}$

Ableitung:

$S \Rightarrow \$T\$ \Rightarrow \$aAT\$ \Rightarrow \$aAaAT\$ \Rightarrow \$aAaAbBT\$ \\ \Rightarrow \$aAaAbB\$ \Rightarrow \$aaAAbB\$ \Rightarrow \$aaAbAB\$ \Rightarrow \$aabAAB\$ \\ \Rightarrow \$aabAA\$b \Rightarrow \$aab\textcolor{red}{A}\$ab \Rightarrow \$aab\textcolor{blue}{\$}aab$

Beispiel für eine Typ 0-Grammatik

$G = (\{S, T, A, B, \$\}, \{a, b\}, P, S)$ mit

$P = \{S \rightarrow \$T\$, T \rightarrow aAT, T \rightarrow bBT, T \rightarrow \epsilon, \$a \rightarrow a\$, \$b \rightarrow b\$, Aa \rightarrow aA, Ab \rightarrow bA, Ba \rightarrow aB, Bb \rightarrow bB, A\$ \rightarrow \$a, B\$ \rightarrow \$b, \$\$ \rightarrow \epsilon\}$

Ableitung:

$$\begin{aligned} S &\Rightarrow \$T\$ \Rightarrow \$aAT\$ \Rightarrow \$aAaAT\$ \Rightarrow \$aAaAbBT\$ \\ &\Rightarrow \$aAaAbB\$ \Rightarrow \$aaAAbB\$ \Rightarrow \$aaAbAB\$ \Rightarrow \$aabAAB\$ \\ &\Rightarrow \$aabAA\$b \Rightarrow \$aabA\$ab \Rightarrow \$aab\$aab \Rightarrow a\$ab\$aab \end{aligned}$$

Beispiel für eine Typ 0-Grammatik

$G = (\{S, T, A, B, \$\}, \{a, b\}, P, S)$ mit

$P = \{S \rightarrow \$T\$, T \rightarrow aAT, T \rightarrow bBT, T \rightarrow \epsilon, \$a \rightarrow a\$, \$b \rightarrow b\$, Aa \rightarrow aA, Ab \rightarrow bA, Ba \rightarrow aB, Bb \rightarrow bB, A\$ \rightarrow \$a, B\$ \rightarrow \$b, \$\$ \rightarrow \epsilon\}$

Ableitung:

$$\begin{aligned} S &\Rightarrow \$T\$ \Rightarrow \$aAT\$ \Rightarrow \$aAaAT\$ \Rightarrow \$aAaAbBT\$ \\ &\Rightarrow \$aAaAbB\$ \Rightarrow \$aaAAbB\$ \Rightarrow \$aaAbAB\$ \Rightarrow \$aabAAB\$ \\ &\Rightarrow \$aabAA\$b \Rightarrow \$aabA\$ab \Rightarrow \$aab\$aab \Rightarrow a\$ab\$aab \\ &\Rightarrow aa\$b\$aab \end{aligned}$$

Beispiel für eine Typ 0-Grammatik

$G = (\{S, T, A, B, \$\}, \{a, b\}, P, S)$ mit

$P = \{S \rightarrow \$T\$, T \rightarrow aAT, T \rightarrow bBT, T \rightarrow \epsilon, \$a \rightarrow a\$, \$b \rightarrow b\$, Aa \rightarrow aA, \\ Ab \rightarrow bA, Ba \rightarrow aB, Bb \rightarrow bB, A\$ \rightarrow \$a, B\$ \rightarrow \$b, \$\$ \rightarrow \epsilon\}$

Ableitung:

$$\begin{aligned} S &\Rightarrow \$T\$ \Rightarrow \$aAT\$ \Rightarrow \$aAaAT\$ \Rightarrow \$aAaAbBT\$ \\ &\Rightarrow \$aAaAbB\$ \Rightarrow \$aaAAbB\$ \Rightarrow \$aaAbAB\$ \Rightarrow \$aabAAB\$ \\ &\Rightarrow \$aabAA\$b \Rightarrow \$aabA\$ab \Rightarrow \$aab\$aab \Rightarrow a\$ab\$aab \\ &\Rightarrow aa\$b\$aab \Rightarrow aab\$\$aab \end{aligned}$$

Beispiel für eine Typ 0-Grammatik

$G = (\{S, T, A, B, \$\}, \{a, b\}, P, S)$ mit

$P = \{S \rightarrow \$T\$, T \rightarrow aAT, T \rightarrow bBT, T \rightarrow \epsilon, \$a \rightarrow a\$, \$b \rightarrow b\$, Aa \rightarrow aA, \\ Ab \rightarrow bA, Ba \rightarrow aB, Bb \rightarrow bB, A\$ \rightarrow \$a, B\$ \rightarrow \$b, \$\$ \rightarrow \epsilon\}$

Ableitung:

$S \Rightarrow \$T\$ \Rightarrow \$aAT\$ \Rightarrow \$aAaAT\$ \Rightarrow \$aAaAbBT\$ \\ \Rightarrow \$aAaAbB\$ \Rightarrow \$aaAAbB\$ \Rightarrow \$aaAbAB\$ \Rightarrow \$aabAAB\$ \\ \Rightarrow \$aabAA\$b \Rightarrow \$aabA\$ab \Rightarrow \$aab\$aab \Rightarrow a\$ab\$aab \\ \Rightarrow aa\$b\$aab \Rightarrow aab\$\$aab \Rightarrow aabaab$

Beispiel für eine Typ 0-Grammatik

$G = (\{S, T, A, B, \$\}, \{a, b\}, P, S)$ mit

$P = \{S \rightarrow \$T\$, T \rightarrow aAT, T \rightarrow bBT, T \rightarrow \epsilon, \$a \rightarrow a\$, \$b \rightarrow b\$, Aa \rightarrow aA, \\ Ab \rightarrow bA, Ba \rightarrow aB, Bb \rightarrow bB, A\$ \rightarrow \$a, B\$ \rightarrow \$b, \$\$ \rightarrow \epsilon\}$

Ableitung:

$$\begin{aligned} S &\Rightarrow \$T\$ \Rightarrow \$aAT\$ \Rightarrow \$aAaAT\$ \Rightarrow \$aAaAbBT\$ \\ &\Rightarrow \$aAaAbB\$ \Rightarrow \$aaAAbB\$ \Rightarrow \$aaAbAB\$ \Rightarrow \$aabAAB\$ \\ &\Rightarrow \$aabAA\$b \Rightarrow \$aabA\$ab \Rightarrow \$aab\$aab \Rightarrow a\$ab\$aab \\ &\Rightarrow aa\$b\$aab \Rightarrow aab\$\$aab \Rightarrow aabaab \end{aligned}$$

$L(G) = ?$

Beispiel für eine Typ 0-Grammatik

$G = (\{S, T, A, B, \$\}, \{a, b\}, P, S)$ mit
 $P = \{S \rightarrow \$T\$, T \rightarrow aAT, T \rightarrow bBT, T \rightarrow \epsilon, \$a \rightarrow a\$, \$b \rightarrow b\$, Aa \rightarrow aA,$
 $Ab \rightarrow bA, Ba \rightarrow aB, Bb \rightarrow bB, A\$ \rightarrow \$a, B\$ \rightarrow \$b, \$\$ \rightarrow \epsilon\}$

Ableitung:

$S \Rightarrow \$T\$ \Rightarrow \$aAT\$ \Rightarrow \$aAaAT\$ \Rightarrow \$aAaAbBT\$$
 $\Rightarrow \$aAaAbB\$ \Rightarrow \$aaAAbB\$ \Rightarrow \$aaAbAB\$ \Rightarrow \$aabAAB\$$
 $\Rightarrow \$aabAA\$b \Rightarrow \$aabA\$ab \Rightarrow \$aab\$aab \Rightarrow a\$ab\aab
 $\Rightarrow aa\$b\$aab \Rightarrow aab\$\$aab \Rightarrow aabaab$

$L(G) = \{ww \mid w \in \{a, b\}^*\}$

Beispiel für eine Typ 0-Grammatik

Satz

Für $G = (\{S, T, A, B, \$\}, \{a, b\}, P, S)$ mit
 $P = \{S \rightarrow \$T\$, T \rightarrow aAT, T \rightarrow bBT, T \rightarrow \epsilon, \$a \rightarrow a\$, \$b \rightarrow b\$, Aa \rightarrow aA,$
 $Ab \rightarrow bA, Ba \rightarrow aB, Bb \rightarrow bB, A\$ \rightarrow \$a, B\$ \rightarrow \$b, \$\$ \rightarrow \epsilon\}$
gilt $L(G) = \{ww \mid w \in \{a, b\}^*\}$.

Beispiel für eine Typ 0-Grammatik

Satz

Für $G = (\{S, T, A, B, \$\}, \{a, b\}, P, S)$ mit
 $P = \{S \rightarrow \$T\$, T \rightarrow aAT, T \rightarrow bBT, T \rightarrow \epsilon, \$a \rightarrow a\$, \$b \rightarrow b\$, Aa \rightarrow aA,$
 $Ab \rightarrow bA, Ba \rightarrow aB, Bb \rightarrow bB, A\$ \rightarrow \$a, B\$ \rightarrow \$b, \$\$ \rightarrow \epsilon\}$
gilt $L(G) = \{ww \mid w \in \{a, b\}^*\}$.

Beweis

“ $\supseteq$ ” Wir zeigen, dass $ww \in L(G)$ für alle $w \in \{a, b\}^*$.

- ▶ Mit $S \rightarrow \$T\$$ wird zunächst eine Umrahmung mit $\$$ erzeugt.

Beispiel für eine Typ 0-Grammatik

Satz

Für $G = (\{S, T, A, B, \$\}, \{a, b\}, P, S)$ mit
 $P = \{S \rightarrow \$T\$, T \rightarrow aAT, T \rightarrow bBT, T \rightarrow \epsilon, \$a \rightarrow a\$, \$b \rightarrow b\$, Aa \rightarrow aA,$
 $Ab \rightarrow bA, Ba \rightarrow aB, Bb \rightarrow bB, A\$ \rightarrow \$a, B\$ \rightarrow \$b, \$\$ \rightarrow \epsilon\}$
gilt $L(G) = \{ww \mid w \in \{a, b\}^*\}$.

Beweis

“ $\supseteq$ ” Wir zeigen, dass $ww \in L(G)$ für alle $w \in \{a, b\}^*$.

- ▶ Mit $S \rightarrow \$T\$$ wird zunächst eine Umrahmung mit $\$$ erzeugt.
- ▶ Mit $T \rightarrow aAT, T \rightarrow bBT, T \rightarrow \epsilon$ wird eine Satzform aus Blöcken aA, bB erzeugt.

Beispiel für eine Typ 0-Grammatik

Satz

Für $G = (\{S, T, A, B, \$\}, \{a, b\}, P, S)$ mit
 $P = \{S \rightarrow \$T\$, T \rightarrow aAT, T \rightarrow bBT, T \rightarrow \epsilon, \$a \rightarrow a\$, \$b \rightarrow b\$, Aa \rightarrow aA,$
 $Ab \rightarrow bA, Ba \rightarrow aB, Bb \rightarrow bB, A\$ \rightarrow \$a, B\$ \rightarrow \$b, \$\$ \rightarrow \epsilon\}$
gilt $L(G) = \{ww \mid w \in \{a, b\}^*\}$.

Beweis

“ $\supseteq$ ” Wir zeigen, dass $ww \in L(G)$ für alle $w \in \{a, b\}^*$.

- ▶ Mit $S \rightarrow \$T\$$ wird zunächst eine Umrahmung mit $\$$ erzeugt.
- ▶ Mit $T \rightarrow aAT, T \rightarrow bBT, T \rightarrow \epsilon$ wird eine Satzform aus Blöcken aA, bB erzeugt.
- ▶ Mit $Aa \rightarrow aA, Ab \rightarrow bA, Ba \rightarrow aB, Bb \rightarrow bB$ kommen A 's und B 's bis vor $\$$.

Beispiel für eine Typ 0-Grammatik

Satz

Für $G = (\{S, T, A, B, \$\}, \{a, b\}, P, S)$ mit
 $P = \{S \rightarrow \$T\$, T \rightarrow aAT, T \rightarrow bBT, T \rightarrow \epsilon, \$a \rightarrow a\$, \$b \rightarrow b\$, Aa \rightarrow aA,$
 $Ab \rightarrow bA, Ba \rightarrow aB, Bb \rightarrow bB, A\$ \rightarrow \$a, B\$ \rightarrow \$b, \$\$ \rightarrow \epsilon\}$
gilt $L(G) = \{ww \mid w \in \{a, b\}^*\}$.

Beweis

“ $\supseteq$ ” Wir zeigen, dass $ww \in L(G)$ für alle $w \in \{a, b\}^*$.

- ▶ Mit $S \rightarrow \$T\$$ wird zunächst eine Umrahmung mit $\$$ erzeugt.
- ▶ Mit $T \rightarrow aAT, T \rightarrow bBT, T \rightarrow \epsilon$ wird eine Satzform aus Blöcken aA, bB erzeugt.
- ▶ Mit $Aa \rightarrow aA, Ab \rightarrow bA, Ba \rightarrow aB, Bb \rightarrow bB$ kommen A 's und B 's bis vor $\$$.
- ▶ Mit $A\$ \rightarrow \$a, B\$ \rightarrow \b werden die A 's und B 's in a 's und b 's verwandelt.

Beispiel für eine Typ 0-Grammatik

Satz

Für $G = (\{S, T, A, B, \$\}, \{a, b\}, P, S)$ mit
 $P = \{S \rightarrow \$T\$, T \rightarrow aAT, T \rightarrow bBT, T \rightarrow \epsilon, \$a \rightarrow a\$, \$b \rightarrow b\$, Aa \rightarrow aA,$
 $Ab \rightarrow bA, Ba \rightarrow aB, Bb \rightarrow bB, A\$ \rightarrow \$a, B\$ \rightarrow \$b, \$\$ \rightarrow \epsilon\}$
gilt $L(G) = \{ww \mid w \in \{a, b\}^*\}$.

Beweis

“ $\supseteq$ ” Wir zeigen, dass $ww \in L(G)$ für alle $w \in \{a, b\}^*$.

- ▶ Mit $S \rightarrow \$T\$$ wird zunächst eine Umrahmung mit $\$ \$$ erzeugt.
- ▶ Mit $T \rightarrow aAT, T \rightarrow bBT, T \rightarrow \epsilon$ wird eine Satzform aus Blöcken aA, bB erzeugt.
- ▶ Mit $Aa \rightarrow aA, Ab \rightarrow bA, Ba \rightarrow aB, Bb \rightarrow bB$ kommen A 's und B 's bis vor $\$$.
- ▶ Mit $A\$ \rightarrow \$a, B\$ \rightarrow \b werden die A 's und B 's in a 's und b 's verwandelt.
- ▶ Mit $\$a \rightarrow a\$, \$b \rightarrow b\$$ wird das linke $\$$ zum rechten geschoben, mit $\$ \$ \rightarrow \epsilon$ werden die beiden $\$$'s dann eliminiert.

Beispiel für eine Typ 0-Grammatik

Satz

Für $G = (\{S, T, A, B, \$\}, \{a, b\}, P, S)$ mit
 $P = \{S \rightarrow \$T\$, T \rightarrow aAT, T \rightarrow bBT, T \rightarrow \epsilon, \$a \rightarrow a\$, \$b \rightarrow b\$, Aa \rightarrow aA,$
 $Ab \rightarrow bA, Ba \rightarrow aB, Bb \rightarrow bB, A\$ \rightarrow \$a, B\$ \rightarrow \$b, \$\$ \rightarrow \epsilon\}$
gilt $L(G) = \{ww \mid w \in \{a, b\}^*\}$.

Beweis

“ $\supseteq$ ” Wir zeigen, dass $ww \in L(G)$ für alle $w \in \{a, b\}^*$.

- ▶ Mit $S \rightarrow \$T\$$ wird zunächst eine Umrahmung mit $\$ \$$ erzeugt.
- ▶ Mit $T \rightarrow aAT, T \rightarrow bBT, T \rightarrow \epsilon$ wird eine Satzform aus Blöcken aA, bB erzeugt.
- ▶ Mit $Aa \rightarrow aA, Ab \rightarrow bA, Ba \rightarrow aB, Bb \rightarrow bB$ kommen A 's und B 's bis vor $\$$.
- ▶ Mit $A\$ \rightarrow \$a, B\$ \rightarrow \b werden die A 's und B 's in a 's und b 's verwandelt.
- ▶ Mit $\$a \rightarrow a\$, \$b \rightarrow b\$$ wird das linke $\$$ zum rechten geschoben, mit $\$ \$ \rightarrow \epsilon$ werden die beiden $\$$'s dann eliminiert.
- ▶ Die relative Lage aller a 's zu b 's bzw. aller A 's zu B 's wird nie geändert.

Beispiel für eine Typ 0-Grammatik

Satz

Für $G = (\{S, T, A, B, \$\}, \{a, b\}, P, S)$ mit
 $P = \{S \rightarrow \$T\$, T \rightarrow aAT, T \rightarrow bBT, T \rightarrow \epsilon, \$a \rightarrow a\$, \$b \rightarrow b\$, Aa \rightarrow aA,$
 $Ab \rightarrow bA, Ba \rightarrow aB, Bb \rightarrow bB, A\$ \rightarrow \$a, B\$ \rightarrow \$b, \$\$ \rightarrow \epsilon\}.$ gilt
 $L(G) = \{ww \mid w \in \{a, b\}^*\}.$

Beweis (Fortsetzung)

“ $\subseteq$ ” Wir zeigen, dass alle Wörter in $L(G)$ von der Form ww sind.

Beispiel für eine Typ 0-Grammatik

Satz

Für $G = (\{S, T, A, B, \$\}, \{a, b\}, P, S)$ mit
 $P = \{S \rightarrow \$T\$, T \rightarrow aAT, T \rightarrow bBT, T \rightarrow \epsilon, \$a \rightarrow a\$, \$b \rightarrow b\$, Aa \rightarrow aA,$
 $Ab \rightarrow bA, Ba \rightarrow aB, Bb \rightarrow bB, A\$ \rightarrow \$a, B\$ \rightarrow \$b, \$\$ \rightarrow \epsilon\}.$ gilt
 $L(G) = \{ww \mid w \in \{a, b\}^*\}.$

Beweis (Fortsetzung)

“ $\subseteq$ ” Wir zeigen, dass alle Wörter in $L(G)$ von der Form ww sind.

Alle Ableitungen müssen mit unwesentlichen Abweichungen von der unter “ $\supseteq$ ” angegebenen Form sein und deshalb zu ww führen. $\square$

Mehrdeutige Grammatiken

Beispiel: $(E, \{*, +, 1, 2\}, P, E)$ mit $P = \{E \rightarrow E * E, E \rightarrow E + E, E \rightarrow 1, E \rightarrow 2\}$

Zwei Ableitungen für $1 + 2 * 2$:

$$\blacktriangleright E \Rightarrow E * E \Rightarrow E + E * E \Rightarrow 1 + E * E \Rightarrow 1 + 2 * E \Rightarrow 1 + 2 * 2$$

$$\blacktriangleright E \Rightarrow E + E \Rightarrow E + E * E \Rightarrow 1 + E * E \Rightarrow 1 + 2 * E \Rightarrow 1 + 2 * 2$$

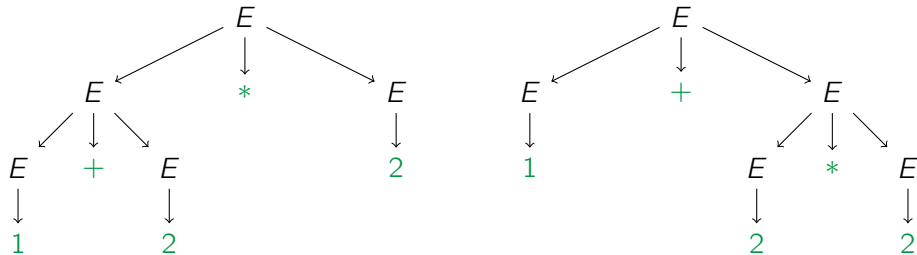
Mehrdeutige Grammatiken

Beispiel: $(E, \{*, +, 1, 2\}, P, E)$ mit $P = \{E \rightarrow E * E, E \rightarrow E + E, E \rightarrow 1, E \rightarrow 2\}$

Zwei Ableitungen für $1 + 2 * 2$:

- ▶ $E \Rightarrow E * E \Rightarrow E + E * E \Rightarrow 1 + E * E \Rightarrow 1 + 2 * E \Rightarrow 1 + 2 * 2$
- ▶ $E \Rightarrow E + E \Rightarrow E + E * E \Rightarrow 1 + E * E \Rightarrow 1 + 2 * E \Rightarrow 1 + 2 * 2$

Syntaxbäume dazu:



Mehrdeutige Grammatiken

Definition

Eine Typ 2-Grammatik ist **mehrdeutig**, wenn es verschieden strukturierte Syntaxbäume für dasselbe Wort w gibt.

Definition

Eine Typ 2-Sprache ist **inhärent mehrdeutig**, wenn es nur mehrdeutige Grammatiken gibt, die diese Sprache erzeugen.

Mehrdeutige Grammatiken

Definition

Eine Typ 2-Grammatik ist **mehrdeutig**, wenn es verschieden strukturierte Syntaxbäume für dasselbe Wort w gibt.

Definition

Eine Typ 2-Sprache ist **inhärent mehrdeutig**, wenn es nur mehrdeutige Grammatiken gibt, die diese Sprache erzeugen.

Die Sprache

$$\{a^m b^m c^n d^n \mid m, n \in \mathbb{N}_{>0}\} \cup \{a^m b^n c^n d^m \mid m, n \in \mathbb{N}_{>0}\}$$

ist inhärent mehrdeutig (Beweis in Hopcroft et al. 2006).

ε -Produktionen: 1. Sonderregel

- Das leere Wort ε kann bisher nicht für Typ 1, 2, 3-Grammatiken erzeugt werden. Produktion $S \rightarrow \varepsilon$ erfüllt die Typ 1-Bedingung $|S| \leq |\varepsilon|$ nicht.

ε -Produktionen: 1. Sonderregel

- Das leere Wort ε kann bisher nicht für Typ 1, 2, 3-Grammatiken erzeugt werden. Produktion $S \rightarrow \varepsilon$ erfüllt die Typ 1-Bedingung $|S| \leq |\varepsilon|$ nicht.

Daher:

1. Sonderregel: ε -Produktion in Typ 1, 2, 3-Grammatiken

Eine Grammatik $G = (V, \Sigma, P, S)$ vom Typ 1, 2 oder 3 darf eine Produktion $S \rightarrow \varepsilon \in P$ enthalten, vorausgesetzt, dass S auf keiner rechten Seite einer Produktion in P vorkommt.

ϵ -Produktionen: 1. Sonderregel

- Das leere Wort ϵ kann bisher nicht für Typ 1, 2, 3-Grammatiken erzeugt werden. Produktion $S \rightarrow \epsilon$ erfüllt die Typ 1-Bedingung $|S| \leq |\epsilon|$ nicht.

Daher:

1. Sonderregel: ϵ -Produktion in Typ 1, 2, 3-Grammatiken

Eine Grammatik $G = (V, \Sigma, P, S)$ vom Typ 1, 2 oder 3 darf eine Produktion $S \rightarrow \epsilon \in P$ enthalten, vorausgesetzt, dass S auf keiner rechten Seite einer Produktion in P vorkommt.

Sonderregel erlaubt nicht:

$$G = (\{S\}, \{a\}, \{S \rightarrow \epsilon, S \rightarrow aSa\}, S)$$

Sonderregel erlaubt:

$$G = (\{S', S\}, \{a\}, \{S' \rightarrow \epsilon, S' \rightarrow aSa, S \rightarrow aSa\}, S')$$

Leeres Wort hinzufügen geht mit 1. Sonderregel immer

Satz (Anwendung von 1. Sonderregel)

Sei $G = (V, \Sigma, P, S)$ vom Typ $i \in \{1, 2, 3\}$ mit $\varepsilon \notin L(G)$. Sei $S' \notin V$.

Dann erzeugt $G' = (V \cup \{S'\}, \Sigma, P \cup \{S' \rightarrow \varepsilon\} \cup \{S' \rightarrow r \mid S \rightarrow r \in P\}, S')$ die Sprache $L(G') = L(G) \cup \{\varepsilon\}$, G' erfüllt die 1. Sonderregel und G' ist vom Typ i .

Leeres Wort hinzufügen geht mit 1. Sonderregel immer

Satz (Anwendung von 1. Sonderregel)

Sei $G = (V, \Sigma, P, S)$ vom Typ $i \in \{1, 2, 3\}$ mit $\varepsilon \notin L(G)$. Sei $S' \notin V$.

Dann erzeugt $G' = (V \cup \{S'\}, \Sigma, P \cup \{S' \rightarrow \varepsilon\} \cup \{S' \rightarrow r \mid S \rightarrow r \in P\}, S')$ die Sprache $L(G') = L(G) \cup \{\varepsilon\}$, G' erfüllt die 1. Sonderregel und G' ist vom Typ i .

Beweis

1. $L(G') = L(G) \cup \{\varepsilon\}$: Da $S' \Rightarrow \varepsilon$, gilt $\varepsilon \in L(G')$.

Für $w \neq \varepsilon$ gilt:

$S \Rightarrow_G^* w$. g.d.w. $S' \Rightarrow_{G'}^* w$. Der jeweils erste Ableitungsschritt muss ausgetauscht werden, d.h. $S \Rightarrow_G r$ vs. $S' \Rightarrow_{G'} r$.

2. Da S' neu ist, kommt S' auf keiner rechten Seite vor.
3. Da $S \rightarrow r \in P$ vom Typ i sind, sind auch $S' \rightarrow r$ vom Typ i . □

ε -Produktionen: 2. Sonderregel

Weitere Sonderregel:

2. Sonderregel: ε -Produktionen in Typ 2- und Typ 3-Grammatiken

Eine Grammatik $G = (V, \Sigma, P, S)$ vom Typ 2 oder 3 darf Produktionen von der Form $A \rightarrow \varepsilon \in P$ enthalten, wo $A \in V \setminus \{S\}$.

ε -Produktionen: 2. Sonderregel

Weitere Sonderregel:

2. Sonderregel: ε -Produktionen in Typ 2- und Typ 3-Grammatiken

Eine Grammatik $G = (V, \Sigma, P, S)$ vom Typ 2 oder 3 darf Produktionen von der Form $A \rightarrow \varepsilon \in P$ enthalten, wo $A \in V \setminus \{S\}$.

Das ist keine echte Erweiterung, denn:

Satz (Entfernen von ε -Produktionen)

Sei $G = (V, \Sigma, P, S)$ eine Typ i -Grammatik (mit 1. und 2. Sonderregel), wo $i \in \{2, 3\}$. Dann gibt es eine Typ i -Grammatik G' mit $L(G') = L(G)$ und G' enthält keine ε -Produktionen außer $S \rightarrow \varepsilon$, falls vorhanden.

ε -Produktionen: 2. Sonderregel

Weitere Sonderregel:

2. Sonderregel: ε -Produktionen in Typ 2- und Typ 3-Grammatiken

Eine Grammatik $G = (V, \Sigma, P, S)$ vom Typ 2 oder 3 darf Produktionen von der Form $A \rightarrow \varepsilon \in P$ enthalten, wo $A \in V \setminus \{S\}$.

Das ist keine echte Erweiterung, denn:

Satz (Entfernen von ε -Produktionen)

Sei $G = (V, \Sigma, P, S)$ eine Typ i -Grammatik (mit 1. und 2. Sonderregel), wo $i \in \{2, 3\}$. Dann gibt es eine Typ i -Grammatik G' mit $L(G') = L(G)$ und G' enthält keine ε -Produktionen außer $S \rightarrow \varepsilon$, falls vorhanden.

Beweis Algorithmus 1 auf späterer Folie erzeugt G' . □

Entfernen von ε -Produktionen

Intuitiver Ansatz:

1. Wiederhole bis alle ε -Produktionen (außer $S \rightarrow \varepsilon$, falls vorhanden) entfernt sind:
 - 1.1 Sei $A \rightarrow \varepsilon \in P$, wo $A \neq S$.
 - 1.2 Entferne $A \rightarrow \varepsilon$.
 - 1.3 Für jede Produktion $B \rightarrow uAv$ füge eine neue Produktion $B \rightarrow uv$ hinzu, die den Ableitungsschritt $A \Rightarrow \varepsilon$ vorwegnimmt.

Entfernen von ε -Produktionen

Intuitiver Ansatz:

1. Wiederhole bis alle ε -Produktionen (außer $S \rightarrow \varepsilon$, falls vorhanden) entfernt sind:
 - 1.1 Sei $A \rightarrow \varepsilon \in P$, wo $A \neq S$.
 - 1.2 Entferne $A \rightarrow \varepsilon$.
 - 1.3 Für jede Produktion $B \rightarrow uAv$ füge eine neue Produktion $B \rightarrow uv$ hinzu, die den Ableitungsschritt $A \Rightarrow \varepsilon$ vorwegnimmt.

Mit anderen Worten: Statt $B \rightarrow uAv$ gefolgt von $A \rightarrow \varepsilon$ anzuwenden, können wir also $B \rightarrow uv$ direkt anwenden. Dann brauchen wir $A \rightarrow \varepsilon$ nicht mehr.

Entfernen von ε -Produktionen

Intuitiver Ansatz:

1. Wiederhole bis alle ε -Produktionen (außer $S \rightarrow \varepsilon$, falls vorhanden) entfernt sind:
 - 1.1 Sei $A \rightarrow \varepsilon \in P$, wo $A \neq S$.
 - 1.2 Entferne $A \rightarrow \varepsilon$.
 - 1.3 Für jede Produktion $B \rightarrow uAv$ füge eine neue Produktion $B \rightarrow uv$ hinzu, die den Ableitungsschritt $A \Rightarrow \varepsilon$ vorwegnimmt.

Mit anderen Worten: Statt $B \rightarrow uAv$ gefolgt von $A \rightarrow \varepsilon$ anzuwenden, können wir also $B \rightarrow uv$ direkt anwenden. Dann brauchen wir $A \rightarrow \varepsilon$ nicht mehr.

Für eine reguläre Produktion $B \rightarrow aA$ wird $B \rightarrow a$ hinzugefügt.
Die Grammatik bleibt regulär.

Entfernen von ε -Produktionen

Intuitiver Ansatz:

1. Wiederhole bis alle ε -Produktionen (außer $S \rightarrow \varepsilon$, falls vorhanden) entfernt sind:
 - 1.1 Sei $A \rightarrow \varepsilon \in P$, wo $A \neq S$.
 - 1.2 Entferne $A \rightarrow \varepsilon$.
 - 1.3 Für jede Produktion $B \rightarrow uAv$ füge eine neue Produktion $B \rightarrow uv$ hinzu, die den Ableitungsschritt $A \Rightarrow \varepsilon$ vorwegnimmt.

Mit anderen Worten: Statt $B \rightarrow uAv$ gefolgt von $A \rightarrow \varepsilon$ anzuwenden, können wir also $B \rightarrow uv$ direkt anwenden. Dann brauchen wir $A \rightarrow \varepsilon$ nicht mehr.

Für eine reguläre Produktion $B \rightarrow aA$ wird $B \rightarrow a$ hinzugefügt.
Die Grammatik bleibt regulär.

Algorithmus 1 basiert auf dieser Idee, entfernt aber alle ε -Produktionen gleichzeitig.

Beispiel für das Entfernen von ε -Produktionen

Typ 2-Grammatik mit ε -Produktion:

$G = (\{A, S\}, \{0, 1\}, P, S)$ mit $P = \{S \rightarrow AA1, A \rightarrow 0, A \rightarrow \varepsilon\}$

Beispiel für das Entfernen von ε -Produktionen

Typ 2-Grammatik mit ε -Produktion:

$G = (\{A, S\}, \{0, 1\}, P, S)$ mit $P = \{S \rightarrow AA1, A \rightarrow 0, A \rightarrow \varepsilon\}$

$L(G) = \{001, 01, 1\}$

Beispiel für das Entfernen von ε -Produktionen

Typ 2-Grammatik **mit** ε -Produktion:

$G = (\{A, S\}, \{0, 1\}, P, S)$ mit $P = \{S \rightarrow AA1, A \rightarrow 0, A \rightarrow \varepsilon\}$

$L(G) = \{001, 01, 1\}$

Äquivalente Typ 2-Grammatik **ohne** ε -Produktion:

$G' = (\{A, S\}, \{0, 1\}, P', S)$ mit $P' = \{S \rightarrow AA1, S \rightarrow A1, S \rightarrow 1, A \rightarrow 0\}$

Beispiel für das Entfernen von ε -Produktionen

Typ 2-Grammatik **mit** ε -Produktion:

$G = (\{A, S\}, \{0, 1\}, P, S)$ mit $P = \{S \rightarrow AA1, A \rightarrow 0, A \rightarrow \varepsilon\}$

$L(G) = \{001, 01, 1\}$

Äquivalente Typ 2-Grammatik **ohne** ε -Produktion:

$G' = (\{A, S\}, \{0, 1\}, P', S)$ mit $P' = \{S \rightarrow AA1, S \rightarrow A1, S \rightarrow 1, A \rightarrow 0\}$

$L(G') = \{001, 01, 1\}$

Beispiel für das Entfernen von ε -Produktionen

Typ 2-Grammatik **mit** ε -Produktion:

$G = (\{A, S\}, \{0, 1\}, P, S)$ mit $P = \{S \rightarrow AA1, A \rightarrow 0, A \rightarrow \varepsilon\}$

$L(G) = \{001, 01, 1\}$

Äquivalente Typ 2-Grammatik **ohne** ε -Produktion:

$G' = (\{A, S\}, \{0, 1\}, P', S)$ mit $P' = \{S \rightarrow AA1, S \rightarrow A1, S \rightarrow 1, A \rightarrow 0\}$

$L(G') = \{001, 01, 1\}$

Die neuen Produktionen $S \rightarrow A1$, $S \rightarrow 1$ nehmen den Ableitungsschritt $A \Rightarrow \varepsilon$ vorweg.

Algorithmus 1: Entfernen von ε -Produktionen

Eingabe: Typ i -Grammatik $G = (V, \Sigma, P, S)$ mit ε -Produktionen, $i \in \{2, 3\}$

Ausgabe: Typ i -Grammatik G' ohne ε -Produktionen (außer $S \rightarrow \varepsilon$, falls vorhanden), sodass $L(G') = L(G)$

Beginn

finde die Menge $W \subseteq V$ aller Variablen A für die gilt $A \Rightarrow^* \varepsilon$:

Beginn

$W := \{A \mid A \rightarrow \varepsilon \in P \text{ und } A \neq S\};$

wiederhole

füge alle A zu W hinzu mit $A \rightarrow A_1 \dots A_n \in P$ und $\forall i : A_i \in W$;

bis sich W nicht mehr ändert;

Ende

$P' := P \setminus \{A \rightarrow \varepsilon \mid A \rightarrow \varepsilon \in P \text{ und } A \neq S\};$

/ lösche Produktionen $A \rightarrow \varepsilon$ */*

wiederhole

für alle Produktionen $B \rightarrow uAv \in P'$ mit $|uv| > 0$ und $A \in W$ **tue**

füge die Produktion $B \rightarrow uv$ zu P' hinzu;

/ für $B \rightarrow u'Av'Aw'$ gibt es (mindestens) zwei Hinzufügungen: Für das Vorkommen von A direkt nach u' als auch für das Vorkommen direkt vor w' */*

Ende

bis sich P' nicht mehr ändert;

gib $G' = (V, \Sigma, P', S)$ als Ergebnisgrammatik aus;

Ende

Weiteres Beispiel für das Entfernen von ε -Produktionen

$G = (\{A, B, C, D, S\}, \{0, 1\}, P, S)$ mit
 $P = \{S \rightarrow 1A, A \rightarrow AB, A \rightarrow DA, A \rightarrow \varepsilon, B \rightarrow 0,$
 $B \rightarrow 1, C \rightarrow AAA, D \rightarrow 1AC\}$

Weiteres Beispiel für das Entfernen von ε -Produktionen

$G = (\{A, B, C, D, S\}, \{0, 1\}, P, S)$ mit
 $P = \{S \rightarrow 1A, A \rightarrow AB, A \rightarrow DA, A \rightarrow \varepsilon, B \rightarrow 0,$
 $B \rightarrow 1, C \rightarrow AAA, D \rightarrow 1AC\}$

1. Finde Menge W der Variablen, die ε herleiten:

$W = \{A, C\}$ da $A \rightarrow \varepsilon$ und $C \rightarrow AAA$

Weiteres Beispiel für das Entfernen von ε -Produktionen

$G = (\{A, B, C, D, S\}, \{0, 1\}, P, S)$ mit
 $P = \{S \rightarrow 1A, A \rightarrow AB, A \rightarrow DA, A \rightarrow \varepsilon, B \rightarrow 0,$
 $B \rightarrow 1, C \rightarrow AAA, D \rightarrow 1AC\}$

1. Finde Menge W der Variablen, die ε herleiten:

$W = \{A, C\}$ da $A \rightarrow \varepsilon$ und $C \rightarrow AAA$

2. Starte mit

$P' = \{S \rightarrow 1A, A \rightarrow AB, A \rightarrow DA, B \rightarrow 0,$
 $B \rightarrow 1, C \rightarrow AAA, D \rightarrow 1AC\}$

Weiteres Beispiel für das Entfernen von ε -Produktionen

$G = (\{A, B, C, D, S\}, \{0, 1\}, P, S)$ mit
 $P = \{S \rightarrow 1A, A \rightarrow AB, A \rightarrow DA, A \rightarrow \varepsilon, B \rightarrow 0,$
 $B \rightarrow 1, C \rightarrow AAA, D \rightarrow 1AC\}$

1. Finde Menge W der Variablen, die ε herleiten:

$W = \{A, C\}$ da $A \rightarrow \varepsilon$ und $C \rightarrow AAA$

2. Starte mit

$P' = \{S \rightarrow 1A, A \rightarrow AB, A \rightarrow DA, B \rightarrow 0,$
 $B \rightarrow 1, C \rightarrow AAA, D \rightarrow 1AC\}$

3. Füge Produktionen für Vorkommen von A und C hinzu:

$P' = \{S \rightarrow 1A, S \rightarrow 1, A \rightarrow AB, A \rightarrow B, A \rightarrow DA, A \rightarrow D,$
 $B \rightarrow 0, B \rightarrow 1, C \rightarrow AAA, C \rightarrow AA, C \rightarrow A,$
 $D \rightarrow 1AC, D \rightarrow 1A, D \rightarrow 1C, D \rightarrow 1\}$

2b

Deterministische endliche Automaten

Prof. Dr. Jasmin Blanchette

Lehr- und Forschungseinheit für
Theoretische Informatik und Theorembeweisen

Stand: 10. Mai 2024
Basiert auf Folien von PD Dr. David Sabel



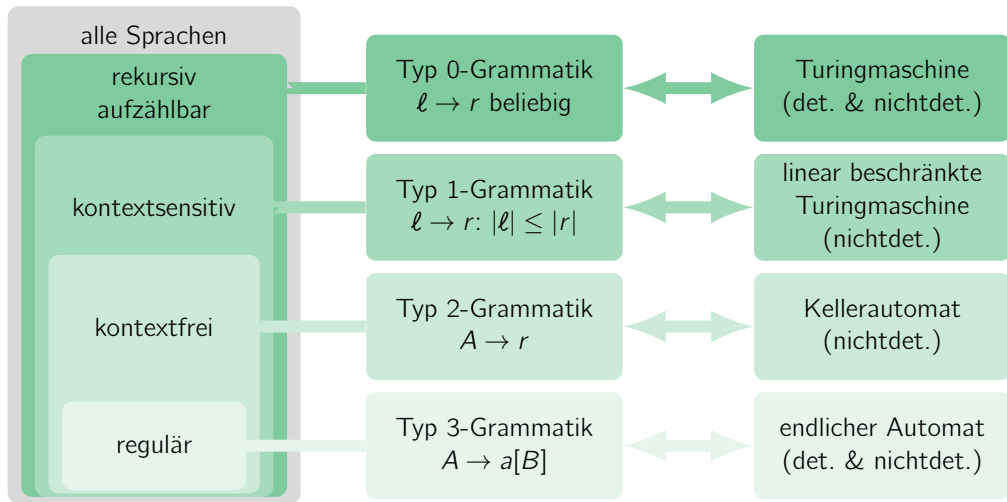
Formalismen, um formale Sprachen zu repräsentieren:

- ▶ **Grammatiken**: Sie **erzeugen** Wörter einer Sprache.
- ▶ **Maschinenmodelle**: Sie **erkennen** Wörter einer Sprache.

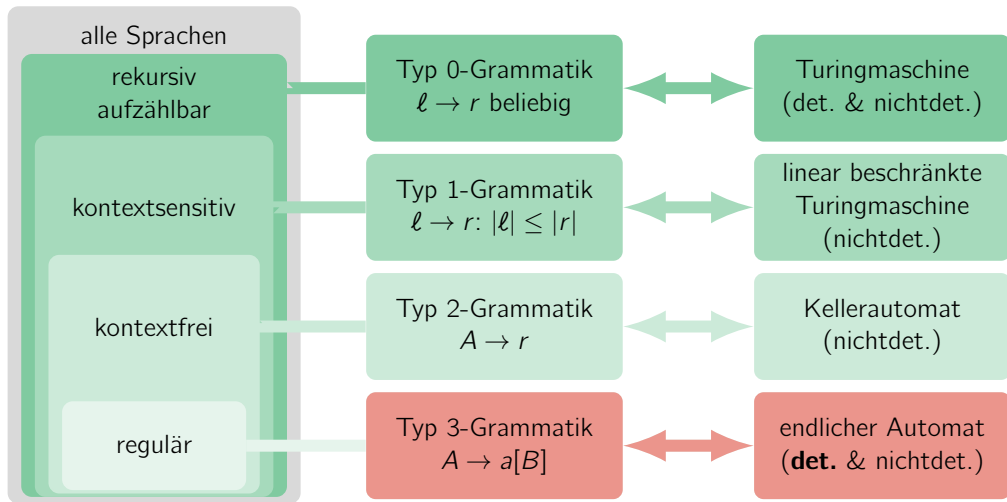
Welcher Formalismus besser ist, hängt von der konkreten Fragestellung ab.

Wichtige Fragestellung: **Welche Maschine akzeptiert welche Sprachklasse?**

Überblick über Grammatiken und Maschinenmodelle



Überblick über Grammatiken und Maschinenmodelle



Wiederholung: Reguläre Sprachen

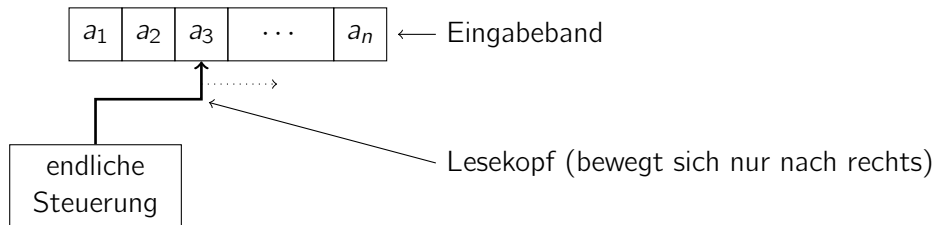
- ▶ Eine Grammatik $G = (V, \Sigma, P, S)$ ist vom Typ 3 (alternativ regulär), wenn alle Produktionen in P von der Form $A \rightarrow r \in P$ sind, wobei $r = a$ oder $r = aA'$ für $a \in \Sigma, A' \in V$.
- ▶ Eine Sprache $L \subseteq \Sigma^*$ ist vom Typ 3 (alternativ regulär), falls es eine Typ 3-Grammatik G gibt, sodass $L(G) = L$ gilt.

Deterministische endliche Automaten

Informelle Kurzfassung:

- ▶ Deterministische endliche Automaten starten im **Startzustand**.
- ▶ Sie lesen zeichenweise ein **Eingabewort**.
- ▶ Sie wechseln dabei den **Zustand** (eindeutig). Es gibt nur endlich viele Zustände.
- ▶ Nach Lesen der Eingabe: **akzeptieren** oder **verwerfen**.
 - ▶ Akzeptieren = in einem **Endzustand**
 - ▶ Verwerfen = nicht in einem **Endzustand**
- ▶ Die **akzeptierte Sprache** besteht aus allen Wörter, für die der Automat akzeptiert.

Illustration eines endlichen Automaten



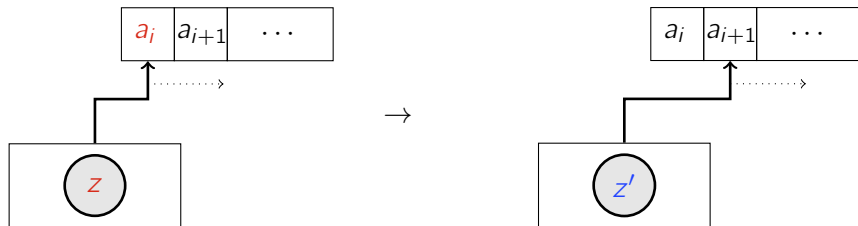
Definition eines DFA

Definition

Ein **deterministischer endlicher Automat** (*deterministic finite automaton*, **DFA**) ist ein 5-Tupel $M = (Z, \Sigma, \delta, z_0, E)$, wobei:

- ▶ Z ist eine endliche Menge von **Zuständen**
- ▶ Σ ist das (endliche) **Eingabealphabet** mit $Z \cap \Sigma = \emptyset$
- ▶ $\delta : Z \times \Sigma \rightarrow Z$ ist die (totale) **Überföhrungsfunktion**
- ▶ $z_0 \in Z$ ist der **Startzustand**
- ▶ $E \subseteq Z$ ist die Menge der **Endzustände**.

Illustration des Zustandsübergangs

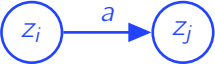


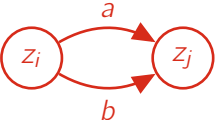
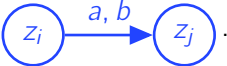
$\delta(z, a_i) = z'$ bedeutet:

Im Zustand z bei Eingabe a_i wechselt der DFA in z' .

Zustandsgraph eines DFA

Für DFA $M = (Z, \Sigma, \delta, z_0, E)$:

- ▶ Für **Zustand** $z \in Z$ gibt es Knoten .
- ▶ **Startzustand** $z_0 \in Z$: eingehender Pfeil .
- ▶ **Endzustände** $z \in E$: doppelte Kreise .
- ▶ **Übergänge** $\delta(z_i, a) = z_j$ als Kante  dargestellt

und statt  zeichnen wir .

Beispiel für einen DFA

DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, \{z_2\})$ mit

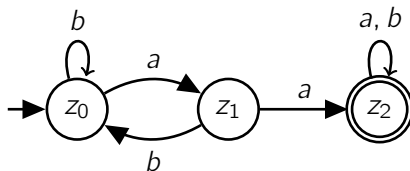
$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(z_1, a) = z_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(z_2, b) = z_2 \end{array}$$

Beispiel für einen DFA

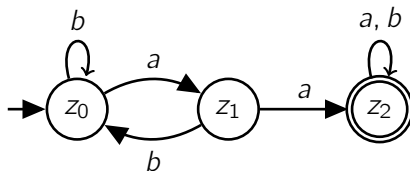
DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, \{z_2\})$ mit

$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(z_1, a) = z_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(z_2, b) = z_2 \end{array}$$

Zustandsgraph zu M :

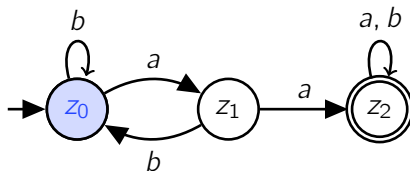


Beispiel für einen akzeptierenden Lauf



Abarbeitung der Eingabe *abbaaa*:

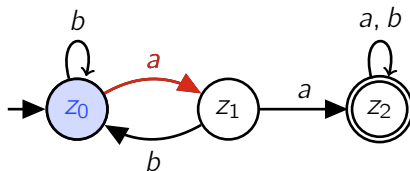
Beispiel für einen akzeptierenden Lauf



Abarbeitung der Eingabe *abbaaa*:

- Starte in z_0 .

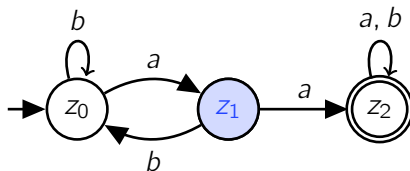
Beispiel für einen akzeptierenden Lauf



Abarbeitung der Eingabe *abbaaa*:

- ▶ Starte in z_0 .
- ▶ Lies a

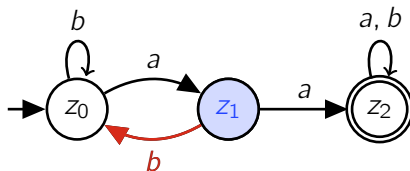
Beispiel für einen akzeptierenden Lauf



Abarbeitung der Eingabe *abbaaa*:

- ▶ Starte in z_0 .
- ▶ Lies a und wechsle in z_1 .

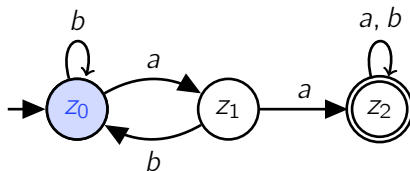
Beispiel für einen akzeptierenden Lauf



Abarbeitung der Eingabe $a**b**aaaa$:

- ▶ Starte in z_0 .
- ▶ Lies a und wechsle in z_1 .
- ▶ Lies b

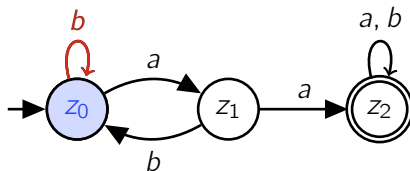
Beispiel für einen akzeptierenden Lauf



Abarbeitung der Eingabe $a**b**aaaa$:

- ▶ Starte in z_0 .
- ▶ Lies a und wechsle in z_1 .
- ▶ Lies b und wechsle in z_0 .

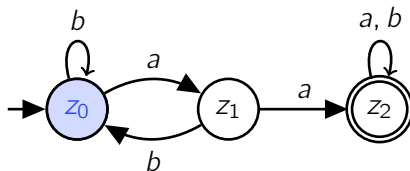
Beispiel für einen akzeptierenden Lauf



Abarbeitung der Eingabe $abbaaa$:

- ▶ Starte in z_0 .
- ▶ Lies a und wechsle in z_1 .
- ▶ Lies b und wechsle in z_0 .
- ▶ Lies b

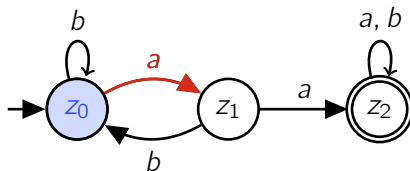
Beispiel für einen akzeptierenden Lauf



Abarbeitung der Eingabe $abbaaa$:

- ▶ Starte in z_0 .
- ▶ Lies a und wechsle in z_1 .
- ▶ Lies b und wechsle in z_0 .
- ▶ Lies b und wechsle in z_0 .

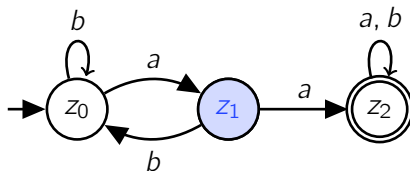
Beispiel für einen akzeptierenden Lauf



Abarbeitung der Eingabe *abb^aaa*:

- ▶ Starte in z_0 .
- ▶ Lies *a* und wechsle in z_1 .
- ▶ Lies *b* und wechsle in z_0 .
- ▶ Lies *b* und wechsle in z_0 .
- ▶ Lies *a*

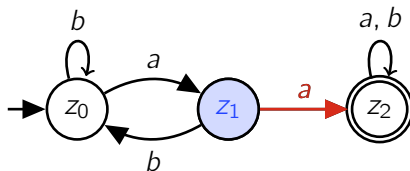
Beispiel für einen akzeptierenden Lauf



Abarbeitung der Eingabe *abb***a***aa*:

- ▶ Starte in z_0 .
- ▶ Lies a und wechsle in z_1 .
- ▶ Lies b und wechsle in z_0 .
- ▶ Lies b und wechsle in z_0 .
- ▶ Lies a und wechsle in z_1 .

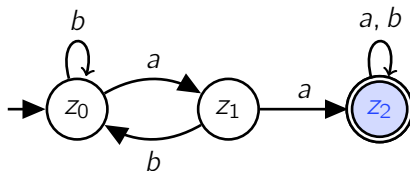
Beispiel für einen akzeptierenden Lauf



Abarbeitung der Eingabe *abba****a****a*:

- ▶ Starte in z_0 .
- ▶ Lies a und wechsle in z_1 .
- ▶ Lies b und wechsle in z_0 .
- ▶ Lies b und wechsle in z_0 .
- ▶ Lies a und wechsle in z_1 .
- ▶ Lies a

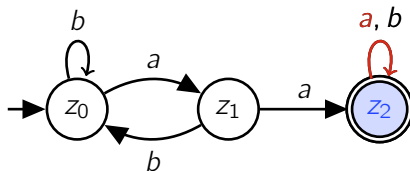
Beispiel für einen akzeptierenden Lauf



Abarbeitung der Eingabe *abba***a***a*:

- ▶ Starte in z_0 .
- ▶ Lies a und wechsle in z_1 .
- ▶ Lies b und wechsle in z_0 .
- ▶ Lies b und wechsle in z_0 .
- ▶ Lies a und wechsle in z_1 .
- ▶ Lies a und wechsle in z_2 .

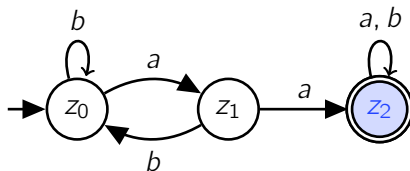
Beispiel für einen akzeptierenden Lauf



Abarbeitung der Eingabe *abb^aaa^a*:

- ▶ Starte in z_0 .
- ▶ Lies a und wechsle in z_1 .
- ▶ Lies b und wechsle in z_0 .
- ▶ Lies b und wechsle in z_0 .
- ▶ Lies a und wechsle in z_1 .
- ▶ Lies a und wechsle in z_2 .
- ▶ Lies a

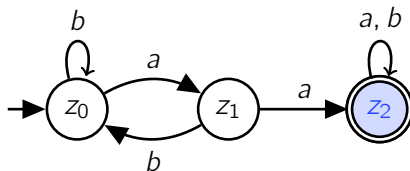
Beispiel für einen akzeptierenden Lauf



Abarbeitung der Eingabe *abb^aaa*:

- ▶ Starte in z_0 .
- ▶ Lies a und wechsle in z_1 .
- ▶ Lies b und wechsle in z_0 .
- ▶ Lies b und wechsle in z_0 .
- ▶ Lies a und wechsle in z_1 .
- ▶ Lies a und wechsle in z_2 .
- ▶ Lies a und wechsle in z_2 .

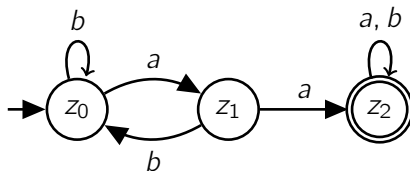
Beispiel für einen akzeptierenden Lauf



Abarbeitung der Eingabe *abbaaa*:

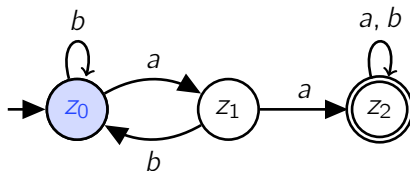
- ▶ Starte in z_0 .
- ▶ Lies a und wechsle in z_1 .
- ▶ Lies b und wechsle in z_0 .
- ▶ Lies b und wechsle in z_0 .
- ▶ Lies a und wechsle in z_1 .
- ▶ Lies a und wechsle in z_2 .
- ▶ Lies a und wechsle in z_2 .
- ▶ Akzeptiere.

Beispiel für einen verwerfenden Lauf



Abarbeitung der Eingabe *bbab*:

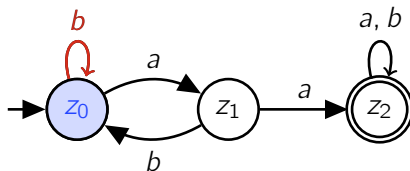
Beispiel für einen verwerfenden Lauf



Abarbeitung der Eingabe $bbab$:

- Starte in z_0 .

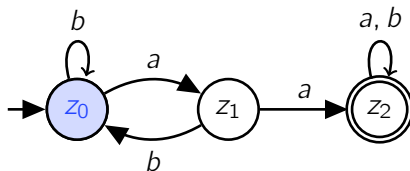
Beispiel für einen verwerfenden Lauf



Abarbeitung der Eingabe $bbab$:

- ▶ Starte in z_0 .
- ▶ Lies b

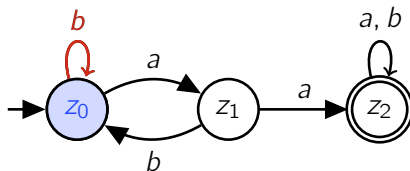
Beispiel für einen verwerfenden Lauf



Abarbeitung der Eingabe $bbab$:

- ▶ Starte in z_0 .
- ▶ Lies b und wechsle in z_0 .

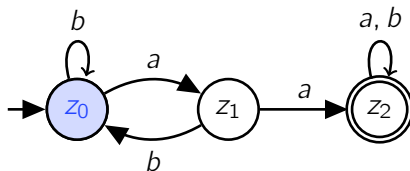
Beispiel für einen verwerfenden Lauf



Abarbeitung der Eingabe $b\textcolor{red}{b}ab$:

- ▶ Starte in z_0 .
- ▶ Lies b und wechsle in z_0 .
- ▶ Lies b

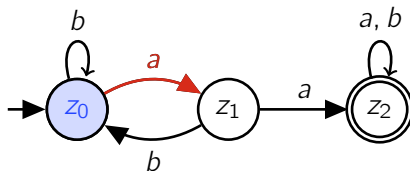
Beispiel für einen verwerfenden Lauf



Abarbeitung der Eingabe $bbab$:

- ▶ Starte in z_0 .
- ▶ Lies b und wechsle in z_0 .
- ▶ Lies b und wechsle in z_0 .

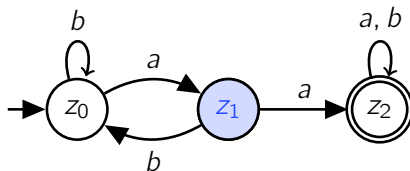
Beispiel für einen verwerfenden Lauf



Abarbeitung der Eingabe $bbab$:

- ▶ Starte in z_0 .
- ▶ Lies b und wechsle in z_0 .
- ▶ Lies b und wechsle in z_0 .
- ▶ Lies a

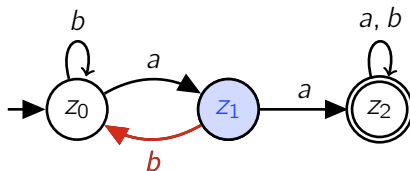
Beispiel für einen verwerfenden Lauf



Abarbeitung der Eingabe *bbab*:

- ▶ Starte in z_0 .
- ▶ Lies b und wechsle in z_0 .
- ▶ Lies b und wechsle in z_0 .
- ▶ Lies a und wechsle in z_1 .

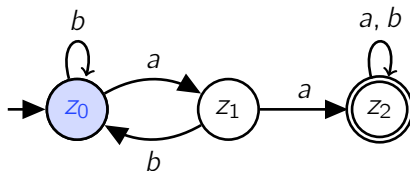
Beispiel für einen verwerfenden Lauf



Abarbeitung der Eingabe *bbab*:

- ▶ Starte in z_0 .
- ▶ Lies b und wechsle in z_0 .
- ▶ Lies b und wechsle in z_0 .
- ▶ Lies a und wechsle in z_1 .
- ▶ Lies b

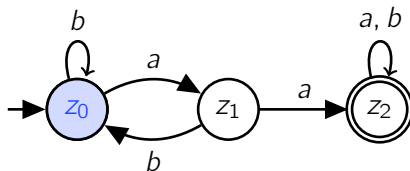
Beispiel für einen verwerfenden Lauf



Abarbeitung der Eingabe *bbab*:

- ▶ Starte in z_0 .
- ▶ Lies b und wechsle in z_0 .
- ▶ Lies b und wechsle in z_0 .
- ▶ Lies a und wechsle in z_1 .
- ▶ Lies b und wechsle in z_0 .

Beispiel für einen verwerfenden Lauf



Abarbeitung der Eingabe *bbab*:

- ▶ Starte in z_0 .
- ▶ Lies b und wechsle in z_0 .
- ▶ Lies b und wechsle in z_0 .
- ▶ Lies a und wechsle in z_1 .
- ▶ Lies b und wechsle in z_0 .
- ▶ **Verwirf.**

Definition

Sei $M = (Z, \Sigma, \delta, z_0, E)$ ein DFA.

Wir definieren $\tilde{\delta} : Z \times \Sigma^* \rightarrow Z$ rekursiv durch

$$\begin{aligned}\tilde{\delta}(z, \epsilon) &:= z \\ \tilde{\delta}(z, aw) &:= \tilde{\delta}(\delta(z, a), w)\end{aligned}$$

Die von M akzeptierte Sprache ist

$$L(M) := \{w \in \Sigma^* \mid \tilde{\delta}(z_0, w) \in E\}$$

Definition

Sei $M = (Z, \Sigma, \delta, z_0, E)$ ein DFA.

Wir definieren $\tilde{\delta} : Z \times \Sigma^* \rightarrow Z$ rekursiv durch

$$\begin{aligned}\tilde{\delta}(z, \epsilon) &:= z \\ \tilde{\delta}(z, aw) &:= \tilde{\delta}(\delta(z, a), w)\end{aligned}$$

Die von M akzeptierte Sprache ist

$$L(M) := \{w \in \Sigma^* \mid \tilde{\delta}(z_0, w) \in E\}$$

Alternativ:

$$\tilde{\delta}(z, a_1 a_2 \dots a_n) = \delta(\dots \delta(\delta(z, a_1), a_2) \dots, a_n)$$

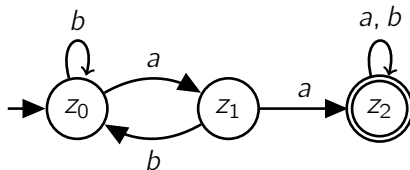
$\tilde{\delta}$ wendet δ solange an, bis das Eingabewort abgearbeitet ist.

Beispiel für die $\tilde{\delta}$ -Funktion

DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, E)$ mit $E = \{z_2\}$ und

$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(z_1, a) = z_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(z_2, b) = z_2 \end{array}$$

Zustandsgraph zu M :



Abarbeitung der Eingabe *abbaaa*:

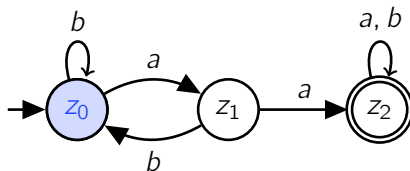
$$\tilde{\delta}(z_0, abbaaa) = \delta(\delta(\delta(\delta(\delta(\delta(z_0, a), b), b), a), a), a), a)$$

Beispiel für die $\tilde{\delta}$ -Funktion

DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, E)$ mit $E = \{z_2\}$ und

$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(z_1, a) = z_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(z_2, b) = z_2 \end{array}$$

Zustandsgraph zu M :



Abarbeitung der Eingabe *abbaaa*:

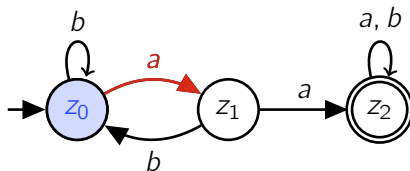
$$\tilde{\delta}(z_0, abbaaa) = \delta(\delta(\delta(\delta(\delta(\delta(z_0, a), b), b), a), a), a), a)$$

Beispiel für die $\tilde{\delta}$ -Funktion

DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, E)$ mit $E = \{z_2\}$ und

$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(z_1, a) = z_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(z_2, b) = z_2 \end{array}$$

Zustandsgraph zu M :



Abarbeitung der Eingabe *abbaaa*:

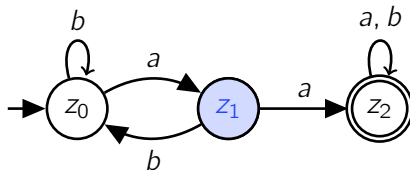
$$\tilde{\delta}(z_0, abbaaa) = \delta(\delta(\delta(\delta(\delta(\delta(z_0, a), b), b), a), a), a), a)$$

Beispiel für die $\tilde{\delta}$ -Funktion

DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, E)$ mit $E = \{z_2\}$ und

$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(z_1, a) = z_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(z_2, b) = z_2 \end{array}$$

Zustandsgraph zu M :



Abarbeitung der Eingabe *abb***aaa**:

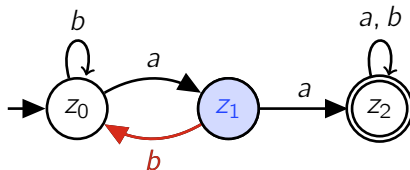
$$\tilde{\delta}(z_0, \text{abb} \mathbf{aaa}) = \delta(\delta(\delta(\delta(\delta(\mathbf{z_1}, b), b), a), a), a), a)$$

Beispiel für die $\tilde{\delta}$ -Funktion

DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, E)$ mit $E = \{z_2\}$ und

$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(z_1, a) = z_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(z_2, b) = z_2 \end{array}$$

Zustandsgraph zu M :



Abarbeitung der Eingabe $a**b**aaaa$:

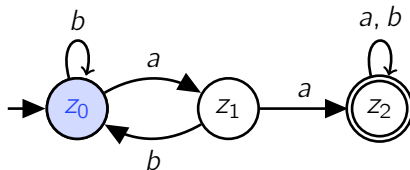
$$\tilde{\delta}(z_0, abbaaa) = \delta(\delta(\delta(\delta(\delta(z_1, \textcolor{red}{b}), b), a), a), a)$$

Beispiel für die $\tilde{\delta}$ -Funktion

DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, E)$ mit $E = \{z_2\}$ und

$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(z_1, a) = z_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(z_2, b) = z_2 \end{array}$$

Zustandsgraph zu M :



Abarbeitung der Eingabe $abbaaa$:

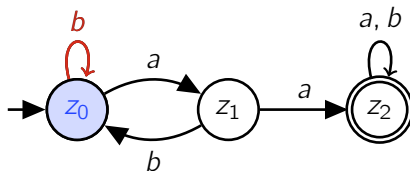
$$\tilde{\delta}(z_0, abbaaa) = \delta(\delta(\delta(\delta(\delta(z_0, b), a), a), a), a), a)$$

Beispiel für die $\tilde{\delta}$ -Funktion

DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, E)$ mit $E = \{z_2\}$ und

$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(z_1, a) = z_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(z_2, b) = z_2 \end{array}$$

Zustandsgraph zu M :



Abarbeitung der Eingabe $abbaaa$:

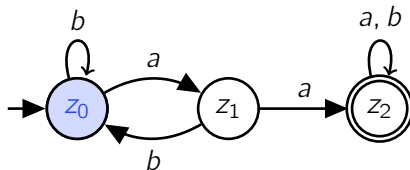
$$\tilde{\delta}(z_0, abbaaa) = \delta(\delta(\delta(\delta(\delta(z_0, b), a), a), a), a), a)$$

Beispiel für die $\tilde{\delta}$ -Funktion

DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, E)$ mit $E = \{z_2\}$ und

$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(z_1, a) = z_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(z_2, b) = z_2 \end{array}$$

Zustandsgraph zu M :



Abarbeitung der Eingabe $abbaaa$:

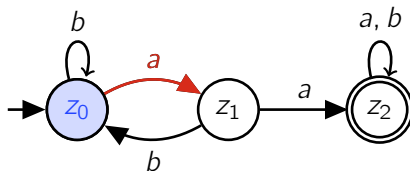
$$\tilde{\delta}(z_0, abbaaa) = \delta(\delta(\delta(\delta(z_0, a), a), b), a), a)$$

Beispiel für die $\tilde{\delta}$ -Funktion

DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, E)$ mit $E = \{z_2\}$ und

$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(z_1, a) = z_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(z_2, b) = z_2 \end{array}$$

Zustandsgraph zu M :



Abarbeitung der Eingabe $abb\textcolor{red}{a}aa$:

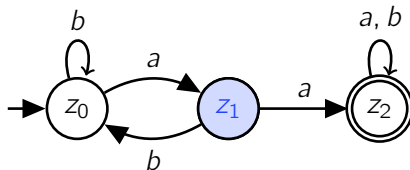
$$\tilde{\delta}(z_0, abb\textcolor{red}{a}aa) = \delta(\delta(\delta(\textcolor{blue}{z}_0, \textcolor{red}{a}), a), a)$$

Beispiel für die $\tilde{\delta}$ -Funktion

DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, E)$ mit $E = \{z_2\}$ und

$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(z_1, a) = z_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(z_2, b) = z_2 \end{array}$$

Zustandsgraph zu M :



Abarbeitung der Eingabe $abb\textcolor{red}{a}aa$:

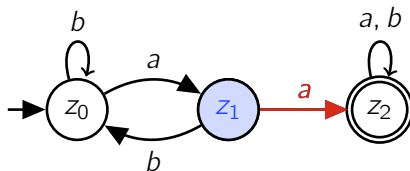
$$\tilde{\delta}(z_0, abb\textcolor{red}{a}aa) = \delta(\delta(\textcolor{blue}{z}_1, a), a)$$

Beispiel für die $\tilde{\delta}$ -Funktion

DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, E)$ mit $E = \{z_2\}$ und

$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(z_1, a) = z_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(z_2, b) = z_2 \end{array}$$

Zustandsgraph zu M :



Abarbeitung der Eingabe $abba$ aa:

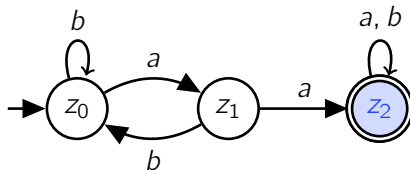
$$\tilde{\delta}(z_0, abbaaa) = \delta(\delta(z_1, a), a)$$

Beispiel für die $\tilde{\delta}$ -Funktion

DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, E)$ mit $E = \{z_2\}$ und

$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(z_1, a) = z_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(z_2, b) = z_2 \end{array}$$

Zustandsgraph zu M :



Abarbeitung der Eingabe *abba***a***a*:

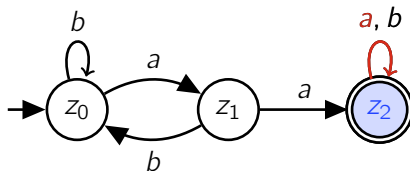
$$\tilde{\delta}(z_0, abb~~a~~aa) = \delta(\textcolor{blue}{z_2}, a)$$

Beispiel für die $\tilde{\delta}$ -Funktion

DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, E)$ mit $E = \{z_2\}$ und

$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(z_1, a) = z_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(z_2, b) = z_2 \end{array}$$

Zustandsgraph zu M :



Abarbeitung der Eingabe $abbaa$ a:

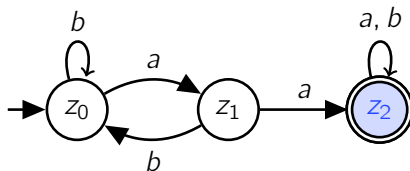
$$\tilde{\delta}(z_0, abbaaa) = \delta(z_2, a)$$

Beispiel für die $\tilde{\delta}$ -Funktion

DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, E)$ mit $E = \{z_2\}$ und

$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(z_1, a) = z_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(z_2, b) = z_2 \end{array}$$

Zustandsgraph zu M :



Abarbeitung der Eingabe *abbaa***a**:

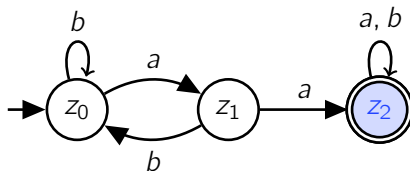
$$\tilde{\delta}(z_0, abbaaa) = z_2$$

Beispiel für die $\tilde{\delta}$ -Funktion

DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, E)$ mit $E = \{z_2\}$ und

$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(z_1, a) = z_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(z_2, b) = z_2 \end{array}$$

Zustandsgraph zu M :



Abarbeitung der Eingabe *abbaaa*:

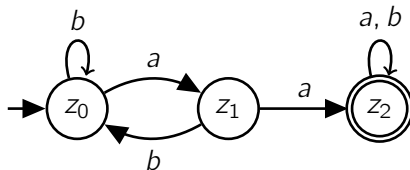
$$\tilde{\delta}(z_0, abbaaa) = z_2 \in E$$

Weiteres Beispiel für die $\tilde{\delta}$ -Funktion

DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, E)$ mit $E = \{z_2\}$ und

$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(z_1, a) = z_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(z_2, b) = z_2 \end{array}$$

Zustandsgraph zu M :



Abarbeitung der Eingabe $bbab$:

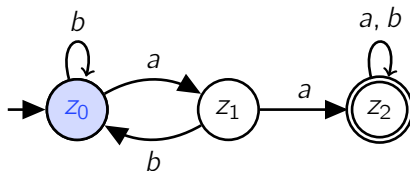
$$\tilde{\delta}(z_0, bbab) = \delta(\delta(\delta(\delta(z_0, b), b), a), b)$$

Weiteres Beispiel für die $\tilde{\delta}$ -Funktion

DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, E)$ mit $E = \{z_2\}$ und

$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(z_1, a) = z_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(z_2, b) = z_2 \end{array}$$

Zustandsgraph zu M :



Abarbeitung der Eingabe $bbab$:

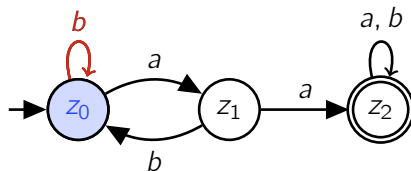
$$\tilde{\delta}(z_0, bbab) = \delta(\delta(\delta(\delta(z_0, b), b), a), b)$$

Weiteres Beispiel für die $\tilde{\delta}$ -Funktion

DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, E)$ mit $E = \{z_2\}$ und

$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(z_1, a) = z_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(z_2, b) = z_2 \end{array}$$

Zustandsgraph zu M :



Abarbeitung der Eingabe $bbab$:

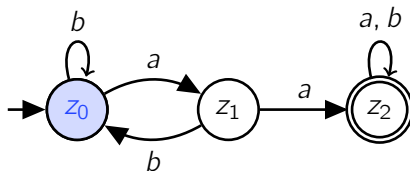
$$\tilde{\delta}(z_0, bbab) = \delta(\delta(\delta(\delta(z_0, b), a), b), b)$$

Weiteres Beispiel für die $\tilde{\delta}$ -Funktion

DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, E)$ mit $E = \{z_2\}$ und

$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(z_1, a) = z_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(z_2, b) = z_2 \end{array}$$

Zustandsgraph zu M :



Abarbeitung der Eingabe $bbab$:

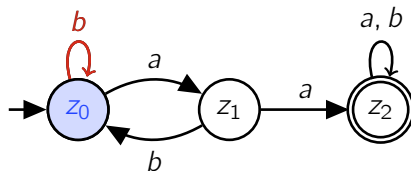
$$\tilde{\delta}(z_0, bbab) = \delta(\delta(\delta(\delta(z_0, b), a), b), a), b)$$

Weiteres Beispiel für die $\tilde{\delta}$ -Funktion

DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, E)$ mit $E = \{z_2\}$ und

$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(z_1, a) = z_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(z_2, b) = z_2 \end{array}$$

Zustandsgraph zu M :



Abarbeitung der Eingabe $b\textcolor{red}{b}ab$:

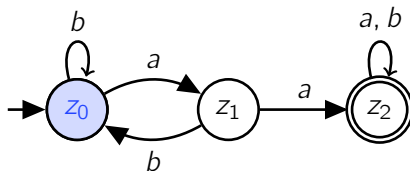
$$\tilde{\delta}(z_0, b\textcolor{red}{b}ab) = \delta(\delta(\delta(\textcolor{blue}{z}_0, \textcolor{red}{b}), a), b)$$

Weiteres Beispiel für die $\tilde{\delta}$ -Funktion

DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, E)$ mit $E = \{z_2\}$ und

$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(z_1, a) = z_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(z_2, b) = z_2 \end{array}$$

Zustandsgraph zu M :



Abarbeitung der Eingabe $bbab$:

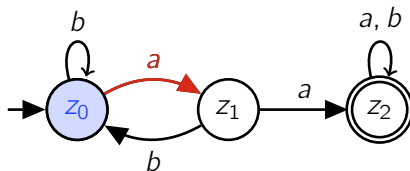
$$\tilde{\delta}(z_0, bbab) = \delta(\delta(z_0, a), b)$$

Weiteres Beispiel für die $\tilde{\delta}$ -Funktion

DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, E)$ mit $E = \{z_2\}$ und

$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(z_1, a) = z_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(z_2, b) = z_2 \end{array}$$

Zustandsgraph zu M :



Abarbeitung der Eingabe $bbab$:

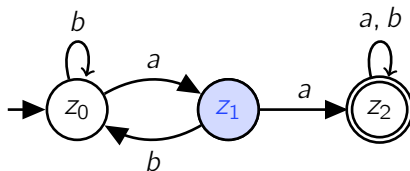
$$\tilde{\delta}(z_0, bbab) = \delta(\delta(z_0, a), b)$$

Weiteres Beispiel für die $\tilde{\delta}$ -Funktion

DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, E)$ mit $E = \{z_2\}$ und

$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(z_1, a) = z_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(z_2, b) = z_2 \end{array}$$

Zustandsgraph zu M :



Abarbeitung der Eingabe $bbab$:

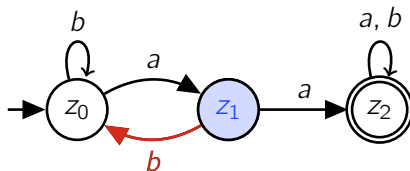
$$\tilde{\delta}(z_0, bbab) = \delta(z_1, b)$$

Weiteres Beispiel für die $\tilde{\delta}$ -Funktion

DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, E)$ mit $E = \{z_2\}$ und

$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(z_1, a) = z_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(z_2, b) = z_2 \end{array}$$

Zustandsgraph zu M :



Abarbeitung der Eingabe $bbab$:

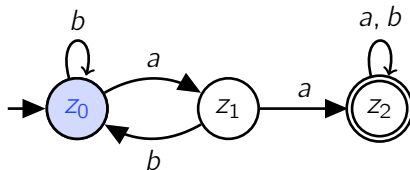
$$\tilde{\delta}(z_0, bbab) = \delta(z_1, b)$$

Weiteres Beispiel für die $\tilde{\delta}$ -Funktion

DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, E)$ mit $E = \{z_2\}$ und

$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(z_1, a) = z_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(z_2, b) = z_2 \end{array}$$

Zustandsgraph zu M :



Abarbeitung der Eingabe $bbab$:

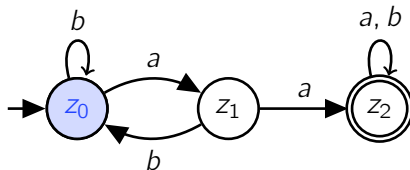
$$\tilde{\delta}(z_0, bbab) = z_0$$

Weiteres Beispiel für die $\tilde{\delta}$ -Funktion

DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, E)$ mit $E = \{z_2\}$ und

$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(z_1, a) = z_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(z_2, b) = z_2 \end{array}$$

Zustandsgraph zu M :



Abarbeitung der Eingabe $bbab$:

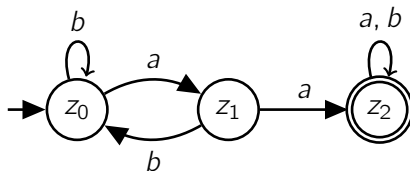
$$\tilde{\delta}(z_0, bbab) = z_0 \notin E$$

Akzeptierte Sprache des Beispiels

DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, E)$ mit $E = \{z_2\}$ und

$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(z_1, a) = z_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(z_2, b) = z_2 \end{array}$$

Zustandsgraph zu M :



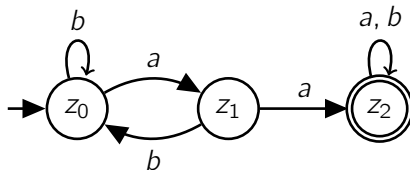
Akzeptierte Sprache = ?

Akzeptierte Sprache des Beispiels

DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, E)$ mit $E = \{z_2\}$ und

$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(z_1, a) = z_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(z_2, b) = z_2 \end{array}$$

Zustandsgraph zu M :



Akzeptierte Sprache = $\{uaav \mid u, v \in \{a, b\}^*\}$

Definition

Seien $M = (Z, \Sigma, \delta, z_0, E)$ ein DFA und $w \in \Sigma^*$ ein Wort der Länge n .

Die Folge von Zuständen $z_0, \dots, z_n$ mit $z_i = \delta(z_{i-1}, w[i])$ für $i \in \{1, \dots, n\}$ ist ein **Lauf** von M für w .

Definition

Seien $M = (Z, \Sigma, \delta, z_0, E)$ ein DFA und $w \in \Sigma^*$ ein Wort der Länge n .

Die Folge von Zuständen $z_0, \dots, z_n$ mit $z_i = \delta(z_{i-1}, w[i])$ für $i \in \{1, \dots, n\}$ ist ein **Lauf** von M für w .

Für einen Lauf schreiben wir auch

$$z_0 \xrightarrow{w[1]} z_1 \xrightarrow{w[2]} \dots \xrightarrow{w[n-1]} z_{n-1} \xrightarrow{w[n]} z_n$$

Beispiele für Läufe

DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, \{z_2\})$ mit

$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(z_1, a) = z_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(z_2, b) = z_2 \end{array}$$

Beispiele für Läufe

DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, \{z_2\})$ mit

$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(z_1, a) = z_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(z_2, b) = z_2 \end{array}$$

Lauf für *abbaaa* (akzeptierend):

$$z_0 \xrightarrow{a} z_1 \xrightarrow{b} z_0 \xrightarrow{b} z_0 \xrightarrow{a} z_1 \xrightarrow{a} z_2 \xrightarrow{a} z_2$$

Beispiele für Läufe

DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, \{z_2\})$ mit

$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(z_1, a) = z_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(z_2, b) = z_2 \end{array}$$

Lauf für *abbaaa* (akzeptierend):

$$z_0 \xrightarrow{a} z_1 \xrightarrow{b} z_0 \xrightarrow{b} z_0 \xrightarrow{a} z_1 \xrightarrow{a} z_2 \xrightarrow{a} z_2$$

Lauf für *bbab* (verwerfend):

$$z_0 \xrightarrow{b} z_0 \xrightarrow{b} z_0 \xrightarrow{a} z_1 \xrightarrow{b} z_0$$

Beispiel für die Konstruktion eines DFA

Konstruiere einen DFA über $\{a, b\}$, der alle Wörter akzeptiert, die mit *abaa* beginnen und mit *bab* enden (z.B. *abaabab*, *abaaaabab*, *abaababab*):

Beispiel für die Konstruktion eines DFA

Konstruiere einen DFA über $\{a, b\}$, der alle Wörter akzeptiert, die mit $abaa$ beginnen und mit bab enden (z.B. *abaabab*, *abaaaabab*, *abaababab*):



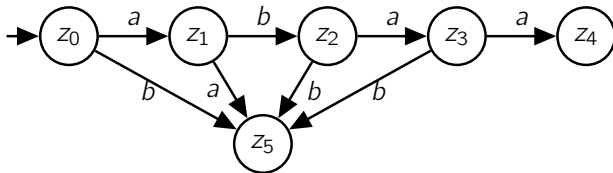
Beispiel für die Konstruktion eines DFA

Konstruiere einen DFA über $\{a, b\}$, der alle Wörter akzeptiert, die mit $abaa$ beginnen und mit bab enden (z.B. *abaabab*, *abaaaabab*, *abaababab*):



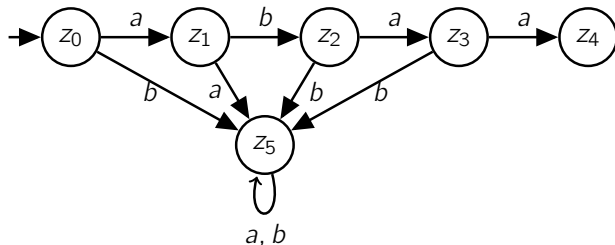
Beispiel für die Konstruktion eines DFA

Konstruiere einen DFA über $\{a, b\}$, der alle Wörter akzeptiert, die mit *abaa* beginnen und mit *bab* enden (z.B. *abaabab*, *abaaaabab*, *abaababab*):



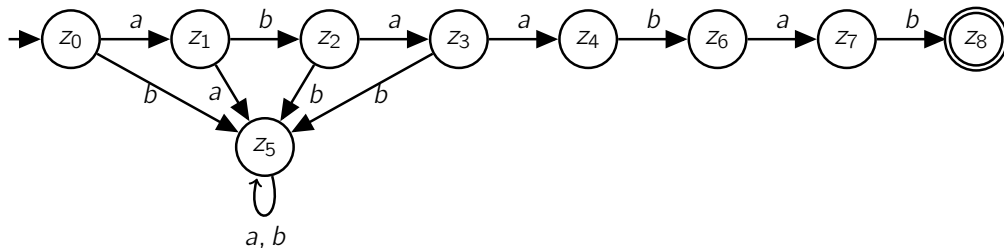
Beispiel für die Konstruktion eines DFA

Konstruiere einen DFA über $\{a, b\}$, der alle Wörter akzeptiert, die mit *abaa* beginnen und mit *bab* enden (z.B. *abaabab*, *abaaaabab*, *abaababab*):



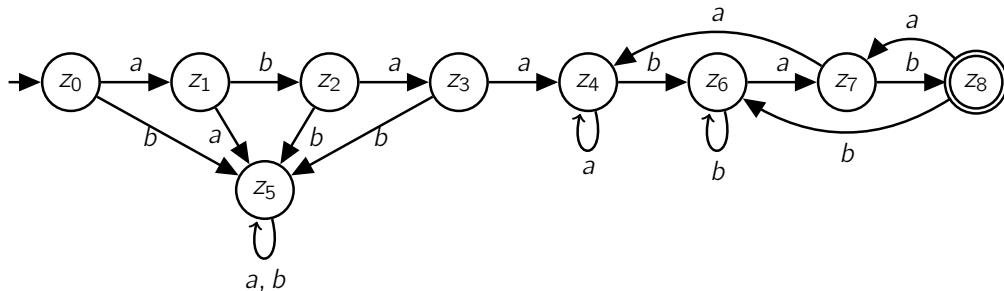
Beispiel für die Konstruktion eines DFA

Konstruiere einen DFA über $\{a, b\}$, der alle Wörter akzeptiert, die mit $abaa$ beginnen und mit bab enden (z.B. *abaabab*, *abaaaabab*, *abaababab*):



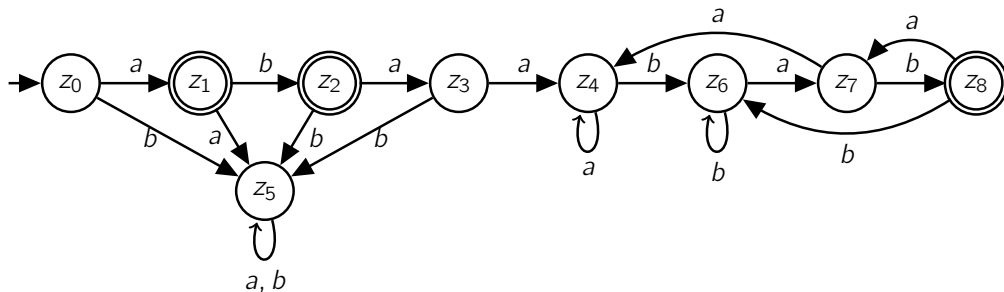
Beispiel für die Konstruktion eines DFA

Konstruiere einen DFA über $\{a, b\}$, der alle Wörter akzeptiert, die mit $abaa$ beginnen und mit bab enden (z.B. *abaabab*, *abaaaabab*, *abaababab*):

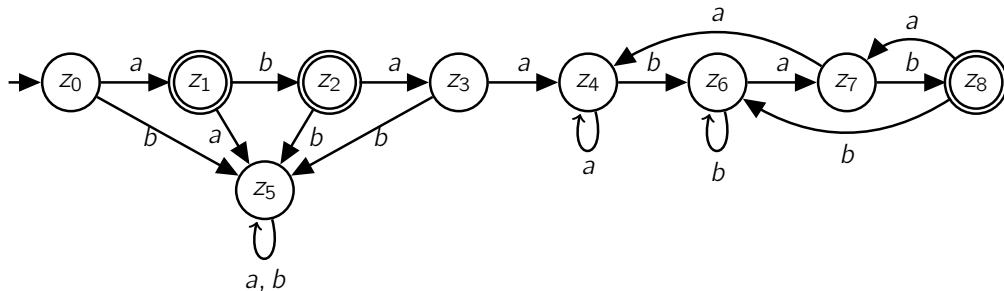


Beispiel für die Konstruktion eines DFA

Zusätzlich die Wörter a und ab akzeptieren:



Beispiel für die Konstruktion eines DFA



Bedeutung der Zustände:

$z_0 = \epsilon$ wurde gelesen

$z_1 = a$ wurde gelesen

$z_2 = ab$ wurde gelesen

$z_3 = aba$ wurde gelesen

$z_4 = abaa$ wurde gelesen

$z_5 =$ ein falsches Präfix wurde gelesen
(Müllzustand)

$z_6 = abaa \dots b$ wurde gelesen

$z_7 = abaa \dots ba$ wurde gelesen

$z_8 = abaa \dots bab$ wurde gelesen

Weiteres Beispiel für die Konstruktion eines DFA

Konstruiere einen DFA über
 $\Sigma = \{+, -, ., 0, \dots, 9\}$,
der alle Wörter akzeptiert,
die Gleitkommazahlen
darstellen
(z.B. $+27$, -3.14 , $.666$):

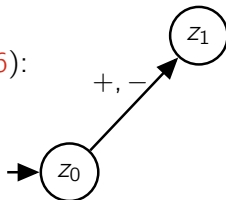
Weiteres Beispiel für die Konstruktion eines DFA

Konstruiere einen DFA über
 $\Sigma = \{+, -, ., 0, \dots, 9\}$,
der alle Wörter akzeptiert,
die Gleitkommazahlen
darstellen
(z.B. $+27$, -3.14 , $.666$):



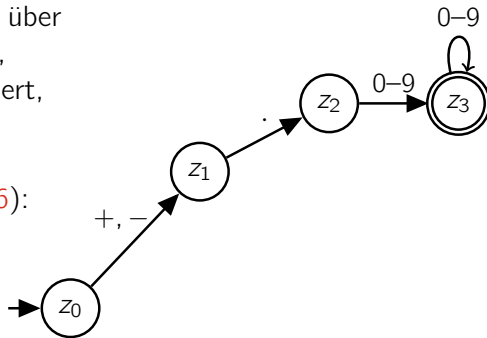
Weiteres Beispiel für die Konstruktion eines DFA

Konstruiere einen DFA über
 $\Sigma = \{+, -, ., 0, \dots, 9\}$,
der alle Wörter akzeptiert,
die Gleitkommazahlen
darstellen
(z.B. $+27$, -3.14 , $.666$):



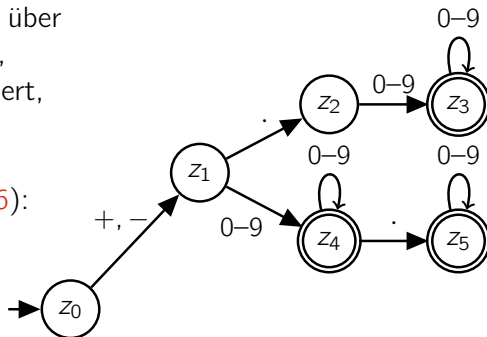
Weiteres Beispiel für die Konstruktion eines DFA

Konstruiere einen DFA über
 $\Sigma = \{+, -, ., 0, \dots, 9\}$,
der alle Wörter akzeptiert,
die Gleitkommazahlen
darstellen
(z.B. $+27$, -3.14 , $.666$):



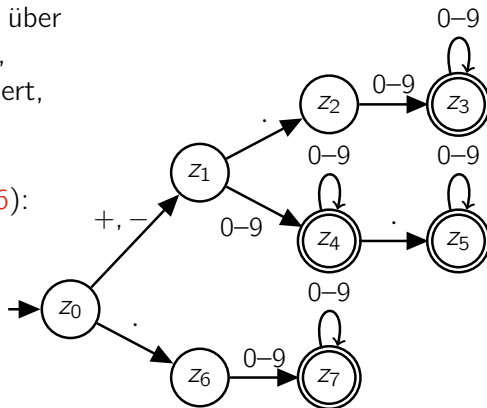
Weiteres Beispiel für die Konstruktion eines DFA

Konstruiere einen DFA über
 $\Sigma = \{+, -, ., 0, \dots, 9\}$,
der alle Wörter akzeptiert,
die Gleitkommazahlen
darstellen
(z.B. $+27$, -3.14 , $.666$):



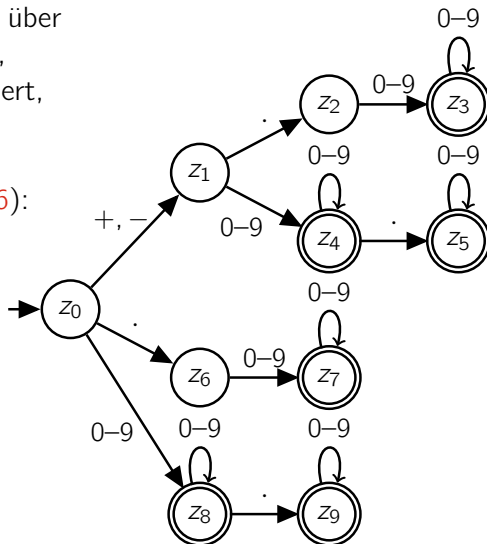
Weiteres Beispiel für die Konstruktion eines DFA

Konstruiere einen DFA über
 $\Sigma = \{+, -, ., 0, \dots, 9\}$,
der alle Wörter akzeptiert,
die Gleitkommazahlen
darstellen
(z.B. $+27$, -3.14 , $.666$):



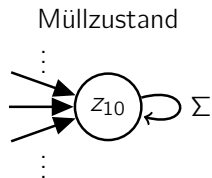
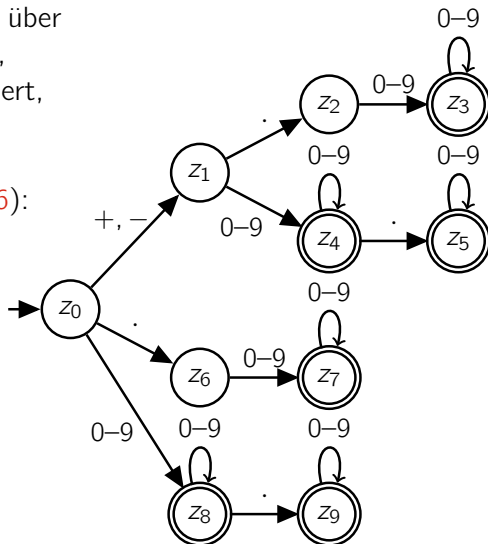
Weiteres Beispiel für die Konstruktion eines DFA

Konstruiere einen DFA über
 $\Sigma = \{+, -, ., 0, \dots, 9\}$,
der alle Wörter akzeptiert,
die Gleitkommazahlen
darstellen
(z.B. $+27$, -3.14 , $.666$):



Weiteres Beispiel für die Konstruktion eines DFA

Konstruiere einen DFA über
 $\Sigma = \{+, -, ., 0, \dots, 9\}$,
der alle Wörter akzeptiert,
die Gleitkommazahlen
darstellen
(z.B. $+27$, -3.14 , $.666$):



Anwendungen von Automaten

- ▶ Automaten werden zur **Spezifikation von Systemen und Protokollen** verwendet.
- ▶ Sie werden zur **Textsuche und Texterkennung** verwendet.
- ▶ Sie dienen als Implementierung für **reguläre Ausdrücke** (mehr hierzu später).

2c

Minimierung von deterministischen endlichen Automaten

Prof. Dr. Jasmin Blanchette

Lehr- und Forschungseinheit für
Theoretische Informatik und Theorembeweisen

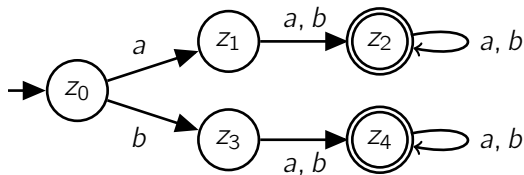
Stand: 11. Juni 2024

Basierend auf Folien von PD Dr. David Sabel



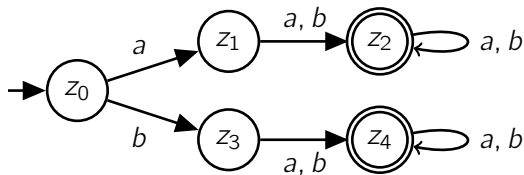
Beispiel für Minimalität

Nicht minimaler DFA:

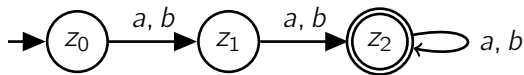


Beispiel für Minimalität

Nicht minimaler DFA:

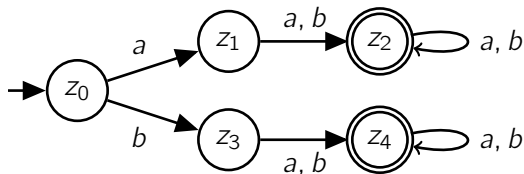


Minimaler DFA:

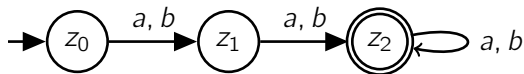


Beispiel für Minimalität

Nicht minimaler DFA:



Minimaler DFA:



Beide DFAs akzeptieren die gleiche Sprache,
aber der zweite hat weniger Zustände.

Vorteile der Minimierung von DFAs

- ▶ Minimale DFAs sind **effizienter zu implementieren**.
- ▶ Minimale DFAs sind oft **einfacher zu verstehen** oder zu analysieren.
- ▶ Da minimale DFAs (bis auf Umbenennung der Zustände) eindeutig sind, können sie **Sprachen exakt darstellen**.

Grundgedanke der Minimierung von DFAs

Schritte:

1. Entferne alle nicht erreichbaren Zustände.
2. Bilde eine Partition (d.h. disjunkte Zerlegung), indem jeweils alle Endzustände und alle Nicht-Endzustände verschmolzen werden.
3. Verfeinere diese Partition schrittweise, bis sie sich nicht mehr verändert.

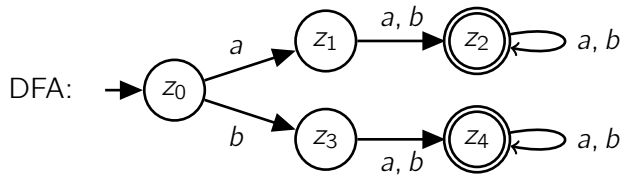
Grundgedanke der Minimierung von DFAs

Schritte:

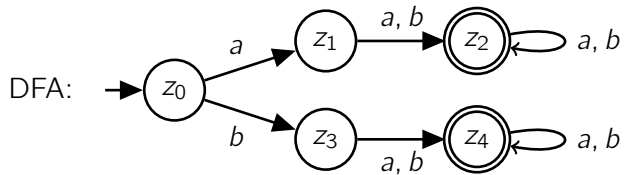
1. Entferne alle nicht erreichbaren Zustände.
2. Bilde eine Partition (d.h. disjunkte Zerlegung), indem jeweils alle Endzustände und alle Nicht-Endzustände verschmolzen werden.
3. Verfeinere diese Partition schrittweise, bis sie sich nicht mehr verändert.

Zwei Ansätze: graphisch und tabellarisch.

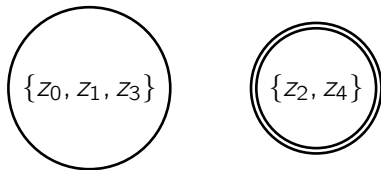
Beispiel für den graphischen Ansatz



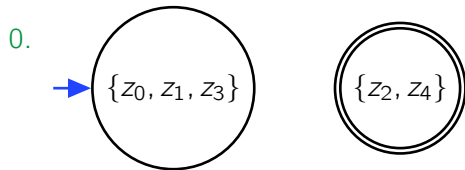
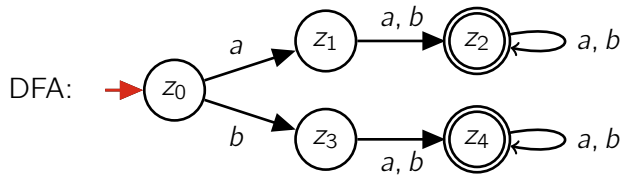
Beispiel für den graphischen Ansatz



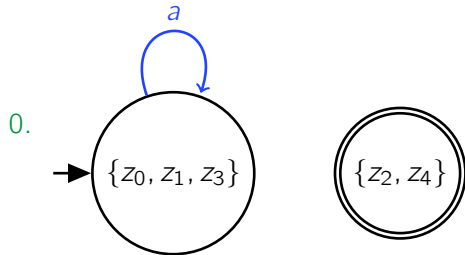
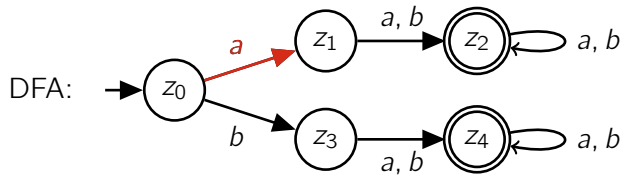
0.



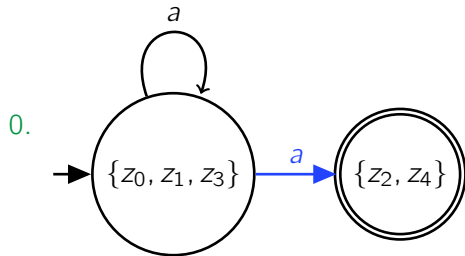
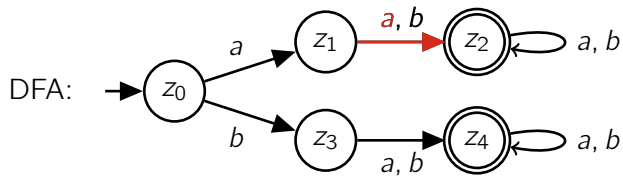
Beispiel für den graphischen Ansatz



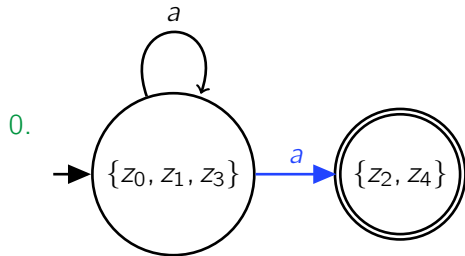
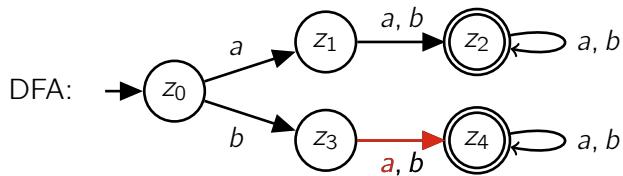
Beispiel für den graphischen Ansatz



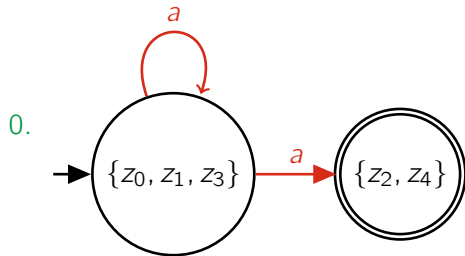
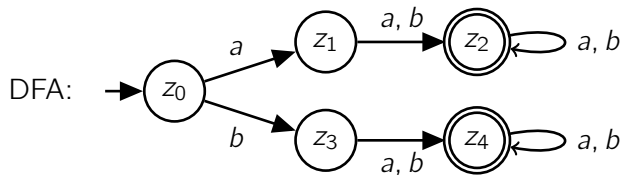
Beispiel für den graphischen Ansatz



Beispiel für den graphischen Ansatz



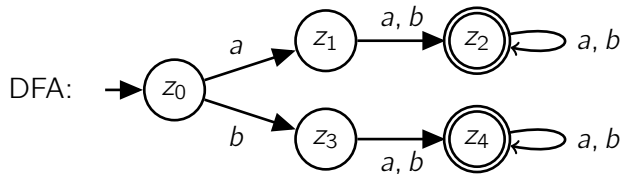
Beispiel für den graphischen Ansatz



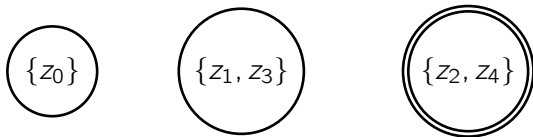
Wird kein DFA

$\{z_0, z_1, z_3\}$ muss gespalten werden,
damit ein DFA entsteht.

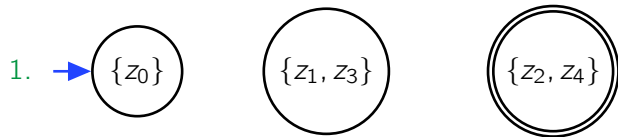
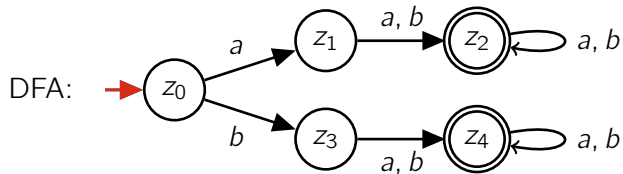
Beispiel für den graphischen Ansatz



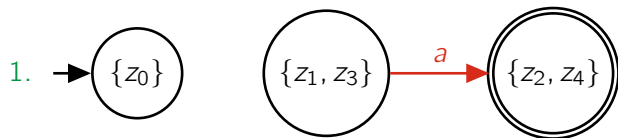
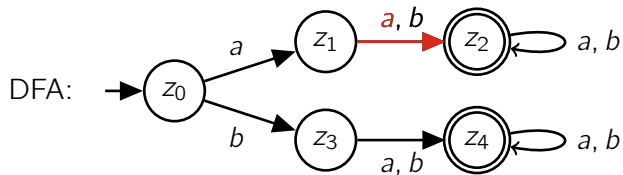
1.



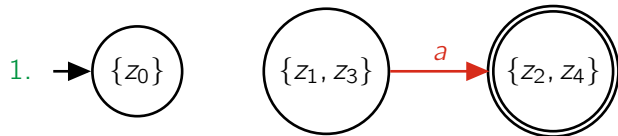
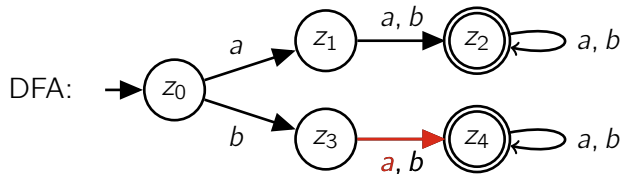
Beispiel für den graphischen Ansatz



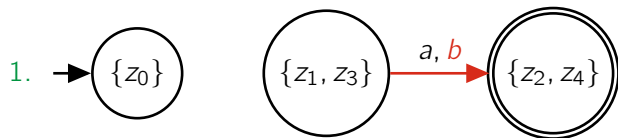
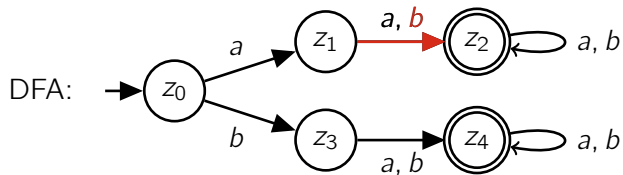
Beispiel für den graphischen Ansatz



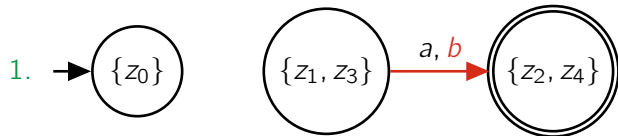
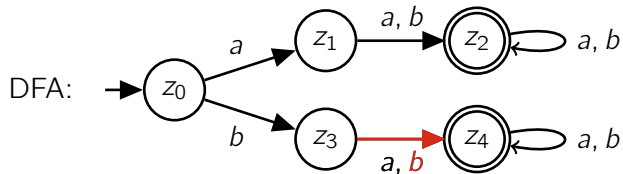
Beispiel für den graphischen Ansatz



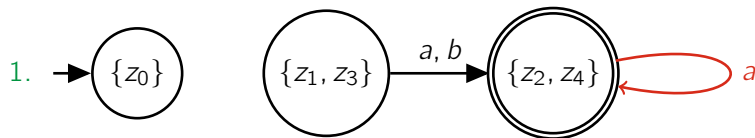
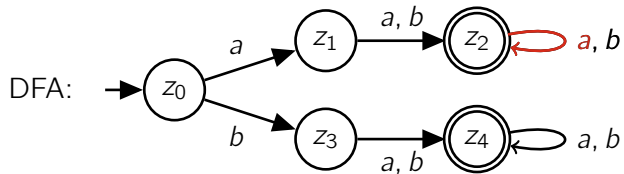
Beispiel für den graphischen Ansatz



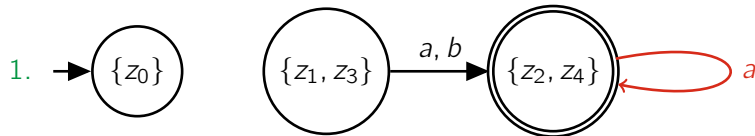
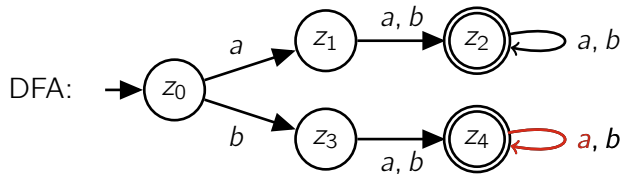
Beispiel für den graphischen Ansatz



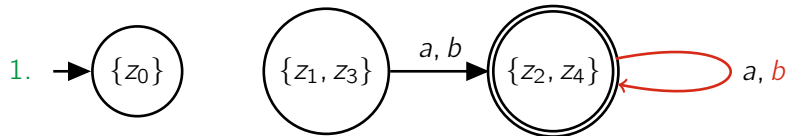
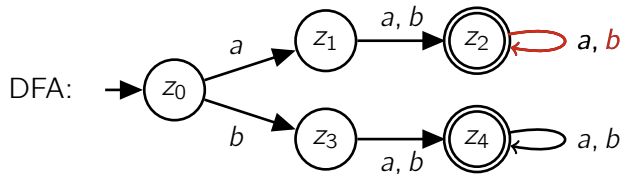
Beispiel für den graphischen Ansatz



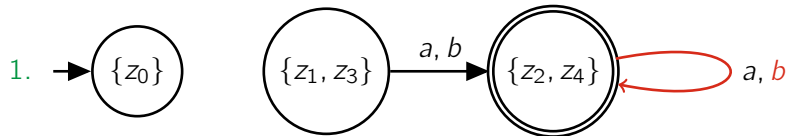
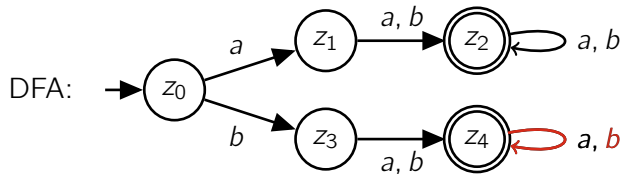
Beispiel für den graphischen Ansatz



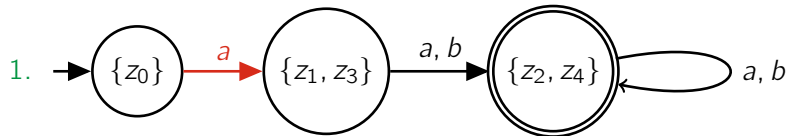
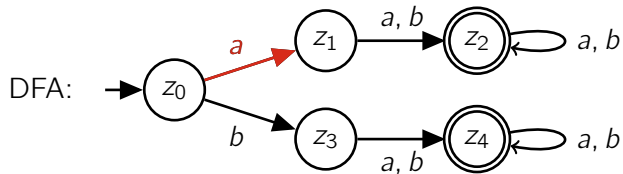
Beispiel für den graphischen Ansatz



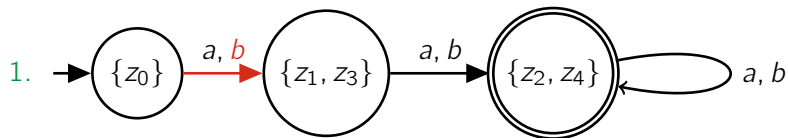
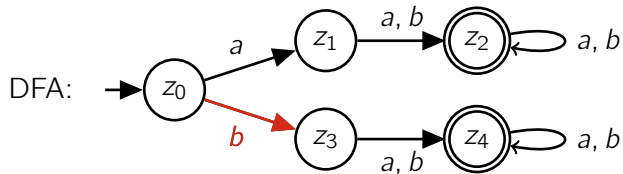
Beispiel für den graphischen Ansatz



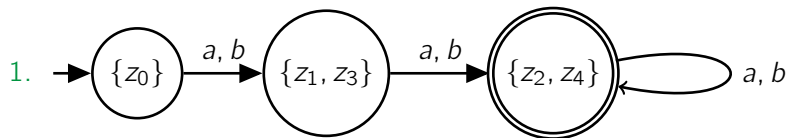
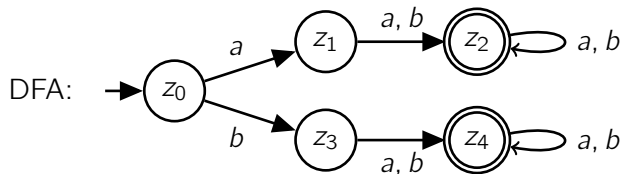
Beispiel für den graphischen Ansatz



Beispiel für den graphischen Ansatz



Beispiel für den graphischen Ansatz



Minimaler DFA

Minimierung eines DFAs mit dem tabellarischen Ansatz

Sei M ein DFA.

Intuitiver Ansatz:

1. Entferne alle nicht erreichbaren Zustände von M .
2. Konstruiere die Partitionstabelle.
3. Bilde den minimalen DFA M' , indem Zustände derselben Klasse verschmolzen werden, basiert auf der letzten Reihe der Partitionstabelle.

Der tabellarische Ansatz

$$\text{DFA } M = (Z, \Sigma, z_0, E)$$

Der tabellarische Ansatz

DFA $M = (Z, \Sigma, z_0, E)$

Schritte:

2.1 Bilde eine Partition $\mathcal{P}$ von Z mit folgenden Klassen:

Endzustände E (falls nicht leer) und Nicht-Endzustände $Z \setminus E$ (falls nicht leer).

Der tabellarische Ansatz

DFA $M = (Z, \Sigma, z_0, E)$

Schritte:

2.1 Bilde eine Partition $\mathcal{P}$ von Z mit folgenden Klassen:

Endzustände E (falls nicht leer) und Nicht-Endzustände $Z \setminus E$ (falls nicht leer).

2.2 Wiederhole bis $\mathcal{P}$ sich nicht mehr verändert:

2.2.1 Für jede Klasse $K \in \mathcal{P}$ mit $|K| \geq 2$ und für jedes $a \in \Sigma$:

2.2.1.1 Berechne für jeden Zustand $z \in K$ die Klasse $L \in \mathcal{P}$, sodass $\delta(z, a) \in L$.

2.2.1.2 Partitioniere K in Teilklassen je nach L .

2.2.1.3 Falls es mehrere Teilklassen gibt, ersetze K in $\mathcal{P}$ durch die Teilklassen.

Der tabellarische Ansatz

DFA $M = (Z, \Sigma, z_0, E)$

Schritte:

2.1 Bilde eine Partition $\mathcal{P}$ von Z mit folgenden Klassen:

Endzustände E (falls nicht leer) und Nicht-Endzustände $Z \setminus E$ (falls nicht leer).

2.2 Wiederhole bis $\mathcal{P}$ sich nicht mehr verändert:

2.2.1 Für jede Klasse $K \in \mathcal{P}$ mit $|K| \geq 2$ und für jedes $a \in \Sigma$:

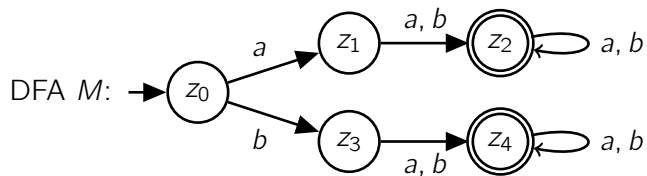
2.2.1.1 Berechne für jeden Zustand $z \in K$ die Klasse $L \in \mathcal{P}$, sodass $\delta(z, a) \in L$.

2.2.1.2 Partitioniere K in Teilklassen je nach L .

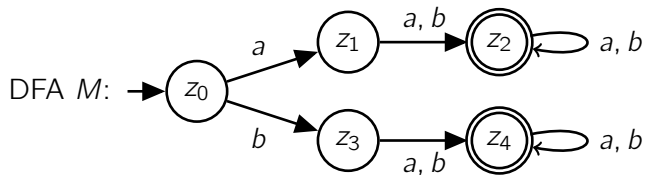
2.2.1.3 Falls es mehrere Teilklassen gibt, ersetze K in $\mathcal{P}$ durch die Teilklassen.

Am Ende besteht $\mathcal{P}$ aus den verschmolzenen Zuständen.

Erstes Beispiel für den tabellarischen Ansatz



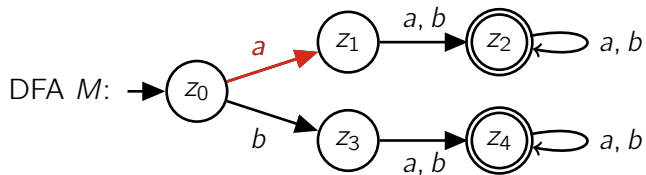
Erstes Beispiel für den tabellarischen Ansatz



0.

z_0	z_1	z_3	z_2	z_4
-------	-------	-------	-------	-------

Erstes Beispiel für den tabellarischen Ansatz

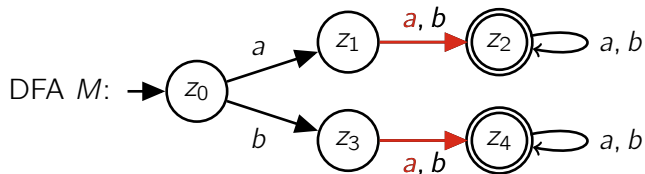


0.

z_0	z_1	z_3	z_2	z_4
-------	-------	-------	-------	-------

► z_0 mit a landet in der ersten Klasse.

Erstes Beispiel für den tabellarischen Ansatz

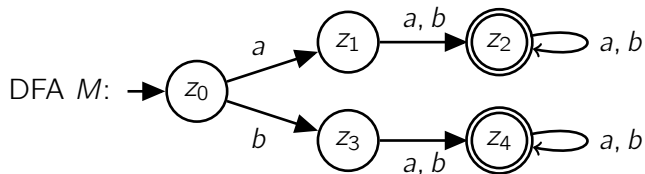


0.

z_0	z_1	z_3	z_2	z_4
-------	-------	-------	-------	-------

- ▶ z_0 mit a landet in der ersten Klasse.
- ▶ z_1 und z_3 mit a landen in der zweiten Klasse.

Erstes Beispiel für den tabellarischen Ansatz

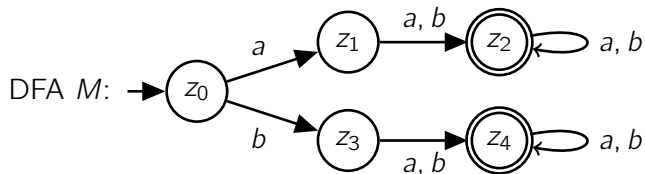


0.

z_0	z_1	z_3	z_2	z_4
-------	-------	-------	-------	-------

- ▶ z_0 mit a landet in der ersten Klasse.
- ▶ z_1 und z_3 mit a landen in der zweiten Klasse.
- ▶ z_0 muss daher von z_1 und z_3 getrennt werden.

Erstes Beispiel für den tabellarischen Ansatz



0.

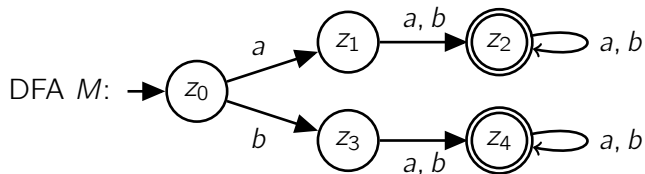
z_0	z_1	z_3	z_2	z_4
-------	-------	-------	-------	-------
- ▶ z_0 mit a landet in der ersten Klasse.
 - ▶ z_1 und z_3 mit a landen in der zweiten Klasse.
 - ▶ z_0 muss daher von z_1 und z_3 getrennt werden.

1.

z_0	z_1	z_3	z_2	z_4
-------	-------	-------	-------	-------

 mit a

Erstes Beispiel für den tabellarischen Ansatz



0.

z_0	z_1	z_3	z_2	z_4
-------	-------	-------	-------	-------

- ▶ z_0 mit a landet in der ersten Klasse.
- ▶ z_1 und z_3 mit a landen in der zweiten Klasse.
- ▶ z_0 muss daher von z_1 und z_3 getrennt werden.

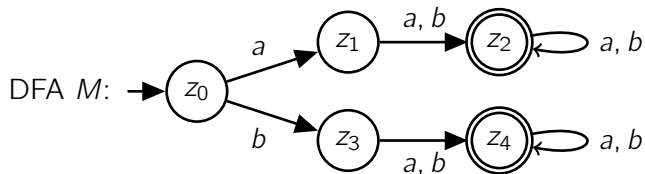
1.

z_0	z_1	z_3	z_2	z_4
-------	-------	-------	-------	-------

 mit a

- ▶ Keine weitere Partition möglich.

Erstes Beispiel für den tabellarischen Ansatz



0.

z_0	z_1	z_3	z_2	z_4
-------	-------	-------	-------	-------

- ▶ z_0 mit a landet in der ersten Klasse.
- ▶ z_1 und z_3 mit a landen in der zweiten Klasse.
- ▶ z_0 muss daher von z_1 und z_3 getrennt werden.

1.

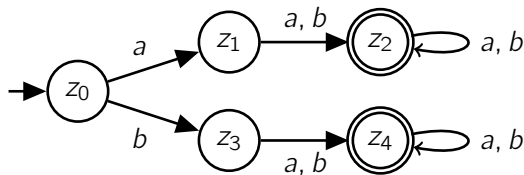
z_0	z_1	z_3	z_2	z_4
-------	-------	-------	-------	-------

 mit a

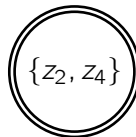
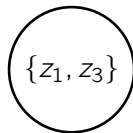
- ▶ Keine weitere Partition möglich.
- ▶ Der minimale DFA hat drei Zustände: $\{z_0\}$, $\{z_1, z_3\}$, $\{z_2, z_4\}$.

Beispiel für die Konstruktion des minimalen DFA

DFA M :

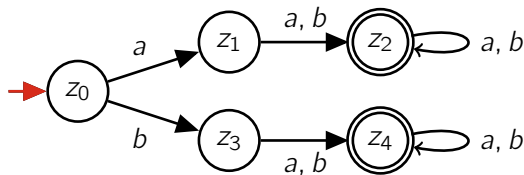


Minimaler DFA M' :

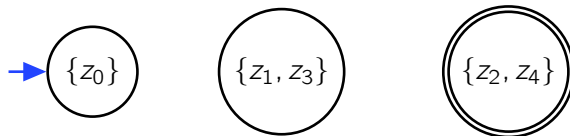


Beispiel für die Konstruktion des minimalen DFA

DFA M :

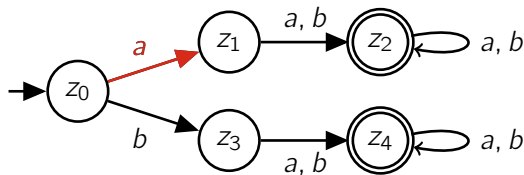


Minimaler DFA M' :

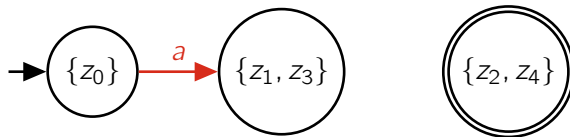


Beispiel für die Konstruktion des minimalen DFA

DFA M :

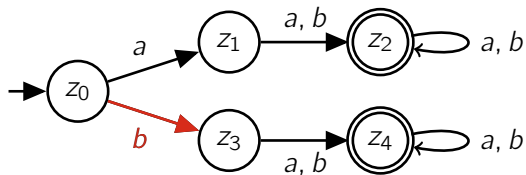


Minimaler DFA M' :

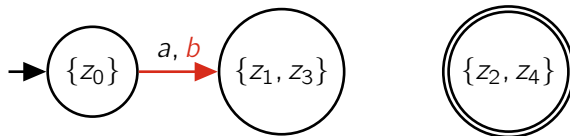


Beispiel für die Konstruktion des minimalen DFA

DFA M :

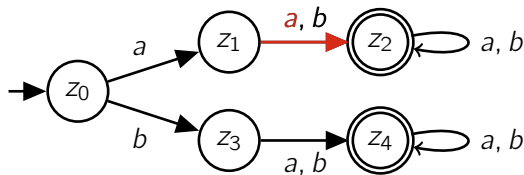


Minimaler DFA M' :

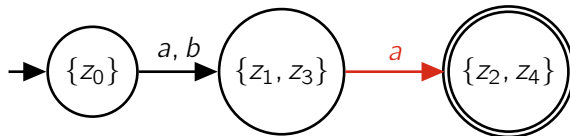


Beispiel für die Konstruktion des minimalen DFA

DFA M :

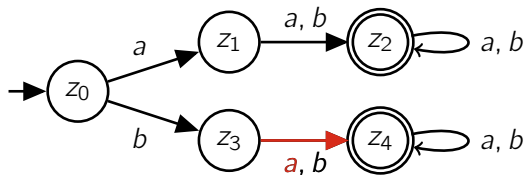


Minimaler DFA M' :

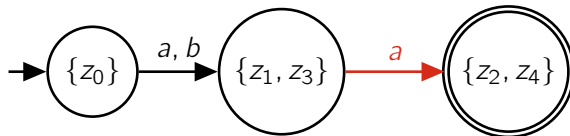


Beispiel für die Konstruktion des minimalen DFA

DFA M :

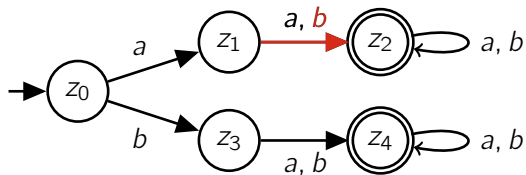


Minimaler DFA M' :

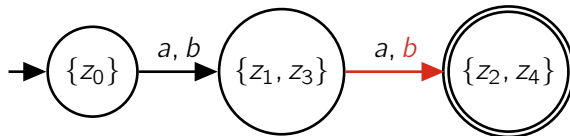


Beispiel für die Konstruktion des minimalen DFA

DFA M :

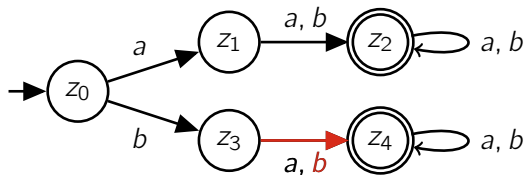


Minimaler DFA M' :

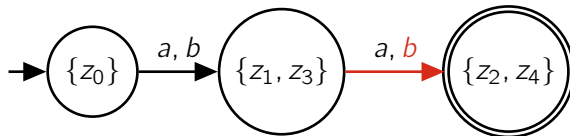


Beispiel für die Konstruktion des minimalen DFA

DFA M :

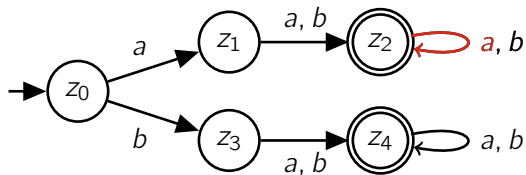


Minimaler DFA M' :

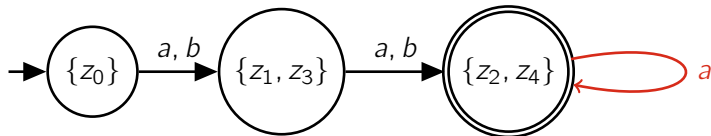


Beispiel für die Konstruktion des minimalen DFA

DFA M :

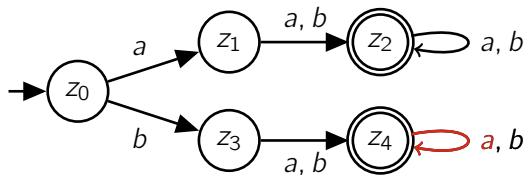


Minimaler DFA M' :

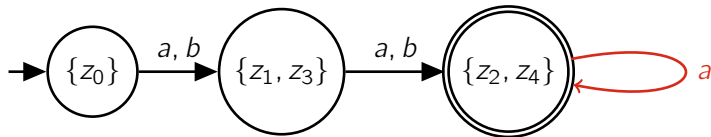


Beispiel für die Konstruktion des minimalen DFA

DFA M :

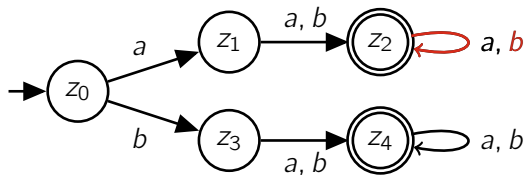


Minimaler DFA M' :

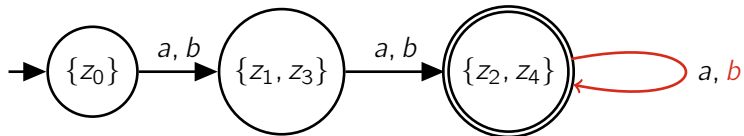


Beispiel für die Konstruktion des minimalen DFA

DFA M :

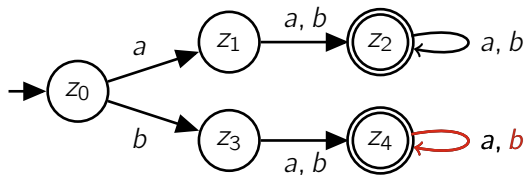


Minimaler DFA M' :

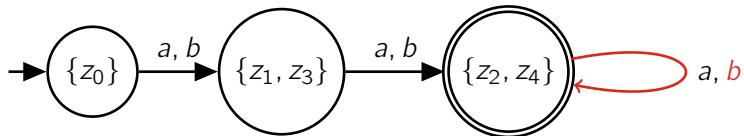


Beispiel für die Konstruktion des minimalen DFA

DFA M :

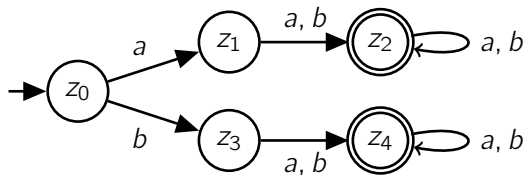


Minimaler DFA M' :

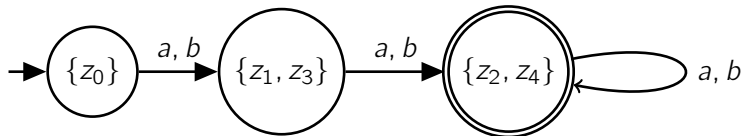


Beispiel für die Konstruktion des minimalen DFA

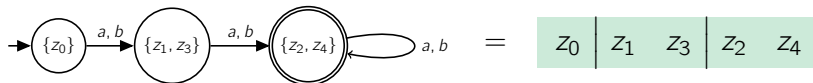
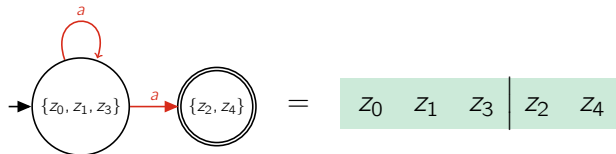
DFA M :



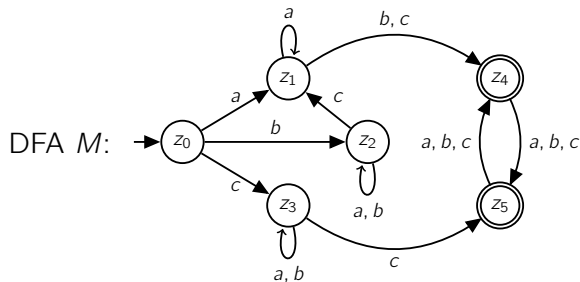
Minimaler DFA M' :



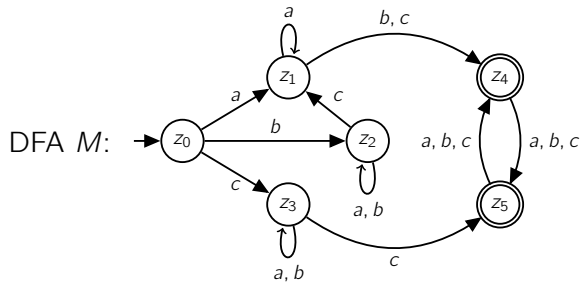
Äquivalenz des graphischen und des tabellarischen Ansatzes



Zweites Beispiel für den tabellarischen Ansatz



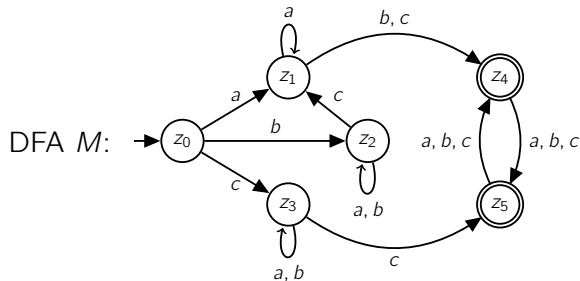
Zweites Beispiel für den tabellarischen Ansatz



Partitionstabelle:

0.	z ₀	z ₁	z ₂	z ₃	z ₄	z ₅
----	----------------	----------------	----------------	----------------	----------------	----------------

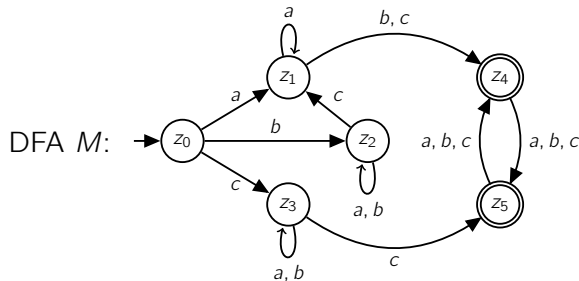
Zweites Beispiel für den tabellarischen Ansatz



Partitionstabelle:

0.	<table><tr><td>z_0</td><td>z_1</td><td>z_2</td><td>z_3</td></tr></table>	z_0	z_1	z_2	z_3	<table><tr><td>z_4</td><td>z_5</td></tr></table>	z_4	z_5	
z_0	z_1	z_2	z_3						
z_4	z_5								
1.	<table><tr><td>z_1</td><td>z_0</td><td>z_2</td><td>z_3</td></tr></table>	z_1	z_0	z_2	z_3	<table><tr><td>z_4</td><td>z_5</td></tr></table>	z_4	z_5	mit b
z_1	z_0	z_2	z_3						
z_4	z_5								

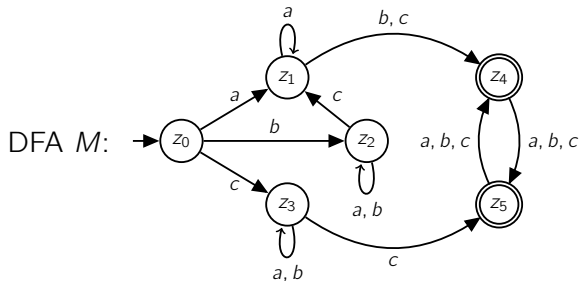
Zweites Beispiel für den tabellarischen Ansatz



Partitionstabelle:

0.	<table><tr><td>z_0</td><td>z_1</td><td>z_2</td><td>z_3</td></tr></table>	z_0	z_1	z_2	z_3	<table><tr><td>z_4</td><td>z_5</td></tr></table>	z_4	z_5			
z_0	z_1	z_2	z_3								
z_4	z_5										
1.	<table><tr><td>z_1</td><td><table><tr><td>z_0</td><td>z_2</td><td>z_3</td></tr></table></td></tr></table>	z_1	<table><tr><td>z_0</td><td>z_2</td><td>z_3</td></tr></table>	z_0	z_2	z_3	<table><tr><td>z_4</td><td>z_5</td></tr></table>	z_4	z_5	mit b	
z_1	<table><tr><td>z_0</td><td>z_2</td><td>z_3</td></tr></table>	z_0	z_2	z_3							
z_0	z_2	z_3									
z_4	z_5										
2.	<table><tr><td>z_1</td><td><table><tr><td>z_0</td><td><table><tr><td>z_2</td><td>z_3</td></tr></table></td></tr></table></td></tr></table>	z_1	<table><tr><td>z_0</td><td><table><tr><td>z_2</td><td>z_3</td></tr></table></td></tr></table>	z_0	<table><tr><td>z_2</td><td>z_3</td></tr></table>	z_2	z_3	<table><tr><td>z_4</td><td>z_5</td></tr></table>	z_4	z_5	mit c
z_1	<table><tr><td>z_0</td><td><table><tr><td>z_2</td><td>z_3</td></tr></table></td></tr></table>	z_0	<table><tr><td>z_2</td><td>z_3</td></tr></table>	z_2	z_3						
z_0	<table><tr><td>z_2</td><td>z_3</td></tr></table>	z_2	z_3								
z_2	z_3										
z_4	z_5										

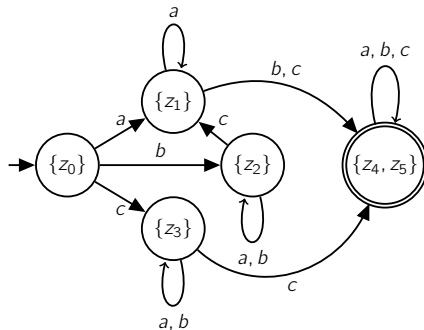
Zweites Beispiel für den tabellarischen Ansatz



Partitionstabelle:

0.	z_0	z_1	z_2	z_3	z_4	z_5	
1.	z_1	z_0	z_2	z_3	z_4	z_5	mit b
2.	z_1	z_0	z_2	z_3	z_4	z_5	mit c

Minimaler DFA M' :



Äquivalenzklassenautomat

Beide Ansätze basieren auf dem Äquivalenzklassenautomaten.

Definition

Sei $M = (Z, \Sigma, \delta, z_0, E)$ ein DFA.

Wir nennen zwei Zustände $z, z' \in Z$ äquivalent und schreiben $z \equiv_M z'$ (alternativ $z \equiv z'$), falls gilt: $\tilde{\delta}(z, w) \in E$ g.d.w. $\tilde{\delta}(z', w) \in E$ für alle $w \in \Sigma^*$.

Äquivalenzklassenautomat

Beide Ansätze basieren auf dem Äquivalenzklassenautomaten.

Definition

Sei $M = (Z, \Sigma, \delta, z_0, E)$ ein DFA.

Wir nennen zwei Zustände $z, z' \in Z$ äquivalent und schreiben $z \equiv_M z'$ (alternativ $z \equiv z'$), falls gilt: $\tilde{\delta}(z, w) \in E$ g.d.w. $\tilde{\delta}(z', w) \in E$ für alle $w \in \Sigma^*$.

Der Äquivalenzklassenautomat zu M ist der DFA $M' = (Z', \Sigma, \delta', z'_0, E')$ mit

$$Z' := \{[z]_{\equiv} \mid z \in Z\}$$

$$z'_0 := [z_0]_{\equiv}$$

$$E' := \{[z]_{\equiv} \mid z \in E\}$$

$$\delta'([z]_{\equiv}, a) := [\delta(z, a)]_{\equiv}$$

Äquivalenzklassenautomat

Beide Ansätze basieren auf dem Äquivalenzklassenautomaten.

Definition

Sei $M = (Z, \Sigma, \delta, z_0, E)$ ein DFA.

Wir nennen zwei Zustände $z, z' \in Z$ äquivalent und schreiben $z \equiv_M z'$ (alternativ $z \equiv z'$), falls gilt: $\tilde{\delta}(z, w) \in E$ g.d.w. $\tilde{\delta}(z', w) \in E$ für alle $w \in \Sigma^*$.

Der Äquivalenzklassenautomat zu M ist der DFA $M' = (Z', \Sigma, \delta', z'_0, E')$ mit

$$\begin{aligned} Z' &:= \{[z]_{\equiv} \mid z \in Z\} \\ z'_0 &:= [z_0]_{\equiv} \\ E' &:= \{[z]_{\equiv} \mid z \in E\} \\ \delta'([z]_{\equiv}, a) &:= [\delta(z, a)]_{\equiv} \end{aligned}$$

Informell: Zwei Zustände sind äquivalent, wenn sie die gleiche „Sprache“ darstellen.

Korrektheit und Minimalität des Äquivalenzklassenautomaten

Satz

Seien $M = (Z, \Sigma, \delta, z_0, E)$ ein DFA und $M' = (Z', \Sigma, \delta', z'_0, E')$ der Äquivalenzklassenautomat zu M . Dann gilt

1. $L(M') = L(M)$.
2. Falls alle Zustände in Z von z_0 erreichbar sind, dann ist M' minimal, d.h. jeder DFA M'' mit $L(M'') = L(M')$ hat mindestens so viele Zustände wie M' .

Korrektheit und Minimalität des Äquivalenzklassenautomaten

Satz

Seien $M = (Z, \Sigma, \delta, z_0, E)$ ein DFA und $M' = (Z', \Sigma, \delta', z'_0, E')$ der Äquivalenzklassenautomat zu M . Dann gilt

1. $L(M') = L(M)$.
2. Falls alle Zustände in Z von z_0 erreichbar sind, dann ist M' minimal, d.h. jeder DFA M'' mit $L(M'') = L(M')$ hat mindestens so viele Zustände wie M' .

Beweis

1. Wir zeigen: $w \in L(M')$ g.d.w. $w \in L(M)$. Sei $w \in \Sigma^*$.

Korrektheit und Minimalität des Äquivalenzklassenautomaten

Satz

Seien $M = (Z, \Sigma, \delta, z_0, E)$ ein DFA und $M' = (Z', \Sigma, \delta', z'_0, E')$ der Äquivalenzklassenautomat zu M . Dann gilt

1. $L(M') = L(M)$.
2. Falls alle Zustände in Z von z_0 erreichbar sind, dann ist M' minimal, d.h. jeder DFA M'' mit $L(M'') = L(M')$ hat mindestens so viele Zustände wie M' .

Beweis

1. Wir zeigen: $w \in L(M')$ g.d.w. $w \in L(M)$. Sei $w \in \Sigma^*$.
 M durchläuft $z_0, \dots, z_{|w|}$ entlang w und akzeptiert w g.d.w. $z_{|w|} \in E$.

Korrektheit und Minimalität des Äquivalenzklassenautomaten

Satz

Seien $M = (Z, \Sigma, \delta, z_0, E)$ ein DFA und $M' = (Z', \Sigma, \delta', z'_0, E')$ der Äquivalenzklassenautomat zu M . Dann gilt

1. $L(M') = L(M)$.
2. Falls alle Zustände in Z von z_0 erreichbar sind, dann ist M' minimal, d.h. jeder DFA M'' mit $L(M'') = L(M')$ hat mindestens so viele Zustände wie M' .

Beweis

1. Wir zeigen: $w \in L(M')$ g.d.w. $w \in L(M)$. Sei $w \in \Sigma^*$.
 M durchläuft $z_0, \dots, z_{|w|}$ entlang w und akzeptiert w g.d.w. $z_{|w|} \in E$.
 M' durchläuft $[z_0]_{\equiv}, \dots, [z_{|w|}]_{\equiv}$ und akzeptiert w g.d.w. $[z_{|w|}]_{\equiv} \in E'$.

Korrektheit und Minimalität des Äquivalenzklassenautomaten

Satz

Seien $M = (Z, \Sigma, \delta, z_0, E)$ ein DFA und $M' = (Z', \Sigma, \delta', z'_0, E')$ der Äquivalenzklassenautomat zu M . Dann gilt

1. $L(M') = L(M)$.
2. Falls alle Zustände in Z von z_0 erreichbar sind, dann ist M' minimal, d.h. jeder DFA M'' mit $L(M'') = L(M')$ hat mindestens so viele Zustände wie M' .

Beweis

1. Wir zeigen: $w \in L(M')$ g.d.w. $w \in L(M)$. Sei $w \in \Sigma^*$.
 M durchläuft $z_0, \dots, z_{|w|}$ entlang w und akzeptiert w g.d.w. $z_{|w|} \in E$.
 M' durchläuft $[z_0]_{\equiv}, \dots, [z_{|w|}]_{\equiv}$ und akzeptiert w g.d.w. $[z_{|w|}]_{\equiv} \in E'$.
Da per Definition $[z_{|w|}]_{\equiv} \in E'$ äquivalent zu $z_{|w|} \in E$ ist, folgt, dass
 $L(M') = L(M)$.

Korrektheit und Minimalität des Äquivalenzklassenautomaten

Satz

Seien $M = (Z, \Sigma, \delta, z_0, E)$ ein DFA und $M' = (Z', \Sigma, \delta', z'_0, E')$ der Äquivalenzklassenautomat zu M . Dann gilt

1. $L(M') = L(M)$.
2. Falls alle Zustände in Z von z_0 erreichbar sind, dann ist M' minimal, d.h. jeder DFA M'' mit $L(M'') = L(M')$ hat mindestens so viele Zustände wie M' .

Beweis (Fortsetzung)

2. Wird in späterer Vorlesung gezeigt (nur FSK).



Algorithmus 3: Berechnung aller äquivalenten Zustände

Eingabe: DFA $M = (Z, \Sigma, \delta, z_0, E)$, der keine unerreichbaren Zustände hat

Ausgabe: Partition $\mathcal{P} = \{[z_1]_{\equiv}, \dots, [z_m]_{\equiv}\}$ von Z

Beginn

initialisiere Partition $\mathcal{P}$ mit E (falls nicht leer) und $Z \setminus E$ (falls nicht leer);

wiederhole

für jedes $K = \{z_1, \dots, z_m\} \in \mathcal{P}$ mit $|m| \geq 2$ **tue**

für jedes $a \in \Sigma$ **tue**

 berechne die Partition $\mathcal{Q}$ von $\{z_1, \dots, z_m\}$ über
 dem Wert von $[\delta(z_i, a)]$ für jedes i ;

$\mathcal{P} := (\mathcal{P} \setminus \{K\}) \cup \mathcal{Q}$;

Ende

Ende

bis sich $\mathcal{P}$ nicht mehr verändert;

return $\mathcal{P}$

Ende

Korrektheit von Algorithmus 3

Satz

Sei $M = (Z, \Sigma, \delta, z_0, E)$ ein DFA, der keine unerreichbaren Zustände hat. Algorithmus 3 berechnet die Äquivalenzklassen bezüglich $\equiv$.

Korrektheit von Algorithmus 3

Satz

Sei $M = (Z, \Sigma, \delta, z_0, E)$ ein DFA, der keine unerreichbaren Zustände hat. Algorithmus 3 berechnet die Äquivalenzklassen bezüglich $\equiv$.

Beweis Wir führen den Beweis in zwei Schritten:

1. Wenn $z \equiv z'$, dann sind z und z' in derselben Klasse von $\mathcal{P}$.
2. Wenn $z \not\equiv z'$, dann sind z und z' in verschiedenen Klassen von $\mathcal{P}$.

Korrektheit des tabellarischen Ansatzes

Beweis (Fortsetzung)

1. Wir zeigen die Kontraposition: Wenn z und z' in verschiedenen Klassen von $\mathcal{P}$ sind, dann $z \not\equiv z'$.

Korrektheit des tabellarischen Ansatzes

Beweis (Fortsetzung)

1. Wir zeigen die Kontraposition: Wenn z und z' in verschiedenen Klassen von $\mathcal{P}$ sind, dann $z \not\equiv z'$. Und um $z \not\equiv z'$ zu zeigen, reicht es, ein Wort w zu finden, sodass $\tilde{\delta}(z, w) \in E$ und $\tilde{\delta}(z', w) \notin E$ oder umgekehrt.

Beweis (Fortsetzung)

1. Wir zeigen die Kontraposition: Wenn z und z' in verschiedenen Klassen von $\mathcal{P}$ sind, dann $z \not\equiv z'$. Und um $z \not\equiv z'$ zu zeigen, reicht es, ein Wort w zu finden, sodass $\tilde{\delta}(z, w) \in E$ und $\tilde{\delta}(z', w) \notin E$ oder umgekehrt.

Durch Induktion über die Anzahl n der Schleifeniterationen bis z und z' getrennt wurden.

Beweis (Fortsetzung)

1. Wir zeigen die Kontraposition: Wenn z und z' in verschiedenen Klassen von $\mathcal{P}$ sind, dann $z \not\equiv z'$. Und um $z \not\equiv z'$ zu zeigen, reicht es, ein Wort w zu finden, sodass $\tilde{\delta}(z, w) \in E$ und $\tilde{\delta}(z', w) \notin E$ oder umgekehrt.

Durch Induktion über die Anzahl n der Schleifeniterationen bis z und z' getrennt wurden.

- Fall $n = 0$: Da z und z' schon vor der ersten Iteration getrennt wurden, müssen $z \in E$ und $z' \notin E$ oder umgekehrt gelten. Wir nehmen $w = \varepsilon$: $\tilde{\delta}(z, \varepsilon) \in E$ und $\tilde{\delta}(z', \varepsilon) \notin E$ oder umgekehrt.

Korrektheit des tabellarischen Ansatzes

Beweis (Fortsetzung)

1. Wir zeigen die Kontraposition: Wenn z und z' in verschiedenen Klassen von $\mathcal{P}$ sind, dann $z \not\equiv z'$. Und um $z \not\equiv z'$ zu zeigen, reicht es, ein Wort w zu finden, sodass $\tilde{\delta}(z, w) \in E$ und $\tilde{\delta}(z', w) \notin E$ oder umgekehrt.

Durch Induktion über die Anzahl n der Schleifeniterationen bis z und z' getrennt wurden.

- Fall $n = 0$: Da z und z' schon vor der ersten Iteration getrennt wurden, müssen $z \in E$ und $z' \notin E$ oder umgekehrt gelten. Wir nehmen $w = \varepsilon$: $\tilde{\delta}(z, \varepsilon) \in E$ und $\tilde{\delta}(z', \varepsilon) \notin E$ oder umgekehrt.
- Fall $n > 0$: Da z und z' in Iteration n getrennt wurden, muss es $a \in \Sigma$ geben, sodass $\delta(z, a)$ und $\delta(z', a)$ schon in Iteration $n - 1$ getrennt waren. Die Induktionshypothese liefert ein Wort w' mit $\tilde{\delta}(\delta(z, a), w') \in E$ und $\tilde{\delta}(\delta(z', a), w') \notin E$ oder umgekehrt. Wir nehmen $w = aw'$: $\tilde{\delta}(z, aw') \in E$ und $\tilde{\delta}(z', aw') \notin E$ oder umgekehrt.

Beweis (Fortsetzung)

2. Wir müssen zeigen, dass wenn $z \neq z'$, dann sind z und z' in zwei verschiedenen Klassen von $\mathcal{P}$. Sei w ein Wort sodass $\tilde{\delta}(z, w) \in E$ und $\tilde{\delta}(z', w) \notin E$ oder umgekehrt.

Durch Induktion über $|w|$.

- Fall $w = \varepsilon$: Dann haben wir $\tilde{\delta}(z, \varepsilon) \in E$ und $\tilde{\delta}(z', \varepsilon) \notin E$ oder umgekehrt. Die Initialisierungsphase sorgt dafür, dass z und z' getrennt werden.
- Fall w ist von der Form aw' : Dann haben wir $\tilde{\delta}(z, aw') = \tilde{\delta}(\delta(z, a), w') \in E$ und $\tilde{\delta}(z', aw') = \tilde{\delta}(\delta(z', a), w') \notin E$ oder umgekehrt. Per Induktionshypothese müssen $\delta(z, a)$ und $\delta(z', a)$ getrennt sein. Der Algorithmus muss z und z' trennen, wenn a betrachtet wird. $\square$

Algorithmus 4: Minimierung von DFAs

Eingabe: DFA $M = (Z, \Sigma, \delta, z_0, E)$

Ausgabe: Minimaler DFA M' mit $L(M') = L(M)$

Beginn

entferne Zustände aus M , die nicht vom Startzustand aus erreichbar sind;
berechne äquivalente Zustände mit Algorithmus 3;
erzeuge den Äquivalenzklassenautomat, indem die berechneten äquivalenten Zustände verschmolzen werden;

Ende

3a

Regularität von deterministischen endlichen Automaten und nichtdeterministische endliche Automaten

Prof. Dr. Jasmin Blanchette

Lehr- und Forschungseinheit für
Theoretische Informatik und Theorembeweisen

Stand: 10. Mai 2024

Basiert auf Folien von PD Dr. David Sabel



Definition

Ein **deterministischer endlicher Automat** (*deterministic finite automaton*, **DFA**) ist ein 5-Tupel $M = (Z, \Sigma, \delta, z_0, E)$, wobei

- ▶ Z ist eine endliche Menge von **Zuständen**
- ▶ Σ ist das (endliche) **Eingabealphabet** mit $Z \cap \Sigma = \emptyset$
- ▶ $\delta : Z \times \Sigma \rightarrow Z$ ist die (totale) **Überföhrungsfunktion**
- ▶ $z_0 \in Z$ ist der **Startzustand**
- ▶ $E \subseteq Z$ ist die Menge der **Endzustände**.

Die akzeptierte Sprache von DFAs ist regulär

Theorem

Sei $M = (Z, \Sigma, \delta, z_0, E)$ ein DFA. Dann ist $L(M)$ regulär.

Die akzeptierte Sprache von DFAs ist regulär

Theorem

Sei $M = (Z, \Sigma, \delta, z_0, E)$ ein DFA. Dann ist $L(M)$ regulär.

Beweis Wir konstruieren für M eine reguläre Grammatik G (mit beiden Sonderregeln), sodass $L(G) = L(M)$.

Die akzeptierte Sprache von DFAs ist regulär

Theorem

Sei $M = (Z, \Sigma, \delta, z_0, E)$ ein DFA. Dann ist $L(M)$ regulär.

Beweis Wir konstruieren für M eine reguläre Grammatik G (mit beiden Sonderregeln), sodass $L(G) = L(M)$.

Sei $G = (V, \Sigma, P, S)$ die Grammatik mit $V := Z$, $S := z_0$ und

$$P := \{z_i \rightarrow az_j \mid \delta(z_i, a) = z_j\}$$

Die akzeptierte Sprache von DFAs ist regulär

Theorem

Sei $M = (Z, \Sigma, \delta, z_0, E)$ ein DFA. Dann ist $L(M)$ regulär.

Beweis Wir konstruieren für M eine reguläre Grammatik G (mit beiden Sonderregeln), sodass $L(G) = L(M)$.

Sei $G = (V, \Sigma, P, S)$ die Grammatik mit $V := Z$, $S := z_0$ und

$$P := \{z_i \rightarrow az_j \mid \delta(z_i, a) = z_j\} \\ \cup \{z_i \rightarrow \varepsilon \mid z_i \in E\}$$

Beispiel für die Konstruktion einer regulären Grammatik

DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, \{z_2\})$ mit

$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(z_1, a) = z_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(z_2, b) = z_2 \end{array}$$

Beispiel für die Konstruktion einer regulären Grammatik

DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, \{z_2\})$ mit

$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(z_1, a) = z_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(z_2, b) = z_2 \end{array}$$

Grammatik dazu: $G = (\{z_0, z_1, z_2\}, \{a, b\}, P, z_0)$ mit

$$P = \{$$

Beispiel für die Konstruktion einer regulären Grammatik

DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, \{z_2\})$ mit

$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(z_1, a) = z_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(z_2, b) = z_2 \end{array}$$

Grammatik dazu: $G = (\{z_0, z_1, z_2\}, \{a, b\}, P, z_0)$ mit

$$P = \{z_0 \rightarrow az_1,$$

Beispiel für die Konstruktion einer regulären Grammatik

DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, \{z_2\})$ mit

$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(z_1, a) = z_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(z_2, b) = z_2 \end{array}$$

Grammatik dazu: $G = (\{z_0, z_1, z_2\}, \{a, b\}, P, z_0)$ mit

$$P = \{z_0 \rightarrow az_1, z_0 \rightarrow bz_0,$$

Beispiel für die Konstruktion einer regulären Grammatik

DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, \{z_2\})$ mit

$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(\textcolor{red}{z}_1, \textcolor{red}{a}) = \textcolor{blue}{z}_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(z_2, b) = z_2 \end{array}$$

Grammatik dazu: $G = (\{z_0, z_1, z_2\}, \{a, b\}, P, z_0)$ mit

$$\begin{aligned} P = \{ & z_0 \rightarrow az_1, z_0 \rightarrow bz_0, \\ & \textcolor{red}{z}_1 \rightarrow \textcolor{red}{a}\textcolor{blue}{z}_2, \end{aligned}$$

Beispiel für die Konstruktion einer regulären Grammatik

DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, \{z_2\})$ mit

$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(z_1, a) = z_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(z_2, b) = z_2 \end{array}$$

Grammatik dazu: $G = (\{z_0, z_1, z_2\}, \{a, b\}, P, z_0)$ mit

$$\begin{array}{l} P = \{z_0 \rightarrow az_1, z_0 \rightarrow bz_0, \\ \quad z_1 \rightarrow az_2, z_1 \rightarrow bz_0, \end{array}$$

Beispiel für die Konstruktion einer regulären Grammatik

DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, \{z_2\})$ mit

$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(z_1, a) = z_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(z_2, b) = z_2 \end{array}$$

Grammatik dazu: $G = (\{z_0, z_1, z_2\}, \{a, b\}, P, z_0)$ mit

$$\begin{aligned} P = \{ & z_0 \rightarrow az_1, z_0 \rightarrow bz_0, \\ & z_1 \rightarrow az_2, z_1 \rightarrow bz_0, \\ & z_2 \rightarrow az_2, \end{aligned}$$

Beispiel für die Konstruktion einer regulären Grammatik

DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, \{z_2\})$ mit

$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(z_1, a) = z_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(\textcolor{red}{z}_2, \textcolor{red}{b}) = \textcolor{blue}{z}_2 \end{array}$$

Grammatik dazu: $G = (\{z_0, z_1, z_2\}, \{a, b\}, P, z_0)$ mit

$$\begin{aligned} P = \{ & z_0 \rightarrow az_1, z_0 \rightarrow bz_0, \\ & z_1 \rightarrow az_2, z_1 \rightarrow bz_0, \\ & z_2 \rightarrow az_2, \textcolor{red}{z}_2 \rightarrow \textcolor{blue}{b}\textcolor{blue}{z}_2, \end{aligned}$$

Beispiel für die Konstruktion einer regulären Grammatik

DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, \{z_2\})$ mit

$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(z_1, a) = z_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(z_2, b) = z_2 \end{array}$$

Grammatik dazu: $G = (\{z_0, z_1, z_2\}, \{a, b\}, P, z_0)$ mit

$$\begin{aligned} P = \{ & z_0 \rightarrow az_1, z_0 \rightarrow bz_0, \\ & z_1 \rightarrow az_2, z_1 \rightarrow bz_0, \\ & z_2 \rightarrow az_2, z_2 \rightarrow bz_2, \\ & \textcolor{red}{z_2} \rightarrow \varepsilon \} \end{aligned}$$

Beispiel für die Konstruktion einer regulären Grammatik

DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, \{z_2\})$ mit

$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(z_1, a) = z_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(z_2, b) = z_2 \end{array}$$

Grammatik dazu: $G = (\{z_0, z_1, z_2\}, \{a, b\}, P, z_0)$ mit

$$\begin{aligned} P = \{ & z_0 \rightarrow az_1, z_0 \rightarrow bz_0, \\ & z_1 \rightarrow az_2, z_1 \rightarrow bz_0, \\ & z_2 \rightarrow az_2, z_2 \rightarrow bz_2, \\ & z_2 \rightarrow \varepsilon \} \end{aligned}$$

Ableitung der Eingabe $abbaaa \in L(M)$:

Beispiel für die Konstruktion einer regulären Grammatik

DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, \{z_2\})$ mit

$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(z_1, a) = z_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(z_2, b) = z_2 \end{array}$$

Grammatik dazu: $G = (\{z_0, z_1, z_2\}, \{a, b\}, P, z_0)$ mit

$$\begin{aligned} P = \{ & z_0 \rightarrow az_1, z_0 \rightarrow bz_0, \\ & z_1 \rightarrow az_2, z_1 \rightarrow bz_0, \\ & z_2 \rightarrow az_2, z_2 \rightarrow bz_2, \\ & z_2 \rightarrow \varepsilon \} \end{aligned}$$

Ableitung der Eingabe $abbaaa \in L(M)$:

z_0

Beispiel für die Konstruktion einer regulären Grammatik

DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, \{z_2\})$ mit

$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(z_1, a) = z_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(z_2, b) = z_2 \end{array}$$

Grammatik dazu: $G = (\{z_0, z_1, z_2\}, \{a, b\}, P, z_0)$ mit

$$\begin{aligned} P = \{ & z_0 \rightarrow az_1, z_0 \rightarrow bz_0, \\ & z_1 \rightarrow az_2, z_1 \rightarrow bz_0, \\ & z_2 \rightarrow az_2, z_2 \rightarrow bz_2, \\ & z_2 \rightarrow \varepsilon \} \end{aligned}$$

Ableitung der Eingabe $abbaaa \in L(M)$:

$$z_0 \Rightarrow az_1$$

Beispiel für die Konstruktion einer regulären Grammatik

DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, \{z_2\})$ mit

$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(z_1, a) = z_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(z_2, b) = z_2 \end{array}$$

Grammatik dazu: $G = (\{z_0, z_1, z_2\}, \{a, b\}, P, z_0)$ mit

$$\begin{aligned} P = \{ & z_0 \rightarrow az_1, z_0 \rightarrow bz_0, \\ & z_1 \rightarrow az_2, \textcolor{red}{z}_1 \rightarrow \textcolor{blue}{b}z_0, \\ & z_2 \rightarrow az_2, z_2 \rightarrow bz_2, \\ & z_2 \rightarrow \varepsilon \} \end{aligned}$$

Ableitung der Eingabe $a\textcolor{blue}{b}baaa \in L(M)$:

$$z_0 \Rightarrow a\textcolor{red}{z}_1 \Rightarrow a\textcolor{blue}{b}z_0$$

Beispiel für die Konstruktion einer regulären Grammatik

DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, \{z_2\})$ mit

$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(z_1, a) = z_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(z_2, b) = z_2 \end{array}$$

Grammatik dazu: $G = (\{z_0, z_1, z_2\}, \{a, b\}, P, z_0)$ mit

$$\begin{aligned} P = \{ & z_0 \rightarrow az_1, \textcolor{red}{z}_0 \rightarrow \textcolor{blue}{b}z_0, \\ & z_1 \rightarrow az_2, z_1 \rightarrow bz_0, \\ & z_2 \rightarrow az_2, z_2 \rightarrow bz_2, \\ & z_2 \rightarrow \varepsilon \} \end{aligned}$$

Ableitung der Eingabe $ab\textcolor{blue}{b}aaa \in L(M)$:

$$z_0 \Rightarrow az_1 \Rightarrow ab\textcolor{red}{z}_0 \Rightarrow ab\textcolor{blue}{b}z_0$$

Beispiel für die Konstruktion einer regulären Grammatik

DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, \{z_2\})$ mit

$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(z_1, a) = z_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(z_2, b) = z_2 \end{array}$$

Grammatik dazu: $G = (\{z_0, z_1, z_2\}, \{a, b\}, P, z_0)$ mit

$$\begin{aligned} P = \{ & z_0 \rightarrow az_1, z_0 \rightarrow bz_0, \\ & z_1 \rightarrow az_2, z_1 \rightarrow bz_0, \\ & z_2 \rightarrow az_2, z_2 \rightarrow bz_2, \\ & z_2 \rightarrow \varepsilon \} \end{aligned}$$

Ableitung der Eingabe $abb $\textcolor{blue}{a}$ aa \in L(M)$:

$$z_0 \Rightarrow az_1 \Rightarrow abz_0 \Rightarrow abb\textcolor{red}{z}_0 \Rightarrow abb\textcolor{blue}{a}z_1$$

Beispiel für die Konstruktion einer regulären Grammatik

DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, \{z_2\})$ mit

$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(z_1, a) = z_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(z_2, b) = z_2 \end{array}$$

Grammatik dazu: $G = (\{z_0, z_1, z_2\}, \{a, b\}, P, z_0)$ mit

$$\begin{aligned} P = \{ & z_0 \rightarrow az_1, z_0 \rightarrow bz_0, \\ & z_1 \rightarrow az_2, z_1 \rightarrow bz_0, \\ & z_2 \rightarrow az_2, z_2 \rightarrow bz_2, \\ & z_2 \rightarrow \varepsilon \} \end{aligned}$$

Ableitung der Eingabe $abbaa \in L(M)$:

$$z_0 \Rightarrow az_1 \Rightarrow abz_0 \Rightarrow abbz_0 \Rightarrow abba z_1 \Rightarrow abba z_2$$

Beispiel für die Konstruktion einer regulären Grammatik

DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, \{z_2\})$ mit

$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(z_1, a) = z_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(z_2, b) = z_2 \end{array}$$

Grammatik dazu: $G = (\{z_0, z_1, z_2\}, \{a, b\}, P, z_0)$ mit

$$\begin{aligned} P = \{ & z_0 \rightarrow az_1, z_0 \rightarrow bz_0, \\ & z_1 \rightarrow az_2, z_1 \rightarrow bz_0, \\ & \textcolor{red}{z}_2 \rightarrow \textcolor{blue}{a}z_2, z_2 \rightarrow bz_2, \\ & z_2 \rightarrow \varepsilon \} \end{aligned}$$

Ableitung der Eingabe $abbaa\textcolor{blue}{a} \in L(M)$:

$$z_0 \Rightarrow az_1 \Rightarrow abz_0 \Rightarrow abbz_0 \Rightarrow abba\textcolor{red}{z}_2 \Rightarrow abbaa\textcolor{blue}{a}z_2$$

Beispiel für die Konstruktion einer regulären Grammatik

DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, \{z_2\})$ mit

$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(z_1, a) = z_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(z_2, b) = z_2 \end{array}$$

Grammatik dazu: $G = (\{z_0, z_1, z_2\}, \{a, b\}, P, z_0)$ mit

$$\begin{aligned} P = \{ & z_0 \rightarrow az_1, z_0 \rightarrow bz_0, \\ & z_1 \rightarrow az_2, z_1 \rightarrow bz_0, \\ & z_2 \rightarrow az_2, z_2 \rightarrow bz_2, \\ & \textcolor{red}{z_2} \rightarrow \textcolor{blue}{\epsilon} \} \end{aligned}$$

Ableitung der Eingabe $abbaaa \in L(M)$:

$$z_0 \Rightarrow az_1 \Rightarrow abz_0 \Rightarrow abbz_0 \Rightarrow abbaaz_1 \Rightarrow abbaaz_2 \Rightarrow abbaaa\textcolor{red}{z_2} \Rightarrow abbaaa$$

Beispiel für die Konstruktion einer regulären Grammatik

DFA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \delta, z_0, \{z_2\})$ mit

$$\begin{array}{lll} \delta(z_0, a) = z_1 & \delta(z_1, a) = z_2 & \delta(z_2, a) = z_2 \\ \delta(z_0, b) = z_0 & \delta(z_1, b) = z_0 & \delta(z_2, b) = z_2 \end{array}$$

Grammatik dazu: $G = (\{z_0, z_1, z_2\}, \{a, b\}, P, z_0)$ mit

$$\begin{aligned} P = \{ & z_0 \rightarrow az_1, z_0 \rightarrow bz_0, \\ & z_1 \rightarrow az_2, z_1 \rightarrow bz_0, \\ & z_2 \rightarrow az_2, z_2 \rightarrow bz_2, \\ & z_2 \rightarrow \varepsilon \} \end{aligned}$$

Ableitung der Eingabe $abbaaa \in L(M)$:

$$z_0 \Rightarrow az_1 \Rightarrow abz_0 \Rightarrow abbz_0 \Rightarrow abbaz_1 \Rightarrow abbaaz_2 \Rightarrow abbaaaz_2 \Rightarrow abbaaa \in L(G)$$

Die akzeptierte Sprache von DFAs ist regulär

Theorem

Sei $M = (Z, \Sigma, \delta, z_0, E)$ ein DFA. Dann ist $L(M)$ regulär.

Beweis (Fortsetzung) Wir zeigen, dass $L(M) = L(G)$,
d.h. $w \in L(M)$ g.d.w. $w \in L(G)$ für ein beliebiges w .

Die akzeptierte Sprache von DFAs ist regulär

Theorem

Sei $M = (Z, \Sigma, \delta, z_0, E)$ ein DFA. Dann ist $L(M)$ regulär.

Beweis (Fortsetzung) Wir zeigen, dass $L(M) = L(G)$,
d.h. $w \in L(M)$ g.d.w. $w \in L(G)$ für ein beliebiges w .

Sei $w = a_1 \cdots a_m$ mit $m \in \mathbb{N}$.

Die akzeptierte Sprache von DFAs ist regulär

Theorem

Sei $M = (Z, \Sigma, \delta, z_0, E)$ ein DFA. Dann ist $L(M)$ regulär.

Beweis (Fortsetzung) Wir zeigen, dass $L(M) = L(G)$,
d.h. $w \in L(M)$ g.d.w. $w \in L(G)$ für ein beliebiges w .

Sei $w = a_1 \cdots a_m$ mit $m \in \mathbb{N}$.

$$a_1 \cdots a_m \in L(M)$$

Die akzeptierte Sprache von DFAs ist regulär

Theorem

Sei $M = (Z, \Sigma, \delta, z_0, E)$ ein DFA. Dann ist $L(M)$ regulär.

Beweis (Fortsetzung) Wir zeigen, dass $L(M) = L(G)$,
d.h. $w \in L(M)$ g.d.w. $w \in L(G)$ für ein beliebiges w .

Sei $w = a_1 \cdots a_m$ mit $m \in \mathbb{N}$.

$$a_1 \cdots a_m \in L(M)$$

g.d.w. es gibt $z_1, \dots, z_m \in Z$ mit $\delta(z_{i-1}, a_i) = z_i$ für $i \in \{1, \dots, m\}$ und $z_m \in E$

Die akzeptierte Sprache von DFAs ist regulär

Theorem

Sei $M = (Z, \Sigma, \delta, z_0, E)$ ein DFA. Dann ist $L(M)$ regulär.

Beweis (Fortsetzung) Wir zeigen, dass $L(M) = L(G)$,
d.h. $w \in L(M)$ g.d.w. $w \in L(G)$ für ein beliebiges w .

Sei $w = a_1 \cdots a_m$ mit $m \in \mathbb{N}$.

$$a_1 \cdots a_m \in L(M)$$

g.d.w. es gibt $z_1, \dots, z_m \in Z$ mit $\delta(z_{i-1}, a_i) = z_i$ für $i \in \{1, \dots, m\}$ und $z_m \in E$

$$a_1 \cdots a_m \in L(G)$$

Die akzeptierte Sprache von DFAs ist regulär

Theorem

Sei $M = (Z, \Sigma, \delta, z_0, E)$ ein DFA. Dann ist $L(M)$ regulär.

Beweis (Fortsetzung) Wir zeigen, dass $L(M) = L(G)$,
d.h. $w \in L(M)$ g.d.w. $w \in L(G)$ für ein beliebiges w .

Sei $w = a_1 \cdots a_m$ mit $m \in \mathbb{N}$.

$$a_1 \cdots a_m \in L(M)$$

g.d.w. es gibt $z_1, \dots, z_m \in Z$ mit $\delta(z_{i-1}, a_i) = z_i$ für $i \in \{1, \dots, m\}$ und $z_m \in E$

$$z_0 \Rightarrow_G^* a_1 \cdots a_m$$

g.d.w. $a_1 \cdots a_m \in L(G)$

Die akzeptierte Sprache von DFAs ist regulär

Theorem

Sei $M = (Z, \Sigma, \delta, z_0, E)$ ein DFA. Dann ist $L(M)$ regulär.

Beweis (Fortsetzung) Wir zeigen, dass $L(M) = L(G)$,
d.h. $w \in L(M)$ g.d.w. $w \in L(G)$ für ein beliebiges w .

Sei $w = a_1 \cdots a_m$ mit $m \in \mathbb{N}$.

$$a_1 \cdots a_m \in L(M)$$

g.d.w. es gibt $z_1, \dots, z_m \in Z$ mit $\delta(z_{i-1}, a_i) = z_i$ für $i \in \{1, \dots, m\}$ und $z_m \in E$

es gibt $z_1, \dots, z_m \in Z$ mit $a_1 \cdots a_{i-1} z_{i-1} \Rightarrow_G a_1 \cdots a_i z_i$ für $i \in \{1, \dots, m\}$
und $a_1 \cdots a_m z_m \Rightarrow_G a_1 \cdots a_m$

g.d.w. $z_0 \Rightarrow_G^* a_1 \cdots a_m$

g.d.w. $a_1 \cdots a_m \in L(G)$

Die akzeptierte Sprache von DFAs ist regulär

Theorem

Sei $M = (Z, \Sigma, \delta, z_0, E)$ ein DFA. Dann ist $L(M)$ regulär.

Beweis (Fortsetzung) Wir zeigen, dass $L(M) = L(G)$,
d.h. $w \in L(M)$ g.d.w. $w \in L(G)$ für ein beliebiges w .

Sei $w = a_1 \cdots a_m$ mit $m \in \mathbb{N}$.

$$a_1 \cdots a_m \in L(M)$$

g.d.w. es gibt $z_1, \dots, z_m \in Z$ mit $\delta(z_{i-1}, a_i) = z_i$ für $i \in \{1, \dots, m\}$ und $z_m \in E$

g.d.w. es gibt $z_1, \dots, z_m \in Z$ mit $a_1 \cdots a_{i-1} z_{i-1} \Rightarrow_G a_1 \cdots a_i z_i$ für $i \in \{1, \dots, m\}$
und $a_1 \cdots a_m z_m \Rightarrow_G a_1 \cdots a_m$

g.d.w. $z_0 \Rightarrow_G^* a_1 \cdots a_m$

g.d.w. $a_1 \cdots a_m \in L(G)$

Die akzeptierte Sprache von DFAs ist regulär

Theorem

Sei $M = (Z, \Sigma, \delta, z_0, E)$ ein DFA. Dann ist $L(M)$ regulär.

Beweis (Fortsetzung) Wir zeigen, dass $L(M) = L(G)$,
d.h. $w \in L(M)$ g.d.w. $w \in L(G)$ für ein beliebiges w .

Sei $w = a_1 \cdots a_m$ mit $m \in \mathbb{N}$.

$$a_1 \cdots a_m \in L(M)$$

g.d.w. es gibt $z_1, \dots, z_m \in Z$ mit $\delta(z_{i-1}, a_i) = z_i$ für $i \in \{1, \dots, m\}$ und $z_m \in E$

g.d.w. es gibt $z_1, \dots, z_m \in Z$ mit $a_1 \cdots a_{i-1} z_{i-1} \Rightarrow_G a_1 \cdots a_i z_i$ für $i \in \{1, \dots, m\}$
und $a_1 \cdots a_m z_m \Rightarrow_G a_1 \cdots a_m$

g.d.w. $z_0 \Rightarrow_G^* a_1 \cdots a_m$

g.d.w. $a_1 \cdots a_m \in L(G)$

Daher gilt $L(M) = L(G)$ und somit ist $L(M)$ regulär. □

Wird jede reguläre Sprache durch einen DFA akzeptiert?

- ▶ Der vorherige Beweis konstruiert:
„für jeden DFA gibt es eine äquivalente reguläre Grammatik“
- ▶ Für die andere Richtung wäre notwendig:
„für jede reguläre Grammatik gibt es einen äquivalenten DFA“

Wird jede reguläre Sprache durch einen DFA akzeptiert?

- ▶ Der vorherige Beweis konstruiert:
„für jeden DFA gibt es eine äquivalente reguläre Grammatik“
- ▶ Für die andere Richtung wäre notwendig:
„für jede reguläre Grammatik gibt es einen äquivalenten DFA“

Problem:

- ▶ Produktionen $A \rightarrow aA_1$ und $A \rightarrow aA_2$ können in Grammatiken vorkommen.
- ▶ Die Konstruktion des DFA ist zunächst unklar.

Wird jede reguläre Sprache durch einen DFA akzeptiert?

- ▶ Der vorherige Beweis konstruiert:
„für jeden DFA gibt es eine äquivalente reguläre Grammatik“
- ▶ Für die andere Richtung wäre notwendig:
„für jede reguläre Grammatik gibt es einen äquivalenten DFA“

Problem:

- ▶ Produktionen $A \rightarrow aA_1$ und $A \rightarrow aA_2$ können in Grammatiken vorkommen.
- ▶ Die Konstruktion des DFA ist zunächst unklar.

Daher: Der Beweis für die andere Richtung **erfolgt auf Umwegen** und verwendet **nichtdeterministische** endliche Automaten (NFAs).

Nichtdeterministische endliche Automaten

Informelle Kurzfassung:

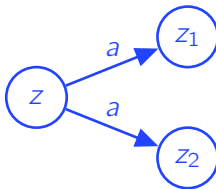
- ▶ Der Zustandswechsel ist nicht eindeutig, sondern **nichtdeterministisch** in einen von mehreren (oder gar keinen) möglichen Zuständen.
- ▶ Der Automat darf sozusagen **raten**, welchen Nachfolgezustand er wählt.

Nichtdeterministische endliche Automaten

Informelle Kurzfassung:

- ▶ Der Zustandswechsel ist nicht eindeutig, sondern **nichtdeterministisch** in einen von mehreren (oder gar keinen) möglichen Zuständen.
- ▶ Der Automat darf sozusagen **raten**, welchen Nachfolgezustand er wählt.

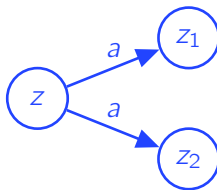
- ▶ Im Zustandsgraph erlaubt:



Nichtdeterministische endliche Automaten

Informelle Kurzfassung:

- ▶ Der Zustandswechsel ist nicht eindeutig, sondern **nichtdeterministisch** in einen von mehreren (oder gar keinen) möglichen Zuständen.
- ▶ Der Automat darf sozusagen **raten**, welchen Nachfolgezustand er wählt.

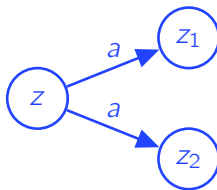


- ▶ Technisch:
 - ▶ Ein **DFA** hat $\delta : Z \times \Sigma \rightarrow Z$ und **einen** Startzustand.
 - ▶ Ein **NFA** hat $\delta : Z \times \Sigma \rightarrow \mathcal{P}(Z)$ und eine **Menge** von Startzuständen.

Nichtdeterministische endliche Automaten

Informelle Kurzfassung:

- ▶ Der Zustandswechsel ist nicht eindeutig, sondern **nichtdeterministisch** in einen von mehreren (oder gar keinen) möglichen Zuständen.
- ▶ Der Automat darf sozusagen **raten**, welchen Nachfolgezustand er wählt.



- ▶ Technisch:
 - ▶ Ein **DFA** hat $\delta : Z \times \Sigma \rightarrow Z$ und **einen** Startzustand.
 - ▶ Ein **NFA** hat $\delta : Z \times \Sigma \rightarrow \mathcal{P}(Z)$ und eine **Menge** von Startzuständen.
- ▶ NFAs machen **manche Konstruktionen einfacher**.

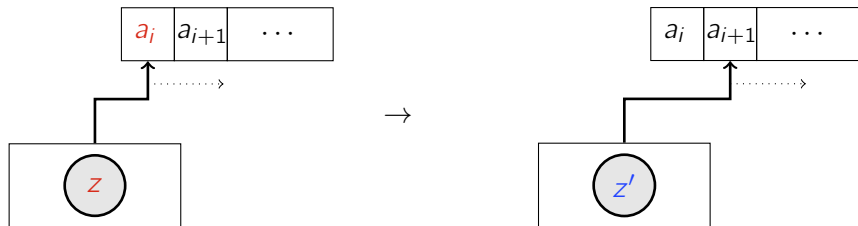
Definition eines NFA

Definition

Ein nichtdeterministischer endlicher Automat (*nondeterministic finite automaton*, NFA) ist ein 5-Tupel $M = (Z, \Sigma, \delta, S, E)$, wobei

- ▶ Z ist eine endliche Menge von Zuständen
- ▶ Σ ist das (endliche) Eingabealphabet mit $Z \cap \Sigma = \emptyset$
- ▶ $\delta : Z \times \Sigma \rightarrow \mathcal{P}(Z)$ ist die Überföhrungsfunktion
- ▶ $S \subseteq Z$ ist die Menge der Startzustände
- ▶ $E \subseteq Z$ ist die Menge der Endzustände.

Illustration des Zustandsübergangs



$z' \in \delta(z, a_i)$ bedeutet:

Im Zustand z bei Eingabe a_i darf der NFA in z' wechseln.

Beispiel für einen NFA

NFA $M = (\{z_0, z_1, z_2, z_3\}, \{a, b, c\}, \delta, \{z_0, z_3\}, \{z_3\})$ mit

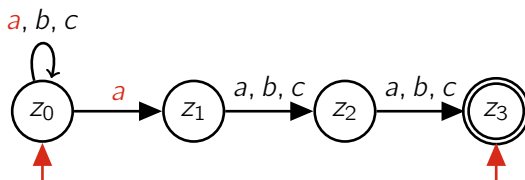
$$\begin{array}{llll} \delta(z_0, a) = \{z_0, z_1\} & \delta(z_1, a) = \{z_2\} & \delta(z_2, a) = \{z_3\} & \delta(z_3, a) = \emptyset \\ \delta(z_0, b) = \{z_0\} & \delta(z_1, b) = \{z_2\} & \delta(z_2, b) = \{z_3\} & \delta(z_3, b) = \emptyset \\ \delta(z_0, c) = \{z_0\} & \delta(z_1, c) = \{z_2\} & \delta(z_2, c) = \{z_3\} & \delta(z_3, c) = \emptyset \end{array}$$

Beispiel für einen NFA

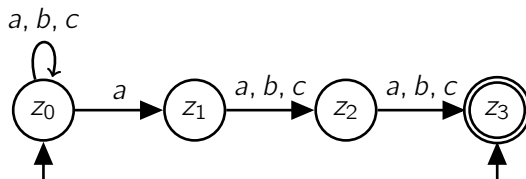
NFA $M = (\{z_0, z_1, z_2, z_3\}, \{a, b, c\}, \delta, \{z_0, z_3\}, \{z_3\})$ mit

$$\begin{array}{llll} \delta(z_0, a) = \{z_0, z_1\} & \delta(z_1, a) = \{z_2\} & \delta(z_2, a) = \{z_3\} & \delta(z_3, a) = \emptyset \\ \delta(z_0, b) = \{z_0\} & \delta(z_1, b) = \{z_2\} & \delta(z_2, b) = \{z_3\} & \delta(z_3, b) = \emptyset \\ \delta(z_0, c) = \{z_0\} & \delta(z_1, c) = \{z_2\} & \delta(z_2, c) = \{z_3\} & \delta(z_3, c) = \emptyset \end{array}$$

Zustandsgraph zu M :

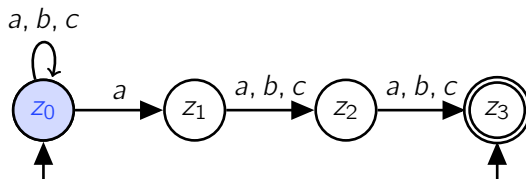


Beispiel für einen akzeptierenden Lauf



Abarbeitung der Eingabe *caaba*:

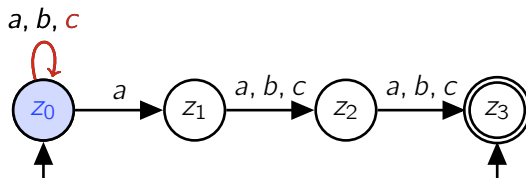
Beispiel für einen akzeptierenden Lauf



Abarbeitung der Eingabe *caaba*:

- Starte in z_0 .

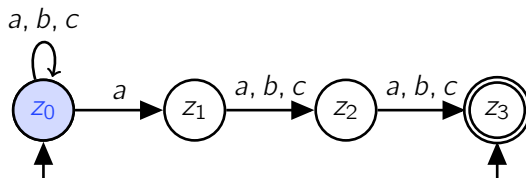
Beispiel für einen akzeptierenden Lauf



Abarbeitung der Eingabe $caaba$:

- ▶ Starte in z_0 .
- ▶ Lies c

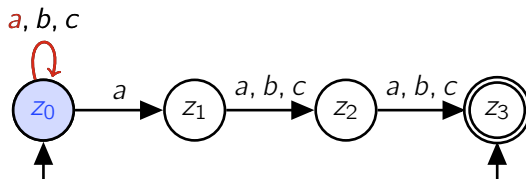
Beispiel für einen akzeptierenden Lauf



Abarbeitung der Eingabe *caaba*:

- ▶ Starte in z_0 .
- ▶ Lies c und wechsle in z_0 .

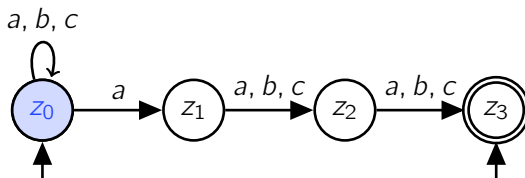
Beispiel für einen akzeptierenden Lauf



Abarbeitung der Eingabe $c\textcolor{red}{a}ba$:

- ▶ Starte in z_0 .
- ▶ Lies c und wechsle in z_0 .
- ▶ Lies a

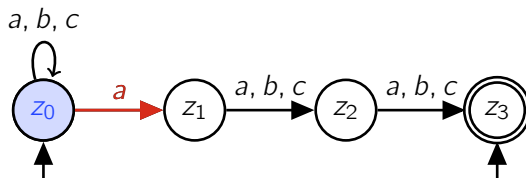
Beispiel für einen akzeptierenden Lauf



Abarbeitung der Eingabe *c***a***ba*:

- ▶ Starte in z_0 .
- ▶ Lies c und wechsle in z_0 .
- ▶ Lies a und wechsle in z_0 .

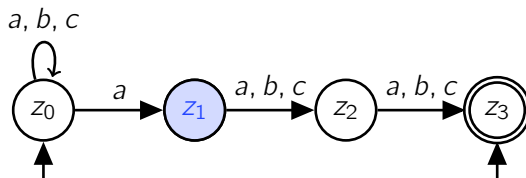
Beispiel für einen akzeptierenden Lauf



Abarbeitung der Eingabe $caaba$:

- ▶ Starte in z_0 .
- ▶ Lies c und wechsle in z_0 .
- ▶ Lies a und wechsle in z_1 .
- ▶ Lies a

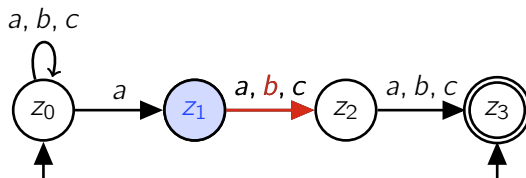
Beispiel für einen akzeptierenden Lauf



Abarbeitung der Eingabe $caaba$:

- ▶ Starte in z_0 .
- ▶ Lies c und wechsle in z_0 .
- ▶ Lies a und wechsle in z_1 .
- ▶ Lies a und wechsle in z_1 .

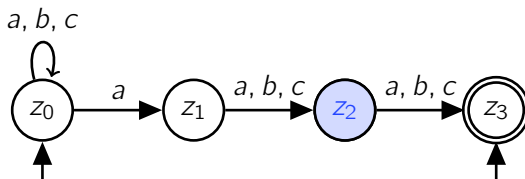
Beispiel für einen akzeptierenden Lauf



Abarbeitung der Eingabe $caab**a**$:

- ▶ Starte in z_0 .
- ▶ Lies c und wechsle in z_0 .
- ▶ Lies a und wechsle in z_1 .
- ▶ Lies b

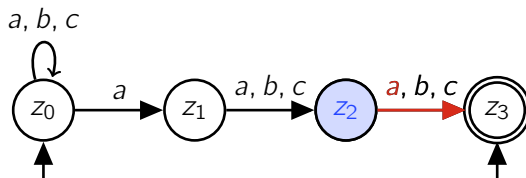
Beispiel für einen akzeptierenden Lauf



Abarbeitung der Eingabe $caaba$:

- ▶ Starte in z_0 .
- ▶ Lies c und wechsle in z_0 .
- ▶ Lies a und wechsle in z_0 .
- ▶ Lies a und wechsle in z_1 .
- ▶ Lies b und wechsle in z_2 .

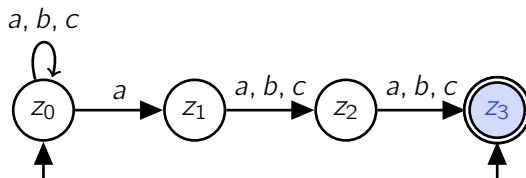
Beispiel für einen akzeptierenden Lauf



Abarbeitung der Eingabe $caaba$:

- ▶ Starte in z_0 .
- ▶ Lies c und wechsle in z_0 .
- ▶ Lies a und wechsle in z_1 .
- ▶ Lies b und wechsle in z_2 .
- ▶ Lies a

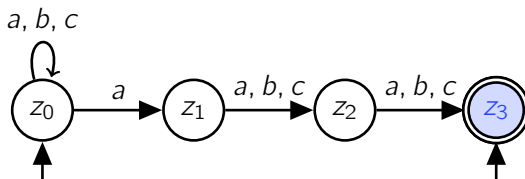
Beispiel für einen akzeptierenden Lauf



Abarbeitung der Eingabe $caaba$:

- ▶ Starte in z_0 .
- ▶ Lies c und wechsle in z_0 .
- ▶ Lies a und wechsle in z_0 .
- ▶ Lies a und wechsle in z_1 .
- ▶ Lies b und wechsle in z_2 .
- ▶ Lies a und wechsle in z_3 .

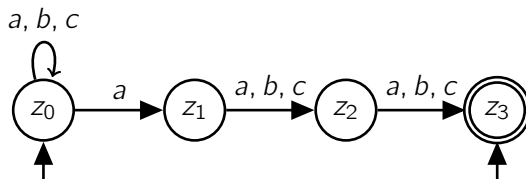
Beispiel für einen akzeptierenden Lauf



Abarbeitung der Eingabe *caaba*:

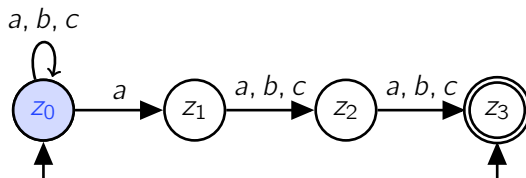
- ▶ Starte in z_0 .
- ▶ Lies c und wechsle in z_0 .
- ▶ Lies a und wechsle in z_1 .
- ▶ Lies b und wechsle in z_2 .
- ▶ Lies a und wechsle in z_3 .
- ▶ Akzeptiere.

Beispiel für einen verworfenden Lauf



Abarbeitung der Eingabe *caaba*:

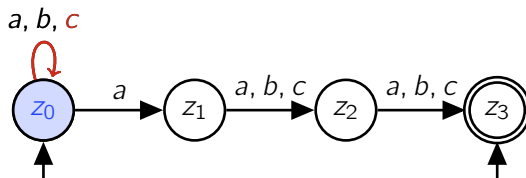
Beispiel für einen verworfenden Lauf



Abarbeitung der Eingabe *caaba*:

- Starte in z_0 .

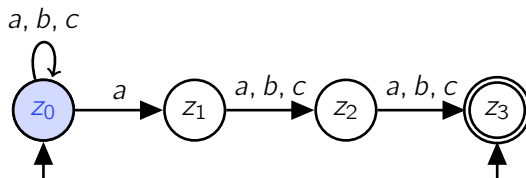
Beispiel für einen verwerfenden Lauf



Abarbeitung der Eingabe $caaba$:

- ▶ Starte in z_0 .
- ▶ Lies c

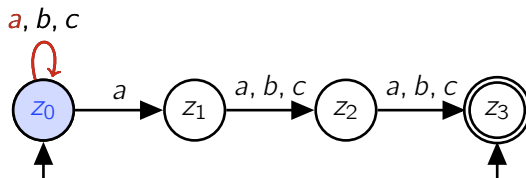
Beispiel für einen verwerfenden Lauf



Abarbeitung der Eingabe *caaba*:

- ▶ Starte in z_0 .
- ▶ Lies c und wechsle in z_0 .

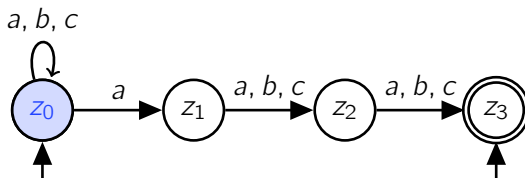
Beispiel für einen verwerfenden Lauf



Abarbeitung der Eingabe $c\textcolor{red}{a}ba$:

- ▶ Starte in z_0 .
- ▶ Lies c und wechsle in z_0 .
- ▶ Lies a

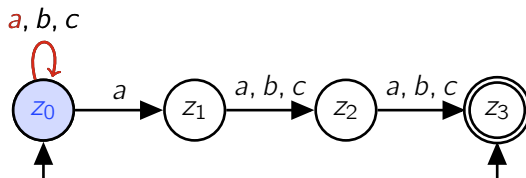
Beispiel für einen verwerfenden Lauf



Abarbeitung der Eingabe $caba$:

- ▶ Starte in z_0 .
- ▶ Lies c und wechsle in z_0 .
- ▶ Lies a und wechsle in z_0 .

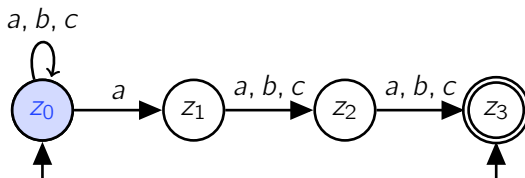
Beispiel für einen verworfenden Lauf



Abarbeitung der Eingabe $caaba$:

- ▶ Starte in z_0 .
- ▶ Lies c und wechsle in z_0 .
- ▶ Lies a und wechsle in z_1 .
- ▶ Lies a

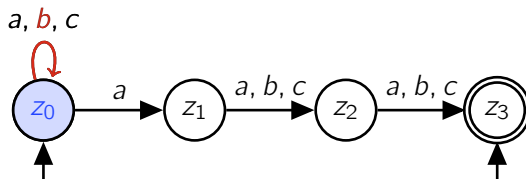
Beispiel für einen verworfenden Lauf



Abarbeitung der Eingabe *caaba*:

- ▶ Starte in z_0 .
- ▶ Lies *c* und wechsle in z_0 .
- ▶ Lies *a* und wechsle in z_1 .
- ▶ Lies *a* und wechsle in z_1 .

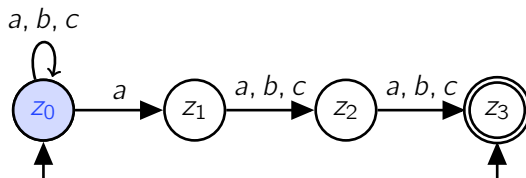
Beispiel für einen verworfenden Lauf



Abarbeitung der Eingabe $caab**a**$:

- ▶ Starte in z_0 .
- ▶ Lies c und wechsle in z_0 .
- ▶ Lies a und wechsle in z_0 .
- ▶ Lies a und wechsle in z_0 .
- ▶ Lies b

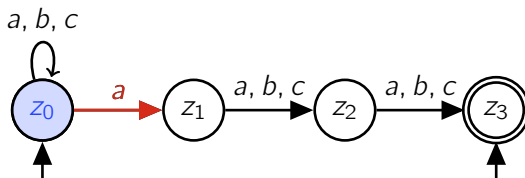
Beispiel für einen verworfenden Lauf



Abarbeitung der Eingabe $caaba$:

- ▶ Starte in z_0 .
- ▶ Lies c und wechsle in z_0 .
- ▶ Lies a und wechsle in z_1 .
- ▶ Lies b und wechsle in z_0 .
- ▶ Lies a und wechsle in z_1 .
- ▶ Lies a und wechsle in z_1 .

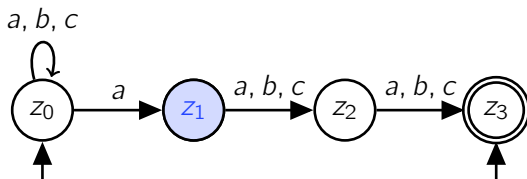
Beispiel für einen verworfenden Lauf



Abarbeitung der Eingabe *caaba*:

- ▶ Starte in z_0 .
- ▶ Lies c und wechsle in z_0 .
- ▶ Lies a und wechsle in z_1 .
- ▶ Lies b und wechsle in z_1 .
- ▶ Lies a und wechsle in z_2 .
- ▶ Lies b und wechsle in z_2 .
- ▶ Lies a und wechsle in z_3 .

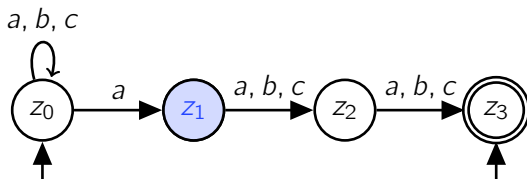
Beispiel für einen verworfenden Lauf



Abarbeitung der Eingabe *caab***a**:

- ▶ Starte in z_0 .
- ▶ Lies c und wechsle in z_0 .
- ▶ Lies a und wechsle in z_0 .
- ▶ Lies a und wechsle in z_0 .
- ▶ Lies b und wechsle in z_0 .
- ▶ Lies a und wechsle in z_1 .

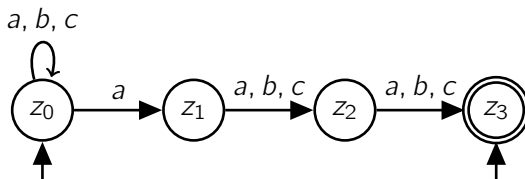
Beispiel für einen verwerfenden Lauf



Abarbeitung der Eingabe *caaba*:

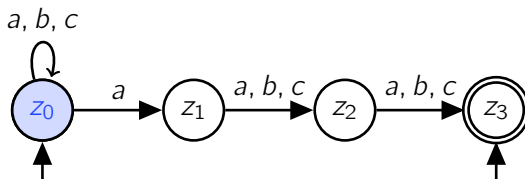
- ▶ Starte in z_0 .
- ▶ Lies c und wechsle in z_0 .
- ▶ Lies a und wechsle in z_0 .
- ▶ Lies a und wechsle in z_0 .
- ▶ Lies b und wechsle in z_0 .
- ▶ Lies a und wechsle in z_1 .
- ▶ **Verwerfe.**

Beispiel für einen steckengebliebenen Lauf



Abarbeitung der Eingabe *caaba*:

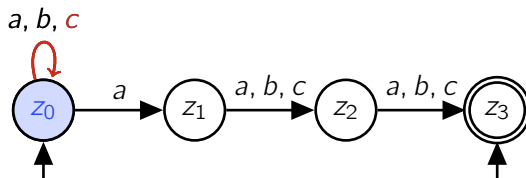
Beispiel für einen steckengebliebenen Lauf



Abarbeitung der Eingabe *caaba*:

- Starte in z_0 .

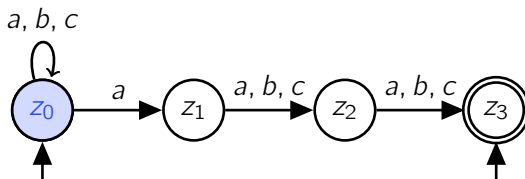
Beispiel für einen steckengebliebenen Lauf



Abarbeitung der Eingabe $caaba$:

- ▶ Starte in z_0 .
- ▶ Lies c

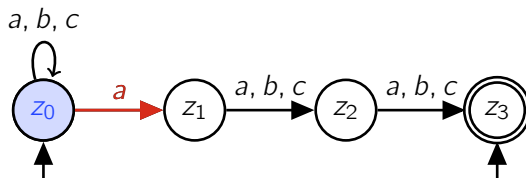
Beispiel für einen steckengebliebenen Lauf



Abarbeitung der Eingabe *caaba*:

- ▶ Starte in z_0 .
- ▶ Lies c und wechsle in z_0 .

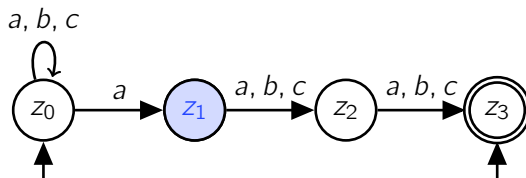
Beispiel für einen steckengebliebenen Lauf



Abarbeitung der Eingabe $c\textcolor{red}{a}ba$:

- ▶ Starte in z_0 .
- ▶ Lies c und wechsle in z_0 .
- ▶ Lies a

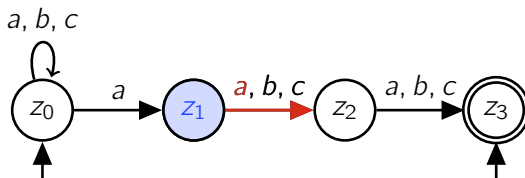
Beispiel für einen steckengebliebenen Lauf



Abarbeitung der Eingabe *c***a***ba*:

- ▶ Starte in z_0 .
- ▶ Lies c und wechsle in z_0 .
- ▶ Lies a und wechsle in z_1 .

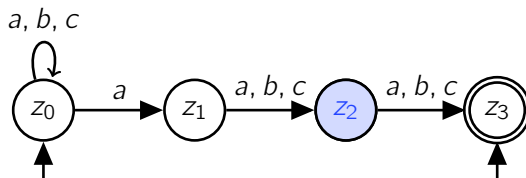
Beispiel für einen steckengebliebenen Lauf



Abarbeitung der Eingabe $caaba$:

- ▶ Starte in z_0 .
- ▶ Lies c und wechsle in z_0 .
- ▶ Lies a und wechsle in z_1 .
- ▶ Lies a

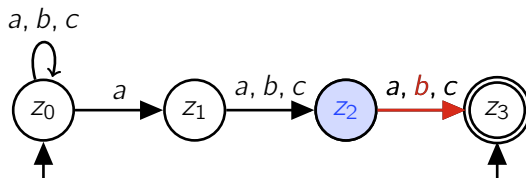
Beispiel für einen steckengebliebenen Lauf



Abarbeitung der Eingabe $caaba$:

- ▶ Starte in z_0 .
- ▶ Lies c und wechsle in z_0 .
- ▶ Lies a und wechsle in z_1 .
- ▶ Lies a und wechsle in z_2 .

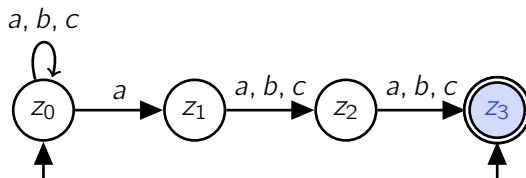
Beispiel für einen steckengebliebenen Lauf



Abarbeitung der Eingabe $ca\textcolor{red}{a}ba$:

- ▶ Starte in z_0 .
- ▶ Lies c und wechsle in z_0 .
- ▶ Lies a und wechsle in z_1 .
- ▶ Lies a und wechsle in z_2 .
- ▶ Lies b

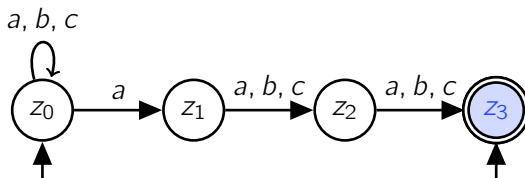
Beispiel für einen steckengebliebenen Lauf



Abarbeitung der Eingabe $caaba$:

- ▶ Starte in z_0 .
- ▶ Lies c und wechsle in z_0 .
- ▶ Lies a und wechsle in z_1 .
- ▶ Lies a und wechsle in z_2 .
- ▶ Lies b und wechsle in z_3 .

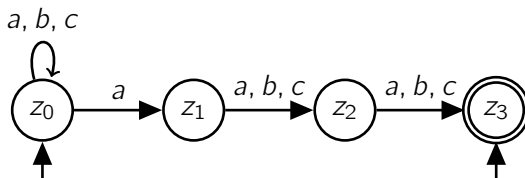
Beispiel für einen steckengebliebenen Lauf



Abarbeitung der Eingabe *caaba*:

- ▶ Starte in z_0 .
- ▶ Lies c und wechsle in z_0 .
- ▶ Lies a und wechsle in z_1 .
- ▶ Lies a und wechsle in z_2 .
- ▶ Lies b und wechsle in z_3 .
- ▶ Lies a und ...?

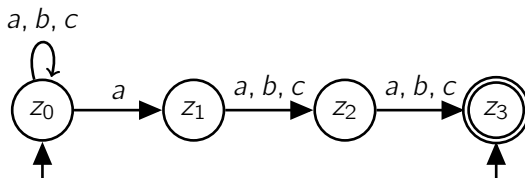
Beispiel für einen steckengebliebenen Lauf



Abarbeitung der Eingabe *caab***a**:

- ▶ Starte in z_0 .
- ▶ Lies c und wechsle in z_0 .
- ▶ Lies a und wechsle in z_1 .
- ▶ Lies a und wechsle in z_2 .
- ▶ Lies b und wechsle in z_3 .
- ▶ Lies a und ...?

Beispiel für einen steckengebliebenen Lauf



Abarbeitung der Eingabe *caaba*:

- ▶ Starte in z_0 .
- ▶ Lies c und wechsle in z_0 .
- ▶ Lies a und wechsle in z_1 .
- ▶ Lies a und wechsle in z_2 .
- ▶ Lies b und wechsle in z_3 .
- ▶ Lies a und ...?
- ▶ **Verwerfe.**

Akzeptanz bei NFAs

Ein Wort w wird vom NFA akzeptiert, wenn es **mindestens einen** Pfad von **einem** Startzustand zu **einem** Endzustand entlang w gibt.

Akzeptanz bei NFAs

Ein Wort w wird vom NFA akzeptiert, wenn es **mindestens einen** Pfad von **einem** Startzustand zu **einem** Endzustand entlang w gibt.

Definition

Sei $M = (Z, \Sigma, \delta, S, E)$ ein NFA.

Wir definieren $\tilde{\delta} : \mathcal{P}(Z) \times \Sigma^* \rightarrow \mathcal{P}(Z)$ rekursiv durch

$$\begin{aligned}\tilde{\delta}(X, \epsilon) &:= X \\ \tilde{\delta}(X, aw) &:= \tilde{\delta}\left(\bigcup_{z \in X} \delta(z, a), w\right)\end{aligned}$$

Akzeptanz bei NFAs

Ein Wort w wird vom NFA akzeptiert, wenn es **mindestens einen** Pfad von **einem** Startzustand zu **einem** Endzustand entlang w gibt.

Definition

Sei $M = (Z, \Sigma, \delta, S, E)$ ein NFA.

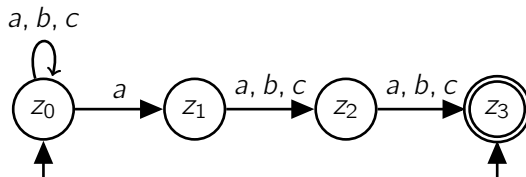
Wir definieren $\tilde{\delta} : \mathcal{P}(Z) \times \Sigma^* \rightarrow \mathcal{P}(Z)$ rekursiv durch

$$\begin{aligned}\tilde{\delta}(X, \epsilon) &:= X \\ \tilde{\delta}(X, aw) &:= \tilde{\delta}\left(\bigcup_{z \in X} \delta(z, a), w\right)\end{aligned}$$

Die von M **akzeptierte Sprache** ist

$$L(M) := \{w \in \Sigma^* \mid \tilde{\delta}(S, w) \cap E \neq \emptyset\}$$

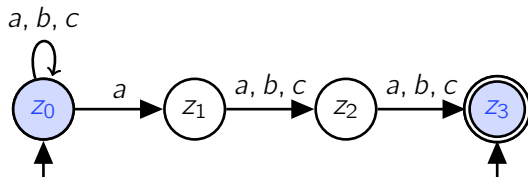
Beispiel für die $\tilde{\delta}$ -Funktion



Abarbeitung der Eingabe *caaba*:

$$\tilde{\delta}(\{z_0, z_3\}, caaba) =$$

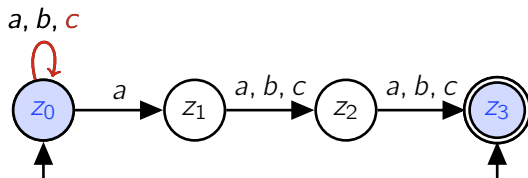
Beispiel für die $\tilde{\delta}$ -Funktion



Abarbeitung der Eingabe **c**aaba:

$$\tilde{\delta}(\{z_0, z_3\}, caaba) = \tilde{\delta}(\bigcup_{z \in \{z_0, z_3\}} \delta(z, \mathbf{c}), aaba)$$

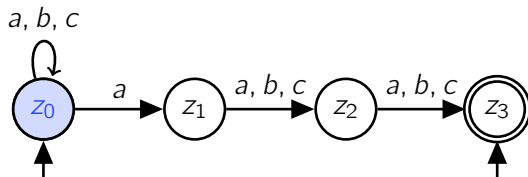
Beispiel für die $\tilde{\delta}$ -Funktion



Abarbeitung der Eingabe $caaba$:

$$\tilde{\delta}(\{z_0, z_3\}, caaba) = \tilde{\delta}(\delta(z_0, c) \cup \delta(z_3, c), aaba)$$

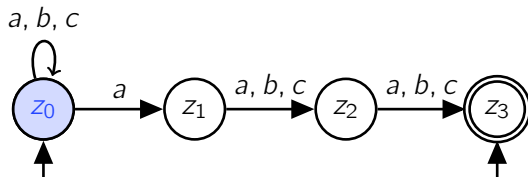
Beispiel für die $\tilde{\delta}$ -Funktion



Abarbeitung der Eingabe **caaba**:

$$\tilde{\delta}(\{z_0, z_3\}, caaba) = \tilde{\delta}(\{z_0\} \cup \emptyset, aaba)$$

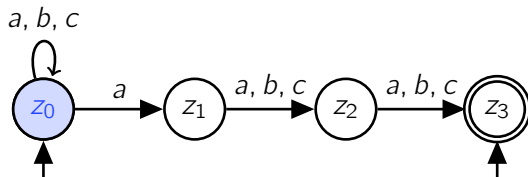
Beispiel für die $\tilde{\delta}$ -Funktion



Abarbeitung der Eingabe **caaba**:

$$\tilde{\delta}(\{z_0, z_3\}, caaba) = \tilde{\delta}(\{z_0\}, aaba)$$

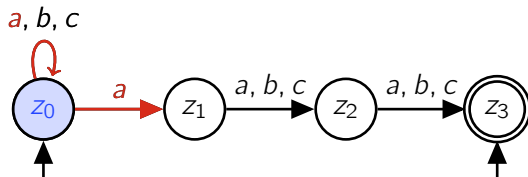
Beispiel für die $\tilde{\delta}$ -Funktion



Abarbeitung der Eingabe $c\textcolor{red}{a}aba$:

$$\tilde{\delta}(\{z_0, z_3\}, c\textcolor{red}{a}aba) = \tilde{\delta}(\bigcup_{z \in \{z_0\}} \delta(z, \textcolor{red}{a}), aba)$$

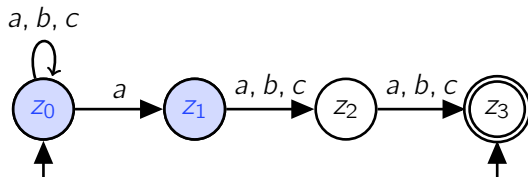
Beispiel für die $\tilde{\delta}$ -Funktion



Abarbeitung der Eingabe $c\textcolor{red}{a}aba$:

$$\tilde{\delta}(\{z_0, z_3\}, c\textcolor{red}{a}aba) = \tilde{\delta}(\delta(\textcolor{blue}{z}_0, \textcolor{red}{a}), aba)$$

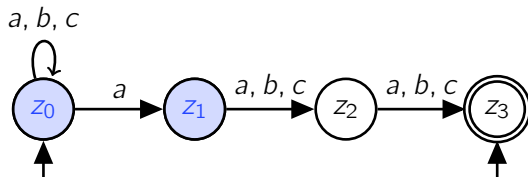
Beispiel für die $\tilde{\delta}$ -Funktion



Abarbeitung der Eingabe $c\textcolor{red}{a}aba$:

$$\tilde{\delta}(\{z_0, z_3\}, c\textcolor{red}{a}aba) = \tilde{\delta}(\{\textcolor{blue}{z}_0, \textcolor{blue}{z}_1\}, aba)$$

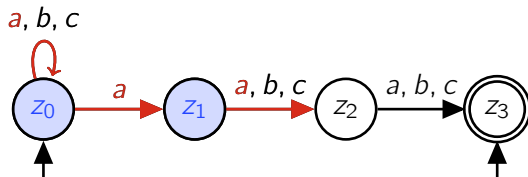
Beispiel für die $\tilde{\delta}$ -Funktion



Abarbeitung der Eingabe $caaba$:

$$\tilde{\delta}(\{z_0, z_3\}, caaba) = \tilde{\delta}(\bigcup_{z \in \{z_0, z_1\}} \delta(z, a), ba)$$

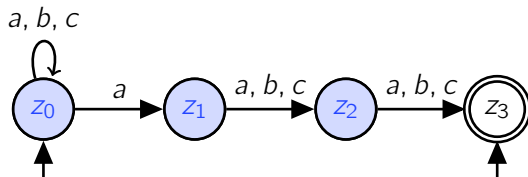
Beispiel für die $\tilde{\delta}$ -Funktion



Abarbeitung der Eingabe $caaba$:

$$\tilde{\delta}(\{z_0, z_3\}, caaba) = \tilde{\delta}(\delta(z_0, a) \cup \delta(z_1, a), ba)$$

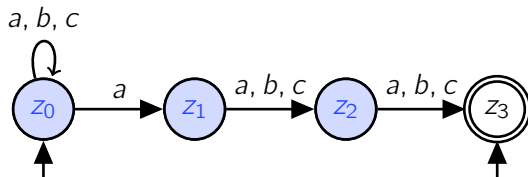
Beispiel für die $\tilde{\delta}$ -Funktion



Abarbeitung der Eingabe $caaba$:

$$\tilde{\delta}(\{z_0, z_3\}, caaba) = \tilde{\delta}(\{z_0, z_1\} \cup \{z_2\}, ba)$$

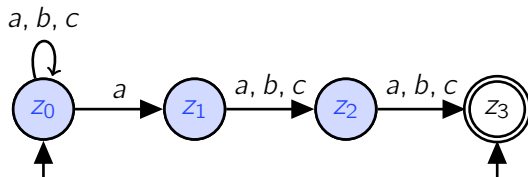
Beispiel für die $\tilde{\delta}$ -Funktion



Abarbeitung der Eingabe $caaba$:

$$\tilde{\delta}(\{z_0, z_3\}, caaba) = \tilde{\delta}(\{z_0, z_1, z_2\}, ba)$$

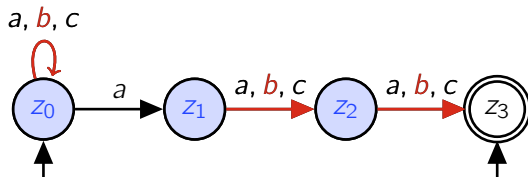
Beispiel für die $\tilde{\delta}$ -Funktion



Abarbeitung der Eingabe $caaba$:

$$\tilde{\delta}(\{z_0, z_3\}, caaba) = \tilde{\delta}(\bigcup_{z \in \{z_0, z_1, z_2\}} \delta(z, \textcolor{red}{b}), a)$$

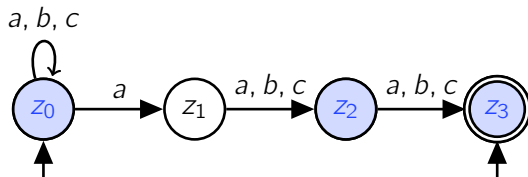
Beispiel für die $\tilde{\delta}$ -Funktion



Abarbeitung der Eingabe $caaba$:

$$\tilde{\delta}(\{z_0, z_3\}, caaba) = \tilde{\delta}(\delta(z_0, b) \cup \delta(z_1, b) \cup \delta(z_2, b), a)$$

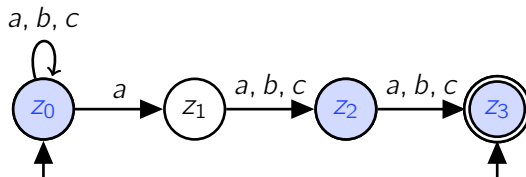
Beispiel für die $\tilde{\delta}$ -Funktion



Abarbeitung der Eingabe $caaba$:

$$\tilde{\delta}(\{z_0, z_3\}, caaba) = \tilde{\delta}(\{z_0\} \cup \{z_2\} \cup \{z_3\}, a)$$

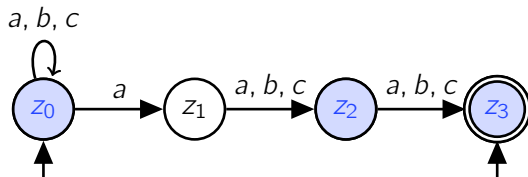
Beispiel für die $\tilde{\delta}$ -Funktion



Abarbeitung der Eingabe $caaba$:

$$\tilde{\delta}(\{z_0, z_3\}, caaba) = \tilde{\delta}(\{z_0, z_2, z_3\}, a)$$

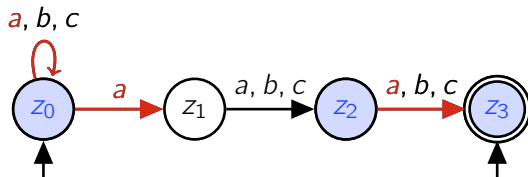
Beispiel für die $\tilde{\delta}$ -Funktion



Abarbeitung der Eingabe $caaba$:

$$\tilde{\delta}(\{z_0, z_3\}, caaba) = \tilde{\delta}(\bigcup_{z \in \{z_0, z_2, z_3\}} \delta(z, a), \varepsilon)$$

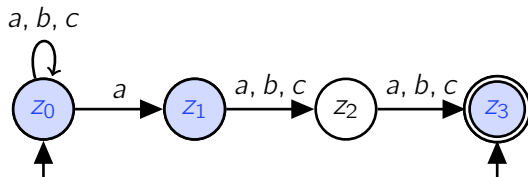
Beispiel für die $\tilde{\delta}$ -Funktion



Abarbeitung der Eingabe $caaba$:

$$\tilde{\delta}(\{z_0, z_3\}, caaba) = \tilde{\delta}(\delta(z_0, a) \cup \delta(z_2, a) \cup \delta(z_3, a), \varepsilon)$$

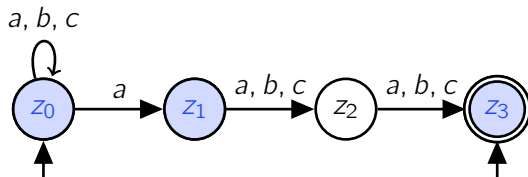
Beispiel für die $\tilde{\delta}$ -Funktion



Abarbeitung der Eingabe *caaba*:

$$\tilde{\delta}(\{z_0, z_3\}, caaba) = \tilde{\delta}(\{z_0, z_1\} \cup \{z_3\} \cup \emptyset, \epsilon)$$

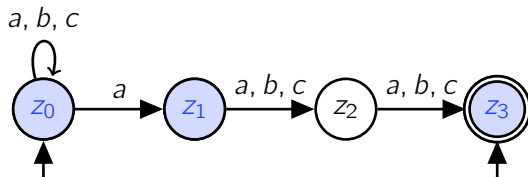
Beispiel für die $\tilde{\delta}$ -Funktion



Abarbeitung der Eingabe *caaba*:

$$\tilde{\delta}(\{z_0, z_3\}, caaba) = \tilde{\delta}(\{z_0, z_1, z_3\}, \epsilon)$$

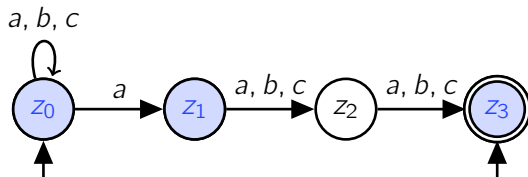
Beispiel für die $\tilde{\delta}$ -Funktion



Abarbeitung der Eingabe $caaba$:

$$\tilde{\delta}(\{z_0, z_3\}, caaba) = \{z_0, z_1, z_3\}$$

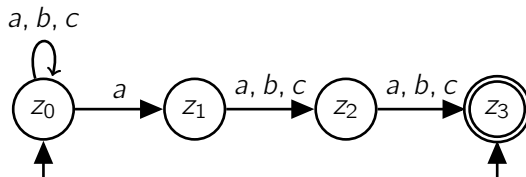
Beispiel für die $\tilde{\delta}$ -Funktion



Abarbeitung der Eingabe *caaba*:

$$\tilde{\delta}(\{z_0, z_3\}, caaba) = \{z_0, z_1, z_3\} \cap \{z_3\} \neq \emptyset$$

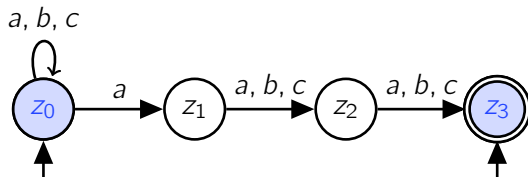
Weiteres Beispiel für die $\tilde{\delta}$ -Funktion



Abarbeitung der Eingabe *bab*:

$$\tilde{\delta}(\{z_0, z_3\}, bab) =$$

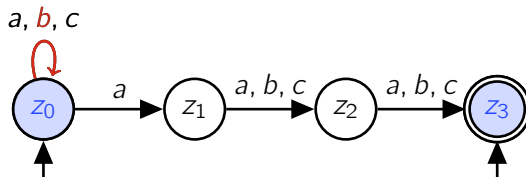
Weiteres Beispiel für die $\tilde{\delta}$ -Funktion



Abarbeitung der Eingabe *bab*:

$$\tilde{\delta}(\{z_0, z_3\}, bab) = \tilde{\delta}(\bigcup_{z \in \{z_0, z_3\}} \delta(z, \textcolor{red}{b}), ab)$$

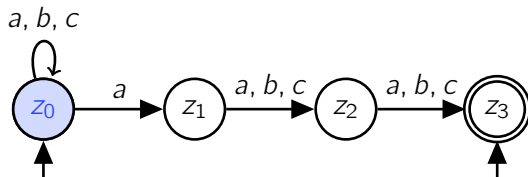
Weiteres Beispiel für die $\tilde{\delta}$ -Funktion



Abarbeitung der Eingabe bab :

$$\tilde{\delta}(\{z_0, z_3\}, bab) = \tilde{\delta}(\delta(z_0, b) \cup \delta(z_3, b), ab)$$

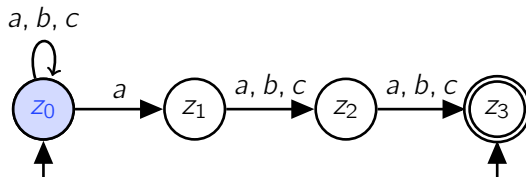
Weiteres Beispiel für die $\tilde{\delta}$ -Funktion



Abarbeitung der Eingabe *bab*:

$$\tilde{\delta}(\{z_0, z_3\}, bab) = \tilde{\delta}(\{z_0\} \cup \emptyset, ab)$$

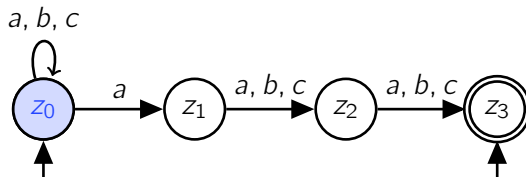
Weiteres Beispiel für die $\tilde{\delta}$ -Funktion



Abarbeitung der Eingabe *bab*:

$$\tilde{\delta}(\{z_0, z_3\}, bab) = \tilde{\delta}(\{z_0\}, ab)$$

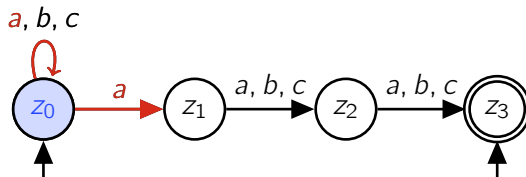
Weiteres Beispiel für die $\tilde{\delta}$ -Funktion



Abarbeitung der Eingabe bab :

$$\tilde{\delta}(\{z_0, z_3\}, bab) = \tilde{\delta}(\bigcup_{z \in \{z_0\}} \delta(z, a), b)$$

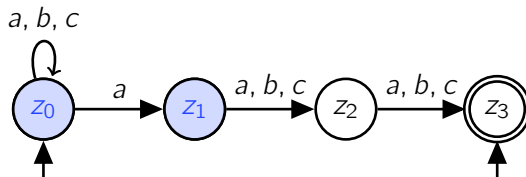
Weiteres Beispiel für die $\tilde{\delta}$ -Funktion



Abarbeitung der Eingabe bab :

$$\tilde{\delta}(\{z_0, z_3\}, bab) = \tilde{\delta}(\delta(z_0, a), b)$$

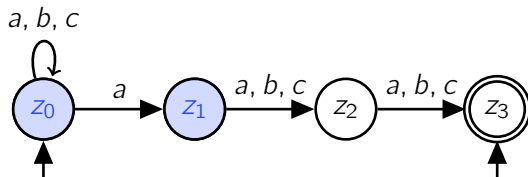
Weiteres Beispiel für die $\tilde{\delta}$ -Funktion



Abarbeitung der Eingabe bab :

$$\tilde{\delta}(\{z_0, z_3\}, bab) = \tilde{\delta}(\{z_0, z_1\}, b)$$

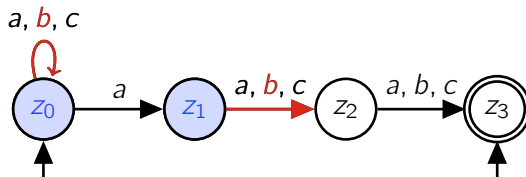
Weiteres Beispiel für die $\tilde{\delta}$ -Funktion



Abarbeitung der Eingabe bab :

$$\tilde{\delta}(\{z_0, z_3\}, bab) = \tilde{\delta}(\bigcup_{z \in \{z_0, z_1\}} \delta(z, \textcolor{red}{b}), \varepsilon)$$

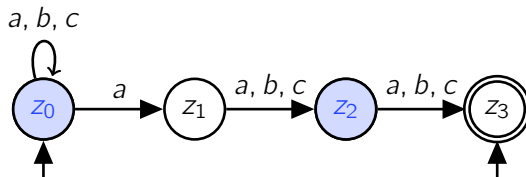
Weiteres Beispiel für die $\tilde{\delta}$ -Funktion



Abarbeitung der Eingabe bab :

$$\tilde{\delta}(\{z_0, z_3\}, bab) = \tilde{\delta}(\delta(z_0, b) \cup \delta(z_1, b), \epsilon)$$

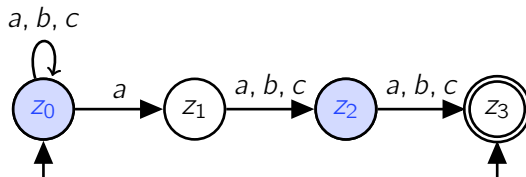
Weiteres Beispiel für die $\tilde{\delta}$ -Funktion



Abarbeitung der Eingabe bab :

$$\tilde{\delta}(\{z_0, z_3\}, bab) = \tilde{\delta}(\{z_0\} \cup \{z_2\}, \epsilon)$$

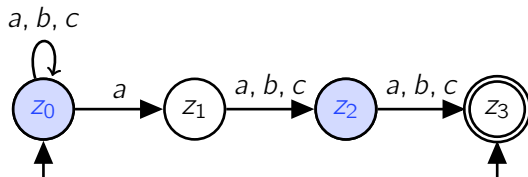
Weiteres Beispiel für die $\tilde{\delta}$ -Funktion



Abarbeitung der Eingabe bab :

$$\tilde{\delta}(\{z_0, z_3\}, bab) = \tilde{\delta}(\{z_0, z_2\}, \epsilon)$$

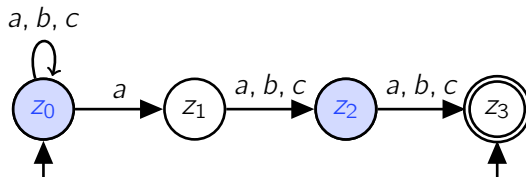
Weiteres Beispiel für die $\tilde{\delta}$ -Funktion



Abarbeitung der Eingabe bab :

$$\tilde{\delta}(\{z_0, z_3\}, bab) = \{z_0, z_2\}$$

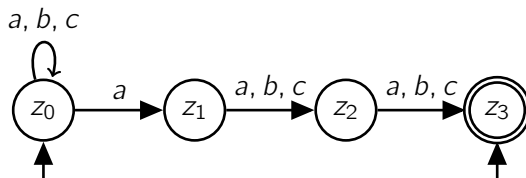
Weiteres Beispiel für die $\tilde{\delta}$ -Funktion



Abarbeitung der Eingabe bab :

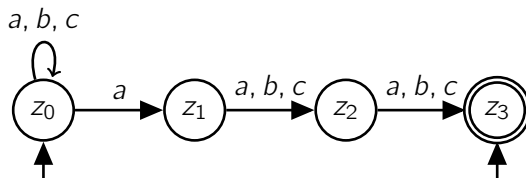
$$\tilde{\delta}(\{z_0, z_3\}, bab) = \{z_0, z_2\} \cap \{z_3\} = \emptyset$$

Akzeptierte Sprache des Beispiels



Akzeptierte Sprache = ?

Akzeptierte Sprache des Beispiels



Akzeptierte Sprache = $\{\epsilon\} \cup \{uav \mid u \in \{a, b, c\}^*, v \in \{a, b, c\}^2\}$

Definition

Sei $M = (Z, \Sigma, \delta, S, E)$ ein NFA und $w \in \Sigma^*$ ein Wort der Länge n .
Eine Folge von Zuständen $z_0, \dots, z_n$ mit $z_0 \in S$ und $z_i \in \delta(z_{i-1}, w[i])$ für $i \in \{1, \dots, n\}$ ist ein **Lauf** von M für w .

Definition

Sei $M = (Z, \Sigma, \delta, S, E)$ ein NFA und $w \in \Sigma^*$ ein Wort der Länge n .
Eine Folge von Zuständen $z_0, \dots, z_n$ mit $z_0 \in S$ und $z_i \in \delta(z_{i-1}, w[i])$ für $i \in \{1, \dots, n\}$ ist ein **Lauf** von M für w .

Für einen Lauf schreiben wir auch

$$z_0 \xrightarrow{w[1]} z_1 \xrightarrow{w[2]} \dots \xrightarrow{w[n-1]} z_{n-1} \xrightarrow{w[n]} z_n$$

Definition

Sei $M = (Z, \Sigma, \delta, S, E)$ ein NFA und $w \in \Sigma^*$ ein Wort der Länge n .
Eine Folge von Zuständen $z_0, \dots, z_n$ mit $z_0 \in S$ und $z_i \in \delta(z_{i-1}, w[i])$ für $i \in \{1, \dots, n\}$ ist ein **Lauf** von M für w .

Für einen Lauf schreiben wir auch

$$z_0 \xrightarrow{w[1]} z_1 \xrightarrow{w[2]} \dots \xrightarrow{w[n-1]} z_{n-1} \xrightarrow{w[n]} z_n$$

Während es bei DFAs genau einen Lauf pro Wort gibt, kann es bei NFAs mehrere geben (oder gar keine).

Beispiel für die Konstruktion eines NFA

Konstruiere einen NFA über $\{a, b\}$, der alle Wörter akzeptiert, die mit *abaa* beginnen und mit *bab* enden (z.B. *abaabab*, *abaaaabab*, *abaababab*):

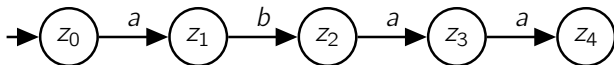
Beispiel für die Konstruktion eines NFA

Konstruiere einen NFA über $\{a, b\}$, der alle Wörter akzeptiert, die mit $abaa$ beginnen und mit bab enden (z.B. *abaabab*, *abaaaabab*, *abaababab*):



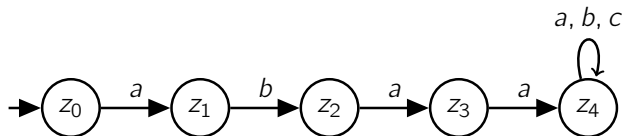
Beispiel für die Konstruktion eines NFA

Konstruiere einen NFA über $\{a, b\}$, der alle Wörter akzeptiert, die mit $abaa$ beginnen und mit bab enden (z.B. *abaabab*, *abaaaabab*, *abaababab*):



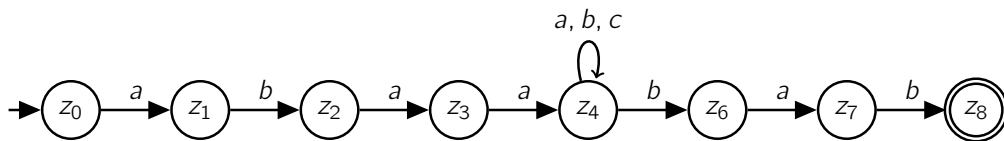
Beispiel für die Konstruktion eines NFA

Konstruiere einen NFA über $\{a, b\}$, der alle Wörter akzeptiert, die mit $abaa$ beginnen und mit bab enden (z.B. *abaabab*, *abaaaabab*, *abaababab*):



Beispiel für die Konstruktion eines NFA

Konstruiere einen NFA über $\{a, b\}$, der alle Wörter akzeptiert, die mit $abaa$ beginnen und mit bab enden (z.B. *abaabab*, *abaaaabab*, *abaababab*):

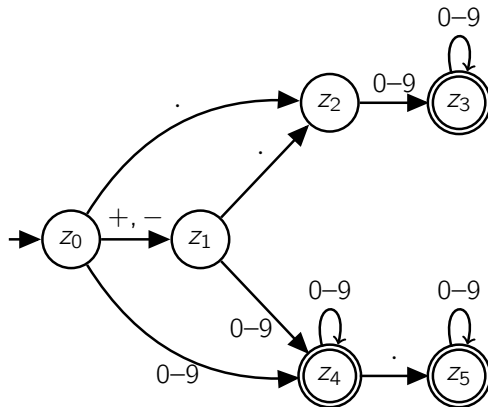


Weiteres Beispiel für die Konstruktion eines NFA

Konstruiere einen NFA über
 $\{+, -, ., 0, \dots, 9\}$,
der alle Wörter akzeptiert,
die Gleitkommazahlen
darstellen
(z.B. $+27$, -3.14 , $.666$):

Weiteres Beispiel für die Konstruktion eines NFA

Konstruiere einen NFA über
 $\{+, -, ., 0, \dots, 9\}$,
der alle Wörter akzeptiert,
die Gleitkommazahlen
darstellen
(z.B. $+27$, -3.14 , $.666$):



3b

Determinisierung von endlichen Automaten

Prof. Dr. Jasmin Blanchette

Lehr- und Forschungseinheit für
Theoretische Informatik und Theorembeweisen

Stand: 10. Mai 2024
Basiert auf Folien von PD Dr. David Sabel



Reguläre Sprachen werden von NFAs akzeptiert

Theorem

Für jede reguläre Sprache L gibt es einen NFA M mit $L(M) = L$.

Reguläre Sprachen werden von NFAs akzeptiert

Theorem

Für jede reguläre Sprache L gibt es einen NFA M mit $L(M) = L$.

Beweis Da L regulär ist, gibt es eine reguläre Grammatik $G = (V, \Sigma, P, S)$ (mit 1. Sonderregel), sodass $L(G) = L$.

Reguläre Sprachen werden von NFAs akzeptiert

Theorem

Für jede reguläre Sprache L gibt es einen NFA M mit $L(M) = L$.

Beweis Da L regulär ist, gibt es eine reguläre Grammatik $G = (V, \Sigma, P, S)$ (mit 1. Sonderregel), sodass $L(G) = L$.

Wir werden einen NFA M konstruieren mit $L(M) = L(G)$.

Reguläre Sprachen werden von NFAs akzeptiert

Theorem

Für jede reguläre Sprache L gibt es einen NFA M mit $L(M) = L$.

Beweis Da L regulär ist, gibt es eine reguläre Grammatik $G = (V, \Sigma, P, S)$ (mit 1. Sonderregel), sodass $L(G) = L$.

Wir werden einen NFA M konstruieren mit $L(M) = L(G)$.

Sei $z_E \notin V$. Sei $M = (Z, \Sigma, \delta, \{S\}, E)$ der NFA mit $Z := V \cup \{z_E\}$ und

Reguläre Sprachen werden von NFAs akzeptiert

Theorem

Für jede reguläre Sprache L gibt es einen NFA M mit $L(M) = L$.

Beweis Da L regulär ist, gibt es eine reguläre Grammatik $G = (V, \Sigma, P, S)$ (mit 1. Sonderregel), sodass $L(G) = L$.

Wir werden einen NFA M konstruieren mit $L(M) = L(G)$.

Sei $z_E \notin V$. Sei $M = (Z, \Sigma, \delta, \{S\}, E)$ der NFA mit $Z := V \cup \{z_E\}$ und

$$E := \{z_E\} \cup \begin{cases} \{S\} & \text{falls } S \rightarrow \varepsilon \in P \\ \emptyset & \text{sonst} \end{cases}$$

Reguläre Sprachen werden von NFAs akzeptiert

Theorem

Für jede reguläre Sprache L gibt es einen NFA M mit $L(M) = L$.

Beweis Da L regulär ist, gibt es eine reguläre Grammatik $G = (V, \Sigma, P, S)$ (mit 1. Sonderregel), sodass $L(G) = L$.

Wir werden einen NFA M konstruieren mit $L(M) = L(G)$.

Sei $z_E \notin V$. Sei $M = (Z, \Sigma, \delta, \{S\}, E)$ der NFA mit $Z := V \cup \{z_E\}$ und

$$E := \{z_E\} \cup \begin{cases} \{S\} & \text{falls } S \rightarrow \varepsilon \in P \\ \emptyset & \text{sonst} \end{cases}$$

$$\delta(A, a) := \{B \mid A \rightarrow aB \in P\}$$

Reguläre Sprachen werden von NFAs akzeptiert

Theorem

Für jede reguläre Sprache L gibt es einen NFA M mit $L(M) = L$.

Beweis Da L regulär ist, gibt es eine reguläre Grammatik $G = (V, \Sigma, P, S)$ (mit 1. Sonderregel), sodass $L(G) = L$.

Wir werden einen NFA M konstruieren mit $L(M) = L(G)$.

Sei $z_E \notin V$. Sei $M = (Z, \Sigma, \delta, \{S\}, E)$ der NFA mit $Z := V \cup \{z_E\}$ und

$$E := \{z_E\} \cup \begin{cases} \{S\} & \text{falls } S \rightarrow \varepsilon \in P \\ \emptyset & \text{sonst} \end{cases}$$

$$\delta(A, a) := \{B \mid A \rightarrow aB \in P\} \\ \cup \{z_E \mid A \rightarrow a \in P\}$$

Reguläre Sprachen werden von NFAs akzeptiert

Theorem

Für jede reguläre Sprache L gibt es einen NFA M mit $L(M) = L$.

Beweis Da L regulär ist, gibt es eine reguläre Grammatik $G = (V, \Sigma, P, S)$ (mit 1. Sonderregel), sodass $L(G) = L$.

Wir werden einen NFA M konstruieren mit $L(M) = L(G)$.

Sei $z_E \notin V$. Sei $M = (Z, \Sigma, \delta, \{S\}, E)$ der NFA mit $Z := V \cup \{z_E\}$ und

$$E := \{z_E\} \cup \begin{cases} \{S\} & \text{falls } S \rightarrow \varepsilon \in P \\ \emptyset & \text{sonst} \end{cases}$$

$$\delta(A, a) := \{B \mid A \rightarrow aB \in P\} \\ \cup \{z_E \mid A \rightarrow a \in P\}$$

$$\delta(z_E, a) := \emptyset$$

Beispiel für die Konstruktion eines NFA aus einer Grammatik

Reguläre Grammatik $G = (V, \Sigma, P, S)$ mit $V = \{S, A, B, C\}$, $\Sigma = \{a, b, c\}$ und

$$\begin{aligned} P = \{ & S \rightarrow aA \mid bA \mid cA \mid aB, \\ & A \rightarrow aA \mid bA \mid cA \mid aB, \\ & B \rightarrow aC \mid bC \mid cC, \\ & C \rightarrow a \mid b \mid c \} \end{aligned}$$

Beispiel für die Konstruktion eines NFA aus einer Grammatik

Reguläre Grammatik $G = (V, \Sigma, P, S)$ mit $V = \{S, A, B, C\}$, $\Sigma = \{a, b, c\}$ und

$$\begin{aligned}P = \{ & S \rightarrow aA \mid bA \mid cA \mid aB, \\ & A \rightarrow aA \mid bA \mid cA \mid aB, \\ & B \rightarrow aC \mid bC \mid cC, \\ & C \rightarrow a \mid b \mid c\}\end{aligned}$$

Ableitung des Wortes *backa*:

$$S \Rightarrow bA \Rightarrow baB \Rightarrow bacC \Rightarrow backa \in L(G)$$

Beispiel für die Konstruktion eines NFA aus einer Grammatik

Reguläre Grammatik $G = (V, \Sigma, P, S)$ mit $V = \{S, A, B, C\}$, $\Sigma = \{a, b, c\}$ und

$$\begin{aligned}P = \{ & S \rightarrow aA \mid bA \mid cA \mid aB, \\ & A \rightarrow aA \mid bA \mid cA \mid aB, \\ & B \rightarrow aC \mid bC \mid cC, \\ & C \rightarrow a \mid b \mid c\}\end{aligned}$$

Ableitung des Wortes *backa*:

$$S \Rightarrow bA \Rightarrow baB \Rightarrow bacC \Rightarrow backa \in L(G)$$

NFA zu G : $M = (\{S, A, B, C, z_E\}, \Sigma, \delta, \{S\}, \{z_E\})$ mit

$$\begin{array}{lllll}\delta(S, a) = \{A, B\} & \delta(A, a) = \{A, B\} & \delta(B, a) = \{C\} & \delta(C, a) = \{z_E\} & \delta(z_E, a) = \emptyset \\ \delta(S, b) = \{A\} & \delta(A, b) = \{A\} & \delta(B, b) = \{C\} & \delta(C, b) = \{z_E\} & \delta(z_E, b) = \emptyset \\ \delta(S, c) = \{A\} & \delta(A, c) = \{A\} & \delta(B, c) = \{C\} & \delta(C, c) = \{z_E\} & \delta(z_E, c) = \emptyset\end{array}$$

Beispiel für die Konstruktion eines NFA aus einer Grammatik

Reguläre Grammatik $G = (V, \Sigma, P, S)$ mit $V = \{S, A, B, C\}$, $\Sigma = \{a, b, c\}$ und

$$\begin{aligned}P = \{ & \textcolor{red}{S} \rightarrow \textcolor{red}{a}\textcolor{blue}{A} \mid bA \mid cA \mid aB, \\ & A \rightarrow aA \mid bA \mid cA \mid aB, \\ & B \rightarrow aC \mid bC \mid cC, \\ & C \rightarrow a \mid b \mid c\}\end{aligned}$$

Ableitung des Wortes *backa*:

$$S \Rightarrow bA \Rightarrow baB \Rightarrow bacC \Rightarrow \textcolor{red}{baca} \in \textcolor{red}{L(G)}$$

NFA zu G : $M = (\{S, A, B, C, z_E\}, \Sigma, \delta, \{S\}, \{z_E\})$ mit

$$\begin{array}{lllll} \delta(\textcolor{red}{S}, \textcolor{red}{a}) = \{\textcolor{blue}{A}, B\} & \delta(A, a) = \{A, B\} & \delta(B, a) = \{C\} & \delta(C, a) = \{z_E\} & \delta(z_E, a) = \emptyset \\ \delta(S, b) = \{A\} & \delta(A, b) = \{A\} & \delta(B, b) = \{C\} & \delta(C, b) = \{z_E\} & \delta(z_E, b) = \emptyset \\ \delta(S, c) = \{A\} & \delta(A, c) = \{A\} & \delta(B, c) = \{C\} & \delta(C, c) = \{z_E\} & \delta(z_E, c) = \emptyset \end{array}$$

Beispiel für die Konstruktion eines NFA aus einer Grammatik

Reguläre Grammatik $G = (V, \Sigma, P, S)$ mit $V = \{S, A, B, C\}$, $\Sigma = \{a, b, c\}$ und

$$\begin{aligned}P = \{ & \textcolor{red}{S} \rightarrow aA \mid bA \mid cA \mid \textcolor{red}{a}\textcolor{blue}{B}, \\ & A \rightarrow aA \mid bA \mid cA \mid aB, \\ & B \rightarrow aC \mid bC \mid cC, \\ & C \rightarrow a \mid b \mid c\}\end{aligned}$$

Ableitung des Wortes *backa*:

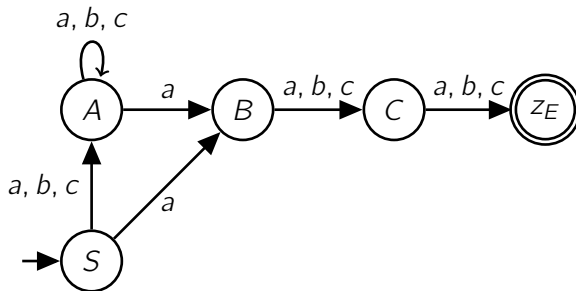
$$S \Rightarrow bA \Rightarrow baB \Rightarrow bacC \Rightarrow \textcolor{red}{baca} \in \textcolor{red}{L(G)}$$

NFA zu G : $M = (\{S, A, B, C, z_E\}, \Sigma, \delta, \{S\}, \{z_E\})$ mit

$$\begin{array}{lllll}\delta(\textcolor{red}{S}, \textcolor{red}{a}) = \{A, \textcolor{blue}{B}\} & \delta(A, a) = \{A, B\} & \delta(B, a) = \{C\} & \delta(C, a) = \{z_E\} & \delta(z_E, a) = \emptyset \\ \delta(S, b) = \{A\} & \delta(A, b) = \{A\} & \delta(B, b) = \{C\} & \delta(C, b) = \{z_E\} & \delta(z_E, b) = \emptyset \\ \delta(S, c) = \{A\} & \delta(A, c) = \{A\} & \delta(B, c) = \{C\} & \delta(C, c) = \{z_E\} & \delta(z_E, c) = \emptyset\end{array}$$

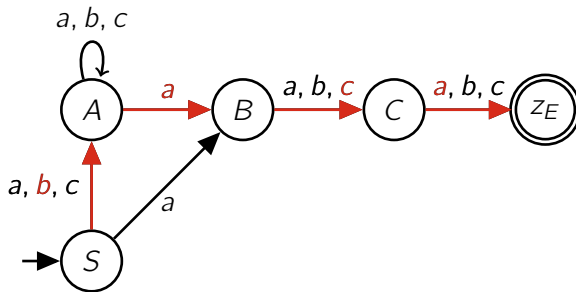
Beispiel für die Konstruktion eines NFA aus einer Grammatik

Zustandsgraph zu M :



Beispiel für die Konstruktion eines NFA aus einer Grammatik

Zustandsgraph zu M :



Eingabe: $baaca \in L(M)$

Reguläre Sprachen werden von NFAs akzeptiert

Theorem

Für jede reguläre Sprache L gibt es einen NFA M mit $L(M) = L$.

Beweis (Fortsetzung) Wir zeigen, dass $L(M) = L(G)$,
d.h. $w \in L(M)$ g.d.w. $w \in L(G)$ für ein beliebiges w .

Reguläre Sprachen werden von NFAs akzeptiert

Theorem

Für jede reguläre Sprache L gibt es einen NFA M mit $L(M) = L$.

Beweis (Fortsetzung) Wir zeigen, dass $L(M) = L(G)$,

d.h. $w \in L(M)$ g.d.w. $w \in L(G)$ für ein beliebiges w .

► Fall $w = \varepsilon$: $\varepsilon \in L(M)$ g.d.w. $S \in E$ g.d.w. $S \rightarrow \varepsilon \in P$ g.d.w. $\varepsilon \in L(G)$.

Reguläre Sprachen werden von NFAs akzeptiert

Theorem

Für jede reguläre Sprache L gibt es einen NFA M mit $L(M) = L$.

Beweis (Fortsetzung) Wir zeigen, dass $L(M) = L(G)$,

d.h. $w \in L(M)$ g.d.w. $w \in L(G)$ für ein beliebiges w .

- ▶ Fall $w = \varepsilon$: $\varepsilon \in L(M)$ g.d.w. $S \in E$ g.d.w. $S \rightarrow \varepsilon \in P$ g.d.w. $\varepsilon \in L(G)$.
- ▶ Fall $w = a_1 \cdots a_n$ mit $n > 0$:

Reguläre Sprachen werden von NFAs akzeptiert

Theorem

Für jede reguläre Sprache L gibt es einen NFA M mit $L(M) = L$.

Beweis (Fortsetzung) Wir zeigen, dass $L(M) = L(G)$,

d.h. $w \in L(M)$ g.d.w. $w \in L(G)$ für ein beliebiges w .

► Fall $w = \varepsilon$: $\varepsilon \in L(M)$ g.d.w. $S \in E$ g.d.w. $S \rightarrow \varepsilon \in P$ g.d.w. $\varepsilon \in L(G)$.

► Fall $w = a_1 \cdots a_n$ mit $n > 0$:

$$a_1 \cdots a_n \in L(M)$$

Reguläre Sprachen werden von NFAs akzeptiert

Theorem

Für jede reguläre Sprache L gibt es einen NFA M mit $L(M) = L$.

Beweis (Fortsetzung) Wir zeigen, dass $L(M) = L(G)$,

d.h. $w \in L(M)$ g.d.w. $w \in L(G)$ für ein beliebiges w .

► Fall $w = \varepsilon$: $\varepsilon \in L(M)$ g.d.w. $S \in E$ g.d.w. $S \rightarrow \varepsilon \in P$ g.d.w. $\varepsilon \in L(G)$.

► Fall $w = a_1 \cdots a_n$ mit $n > 0$:

$$a_1 \cdots a_n \in L(M)$$

g.d.w. es gibt einen Lauf $S \xrightarrow{a_1} A_1 \xrightarrow{a_2} \cdots \xrightarrow{a_{n-1}} A_{n-1} \xrightarrow{a_n} z_E$

Reguläre Sprachen werden von NFAs akzeptiert

Theorem

Für jede reguläre Sprache L gibt es einen NFA M mit $L(M) = L$.

Beweis (Fortsetzung) Wir zeigen, dass $L(M) = L(G)$,

d.h. $w \in L(M)$ g.d.w. $w \in L(G)$ für ein beliebiges w .

► Fall $w = \varepsilon$: $\varepsilon \in L(M)$ g.d.w. $S \in E$ g.d.w. $S \rightarrow \varepsilon \in P$ g.d.w. $\varepsilon \in L(G)$.

► Fall $w = a_1 \cdots a_n$ mit $n > 0$:

$$a_1 \cdots a_n \in L(M)$$

g.d.w. es gibt einen Lauf $S \xrightarrow{a_1} A_1 \xrightarrow{a_2} \cdots \xrightarrow{a_{n-1}} A_{n-1} \xrightarrow{a_n} z_E$

g.d.w. es gibt Zustände $A_1, \dots, A_{n-1}$ mit $A_1 \in \delta(S, a_1)$, $A_i \in \delta(A_{i-1}, a_i)$
für $i \in \{2, \dots, n-1\}$ und $z_E \in \delta(A_{n-1}, a_n)$

Reguläre Sprachen werden von NFAs akzeptiert

Theorem

Für jede reguläre Sprache L gibt es einen NFA M mit $L(M) = L$.

Beweis (Fortsetzung) Wir zeigen, dass $L(M) = L(G)$,

d.h. $w \in L(M)$ g.d.w. $w \in L(G)$ für ein beliebiges w .

► Fall $w = \varepsilon$: $\varepsilon \in L(M)$ g.d.w. $S \in E$ g.d.w. $S \rightarrow \varepsilon \in P$ g.d.w. $\varepsilon \in L(G)$.

► Fall $w = a_1 \cdots a_n$ mit $n > 0$:

$$a_1 \cdots a_n \in L(M)$$

g.d.w. es gibt einen Lauf $S \xrightarrow{a_1} A_1 \xrightarrow{a_2} \cdots \xrightarrow{a_{n-1}} A_{n-1} \xrightarrow{a_n} z_E$

g.d.w. es gibt Zustände $A_1, \dots, A_{n-1}$ mit $A_1 \in \delta(S, a_1)$, $A_i \in \delta(A_{i-1}, a_i)$
für $i \in \{2, \dots, n-1\}$ und $z_E \in \delta(A_{n-1}, a_n)$

$$a_1 \cdots a_n \in L(G)$$

Reguläre Sprachen werden von NFAs akzeptiert

Theorem

Für jede reguläre Sprache L gibt es einen NFA M mit $L(M) = L$.

Beweis (Fortsetzung) Wir zeigen, dass $L(M) = L(G)$,

d.h. $w \in L(M)$ g.d.w. $w \in L(G)$ für ein beliebiges w .

► Fall $w = \varepsilon$: $\varepsilon \in L(M)$ g.d.w. $S \in E$ g.d.w. $S \rightarrow \varepsilon \in P$ g.d.w. $\varepsilon \in L(G)$.

► Fall $w = a_1 \cdots a_n$ mit $n > 0$:

$$a_1 \cdots a_n \in L(M)$$

g.d.w. es gibt einen Lauf $S \xrightarrow{a_1} A_1 \xrightarrow{a_2} \cdots \xrightarrow{a_{n-1}} A_{n-1} \xrightarrow{a_n} z_E$

g.d.w. es gibt Zustände $A_1, \dots, A_{n-1}$ mit $A_1 \in \delta(S, a_1)$, $A_i \in \delta(A_{i-1}, a_i)$
für $i \in \{2, \dots, n-1\}$ und $z_E \in \delta(A_{n-1}, a_n)$

$$S \Rightarrow_G a_1 A_1 \Rightarrow_G \cdots \Rightarrow_G a_1 \cdots a_{n-1} A_{n-1} \Rightarrow_G a_1 \cdots a_n$$

g.d.w. $a_1 \cdots a_n \in L(G)$

Reguläre Sprachen werden von NFAs akzeptiert

Theorem

Für jede reguläre Sprache L gibt es einen NFA M mit $L(M) = L$.

Beweis (Fortsetzung) Wir zeigen, dass $L(M) = L(G)$,

d.h. $w \in L(M)$ g.d.w. $w \in L(G)$ für ein beliebiges w .

► Fall $w = \varepsilon$: $\varepsilon \in L(M)$ g.d.w. $S \in E$ g.d.w. $S \rightarrow \varepsilon \in P$ g.d.w. $\varepsilon \in L(G)$.

► Fall $w = a_1 \cdots a_n$ mit $n > 0$:

$a_1 \cdots a_n \in L(M)$

g.d.w. es gibt einen Lauf $S \xrightarrow{a_1} A_1 \xrightarrow{a_2} \cdots \xrightarrow{a_{n-1}} A_{n-1} \xrightarrow{a_n} z_E$

g.d.w. es gibt Zustände $A_1, \dots, A_{n-1}$ mit $A_1 \in \delta(S, a_1)$, $A_i \in \delta(A_{i-1}, a_i)$
für $i \in \{2, \dots, n-1\}$ und $z_E \in \delta(A_{n-1}, a_n)$

g.d.w. $S \Rightarrow_G a_1 A_1 \Rightarrow_G \cdots \Rightarrow_G a_1 \cdots a_{n-1} A_{n-1} \Rightarrow_G a_1 \cdots a_n$

g.d.w. $a_1 \cdots a_n \in L(G)$

Reguläre Sprachen werden von NFAs akzeptiert

Theorem

Für jede reguläre Sprache L gibt es einen NFA M mit $L(M) = L$.

Beweis (Fortsetzung) Wir zeigen, dass $L(M) = L(G)$,

d.h. $w \in L(M)$ g.d.w. $w \in L(G)$ für ein beliebiges w .

► Fall $w = \varepsilon$: $\varepsilon \in L(M)$ g.d.w. $S \in E$ g.d.w. $S \rightarrow \varepsilon \in P$ g.d.w. $\varepsilon \in L(G)$.

► Fall $w = a_1 \cdots a_n$ mit $n > 0$:

$$a_1 \cdots a_n \in L(M)$$

g.d.w. es gibt einen Lauf $S \xrightarrow{a_1} A_1 \xrightarrow{a_2} \cdots \xrightarrow{a_{n-1}} A_{n-1} \xrightarrow{a_n} z_E$

g.d.w. es gibt Zustände $A_1, \dots, A_{n-1}$ mit $A_1 \in \delta(S, a_1)$, $A_i \in \delta(A_{i-1}, a_i)$
für $i \in \{2, \dots, n-1\}$ und $z_E \in \delta(A_{n-1}, a_n)$

g.d.w. $S \Rightarrow_G a_1 A_1 \Rightarrow_G \cdots \Rightarrow_G a_1 \cdots a_{n-1} A_{n-1} \Rightarrow_G a_1 \cdots a_n$

g.d.w. $a_1 \cdots a_n \in L(G)$

Daher gibt es einen NFA M mit $L(M) = L(G) = L$. □

Theorem (Rabin und Scott 1959)

Jede von einem NFA akzeptierte Sprache ist auch durch einen DFA akzeptierbar.

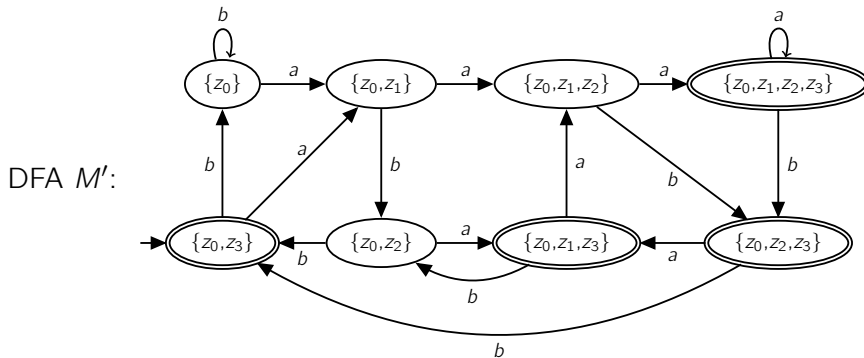
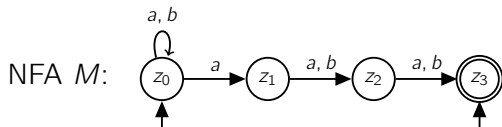
Theorem (Rabin und Scott 1959)

Jede von einem NFA akzeptierte Sprache ist auch durch einen DFA akzeptierbar.

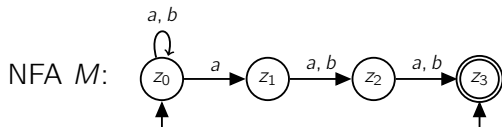
Der Beweis basiert auf der [Potenzmengenkonstruktion](#):

- ▶ Konstruiere für einen gegebenen NFA M einen DFA M' , sodass sich der DFA **alle** Zustände merkt, in denen der NFA sein könnte.
- ▶ Jede Teilmenge von Zuständen des NFAs wird zu einem Zustand des DFA.
- ▶ Da Z endlich ist, gibt es nur endlich viele Teilmengen.

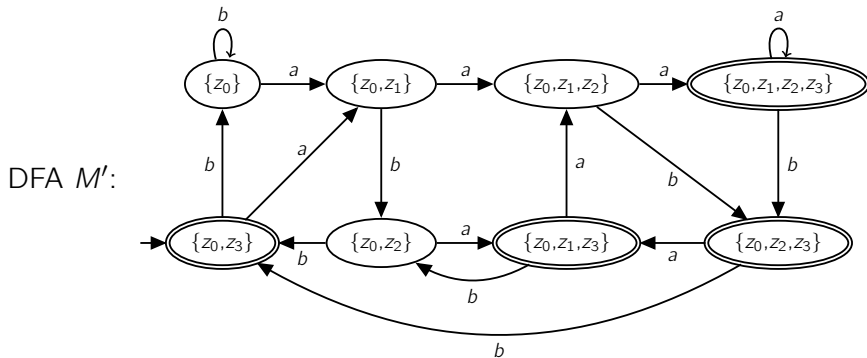
Beispiel für die Potenzmengenkonstruktion



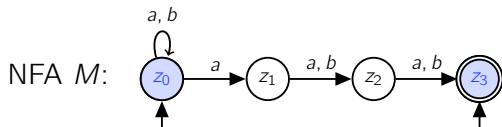
Beispiel für die Potenzmengenkonstruktion



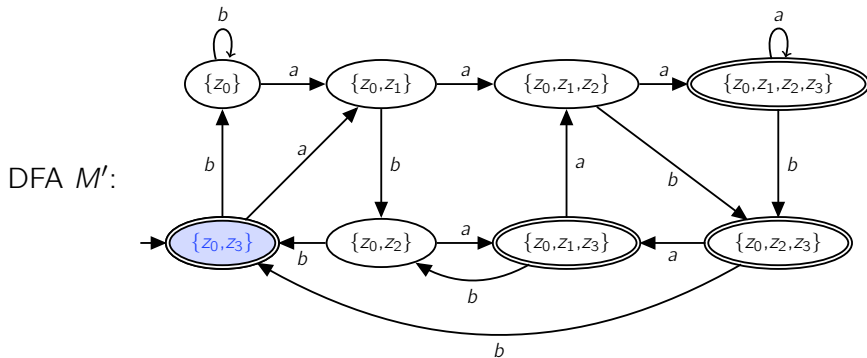
Eingabe: *babbabb*



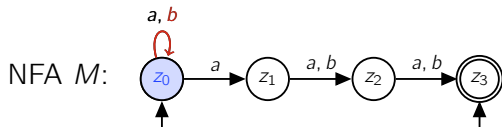
Beispiel für die Potenzmengenkonstruktion



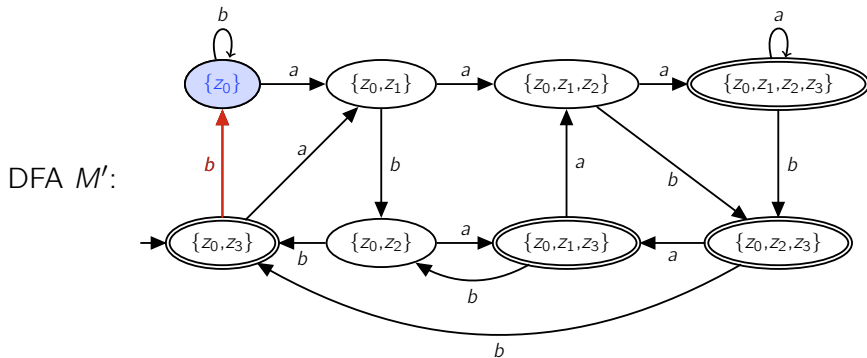
Eingabe: *babbabb*



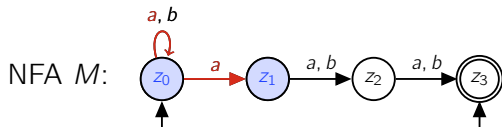
Beispiel für die Potenzmengenkonstruktion



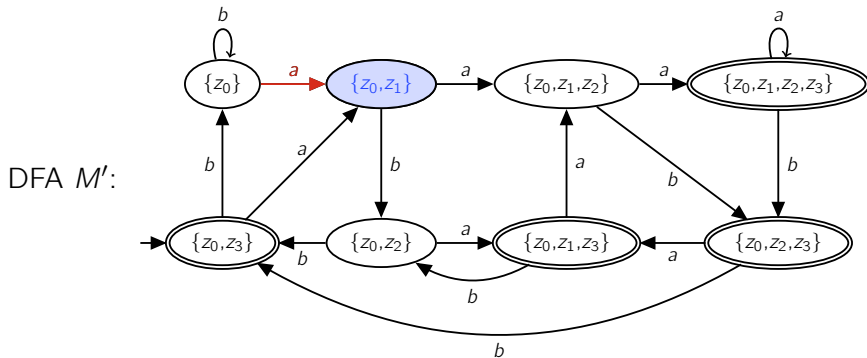
Eingabe: *babbabb*



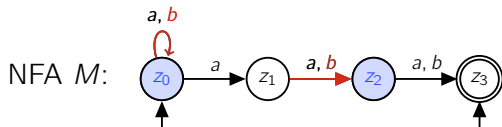
Beispiel für die Potenzmengenkonstruktion



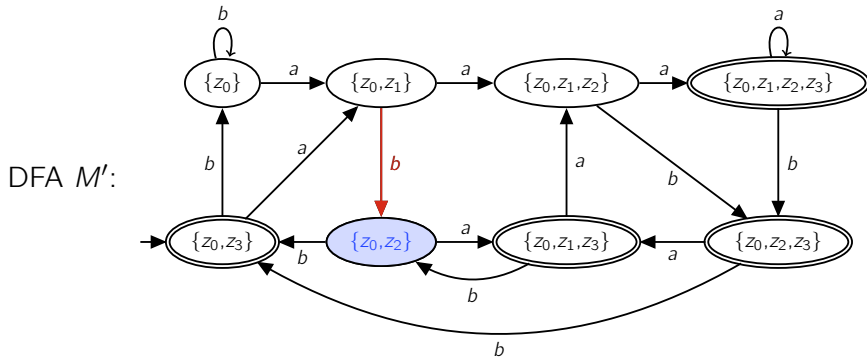
Eingabe: $bababb$



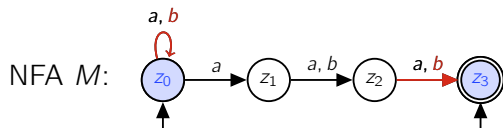
Beispiel für die Potenzmengenkonstruktion



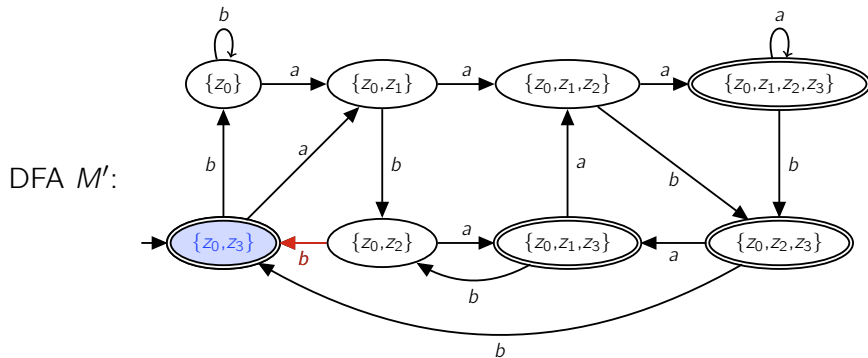
Eingabe: $bababb$



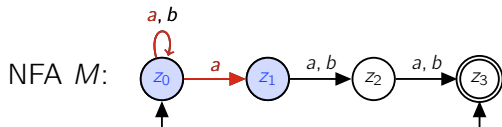
Beispiel für die Potenzmengenkonstruktion



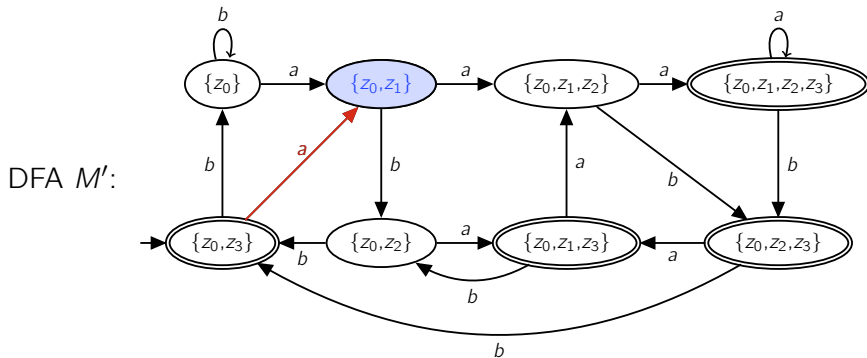
Eingabe: *bab****b****abb*



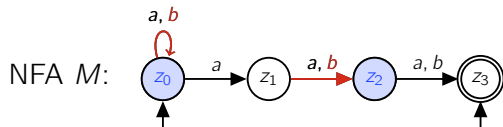
Beispiel für die Potenzmengenkonstruktion



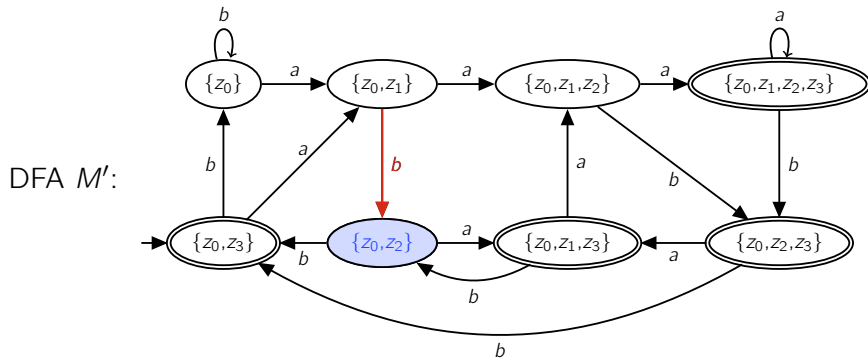
Eingabe: *babbabb*



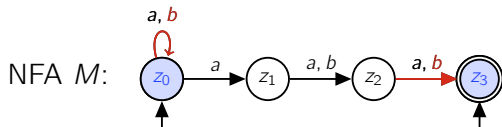
Beispiel für die Potenzmengenkonstruktion



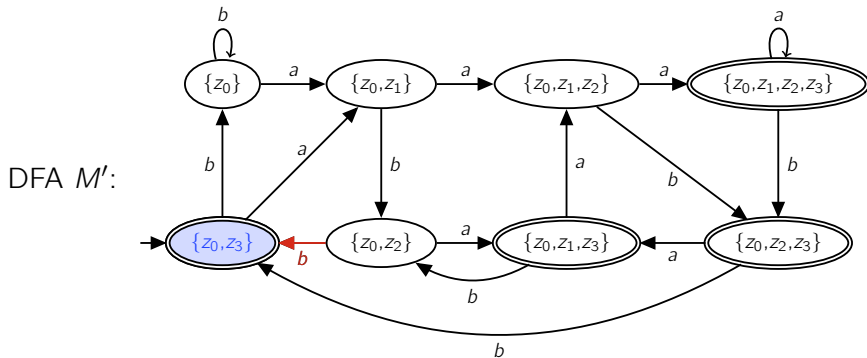
Eingabe: *babba****b***



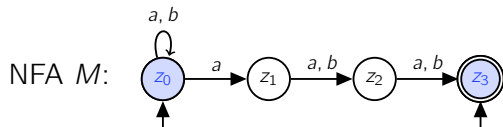
Beispiel für die Potenzmengenkonstruktion



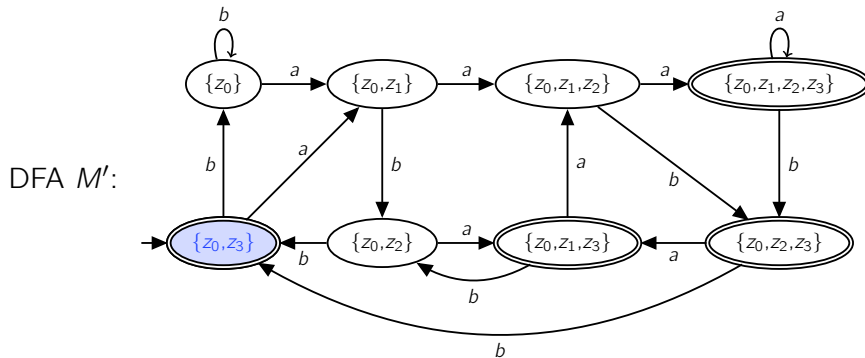
Eingabe: *babbab****b***



Beispiel für die Potenzmengenkonstruktion



Eingabe: $babbabb \in L(M)$
 $\in L(M')$



Potenzmengenkonstruktion

Für den NFA $M = (Z, \Sigma, \delta, S, E)$ definieren wir den DFA $M' = (Z', \Sigma, \delta', S', E')$ mit

- ▶ $Z' := \mathcal{P}(Z)$
- ▶ $S' := S$
- ▶ $E' := \{X \in Z' \mid E \cap X \neq \emptyset\}$
- ▶ $\delta'(X, a) := (\bigcup_{z \in X} \delta(z, a)) = \widetilde{\delta}(X, a)$

Potenzmengenkonstruktion

Für den NFA $M = (Z, \Sigma, \delta, S, E)$ definieren wir den DFA $M' = (Z', \Sigma, \delta', S', E')$ mit

► $Z' := \mathcal{P}(Z)$

Die Zustandsmenge ist die Potenzmenge von Z .

► $S' := S$

Der Startzustand ist die Menge S aller Startzustände von M .

► $E' := \{X \in Z' \mid E \cap X \neq \emptyset\}$

Jede Menge, die mindestens einen Endzustand von M enthält, ist Endzustand.

► $\delta'(X, a) := (\bigcup_{z \in X} \delta(z, a)) = \tilde{\delta}(X, a)$

Die Übergangsfunktion berechnet alle von einem Zustand in X aus über a erreichbaren Zustände.

Theorem (Rabin und Scott 1959)

Jede von einem NFA akzeptierte Sprache ist auch durch einen DFA akzeptierbar.

Theorem (Rabin und Scott 1959)

Jede von einem NFA akzeptierte Sprache ist auch durch einen DFA akzeptierbar.

Beweis Wir zeigen, dass $L(M') = L(M)$,
d.h. $w \in L(M)$ g.d.w. $w \in L(M')$ für ein beliebiges w .

Theorem (Rabin und Scott 1959)

Jede von einem NFA akzeptierte Sprache ist auch durch einen DFA akzeptierbar.

Beweis Wir zeigen, dass $L(M') = L(M)$,
d.h. $w \in L(M)$ g.d.w. $w \in L(M')$ für ein beliebiges w .

► Fall $w = \varepsilon$: $\varepsilon \in L(M)$ g.d.w. $S \cap E \neq \emptyset$ g.d.w. $S \in E'$ g.d.w. $\varepsilon \in L(M')$.

Beweis (Fortsetzung)

► Fall $w = a_1 \cdots a_n$ mit $n > 0$:

Zu zeigen: $a_1 \cdots a_n \in L(M)$ g.d.w. $a_1 \cdots a_n \in L(M')$.

NFAs in DFAs transformieren

Beweis (Fortsetzung)

► Fall $w = a_1 \cdots a_n$ mit $n > 0$:

Zu zeigen: $a_1 \cdots a_n \in L(M)$ g.d.w. $a_1 \cdots a_n \in L(M')$.

$$a_1 \cdots a_n \in L(M)$$

NFAs in DFAs transformieren

Beweis (Fortsetzung)

- Fall $w = a_1 \cdots a_n$ mit $n > 0$:

Zu zeigen: $a_1 \cdots a_n \in L(M)$ g.d.w. $a_1 \cdots a_n \in L(M')$.

$$a_1 \cdots a_n \in L(M)$$

g.d.w. $\tilde{\delta}(S, a_1 \cdots a_n) \cap E \neq \emptyset$

NFAs in DFAs transformieren

Beweis (Fortsetzung)

- Fall $w = a_1 \cdots a_n$ mit $n > 0$:

Zu zeigen: $a_1 \cdots a_n \in L(M)$ g.d.w. $a_1 \cdots a_n \in L(M')$.

$$a_1 \cdots a_n \in L(M)$$

g.d.w. $\tilde{\delta}(S, a_1 \cdots a_n) \cap E \neq \emptyset$

g.d.w. es gibt $Z_1, \dots, Z_n \subseteq Z$ mit $(\bigcup_{z \in S} \delta(z, a_1)) = Z_1$,
 $(\bigcup_{z \in Z_{i-1}} \delta(z, a_i)) = Z_i$ für $i \in \{2, \dots, n\}$ und $Z_n \cap E \neq \emptyset$

NFAs in DFAs transformieren

Beweis (Fortsetzung)

- Fall $w = a_1 \cdots a_n$ mit $n > 0$:

Zu zeigen: $a_1 \cdots a_n \in L(M)$ g.d.w. $a_1 \cdots a_n \in L(M')$.

$$a_1 \cdots a_n \in L(M)$$

g.d.w. $\tilde{\delta}(S, a_1 \cdots a_n) \cap E \neq \emptyset$

g.d.w. es gibt $Z_1, \dots, Z_n \subseteq Z$ mit $(\bigcup_{z \in S} \delta(z, a_1)) = Z_1$,
 $(\bigcup_{z \in Z_{i-1}} \delta(z, a_i)) = Z_i$ für $i \in \{2, \dots, n\}$ und $Z_n \cap E \neq \emptyset$

$$a_1 \cdots a_n \in L(M')$$

NFAs in DFAs transformieren

Beweis (Fortsetzung)

- Fall $w = a_1 \cdots a_n$ mit $n > 0$:

Zu zeigen: $a_1 \cdots a_n \in L(M)$ g.d.w. $a_1 \cdots a_n \in L(M')$.

$$a_1 \cdots a_n \in L(M)$$

g.d.w. $\tilde{\delta}(S, a_1 \cdots a_n) \cap E \neq \emptyset$

g.d.w. es gibt $Z_1, \dots, Z_n \subseteq Z$ mit $(\bigcup_{z \in S} \delta(z, a_1)) = Z_1$,
 $(\bigcup_{z \in Z_{i-1}} \delta(z, a_i)) = Z_i$ für $i \in \{2, \dots, n\}$ und $Z_n \cap E \neq \emptyset$

$$\tilde{\delta}'(S', a_1 \cdots a_n) \in E'$$

g.d.w. $a_1 \cdots a_n \in L(M')$

NFAs in DFAs transformieren

Beweis (Fortsetzung)

- Fall $w = a_1 \cdots a_n$ mit $n > 0$:

Zu zeigen: $a_1 \cdots a_n \in L(M)$ g.d.w. $a_1 \cdots a_n \in L(M')$.

$$a_1 \cdots a_n \in L(M)$$

g.d.w. $\tilde{\delta}(S, a_1 \cdots a_n) \cap E \neq \emptyset$

g.d.w. es gibt $Z_1, \dots, Z_n \subseteq Z$ mit $(\bigcup_{z \in S} \delta(z, a_1)) = Z_1$,
 $(\bigcup_{z \in Z_{i-1}} \delta(z, a_i)) = Z_i$ für $i \in \{2, \dots, n\}$ und $Z_n \cap E \neq \emptyset$

es gibt $Z_1, \dots, Z_n \subseteq Z$ mit $\delta'(S, a_1) = Z_1$,

$\delta'(Z_{i-1}, a_i) = Z_i$ für $i \in \{2, \dots, n\}$ und $Z_n \cap E \neq \emptyset$

g.d.w. $\tilde{\delta}'(S', a_1 \cdots a_n) \in E'$

g.d.w. $a_1 \cdots a_n \in L(M')$

NFAs in DFAs transformieren

Beweis (Fortsetzung)

► Fall $w = a_1 \cdots a_n$ mit $n > 0$:

Zu zeigen: $a_1 \cdots a_n \in L(M)$ g.d.w. $a_1 \cdots a_n \in L(M')$.

$$a_1 \cdots a_n \in L(M)$$

g.d.w. $\tilde{\delta}(S, a_1 \cdots a_n) \cap E \neq \emptyset$

g.d.w. es gibt $Z_1, \dots, Z_n \subseteq Z$ mit $(\bigcup_{z \in S} \delta(z, a_1)) = Z_1$,
 $(\bigcup_{z \in Z_{i-1}} \delta(z, a_i)) = Z_i$ für $i \in \{2, \dots, n\}$ und $Z_n \cap E \neq \emptyset$

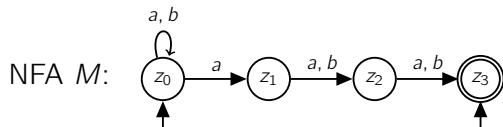
g.d.w. es gibt $Z_1, \dots, Z_n \subseteq Z$ mit $\delta'(S, a_1) = Z_1$,
 $\delta'(Z_{i-1}, a_i) = Z_i$ für $i \in \{2, \dots, n\}$ und $Z_n \cap E \neq \emptyset$

g.d.w. $\tilde{\delta}'(S', a_1 \cdots a_n) \in E'$

g.d.w. $a_1 \cdots a_n \in L(M')$

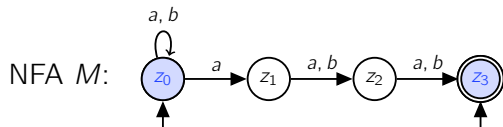


Beispiel für die Ausführung der Potenzmengenkonstruktion

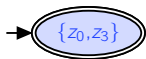


DFA M' :

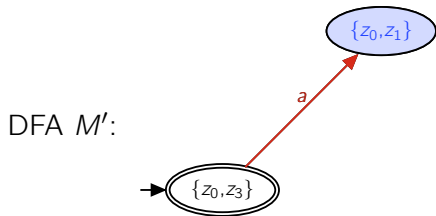
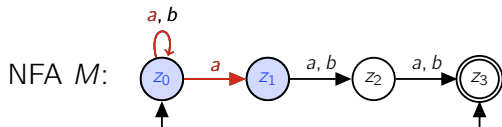
Beispiel für die Ausführung der Potenzmengenkonstruktion



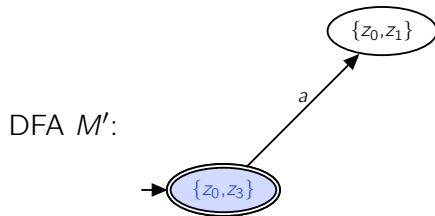
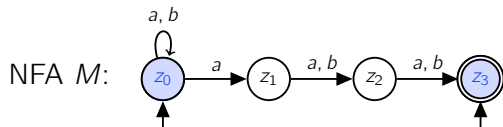
DFA M' :



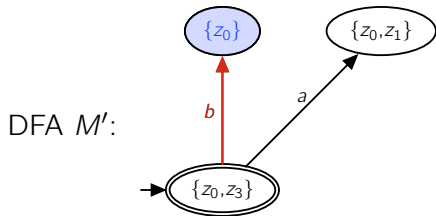
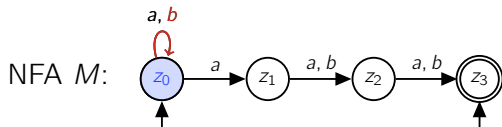
Beispiel für die Ausführung der Potenzmengenkonstruktion



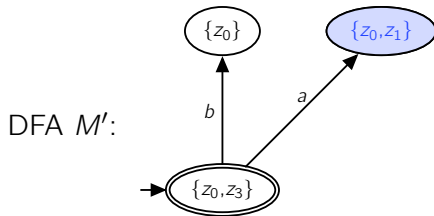
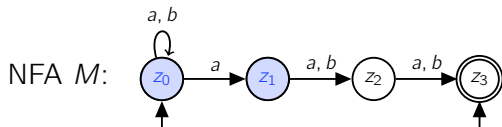
Beispiel für die Ausführung der Potenzmengenkonstruktion



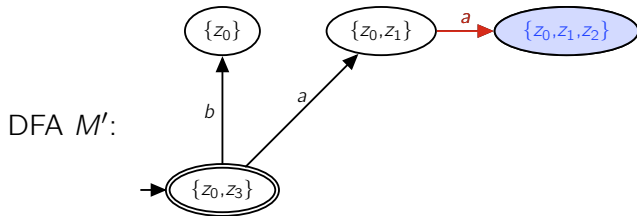
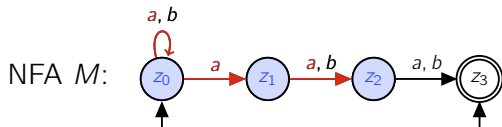
Beispiel für die Ausführung der Potenzmengenkonstruktion



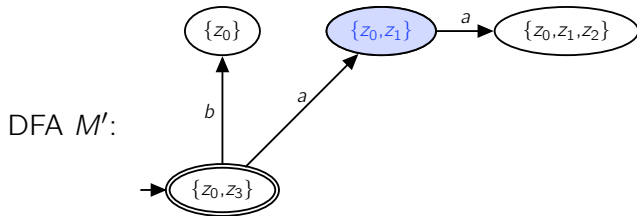
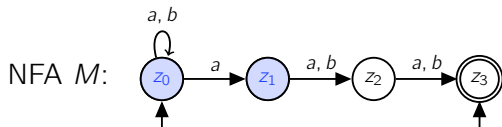
Beispiel für die Ausführung der Potenzmengenkonstruktion



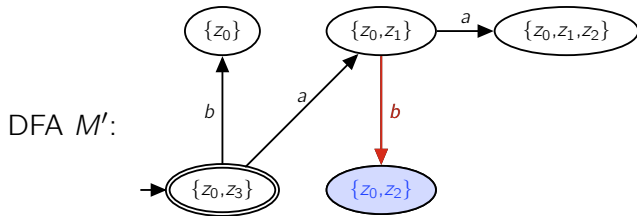
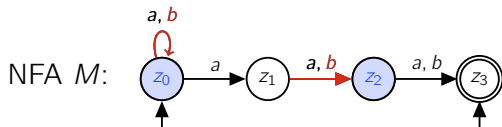
Beispiel für die Ausführung der Potenzmengenkonstruktion



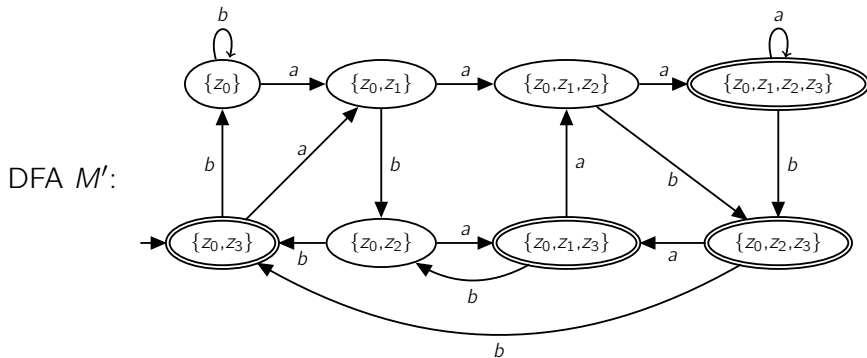
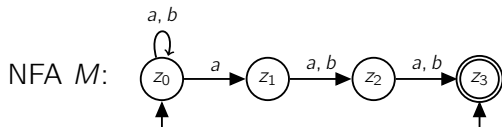
Beispiel für die Ausführung der Potenzmengenkonstruktion



Beispiel für die Ausführung der Potenzmengenkonstruktion



Beispiel für die Ausführung der Potenzmengenkonstruktion



Äquivalenz von regulären Sprachen und von DFAs bzw. NFAs

Theorem

DFAs und NFAs akzeptieren genau die regulären Sprachen.

Äquivalenz von regulären Sprachen und von DFAs bzw. NFAs

Theorem

DFAs und NFAs akzeptieren genau die regulären Sprachen.

Beweis

⊇ Für jede reguläre Sprache gibt es einen DFA bzw. NFA, der sie erkennt.

Das folgt aus:

- ▶ Für jede reguläre Sprache L gibt es einen NFA M mit $L(M) = L$.
- ▶ Jede von einem NFA akzeptierte Sprache ist auch durch einen DFA akzeptierbar.

Äquivalenz von regulären Sprachen und von DFAs bzw. NFAs

Theorem

DFAs und NFAs akzeptieren genau die regulären Sprachen.

Beweis

⊇ Für jede reguläre Sprache gibt es einen DFA bzw. NFA, der sie erkennt.

Das folgt aus:

- ▶ Für jede reguläre Sprache L gibt es einen NFA M mit $L(M) = L$.
- ▶ Jede von einem NFA akzeptierte Sprache ist auch durch einen DFA akzeptierbar.

⊆ Die akzeptierte Sprache eines DFA bzw. NFA ist regulär.

Das folgt aus:

- ▶ Jede von einem NFA akzeptierte Sprache ist auch durch einen DFA akzeptierbar.
- ▶ Sei M ein DFA. Dann ist $L(M)$ regulär. □

Größe des DFA vs. NFA

- ▶ Sei ein NFA mit n Zuständen.
- ▶ Der durch die Potenzmengenkonstruktion erstellte DFA hat 2^n Zustände.
Der Platz explodiert uns.
- ▶ Geht es besser?

Größe des DFA vs. NFA

- ▶ Sei ein NFA mit n Zuständen.
- ▶ Der durch die Potenzmengenkonstruktion erstellte DFA hat 2^n Zustände.
Der Platz explodiert uns.
- ▶ Geht es besser? Nicht wirklich.

Lemma

Sei $L = \{uav \mid u \in \Sigma^*, v \in \Sigma^{n-1}\}$ für $\Sigma = \{a, b\}$ und $n \in \mathbb{N}_{>0}$
(d.h. die Sprache aller Wörter aus Σ^* , die an n -letzter Stelle ein a haben).

1. Es gibt einen **NFA** M mit $L(M) = L$ und M hat $n + 1$ Zustände.
2. Jeder **DFA** M' mit $L(M') = L$ hat mindestens 2^n Zustände.

Größe des DFA vs. NFA

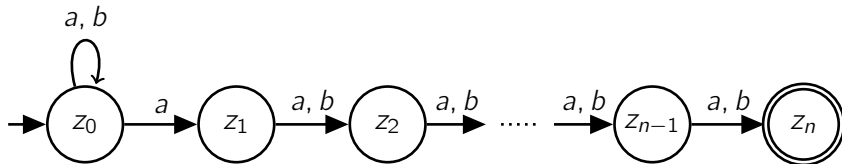
Lemma

Sei $L = \{uav \mid u \in \Sigma^*, v \in \Sigma^{n-1}\}$ für $\Sigma = \{a, b\}$ und $n \in \mathbb{N}_{>0}$
(d.h. die Sprache aller Wörter aus Σ^* , die an n -letzter Stelle ein a haben).

1. Es gibt einen NFA M mit $L(M) = L$ und M hat $n + 1$ Zustände.
2. Jeder DFA M' mit $L(M') = L$ hat mindestens 2^n Zustände.

Beweis

1. Sei M der folgende NFA:



Beweis (Fortsetzung)

2. Durch Widerspruch.

Beweis (Fortsetzung)

2. Durch Widerspruch.

Wir nehmen an, es gibt einen DFA $M' = (Z, \Sigma, \delta, z_0, E)$ mit $L(M') = L = \{uav \mid u \in \Sigma^*, v \in \Sigma^{n-1}\}$ und $|Z| < 2^n$.

Beweis (Fortsetzung)

2. Durch Widerspruch.

Wir nehmen an, es gibt einen DFA $M' = (Z, \Sigma, \delta, z_0, E)$ mit $L(M') = L = \{uav \mid u \in \Sigma^*, v \in \Sigma^{n-1}\}$ und $|Z| < 2^n$.

Die Menge Σ^n enthält 2^n Wörter der Länge n und da $|Z| < 2^n$, muss es $w, w' \in \Sigma^n$ geben mit $w \neq w'$ und $\tilde{\delta}(z_0, w) = \tilde{\delta}(z_0, w') = z$.

Beweis (Fortsetzung)

2. Durch Widerspruch.

Wir nehmen an, es gibt einen DFA $M' = (Z, \Sigma, \delta, z_0, E)$ mit $L(M') = L = \{uav \mid u \in \Sigma^*, v \in \Sigma^{n-1}\}$ und $|Z| < 2^n$.

Die Menge Σ^n enthält 2^n Wörter der Länge n und da $|Z| < 2^n$, muss es $w, w' \in \Sigma^n$ geben mit $w \neq w'$ und $\tilde{\delta}(z_0, w) = \tilde{\delta}(z_0, w') = z$.

Sei $j \in \{1, \dots, n\}$ die erste Position, an der sich w und w' unterscheiden.

Beweis (Fortsetzung)

2. Durch Widerspruch.

Wir nehmen an, es gibt einen DFA $M' = (Z, \Sigma, \delta, z_0, E)$ mit $L(M') = L = \{uav \mid u \in \Sigma^*, v \in \Sigma^{n-1}\}$ und $|Z| < 2^n$.

Die Menge Σ^n enthält 2^n Wörter der Länge n und da $|Z| < 2^n$, muss es $w, w' \in \Sigma^n$ geben mit $w \neq w'$ und $\tilde{\delta}(z_0, w) = \tilde{\delta}(z_0, w') = z$.

Sei $j \in \{1, \dots, n\}$ die erste Position, an der sich w und w' unterscheiden.

w ist von der Form uav und w' ist von der Form ubv' (oder umgekehrt) mit $|v| = |v'| = n - j$. Da $\tilde{\delta}(z_0, uav) = \tilde{\delta}(z_0, ubv') = z$, muss dann gelten $\tilde{\delta}(z_0, uavb^{j-1}) = \tilde{\delta}(z_0, ubv'b^{j-1}) = z'$ für einen Zustand z' .

Beweis (Fortsetzung)

2. Durch Widerspruch.

Wir nehmen an, es gibt einen DFA $M' = (Z, \Sigma, \delta, z_0, E)$ mit $L(M') = L = \{uav \mid u \in \Sigma^*, v \in \Sigma^{n-1}\}$ und $|Z| < 2^n$.

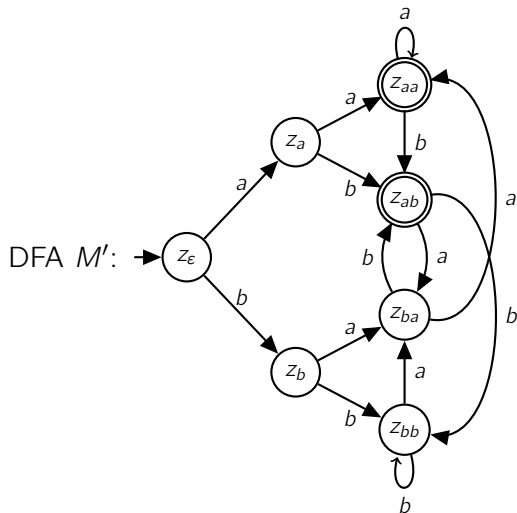
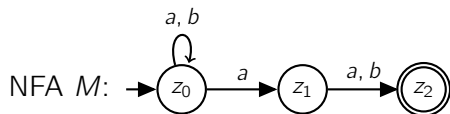
Die Menge Σ^n enthält 2^n Wörter der Länge n und da $|Z| < 2^n$, muss es $w, w' \in \Sigma^n$ geben mit $w \neq w'$ und $\tilde{\delta}(z_0, w) = \tilde{\delta}(z_0, w') = z$.

Sei $j \in \{1, \dots, n\}$ die erste Position, an der sich w und w' unterscheiden.

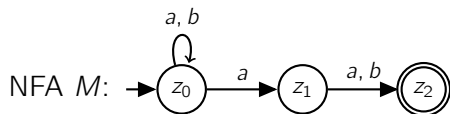
w ist von der Form uav und w' ist von der Form ubv' (oder umgekehrt) mit $|v| = |v'| = n - j$. Da $\tilde{\delta}(z_0, uav) = \tilde{\delta}(z_0, ubv') = z$, muss dann gelten $\tilde{\delta}(z_0, uavb^{j-1}) = \tilde{\delta}(z_0, ubv'b^{j-1}) = z'$ für einen Zustand z' .

Aber $uavb^{j-1} \in L$ und $ubv'b^{j-1} \notin L$, daher $z' \in E$ und $z' \notin E$. Widerspruch. $\square$

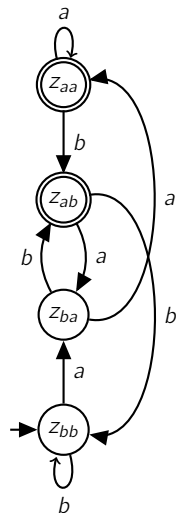
Beispiel wenn $n = 2$



Beispiel wenn $n = 2$



DFA M' (minimiert):



3c

Nichtdeterministische endliche Automaten mit ϵ -Übergängen

Prof. Dr. Jasmin Blanchette

Lehr- und Forschungseinheit für
Theoretische Informatik und Theorembeweisen

Stand: 10. Mai 2024

Basiert auf Folien von PD Dr. David Sabel



NFAs mit ε -Übergängen

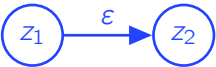
Informelle Kurzfassung:

- ▶ ε -Übergänge erlauben Zustandswechsel **ohne Lesen eines Zeichens**.
Es wird sozusagen ε gelesen.

NFAs mit ε -Übergängen

Informelle Kurzfassung:

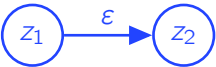
- ▶ ε -Übergänge erlauben Zustandswechsel **ohne Lesen eines Zeichens**.
Es wird sozusagen ε gelesen.

- ▶ Im Zustandsgraph erlaubt: A diagram showing two states, z_1 and z_2 , represented as blue circles. A blue arrow points from z_1 to z_2 , with the label ε above it.

NFAs mit ε -Übergängen

Informelle Kurzfassung:

- ▶ ε -Übergänge erlauben Zustandswechsel **ohne Lesen eines Zeichens**.
Es wird sozusagen ε gelesen.

- ▶ Im Zustandsgraph erlaubt: A diagram showing two states, z_1 and z_2 , each enclosed in a blue circle. A blue arrow points from z_1 to z_2 , with the label ε positioned above the arrow.

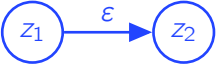
- ▶ Technisch:

- ▶ Ein **NFA ohne ε -Übergänge** hat $\delta : Z \times \Sigma \rightarrow \mathcal{P}(Z)$.
- ▶ Ein **NFA mit ε -Übergängen** hat $\delta : Z \times (\Sigma \cup \{\varepsilon\}) \rightarrow \mathcal{P}(Z)$.

NFAs mit ε -Übergängen

Informelle Kurzfassung:

- ▶ ε -Übergänge erlauben Zustandswechsel **ohne Lesen eines Zeichens**.
Es wird sozusagen ε gelesen.

- ▶ Im Zustandsgraph erlaubt: A diagram showing two states, z_1 and z_2 , represented as blue circles. A blue arrow points from z_1 to z_2 , with the label ε above it.

- ▶ Technisch:

- ▶ Ein **NFA ohne ε -Übergänge** hat $\delta : Z \times \Sigma \rightarrow \mathcal{P}(Z)$.
 - ▶ Ein **NFA mit ε -Übergängen** hat $\delta : Z \times (\Sigma \cup \{\varepsilon\}) \rightarrow \mathcal{P}(Z)$.
- ▶ Die Ausdruckskraft ändert sich mit ε -Übergängen nicht.
- ▶ ε -Übergänge machen manche Konstruktionen einfacher.

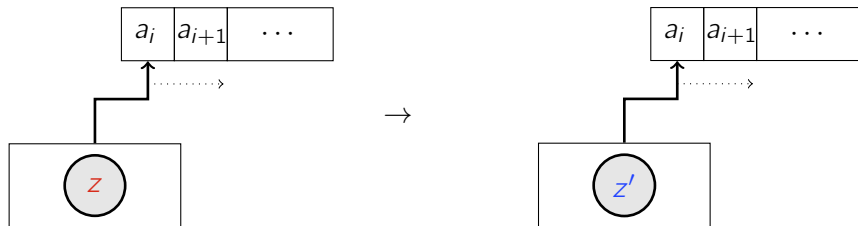
Definition eines NFA mit ε -Übergängen

Definition

Ein nichtdeterministischer endlicher Automat mit ε -Übergängen (NFA mit ε -Übergängen) ist ein 5-Tupel $M = (Z, \Sigma, \delta, S, E)$, wobei

- ▶ Z ist eine endliche Menge von Zuständen
- ▶ Σ ist das (endliche) Eingabealphabet mit $Z \cap \Sigma = \emptyset$
- ▶ $\delta : Z \times (\Sigma \cup \{\varepsilon\}) \rightarrow \mathcal{P}(Z)$ ist die Überföhrungsfunktion
- ▶ $S \subseteq Z$ ist die Menge der Startzustände
- ▶ $E \subseteq Z$ ist die Menge der Endzustände.

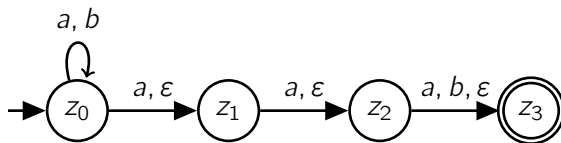
Illustration eines Zustandsübergangs mit ϵ



$z' \in \delta(z, \epsilon)$ bedeutet:

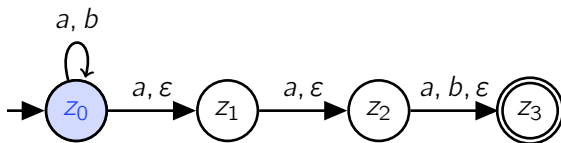
Im Zustand z darf der Automat in z' wechseln, ohne dass der Lesekopf sich bewegt.

Beispiel für einen akzeptierenden Lauf



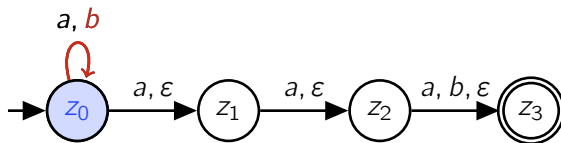
Abarbeitung der Eingabe *baab*

Beispiel für einen akzeptierenden Lauf



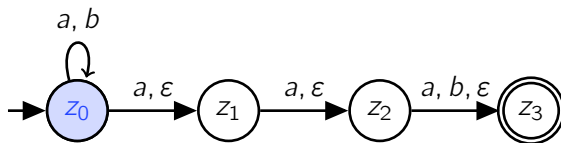
Abarbeitung der Eingabe *baab*

Beispiel für einen akzeptierenden Lauf



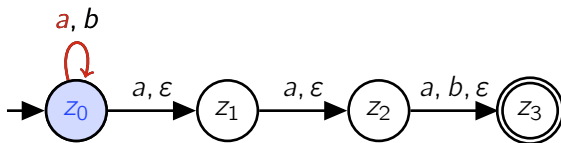
Abarbeitung der Eingabe $baab$

Beispiel für einen akzeptierenden Lauf



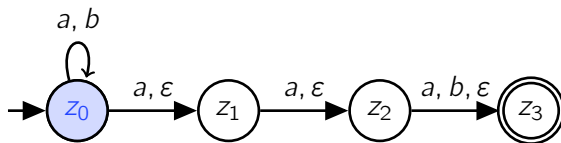
Abarbeitung der Eingabe *baab*

Beispiel für einen akzeptierenden Lauf



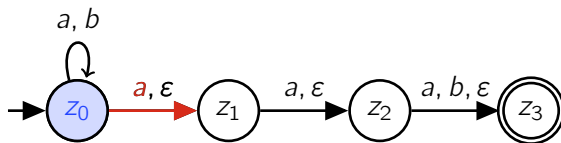
Abarbeitung der Eingabe $b\textcolor{red}{a}ab$

Beispiel für einen akzeptierenden Lauf



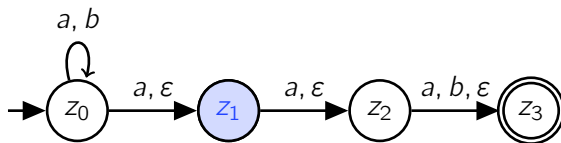
Abarbeitung der Eingabe $b\textcolor{red}{a}ab$

Beispiel für einen akzeptierenden Lauf



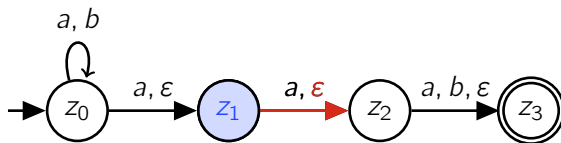
Abarbeitung der Eingabe $baab$

Beispiel für einen akzeptierenden Lauf



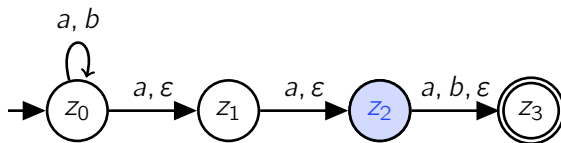
Abarbeitung der Eingabe $baab$

Beispiel für einen akzeptierenden Lauf



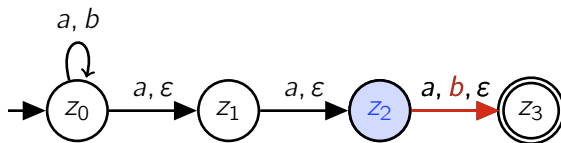
Abarbeitung der Eingabe $baab$

Beispiel für einen akzeptierenden Lauf



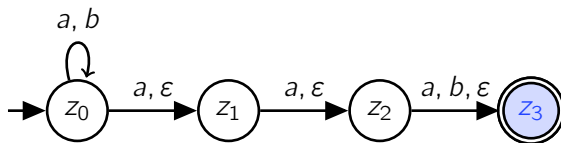
Abarbeitung der Eingabe $baab$

Beispiel für einen akzeptierenden Lauf



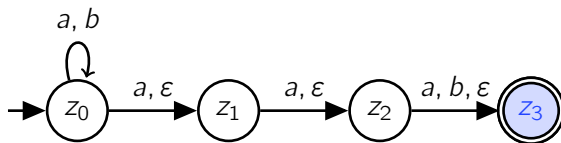
Abarbeitung der Eingabe $baab$

Beispiel für einen akzeptierenden Lauf



Abarbeitung der Eingabe *baa****b***

Beispiel für einen akzeptierenden Lauf



Abarbeitung der Eingabe *baab*

Die ε -Hülle fügt für einen Zustand alle durch ε -Übergänge erreichbaren Zustände hinzu.

Die ε -Hülle fügt für einen Zustand alle durch ε -Übergänge erreichbaren Zustände hinzu.

Definition

Sei $M = (Z, \Sigma, \delta, S, E)$ ein NFA mit ε -Übergängen.

Die ε -Hülle $\varepsilon\text{-Hülle}(z)$ eines Zustands $z \in Z$ ist definiert als die kleinste Menge von Zuständen, welche erfüllt:

1. $z \in \varepsilon\text{-Hülle}(z)$.
2. Wenn $z' \in \varepsilon\text{-Hülle}(z)$ und $z'' \in \delta(z', \varepsilon)$, dann ist auch $z'' \in \varepsilon\text{-Hülle}(z)$.

Die ε -Hülle fügt für einen Zustand alle durch ε -Übergänge erreichbaren Zustände hinzu.

Definition

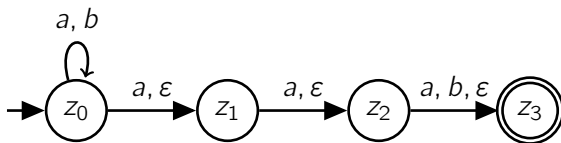
Sei $M = (Z, \Sigma, \delta, S, E)$ ein NFA mit ε -Übergängen.

Die ε -Hülle $\varepsilon\text{-Hülle}(z)$ eines Zustands $z \in Z$ ist definiert als die kleinste Menge von Zuständen, welche erfüllt:

1. $z \in \varepsilon\text{-Hülle}(z)$.
2. Wenn $z' \in \varepsilon\text{-Hülle}(z)$ und $z'' \in \delta(z', \varepsilon)$, dann ist auch $z'' \in \varepsilon\text{-Hülle}(z)$.

Für eine Menge $X \subseteq Z$ definieren wir $\varepsilon\text{-Hülle}(X) := \bigcup_{z \in X} \varepsilon\text{-Hülle}(z)$.

Beispiel für die ε -Hülle



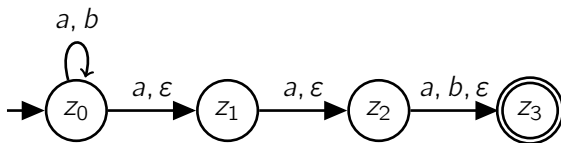
ε -Hülle(z_0) = ?

ε -Hülle(z_1) = ?

ε -Hülle(z_2) = ?

ε -Hülle(z_3) = ?

Beispiel für die ε -Hülle



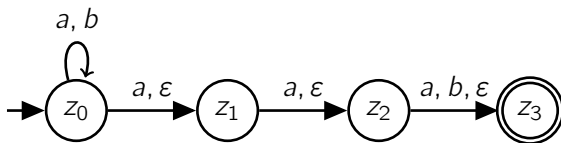
$$\varepsilon\text{-H\ddot{u}lle}(z_0) = \{z_0, z_1, z_2, z_3\}$$

$$\varepsilon\text{-H\ddot{u}lle}(z_1) = ?$$

$$\varepsilon\text{-H\ddot{u}lle}(z_2) = ?$$

$$\varepsilon\text{-H\ddot{u}lle}(z_3) = ?$$

Beispiel für die ε -Hülle



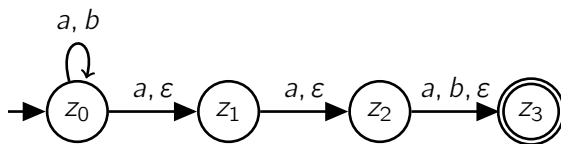
$$\varepsilon\text{-H\ddot{u}lle}(z_0) = \{z_0, z_1, z_2, z_3\}$$

$$\varepsilon\text{-H\ddot{u}lle}(z_1) = \{z_1, z_2, z_3\}$$

$$\varepsilon\text{-H\ddot{u}lle}(z_2) = ?$$

$$\varepsilon\text{-H\ddot{u}lle}(z_3) = ?$$

Beispiel für die ε -Hülle



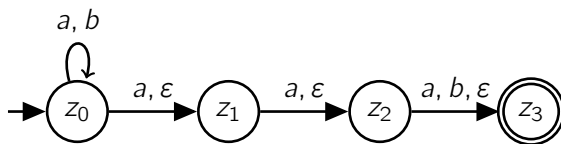
$$\varepsilon\text{-H\ddot{u}lle}(z_0) = \{z_0, z_1, z_2, z_3\}$$

$$\varepsilon\text{-H\ddot{u}lle}(z_1) = \{z_1, z_2, z_3\}$$

$$\varepsilon\text{-H\ddot{u}lle}(z_2) = \{z_2, z_3\}$$

$$\varepsilon\text{-H\ddot{u}lle}(z_3) = ?$$

Beispiel für die ε -Hülle



$$\varepsilon\text{-H\ddot{u}lle}(z_0) = \{z_0, z_1, z_2, z_3\}$$

$$\varepsilon\text{-H\ddot{u}lle}(z_1) = \{z_1, z_2, z_3\}$$

$$\varepsilon\text{-H\ddot{u}lle}(z_2) = \{z_2, z_3\}$$

$$\varepsilon\text{-H\ddot{u}lle}(z_3) = \{z_3\}$$

Akzeptierte Sprache eines NFA mit ε -Übergängen

Sei $M = (Z, \Sigma, \delta, S, E)$ ein NFA mit ε -Übergängen.

Wir definieren $\tilde{\delta} : \mathcal{P}(Z) \times \Sigma^* \rightarrow \mathcal{P}(Z)$ rekursiv durch

$$\begin{aligned}\tilde{\delta}(X, \varepsilon) &:= X \\ \tilde{\delta}(X, aw) &:= \tilde{\delta}\left(\bigcup_{z \in X} \varepsilon\text{-Hülle}(\delta(z, a)), w\right)\end{aligned}$$

Akzeptanz bei NFAs mit ε -Übergängen

Akzeptierte Sprache eines NFA mit ε -Übergängen

Sei $M = (Z, \Sigma, \delta, S, E)$ ein NFA mit ε -Übergängen.

Wir definieren $\tilde{\delta} : \mathcal{P}(Z) \times \Sigma^* \rightarrow \mathcal{P}(Z)$ rekursiv durch

$$\begin{aligned}\tilde{\delta}(X, \varepsilon) &:= X \\ \tilde{\delta}(X, aw) &:= \tilde{\delta}\left(\bigcup_{z \in X} \varepsilon\text{-Hülle}(\delta(z, a)), w\right)\end{aligned}$$

Die von M akzeptierte Sprache ist

$$L(M) := \{w \in \Sigma^* \mid \tilde{\delta}(\varepsilon\text{-Hülle}(S), w) \cap E \neq \emptyset\}$$

Entfernen von ε -Übergängen

Intuitiver Ansatz:

1. Markiere für jeden Pfad von der Form



den Zustand z_n als Startzustand:



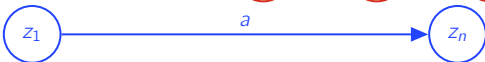
Entfernen von ϵ -Übergängen

Intuitiver Ansatz:

1. Markiere für jeden Pfad von der Form 

den Zustand z_n als Startzustand:



2. Füge für jeden Pfad von der Form 
einen Übergang  hinzu.

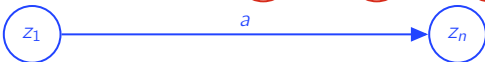
Entfernen von ε -Übergängen

Intuitiver Ansatz:

1. Markiere für jeden Pfad von der Form 

den Zustand z_n als Startzustand:



2. Füge für jeden Pfad von der Form 
einen Übergang  hinzu.

3. Entferne alle ε -Übergänge.

Sprache von NFAs mit ε -Übergängen

Satz

NFAs mit ε -Übergängen akzeptieren genau die regulären Sprachen.

Sprache von NFAs mit ϵ -Übergängen

Satz

NFAs mit ϵ -Übergängen akzeptieren genau die regulären Sprachen.

Beweis

- ⊇ Wir zeigen zuerst, dass jede reguläre Sprache L von einem NFA mit ϵ -Übergängen akzeptiert wird.

Sprache von NFAs mit ϵ -Übergängen

Satz

NFAs mit ϵ -Übergängen akzeptieren genau die regulären Sprachen.

Beweis

⊇ Wir zeigen zuerst, dass jede reguläre Sprache L von einem NFA mit ϵ -Übergängen akzeptiert wird.

L wird von einem NFA $M = (Z, \Sigma, \delta, S, E)$ ohne ϵ -Übergänge akzeptiert. Sei $M' = (Z, \Sigma, \delta', S, E)$ ein NFA mit ϵ -Übergängen, wobei

$$\begin{aligned}\delta'(z, a) &:= \delta(z, a) \quad \text{für } a \in \Sigma \\ \delta'(z, \epsilon) &:= \emptyset\end{aligned}$$

Sprache von NFAs mit ϵ -Übergängen

Satz

NFAs mit ϵ -Übergängen akzeptieren genau die regulären Sprachen.

Beweis

⊇ Wir zeigen zuerst, dass jede reguläre Sprache L von einem NFA mit ϵ -Übergängen akzeptiert wird.

L wird von einem NFA $M = (Z, \Sigma, \delta, S, E)$ ohne ϵ -Übergänge akzeptiert. Sei $M' = (Z, \Sigma, \delta', S, E)$ ein NFA mit ϵ -Übergängen, wobei

$$\begin{aligned}\delta'(z, a) &:= \delta(z, a) \quad \text{für } a \in \Sigma \\ \delta'(z, \epsilon) &:= \emptyset\end{aligned}$$

Offensichtlich gilt $L(M') = L(M) = L$.

Beweis (Fortsetzung)

- $\subseteq$ Sei $M = (Z, \Sigma, \delta, S, E)$ ein NFA mit ε -Übergängen.
Wir zeigen, dass $L(M)$ ist regulär.

Beweis (Fortsetzung)

⊆ Sei $M = (Z, \Sigma, \delta, S, E)$ ein NFA mit ϵ -Übergängen.

Wir zeigen, dass $L(M)$ ist regulär.

Wir konstruieren einen NFA M' ohne ϵ -Übergänge mit $L(M') = L(M)$. Dann ist $L(M')$ regulär, daher ist auch $L(M)$ regulär.

Sprache von NFAs mit ε -Übergängen

Beweis (Fortsetzung)

$\subseteq$ Sei $M = (Z, \Sigma, \delta, S, E)$ ein NFA mit ε -Übergängen.

Wir zeigen, dass $L(M)$ ist regulär.

Wir konstruieren einen NFA M' ohne ε -Übergänge mit $L(M') = L(M)$. Dann ist $L(M')$ regulär, daher ist auch $L(M)$ regulär.

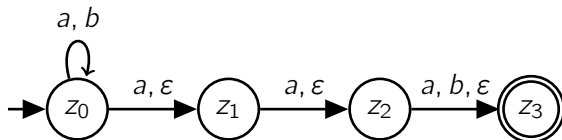
Sei $M' = (Z, \Sigma, \delta', S', E)$ mit

$$S' := \varepsilon\text{-Hülle}(S)$$

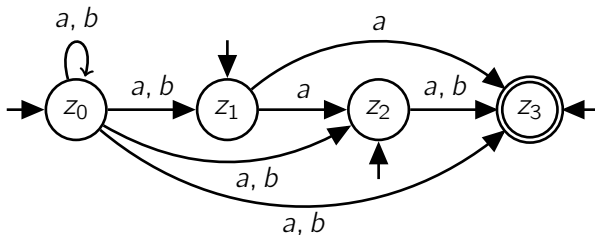
$$\delta'(z, a) := \varepsilon\text{-Hülle}(\delta(z, a))$$

Beispiel für die Elimination von ϵ -Übergängen

NFA M mit ϵ -Übergängen:

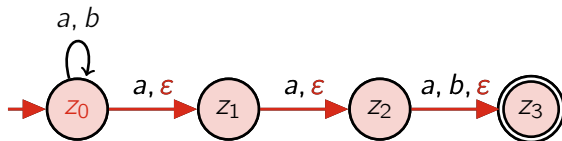


NFA M' ohne ϵ -Übergänge:

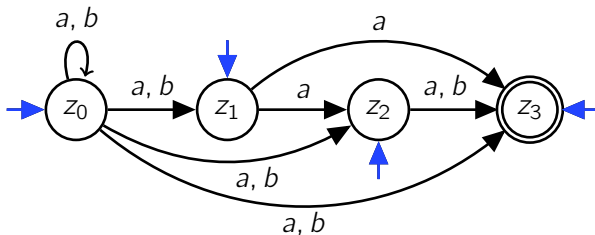


Beispiel für die Elimination von ϵ -Übergängen

NFA M mit ϵ -Übergängen:

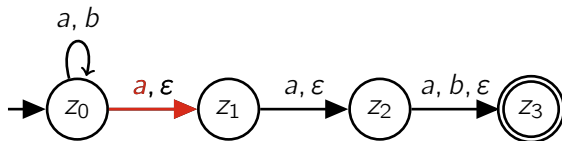


NFA M' ohne ϵ -Übergänge:

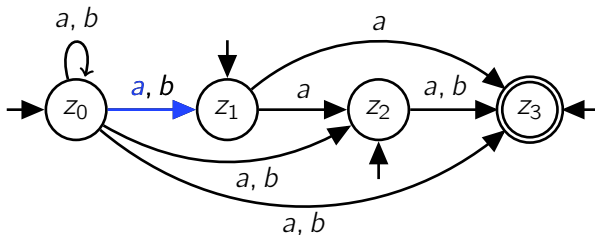


Beispiel für die Elimination von ϵ -Übergängen

NFA M mit ϵ -Übergängen:

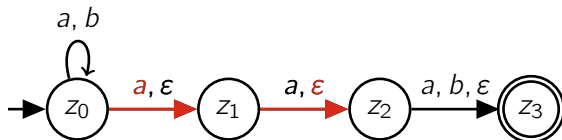


NFA M' ohne ϵ -Übergänge:

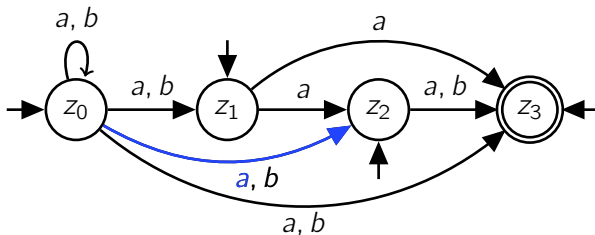


Beispiel für die Elimination von ϵ -Übergängen

NFA M mit ϵ -Übergängen:

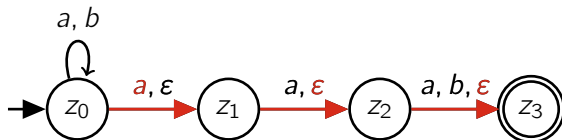


NFA M' ohne ϵ -Übergänge:

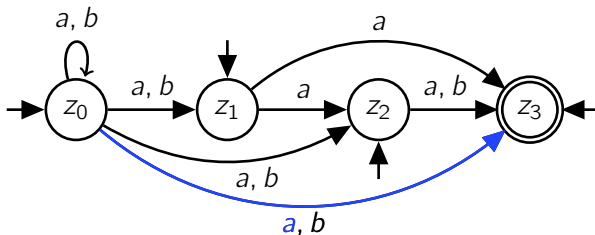


Beispiel für die Elimination von ϵ -Übergängen

NFA M mit ϵ -Übergängen:



NFA M' ohne ϵ -Übergänge:



Beweis (Fortsetzung)

Wir müssen zeigen, dass $L(M') = L(M)$, d.h. $w \in L(M')$ g.d.w. $w \in L(M)$.

Beweis (Fortsetzung)

Wir müssen zeigen, dass $L(M') = L(M)$, d.h. $w \in L(M')$ g.d.w. $w \in L(M)$.

$w \in L(M')$

Sprache von NFAs mit ε -Übergängen

Beweis (Fortsetzung)

Wir müssen zeigen, dass $L(M') = L(M)$, d.h. $w \in L(M')$ g.d.w. $w \in L(M)$.

$w \in L(M')$
g.d.w. $\tilde{\delta}'(\varepsilon\text{-Hülle}(S), w) \cap E \neq \emptyset$

Sprache von NFAs mit ε -Übergängen

Beweis (Fortsetzung)

Wir müssen zeigen, dass $L(M') = L(M)$, d.h. $w \in L(M')$ g.d.w. $w \in L(M)$.

$w \in L(M')$
g.d.w. $\tilde{\delta}'(\varepsilon\text{-Hülle}(S), w) \cap E \neq \emptyset$

$w \in L(M)$

Sprache von NFAs mit ε -Übergängen

Beweis (Fortsetzung)

Wir müssen zeigen, dass $L(M') = L(M)$, d.h. $w \in L(M')$ g.d.w. $w \in L(M)$.

$w \in L(M')$
g.d.w. $\tilde{\delta}'(\varepsilon\text{-Hülle}(S), w) \cap E \neq \emptyset$

$\tilde{\delta}(\varepsilon\text{-Hülle}(S), w) \cap E \neq \emptyset$
g.d.w. $w \in L(M)$

Sprache von NFAs mit ε -Übergängen

Beweis (Fortsetzung)

Wir müssen zeigen, dass $L(M') = L(M)$, d.h. $w \in L(M')$ g.d.w. $w \in L(M)$.

$w \in L(M')$
g.d.w. $\tilde{\delta}'(\varepsilon\text{-Hülle}(S), w) \cap E \neq \emptyset$

g.d.w. $\tilde{\delta}(\varepsilon\text{-Hülle}(S), w) \cap E \neq \emptyset$ (weil $\tilde{\delta}' = \tilde{\delta}$)

g.d.w. $w \in L(M)$

Beweis (Fortsetzung)

Es bleibt zu zeigen, dass $\tilde{\delta}' = \tilde{\delta}$, d.h. $\tilde{\delta}'(X, w) = \tilde{\delta}(X, w)$ für alle X, w .

Wir verwenden Induktion über die Wortlänge $|w|$.

Beweis (Fortsetzung)

Es bleibt zu zeigen, dass $\tilde{\delta}' = \tilde{\delta}$, d.h. $\tilde{\delta}'(X, w) = \tilde{\delta}(X, w)$ für alle X, w .

Wir verwenden Induktion über die Wortlänge $|w|$.

- Fall $w = \varepsilon$: $\tilde{\delta}'(X, \varepsilon) = X = \tilde{\delta}(X, \varepsilon)$.

Beweis (Fortsetzung)

Es bleibt zu zeigen, dass $\tilde{\delta}' = \tilde{\delta}$, d.h. $\tilde{\delta}'(X, w) = \tilde{\delta}(X, w)$ für alle X, w .

Wir verwenden Induktion über die Wortlänge $|w|$.

- ▶ Fall $w = \varepsilon$: $\tilde{\delta}'(X, \varepsilon) = X = \tilde{\delta}(X, \varepsilon)$.
- ▶ Fall $w = aw'$: Die IH für w' ist $\tilde{\delta}'(X, w') = \tilde{\delta}(X, w')$.

Beweis (Fortsetzung)

Es bleibt zu zeigen, dass $\tilde{\delta}' = \tilde{\delta}$, d.h. $\tilde{\delta}'(X, w) = \tilde{\delta}(X, w)$ für alle X, w .

Wir verwenden Induktion über die Wortlänge $|w|$.

- ▶ Fall $w = \varepsilon$: $\tilde{\delta}'(X, \varepsilon) = X = \tilde{\delta}(X, \varepsilon)$.
- ▶ Fall $w = aw'$: Die IH für w' ist $\tilde{\delta}'(X, w') = \tilde{\delta}(X, w')$.

$$\tilde{\delta}'(X, aw')$$

Sprache von NFAs mit ε -Übergängen

Beweis (Fortsetzung)

Es bleibt zu zeigen, dass $\tilde{\delta}' = \tilde{\delta}$, d.h. $\tilde{\delta}'(X, w) = \tilde{\delta}(X, w)$ für alle X, w .

Wir verwenden Induktion über die Wortlänge $|w|$.

- ▶ Fall $w = \varepsilon$: $\tilde{\delta}'(X, \varepsilon) = X = \tilde{\delta}(X, \varepsilon)$.
- ▶ Fall $w = aw'$: Die IH für w' ist $\tilde{\delta}'(X, w') = \tilde{\delta}(X, w')$.

$$\begin{aligned} & \tilde{\delta}'(X, aw') \\ = & \tilde{\delta}'(\bigcup_{z \in X} \delta'(z, a), w') \end{aligned} \quad (\text{Def. } \tilde{\delta}', \text{ für NFA ohne } \varepsilon\text{-Übergänge})$$

Sprache von NFAs mit ε -Übergängen

Beweis (Fortsetzung)

Es bleibt zu zeigen, dass $\tilde{\delta}' = \tilde{\delta}$, d.h. $\tilde{\delta}'(X, w) = \tilde{\delta}(X, w)$ für alle X, w .

Wir verwenden Induktion über die Wortlänge $|w|$.

- Fall $w = \varepsilon$: $\tilde{\delta}'(X, \varepsilon) = X = \tilde{\delta}(X, \varepsilon)$.
- Fall $w = aw'$: Die IH für w' ist $\tilde{\delta}'(X, w') = \tilde{\delta}(X, w')$.

$$\begin{aligned} & \tilde{\delta}'(X, aw') \\ &= \tilde{\delta}'\left(\bigcup_{z \in X} \delta'(z, a), w'\right) && \text{(Def. } \tilde{\delta}', \text{ für NFA ohne } \varepsilon\text{-Übergänge)} \\ &= \tilde{\delta}'\left(\bigcup_{z \in X} \varepsilon\text{-Hülle}(\delta(z, a)), w'\right) && \text{(Def. } \delta') \end{aligned}$$

Sprache von NFAs mit ε -Übergängen

Beweis (Fortsetzung)

Es bleibt zu zeigen, dass $\tilde{\delta}' = \tilde{\delta}$, d.h. $\tilde{\delta}'(X, w) = \tilde{\delta}(X, w)$ für alle X, w .

Wir verwenden Induktion über die Wortlänge $|w|$.

- Fall $w = \varepsilon$: $\tilde{\delta}'(X, \varepsilon) = X = \tilde{\delta}(X, \varepsilon)$.
- Fall $w = aw'$: Die IH für w' ist $\tilde{\delta}'(X, w') = \tilde{\delta}(X, w')$.

$$\begin{aligned} & \tilde{\delta}'(X, aw') \\ &= \tilde{\delta}'\left(\bigcup_{z \in X} \delta'(z, a), w'\right) && \text{(Def. } \tilde{\delta}', \text{ für NFA ohne } \varepsilon\text{-Übergänge)} \\ &= \tilde{\delta}'\left(\bigcup_{z \in X} \varepsilon\text{-Hülle}(\delta(z, a)), w'\right) && \text{(Def. } \delta') \end{aligned}$$

$$\tilde{\delta}(X, aw')$$

Sprache von NFAs mit ε -Übergängen

Beweis (Fortsetzung)

Es bleibt zu zeigen, dass $\tilde{\delta}' = \tilde{\delta}$, d.h. $\tilde{\delta}'(X, w) = \tilde{\delta}(X, w)$ für alle X, w .

Wir verwenden Induktion über die Wortlänge $|w|$.

- Fall $w = \varepsilon$: $\tilde{\delta}'(X, \varepsilon) = X = \tilde{\delta}(X, \varepsilon)$.
- Fall $w = aw'$: Die IH für w' ist $\tilde{\delta}'(X, w') = \tilde{\delta}(X, w')$.

$$\begin{aligned} & \tilde{\delta}'(X, aw') \\ &= \tilde{\delta}'\left(\bigcup_{z \in X} \delta'(z, a), w'\right) && \text{(Def. } \tilde{\delta}', \text{ für NFA ohne } \varepsilon\text{-Übergänge)} \\ &= \tilde{\delta}'\left(\bigcup_{z \in X} \varepsilon\text{-Hülle}(\delta(z, a)), w'\right) && \text{(Def. } \delta') \\ &= \tilde{\delta}\left(\bigcup_{z \in X} \varepsilon\text{-Hülle}(\delta(z, a)), w'\right) \\ &= \tilde{\delta}(X, aw') && \text{(Def. } \tilde{\delta}, \text{ für NFA mit } \varepsilon\text{-Übergängen)} \end{aligned}$$

Sprache von NFAs mit ε -Übergängen

Beweis (Fortsetzung)

Es bleibt zu zeigen, dass $\tilde{\delta}' = \tilde{\delta}$, d.h. $\tilde{\delta}'(X, w) = \tilde{\delta}(X, w)$ für alle X, w .

Wir verwenden Induktion über die Wortlänge $|w|$.

- Fall $w = \varepsilon$: $\tilde{\delta}'(X, \varepsilon) = X = \tilde{\delta}(X, \varepsilon)$.
- Fall $w = aw'$: Die IH für w' ist $\tilde{\delta}'(X, w') = \tilde{\delta}(X, w')$.

$$\begin{aligned} & \tilde{\delta}'(X, aw') \\ &= \tilde{\delta}'\left(\bigcup_{z \in X} \delta'(z, a), w'\right) && \text{(Def. } \tilde{\delta}', \text{ für NFA ohne } \varepsilon\text{-Übergänge)} \\ &= \tilde{\delta}'\left(\bigcup_{z \in X} \varepsilon\text{-Hülle}(\delta(z, a)), w'\right) && \text{(Def. } \delta') \\ &= \tilde{\delta}\left(\bigcup_{z \in X} \varepsilon\text{-Hülle}(\delta(z, a)), w'\right) && \text{(IH)} \\ &= \tilde{\delta}(X, aw') && \text{(Def. } \tilde{\delta}, \text{ für NFA mit } \varepsilon\text{-Übergängen)} \end{aligned}$$

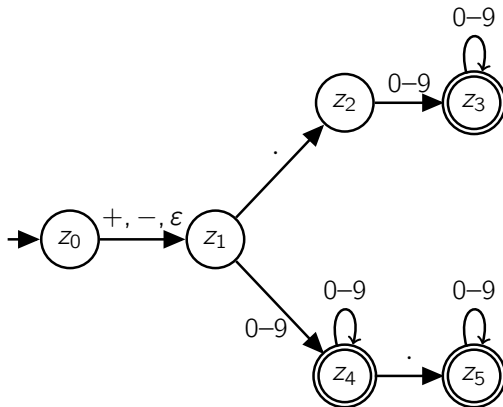
□

Beispiel für die Konstruktion eines NFA mit ε -Übergängen

Konstruiere einen NFA mit
 ε -Übergängen über
 $\{+, -, ., 0, \dots, 9\}$,
der alle Wörter akzeptiert,
die Gleitkommazahlen
darstellen
(z.B. $+27$, -3.14 , $.666$):

Beispiel für die Konstruktion eines NFA mit ε -Übergängen

Konstruiere einen NFA mit ε -Übergängen über $\{+, -, ., 0, \dots, 9\}$, der alle Wörter akzeptiert, die Gleitkommazahlen darstellen (z.B. $+27$, -3.14 , $.666$):



Eindeutige Start- und Endzustände

Satz

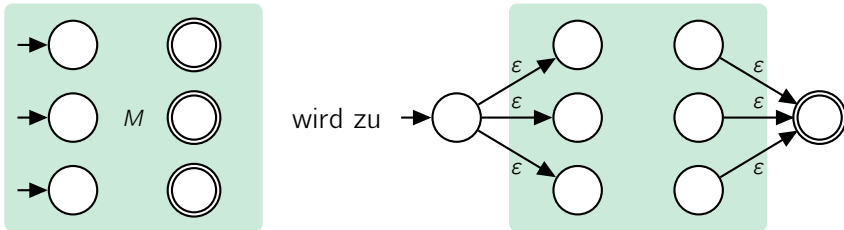
Für jeden NFA M mit ε -Übergängen gibt es einen NFA M' mit ε -Übergängen, sodass $L(M') = L(M)$ und M' genau einen Startzustand und genau einen Endzustand hat, wobei diese beiden Zustände verschieden sind.

Eindeutige Start- und Endzustände

Satz

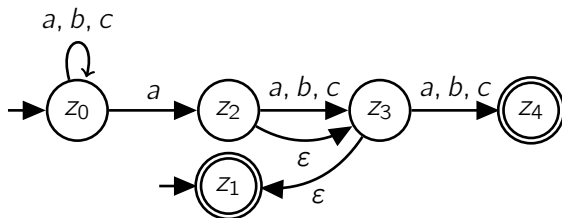
Für jeden NFA M mit ε -Übergängen gibt es einen NFA M' mit ε -Übergängen, sodass $L(M') = L(M)$ und M' genau einen Startzustand und genau einen Endzustand hat, wobei diese beiden Zustände verschieden sind.

Beweis Konstruiere M' aus M , durch Hinzufügen eines neuen Start- und eines neuen Endzustands mit ε -Übergängen:

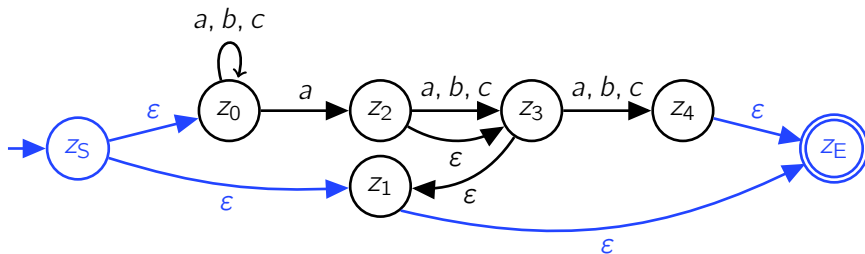


□

Beispiel für eindeutige Start- und Endzustände



wird zu



Christopher Schaffner hat ein Web-Tool zum Üben und Anschauen entwickelt:

chrisschaffner.github.io/graphicalInterfaceForGrammars/

4a

Reguläre Ausdrücke

Prof. Dr. Jasmin Blanchette

Lehr- und Forschungseinheit für
Theoretische Informatik und Theorembeweisen

Stand: 10. Mai 2024

Basiert auf Folien von PD Dr. David Sabel und Dr. Jan Johannsen



Informelle Kurzfassung:

- ▶ Reguläre Ausdrücke sind (wie Automaten und Grammatiken) ein Formalismus zur Repräsentation von Sprachen.
- ▶ Sie bieten eine kompakte Notation für reguläre Sprachen an.
- ▶ In der Praxis werden Sie verwendet, um Text zu erkennen oder zu suchen.

WHENEVER I LEARN A
NEW SKILL I CONCOCT
ELABORATE FANTASY
SCENARIOS WHERE IT
LETS ME SAVE THE DAY.

OH NO! THE KILLER
MUST HAVE FOLLOWED
HER ON VACATION!



BUT TO FIND THEM WE'D HAVE TO SEARCH
THROUGH 200 MB OF EMAILS LOOKING FOR
SOMETHING FORMATTED LIKE AN ADDRESS!



IT'S HOPELESS!

EVERYBODY STAND BACK.



I KNOW REGULAR
EXPRESSIONS.



xkcd.com/208/

Definition

Sei Σ ein Alphabet. Die regulären Ausdrücke über Σ sind definiert durch folgende Regeln (und nur diese):

- ▶ $\emptyset$ ist ein regulärer Ausdruck.
- ▶ ϵ ist ein regulärer Ausdruck.
- ▶ a mit $a \in \Sigma$ ist ein regulärer Ausdruck.
- ▶ Wenn α und β reguläre Ausdrücke sind, dann auch $\alpha\beta$ (alternativ $\alpha \cdot \beta$).
- ▶ Wenn α und β reguläre Ausdrücke sind, dann auch $(\alpha|\beta)$.
- ▶ Wenn α ein regulärer Ausdruck ist, dann auch $(\alpha)^*$.

Erzeugte Sprache eines regulären Ausdrucks

Definition

Die von einem regulären Ausdruck α **erzeugte Sprache** $L(\alpha)$ ist rekursiv definiert:

$$L(\emptyset) := \emptyset$$

$$L(\varepsilon) := \{\varepsilon\}$$

$$L(a) := \{a\} \quad \text{für } a \in \Sigma$$

$$L(\alpha\beta) := L(\alpha)L(\beta) = \{uv \mid u \in L(\alpha), v \in L(\beta)\}$$

$$L(\alpha|\beta) := L(\alpha) \cup L(\beta)$$

$$L((\alpha)^*) := L(\alpha)^*$$

Erzeugte Sprache eines regulären Ausdrucks

Definition

Die von einem regulären Ausdruck α **erzeugte Sprache** $L(\alpha)$ ist rekursiv definiert:

$$L(\emptyset) := \emptyset$$

$$L(\varepsilon) := \{\varepsilon\}$$

$$L(a) := \{a\} \quad \text{für } a \in \Sigma$$

$$L(\alpha\beta) := L(\alpha)L(\beta) = \{uv \mid u \in L(\alpha), v \in L(\beta)\}$$

$$L(\alpha|\beta) := L(\alpha) \cup L(\beta)$$

$$L((\alpha)^*) := L(\alpha)^*$$

Für alle regulären Ausdrücke α, β, γ gilt $L((\alpha|\beta)|\gamma) = L(\alpha|(\beta|\gamma))$.

Daher lassen wir Klammern weg und schreiben $(\alpha_1|\alpha_2|\cdots|\alpha_n)$.

Beispiele für reguläre Ausdrücke

1. $(a|b)^*aa(a|b)^*$

erzeugt ?

2. $(\epsilon|((a|b|c)^*a(a|b|c)(a|b|c)(a|b|c)))$

erzeugt ?

3. $((0|1|2|3|4|5|6|7|8|9)|1(0|1|2|3|4|5|6|7|8|9)|(2(0|1|2|3))):$
 $((0|1|2|3|4|5)(0|1|2|3|4|5|6|7|8|9))$

erzeugt ?

Beispiele für reguläre Ausdrücke

1. $(a|b)^*aa(a|b)^*$

erzeugt alle Wörter über $\{a, b\}$, die zwei aufeinanderfolgende a 's enthalten.

2. $(\epsilon|((a|b|c)^*a(a|b|c)(a|b|c)(a|b|c)))$

erzeugt ?

3. $((0|1|2|3|4|5|6|7|8|9)|1(0|1|2|3|4|5|6|7|8|9)|(2(0|1|2|3))):$
 $((0|1|2|3|4|5)(0|1|2|3|4|5|6|7|8|9))$

erzeugt ?

Beispiele für reguläre Ausdrücke

1. $(a|b)^*aa(a|b)^*$

erzeugt alle Wörter über $\{a, b\}$, die zwei aufeinanderfolgende a 's enthalten.

2. $(\epsilon|((a|b|c)^*a(a|b|c)(a|b|c)(a|b|c)))$

erzeugt alle Wörter über $\{a, b, c\}$, die an viertletzter Stelle ein a haben sowie das leere Wort.

3. $((0|1|2|3|4|5|6|7|8|9)|1(0|1|2|3|4|5|6|7|8|9)|(2(0|1|2|3)))):$
 $((0|1|2|3|4|5)(0|1|2|3|4|5|6|7|8|9))$

erzeugt ?

Beispiele für reguläre Ausdrücke

1. $(a|b)^*aa(a|b)^*$
erzeugt alle Wörter über $\{a, b\}$, die zwei aufeinanderfolgende a 's enthalten.
2. $(\epsilon|((a|b|c)^*a(a|b|c)(a|b|c)(a|b|c)))$
erzeugt alle Wörter über $\{a, b, c\}$, die an viertletzter Stelle ein a haben sowie das leere Wort.
3. $((0|1|2|3|4|5|6|7|8|9)|1(0|1|2|3|4|5|6|7|8|9)|(2(0|1|2|3)))):$
 $((0|1|2|3|4|5)(0|1|2|3|4|5|6|7|8|9))$
erzeugt alle Wörter über $\{:, 0, 1, \dots, 9\}$, die Uhrzeiten im 24-Stunden-Format entsprechen.

Beispiele für reguläre Ausdrücke

Geben Sie reguläre Ausdrücke an, welche die folgenden Sprachen erzeugen.

1. alle Wörter über $\{a, b\}$, die das Teilwort *abba* enthalten:

?

2. alle Wörter über $\{a, b\}$, die das Teilwort *aba* mindestens zweimal enthalten:

?

3. alle Wörter über $\{a, b\}$, die das Teilwort *aaa* **nicht** enthalten:

?

4. alle Wörter einer endlichen Sprache $L = \{w_1, \dots, w_n\}$ mit $n \geq 1$:

?

Beispiele für reguläre Ausdrücke

Geben Sie reguläre Ausdrücke an, welche die folgenden Sprachen erzeugen.

1. alle Wörter über $\{a, b\}$, die das Teilwort $abba$ enthalten:

$(a|b)^*abba(a|b)^*$

2. alle Wörter über $\{a, b\}$, die das Teilwort aba mindestens zweimal enthalten:

?

3. alle Wörter über $\{a, b\}$, die das Teilwort aaa nicht enthalten:

?

4. alle Wörter einer endlichen Sprache $L = \{w_1, \dots, w_n\}$ mit $n \geq 1$:

?

Beispiele für reguläre Ausdrücke

Geben Sie reguläre Ausdrücke an, welche die folgenden Sprachen erzeugen.

1. alle Wörter über $\{a, b\}$, die das Teilwort $abba$ enthalten:

$(a|b)^* abba (a|b)^*$

2. alle Wörter über $\{a, b\}$, die das Teilwort aba mindestens zweimal enthalten:

$((a|b)^* aba (a|b)^* aba (a|b)^* | (a|b)^* ababa (a|b)^*)$

3. alle Wörter über $\{a, b\}$, die das Teilwort aaa nicht enthalten:

?

4. alle Wörter einer endlichen Sprache $L = \{w_1, \dots, w_n\}$ mit $n \geq 1$:

?

Beispiele für reguläre Ausdrücke

Geben Sie reguläre Ausdrücke an, welche die folgenden Sprachen erzeugen.

1. alle Wörter über $\{a, b\}$, die das Teilwort $abba$ enthalten:
 $(a|b)^* abba (a|b)^*$
2. alle Wörter über $\{a, b\}$, die das Teilwort aba mindestens zweimal enthalten:
 $((a|b)^* aba (a|b)^* aba (a|b)^* | (a|b)^* ababa (a|b)^*)$
3. alle Wörter über $\{a, b\}$, die das Teilwort aaa **nicht** enthalten:
 $(b|ab|aab)^* (\epsilon|a|aa)$
4. alle Wörter einer endlichen Sprache $L = \{w_1, \dots, w_n\}$ mit $n \geq 1$:
?

Beispiele für reguläre Ausdrücke

Geben Sie reguläre Ausdrücke an, welche die folgenden Sprachen erzeugen.

1. alle Wörter über $\{a, b\}$, die das Teilwort $abba$ enthalten:
 $(a|b)^* abba (a|b)^*$
2. alle Wörter über $\{a, b\}$, die das Teilwort aba mindestens zweimal enthalten:
 $((a|b)^* aba (a|b)^* aba (a|b)^* | (a|b)^* ababa (a|b)^*)$
3. alle Wörter über $\{a, b\}$, die das Teilwort aaa **nicht** enthalten:
 $(b|ab|aab)^* (\epsilon|a|aa)$
4. alle Wörter einer endlichen Sprache $L = \{w_1, \dots, w_n\}$ mit $n \geq 1$:
 $(w_1 | \dots | w_n)$

Regeln zum Vereinfachen von regulären Ausdrücken

Kommutativität:

$$(\alpha|\beta) = (\beta|\alpha)$$

Neutrale Elemente:

$$(\emptyset|\alpha) = (\alpha|\emptyset) = \alpha$$

Absorption:

$$\emptyset\alpha = \alpha\emptyset = \emptyset$$

Distributivität:

$$\alpha(\beta|\gamma) = \alpha\beta|\alpha\gamma$$

Gesetze über Kleene-Stern:

$$((\alpha)^*)^* = (\alpha)^* \quad (\emptyset)^* = \varepsilon \quad (\varepsilon)^* = \varepsilon$$

Sprache von regulären Ausdrücken

Theorem (Satz von Kleene)

Reguläre Ausdrücke erzeugen genau die regulären Sprachen.

Beweis

⊆ Wir zeigen, dass die erzeugte Sprache eines regulären Ausdrucks regulär ist.

Sprache von regulären Ausdrücken

Theorem (Satz von Kleene)

Reguläre Ausdrücke erzeugen genau die regulären Sprachen.

Beweis

⊆ Wir zeigen, dass die erzeugte Sprache eines regulären Ausdrucks regulär ist.

Wir konstruieren für einen regulären Ausdruck α einen NFA M_α mit ϵ -Übergängen und eindeutigen Start- und Endzuständen, sodass $L(M_\alpha) = L(\alpha)$.

Sprache von regulären Ausdrücken

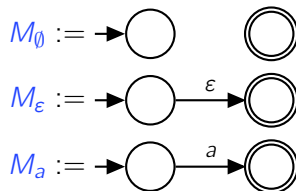
Theorem (Satz von Kleene)

Reguläre Ausdrücke erzeugen genau die regulären Sprachen.

Beweis

⊆ Wir zeigen, dass die erzeugte Sprache eines regulären Ausdrucks regulär ist.

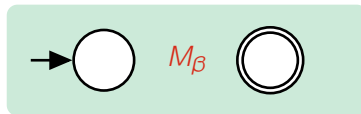
Wir konstruieren für einen regulären Ausdruck α einen NFA M_α mit ε -Übergängen und eindeutigen Start- und Endzuständen, sodass $L(M_\alpha) = L(\alpha)$.



Sprache von regulären Ausdrücken

Beweis (Fortsetzung)

Seien



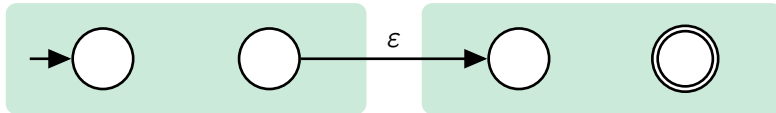
Sprache von regulären Ausdrücken

Beweis (Fortsetzung)

Seien



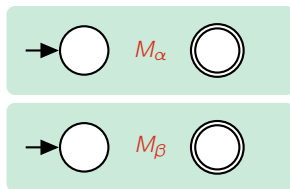
$M_{\alpha\beta} :=$



Sprache von regulären Ausdrücken

Beweis (Fortsetzung)

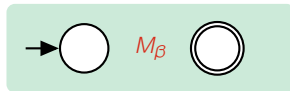
Seien



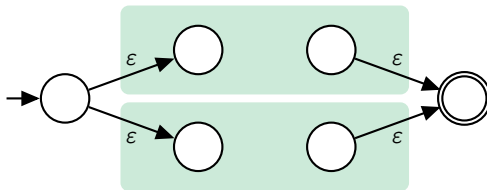
Sprache von regulären Ausdrücken

Beweis (Fortsetzung)

Seien



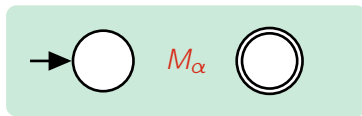
$M_{(\alpha|\beta)} :=$



Sprache von regulären Ausdrücken

Beweis (Fortsetzung)

Sei



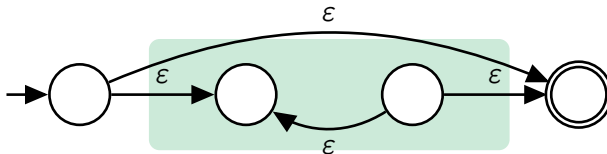
Sprache von regulären Ausdrücken

Beweis (Fortsetzung)

Sei



$M_{(\alpha)^*} :=$



Beispiel für den NFA zu einem regulären Ausdruck

Regulärer Ausdruck:

$(\varepsilon|(a|b)^*b(a|b))$

NFA mit ε -Übergängen:

Beispiel für den NFA zu einem regulären Ausdruck

Regulärer Ausdruck:

$(\epsilon|(a|b)^*b(a|b))$

NFA mit ϵ -Übergängen:

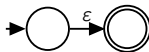


Beispiel für den NFA zu einem regulären Ausdruck

Regulärer Ausdruck:

$(\varepsilon|(a|b)^*b(a|b))$

NFA mit ε -Übergängen:

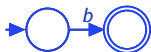
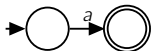
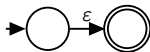


Beispiel für den NFA zu einem regulären Ausdruck

Regulärer Ausdruck:

$(\varepsilon|(a|b)^*b(a|b))$

NFA mit ε -Übergängen:

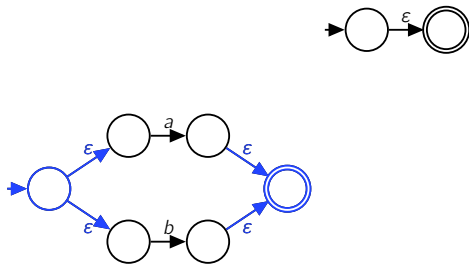


Beispiel für den NFA zu einem regulären Ausdruck

Regulärer Ausdruck:

$(\varepsilon | (a|b)^* b(a|b))$

NFA mit ε -Übergängen:

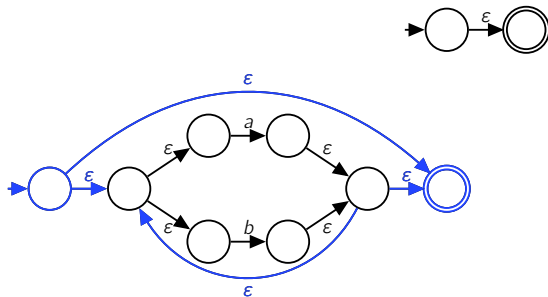


Beispiel für den NFA zu einem regulären Ausdruck

Regulärer Ausdruck:

$(\varepsilon | (a|b)^* b(a|b))$

NFA mit ε -Übergängen:

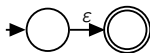
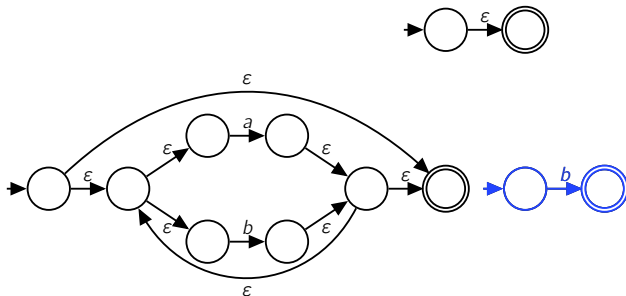


Beispiel für den NFA zu einem regulären Ausdruck

Regulärer Ausdruck:

$(\epsilon | (a|b)^* b (a|b))$

NFA mit ϵ -Übergängen:

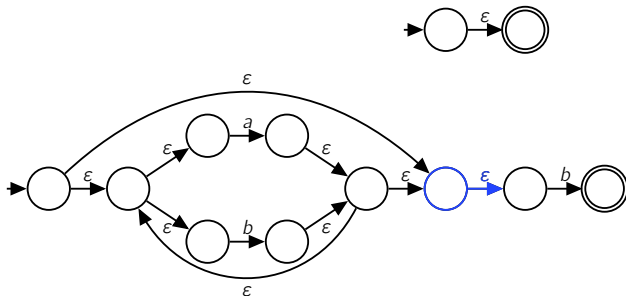


Beispiel für den NFA zu einem regulären Ausdruck

Regulärer Ausdruck:

$(\epsilon | (a|b)^* b(a|b))$

NFA mit ϵ -Übergängen:

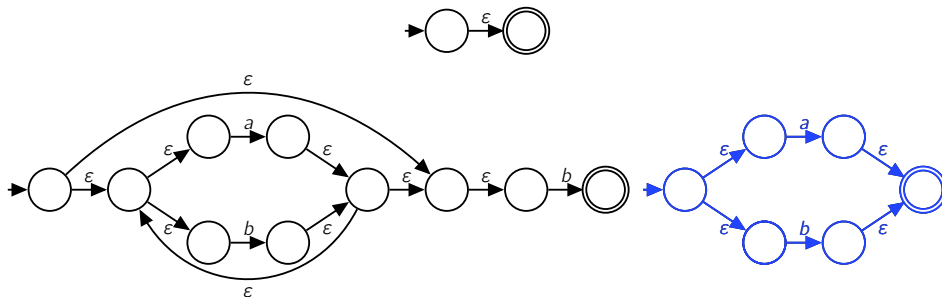


Beispiel für den NFA zu einem regulären Ausdruck

Regulärer Ausdruck:

$(\varepsilon|(a|b)^*b(a|b))$

NFA mit ε -Übergängen:

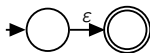
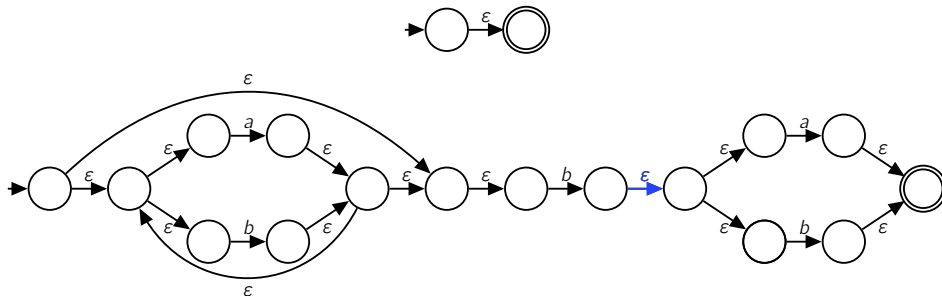


Beispiel für den NFA zu einem regulären Ausdruck

Regulärer Ausdruck:

$(\epsilon | (a|b)^* b(a|b))$

NFA mit ϵ -Übergängen:

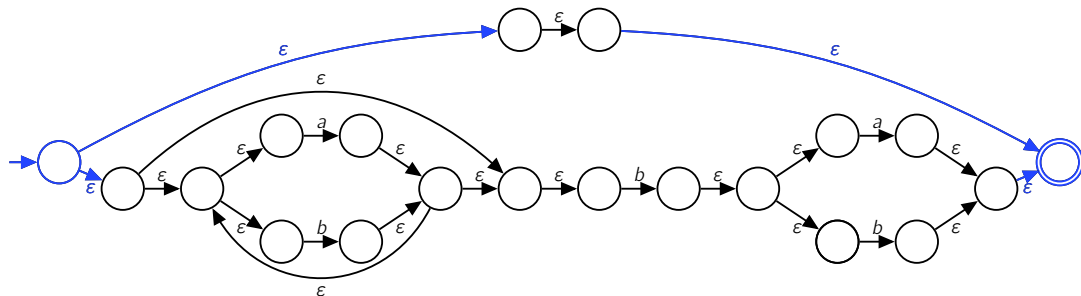


Beispiel für den NFA zu einem regulären Ausdruck

Regulärer Ausdruck:

$(\epsilon|(a|b)^*b(a|b))$

NFA mit ϵ -Übergängen:

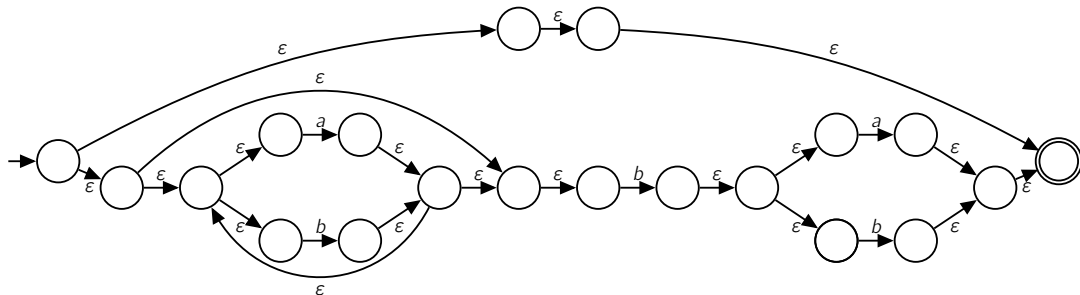


Beispiel für den NFA zu einem regulären Ausdruck

Regulärer Ausdruck:

$$(\varepsilon|(a|b)^*b(a|b))$$

NFA mit ε -Übergängen:



Sprache von regulären Ausdrücken

Beweis (Fortsetzung)

Es bleibt zu zeigen, dass $L(M_\alpha) = L(\alpha)$ für jeden regulären Ausdruck α gilt. Da $L(M_\alpha)$ regulär ist, heißt es dann, dass $L(\alpha)$ auch regulär ist.

Sprache von regulären Ausdrücken

Beweis (Fortsetzung)

Es bleibt zu zeigen, dass $L(M_\alpha) = L(\alpha)$ für jeden regulären Ausdruck α gilt. Da $L(M_\alpha)$ regulär ist, heißt es dann, dass $L(\alpha)$ auch regulär ist.

Induktion über die Größe von α .

Sprache von regulären Ausdrücken

Beweis (Fortsetzung)

Es bleibt zu zeigen, dass $L(M_\alpha) = L(\alpha)$ für jeden regulären Ausdruck α gilt. Da $L(M_\alpha)$ regulär ist, heißt es dann, dass $L(\alpha)$ auch regulär ist.

Induktion über die Größe von α .

- Fall α ist $\emptyset$, ε oder a : Offensichtlich.

Sprache von regulären Ausdrücken

Beweis (Fortsetzung)

Es bleibt zu zeigen, dass $L(M_\alpha) = L(\alpha)$ für jeden regulären Ausdruck α gilt. Da $L(M_\alpha)$ regulär ist, heißt es dann, dass $L(\alpha)$ auch regulär ist.

Induktion über die Größe von α .

- ▶ Fall α ist $\emptyset$, ε oder a : Offensichtlich.
- ▶ Fall α ist $\beta\gamma$: Die Induktionshypothese liefert $L(M_\beta) = L(\beta)$ und $L(M_\gamma) = L(\gamma)$. Die Konstruktion von $M_{\beta\gamma}$ sicherstellt, dass $L(M_{\beta\gamma}) = L(M_\beta)L(M_\gamma)$. Daher $L(M_{\beta\gamma}) = L(M_\beta)L(M_\gamma) = L(\beta)L(\gamma) = L(\beta\gamma)$.

Sprache von regulären Ausdrücken

Beweis (Fortsetzung)

Es bleibt zu zeigen, dass $L(M_\alpha) = L(\alpha)$ für jeden regulären Ausdruck α gilt. Da $L(M_\alpha)$ regulär ist, heißt es dann, dass $L(\alpha)$ auch regulär ist.

Induktion über die Größe von α .

- ▶ Fall α ist $\emptyset$, ε oder a : Offensichtlich.
- ▶ Fall α ist $\beta\gamma$: Die Induktionshypothese liefert $L(M_\beta) = L(\beta)$ und $L(M_\gamma) = L(\gamma)$. Die Konstruktion von $M_{\beta\gamma}$ sicherstellt, dass $L(M_{\beta\gamma}) = L(M_\beta)L(M_\gamma)$. Daher $L(M_{\beta\gamma}) = L(M_\beta)L(M_\gamma) = L(\beta)L(\gamma) = L(\beta\gamma)$.
- ▶ Fall α ist $(\beta|\gamma)$: Die Induktionshypothese liefert $L(M_\beta) = L(\beta)$ und $L(M_\gamma) = L(\gamma)$. Die Konstruktion von $M_{\beta|\gamma}$ sicherstellt, dass $L(M_{(\beta|\gamma)}) = L(M_\beta) \cup L(M_\gamma)$. Daher $L(M_{(\beta|\gamma)}) = L(M_\beta) \cup L(M_\gamma) = L(\beta) \cup L(\gamma) = L(\beta|\gamma)$.

Sprache von regulären Ausdrücken

Beweis (Fortsetzung)

Es bleibt zu zeigen, dass $L(M_\alpha) = L(\alpha)$ für jeden regulären Ausdruck α gilt. Da $L(M_\alpha)$ regulär ist, heißt es dann, dass $L(\alpha)$ auch regulär ist.

Induktion über die Größe von α .

- ▶ Fall α ist $\emptyset$, ε oder a : Offensichtlich.
- ▶ Fall α ist $\beta\gamma$: Die Induktionshypothese liefert $L(M_\beta) = L(\beta)$ und $L(M_\gamma) = L(\gamma)$. Die Konstruktion von $M_{\beta\gamma}$ sicherstellt, dass $L(M_{\beta\gamma}) = L(M_\beta)L(M_\gamma)$. Daher $L(M_{\beta\gamma}) = L(M_\beta)L(M_\gamma) = L(\beta)L(\gamma) = L(\beta\gamma)$.
- ▶ Fall α ist $(\beta|\gamma)$: Die Induktionshypothese liefert $L(M_\beta) = L(\beta)$ und $L(M_\gamma) = L(\gamma)$. Die Konstruktion von $M_{\beta|\gamma}$ sicherstellt, dass $L(M_{(\beta|\gamma)}) = L(M_\beta) \cup L(M_\gamma)$. Daher $L(M_{(\beta|\gamma)}) = L(M_\beta) \cup L(M_\gamma) = L(\beta) \cup L(\gamma) = L(\beta|\gamma)$.
- ▶ Fall α ist $(\beta)^*$: Die Induktionshypothese liefert $L(M_\beta) = L(\beta)$. Die Konstruktion von $M_{(\beta)^*}$ sicherstellt, dass $L(M_{(\beta)^*}) = L(M_\beta)^*$. Daher $L(M_{(\beta)^*}) = L(M_\beta)^* = L(\beta)^* = L((\beta)^*)$.



Sprache von regulären Ausdrücken

Theorem (Satz von Kleene)

Reguläre Ausdrücke erzeugen genau die regulären Sprachen.

Beweis (Fortsetzung)

⊇ Für jede reguläre Sprache L gibt es einen regulären Ausdruck α mit $L(\alpha) = L$.

Sprache von regulären Ausdrücken

Theorem (Satz von Kleene)

Reguläre Ausdrücke erzeugen genau die regulären Sprachen.

Beweis (Fortsetzung)

⊇ Für jede reguläre Sprache L gibt es einen regulären Ausdruck α mit $L(\alpha) = L$.

Sei DFA $M = (\{z_1, \dots, z_n\}, \Sigma, \delta, z_1, E)$ mit $L(M) = L$.

Sprache von regulären Ausdrücken

Theorem (Satz von Kleene)

Reguläre Ausdrücke erzeugen genau die regulären Sprachen.

Beweis (Fortsetzung)

⊇ Für jede reguläre Sprache L gibt es einen regulären Ausdruck α mit $L(\alpha) = L$.

Sei DFA $M = (\{z_1, \dots, z_n\}, \Sigma, \delta, z_1, E)$ mit $L(M) = L$.

Wir konstruieren für M einen regulären Ausdruck α , sodass $L(\alpha) = L(M) = L$.

Sprache von regulären Ausdrücken

Theorem (Satz von Kleene)

Reguläre Ausdrücke erzeugen genau die regulären Sprachen.

Beweis (Fortsetzung)

⊇ Für jede reguläre Sprache L gibt es einen regulären Ausdruck α mit $L(\alpha) = L$.

Sei DFA $M = (\{z_1, \dots, z_n\}, \Sigma, \delta, z_1, E)$ mit $L(M) = L$.

Wir konstruieren für M einen regulären Ausdruck α , sodass $L(\alpha) = L(M) = L$.

Hilfsmittel: $\alpha_{i,j}^k$ erzeugt die Wörter, die von Zustand z_i zu Zustand z_j führen, ohne dabei Zwischenzustände mit Index größer als k zu nutzen.

Sprache von regulären Ausdrücken

Theorem (Satz von Kleene)

Reguläre Ausdrücke erzeugen genau die regulären Sprachen.

Beweis (Fortsetzung)

⊇ Für jede reguläre Sprache L gibt es einen regulären Ausdruck α mit $L(\alpha) = L$.

Sei DFA $M = (\{z_1, \dots, z_n\}, \Sigma, \delta, z_1, E)$ mit $L(M) = L$.

Wir konstruieren für M einen regulären Ausdruck α , sodass $L(\alpha) = L(M) = L$.

Hilfsmittel: $\alpha_{i,j}^k$ erzeugt die Wörter, die von Zustand z_i zu Zustand z_j führen, ohne dabei Zwischenzustände mit Index größer als k zu nutzen.

Daher

$$\alpha := \begin{cases} \emptyset & \text{falls } E = \emptyset \\ (\alpha_{1,j_1}^n | \dots | \alpha_{1,j_m}^n) & \text{falls } E = \{z_{j_1}, \dots, z_{j_m}\} \end{cases}$$

Sprache von regulären Ausdrücken

Beweis (Fortsetzung)

Zur Erinnerung: $\alpha_{i,j}^k$ erzeugt die Wörter, die von Zustand z_i zu Zustand z_j führen, ohne dabei Zwischenzustände mit Index größer als k zu nutzen.

Sprache von regulären Ausdrücken

Beweis (Fortsetzung)

Zur Erinnerung: $\alpha_{i,j}^k$ erzeugt die Wörter, die von Zustand z_i zu Zustand z_j führen, ohne dabei Zwischenzustände mit Index größer als k zu nutzen.

Wir müssen $\alpha_{i,j}^k$ konstruieren. Sei $\{a_1, \dots, a_q\} = \{a \in \Sigma \mid \delta(z_i, a) = z_j\}$.

Sprache von regulären Ausdrücken

Beweis (Fortsetzung)

Zur Erinnerung: $\alpha_{i,j}^k$ erzeugt die Wörter, die von Zustand z_i zu Zustand z_j führen, ohne dabei Zwischenzustände mit Index größer als k zu nutzen.

Wir müssen $\alpha_{i,j}^k$ konstruieren. Sei $\{a_1, \dots, a_q\} = \{a \in \Sigma \mid \delta(z_i, a) = z_j\}$.

$$\alpha_{i,j}^0 := \begin{cases} \emptyset & \text{falls } i \neq j \text{ und } q = 0 \\ (a_1 | \dots | a_q) & \text{falls } i \neq j \text{ und } q > 0 \\ (\varepsilon | a_1 | \dots | a_q) & \text{falls } i = j \end{cases}$$

$$\alpha_{i,j}^{k+1} := (\alpha_{i,j}^k | \alpha_{i,k+1}^k (\alpha_{k+1,k+1}^k)^* \alpha_{k+1,j}^k)$$

Sprache von regulären Ausdrücken

Beweis (Fortsetzung)

Zur Erinnerung: $\alpha_{i,j}^k$ erzeugt die Wörter, die von Zustand z_i zu Zustand z_j führen, ohne dabei Zwischenzustände mit Index größer als k zu nutzen.

Wir müssen $\alpha_{i,j}^k$ konstruieren. Sei $\{a_1, \dots, a_q\} = \{a \in \Sigma \mid \delta(z_i, a) = z_j\}$.

$$\alpha_{i,j}^0 := \begin{cases} \emptyset & \text{falls } i \neq j \text{ und } q = 0 \\ (a_1 | \dots | a_q) & \text{falls } i \neq j \text{ und } q > 0 \\ (\varepsilon | a_1 | \dots | a_q) & \text{falls } i = j \end{cases}$$

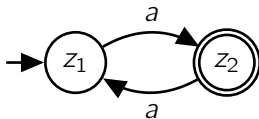
$$\alpha_{i,j}^{k+1} := (\alpha_{i,j}^k | \alpha_{i,k+1}^k (\alpha_{k+1,k+1}^k)^* \alpha_{k+1,j}^k)$$

Denn entweder läuft M ohne Zustand z_{k+1} zu besuchen (vgl. $\alpha_{i,j}^k$) oder der „Lauf“ kann in drei Teile gespalten werden:

1. „Lauf“ von z_i bis zum ersten Besuch des Zustands z_{k+1} (vgl. $\alpha_{i,k+1}^k$)
2. mehrmaliges, zyklisches Besuchen von $k+1$ (vgl. $\alpha_{k+1,k+1}^k$)
3. letztmaliges Verlassen von z_{k+1} und „Lauf“ bis zu z_j (vgl. $\alpha_{k+1,j}^k$).

Beispiel für den regulären Ausdruck zu einem DFA

DFA:



Regulärer Ausdruck dazu:

$$\begin{aligned}\alpha_{1,2}^2 &= (\alpha_{1,2}^1 | \alpha_{1,2}^1 (\alpha_{2,2}^1)^* \alpha_{2,2}^1) \\ &= ((a|\varepsilon(\varepsilon)^*a) | (a|\varepsilon(\varepsilon)^*a) (\varepsilon|a(\varepsilon)^*a)^* (\varepsilon|a(\varepsilon)^*a))\end{aligned}$$

denn

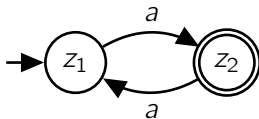
$$\alpha_{1,2}^1 = (\alpha_{1,2}^0 | \alpha_{1,1}^0 (\alpha_{1,1}^0)^* \alpha_{1,2}^0) = (a|\varepsilon(\varepsilon)^*a)$$

$$\alpha_{2,2}^1 = (\alpha_{2,2}^0 | \alpha_{2,1}^0 (\alpha_{1,1}^0)^* \alpha_{1,2}^0) = (\varepsilon|a(\varepsilon)^*a)$$

$$\alpha_{1,1}^0 = \varepsilon \quad \alpha_{2,2}^0 = \varepsilon \quad \alpha_{1,2}^0 = a \quad \alpha_{2,1}^0 = a$$

Beispiel für den regulären Ausdruck zu einem DFA

DFA:



Regulärer Ausdruck dazu:

$$\begin{aligned}\alpha_{1,2}^2 &= (\alpha_{1,2}^1 | \alpha_{1,2}^1 (\alpha_{2,2}^1)^* \alpha_{2,2}^1) \\ &= ((a|\varepsilon(\varepsilon)^*a) | (a|\varepsilon(\varepsilon)^*a)(\varepsilon|a(\varepsilon)^*a)^*(\varepsilon|a(\varepsilon)^*a)) \\ &\text{äquivalent zu } a(aa)^* \text{ (durch Vereinfachung)}\end{aligned}$$

denn

$$\alpha_{1,2}^1 = (\alpha_{1,2}^0 | \alpha_{1,1}^0 (\alpha_{1,1}^0)^* \alpha_{1,2}^0) = (a|\varepsilon(\varepsilon)^*a)$$

$$\alpha_{2,2}^1 = (\alpha_{2,2}^0 | \alpha_{2,1}^0 (\alpha_{1,1}^0)^* \alpha_{1,2}^0) = (\varepsilon|a(\varepsilon)^*a)$$

$$\alpha_{1,1}^0 = \varepsilon \quad \alpha_{2,2}^0 = \varepsilon \quad \alpha_{1,2}^0 = a \quad \alpha_{2,1}^0 = a$$

Sprache von regulären Ausdrücken

Beweis (Fortsetzung)

Es bleibt zu zeigen, dass $L(\alpha) = L(M)$.

Sprache von regulären Ausdrücken

Beweis (Fortsetzung)

Es bleibt zu zeigen, dass $L(\alpha) = L(M)$.

Für $w \in \Sigma^*$ und z_i, z_j mit $\tilde{\delta}(z_i, w) = z_j$ sei $visit_i(w) = q_1, \dots, q_m$ die Folge der besuchten Zustände (wobei $q_1 = z_i$ und $q_m = z_j$).

Sprache von regulären Ausdrücken

Beweis (Fortsetzung)

Es bleibt zu zeigen, dass $L(\alpha) = L(M)$.

Für $w \in \Sigma^*$ und z_i, z_j mit $\tilde{\delta}(z_i, w) = z_j$ sei $visit_i(w) = q_1, \dots, q_m$ die Folge der besuchten Zustände (wobei $q_1 = z_i$ und $q_m = z_j$).

Wir definieren:

$$L_{i,j}^k = \left\{ w \in \Sigma^* \mid \begin{array}{l} \tilde{\delta}(z_i, w) = z_j \text{ und } visit_i(w) = q_1, \dots, q_m, \\ \text{sodass für } 1 < l < m: \text{ wenn } q_l = z_p \text{ dann } p \leq k \end{array} \right\}$$

$L_{i,j}^k$ enthält die Wörter, die von Zustand z_i zu Zustand z_j führen ohne dabei Zwischenzustände mit Index größer als k zu benutzen.

Sprache von regulären Ausdrücken

Beweis (Fortsetzung)

Es bleibt zu zeigen, dass $L(\alpha) = L(M)$.

Für $w \in \Sigma^*$ und z_i, z_j mit $\tilde{\delta}(z_i, w) = z_j$ sei $visit_i(w) = q_1, \dots, q_m$ die Folge der besuchten Zustände (wobei $q_1 = z_i$ und $q_m = z_j$).

Wir definieren:

$$L_{i,j}^k = \left\{ w \in \Sigma^* \mid \begin{array}{l} \tilde{\delta}(z_i, w) = z_j \text{ und } visit_i(w) = q_1, \dots, q_m, \\ \text{sodass für } 1 < l < m: \text{ wenn } q_l = z_p \text{ dann } p \leq k \end{array} \right\}$$

$L_{i,j}^k$ enthält die Wörter, die von Zustand z_i zu Zustand z_j führen ohne dabei Zwischenzustände mit Index größer als k zu benutzen.

Mit Induktion über k können wir zeigen, dass $L(\alpha_{i,j}^k) = L_{i,j}^k$ für jedes $i, j, k \in \{1, \dots, n\}$. Siehe Skript.

Beweis (Fortsetzung)

Sei $E = \{z_{j_1}, \dots, z_{j_m}\}$.

Beweis (Fortsetzung)

Sei $E = \{z_{j_1}, \dots, z_{j_m}\}$.

Falls $m = 0$, dann $L(\alpha) = L(\emptyset) = \emptyset = L(M)$.

Beweis (Fortsetzung)

Sei $E = \{z_{j_1}, \dots, z_{j_m}\}$.

Falls $m = 0$, dann $L(\alpha) = L(\emptyset) = \emptyset = L(M)$.

Falls $m > 0$, dann

$L(\alpha)$

Beweis (Fortsetzung)

Sei $E = \{z_{j_1}, \dots, z_{j_m}\}$.

Falls $m = 0$, dann $L(\alpha) = L(\emptyset) = \emptyset = L(M)$.

Falls $m > 0$, dann

$$\begin{aligned} & L(\alpha) \\ = & L(\alpha_{1,j_1}^n | \dots | \alpha_{1,j_m}^n) \end{aligned}$$

Beweis (Fortsetzung)

Sei $E = \{z_{j_1}, \dots, z_{j_m}\}$.

Falls $m = 0$, dann $L(\alpha) = L(\emptyset) = \emptyset = L(M)$.

Falls $m > 0$, dann

$$\begin{aligned} & L(\alpha) \\ &= L(\alpha_{1,j_1}^n | \dots | \alpha_{1,j_m}^n) \\ &= L(\alpha_{1,j_1}^n) \cup \dots \cup L(\alpha_{1,j_m}^n) \end{aligned}$$

Beweis (Fortsetzung)

Sei $E = \{z_{j_1}, \dots, z_{j_m}\}$.

Falls $m = 0$, dann $L(\alpha) = L(\emptyset) = \emptyset = L(M)$.

Falls $m > 0$, dann

$$\begin{aligned} & L(\alpha) \\ &= L(\alpha_{1,j_1}^n | \dots | \alpha_{1,j_m}^n) \\ &= L(\alpha_{1,j_1}^n) \cup \dots \cup L(\alpha_{1,j_m}^n) \\ &= L_{1,j_1}^n \cup \dots \cup L_{1,j_m}^n \end{aligned}$$

Beweis (Fortsetzung)

Sei $E = \{z_{j_1}, \dots, z_{j_m}\}$.

Falls $m = 0$, dann $L(\alpha) = L(\emptyset) = \emptyset = L(M)$.

Falls $m > 0$, dann

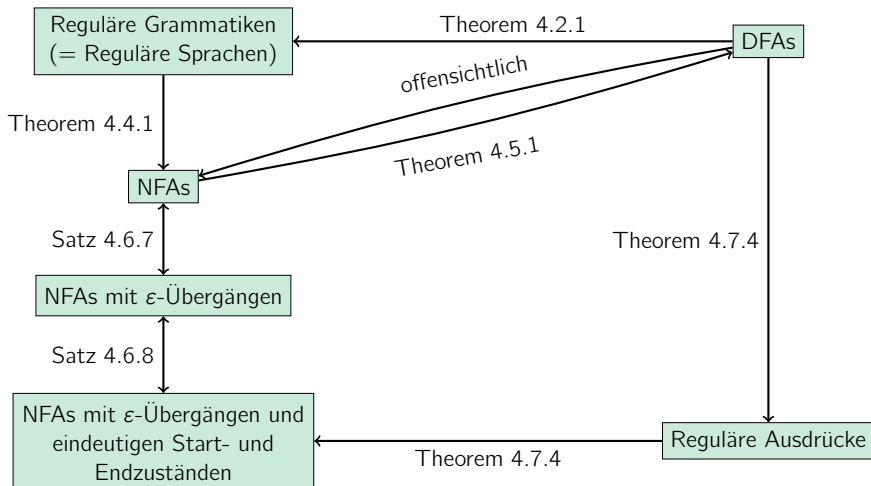
$$\begin{aligned} & L(\alpha) \\ &= L(\alpha_{1,j_1}^n | \dots | \alpha_{1,j_m}^n) \\ &= L(\alpha_{1,j_1}^n) \cup \dots \cup L(\alpha_{1,j_m}^n) \\ &= L_{1,j_1}^n \cup \dots \cup L_{1,j_m}^n \\ &= L(M) \end{aligned}$$



Anwendungen von regulären Ausdrücken

- ▶ Reguläre Ausdrücke werden zur **lexikalischen Analyse** von Programmiersprachen und Domain Specific Languages verwendet, aber auch für Textsuche und -ersetzung in Texteditoren sowie in Befehlszeilenprogramm wie `sed` und `AWK`.
- ▶ Tools wie **lex** (für C/C++), **ANTLR** (für Java) und **PLY** (für Python) generieren lexikalische Analyser („Lexer“) aus regulären Ausdrücken.
- ▶ Moderne Programmiersprachen unterstützen reguläre Ausdrücke entweder nativ oder mithilfe einer Bibliothek.

Zusammenfassung der Formalismen für reguläre Sprachen



Kanten ($\rightarrow$) zeigen Kodierungen.

4b

Das Pumping-Lemma für reguläre Sprachen

Prof. Dr. Jasmin Blanchette

Lehr- und Forschungseinheit für
Theoretische Informatik und Theorembeweisen

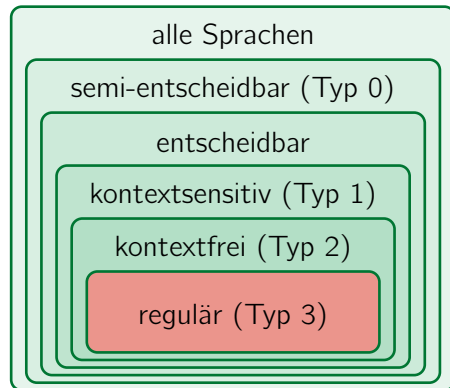
Stand: 14. Mai 2024
Basierend auf Folien von PD Dr. David Sabel



Hintergrund zum Pumping-Lemma

Formalismen zur Darstellung von
regulären Sprachen:

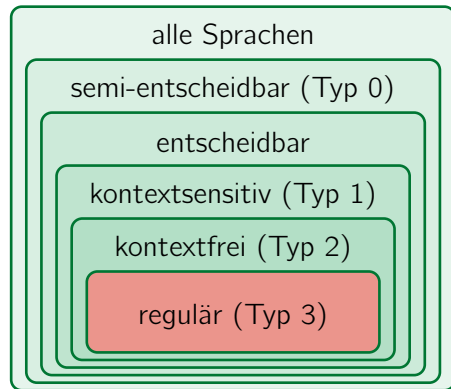
- ▶ reguläre Grammatiken
- ▶ endliche Automaten
- ▶ reguläre Ausdrücke



Hintergrund zum Pumping-Lemma

Formalismen zur Darstellung von
regulären Sprachen:

- ▶ reguläre Grammatiken
- ▶ endliche Automaten
- ▶ reguläre Ausdrücke



Wie zeigt man, dass eine formale Sprache **nicht regulär** ist?

Das **Pumping-Lemma** ist ein Werkzeug dafür.

Exkurs: Beweistechniken für Quantoren

Beweise für $\forall$ und $\exists$ variieren, je nachdem, ob die Aussage zu beweisen ist oder angenommen wird.

Auszug aus Aufgabenblatt 0:

	Um eine Aussage dieser Form zu beweisen ...	Wenn eine Aussage dieser Form angenommen wird ...
$\forall x, P(x)$	beweise, dass $P(a)$ für ein beliebiges a gilt	nimm $P(a)$ für jedes konkrete a an
$\exists x, P(x)$	gib ein konkretes a an und beweise $P(a)$	nimm ein beliebiges a an, für das $P(a)$ gilt

Definition

Eine mit $>$ ausgestattete Menge A hat die **EGE-Eigenschaft**, wenn gilt:
Für jedes $a \in A$ gibt es ein $b \in A$, sodass $b > a$
(d.h. $\forall a \in A, \exists b \in A, b > a$).

(„EGE“ steht für „*exists greater element*“.)

Exkurs: Beispiel für Beweistechniken für Quantoren

Satz

Die natürlichen Zahlen $\mathbb{N}$ mit $>$ haben die EGE-Eigenschaft.

Exkurs: Beispiel für Beweistechniken für Quantoren

Satz

Die natürlichen Zahlen $\mathbb{N}$ mit $>$ haben die EGE-Eigenschaft.

Beweis Wir zeigen: Für jede Zahl $a \in \mathbb{N}$ gibt es eine Zahl $b \in \mathbb{N}$, sodass $b > a$.

Exkurs: Beispiel für Beweistechniken für Quantoren

Satz

Die natürlichen Zahlen $\mathbb{N}$ mit $>$ haben die EGE-Eigenschaft.

Beweis Wir zeigen: Für jede Zahl $a \in \mathbb{N}$ gibt es eine Zahl $b \in \mathbb{N}$, sodass $b > a$.
Sei $a \in \mathbb{N}$ beliebig. Wir müssen dann beweisen, dass es $b \in \mathbb{N}$ gibt, sodass $b > a$.

Exkurs: Beispiel für Beweistechniken für Quantoren

Satz

Die natürlichen Zahlen $\mathbb{N}$ mit $>$ haben die EGE-Eigenschaft.

Beweis Wir zeigen: Für jede Zahl $a \in \mathbb{N}$ gibt es eine Zahl $b \in \mathbb{N}$, sodass $b > a$.
Sei $a \in \mathbb{N}$ beliebig. Wir müssen dann beweisen, dass es $b \in \mathbb{N}$ gibt, sodass $b > a$.
Wir wählen $b = a + 1$ und beweisen $a + 1 > a$, was offensichtlich gilt. $\square$

Exkurs: Beispiel für Beweistechniken für Quantoren

Satz

Die natürlichen Zahlen $\mathbb{N}$ mit $>$ haben die EGE-Eigenschaft.

Beweis Wir zeigen: Für jede Zahl $a \in \mathbb{N}$ gibt es eine Zahl $b \in \mathbb{N}$, sodass $b > a$.
Sei $a \in \mathbb{N}$ beliebig. Wir müssen dann beweisen, dass es $b \in \mathbb{N}$ gibt, sodass $b > a$.
Wir wählen $b = a + 1$ und beweisen $a + 1 > a$, was offensichtlich gilt. $\square$

Satz

Die nicht positiven ganzen Zahlen $\mathbb{Z}_{\leq 0}$ mit $>$ haben nicht die EGE-Eigenschaft.

Exkurs: Beispiel für Beweistechniken für Quantoren

Satz

Die natürlichen Zahlen $\mathbb{N}$ mit $>$ haben die EGE-Eigenschaft.

Beweis Wir zeigen: Für jede Zahl $a \in \mathbb{N}$ gibt es eine Zahl $b \in \mathbb{N}$, sodass $b > a$.
Sei $a \in \mathbb{N}$ beliebig. Wir müssen dann beweisen, dass es $b \in \mathbb{N}$ gibt, sodass $b > a$.
Wir wählen $b = a + 1$ und beweisen $a + 1 > a$, was offensichtlich gilt. $\square$

Satz

Die nicht positiven ganzen Zahlen $\mathbb{Z}_{\leq 0}$ mit $>$ haben nicht die EGE-Eigenschaft.

Beweis Durch Widerspruch. Wir nehmen an, dass $\mathbb{Z}_{\leq 0}$ die EGE-Eigenschaft hat.

Exkurs: Beispiel für Beweistechniken für Quantoren

Satz

Die natürlichen Zahlen $\mathbb{N}$ mit $>$ haben die EGE-Eigenschaft.

Beweis Wir zeigen: Für jede Zahl $a \in \mathbb{N}$ gibt es eine Zahl $b \in \mathbb{N}$, sodass $b > a$.
Sei $a \in \mathbb{N}$ beliebig. Wir müssen dann beweisen, dass es $b \in \mathbb{N}$ gibt, sodass $b > a$.
Wir wählen $b = a + 1$ und beweisen $a + 1 > a$, was offensichtlich gilt. $\square$

Satz

Die nicht positiven ganzen Zahlen $\mathbb{Z}_{\leq 0}$ mit $>$ haben nicht die EGE-Eigenschaft.

Beweis Durch Widerspruch. Wir nehmen an, dass $\mathbb{Z}_{\leq 0}$ die EGE-Eigenschaft hat.
D.h. für jede Zahl $a \in \mathbb{Z}_{\leq 0}$ gibt es eine Zahl $b \in \mathbb{Z}_{\leq 0}$, sodass $b > a$.

Exkurs: Beispiel für Beweistechniken für Quantoren

Satz

Die natürlichen Zahlen $\mathbb{N}$ mit $>$ haben die EGE-Eigenschaft.

Beweis Wir zeigen: Für jede Zahl $a \in \mathbb{N}$ gibt es eine Zahl $b \in \mathbb{N}$, sodass $b > a$.
Sei $a \in \mathbb{N}$ beliebig. Wir müssen dann beweisen, dass es $b \in \mathbb{N}$ gibt, sodass $b > a$.
Wir wählen $b = a + 1$ und beweisen $a + 1 > a$, was offensichtlich gilt. $\square$

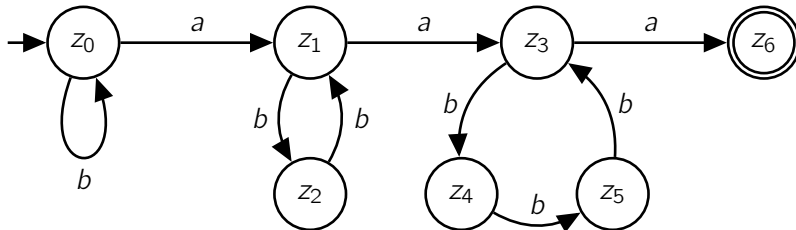
Satz

Die nicht positiven ganzen Zahlen $\mathbb{Z}_{\leq 0}$ mit $>$ haben nicht die EGE-Eigenschaft.

Beweis Durch Widerspruch. Wir nehmen an, dass $\mathbb{Z}_{\leq 0}$ die EGE-Eigenschaft hat.
D.h. für jede Zahl $a \in \mathbb{Z}_{\leq 0}$ gibt es eine Zahl $b \in \mathbb{Z}_{\leq 0}$, sodass $b > a$.
Wir wählen $a = 0$. Es gibt also eine Zahl $b \in \mathbb{Z}_{\leq 0}$, sodass $b > 0$. Aber 0 ist die größte Zahl in $\mathbb{Z}_{\leq 0}$. Widerspruch. $\square$

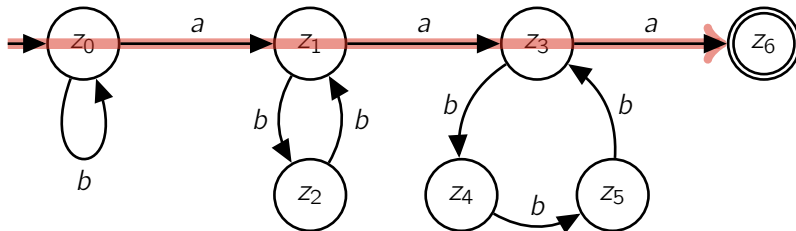
Intuition hinter dem Pumping-Lemma

Automat M :



Intuition hinter dem Pumping-Lemma

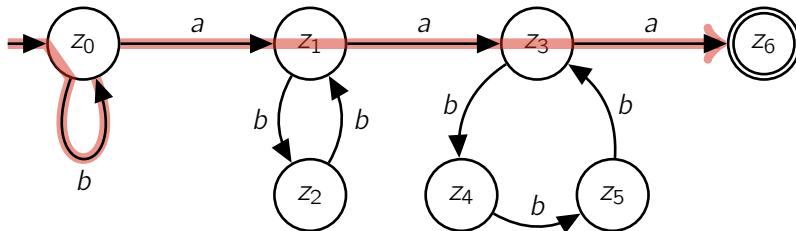
Automat M :



Von M erkannte Wörter der Länge 3,

Intuition hinter dem Pumping-Lemma

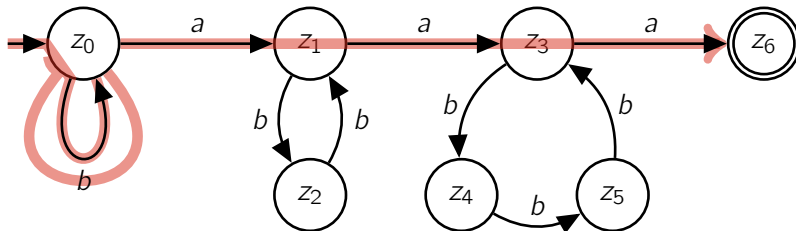
Automat M :



Von M erkannte Wörter der Länge 3, 4,

Intuition hinter dem Pumping-Lemma

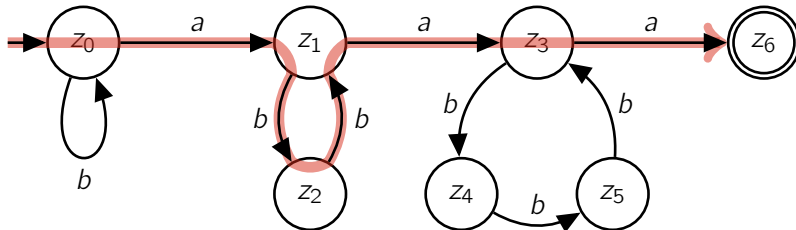
Automat M :



Von M erkannte Wörter der Länge 3, 4, 5,

Intuition hinter dem Pumping-Lemma

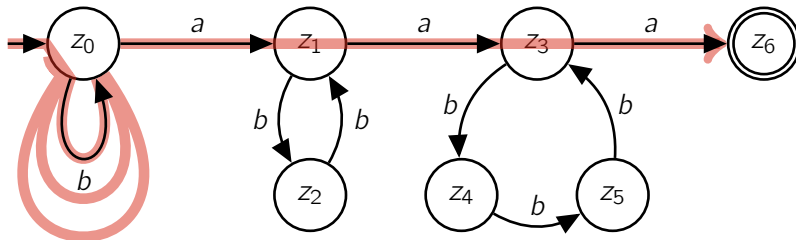
Automat M :



Von M erkannte Wörter der Länge 3, 4, 5,

Intuition hinter dem Pumping-Lemma

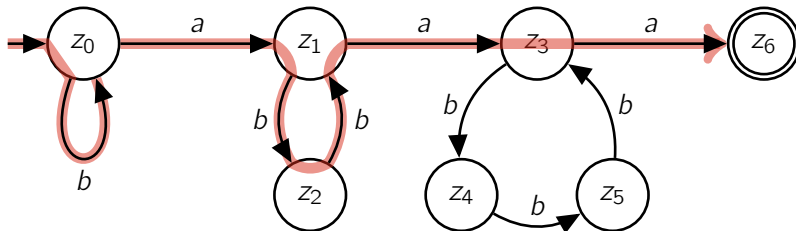
Automat M :



Von M erkannte Wörter der Länge 3, 4, 5, 6, ...

Intuition hinter dem Pumping-Lemma

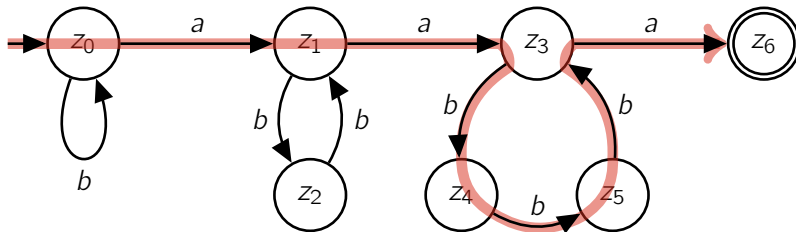
Automat M :



Von M erkannte Wörter der Länge 3, 4, 5, 6, ...

Intuition hinter dem Pumping-Lemma

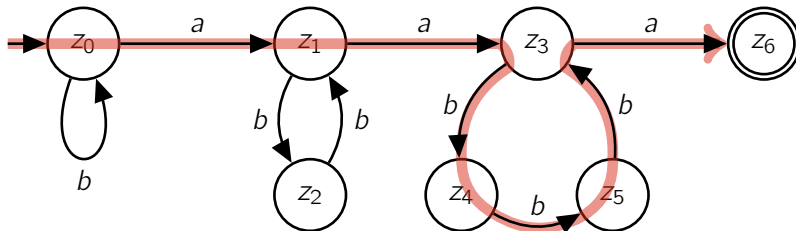
Automat M :



Von M erkannte Wörter der Länge 3, 4, 5, 6, ...

Intuition hinter dem Pumping-Lemma

Automat M :

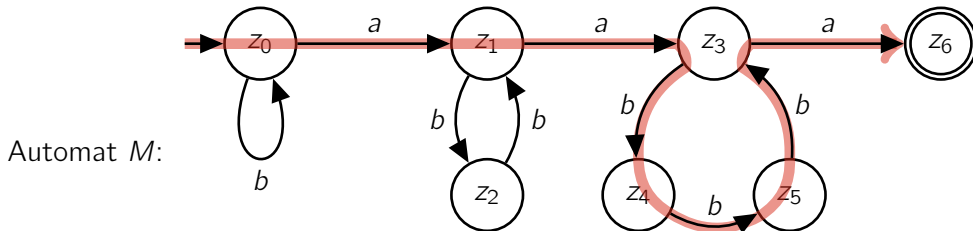


Von M erkannte Wörter der Länge 3, 4, 5, 6, ...

Beobachtungen:

1. Jedes Wort $z \in L(M)$ der Länge > 3 muss mindestens **eine Schleife durchlaufen**.

Intuition hinter dem Pumping-Lemma



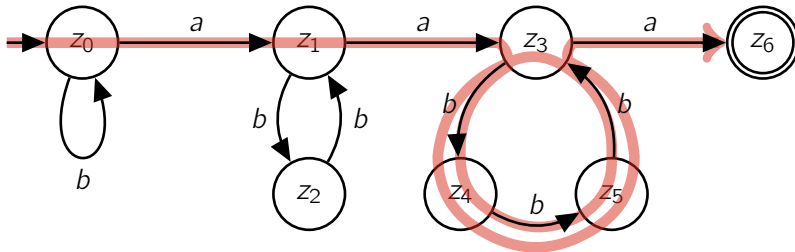
Von M erkannte Wörter der Länge 3, 4, 5, 6, ...

Beobachtungen:

1. Jedes Wort $z \in L(M)$ der Länge > 3 muss mindestens **eine Schleife durchlaufen**.
2. Wenn die **Schleife mehrfach durchgelaufen** wird, wird das entsprechende Wort immer noch erkannt. D.h. Wörter in $L(M)$ mit Länge > 3 können wir **aufpumpen** und verbleiben in $L(M)$.

Intuition hinter dem Pumping-Lemma

Automat M :

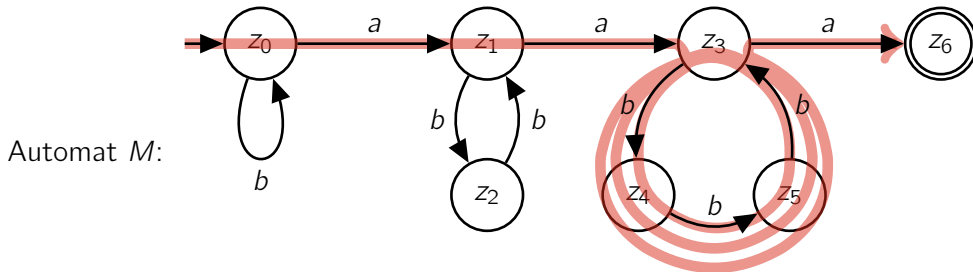


Von M erkannte Wörter der Länge 3, 4, 5, 6, ...

Beobachtungen:

1. Jedes Wort $z \in L(M)$ der Länge > 3 muss mindestens **eine Schleife durchlaufen**.
2. Wenn die **Schleife mehrfach durchgelaufen** wird, wird das entsprechende Wort immer noch erkannt. D.h. Wörter in $L(M)$ mit Länge > 3 können wir **aufpumpen** und verbleiben in $L(M)$.

Intuition hinter dem Pumping-Lemma

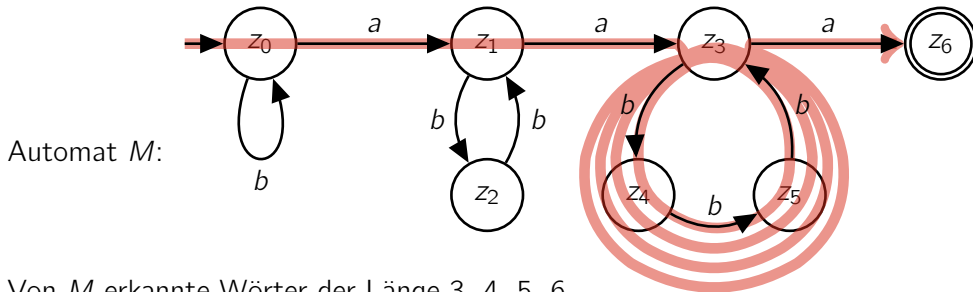


Von M erkannte Wörter der Länge 3, 4, 5, 6, ...

Beobachtungen:

1. Jedes Wort $z \in L(M)$ der Länge > 3 muss mindestens **eine Schleife durchlaufen**.
2. Wenn die **Schleife mehrfach durchgelaufen** wird, wird das entsprechende Wort immer noch erkannt. D.h. Wörter in $L(M)$ mit Länge > 3 können wir **aufpumpen** und verbleiben in $L(M)$.

Intuition hinter dem Pumping-Lemma



Von M erkannte Wörter der Länge 3, 4, 5, 6, ...

Beobachtungen:

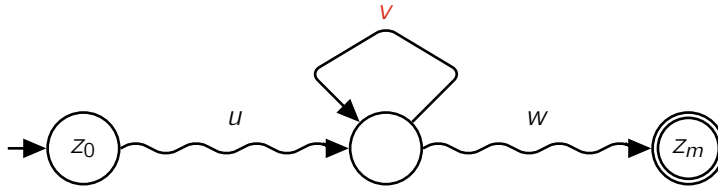
1. Jedes Wort $z \in L(M)$ der Länge > 3 muss mindestens **eine Schleife durchlaufen**.
2. Wenn die **Schleife mehrfach durchgelaufen** wird, wird das entsprechende Wort immer noch erkannt. D.h. Wörter in $L(M)$ mit Länge > 3 können wir **aufpumpen** und verbleiben in $L(M)$.

Intuition hinter dem Pumping-Lemma

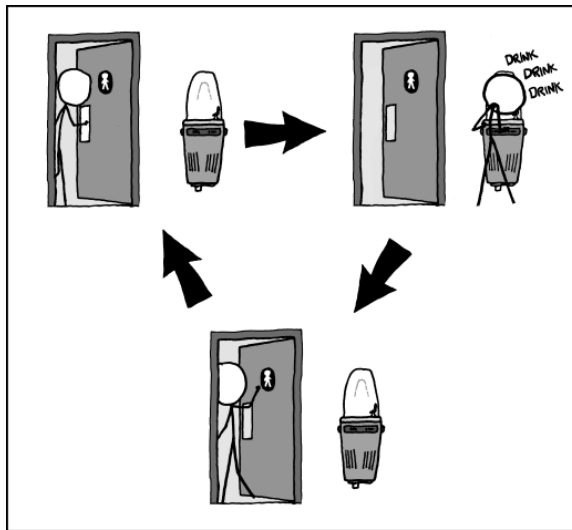
Gilt das allgemein?

Intuition hinter dem Pumping-Lemma

Gilt das allgemein? Ja



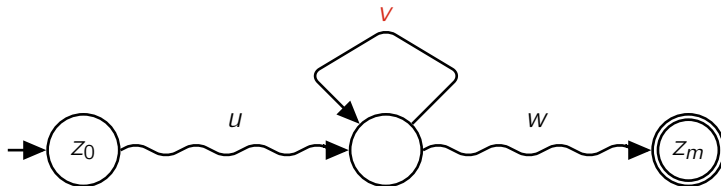
- ▶ Wenn ein DFA n Zustände hat, dann müssen akzeptierte Wörter der Länge $\geq n$ eine Schleife durchlaufen.
- ▶ Diese Wörter kann man aufpumpen: $uvw, uvvw, uvvww, \dots$.
Man kann auch die Schleife überspringen: uw .
Allgemein: $uv^i w$ für $i \in \mathbb{N}$ liegt in der erkannten Sprache.



I AVOID DRINKING FOUNTAINS OUTSIDE BATHROOMS
BECAUSE I'M AFRAID OF GETTING TRAPPED IN A LOOP.

xkcd.com/986/

Die Pumping-Eigenschaft für reguläre Sprachen



Definition

Eine Sprache L hat die **Pumping-Eigenschaft** (für reguläre Sprachen), wenn gilt: Es gibt eine Zahl $n \in \mathbb{N}_{>0}$, sodass jedes Wort $z \in L$, welches Mindestlänge n hat (d.h. $|z| \geq n$), als $z = uvvw$ geschrieben werden kann, sodass gilt:

1. $|uv| \leq n$
2. $|v| \geq 1$
3. für alle $i \in \mathbb{N}$: $uv^i w \in L$.

Das Pumping-Lemma für reguläre Sprachen

Lemma (Pumping-Lemma)

Jede reguläre Sprache hat die **Pumping-Eigenschaft**.

Das Pumping-Lemma für reguläre Sprachen

Lemma (Pumping-Lemma)

Jede reguläre Sprache hat die Pumping-Eigenschaft.

Beweis Sei L eine reguläre Sprache. Wir zeigen, dass es eine Zahl $n \in \mathbb{N}_{>0}$ gibt, sodass jedes $z \in L$ mit $|z| \geq n$ als $z = uvw$ geschrieben werden kann, sodass $|uv| \leq n$, $|v| \geq 1$ und $uv^i w \in L$ und für alle $i \in \mathbb{N}$.

Das Pumping-Lemma für reguläre Sprachen

Lemma (Pumping-Lemma)

Jede reguläre Sprache hat die Pumping-Eigenschaft.

Beweis Sei L eine reguläre Sprache. Wir zeigen, dass es eine Zahl $n \in \mathbb{N}_{>0}$ gibt, sodass jedes $z \in L$ mit $|z| \geq n$ als $z = uvw$ geschrieben werden kann, sodass $|uv| \leq n$, $|v| \geq 1$ und $uv^i w \in L$ und für alle $i \in \mathbb{N}$.

Sei $M = (Z, \Sigma, \delta, z_0, E)$ ein DFA mit $L(M) = L$.

Das Pumping-Lemma für reguläre Sprachen

Lemma (Pumping-Lemma)

Jede reguläre Sprache hat die Pumping-Eigenschaft.

Beweis Sei L eine reguläre Sprache. Wir zeigen, dass es eine Zahl $n \in \mathbb{N}_{>0}$ gibt, sodass jedes $z \in L$ mit $|z| \geq n$ als $z = uvw$ geschrieben werden kann, sodass $|uv| \leq n$, $|v| \geq 1$ und $uv^i w \in L$ und für alle $i \in \mathbb{N}$.

Sei $M = (Z, \Sigma, \delta, z_0, E)$ ein DFA mit $L(M) = L$.

Wir wählen $n = |Z|$. Sei $z \in L$ mit $|z| \geq n$.

Das Pumping-Lemma für reguläre Sprachen

Lemma (Pumping-Lemma)

Jede reguläre Sprache hat die Pumping-Eigenschaft.

Beweis Sei L eine reguläre Sprache. Wir zeigen, dass es eine Zahl $n \in \mathbb{N}_{>0}$ gibt, sodass jedes $z \in L$ mit $|z| \geq n$ als $z = uvw$ geschrieben werden kann, sodass $|uv| \leq n$, $|v| \geq 1$ und $uv^i w \in L$ und für alle $i \in \mathbb{N}$.

Sei $M = (Z, \Sigma, \delta, z_0, E)$ ein DFA mit $L(M) = L$.

Wir wählen $n = |Z|$. Sei $z \in L$ mit $|z| \geq n$.

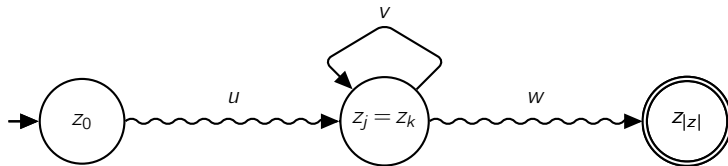
Sei $z_0, \dots, z_{|z|}$ der Lauf für z , mit $z_{|z|} \in E$.

Spätestens nach Lesen von $n (= |Z|)$ Zeichen wird einen Zustand erneut besucht.

Sei z_k (mit $k \leq n$) der erste Zustand, der bereits besucht wurde.

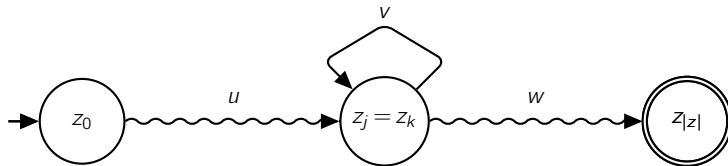
Das Pumping-Lemma für reguläre Sprachen

Beweis (Fortsetzung) Daher gibt es $j < k$, sodass $z_k = z_j$, k ist minimal und $z = uvw$ mit



Das Pumping-Lemma für reguläre Sprachen

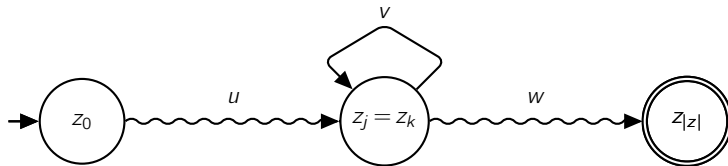
Beweis (Fortsetzung) Daher gibt es $j < k$, sodass $z_k = z_j$, k ist minimal und $z = uvw$ mit



Wir zeigen nun die drei geforderten Eigenschaften der Zerlegung:

Das Pumping-Lemma für reguläre Sprachen

Beweis (Fortsetzung) Daher gibt es $j < k$, sodass $z_k = z_j$, k ist minimal und $z = uvw$ mit

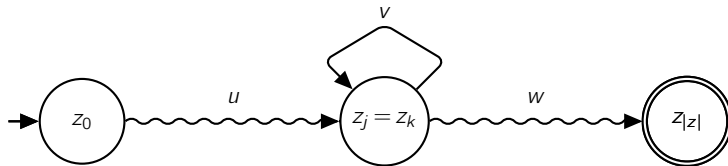


Wir zeigen nun die drei geforderten Eigenschaften der Zerlegung:

- $|v| \geq 1$: folgt aus $j < k$.

Das Pumping-Lemma für reguläre Sprachen

Beweis (Fortsetzung) Daher gibt es $j < k$, sodass $z_k = z_j$, k ist minimal und $z = uvw$ mit

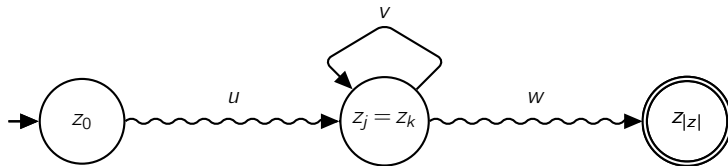


Wir zeigen nun die drei geforderten Eigenschaften der Zerlegung:

- ▶ $|v| \geq 1$: folgt aus $j < k$.
- ▶ $|uv| \leq n$: folgt aus $k \leq n$.

Das Pumping-Lemma für reguläre Sprachen

Beweis (Fortsetzung) Daher gibt es $j < k$, sodass $z_k = z_j$, k ist minimal und $z = uvw$ mit



Wir zeigen nun die drei geforderten Eigenschaften der Zerlegung:

- ▶ $|v| \geq 1$: folgt aus $j < k$.
- ▶ $|uv| \leq n$: folgt aus $k \leq n$.
- ▶ $uv^i w \in L(M)$ für jedes $i \in \mathbb{N}$: lässt sich im obigen Diagramm lesen.

□

Das Paradoxon der endlichen Sprachen

Endliche Sprachen sind regulär.

Warum erfüllen sie die Pumping-Eigenschaft?

Das Paradoxon der endlichen Sprachen

Endliche Sprachen sind regulär.

Warum erfüllen sie die Pumping-Eigenschaft?

Definition

Eine Sprache L hat die Pumping-Eigenschaft (für reguläre Sprachen), wenn gilt:
Es gibt eine Zahl $n \in \mathbb{N}_{>0}$, sodass jedes Wort $z \in L$, welches Mindestlänge n hat (d.h. $|z| \geq n$), als $z = uvw$ geschrieben werden kann, sodass gilt:

1. $|uv| \leq n$
2. $|v| \geq 1$
3. für alle $i \in \mathbb{N}$: $uv^i w \in L$.

Das Paradoxon der endlichen Sprachen

Endliche Sprachen sind regulär.

Warum erfüllen sie die Pumping-Eigenschaft?

Definition

Eine Sprache L hat die Pumping-Eigenschaft (für reguläre Sprachen), wenn gilt:
Es gibt eine Zahl $n \in \mathbb{N}_{>0}$, sodass jedes Wort $z \in L$, welches Mindestlänge n hat (d.h. $|z| \geq n$), als $z = uvw$ geschrieben werden kann, sodass gilt:

1. $|uv| \leq n$
2. $|v| \geq 1$
3. für alle $i \in \mathbb{N}$: $uv^i w \in L$.

Wähle n größer als die Länge des längsten Worts.

Dann ist die Eigenschaft trivial erfüllt.

Anwendung des Pumping-Lemmas

Sei L eine Sprache, die wir als nicht regulär beweisen wollen.

Anwendung des Pumping-Lemmas

Sei L eine Sprache, die wir als nicht regulär beweisen wollen.

Pumping-Lemma:

L ist regulär $\implies L$ hat die Pumping-Eigenschaft

Anwendung des Pumping-Lemmas

Sei L eine Sprache, die wir als nicht regulär beweisen wollen.

Pumping-Lemma:

L ist regulär $\implies L$ hat die Pumping-Eigenschaft

Kontraposition:

L hat nicht die Pumping-Eigenschaft $\implies L$ ist nicht regulär

Anwendung des Pumping-Lemmas

Sei L eine Sprache, die wir als nicht regulär beweisen wollen.

Pumping-Lemma:

L ist regulär $\implies L$ hat die Pumping-Eigenschaft

Kontraposition:

L hat nicht die Pumping-Eigenschaft $\implies L$ ist nicht regulär

Beweisstrategie für die Aussage „ L ist nicht regulär“:

1. Durch die Kontraposition reicht es zu zeigen, dass L die Pumping-Eigenschaft nicht hat.
2. Zeige dies durch Widerspruch: Nehme an, dass L die Pumping-Eigenschaft hat.
3. Leite einen Widerspruch her.
4. D.h. L ist nicht regulär.

Nichtregularität zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{a^j b^j \mid j \in \mathbb{N}\}$ ist nicht regulär.

Nichtregularität zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{a^j b^j \mid j \in \mathbb{N}\}$ ist nicht regulär.

Beweis Um Nichtregularität von L zu zeigen, reicht es durch das Pumping-Lemma zu zeigen, dass L die Pumping-Eigenschaft nicht hat.

Nichtregularität zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{a^j b^j \mid j \in \mathbb{N}\}$ ist nicht regulär.

Beweis Um Nichtregularität von L zu zeigen, reicht es durch das Pumping-Lemma zu zeigen, dass L die Pumping-Eigenschaft nicht hat.

Durch Widerspruch: Wir nehmen an, dass L die Pumping-Eigenschaft hat.

Nichtregularität zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{a^j b^j \mid j \in \mathbb{N}\}$ ist nicht regulär.

Beweis Um Nichtregularität von L zu zeigen, reicht es durch das Pumping-Lemma zu zeigen, dass L die Pumping-Eigenschaft nicht hat.

Durch Widerspruch: Wir nehmen an, dass L die Pumping-Eigenschaft hat.

Sei $n \in \mathbb{N}_{>0}$ beliebig.

Nichtregularität zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{a^j b^j \mid j \in \mathbb{N}\}$ ist nicht regulär.

Beweis Um Nichtregularität von L zu zeigen, reicht es durch das Pumping-Lemma zu zeigen, dass L die Pumping-Eigenschaft nicht hat.

Durch Widerspruch: Wir nehmen an, dass L die Pumping-Eigenschaft hat.

Sei $n \in \mathbb{N}_{>0}$ beliebig.

Wir wählen $z \in L$ als $z = a^n b^n$. Damit ist auch $|z| \geq n$ erfüllt.

Nichtregularität zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{a^j b^j \mid j \in \mathbb{N}\}$ ist nicht regulär.

Beweis Um Nichtregularität von L zu zeigen, reicht es durch das Pumping-Lemma zu zeigen, dass L die Pumping-Eigenschaft nicht hat.

Durch Widerspruch: Wir nehmen an, dass L die Pumping-Eigenschaft hat.

Sei $n \in \mathbb{N}_{>0}$ beliebig.

Wir wählen $z \in L$ als $z = a^n b^n$. Damit ist auch $|z| \geq n$ erfüllt.

Sei $z = uvw$ eine beliebige Zerlegung von z , sodass $|uv| \leq n$, $|v| \geq 1$ und $uv^i w \in L$ für jedes $i \in \mathbb{N}$. Dann ist $u = a^r$, $v = a^s$ mit $r + s \leq n$, $s \geq 1$ und $w = a^t b^n$ mit $r + s + t = n$.

Nichtregularität zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{a^j b^j \mid j \in \mathbb{N}\}$ ist nicht regulär.

Beweis Um Nichtregularität von L zu zeigen, reicht es durch das Pumping-Lemma zu zeigen, dass L die Pumping-Eigenschaft nicht hat.

Durch Widerspruch: Wir nehmen an, dass L die Pumping-Eigenschaft hat.

Sei $n \in \mathbb{N}_{>0}$ beliebig.

Wir wählen $z \in L$ als $z = a^n b^n$. Damit ist auch $|z| \geq n$ erfüllt.

Sei $z = uvw$ eine beliebige Zerlegung von z , sodass $|uv| \leq n$, $|v| \geq 1$ und $uv^i w \in L$ für jedes $i \in \mathbb{N}$. Dann ist $u = a^r$, $v = a^s$ mit $r + s \leq n$, $s \geq 1$ und $w = a^t b^n$ mit $r + s + t = n$.

Wir betrachten das Wort uv^2w . Es gilt $uv^2w = a^r a^s a^s a^t b^n = a^{n+s} b^n \notin L$, da $s \geq 1$.

Nach Annahme gilt aber $uv^i w \in L$ für jedes $i \in \mathbb{N}$, insbesondere für $i = 2$.

Widerspruch. □

Nichtregularität zeigen mit dem Pumping-Lemma

Es geht kürzer:

Satz

Die Sprache $L = \{a^j b^j \mid j \in \mathbb{N}\}$ ist nicht regulär.

Beweis Mit dem Pumping-Lemma.

Sei $n \in \mathbb{N}_{>0}$ beliebig.

Wir wählen $z \in L$ als $z = a^n b^n$ mit $|z| \geq n$.

Sei $z = uvw$ eine beliebige Zerlegung von z , sodass $|uv| \leq n$, $|v| \geq 1$ und $uv^i w \in L$ für jedes $i \in \mathbb{N}$. Dann ist $u = a^r$, $v = a^s$ mit $r + s \leq n$, $s \geq 1$ und $w = a^t b^n$ mit $r + s + t = n$.

Wir wählen $i = 2$. Es gilt $uv^2 w = a^r a^s a^s a^t b^n = a^{n+s} b^n \notin L$, da $s \geq 1$.

Widerspruch. □

Das Pumping-Lemma als Spiel

Sei L die Sprache.

Schritte:

1. Der **Gegner** wählt die Zahl $n \in \mathbb{N}_{>0}$.
2. **Wir** wählen das Wort $z \in L$ mit $|z| \geq n$.
3. Der **Gegner** wählt die Zerlegung $z = uvw$ mit $|uv| \leq n$ und $|v| \geq 1$.
4. **Wir** gewinnen das Spiel, wenn wir ein $i \in \mathbb{N}$ angeben können, sodass $uv^i w \notin L$.

Das Pumping-Lemma als Spiel

Sei L die Sprache.

Schritte:

1. Der **Gegner** wählt die Zahl $n \in \mathbb{N}_{>0}$.
2. **Wir** wählen das Wort $z \in L$ mit $|z| \geq n$.
3. Der **Gegner** wählt die Zerlegung $z = uvw$ mit $|uv| \leq n$ und $|v| \geq 1$.
4. **Wir** gewinnen das Spiel, wenn wir ein $i \in \mathbb{N}$ angeben können, sodass $uv^i w \notin L$.

Wenn **wir** das Spiel **für alle Wahlmöglichkeiten des Gegners gewinnen**, dann haben wir nachgewiesen, dass L nicht regulär ist.

Das Pumping-Lemma als Spiel

Satz

Die Sprache $L = \{a^j b^j \mid j \in \mathbb{N}\}$ ist nicht regulär.

Beweis Wir zeigen, dass wir das eben eingeführte Spiel stets gewinnen:

1. Sei $n \in \mathbb{N}_{>0}$ vom **Gegner** gewählt.
2. **Wir** wählen $z \in L$ als $z = a^n b^n$ mit $|z| \geq n$.
3. Der **Gegner** wählt die Zerlegung $z = uvw$, sodass $|uv| \leq n$ und $|v| \geq 1$. Dann ist $u = a^r$, $v = a^s$ mit $r + s \leq n$, $s \geq 1$ und $w = a^t b^n$ mit $r + s + t = n$.
4. **Wir** wählen $i = 2$. Es gilt $uv^2w = a^r a^s a^s a^t b^n = a^{n+s} b^n \notin L$, da $s \geq 1$.
Also gewinnen wir. □

Nichtregularität zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{a^p \mid p \text{ ist Primzahl}\}$ ist nicht regulär.

Nichtregularität zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{a^p \mid p \text{ ist Primzahl}\}$ ist nicht regulär.

Beweis Mit dem Pumping-Lemma.

Nichtregularität zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{a^p \mid p \text{ ist Primzahl}\}$ ist nicht regulär.

Beweis Mit dem Pumping-Lemma.

Sei $n \in \mathbb{N}_{>0}$ beliebig.

Nichtregularität zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{a^p \mid p \text{ ist Primzahl}\}$ ist nicht regulär.

Beweis Mit dem Pumping-Lemma.

Sei $n \in \mathbb{N}_{>0}$ beliebig.

Wir wählen $z \in L$ als $z = a^p$, wobei p die nächste Primzahl ist, die größer gleich n ist. Wir haben $|z| = p \geq n$.

Nichtregularität zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{a^p \mid p \text{ ist Primzahl}\}$ ist nicht regulär.

Beweis Mit dem Pumping-Lemma.

Sei $n \in \mathbb{N}_{>0}$ beliebig.

Wir wählen $z \in L$ als $z = a^p$, wobei p die nächste Primzahl ist, die größer gleich n ist. Wir haben $|z| = p \geq n$.

Sei $z = uvw$ eine beliebige Zerlegung von z , sodass $|uv| \leq n$, $|v| \geq 1$ und $uv^i w \in L$ für jedes $i \in \mathbb{N}$. Dann $u = a^r$, $v = a^s$ (mit $s \geq 1$) und $w = a^t$.

Nichtregularität zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{a^p \mid p \text{ ist Primzahl}\}$ ist nicht regulär.

Beweis Mit dem Pumping-Lemma.

Sei $n \in \mathbb{N}_{>0}$ beliebig.

Wir wählen $z \in L$ als $z = a^p$, wobei p die nächste Primzahl ist, die größer gleich n ist. Wir haben $|z| = p \geq n$.

Sei $z = uvw$ eine beliebige Zerlegung von z , sodass $|uv| \leq n$, $|v| \geq 1$ und $uv^i w \in L$ für jedes $i \in \mathbb{N}$. Dann $u = a^r$, $v = a^s$ (mit $s \geq 1$) und $w = a^t$.

Wir wählen $i = p + 1$. Es gilt $uv^{p+1}w \notin L$, denn

$uv^{p+1}w = a^r(a^s)^{p+1}a^t = a^{r+s \cdot (p+1)+t} = a^{r+s \cdot p+s+t} = a^{s \cdot p+p} = a^{p \cdot (s+1)}$ und für $s \geq 1$ folgt, dass $p \cdot (s + 1)$ keine Primzahl sein kann. Widerspruch. $\square$

Nichtregularität zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{a^{n^2} \mid n \in \mathbb{N}\}$ ist nicht regulär.

Beweis Mit dem Pumping-Lemma.

Nichtregularität zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{a^{n^2} \mid n \in \mathbb{N}\}$ ist nicht regulär.

Beweis Mit dem Pumping-Lemma.

Sei $n \in \mathbb{N}_{>0}$ beliebig.

Nichtregularität zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{a^{n^2} \mid n \in \mathbb{N}\}$ ist nicht regulär.

Beweis Mit dem Pumping-Lemma.

Sei $n \in \mathbb{N}_{>0}$ beliebig.

Wir wählen $z = a^{n^2} \in L$ mit $|z| = n^2 \geq n$.

Nichtregularität zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{a^{n^2} \mid n \in \mathbb{N}\}$ ist nicht regulär.

Beweis Mit dem Pumping-Lemma.

Sei $n \in \mathbb{N}_{>0}$ beliebig.

Wir wählen $z = a^{n^2} \in L$ mit $|z| = n^2 \geq n$.

Sei $z = uvw$ eine beliebige Zerlegung von z , sodass $|uv| \leq n$, $|v| \geq 1$ und $uv^i w \in L$ für jedes $i \in \mathbb{N}$.

Nichtregularität zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{a^{n^2} \mid n \in \mathbb{N}\}$ ist nicht regulär.

Beweis Mit dem Pumping-Lemma.

Sei $n \in \mathbb{N}_{>0}$ beliebig.

Wir wählen $z = a^{n^2} \in L$ mit $|z| = n^2 \geq n$.

Sei $z = uvw$ eine beliebige Zerlegung von z , sodass $|uv| \leq n$, $|v| \geq 1$ und $uv^i w \in L$ für jedes $i \in \mathbb{N}$.

Wir wählen $i = 2$. Wir betrachten $uv^2w = a^k$.

- ▶ $1 + n^2 \leq k$ (denn $|v| \geq 1$)
- ▶ $k \leq n^2 + n$ (denn $|uv| \leq n$ und daher $|v| \leq n$)

Nichtregularität zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{a^{n^2} \mid n \in \mathbb{N}\}$ ist nicht regulär.

Beweis Mit dem Pumping-Lemma.

Sei $n \in \mathbb{N}_{>0}$ beliebig.

Wir wählen $z = a^{n^2} \in L$ mit $|z| = n^2 \geq n$.

Sei $z = uvw$ eine beliebige Zerlegung von z , sodass $|uv| \leq n$, $|v| \geq 1$ und $uv^i w \in L$ für jedes $i \in \mathbb{N}$.

Wir wählen $i = 2$. Wir betrachten $uv^2w = a^k$.

- ▶ $1 + n^2 \leq k$ (denn $|v| \geq 1$)
- ▶ $k \leq n^2 + n$ (denn $|uv| \leq n$ und daher $|v| \leq n$)

D.h. wir haben $n^2 < k \leq n^2 + n = (n+1) \cdot n < (n+1)^2$.

Dann kann k keine Quadratzahl sein. Daher gilt $uv^2w \notin L$. Widerspruch. □

Nichtregularität zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{w \in \{a, b\}^* \mid w \text{ ist ein Palindrom}\}$ ist nicht regulär.

Beweis Mit dem Pumping-Lemma.

Nichtregularität zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{w \in \{a, b\}^* \mid w \text{ ist ein Palindrom}\}$ ist nicht regulär.

Beweis Mit dem Pumping-Lemma.

Sei $n \in \mathbb{N}_{>0}$ beliebig.

Nichtregularität zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{w \in \{a, b\}^* \mid w \text{ ist ein Palindrom}\}$ ist nicht regulär.

Beweis Mit dem Pumping-Lemma.

Sei $n \in \mathbb{N}_{>0}$ beliebig.

Wir wählen $z = a^n b a^n \in L$ mit $|w| = 2n + 1 \geq n$.

Nichtregularität zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{w \in \{a, b\}^* \mid w \text{ ist ein Palindrom}\}$ ist nicht regulär.

Beweis Mit dem Pumping-Lemma.

Sei $n \in \mathbb{N}_{>0}$ beliebig.

Wir wählen $z = a^n b a^n \in L$ mit $|w| = 2n + 1 \geq n$.

Sei $z = uvw$ eine beliebige Zerlegung von z , sodass $|uv| \leq n$, $|v| \geq 1$ und $uv^i w \in L$ für jedes $i \in \mathbb{N}$.

Nichtregularität zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{w \in \{a, b\}^* \mid w \text{ ist ein Palindrom}\}$ ist nicht regulär.

Beweis Mit dem Pumping-Lemma.

Sei $n \in \mathbb{N}_{>0}$ beliebig.

Wir wählen $z = a^n b a^n \in L$ mit $|z| = 2n + 1 \geq n$.

Sei $z = uvw$ eine beliebige Zerlegung von z , sodass $|uv| \leq n$, $|v| \geq 1$ und $uv^i w \in L$ für jedes $i \in \mathbb{N}$.

Wir wählen $i = 0$. Es gilt $uv^0 w \notin L$, denn $uv^0 w = a^k b a^n$ mit $k = n - |v| < n$ ist kein Palindrom. Widerspruch. $\square$

Pumping-Eigenschaft ist nicht hinreichend

Lemma

Es gibt Sprachen, die die Pumping-Eigenschaft haben aber nicht regulär sind.
Die Sprache $L = \{a^j b^k c^k \mid j, k \in \mathbb{N}_{>0}\}$ ist eine solche Sprache.

Beweis

1. Wir zeigen, dass L die Pumping-Eigenschaft hat.

Pumping-Eigenschaft ist nicht hinreichend

Lemma

Es gibt Sprachen, die die Pumping-Eigenschaft haben aber nicht regulär sind.
Die Sprache $L = \{a^j b^k c^k \mid j, k \in \mathbb{N}_{>0}\}$ ist eine solche Sprache.

Beweis

1. Wir zeigen, dass L die Pumping-Eigenschaft hat.

Wir wählen $n = 1$.

Pumping-Eigenschaft ist nicht hinreichend

Lemma

Es gibt Sprachen, die die Pumping-Eigenschaft haben aber nicht regulär sind.
Die Sprache $L = \{a^j b^k c^k \mid j, k \in \mathbb{N}_{>0}\}$ ist eine solche Sprache.

Beweis

1. Wir zeigen, dass L die Pumping-Eigenschaft hat.

Wir wählen $n = 1$.

Sei $z \in L$ ein Wort mit $|z| \geq n$.

Pumping-Eigenschaft ist nicht hinreichend

Lemma

Es gibt Sprachen, die die Pumping-Eigenschaft haben aber nicht regulär sind. Die Sprache $L = \{a^j b^k c^k \mid j, k \in \mathbb{N}_{>0}\}$ ist eine solche Sprache.

Beweis

1. Wir zeigen, dass L die Pumping-Eigenschaft hat.

Wir wählen $n = 1$.

Sei $z \in L$ ein Wort mit $|z| \geq n$.

Da z von der Form $a^j b^k c^k$ ist, dann muss $j > 0$ gelten und wir zerlegen $z = uvw$ mit $u = \varepsilon$, $v = a$, $w = a^{j-1} b^k c^k$. Da $|v| = 1$, $|uv| \leq n$ und $uv^i w = a^{j+i-1} b^k c^k \in L$ für alle $i \in \mathbb{N}$, hat L die Pumping-Eigenschaft.

Pumping-Eigenschaft ist nicht hinreichend

Lemma

Es gibt Sprachen, die die Pumping-Eigenschaft haben aber nicht regulär sind. Die Sprache $L = \{a^j b^k c^k \mid j, k \in \mathbb{N}_{>0}\}$ ist eine solche Sprache.

Beweis

1. Wir zeigen, dass L die Pumping-Eigenschaft hat.

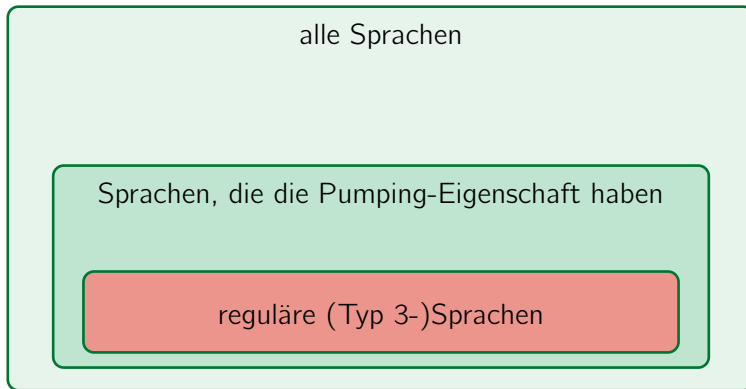
Wir wählen $n = 1$.

Sei $z \in L$ ein Wort mit $|z| \geq n$.

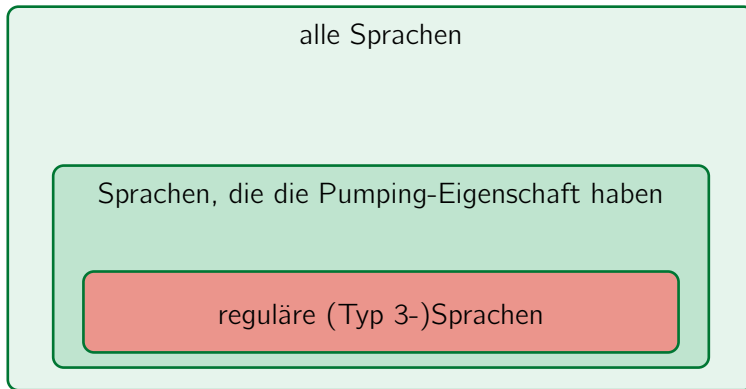
Da z von der Form $a^j b^k c^k$ ist, dann muss $j > 0$ gelten und wir zerlegen $z = uvw$ mit $u = \varepsilon$, $v = a$, $w = a^{j-1} b^k c^k$. Da $|v| = 1$, $|uv| \leq n$ und $uv^i w = a^{j+i-1} b^k c^k \in L$ für alle $i \in \mathbb{N}$, hat L die Pumping-Eigenschaft.

2. Beweis, dass L nicht regulär ist, folgt in der nächsten Vorlesung (nur FSK). $\square$

Pumping-Eigenschaft ist nicht hinreichend



Pumping-Eigenschaft ist nicht hinreichend



Wichtige Konsequenz:

- Das Pumping-Lemma kann **nicht** verwendet werden, um zu zeigen, dass eine Sprache regulär ist.

Zusammenfassung vom Pumping-Lemma

Bezug zu Regularität:

- ▶ Das Pumping-Lemma gibt **eine notwendige Bedingung** für reguläre Sprachen.
Sehr informell: *Wörter einer regulären Sprache können an einer Stelle aufgepumpt werden, wenn sie lang genug sind.*
- ▶ Das Pumping-Lemma gibt **keine hinreichende Bedingung** für reguläre Sprachen,
d.h. Regularität kann **nicht** mit dem Pumping-Lemma gezeigt werden.

Anwendung:

- ▶ L hat **nicht** die Pumping-Eigenschaft $\implies L$ ist **nicht** regulär
- ▶ Dies funktioniert nicht für jede nicht reguläre Sprache.

4b

Eigenschaften von regulären Sprachen

Prof. Dr. Jasmin Blanchette

Lehr- und Forschungseinheit für
Theoretische Informatik und Theorembeweisen

Stand: 6. Mai 2024

Basiert auf Folien von PD Dr. David Sabel



Satz

Die regulären Sprachen sind abgeschlossen bezüglich Vereinigung, Produkt und Kleeneschem Abschluss. D.h. wenn L_1, L_2 regulär sind, dann sind $L_1 \cup L_2$, $L_1 L_2$ und L_1^* regulär.

Satz

Die regulären Sprachen sind abgeschlossen bezüglich Vereinigung, Produkt und Kleeneschem Abschluss. D.h. wenn L_1, L_2 regulär sind, dann sind $L_1 \cup L_2$, $L_1 L_2$ und L_1^* regulär.

Beweis Vereinigung, Produkt und Kleenescher Abschluss werden durch reguläre Ausdrücke erzeugt und sind daher reguläre Sprachen.

Satz

Die regulären Sprachen sind abgeschlossen bezüglich Vereinigung, Produkt und Kleeneschem Abschluss. D.h. wenn L_1, L_2 regulär sind, dann sind $L_1 \cup L_2$, $L_1 L_2$ und L_1^* regulär.

Beweis Vereinigung, Produkt und Kleenescher Abschluss werden durch reguläre Ausdrücke erzeugt und sind daher reguläre Sprachen.

Seien reguläre Ausdrücke α_1, α_2 mit $L(\alpha_i) = L_i$ für $i \in \{1, 2\}$.

Satz

Die regulären Sprachen sind abgeschlossen bezüglich Vereinigung, Produkt und Kleeneschem Abschluss. D.h. wenn L_1, L_2 regulär sind, dann sind $L_1 \cup L_2$, $L_1 L_2$ und L_1^* regulär.

Beweis Vereinigung, Produkt und Kleenescher Abschluss werden durch reguläre Ausdrücke erzeugt und sind daher reguläre Sprachen.

Seien reguläre Ausdrücke α_1, α_2 mit $L(\alpha_i) = L_i$ für $i \in \{1, 2\}$.

- ▶ $(\alpha_1 | \alpha_2)$ erzeugt $L(\alpha_1 | \alpha_2) = L(\alpha_1) \cup L(\alpha_2) = L_1 \cup L_2$.
- ▶ $\alpha_1 \alpha_2$ erzeugt $L(\alpha_1 \alpha_2) = L(\alpha_1) L(\alpha_2) = L_1 L_2$.
- ▶ $(\alpha_1)^*$ erzeugt $L(\alpha_1^*) = L(\alpha_1)^* = L_1^*$. □

Satz

Die regulären Sprachen sind abgeschlossen bezüglich **Komplementbildung**. D.h. wenn L regulär ist, dann ist das Komplement $\overline{L}$ regulär.

Satz

Die regulären Sprachen sind abgeschlossen bezüglich **Komplementbildung**. D.h. wenn L regulär ist, dann ist das Komplement $\bar{L}$ regulär.

Beweis Sei $M = (Z, \Sigma, \delta, z_0, E)$ ein DFA, der L akzeptiert.

Satz

Die regulären Sprachen sind abgeschlossen bezüglich **Komplementbildung**. D.h. wenn L regulär ist, dann ist das Komplement $\bar{L}$ regulär.

Beweis Sei $M = (Z, \Sigma, \delta, z_0, E)$ ein DFA, der L akzeptiert.

Dann akzeptiert $\bar{M} = (Z, \Sigma, \delta, z_0, Z \setminus E)$ die Sprache $\bar{L}$:

Offensichtlich gilt $\tilde{\delta}(z_0, w) \in Z \setminus E$ g.d.w. $\tilde{\delta}(z_0, w) \notin E$.

Satz

Die regulären Sprachen sind abgeschlossen bezüglich **Komplementbildung**. D.h. wenn L regulär ist, dann ist das Komplement $\bar{L}$ regulär.

Beweis Sei $M = (Z, \Sigma, \delta, z_0, E)$ ein DFA, der L akzeptiert.

Dann akzeptiert $\bar{M} = (Z, \Sigma, \delta, z_0, Z \setminus E)$ die Sprache $\bar{L}$:

Offensichtlich gilt $\tilde{\delta}(z_0, w) \in Z \setminus E$ g.d.w. $\tilde{\delta}(z_0, w) \notin E$.

Daher ist $\bar{L}$ regulär.



Satz

Die regulären Sprachen sind abgeschlossen bezüglich **Schnitt**. D.h. wenn L_1, L_2 regulär sind, dann ist $L_1 \cap L_2$ regulär.

Satz

Die regulären Sprachen sind abgeschlossen bezüglich **Schnitt**. D.h. wenn L_1, L_2 regulär sind, dann ist $L_1 \cap L_2$ regulär.

Beweis Dies folgt aus $L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}$ und da reguläre Sprachen abgeschlossen bezüglich Vereinigung und Komplementbildung sind. $\square$

Satz

Die regulären Sprachen sind abgeschlossen bezüglich **Schnitt**. D.h. wenn L_1, L_2 regulär sind, dann ist $L_1 \cap L_2$ regulär.

Alternativer Beweis

Seien $M_1 = (Z_1, \Sigma, \delta_1, z_{01}, E_1)$ und $M_2 = (Z_2, \Sigma, \delta_2, z_{02}, E_2)$ DFAs, die $L_1 = L(M_1)$ und $L_2 = L(M_2)$ akzeptieren.

Der **Produktautomat** von M_1 und M_2 ist der DFA

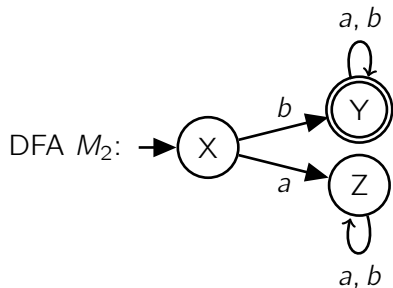
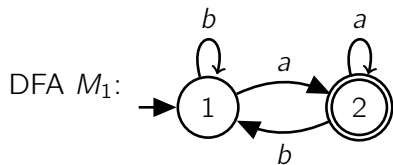
$M_1 \times M_2 = (Z_1 \times Z_2, \Sigma, \delta, (z_{01}, z_{02}), E_1 \times E_2)$ mit

$$\delta((z_1, z_2), a) := (\delta_1(z_1, a), \delta_2(z_2, a)) \quad \text{für alle } a \in \Sigma \text{ und } (z_1, z_2) \in Z_1 \times Z_2$$

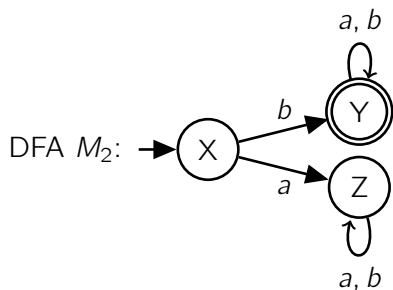
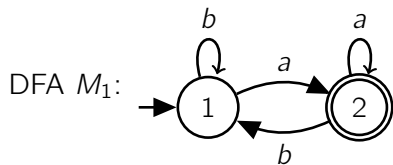
M akzeptiert $L_1 \cap L_2$, denn es gilt:

$\tilde{\delta}((z_{01}, z_{02}), w) \in E_1 \times E_2$ g.d.w. $\tilde{\delta}_1(z_{01}, w) \in E_1$ und $\tilde{\delta}_2(z_{02}, w) \in E_2$. □

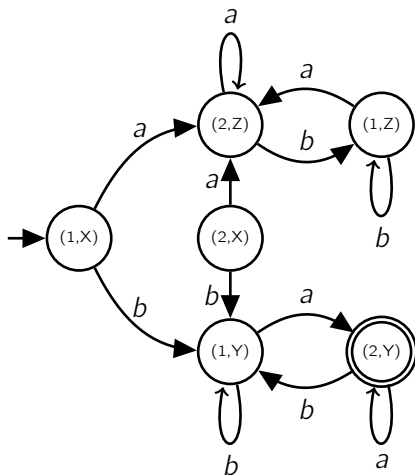
Beispiel für den Produktautomaten



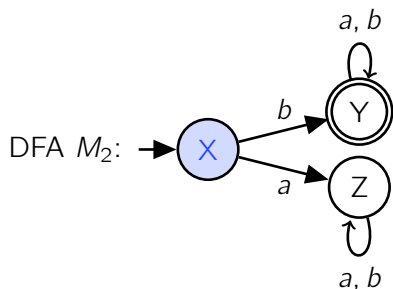
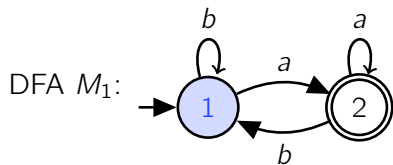
Beispiel für den Produktautomaten



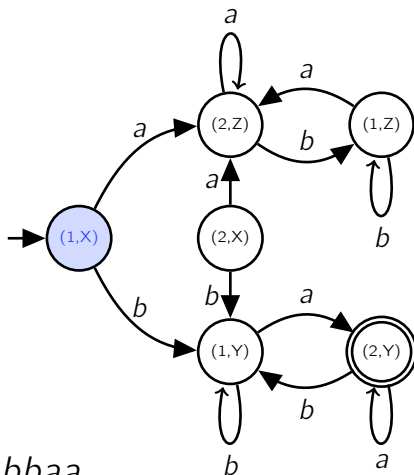
Produktautomat $M_1 \times M_2$:



Beispiel für den Produktautomaten

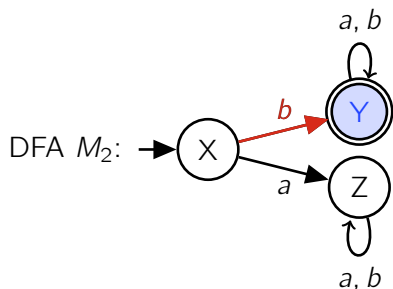
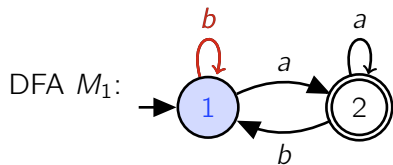


Produktautomat $M_1 \times M_2$:

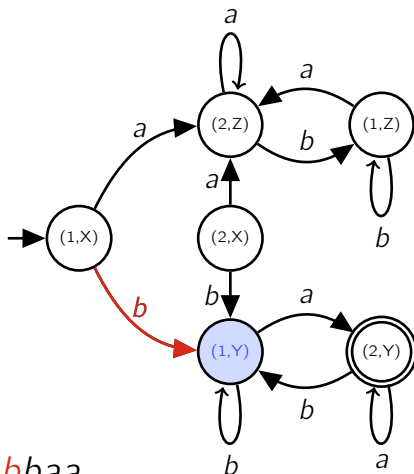


Wort: *bbaa*

Beispiel für den Produktautomaten

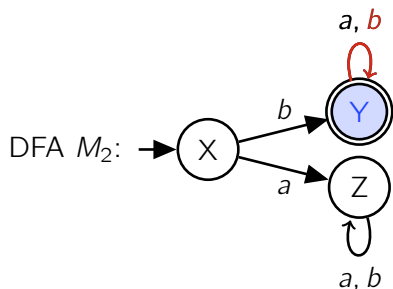
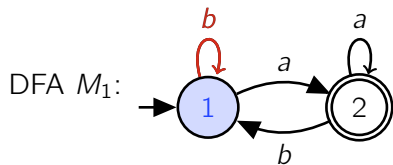


Produktautomat $M_1 \times M_2$:

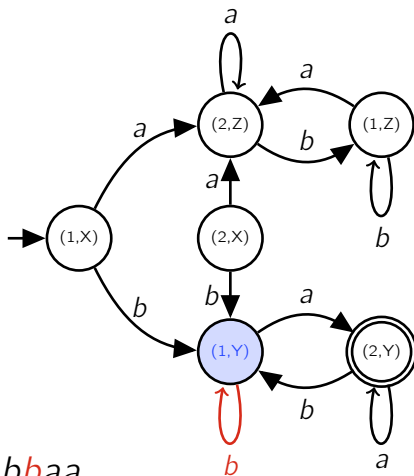


Wort: **b**baa

Beispiel für den Produktautomaten

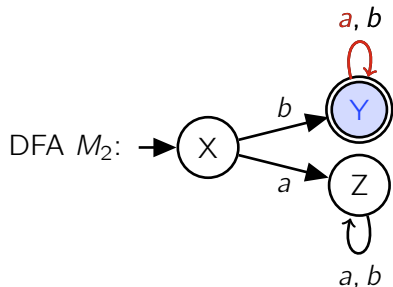
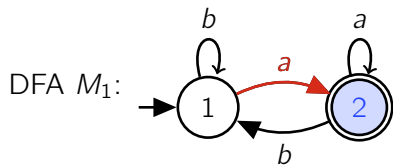


Produktautomat $M_1 \times M_2$:

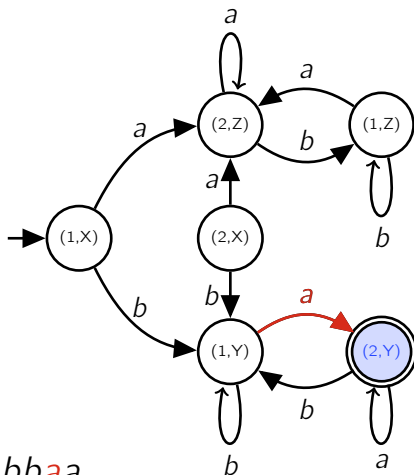


Wort: $bbaa$

Beispiel für den Produktautomaten

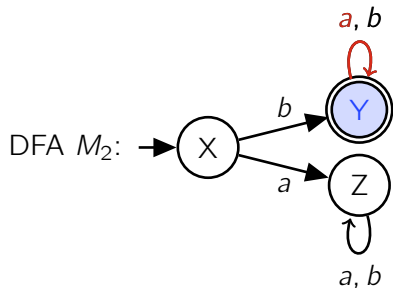
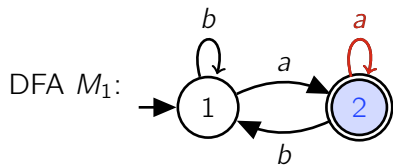


Produktautomat $M_1 \times M_2$:

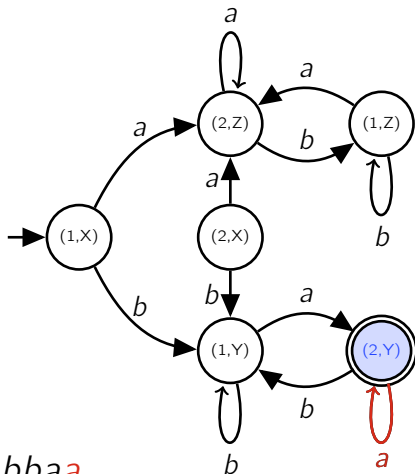


Wort: bb aa

Beispiel für den Produktautomaten

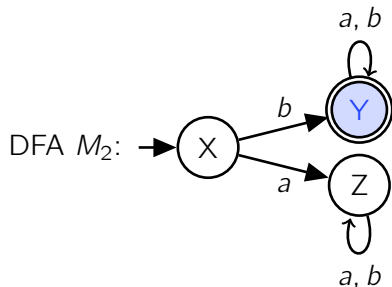
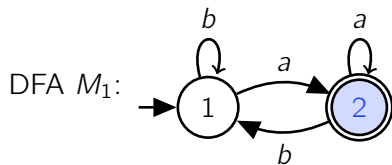


Produktautomat $M_1 \times M_2$:

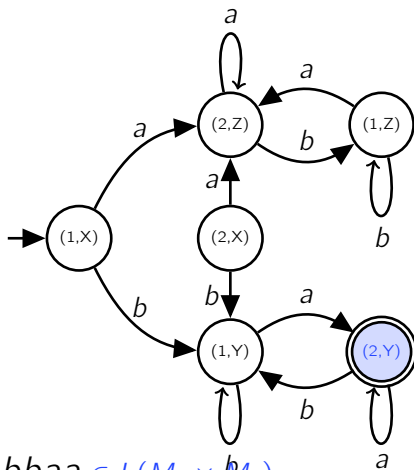


Wort: *bbaa*

Beispiel für den Produktautomaten



Produktautomat $M_1 \times M_2$:



Wort: $bbaa \in L(M_1 \times M_2)$

Theorem (Abschlusseigenschaften der regulären Sprachen)

Die regulären Sprachen sind abgeschlossen bezüglich Vereinigung, Schnitt, Komplementbildung, Produkt und Kleeneschem Abschluss.

Nichtregularität zeigen mit Abschlusseigenschaften

Anleitung zum Widerlegen der Regularität von L mit Abschlusseigenschaften:

1. Nehme an, dass L regulär ist.
2. Operiere auf L unter Erhaltung der Regularität:
vereinige, schneide, komplementiere, multipliziere L mit bekannt regulärer Sprache, bilde Kleeneschen Abschluss.
3. Kommt dabei eine bekannt **nicht reguläre** Sprache heraus,
dann hat man einen **Widerspruch** und die Annahme war falsch.
Daher ist L dann nicht regulär.

Nichtregularität zeigen mit Abschlusseigenschaften

Satz

Die Sprache $L = \{a^n \mid n \text{ ist keine Primzahl}\}$ ist nicht regulär.

Nichtregularität zeigen mit Abschlusseigenschaften

Satz

Die Sprache $L = \{a^n \mid n \text{ ist keine Primzahl}\}$ ist nicht regulär.

Beweis Durch Widerspruch. Wir nehmen an, L ist regulär.

Nichtregularität zeigen mit Abschlusseigenschaften

Satz

Die Sprache $L = \{a^n \mid n \text{ ist keine Primzahl}\}$ ist nicht regulär.

Beweis Durch Widerspruch. Wir nehmen an, L ist regulär.
Dann ist $\overline{L}$ auch regulär.

Nichtregularität zeigen mit Abschlusseigenschaften

Satz

Die Sprache $L = \{a^n \mid n \text{ ist keine Primzahl}\}$ ist nicht regulär.

Beweis Durch Widerspruch. Wir nehmen an, L ist regulär.

Dann ist $\bar{L}$ auch regulär.

$\bar{L} = \{a\}^* \setminus L = \{a^n \mid n \text{ ist eine Primzahl}\}$ ist aber nicht regulär (bereits gezeigt).

Widerspruch. □

Nichtregularität zeigen mit Abschlusseigenschaften

Satz

Die Sprache $L = \{w \in \{a, b\}^* \mid \#_a(w) = \#_b(w)\}$ ist nicht regulär.

Nichtregularität zeigen mit Abschlusseigenschaften

Satz

Die Sprache $L = \{w \in \{a, b\}^* \mid \#_a(w) = \#_b(w)\}$ ist nicht regulär.

Beweis Durch Widerspruch. Wir nehmen an, L ist regulär.

Nichtregularität zeigen mit Abschlusseigenschaften

Satz

Die Sprache $L = \{w \in \{a, b\}^* \mid \#_a(w) = \#_b(w)\}$ ist nicht regulär.

Beweis Durch Widerspruch. Wir nehmen an, L ist regulär.

Die Sprache $L' = \{a^n b^m \mid n, m \in \mathbb{N}\}$ ist regulär, da der reguläre Ausdruck $a^* b^*$ sie erzeugt.

Nichtregularität zeigen mit Abschlusseigenschaften

Satz

Die Sprache $L = \{w \in \{a, b\}^* \mid \#_a(w) = \#_b(w)\}$ ist nicht regulär.

Beweis Durch Widerspruch. Wir nehmen an, L ist regulär.

Die Sprache $L' = \{a^n b^m \mid n, m \in \mathbb{N}\}$ ist regulär, da der reguläre Ausdruck $a^* b^*$ sie erzeugt.

Da L und L' regulär sind, ist auch $L \cap L'$ regulär.

Nichtregularität zeigen mit Abschlusseigenschaften

Satz

Die Sprache $L = \{w \in \{a, b\}^* \mid \#_a(w) = \#_b(w)\}$ ist nicht regulär.

Beweis Durch Widerspruch. Wir nehmen an, L ist regulär.

Die Sprache $L' = \{a^n b^m \mid n, m \in \mathbb{N}\}$ ist regulär, da der reguläre Ausdruck $a^* b^*$ sie erzeugt.

Da L und L' regulär sind, ist auch $L \cap L'$ regulär.

$L \cap L' = \{a^j b^j \mid j \in \mathbb{N}\}$ ist aber nicht regulär (bereits gezeigt). Widerspruch. □

Wortproblem für reguläre Grammatiken

Definition

Das **Wortproblem** für Typ i -Grammatiken ist die Frage, ob für eine gegebene Typ i -Grammatik $G = (V, \Sigma, P, S)$ und ein Wort $w \in \Sigma^*$ $w \in L(G)$ gilt oder nicht.

Wortproblem für reguläre Grammatiken

Definition

Das **Wortproblem** für Typ i -Grammatiken ist die Frage, ob für eine gegebene Typ i -Grammatik $G = (V, \Sigma, P, S)$ und ein Wort $w \in \Sigma^*$ $w \in L(G)$ gilt oder nicht.

Satz

Das Wortproblem für reguläre Grammatiken ist entscheidbar:
Es gibt einen Algorithmus, der bei Eingabe von regulärer Grammatik G und Wort w nach endlicher Zeit entscheidet, ob $w \in L(G)$ gilt oder nicht.

Wortproblem für reguläre Grammatiken

Definition

Das **Wortproblem** für Typ i -Grammatiken ist die Frage, ob für eine gegebene Typ i -Grammatik $G = (V, \Sigma, P, S)$ und ein Wort $w \in \Sigma^*$ $w \in L(G)$ gilt oder nicht.

Satz

Das Wortproblem für reguläre Grammatiken ist entscheidbar:
Es gibt einen Algorithmus, der bei Eingabe von regulärer Grammatik G und Wort w nach endlicher Zeit entscheidet, ob $w \in L(G)$ gilt oder nicht.

Beweis Sei G eine reguläre Grammatik und sei M ein DFA mit $L(M) = L(G)$.
Für M ist das Wortproblem in Linearzeit in der Länge des Wortes lösbar, denn die Berechnung von $\tilde{\delta}(z_0, w)$ braucht für einen DFA nur $|w|$ Schritte. □

Leerheitsproblem für reguläre Grammatiken

Satz

Das Leerheitsproblem für reguläre Grammatiken ist entscheidbar.

Leerheitsproblem für reguläre Grammatiken

Satz

Das Leerheitsproblem für reguläre Grammatiken ist entscheidbar.

Beweis Sei G eine reguläre Grammatik und sei M ein DFA mit $L(M) = L(G)$.

Leerheitsproblem für reguläre Grammatiken

Satz

Das Leerheitsproblem für reguläre Grammatiken ist entscheidbar.

Beweis Sei G eine reguläre Grammatik und sei M ein DFA mit $L(M) = L(G)$.

Dann gilt $L(M) = \emptyset$ g.d.w. es keinen Pfad vom Startzustand zu einem Endzustand in M gibt.

Leerheitsproblem für reguläre Grammatiken

Satz

Das Leerheitsproblem für reguläre Grammatiken ist entscheidbar.

Beweis Sei G eine reguläre Grammatik und sei M ein DFA mit $L(M) = L(G)$.

Dann gilt $L(M) = \emptyset$ g.d.w. es keinen Pfad vom Startzustand zu einem Endzustand in M gibt.

Dies kann man leicht mit einer Tiefensuche (Depth-First-Search) auf dem Zustandsgraph von M prüfen. □

Endlichkeitsproblem für reguläre Grammatiken

Satz

Das Endlichkeitsproblem für reguläre Grammatiken ist entscheidbar.

Endlichkeitsproblem für reguläre Grammatiken

Satz

Das Endlichkeitsproblem für reguläre Grammatiken ist entscheidbar.

Beweis Sei G eine reguläre Grammatik und sei M ein DFA mit $L(M) = L(G)$.

Endlichkeitsproblem für reguläre Grammatiken

Satz

Das Endlichkeitsproblem für reguläre Grammatiken ist entscheidbar.

Beweis Sei G eine reguläre Grammatik und sei M ein DFA mit $L(M) = L(G)$.

Es gilt $|L(M)| < \infty$ g.d.w. es keinen Pfad vom Startzustand zu einem Endzustand gibt, der eine Schleife enthält.

Endlichkeitsproblem für reguläre Grammatiken

Satz

Das Endlichkeitsproblem für reguläre Grammatiken ist entscheidbar.

Beweis Sei G eine reguläre Grammatik und sei M ein DFA mit $L(M) = L(G)$.

Es gilt $|L(M)| < \infty$ g.d.w. es keinen Pfad vom Startzustand zu einem Endzustand gibt, der eine Schleife enthält.

Prüfe dies mit einer Tiefensuche auf dem Zustandsgraph von M . □

Schnittproblem für reguläre Grammatiken

Satz

Das Schnittproblem für reguläre Grammatiken ist entscheidbar.

Schnittproblem für reguläre Grammatiken

Satz

Das Schnittproblem für reguläre Grammatiken ist entscheidbar.

Beweis Seien G_1, G_2 reguläre Grammatiken und seien M_1, M_2 DFAs mit $L(M_i) = L(G_i)$ für $i \in \{1, 2\}$.

Schnittproblem für reguläre Grammatiken

Satz

Das Schnittproblem für reguläre Grammatiken ist entscheidbar.

Beweis Seien G_1, G_2 reguläre Grammatiken und seien M_1, M_2 DFAs mit $L(M_i) = L(G_i)$ für $i \in \{1, 2\}$.

Konstruiere den Produktautomaten $M_1 \times M_2$ mit $L(M_1 \times M_2) = L(M_1) \cap L(M_2)$.

Schnittproblem für reguläre Grammatiken

Satz

Das Schnittproblem für reguläre Grammatiken ist entscheidbar.

Beweis Seien G_1, G_2 reguläre Grammatiken und seien M_1, M_2 DFAs mit $L(M_i) = L(G_i)$ für $i \in \{1, 2\}$.

Konstruiere den Produktautomaten $M_1 \times M_2$ mit $L(M_1 \times M_2) = L(M_1) \cap L(M_2)$.

Prüfe das Leerheitsproblem für $L(M_1 \times M_2)$. □

Äquivalenzproblem für reguläre Grammatiken

Satz

Das Äquivalenzproblem für reguläre Grammatiken ist entscheidbar.

Äquivalenzproblem für reguläre Grammatiken

Satz

Das Äquivalenzproblem für reguläre Grammatiken ist entscheidbar.

Beweis Seien G_1, G_2 reguläre Grammatiken und seien M_1, M_2 DFAs mit $L(M_i) = L(G_i)$ für $i \in \{1, 2\}$.

Äquivalenzproblem für reguläre Grammatiken

Satz

Das Äquivalenzproblem für reguläre Grammatiken ist entscheidbar.

Beweis Seien G_1, G_2 reguläre Grammatiken und seien M_1, M_2 DFAs mit $L(M_i) = L(G_i)$ für $i \in \{1, 2\}$.

Minimiere M_1 und M_2 .

Äquivalenzproblem für reguläre Grammatiken

Satz

Das Äquivalenzproblem für reguläre Grammatiken ist entscheidbar.

Beweis Seien G_1, G_2 reguläre Grammatiken und seien M_1, M_2 DFAs mit $L(M_i) = L(G_i)$ für $i \in \{1, 2\}$.

Minimiere M_1 und M_2 .

Prüfe die minimalen DFAs auf Gleichheit bis auf Umbenennung.



4c

Das Pumping-Lemma für reguläre Sprachen

Prof. Dr. Jasmin Blanchette

Lehr- und Forschungseinheit für
Theoretische Informatik und Theorembeweisen

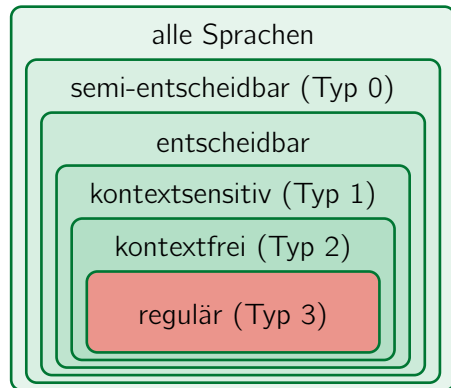
Stand: 6. Mai 2024
Basiert auf Folien von PD Dr. David Sabel



Hintergrund zum Pumping-Lemma

Formalismen zur Darstellung von
regulären Sprachen:

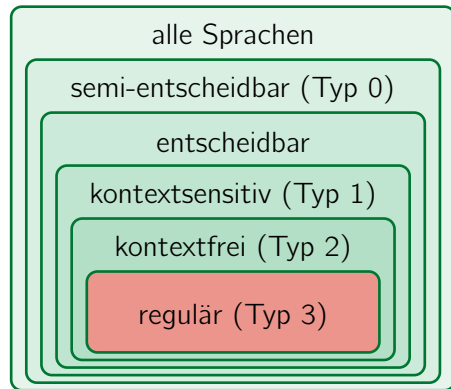
- ▶ reguläre Grammatiken
- ▶ endliche Automaten
- ▶ reguläre Ausdrücke



Hintergrund zum Pumping-Lemma

Formalismen zur Darstellung von
regulären Sprachen:

- ▶ reguläre Grammatiken
- ▶ endliche Automaten
- ▶ reguläre Ausdrücke



Wie zeigt man, dass eine formale Sprache **nicht regulär** ist?

Das **Pumping-Lemma** ist ein Werkzeug dafür.

Exkurs: Beweistechniken für Quantoren

Beweise für $\forall$ und $\exists$ variieren, je nachdem, ob die Aussage zu beweisen ist oder angenommen wird.

Auszug aus Aufgabenblatt 0:

	Um eine Aussage dieser Form zu beweisen ...	Wenn eine Aussage dieser Form angenommen wird ...
$\forall x, P(x)$	beweise, dass $P(a)$ für ein beliebiges a gilt	nimm $P(a)$ für jedes konkrete a an
$\exists x, P(x)$	gib ein konkretes a an und beweise $P(a)$	nimm ein beliebiges a an, für das $P(a)$ gilt

Definition

Eine mit $>$ ausgestattete Menge A hat die **EGE-Eigenschaft**, wenn gilt:
Für jedes $a \in A$ gibt es ein $b \in A$, sodass $b > a$
(d.h. $\forall a \in A, \exists b \in A, b > a$).

(„EGE“ steht für „*exists greater element*“.)

Exkurs: Beispiel für Beweistechniken für Quantoren

Satz

Die natürlichen Zahlen $\mathbb{N}$ mit $>$ haben die EGE-Eigenschaft.

Exkurs: Beispiel für Beweistechniken für Quantoren

Satz

Die natürlichen Zahlen $\mathbb{N}$ mit $>$ haben die EGE-Eigenschaft.

Beweis Wir zeigen: Für jede Zahl $a \in \mathbb{N}$ gibt es eine Zahl $b \in \mathbb{N}$, sodass $b > a$.

Exkurs: Beispiel für Beweistechniken für Quantoren

Satz

Die natürlichen Zahlen $\mathbb{N}$ mit $>$ haben die EGE-Eigenschaft.

Beweis Wir zeigen: Für jede Zahl $a \in \mathbb{N}$ gibt es eine Zahl $b \in \mathbb{N}$, sodass $b > a$.
Sei $a \in \mathbb{N}$ beliebig. Wir müssen dann beweisen, dass es $b \in \mathbb{N}$ gibt, sodass $b > a$.

Exkurs: Beispiel für Beweistechniken für Quantoren

Satz

Die natürlichen Zahlen $\mathbb{N}$ mit $>$ haben die EGE-Eigenschaft.

Beweis Wir zeigen: Für jede Zahl $a \in \mathbb{N}$ gibt es eine Zahl $b \in \mathbb{N}$, sodass $b > a$.
Sei $a \in \mathbb{N}$ beliebig. Wir müssen dann beweisen, dass es $b \in \mathbb{N}$ gibt, sodass $b > a$.
Wir wählen $b = a + 1$ und beweisen $a + 1 > a$, was offensichtlich gilt. $\square$

Exkurs: Beispiel für Beweistechniken für Quantoren

Satz

Die natürlichen Zahlen $\mathbb{N}$ mit $>$ haben die EGE-Eigenschaft.

Beweis Wir zeigen: Für jede Zahl $a \in \mathbb{N}$ gibt es eine Zahl $b \in \mathbb{N}$, sodass $b > a$.
Sei $a \in \mathbb{N}$ beliebig. Wir müssen dann beweisen, dass es $b \in \mathbb{N}$ gibt, sodass $b > a$.
Wir wählen $b = a + 1$ und beweisen $a + 1 > a$, was offensichtlich gilt. $\square$

Satz

Die nicht positiven ganzen Zahlen $\mathbb{Z}_{\leq 0}$ mit $>$ haben nicht die EGE-Eigenschaft.

Exkurs: Beispiel für Beweistechniken für Quantoren

Satz

Die natürlichen Zahlen $\mathbb{N}$ mit $>$ haben die EGE-Eigenschaft.

Beweis Wir zeigen: Für jede Zahl $a \in \mathbb{N}$ gibt es eine Zahl $b \in \mathbb{N}$, sodass $b > a$.
Sei $a \in \mathbb{N}$ beliebig. Wir müssen dann beweisen, dass es $b \in \mathbb{N}$ gibt, sodass $b > a$.
Wir wählen $b = a + 1$ und beweisen $a + 1 > a$, was offensichtlich gilt. $\square$

Satz

Die nicht positiven ganzen Zahlen $\mathbb{Z}_{\leq 0}$ mit $>$ haben nicht die EGE-Eigenschaft.

Beweis Durch Widerspruch. Wir nehmen an, dass $\mathbb{Z}_{\leq 0}$ die EGE-Eigenschaft hat.

Exkurs: Beispiel für Beweistechniken für Quantoren

Satz

Die natürlichen Zahlen $\mathbb{N}$ mit $>$ haben die EGE-Eigenschaft.

Beweis Wir zeigen: Für jede Zahl $a \in \mathbb{N}$ gibt es eine Zahl $b \in \mathbb{N}$, sodass $b > a$.
Sei $a \in \mathbb{N}$ beliebig. Wir müssen dann beweisen, dass es $b \in \mathbb{N}$ gibt, sodass $b > a$.
Wir wählen $b = a + 1$ und beweisen $a + 1 > a$, was offensichtlich gilt. $\square$

Satz

Die nicht positiven ganzen Zahlen $\mathbb{Z}_{\leq 0}$ mit $>$ haben nicht die EGE-Eigenschaft.

Beweis Durch Widerspruch. Wir nehmen an, dass $\mathbb{Z}_{\leq 0}$ die EGE-Eigenschaft hat.
D.h. für jede Zahl $a \in \mathbb{Z}_{\leq 0}$ gibt es eine Zahl $b \in \mathbb{Z}_{\leq 0}$, sodass $b > a$.

Exkurs: Beispiel für Beweistechniken für Quantoren

Satz

Die natürlichen Zahlen $\mathbb{N}$ mit $>$ haben die EGE-Eigenschaft.

Beweis Wir zeigen: Für jede Zahl $a \in \mathbb{N}$ gibt es eine Zahl $b \in \mathbb{N}$, sodass $b > a$.
Sei $a \in \mathbb{N}$ beliebig. Wir müssen dann beweisen, dass es $b \in \mathbb{N}$ gibt, sodass $b > a$.
Wir wählen $b = a + 1$ und beweisen $a + 1 > a$, was offensichtlich gilt. $\square$

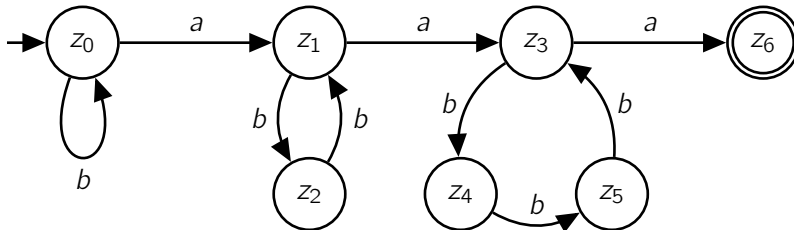
Satz

Die nicht positiven ganzen Zahlen $\mathbb{Z}_{\leq 0}$ mit $>$ haben nicht die EGE-Eigenschaft.

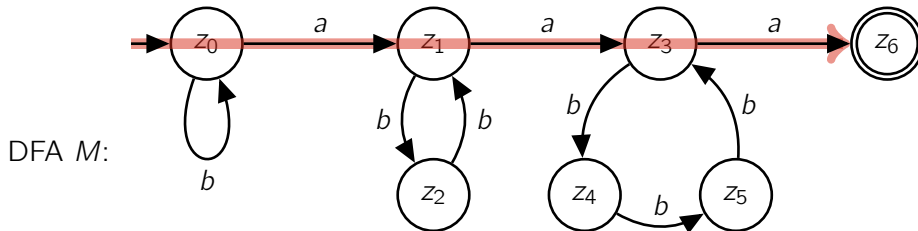
Beweis Durch Widerspruch. Wir nehmen an, dass $\mathbb{Z}_{\leq 0}$ die EGE-Eigenschaft hat.
D.h. für jede Zahl $a \in \mathbb{Z}_{\leq 0}$ gibt es eine Zahl $b \in \mathbb{Z}_{\leq 0}$, sodass $b > a$.
Wir wählen $a = 0$. Es gibt also eine Zahl $b \in \mathbb{Z}_{\leq 0}$, sodass $b > 0$. Aber 0 ist die größte Zahl in $\mathbb{Z}_{\leq 0}$. Widerspruch. $\square$

Intuition hinter dem Pumping-Lemma

DFA M :

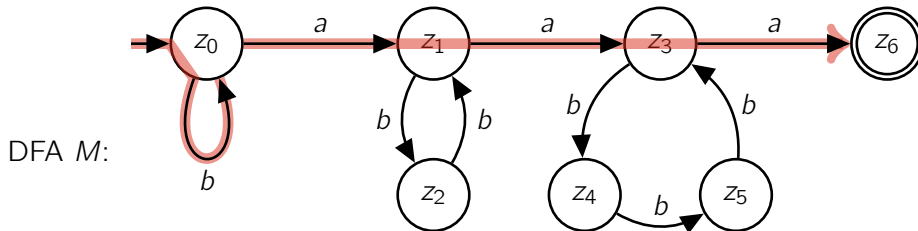


Intuition hinter dem Pumping-Lemma



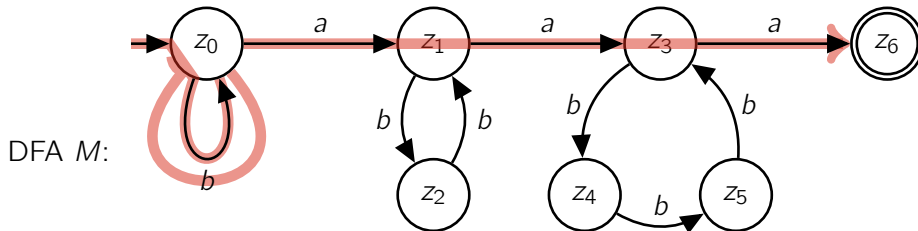
Von M erkannte Wörter der Länge 3,

Intuition hinter dem Pumping-Lemma



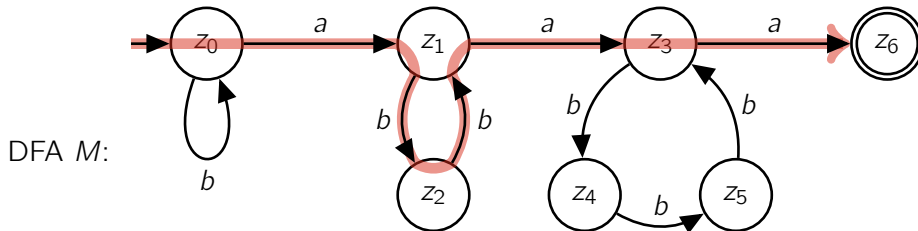
Von M erkannte Wörter der Länge 3, 4,

Intuition hinter dem Pumping-Lemma



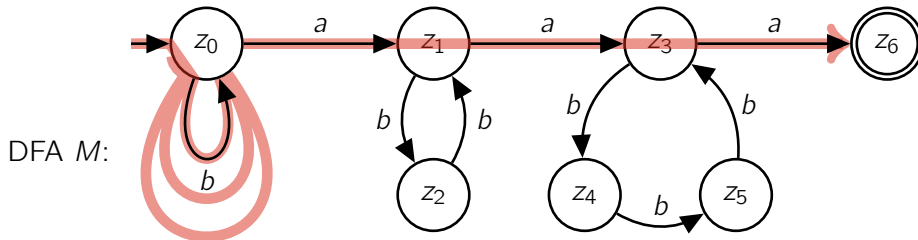
Von M erkannte Wörter der Länge 3, 4, 5,

Intuition hinter dem Pumping-Lemma



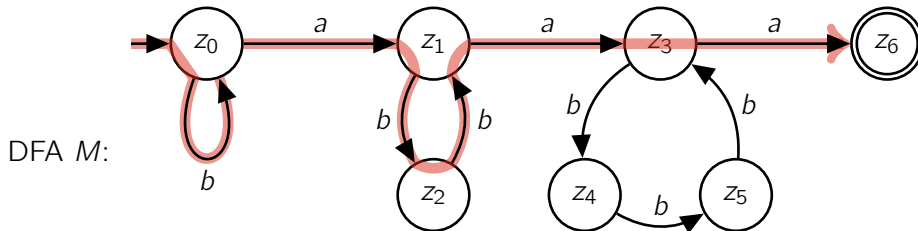
Von M erkannte Wörter der Länge 3, 4, 5,

Intuition hinter dem Pumping-Lemma



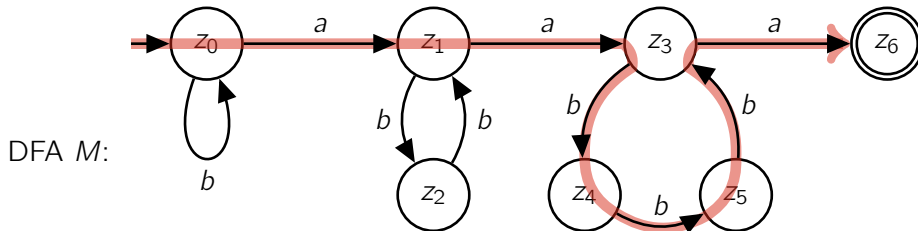
Von M erkannte Wörter der Länge 3, 4, 5, 6, ...

Intuition hinter dem Pumping-Lemma



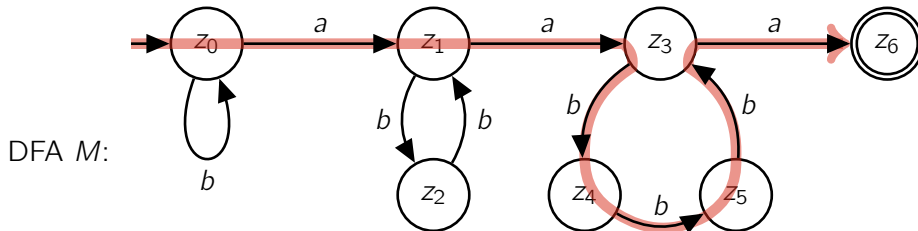
Von M erkannte Wörter der Länge 3, 4, 5, 6, ...

Intuition hinter dem Pumping-Lemma



Von M erkannte Wörter der Länge 3, 4, 5, 6, ...

Intuition hinter dem Pumping-Lemma

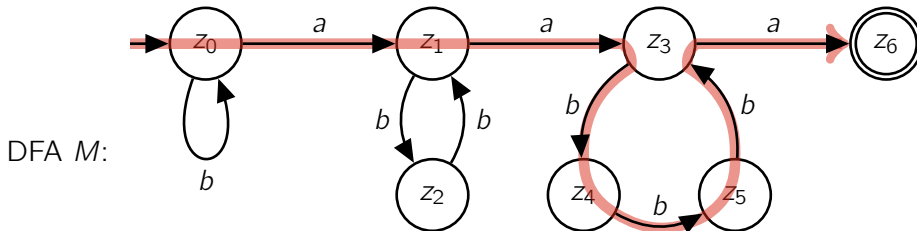


Von M erkannte Wörter der Länge 3, 4, 5, 6, ...

Beobachtungen:

1. Jedes Wort $z \in L(M)$ der Länge > 3 muss mindestens **eine Schleife durchlaufen**.

Intuition hinter dem Pumping-Lemma

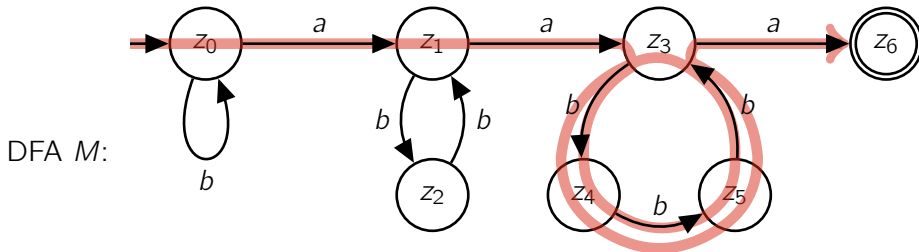


Von M erkannte Wörter der Länge 3, 4, 5, 6, ...

Beobachtungen:

1. Jedes Wort $z \in L(M)$ der Länge > 3 muss mindestens **eine Schleife durchlaufen**.
2. Wenn die **Schleife mehrfach durchgelaufen** wird, wird das entsprechende Wort immer noch erkannt. D.h. Wörter in $L(M)$ mit Länge > 3 können wir **aufpumpen** und verbleiben in $L(M)$.

Intuition hinter dem Pumping-Lemma

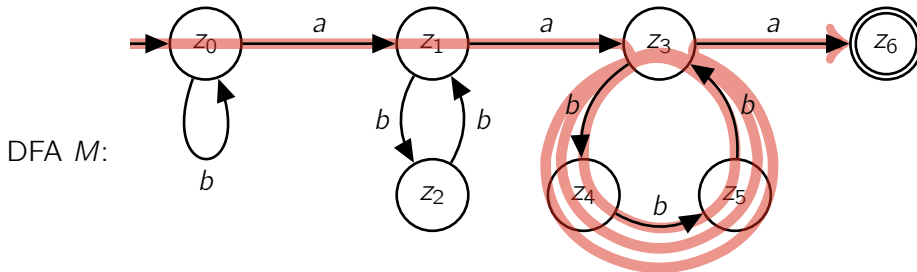


Von M erkannte Wörter der Länge 3, 4, 5, 6, ...

Beobachtungen:

1. Jedes Wort $z \in L(M)$ der Länge > 3 muss mindestens **eine Schleife durchlaufen**.
2. Wenn die **Schleife mehrfach durchgelaufen** wird, wird das entsprechende Wort immer noch erkannt. D.h. Wörter in $L(M)$ mit Länge > 3 können wir **aufpumpen** und verbleiben in $L(M)$.

Intuition hinter dem Pumping-Lemma

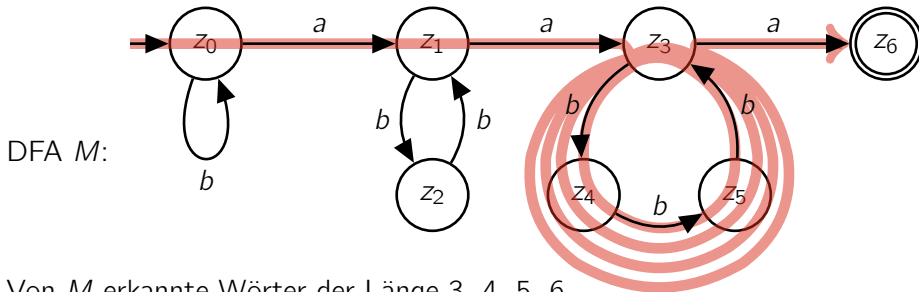


Von M erkannte Wörter der Länge 3, 4, 5, 6, ...

Beobachtungen:

1. Jedes Wort $z \in L(M)$ der Länge > 3 muss mindestens **eine Schleife durchlaufen**.
2. Wenn die **Schleife mehrfach durchgelaufen** wird, wird das entsprechende Wort immer noch erkannt. D.h. Wörter in $L(M)$ mit Länge > 3 können wir **aufpumpen** und verbleiben in $L(M)$.

Intuition hinter dem Pumping-Lemma



Von M erkannte Wörter der Länge 3, 4, 5, 6, ...

Beobachtungen:

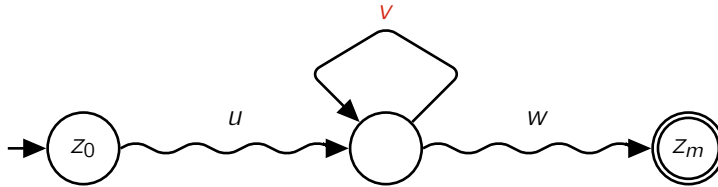
1. Jedes Wort $z \in L(M)$ der Länge > 3 muss mindestens **eine Schleife durchlaufen**.
2. Wenn die **Schleife mehrfach durchgelaufen** wird, wird das entsprechende Wort immer noch erkannt. D.h. Wörter in $L(M)$ mit Länge > 3 können wir **aufpumpen** und verbleiben in $L(M)$.

Intuition hinter dem Pumping-Lemma

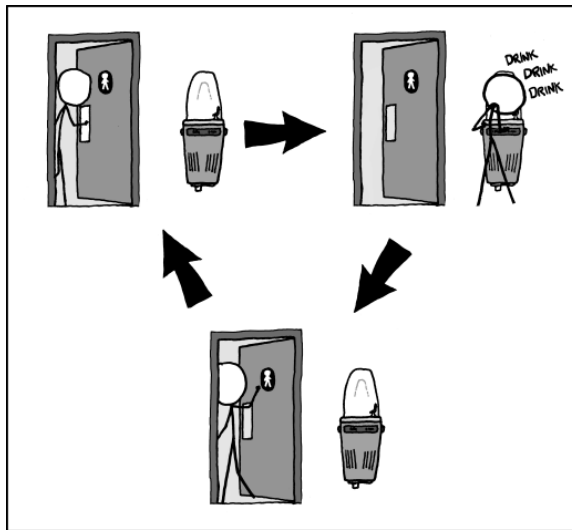
Gilt das allgemein?

Intuition hinter dem Pumping-Lemma

Gilt das allgemein? Ja



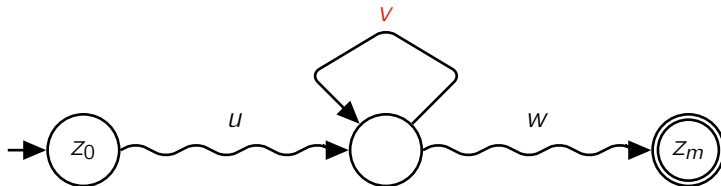
- ▶ Wenn ein DFA n Zustände hat, dann müssen akzeptierte Wörter der Länge $\geq n$ eine Schleife durchlaufen.
- ▶ Diese Wörter kann man aufpumpen: $uvw, uvvw, uvvww, \dots$.
Man kann auch die Schleife überspringen: uw .
Allgemein: $uv^i w$ für $i \in \mathbb{N}$ liegt in der erkannten Sprache.



I AVOID DRINKING FOUNTAINS OUTSIDE BATHROOMS
BECAUSE I'M AFRAID OF GETTING TRAPPED IN A LOOP.

xkcd.com/986/

Die Pumping-Eigenschaft für reguläre Sprachen



Definition

Eine Sprache L hat die **Pumping-Eigenschaft** (für reguläre Sprachen), wenn gilt: Es gibt eine Zahl $n \in \mathbb{N}_{>0}$, sodass jedes Wort $z \in L$, welches Mindestlänge n hat (d.h. $|z| \geq n$), als $z = uvvw$ geschrieben werden kann, sodass gilt:

1. $|uv| \leq n$
2. $|v| \geq 1$
3. für alle $i \in \mathbb{N}$: $uv^i w \in L$.

Das Pumping-Lemma für reguläre Sprachen

Lemma (Pumping-Lemma)

Jede reguläre Sprache hat die **Pumping-Eigenschaft**.

Das Pumping-Lemma für reguläre Sprachen

Lemma (Pumping-Lemma)

Jede reguläre Sprache hat die Pumping-Eigenschaft.

Beweis Sei L eine reguläre Sprache. Wir zeigen, dass es eine Zahl $n \in \mathbb{N}_{>0}$ gibt, sodass jedes $z \in L$ mit $|z| \geq n$ als $z = uvw$ geschrieben werden kann, sodass $|uv| \leq n$, $|v| \geq 1$ und $uv^i w \in L$ und für alle $i \in \mathbb{N}$.

Das Pumping-Lemma für reguläre Sprachen

Lemma (Pumping-Lemma)

Jede reguläre Sprache hat die Pumping-Eigenschaft.

Beweis Sei L eine reguläre Sprache. Wir zeigen, dass es eine Zahl $n \in \mathbb{N}_{>0}$ gibt, sodass jedes $z \in L$ mit $|z| \geq n$ als $z = uvw$ geschrieben werden kann, sodass $|uv| \leq n$, $|v| \geq 1$ und $uv^i w \in L$ und für alle $i \in \mathbb{N}$.

Sei $M = (Z, \Sigma, \delta, z_0, E)$ ein DFA mit $L(M) = L$.

Das Pumping-Lemma für reguläre Sprachen

Lemma (Pumping-Lemma)

Jede reguläre Sprache hat die Pumping-Eigenschaft.

Beweis Sei L eine reguläre Sprache. Wir zeigen, dass es eine Zahl $n \in \mathbb{N}_{>0}$ gibt, sodass jedes $z \in L$ mit $|z| \geq n$ als $z = uvw$ geschrieben werden kann, sodass $|uv| \leq n$, $|v| \geq 1$ und $uv^i w \in L$ und für alle $i \in \mathbb{N}$.

Sei $M = (Z, \Sigma, \delta, z_0, E)$ ein DFA mit $L(M) = L$.

Wir wählen $n = |Z|$. Sei $z \in L$ mit $|z| \geq n$.

Das Pumping-Lemma für reguläre Sprachen

Lemma (Pumping-Lemma)

Jede reguläre Sprache hat die Pumping-Eigenschaft.

Beweis Sei L eine reguläre Sprache. Wir zeigen, dass es eine Zahl $n \in \mathbb{N}_{>0}$ gibt, sodass jedes $z \in L$ mit $|z| \geq n$ als $z = uvw$ geschrieben werden kann, sodass $|uv| \leq n$, $|v| \geq 1$ und $uv^i w \in L$ und für alle $i \in \mathbb{N}$.

Sei $M = (Z, \Sigma, \delta, z_0, E)$ ein DFA mit $L(M) = L$.

Wir wählen $n = |Z|$. Sei $z \in L$ mit $|z| \geq n$.

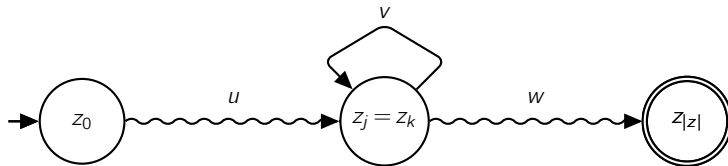
Sei $z_0, \dots, z_{|z|}$ der Lauf für z , mit $z_{|z|} \in E$.

Spätestens nach Lesen von $n (= |Z|)$ Zeichen wird einen Zustand erneut besucht.

Sei z_k (mit $k \leq n$) der erste Zustand, der bereits besucht wurde.

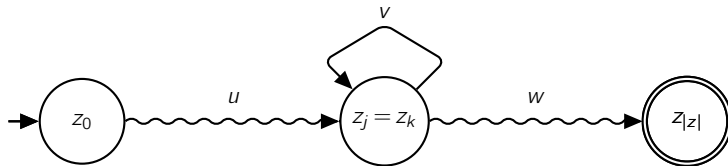
Das Pumping-Lemma für reguläre Sprachen

Beweis (Fortsetzung) Daher gibt es $j < k$, sodass $z_k = z_j$, k ist minimal und $z = uvw$ mit



Das Pumping-Lemma für reguläre Sprachen

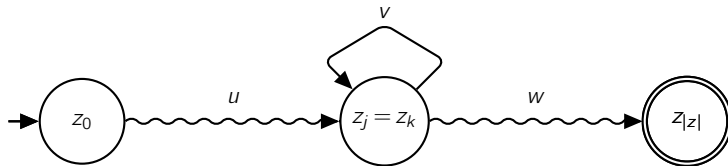
Beweis (Fortsetzung) Daher gibt es $j < k$, sodass $z_k = z_j$, k ist minimal und $z = uvw$ mit



Wir zeigen nun die drei geforderten Eigenschaften der Zerlegung:

Das Pumping-Lemma für reguläre Sprachen

Beweis (Fortsetzung) Daher gibt es $j < k$, sodass $z_k = z_j$, k ist minimal und $z = uvw$ mit

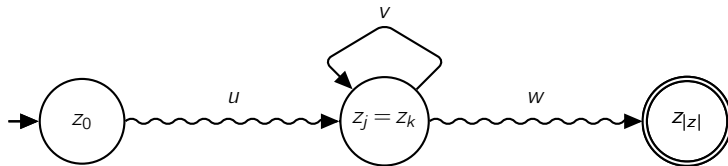


Wir zeigen nun die drei geforderten Eigenschaften der Zerlegung:

- $|v| \geq 1$: folgt aus $j < k$.

Das Pumping-Lemma für reguläre Sprachen

Beweis (Fortsetzung) Daher gibt es $j < k$, sodass $z_k = z_j$, k ist minimal und $z = uvw$ mit

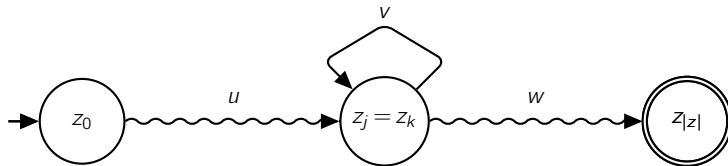


Wir zeigen nun die drei geforderten Eigenschaften der Zerlegung:

- ▶ $|v| \geq 1$: folgt aus $j < k$.
- ▶ $|uv| \leq n$: folgt aus $k \leq n$.

Das Pumping-Lemma für reguläre Sprachen

Beweis (Fortsetzung) Daher gibt es $j < k$, sodass $z_k = z_j$, k ist minimal und $z = uvw$ mit



Wir zeigen nun die drei geforderten Eigenschaften der Zerlegung:

- ▶ $|v| \geq 1$: folgt aus $j < k$.
- ▶ $|uv| \leq n$: folgt aus $k \leq n$.
- ▶ $uv^i w \in L(M)$ für jedes $i \in \mathbb{N}$: lässt sich im obigen Diagramm lesen.

□

Das Paradoxon der endlichen Sprachen

Endliche Sprachen sind regulär.

Warum erfüllen sie die Pumping-Eigenschaft?

Das Paradoxon der endlichen Sprachen

Endliche Sprachen sind regulär.

Warum erfüllen sie die Pumping-Eigenschaft?

Definition

Eine Sprache L hat die Pumping-Eigenschaft (für reguläre Sprachen), wenn gilt:
Es gibt eine Zahl $n \in \mathbb{N}_{>0}$, sodass jedes Wort $z \in L$, welches Mindestlänge n hat (d.h. $|z| \geq n$), als $z = uvw$ geschrieben werden kann, sodass gilt:

1. $|uv| \leq n$
2. $|v| \geq 1$
3. für alle $i \in \mathbb{N}$: $uv^i w \in L$.

Das Paradoxon der endlichen Sprachen

Endliche Sprachen sind regulär.

Warum erfüllen sie die Pumping-Eigenschaft?

Definition

Eine Sprache L hat die Pumping-Eigenschaft (für reguläre Sprachen), wenn gilt:
Es gibt eine Zahl $n \in \mathbb{N}_{>0}$, sodass jedes Wort $z \in L$, welches Mindestlänge n hat (d.h. $|z| \geq n$), als $z = uvw$ geschrieben werden kann, sodass gilt:

1. $|uv| \leq n$
2. $|v| \geq 1$
3. für alle $i \in \mathbb{N}$: $uv^i w \in L$.

Wähle n größer als die Länge des längsten Worts.

Dann ist die Eigenschaft trivial erfüllt.

Anwendung des Pumping-Lemmas

Sei eine Sprache L , die wir als nicht regulär beweisen wollen.

Anwendung des Pumping-Lemmas

Sei eine Sprache L , die wir als nicht regulär beweisen wollen.

Pumping-Lemma:

L ist regulär $\implies L$ hat die Pumping-Eigenschaft

Anwendung des Pumping-Lemmas

Sei eine Sprache L , die wir als nicht regulär beweisen wollen.

Pumping-Lemma:

L ist regulär $\implies L$ hat die Pumping-Eigenschaft

Kontraposition:

L hat nicht die Pumping-Eigenschaft $\implies L$ ist nicht regulär

Anwendung des Pumping-Lemmas

Sei eine Sprache L , die wir als nicht regulär beweisen wollen.

Pumping-Lemma:

L ist regulär $\implies L$ hat die Pumping-Eigenschaft

Kontraposition:

L hat nicht die Pumping-Eigenschaft $\implies L$ ist nicht regulär

Beweisstrategie für die Aussage „ L nicht regulär“:

1. Durch die Kontraposition reicht es zu zeigen, dass L die Pumping-Eigenschaft nicht hat.
2. Zeige dies durch Widerspruch: Nehme an, dass L die Pumping-Eigenschaft hat.
3. Leite einen Widerspruch her.
4. D.h. L ist nicht regulär.

Nichtregularität zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{a^j b^j \mid j \in \mathbb{N}\}$ ist nicht regulär.

Nichtregularität zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{a^j b^j \mid j \in \mathbb{N}\}$ ist nicht regulär.

Beweis Um Nichtregularität von L zu zeigen, reicht es durch das Pumping-Lemma zu zeigen, dass L die Pumping-Eigenschaft nicht hat.

Nichtregularität zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{a^j b^j \mid j \in \mathbb{N}\}$ ist nicht regulär.

Beweis Um Nichtregularität von L zu zeigen, reicht es durch das Pumping-Lemma zu zeigen, dass L die Pumping-Eigenschaft nicht hat.

Durch Widerspruch: Wir nehmen an, dass L die Pumping-Eigenschaft hat.

Nichtregularität zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{a^j b^j \mid j \in \mathbb{N}\}$ ist nicht regulär.

Beweis Um Nichtregularität von L zu zeigen, reicht es durch das Pumping-Lemma zu zeigen, dass L die Pumping-Eigenschaft nicht hat.

Durch Widerspruch: Wir nehmen an, dass L die Pumping-Eigenschaft hat.

Sei $n \in \mathbb{N}_{>0}$ beliebig.

Nichtregularität zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{a^j b^j \mid j \in \mathbb{N}\}$ ist nicht regulär.

Beweis Um Nichtregularität von L zu zeigen, reicht es durch das Pumping-Lemma zu zeigen, dass L die Pumping-Eigenschaft nicht hat.

Durch Widerspruch: Wir nehmen an, dass L die Pumping-Eigenschaft hat.

Sei $n \in \mathbb{N}_{>0}$ beliebig.

Wir wählen $z \in L$ als $z = a^n b^n$. Damit ist auch $|z| \geq n$ erfüllt.

Nichtregularität zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{a^j b^j \mid j \in \mathbb{N}\}$ ist nicht regulär.

Beweis Um Nichtregularität von L zu zeigen, reicht es durch das Pumping-Lemma zu zeigen, dass L die Pumping-Eigenschaft nicht hat.

Durch Widerspruch: Wir nehmen an, dass L die Pumping-Eigenschaft hat.

Sei $n \in \mathbb{N}_{>0}$ beliebig.

Wir wählen $z \in L$ als $z = a^n b^n$. Damit ist auch $|z| \geq n$ erfüllt.

Sei $z = uvw$ eine beliebige Zerlegung von z , sodass $|uv| \leq n$, $|v| \geq 1$ und $uv^i w \in L$ für jedes $i \in \mathbb{N}$. Dann ist $u = a^r$, $v = a^s$ mit $r + s \leq n$, $s \geq 1$ und $w = a^t b^n$ mit $r + s + t = n$.

Nichtregularität zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{a^j b^j \mid j \in \mathbb{N}\}$ ist nicht regulär.

Beweis Um Nichtregularität von L zu zeigen, reicht es durch das Pumping-Lemma zu zeigen, dass L die Pumping-Eigenschaft nicht hat.

Durch Widerspruch: Wir nehmen an, dass L die Pumping-Eigenschaft hat.

Sei $n \in \mathbb{N}_{>0}$ beliebig.

Wir wählen $z \in L$ als $z = a^n b^n$. Damit ist auch $|z| \geq n$ erfüllt.

Sei $z = uvw$ eine beliebige Zerlegung von z , sodass $|uv| \leq n$, $|v| \geq 1$ und $uv^i w \in L$ für jedes $i \in \mathbb{N}$. Dann ist $u = a^r$, $v = a^s$ mit $r + s \leq n$, $s \geq 1$ und $w = a^t b^n$ mit $r + s + t = n$.

Wir betrachten das Wort uv^2w . Es gilt $uv^2w = a^r a^s a^s a^t b^n = a^{n+s} b^n \notin L$, da $s \geq 1$.

Nach Annahme gilt aber $uv^i w \in L$ für jedes $i \in \mathbb{N}$, insbesondere für $i = 2$.

Widerspruch. □

Nichtregularität zeigen mit dem Pumping-Lemma

Es geht kürzer:

Satz

Die Sprache $L = \{a^j b^j \mid j \in \mathbb{N}\}$ ist nicht regulär.

Beweis Mit dem Pumping-Lemma.

Sei $n \in \mathbb{N}_{>0}$ beliebig.

Wir wählen $z \in L$ als $z = a^n b^n$ mit $|z| \geq n$.

Sei $z = uvw$ eine beliebige Zerlegung von z , sodass $|uv| \leq n$, $|v| \geq 1$ und $uv^i w \in L$ für jedes $i \in \mathbb{N}$. Dann ist $u = a^r$, $v = a^s$ mit $r + s \leq n$, $s \geq 1$ und $w = a^t b^n$ mit $r + s + t = n$.

Wir wählen $i = 2$. Es gilt $uv^2 w = a^r a^s a^s a^t b^n = a^{n+s} b^n \notin L$, da $s \geq 1$.

Widerspruch. □

Das Pumping-Lemma als Spiel

Sei L die Sprache.

Schritte:

1. Der **Gegner** wählt die Zahl $n \in \mathbb{N}_{>0}$.
2. **Wir** wählen das Wort $z \in L$ mit $|z| \geq n$.
3. Der **Gegner** wählt die Zerlegung $z = uvw$ mit $|uv| \leq n$ und $|v| \geq 1$.
4. **Wir** gewinnen das Spiel, wenn wir ein $i \in \mathbb{N}$ angeben können, sodass $uv^i w \notin L$.

Das Pumping-Lemma als Spiel

Sei L die Sprache.

Schritte:

1. Der **Gegner** wählt die Zahl $n \in \mathbb{N}_{>0}$.
2. **Wir** wählen das Wort $z \in L$ mit $|z| \geq n$.
3. Der **Gegner** wählt die Zerlegung $z = uvw$ mit $|uv| \leq n$ und $|v| \geq 1$.
4. **Wir** gewinnen das Spiel, wenn wir ein $i \in \mathbb{N}$ angeben können, sodass $uv^i w \notin L$.

Wenn **wir** das Spiel **für alle Wahlmöglichkeiten des Gegners gewinnen**, dann haben wir nachgewiesen, dass L nicht regulär ist.

Das Pumping-Lemma als Spiel

Satz

Die Sprache $L = \{a^j b^j \mid j \in \mathbb{N}\}$ ist nicht regulär.

Beweis Wir zeigen, dass wir das eben eingeführte Spiel stets gewinnen:

1. Sei $n \in \mathbb{N}_{>0}$ vom **Gegner** gewählt.
2. **Wir** wählen $z \in L$ als $z = a^n b^n$ mit $|z| \geq n$.
3. Der **Gegner** wählt die Zerlegung $z = uvw$, sodass $|uv| \leq n$ und $|v| \geq 1$. Dann ist $u = a^r$, $v = a^s$ mit $r + s \leq n$, $s \geq 1$ und $w = a^t b^n$ mit $r + s + t = n$.
4. **Wir** wählen $i = 2$. Es gilt $uv^2w = a^r a^s a^s a^t b^n = a^{n+s} b^n \notin L$, da $s \geq 1$.
Also gewinnen wir. □

Nichtregularität zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{a^p \mid p \text{ ist Primzahl}\}$ ist nicht regulär.

Nichtregularität zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{a^p \mid p \text{ ist Primzahl}\}$ ist nicht regulär.

Beweis Mit dem Pumping-Lemma.

Nichtregularität zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{a^p \mid p \text{ ist Primzahl}\}$ ist nicht regulär.

Beweis Mit dem Pumping-Lemma.

Sei $n \in \mathbb{N}_{>0}$ beliebig.

Nichtregularität zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{a^p \mid p \text{ ist Primzahl}\}$ ist nicht regulär.

Beweis Mit dem Pumping-Lemma.

Sei $n \in \mathbb{N}_{>0}$ beliebig.

Wir wählen $z \in L$ als $z = a^p$, wobei p die nächste Primzahl ist, die größer gleich n ist. Wir haben $|z| = p \geq n$.

Nichtregularität zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{a^p \mid p \text{ ist Primzahl}\}$ ist nicht regulär.

Beweis Mit dem Pumping-Lemma.

Sei $n \in \mathbb{N}_{>0}$ beliebig.

Wir wählen $z \in L$ als $z = a^p$, wobei p die nächste Primzahl ist, die größer gleich n ist. Wir haben $|z| = p \geq n$.

Sei $z = uvw$ eine beliebige Zerlegung von z , sodass $|uv| \leq n$, $|v| \geq 1$ und $uv^i w \in L$ für jedes $i \in \mathbb{N}$. Dann $u = a^r$, $v = a^s$ (mit $s \geq 1$) und $w = a^t$.

Nichtregularität zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{a^p \mid p \text{ ist Primzahl}\}$ ist nicht regulär.

Beweis Mit dem Pumping-Lemma.

Sei $n \in \mathbb{N}_{>0}$ beliebig.

Wir wählen $z \in L$ als $z = a^p$, wobei p die nächste Primzahl ist, die größer gleich n ist. Wir haben $|z| = p \geq n$.

Sei $z = uvw$ eine beliebige Zerlegung von z , sodass $|uv| \leq n$, $|v| \geq 1$ und $uv^i w \in L$ für jedes $i \in \mathbb{N}$. Dann $u = a^r$, $v = a^s$ (mit $s \geq 1$) und $w = a^t$.

Wir wählen $i = p + 1$. Es gilt $uv^{p+1}w \notin L$, denn

$uv^{p+1}w = a^r(a^s)^{p+1}a^t = a^{r+s \cdot (p+1)+t} = a^{r+s \cdot p+s+t} = a^{s \cdot p+p} = a^{p \cdot (s+1)}$ und für $s \geq 1$ folgt, dass $p \cdot (s + 1)$ keine Primzahl sein kann. Widerspruch. $\square$

Nichtregularität zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{a^{n^2} \mid n \in \mathbb{N}\}$ ist nicht regulär.

Beweis Mit dem Pumping-Lemma.

Nichtregularität zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{a^{n^2} \mid n \in \mathbb{N}\}$ ist nicht regulär.

Beweis Mit dem Pumping-Lemma.

Sei $n \in \mathbb{N}_{>0}$ beliebig.

Nichtregularität zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{a^{n^2} \mid n \in \mathbb{N}\}$ ist nicht regulär.

Beweis Mit dem Pumping-Lemma.

Sei $n \in \mathbb{N}_{>0}$ beliebig.

Wir wählen $z = a^{n^2} \in L$ mit $|z| = n^2 \geq n$.

Nichtregularität zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{a^{n^2} \mid n \in \mathbb{N}\}$ ist nicht regulär.

Beweis Mit dem Pumping-Lemma.

Sei $n \in \mathbb{N}_{>0}$ beliebig.

Wir wählen $z = a^{n^2} \in L$ mit $|z| = n^2 \geq n$.

Sei $z = uvw$ eine beliebige Zerlegung von z , sodass $|uv| \leq n$, $|v| \geq 1$ und $uv^i w \in L$ für jedes $i \in \mathbb{N}$.

Nichtregularität zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{a^{n^2} \mid n \in \mathbb{N}\}$ ist nicht regulär.

Beweis Mit dem Pumping-Lemma.

Sei $n \in \mathbb{N}_{>0}$ beliebig.

Wir wählen $z = a^{n^2} \in L$ mit $|z| = n^2 \geq n$.

Sei $z = uvw$ eine beliebige Zerlegung von z , sodass $|uv| \leq n$, $|v| \geq 1$ und $uv^i w \in L$ für jedes $i \in \mathbb{N}$.

Wir wählen $i = 2$. Wir betrachten $uv^2w = a^k$.

- ▶ $1 + n^2 \leq k$ (denn $|v| \geq 1$)
- ▶ $k \leq n^2 + n$ (denn $|uv| \leq n$ und daher $|v| \leq n$)

Nichtregularität zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{a^{n^2} \mid n \in \mathbb{N}\}$ ist nicht regulär.

Beweis Mit dem Pumping-Lemma.

Sei $n \in \mathbb{N}_{>0}$ beliebig.

Wir wählen $z = a^{n^2} \in L$ mit $|z| = n^2 \geq n$.

Sei $z = uvw$ eine beliebige Zerlegung von z , sodass $|uv| \leq n$, $|v| \geq 1$ und $uv^i w \in L$ für jedes $i \in \mathbb{N}$.

Wir wählen $i = 2$. Wir betrachten $uv^2w = a^k$.

- ▶ $1 + n^2 \leq k$ (denn $|v| \geq 1$)
- ▶ $k \leq n^2 + n$ (denn $|uv| \leq n$ und daher $|v| \leq n$)

D.h. wir haben $n^2 < k \leq n^2 + n = (n+1) \cdot n < (n+1)^2$.

Dann kann k keine Quadratzahl sein. Daher gilt $uv^2w \notin L$. Widerspruch. □

Nichtregularität zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{w \in \{a, b\}^* \mid w \text{ ist ein Palindrom}\}$ ist nicht regulär.

Beweis Mit dem Pumping-Lemma.

Nichtregularität zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{w \in \{a, b\}^* \mid w \text{ ist ein Palindrom}\}$ ist nicht regulär.

Beweis Mit dem Pumping-Lemma.

Sei $n \in \mathbb{N}_{>0}$ beliebig.

Nichtregularität zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{w \in \{a, b\}^* \mid w \text{ ist ein Palindrom}\}$ ist nicht regulär.

Beweis Mit dem Pumping-Lemma.

Sei $n \in \mathbb{N}_{>0}$ beliebig.

Wir wählen $z = a^n b a^n \in L$ mit $|w| = 2n + 1 \geq n$.

Nichtregularität zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{w \in \{a, b\}^* \mid w \text{ ist ein Palindrom}\}$ ist nicht regulär.

Beweis Mit dem Pumping-Lemma.

Sei $n \in \mathbb{N}_{>0}$ beliebig.

Wir wählen $z = a^n b a^n \in L$ mit $|w| = 2n + 1 \geq n$.

Sei $z = uvw$ eine beliebige Zerlegung von z , sodass $|uv| \leq n$, $|v| \geq 1$ und $uv^i w \in L$ für jedes $i \in \mathbb{N}$.

Nichtregularität zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{w \in \{a, b\}^* \mid w \text{ ist ein Palindrom}\}$ ist nicht regulär.

Beweis Mit dem Pumping-Lemma.

Sei $n \in \mathbb{N}_{>0}$ beliebig.

Wir wählen $z = a^n b a^n \in L$ mit $|z| = 2n + 1 \geq n$.

Sei $z = uvw$ eine beliebige Zerlegung von z , sodass $|uv| \leq n$, $|v| \geq 1$ und $uv^i w \in L$ für jedes $i \in \mathbb{N}$.

Wir wählen $i = 0$. Es gilt $uv^0 w \notin L$, denn $uv^0 w = a^k b a^n$ mit $k = n - |v| < n$ ist kein Palindrom. Widerspruch. $\square$

Pumping-Eigenschaft ist nicht hinreichend

Lemma

Es gibt Sprachen, die die Pumping-Eigenschaft haben aber nicht regulär sind.
Die Sprache $L = \{a^j b^k c^k \mid j, k \in \mathbb{N}_{>0}\}$ ist eine solche Sprache.

Beweis

1. Wir zeigen, dass L die Pumping-Eigenschaft hat.

Pumping-Eigenschaft ist nicht hinreichend

Lemma

Es gibt Sprachen, die die Pumping-Eigenschaft haben aber nicht regulär sind.
Die Sprache $L = \{a^j b^k c^k \mid j, k \in \mathbb{N}_{>0}\}$ ist eine solche Sprache.

Beweis

1. Wir zeigen, dass L die Pumping-Eigenschaft hat.

Wir wählen $n = 1$.

Pumping-Eigenschaft ist nicht hinreichend

Lemma

Es gibt Sprachen, die die Pumping-Eigenschaft haben aber nicht regulär sind.
Die Sprache $L = \{a^j b^k c^k \mid j, k \in \mathbb{N}_{>0}\}$ ist eine solche Sprache.

Beweis

1. Wir zeigen, dass L die Pumping-Eigenschaft hat.

Wir wählen $n = 1$.

Sei $z \in L$ ein Wort mit $|z| \geq n$.

Pumping-Eigenschaft ist nicht hinreichend

Lemma

Es gibt Sprachen, die die Pumping-Eigenschaft haben aber nicht regulär sind. Die Sprache $L = \{a^j b^k c^k \mid j, k \in \mathbb{N}_{>0}\}$ ist eine solche Sprache.

Beweis

1. Wir zeigen, dass L die Pumping-Eigenschaft hat.

Wir wählen $n = 1$.

Sei $z \in L$ ein Wort mit $|z| \geq n$.

Da z von der Form $a^j b^k c^k$ ist, dann muss $j > 0$ gelten und wir zerlegen $z = uvw$ mit $u = \varepsilon$, $v = a$, $w = a^{j-1} b^k c^k$. Da $|v| = 1$, $|uv| \leq n$ und $uv^i w = a^{j+i-1} b^k c^k \in L$ für alle $i \in \mathbb{N}$, hat L die Pumping-Eigenschaft.

Pumping-Eigenschaft ist nicht hinreichend

Lemma

Es gibt Sprachen, die die Pumping-Eigenschaft haben aber nicht regulär sind. Die Sprache $L = \{a^j b^k c^k \mid j, k \in \mathbb{N}_{>0}\}$ ist eine solche Sprache.

Beweis

1. Wir zeigen, dass L die Pumping-Eigenschaft hat.

Wir wählen $n = 1$.

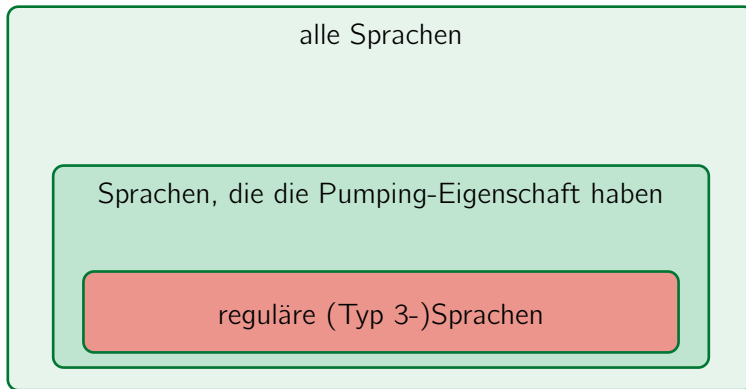
Sei $z \in L$ ein Wort mit $|z| \geq n$.

Da z von der Form $a^j b^k c^k$ ist, dann muss $j > 0$ gelten und wir zerlegen $z = uvw$ mit $u = \varepsilon$, $v = a$, $w = a^{j-1} b^k c^k$. Da $|v| = 1$, $|uv| \leq n$ und $uv^i w = a^{j+i-1} b^k c^k \in L$ für alle $i \in \mathbb{N}$, hat L die Pumping-Eigenschaft.

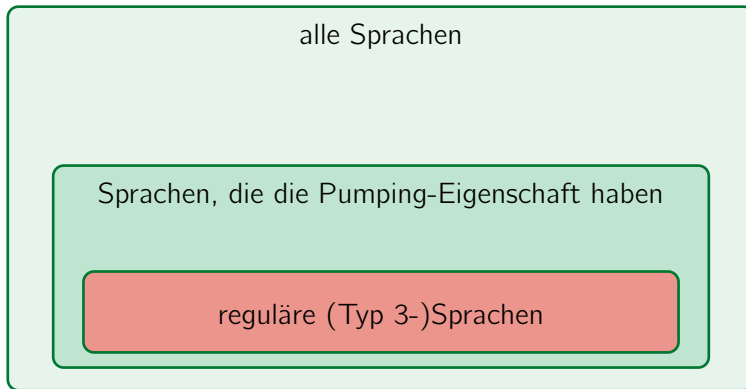
2. Beweis, dass L nicht regulär ist folgt später (nur FSK).



Pumping-Eigenschaft ist nicht hinreichend



Pumping-Eigenschaft ist nicht hinreichend



Wichtige Konsequenz:

- Das Pumping-Lemma kann **nicht** verwendet werden, um zu zeigen, dass eine Sprache regulär ist.

Zusammenfassung vom Pumping-Lemma

Bezug zu Regularität:

- ▶ Das Pumping-Lemma gibt **eine notwendige Bedingung** für reguläre Sprachen.
Sehr informell: *Wörter einer regulären Sprache können an einer Stelle aufgepumpt werden, wenn sie lang genug sind.*
- ▶ Das Pumping-Lemma gibt **keine hinreichende Bedingung** für reguläre Sprachen, d.h. Regularität kann **nicht** mit dem Pumping-Lemma gezeigt werden.

Anwendung:

- ▶ L hat **nicht** die Pumping-Eigenschaft $\implies L$ **nicht** regulär
- ▶ Dies funktioniert nicht für jede nicht reguläre Sprache.

4c

Eigenschaften von regulären Sprachen

Prof. Dr. Jasmin Blanchette

Lehr- und Forschungseinheit für
Theoretische Informatik und Theorembeweisen

Stand: 10. Mai 2024
Basiert auf Folien von PD Dr. David Sabel



Satz

Die regulären Sprachen sind abgeschlossen bezüglich Vereinigung, Produkt und Kleeneschem Abschluss. D.h. wenn L_1, L_2 regulär sind, dann sind $L_1 \cup L_2$, $L_1 L_2$ und L_1^* regulär.

Satz

Die regulären Sprachen sind abgeschlossen bezüglich Vereinigung, Produkt und Kleeneschem Abschluss. D.h. wenn L_1, L_2 regulär sind, dann sind $L_1 \cup L_2$, $L_1 L_2$ und L_1^* regulär.

Beweis Vereinigung, Produkt und Kleenescher Abschluss werden durch reguläre Ausdrücke erzeugt und sind daher reguläre Sprachen.

Satz

Die regulären Sprachen sind abgeschlossen bezüglich Vereinigung, Produkt und Kleeneschem Abschluss. D.h. wenn L_1, L_2 regulär sind, dann sind $L_1 \cup L_2$, $L_1 L_2$ und L_1^* regulär.

Beweis Vereinigung, Produkt und Kleenescher Abschluss werden durch reguläre Ausdrücke erzeugt und sind daher reguläre Sprachen.

Seien reguläre Ausdrücke α_1, α_2 mit $L(\alpha_i) = L_i$ für $i \in \{1, 2\}$.

Satz

Die regulären Sprachen sind abgeschlossen bezüglich Vereinigung, Produkt und Kleeneschem Abschluss. D.h. wenn L_1, L_2 regulär sind, dann sind $L_1 \cup L_2$, $L_1 L_2$ und L_1^* regulär.

Beweis Vereinigung, Produkt und Kleenescher Abschluss werden durch reguläre Ausdrücke erzeugt und sind daher reguläre Sprachen.

Seien reguläre Ausdrücke α_1, α_2 mit $L(\alpha_i) = L_i$ für $i \in \{1, 2\}$.

- ▶ $(\alpha_1 | \alpha_2)$ erzeugt $L(\alpha_1 | \alpha_2) = L(\alpha_1) \cup L(\alpha_2) = L_1 \cup L_2$.
- ▶ $\alpha_1 \alpha_2$ erzeugt $L(\alpha_1 \alpha_2) = L(\alpha_1) L(\alpha_2) = L_1 L_2$.
- ▶ $(\alpha_1)^*$ erzeugt $L(\alpha_1^*) = L(\alpha_1)^* = L_1^*$. □

Satz

Die regulären Sprachen sind abgeschlossen bezüglich **Komplementbildung**. D.h. wenn L regulär ist, dann ist das Komplement $\bar{L}$ regulär.

Satz

Die regulären Sprachen sind abgeschlossen bezüglich **Komplementbildung**. D.h. wenn L regulär ist, dann ist das Komplement $\bar{L}$ regulär.

Beweis Sei $M = (Z, \Sigma, \delta, z_0, E)$ ein DFA, der L akzeptiert.

Satz

Die regulären Sprachen sind abgeschlossen bezüglich **Komplementbildung**. D.h. wenn L regulär ist, dann ist das Komplement $\bar{L}$ regulär.

Beweis Sei $M = (Z, \Sigma, \delta, z_0, E)$ ein DFA, der L akzeptiert.

Dann akzeptiert $\bar{M} = (Z, \Sigma, \delta, z_0, Z \setminus E)$ die Sprache $\bar{L}$:

Offensichtlich gilt $\tilde{\delta}(z_0, w) \in Z \setminus E$ g.d.w. $\tilde{\delta}(z_0, w) \notin E$.

Satz

Die regulären Sprachen sind abgeschlossen bezüglich **Komplementbildung**. D.h. wenn L regulär ist, dann ist das Komplement $\bar{L}$ regulär.

Beweis Sei $M = (Z, \Sigma, \delta, z_0, E)$ ein DFA, der L akzeptiert.

Dann akzeptiert $\bar{M} = (Z, \Sigma, \delta, z_0, Z \setminus E)$ die Sprache $\bar{L}$:

Offensichtlich gilt $\tilde{\delta}(z_0, w) \in Z \setminus E$ g.d.w. $\tilde{\delta}(z_0, w) \notin E$.

Daher ist $\bar{L}$ regulär.



Satz

Die regulären Sprachen sind abgeschlossen bezüglich **Schnitt**. D.h. wenn L_1, L_2 regulär sind, dann ist $L_1 \cap L_2$ regulär.

Satz

Die regulären Sprachen sind abgeschlossen bezüglich **Schnitt**. D.h. wenn L_1, L_2 regulär sind, dann ist $L_1 \cap L_2$ regulär.

Beweis Dies folgt aus $L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}$ und da reguläre Sprachen abgeschlossen bezüglich Vereinigung und Komplementbildung sind. $\square$

Satz

Die regulären Sprachen sind abgeschlossen bezüglich **Schnitt**. D.h. wenn L_1, L_2 regulär sind, dann ist $L_1 \cap L_2$ regulär.

Alternativer Beweis

Seien $M_1 = (Z_1, \Sigma, \delta_1, z_{01}, E_1)$ und $M_2 = (Z_2, \Sigma, \delta_2, z_{02}, E_2)$ DFAs, die $L_1 = L(M_1)$ und $L_2 = L(M_2)$ akzeptieren.

Der **Produktautomat** von M_1 und M_2 ist der DFA

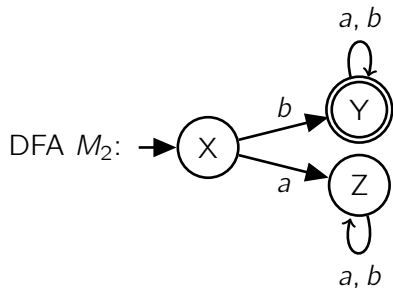
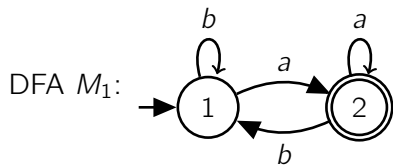
$M_1 \times M_2 = (Z_1 \times Z_2, \Sigma, \delta, (z_{01}, z_{02}), E_1 \times E_2)$ mit

$$\delta((z_1, z_2), a) := (\delta_1(z_1, a), \delta_2(z_2, a)) \quad \text{für alle } a \in \Sigma \text{ und } (z_1, z_2) \in Z_1 \times Z_2$$

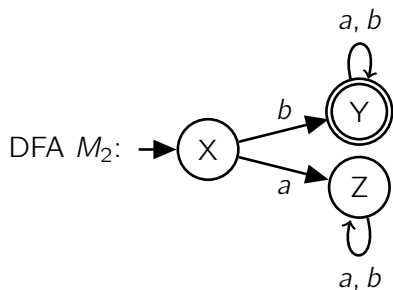
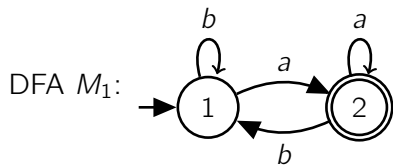
M akzeptiert $L_1 \cap L_2$, denn es gilt:

$\tilde{\delta}((z_{01}, z_{02}), w) \in E_1 \times E_2$ g.d.w. $\tilde{\delta}_1(z_{01}, w) \in E_1$ und $\tilde{\delta}_2(z_{02}, w) \in E_2$. □

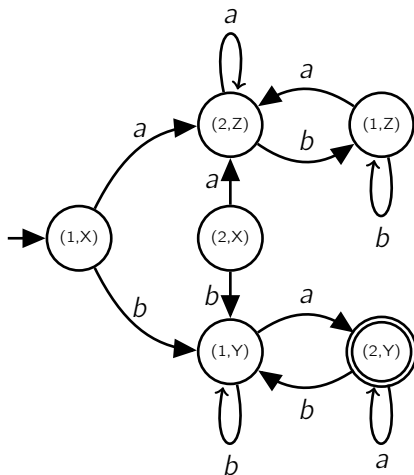
Beispiel für den Produktautomaten



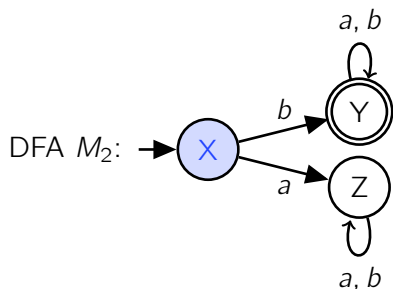
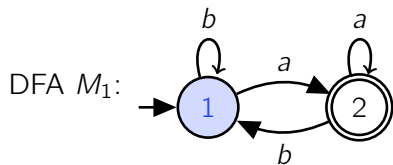
Beispiel für den Produktautomaten



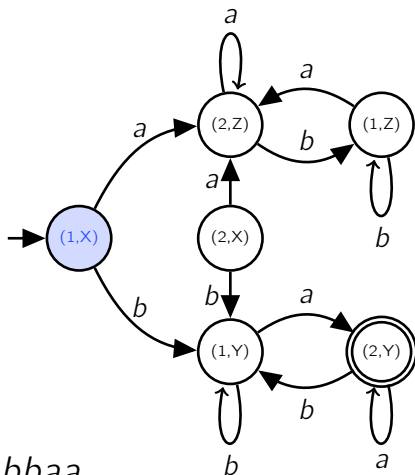
Produktautomat $M_1 \times M_2$:



Beispiel für den Produktautomaten

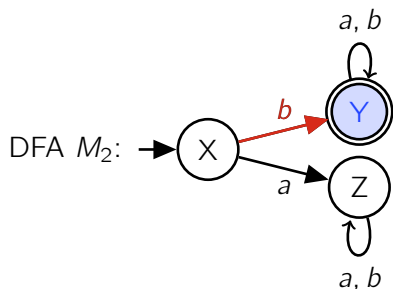
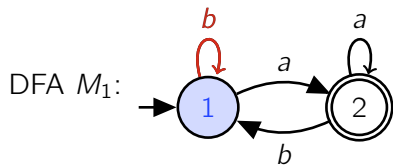


Produktautomat $M_1 \times M_2$:

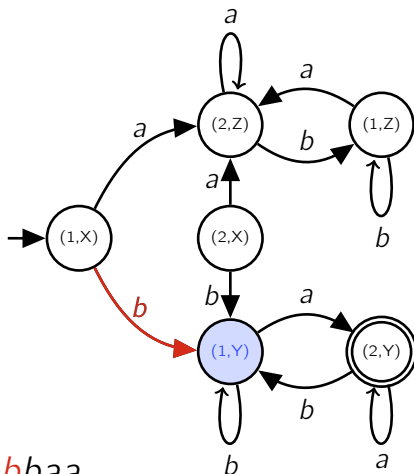


Wort: *bbaa*

Beispiel für den Produktautomaten

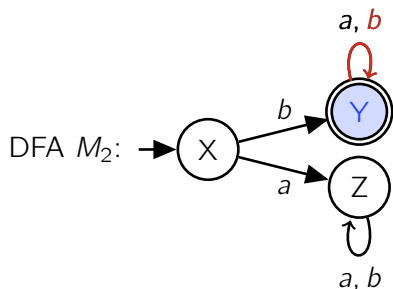
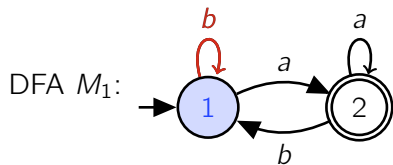


Produktautomat $M_1 \times M_2$:

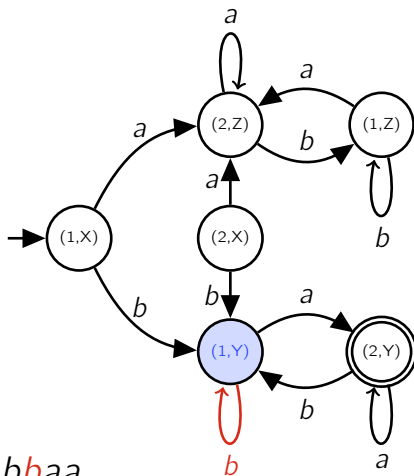


Wort: **b**baa

Beispiel für den Produktautomaten

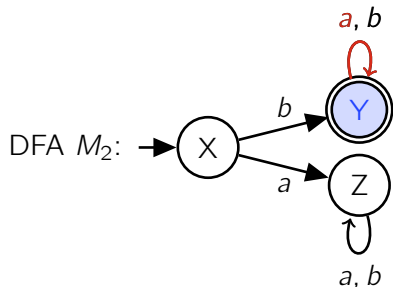
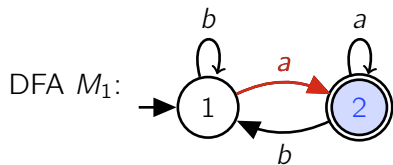


Produktautomat $M_1 \times M_2$:

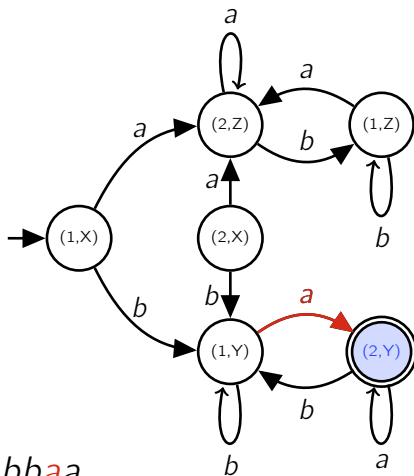


Wort: $bbaa$

Beispiel für den Produktautomaten

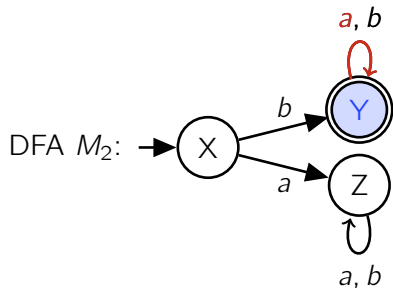
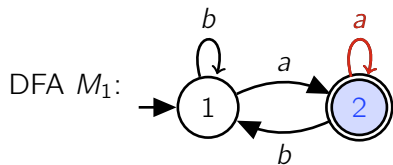


Produktautomat $M_1 \times M_2$:

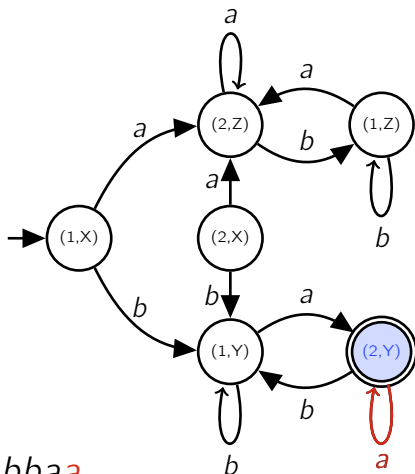


Wort: bb aa

Beispiel für den Produktautomaten

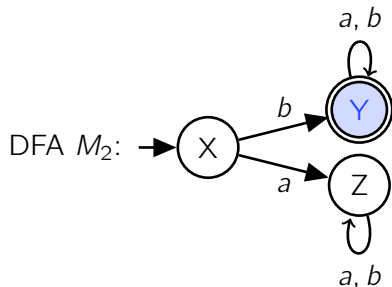
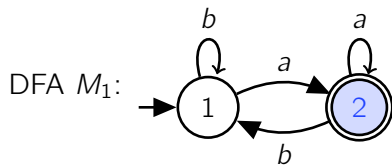


Produktautomat $M_1 \times M_2$:

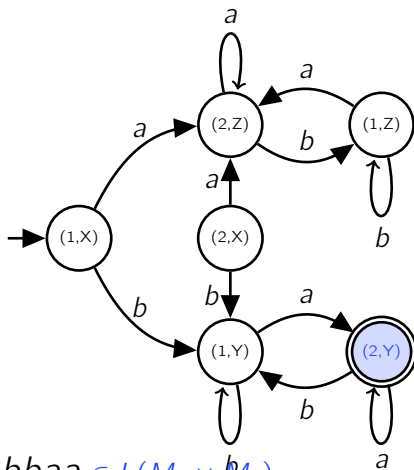


Wort: *bbaa*

Beispiel für den Produktautomaten



Produktautomat $M_1 \times M_2$:



Wort: $bbaa \in L(M_1 \times M_2)$

Theorem (Abschlusseigenschaften der regulären Sprachen)

Die regulären Sprachen sind abgeschlossen bezüglich **Vereinigung**, **Schnitt**, **Komplementbildung**, **Produkt** und **Kleeneschem Abschluss**.

Nichtregularität zeigen mit Abschlusseigenschaften

Anleitung zum Widerlegen der Regularität von L mit Abschlusseigenschaften:

1. Nehme an, dass L regulär ist.
2. Operiere auf L unter Erhaltung der Regularität:
vereinige, schneide, komplementiere, multipliziere L mit bekannt regulärer Sprache, bilde Kleeneschen Abschluss.
3. Kommt dabei eine bekannt **nicht reguläre** Sprache heraus,
dann hat man einen **Widerspruch** und die Annahme war falsch.
Daher ist L dann nicht regulär.

Nichtregularität zeigen mit Abschlusseigenschaften

Satz

Die Sprache $L = \{a^n \mid n \text{ ist keine Primzahl}\}$ ist nicht regulär.

Nichtregularität zeigen mit Abschlusseigenschaften

Satz

Die Sprache $L = \{a^n \mid n \text{ ist keine Primzahl}\}$ ist nicht regulär.

Beweis Durch Widerspruch. Wir nehmen an, L ist regulär.

Nichtregularität zeigen mit Abschlusseigenschaften

Satz

Die Sprache $L = \{a^n \mid n \text{ ist keine Primzahl}\}$ ist nicht regulär.

Beweis Durch Widerspruch. Wir nehmen an, L ist regulär.
Dann ist $\overline{L}$ auch regulär.

Nichtregularität zeigen mit Abschlusseigenschaften

Satz

Die Sprache $L = \{a^n \mid n \text{ ist keine Primzahl}\}$ ist nicht regulär.

Beweis Durch Widerspruch. Wir nehmen an, L ist regulär.

Dann ist $\bar{L}$ auch regulär.

$\bar{L} = \{a\}^* \setminus L = \{a^n \mid n \text{ ist eine Primzahl}\}$ ist aber nicht regulär (bereits gezeigt).

Widerspruch. □

Nichtregularität zeigen mit Abschlusseigenschaften

Satz

Die Sprache $L = \{w \in \{a, b\}^* \mid \#_a(w) = \#_b(w)\}$ ist nicht regulär.

Nichtregularität zeigen mit Abschlusseigenschaften

Satz

Die Sprache $L = \{w \in \{a, b\}^* \mid \#_a(w) = \#_b(w)\}$ ist nicht regulär.

Beweis Durch Widerspruch. Wir nehmen an, L ist regulär.

Nichtregularität zeigen mit Abschlusseigenschaften

Satz

Die Sprache $L = \{w \in \{a, b\}^* \mid \#_a(w) = \#_b(w)\}$ ist nicht regulär.

Beweis Durch Widerspruch. Wir nehmen an, L ist regulär.

Die Sprache $L' = \{a^m b^n \mid m, n \in \mathbb{N}\}$ ist regulär, da der reguläre Ausdruck $a^* b^*$ sie erzeugt.

Nichtregularität zeigen mit Abschlusseigenschaften

Satz

Die Sprache $L = \{w \in \{a, b\}^* \mid \#_a(w) = \#_b(w)\}$ ist nicht regulär.

Beweis Durch Widerspruch. Wir nehmen an, L ist regulär.

Die Sprache $L' = \{a^m b^n \mid m, n \in \mathbb{N}\}$ ist regulär, da der reguläre Ausdruck $a^* b^*$ sie erzeugt.

Da L und L' regulär sind, ist auch $L \cap L'$ regulär.

Nichtregularität zeigen mit Abschlusseigenschaften

Satz

Die Sprache $L = \{w \in \{a, b\}^* \mid \#_a(w) = \#_b(w)\}$ ist nicht regulär.

Beweis Durch Widerspruch. Wir nehmen an, L ist regulär.

Die Sprache $L' = \{a^m b^n \mid m, n \in \mathbb{N}\}$ ist regulär, da der reguläre Ausdruck $a^* b^*$ sie erzeugt.

Da L und L' regulär sind, ist auch $L \cap L'$ regulär.

$L \cap L' = \{a^j b^j \mid j \in \mathbb{N}\}$ ist aber nicht regulär (bereits gezeigt). Widerspruch. □

Wortproblem für reguläre Grammatiken

Definition

Das **Wortproblem** für Typ i -Grammatiken ist die Frage, ob für eine gegebene Typ i -Grammatik $G = (V, \Sigma, P, S)$ und ein Wort $w \in \Sigma^*$ $w \in L(G)$ gilt oder nicht.

Wortproblem für reguläre Grammatiken

Definition

Das **Wortproblem** für Typ i -Grammatiken ist die Frage, ob für eine gegebene Typ i -Grammatik $G = (V, \Sigma, P, S)$ und ein Wort $w \in \Sigma^*$ $w \in L(G)$ gilt oder nicht.

Satz

Das Wortproblem für reguläre Grammatiken ist entscheidbar:
Es gibt einen Algorithmus, der bei Eingabe von regulärer Grammatik G und Wort w nach endlicher Zeit entscheidet, ob $w \in L(G)$ gilt oder nicht.

Wortproblem für reguläre Grammatiken

Definition

Das **Wortproblem** für Typ i -Grammatiken ist die Frage, ob für eine gegebene Typ i -Grammatik $G = (V, \Sigma, P, S)$ und ein Wort $w \in \Sigma^*$ $w \in L(G)$ gilt oder nicht.

Satz

Das Wortproblem für reguläre Grammatiken ist entscheidbar:
Es gibt einen Algorithmus, der bei Eingabe von regulärer Grammatik G und Wort w nach endlicher Zeit entscheidet, ob $w \in L(G)$ gilt oder nicht.

Beweis Sei G eine reguläre Grammatik und sei M ein DFA mit $L(M) = L(G)$.
Für M ist das Wortproblem in Linearzeit in der Länge des Wortes lösbar, denn die Berechnung von $\tilde{\delta}(z_0, w)$ braucht für einen DFA nur $|w|$ Schritte. □

Leerheitsproblem für reguläre Grammatiken

Satz

Das Leerheitsproblem für reguläre Grammatiken ist entscheidbar.

Leerheitsproblem für reguläre Grammatiken

Satz

Das Leerheitsproblem für reguläre Grammatiken ist entscheidbar.

Beweis Sei G eine reguläre Grammatik und sei M ein DFA mit $L(M) = L(G)$.

Leerheitsproblem für reguläre Grammatiken

Satz

Das Leerheitsproblem für reguläre Grammatiken ist entscheidbar.

Beweis Sei G eine reguläre Grammatik und sei M ein DFA mit $L(M) = L(G)$.

Dann gilt $L(M) = \emptyset$ g.d.w. es keinen Pfad vom Startzustand zu einem Endzustand in M gibt.

Leerheitsproblem für reguläre Grammatiken

Satz

Das Leerheitsproblem für reguläre Grammatiken ist entscheidbar.

Beweis Sei G eine reguläre Grammatik und sei M ein DFA mit $L(M) = L(G)$.

Dann gilt $L(M) = \emptyset$ g.d.w. es keinen Pfad vom Startzustand zu einem Endzustand in M gibt.

Dies kann man leicht mit einer Tiefensuche (Depth-First-Search) auf dem Zustandsgraphen von M prüfen. □

Endlichkeitsproblem für reguläre Grammatiken

Satz

Das Endlichkeitsproblem für reguläre Grammatiken ist entscheidbar.

Endlichkeitsproblem für reguläre Grammatiken

Satz

Das Endlichkeitsproblem für reguläre Grammatiken ist entscheidbar.

Beweis Sei G eine reguläre Grammatik und sei M ein DFA mit $L(M) = L(G)$.

Endlichkeitsproblem für reguläre Grammatiken

Satz

Das Endlichkeitsproblem für reguläre Grammatiken ist entscheidbar.

Beweis Sei G eine reguläre Grammatik und sei M ein DFA mit $L(M) = L(G)$.

Es gilt $|L(M)| < \infty$ g.d.w. es keinen Pfad vom Startzustand zu einem Endzustand gibt, der eine Schleife enthält.

Endlichkeitsproblem für reguläre Grammatiken

Satz

Das Endlichkeitsproblem für reguläre Grammatiken ist entscheidbar.

Beweis Sei G eine reguläre Grammatik und sei M ein DFA mit $L(M) = L(G)$.

Es gilt $|L(M)| < \infty$ g.d.w. es keinen Pfad vom Startzustand zu einem Endzustand gibt, der eine Schleife enthält.

Prüfe dies mit einer Tiefensuche auf dem Zustandsgraphen von M . □

Schnittproblem für reguläre Grammatiken

Satz

Das Schnittproblem für reguläre Grammatiken ist entscheidbar.

Schnittproblem für reguläre Grammatiken

Satz

Das Schnittproblem für reguläre Grammatiken ist entscheidbar.

Beweis Seien G_1, G_2 reguläre Grammatiken und seien M_1, M_2 DFAs mit $L(M_i) = L(G_i)$ für $i \in \{1, 2\}$.

Schnittproblem für reguläre Grammatiken

Satz

Das Schnittproblem für reguläre Grammatiken ist entscheidbar.

Beweis Seien G_1, G_2 reguläre Grammatiken und seien M_1, M_2 DFAs mit $L(M_i) = L(G_i)$ für $i \in \{1, 2\}$.

Konstruiere den Produktautomaten $M_1 \times M_2$ mit $L(M_1 \times M_2) = L(M_1) \cap L(M_2)$.

Schnittproblem für reguläre Grammatiken

Satz

Das Schnittproblem für reguläre Grammatiken ist entscheidbar.

Beweis Seien G_1, G_2 reguläre Grammatiken und seien M_1, M_2 DFAs mit $L(M_i) = L(G_i)$ für $i \in \{1, 2\}$.

Konstruiere den Produktautomaten $M_1 \times M_2$ mit $L(M_1 \times M_2) = L(M_1) \cap L(M_2)$.

Prüfe das Leerheitsproblem für $L(M_1 \times M_2)$. □

Äquivalenzproblem für reguläre Grammatiken

Satz

Das Äquivalenzproblem für reguläre Grammatiken ist entscheidbar.

Äquivalenzproblem für reguläre Grammatiken

Satz

Das Äquivalenzproblem für reguläre Grammatiken ist entscheidbar.

Beweis Seien G_1, G_2 reguläre Grammatiken und seien M_1, M_2 DFAs mit $L(M_i) = L(G_i)$ für $i \in \{1, 2\}$.

Äquivalenzproblem für reguläre Grammatiken

Satz

Das Äquivalenzproblem für reguläre Grammatiken ist entscheidbar.

Beweis Seien G_1, G_2 reguläre Grammatiken und seien M_1, M_2 DFAs mit $L(M_i) = L(G_i)$ für $i \in \{1, 2\}$.

Minimiere M_1 und M_2 .

Äquivalenzproblem für reguläre Grammatiken

Satz

Das Äquivalenzproblem für reguläre Grammatiken ist entscheidbar.

Beweis Seien G_1, G_2 reguläre Grammatiken und seien M_1, M_2 DFAs mit $L(M_i) = L(G_i)$ für $i \in \{1, 2\}$.

Minimiere M_1 und M_2 .

Prüfe die minimalen DFAs auf Gleichheit bis auf Umbenennung.



5a

Der Satz von Myhill und Nerode

Prof. Dr. Jasmin Blanchette

Lehr- und Forschungseinheit für
Theoretische Informatik und TheorembeweisenStand: 11. Juni 2024
Basierend auf Folien von PD Dr. David Sabel

Die Nerode-Relation

Definition

Sei L eine Sprache über Σ . Die **Nerode-Relation** $\sim_L \subseteq \Sigma^* \times \Sigma^*$ zu L ist definiert für alle Wörter $u, v \in \Sigma^*$, sodass $u \sim_L v$ wenn

für jedes $w \in \Sigma^*$: $uw \in L$ g.d.w. $vw \in L$

Die Nerode-Relation

Definition

Sei L eine Sprache über Σ . Die **Nerode-Relation** $\sim_L \subseteq \Sigma^* \times \Sigma^*$ zu L ist definiert für alle Wörter $u, v \in \Sigma^*$, sodass $u \sim_L v$ wenn

für jedes $w \in \Sigma^*$: $uw \in L$ g.d.w. $vw \in L$

Informell: $u \sim_L v$, wenn sich ihr Enthaltensein in L gleich verhält bezüglich beliebiger Erweiterung um dasselbe Suffix.

Die Nerode-Relation

Definition

Sei L eine Sprache über Σ . Die **Nerode-Relation** $\sim_L \subseteq \Sigma^* \times \Sigma^*$ zu L ist definiert für alle Wörter $u, v \in \Sigma^*$, sodass $u \sim_L v$ wenn

für jedes $w \in \Sigma^*$: $uw \in L$ g.d.w. $vw \in L$

Informell: $u \sim_L v$, wenn sich ihr Enthaltensein in L gleich verhält bezüglich beliebiger Erweiterung um dasselbe Suffix.

Sei die Sprache $L = \{ab^n \mid n \in \mathbb{N}\}$ über $\Sigma = \{a, b\}$.

1. Gilt $a \sim_L ab$?
2. Gilt $aa u \sim_L bv$ mit $u, v \in \Sigma^*$?
3. Gilt $\varepsilon \sim_L a$?
4. Gilt $aa u \sim_L aa v$ mit $u, v \in \{b\}^*$?
5. Gilt $ab^i \sim_L ab^j$?
6. Gilt $a \sim_L b$?

Die Nerode-Relation

Definition

Sei L eine Sprache über Σ . Die **Nerode-Relation** $\sim_L \subseteq \Sigma^* \times \Sigma^*$ zu L ist definiert für alle Wörter $u, v \in \Sigma^*$, sodass $u \sim_L v$ wenn

für jedes $w \in \Sigma^*$: $uw \in L$ g.d.w. $vw \in L$

Informell: $u \sim_L v$, wenn sich ihr Enthaltensein in L gleich verhält bezüglich beliebiger Erweiterung um dasselbe Suffix.

Sei die Sprache $L = \{ab^n \mid n \in \mathbb{N}\}$ über $\Sigma = \{a, b\}$.

1. Gilt $a \sim_L ab$? **Ja**
2. Gilt $aa u \sim_L bv$ mit $u, v \in \Sigma^*$?
3. Gilt $\varepsilon \sim_L a$?
4. Gilt $aa u \sim_L aa v$ mit $u, v \in \{b\}^*$?
5. Gilt $ab^i \sim_L ab^j$?
6. Gilt $a \sim_L b$?

Die Nerode-Relation

Definition

Sei L eine Sprache über Σ . Die **Nerode-Relation** $\sim_L \subseteq \Sigma^* \times \Sigma^*$ zu L ist definiert für alle Wörter $u, v \in \Sigma^*$, sodass $u \sim_L v$ wenn

für jedes $w \in \Sigma^*$: $uw \in L$ g.d.w. $vw \in L$

Informell: $u \sim_L v$, wenn sich ihr Enthaltensein in L gleich verhält bezüglich beliebiger Erweiterung um dasselbe Suffix.

Sei die Sprache $L = \{ab^n \mid n \in \mathbb{N}\}$ über $\Sigma = \{a, b\}$.

1. Gilt $a \sim_L ab$? **Ja**
2. Gilt $aa u \sim_L bv$ mit $u, v \in \Sigma^*$? **Ja**
3. Gilt $\varepsilon \sim_L a$?
4. Gilt $aa u \sim_L aa v$ mit $u, v \in \{b\}^*$?
5. Gilt $ab^i \sim_L ab^j$?
6. Gilt $a \sim_L b$?

Die Nerode-Relation

Definition

Sei L eine Sprache über Σ . Die **Nerode-Relation** $\sim_L \subseteq \Sigma^* \times \Sigma^*$ zu L ist definiert für alle Wörter $u, v \in \Sigma^*$, sodass $u \sim_L v$ wenn

für jedes $w \in \Sigma^*$: $uw \in L$ g.d.w. $vw \in L$

Informell: $u \sim_L v$, wenn sich ihr Enthaltensein in L gleich verhält bezüglich beliebiger Erweiterung um dasselbe Suffix.

Sei die Sprache $L = \{ab^n \mid n \in \mathbb{N}\}$ über $\Sigma = \{a, b\}$.

- | | |
|--|---|
| 1. Gilt $a \sim_L ab$? Ja | 4. Gilt $aa u \sim_L aa v$ mit $u, v \in \{b\}^*$? |
| 2. Gilt $aa u \sim_L bv$ mit $u, v \in \Sigma^*$? Ja | 5. Gilt $ab^i \sim_L ab^j$? |
| 3. Gilt $\varepsilon \sim_L a$? Nein (z.B. $w = \varepsilon$) | 6. Gilt $a \sim_L b$? |

Die Nerode-Relation

Definition

Sei L eine Sprache über Σ . Die **Nerode-Relation** $\sim_L \subseteq \Sigma^* \times \Sigma^*$ zu L ist definiert für alle Wörter $u, v \in \Sigma^*$, sodass $u \sim_L v$ wenn

für jedes $w \in \Sigma^*$: $uw \in L$ g.d.w. $vw \in L$

Informell: $u \sim_L v$, wenn sich ihr Enthaltensein in L gleich verhält bezüglich beliebiger Erweiterung um dasselbe Suffix.

Sei die Sprache $L = \{ab^n \mid n \in \mathbb{N}\}$ über $\Sigma = \{a, b\}$.

- | | |
|--|---|
| 1. Gilt $a \sim_L ab$? Ja | 4. Gilt $aa u \sim_L aa v$ mit $u, v \in \{b\}^*$? Ja |
| 2. Gilt $aa u \sim_L bv$ mit $u, v \in \Sigma^*$? Ja | 5. Gilt $ab^i \sim_L ab^j$? |
| 3. Gilt $\varepsilon \sim_L a$? Nein (z.B. $w = \varepsilon$) | 6. Gilt $a \sim_L b$? |

Die Nerode-Relation

Definition

Sei L eine Sprache über Σ . Die **Nerode-Relation** $\sim_L \subseteq \Sigma^* \times \Sigma^*$ zu L ist definiert für alle Wörter $u, v \in \Sigma^*$, sodass $u \sim_L v$ wenn

für jedes $w \in \Sigma^*$: $uw \in L$ g.d.w. $vw \in L$

Informell: $u \sim_L v$, wenn sich ihr Enthaltensein in L gleich verhält bezüglich beliebiger Erweiterung um dasselbe Suffix.

Sei die Sprache $L = \{ab^n \mid n \in \mathbb{N}\}$ über $\Sigma = \{a, b\}$.

- | | |
|--|---|
| 1. Gilt $a \sim_L ab$? Ja | 4. Gilt $aa u \sim_L aa v$ mit $u, v \in \{b\}^*$? Ja |
| 2. Gilt $aa u \sim_L bv$ mit $u, v \in \Sigma^*$? Ja | 5. Gilt $ab^i \sim_L ab^j$? Ja |
| 3. Gilt $\varepsilon \sim_L a$? Nein (z.B. $w = \varepsilon$) | 6. Gilt $a \sim_L b$? |

Die Nerode-Relation

Definition

Sei L eine Sprache über Σ . Die **Nerode-Relation** $\sim_L \subseteq \Sigma^* \times \Sigma^*$ zu L ist definiert für alle Wörter $u, v \in \Sigma^*$, sodass $u \sim_L v$ wenn

für jedes $w \in \Sigma^*$: $uw \in L$ g.d.w. $vw \in L$

Informell: $u \sim_L v$, wenn sich ihr Enthaltensein in L gleich verhält bezüglich beliebiger Erweiterung um dasselbe Suffix.

Sei die Sprache $L = \{ab^n \mid n \in \mathbb{N}\}$ über $\Sigma = \{a, b\}$.

- | | |
|--|---|
| 1. Gilt $a \sim_L ab$? Ja | 4. Gilt $aa u \sim_L aa v$ mit $u, v \in \{b\}^*$? Ja |
| 2. Gilt $aa u \sim_L bv$ mit $u, v \in \Sigma^*$? Ja | 5. Gilt $ab^i \sim_L ab^j$? Ja |
| 3. Gilt $\varepsilon \sim_L a$? Nein (z.B. $w = \varepsilon$) | 6. Gilt $a \sim_L b$? Nein (z.B. $w = \varepsilon$) |

Die Nerode-Relation als Äquivalenzrelation

Satz

Die Nerode-Relation ist eine Äquivalenzrelation.

Die Nerode-Relation als Äquivalenzrelation

Satz

Die Nerode-Relation ist eine Äquivalenzrelation.

Beweis Der Satz folgt daraus, dass „g.d.w.“ eine Äquivalenzrelation ist. □

Die Nerode-Relation als Äquivalenzrelation

Satz

Die Nerode-Relation ist eine Äquivalenzrelation.

Beweis Der Satz folgt daraus, dass „g.d.w.“ eine Äquivalenzrelation ist. □

Zur Erinnerung:

- ▶ Die **Äquivalenzklasse** $[u]_{\sim_L}$ ist definiert als $\{v \mid u \sim_L v\}$.
- ▶ Der **Index** einer Äquivalenzrelation ist die **Anzahl ihrer disjunkten Äquivalenzklassen**: $\Sigma^* = [u_1]_{\sim_L} \cup [u_2]_{\sim_L} \cup \dots$.
- ▶ Der Index kann unendlich sein.

Beispiel für den Index der Nerode-Relation

Sei $L = \{ab^n \mid n \in \mathbb{N}\}$ eine Sprache über $\Sigma = \{a, b\}$.

$\text{Index}(\sim_L) = ?$

Beispiel für den Index der Nerode-Relation

Sei $L = \{ab^n \mid n \in \mathbb{N}\}$ eine Sprache über $\Sigma = \{a, b\}$.

$\text{Index}(\sim_L) = ?$

► $[\varepsilon]_{\sim_L} = \{\varepsilon\}$

Beispiel für den Index der Nerode-Relation

Sei $L = \{ab^n \mid n \in \mathbb{N}\}$ eine Sprache über $\Sigma = \{a, b\}$.

$\text{Index}(\sim_L) = ?$

- ▶ $[\varepsilon]_{\sim_L} = \{\varepsilon\}$
- ▶ $[a]_{\sim_L} = \{a, ab, abb, abbb, \dots\}$

Beispiel für den Index der Nerode-Relation

Sei $L = \{ab^n \mid n \in \mathbb{N}\}$ eine Sprache über $\Sigma = \{a, b\}$.

$\text{Index}(\sim_L) = ?$

- ▶ $[\varepsilon]_{\sim_L} = \{\varepsilon\}$
- ▶ $[a]_{\sim_L} = \{a, ab, abb, abbb, \dots\}$
- ▶ $[aa]_{\sim_L} = \{aa u \mid u \in \Sigma^*\}$
 $\cup \{bu \mid u \in \Sigma^*\}$
 $\cup \{abuav \mid u, v \in \Sigma^*\}$

Beispiel für den Index der Nerode-Relation

Sei $L = \{ab^n \mid n \in \mathbb{N}\}$ eine Sprache über $\Sigma = \{a, b\}$.

$Index(\sim_L) = ?$

- ▶ $[\epsilon]_{\sim_L} = \{\epsilon\}$
- ▶ $[a]_{\sim_L} = \{a, ab, abb, abbb, \dots\}$
- ▶ $[aa]_{\sim_L} = \{aa u \mid u \in \Sigma^*\}$
 $\cup \{bu \mid u \in \Sigma^*\}$
 $\cup \{abuav \mid u, v \in \Sigma^*\}$

$$\Sigma^* = [\epsilon]_{\sim_L} \cup [a]_{\sim_L} \cup [aa]_{\sim_L}$$

Beispiel für den Index der Nerode-Relation

Sei $L = \{ab^n \mid n \in \mathbb{N}\}$ eine Sprache über $\Sigma = \{a, b\}$.

$$\text{Index}(\sim_L) = 3$$

- ▶ $[\varepsilon]_{\sim_L} = \{\varepsilon\}$
- ▶ $[a]_{\sim_L} = \{a, ab, abb, abbb, \dots\}$
- ▶ $[aa]_{\sim_L} = \{aa u \mid u \in \Sigma^*\}$
 $\cup \{bu \mid u \in \Sigma^*\}$
 $\cup \{abuav \mid u, v \in \Sigma^*\}$

$$\Sigma^* = [\varepsilon]_{\sim_L} \cup [a]_{\sim_L} \cup [aa]_{\sim_L}$$

Definition

Sei L eine Sprache über Σ mit $\text{Index}(\sim_L) = n \in \mathbb{N}_{>0}$.

Seien $\Sigma^* = [u_1]_{\sim_L} \cup \dots \cup [u_n]_{\sim_L}$.

Der **Nerode-Automat** ist der DFA $M = (Z, \Sigma, \delta, [\varepsilon]_{\sim_L}, E)$ mit

$$\begin{aligned} Z &:= \{[u_1]_{\sim_L}, \dots, [u_n]_{\sim_L}\} \\ \delta([u_i]_{\sim_L}, a) &:= [u_i a]_{\sim_L} \\ E &:= \{[u_i]_{\sim_L} \mid u_i \in L, i \in \{1, \dots, n\}\} \end{aligned}$$

Definition

Sei L eine Sprache über Σ mit $\text{Index}(\sim_L) = n \in \mathbb{N}_{>0}$.

Seien $\Sigma^* = [u_1]_{\sim_L} \cup \dots \cup [u_n]_{\sim_L}$.

Der **Nerode-Automat** ist der DFA $M = (Z, \Sigma, \delta, [\varepsilon]_{\sim_L}, E)$ mit

$$\begin{aligned} Z &:= \{[u_1]_{\sim_L}, \dots, [u_n]_{\sim_L}\} \\ \delta([u_i]_{\sim_L}, a) &:= [u_i a]_{\sim_L} \\ E &:= \{[u_i]_{\sim_L} \mid u_i \in L, i \in \{1, \dots, n\}\} \end{aligned}$$

Wir werden zeigen, dass der Nerode-Automat **minimal** ist. Aber im Gegensatz zum Äquivalenzklassenautomaten kann man ihn im Allgemeinen **nicht berechnen**.

Beispiel für den Nerode-Automaten

Sprache $L = \{ab^n \mid n \in \mathbb{N}\}$ über $\Sigma = \{a, b\}^*$ mit

- ▶ $[\varepsilon]_{\sim_L} = \{\varepsilon\}$
- ▶ $[a]_{\sim_L} = \{a, ab, abb, abbb, \dots\}$
- ▶ $[aa]_{\sim_L} = \{aa u \mid u \in \Sigma^*\}$
 $\cup \{bu \mid u \in \Sigma^*\}$
 $\cup \{abuav \mid u, v \in \Sigma^*\}$

$$\Sigma^* = [\varepsilon]_{\sim_L} \cup [a]_{\sim_L} \cup [aa]_{\sim_L}$$

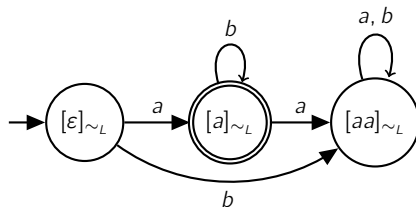
Beispiel für den Nerode-Automaten

Sprache $L = \{ab^n \mid n \in \mathbb{N}\}$ über $\Sigma = \{a, b\}^*$ mit

- ▶ $[\varepsilon]_{\sim_L} = \{\varepsilon\}$
- ▶ $[a]_{\sim_L} = \{a, ab, abb, abbb, \dots\}$
- ▶ $[aa]_{\sim_L} = \{aa u \mid u \in \Sigma^*\}$
 $\cup \{bu \mid u \in \Sigma^*\}$
 $\cup \{abuav \mid u, v \in \Sigma^*\}$

$$\Sigma^* = [\varepsilon]_{\sim_L} \cup [a]_{\sim_L} \cup [aa]_{\sim_L}$$

Nerode-Automat zu L :



Der Satz von Myhill und Nerode

Unabhängig bewiesen von John Myhill (1957) und Anil Nerode (1958):

Theorem (Satz von Myhill und Nerode)

Eine Sprache L ist genau dann regulär, wenn der Index von $\sim_L$ endlich ist.

Der Satz von Myhill und Nerode

Unabhängig bewiesen von John Myhill (1957) und Anil Nerode (1958):

Theorem (Satz von Myhill und Nerode)

Eine Sprache L ist genau dann regulär, wenn der Index von $\sim_L$ endlich ist.

Damit haben wir eine **genaue** Charakterisierung der regulären Sprachen (im Gegensatz zum Pumping-Lemma).

Der Satz von Myhill und Nerode

Beweis

1. Wir zeigen zuerst, dass wenn L regulär ist, dann ist der Index von $\sim_L$ endlich.

Der Satz von Myhill und Nerode

Beweis

1. Wir zeigen zuerst, dass wenn L regulär ist, dann ist der Index von $\sim_L$ endlich.

Sei L regulär und $M = (Z, \Sigma, \delta, z_0, E)$ ein DFA mit $L(M) = L$.

Der Satz von Myhill und Nerode

Beweis

1. Wir zeigen zuerst, dass wenn L regulär ist, dann ist der Index von $\sim_L$ endlich.

Sei L regulär und $M = (Z, \Sigma, \delta, z_0, E)$ ein DFA mit $L(M) = L$.

Sei $\approx_M \subseteq \Sigma^* \times \Sigma^*$ definiert, sodass $u \approx_M v$ wenn $\tilde{\delta}(z_0, u) = \tilde{\delta}(z_0, v)$.

D.h. $u \approx_M v$, wenn u, v auf M zum gleichen Zustand führen.

Der Satz von Myhill und Nerode

Beweis

1. Wir zeigen zuerst, dass wenn L regulär ist, dann ist der Index von $\sim_L$ endlich.

Sei L regulär und $M = (Z, \Sigma, \delta, z_0, E)$ ein DFA mit $L(M) = L$.

Sei $\approx_M \subseteq \Sigma^* \times \Sigma^*$ definiert, sodass $u \approx_M v$ wenn $\tilde{\delta}(z_0, u) = \tilde{\delta}(z_0, v)$.

D.h. $u \approx_M v$, wenn u, v auf M zum gleichen Zustand führen.

$\approx_M$ ist eine Äquivalenzrelation.

Der Satz von Myhill und Nerode

Beweis

1. Wir zeigen zuerst, dass wenn L regulär ist, dann ist der Index von $\sim_L$ endlich.

Sei L regulär und $M = (Z, \Sigma, \delta, z_0, E)$ ein DFA mit $L(M) = L$.

Sei $\approx_M \subseteq \Sigma^* \times \Sigma^*$ definiert, sodass $u \approx_M v$ wenn $\tilde{\delta}(z_0, u) = \tilde{\delta}(z_0, v)$.

D.h. $u \approx_M v$, wenn u, v auf M zum gleichen Zustand führen.

$\approx_M$ ist eine Äquivalenzrelation.

$\text{Index}(\approx_M) \leq |Z|$ (und damit endlich).

Der Satz von Myhill und Nerode

Beweis

1. Wir zeigen zuerst, dass wenn L regulär ist, dann ist der Index von $\sim_L$ endlich.

Sei L regulär und $M = (Z, \Sigma, \delta, z_0, E)$ ein DFA mit $L(M) = L$.

Sei $\approx_M \subseteq \Sigma^* \times \Sigma^*$ definiert, sodass $u \approx_M v$ wenn $\tilde{\delta}(z_0, u) = \tilde{\delta}(z_0, v)$.

D.h. $u \approx_M v$, wenn u, v auf M zum gleichen Zustand führen.

$\approx_M$ ist eine Äquivalenzrelation.

$\text{Index}(\approx_M) \leq |Z|$ (und damit endlich).

Wir zeigen: aus $u \approx_M v$ folgt $u \sim_L v$ (d.h. $\approx_M$ verfeinert $\sim_L$).

Dann gilt auch $\text{Index}(\sim_L) \leq \text{Index}(\approx_M) \leq |Z|$ (und damit endlich).

Der Satz von Myhill und Nerode

Beweis

1. Wir zeigen zuerst, dass wenn L regulär ist, dann ist der Index von $\sim_L$ endlich.

Sei L regulär und $M = (Z, \Sigma, \delta, z_0, E)$ ein DFA mit $L(M) = L$.

Sei $\approx_M \subseteq \Sigma^* \times \Sigma^*$ definiert, sodass $u \approx_M v$ wenn $\tilde{\delta}(z_0, u) = \tilde{\delta}(z_0, v)$.

D.h. $u \approx_M v$, wenn u, v auf M zum gleichen Zustand führen.

$\approx_M$ ist eine Äquivalenzrelation.

$Index(\approx_M) \leq |Z|$ (und damit endlich).

Wir zeigen: aus $u \approx_M v$ folgt $u \sim_L v$ (d.h. $\approx_M$ verfeinert $\sim_L$).

Dann gilt auch $Index(\sim_L) \leq Index(\approx_M) \leq |Z|$ (und damit endlich).

Seien $u \approx_M v$ und $w \in \Sigma^*$. Dann gilt

$$\tilde{\delta}(z_0, uw) = \tilde{\delta}(\tilde{\delta}(z_0, u), w) = \tilde{\delta}(\tilde{\delta}(z_0, v), w) = \tilde{\delta}(z_0, vw)$$

und damit $uw \in L$ g.d.w. $vw \in L$. Da w beliebig war, zeigt dies $u \sim_L v$.

Beweis

2. Wir zeigen nun, dass wenn der Index von $\sim_L$ endlich ist, dann ist L regulär.

Beweis

- Wir zeigen nun, dass wenn der Index von $\sim_L$ endlich ist, dann ist L regulär.
Sei der Index von $\sim_L$ endlich.

Beweis

2. Wir zeigen nun, dass wenn der Index von $\sim_L$ endlich ist, dann ist L regulär.

Sei der Index von $\sim_L$ endlich.

Sei $M = (Z, \Sigma, \delta, [\varepsilon]_{\sim_L}, E)$ der Nerode-Automat. Wir zeigen $L(M) = L$.
Somit ist L regulär.

Beweis

2. Wir zeigen nun, dass wenn der Index von $\sim_L$ endlich ist, dann ist L regulär.

Sei der Index von $\sim_L$ endlich.

Sei $M = (Z, \Sigma, \delta, [\varepsilon]_{\sim_L}, E)$ der Nerode-Automat. Wir zeigen $L(M) = L$.

Somit ist L regulär.

$$w \in L(M)$$

Der Satz von Myhill und Nerode

Beweis

2. Wir zeigen nun, dass wenn der Index von $\sim_L$ endlich ist, dann ist L regulär.

Sei der Index von $\sim_L$ endlich.

Sei $M = (Z, \Sigma, \delta, [\varepsilon]_{\sim_L}, E)$ der Nerode-Automat. Wir zeigen $L(M) = L$.

Somit ist L regulär.

$w \in L(M)$
g.d.w. $\tilde{\delta}([\varepsilon]_{\sim_L}, w) \in E$

Der Satz von Myhill und Nerode

Beweis

2. Wir zeigen nun, dass wenn der Index von $\sim_L$ endlich ist, dann ist L regulär.

Sei der Index von $\sim_L$ endlich.

Sei $M = (Z, \Sigma, \delta, [\varepsilon]_{\sim_L}, E)$ der Nerode-Automat. Wir zeigen $L(M) = L$.

Somit ist L regulär.

$$w \in L(M)$$

$$\text{g.d.w. } \tilde{\delta}([\varepsilon]_{\sim_L}, w) \in E$$

$$\text{g.d.w. } [w]_{\sim_L} \in E$$

Der Satz von Myhill und Nerode

Beweis

2. Wir zeigen nun, dass wenn der Index von $\sim_L$ endlich ist, dann ist L regulär.

Sei der Index von $\sim_L$ endlich.

Sei $M = (Z, \Sigma, \delta, [\varepsilon]_{\sim_L}, E)$ der Nerode-Automat. Wir zeigen $L(M) = L$.

Somit ist L regulär.

$w \in L(M)$
g.d.w. $\tilde{\delta}([\varepsilon]_{\sim_L}, w) \in E$
g.d.w. $[w]_{\sim_L} \in E$
g.d.w. $w \in L$



Anwendung des Satzes von Myhill und Nerode

Da der Satz von Myhill und Nerode eine **hinreichende und notwendige** Bedingung für reguläre Sprachen liefert, kann man ihn verwenden, um

1. Regularität nachzuweisen: Zeige $Index(\sim_L)$ ist endlich.
2. Nichtregularität nachzuweisen: Zeige $Index(\sim_L)$ ist unendlich.

Anwendung des Satzes von Myhill und Nerode

Da der Satz von Myhill und Nerode eine **hinreichende und notwendige** Bedingung für reguläre Sprachen liefert, kann man ihn verwenden, um

1. Regularität nachzuweisen: Zeige $\text{Index}(\sim_L)$ ist endlich.

Oft einfacher:

- ▶ *Gib endlichen Automaten an, der L akzeptiert.*
- ▶ *Gib regulären Ausdruck an, der L erzeugt.*
- ▶ *Gib reguläre Grammatik an, die L erzeugt.*

2. Nichtregularität nachzuweisen: Zeige $\text{Index}(\sim_L)$ ist unendlich.

Anwendung des Satzes von Myhill und Nerode

Da der Satz von Myhill und Nerode eine **hinreichende und notwendige** Bedingung für reguläre Sprachen liefert, kann man ihn verwenden, um

1. Regularität nachzuweisen: Zeige $\text{Index}(\sim_L)$ ist endlich.

Oft einfacher:

- ▶ *Gib endlichen Automaten an, der L akzeptiert.*
- ▶ *Gib regulären Ausdruck an, der L erzeugt.*
- ▶ *Gib reguläre Grammatik an, die L erzeugt.*

2. Nichtregularität nachzuweisen: Zeige $\text{Index}(\sim_L)$ ist unendlich.

Wird insbesondere dann benutzt, wenn das Pumping-Lemma nicht funktioniert.

Wie zeigt man $\text{Index}(\sim_L) = \infty$?

Grundgedanke:

- Finde unendlich viele Äquivalenzklassen $[u_1]_{\sim_L}, [u_2]_{\sim_L}, \dots$, die paarweise verschieden sind ($[u_i]_{\sim_L} \neq [u_j]_{\sim_L}$ für $i \neq j$).

Wie zeigt man $\text{Index}(\sim_L) = \infty$?

Grundgedanke:

- Finde unendlich viele Äquivalenzklassen $[u_1]_{\sim_L}, [u_2]_{\sim_L}, \dots$, die paarweise verschieden sind ($[u_i]_{\sim_L} \neq [u_j]_{\sim_L}$ für $i \neq j$).

Rezept:

- Finde für $i = 1, 2, \dots$ Wörter u_i und w_i , sodass $u_i w_i \in L$ aber $u_j w_i \notin L$ für alle $i \neq j$.

Wie zeigt man $\text{Index}(\sim_L) = \infty$?

Grundgedanke:

- Finde unendlich viele Äquivalenzklassen $[u_1]_{\sim_L}, [u_2]_{\sim_L}, \dots$, die paarweise verschieden sind ($[u_i]_{\sim_L} \neq [u_j]_{\sim_L}$ für $i \neq j$).

Rezept:

- Finde für $i = 1, 2, \dots$ Wörter u_i und w_i , sodass $u_i w_i \in L$ aber $u_j w_i \notin L$ für alle $i \neq j$.

Dann sind $u_i \not\sim_L u_j$ für alle $i \neq j$.

Wie zeigt man $\text{Index}(\sim_L) = \infty$?

Grundgedanke:

- Finde unendlich viele Äquivalenzklassen $[u_1]_{\sim_L}, [u_2]_{\sim_L}, \dots$, die paarweise verschieden sind ($[u_i]_{\sim_L} \neq [u_j]_{\sim_L}$ für $i \neq j$).

Rezept:

- Finde für $i = 1, 2, \dots$ Wörter u_i und w_i , sodass $u_i w_i \in L$ aber $u_j w_i \notin L$ für alle $i \neq j$.

Dann sind $u_i \not\sim_L u_j$ für alle $i \neq j$.

Daher sind $[u_1]_{\sim_L}, [u_2]_{\sim_L}, \dots$ paarweise disjunkt.

Wie zeigt man $\text{Index}(\sim_L) = \infty$?

Grundgedanke:

- Finde unendlich viele Äquivalenzklassen $[u_1]_{\sim_L}, [u_2]_{\sim_L}, \dots$, die paarweise verschieden sind ($[u_i]_{\sim_L} \neq [u_j]_{\sim_L}$ für $i \neq j$).

Rezept:

- Finde für $i = 1, 2, \dots$ Wörter u_i und w_i , sodass $u_i w_i \in L$ aber $u_j w_i \notin L$ für alle $i \neq j$.

Dann sind $u_i \not\sim_L u_j$ für alle $i \neq j$.

Daher sind $[u_1]_{\sim_L}, [u_2]_{\sim_L}, \dots$ paarweise disjunkt.

Beachte: Es ist hierfür **nicht** notwendig, alle Äquivalenzklassen der disjunkten Zerlegung von Σ^* zu finden.

Nichtregularität zeigen mit dem Satz von Myhill und Nerode

Satz

Die Sprache $L = \{a^n b^n \mid n \in \mathbb{N}_{>0}\}$ ist nicht regulär.

Beweis Für $u_i = a^i b$ für $i \in \mathbb{N}_{>0}$ gilt: $[u_i]_{\sim_L}$ besteht aus allen Wörtern, denen noch $i - 1$ b 's fehlen, um in L enthalten zu sein.

Nichtregularität zeigen mit dem Satz von Myhill und Nerode

Satz

Die Sprache $L = \{a^n b^n \mid n \in \mathbb{N}_{>0}\}$ ist nicht regulär.

Beweis Für $u_i = a^i b$ für $i \in \mathbb{N}_{>0}$ gilt: $[u_i]_{\sim_L}$ besteht aus allen Wörtern, denen noch $i - 1$ b 's fehlen, um in L enthalten zu sein. D.h.

$$\begin{aligned}[u_1]_{\sim_L} &= \{ab, a^2b^2, \dots\} = L \\[u_2]_{\sim_L} &= \{a^2b, a^3b^2, a^4b^3, \dots\} \\[u_3]_{\sim_L} &= \{a^3b, a^4b^2, a^5b^3, \dots\} \\&\vdots\end{aligned}$$

Nichtregularität zeigen mit dem Satz von Myhill und Nerode

Satz

Die Sprache $L = \{a^n b^n \mid n \in \mathbb{N}_{>0}\}$ ist nicht regulär.

Beweis Für $u_i = a^i b$ für $i \in \mathbb{N}_{>0}$ gilt: $[u_i]_{\sim_L}$ besteht aus allen Wörtern, denen noch $i - 1$ b 's fehlen, um in L enthalten zu sein. D.h.

$$\begin{aligned}[u_1]_{\sim_L} &= \{ab, a^2b^2, \dots\} = L \\[u_2]_{\sim_L} &= \{a^2b, a^3b^2, a^4b^3, \dots\} \\[u_3]_{\sim_L} &= \{a^3b, a^4b^2, a^5b^3, \dots\} \\&\vdots\end{aligned}$$

Sei $w_i = b^{i-1}$ für $i \in \mathbb{N}_{>0}$.

Nichtregularität zeigen mit dem Satz von Myhill und Nerode

Satz

Die Sprache $L = \{a^n b^n \mid n \in \mathbb{N}_{>0}\}$ ist nicht regulär.

Beweis Für $u_i = a^i b$ für $i \in \mathbb{N}_{>0}$ gilt: $[u_i]_{\sim_L}$ besteht aus allen Wörtern, denen noch $i - 1$ b 's fehlen, um in L enthalten zu sein. D.h.

$$\begin{aligned}[u_1]_{\sim_L} &= \{ab, a^2b^2, \dots\} = L \\[u_2]_{\sim_L} &= \{a^2b, a^3b^2, a^4b^3, \dots\} \\[u_3]_{\sim_L} &= \{a^3b, a^4b^2, a^5b^3, \dots\} \\&\vdots\end{aligned}$$

Sei $w_i = b^{i-1}$ für $i \in \mathbb{N}_{>0}$.

$u_i w_i = a^i b^i \in L$, aber $u_j w_i = a^j b^i \notin L$ für $i \neq j$.

Nichtregularität zeigen mit dem Satz von Myhill und Nerode

Satz

Die Sprache $L = \{a^n b^n \mid n \in \mathbb{N}_{>0}\}$ ist nicht regulär.

Beweis Für $u_i = a^i b$ für $i \in \mathbb{N}_{>0}$ gilt: $[u_i]_{\sim_L}$ besteht aus allen Wörtern, denen noch $i - 1$ b 's fehlen, um in L enthalten zu sein. D.h.

$$\begin{aligned}[u_1]_{\sim_L} &= \{ab, a^2b^2, \dots\} = L \\[u_2]_{\sim_L} &= \{a^2b, a^3b^2, a^4b^3, \dots\} \\[u_3]_{\sim_L} &= \{a^3b, a^4b^2, a^5b^3, \dots\} \\&\vdots\end{aligned}$$

Sei $w_i = b^{i-1}$ für $i \in \mathbb{N}_{>0}$.

$u_i w_i = a^i b^i \in L$, aber $u_j w_i = a^j b^i \notin L$ für $i \neq j$.

Daher gilt $[u_i]_{\sim_L} \neq [u_j]_{\sim_L}$ für $i \neq j$ und $\text{Index}(\sim_L) = \infty$.

Nichtregularität zeigen mit dem Satz von Myhill und Nerode

Satz

Die Sprache $L = \{a^n b^n \mid n \in \mathbb{N}_{>0}\}$ ist nicht regulär.

Beweis Für $u_i = a^i b$ für $i \in \mathbb{N}_{>0}$ gilt: $[u_i]_{\sim_L}$ besteht aus allen Wörtern, denen noch $i - 1$ b 's fehlen, um in L enthalten zu sein. D.h.

$$\begin{aligned}[u_1]_{\sim_L} &= \{ab, a^2b^2, \dots\} = L \\[u_2]_{\sim_L} &= \{a^2b, a^3b^2, a^4b^3, \dots\} \\[u_3]_{\sim_L} &= \{a^3b, a^4b^2, a^5b^3, \dots\} \\&\vdots\end{aligned}$$

Sei $w_i = b^{i-1}$ für $i \in \mathbb{N}_{>0}$.

$u_i w_i = a^i b^i \in L$, aber $u_j w_i = a^j b^i \notin L$ für $i \neq j$.

Daher gilt $[u_i]_{\sim_L} \neq [u_j]_{\sim_L}$ für $i \neq j$ und $\text{Index}(\sim_L) = \infty$.

Mit dem Satz von Myhill und Nerode folgt, dass L nicht regulär ist. □

Eine nicht reguläre Sprache, die die Pumping-Eigenschaft hat

Satz

Die Sprache $L = \{a^j b^k c^k \mid j, k \in \mathbb{N}_{>0}\}$ ist nicht regulär.

Eine nicht reguläre Sprache, die die Pumping-Eigenschaft hat

Satz

Die Sprache $L = \{a^j b^k c^k \mid j, k \in \mathbb{N}_{>0}\}$ ist nicht regulär.

Beweis Sei $u_i = ab^i c$ für $i \in \mathbb{N}_{>0}$.

Eine nicht reguläre Sprache, die die Pumping-Eigenschaft hat

Satz

Die Sprache $L = \{a^j b^k c^k \mid j, k \in \mathbb{N}_{>0}\}$ ist nicht regulär.

Beweis Sei $u_i = ab^i c$ für $i \in \mathbb{N}_{>0}$.

Sei $w_i = c^{i-1}$.

Eine nicht reguläre Sprache, die die Pumping-Eigenschaft hat

Satz

Die Sprache $L = \{a^j b^k c^k \mid j, k \in \mathbb{N}_{>0}\}$ ist nicht regulär.

Beweis Sei $u_i = ab^i c$ für $i \in \mathbb{N}_{>0}$.

Sei $w_i = c^{i-1}$.

$u_i w_i = ab^i c^i \in L$, aber $u_j w_i = ab^j c^i \notin L$ für $i \neq j$.

Eine nicht reguläre Sprache, die die Pumping-Eigenschaft hat

Satz

Die Sprache $L = \{a^j b^k c^k \mid j, k \in \mathbb{N}_{>0}\}$ ist nicht regulär.

Beweis Sei $u_i = ab^i c$ für $i \in \mathbb{N}_{>0}$.

Sei $w_i = c^{i-1}$.

$u_i w_i = ab^i c^i \in L$, aber $u_j w_i = ab^j c^i \notin L$ für $i \neq j$.

Daher sind alle $[u_i]_{\sim_L}$ disjunkt und $\text{Index}(\sim_L) = \infty$.

Eine nicht reguläre Sprache, die die Pumping-Eigenschaft hat

Satz

Die Sprache $L = \{a^j b^k c^k \mid j, k \in \mathbb{N}_{>0}\}$ ist nicht regulär.

Beweis Sei $u_i = ab^i c$ für $i \in \mathbb{N}_{>0}$.

Sei $w_i = c^{i-1}$.

$u_i w_i = ab^i c^i \in L$, aber $u_j w_i = ab^j c^i \notin L$ für $i \neq j$.

Daher sind alle $[u_i]_{\sim_L}$ disjunkt und $\text{Index}(\sim_L) = \infty$.

Mit dem Satz von Myhill und Nerode folgt, dass L nicht regulär ist. □

Regularität zeigen mit dem Satz von Myhill und Nerode

Sei $L = \{ab^n \mid n \in \mathbb{N}\}$ eine Sprache über $\Sigma = \{a, b\}$.

Die disjunkten Äquivalenzklassen von $\sim_L$ sind

- ▶ $[\varepsilon]_{\sim_L} = \{\varepsilon\}$
- ▶ $[a]_{\sim_L} = \{a, ab, abb, abbb, \dots\}$
- ▶ $[aa]_{\sim_L} = \{aa u \mid u \in \Sigma^*\}$
 $\cup \{bu \mid u \in \Sigma^*\}$
 $\cup \{abuav \mid u, v \in \Sigma^*\}$

Regularität zeigen mit dem Satz von Myhill und Nerode

Sei $L = \{ab^n \mid n \in \mathbb{N}\}$ eine Sprache über $\Sigma = \{a, b\}$.

Die disjunkten Äquivalenzklassen von $\sim_L$ sind

- ▶ $[\varepsilon]_{\sim_L} = \{\varepsilon\}$
- ▶ $[a]_{\sim_L} = \{a, ab, abb, abbb, \dots\}$
- ▶ $[aa]_{\sim_L} = \{aa u \mid u \in \Sigma^*\}$
 $\cup \{bu \mid u \in \Sigma^*\}$
 $\cup \{abuav \mid u, v \in \Sigma^*\}$

Daher folgt $\text{Index}(\sim_L) = 3$ und mit dem Satz von Myhill und Nerode folgt, dass L regulär ist.

Satz

Der Nerode-Automat zu einer regulären Sprache L ist ein **minimaler DFA**, d.h. jeder DFA M' , mit $L(M') = L$ hat mindestens so viele Zustände wie der **Nerode-Automat**. Zudem sind alle minimalen DFAs bis auf Umbenennung von Zuständen identisch.

Beweis

1. Wir zeigen zuerst Minimalität.

Satz

Der Nerode-Automat zu einer regulären Sprache L ist ein **minimaler DFA**, d.h. jeder DFA M' , mit $L(M') = L$ hat mindestens so viele Zustände wie der **Nerode-Automat**. Zudem sind alle minimalen DFAs bis auf Umbenennung von Zuständen identisch.

Beweis

1. Wir zeigen zuerst Minimalität.

Seien $M = (Z, \Sigma, \delta, z_0, E)$ der Nerode-Automat zur Sprache L und $M' = (Z', \Sigma, \delta', z'_0, E')$ ein beliebiger DFA mit $L(M') = L$.

Satz

Der Nerode-Automat zu einer regulären Sprache L ist ein **minimaler DFA**, d.h. jeder DFA M' , mit $L(M') = L$ hat mindestens so viele Zustände wie der **Nerode-Automat**. Zudem sind alle minimalen DFAs bis auf Umbenennung von Zuständen identisch.

Beweis

1. Wir zeigen zuerst Minimalität.

Seien $M = (Z, \Sigma, \delta, z_0, E)$ der Nerode-Automat zur Sprache L und $M' = (Z', \Sigma, \delta', z'_0, E')$ ein beliebiger DFA mit $L(M') = L$.

Wir zeigen $|Z'| \geq |Z|$.

Satz

Der Nerode-Automat zu einer regulären Sprache L ist ein **minimaler DFA**, d.h. jeder DFA M' , mit $L(M') = L$ hat mindestens so viele Zustände wie der **Nerode-Automat**. Zudem sind alle minimalen DFAs bis auf Umbenennung von Zuständen identisch.

Beweis

1. Wir zeigen zuerst Minimalität.

Seien $M = (Z, \Sigma, \delta, z_0, E)$ der Nerode-Automat zur Sprache L und $M' = (Z', \Sigma, \delta', z'_0, E')$ ein beliebiger DFA mit $L(M') = L$.

Wir zeigen $|Z'| \geq |Z|$.

Der Beweis des Satzes von Myhill und Nerode zeigt: $\approx_{M'}$ verfeinert $\sim_L$.

Satz

Der Nerode-Automat zu einer regulären Sprache L ist ein **minimaler DFA**, d.h. jeder DFA M' , mit $L(M') = L$ hat mindestens so viele Zustände wie der **Nerode-Automat**. Zudem sind alle minimalen DFAs bis auf Umbenennung von Zuständen identisch.

Beweis

1. Wir zeigen zuerst Minimalität.

Seien $M = (Z, \Sigma, \delta, z_0, E)$ der Nerode-Automat zur Sprache L und $M' = (Z', \Sigma, \delta', z'_0, E')$ ein beliebiger DFA mit $L(M') = L$.

Wir zeigen $|Z'| \geq |Z|$.

Der Beweis des Satzes von Myhill und Nerode zeigt: $\approx_{M'}$ verfeinert $\sim_L$.
Zudem $\text{Index}(\sim_L) = \text{Index}(\approx_M)$.

Satz

Der Nerode-Automat zu einer regulären Sprache L ist ein **minimaler DFA**, d.h. jeder DFA M' , mit $L(M') = L$ hat mindestens so viele Zustände wie der **Nerode-Automat**. Zudem sind alle minimalen DFAs bis auf Umbenennung von Zuständen identisch.

Beweis

- Wir zeigen zuerst Minimalität.

Seien $M = (Z, \Sigma, \delta, z_0, E)$ der Nerode-Automat zur Sprache L und $M' = (Z', \Sigma, \delta', z'_0, E')$ ein beliebiger DFA mit $L(M') = L$.

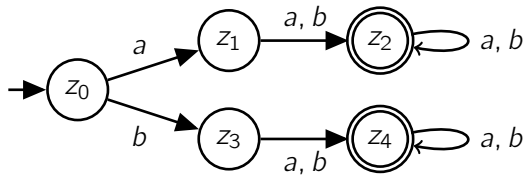
Wir zeigen $|Z'| \geq |Z|$.

Der Beweis des Satzes von Myhill und Nerode zeigt: $\approx_{M'}$ verfeinert $\sim_L$.
Zudem $\text{Index}(\sim_L) = \text{Index}(\approx_M)$.

Daher ist $\text{Index}(\approx_{M'}) \geq \text{Index}(\approx_M)$ und daher auch
 $|Z'| \geq \text{Index}(\approx_{M'}) \geq \text{Index}(\approx_M) = \text{Index}(\sim_L) = |Z|$.

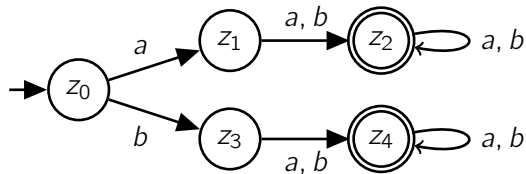
Beispiel für Minimalität

Beliebiger DFA M :

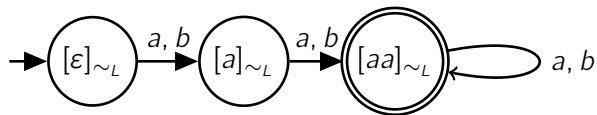


Beispiel für Minimalität

Beliebiger DFA M :



Nerode-Automat M' :



Beweis (Fortsetzung)

2. Wir zeigen nun, dass alle minimalen DFAs bis auf Umbenennung von Zuständen identisch sind.

Beweis (Fortsetzung)

2. Wir zeigen nun, dass alle minimalen DFAs bis auf Umbenennung von Zuständen identisch sind.

Sei $M = (Z, \Sigma, \delta, z_0, E)$ der Nerode-Automat zur Sprache L und $M' = (Z', \Sigma, \delta', z'_0, E')$ ein beliebiger minimaler DFA mit $L(M') = L$.

Beweis (Fortsetzung)

- Wir zeigen nun, dass alle minimalen DFAs bis auf Umbenennung von Zuständen identisch sind.

Sei $M = (Z, \Sigma, \delta, z_0, E)$ der Nerode-Automat zur Sprache L und $M' = (Z', \Sigma, \delta', z'_0, E')$ ein beliebiger minimaler DFA mit $L(M') = L$.

Minimalität impliziert $|Z'| = |Z|$.

Beweis (Fortsetzung)

- Wir zeigen nun, dass alle minimalen DFAs bis auf Umbenennung von Zuständen identisch sind.

Sei $M = (Z, \Sigma, \delta, z_0, E)$ der Nerode-Automat zur Sprache L und $M' = (Z', \Sigma, \delta', z'_0, E')$ ein beliebiger minimaler DFA mit $L(M') = L$.

Minimalität impliziert $|Z'| = |Z|$.

Da $\approx_{M'} \sim_L$ verfeinert, gilt entweder $\text{Index}(\approx_{M'}) > \text{Index}(\sim_L)$ oder $\approx_{M'} = \sim_L$. Im ersten Fall hätten wir $|Z'| \geq \text{Index}(\approx_{M'}) > \text{Index}(\sim_L) = |Z|$. Widerspruch. Daher $\approx_{M'} = \sim_L$.

Beweis (Fortsetzung)

2. Wir zeigen nun, dass alle minimalen DFAs bis auf Umbenennung von Zuständen identisch sind.

Sei $M = (Z, \Sigma, \delta, z_0, E)$ der Nerode-Automat zur Sprache L und $M' = (Z', \Sigma, \delta', z'_0, E')$ ein beliebiger minimaler DFA mit $L(M') = L$.

Minimalität impliziert $|Z'| = |Z|$.

Da $\approx_{M'} \sim_L$ verfeinert, gilt entweder $\text{Index}(\approx_{M'}) > \text{Index}(\sim_L)$ oder $\approx_{M'} = \sim_L$. Im ersten Fall hätten wir $|Z'| \geq \text{Index}(\approx_{M'}) > \text{Index}(\sim_L) = |Z|$. Widerspruch.

Daher $\approx_{M'} = \sim_L$.

Da $\approx_{M'} = \sim_L = \approx_M$, folgt dass M und M' strukturell identisch sind (siehe Skript).



Minimale NFAs

Wir haben gezeigt, dass alle **minimalen DFAs** bis auf Umbenennung **identisch** sind.

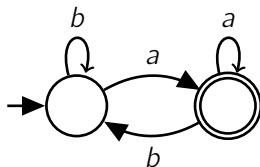
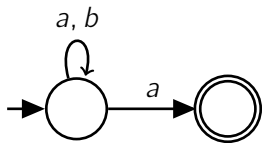
Minimale NFAs

Wir haben gezeigt, dass alle **minimalen DFAs** bis auf Umbenennung **identisch** sind.
Für **NFAs** gilt das im Allgemeinen **nicht**.

Minimale NFAs

Wir haben gezeigt, dass alle **minimalen DFAs** bis auf Umbenennung **identisch** sind.
Für **NFAs** gilt das im Allgemeinen **nicht**.

Gegenbeispiel:



Beide NFAs erkennen $\{ua \mid u \in \{a, b\}^*\}$ und haben eine minimale Zustandsanzahl, sind aber **strukturell verschieden**.

Wiederholung: Äquivalenzklassenautomat

Definition

Sei $M = (Z, \Sigma, \delta, z_0, E)$ ein DFA.

Wir nennen zwei Zustände $z, z' \in Z$ äquivalent und schreiben $z \equiv_M z'$ (alternativ $z \equiv z'$), falls gilt: $\tilde{\delta}(z, w) \in E$ g.d.w. $\tilde{\delta}(z', w) \in E$ für alle $w \in \Sigma^*$.

Der Äquivalenzklassenautomat zu M ist der DFA $M' = (Z', \Sigma, \delta', z'_0, E')$ mit

$$Z' := \{[z]_{\equiv} \mid z \in Z\}$$

$$z'_0 := [z_0]_{\equiv}$$

$$E' := \{[z]_{\equiv} \mid z \in E\}$$

$$\delta'([z]_{\equiv}, a) := [\delta(z, a)]_{\equiv}$$

Informell: Zwei Zustände sind äquivalent, wenn sie die gleiche „Sprache“ darstellen.

Korrektheit und Minimalität des Äquivalenzklassenautomaten

Satz

Seien $M = (Z, \Sigma, \delta, z_0, E)$ ein DFA und $M' = (Z', \Sigma, \delta', z'_0, E')$ der Äquivalenzklassenautomat zu M . Dann gilt

1. $L(M') = L(M)$.
2. Falls alle Zustände in Z von z_0 erreichbar sind, dann ist M' minimal, d.h. jeder DFA M'' mit $L(M'') = L(M')$ hat mindestens so viele Zustände wie M' .

Korrektheit und Minimalität des Äquivalenzklassenautomaten

Satz

Seien $M = (Z, \Sigma, \delta, z_0, E)$ ein DFA und $M' = (Z', \Sigma, \delta', z'_0, E')$ der Äquivalenzklassenautomat zu M . Dann gilt

1. $L(M') = L(M)$.
2. Falls alle Zustände in Z von z_0 erreichbar sind, dann ist M' minimal, d.h. jeder DFA M'' mit $L(M'') = L(M')$ hat mindestens so viele Zustände wie M' .

Beweis

1. Wurde in früherer Vorlesung gezeigt.

Korrektheit und Minimalität des Äquivalenzklassenautomaten

Beweis (Fortsetzung)

- Wir müssen zeigen, dass falls alle Zustände in Z von z_0 erreichbar sind, dann ist M' minimal, d.h. jeder DFA M'' mit $L(M'') = L(M')$ hat mindestens so viele Zustände wie M' .

Korrektheit und Minimalität des Äquivalenzklassenautomaten

Beweis (Fortsetzung)

- Wir müssen zeigen, dass falls alle Zustände in Z von z_0 erreichbar sind, dann ist M' minimal, d.h. jeder DFA M'' mit $L(M'') = L(M')$ hat mindestens so viele Zustände wie M' .

Sei $L = L(M') = L(M)$.

Korrektheit und Minimalität des Äquivalenzklassenautomaten

Beweis (Fortsetzung)

- Wir müssen zeigen, dass falls alle Zustände in Z von z_0 erreichbar sind, dann ist M' minimal, d.h. jeder DFA M'' mit $L(M'') = L(M')$ hat mindestens so viele Zustände wie M' .

Sei $L = L(M') = L(M)$.

Da alle $z \in Z$ erreichbar sind, sind alle $z' \in Z'$ erreichbar.

Korrektheit und Minimalität des Äquivalenzklassenautomaten

Beweis (Fortsetzung)

2. Wir müssen zeigen, dass falls alle Zustände in Z von z_0 erreichbar sind, dann ist M' minimal, d.h. jeder DFA M'' mit $L(M'') = L(M')$ hat mindestens so viele Zustände wie M' .

Sei $L = L(M') = L(M)$.

Da alle $z \in Z$ erreichbar sind, sind alle $z' \in Z'$ erreichbar.

Für Minimalität genügt es zu zeigen, dass M' nicht mehr Zustände hat als der Nerode-Automat: $|Z'| \leq \text{Index}(\sim_L)$.

Korrektheit und Minimalität des Äquivalenzklassenautomaten

Beweis (Fortsetzung)

2. Wir müssen zeigen, dass falls alle Zustände in Z von z_0 erreichbar sind, dann ist M' minimal, d.h. jeder DFA M'' mit $L(M'') = L(M')$ hat mindestens so viele Zustände wie M' .

Sei $L = L(M') = L(M)$.

Da alle $z \in Z$ erreichbar sind, sind alle $z' \in Z'$ erreichbar.

Für Minimalität genügt es zu zeigen, dass M' nicht mehr Zustände hat als der Nerode-Automat: $|Z'| \leq \text{Index}(\sim_L)$.

Wir zeigen gleich für jede Äquivalenzklasse $[v]_{\sim_L}$:

Für alle $u \in [v]_{\sim_L}$ ist $\tilde{\delta}(z'_0, u)$ derselbe Zustand.

Dann gibt es nicht mehr als $\text{Index}(\sim_L)$ erreichbare Zustände.

Korrektheit und Minimalität des Äquivalenzklassenautomaten

Beweis (Fortsetzung)

- Wir müssen zeigen, dass falls alle Zustände in Z von z_0 erreichbar sind, dann ist M' minimal, d.h. jeder DFA M'' mit $L(M'') = L(M')$ hat mindestens so viele Zustände wie M' .

Sei $L = L(M') = L(M)$.

Da alle $z \in Z$ erreichbar sind, sind alle $z' \in Z'$ erreichbar.

Für Minimalität genügt es zu zeigen, dass M' nicht mehr Zustände hat als der Nerode-Automat: $|Z'| \leq \text{Index}(\sim_L)$.

Wir zeigen gleich für jede Äquivalenzklasse $[v]_{\sim_L}$:

Für alle $u \in [v]_{\sim_L}$ ist $\tilde{\delta}(z'_0, u)$ derselbe Zustand.

Dann gibt es nicht mehr als $\text{Index}(\sim_L)$ erreichbare Zustände.

Da alle $z' \in Z'$ erreichbar sind, gilt damit $|Z'| \leq \text{Index}(\sim_L)$.

Korrektheit und Minimalität des Äquivalenzklassenautomaten

Beweis (Fortsetzung)

Seien $u, u' \in [v]_{\sim_L}$, d.h. $u \sim_L u'$. Dann gilt

Korrektheit und Minimalität des Äquivalenzklassenautomaten

Beweis (Fortsetzung)

Seien $u, u' \in [v]_{\sim_L}$, d.h. $u \sim_L u'$. Dann gilt

$uw \in L$ g.d.w. $u'w \in L$ für jedes $w \in \Sigma^*$

Korrektheit und Minimalität des Äquivalenzklassenautomaten

Beweis (Fortsetzung)

Seien $u, u' \in [v]_{\sim_L}$, d.h. $u \sim_L u'$. Dann gilt

$uw \in L$ g.d.w. $u'w \in L$ für jedes $w \in \Sigma^*$

woraus folgt: $\tilde{\delta}'(z'_0, uw) \in E'$ g.d.w. $\tilde{\delta}'(z'_0, u'w) \in E'$ für jedes $w \in \Sigma^*$

Korrektheit und Minimalität des Äquivalenzklassenautomaten

Beweis (Fortsetzung)

Seien $u, u' \in [v]_{\sim_L}$, d.h. $u \sim_L u'$. Dann gilt

$uw \in L$ g.d.w. $u'w \in L$ für jedes $w \in \Sigma^*$

woraus folgt: $\tilde{\delta}'(z'_0, uw) \in E'$ g.d.w. $\tilde{\delta}'(z'_0, u'w) \in E'$ für jedes $w \in \Sigma^*$

woraus folgt: $\tilde{\delta}'(\tilde{\delta}'(z'_0, u), w) \in E'$ g.d.w. $\tilde{\delta}'(\tilde{\delta}'(z'_0, u'), w) \in E'$ für jedes $w \in \Sigma^*$

Korrektheit und Minimalität des Äquivalenzklassenautomaten

Beweis (Fortsetzung)

Seien $u, u' \in [v]_{\sim_L}$, d.h. $u \sim_L u'$. Dann gilt

$uw \in L$ g.d.w. $u'w \in L$ für jedes $w \in \Sigma^*$

woraus folgt: $\tilde{\delta}'(z'_0, uw) \in E'$ g.d.w. $\tilde{\delta}'(z'_0, u'w) \in E'$ für jedes $w \in \Sigma^*$

woraus folgt: $\tilde{\delta}'(\tilde{\delta}'(z'_0, u), w) \in E'$ g.d.w. $\tilde{\delta}'(\tilde{\delta}'(z'_0, u'), w) \in E'$ für jedes $w \in \Sigma^*$

woraus folgt: $\tilde{\delta}'(z'_0, u) = \tilde{\delta}'(z'_0, u')$ □

5b

Die Chomsky-Normalform

Prof. Dr. Jasmin Blanchette

Lehr- und Forschungseinheit für
Theoretische Informatik und Theorembeweisen

Stand: 14. Mai 2024

Basierend auf Folien von PD Dr. David Sabel



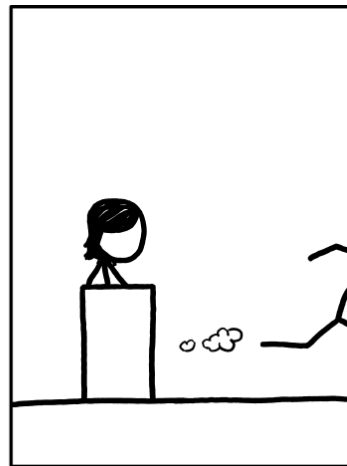
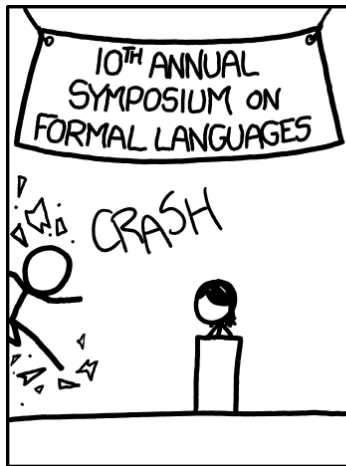
Zusammenfassung:

- ▶ **Kontextfreie Sprachen** (*context-free languages*, **CFLs**) werden von **kontextfreien Grammatiken** (*context-free grammars*, **CFGs**) erzeugt.
- ▶ Das sind die **Typ 2-Sprachen** bzw. die **Typ 2-Grammatiken**.
- ▶ Bedingung: Alle linken Seiten der Produktionen bestehen aus genau einer Variablen, d.h. die Produktionen sind von der Form $A \rightarrow r$.

Zusammenfassung:

- ▶ **Kontextfreie Sprachen** (*context-free languages*, **CFLs**) werden von **kontextfreien Grammatiken** (*context-free grammars*, **CFGs**) erzeugt.
- ▶ Das sind die **Typ 2-Sprachen** bzw. die **Typ 2-Grammatiken**.
- ▶ Bedingung: Alle linken Seiten der Produktionen bestehen aus genau einer Variablen, d.h. die Produktionen sind von der Form $A \rightarrow r$.

Kontextfreie Sprachen sind insbesondere nützlich um Sprachen mit Klammerungen zu beschreiben, z.B. Programmiersprachen.



xkcd.com/1090/

Beispiele für kontextfreie Sprachen

Die CFG $G_1 = (\{S, T\}, \{a, b\}, P_1, S)$ mit

$$P_1 := \{S \rightarrow \varepsilon \mid T, \\ T \rightarrow aTb \mid ab\}$$

erzeugt die Sprache $\{a^j b^j \mid j \in \mathbb{N}\}$, die daher kontextfrei ist.

Beispiele für kontextfreie Sprachen

Die CFG $G_1 = (\{S, T\}, \{a, b\}, P_1, S)$ mit

$$P_1 := \{S \rightarrow \varepsilon \mid T, \\ T \rightarrow aTb \mid ab\}$$

erzeugt die Sprache $\{a^j b^j \mid j \in \mathbb{N}\}$, die daher kontextfrei ist.

Die CFG $G_2 = (\{E, M, A, Z, R\}, \{+, *, (,)\} \cup \{0, \dots, 9\}, P_2, E)$ mit

$$P_2 := \{E \rightarrow M \mid E + M, \\ M \rightarrow A \mid M * A, \\ A \rightarrow Z \mid (E), \\ Z \rightarrow 1R \mid \dots \mid 9R, \\ R \rightarrow 0R \mid \dots \mid 9R \mid \varepsilon\}$$

erzeugt einfache **arithmetische Ausdrücke** nach der „Punkt vor Strich“-Regel.

Normalformen von Grammatiken

- ▶ **Normalformen** fordern eine spezielle Form der Produktionen.
- ▶ Sie sind nützlich, wenn man Grammatiken analysiert oder Algorithmen auf Grammatiken formuliert: Man muss dann nur diese Form (statt aller erlaubten) von Produktionen betrachten.
- ▶ Wir betrachten zwei Normalformen:
die **Chomsky-Normalform** und die **Greibach-Normalform**.

Die Chomsky-Normalform

Definition

Eine CFG $G = (V, \Sigma, P, S)$ ist in **Chomsky-Normalform**, wenn für jede Produktion $A \rightarrow w \in P$ gilt: $w = a \in \Sigma$ oder $w = BC$ mit $B, C \in V$.

Die Chomsky-Normalform

Definition

Eine CFG $G = (V, \Sigma, P, S)$ ist in **Chomsky-Normalform**, wenn für jede Produktion $A \rightarrow w \in P$ gilt: $w = a \in \Sigma$ oder $w = BC$ mit $B, C \in V$.

Beispiele:

- ▶ Die CFG $G = (\{A\}, \{ (,), [,] \}, \{ A \rightarrow (A) \mid () \mid [A] \mid [] \mid AA \}, A)$ ist **nicht** in Chomsky-Normalform. Nur die Produktion $A \rightarrow AA$ passt zum vorgeschriebenen Format.

Die Chomsky-Normalform

Definition

Eine CFG $G = (V, \Sigma, P, S)$ ist in **Chomsky-Normalform**, wenn für jede Produktion $A \rightarrow w \in P$ gilt: $w = a \in \Sigma$ oder $w = BC$ mit $B, C \in V$.

Beispiele:

- Die CFG $G = (\{A\}, \{ (,), [,] \}, \{ A \rightarrow (A) \mid () \mid [A] \mid [] \mid AA \}, A)$ ist **nicht** in Chomsky-Normalform. Nur die Produktion $A \rightarrow AA$ passt zum vorgeschriebenen Format.
- Die CFG $G' = (\{A, B, C, D, E, F, G\}, \{ (,), [,] \}, P, A)$ mit

$$P := \{ A \rightarrow BF \mid BC \mid DG \mid DE \mid AA, \\ B \rightarrow (, C \rightarrow), D \rightarrow [, E \rightarrow], F \rightarrow AC, G \rightarrow AE \}$$

ist in Chomsky-Normalform, und erzeugt dieselbe Sprache wie G .

Eigenschaften der Chomsky-Normalform

Für CFG G in Chomsky-Normalform gilt, dass Ableitungen eines Worts $w \in L(G)$ immer genau aus $2 \cdot |w| - 1$ Ableitungsschritten bestehen.

Zudem sind die dazugehörigen Syntaxbäume immer Binärbäume (Bäume, wobei jeder Knoten 0 oder 2 Kinder hat), bis auf die letzte Schicht, die Produktionen $A \rightarrow a$ anwendet.

Eigenschaften der Chomsky-Normalform

Für CFG G in Chomsky-Normalform gilt, dass Ableitungen eines Worts $w \in L(G)$ immer genau aus $2 \cdot |w| - 1$ Ableitungsschritten bestehen.

Zudem sind die dazugehörigen Syntaxbäume immer Binärbäume (Bäume, wobei jeder Knoten 0 oder 2 Kinder hat), bis auf die letzte Schicht, die Produktionen $A \rightarrow a$ anwendet.

Sei die CFG $G' = (\{A, B, C, D, E, F, G\}, \{ (,), [,] \}, P, A)$ mit

$$P := \{ A \rightarrow BF \mid BC \mid DG \mid DE \mid AA, \\ B \rightarrow (, C \rightarrow), D \rightarrow [, E \rightarrow], F \rightarrow AC, G \rightarrow AE \}$$

Eigenschaften der Chomsky-Normalform

Für CFG G in Chomsky-Normalform gilt, dass Ableitungen eines Worts $w \in L(G)$ immer genau aus $2 \cdot |w| - 1$ **Ableitungsschritten** bestehen.

Zudem sind die dazugehörigen Syntaxbäume immer **Binärbäume** (Bäume, wobei jeder Knoten 0 oder 2 Kinder hat), bis auf die letzte Schicht, die Produktionen $A \rightarrow a$ anwendet.

Sei die CFG $G' = (\{A, B, C, D, E, F, G\}, \{ (,), [,] \}, P, A)$ mit

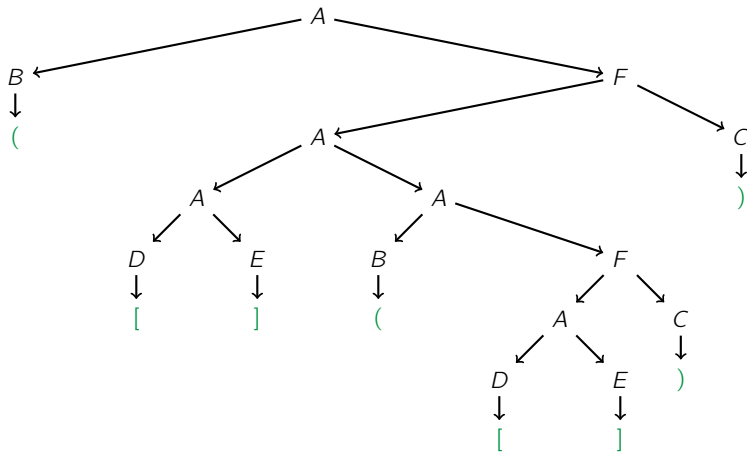
$$P := \{ A \rightarrow BF \mid BC \mid DG \mid DE \mid AA, \\ B \rightarrow (, C \rightarrow), D \rightarrow [, E \rightarrow], F \rightarrow AC, G \rightarrow AE \}$$

Ableitung von $([]([]))$:

$$\begin{aligned} A &\Rightarrow BF \Rightarrow (F \Rightarrow (AC \Rightarrow (AAC \Rightarrow (DEAC \Rightarrow ([EAC \\ &\Rightarrow ([]AC \Rightarrow ([]BFC \Rightarrow ([](FC \Rightarrow ([](ACC \Rightarrow ([](DECC \\ &\Rightarrow ([]([ECC \Rightarrow ([]([]CC \Rightarrow ([]([])C \Rightarrow ([]([])) \end{aligned}$$

Eigenschaften der Chomsky-Normalform

Der Syntaxbaum zur vorherigen Ableitung:



Herstellen der Chomsky-Normalform

Theorem

Für jede CFG G mit $\varepsilon \notin L(G)$ kann eine CFG G' in Chomsky-Normalform berechnet werden, sodass $L(G') = L(G)$ gilt.

Herstellen der Chomsky-Normalform

Theorem

Für jede CFG G mit $\varepsilon \notin L(G)$ kann eine CFG G' in Chomsky-Normalform berechnet werden, sodass $L(G') = L(G)$ gilt.

Grundgedanke der Prozedur:

1. Entferne ε -Produktionen (wie in früherer Vorlesung).
2. Entferne Produktionen von der Form $A \rightarrow B$ („Entfernen von Einheitsproduktionen“).
3. Ersetze alle Terminale a in rechten Seiten, die nicht nur aus a bestehen durch neue Variablen A und füge Produktionen $A \rightarrow a$ hinzu.
4. Zerlege alle Produktionen von der Form $A \rightarrow B_1 \cdots B_m$ mit $m > 2$ in mehrere:
 $A \rightarrow B_1 C_1, C_1 \rightarrow B_2 C_2, \dots, C_{m-2} \rightarrow B_{m-1} B_m$.

Beispiel für die Herstellung der Chomsky-Normalform

0. Eingabe: CFG $G = (\{A\}, \{ (,), [,] \}, \{ A \rightarrow (A) \mid () \mid [A] \mid [] \mid AA \}, A)$.

Beispiel für die Herstellung der Chomsky-Normalform

0. Eingabe: CFG $G = (\{A\}, \{ (,), [,] \}, \{ A \rightarrow (A) \mid () \mid [A] \mid [] \mid AA \}, A)$.
1. Es gibt keine ε -Produktionen zu entfernen.

Beispiel für die Herstellung der Chomsky-Normalform

0. Eingabe: CFG $G = (\{A\}, \{ (,), [,] \}, \{ A \rightarrow (A) \mid () \mid [A] \mid [] \mid AA \}, A)$.
1. Es gibt keine ε -Produktionen zu entfernen.
2. Es gibt keine Einheitsproduktionen zu entfernen.

Beispiel für die Herstellung der Chomsky-Normalform

0. Eingabe: CFG $G = (\{A\}, \{ (,), [,] \}, \{ A \rightarrow (A) \mid () \mid [A] \mid [] \mid AA \}, A)$.
1. Es gibt keine ε -Produktionen zu entfernen.
2. Es gibt keine Einheitsproduktionen zu entfernen.
3. Das Ersetzen von Terminale CFG führt zu
 $G_3 = (\{A, B, C, D, E\}, \{ (,), [,] \}, P_3, A)$ mit

$$P_3 := \{ A \rightarrow BAC \mid BC \mid DAE \mid DE \mid AA \} \\ B \rightarrow (, C \rightarrow), D \rightarrow [, E \rightarrow] \}$$

Beispiel für die Herstellung der Chomsky-Normalform

0. Eingabe: CFG $G = (\{A\}, \{ (,), [,] \}, \{ A \rightarrow (A) \mid () \mid [A] \mid [] \mid AA \}, A)$.
1. Es gibt keine ε -Produktionen zu entfernen.
2. Es gibt keine Einheitsproduktionen zu entfernen.
3. Das Ersetzen von Terminale CFG führt zu $G_3 = (\{A, B, C, D, E\}, \{ (,), [,] \}, P_3, A)$ mit

$$P_3 := \{ A \rightarrow BAC \mid BC \mid DAE \mid DE \mid AA \} \\ B \rightarrow (, C \rightarrow), D \rightarrow [, E \rightarrow] \}$$

4. Die Zerlegung von langen rechten Seiten führt zu $G' = (\{A, B, C, D, E, F, G\}, \{ (,), [,] \}, P', A)$ mit

$$P' := \{ A \rightarrow BF \mid BC \mid DG \mid DE \mid AA, \\ B \rightarrow (, C \rightarrow), D \rightarrow [, E \rightarrow], F \rightarrow AC, G \rightarrow AE \}$$

Entfernen von Einheitsproduktionen

Intuitiv ist klar, dass $A \rightarrow B$ entfernt werden kann:
Wenn erst $A \rightarrow B$, dann $B \rightarrow w$ angewendet wird,
kann man auch gleich $A \rightarrow w$ anwenden.

Entfernen von Einheitsproduktionen

Intuitiv ist klar, dass $A \rightarrow B$ entfernt werden kann:
Wenn erst $A \rightarrow B$, dann $B \rightarrow w$ angewendet wird,
kann man auch gleich $A \rightarrow w$ anwenden.

Algorithmisch zu beachten:

- ▶ Eliminiere in der richtigen Reihenfolge: Wenn $A \rightarrow B$ und $B \rightarrow C$, dann ist Ersetzen von $A \rightarrow B$ durch $A \rightarrow C$ nicht zielführend.
- ▶ Zyklen $A_1 \rightarrow A_2, A_2 \rightarrow A_3, \dots, A_n \rightarrow A_1$ müssen vorher entfernt werden.

Algorithmus 5: Entfernen von Einheitsproduktionen

Eingabe: Eine CFG $G = (V, \Sigma, P, S)$

Beginn

Erzeuge gerichteten Graph $D = (V, E)$, mit $(A, B) \in E$ für jede Einheitsproduktion $A \rightarrow B \in P$;

solange es einen Zyklus $(A_1, A_2), \dots, (A_{n-1}, A_n), (A_n, A_1) \in E$ gibt **tue**

```
     $P := P \setminus \{A_1 \rightarrow A_2, \dots, A_{n-1} \rightarrow A_n, A_n \rightarrow A_1\};$            /* entferne zyklische Regeln */  
     $P := P[A_1/A_2, \dots, A_1/A_n];$            /* ersetze alle Vorkommen von  $A_i$  durch  $A_1$  für  $i = 2, \dots, n$  */  
     $V := V \setminus \{A_2, A_3, \dots, A_n\};$            /* lösche  $A_2, \dots, A_n$  */  
     $S := S[A_1/A_2, \dots, A_1/A_n];$            /* ersetze Startsymbol durch  $A_1$ , falls es  $A_i$ ,  $2 \leq i \leq n$  war */  
     $E := E \setminus \{(A_1, A_2), \dots, (A_{n-1}, A_n), (A_n, A_1)\};$            /* entferne Zyklus aus Graph */  
     $E := E[A_1/A_2, \dots, A_1/A_n];$            /* ersetze  $A_i$  durch  $A_1$  für  $i = 2, \dots, n$  in anderen Kanten */
```

Sortiere D topologisch und nummeriere die Variablen in V durch (und benenne entsprechend in E, P, S um),
sodass gilt: $A_i \rightarrow A_j$ impliziert $i < j$;

Sei $V = \{A_1, \dots, A_k\}$;

für $i = k$ **bis** 1 **tue**

wenn $A_i \rightarrow A_j \in P$ **dann**

Seien $A_j \rightarrow w_1, \dots, A_j \rightarrow w_m$ alle Produktionen mit A_j als linker Seite;

```
     $P := P \cup \{A_i \rightarrow w_1, \dots, A_i \rightarrow w_m\};$            /* füge  $A_i \rightarrow w_k$  hinzu für  $A_i \rightarrow A_j$  und  $A_j \rightarrow w_k$  */
```

```
     $P := P \setminus \{A_i \rightarrow A_j\};$            /* entferne  $A_i \rightarrow A_j$  */
```

Gib die so entstandene Grammatik als G' aus;

Satz

Algorithmus 5 berechnet bei Eingabe einer CFG G mit $\varepsilon \notin L(G)$ eine CFG G' , die keine Einheitsproduktionen hat, sodass gilt $L(G') = L(G)$.

Wenn G keine ε -Produktionen hat, dann hat auch G' keine ε -Produktionen.

Satz

Algorithmus 5 berechnet bei Eingabe einer CFG G mit $\varepsilon \notin L(G)$ eine CFG G' , die keine Einheitsproduktionen hat, sodass gilt $L(G') = L(G)$.

Wenn G keine ε -Produktionen hat, dann hat auch G' keine ε -Produktionen.

Beweis Das Entfernen eines Zyklus ändert die erzeugte Sprache nicht.

Satz

Algorithmus 5 berechnet bei Eingabe einer CFG G mit $\varepsilon \notin L(G)$ eine CFG G' , die keine Einheitsproduktionen hat, sodass gilt $L(G') = L(G)$.

Wenn G keine ε -Produktionen hat, dann hat auch G' keine ε -Produktionen.

Beweis Das Entfernen eines Zyklus ändert die erzeugte Sprache nicht.

Das Entfernen einer Einheitsproduktion $A_i \rightarrow A_j$ in der rückwärts-laufenden für-Schleife ändert die erzeugte Sprache nicht. Dies ist eine Instanz der Operation „Inlining von Produktionen“, die wir gleich korrekt beweisen werden.

Beweis (Fortsetzung) Es werden keine Einheitsproduktionen eingeführt, da die Produktionen topologisch sortiert behandelt werden. Wenn $A_i \rightarrow A_j$ entfernt wird, wurden **vorher** alle Einheitsproduktionen $A_j \rightarrow A_k$ entfernt. D.h. zu diesem Zeitpunkt gilt: für alle Produktionen $A_j \rightarrow w$ besteht w nicht nur aus einer Variablen.

Beweis (Fortsetzung) Es werden keine Einheitsproduktionen eingeführt, da die Produktionen topologisch sortiert behandelt werden. Wenn $A_i \rightarrow A_j$ entfernt wird, wurden **vorher** alle Einheitsproduktionen $A_j \rightarrow A_k$ entfernt. D.h. zu diesem Zeitpunkt gilt: für alle Produktionen $A_j \rightarrow w$ besteht w nicht nur aus einer Variablen. Es werden offensichtlich nie ε -Produktionen eingeführt.

Korrektheit von Algorithmus 5

Beweis (Fortsetzung) Es werden keine Einheitsproduktionen eingeführt, da die Produktionen topologisch sortiert behandelt werden. Wenn $A_i \rightarrow A_j$ entfernt wird, wurden **vorher** alle Einheitsproduktionen $A_j \rightarrow A_k$ entfernt. D.h. zu diesem Zeitpunkt gilt: für alle Produktionen $A_j \rightarrow w$ besteht w nicht nur aus einer Variablen.

Es werden offensichtlich nie ε -Produktionen eingeführt.

Der Algorithmus terminiert:

- ▶ Die solange-Schleife terminiert, da jede Iteration die Anzahl der Zyklen strikt verkleinert.
- ▶ Die für-Schleife terminiert offensichtlich. □

Lemma (Inlining von Produktionen)

Sei $G = (V, \Sigma, P, S)$ eine CFG mit

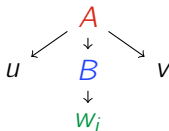
- ▶ $A \rightarrow uBv \in P$
- ▶ $B \rightarrow w_1 \mid \dots \mid w_n$ alle Regeln mit B als linker Seite

und sei $G' = (V, \Sigma, (P \setminus \{A \rightarrow uBv\}) \cup \{A \rightarrow uw_1v \mid \dots \mid uw_nv\}, S)$.

Dann gilt $L(G') = L(G)$.

Inlining von Produktionen

Beweis Das folgt durch Modifizieren der Syntaxbäume zur Ableitung mit G bzw. G' :
Tausche alle Baumabschnitte



im Syntaxbaum mit G durch



im Syntaxbaum mit G' .

□

Algorithmus 6: Herstellen der Chomsky-Normalform

Eingabe: CFG G mit $\varepsilon \notin L(G)$

Ausgabe: CFG G' in Chomsky-Normalform mit $L(G') = L(G)$

Beginn

Entferne die ε -Produktionen in G mit Algorithmus 1 und
entferne anschließend die Einheitsproduktionen mit Algorithmus 5;

Sei $G' = (V', \Sigma, P', S')$ die entstandene Grammatik;

für alle $a \in \Sigma$ **tue**

Sei B eine neue Variable;

/ Führe neue Variable B für a ein, und ersetze Vorkommen von a durch B */*

$G' := (V' \cup \{B\}, \Sigma, \{A \rightarrow w[B/a] \mid A \rightarrow w \in P' \text{ und } |w| > 1\}$
 $\quad \cup \{A \rightarrow w \mid A \rightarrow w \in P' \text{ und } |w| = 1\} \cup \{B \rightarrow a\}, S)$

/ Nun sind alle Regeln von der Form $A \rightarrow a$ oder $A \rightarrow B_1 \cdots B_m$ mit $m \geq 2$ */*

für alle $A \rightarrow B_1 \cdots B_m \in P'$ mit $m > 2$ **tue**

Seien $C_1, \dots, C_{m-2}$ neue Variablen;

$V' := V' \cup \{C_1, \dots, C_{m-2}\};$

/ Ersetze in P' die Produktion $A \rightarrow B_1 \cdots B_m$ durch neue Regeln */*

$P' := (P' \setminus \{A \rightarrow B_1 \cdots B_m\})$
 $\quad \cup \{A \rightarrow B_1 C_1\} \cup \{C_i \rightarrow B_{i+1} C_{i+1} \mid \text{für } i = 1, \dots, m-3\} \cup \{C_{m-2} \rightarrow B_{m-1} B_m\};$

Theorem

Für jede CFG G mit $\varepsilon \notin L(G)$ kann eine CFG G' in Chomsky-Normalform berechnet werden, sodass $L(G') = L(G)$ gilt.

Theorem

Für jede CFG G mit $\varepsilon \notin L(G)$ kann eine CFG G' in Chomsky-Normalform berechnet werden, sodass $L(G') = L(G)$ gilt.

Beweis Algorithmus 6 berechnet eine solche äquivalente CFG.

Korrektheit von Algorithmus 6

Theorem

Für jede CFG G mit $\varepsilon \notin L(G)$ kann eine CFG G' in Chomsky-Normalform berechnet werden, sodass $L(G') = L(G)$ gilt.

Beweis Algorithmus 6 berechnet eine solche äquivalente CFG.

Die Schritte „Entfernen von ε -Produktionen“ und „Entfernen von Einheitsproduktionen“ haben wir als korrekt gezeigt.

Korrektheit von Algorithmus 6

Theorem

Für jede CFG G mit $\varepsilon \notin L(G)$ kann eine CFG G' in Chomsky-Normalform berechnet werden, sodass $L(G') = L(G)$ gilt.

Beweis Algorithmus 6 berechnet eine solche äquivalente CFG.

Die Schritte „Entfernen von ε -Produktionen“ und „Entfernen von Einheitsproduktionen“ haben wir als korrekt gezeigt.

Die Schritte „Einführen von Produktionen $B \rightarrow a$ “ und „Behandlung von $A \rightarrow B_1 \cdots B_m$ “ sind Varianten der korrekten Operation „Boxing von Satzformen“, die wir gleich korrekt beweisen werden.

Korrektheit von Algorithmus 6

Theorem

Für jede CFG G mit $\varepsilon \notin L(G)$ kann eine CFG G' in Chomsky-Normalform berechnet werden, sodass $L(G') = L(G)$ gilt.

Beweis Algorithmus 6 berechnet eine solche äquivalente CFG.

Die Schritte „Entfernen von ε -Produktionen“ und „Entfernen von Einheitsproduktionen“ haben wir als korrekt gezeigt.

Die Schritte „Einführen von Produktionen $B \rightarrow a$ “ und „Behandlung von $A \rightarrow B_1 \cdots B_m$ “ sind Varianten der korrekten Operation „Boxing von Satzformen“, die wir gleich korrekt beweisen werden.

Am Ende der Berechnung haben alle Produktionen die gewünschte Form. □

Lemma (Boxing von Satzformen)

Sei G eine CFG mit $G = (V, \Sigma, P \cup \{A \rightarrow w_1 \cdots w_n\}, S)$.

Seien $B_1, \dots, B_n$ neue Variablen (d.h. $V \cap \{B_1, \dots, B_n\} = \emptyset$) und

$G' = (V \cup \{B_1, \dots, B_n\}, \Sigma, P \cup \{A \rightarrow B_1 \cdots B_n, B_1 \rightarrow w_1, \dots, B_n \rightarrow w_n\}, S)$.

Dann gilt $L(G') = L(G)$.

Boxing von Satzformen

Lemma (Boxing von Satzformen)

Sei G eine CFG mit $G = (V, \Sigma, P \cup \{A \rightarrow w_1 \cdots w_n\}, S)$.

Seien $B_1, \dots, B_n$ neue Variablen (d.h. $V \cap \{B_1, \dots, B_n\} = \emptyset$) und

$G' = (V \cup \{B_1, \dots, B_n\}, \Sigma, P \cup \{A \rightarrow B_1 \cdots B_n, B_1 \rightarrow w_1, \dots, B_n \rightarrow w_n\}, S)$.

Dann gilt $L(G') = L(G)$.

Beweis

⊇ Wir zeigen: $w \in L(G)$ impliziert $w \in L(G')$.

Konstruiere aus $S \Rightarrow_G^* w$ die Ableitung $S \Rightarrow_{G'}^* w$: Übersetze jeden Schritt

$uAv \Rightarrow_G uw_1 \cdots w_nv$ in $uAv \Rightarrow_{G'} uB_1 \cdots B_nv \Rightarrow_{G'}^n uw_1 \cdots w_nv$.

Beweis (Fortsetzung)

⊆ Wir zeigen: $w \in L(G')$ impliziert $w \in L(G)$.

Betrachte den Syntaxbaum für $S \Rightarrow_{G'}^* w$.

Identifiziere die Anwendungen der Regeln $A \rightarrow B_1 \cdots B_n$ und $B_i \rightarrow w_i$, und modifiziere den Syntaxbaum durch Anwendung der Regel $A \rightarrow w_1 \cdots w_n$.

Lies Ableitung $S \Rightarrow_G^* w$ ab.



Weiteres Beispiel für die Herstellung der Chomsky-Normalform

Eingabe: $G_0 = (\{A, B, C, D, S\}, \{0, 1\}, P_0, S)$ mit
 $P_0 = \{S \rightarrow 1A, A \rightarrow AB, A \rightarrow DA, A \rightarrow \varepsilon,$
 $B \rightarrow 0, B \rightarrow 1, C \rightarrow AAA, D \rightarrow 1AC\}$

Schritt 1: Entfernen von ε -Produktionen

$G_0 = (\{A, B, C, D, S\}, \{0, 1\}, P_0, S)$ mit
 $P_0 = \{S \rightarrow 1A, A \rightarrow AB, A \rightarrow DA, A \rightarrow \varepsilon, B \rightarrow 0,$
 $B \rightarrow 1, C \rightarrow AAA, D \rightarrow 1AC\}$

Schritt 1: Entfernen von ε -Produktionen

$G_0 = (\{A, B, C, D, S\}, \{0, 1\}, P_0, S)$ mit
 $P_0 = \{S \rightarrow 1A, A \rightarrow AB, A \rightarrow DA, A \rightarrow \varepsilon, B \rightarrow 0, \\ B \rightarrow 1, C \rightarrow AAA, D \rightarrow 1AC\}$

- Finde Menge W der Variablen, die ε herleiten:
 $W = \{A, C\}$ da $A \rightarrow \varepsilon$ und $C \rightarrow AAA$

Schritt 1: Entfernen von ε -Produktionen

$G_0 = (\{A, B, C, D, S\}, \{0, 1\}, P_0, S)$ mit
 $P_0 = \{S \rightarrow 1A, A \rightarrow AB, A \rightarrow DA, A \rightarrow \varepsilon, B \rightarrow 0, \\ B \rightarrow 1, C \rightarrow AAA, D \rightarrow 1AC\}$

- Finde Menge W der Variablen, die ε herleiten:
 $W = \{A, C\}$ da $A \rightarrow \varepsilon$ und $C \rightarrow AAA$

- Starte mit

$G_1 = (\{A, B, C, D, S\}, \{0, 1\}, P_1, S)$ mit
 $P_1 = \{S \rightarrow 1A, A \rightarrow AB, A \rightarrow DA, B \rightarrow 0, \\ B \rightarrow 1, C \rightarrow AAA, D \rightarrow 1AC\}$

Schritt 1: Entfernen von ε -Produktionen

$G_0 = (\{A, B, C, D, S\}, \{0, 1\}, P_0, S)$ mit
 $P_0 = \{S \rightarrow 1A, A \rightarrow AB, A \rightarrow DA, A \rightarrow \varepsilon, B \rightarrow 0, \\ B \rightarrow 1, C \rightarrow AAA, D \rightarrow 1AC\}$

- Finde Menge W der Variablen, die ε herleiten:
 $W = \{A, C\}$ da $A \rightarrow \varepsilon$ und $C \rightarrow AAA$

- Starte mit

$G_1 = (\{A, B, C, D, S\}, \{0, 1\}, P_1, S)$ mit
 $P_1 = \{S \rightarrow 1A, A \rightarrow AB, A \rightarrow DA, B \rightarrow 0, \\ B \rightarrow 1, C \rightarrow AAA, D \rightarrow 1AC\}$

- Füge Produktionen für Vorkommen von A und C hinzu:

$P_1 = \{S \rightarrow 1A, S \rightarrow 1, A \rightarrow AB, A \rightarrow B, A \rightarrow DA, A \rightarrow D, \\ B \rightarrow 0, B \rightarrow 1, C \rightarrow AAA, C \rightarrow AA, C \rightarrow A, \\ D \rightarrow 1AC, D \rightarrow 1A, D \rightarrow 1C, D \rightarrow 1\}$

Schritt 2: Entfernen von Einheitsproduktionen

$$P_1 = \{S \rightarrow 1A, S \rightarrow 1, A \rightarrow AB, A \rightarrow B, A \rightarrow DA, A \rightarrow D, \\ B \rightarrow 0, B \rightarrow 1, C \rightarrow AAA, C \rightarrow AA, C \rightarrow A, \\ D \rightarrow 1AC, D \rightarrow 1A, D \rightarrow 1C, D \rightarrow 1\}$$

Schritt 2: Entfernen von Einheitsproduktionen

$$P_1 = \{S \rightarrow 1A, S \rightarrow 1, A \rightarrow AB, A \rightarrow B, A \rightarrow DA, A \rightarrow D, \\ B \rightarrow 0, B \rightarrow 1, C \rightarrow AAA, C \rightarrow AA, C \rightarrow A, \\ D \rightarrow 1AC, D \rightarrow 1A, D \rightarrow 1C, D \rightarrow 1\}$$

- Der gerichtete Graph ist $D' = (\{S, A, B, C, D\}, \{(A, B), (A, D), (C, A)\})$.

Schritt 2: Entfernen von Einheitsproduktionen

$$P_1 = \{S \rightarrow 1A, S \rightarrow 1, A \rightarrow AB, A \rightarrow B, A \rightarrow DA, A \rightarrow D, \\ B \rightarrow 0, B \rightarrow 1, C \rightarrow AAA, C \rightarrow AA, C \rightarrow A, \\ D \rightarrow 1AC, D \rightarrow 1A, D \rightarrow 1C, D \rightarrow 1\}$$

- Der gerichtete Graph ist $D' = (\{S, A, B, C, D\}, \{(A, B), (A, D), (C, A)\})$.
- Es gibt keine Zyklen, wir erhalten $G_2 = G_1$.

Schritt 2: Entfernen von Einheitsproduktionen

$$P_1 = \{S \rightarrow 1A, S \rightarrow 1, A \rightarrow AB, A \rightarrow B, A \rightarrow DA, A \rightarrow D, \\ B \rightarrow 0, B \rightarrow 1, C \rightarrow AAA, C \rightarrow AA, C \rightarrow A, \\ D \rightarrow 1AC, D \rightarrow 1A, D \rightarrow 1C, D \rightarrow 1\}$$

- ▶ Der gerichtete Graph ist $D' = (\{S, A, B, C, D\}, \{(A, B), (A, D), (C, A)\})$.
- ▶ Es gibt keine Zyklen, wir erhalten $G_2 = G_1$.
- ▶ Sortieren und Umbenennen der Variablen, sodass „ $A_i \rightarrow A_j$ impliziert $i < j$ “ gilt, erfordert eine Umbenennung ρ , welche die Beziehungen $A < B, A < D, C < A$ erzeugt.

Schritt 2: Entfernen von Einheitsproduktionen

$$P_1 = \{S \rightarrow 1A, S \rightarrow 1, A \rightarrow AB, A \rightarrow B, A \rightarrow DA, A \rightarrow D, \\ B \rightarrow 0, B \rightarrow 1, C \rightarrow AAA, C \rightarrow AA, C \rightarrow A, \\ D \rightarrow 1AC, D \rightarrow 1A, D \rightarrow 1C, D \rightarrow 1\}$$

- ▶ Der gerichtete Graph ist $D' = (\{S, A, B, C, D\}, \{(A, B), (A, D), (C, A)\})$.
- ▶ Es gibt keine Zyklen, wir erhalten $G_2 = G_1$.
- ▶ Sortieren und Umbenennen der Variablen, sodass „ $A_i \rightarrow A_j$ impliziert $i < j$ “ gilt, erfordert eine Umbenennung ρ , welche die Beziehungen $A < B$, $A < D$, $C < A$ erzeugt.
- ▶ Wir wählen ρ mit $\rho(C) = A_1$, $\rho(A) = A_2$, $\rho(B) = A_3$, $\rho(D) = A_4$, $\rho(S) = A_5$.

Schritt 2: Entfernen von Einheitsproduktionen

$$P_1 = \{S \rightarrow 1A, S \rightarrow 1, A \rightarrow AB, A \rightarrow B, A \rightarrow DA, A \rightarrow D, \\ B \rightarrow 0, B \rightarrow 1, C \rightarrow AAA, C \rightarrow AA, C \rightarrow A, \\ D \rightarrow 1AC, D \rightarrow 1A, D \rightarrow 1C, D \rightarrow 1\}$$

- ▶ Der gerichtete Graph ist $D' = (\{S, A, B, C, D\}, \{(A, B), (A, D), (C, A)\})$.
- ▶ Es gibt keine Zyklen, wir erhalten $G_2 = G_1$.
- ▶ Sortieren und Umbenennen der Variablen, sodass „ $A_i \rightarrow A_j$ impliziert $i < j$ “ gilt, erfordert eine Umbenennung ρ , welche die Beziehungen $A < B$, $A < D$, $C < A$ erzeugt.
- ▶ Wir wählen ρ mit $\rho(C) = A_1$, $\rho(A) = A_2$, $\rho(B) = A_3$, $\rho(D) = A_4$, $\rho(S) = A_5$.
- ▶ Das liefert uns $G_3 = (\{A_1, A_2, A_3, A_4, A_5\}, \Sigma, P_3, A_5)$ mit
$$P_3 = \{A_5 \rightarrow 1A_2, A_5 \rightarrow 1, A_2 \rightarrow A_2A_3, A_2 \rightarrow A_3, A_2 \rightarrow A_4A_2, \\ A_2 \rightarrow A_4, A_3 \rightarrow 0, A_3 \rightarrow 1, A_1 \rightarrow A_2A_2A_2, A_1 \rightarrow A_2A_2, \\ A_1 \rightarrow A_2, A_4 \rightarrow 1A_2A_1, A_4 \rightarrow 1A_2, A_4 \rightarrow 1A_1, A_4 \rightarrow 1\}$$

Schritt 2: Entfernen von Einheitsproduktionen

$$P_3 = \{A_5 \rightarrow 1A_2, A_5 \rightarrow 1, A_2 \rightarrow A_2A_3, A_2 \rightarrow A_3, A_2 \rightarrow A_4A_2, \\ A_2 \rightarrow A_4, A_3 \rightarrow 0, A_3 \rightarrow 1, A_1 \rightarrow A_2A_2A_2, A_1 \rightarrow A_2A_2, \\ A_1 \rightarrow A_2, A_4 \rightarrow 1A_2A_1, A_4 \rightarrow 1A_2, A_4 \rightarrow 1A_1, A_4 \rightarrow 1\}$$

- Nun läuft die für-Schleife für i von 5 bis 1:

Schritt 2: Entfernen von Einheitsproduktionen

$$P_3 = \{A_5 \rightarrow 1A_2, A_5 \rightarrow 1, A_2 \rightarrow A_2A_3, A_2 \rightarrow A_3, A_2 \rightarrow A_4A_2, \\ A_2 \rightarrow A_4, A_3 \rightarrow 0, A_3 \rightarrow 1, A_1 \rightarrow A_2A_2A_2, A_1 \rightarrow A_2A_2, \\ A_1 \rightarrow A_2, A_4 \rightarrow 1A_2A_1, A_4 \rightarrow 1A_2, A_4 \rightarrow 1A_1, A_4 \rightarrow 1\}$$

- Nun läuft die für-Schleife für i von 5 bis 1:
 - Für $i = 5, i = 4, i = 3$ gibt es jeweils keine Produktion der Form $A_i \rightarrow A_j$.

Schritt 2: Entfernen von Einheitsproduktionen

$$P_3 = \{A_5 \rightarrow 1A_2, A_5 \rightarrow 1, A_2 \rightarrow A_2A_3, A_2 \rightarrow A_3, A_2 \rightarrow A_4A_2, \\ A_2 \rightarrow A_4, A_3 \rightarrow 0, A_3 \rightarrow 1, A_1 \rightarrow A_2A_2A_2, A_1 \rightarrow A_2A_2, \\ A_1 \rightarrow A_2, A_4 \rightarrow 1A_2A_1, A_4 \rightarrow 1A_2, A_4 \rightarrow 1A_1, A_4 \rightarrow 1\}$$

► Nun läuft die für-Schleife für i von 5 bis 1:

► Für $i = 5, i = 4, i = 3$ gibt es jeweils keine Produktion der Form $A_i \rightarrow A_j$.

► Für $i = 2$ wird $A_2 \rightarrow A_3$ ersetzt durch $A_2 \rightarrow 0, A_2 \rightarrow 1$, und es wird $A_2 \rightarrow A_4$ ersetzt durch $A_2 \rightarrow 1A_2A_1, A_2 \rightarrow 1A_2, A_2 \rightarrow 1A_1$ und $A_2 \rightarrow 1$.
Danach ist

$$P_4 = \{A_5 \rightarrow 1A_2, A_5 \rightarrow 1, A_2 \rightarrow A_2A_3, A_2 \rightarrow 0, A_2 \rightarrow 1, A_2 \rightarrow A_4A_2, \\ A_2 \rightarrow 1A_2A_1, A_2 \rightarrow 1A_2, A_2 \rightarrow 1A_1, A_3 \rightarrow 0, A_3 \rightarrow 1, \\ A_1 \rightarrow A_2A_2A_2, A_1 \rightarrow A_2A_2, A_1 \rightarrow A_2, A_4 \rightarrow 1A_2A_1, \\ A_4 \rightarrow 1A_2, A_4 \rightarrow 1A_1, A_4 \rightarrow 1\}$$

Schritt 2: Entfernen von Einheitsproduktionen

$$P_3 = \{A_5 \rightarrow 1A_2, A_5 \rightarrow 1, A_2 \rightarrow A_2A_3, A_2 \rightarrow A_3, A_2 \rightarrow A_4A_2, \\ A_2 \rightarrow A_4, A_3 \rightarrow 0, A_3 \rightarrow 1, A_1 \rightarrow A_2A_2A_2, A_1 \rightarrow A_2A_2, \\ A_1 \rightarrow A_2, A_4 \rightarrow 1A_2A_1, A_4 \rightarrow 1A_2, A_4 \rightarrow 1A_1, A_4 \rightarrow 1\}$$

► Nun läuft die für-Schleife für i von 5 bis 1:

► Für $i = 5, i = 4, i = 3$ gibt es jeweils keine Produktion der Form $A_i \rightarrow A_j$.

► Für $i = 2$ wird $A_2 \rightarrow A_3$ ersetzt durch $A_2 \rightarrow 0, A_2 \rightarrow 1$, und es wird $A_2 \rightarrow A_4$ ersetzt durch $A_2 \rightarrow 1A_2A_1, A_2 \rightarrow 1A_2, A_2 \rightarrow 1A_1$ und $A_2 \rightarrow 1$.
Danach ist

$$P_4 = \{A_5 \rightarrow 1A_2, A_5 \rightarrow 1, A_2 \rightarrow A_2A_3, A_2 \rightarrow 0, A_2 \rightarrow 1, A_2 \rightarrow A_4A_2, \\ A_2 \rightarrow 1A_2A_1, A_2 \rightarrow 1A_2, A_2 \rightarrow 1A_1, A_3 \rightarrow 0, A_3 \rightarrow 1, \\ A_1 \rightarrow A_2A_2A_2, A_1 \rightarrow A_2A_2, A_1 \rightarrow A_2, A_4 \rightarrow 1A_2A_1, \\ A_4 \rightarrow 1A_2, A_4 \rightarrow 1A_1, A_4 \rightarrow 1\}$$

► Für $i = 1$ wird $A_1 \rightarrow A_2$ ersetzt durch $A_1 \rightarrow A_2A_3, A_1 \rightarrow 0, A_1 \rightarrow 1, A_1 \rightarrow A_4A_2, \\ A_1 \rightarrow 1A_2A_1, A_1 \rightarrow 1A_2$ und $A_1 \rightarrow 1A_1$.

Schritt 2: Entfernen von Einheitsproduktionen

- Daher ist die Grammatik nach Entfernen von Einheitsproduktionen:

$G_5 = (V_5, \Sigma, P_5, A_5)$ mit $V_5 = \{A_1, A_2, A_3, A_4, A_5\}$ und

$P_5 = \{A_5 \rightarrow 1A_2, A_5 \rightarrow 1, A_2 \rightarrow A_2A_3, A_2 \rightarrow 0, A_2 \rightarrow 1, A_2 \rightarrow A_4A_2, A_2 \rightarrow 1A_2A_1, A_2 \rightarrow 1A_2, A_2 \rightarrow 1A_1, A_3 \rightarrow 0, A_3 \rightarrow 1, A_1 \rightarrow A_2A_2A_2, A_1 \rightarrow A_2A_2, A_1 \rightarrow A_2A_3, A_1 \rightarrow 0, A_1 \rightarrow 1, A_1 \rightarrow A_4A_2, A_1 \rightarrow 1A_2A_1, A_1 \rightarrow 1A_2, A_1 \rightarrow 1A_1, A_4 \rightarrow 1A_2A_1, A_4 \rightarrow 1A_2, A_4 \rightarrow 1A_1, A_4 \rightarrow 1\}$

Schritt 3: Terminale durch Variablen darstellen

- Füge $B_0 \rightarrow 0$ und $B_1 \rightarrow 1$ hinzu und ersetze in rechten Seiten mit Wortlänge > 1 :

$$P_6 = \{B_0 \rightarrow 0, B_1 \rightarrow 1, A_5 \rightarrow B_1 A_2, A_5 \rightarrow 1, A_2 \rightarrow A_2 A_3, A_2 \rightarrow 0, \\ A_2 \rightarrow 1, A_2 \rightarrow A_4 A_2, A_2 \rightarrow B_1 A_2 A_1, A_2 \rightarrow B_1 A_2, \\ A_2 \rightarrow B_1 A_1, A_3 \rightarrow 0, A_3 \rightarrow 1, A_1 \rightarrow A_2 A_2 A_2, A_1 \rightarrow A_2 A_2, \\ A_1 \rightarrow A_2 A_3, A_1 \rightarrow 0, A_1 \rightarrow 1, A_1 \rightarrow A_4 A_2, A_1 \rightarrow B_1 A_2 A_1, \\ A_1 \rightarrow B_1 A_2, A_1 \rightarrow B_1 A_1, A_4 \rightarrow B_1 A_2 A_1, \\ A_4 \rightarrow B_1 A_2, A_4 \rightarrow B_1 A_1, A_4 \rightarrow 1\}$$

Schritt 4: Rechte Seiten zerlegen

- Zerlege rechte Seiten mit Wortlänge > 2 :

Ergibt $G_7 = (V_7, \Sigma, P_7, A_5)$, wobei

$$V_7 = \{A_1, A_2, A_3, A_4, A_5, B_0, B_1, C_1, C_2, C_3, C_4\}$$

$$P_7 = \{B_0 \rightarrow 0, B_1 \rightarrow 1, A_5 \rightarrow B_1 A_2, A_5 \rightarrow 1, A_2 \rightarrow A_2 A_3, A_2 \rightarrow 0, \\ A_2 \rightarrow 1, A_2 \rightarrow A_4 A_2, A_2 \rightarrow B_1 C_1, C_1 \rightarrow A_2 A_1, \\ A_2 \rightarrow B_1 A_2, A_2 \rightarrow B_1 A_1, A_3 \rightarrow 0, A_3 \rightarrow 1, A_1 \rightarrow A_2 C_2, \\ C_2 \rightarrow A_2 A_2, A_1 \rightarrow A_2 A_2, A_1 \rightarrow A_2 A_3, A_1 \rightarrow 0, A_1 \rightarrow 1, \\ A_1 \rightarrow A_4 A_2, A_1 \rightarrow B_1 C_3, C_3 \rightarrow A_2 A_1, A_1 \rightarrow B_1 A_2, \\ A_1 \rightarrow B_1 A_1, A_4 \rightarrow B_1 C_4, C_4 \rightarrow A_2 A_1, A_4 \rightarrow B_1 A_2, \\ A_4 \rightarrow B_1 A_1, A_4 \rightarrow 1\}$$

Schritt 4: Rechte Seiten zerlegen

- Zerlege rechte Seiten mit Wortlänge > 2 :

Ergibt $G_7 = (V_7, \Sigma, P_7, A_5)$, wobei

$$V_7 = \{A_1, A_2, A_3, A_4, A_5, B_0, B_1, C_1, C_2, C_3, C_4\}$$

$$P_7 = \{B_0 \rightarrow 0, B_1 \rightarrow 1, A_5 \rightarrow B_1 A_2, A_5 \rightarrow 1, A_2 \rightarrow A_2 A_3, A_2 \rightarrow 0, \\ A_2 \rightarrow 1, A_2 \rightarrow A_4 A_2, A_2 \rightarrow B_1 C_1, C_1 \rightarrow A_2 A_1, \\ A_2 \rightarrow B_1 A_2, A_2 \rightarrow B_1 A_1, A_3 \rightarrow 0, A_3 \rightarrow 1, A_1 \rightarrow A_2 C_2, \\ C_2 \rightarrow A_2 A_2, A_1 \rightarrow A_2 A_2, A_1 \rightarrow A_2 A_3, A_1 \rightarrow 0, A_1 \rightarrow 1, \\ A_1 \rightarrow A_4 A_2, A_1 \rightarrow B_1 C_3, C_3 \rightarrow A_2 A_1, A_1 \rightarrow B_1 A_2, \\ A_1 \rightarrow B_1 A_1, A_4 \rightarrow B_1 C_4, C_4 \rightarrow A_2 A_1, A_4 \rightarrow B_1 A_2, \\ A_4 \rightarrow B_1 A_1, A_4 \rightarrow 1\}$$

G_7 ist in Chomsky-Normalform.

5c

Das Pumping-Lemma für kontextfreie Sprachen

Prof. Dr. Jasmin Blanchette

Lehr- und Forschungseinheit für
Theoretische Informatik und Theorembeweisen

Stand: 28. Mai 2024
Basierend auf Folien von PD Dr. David Sabel

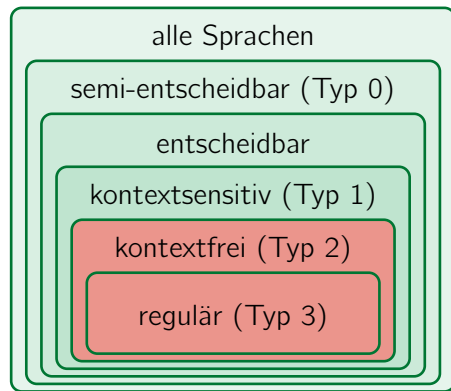


Hintergrund zum Pumping-Lemma

Wir lernen eine Methode kennen zum Widerlegen der Kontextfreiheit:
das **Pumping-Lemma** für kontextfreie Sprachen.

Es gibt allgemeinere Formulierungen, z.B.

- ▶ Ogdens Lemma (ist im Skript, aber kein Prüfungstoff)
- ▶ das Interchange-Lemma.



Pumping-Lemmas im Vergleich

- ▶ Pumping-Lemma für reguläre Sprachen:
Jede reguläre Sprache erfüllt die Pumping-Eigenschaft für reguläre Sprachen.

Pumping-Lemmas im Vergleich

- ▶ Pumping-Lemma für **reguläre** Sprachen:
Jede **reguläre** Sprache erfüllt die Pumping-Eigenschaft für **reguläre** Sprachen.
- ▶ Pumping-Lemma für **kontextfreie** Sprachen:
Jede **kontextfreie** Sprache erfüllt die Pumping-Eigenschaft für **kontextfreie** Sprachen.

Pumping-Lemmas im Vergleich

- ▶ Pumping-Lemma für **reguläre** Sprachen:
Jede **reguläre** Sprache erfüllt die Pumping-Eigenschaft für **reguläre** Sprachen.
- ▶ Pumping-Lemma für **kontextfreie** Sprachen:
Jede **kontextfreie** Sprache erfüllt die Pumping-Eigenschaft für **kontextfreie** Sprachen.
- ▶ Pumping-Eigenschaft für **reguläre** Sprachen, informell:
Man kann Wörter an **einer** Stelle aufpumpen
und in der Sprache verbleiben ($uv^i w \in L$ für alle $i \in \mathbb{N}$).

Pumping-Lemmas im Vergleich

- ▶ Pumping-Lemma für **reguläre** Sprachen:
Jede **reguläre** Sprache erfüllt die Pumping-Eigenschaft für **reguläre** Sprachen.
- ▶ Pumping-Lemma für **kontextfreie** Sprachen:
Jede **kontextfreie** Sprache erfüllt die Pumping-Eigenschaft für **kontextfreie** Sprachen.
- ▶ Pumping-Eigenschaft für **reguläre** Sprachen, informell:
Man kann Wörter an **einer** Stelle aufpumpen
und in der Sprache verbleiben ($uv^i w \in L$ für alle $i \in \mathbb{N}$).
- ▶ Pumping-Eigenschaft für **kontextfreie** Sprachen, informell:
Man kann Wörter an **zwei** Stellen gleichzeitig aufpumpen
und in der Sprache verbleiben ($uv^i wx^i y \in L$ für alle $i \in \mathbb{N}$).

Die Pumping-Eigenschaft für kontextfreie Sprachen

Definition

Eine Sprache L hat die **Pumping-Eigenschaft** (für kontextfreie Sprachen), wenn gilt:
Es gibt eine Zahl $n \in \mathbb{N}_{>0}$, sodass jedes Wort $z \in L$, welches Mindestlänge n hat (d.h. $|z| \geq n$), als $z = uvwx y$ geschrieben werden kann, sodass gilt:

1. $|vx| \geq 1$
2. $|vwx| \leq n$
3. für alle $i \in \mathbb{N}$: $uv^iwx^iy \in L$.

Das Pumping-Lemma für kontextfreie Sprachen

Lemma (Pumping-Lemma)

Jede kontextfreie Sprache hat die [Pumping-Eigenschaft](#).

Das Pumping-Lemma für kontextfreie Sprachen

Lemma (Pumping-Lemma)

Jede kontextfreie Sprache hat die [Pumping-Eigenschaft](#).

Beweis Sei $G = (V, \Sigma, P, S)$ eine CFG in Chomsky-Normalform mit $L(G) = L \setminus \{\varepsilon\}$.

Das Pumping-Lemma für kontextfreie Sprachen

Lemma (Pumping-Lemma)

Jede kontextfreie Sprache hat die [Pumping-Eigenschaft](#).

Beweis Sei $G = (V, \Sigma, P, S)$ eine CFG in Chomsky-Normalform mit $L(G) = L \setminus \{\varepsilon\}$.
Wir wählen $n = 2^{|V|}$.

Das Pumping-Lemma für kontextfreie Sprachen

Lemma (Pumping-Lemma)

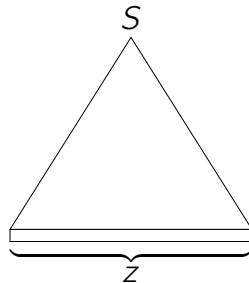
Jede kontextfreie Sprache hat die **Pumping-Eigenschaft**.

Beweis Sei $G = (V, \Sigma, P, S)$ eine CFG in Chomsky-Normalform mit $L(G) = L \setminus \{\varepsilon\}$.

Wir wählen $n = 2^{|V|}$.

Sei $z \in L$ ein beliebiges Wort mit $|z| \geq n$.

Betrachte den Syntaxbaum von z .



Das Pumping-Lemma für kontextfreie Sprachen

Lemma (Pumping-Lemma)

Jede kontextfreie Sprache hat die **Pumping-Eigenschaft**.

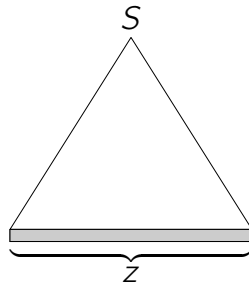
Beweis Sei $G = (V, \Sigma, P, S)$ eine CFG in Chomsky-Normalform mit $L(G) = L \setminus \{\varepsilon\}$.

Wir wählen $n = 2^{|V|}$.

Sei $z \in L$ ein beliebiges Wort mit $|z| \geq n$.

Betrachte den Syntaxbaum von z .

Da G in Chomsky-Normalform ist, ist der Syntaxbaum binär, bis auf die letzte Schicht, die Produktionen $A \rightarrow a$ anwendet.



Das Pumping-Lemma für kontextfreie Sprachen

Lemma (Pumping-Lemma)

Jede kontextfreie Sprache hat die **Pumping-Eigenschaft**.

Beweis Sei $G = (V, \Sigma, P, S)$ eine CFG in Chomsky-Normalform mit $L(G) = L \setminus \{\varepsilon\}$.

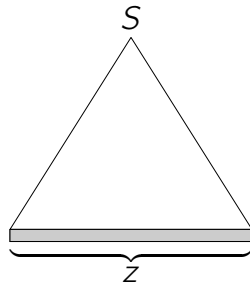
Wir wählen $n = 2^{|V|}$.

Sei $z \in L$ ein beliebiges Wort mit $|z| \geq n$.

Betrachte den Syntaxbaum von z .

Da G in Chomsky-Normalform ist, ist der Syntaxbaum binär, bis auf die letzte Schicht, die Produktionen $A \rightarrow a$ anwendet.

Der Baum ohne letzte Schicht hat $|z| \geq n = 2^{|V|}$ Blätter.



Das Pumping-Lemma für kontextfreie Sprachen

Lemma (Pumping-Lemma)

Jede kontextfreie Sprache hat die **Pumping-Eigenschaft**.

Beweis Sei $G = (V, \Sigma, P, S)$ eine CFG in Chomsky-Normalform mit $L(G) = L \setminus \{\varepsilon\}$.

Wir wählen $n = 2^{|V|}$.

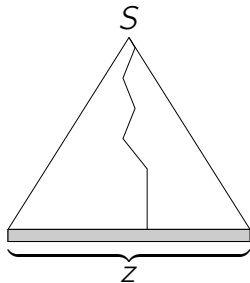
Sei $z \in L$ ein beliebiges Wort mit $|z| \geq n$.

Betrachte den Syntaxbaum von z .

Da G in Chomsky-Normalform ist, ist der Syntaxbaum binär, bis auf die letzte Schicht, die Produktionen $A \rightarrow a$ anwendet.

Der Baum ohne letzte Schicht hat $|z| \geq n = 2^{|V|}$ Blätter.

Daher hat der längste Pfad von der Wurzel zum Blatt eine Länge $\geq |V|$ (siehe Lemma). Dieser besteht aus $\geq |V| + 1$ Knoten und jeder Knoten ist mit einer Variablen markiert.



Lemma über Binärbäume

Lemma

Sei B ein Binärbaum mit $\geq 2^k$ Blättern. Dann hat B einen Pfad der Länge $\geq k$.

Lemma über Binärbäume

Lemma

Sei B ein Binärbaum mit $\geq 2^k$ Blättern. Dann hat B einen Pfad der Länge $\geq k$.

Beweis Durch Induktion über k .

Lemma über Binärbäume

Lemma

Sei B ein Binärbaum mit $\geq 2^k$ Blättern. Dann hat B einen Pfad der Länge $\geq k$.

Beweis Durch Induktion über k .

- **Fall $k = 0$:** Ein Baum mit $2^k = 2^0 = 1$ Blatt besteht genau aus diesem Blatt und hat einen Pfad der Länge $0 \geq 0$.

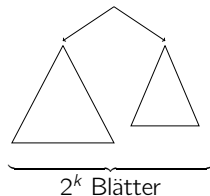
Lemma über Binärbäume

Lemma

Sei B ein Binärbaum mit $\geq 2^k$ Blättern. Dann hat B einen Pfad der Länge $\geq k$.

Beweis Durch Induktion über k .

- **Fall $k = 0$:** Ein Baum mit $2^k = 2^0 = 1$ Blatt besteht genau aus diesem Blatt und hat einen Pfad der Länge $0 \geq 0$.
- **Fall $k > 0$:** Einer der beiden Teilbäume unter der Wurzel hat $\geq 2^{k-1}$ Blätter.



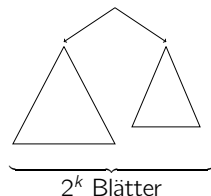
Lemma über Binärbäume

Lemma

Sei B ein Binärbaum mit $\geq 2^k$ Blättern. Dann hat B einen Pfad der Länge $\geq k$.

Beweis Durch Induktion über k .

- **Fall $k = 0$:** Ein Baum mit $2^k = 2^0 = 1$ Blatt besteht genau aus diesem Blatt und hat einen Pfad der Länge $0 \geq 0$.
- **Fall $k > 0$:** Einer der beiden Teilbäume unter der Wurzel hat $\geq 2^{k-1}$ Blätter. Per Induktionshypothese hat dieser einen Pfad der Länge $\geq k - 1$.



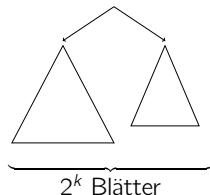
Lemma über Binärbäume

Lemma

Sei B ein Binärbaum mit $\geq 2^k$ Blättern. Dann hat B einen Pfad der Länge $\geq k$.

Beweis Durch Induktion über k .

- **Fall $k = 0$:** Ein Baum mit $2^k = 2^0 = 1$ Blatt besteht genau aus diesem Blatt und hat einen Pfad der Länge $0 \geq 0$.
- **Fall $k > 0$:** Einer der beiden Teilbäume unter der Wurzel hat $\geq 2^{k-1}$ Blätter. Per Induktionshypothese hat dieser einen Pfad der Länge $\geq k - 1$. Daher hat der gesamte Baum einen Pfad der Länge $\geq k$. □

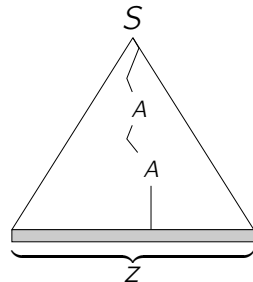


Das Pumping-Lemmas für kontextfreie Sprachen

Lemma (Pumping-Lemma)

Jede kontextfreie Sprache hat die [Pumping-Eigenschaft](#).

Beweis (Fortsetzung) Wir erinnern uns, dass der längste Pfad von der Wurzel aus $\geq |V| + 1$ Knoten besteht und dass jeder Knoten mit einer Variablen markiert ist.



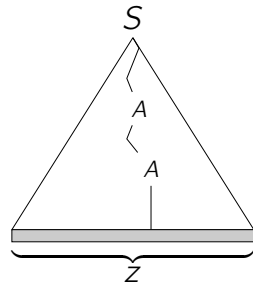
Das Pumping-Lemmas für kontextfreie Sprachen

Lemma (Pumping-Lemma)

Jede kontextfreie Sprache hat die **Pumping-Eigenschaft**.

Beweis (Fortsetzung) Wir erinnern uns, dass der längste Pfad von der Wurzel aus $\geq |V| + 1$ Knoten besteht und dass jeder Knoten mit einer Variablen markiert ist.

Da es nur $|V|$ Variablen gibt, kommt mindestens eine Variable mehrfach auf diesem Pfad vor.



Das Pumping-Lemmas für kontextfreie Sprachen

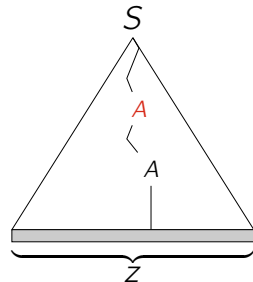
Lemma (Pumping-Lemma)

Jede kontextfreie Sprache hat die **Pumping-Eigenschaft**.

Beweis (Fortsetzung) Wir erinnern uns, dass der längste Pfad von der Wurzel aus $\geq |V| + 1$ Knoten besteht und dass jeder Knoten mit einer Variablen markiert ist.

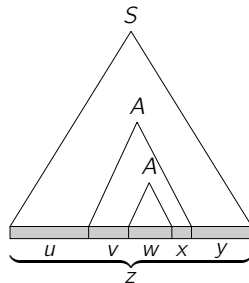
Da es nur $|V|$ Variablen gibt, kommt mindestens eine Variable mehrfach auf diesem Pfad vor.

Wähle die Vorkommen der Variablen so, dass das obere Vorkommen am tiefsten ist. Sei A die Variable.



Das Pumping-Lemmas für kontextfreie Sprachen

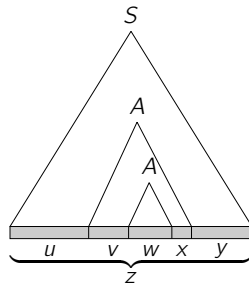
Beweis (Fortsetzung) Betrachte die Teilbäume, die jeweils A als Wurzel haben.



Das Pumping-Lemmas für kontextfreie Sprachen

Beweis (Fortsetzung) Betrachte die Teilbäume, die jeweils A als Wurzel haben.

Sie entsprechen Ableitungen von Teilwörtern von z .

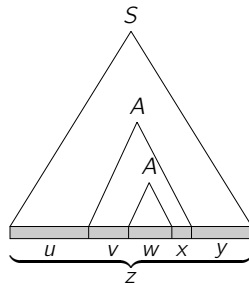


Das Pumping-Lemmas für kontextfreie Sprachen

Beweis (Fortsetzung) Betrachte die Teilbäume, die jeweils A als Wurzel haben.

Sie entsprechen Ableitungen von Teilwörtern von z .

Der Teilbaum mit dem unteren A als Wurzel erzeugt ein Teilwort des Teilbaums mit dem oberen A als Wurzel.



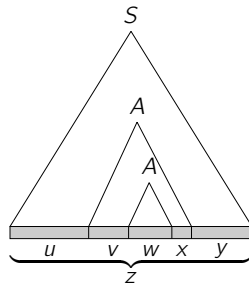
Das Pumping-Lemmas für kontextfreie Sprachen

Beweis (Fortsetzung) Betrachte die Teilbäume, die jeweils A als Wurzel haben.

Sie entsprechen Ableitungen von Teilwörtern von z .

Der Teilbaum mit dem unteren A als Wurzel erzeugt ein Teilwort des Teilbaums mit dem oberen A als Wurzel.

Wir wählen die Zerlegung $z = uvwxy$, wobei vw vom oberen A und w vom unteren A erzeugt wird.

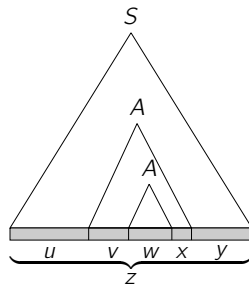


Das Pumping-Lemmas für kontextfreie Sprachen

Beweis (Fortsetzung) Wir zeigen nun die drei geforderten Eigenschaften der Zerlegung.

- $|vx| \geq 1$: Es gilt $|w| \geq 1$, da Variablen einer Grammatik in Chomsky-Normalform nur Wörter mit Länge ≥ 1 ableiten.

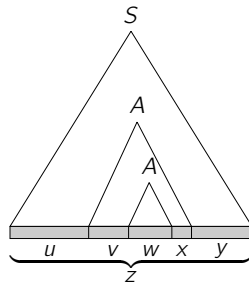
vw muss echt länger sein als w , da das obere A über dem unteren A steht. Daher folgt $|v| \geq 1$ oder $|x| \geq 1$, d.h. $|vx| \geq 1$.



Das Pumping-Lemmas für kontextfreie Sprachen

Beweis (Fortsetzung)

- $|vwx| \leq n$: Da wir das tiefste Vorkommen der wiederholten Variable gewählt haben, kann der Pfad vom oberen A bis zur Blattebene nur aus $\leq |V| + 1$ Knoten bestehen und die Länge $\leq |V|$ haben.

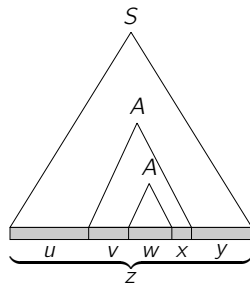


Das Pumping-Lemmas für kontextfreie Sprachen

Beweis (Fortsetzung)

- $|vwx| \leq n$: Da wir das tiefste Vorkommen der wiederholten Variable gewählt haben, kann der Pfad vom oberen A bis zur Blattebene nur aus $\leq |V| + 1$ Knoten bestehen und die Länge $\leq |V|$ haben.

Da der Pfad von der Wurzel maximaler Länge ist, müssen andere Pfade vom oberen A bis zur Blattebene kürzer oder gleich lang sein.



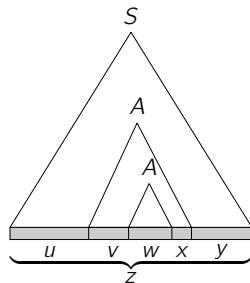
Das Pumping-Lemmas für kontextfreie Sprachen

Beweis (Fortsetzung)

- $|vwx| \leq n$: Da wir das tiefste Vorkommen der wiederholten Variable gewählt haben, kann der Pfad vom oberen A bis zur Blattebene nur aus $\leq |V| + 1$ Knoten bestehen und die Länge $\leq |V|$ haben.

Da der Pfad von der Wurzel maximaler Länge ist, müssen andere Pfade vom oberen A bis zur Blattebene kürzer oder gleich lang sein.

Daraus folgt: $|vwx| \leq 2^{|V|} = n$ (siehe Lemma).



Lemma über Binärbäume

Lemma

Sei B ein Binärbaum von dem alle Pfade eine Länge $\leq k$ haben.
Dann hat $B \leq 2^k$ Blätter.

Lemma über Binärbäume

Lemma

Sei B ein Binärbaum von dem alle Pfade eine Länge $\leq k$ haben.
Dann hat $B \leq 2^k$ Blätter.

Beweis Durch Induktion über k .

Lemma über Binärbäume

Lemma

Sei B ein Binärbaum von dem alle Pfade eine Länge $\leq k$ haben.
Dann hat $B \leq 2^k$ Blätter.

Beweis Durch Induktion über k .

► Fall $k = 0$: B besteht nur aus $2^0 = 1$ Blatt.

Lemma über Binärbäume

Lemma

Sei B ein Binärbaum von dem alle Pfade eine Länge $\leq k$ haben.
Dann hat $B \leq 2^k$ Blätter.

Beweis Durch Induktion über k .

- ▶ Fall $k = 0$: B besteht nur aus $2^0 = 1$ Blatt.
- ▶ Fall $k > 0$: B hat zwei Teilbäume unter der Wurzel, von denen alle Pfade eine Länge $\leq k - 1$ haben.

Lemma über Binärbäume

Lemma

Sei B ein Binärbaum von dem alle Pfade eine Länge $\leq k$ haben.
Dann hat $B \leq 2^k$ Blätter.

Beweis Durch Induktion über k .

- ▶ Fall $k = 0$: B besteht nur aus $2^0 = 1$ Blatt.
- ▶ Fall $k > 0$: B hat zwei Teilbäume unter der Wurzel, von denen alle Pfade eine Länge $\leq k - 1$ haben.

Durch die Induktionshypothese haben die beiden Teilbäume jeweils $\leq 2^{k-1}$ Blätter.

Lemma über Binärbäume

Lemma

Sei B ein Binärbaum von dem alle Pfade eine Länge $\leq k$ haben.
Dann hat $B \leq 2^k$ Blätter.

Beweis Durch Induktion über k .

- ▶ Fall $k = 0$: B besteht nur aus $2^0 = 1$ Blatt.
- ▶ Fall $k > 0$: B hat zwei Teilbäume unter der Wurzel, von denen alle Pfade eine Länge $\leq k - 1$ haben.

Durch die Induktionshypothese haben die beiden Teilbäume jeweils $\leq 2^{k-1}$ Blätter.

B als Ganzes hat dann $\leq 2^{k-1} + 2^{k-1} = 2^k$ Blätter.



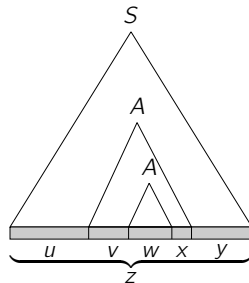
Das Pumping-Lemmas für kontextfreie Sprachen

Lemma (Pumping-Lemma)

Jede kontextfreie Sprache hat die **Pumping-Eigenschaft**.

Beweis (Fortsetzung)

- für alle $i \in \mathbb{N}$: $uv^iwx^iy \in L$: Aus dem Baum folgt:
 $A \Rightarrow^* w$ und $A \Rightarrow^* vAx$ und daher kann man auch
 $A \Rightarrow^* v^iwx^i$ für alle $i \in \mathbb{N}$ ableiten.



Das Pumping-Lemmas für kontextfreie Sprachen

Lemma (Pumping-Lemma)

Jede kontextfreie Sprache hat die **Pumping-Eigenschaft**.

Beweis (Fortsetzung)

- für alle $i \in \mathbb{N}$: $uv^iwx^iy \in L$: Aus dem Baum folgt:
 $A \Rightarrow^* w$ und $A \Rightarrow^* vAx$ und daher kann man auch
 $A \Rightarrow^* v^iwx^i$ für alle $i \in \mathbb{N}$ ableiten.

Schließlich folgt daraus $S \Rightarrow^* uv^iwx^iy$ für alle $i \in \mathbb{N}$. $\square$

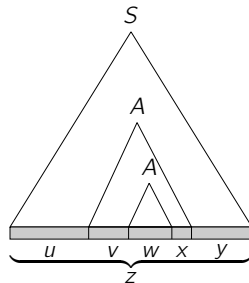
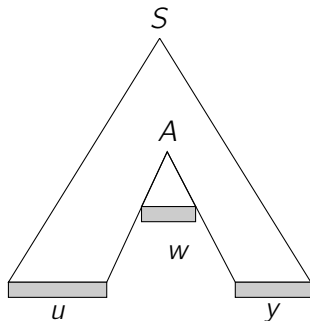
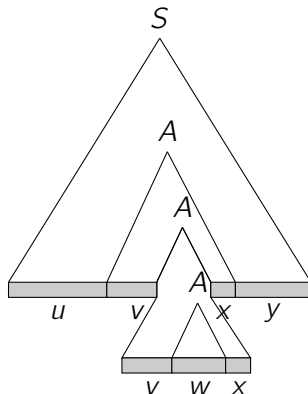


Illustration des Aufpumpens



uv^0wx^0y



uv^2wx^2y

Anwendung des Pumping-Lemmas

Sei L eine Sprache, die wir als nicht kontextfrei beweisen wollen.

Anwendung des Pumping-Lemmas

Sei L eine Sprache, die wir als nicht kontextfrei beweisen wollen.

Pumping-Lemma:

L ist kontextfrei $\implies L$ hat die Pumping-Eigenschaft

Anwendung des Pumping-Lemmas

Sei L eine Sprache, die wir als nicht kontextfrei beweisen wollen.

Pumping-Lemma:

L ist kontextfrei $\implies L$ hat die Pumping-Eigenschaft

Kontraposition:

L hat **nicht** die Pumping-Eigenschaft $\implies L$ ist **nicht** kontextfrei

Anwendung des Pumping-Lemmas

Sei L eine Sprache, die wir als nicht kontextfrei beweisen wollen.

Pumping-Lemma:

L ist kontextfrei $\implies L$ hat die Pumping-Eigenschaft

Kontraposition:

L hat **nicht** die Pumping-Eigenschaft $\implies L$ ist **nicht** kontextfrei

Beweisstrategie für die Aussage „ L ist **nicht** kontextfrei“:

1. Durch die Kontraposition reicht es zu zeigen, dass L die Pumping-Eigenschaft **nicht** hat.
2. Zeige dies durch Widerspruch: Nehme an, dass L die Pumping-Eigenschaft hat.
3. Leite einen Widerspruch her.
4. D.h. L ist **nicht** kontextfrei.

Nichtkontextfreiheit zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{a^j b^j c^j \mid j \in \mathbb{N}\}$ ist nicht kontextfrei.

Nichtkontextfreiheit zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{a^j b^j c^j \mid j \in \mathbb{N}\}$ ist nicht kontextfrei.

Beweis Mit dem Pumping-Lemma.

Nichtkontextfreiheit zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{a^j b^j c^j \mid j \in \mathbb{N}\}$ ist nicht kontextfrei.

Beweis Mit dem Pumping-Lemma.

Sei $n \in \mathbb{N}_{>0}$ beliebig.

Nichtkontextfreiheit zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{a^j b^j c^j \mid j \in \mathbb{N}\}$ ist nicht kontextfrei.

Beweis Mit dem Pumping-Lemma.

Sei $n \in \mathbb{N}_{>0}$ beliebig.

Wir wählen $z \in L$ als $z = a^n b^n c^n$ mit $|z| \geq n$.

Nichtkontextfreiheit zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{a^j b^j c^j \mid j \in \mathbb{N}\}$ ist nicht kontextfrei.

Beweis Mit dem Pumping-Lemma.

Sei $n \in \mathbb{N}_{>0}$ beliebig.

Wir wählen $z \in L$ als $z = a^n b^n c^n$ mit $|z| \geq n$.

Sei $z = uvwxy$ eine beliebige Zerlegung von z , sodass $|vx| \geq 1$, $|vwx| \leq n$ und $uv^i wx^i y \in L$ für jedes $i \in \mathbb{N}$. Wir wählen $i = 0$.

Nichtkontextfreiheit zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{a^j b^j c^j \mid j \in \mathbb{N}\}$ ist nicht kontextfrei.

Beweis Mit dem Pumping-Lemma.

Sei $n \in \mathbb{N}_{>0}$ beliebig.

Wir wählen $z \in L$ als $z = a^n b^n c^n$ mit $|z| \geq n$.

Sei $z = uvwxy$ eine beliebige Zerlegung von z , sodass $|vx| \geq 1$, $|vwx| \leq n$ und $uv^i wx^i y \in L$ für jedes $i \in \mathbb{N}$. Wir wählen $i = 0$.

- **Fall 1:** vwx ist von der Form $a^i b^j$, $i + j \leq n$. Da $|vx| \geq 1$, gilt $\#_a(vx) \geq 1$ oder $\#_b(vx) \geq 1$, aber $\#_c(vx) = 0$. Damit folgt $uv^0 wx^0 y \notin L$. Widerspruch.
- **Fall 2:** vwx ist von der Form $b^i c^j$, $i + j \leq n$. Da $|vx| \geq 1$, gilt $\#_b(vx) \geq 1$ oder $\#_c(vx) \geq 1$, aber $\#_a(vx) = 0$. Damit folgt $uv^0 wx^0 y \notin L$. Widerspruch.

Andere Fälle sind nicht möglich. □

Das Pumping-Lemma als Spiel

Sei L die Sprache.

Schritte:

1. Der **Gegner** wählt die Zahl $n \in \mathbb{N}_{>0}$.
2. **Wir** wählen das Wort $z \in L$ mit $|z| \geq n$.
3. Der **Gegner** wählt die Zerlegung $z = uvwxy$ mit $|vx| \geq 1$ und $|vwx| \leq n$.
4. **Wir** gewinnen das Spiel, wenn wir ein $i \in \mathbb{N}$ angeben können, sodass $uv^iwx^iy \notin L$.

Das Pumping-Lemma als Spiel

Sei L die Sprache.

Schritte:

1. Der **Gegner** wählt die Zahl $n \in \mathbb{N}_{>0}$.
2. **Wir** wählen das Wort $z \in L$ mit $|z| \geq n$.
3. Der **Gegner** wählt die Zerlegung $z = uvwxy$ mit $|vx| \geq 1$ und $|vwx| \leq n$.
4. **Wir** gewinnen das Spiel, wenn wir ein $i \in \mathbb{N}$ angeben können, sodass $uv^iwx^iy \notin L$.

Wenn **wir** das Spiel **für alle Wahlmöglichkeiten des Gegners gewinnen**, dann haben wir nachgewiesen, dass L nicht kontextfrei ist.

Das Pumping-Lemma als Spiel

Satz

Die Sprache $L = \{a^j b^j c^j \mid j \in \mathbb{N}\}$ ist nicht kontextfrei.

Beweis Wir zeigen, dass wir das eben eingeführte Spiel stets gewinnen:

1. Sei $n \in \mathbb{N}_{>0}$ vom **Gegner** gewählt.
2. **Wir** wählen $z \in L$ als $z = a^n b^n c^n$ mit $|z| \geq n$.
3. Der **Gegner** wählt die Zerlegung $z = uvwxy$, sodass $|vx| \geq 1$ und $|vwx| \leq n$.
4. **Wir** wählen $i = 0$.
 - **Fall 1:** vwx ist von der Form $a^i b^j$, $i + j \leq n$. Da $|vx| \geq 1$, gilt $\#_a(vx) \geq 1$ oder $\#_b(vx) \geq 1$, aber $\#_c(vx) = 0$. Damit folgt $uv^0wx^0y \notin L$.
 - **Fall 2:** vwx ist von der Form $b^i c^j$, $i + j \leq n$. Da $|vx| \geq 1$, gilt $\#_b(vx) \geq 1$ oder $\#_c(vx) \geq 1$, aber $\#_a(vx) = 0$. Damit folgt $uv^0wx^0y \notin L$.

Andere Fälle sind nicht möglich. Also gewinnen wir. □

Nichtkontextfreiheit zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{a^i b^j c^i d^j \mid i, j \in \mathbb{N}_{>0}\}$ ist nicht kontextfrei.

Nichtkontextfreiheit zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{a^i b^j c^i d^j \mid i, j \in \mathbb{N}_{>0}\}$ ist nicht kontextfrei.

Beweis Mit dem Pumping-Lemma.

Nichtkontextfreiheit zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{a^i b^j c^i d^j \mid i, j \in \mathbb{N}_{>0}\}$ ist nicht kontextfrei.

Beweis Mit dem Pumping-Lemma.

Sei $n \in \mathbb{N}_{>0}$ beliebig.

Nichtkontextfreiheit zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{a^i b^j c^i d^j \mid i, j \in \mathbb{N}_{>0}\}$ ist nicht kontextfrei.

Beweis Mit dem Pumping-Lemma.

Sei $n \in \mathbb{N}_{>0}$ beliebig.

Wir wählen $z \in L$ als $z = a^n b^n c^n d^n$ mit $|z| \geq n$.

Nichtkontextfreiheit zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{a^i b^j c^i d^j \mid i, j \in \mathbb{N}_{>0}\}$ ist nicht kontextfrei.

Beweis Mit dem Pumping-Lemma.

Sei $n \in \mathbb{N}_{>0}$ beliebig.

Wir wählen $z \in L$ als $z = a^n b^n c^n d^n$ mit $|z| \geq n$.

Sei $z = uvwxy$ eine beliebige Zerlegung von z , sodass $|vx| \geq 1$, $|vwx| \leq n$ und $uv^i wx^i y \in L$ für jedes $i \in \mathbb{N}$. Wir wählen $i = 0$.

Nichtkontextfreiheit zeigen mit dem Pumping-Lemma

Satz

Die Sprache $L = \{a^i b^j c^i d^j \mid i, j \in \mathbb{N}_{>0}\}$ ist nicht kontextfrei.

Beweis Mit dem Pumping-Lemma.

Sei $n \in \mathbb{N}_{>0}$ beliebig.

Wir wählen $z \in L$ als $z = a^n b^n c^n d^n$ mit $|z| \geq n$.

Sei $z = uvwxy$ eine beliebige Zerlegung von z , sodass $|vx| \geq 1$, $|vwx| \leq n$ und $uv^i wx^i y \in L$ für jedes $i \in \mathbb{N}$. Wir wählen $i = 0$.

- **Fall 1:** $vwx = a^i b^j$ mit $i + j \leq n$. Da $|vx| \geq 1$, gilt $\#_a(vx) + \#_b(vx) \geq 1$ und $uv^0 wx^0 y = uwy = a^{i'} b^{j'} c^n d^n$ und $i' < n$ oder $j' < n$, d.h. $uv^0 wx^0 y \notin L$.
Widerspruch.

Nichtkontextfreiheit zeigen mit dem Pumping-Lemma

Beweis (Fortsetzung)

- **Fall 2:** $vwx = b^i c^j$ mit $i + j \leq n$. Da $|vx| \geq 1$, gilt $\#_b(vx) + \#_c(vx) \geq 1$ und $uv^0wx^0y = uwy = a^n b^{i'} c^{j'} d^n$ und $i' < n$ oder $j' < n$, d.h. $uv^0wx^0y \notin L$.
Widerspruch.
- **Fall 3:** $vwx = c^i d^j$ mit $i + j \leq n$. Da $|vx| \geq 1$, gilt $\#_c(vx) + \#_d(vx) \geq 1$ und $uv^0wx^0y = uwy = a^n b^n c^{i'} d^{j'}$ und $i' < n$ oder $j' < n$, d.h. $uv^0wx^0y \notin L$.
Widerspruch.

Andere Fälle sind nicht möglich.



Satz

Sei L eine Sprache über einem unären Alphabet (d.h. $|\Sigma| = 1$). Dann ist L genau dann regulär, wenn L kontextfrei ist.

Beweis:

- ▶ Wenn L regulär ist, dann ist L auch kontextfrei.
- ▶ Die Rückrichtung ist im Skript, aber kein Prüfungsstoff.
(Der Beweis verwendet die Pumping-Eigenschaft für kontextfreie Sprachen.)

Satz

Die Sprachen

$$L_1 = \{a^p \mid p \text{ ist eine Primzahl}\}$$

$$L_2 = \{a^n \mid n \text{ ist keine Primzahl}\}$$

$$L_3 = \{a^n \mid n \text{ ist eine Quadratzahl}\}$$

$$L_4 = \{a^{2^n} \mid n \in \mathbb{N}\}$$

sind allesamt nicht kontextfrei.

Beispiele für unäre Alphabete

Satz

Die Sprachen

$$L_1 = \{a^p \mid p \text{ ist eine Primzahl}\}$$

$$L_2 = \{a^n \mid n \text{ ist keine Primzahl}\}$$

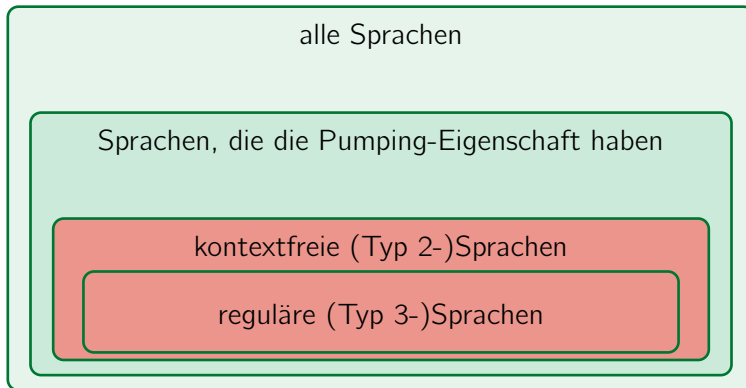
$$L_3 = \{a^n \mid n \text{ ist eine Quadratzahl}\}$$

$$L_4 = \{a^{2^n} \mid n \in \mathbb{N}\}$$

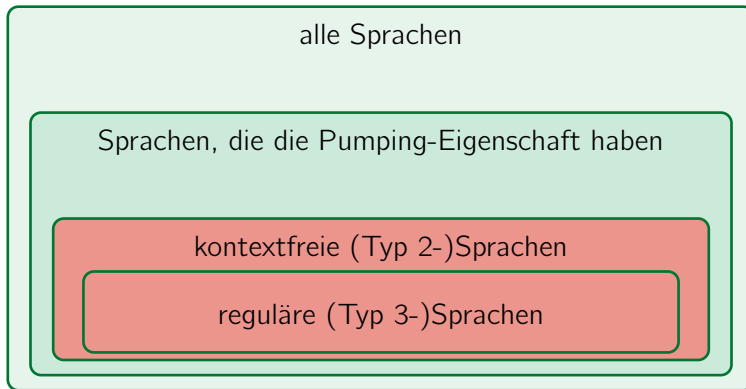
sind allesamt nicht kontextfrei.

Beweis Wir haben für alle vier Sprachen gezeigt, dass sie nicht regulär sind. Da sie alle über einem unären Alphabet definiert sind, sind sie auch nicht kontextfrei. $\square$

Pumping-Eigenschaft ist nicht hinreichend



Pumping-Eigenschaft ist nicht hinreichend



Wichtige Konsequenz:

- ▶ Das Pumping-Lemma kann **nicht** verwendet werden, um zu zeigen, dass eine Sprache kontextfrei ist.

Zusammenfassung vom Pumping-Lemma

Bezug zu Kontextfreiheit:

- ▶ Das Pumping-Lemma gibt **eine notwendige Bedingung** für kontextfreie Sprachen. Sehr informell: *Wörter einer kontextfreien Sprache können an zwei Stellen aufgepumpt werden, wenn sie lang genug sind.*
- ▶ Das Pumping-Lemma gibt **keine hinreichende Bedingung** für kontextfreie Sprachen, d.h. Kontextfreiheit kann **nicht** mit dem Pumping-Lemma gezeigt werden.

Anwendung:

- ▶ L hat **nicht** die Pumping-Eigenschaft $\implies L$ ist **nicht** kontextfrei
- ▶ Dies funktioniert nicht für jede nicht kontextfreie Sprache.

6a

**Die Greibach-Normalform und Eigenschaften
von kontextfreien Sprachen**

Prof. Dr. Jasmin Blanchette

Lehr- und Forschungseinheit für
Theoretische Informatik und Theorembeweisen

Stand: 4. Juni 2024

Basierend auf Folien von PD Dr. David Sabel



Satz

Das Leerheitsproblem für kontextfreie Grammatiken ist entscheidbar.

Satz

Das Leerheitsproblem für kontextfreie Grammatiken ist entscheidbar.

Beweis Sei $G = (V, \Sigma, P, S)$ eine CFG mit 1. Sonderregel.

Satz

Das Leerheitsproblem für kontextfreie Grammatiken ist entscheidbar.

Beweis Sei $G = (V, \Sigma, P, S)$ eine CFG mit 1. Sonderregel.

Prüfe zunächst, ob $S \rightarrow \varepsilon \in P$. Wenn ja, dann ist $L(G)$ nicht leer.

Satz

Das Leerheitsproblem für kontextfreie Grammatiken ist entscheidbar.

Beweis Sei $G = (V, \Sigma, P, S)$ eine CFG mit 1. Sonderregel.

Prüfe zunächst, ob $S \rightarrow \varepsilon \in P$. Wenn ja, dann ist $L(G)$ nicht leer.

Sonst: Sei $G' = (V', \Sigma, P', S')$ eine CFG in Chomsky-Normalform mit $L(G') = L(G)$.

Satz

Das Leerheitsproblem für kontextfreie Grammatiken ist entscheidbar.

Beweis Sei $G = (V, \Sigma, P, S)$ eine CFG mit 1. Sonderregel.

Prüfe zunächst, ob $S \rightarrow \varepsilon \in P$. Wenn ja, dann ist $L(G)$ nicht leer.

Sonst: Sei $G' = (V', \Sigma, P', S')$ eine CFG in Chomsky-Normalform mit $L(G') = L(G)$.

Der folgende Algorithmus markiert alle $A \in V'$ mit $\{w \in \Sigma^* \mid A \Rightarrow_{G'}^* w\} \neq \emptyset$.

Satz

Das Leerheitsproblem für kontextfreie Grammatiken ist entscheidbar.

Beweis Sei $G = (V, \Sigma, P, S)$ eine CFG mit 1. Sonderregel.

Prüfe zunächst, ob $S \rightarrow \varepsilon \in P$. Wenn ja, dann ist $L(G)$ nicht leer.

Sonst: Sei $G' = (V', \Sigma, P', S')$ eine CFG in Chomsky-Normalform mit $L(G') = L(G)$.

Der folgende Algorithmus markiert alle $A \in V'$ mit $\{w \in \Sigma^* \mid A \Rightarrow_{G'}^* w\} \neq \emptyset$.

Prüfe, ob S' markiert wird. Wenn ja, dann ist $L(G) = L(G')$ nicht leer. $\square$

Algorithmus 9: Markierung der „nichtleeren“ Variablen

Eingabe: Grammatik $G = (V, \Sigma, P, S)$ in Chomsky-Normalform

Ausgabe: Menge $W \subseteq V$ aller Variablen, die nicht die leere Sprache erzeugen

Beginn

$W := \{A \in V \mid A \rightarrow a \in P, a \in \Sigma\};$

wiederhole

$W_{\text{alt}} := W;$

$W := W_{\text{alt}} \cup \{A \mid A \rightarrow BC \in P, B \in W_{\text{alt}}, C \in W_{\text{alt}}\};$

bis $W = W_{\text{alt}};$

return W

Beispiel für die Markierung der „nichtleeren“ Variablen

Sei $G = (\{S, X, Y, Z\}, \{a, b\}, \{S \rightarrow XY, Z \rightarrow ZZ \mid a \mid b, X \rightarrow ZZ \mid YY, Y \rightarrow ZY\})$.

Beispiel für die Markierung der „nichtleeren“ Variablen

Sei $G = (\{S, X, Y, Z\}, \{a, b\}, \{S \rightarrow XY, Z \rightarrow ZZ \mid a \mid b, X \rightarrow ZZ \mid YY, Y \rightarrow ZY\})$.

Ausführung von Algorithmus 9:

$$0. W := \{A \in V \mid A \rightarrow a \in P\} = \{Z\}$$

Beispiel für die Markierung der „nichtleeren“ Variablen

Sei $G = (\{S, X, Y, Z\}, \{a, b\}, \{S \rightarrow XY, Z \rightarrow ZZ \mid a \mid b, X \rightarrow ZZ \mid YY, Y \rightarrow ZY\})$.

Ausführung von Algorithmus 9:

$$0. W := \{A \in V \mid A \rightarrow a \in P\} = \{Z\}$$

$$1. W_{\text{alt}} := W = \{Z\}$$

$$W := W_{\text{alt}} \cup \{A \mid A \rightarrow BC \in P, B, C \in W_{\text{alt}}\} = \{Z\} \cup \{X, Z\} = \{X, Z\}$$

$$W \neq W_{\text{alt}}$$

Beispiel für die Markierung der „nichtleeren“ Variablen

Sei $G = (\{S, X, Y, Z\}, \{a, b\}, \{S \rightarrow XY, Z \rightarrow ZZ \mid a \mid b, X \rightarrow ZZ \mid YY, Y \rightarrow ZY\})$.

Ausführung von Algorithmus 9:

0. $W := \{A \in V \mid A \rightarrow a \in P\} = \{Z\}$

1. $W_{\text{alt}} := W = \{Z\}$

$$W := W_{\text{alt}} \cup \{A \mid A \rightarrow BC \in P, B, C \in W_{\text{alt}}\} = \{Z\} \cup \{X, Z\} = \{X, Z\}$$

$$W \neq W_{\text{alt}}$$

2. $W_{\text{alt}} := W = \{X, Z\}$

$$W := W_{\text{alt}} \cup \{A \mid A \rightarrow BC \in P, B, C \in W_{\text{alt}}\} = \{X, Z\} \cup \{X, Z\} = \{X, Z\}$$

$$W = W_{\text{alt}}$$

Beispiel für die Markierung der „nichtleeren“ Variablen

Sei $G = (\{S, X, Y, Z\}, \{a, b\}, \{S \rightarrow XY, Z \rightarrow ZZ \mid a \mid b, X \rightarrow ZZ \mid YY, Y \rightarrow ZY\})$.

Ausführung von Algorithmus 9:

0. $W := \{A \in V \mid A \rightarrow a \in P\} = \{Z\}$

1. $W_{\text{alt}} := W = \{Z\}$

$$W := W_{\text{alt}} \cup \{A \mid A \rightarrow BC \in P, B, C \in W_{\text{alt}}\} = \{Z\} \cup \{X, Z\} = \{X, Z\}$$

$$W \neq W_{\text{alt}}$$

2. $W_{\text{alt}} := W = \{X, Z\}$

$$W := W_{\text{alt}} \cup \{A \mid A \rightarrow BC \in P, B, C \in W_{\text{alt}}\} = \{X, Z\} \cup \{X, Z\} = \{X, Z\}$$

$$W = W_{\text{alt}}$$

3. return $W = \{X, Z\}$

Beispiel für die Markierung der „nichtleeren“ Variablen

Sei $G = (\{S, X, Y, Z\}, \{a, b\}, \{S \rightarrow XY, Z \rightarrow ZZ \mid a \mid b, X \rightarrow ZZ \mid YY, Y \rightarrow ZY\})$.

Ausführung von Algorithmus 9:

0. $W := \{A \in V \mid A \rightarrow a \in P\} = \{Z\}$

1. $W_{\text{alt}} := W = \{Z\}$

$$W := W_{\text{alt}} \cup \{A \mid A \rightarrow BC \in P, B, C \in W_{\text{alt}}\} = \{Z\} \cup \{X, Z\} = \{X, Z\}$$

$$W \neq W_{\text{alt}}$$

2. $W_{\text{alt}} := W = \{X, Z\}$

$$W := W_{\text{alt}} \cup \{A \mid A \rightarrow BC \in P, B, C \in W_{\text{alt}}\} = \{X, Z\} \cup \{X, Z\} = \{X, Z\}$$

$$W = W_{\text{alt}}$$

3. return $W = \{X, Z\}$

Da $S \notin W$ folgt, dass G die leere Sprache erzeugt.

Definition

Eine CFG $G = (V, \Sigma, P, S)$ ist in [Greibach-Normalform](#), falls alle Produktionen in P von der Form $A \rightarrow aB_1B_2 \dots B_j$ sind, mit $j \geq 0$, $a \in \Sigma$ und $A, B_1, \dots, B_j \in V$.

Definition

Eine CFG $G = (V, \Sigma, P, S)$ ist in [Greibach-Normalform](#), falls alle Produktionen in P von der Form $A \rightarrow aB_1B_2 \dots B_j$ sind, mit $j \geq 0$, $a \in \Sigma$ und $A, B_1, \dots, B_j \in V$.

Die Normalform ist benannt nach Sheila A. Greibach.

Definition

Eine CFG $G = (V, \Sigma, P, S)$ ist in **Greibach-Normalform**, falls alle Produktionen in P von der Form $A \rightarrow aB_1B_2 \dots B_j$ sind, mit $j \geq 0$, $a \in \Sigma$ und $A, B_1, \dots, B_j \in V$.

Die Normalform ist benannt nach Sheila A. Greibach.

Reguläre Grammatiken sind ein Spezialfall der Greibach-Normalform:

Dort ist nur $j \in \{0, 1\}$ erlaubt.

Die Greibach-Normalform

Definition

Eine CFG $G = (V, \Sigma, P, S)$ ist in **Greibach-Normalform**, falls alle Produktionen in P von der Form $A \rightarrow aB_1B_2 \dots B_j$ sind, mit $j \geq 0$, $a \in \Sigma$ und $A, B_1, \dots, B_j \in V$.

Die Normalform ist benannt nach Sheila A. Greibach.

Reguläre Grammatiken sind ein Spezialfall der Greibach-Normalform:

Dort ist nur $j \in \{0, 1\}$ erlaubt.

Die Greibach-Normalform wird u.a. verwendet, um zu zeigen, dass kontextfreie Sprachen genau von den nichtdeterministischen Kellerautomaten erkannt werden (nächste Vorlesung).

Herstellen der Greibach-Normalform

Satz

Für jede CFG G mit $\varepsilon \notin L(G)$ kann eine CFG G' in Greibach-Normalform berechnet werden, sodass $L(G') = L(G)$ gilt.

Herstellen der Greibach-Normalform

Grundgedanke der Prozedur:

1. Bringe die Eingabe in Chomsky-Normalform.

Herstellen der Greibach-Normalform

Grundgedanke der Prozedur:

1. Bringe die Eingabe in Chomsky-Normalform.
2. Nummeriere die Variablen $A_1, \dots, A_n$ durch.

Herstellen der Greibach-Normalform

Grundgedanke der Prozedur:

1. Bringe die Eingabe in Chomsky-Normalform.
2. Nummeriere die Variablen $A_1, \dots, A_n$ durch.
3. Für $i := 1$ bis n :
 - 3.1 Für jedes $j := 1$ bis $i - 1$, ersetze A_j in $A_i \rightarrow A_j u$ durch alle möglichen rechten Seiten w von $A_j \rightarrow w$ („Inlining von Produktionen“).
 - 3.2 Ersetze jede Produktion $A_i \rightarrow A_j u$ („Entfernen der Linksrekursion“) mithilfe von B_j .

Herstellen der Greibach-Normalform

Grundgedanke der Prozedur:

1. Bringe die Eingabe in Chomsky-Normalform.
2. Nummeriere die Variablen $A_1, \dots, A_n$ durch.
3. Für $i := 1$ bis n :
 - 3.1 Für jedes $j := 1$ bis $i - 1$, ersetze A_j in $A_i \rightarrow A_j u$ durch alle möglichen rechten Seiten w von $A_j \rightarrow w$ („Inlining von Produktionen“).
 - 3.2 Ersetze jede Produktion $A_i \rightarrow A_j u$ („Entfernen der Linksrekursion“) mithilfe von B_j .
(Nun gilt für $A_i \rightarrow A_j u$ stets $i < j$. Damit gilt für $A_n \rightarrow u$, dass u mit einem Terminal beginnt. Die nächste Schleife ersetzt alle restlichen $A_i \rightarrow A_j u$.)

Herstellen der Greibach-Normalform

Grundgedanke der Prozedur:

1. Bringe die Eingabe in Chomsky-Normalform.
2. Nummeriere die Variablen $A_1, \dots, A_n$ durch.
3. Für $i := 1$ bis n :
 - 3.1 Für jedes $j := 1$ bis $i - 1$, ersetze A_j in $A_i \rightarrow A_j u$ durch alle möglichen rechten Seiten w von $A_j \rightarrow w$ („Inlining von Produktionen“).
 - 3.2 Ersetze jede Produktion $A_i \rightarrow A_j u$ („Entfernen der Linksrekursion“) mithilfe von B_j .
(Nun gilt für $A_i \rightarrow A_j u$ stets $i < j$. Damit gilt für $A_n \rightarrow u$, dass u mit einem Terminal beginnt. Die nächste Schleife ersetzt alle restlichen $A_i \rightarrow A_j u$.)
4. Für $i = n - 1$ bis 1:
 - 3.1 Ersetze A_j in $A_i \rightarrow A_j u$ (wo $j > i$) durch alle möglichen rechten Seiten w von $A_j \rightarrow w$ („Inlining von Produktionen“).
(w fängt mit einem Terminal an, da die Schleife absteigend läuft.)

Herstellen der Greibach-Normalform

Grundgedanke der Prozedur:

1. Bringe die Eingabe in Chomsky-Normalform.
2. Nummeriere die Variablen $A_1, \dots, A_n$ durch.
3. Für $i := 1$ bis n :
 - 3.1 Für jedes $j := 1$ bis $i - 1$, ersetze A_j in $A_i \rightarrow A_j u$ durch alle möglichen rechten Seiten w von $A_j \rightarrow w$ („Inlining von Produktionen“).
 - 3.2 Ersetze jede Produktion $A_i \rightarrow A_j u$ („Entfernen der Linksrekursion“) mithilfe von B_i .
(Nun gilt für $A_i \rightarrow A_j u$ stets $i < j$. Damit gilt für $A_n \rightarrow u$, dass u mit einem Terminal beginnt. Die nächste Schleife ersetzt alle restlichen $A_i \rightarrow A_j u$.)
4. Für $i = n - 1$ bis 1:
 - 3.1 Ersetze A_j in $A_i \rightarrow A_j u$ (wo $j > i$) durch alle möglichen rechten Seiten w von $A_j \rightarrow w$ („Inlining von Produktionen“).
(w fängt mit einem Terminal an, da die Schleife absteigend läuft.)
5. Für jede Produktion $B_i \rightarrow A_j u$, ersetze A_j durch alle möglichen rechten Seiten w von $A_j \rightarrow w$ („Inlining von Produktionen“).

Beispiel für die Herstellung der Greibach-Normalform

Sei $G_0 = (\{S, T\}, \{a\}, \{S \rightarrow TT, T \rightarrow TS \mid a\}, S)$ eine CFG.

1. G_0 befindet sich bereits in Chomsky-Normalform.

Beispiel für die Herstellung der Greibach-Normalform

Sei $G_0 = (\{S, T\}, \{a\}, \{S \rightarrow TT, T \rightarrow TS \mid a\}, S)$ eine CFG.

1. G_0 befindet sich bereits in Chomsky-Normalform.
2. Das Durchnummerieren der Variablen führt z.B. zu
 $G_2 = (\{A_1, A_2\}, \{a\}, \{A_1 \rightarrow A_2A_2, A_2 \rightarrow A_2A_1 \mid a\}, A_1).$

Beispiel für die Herstellung der Greibach-Normalform

Sei $G_0 = (\{S, T\}, \{a\}, \{S \rightarrow TT, T \rightarrow TS \mid a\}, S)$ eine CFG.

1. G_0 befindet sich bereits in Chomsky-Normalform.
2. Das Durchnummerieren der Variablen führt z.B. zu
 $G_2 = (\{A_1, A_2\}, \{a\}, \{A_1 \rightarrow A_2A_2, A_2 \rightarrow A_2A_1 \mid a\}, A_1)$.
3. Ersetze linksrekursive $A_2 \rightarrow A_2A_1 \mid a$ durch $A_2 \rightarrow aB_2 \mid a, B_2 \rightarrow A_1B_2 \mid A_1$.
Dies führt zu
 $G_3 = (\{A_1, A_2\}, \{a\}, \{A_1 \rightarrow A_2A_2, A_2 \rightarrow aB_2 \mid a, B_2 \rightarrow A_1B_2 \mid A_1\}, A_1)$.

Beispiel für die Herstellung der Greibach-Normalform

Sei $G_0 = (\{S, T\}, \{a\}, \{S \rightarrow TT, T \rightarrow TS \mid a\}, S)$ eine CFG.

1. G_0 befindet sich bereits in Chomsky-Normalform.
2. Das Durchnummerieren der Variablen führt z.B. zu
 $G_2 = (\{A_1, A_2\}, \{a\}, \{A_1 \rightarrow A_2A_2, A_2 \rightarrow A_2A_1 \mid a\}, A_1)$.
3. Ersetze linksrekursive $A_2 \rightarrow A_2A_1 \mid a$ durch $A_2 \rightarrow aB_2 \mid a, B_2 \rightarrow A_1B_2 \mid A_1$.
Dies führt zu
 $G_3 = (\{A_1, A_2\}, \{a\}, \{A_1 \rightarrow A_2A_2, A_2 \rightarrow aB_2 \mid a, B_2 \rightarrow A_1B_2 \mid A_1\}, A_1)$.
4. Ersetze mithilfe Inlining $A_1 \rightarrow A_2A_2$ durch $A_1 \rightarrow aB_2A_2 \mid aA_2$. Dies führt zu
 $G_4 = (\{A_1, A_2\}, \{a\}, \{A_1 \rightarrow aB_2A_2 \mid aA_2, A_2 \rightarrow aB_2 \mid a, B_2 \rightarrow A_1B_2 \mid A_1\}, A_1)$.

Beispiel für die Herstellung der Greibach-Normalform

Sei $G_0 = (\{S, T\}, \{a\}, \{S \rightarrow TT, T \rightarrow TS \mid a\}, S)$ eine CFG.

1. G_0 befindet sich bereits in Chomsky-Normalform.
2. Das Durchnummerieren der Variablen führt z.B. zu
 $G_2 = (\{A_1, A_2\}, \{a\}, \{A_1 \rightarrow A_2A_2, A_2 \rightarrow A_2A_1 \mid a\}, A_1)$.
3. Ersetze linksrekursive $A_2 \rightarrow A_2A_1 \mid a$ durch $A_2 \rightarrow aB_2 \mid a, B_2 \rightarrow A_1B_2 \mid A_1$.
Dies führt zu
 $G_3 = (\{A_1, A_2\}, \{a\}, \{A_1 \rightarrow A_2A_2, A_2 \rightarrow aB_2 \mid a, B_2 \rightarrow A_1B_2 \mid A_1\}, A_1)$.
4. Ersetze mithilfe Inlining $A_1 \rightarrow A_2A_2$ durch $A_1 \rightarrow aB_2A_2 \mid aA_2$. Dies führt zu
 $G_4 = (\{A_1, A_2\}, \{a\}, \{A_1 \rightarrow aB_2A_2 \mid aA_2, A_2 \rightarrow aB_2 \mid a, B_2 \rightarrow A_1B_2 \mid A_1\}, A_1)$.
5. Ersetze mithilfe Inlining $B_2 \rightarrow A_1B_2$ durch $B_2 \rightarrow aB_2A_2B_2 \mid aA_2B_2$ und
 $B_2 \rightarrow A_1$ durch $B_2 \rightarrow aB_2A_2 \mid aA_2$. Dies führt zu
 $G_5 = (\{A_1, A_2\}, \{a\}, \{A_1 \rightarrow aB_2A_2 \mid aA_2, A_2 \rightarrow aB_2 \mid a, B_2 \rightarrow aB_2A_2B_2 \mid aA_2B_2 \mid aB_2A_2 \mid aA_2\}, A_1)$.

Links- und rechtsrekursive Produktion

Definition

Seien $G = (V, \Sigma, P, S)$, $A \in V$, $u \in (V \cup \Sigma)^*$. Eine Produktion ist

- ▶ linksrekursiv, wenn sie von der Form $A \rightarrow Au$ ist
- ▶ rechtsrekursiv, wenn sie von der Form $A \rightarrow uA$ ist.

Lemma (Entfernen der Linksrekursion)

Sei $G = (V, \Sigma, Q \cup \underbrace{\{A \rightarrow Au_1 \mid \dots \mid Au_n \mid w_1 \mid \dots \mid w_m\}}_R, S)$ eine CFG mit

- ▶ R sind alle Produktionen mit A als linker Seite
- ▶ die Satzformen $w_1, \dots, w_m$ beginnen alle nicht mit A .

Entfernen der Linksrekursion

Lemma (Entfernen der Linksrekursion)

Sei $G = (V, \Sigma, Q \cup \underbrace{\{A \rightarrow Au_1 \mid \dots \mid Au_n \mid w_1 \mid \dots \mid w_m\}}_R, S)$ eine CFG mit

- ▶ R sind alle Produktionen mit A als linker Seite
- ▶ die Satzformen $w_1, \dots, w_m$ beginnen alle nicht mit A .

Es gilt $L(G') = L(G)$ für $G' = (V \cup \{B\}, \Sigma, Q \cup R', S)$, wobei B eine neue Variable ist und

$$R' := \{A \rightarrow w_1 B \mid \dots \mid w_m B \mid w_1 \mid \dots \mid w_m, \\ B \rightarrow u_1 B \mid \dots \mid u_n B \mid u_1 \mid \dots \mid u_n\}$$

Entfernen der Linksrekursion

Lemma (Entfernen der Linksrekursion)

Sei $G = (V, \Sigma, Q \cup \underbrace{\{A \rightarrow Au_1 \mid \dots \mid Au_n \mid w_1 \mid \dots \mid w_m\}}_R, S)$ eine CFG mit

- ▶ R sind alle Produktionen mit A als linker Seite
- ▶ die Satzformen $w_1, \dots, w_m$ beginnen alle nicht mit A .

Es gilt $L(G') = L(G)$ für $G' = (V \cup \{B\}, \Sigma, Q \cup R', S)$, wobei B eine neue Variable ist und

$$R' := \{A \rightarrow w_1 B \mid \dots \mid w_m B \mid w_1 \mid \dots \mid w_m, \\ B \rightarrow u_1 B \mid \dots \mid u_n B \mid u_1 \mid \dots \mid u_n\}$$

Beweis Siehe Skript.



Beispiel für das Entfernen der Linksrekursion

Sei die CFG $G = (\{A, C\}, \{b, c, d\}, \{A \rightarrow ACA \mid bb, C \rightarrow Ccc \mid d\}, A)$.

Beispiel für das Entfernen der Linksrekursion

Sei die CFG $G = (\{A, C\}, \{b, c, d\}, \{A \rightarrow ACA \mid bb, C \rightarrow Ccc \mid d\}, A)$.

Entfernen der Linksrekursion für A ergibt

$$G' = (\{A, B, C\}, \{b, c, d\}, \{A \rightarrow bbB \mid bb, B \rightarrow CAB \mid CA, C \rightarrow Ccc \mid d\}, A)$$

Beispiel für das Entfernen der Linksrekursion

Sei die CFG $G = (\{A, C\}, \{b, c, d\}, \{A \rightarrow ACA \mid bb, C \rightarrow Ccc \mid d\}, A)$.

Entfernen der Linksrekursion für A ergibt

$$G' = (\{A, B, C\}, \{b, c, d\}, \{A \rightarrow bbB \mid bb, B \rightarrow CAB \mid CA, C \rightarrow Ccc \mid d\}, A)$$

Anschließendes Entfernen der Linksrekursion für C ergibt

$$G'' = (\{A, B, C, D\}, \{b, c, d\}, \\ \{A \rightarrow bbB \mid bb, B \rightarrow CAB \mid CA, C \rightarrow dD \mid d, D \rightarrow ccD \mid cc\}, A)$$

Algorithmus 7: Herstellen der Greibach-Normalform

Eingabe: CFG $G = (\{A_1, \dots, A_n\}, \Sigma, P, A_i)$ in Chomsky-Normalform mit $\varepsilon \notin L(G)$

Ausgabe: CFG G' in Greibach-Normalform mit $L(G) = L(G')$

Beginn

für $i = 1$ *bis* n **tue**

für $j = 1$ *bis* $i - 1$ **tue**

für alle $A_i \rightarrow A_j u \in P$ **tue**

 Seien $A_j \rightarrow w_1 \mid \dots \mid w_m$ alle Produktionen in P mit A_j als linker Seite;

 Ersetze $A_i \rightarrow A_j u$ durch $A_i \rightarrow w_1 u \mid \dots \mid w_m u$ in P ;

wenn $A_i \rightarrow A_i u \in P$ **dann**

 Eliminiere solche Produktionen mit der Operation „Entfernen der Linksrekursion“;

 Sei B_i die dabei neu erzeugte Variable;

für $i = n - 1$ *bis* 1 **tue**

für alle $A_i \rightarrow A_j u \in P, j > i$ **tue**

 Seien $A_j \rightarrow w_1 \mid \dots \mid w_m$ alle Produktionen in P mit A_j als linker Seite;

 Ersetze $A_i \rightarrow A_j u$ durch $A_i \rightarrow w_1 u \mid \dots \mid w_m u$ in P ;

für $i = 1$ *bis* n **tue**

für alle $B_i \rightarrow A_j u \in P$ **tue**

 Seien $A_j \rightarrow w_1 \mid \dots \mid w_m$ alle Produktionen in P mit A_j als linker Seite;

 Ersetze $B_i \rightarrow A_j u$ durch $B_i \rightarrow w_1 u \mid \dots \mid w_m u$ in P ;

Satz

Für jede CFG G mit $\varepsilon \notin L(G)$ kann eine CFG G' in Greibach-Normalform berechnet werden, sodass $L(G') = L(G)$ gilt.

Satz

Für jede CFG G mit $\varepsilon \notin L(G)$ kann eine CFG G' in Greibach-Normalform berechnet werden, sodass $L(G') = L(G)$ gilt.

Beweis Korrektheit folgt durch Korrektheit der Operationen „Inlining von Produktionen“ und „Entfernen der Linksrekursion“, mithilfe von geeigneten Invarianten. □

Abschlusseigenschaften der kontextfreien Sprachen

Theorem

Die kontextfreien Sprachen sind abgeschlossen bezüglich **Vereinigung**, **Produkt** und **Kleeneschem Abschluss**.

Abschlusseigenschaften der kontextfreien Sprachen

Theorem

Die kontextfreien Sprachen sind abgeschlossen bezüglich Vereinigung, Produkt und Kleeneschem Abschluss.

Beweis Seien L_1, L_2 CFLs und $G_i = (V_i, \Sigma_i, P_i, S_i)$ CFGs mit $L(G_i) = L_i$ für $i = 1, 2$.
O.B.d.A. sei $V_1 \cap V_2 = \emptyset$.

Abschlusseigenschaften der kontextfreien Sprachen

Theorem

Die kontextfreien Sprachen sind abgeschlossen bezüglich Vereinigung, Produkt und Kleeneschem Abschluss.

Beweis Seien L_1, L_2 CFLs und $G_i = (V_i, \Sigma_i, P_i, S_i)$ CFGs mit $L(G_i) = L_i$ für $i = 1, 2$.
O.B.d.A. sei $V_1 \cap V_2 = \emptyset$.

Seien S, S' neue Variablen (d.h. $\{S, S'\} \cap (V_1 \cup V_2) = \emptyset$).

Abschlusseigenschaften der kontextfreien Sprachen

Theorem

Die kontextfreien Sprachen sind abgeschlossen bezüglich Vereinigung, Produkt und Kleeneschem Abschluss.

Beweis Seien L_1, L_2 CFLs und $G_i = (V_i, \Sigma_i, P_i, S_i)$ CFGs mit $L(G_i) = L_i$ für $i = 1, 2$.
O.B.d.A. sei $V_1 \cap V_2 = \emptyset$.

Seien S, S' neue Variablen (d.h. $\{S, S'\} \cap (V_1 \cup V_2) = \emptyset$).

1. **Vereinigung:** Sei $G_U = (V_1 \cup V_2 \cup \{S\}, \Sigma_1 \cup \Sigma_2, P_1 \cup P_2 \cup \{S \rightarrow S_1 \mid S_2\}, S)$.
Dann gilt $L(G_U) = L(G_1) \cup L(G_2) = L_1 \cup L_2$.

Abschlusseigenschaften der kontextfreien Sprachen

Theorem

Die kontextfreien Sprachen sind abgeschlossen bezüglich **Vereinigung**, **Produkt** und **Kleeneschem Abschluss**.

Beweis Seien L_1, L_2 CFLs und $G_i = (V_i, \Sigma_i, P_i, S_i)$ CFGs mit $L(G_i) = L_i$ für $i = 1, 2$.
O.B.d.A. sei $V_1 \cap V_2 = \emptyset$.

Seien S, S' neue Variablen (d.h. $\{S, S'\} \cap (V_1 \cup V_2) = \emptyset$).

- Vereinigung:** Sei $G_U = (V_1 \cup V_2 \cup \{S\}, \Sigma_1 \cup \Sigma_2, P_1 \cup P_2 \cup \{S \rightarrow S_1 \mid S_2\}, S)$.
Dann gilt $L(G_U) = L(G_1) \cup L(G_2) = L_1 \cup L_2$.
- Produkt:** Sei $G_\cdot = (V_1 \cup V_2 \cup \{S\}, \Sigma_1 \cup \Sigma_2, P_1 \cup P_2 \cup \{S \rightarrow S_1 S_2\}, S)$.
Dann gilt $L(G_\cdot) = L(G_1)L(G_2) = L_1 L_2$.

Abschlusseigenschaften der kontextfreien Sprachen

Theorem

Die kontextfreien Sprachen sind abgeschlossen bezüglich **Vereinigung**, **Produkt** und **Kleeneschem Abschluss**.

Beweis Seien L_1, L_2 CFLs und $G_i = (V_i, \Sigma_i, P_i, S_i)$ CFGs mit $L(G_i) = L_i$ für $i = 1, 2$.
O.B.d.A. sei $V_1 \cap V_2 = \emptyset$.

Seien S, S' neue Variablen (d.h. $\{S, S'\} \cap (V_1 \cup V_2) = \emptyset$).

- Vereinigung:** Sei $G_U = (V_1 \cup V_2 \cup \{S\}, \Sigma_1 \cup \Sigma_2, P_1 \cup P_2 \cup \{S \rightarrow S_1 \mid S_2\}, S)$.
Dann gilt $L(G_U) = L(G_1) \cup L(G_2) = L_1 \cup L_2$.
- Produkt:** Sei $G_\cdot = (V_1 \cup V_2 \cup \{S\}, \Sigma_1 \cup \Sigma_2, P_1 \cup P_2 \cup \{S \rightarrow S_1 S_2\}, S)$.
Dann gilt $L(G_\cdot) = L(G_1)L(G_2) = L_1 L_2$.
- Kleenescher Abschluss:** Sei $G_* = (V_1 \cup \{S, S'\}, \Sigma, P', S')$ mit
 $P' = (P \setminus \{S_1 \rightarrow \varepsilon\}) \cup \{S' \rightarrow \varepsilon, S' \rightarrow S, S \rightarrow SS, S \rightarrow S_1\}$.
Dann gilt $L(G_*) = L(G_1)^*$. □

Abschlusseigenschaften der kontextfreien Sprachen

Theorem

Die kontextfreien Sprachen sind **nicht** abgeschlossen unter **Schnitt** und **Komplement**.

Abschlusseigenschaften der kontextfreien Sprachen

Theorem

Die kontextfreien Sprachen sind **nicht** abgeschlossen unter **Schnitt** und **Komplement**.

Beweis

1. **Schnitt:** Seien $L_1 = \{a^n b^m c^m \mid m, n \in \mathbb{N}\}$ und $L_2 = \{a^m b^m c^n \mid m, n \in \mathbb{N}\}$.

Abschlusseigenschaften der kontextfreien Sprachen

Theorem

Die kontextfreien Sprachen sind **nicht** abgeschlossen unter **Schnitt** und **Komplement**.

Beweis

1. **Schnitt:** Seien $L_1 = \{a^n b^m c^m \mid m, n \in \mathbb{N}\}$ und $L_2 = \{a^m b^m c^n \mid m, n \in \mathbb{N}\}$.

Seien

$$G_1 = (\{A, D, S\}, \{a, b, c\}, \{S \rightarrow AD, A \rightarrow \varepsilon \mid aA, D \rightarrow \varepsilon \mid bDc\}, S)$$

$$G_2 = (\{C, D, S\}, \{a, b, c\}, \{S \rightarrow DC, \varepsilon \mid C \rightarrow cC, D \rightarrow \varepsilon \mid aDb\}, S)$$

Abschlusseigenschaften der kontextfreien Sprachen

Theorem

Die kontextfreien Sprachen sind **nicht** abgeschlossen unter **Schnitt** und **Komplement**.

Beweis

1. **Schnitt:** Seien $L_1 = \{a^n b^m c^m \mid m, n \in \mathbb{N}\}$ und $L_2 = \{a^m b^m c^n \mid m, n \in \mathbb{N}\}$.

Seien

$$G_1 = (\{A, D, S\}, \{a, b, c\}, \{S \rightarrow AD, A \rightarrow \varepsilon \mid aA, D \rightarrow \varepsilon \mid bDc\}, S)$$

$$G_2 = (\{C, D, S\}, \{a, b, c\}, \{S \rightarrow DC, \varepsilon \mid C \rightarrow cC, D \rightarrow \varepsilon \mid aDb\}, S)$$

$L(G_i) = L_i$ für $i \in \{1, 2\}$, daher sind L_1 und L_2 beide kontextfrei.

Abschlusseigenschaften der kontextfreien Sprachen

Theorem

Die kontextfreien Sprachen sind **nicht** abgeschlossen unter **Schnitt** und **Komplement**.

Beweis

1. **Schnitt:** Seien $L_1 = \{a^n b^m c^m \mid m, n \in \mathbb{N}\}$ und $L_2 = \{a^m b^m c^n \mid m, n \in \mathbb{N}\}$.

Seien

$$G_1 = (\{A, D, S\}, \{a, b, c\}, \{S \rightarrow AD, A \rightarrow \varepsilon \mid aA, D \rightarrow \varepsilon \mid bDc\}, S)$$

$$G_2 = (\{C, D, S\}, \{a, b, c\}, \{S \rightarrow DC, \varepsilon \mid C \rightarrow cC, D \rightarrow \varepsilon \mid aDb\}, S)$$

$L(G_i) = L_i$ für $i \in \{1, 2\}$, daher sind L_1 und L_2 beide kontextfrei.

$L_1 \cap L_2 = \{a^n b^n c^n \mid n \in \mathbb{N}\}$ ist aber bekanntlich nicht kontextfrei.

Abschlusseigenschaften der kontextfreien Sprachen

Theorem

Die kontextfreien Sprachen sind **nicht** abgeschlossen unter **Schnitt** und **Komplement**.

Beweis

1. **Schnitt:** Seien $L_1 = \{a^n b^m c^m \mid m, n \in \mathbb{N}\}$ und $L_2 = \{a^m b^m c^n \mid m, n \in \mathbb{N}\}$.

Seien

$$G_1 = (\{A, D, S\}, \{a, b, c\}, \{S \rightarrow AD, A \rightarrow \varepsilon \mid aA, D \rightarrow \varepsilon \mid bDc\}, S)$$

$$G_2 = (\{C, D, S\}, \{a, b, c\}, \{S \rightarrow DC, \varepsilon \mid C \rightarrow cC, D \rightarrow \varepsilon \mid aDb\}, S)$$

$L(G_i) = L_i$ für $i \in \{1, 2\}$, daher sind L_1 und L_2 beide kontextfrei.

$L_1 \cap L_2 = \{a^n b^n c^n \mid n \in \mathbb{N}\}$ ist aber bekanntlich nicht kontextfrei.

Daher sind die CFLs nicht abgeschlossen bezüglich Schnitt.

Beweis (Fortsetzung)

2. **Komplement:** Durch Widerspruch. Wir nehmen an, dass wenn L CFL ist, dann ist auch $\overline{L}$ CFL.

Beweis (Fortsetzung)

2. **Komplement:** Durch Widerspruch. Wir nehmen an, dass wenn L CFL ist, dann ist auch $\overline{L}$ CFL.

Seien L_1, L_2 CFLs. Dann ist auch $\overline{\overline{L_1} \cup \overline{L_2}}$ CFL
(da CFLs abgeschlossen bezüglich $\overline{}$ und $\cup$).

Beweis (Fortsetzung)

2. **Komplement:** Durch Widerspruch. Wir nehmen an, dass wenn L CFL ist, dann ist auch $\overline{L}$ CFL.

Seien L_1, L_2 CFLs. Dann ist auch $\overline{\overline{L_1} \cup \overline{L_2}}$ CFL
(da CFLs abgeschlossen bezüglich $\overline{}$ und $\cup$).

$\overline{\overline{L_1} \cup \overline{L_2}} = L_1 \cap L_2$. Dies würde heißen, dass die CFLs abgeschlossen bezüglich Schnitt sind. Wir haben aber unter Punkt 1 genau das Gegenteil beweisen.
Widerspruch. □

6b

Der CYK-Algorithmus

Prof. Dr. Jasmin Blanchette

Lehr- und Forschungseinheit für
Theoretische Informatik und Theorembeweisen

Stand: 29. Mai 2024
Basierend auf Folien von PD Dr. David Sabel



Wiederholung: Entscheidbarkeit

Definition

Eine Sprache L ist **entscheidbar**, wenn es einen Algorithmus gibt, der bei Eingabe eines Wortes w in endlicher Zeit feststellt, ob $w \in L$ gilt oder nicht.

Wortproblem für Typ 2-Grammatiken

Definition

Das **Wortproblem** für Typ i -Grammatiken ist die Frage, ob für eine gegebene Typ i -Grammatik $G = (V, \Sigma, P, S)$ und ein Wort $w \in \Sigma^*$ $w \in L(G)$ gilt oder nicht.

Wortproblem für Typ 2-Grammatiken

Definition

Das **Wortproblem** für Typ i -Grammatiken ist die Frage, ob für eine gegebene Typ i -Grammatik $G = (V, \Sigma, P, S)$ und ein Wort $w \in \Sigma^*$ $w \in L(G)$ gilt oder nicht.

Satz

Das Wortproblem für Typ 2-Grammatiken ist entscheidbar:

Es gibt einen Algorithmus, der bei Eingabe von Typ 2-Grammatik G und Wort w nach endlicher Zeit entscheidet, ob $w \in L(G)$ gilt oder nicht. Zudem entscheidet er das Wortproblem in Polynomialzeit.

Grundgedanke des Algorithmus

Der **CYK-Algorithmus** von Cocke, Younger und Kasami ist ein Polynomialzeitalgorithmus für das Wortproblem für Typ 2-Grammatiken.

Grundgedanke des Algorithmus

Der **CYK-Algorithmus** von Cocke, Younger und Kasami ist ein Polynomialzeitalgorithmus für das Wortproblem für Typ 2-Grammatiken.

Eingabe: Eine CFG $G = (V, \Sigma, P, S)$ in Chomsky-Normalform und ein Wort $w \in \Sigma^+$.

Grundgedanke des Algorithmus

Der **CYK-Algorithmus** von Cocke, Younger und Kasami ist ein Polynomialzeitalgorithmus für das Wortproblem für Typ 2-Grammatiken.

Eingabe: Eine CFG $G = (V, \Sigma, P, S)$ in Chomsky-Normalform und ein Wort $w \in \Sigma^+$.

G ist in **Chomsky-Normalform**, wenn für jede Produktion $A \rightarrow w \in P$ gilt:
 $w = a \in \Sigma$ oder $w = BC$ mit $B, C \in V$.

Für jede CFG G mit $\varepsilon \notin L(G)$ kann eine Chomsky-Normalform G' mit $L(G') = L(G)$ berechnet werden.

Grundgedanke des Algorithmus

Der **CYK-Algorithmus** von Cocke, Younger und Kasami ist ein Polynomialzeitalgorithmus für das Wortproblem für Typ 2-Grammatiken.

Eingabe: Eine CFG $G = (V, \Sigma, P, S)$ in Chomsky-Normalform und ein Wort $w \in \Sigma^+$.

G ist in **Chomsky-Normalform**, wenn für jede Produktion $A \rightarrow w \in P$ gilt:
 $w = a \in \Sigma$ oder $w = BC$ mit $B, C \in V$.

Für jede CFG G mit $\varepsilon \notin L(G)$ kann eine Chomsky-Normalform G' mit $L(G') = L(G)$ berechnet werden.

Ausgabe: **ja**, wenn $w \in L(G)$, sonst **nein**.

Grundgedanke des Algorithmus

Der **CYK-Algorithmus** von Cocke, Younger und Kasami ist ein Polynomialzeitalgorithmus für das Wortproblem für Typ 2-Grammatiken.

Eingabe: Eine CFG $G = (V, \Sigma, P, S)$ in Chomsky-Normalform und ein Wort $w \in \Sigma^+$.

G ist in **Chomsky-Normalform**, wenn für jede Produktion $A \rightarrow w \in P$ gilt:
 $w = a \in \Sigma$ oder $w = BC$ mit $B, C \in V$.

Für jede CFG G mit $\varepsilon \notin L(G)$ kann eine Chomsky-Normalform G' mit $L(G') = L(G)$ berechnet werden.

Ausgabe: **ja**, wenn $w \in L(G)$, sonst **nein**.

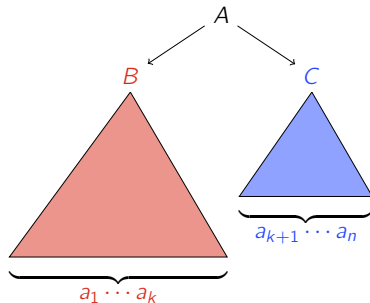
Grober Ansatz:

1. Teste für jede Variable A und Teilwort u von w , ob sie es erzeugt.
2. Verwende Test zum Prüfen, ob das Startsymbol S das Wort w erzeugt.

Grundgedanke des CYK-Algorithmus

Prüfe, ob $A \in V$ ein Wort $u = a_1 \cdots a_n$ ($n \geq 1$) erzeugt:

- ▶ Wenn $u = a_1 \in \Sigma$, dann prüfe ob $A \rightarrow a_1 \in P$.
- ▶ Anderenfalls ($n > 1$) kann u nur erzeugt werden, wenn
 - ▶ es eine Produktion $A \rightarrow BC \in P$ gibt und
 - ▶ es einen Index $1 \leq k < n$ gibt, sodass
 B erzeugt $a_1 \cdots a_k$ und
 C erzeugt $a_{k+1} \cdots a_n$.



- ▶ Daher prüfe für **alle** $A \rightarrow BC \in P$ und **alle** k mit $1 \leq k < n$ **rekursiv**, ob B das Wort $a_1 \cdots a_k$ und C das Wort $a_{k+1} \cdots a_n$ erzeugt.

Beispiel für naives rekursives Suchen

Seien die CFG $G = (V, \Sigma, P, S)$ mit

$$P = \{ S \rightarrow AB \mid BA, \\ A \rightarrow AA \mid AB \mid a, \\ B \rightarrow BB \mid b$$

und das Wort $bbbaab$.

Beispiel für naives rekursives Suchen

Seien die CFG $G = (V, \Sigma, P, S)$ mit

$$P = \{S \rightarrow AB \mid BA, \\ A \rightarrow AA \mid AB \mid a, \\ B \rightarrow BB \mid b\}$$

und das Wort $bbbaab$.

S erzeugt $bbbaab$, denn $S \rightarrow BA \in P$ und

- ▶ B erzeugt bbb , denn $B \rightarrow BB \in P$ und
 - ▶ B erzeugt bb , denn $B \rightarrow BB \in P$ und
 - ▶ B erzeugt b , denn $B \rightarrow b \in P$
 - ▶ B erzeugt b , denn $B \rightarrow b \in P$
 - ▶ B erzeugt b , denn $B \rightarrow b \in P$
- ▶ A erzeugt aab , denn $A \rightarrow AB \in P$ und
 - ▶ A erzeugt aa , denn $A \rightarrow AA \in P$ und
 - ▶ A erzeugt a , denn $A \rightarrow a \in P$ und
 - ▶ A erzeugt a , denn $A \rightarrow a \in P$
 - ▶ B erzeugt b , denn $B \rightarrow b \in P$.

Beispiel für naives rekursives Suchen

Seien die CFG $G = (V, \Sigma, P, S)$ mit

$$P = \{S \rightarrow AB \mid BA, \\ A \rightarrow AA \mid AB \mid a, \\ B \rightarrow BB \mid b\}$$

und das Wort $bbbaab$.

S erzeugt $bbbaab$, denn $S \rightarrow BA \in P$ und

- ▶ B erzeugt bbb , denn $B \rightarrow BB \in P$ und
 - ▶ B erzeugt bb , denn $B \rightarrow BB \in P$ und
 - ▶ B erzeugt b , denn $B \rightarrow b \in P$
 - ▶ B erzeugt b , denn $B \rightarrow b \in P$
 - ▶ B erzeugt b , denn $B \rightarrow b \in P$
- ▶ A erzeugt aab , denn $A \rightarrow AB \in P$ und
 - ▶ A erzeugt aa , denn $A \rightarrow AA \in P$ und
 - ▶ A erzeugt a , denn $A \rightarrow a \in P$ und
 - ▶ A erzeugt a , denn $A \rightarrow a \in P$
 - ▶ B erzeugt b , denn $B \rightarrow b \in P$.

Beispiel für naives rekursives Suchen

Seien die CFG $G = (V, \Sigma, P, S)$ mit

$$P = \{S \rightarrow AB \mid BA, \\ A \rightarrow AA \mid AB \mid a, \\ B \rightarrow BB \mid b\}$$

und das Wort $bbbaab$.

S erzeugt $bbbaab$, denn $S \rightarrow BA \in P$ und

- ▶ B erzeugt bbb , denn $B \rightarrow BB \in P$ und
 - ▶ B erzeugt bb , denn $B \rightarrow BB \in P$ und
 - ▶ B erzeugt b , denn $B \rightarrow b \in P$
 - ▶ B erzeugt b , denn $B \rightarrow b \in P$
 - ▶ B erzeugt b , denn $B \rightarrow b \in P$
- ▶ A erzeugt aab , denn $A \rightarrow AB \in P$ und
 - ▶ A erzeugt aa , denn $A \rightarrow AA \in P$ und
 - ▶ A erzeugt a , denn $A \rightarrow a \in P$ und
 - ▶ A erzeugt a , denn $A \rightarrow a \in P$
 - ▶ B erzeugt b , denn $B \rightarrow b \in P$.

Um an Effizienz zu gewinnen:

Statt Rekursion verwende [dynamische Programmierung](#).

Grundgedanke des CYK-Algorithmus

Der Algorithmus berechnet Mengen $V(i, j) \subseteq V$, sodass

$$V(i, j) := \{A \in V \mid A \Rightarrow^* a_i \cdots a_{i+j-1}\}$$

Informell: $V(i, j)$ enthält alle Variablen $A \in V$, die $a_i \cdots a_{i+j-1}$ (= das Teilwort von w ab Position i mit Länge j) erzeugen.

Grundgedanke des CYK-Algorithmus

Der Algorithmus berechnet Mengen $V(i, j) \subseteq V$, sodass

$$V(i, j) := \{A \in V \mid A \Rightarrow^* a_i \cdots a_{i+j-1}\}$$

Informell: $V(i, j)$ enthält alle Variablen $A \in V$, die $a_i \cdots a_{i+j-1}$ (= das Teilwort von w ab Position i mit Länge j) erzeugen.

Schritte:

1. Beginne mit $V(i, 1) = \{A \mid A \rightarrow a_i \in P\}$.

Grundgedanke des CYK-Algorithmus

Der Algorithmus berechnet Mengen $V(i, j) \subseteq V$, sodass

$$V(i, j) := \{A \in V \mid A \Rightarrow^* a_i \cdots a_{i+j-1}\}$$

Informell: $V(i, j)$ enthält alle Variablen $A \in V$, die $a_i \cdots a_{i+j-1}$ (= das Teilwort von w ab Position i mit Länge j) erzeugen.

Schritte:

1. Beginne mit $V(i, 1) = \{A \mid A \rightarrow a_i \in P\}$.
2. Berechne $V(i, j)$ für $j = 2$ bis n . Für $j > 1$ gilt:

$$A \in V(i, j) \text{ g.d.w. es gibt } k \in \{1, 2, \dots, j-1\}, \text{ sodass} \\ A \rightarrow BC \in P, B \in V(i, k) \text{ und } C \in V(i+k, j-k)$$

Grundgedanke des CYK-Algorithmus

Der Algorithmus berechnet Mengen $V(i, j) \subseteq V$, sodass

$$V(i, j) := \{A \in V \mid A \Rightarrow^* a_i \cdots a_{i+j-1}\}$$

Informell: $V(i, j)$ enthält alle Variablen $A \in V$, die $a_i \cdots a_{i+j-1}$ (= das Teilwort von w ab Position i mit Länge j) erzeugen.

Schritte:

1. Beginne mit $V(i, 1) = \{A \mid A \rightarrow a_i \in P\}$.
2. Berechne $V(i, j)$ für $j = 2$ bis n . Für $j > 1$ gilt:

$$A \in V(i, j) \text{ g.d.w. es gibt } k \in \{1, 2, \dots, j-1\}, \text{ sodass} \\ A \rightarrow BC \in P, B \in V(i, k) \text{ und } C \in V(i+k, j-k)$$

3. Prüfe, ob $S \in V(1, n)$.

Beispiel für den CYK-Algorithmus

Seien $w = bbd dc$ und $G = (\{S, A, B, C, D, E\}, \{b, c, d\}, P, S)$ mit
 $P = \{S \rightarrow AC, A \rightarrow BE, A \rightarrow BD, E \rightarrow AD, C \rightarrow c, B \rightarrow b, D \rightarrow d\}$.

		b	b	d	d	c
		$\xrightarrow{i}$				
	$V(i, j)$	1	2	3	4	5
$j \downarrow$	1					
	2					
	3					
	4					
	5					

1. Schritt: Füllen der $V(i, 1)$ -Einträge

Beispiel für den CYK-Algorithmus

Seien $w = bbddc$ und $G = (\{S, A, B, C, D, E\}, \{b, c, d\}, P, S)$ mit
 $P = \{S \rightarrow AC, A \rightarrow BE, A \rightarrow BD, E \rightarrow AD, C \rightarrow c, \textcolor{red}{B} \rightarrow \textcolor{red}{b}, D \rightarrow d\}$.

		b	b	d	d	c
		$\xrightarrow{i}$				
	$V(i,j)$	1	2	3	4	5
j ↓	1	B				
	2					
	3					
	4					
	5					

1. Schritt: Füllen der $V(i, 1)$ -Einträge

Beispiel für den CYK-Algorithmus

Seien $w = bbddc$ und $G = (\{S, A, B, C, D, E\}, \{b, c, d\}, P, S)$ mit
 $P = \{S \rightarrow AC, A \rightarrow BE, A \rightarrow BD, E \rightarrow AD, C \rightarrow c, \textcolor{red}{B} \rightarrow \textcolor{red}{b}, D \rightarrow d\}$.

		b	b	d	d	c
		$\xrightarrow{i}$				
	$V(i,j)$	1	2	3	4	5
$j \downarrow$	1	B	B			
	2					
	3					
	4					
	5					

1. Schritt: Füllen der $V(i, 1)$ -Einträge

Beispiel für den CYK-Algorithmus

Seien $w = bbddc$ und $G = (\{S, A, B, C, D, E\}, \{b, c, d\}, P, S)$ mit
 $P = \{S \rightarrow AC, A \rightarrow BE, A \rightarrow BD, E \rightarrow AD, C \rightarrow c, B \rightarrow b, D \rightarrow d\}$.

		b	b	d	d	c
				i		
	$V(i, j)$	1	2	3	4	5
	1	B	B	D		
	2					
	3					
	4					
	5					
j						

1. Schritt: Füllen der $V(i, 1)$ -Einträge

Beispiel für den CYK-Algorithmus

Seien $w = bbddc$ und $G = (\{S, A, B, C, D, E\}, \{b, c, d\}, P, S)$ mit
 $P = \{S \rightarrow AC, A \rightarrow BE, A \rightarrow BD, E \rightarrow AD, C \rightarrow c, B \rightarrow b, D \rightarrow d\}$.

		b	b	d	d	c
		$\xrightarrow{i}$				
	$V(i,j)$	1	2	3	4	5
$j \downarrow$	1	B	B	D	D	
	2					
	3					
	4					
	5					

1. Schritt: Füllen der $V(i, 1)$ -Einträge

Beispiel für den CYK-Algorithmus

Seien $w = bbddc$ und $G = (\{S, A, B, C, D, E\}, \{b, c, d\}, P, S)$ mit
 $P = \{S \rightarrow AC, A \rightarrow BE, A \rightarrow BD, E \rightarrow AD, \textcolor{red}{C} \rightarrow \textcolor{red}{c}, B \rightarrow b, D \rightarrow d\}$.

		b	b	d	d	c
				i		
	$V(i, j)$	1	2	3	4	5
	1	B	B	D	D	C
	2					
	3					
	4					
	5					

j ↓

1. Schritt: Füllen der $V(i, 1)$ -Einträge

Beispiel für den CYK-Algorithmus

Seien $w = bbd dc$ und $G = (\{S, A, B, C, D, E\}, \{b, c, d\}, P, S)$ mit
 $P = \{S \rightarrow AC, A \rightarrow BE, A \rightarrow BD, E \rightarrow AD, C \rightarrow c, B \rightarrow b, D \rightarrow d\}$.

		b	b	d	d	c
		$\xrightarrow{i}$				
	$V(i,j)$	1	2	3	4	5
j $\downarrow$	1	B	B	D	D	C
	2					
	3					
	4					
	5					

2. Schritt:

Beispiel für den CYK-Algorithmus

Seien $w = bbddc$ und $G = (\{S, A, B, C, D, E\}, \{b, c, d\}, P, S)$ mit $P = \{S \rightarrow AC, A \rightarrow BE, A \rightarrow BD, E \rightarrow AD, C \rightarrow c, B \rightarrow b, D \rightarrow d\}$.

		b	b	d	d	c
		$\xrightarrow{i}$				
	$V(i,j)$	1	2	3	4	5
j	1	B	B	D	D	C
	2					
	3					
	4					
	5					

2. Schritt: $j = 2, i = 1, k = 1: V(1,2) := V(1,2) \cup \left\{ A \mid \begin{array}{l} A \rightarrow BC, \\ B \in V(1,1), \\ C \in V(2,1) \end{array} \right\}$

Beispiel für den CYK-Algorithmus

Seien $w = bbddc$ und $G = (\{S, A, B, C, D, E\}, \{b, c, d\}, P, S)$ mit $P = \{S \rightarrow AC, A \rightarrow BE, A \rightarrow BD, E \rightarrow AD, C \rightarrow c, B \rightarrow b, D \rightarrow d\}$.

		b	b	d	d	c
				i		
	$V(i, j)$	1	2	3	4	5
	1	B	B	D	D	C
	2		A			
	3					
	4					
	5					
j						

2. Schritt: $j = 2, i = 2, k = 1: V(2, 2) := V(2, 2) \cup \left\{ A \mid \begin{array}{l} A \rightarrow BC, \\ B \in V(2, 1), \\ C \in V(3, 1) \end{array} \right\}$

Beispiel für den CYK-Algorithmus

Seien $w = bbddc$ und $G = (\{S, A, B, C, D, E\}, \{b, c, d\}, P, S)$ mit $P = \{S \rightarrow AC, A \rightarrow BE, A \rightarrow BD, E \rightarrow AD, C \rightarrow c, B \rightarrow b, D \rightarrow d\}$.

		b	b	d	d	c
				i		
	$V(i, j)$	1	2	3	4	5
	1	B	B	D	D	C
	2		A			
	3					
	4					
	5					
j						

2. Schritt: $j = 2, i = 3, k = 1: V(3, 2) := V(3, 2) \cup \left\{ A \mid \begin{array}{l} A \rightarrow BC, \\ B \in V(3, 1), \\ C \in V(4, 1) \end{array} \right\}$

Beispiel für den CYK-Algorithmus

Seien $w = bbddc$ und $G = (\{S, A, B, C, D, E\}, \{b, c, d\}, P, S)$ mit $P = \{S \rightarrow AC, A \rightarrow BE, A \rightarrow BD, E \rightarrow AD, C \rightarrow c, B \rightarrow b, D \rightarrow d\}$.

		b	b	d	d	c
				i		
	$V(i, j)$	1	2	3	4	5
	1	B	B	D	D	C
	2		A			
	3					
	4					
	5					
j						

2. Schritt: $j = 2, i = 4, k = 1: V(4, 2) := V(4, 2) \cup \left\{ A \mid \begin{array}{l} A \rightarrow BC, \\ B \in V(4, 1), \\ C \in V(5, 1) \end{array} \right\}$

Beispiel für den CYK-Algorithmus

Seien $w = bbddc$ und $G = (\{S, A, B, C, D, E\}, \{b, c, d\}, P, S)$ mit
 $P = \{S \rightarrow AC, A \rightarrow BE, A \rightarrow BD, E \rightarrow AD, C \rightarrow c, B \rightarrow b, D \rightarrow d\}$.

		b	b	d	d	c
		$\xrightarrow{i}$				
	$V(i,j)$	1	2	3	4	5
j	1	B	B	D	D	C
	2		A			
	3					
	4					
	5					

2. Schritt: $j = 3, i = 1, k = 1: V(1,3) := V(1,3) \cup \left\{ A \mid \begin{array}{l} A \rightarrow BC, \\ B \in V(1,1), \\ C \in V(2,2) \end{array} \right\}$

Beispiel für den CYK-Algorithmus

Seien $w = bbddc$ und $G = (\{S, A, B, C, D, E\}, \{b, c, d\}, P, S)$ mit
 $P = \{S \rightarrow AC, A \rightarrow BE, A \rightarrow BD, E \rightarrow AD, C \rightarrow c, B \rightarrow b, D \rightarrow d\}$.

		b	b	d	d	c
				i		
		$\longrightarrow$				
$V(i, j)$		1	2	3	4	5
j	1	B	B	D	D	C
	2		A			
	3					
	4					
	5					

2. Schritt: $j = 3, i = 1, k = 2$: $V(1, 3) := V(1, 3) \cup \left\{ A \mid \begin{array}{l} A \rightarrow BC, \\ B \in V(1, 2), \\ C \in V(3, 1) \end{array} \right\}$

Beispiel für den CYK-Algorithmus

Seien $w = bbddc$ und $G = (\{S, A, B, C, D, E\}, \{b, c, d\}, P, S)$ mit $P = \{S \rightarrow AC, A \rightarrow BE, A \rightarrow BD, E \rightarrow AD, C \rightarrow c, B \rightarrow b, D \rightarrow d\}$.

		b	b	d	d	c
				i		
	$V(i, j)$	1	2	3	4	5
	1	B	B	D	D	C
	2		A			
	3					
	4					
	5					
j						

2. Schritt: $j = 3, i = 2, k = 1: V(2, 3) := V(2, 3) \cup \left\{ A \mid \begin{array}{l} A \rightarrow BC, \\ B \in V(2, 1), \\ C \in V(3, 2) \end{array} \right\}$

Beispiel für den CYK-Algorithmus

Seien $w = bbddc$ und $G = (\{S, A, B, C, D, E\}, \{b, c, d\}, P, S)$ mit
 $P = \{S \rightarrow AC, A \rightarrow BE, A \rightarrow BD, E \rightarrow AD, C \rightarrow c, B \rightarrow b, D \rightarrow d\}$.

		b	b	d	d	c
				i		
	$V(i, j)$	1	2	3	4	5
	1	B	B	D	D	C
	2		A			
	3		E			
	4					
	5					
j						

2. Schritt: $j = 3, i = 2, k = 2: V(2, 3) := V(2, 3) \cup \left\{ A \mid \begin{array}{l} A \rightarrow BC, \\ B \in V(2, 2), \\ C \in V(4, 1) \end{array} \right\}$

Beispiel für den CYK-Algorithmus

Seien $w = bbddc$ und $G = (\{S, A, B, C, D, E\}, \{b, c, d\}, P, S)$ mit $P = \{S \rightarrow AC, A \rightarrow BE, A \rightarrow BD, E \rightarrow AD, C \rightarrow c, B \rightarrow b, D \rightarrow d\}$.

		b	b	d	d	c
				i		
	$V(i, j)$	1	2	3	4	5
	1	B	B	D	D	C
	2		A			
	3		E			
	4					
	5					
j						

2. Schritt: $j = 3, i = 3, k = 1: V(3, 3) := V(3, 3) \cup \left\{ A \mid \begin{array}{l} A \rightarrow BC, \\ B \in V(3, 1), \\ C \in V(4, 2) \end{array} \right\}$

Beispiel für den CYK-Algorithmus

Seien $w = bbddc$ und $G = (\{S, A, B, C, D, E\}, \{b, c, d\}, P, S)$ mit $P = \{S \rightarrow AC, A \rightarrow BE, A \rightarrow BD, E \rightarrow AD, C \rightarrow c, B \rightarrow b, D \rightarrow d\}$.

		b	b	d	d	c
				i		
	$V(i,j)$	1	2	3	4	5
	1	B	B	D	D	C
	2		A			
	3		E			
	4					
	5					

2. Schritt: $j = 3, i = 3, k = 2: V(3,3) := V(3,3) \cup \left\{ A \mid \begin{array}{l} A \rightarrow BC, \\ B \in V(3,2), \\ C \in V(5,1) \end{array} \right\}$

Beispiel für den CYK-Algorithmus

Seien $w = bbd dc$ und $G = (\{S, A, B, C, D, E\}, \{b, c, d\}, P, S)$ mit $P = \{S \rightarrow AC, A \rightarrow BE, A \rightarrow BD, E \rightarrow AD, C \rightarrow c, B \rightarrow b, D \rightarrow d\}$.

		b	b	d	d	c
				i		
	$V(i, j)$	1	2	3	4	5
	1	B	B	D	D	C
	2		A			
	3		E			
	4	A				
	5					

2. Schritt: $j = 4, i = 1, k = 1: V(1, 4) := V(1, 4) \cup \left\{ A \mid \begin{array}{l} A \rightarrow BC, \\ B \in V(1, 1), \\ C \in V(2, 3) \end{array} \right\}$

Beispiel für den CYK-Algorithmus

Seien $w = b b d d c$ und $G = (\{S, A, B, C, D, E\}, \{b, c, d\}, P, S)$ mit
 $P = \{S \rightarrow AC, A \rightarrow BE, A \rightarrow BD, E \rightarrow AD, C \rightarrow c, B \rightarrow b, D \rightarrow d\}$.

		b	b	d	d	c
		$\xrightarrow{i}$				
	$V(i, j)$	1	2	3	4	5
$j \downarrow$	1	B	B	D	D	C
	2		A			
	3		E			
	4	A				
	5					

2. Schritt: $j = 4, i = 1, k = 2: V(1, 4) := V(1, 4) \cup \left\{ A \mid \begin{array}{l} A \rightarrow BC, \\ B \in V(1, 2), \\ C \in V(3, 2) \end{array} \right\}$

Beispiel für den CYK-Algorithmus

Seien $w = b b d d c$ und $G = (\{S, A, B, C, D, E\}, \{b, c, d\}, P, S)$ mit
 $P = \{S \rightarrow AC, A \rightarrow BE, A \rightarrow BD, E \rightarrow AD, C \rightarrow c, B \rightarrow b, D \rightarrow d\}$.

		b	b	d	d	c
		$\xrightarrow{i}$				
	$V(i,j)$	1	2	3	4	5
$j \downarrow$	1	B	B	D	D	C
	2		A			
	3		E			
	4	A				
	5					

2. Schritt: $j = 4, i = 1, k = 3$: $V(1, 4) := V(1, 4) \cup \left\{ A \mid \begin{array}{l} A \rightarrow BC, \\ B \in V(1, 3), \\ C \in V(4, 1) \end{array} \right\}$

Beispiel für den CYK-Algorithmus

Seien $w = b b d d c$ und $G = (\{S, A, B, C, D, E\}, \{b, c, d\}, P, S)$ mit $P = \{S \rightarrow AC, A \rightarrow BE, A \rightarrow BD, E \rightarrow AD, C \rightarrow c, B \rightarrow b, D \rightarrow d\}$.

		b	b	d	d	c
				i		
	$V(i, j)$	1	2	3	4	5
	1	B	B	D	D	C
	2		A			
	3		E			
	4	A				
	5					

2. Schritt: $j = 4, i = 2, k = 1: V(2, 4) := V(2, 4) \cup \left\{ A \mid \begin{array}{l} A \rightarrow BC, \\ B \in V(2, 1), \\ C \in V(3, 3) \end{array} \right\}$

Beispiel für den CYK-Algorithmus

Seien $w = b b d d c$ und $G = (\{S, A, B, C, D, E\}, \{b, c, d\}, P, S)$ mit $P = \{S \rightarrow AC, A \rightarrow BE, A \rightarrow BD, E \rightarrow AD, C \rightarrow c, B \rightarrow b, D \rightarrow d\}$.

		b	b	d	d	c
				i		
	$V(i, j)$	1	2	3	4	5
	1	B	B	D	D	C
	2		A			
	3		E			
	4	A				
	5					

2. Schritt: $j = 4, i = 2, k = 2: V(2, 4) := V(2, 4) \cup \left\{ A \mid \begin{array}{l} A \rightarrow BC, \\ B \in V(2, 2), \\ C \in V(4, 2) \end{array} \right\}$

Beispiel für den CYK-Algorithmus

Seien $w = bbd dc$ und $G = (\{S, A, B, C, D, E\}, \{b, c, d\}, P, S)$ mit $P = \{S \rightarrow AC, A \rightarrow BE, A \rightarrow BD, E \rightarrow AD, C \rightarrow c, B \rightarrow b, D \rightarrow d\}$.

		b	b	d	d	c
				i		
	$V(i, j)$	1	2	3	4	5
	1	B	B	D	D	C
	2		A			
	3		E			
	4	A				
	5					

2. Schritt: $j = 4, i = 2, k = 3: V(2, 4) := V(2, 4) \cup \left\{ A \mid \begin{array}{l} A \rightarrow BC, \\ B \in V(2, 3), \\ C \in V(5, 1) \end{array} \right\}$

Beispiel für den CYK-Algorithmus

Seien $w = bbd dc$ und $G = (\{S, A, B, C, D, E\}, \{b, c, d\}, P, S)$ mit $P = \{S \rightarrow AC, A \rightarrow BE, A \rightarrow BD, E \rightarrow AD, C \rightarrow c, B \rightarrow b, D \rightarrow d\}$.

		b	b	d	d	c
		$\xrightarrow{i}$				
	$V(i, j)$	1	2	3	4	5
j	1	B	B	D	D	C
	2		A			
	3		E			
	4	A				
	5					

2. Schritt: $j = 5, i = 1, k = 1: V(1, 5) := V(1, 5) \cup \left\{ A \mid \begin{array}{l} A \rightarrow BC, \\ B \in V(1, 1), \\ C \in V(2, 4) \end{array} \right\}$

Beispiel für den CYK-Algorithmus

Seien $w = b b d d c$ und $G = (\{S, A, B, C, D, E\}, \{b, c, d\}, P, S)$ mit $P = \{S \rightarrow AC, A \rightarrow BE, A \rightarrow BD, E \rightarrow AD, C \rightarrow c, B \rightarrow b, D \rightarrow d\}$.

		b	b	d	d	c
		$\xrightarrow{i}$				
	$V(i, j)$	1	2	3	4	5
$j \downarrow$	1	B	B	D	D	C
	2		A			
	3		E			
	4	A				
	5					

2. Schritt: $j = 5, i = 1, k = 2: V(1, 5) := V(1, 5) \cup \left\{ A \mid \begin{array}{l} A \rightarrow BC, \\ B \in V(1, 2), \\ C \in V(3, 3) \end{array} \right\}$

Beispiel für den CYK-Algorithmus

Seien $w = bbd dc$ und $G = (\{S, A, B, C, D, E\}, \{b, c, d\}, P, S)$ mit $P = \{S \rightarrow AC, A \rightarrow BE, A \rightarrow BD, E \rightarrow AD, C \rightarrow c, B \rightarrow b, D \rightarrow d\}$.

		b	b	d	d	c
		$\xrightarrow{i}$				
	$V(i, j)$	1	2	3	4	5
$j \downarrow$	1	B	B	D	D	C
	2		A			
	3		E			
	4	A				
	5					

2. Schritt: $j = 5, i = 1, k = 3: V(1, 5) := V(1, 5) \cup \left\{ A \mid \begin{array}{l} A \rightarrow BC, \\ B \in V(1, 3), \\ C \in V(4, 2) \end{array} \right\}$

Beispiel für den CYK-Algorithmus

Seien $w = bbd dc$ und $G = (\{S, A, B, C, D, E\}, \{b, c, d\}, P, S)$ mit
 $P = \{S \rightarrow AC, A \rightarrow BE, A \rightarrow BD, E \rightarrow AD, C \rightarrow c, B \rightarrow b, D \rightarrow d\}$.

		b	b	d	d	c
		$\xrightarrow{i}$				
	$V(i,j)$	1	2	3	4	5
$j \downarrow$	1	B	B	D	D	C
	2		A			
	3		E			
	4	A				
	5	S				

2. Schritt: $j = 5, i = 1, k = 4: V(1, 5) := V(1, 5) \cup \left\{ A \mid \begin{array}{l} A \rightarrow BC, \\ B \in V(1, 4), \\ C \in V(5, 1) \end{array} \right\}$

Beispiel für den CYK-Algorithmus

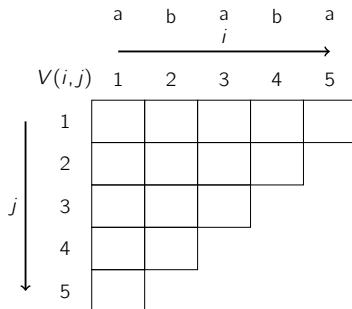
Seien $w = bbddc$ und $G = (\{S, A, B, C, D, E\}, \{b, c, d\}, P, S)$ mit
 $P = \{S \rightarrow AC, A \rightarrow BE, A \rightarrow BD, E \rightarrow AD, C \rightarrow c, B \rightarrow b, D \rightarrow d\}$.

		b	b	d	d	c
		$\xrightarrow{i}$				
	$V(i,j)$	1	2	3	4	5
$j \downarrow$	1	B	B	D	D	C
	2		A			
	3		E			
	4	A				
	5	S				

3. Schritt: Da $S \in V(1, 5)$, gilt $w \in L(G)$.

Weiteres Beispiel für den CYK-Algorithmus

Seien $w = ababa$ und $G = (\{S, T, U\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow TU, S \rightarrow UT, T \rightarrow TT, T \rightarrow TU, T \rightarrow a, U \rightarrow UU, U \rightarrow b\}$.



1. Schritt: Füllen der $V(i, 1)$ -Einträge

Weiteres Beispiel für den CYK-Algorithmus

Seien $w = ababa$ und $G = (\{S, T, U\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow TU, S \rightarrow UT, T \rightarrow TT, T \rightarrow TU, T \rightarrow a, U \rightarrow UU, U \rightarrow b\}$.

		a	b	a	b	a
				i		
	$V(i, j)$	1	2	3	4	5
	1	T				
	2					
	3					
	4					
	5					

j ↓

1. Schritt: Füllen der $V(i, 1)$ -Einträge

Weiteres Beispiel für den CYK-Algorithmus

Seien $w = ababa$ und $G = (\{S, T, U\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow TU, S \rightarrow UT, T \rightarrow TT, T \rightarrow TU, T \rightarrow a, U \rightarrow UU, U \rightarrow b\}$.

		a	b	a	b	a
				i		
	$V(i, j)$	1	2	3	4	5
	1	T	U			
	2					
	3					
	4					
	5					

j ↓

1. Schritt: Füllen der $V(i, 1)$ -Einträge

Weiteres Beispiel für den CYK-Algorithmus

Seien $w = ababa$ und $G = (\{S, T, U\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow TU, S \rightarrow UT, T \rightarrow TT, T \rightarrow TU, T \rightarrow a, U \rightarrow UU, U \rightarrow b\}$.

		a	b	a	b	a
				i		
$V(i, j)$		1	2	3	4	5
j	1	T	U	T		
	2					
	3					
	4					
	5					

1. Schritt: Füllen der $V(i, 1)$ -Einträge

Weiteres Beispiel für den CYK-Algorithmus

Seien $w = ababa$ und $G = (\{S, T, U\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow TU, S \rightarrow UT, T \rightarrow TT, T \rightarrow TU, T \rightarrow a, U \rightarrow UU, U \rightarrow b\}$.

		a	b	a	b	a
		$\xrightarrow{i}$				
	$V(i, j)$	1	2	3	4	5
$j \downarrow$	1	T	U	T	U	
	2					
	3					
	4					
	5					

1. Schritt: Füllen der $V(i, 1)$ -Einträge

Weiteres Beispiel für den CYK-Algorithmus

Seien $w = ababa$ und $G = (\{S, T, U\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow TU, S \rightarrow UT, T \rightarrow TT, T \rightarrow TU, T \rightarrow a, U \rightarrow UU, U \rightarrow b\}$.

		a	b	a	b	a
				i		
	$V(i, j)$	1	2	3	4	5
	1	T	U	T	U	T
	2					
	3					
	4					
	5					

j

1. Schritt: Füllen der $V(i, 1)$ -Einträge

Weiteres Beispiel für den CYK-Algorithmus

Seien $w = ababa$ und $G = (\{S, T, U\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow TU, S \rightarrow UT, T \rightarrow TT, T \rightarrow TU, T \rightarrow a, U \rightarrow UU, U \rightarrow b\}$.

		a	b	a	b	a
		$\xrightarrow{i}$				
$V(i,j)$		1	2	3	4	5
$j \downarrow$	1	T	U	T	U	T
	2					
	3					
	4					
	5					

2. Schritt:

Weiteres Beispiel für den CYK-Algorithmus

Seien $w = ababa$ und $G = (\{S, T, U\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow TU, S \rightarrow UT, T \rightarrow TT, T \rightarrow TU, T \rightarrow a, U \rightarrow UU, U \rightarrow b\}.$

		a	b	a	b	a
		$\xrightarrow{i}$				
$V(i,j)$		1	2	3	4	5
$j \downarrow$	1	T	U	T	U	T
	2	S, T				
	3					
	4					
	5					

2. Schritt: $j = 2, i = 1, k = 1: V(1, 2) := V(1, 2) \cup \left\{ A \mid \begin{array}{l} A \rightarrow BC, \\ B \in V(1, 1), \\ C \in V(2, 1) \end{array} \right\}$

Weiteres Beispiel für den CYK-Algorithmus

Seien $w = ababa$ und $G = (\{S, T, U\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow TU, S \rightarrow UT, T \rightarrow TT, T \rightarrow TU, T \rightarrow a, U \rightarrow UU, U \rightarrow b\}$.

		a	b	a	b	a
		$\xrightarrow{i}$				
$V(i, j)$		1	2	3	4	5
$j \downarrow$	1	T	U	T	U	T
	2	S, T	S			
	3					
	4					
	5					

2. Schritt: $j = 2, i = 2, k = 1: V(2, 2) := V(2, 2) \cup \left\{ A \mid \begin{array}{l} A \rightarrow BC, \\ B \in V(2, 1), \\ C \in V(3, 1) \end{array} \right\}$

Weiteres Beispiel für den CYK-Algorithmus

Seien $w = ababa$ und $G = (\{S, T, U\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow TU, S \rightarrow UT, T \rightarrow TT, T \rightarrow TU, T \rightarrow a, U \rightarrow UU, U \rightarrow b\}$.

		a	b	a	b	a
				i		
	$V(i, j)$	1	2	3	4	5
	1	T	U	T	U	T
	2	S, T	S	S, T		
	3					
	4					
	5					

2. Schritt: $j = 2, i = 3, k = 1: V(3, 2) := V(3, 2) \cup \left\{ A \mid \begin{array}{l} A \rightarrow BC, \\ B \in V(3, 1), \\ C \in V(4, 1) \end{array} \right\}$

Weiteres Beispiel für den CYK-Algorithmus

Seien $w = ababa$ und $G = (\{S, T, U\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow TU, S \rightarrow UT, T \rightarrow TT, T \rightarrow TU, T \rightarrow a, U \rightarrow UU, U \rightarrow b\}$.

		a	b	a	b	a
		$\xrightarrow{i}$				
	$V(i, j)$	1	2	3	4	5
j	1	T	U	T	U	T
	2	S, T	S	S, T	S	
	3					
	4					
	5					

2. Schritt: $j = 2, i = 4, k = 1: V(4, 2) := V(4, 2) \cup \left\{ A \mid \begin{array}{l} A \rightarrow BC, \\ B \in V(4, 1), \\ C \in V(5, 1) \end{array} \right\}$

Weiteres Beispiel für den CYK-Algorithmus

Seien $w = ababa$ und $G = (\{S, T, U\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow TU, S \rightarrow UT, T \rightarrow TT, T \rightarrow TU, T \rightarrow a, U \rightarrow UU, U \rightarrow b\}$.

		a	b	a	b	a
		$\xrightarrow{i}$				
	$V(i, j)$	1	2	3	4	5
j	1	T	U	T	U	T
	2	S, T	S	S, T	S	
	3					
	4					
	5					

2. Schritt: $j = 3, i = 1, k = 1$: $V(1, 3) := V(1, 3) \cup \left\{ A \mid \begin{array}{l} A \rightarrow BC, \\ B \in V(1, 1), \\ C \in V(2, 2) \end{array} \right\}$

Weiteres Beispiel für den CYK-Algorithmus

Seien $w = ababa$ und $G = (\{S, T, U\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow TU, S \rightarrow UT, T \rightarrow TT, T \rightarrow TU, T \rightarrow a, U \rightarrow UU, U \rightarrow b\}$.

		a	b	a	b	a
		$\xrightarrow{i}$				
$V(i, j)$		1	2	3	4	5
1		T	U	T	U	T
2		S, T	S	S, T	S	
3		T				
4						
5						

2. Schritt: $j = 3, i = 1, k = 2: V(1, 3) := V(1, 3) \cup \left\{ A \mid \begin{array}{l} A \rightarrow BC, \\ B \in V(1, 2), \\ C \in V(3, 1) \end{array} \right\}$

Weiteres Beispiel für den CYK-Algorithmus

Seien $w = ababa$ und $G = (\{S, T, U\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow TU, S \rightarrow UT, T \rightarrow TT, T \rightarrow TU, T \rightarrow a, U \rightarrow UU, U \rightarrow b\}$.

		a	b	a	b	a
		$\xrightarrow{i}$				
$V(i, j)$		1	2	3	4	5
$j \downarrow$	1	T	U	T	U	T
	2	S, T	S	S, T	S	
	3	T	S			
	4					
	5					

2. Schritt: $j = 3, i = 2, k = 1: V(2, 3) := V(2, 3) \cup \left\{ A \mid \begin{array}{l} A \rightarrow BC, \\ B \in V(2, 1), \\ C \in V(3, 2) \end{array} \right\}$

Weiteres Beispiel für den CYK-Algorithmus

Seien $w = ababa$ und $G = (\{S, T, U\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow TU, S \rightarrow UT, T \rightarrow TT, T \rightarrow TU, T \rightarrow a, U \rightarrow UU, U \rightarrow b\}$.

		a	b	a	b	a
		$\xrightarrow{i}$				
$V(i, j)$		1	2	3	4	5
$j \downarrow$	1	T	U	T	U	T
	2	S, T	S	S, T	S	
	3	T	S			
	4					
	5					

2. Schritt: $j = 3, i = 2, k = 2$: $V(2, 3) := V(2, 3) \cup \left\{ A \mid \begin{array}{l} A \rightarrow BC, \\ B \in V(2, 2), \\ C \in V(4, 1) \end{array} \right\}$

Weiteres Beispiel für den CYK-Algorithmus

Seien $w = ababa$ und $G = (\{S, T, U\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow TU, S \rightarrow UT, T \rightarrow TT, T \rightarrow TU, T \rightarrow a, U \rightarrow UU, U \rightarrow b\}$.

		a	b	a	b	a
		$\xrightarrow{i}$				
$V(i,j)$		1	2	3	4	5
$j \downarrow$	1	T	U	T	U	T
	2	S, T	S	S, T	S	
	3	T	S			
	4					
	5					

2. Schritt: $j = 3, i = 3, k = 1: V(3, 3) := V(3, 3) \cup \left\{ A \mid \begin{array}{l} A \rightarrow BC, \\ B \in V(3, 1), \\ C \in V(4, 2) \end{array} \right\}$

Weiteres Beispiel für den CYK-Algorithmus

Seien $w = ababa$ und $G = (\{S, T, U\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow TU, S \rightarrow UT, T \rightarrow TT, T \rightarrow TU, T \rightarrow a, U \rightarrow UU, U \rightarrow b\}$.

		a	b	a	b	a
		$\xrightarrow{i}$				
	$V(i, j)$	1	2	3	4	5
j	1	T	U	T	U	T
	2	S, T	S	S, T	S	
	3	T	S	T		
	4					
	5					

2. Schritt: $j = 3, i = 3, k = 2: V(3, 3) := V(3, 3) \cup \left\{ A \mid \begin{array}{l} A \rightarrow BC, \\ B \in V(3, 2), \\ C \in V(5, 1) \end{array} \right\}$

Weiteres Beispiel für den CYK-Algorithmus

Seien $w = ababa$ und $G = (\{S, T, U\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow TU, S \rightarrow UT, T \rightarrow TT, T \rightarrow TU, T \rightarrow a, U \rightarrow UU, U \rightarrow b\}$.

		a	b	a	b	a
		$\xrightarrow{i}$				
	$V(i, j)$	1	2	3	4	5
j	1	T	U	T	U	T
	2	S, T	S	S, T	S	
	3	T	S	T		
	4					
	5					

2. Schritt: $j = 4, i = 1, k = 1: V(1, 4) := V(1, 4) \cup \left\{ A \mid \begin{array}{l} A \rightarrow BC, \\ B \in V(1, 1), \\ C \in V(2, 3) \end{array} \right\}$

Weiteres Beispiel für den CYK-Algorithmus

Seien $w = ababa$ und $G = (\{S, T, U\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow TU, S \rightarrow UT, T \rightarrow TT, T \rightarrow TU, T \rightarrow a, U \rightarrow UU, U \rightarrow b\}$.

		a	b	a	b	a
		$\xrightarrow{\quad i \quad}$				
$V(i, j)$		1	2	3	4	5
$j \downarrow$	1	T	U	T	U	T
	2	S, T	S	S, T	S	
	3	T	S	T		
	4	T				
	5					

2. Schritt: $j = 4, i = 1, k = 2$: $V(1, 4) := V(1, 4) \cup \left\{ A \mid \begin{array}{l} A \rightarrow BC, \\ B \in V(1, 2), \\ C \in V(3, 2) \end{array} \right\}$

Weiteres Beispiel für den CYK-Algorithmus

Seien $w = ababa$ und $G = (\{S, T, U\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow TU, S \rightarrow UT, T \rightarrow TT, T \rightarrow TU, T \rightarrow a, U \rightarrow UU, U \rightarrow b\}.$

		a	b	a	b	a
		$\xrightarrow{i}$				
	$V(i, j)$	1	2	3	4	5
$j \downarrow$	1	T	U	T	U	T
	2	S, T	S	S, T	S	
	3	T	S	T		
	4	S, T				
	5					

2. Schritt: $j = 4, i = 1, k = 3: V(1, 4) := V(1, 4) \cup \left\{ A \mid \begin{array}{l} A \rightarrow BC, \\ B \in V(1, 3), \\ C \in V(4, 1) \end{array} \right\}$

Weiteres Beispiel für den CYK-Algorithmus

Seien $w = ababa$ und $G = (\{S, T, U\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow TU, S \rightarrow UT, T \rightarrow TT, T \rightarrow TU, T \rightarrow a, U \rightarrow UU, U \rightarrow b\}$.

		a	b	a	b	a
		$\xrightarrow{i}$				
$V(i, j)$		1	2	3	4	5
$j \downarrow$	1	T	U	T	U	T
	2	S, T	S	S, T	S	
	3	T	S	T		
	4	S, T	S			
	5					

2. Schritt: $j = 4, i = 2, k = 1: V(2, 4) := V(2, 4) \cup \left\{ A \mid \begin{array}{l} A \rightarrow BC, \\ B \in V(2, 1), \\ C \in V(3, 3) \end{array} \right\}$

Weiteres Beispiel für den CYK-Algorithmus

Seien $w = ababa$ und $G = (\{S, T, U\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow TU, S \rightarrow UT, T \rightarrow TT, T \rightarrow TU, T \rightarrow a, U \rightarrow UU, U \rightarrow b\}$.

		a	b	a	b	a
		$\xrightarrow{i}$				
$V(i, j)$		1	2	3	4	5
$j \downarrow$	1	T	U	T	U	T
	2	S, T	S	S, T	S	
	3	T	S	T		
	4	S, T	S			
	5					

2. Schritt: $j = 4, i = 2, k = 2$: $V(2, 4) := V(2, 4) \cup \left\{ A \mid \begin{array}{l} A \rightarrow BC, \\ B \in V(2, 2), \\ C \in V(4, 2) \end{array} \right\}$

Weiteres Beispiel für den CYK-Algorithmus

Seien $w = ababa$ und $G = (\{S, T, U\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow TU, S \rightarrow UT, T \rightarrow TT, T \rightarrow TU, T \rightarrow a, U \rightarrow UU, U \rightarrow b\}$.

		a	b	a	b	a
		$\xrightarrow{i}$				
$V(i, j)$		1	2	3	4	5
$j \downarrow$	1	T	U	T	U	T
	2	S, T	S	S, T	S	
	3	T	S	T		
	4	S, T	S			
	5					

2. Schritt: $j = 4, i = 2, k = 3$: $V(2, 4) := V(2, 4) \cup \left\{ A \mid \begin{array}{l} A \rightarrow BC, \\ B \in V(2, 3), \\ C \in V(5, 1) \end{array} \right\}$

Weiteres Beispiel für den CYK-Algorithmus

Seien $w = ababa$ und $G = (\{S, T, U\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow TU, S \rightarrow UT, T \rightarrow TT, T \rightarrow TU, T \rightarrow a, U \rightarrow UU, U \rightarrow b\}$.

		a	b	a	b	a
		$\xrightarrow{i}$				
$V(i, j)$		1	2	3	4	5
$j \downarrow$	1	T	U	T	U	T
	2	S, T	S	S, T	S	
	3	T	S	T		
	4	S, T	S			
	5					

2. Schritt: $j = 5, i = 1, k = 1: V(1, 5) := V(1, 5) \cup \left\{ A \mid \begin{array}{l} A \rightarrow BC, \\ B \in V(1, 1), \\ C \in V(2, 4) \end{array} \right\}$

Weiteres Beispiel für den CYK-Algorithmus

Seien $w = ababa$ und $G = (\{S, T, U\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow TU, S \rightarrow UT, T \rightarrow TT, T \rightarrow TU, T \rightarrow a, U \rightarrow UU, U \rightarrow b\}$.

		a	b	a	b	a
		$\xrightarrow{i}$				
$V(i, j)$		1	2	3	4	5
j	1	T	U	T	U	T
	2	S, T	S	S, T	S	
	3	T	S	T		
	4	S, T	S			
	5	T				

2. Schritt: $j = 5, i = 1, k = 2: V(1, 5) := V(1, 5) \cup \left\{ A \mid \begin{array}{l} A \rightarrow BC, \\ B \in V(1, 2), \\ C \in V(3, 3) \end{array} \right\}$

Weiteres Beispiel für den CYK-Algorithmus

Seien $w = ababa$ und $G = (\{S, T, U\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow TU, S \rightarrow UT, T \rightarrow TT, T \rightarrow TU, T \rightarrow a, U \rightarrow UU, U \rightarrow b\}$.

		a	b	a	b	a
		$\xrightarrow{i}$				
	$V(i, j)$	1	2	3	4	5
j	1	T	U	T	U	T
	2	S, T	S	S, T	S	
	3	T	S	T		
	4	S, T	S			
	5	T				

2. Schritt: $j = 5, i = 1, k = 3$: $V(1, 5) := V(1, 5) \cup \left\{ A \mid \begin{array}{l} A \rightarrow BC, \\ B \in V(1, 3), \\ C \in V(4, 2) \end{array} \right\}$

Weiteres Beispiel für den CYK-Algorithmus

Seien $w = ababa$ und $G = (\{S, T, U\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow TU, S \rightarrow UT, T \rightarrow TT, T \rightarrow TU, T \rightarrow a, U \rightarrow UU, U \rightarrow b\}$.

		a	b	a	b	a
		$\xrightarrow{i}$				
	$V(i, j)$	1	2	3	4	5
j	1	T	U	T	U	T
	2	S, T	S	S, T	S	
	3	T	S	T		
	4	S, T	S			
	5	T				

2. Schritt: $j = 5, i = 1, k = 4$: $V(1, 5) := V(1, 5) \cup \left\{ A \mid \begin{array}{l} A \rightarrow BC, \\ B \in V(1, 4), \\ C \in V(5, 1) \end{array} \right\}$

Weiteres Beispiel für den CYK-Algorithmus

Seien $w = ababa$ und $G = (\{S, T, U\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow TU, S \rightarrow UT, T \rightarrow TT, T \rightarrow TU, T \rightarrow a, U \rightarrow UU, U \rightarrow b\}$.

		a	b	a	b	a
		$\xrightarrow{i}$				
$V(i, j)$		1	2	3	4	5
$j \downarrow$	1	T	U	T	U	T
	2	S, T	S	S, T	S	
	3	T	S	T		
	4	S, T	S			
	5	T				

3. Schritt: Da $S \notin V(1, 5)$, gilt $w \notin L(G)$.

Algorithmus 8: CYK-Algorithmus

Eingabe: CFG $G = (V, \Sigma, P, S)$ in Chomsky-Normalform und Wort $w = a_1 \cdots a_n \in \Sigma^+$

Ausgabe: Ja, wenn $w \in L(G)$, und Nein, wenn $w \notin L(G)$

Beginn

```
für  $i = 1$  bis  $n$  tue
└  $V(i, 1) := \{A \in V \mid A \rightarrow a_i \in P\}$ 
für  $j = 2$  bis  $n$  tue
└ für  $i = 1$  bis  $n + 1 - j$  tue
    └  $V(i, j) := \emptyset;$ 
        └ für  $k = 1$  bis  $j - 1$  tue
            └  $V(i, j) := V(i, j) \cup \left\{ A \in V \mid \begin{array}{l} A \rightarrow BC \in P, \\ B \in V(i, k), \\ C \in V(i + k, j - k) \end{array} \right\}$ 
└ wenn  $S \in V(1, n)$  dann
    └ return Ja
sonst
└ return Nein
```

Theorem

Das Wortproblem für Typ 2-Grammatiken ist entscheidbar:

Es gibt einen Algorithmus, der bei Eingabe von Typ 2-Grammatik G und Wort w nach endlicher Zeit entscheidet, ob $w \in L(G)$ gilt oder nicht. Zudem entscheidet er das Wortproblem in Polynomialzeit.

Theorem

Das Wortproblem für Typ 2-Grammatiken ist entscheidbar:

Es gibt einen Algorithmus, der bei Eingabe von Typ 2-Grammatik G und Wort w nach endlicher Zeit entscheidet, ob $w \in L(G)$ gilt oder nicht. Zudem entscheidet er das Wortproblem in Polynomialzeit.

Beweis Algorithmus 8 ist eine Entscheidungsprozedur.

Theorem

Das Wortproblem für Typ 2-Grammatiken ist entscheidbar:

Es gibt einen Algorithmus, der bei Eingabe von Typ 2-Grammatik G und Wort w nach endlicher Zeit entscheidet, ob $w \in L(G)$ gilt oder nicht. Zudem entscheidet er das Wortproblem in Polynomialzeit.

Beweis Algorithmus 8 ist eine Entscheidungsprozedur.

Er besteht aus drei geschachtelte für-Schleifen. Im Inneren wird noch über alle Produktionen aus P iteriert. Die Laufzeitkomplexität kann daher mit $O(n^3 \cdot |P|)$ abgeschätzt werden. □

Tobias Lindebar hat ein Web-Tool zum Üben und Anschauen entwickelt:

www.cip.ifi.lmu.de/~lindebar/

Das Endlichkeitsproblem

Das **Endlichkeitsproblem** für Typ i -Grammatiken ist die Frage, ob für eine gegebene Typ i -Grammatik G die Ungleichheit $|L(G)| < \infty$ gilt.

Das Endlichkeitsproblem

Das **Endlichkeitsproblem** für Typ i -Grammatiken ist die Frage, ob für eine gegebene Typ i -Grammatik G die Ungleichheit $|L(G)| < \infty$ gilt.

Satz

Das Endlichkeitsproblem für kontextfreie Grammatiken ist entscheidbar.

Das Endlichkeitsproblem

Das **Endlichkeitsproblem** für Typ i -Grammatiken ist die Frage, ob für eine gegebene Typ i -Grammatik G die Ungleichheit $|L(G)| < \infty$ gilt.

Satz

Das Endlichkeitsproblem für kontextfreie Grammatiken ist entscheidbar.

Wir brauchen zuerst ein Lemma.

Entscheiden des Endlichkeitsproblems

Lemma

Sei G eine CFG in Chomsky-Normalform.

Sei n die Zahl aus dem Pumping-Lemma für kontextfreie Sprachen.

Es gilt $|L(G)| = \infty$ g.d.w. es ein Wort $z \in L(G)$ mit $n \leq |z| < 2n$ gibt.

Entscheiden des Endlichkeitsproblems

Lemma

Sei G eine CFG in Chomsky-Normalform.

Sei n die Zahl aus dem Pumping-Lemma für kontextfreie Sprachen.

Es gilt $|L(G)| = \infty$ g.d.w. es ein Wort $z \in L(G)$ mit $n \leq |z| < 2n$ gibt.

Beweis Siehe Skript (nur FSK).



Entscheiden des Endlichkeitsproblems

Satz

Das Endlichkeitsproblem für kontextfreie Grammatiken ist entscheidbar.

Entscheiden des Endlichkeitsproblems

Satz

Das Endlichkeitsproblem für kontextfreie Grammatiken ist entscheidbar.

Beweis Entscheidungsprozedur:

Entscheiden des Endlichkeitsproblems

Satz

Das Endlichkeitsproblem für kontextfreie Grammatiken ist entscheidbar.

Beweis Entscheidungsprozedur:

1. Berechne die Zahl n .

Entscheiden des Endlichkeitsproblems

Satz

Das Endlichkeitsproblem für kontextfreie Grammatiken ist entscheidbar.

Beweis Entscheidungsprozedur:

1. Berechne die Zahl n .
2. Teste mit dem CYK-Algorithmus für alle Wörter $w \in \Sigma^+$ der Länge $n \leq |w| < 2n$, ob $w \in L(G)$ gilt.

Entscheiden des Endlichkeitsproblems

Satz

Das Endlichkeitsproblem für kontextfreie Grammatiken ist entscheidbar.

Beweis Entscheidungsprozedur:

1. Berechne die Zahl n .
2. Teste mit dem CYK-Algorithmus für alle Wörter $w \in \Sigma^+$ der Länge $n \leq |w| < 2n$, ob $w \in L(G)$ gilt.
3. Wenn $w \in L(G)$ für eines der Wörter gilt, dann $|L(G)| = \infty$.

Entscheiden des Endlichkeitsproblems

Satz

Das Endlichkeitsproblem für kontextfreie Grammatiken ist entscheidbar.

Beweis Entscheidungsprozedur:

1. Berechne die Zahl n .
2. Teste mit dem CYK-Algorithmus für alle Wörter $w \in \Sigma^+$ der Länge $n \leq |w| < 2n$, ob $w \in L(G)$ gilt.
3. Wenn $w \in L(G)$ für eines der Wörter gilt, dann $|L(G)| = \infty$.
4. Sonst $|L(G)| < \infty$.



6c

Kellerautomaten

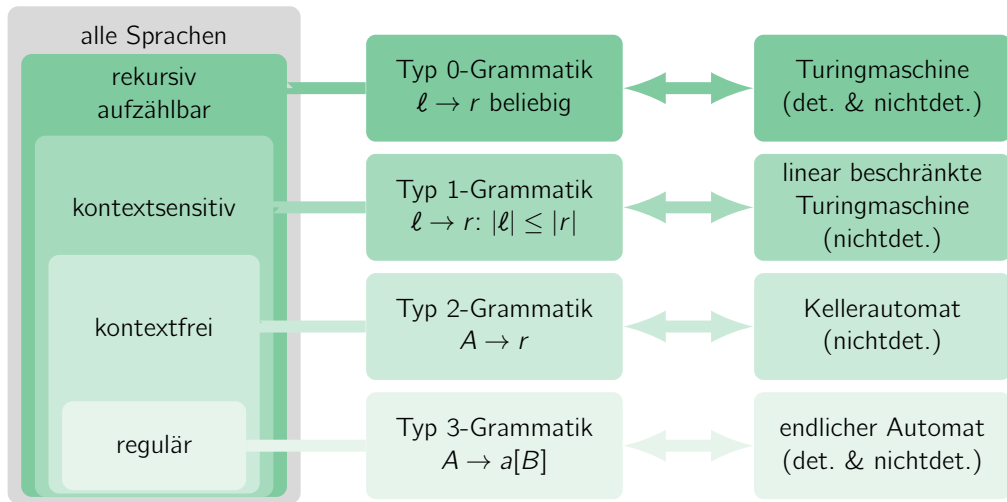
Prof. Dr. Jasmin Blanchette

Lehr- und Forschungseinheit für
Theoretische Informatik und Theorembeweisen

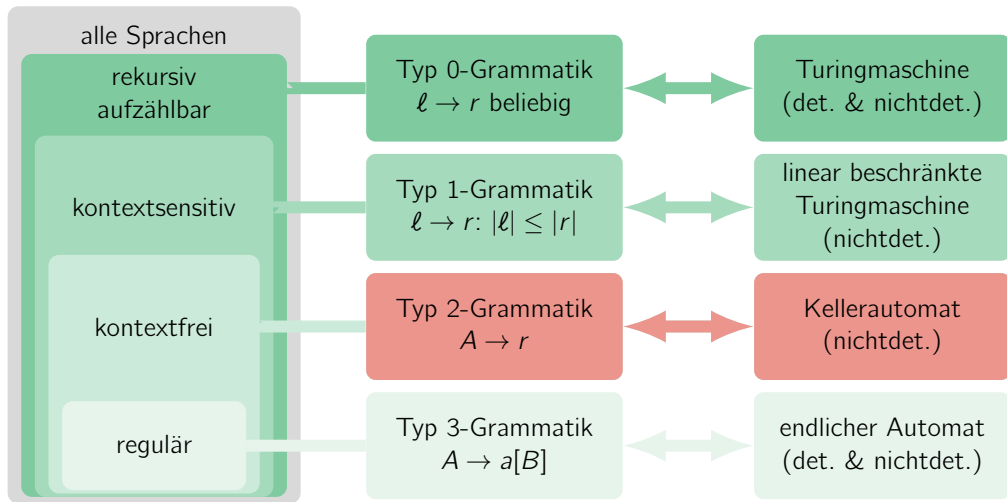
Stand: 29. Mai 2024
Basierend auf Folien von PD Dr. David Sabel



Überblick über Grammatiken und Maschinenmodelle



Überblick über Grammatiken und Maschinenmodelle



Begrenzungen der endlichen Automaten

- ▶ DFAs und NFAs haben fast **keinen Speicher**:
Der einzige Speicher dort sind die **Zustände**, daher **endlicher** Speicher.

Begrenzungen der endlichen Automaten

- ▶ DFAs und NFAs haben fast **keinen Speicher**:
Der einzige Speicher dort sind die **Zustände**, daher **endlicher** Speicher.
- ▶ Daher ist es z.B. unmöglich, $\{w\$ \overline{w} \mid w \in \{a, b, c\}^*\}$ zu erkennen:
Man müsste beim Lesen von w alle gelesenen Zeichen speichern, um sie dann beim Lesen von $\overline{w}$ (d.h. w rückwärts) zu vergleichen.

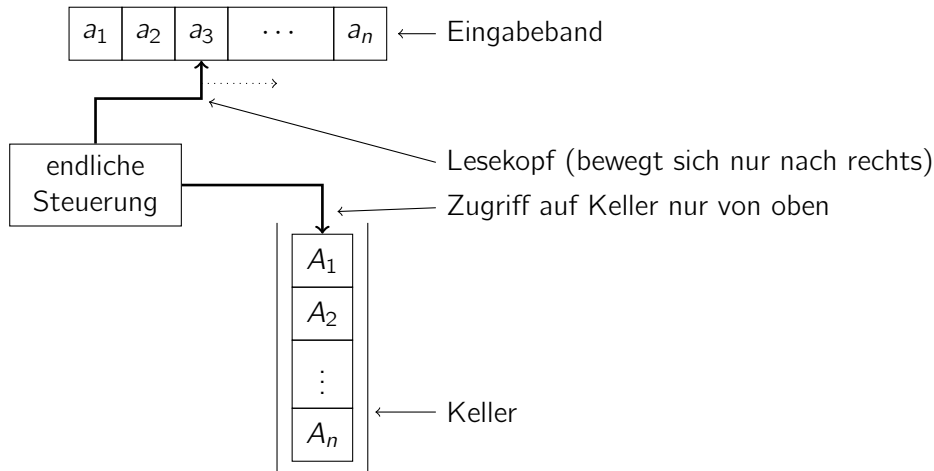
Kellerautomaten

Informelle Kurzfassung:

- ▶ Kellerautomaten fügen einen **beliebig großen** Kellerspeicher hinzu (Stack, LIFO-Speicher, last-in-first-out-Speicher).
- ▶ Der Keller ist ein Stapel, auf den **nur von oben** zugegriffen werden kann.
- ▶ Zustandsübergang:

	NFA mit ϵ -Übergängen	Kellerautomat
Eingabe	Zustand und Zeichen-oder- ϵ	Zustand, Zeichen-oder- ϵ und oberstes Symbol im Keller
Ausgabe	nächster Zustand	nächster Zustand und Sequenz von Kellersymbolen, die das erste Symbol ersetzen

Illustration eines Kellerautomaten



Definition eines Kellerautomaten

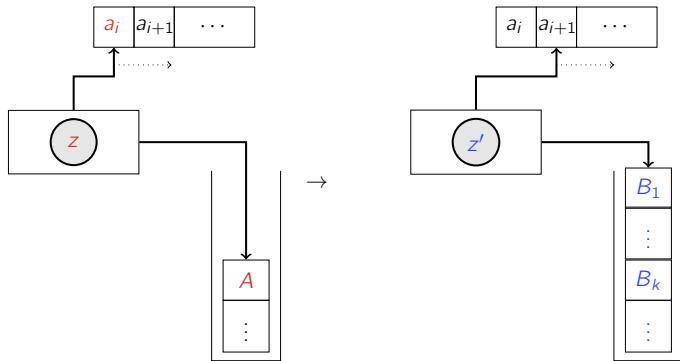
Definition

Ein (nichtdeterministischer) Kellerautomat (pushdown automaton, PDA) ist ein 6-Tupel $M = (Z, \Sigma, \Gamma, \delta, z_0, \#)$, wobei:

- ▶ Z ist eine endliche Menge von Zuständen
- ▶ Σ ist das (endliche) Eingabealphabet
- ▶ Γ ist das (endliche) Kelleralphabet
- ▶ $\delta : Z \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \rightarrow \mathcal{P}_e(Z \times \Gamma^*)$ ist die Überföhrungsfunktion
- ▶ $z_0 \in Z$ ist der Startzustand
- ▶ $\# \in \Gamma$ ist das Startsymbol im Keller.

$\mathcal{P}_e(X)$ steht für die Menge der endlichen Teilmengen von X .

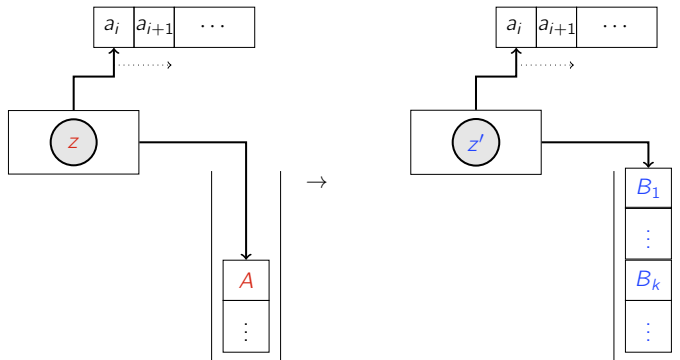
Illustration des Zustandsübergangs mit $a \in \Sigma$



$(z', B_1 \dots B_k) \in \delta(z, a_i, A)$ bedeutet:

Im Zustand z bei Eingabe a_i und A oben auf dem Keller darf der PDA in z' wechseln.
Dabei wird A durch $B_1 \dots B_k$ ($k \geq 0$) ersetzt, wobei B_1 oben liegt.

Illustration des Zustandsübergangs mit ε



$(z', B_1 \dots B_k) \in \delta(z, \varepsilon, A)$ bedeutet:

Im Zustand z bei A oben auf dem Keller darf der PDA in z' wechseln, ohne dass der Lesekopf sich bewegt.

Dabei wird A durch $B_1 \dots B_k$ ($k \geq 0$) ersetzt, wobei B_1 oben liegt.

Mit unserer Definition:

- ▶ PDAs sind nichtdeterministisch.
- ▶ PDAs erlauben ϵ -Übergänge.
- ▶ PDAs haben keine Endzustände.

Mit unserer Definition:

- ▶ PDAs sind nichtdeterministisch.
- ▶ PDAs erlauben ϵ -Übergänge.
- ▶ PDAs haben keine Endzustände.

Zusätzlich:

- ▶ PDAs akzeptieren, wenn die Eingabe verarbeitet ist und der Keller leer ist.
- ▶ Am Anfang enthält der Keller #.

- ▶ Während einer Berechnung mit dem PDA werden der aktuelle Zustand, die Resteingabe und der aktuelle Kellerinhalt buchführt.
- ▶ Diese Informationen werden durch eine PDA-Konfiguration dargestellt.

- ▶ Während einer Berechnung mit dem PDA werden der aktuelle Zustand, die Resteingabe und der aktuelle Kellerinhalt buchführt.
- ▶ Diese Informationen werden durch eine PDA-Konfiguration dargestellt.

Definition

Sei $M = (Z, \Sigma, \Gamma, \delta, z_0, \#)$ ein PDA.

Eine **Konfiguration** von M ist ein Tripel $(z, w, W) \in Z \times \Sigma^* \times \Gamma^*$, wobei:

- ▶ z ist der **aktuelle Zustand**
- ▶ w ist die **Resteingabe**
- ▶ W ist der **aktuelle Kellerinhalt**.

Übergangsrelation eines PDAs

Definition

Sei $M = (Z, \Sigma, \Gamma, \delta, z_0, \#)$ ein PDA.

Die Relation $\vdash_M \subseteq (Z \times \Sigma^* \times \Gamma^*) \times (Z \times \Sigma^* \times \Gamma^*)$ ist definiert durch

- ▶ $(z, a_1 \cdots a_n, A_1 \cdots A_m) \vdash_M (z', a_2 \cdots a_n, WA_2 \cdots A_m)$ falls $(z', W) \in \delta(z, a_1, A_1)$
- ▶ $(z, a_1 \cdots a_n, A_1 \cdots A_m) \vdash_M (z', a_1 \cdots a_n, WA_2 \cdots A_m)$ falls $(z', W) \in \delta(z, \varepsilon, A_1)$.

Übergangsrelation eines PDAs

Definition

Sei $M = (Z, \Sigma, \Gamma, \delta, z_0, \#)$ ein PDA.

Die Relation $\vdash_M \subseteq (Z \times \Sigma^* \times \Gamma^*) \times (Z \times \Sigma^* \times \Gamma^*)$ ist definiert durch

- ▶ $(z, a_1 \cdots a_n, A_1 \cdots A_m) \vdash_M (z', a_2 \cdots a_n, WA_2 \cdots A_m)$ falls $(z', W) \in \delta(z, a_1, A_1)$
- ▶ $(z, a_1 \cdots a_n, A_1 \cdots A_m) \vdash_M (z', a_1 \cdots a_n, WA_2 \cdots A_m)$ falls $(z', W) \in \delta(z, \varepsilon, A_1)$.

Weitere Notationen:

- ▶ $\vdash_M^*$ ist die reflexiv-transitive Hülle von $\vdash_M$.
- ▶ $\vdash_M^i$ ist die i -fache Anwendung von $\vdash_M$.
- ▶ Wenn M eindeutig ist, schreiben wir $\vdash$ statt $\vdash_M$.

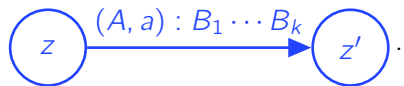
Definition

Sei $M = (Z, \Sigma, \Gamma, \delta, z_0, \#)$ ein PDA. Die von M akzeptierte Sprache ist

$$L(M) := \{w \in \Sigma^* \mid (z_0, w, \#) \vdash^* (z, \varepsilon, \varepsilon) \text{ für ein } z \in Z\}$$

Zustandsgraph eines PDA

- ▶ Die Darstellung als Zustandsgraph ist analog zu der der NFAs mit ε -Übergängen.
- ▶ Für $(z', B_1 \cdots B_k) \in \delta(z, a, A)$ (mit $a \in \Sigma \cup \{\varepsilon\}$) zeichnen wir



- ▶ Beachte, dass das Startsymbol im Keller bekannt sein muss (üblicherweise $\#$).

Beispiel für einen PDA

PDA $M = (\{z_0, z_1\}, \{a, b\}, \{B, \#\}, \delta, z_0, \#)$ mit

$$\begin{array}{lll} \delta(z_0, a, \#) = \{(z_0, B\#)\} & \delta(z_0, b, B) = \{(z_1, \varepsilon)\} & \delta(z_0, \varepsilon, \#) = \{(z_0, \varepsilon)\} \\ \delta(z_0, a, B) = \{(z_0, BB)\} & \delta(z_1, b, B) = \{(z_1, \varepsilon)\} & \delta(z_1, \varepsilon, \#) = \{(z_1, \varepsilon)\} \end{array}$$

und $\delta(z_i, c, A) = \emptyset$ sonst (für $c \in \{a, b, \varepsilon\}$).

Beispiel für einen PDA

PDA $M = (\{z_0, z_1\}, \{a, b\}, \{B, \#\}, \delta, z_0, \#)$ mit

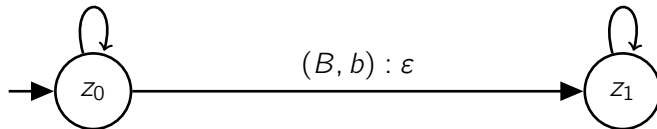
$$\begin{array}{lll} \delta(z_0, a, \#) = \{(z_0, B\#)\} & \delta(z_0, b, B) = \{(z_1, \varepsilon)\} & \delta(z_0, \varepsilon, \#) = \{(z_0, \varepsilon)\} \\ \delta(z_0, a, B) = \{(z_0, BB)\} & \delta(z_1, b, B) = \{(z_1, \varepsilon)\} & \delta(z_1, \varepsilon, \#) = \{(z_1, \varepsilon)\} \end{array}$$

und $\delta(z_i, c, A) = \emptyset$ sonst (für $c \in \{a, b, \varepsilon\}$).

Zustandsgraph dazu:

$(\#, a) : B\#, (\#, \varepsilon) : \varepsilon, (B, a) : BB$

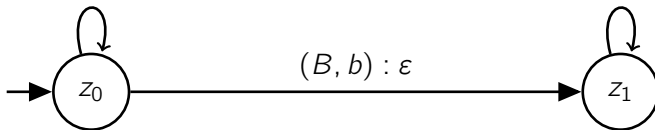
$(B, b) : \varepsilon, (\#, \varepsilon) : \varepsilon$



Beispiel für einen akzeptierenden Lauf

$(\#, a) : B\#, (\#, \varepsilon) : \varepsilon, (B, a) : BB$

$(B, b) : \varepsilon, (\#, \varepsilon) : \varepsilon$



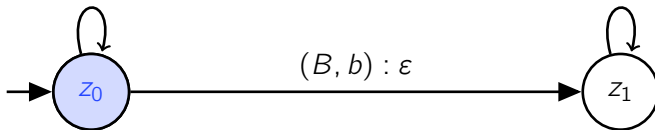
Eingabe: *aabb*

Keller: #

Beispiel für einen akzeptierenden Lauf

$(\#, a) : B\#, (\#, \varepsilon) : \varepsilon, (B, a) : BB$

$(B, b) : \varepsilon, (\#, \varepsilon) : \varepsilon$



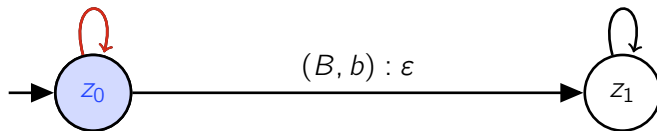
Eingabe: *aabb*

Keller: *#*

Beispiel für einen akzeptierenden Lauf

$(\#, a) : B\#$, $(\#, \varepsilon) : \varepsilon$, $(B, a) : BB$

$(B, b) : \varepsilon$, $(\#, \varepsilon) : \varepsilon$



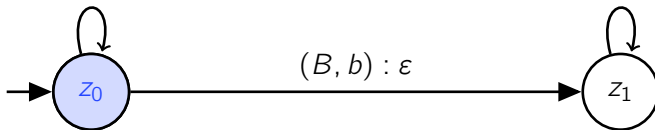
Eingabe: $aabb$

Keller: $\#$

Beispiel für einen akzeptierenden Lauf

$(\#, a) : B\#, (\#, \varepsilon) : \varepsilon, (B, a) : BB$

$(B, b) : \varepsilon, (\#, \varepsilon) : \varepsilon$



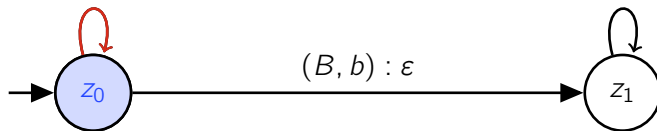
Eingabe: *a*abb

Keller: *B*#

Beispiel für einen akzeptierenden Lauf

$(\#, a) : B\#, (\#, \varepsilon) : \varepsilon, (B, a) : BB$

$(B, b) : \varepsilon, (\#, \varepsilon) : \varepsilon$



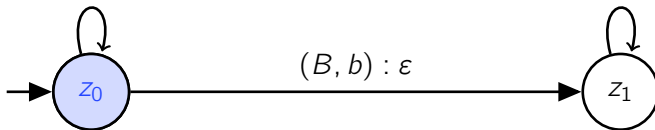
Eingabe: $aab\bar{b}$

Keller: $B\#$

Beispiel für einen akzeptierenden Lauf

$(\#, a) : B\#, (\#, \varepsilon) : \varepsilon, (B, a) : BB$

$(B, b) : \varepsilon, (\#, \varepsilon) : \varepsilon$



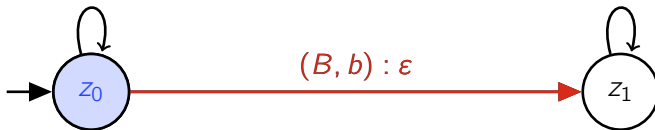
Eingabe: $aab\bar{b}$

Keller: $BB\#$

Beispiel für einen akzeptierenden Lauf

$(\#, a) : B\#, (\#, \varepsilon) : \varepsilon, (B, a) : BB$

$(B, b) : \varepsilon, (\#, \varepsilon) : \varepsilon$



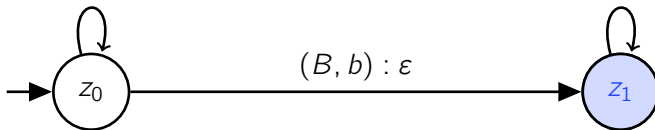
Eingabe: $aa**b**$

Keller: $**B**B\#$

Beispiel für einen akzeptierenden Lauf

$(\#, a) : B\#, (\#, \varepsilon) : \varepsilon, (B, a) : BB$

$(B, b) : \varepsilon, (\#, \varepsilon) : \varepsilon$



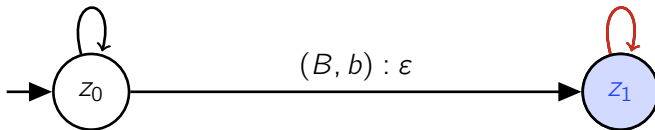
Eingabe: $aa**b**b$

Keller: $B\#$

Beispiel für einen akzeptierenden Lauf

$(\#, a) : B\#, (\#, \varepsilon) : \varepsilon, (B, a) : BB$

$(B, b) : \varepsilon, (\#, \varepsilon) : \varepsilon$



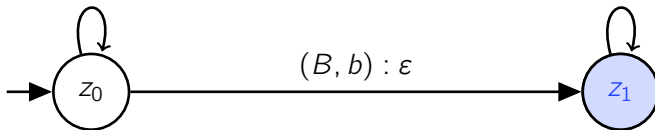
Eingabe: aab

Keller: $B\#$

Beispiel für einen akzeptierenden Lauf

$(\#, a) : B\#, (\#, \varepsilon) : \varepsilon, (B, a) : BB$

$(B, b) : \varepsilon, (\#, \varepsilon) : \varepsilon$



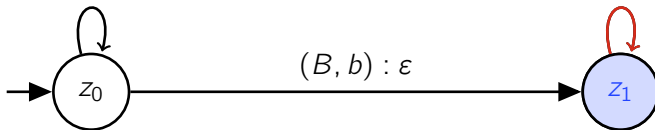
Eingabe: *aab***b**

Keller: #

Beispiel für einen akzeptierenden Lauf

$(\#, a) : B\#$, $(\#, \varepsilon) : \varepsilon$, $(B, a) : BB$

$(B, b) : \varepsilon$, $(\#, \varepsilon) : \varepsilon$



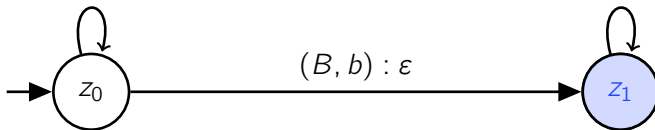
Eingabe: *aabb*

Keller: *#*

Beispiel für einen akzeptierenden Lauf

$(\#, a) : B\#, (\#, \varepsilon) : \varepsilon, (B, a) : BB$

$(B, b) : \varepsilon, (\#, \varepsilon) : \varepsilon$



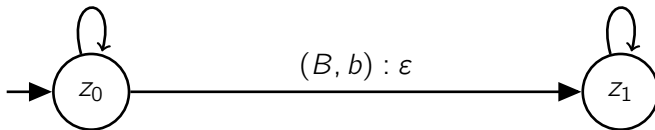
Eingabe: *aabb*

Keller: ε

Beispiel für einen akzeptierenden Lauf

$(\#, a) : B\#, (\#, \varepsilon) : \varepsilon, (B, a) : BB$

$(B, b) : \varepsilon, (\#, \varepsilon) : \varepsilon$



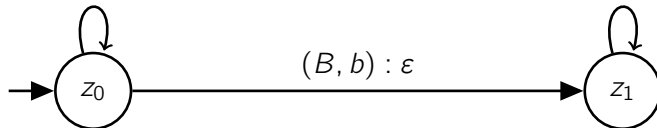
Eingabe: $aabb \in L(M)$

Keller: ε

Akzeptierte Sprache des Beispiels

$(\#, a) : B\#$, $(\#, \varepsilon) : \varepsilon$, $(B, a) : BB$

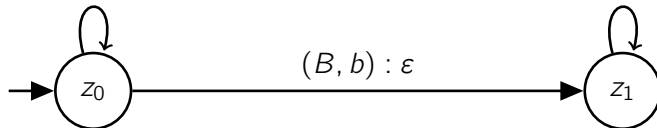
$(B, b) : \varepsilon$, $(\#, \varepsilon) : \varepsilon$



Akzeptierte Sprache des Beispiels

$(\#, a) : B\#, (\#, \varepsilon) : \varepsilon, (B, a) : BB$

$(B, b) : \varepsilon, (\#, \varepsilon) : \varepsilon$

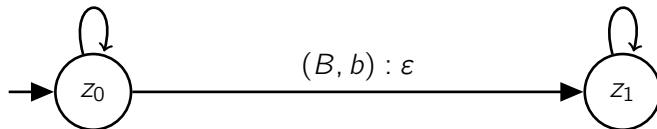


$L(M) = \{a^i b^i \mid i \in \mathbb{N}\}$ denn:

Akzeptierte Sprache des Beispiels

$(\#, a) : B\#, (\#, \varepsilon) : \varepsilon, (B, a) : BB$

$(B, b) : \varepsilon, (\#, \varepsilon) : \varepsilon$



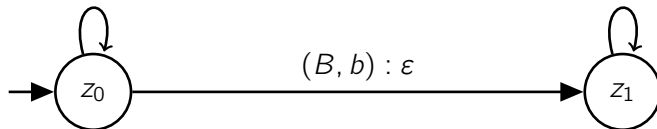
$L(M) = \{a^i b^i \mid i \in \mathbb{N}\}$ denn:

- M akzeptiert das Wort ε , denn $(z_0, \varepsilon, \#) \vdash (z_0, \varepsilon, \varepsilon)$.

Akzeptierte Sprache des Beispiels

$(\#, a) : B\#, (\#, \varepsilon) : \varepsilon, (B, a) : BB$

$(B, b) : \varepsilon, (\#, \varepsilon) : \varepsilon$



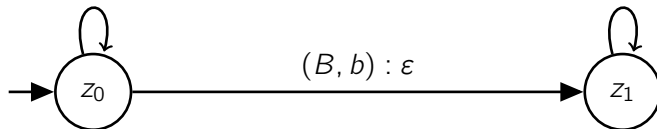
$L(M) = \{a^i b^i \mid i \in \mathbb{N}\}$ denn:

- ▶ M akzeptiert das Wort ε , denn $(z_0, \varepsilon, \#) \vdash (z_0, \varepsilon, \varepsilon)$.
- ▶ M akzeptiert das Wort $a^i b^i$ für $i > 0$, da $(z_0, a^i b^i, \#) \vdash (z_0, a^{i-1} b^i, B\#) \vdash^* (z_0, b^i, B^i \#) \vdash (z_1, b^{i-1}, B^{i-1} \#) \vdash^* (z_1, \varepsilon, \#) \vdash (z_1, \varepsilon, \varepsilon)$.

Akzeptierte Sprache des Beispiels

$(\#, a) : B\#, (\#, \varepsilon) : \varepsilon, (B, a) : BB$

$(B, b) : \varepsilon, (\#, \varepsilon) : \varepsilon$



$L(M) = \{a^i b^i \mid i \in \mathbb{N}\}$ denn:

- ▶ M akzeptiert das Wort ε , denn $(z_0, \varepsilon, \#) \vdash (z_0, \varepsilon, \varepsilon)$.
- ▶ M akzeptiert das Wort $a^i b^i$ für $i > 0$, da $(z_0, a^i b^i, \#) \vdash (z_0, a^{i-1} b^i, B\#) \vdash^* (z_0, b^i, B^i \#) \vdash (z_1, b^{i-1}, B^{i-1} \#) \vdash^* (z_1, \varepsilon, \#) \vdash (z_1, \varepsilon, \varepsilon)$.
- ▶ Andere Wörter werden nicht akzeptiert.

Weiteres Beispiel für einen PDA

Sei $M = (\{z_0, z_1\}, \{a, b\}, \{A, B, \#\}, \delta, z_0, \#)$ mit

$$\delta(z_0, a, \#) = \{(z_0, A\#), (z_1, \#)\}$$

$$\delta(z_0, b, \#) = \{(z_0, B\#), (z_1, \#)\}$$

$$\delta(z_0, a, A) = \{(z_0, AA), (z_1, A)\}$$

$$\delta(z_0, b, A) = \{(z_0, BA), (z_1, A)\}$$

$$\delta(z_0, a, B) = \{(z_0, AB), (z_1, B)\}$$

$$\delta(z_0, b, B) = \{(z_0, BB), (z_1, B)\}$$

$$\delta(z_0, \varepsilon, A) = \{(z_1, A)\}$$

$$\delta(z_0, \varepsilon, B) = \{(z_1, B)\}$$

$$\delta(z_0, \varepsilon, \#) = \{(z_1, \#)\}$$

$$\delta(z_1, a, A) = \{(z_1, \varepsilon)\}$$

$$\delta(z_1, b, B) = \{(z_1, \varepsilon)\}$$

$$\delta(z_1, \varepsilon, \#) = \{(z_1, \varepsilon)\}$$

und $\delta(z_i, c, C) = \emptyset$ sonst (für $c \in \{a, b, \varepsilon\}$). Zustandsgraph dazu:

$(\#, a) : A\#, (\#, b) : B\#,$

$(A, a) : AA, (A, b) : BA,$

$(B, a) : AB, (B, b) : BB$

$(\#, a) : \#, (\#, b) : \#, (A, a) : A,$

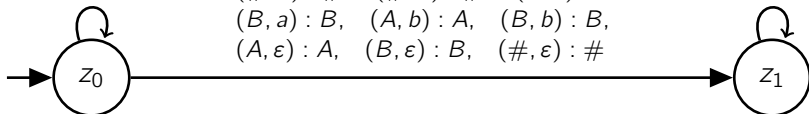
$(B, a) : B, (A, b) : A, (B, b) : B,$

$(A, \varepsilon) : A, (B, \varepsilon) : B, (\#, \varepsilon) : \#$

$(A, a) : \varepsilon,$

$(B, b) : \varepsilon,$

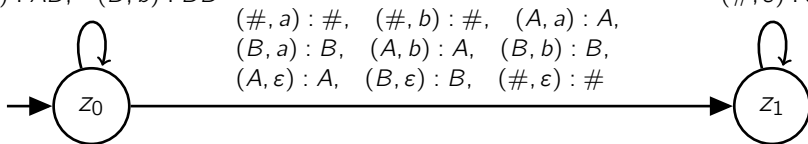
$(\#, \varepsilon) : \varepsilon$



Akzeptierte Sprache des weiteren Beispiels

$(\#, a) : A\#$, $(\#, b) : B\#$,
 $(A, a) : AA$, $(A, b) : BA$,
 $(B, a) : AB$, $(B, b) : BB$

$(A, a) : \varepsilon$,
 $(B, b) : \varepsilon$,
 $(\#, \varepsilon) : \varepsilon$

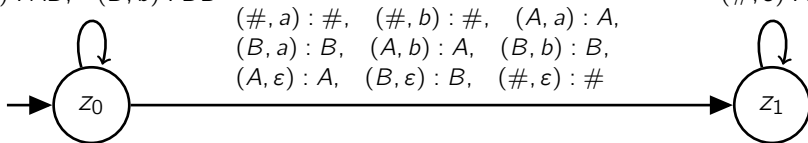


$L(M) = \{w \in \{a, b\}^* \mid w \text{ ist ein Palindrom}\}$ denn:

Akzeptierte Sprache des weiteren Beispiels

$(\#, a) : A\#$, $(\#, b) : B\#$,
 $(A, a) : AA$, $(A, b) : BA$,
 $(B, a) : AB$, $(B, b) : BB$

$(A, a) : \varepsilon$,
 $(B, b) : \varepsilon$,
 $(\#, \varepsilon) : \varepsilon$



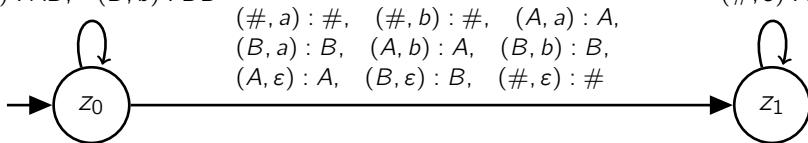
$L(M) = \{w \in \{a, b\}^* \mid w \text{ ist ein Palindrom}\}$ denn:

- In z_0 werden die gelesenen Zeichen (als A, B) auf den Keller gelegt.

Akzeptierte Sprache des weiteren Beispiels

$(\#, a) : A\#$, $(\#, b) : B\#$,
 $(A, a) : AA$, $(A, b) : BA$,
 $(B, a) : AB$, $(B, b) : BB$

$(A, a) : \varepsilon$,
 $(B, b) : \varepsilon$,
 $(\#, \varepsilon) : \varepsilon$



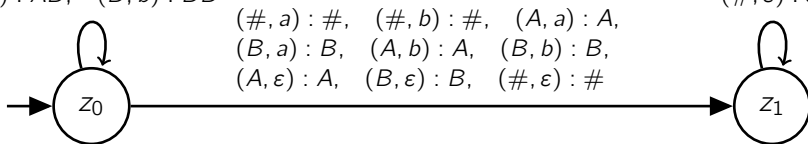
$L(M) = \{w \in \{a, b\}^* \mid w \text{ ist ein Palindrom}\}$ denn:

- ▶ In z_0 werden die gelesenen Zeichen (als A, B) auf den Keller gelegt.
- ▶ In z_1 werden sie dann wieder abgearbeitet (durch Lesen von a, b).

Akzeptierte Sprache des weiteren Beispiels

$(\#, a) : A\#$, $(\#, b) : B\#$,
 $(A, a) : AA$, $(A, b) : BA$,
 $(B, a) : AB$, $(B, b) : BB$

$(A, a) : \varepsilon$,
 $(B, b) : \varepsilon$,
 $(\#, \varepsilon) : \varepsilon$



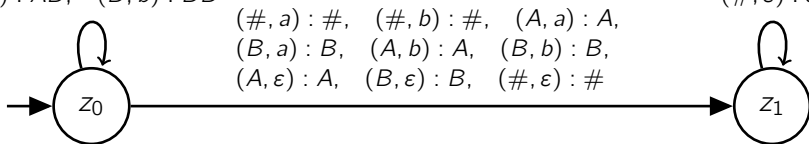
$L(M) = \{w \in \{a, b\}^* \mid w \text{ ist ein Palindrom}\}$ denn:

- ▶ In z_0 werden die gelesenen Zeichen (als A, B) auf den Keller gelegt.
- ▶ In z_1 werden sie dann wieder abgearbeitet (durch Lesen von a, b).
- ▶ Wechsel von z_0 zu z_1 mit einem Zeichen (für Palindrome $ua\bar{u}$, $ub\bar{u}$) oder mit ε (für Palindrome $u\bar{u}$).

Akzeptierte Sprache des weiteren Beispiels

$(\#, a) : A\#$, $(\#, b) : B\#$,
 $(A, a) : AA$, $(A, b) : BA$,
 $(B, a) : AB$, $(B, b) : BB$

$(A, a) : \varepsilon$,
 $(B, b) : \varepsilon$,
 $(\#, \varepsilon) : \varepsilon$



$L(M) = \{w \in \{a, b\}^* \mid w \text{ ist ein Palindrom}\}$ denn:

- ▶ In z_0 werden die gelesenen Zeichen (als A, B) auf den Keller gelegt.
- ▶ In z_1 werden sie dann wieder abgearbeitet (durch Lesen von a, b).
- ▶ Wechsel von z_0 zu z_1 mit einem Zeichen (für Palindrome $ua\bar{u}$, $ub\bar{u}$) oder mit ε (für Palindrome $u\bar{u}$).
- ▶ Der Nichtdeterminismus entdeckt den richtigen Zeitpunkt des Wechsels.

PDAs mit Erzeugung von höchstens zwei Kellersymbolen

Lemma

Für jeden PDA $M = (Z, \Sigma, \Gamma, \delta, z_0, \#)$ gibt es einen PDA $M' = (Z, \Sigma, \Gamma', \delta', z_0, \#)$ mit $L(M') = L(M)$, sodass wenn $(z', B_1 \cdots B_k) \in \delta'(z, a, A)$ (für $a \in \Sigma \cup \{\epsilon\}$), dann ist $k \leq 2$.

Beweis Transformiere M in M' wie folgt.

PDAs mit Erzeugung von höchstens zwei Kellersymbolen

Lemma

Für jeden PDA $M = (Z, \Sigma, \Gamma, \delta, z_0, \#)$ gibt es einen PDA $M' = (Z, \Sigma, \Gamma', \delta', z_0, \#)$ mit $L(M') = L(M)$, sodass wenn $(z', B_1 \cdots B_k) \in \delta'(z, a, A)$ (für $a \in \Sigma \cup \{\epsilon\}$), dann ist $k \leq 2$.

Beweis Transformiere M in M' wie folgt.

Sei $(z', B_1 \cdots B_k) \in \delta(z, a, A)$ mit $A \in \Gamma$ und $a \in \Sigma \cup \{\epsilon\}$.

PDAs mit Erzeugung von höchstens zwei Kellersymbolen

Lemma

Für jeden PDA $M = (Z, \Sigma, \Gamma, \delta, z_0, \#)$ gibt es einen PDA $M' = (Z, \Sigma, \Gamma', \delta', z_0, \#)$ mit $L(M') = L(M)$, sodass wenn $(z', B_1 \cdots B_k) \in \delta'(z, a, A)$ (für $a \in \Sigma \cup \{\epsilon\}$), dann ist $k \leq 2$.

Beweis Transformiere M in M' wie folgt.

Sei $(z', B_1 \cdots B_k) \in \delta(z, a, A)$ mit $A \in \Gamma$ und $a \in \Sigma \cup \{\epsilon\}$.

► Falls $k \leq 2$, dann $(z', B_1 \cdots B_k) \in \delta'(z, a, A)$.

PDAs mit Erzeugung von höchstens zwei Kellersymbolen

Lemma

Für jeden PDA $M = (Z, \Sigma, \Gamma, \delta, z_0, \#)$ gibt es einen PDA $M' = (Z, \Sigma, \Gamma', \delta', z_0, \#)$ mit $L(M') = L(M)$, sodass wenn $(z', B_1 \cdots B_k) \in \delta'(z, a, A)$ (für $a \in \Sigma \cup \{\varepsilon\}$), dann ist $k \leq 2$.

Beweis Transformiere M in M' wie folgt.

Sei $(z', B_1 \cdots B_k) \in \delta(z, a, A)$ mit $A \in \Gamma$ und $a \in \Sigma \cup \{\varepsilon\}$.

- ▶ Falls $k \leq 2$, dann $(z', B_1 \cdots B_k) \in \delta'(z, a, A)$.
- ▶ Falls $k > 2$, dann
 - ▶ $(z, C_k B_k) \in \delta'(z, a, A)$
 - ▶ $\delta'(z, \varepsilon, C_i) = \{(z, C_{i-1} B_{i-1})\}$ für jedes i mit $4 \leq i \leq k$
 - ▶ $\delta'(z, \varepsilon, C_3) = \{(z', B_1 B_2)\}$

wobei $C_3, \dots, C_k \in \Gamma'$ neue Kellersymbole sind.

(Diese werden jeweils neu erzeugt pro ersetztem Eintrag.)



Definition

Ein (nichtdeterministischer) Kellerautomat mit Endzuständen (PDA mit Endzuständen) ist ein 7-Tupel $M = (Z, \Sigma, \Gamma, \delta, z_0, \#, E)$, wobei:

- ▶ Z ist eine endliche Menge von Zuständen
- ▶ Σ ist das (endliche) Eingabealphabet
- ▶ Γ ist das (endliche) Kelleralphabet
- ▶ $\delta : Z \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \rightarrow \mathcal{P}_e(Z \times \Gamma^*)$ ist die Überföhrungsfunktion
- ▶ $z_0 \in Z$ ist der Startzustand
- ▶ $\# \in \Gamma$ ist das Startsymbol im Keller und
- ▶ $E \subseteq Z$ ist die Menge der Endzustände.

Definition

Sei $M = (Z, \Sigma, \Gamma, \delta, z_0, \#, E)$ ein PDA mit Endzuständen.

Die von M akzeptierte Sprache ist

$$L(M) := \{w \in \Sigma^* \mid (z_0, w, \#) \vdash^* (z, \varepsilon, W) \text{ für } z \in E \text{ und } W \in \Gamma^*\}$$

Äquivalenz von PDAs mit und ohne Endzustände

Lemma

Für jeden PDA mit Endzuständen M kann ein PDA M' ohne Endzustände konstruiert werden, sodass $L(M') = L(M)$ gilt.

Lemma

Für jeden PDA M ohne Endzustände kann ein PDA mit Endzuständen M' konstruiert werden, sodass $L(M') = L(M)$ gilt.

Satz

PDAs mit Endzuständen und PDAs ohne Endzustände sind äquivalente Formalismen.

Beweis Siehe Skript.



Anwendungen von Kellerautomaten

- ▶ Sie dienen als Implementierung für Tools wie [yacc](#), [ANTLR](#) und [PLY](#), die syntaktische Analyser ("Parser") generieren.
- ▶ Sie werden für die Analyse von Netzwerken verwendet.
- ▶ Sie werden für die automatische Verifikation von (z.B. C++- oder Java-)Programmen, inklusive Prozeduren und Rekursion, verwendet.

7a

**Äquivalenz von kontextfreien Sprachen und
von Kellerautomaten**

Prof. Dr. Jasmin Blanchette

Lehr- und Forschungseinheit für
Theoretische Informatik und Theorembeweisen

Stand: 16. Juli 2024

Basierend auf Folien von PD Dr. David Sabel



Definition

Ein (nichtdeterministischer) Kellerautomat (pushdown automaton, PDA) ist ein 6-Tupel $M = (Z, \Sigma, \Gamma, \delta, z_0, \#)$, wobei:

- ▶ Z ist eine endliche Menge von Zuständen
- ▶ Σ ist das (endliche) Eingabealphabet
- ▶ Γ ist das (endliche) Kelleralphabet
- ▶ $\delta : Z \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \rightarrow \mathcal{P}_e(Z \times \Gamma^*)$ ist die Überföhrungsfunktion
- ▶ $z_0 \in Z$ ist der Startzustand
- ▶ $\# \in \Gamma$ ist das Startsymbol im Keller.

Kontextfreie Sprachen werden von PDAs akzeptiert

Satz

Für jede kontextfreie Sprache L gibt es einen PDA M mit $L(M) = L$.

Beweis Sei $G = (V, \Sigma, P, S)$ mit $L(G) = L \setminus \{\epsilon\}$ eine kontextfreie Grammatik in Greibach-Normalform.

Kontextfreie Sprachen werden von PDAs akzeptiert

Satz

Für jede kontextfreie Sprache L gibt es einen PDA M mit $L(M) = L$.

Beweis Sei $G = (V, \Sigma, P, S)$ mit $L(G) = L \setminus \{\varepsilon\}$ eine kontextfreie Grammatik in Greibach-Normalform.

Grundgedanke:

- ▶ Wir definieren M , sodass er eine Linksableitung $S \Rightarrow^* a_1 \cdots a_n$ simuliert.
- ▶ Da G in Greibach-Normalform ist, ist eine Linksableitung nach i Schritten immer von der Form $S \Rightarrow^i a_1 \cdots a_i B_1 \cdots B_j$.
- ▶ Beginne Simulation mit Eingabe $a_1 \cdots a_n$ und S auf dem Keller.
- ▶ Nach i Schritten ist $a_1 \cdots a_i$ verarbeitet und $B_1 \cdots B_j$ auf dem Keller.
- ▶ Insbesondere ist nach n Schritten $a_1 \cdots a_n$ verarbeitet und ε auf dem Keller.

Kontextfreie Sprachen werden von PDAs akzeptiert

Beweis (Fortsetzung) Sei $G = (V, \Sigma, P, S)$ mit $L(G) = L \setminus \{\varepsilon\}$ eine kontextfreie Grammatik in Greibach-Normalform.

Sei $M = (\{z_0\}, \Sigma, V, \delta, z_0, S)$ ein PDA, sodass

$$\delta(z_0, a, A) := \{(z_0, B_1 \cdots B_n) \mid (A \rightarrow aB_1 \cdots B_n) \in P\}$$

$$\delta(z_0, \varepsilon, A) := \begin{cases} \{(z_0, \varepsilon)\} & \text{falls } \varepsilon \in L \text{ und } A = S \\ \emptyset & \text{sonst} \end{cases}$$

Im Skript wird $L(M) = L$ gezeigt.



Beispiel für die Konstruktion eines PDA aus einer Grammatik

Kontextfreie Grammatik in Greibach-Normalform:

$$G = (\{S, B, C\}, \{a, b\}, \{S \rightarrow aB, B \rightarrow b \mid aBC, C \rightarrow b\}, S)$$

Beispiel für die Konstruktion eines PDA aus einer Grammatik

Kontextfreie Grammatik in Greibach-Normalform:

$$G = (\{S, B, C\}, \{a, b\}, \{S \rightarrow aB, B \rightarrow b \mid aBC, C \rightarrow b\}, S)$$

PDA zu G : $M = (\{z_0\}, \Sigma, V, \delta, z_0, S)$ mit

$$\begin{aligned} \delta(z_0, a, S) &:= \{(z_0, B)\} & \delta(z_0, b, B) &:= \{(z_0, \varepsilon)\} & \delta(z_0, a, B) &:= \{(z_0, BC)\} \\ \delta(z_0, b, C) &:= \{(z_0, \varepsilon)\} & \delta(z_0, c, A) &:= \emptyset \text{ sonst (für } c \in \Sigma \cup \{\varepsilon\}) \end{aligned}$$

Beispiel für die Konstruktion eines PDA aus einer Grammatik

Kontextfreie Grammatik in Greibach-Normalform:

$$G = (\{S, B, C\}, \{a, b\}, \{S \rightarrow aB, B \rightarrow b \mid aBC, C \rightarrow b\}, S)$$

PDA zu G : $M = (\{z_0\}, \Sigma, V, \delta, z_0, S)$ mit

$$\begin{aligned} \delta(z_0, a, S) &:= \{(z_0, B)\} & \delta(z_0, b, B) &:= \{(z_0, \varepsilon)\} & \delta(z_0, a, B) &:= \{(z_0, BC)\} \\ \delta(z_0, b, C) &:= \{(z_0, \varepsilon)\} & \delta(z_0, c, A) &:= \emptyset \text{ sonst (für } c \in \Sigma \cup \{\varepsilon\}) \end{aligned}$$

Eine Konfigurationsfolge von M für die Eingabe $aaabbb$ ist

$$(z_0, aaabbb, S)$$

Beispiel für die Konstruktion eines PDA aus einer Grammatik

Kontextfreie Grammatik in Greibach-Normalform:

$$G = (\{S, B, C\}, \{a, b\}, \{S \rightarrow aB, B \rightarrow b \mid aBC, C \rightarrow b\}, S)$$

PDA zu G : $M = (\{z_0\}, \Sigma, V, \delta, z_0, S)$ mit

$$\begin{aligned} \delta(z_0, a, S) &:= \{(z_0, B)\} & \delta(z_0, b, B) &:= \{(z_0, \varepsilon)\} & \delta(z_0, a, B) &:= \{(z_0, BC)\} \\ \delta(z_0, b, C) &:= \{(z_0, \varepsilon)\} & \delta(z_0, c, A) &:= \emptyset \text{ sonst (für } c \in \Sigma \cup \{\varepsilon\}) \end{aligned}$$

Eine Konfigurationsfolge von M für die Eingabe $aaabbb$ ist

$$(z_0, aaabbb, S) \vdash (z_0, aabbb, B)$$

Beispiel für die Konstruktion eines PDA aus einer Grammatik

Kontextfreie Grammatik in Greibach-Normalform:

$$G = (\{S, B, C\}, \{a, b\}, \{S \rightarrow aB, B \rightarrow b \mid aBC, C \rightarrow b\}, S)$$

PDA zu G : $M = (\{z_0\}, \Sigma, V, \delta, z_0, S)$ mit

$$\begin{aligned} \delta(z_0, a, S) &:= \{(z_0, B)\} & \delta(z_0, b, B) &:= \{(z_0, \varepsilon)\} & \delta(z_0, a, B) &:= \{(z_0, BC)\} \\ \delta(z_0, b, C) &:= \{(z_0, \varepsilon)\} & \delta(z_0, c, A) &:= \emptyset \text{ sonst (für } c \in \Sigma \cup \{\varepsilon\}) \end{aligned}$$

Eine Konfigurationsfolge von M für die Eingabe $aaabbb$ ist

$$(z_0, aaabbb, S) \vdash (z_0, aabbb, B) \vdash (z_0, abbb, BC)$$

Beispiel für die Konstruktion eines PDA aus einer Grammatik

Kontextfreie Grammatik in Greibach-Normalform:

$$G = (\{S, B, C\}, \{a, b\}, \{S \rightarrow aB, B \rightarrow b \mid aBC, C \rightarrow b\}, S)$$

PDA zu G : $M = (\{z_0\}, \Sigma, V, \delta, z_0, S)$ mit

$$\begin{aligned} \delta(z_0, a, S) &:= \{(z_0, B)\} & \delta(z_0, b, B) &:= \{(z_0, \varepsilon)\} & \delta(z_0, a, B) &:= \{(z_0, BC)\} \\ \delta(z_0, b, C) &:= \{(z_0, \varepsilon)\} & \delta(z_0, c, A) &:= \emptyset \text{ sonst (für } c \in \Sigma \cup \{\varepsilon\}) \end{aligned}$$

Eine Konfigurationsfolge von M für die Eingabe $aaabbb$ ist

$$\begin{aligned} (z_0, aaabbb, S) &\vdash (z_0, aabbb, B) \vdash (z_0, abbb, BC) \\ &\vdash (z_0, bbb, BCC) \end{aligned}$$

Beispiel für die Konstruktion eines PDA aus einer Grammatik

Kontextfreie Grammatik in Greibach-Normalform:

$$G = (\{S, B, C\}, \{a, b\}, \{S \rightarrow aB, B \rightarrow b \mid aBC, C \rightarrow b\}, S)$$

PDA zu G : $M = (\{z_0\}, \Sigma, V, \delta, z_0, S)$ mit

$$\begin{aligned} \delta(z_0, a, S) &:= \{(z_0, B)\} & \delta(z_0, b, B) &:= \{(z_0, \epsilon)\} & \delta(z_0, a, B) &:= \{(z_0, BC)\} \\ \delta(z_0, b, C) &:= \{(z_0, \epsilon)\} & \delta(z_0, c, A) &:= \emptyset \text{ sonst (für } c \in \Sigma \cup \{\epsilon\}) \end{aligned}$$

Eine Konfigurationsfolge von M für die Eingabe $aaabbb$ ist

$$\begin{aligned} (z_0, aaabbb, S) &\vdash (z_0, aabbb, B) \vdash (z_0, abbb, BC) \\ &\vdash (z_0, bbb, BCC) \vdash (z_0, bb, CC) \end{aligned}$$

Beispiel für die Konstruktion eines PDA aus einer Grammatik

Kontextfreie Grammatik in Greibach-Normalform:

$$G = (\{S, B, C\}, \{a, b\}, \{S \rightarrow aB, B \rightarrow b \mid aBC, \textcolor{red}{C} \rightarrow \textcolor{red}{b}\}, S)$$

PDA zu G : $M = (\{z_0\}, \Sigma, V, \delta, z_0, S)$ mit

$$\begin{aligned} \delta(z_0, a, S) &:= \{(z_0, B)\} & \delta(z_0, b, B) &:= \{(z_0, \varepsilon)\} & \delta(z_0, a, B) &:= \{(z_0, BC)\} \\ \delta(z_0, \textcolor{red}{b}, \textcolor{red}{C}) &:= \{(z_0, \textcolor{blue}{\varepsilon})\} & \delta(z_0, c, A) &:= \emptyset \text{ sonst (für } c \in \Sigma \cup \{\varepsilon\}) \end{aligned}$$

Eine Konfigurationsfolge von M für die Eingabe $aaabbb$ ist

$$\begin{aligned} &(z_0, aaabbb, S) \vdash (z_0, aabbb, B) \vdash (z_0, abbb, BC) \\ &\vdash (z_0, bbb, BCC) \vdash (z_0, \textcolor{red}{bb}, \textcolor{red}{CC}) \vdash (z_0, b, C) \end{aligned}$$

Beispiel für die Konstruktion eines PDA aus einer Grammatik

Kontextfreie Grammatik in Greibach-Normalform:

$$G = (\{S, B, C\}, \{a, b\}, \{S \rightarrow aB, B \rightarrow b \mid aBC, \textcolor{red}{C} \rightarrow \textcolor{red}{b}\}, S)$$

PDA zu G : $M = (\{z_0\}, \Sigma, V, \delta, z_0, S)$ mit

$$\begin{aligned} \delta(z_0, a, S) &:= \{(z_0, B)\} & \delta(z_0, b, B) &:= \{(z_0, \varepsilon)\} & \delta(z_0, a, B) &:= \{(z_0, BC)\} \\ \delta(z_0, \textcolor{red}{b}, \textcolor{red}{C}) &:= \{(z_0, \textcolor{blue}{\varepsilon})\} & \delta(z_0, c, A) &:= \emptyset \text{ sonst (für } c \in \Sigma \cup \{\varepsilon\}) \end{aligned}$$

Eine Konfigurationsfolge von M für die Eingabe $aaabbb$ ist

$$\begin{aligned} (z_0, aaabbb, S) &\vdash (z_0, aabbb, B) \vdash (z_0, abbb, BC) \\ &\vdash (z_0, bbb, BCC) \vdash (z_0, bb, CC) \vdash (z_0, \textcolor{red}{b}, \textcolor{red}{C}) \vdash (z_0, \varepsilon, \varepsilon) \end{aligned}$$

Beispiel für die Konstruktion eines PDA aus einer Grammatik

Kontextfreie Grammatik in Greibach-Normalform

$$G = (\{S, B, C\}, \{a, b\}, \{S \rightarrow aB, B \rightarrow b \mid aBC, C \rightarrow b\}, S)$$

Eine Konfigurationsfolge von M für die Eingabe $aaabbb$ ist

Die entsprechende Linksableitung von G ist

Beispiel für die Konstruktion eines PDA aus einer Grammatik

Kontextfreie Grammatik in Greibach-Normalform

$$G = (\{S, B, C\}, \{a, b\}, \{S \rightarrow aB, B \rightarrow b \mid aBC, C \rightarrow b\}, S)$$

Eine Konfigurationsfolge von M für die Eingabe $aaabbb$ ist

$$(z_0, aaabbb, S)$$

Die entsprechende Linksableitung von G ist

$$S$$

Beispiel für die Konstruktion eines PDA aus einer Grammatik

Kontextfreie Grammatik in Greibach-Normalform

$$G = (\{S, B, C\}, \{a, b\}, \{S \rightarrow aB, B \rightarrow b \mid aBC, C \rightarrow b\}, S)$$

Eine Konfigurationsfolge von M für die Eingabe $aaabbb$ ist

$$(z_0, aaabbb, S) \vdash (z_0, aabbb, B)$$

Die entsprechende Linksableitung von G ist

$$S \Rightarrow aB$$

Beispiel für die Konstruktion eines PDA aus einer Grammatik

Kontextfreie Grammatik in Greibach-Normalform

$$G = (\{S, B, C\}, \{a, b\}, \{S \rightarrow aB, B \rightarrow b \mid aBC, C \rightarrow b\}, S)$$

Eine Konfigurationsfolge von M für die Eingabe $aaabbb$ ist

$$(z_0, aaabbb, S) \vdash (z_0, aabbb, B) \vdash (z_0, abbb, BC)$$

Die entsprechende Linksableitung von G ist

$$S \Rightarrow aB \Rightarrow aaBC$$

Beispiel für die Konstruktion eines PDA aus einer Grammatik

Kontextfreie Grammatik in Greibach-Normalform

$$G = (\{S, B, C\}, \{a, b\}, \{S \rightarrow aB, B \rightarrow b \mid aBC, C \rightarrow b\}, S)$$

Eine Konfigurationsfolge von M für die Eingabe $aaabbb$ ist

$$\begin{aligned} & (z_0, aaabbb, S) \vdash (z_0, aabbb, B) \vdash (z_0, abbb, BC) \\ & \vdash (z_0, bbb, BCC) \end{aligned}$$

Die entsprechende Linksableitung von G ist

$$S \Rightarrow aB \Rightarrow aaBC \Rightarrow aaBC$$

Beispiel für die Konstruktion eines PDA aus einer Grammatik

Kontextfreie Grammatik in Greibach-Normalform

$$G = (\{S, B, C\}, \{a, b\}, \{S \rightarrow aB, B \rightarrow b \mid aBC, C \rightarrow b\}, S)$$

Eine Konfigurationsfolge von M für die Eingabe $aaabbb$ ist

$$\begin{aligned} & (z_0, aaabbb, S) \vdash (z_0, aabbb, B) \vdash (z_0, abbb, BC) \\ & \vdash (z_0, bbb, BCC) \vdash (z_0, bb, CC) \end{aligned}$$

Die entsprechende Linksableitung von G ist

$$S \Rightarrow aB \Rightarrow aaBC \Rightarrow aaaBCC \Rightarrow aaabCC$$

Beispiel für die Konstruktion eines PDA aus einer Grammatik

Kontextfreie Grammatik in Greibach-Normalform

$$G = (\{S, B, C\}, \{a, b\}, \{S \rightarrow aB, B \rightarrow b \mid aBC, C \rightarrow b\}, S)$$

Eine Konfigurationsfolge von M für die Eingabe $aaabbb$ ist

$$\begin{aligned} & (z_0, aaabbb, S) \vdash (z_0, aabbb, B) \vdash (z_0, abbb, BC) \\ & \vdash (z_0, bbb, BCC) \vdash (z_0, bb, CC) \vdash (z_0, b, C) \end{aligned}$$

Die entsprechende Linksableitung von G ist

$$S \Rightarrow aB \Rightarrow aaBC \Rightarrow aaaBCC \Rightarrow aaabCC \Rightarrow aaab bC$$

Beispiel für die Konstruktion eines PDA aus einer Grammatik

Kontextfreie Grammatik in Greibach-Normalform

$$G = (\{S, B, C\}, \{a, b\}, \{S \rightarrow aB, B \rightarrow b \mid aBC, C \rightarrow b\}, S)$$

Eine Konfigurationsfolge von M für die Eingabe $aaabbb$ ist

$$\begin{aligned} & (z_0, aaabbb, S) \vdash (z_0, aabbb, B) \vdash (z_0, abbb, BC) \\ & \vdash (z_0, bbb, BCC) \vdash (z_0, bb, CC) \vdash (z_0, b, C) \vdash (z_0, \varepsilon, \varepsilon) \end{aligned}$$

Die entsprechende Linksableitung von G ist

$$S \Rightarrow aB \Rightarrow aaBC \Rightarrow aaaBCC \Rightarrow aaabCC \Rightarrow aaabbC \Rightarrow aaabbb$$

Die akzeptierte Sprache von PDAs ist kontextfrei

Satz

Sei $M = (Z, \Sigma, V, \delta, z_0, S)$ ein PDA. Dann ist $L(M)$ kontextfrei.

Die akzeptierte Sprache von PDAs ist kontextfrei

Satz

Sei $M = (Z, \Sigma, V, \delta, z_0, S)$ ein PDA. Dann ist $L(M)$ kontextfrei.

Beweis Grundgedanke:

- ▶ Nehme o.B.d.A. an, dass M maximal 2 Kellersymbole erzeugt.
- ▶ Definiere Grammatik mit der sogenannten **Tripelkonstruktion**:
 - ▶ Die Variablen der Grammatik sind Tripel $\langle z', A, z \rangle$, die alle Wörter w erzeugen, die den PDA von z' mit Kellerinhalt A und Wort w zu z und leeren Keller führen.

Die akzeptierte Sprache von PDAs ist kontextfrei

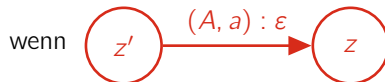
Satz

Sei $M = (Z, \Sigma, V, \delta, z_0, S)$ ein PDA. Dann ist $L(M)$ kontextfrei.

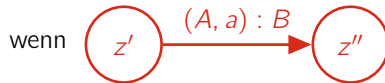
Beweis (Fortsetzung)

- Die Produktionen sind (für $a \in \Sigma \cup \{\varepsilon\}$)

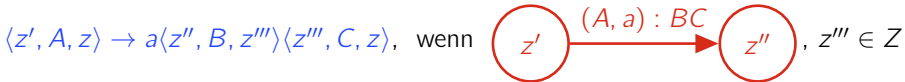
$$\langle z', A, z \rangle \rightarrow a,$$



$$\langle z', A, z \rangle \rightarrow a \langle z'', B, z \rangle,$$



$$\langle z', A, z \rangle \rightarrow a \langle z'', B, z''' \rangle \langle z''', C, z \rangle,$$



Die akzeptierte Sprache von PDAs ist kontextfrei

Satz

Sei $M = (Z, \Sigma, V, \delta, z_0, S)$ ein PDA. Dann ist $L(M)$ kontextfrei.

Beweis (Fortsetzung) Nehme o.B.d.A. an, dass M ein PDA mit $k \leq 2$ für alle $(z', B_1 \cdots B_k) \in \delta(z, a, A)$ (und $a \in \Sigma \cup \{\epsilon\}$) ist.

Die akzeptierte Sprache von PDAs ist kontextfrei

Satz

Sei $M = (Z, \Sigma, V, \delta, z_0, S)$ ein PDA. Dann ist $L(M)$ kontextfrei.

Beweis (Fortsetzung) Nehme o.B.d.A. an, dass M ein PDA mit $k \leq 2$ für alle $(z', B_1 \cdots B_k) \in \delta(z, a, A)$ (und $a \in \Sigma \cup \{\varepsilon\}$) ist.

Konstruiere $G = (V, \Sigma, P, S)$ (mit 2. Sonderregel), wobei S ein neues Symbol ist und

$$V := \{S\} \cup \{\langle z_i, A, z_j \rangle \mid z_i, z_j \in Z, A \in \Gamma\}$$

$$P := \{S \rightarrow \langle z_0, \#, z \rangle \mid z \in Z\}$$

$$\cup \{\langle z', A, z \rangle \rightarrow a \mid (z, \varepsilon) \in \delta(z', a, A), a \in \Sigma \cup \{\varepsilon\}, A \in \Gamma\}$$

$$\cup \{\langle z', A, z \rangle \rightarrow a \langle z'', B, z \rangle \mid (z'', B) \in \delta(z', a, A), z \in Z, a \in \Sigma \cup \{\varepsilon\}, A \in \Gamma\}$$

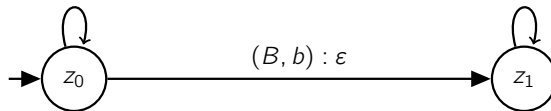
$$\cup \{\langle z', A, z \rangle \rightarrow a \langle z'', B, z''' \rangle \langle z''', C, z \rangle \mid (z'', BC) \in \delta(z', a, A), z, z''' \in Z, a \in \Sigma \cup \{\varepsilon\}, A \in \Gamma\}$$

Beispiel für die Tripelkonstruktion

PDA M :

$(\#, a) : B\#, (\#, \epsilon) : \epsilon, (B, a) : BB$

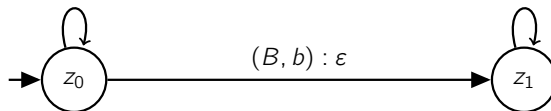
$(B, b) : \epsilon, (\#, \epsilon) : \epsilon$



Beispiel für die Tripelkonstruktion

PDA M :

$(\#, a) : B\#, (\#, \epsilon) : \epsilon, (B, a) : BB$ $(B, b) : \epsilon, (\#, \epsilon) : \epsilon$



Grammatik zu M : $G = (V, \Sigma, P, S)$ mit

$$V = \{S, \langle z_0, B, z_0 \rangle, \langle z_0, B, z_1 \rangle, \langle z_1, B, z_0 \rangle, \langle z_1, B, z_1 \rangle, \\ \langle z_0, \#, z_0 \rangle, \langle z_0, \#, z_1 \rangle, \langle z_1, \#, z_0 \rangle, \langle z_1, \#, z_1 \rangle\}$$

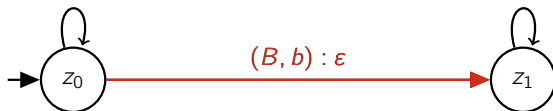
$$P = \{S \rightarrow \langle z_0, \#, z_0 \rangle, S \rightarrow \langle z_0, \#, z_1 \rangle\} \\ \cup \{\langle z_0, B, z_1 \rangle \rightarrow b, \langle z_1, B, z_1 \rangle \rightarrow b, \langle z_0, \#, z_0 \rangle \rightarrow \epsilon, \langle z_1, \#, z_1 \rangle \rightarrow \epsilon\} \\ \cup \{\langle z_0, \#, z_0 \rangle \rightarrow a\langle z_0, B, z_0 \rangle\langle z_0, \#, z_0 \rangle, \langle z_0, \#, z_0 \rangle \rightarrow a\langle z_0, B, z_1 \rangle\langle z_1, \#, z_0 \rangle, \\ \langle z_0, \#, z_1 \rangle \rightarrow a\langle z_0, B, z_0 \rangle\langle z_0, \#, z_1 \rangle, \langle z_0, \#, z_1 \rangle \rightarrow a\langle z_0, B, z_1 \rangle\langle z_1, \#, z_1 \rangle, \\ \langle z_0, B, z_0 \rangle \rightarrow a\langle z_0, B, z_0 \rangle\langle z_0, B, z_0 \rangle, \langle z_0, B, z_1 \rangle \rightarrow a\langle z_0, B, z_0 \rangle\langle z_0, B, z_1 \rangle, \\ \langle z_0, B, z_0 \rangle \rightarrow a\langle z_0, B, z_1 \rangle\langle z_1, B, z_0 \rangle, \langle z_0, B, z_1 \rangle \rightarrow a\langle z_0, B, z_1 \rangle\langle z_1, B, z_1 \rangle\}$$

Beispiel für die Tripelkonstruktion

PDA M :

$(\#, a) : B\#, (\#, \epsilon) : \epsilon, (B, a) : BB$

$(B, b) : \epsilon, (\#, \epsilon) : \epsilon$



Grammatik zu M : $G = (V, \Sigma, P, S)$ mit

$$V = \{S, \langle z_0, B, z_0 \rangle, \langle z_0, B, z_1 \rangle, \langle z_1, B, z_0 \rangle, \langle z_1, B, z_1 \rangle, \\ \langle z_0, \#, z_0 \rangle, \langle z_0, \#, z_1 \rangle, \langle z_1, \#, z_0 \rangle, \langle z_1, \#, z_1 \rangle\}$$

$$P = \{S \rightarrow \langle z_0, \#, z_0 \rangle, S \rightarrow \langle z_0, \#, z_1 \rangle\}$$

$$\cup \{\langle z_0, B, z_1 \rangle \rightarrow b, \langle z_1, B, z_1 \rangle \rightarrow b, \langle z_0, \#, z_0 \rangle \rightarrow \epsilon, \langle z_1, \#, z_1 \rangle \rightarrow \epsilon\}$$

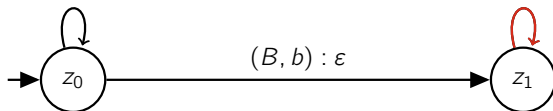
$$\cup \{\langle z_0, \#, z_0 \rangle \rightarrow a\langle z_0, B, z_0 \rangle\langle z_0, \#, z_0 \rangle, \langle z_0, \#, z_0 \rangle \rightarrow a\langle z_0, B, z_1 \rangle\langle z_1, \#, z_0 \rangle, \\ \langle z_0, \#, z_1 \rangle \rightarrow a\langle z_0, B, z_0 \rangle\langle z_0, \#, z_1 \rangle, \langle z_0, \#, z_1 \rangle \rightarrow a\langle z_0, B, z_1 \rangle\langle z_1, \#, z_1 \rangle, \\ \langle z_0, B, z_0 \rangle \rightarrow a\langle z_0, B, z_0 \rangle\langle z_0, B, z_0 \rangle, \langle z_0, B, z_1 \rangle \rightarrow a\langle z_0, B, z_0 \rangle\langle z_0, B, z_1 \rangle, \\ \langle z_0, B, z_0 \rangle \rightarrow a\langle z_0, B, z_1 \rangle\langle z_1, B, z_0 \rangle, \langle z_0, B, z_1 \rangle \rightarrow a\langle z_0, B, z_1 \rangle\langle z_1, B, z_1 \rangle\}$$

Beispiel für die Tripelkonstruktion

PDA M :

$(\#, a) : B\#, (\#, \epsilon) : \epsilon, (B, a) : BB$

$(B, b) : \epsilon, (\#, \epsilon) : \epsilon$



Grammatik zu M : $G = (V, \Sigma, P, S)$ mit

$$V = \{S, \langle z_0, B, z_0 \rangle, \langle z_0, B, z_1 \rangle, \langle z_1, B, z_0 \rangle, \langle z_1, B, z_1 \rangle, \\ \langle z_0, \#, z_0 \rangle, \langle z_0, \#, z_1 \rangle, \langle z_1, \#, z_0 \rangle, \langle z_1, \#, z_1 \rangle\}$$

$$P = \{S \rightarrow \langle z_0, \#, z_0 \rangle, S \rightarrow \langle z_0, \#, z_1 \rangle\}$$

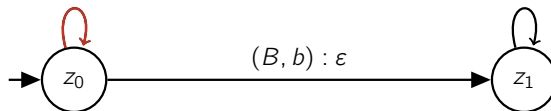
$$\cup \{\langle z_0, B, z_1 \rangle \rightarrow b, \langle z_1, B, z_1 \rangle \rightarrow b, \langle z_0, \#, z_0 \rangle \rightarrow \epsilon, \langle z_1, \#, z_1 \rangle \rightarrow \epsilon\}$$

$$\cup \{\langle z_0, \#, z_0 \rangle \rightarrow a \langle z_0, B, z_0 \rangle \langle z_0, \#, z_0 \rangle, \langle z_0, \#, z_0 \rangle \rightarrow a \langle z_0, B, z_1 \rangle \langle z_1, \#, z_0 \rangle, \\ \langle z_0, \#, z_1 \rangle \rightarrow a \langle z_0, B, z_0 \rangle \langle z_0, \#, z_1 \rangle, \langle z_0, \#, z_1 \rangle \rightarrow a \langle z_0, B, z_1 \rangle \langle z_1, \#, z_1 \rangle, \\ \langle z_0, B, z_0 \rangle \rightarrow a \langle z_0, B, z_0 \rangle \langle z_0, B, z_0 \rangle, \langle z_0, B, z_1 \rangle \rightarrow a \langle z_0, B, z_0 \rangle \langle z_0, B, z_1 \rangle, \\ \langle z_0, B, z_0 \rangle \rightarrow a \langle z_0, B, z_1 \rangle \langle z_1, B, z_0 \rangle, \langle z_0, B, z_1 \rangle \rightarrow a \langle z_0, B, z_1 \rangle \langle z_1, B, z_1 \rangle\}$$

Beispiel für die Tripelkonstruktion

PDA M :

$(\#, a) : B\#, (\#, \epsilon) : \epsilon, (B, a) : BB$ $(B, b) : \epsilon, (\#, \epsilon) : \epsilon$



Grammatik zu M : $G = (V, \Sigma, P, S)$ mit

$$V = \{S, \langle z_0, B, z_0 \rangle, \langle z_0, B, z_1 \rangle, \langle z_1, B, z_0 \rangle, \langle z_1, B, z_1 \rangle, \\ \langle z_0, \#, z_0 \rangle, \langle z_0, \#, z_1 \rangle, \langle z_1, \#, z_0 \rangle, \langle z_1, \#, z_1 \rangle\}$$

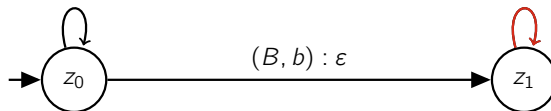
$$P = \{S \rightarrow \langle z_0, \#, z_0 \rangle, S \rightarrow \langle z_0, \#, z_1 \rangle\} \\ \cup \{\langle z_0, B, z_1 \rangle \rightarrow b, \langle z_1, B, z_1 \rangle \rightarrow b, \langle z_0, \#, z_0 \rangle \rightarrow \epsilon, \langle z_1, \#, z_1 \rangle \rightarrow \epsilon\} \\ \cup \{\langle z_0, \#, z_0 \rangle \rightarrow a\langle z_0, B, z_0 \rangle\langle z_0, \#, z_0 \rangle, \langle z_0, \#, z_0 \rangle \rightarrow a\langle z_0, B, z_1 \rangle\langle z_1, \#, z_0 \rangle, \\ \langle z_0, \#, z_1 \rangle \rightarrow a\langle z_0, B, z_0 \rangle\langle z_0, \#, z_1 \rangle, \langle z_0, \#, z_1 \rangle \rightarrow a\langle z_0, B, z_1 \rangle\langle z_1, \#, z_1 \rangle, \\ \langle z_0, B, z_0 \rangle \rightarrow a\langle z_0, B, z_0 \rangle\langle z_0, B, z_0 \rangle, \langle z_0, B, z_1 \rangle \rightarrow a\langle z_0, B, z_0 \rangle\langle z_0, B, z_1 \rangle, \\ \langle z_0, B, z_0 \rangle \rightarrow a\langle z_0, B, z_1 \rangle\langle z_1, B, z_0 \rangle, \langle z_0, B, z_1 \rangle \rightarrow a\langle z_0, B, z_1 \rangle\langle z_1, B, z_1 \rangle\}$$

Beispiel für die Tripelkonstruktion

PDA M :

$(\#, a) : B\#, (\#, \epsilon) : \epsilon, (B, a) : BB$

$(B, b) : \epsilon, (\#, \epsilon) : \epsilon$



Grammatik zu M : $G = (V, \Sigma, P, S)$ mit

$$V = \{S, \langle z_0, B, z_0 \rangle, \langle z_0, B, z_1 \rangle, \langle z_1, B, z_0 \rangle, \langle z_1, B, z_1 \rangle, \\ \langle z_0, \#, z_0 \rangle, \langle z_0, \#, z_1 \rangle, \langle z_1, \#, z_0 \rangle, \langle z_1, \#, z_1 \rangle\}$$

$$P = \{S \rightarrow \langle z_0, \#, z_0 \rangle, S \rightarrow \langle z_0, \#, z_1 \rangle\}$$

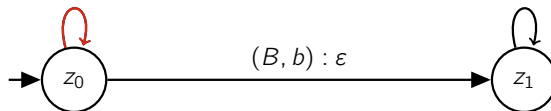
$$\cup \{\langle z_0, B, z_1 \rangle \rightarrow b, \langle z_1, B, z_1 \rangle \rightarrow b, \langle z_0, \#, z_0 \rangle \rightarrow \epsilon, \langle z_1, \#, z_1 \rangle \rightarrow \epsilon\}$$

$$\cup \{\langle z_0, \#, z_0 \rangle \rightarrow a \langle z_0, B, z_0 \rangle \langle z_0, \#, z_0 \rangle, \langle z_0, \#, z_0 \rangle \rightarrow a \langle z_0, B, z_1 \rangle \langle z_1, \#, z_0 \rangle, \\ \langle z_0, \#, z_1 \rangle \rightarrow a \langle z_0, B, z_0 \rangle \langle z_0, \#, z_1 \rangle, \langle z_0, \#, z_1 \rangle \rightarrow a \langle z_0, B, z_1 \rangle \langle z_1, \#, z_1 \rangle, \\ \langle z_0, B, z_0 \rangle \rightarrow a \langle z_0, B, z_0 \rangle \langle z_0, B, z_0 \rangle, \langle z_0, B, z_1 \rangle \rightarrow a \langle z_0, B, z_0 \rangle \langle z_0, B, z_1 \rangle, \\ \langle z_0, B, z_0 \rangle \rightarrow a \langle z_0, B, z_1 \rangle \langle z_1, B, z_0 \rangle, \langle z_0, B, z_1 \rangle \rightarrow a \langle z_0, B, z_1 \rangle \langle z_1, B, z_1 \rangle\}$$

Beispiel für die Tripelkonstruktion

PDA M :

$(\#, a) : B\#, (\#, \epsilon) : \epsilon, (B, a) : BB$ $(B, b) : \epsilon, (\#, \epsilon) : \epsilon$



Grammatik zu M : $G = (V, \Sigma, P, S)$ mit

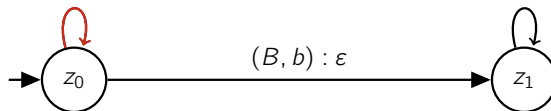
$$V = \{S, \langle z_0, B, z_0 \rangle, \langle z_0, B, z_1 \rangle, \langle z_1, B, z_0 \rangle, \langle z_1, B, z_1 \rangle, \\ \langle z_0, \#, z_0 \rangle, \langle z_0, \#, z_1 \rangle, \langle z_1, \#, z_0 \rangle, \langle z_1, \#, z_1 \rangle\}$$

$$P = \{S \rightarrow \langle z_0, \#, z_0 \rangle, S \rightarrow \langle z_0, \#, z_1 \rangle\} \\ \cup \{\langle z_0, B, z_1 \rangle \rightarrow b, \langle z_1, B, z_1 \rangle \rightarrow b, \langle z_0, \#, z_0 \rangle \rightarrow \epsilon, \langle z_1, \#, z_1 \rangle \rightarrow \epsilon\} \\ \cup \{\langle z_0, \#, z_0 \rangle \rightarrow a\langle z_0, B, z_0 \rangle\langle z_0, \#, z_0 \rangle, \langle z_0, \#, z_0 \rangle \rightarrow a\langle z_0, B, z_1 \rangle\langle z_1, \#, z_0 \rangle, \\ \langle z_0, \#, z_1 \rangle \rightarrow a\langle z_0, B, z_0 \rangle\langle z_0, \#, z_1 \rangle, \langle z_0, \#, z_1 \rangle \rightarrow a\langle z_0, B, z_1 \rangle\langle z_1, \#, z_1 \rangle, \\ \langle z_0, B, z_0 \rangle \rightarrow a\langle z_0, B, z_0 \rangle\langle z_0, B, z_0 \rangle, \langle z_0, B, z_1 \rangle \rightarrow a\langle z_0, B, z_0 \rangle\langle z_0, B, z_1 \rangle, \\ \langle z_0, B, z_0 \rangle \rightarrow a\langle z_0, B, z_1 \rangle\langle z_1, B, z_0 \rangle, \langle z_0, B, z_1 \rangle \rightarrow a\langle z_0, B, z_1 \rangle\langle z_1, B, z_1 \rangle\}$$

Beispiel für die Tripelkonstruktion

PDA M :

$(\#, a) : B\#, (\#, \epsilon) : \epsilon, (B, a) : BB$ $(B, b) : \epsilon, (\#, \epsilon) : \epsilon$



Grammatik zu M : $G = (V, \Sigma, P, S)$ mit

$$V = \{S, \langle z_0, B, z_0 \rangle, \langle z_0, B, z_1 \rangle, \langle z_1, B, z_0 \rangle, \langle z_1, B, z_1 \rangle, \\ \langle z_0, \#, z_0 \rangle, \langle z_0, \#, z_1 \rangle, \langle z_1, \#, z_0 \rangle, \langle z_1, \#, z_1 \rangle\}$$

$$P = \{S \rightarrow \langle z_0, \#, z_0 \rangle, S \rightarrow \langle z_0, \#, z_1 \rangle\} \\ \cup \{\langle z_0, B, z_1 \rangle \rightarrow b, \langle z_1, B, z_1 \rangle \rightarrow b, \langle z_0, \#, z_0 \rangle \rightarrow \epsilon, \langle z_1, \#, z_1 \rangle \rightarrow \epsilon\} \\ \cup \{\langle z_0, \#, z_0 \rangle \rightarrow a\langle z_0, B, z_0 \rangle\langle z_0, \#, z_0 \rangle, \langle z_0, \#, z_0 \rangle \rightarrow a\langle z_0, B, z_1 \rangle\langle z_1, \#, z_0 \rangle, \\ \langle z_0, \#, z_1 \rangle \rightarrow a\langle z_0, B, z_0 \rangle\langle z_0, \#, z_1 \rangle, \langle z_0, \#, z_1 \rangle \rightarrow a\langle z_0, B, z_1 \rangle\langle z_1, \#, z_1 \rangle, \\ \langle z_0, B, z_0 \rangle \rightarrow a\langle z_0, B, z_0 \rangle\langle z_0, B, z_0 \rangle, \langle z_0, B, z_1 \rangle \rightarrow a\langle z_0, B, z_0 \rangle\langle z_0, B, z_1 \rangle, \\ \langle z_0, B, z_0 \rangle \rightarrow a\langle z_0, B, z_1 \rangle\langle z_1, B, z_0 \rangle, \langle z_0, B, z_1 \rangle \rightarrow a\langle z_0, B, z_1 \rangle\langle z_1, B, z_1 \rangle\}$$

Beispiel für die Tripelkonstruktion

Diese Grammatik kann man noch vereinfachen, indem man untersucht, welche Produktionen nie in einer erfolgreichen Ableitung verwendet werden können.

Das Ergebnis ist $G' = (V', \Sigma, P', S)$, wobei

$$V' = \{S, \langle z_0, B, z_1 \rangle, \langle z_1, B, z_1 \rangle, \langle z_0, \#, z_0 \rangle, \langle z_0, \#, z_1 \rangle, \langle z_1, \#, z_1 \rangle\}$$

$$P' = \{S \rightarrow \langle z_0, \#, z_0 \rangle, \\ S \rightarrow \langle z_0, \#, z_1 \rangle\}$$

$$\cup \{\langle z_0, B, z_1 \rangle \rightarrow b, \\ \langle z_1, B, z_1 \rangle \rightarrow b, \\ \langle z_0, \#, z_0 \rangle \rightarrow \varepsilon, \\ \langle z_1, \#, z_1 \rangle \rightarrow \varepsilon\}$$

$$\cup \{\langle z_0, \#, z_1 \rangle \rightarrow a\langle z_0, B, z_1 \rangle\langle z_1, \#, z_1 \rangle, \\ \langle z_0, B, z_1 \rangle \rightarrow a\langle z_0, B, z_1 \rangle\langle z_1, B, z_1 \rangle\}$$

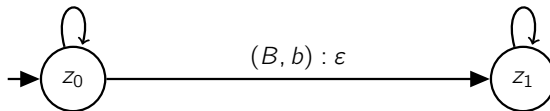
Diese Grammatik kann man noch weiter vereinfachen (siehe Skript).

Beispiel für die Tripelkonstruktion

PDA M :

$(\#, a) : B\#, (\#, \varepsilon) : \varepsilon, (B, a) : BB$

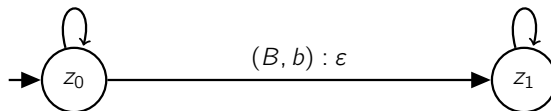
$(B, b) : \varepsilon, (\#, \varepsilon) : \varepsilon$



Beispiel für die Tripelkonstruktion

PDA M :

$(\#, a) : B\#, (\#, \varepsilon) : \varepsilon, (B, a) : BB$ $(B, b) : \varepsilon, (\#, \varepsilon) : \varepsilon$



Eine Konfigurationsfolge von M für die Eingabe $aabb$ ist

$(z_0, aabb, \#)$

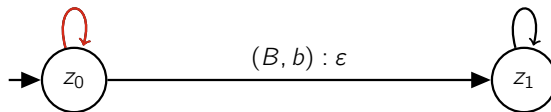
Eine entsprechende Linksableitung von G ist

$S \Rightarrow \langle z_0, \#, z_1 \rangle$

Beispiel für die Tripelkonstruktion

PDA M :

$(\#, a) : B\#, (\#, \varepsilon) : \varepsilon, (B, a) : BB$ $(B, b) : \varepsilon, (\#, \varepsilon) : \varepsilon$



Eine Konfigurationsfolge von M für die Eingabe $aabb$ ist

$$(z_0, aabb, \#) \vdash (z_0, abb, B\#)$$

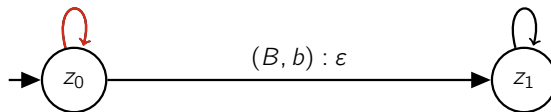
Eine entsprechende Linksableitung von G ist

$$S \Rightarrow \langle z_0, \#, z_1 \rangle \Rightarrow a \langle z_0, B, z_1 \rangle \langle z_1, \#, z_1 \rangle$$

Beispiel für die Tripelkonstruktion

PDA M :

$(\#, a) : B\#, (\#, \varepsilon) : \varepsilon, (B, a) : BB$ $(B, b) : \varepsilon, (\#, \varepsilon) : \varepsilon$



Eine Konfigurationsfolge von M für die Eingabe $aabb$ ist

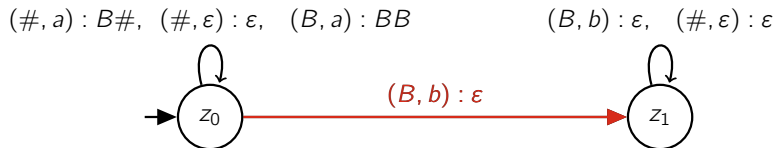
$(z_0, aabb, \#) \vdash (z_0, abb, B\#) \vdash (z_0, bb, BB\#)$

Eine entsprechende Linksableitung von G ist

$S \Rightarrow \langle z_0, \#, z_1 \rangle \Rightarrow a\langle z_0, B, z_1 \rangle \langle z_1, \#, z_1 \rangle \Rightarrow aa\langle z_0, B, z_1 \rangle \langle z_1, B, z_1 \rangle \langle z_1, \#, z_1 \rangle$

Beispiel für die Tripelkonstruktion

PDA M :



Eine Konfigurationsfolge von M für die Eingabe $aabb$ ist

$$(z_0, aabb, \#) \vdash (z_0, abb, B\#) \vdash (z_0, \textcolor{red}{bb}, \textcolor{red}{B}B\#) \vdash (z_1, b, B\#)$$

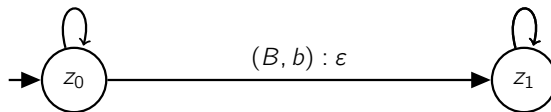
Eine entsprechende Linksableitung von G ist

$$\begin{aligned} S &\Rightarrow \langle z_0, \#, z_1 \rangle \Rightarrow a \langle z_0, B, z_1 \rangle \langle z_1, \#, z_1 \rangle \Rightarrow aa \langle \textcolor{red}{z_0}, \textcolor{red}{B}, \textcolor{red}{z_1} \rangle \langle z_1, B, z_1 \rangle \langle z_1, \#, z_1 \rangle \\ &\Rightarrow aab \langle z_1, B, z_1 \rangle \langle z_1, \#, z_1 \rangle \end{aligned}$$

Beispiel für die Tripelkonstruktion

PDA M :

$(\#, a) : B\#, (\#, \varepsilon) : \varepsilon, (B, a) : BB$ $(B, b) : \varepsilon, (\#, \varepsilon) : \varepsilon$



Eine Konfigurationsfolge von M für die Eingabe $aabb$ ist

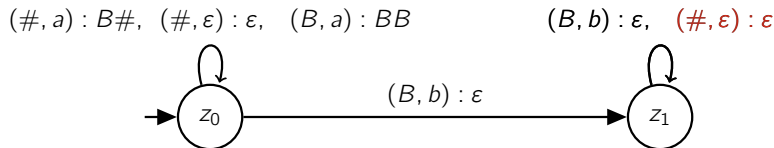
$(z_0, aabb, \#) \vdash (z_0, abb, B\#) \vdash (z_0, bb, BB\#) \vdash (z_1, \textcolor{red}{b}, \textcolor{red}{B}\#) \vdash (z_1, \varepsilon, \#)$

Eine entsprechende Linksableitung von G ist

$S \Rightarrow \langle z_0, \#, z_1 \rangle \Rightarrow a \langle z_0, B, z_1 \rangle \langle z_1, \#, z_1 \rangle \Rightarrow aa \langle z_0, B, z_1 \rangle \langle z_1, B, z_1 \rangle \langle z_1, \#, z_1 \rangle$
 $\Rightarrow aab \langle \textcolor{red}{z}_1, \textcolor{red}{B}, \textcolor{red}{z}_1 \rangle \langle z_1, \#, z_1 \rangle \Rightarrow aab \textcolor{blue}{b} \langle z_1, \#, z_1 \rangle$

Beispiel für die Tripelkonstruktion

PDA M :



Eine Konfigurationsfolge von M für die Eingabe $aabb$ ist

$$(z_0, aabb, \#) \vdash (z_0, abb, B\#) \vdash (z_0, bb, BB\#) \vdash (z_1, b, B\#) \vdash (z_1, \varepsilon, \#) \vdash (z_1, \varepsilon, \varepsilon)$$

Eine entsprechende Linksableitung von G ist

$$\begin{aligned} S &\Rightarrow \langle z_0, \#, z_1 \rangle \Rightarrow a \langle z_0, B, z_1 \rangle \langle z_1, \#, z_1 \rangle \Rightarrow aa \langle z_0, B, z_1 \rangle \langle z_1, B, z_1 \rangle \langle z_1, \#, z_1 \rangle \\ &\Rightarrow aab \langle z_1, B, z_1 \rangle \langle z_1, \#, z_1 \rangle \Rightarrow aabb \langle z_1, \#, z_1 \rangle \Rightarrow aabb \end{aligned}$$

Die akzeptierte Sprache von PDAs ist kontextfrei

Beweis (Fortsetzung) Wir zeigen die Verallgemeinerung:

$\langle z', A, z \rangle \Rightarrow_G^* w$ g.d.w. $(z', w, A) \vdash_M^* (z, \varepsilon, \varepsilon)$.

Die akzeptierte Sprache von PDAs ist kontextfrei

Beweis (Fortsetzung) Wir zeigen die Verallgemeinerung:

$\langle z', A, z \rangle \Rightarrow_G^* w$ g.d.w. $(z', w, A) \vdash_M^* (z, \varepsilon, \varepsilon)$.

Da $S \rightarrow \langle z_0, \#, z \rangle \in P$ folgt: $w \in L(G)$ g.d.w. $w \in L(M)$, d.h. $L(G) = L(M)$.

$\Rightarrow$ Sei $\langle z', A, z \rangle \Rightarrow_G^i w$ eine Linksableitung. Wir verwenden Induktion über i .

Die akzeptierte Sprache von PDAs ist kontextfrei

Beweis (Fortsetzung) Wir zeigen die Verallgemeinerung:

$\langle z', A, z \rangle \Rightarrow_G^* w$ g.d.w. $(z', w, A) \vdash_M^* (z, \varepsilon, \varepsilon)$.

Da $S \rightarrow \langle z_0, \#, z \rangle \in P$ folgt: $w \in L(G)$ g.d.w. $w \in L(M)$, d.h. $L(G) = L(M)$.

$\Rightarrow$ Sei $\langle z', A, z \rangle \Rightarrow_G^i w$ eine Linksableitung. Wir verwenden Induktion über i .

► Fall $i = 0$: Unmöglich.

Die akzeptierte Sprache von PDAs ist kontextfrei

Beweis (Fortsetzung) Wir zeigen die Verallgemeinerung:

$\langle z', A, z \rangle \Rightarrow_G^* w$ g.d.w. $(z', w, A) \vdash_M^* (z, \varepsilon, \varepsilon)$.

Da $S \rightarrow \langle z_0, \#, z \rangle \in P$ folgt: $w \in L(G)$ g.d.w. $w \in L(M)$, d.h. $L(G) = L(M)$.

$\Rightarrow$ Sei $\langle z', A, z \rangle \Rightarrow_G^i w$ eine Linksableitung. Wir verwenden Induktion über i .

► Fall $i = 0$: Unmöglich.

► Fall $i = 1$: Sei $\langle z', A, z \rangle \Rightarrow_G w$. Die verwendete Produktion muss $\langle z', A, z \rangle \rightarrow a$ sein. Dann muss $(z, \varepsilon) \in \delta(z', a, A)$ gelten und damit gilt: $(z', a, A) \vdash (z, \varepsilon, \varepsilon)$.

Die akzeptierte Sprache von PDAs ist kontextfrei

Beweis (Fortsetzung) Wir zeigen die Verallgemeinerung:

$\langle z', A, z \rangle \Rightarrow_G^* w$ g.d.w. $(z', w, A) \vdash_M^* (z, \varepsilon, \varepsilon)$.

Da $S \rightarrow \langle z_0, \#, z \rangle \in P$ folgt: $w \in L(G)$ g.d.w. $w \in L(M)$, d.h. $L(G) = L(M)$.

$\Rightarrow$ Sei $\langle z', A, z \rangle \Rightarrow_G^i w$ eine Linksableitung. Wir verwenden Induktion über i .

- ▶ Fall $i = 0$: Unmöglich.
- ▶ Fall $i = 1$: Sei $\langle z', A, z \rangle \Rightarrow_G w$. Die verwendete Produktion muss $\langle z', A, z \rangle \rightarrow a$ sein. Dann muss $(z, \varepsilon) \in \delta(z', a, A)$ gelten und damit gilt: $(z', a, A) \vdash (z, \varepsilon, \varepsilon)$.
- ▶ Fall $i > 1$: Sei $\langle z', A, z \rangle \Rightarrow_G u \Rightarrow_G^{i-1} w$ mit $i - 1 > 0$.

Die akzeptierte Sprache von PDAs ist kontextfrei

Beweis (Fortsetzung) Wir zeigen die Verallgemeinerung:

$\langle z', A, z \rangle \Rightarrow_G^* w$ g.d.w. $(z', w, A) \vdash_M^* (z, \varepsilon, \varepsilon)$.

Da $S \rightarrow \langle z_0, \#, z \rangle \in P$ folgt: $w \in L(G)$ g.d.w. $w \in L(M)$, d.h. $L(G) = L(M)$.

$\Rightarrow$ Sei $\langle z', A, z \rangle \Rightarrow_G^i w$ eine Linksableitung. Wir verwenden Induktion über i .

- ▶ Fall $i = 0$: Unmöglich.
- ▶ Fall $i = 1$: Sei $\langle z', A, z \rangle \Rightarrow_G w$. Die verwendete Produktion muss $\langle z', A, z \rangle \rightarrow a$ sein. Dann muss $(z, \varepsilon) \in \delta(z', a, A)$ gelten und damit gilt: $(z', a, A) \vdash (z, \varepsilon, \varepsilon)$.
- ▶ Fall $i > 1$: Sei $\langle z', A, z \rangle \Rightarrow_G u \Rightarrow_G^{i-1} w$ mit $i - 1 > 0$.
 - ▶ Wenn $u = a \in \Sigma \cup \{\varepsilon\}$, dann kann $i - 1 > 0$ nicht gelten.

Die akzeptierte Sprache von PDAs ist kontextfrei

Beweis (Fortsetzung) Wir zeigen die Verallgemeinerung:

$\langle z', A, z \rangle \Rightarrow_G^* w$ g.d.w. $(z', w, A) \vdash_M^* (z, \varepsilon, \varepsilon)$.

Da $S \rightarrow \langle z_0, \#, z \rangle \in P$ folgt: $w \in L(G)$ g.d.w. $w \in L(M)$, d.h. $L(G) = L(M)$.

$\Rightarrow$ Sei $\langle z', A, z \rangle \Rightarrow_G^i w$ eine Linksableitung. Wir verwenden Induktion über i .

- ▶ Fall $i = 0$: Unmöglich.
- ▶ Fall $i = 1$: Sei $\langle z', A, z \rangle \Rightarrow_G w$. Die verwendete Produktion muss $\langle z', A, z \rangle \rightarrow a$ sein. Dann muss $(z, \varepsilon) \in \delta(z', a, A)$ gelten und damit gilt: $(z', a, A) \vdash (z, \varepsilon, \varepsilon)$.
- ▶ Fall $i > 1$: Sei $\langle z', A, z \rangle \Rightarrow_G u \Rightarrow_G^{i-1} w$ mit $i - 1 > 0$.
 - ▶ Wenn $u = a \in \Sigma \cup \{\varepsilon\}$, dann kann $i - 1 > 0$ nicht gelten.
 - ▶ Wenn $u = a\langle z'', B, z \rangle$, dann $(z'', B) \in \delta(z', a, A)$ und $u = a\langle z'', B, z \rangle \Rightarrow^{i-1} aw' = w$.

Die akzeptierte Sprache von PDAs ist kontextfrei

Beweis (Fortsetzung) Wir zeigen die Verallgemeinerung:

$\langle z', A, z \rangle \Rightarrow_G^* w$ g.d.w. $(z', w, A) \vdash_M^* (z, \varepsilon, \varepsilon)$.

Da $S \rightarrow \langle z_0, \#, z \rangle \in P$ folgt: $w \in L(G)$ g.d.w. $w \in L(M)$, d.h. $L(G) = L(M)$.

$\Rightarrow$ Sei $\langle z', A, z \rangle \Rightarrow_G^i w$ eine Linksableitung. Wir verwenden Induktion über i .

- ▶ Fall $i = 0$: Unmöglich.
- ▶ Fall $i = 1$: Sei $\langle z', A, z \rangle \Rightarrow_G w$. Die verwendete Produktion muss $\langle z', A, z \rangle \rightarrow a$ sein. Dann muss $(z, \varepsilon) \in \delta(z', a, A)$ gelten und damit gilt: $(z', a, A) \vdash (z, \varepsilon, \varepsilon)$.
- ▶ Fall $i > 1$: Sei $\langle z', A, z \rangle \Rightarrow_G u \Rightarrow_G^{i-1} w$ mit $i - 1 > 0$.
 - ▶ Wenn $u = a \in \Sigma \cup \{\varepsilon\}$, dann kann $i - 1 > 0$ nicht gelten.
 - ▶ Wenn $u = a\langle z'', B, z \rangle$, dann $(z'', B) \in \delta(z', a, A)$ und $u = a\langle z'', B, z \rangle \Rightarrow^{i-1} aw' = w$.

Dann gilt $\langle z'', B, z \rangle \Rightarrow^{i-1} w'$ und die Induktionshypothese liefert

$(z'', w', B) \vdash_M^* (z, \varepsilon, \varepsilon)$. Mit $(z'', B) \in \delta(z', a, A)$ zeigt dies

$(z', \underbrace{aw'}_w, A) \vdash_M (z'', w', B) \vdash_M^* (z, \varepsilon, \varepsilon)$.

Die akzeptierte Sprache von PDAs ist kontextfrei

Beweis (Fortsetzung) Wir zeigen die Verallgemeinerung:

$\langle z', A, z \rangle \Rightarrow_G^* w$ g.d.w. $(z', w, A) \vdash_M^* (z, \varepsilon, \varepsilon)$.

Da $S \rightarrow \langle z_0, \#, z \rangle \in P$ folgt: $w \in L(G)$ g.d.w. $w \in L(M)$, d.h. $L(G) = L(M)$.

$\Rightarrow$ Sei $\langle z', A, z \rangle \Rightarrow_G^i w$ eine Linksableitung. Wir verwenden Induktion über i .

- ▶ Fall $i = 0$: Unmöglich.
- ▶ Fall $i = 1$: Sei $\langle z', A, z \rangle \Rightarrow_G w$. Die verwendete Produktion muss $\langle z', A, z \rangle \rightarrow a$ sein. Dann muss $(z, \varepsilon) \in \delta(z', a, A)$ gelten und damit gilt: $(z', a, A) \vdash (z, \varepsilon, \varepsilon)$.
- ▶ Fall $i > 1$: Sei $\langle z', A, z \rangle \Rightarrow_G u \Rightarrow_G^{i-1} w$ mit $i - 1 > 0$.
 - ▶ Wenn $u = a \in \Sigma \cup \{\varepsilon\}$, dann kann $i - 1 > 0$ nicht gelten.
 - ▶ Wenn $u = a\langle z'', B, z \rangle$, dann $(z'', B) \in \delta(z', a, A)$ und $u = a\langle z'', B, z \rangle \Rightarrow^{i-1} aw' = w$.
Dann gilt $\langle z'', B, z \rangle \Rightarrow^{i-1} w'$ und die Induktionshypothese liefert $(z'', w', B) \vdash_M^* (z, \varepsilon, \varepsilon)$. Mit $(z'', B) \in \delta(z', a, A)$ zeigt dies $(z', \underbrace{aw'}_w, A) \vdash_M (z'', w', B) \vdash_M^* (z, \varepsilon, \varepsilon)$.
- ▶ Der Fall $u = a\langle z'', B, z''' \rangle \langle z''', C, z \rangle$ ist komplizierter aber ähnlich (siehe Skript).

Die akzeptierte Sprache von PDAs ist kontextfrei

$\Leftarrow$ Sei $(z', w, A) \vdash_M^i (z, \varepsilon, \varepsilon)$. Wir zeigen $\langle z', A, z \rangle \Rightarrow_G^* w$ mit Induktion über i .

Die akzeptierte Sprache von PDAs ist kontextfrei

- ← Sei $(z', w, A) \vdash_M^i (z, \varepsilon, \varepsilon)$. Wir zeigen $\langle z', A, z \rangle \Rightarrow_G^* w$ mit Induktion über i .
- Fall $i = 0$: Unmöglich.

Die akzeptierte Sprache von PDAs ist kontextfrei

- ← Sei $(z', w, A) \vdash_M^i (z, \varepsilon, \varepsilon)$. Wir zeigen $\langle z', A, z \rangle \Rightarrow_G^* w$ mit Induktion über i .
- ▶ Fall $i = 0$: Unmöglich.
 - ▶ Fall $i = 1$: Dann gilt $w = a \in \Sigma \cup \{\varepsilon\}$ und $(z, \varepsilon) \in \delta(z', a, A)$.
Damit gibt es $\langle z', A, z \rangle \rightarrow a \in P$ und daher $\langle z', A, z \rangle \Rightarrow_G a$.

Die akzeptierte Sprache von PDAs ist kontextfrei

- ← Sei $(z', w, A) \vdash_M^i (z, \varepsilon, \varepsilon)$. Wir zeigen $\langle z', A, z \rangle \Rightarrow_G^* w$ mit Induktion über i .
- ▶ Fall $i = 0$: Unmöglich.
 - ▶ Fall $i = 1$: Dann gilt $w = a \in \Sigma \cup \{\varepsilon\}$ und $(z, \varepsilon) \in \delta(z', a, A)$.
Damit gibt es $\langle z', A, z \rangle \rightarrow a \in P$ und daher $\langle z', A, z \rangle \Rightarrow_G a$.
 - ▶ Fall $i > 1$: Dann $w = aw'$, $(z', aw', A) \vdash (z'', w', W) \vdash_M^{i-1} (z, \varepsilon, \varepsilon)$ für $i - 1 > 0$,
 $a \in \Sigma \cup \{\varepsilon\}$ und $W = \varepsilon$, $W = B$ oder $W = BC$.

Die akzeptierte Sprache von PDAs ist kontextfrei

- $\Leftarrow$ Sei $(z', w, A) \vdash_M^i (z, \varepsilon, \varepsilon)$. Wir zeigen $\langle z', A, z \rangle \Rightarrow_G^* w$ mit Induktion über i .
- ▶ Fall $i = 0$: Unmöglich.
 - ▶ Fall $i = 1$: Dann gilt $w = a \in \Sigma \cup \{\varepsilon\}$ und $(z, \varepsilon) \in \delta(z', a, A)$.
Damit gibt es $\langle z', A, z \rangle \rightarrow a \in P$ und daher $\langle z', A, z \rangle \Rightarrow_G a$.
 - ▶ Fall $i > 1$: Dann $w = aw'$, $(z', aw', A) \vdash_M^{i-1} (z, \varepsilon, \varepsilon)$ für $i - 1 > 0$,
 $a \in \Sigma \cup \{\varepsilon\}$ und $W = \varepsilon$, $W = B$ oder $W = BC$.

Wir betrachten alle drei Fälle für W einzeln:

- ▶ Fall $W = \varepsilon$: Dieser Fall ist nicht möglich, da $i - 1 > 0$ nicht gelten kann.

Die akzeptierte Sprache von PDAs ist kontextfrei

$\Leftarrow$ Sei $(z', w, A) \vdash_M^i (z, \varepsilon, \varepsilon)$. Wir zeigen $\langle z', A, z \rangle \Rightarrow_G^* w$ mit Induktion über i .

- ▶ Fall $i = 0$: Unmöglich.
- ▶ Fall $i = 1$: Dann gilt $w = a \in \Sigma \cup \{\varepsilon\}$ und $(z, \varepsilon) \in \delta(z', a, A)$.
Damit gibt es $\langle z', A, z \rangle \rightarrow a \in P$ und daher $\langle z', A, z \rangle \Rightarrow_G a$.
- ▶ Fall $i > 1$: Dann $w = aw'$, $(z', aw', A) \vdash (z'', w', W) \vdash_M^{i-1} (z, \varepsilon, \varepsilon)$ für $i - 1 > 0$,
 $a \in \Sigma \cup \{\varepsilon\}$ und $W = \varepsilon$, $W = B$ oder $W = BC$.

Wir betrachten alle drei Fälle für W einzeln:

- ▶ Fall $W = \varepsilon$: Dieser Fall ist nicht möglich, da $i - 1 > 0$ nicht gelten kann.
- ▶ Fall $W = B$: Dann ist $\langle z', A, z \rangle \rightarrow a \langle z'', B, z \rangle \in P$.
Da $(z'', w', B) \vdash_M^{i-1} (z, \varepsilon, \varepsilon)$, liefert die Induktionshypothese $\langle z'', B, z \rangle \Rightarrow_G^* w'$
und daher: $\langle z', A, z \rangle \Rightarrow_G a \langle z'', B, z \rangle \Rightarrow_G^* aw' = w$.

Die akzeptierte Sprache von PDAs ist kontextfrei

← Sei $\langle z', w, A \rangle \vdash_M^i (z, \varepsilon, \varepsilon)$. Wir zeigen $\langle z', A, z \rangle \Rightarrow_G^* w$ mit Induktion über i .

- ▶ Fall $i = 0$: Unmöglich.
- ▶ Fall $i = 1$: Dann gilt $w = a \in \Sigma \cup \{\varepsilon\}$ und $(z, \varepsilon) \in \delta(z', a, A)$.
Damit gibt es $\langle z', A, z \rangle \rightarrow a \in P$ und daher $\langle z', A, z \rangle \Rightarrow_G a$.
- ▶ Fall $i > 1$: Dann $w = aw'$, $\langle z', aw', A \rangle \vdash (z'', w', W) \vdash_M^{i-1} (z, \varepsilon, \varepsilon)$ für $i - 1 > 0$,
 $a \in \Sigma \cup \{\varepsilon\}$ und $W = \varepsilon$, $W = B$ oder $W = BC$.

Wir betrachten alle drei Fälle für W einzeln:

- ▶ Fall $W = \varepsilon$: Dieser Fall ist nicht möglich, da $i - 1 > 0$ nicht gelten kann.
- ▶ Fall $W = B$: Dann ist $\langle z', A, z \rangle \rightarrow a \langle z'', B, z \rangle \in P$.
Da $\langle z'', w', B \rangle \vdash_M^{i-1} (z, \varepsilon, \varepsilon)$, liefert die Induktionshypothese $\langle z'', B, z \rangle \Rightarrow_G^* w'$
und daher: $\langle z', A, z \rangle \Rightarrow_G a \langle z'', B, z \rangle \Rightarrow_G^* aw' = w$.
- ▶ Fall $W = BC$: Dieser Fall ist komplizierter aber ähnlich (siehe Skript). □

Äquivalenz von kontextfreien Sprachen und von Kellerautomaten

Theorem

Kellerautomaten erkennen genau die kontextfreien Sprachen.

Äquivalenz von kontextfreien Sprachen und von Kellerautomaten

Theorem

Kellerautomaten erkennen genau die kontextfreien Sprachen.

Beweis Dies folgt aus den obigen Sätzen.



Ausdruckskraft von PDAs mit einem Zustand

Die bisherigen Beweise zeigen auch, dass man PDAs auf PDAs mit **genau einem Zustand** einschränken kann.

Ausdruckskraft von PDAs mit einem Zustand

Die bisherigen Beweise zeigen auch, dass man PDAs auf PDAs mit **genau einem Zustand** einschränken kann.

Sei M ein PDA.

1. Transformiere PDA M in Grammatik G mit $L(G) = L(M)$.
2. Transformiere Grammatik G in Grammatik G' in Greibach-Normalform (mit $L(G') = L(G) \setminus \{\epsilon\}$).
3. Transformiere Grammatik G' in PDA M' mit $L(M') = L(G)$.

PDA M' hat nur einen Zustand.

7b

Deterministische Kellerautomaten

Prof. Dr. Jasmin Blanchette

Lehr- und Forschungseinheit für
Theoretische Informatik und Theorembeweisen

Stand: 4. Juni 2024
Basierend auf Folien von PD Dr. David Sabel



DPDAs und deterministisch kontextfreie Sprachen

- ▶ Deterministisch kontextfreie Sprachen sind definiert durch **deterministische** Kellerautomaten mit **Akzeptanz durch Endzustände**. (Akzeptanz durch leeren Keller wäre schwächer.)
- ▶ **ϵ -Übergänge** sind erlaubt, aber nur wenn es keinen anderen Übergang (mit demselben Kellersymbol und einem beliebigen Terminalzeichen) gibt.

DPDAs und deterministisch kontextfreie Sprachen

- ▶ Deterministisch kontextfreie Sprachen sind definiert durch **deterministische** Kellerautomaten mit **Akzeptanz durch Endzustände**. (Akzeptanz durch leeren Keller wäre schwächer.)
- ▶ **ϵ -Übergänge** sind erlaubt, aber nur wenn es keinen anderen Übergang (mit demselben Kellersymbol und einem beliebigen Terminalzeichen) gibt.

Definition

Ein Kellerautomat mit Endzuständen $M = (Z, \Sigma, \Gamma, \delta, z_0, \#, E)$ ist **deterministisch** (ein **DPDA**) wenn für alle $(z, a, A) \in Z \times \Sigma \times \Gamma$ gilt

$$|\delta(z, a, A)| + |\delta(z, \epsilon, A)| \leq 1$$

Die von DPDAs akzeptierten Sprachen heißen **deterministisch kontextfrei**.

Beispiel für einen DPDA

Satz

Die Sprache $\{w\$ \overline{w} \mid w \in \{a, b\}^*\}$ ist deterministisch kontextfrei.

Beispiel für einen DPDA

Satz

Die Sprache $\{w\$ \overline{w} \mid w \in \{a, b\}^*\}$ ist deterministisch kontextfrei.

Beweis Betrachte den DPDA $M = (\{z_0, z_1, z_2\}, \{a, b, \$\}, \{\#, A, B\}, \delta, z_0, \#, \{z_2\})$ mit

$$\delta(z_0, a, \#) = \{(z_0, A\#)\}$$

$$\delta(z_0, b, \#) = \{(z_0, B\#)\}$$

$$\delta(z_0, a, A) = \{(z_0, AA)\}$$

$$\delta(z_0, b, A) = \{(z_0, BA)\}$$

$$\delta(z_0, a, B) = \{(z_0, AB)\}$$

$$\delta(z_0, b, B) = \{(z_0, BB)\}$$

$$\delta(z_0, \$, \#) = \{(z_1, \#)\}$$

$$\delta(z_0, \$, A) = \{(z_1, A)\}$$

$$\delta(z_0, \$, B) = \{(z_1, B)\}$$

$$\delta(z_1, a, A) = \{(z_1, \varepsilon)\}$$

$$\delta(z_1, b, B) = \{(z_1, \varepsilon)\}$$

$$\delta(z_1, \varepsilon, \#) = \{(z_2, \varepsilon)\}$$

und $\delta(z_i, c, C) = \emptyset$ sonst (für $c \in \{a, b, \$, \varepsilon\}$).



Beispiel für einen DPDA

Satz

Die Sprache $\{w\$ \overline{w} \mid w \in \{a, b\}^*\}$ ist deterministisch kontextfrei.

Beweis Betrachte den DPDA $M = (\{z_0, z_1, z_2\}, \{a, b, \$\}, \{\#, A, B\}, \delta, z_0, \#, \{z_2\})$ mit

$$\delta(z_0, a, \#) = \{(z_0, A\#)\}$$

$$\delta(z_0, \$, \#) = \{(z_1, \#)\}$$

$$\delta(z_0, b, \#) = \{(z_0, B\#)\}$$

$$\delta(z_0, \$, A) = \{(z_1, A)\}$$

$$\delta(z_0, a, A) = \{(z_0, AA)\}$$

$$\delta(z_0, \$, B) = \{(z_1, B)\}$$

$$\delta(z_0, b, A) = \{(z_0, BA)\}$$

$$\delta(z_1, a, A) = \{(z_1, \varepsilon)\}$$

$$\delta(z_0, a, B) = \{(z_0, AB)\}$$

$$\delta(z_1, b, B) = \{(z_1, \varepsilon)\}$$

$$\delta(z_0, b, B) = \{(z_0, BB)\}$$

$$\delta(z_1, \varepsilon, \#) = \{(z_2, \varepsilon)\}$$

und $\delta(z_i, c, C) = \emptyset$ sonst (für $c \in \{a, b, \$, \varepsilon\}$).

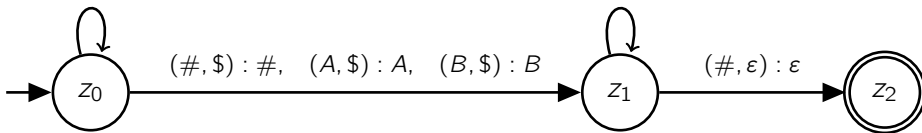
□

Beachte: $\{w\overline{w} \mid w \in \{a, b\}^*\}$ ist **nicht** deterministisch kontextfrei aber kontextfrei.

Beispiel für einen DPDA

$(\#, a) : A\#$, $(\#, b) : B\#$,
 $(A, a) : AA$, $(A, b) : BA$,
 $(B, a) : AB$, $(B, b) : BB$

$(A, a) : \epsilon$,
 $(B, b) : \epsilon$



Weiteres Beispiel für einen DPDA

Satz

Die Sprache $\{a^i b^i \mid i \in \mathbb{N}_{>0}\}$ ist deterministisch kontextfrei.

Weiteres Beispiel für einen DPDA

Satz

Die Sprache $\{a^i b^i \mid i \in \mathbb{N}_{>0}\}$ ist deterministisch kontextfrei.

Beweis: Betrachte den DPDA $M = (\{z_0, z_1, z_2\}, \{a, b\}, \{\#, B\}, \delta, z_0, \#, \{z_2\})$ mit

$$\delta(z_0, a, \#) = \{(z_0, B\#)\}$$

$$\delta(z_0, a, B) = \{(z_0, BB)\}$$

$$\delta(z_0, b, B) = \{(z_1, \varepsilon)\}$$

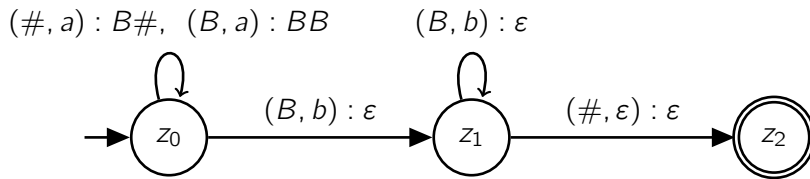
$$\delta(z_1, b, B) = \{(z_1, \varepsilon)\}$$

$$\delta(z_1, \varepsilon, \#) = \{(z_2, \varepsilon)\}$$

und $\delta(z_i, c, A) = \emptyset$ sonst (für $c \in \{a, b, \varepsilon\}$).



Weiteres Beispiel für einen DPDA



Theorem

1. Für deterministisch kontextfreie Sprachen gibt es eindeutige Grammatiken.
2. Deterministisch kontextfreie Sprachen sind unter Komplementbildung abgeschlossen.

Theorem

1. Für deterministisch kontextfreie Sprachen gibt es eindeutige Grammatiken.
2. Deterministisch kontextfreie Sprachen sind unter Komplementbildung abgeschlossen.

Beweis Siehe Literatur.



Weitere Eigenschaften deterministischer kontextfreier Sprachen

Satz

Die deterministischen kontextfreien Sprachen sind **nicht** abgeschlossen bezüglich Vereinigung und Schnitt.

Beweis

- **Schnitt:** Die Sprachen $L_1 = \{a^n b^n c^m \mid n, m \in \mathbb{N}_{>0}\}$ und $L_2 = \{a^n b^m c^m \mid n, m \in \mathbb{N}_{>0}\}$ sind deterministisch kontextfrei.

Satz

Die deterministischen kontextfreien Sprachen sind **nicht** abgeschlossen bezüglich Vereinigung und Schnitt.

Beweis

- **Schnitt:** Die Sprachen $L_1 = \{a^n b^n c^m \mid n, m \in \mathbb{N}_{>0}\}$ und $L_2 = \{a^n b^m c^m \mid n, m \in \mathbb{N}_{>0}\}$ sind deterministisch kontextfrei.
 $L_1 \cap L_2 = \{a^n b^n c^n \mid n \in \mathbb{N}_{>0}\}$ ist nicht kontextfrei.

Weitere Eigenschaften deterministischer kontextfreier Sprachen

Satz

Die deterministischen kontextfreien Sprachen sind **nicht** abgeschlossen bezüglich Vereinigung und Schnitt.

Beweis

- ▶ **Vereinigung:** Durch Widerspruch. Nehme an, die deterministischen kontextfreien Sprachen sind abgeschlossen bezüglich Vereinigung.

Weitere Eigenschaften deterministischer kontextfreier Sprachen

Satz

Die deterministischen kontextfreien Sprachen sind **nicht** abgeschlossen bezüglich Vereinigung und Schnitt.

Beweis

- **Vereinigung:** Durch Widerspruch. Nehme an, die deterministischen kontextfreien Sprachen sind abgeschlossen bezüglich Vereinigung.

Da die deterministischen kontextfreien Sprachen auch abgeschlossen bezüglich Komplement sind, folgt aus $L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}$, dass die deterministischen kontextfreien Sprachen abgeschlossen bezüglich Schnitt sind. Widerspruch. $\square$

Weitere Eigenschaften deterministischer kontextfreier Sprachen

Satz

Der Schnitt einer kontextfreien Sprachen L mit einer regulären Sprache L' ist kontextfrei. Wenn L deterministisch kontextfrei ist, dann ist der Schnitt das auch.

Weitere Eigenschaften deterministischer kontextfreier Sprachen

Satz

Der Schnitt einer kontextfreien Sprachen L mit einer regulären Sprache L' ist kontextfrei. Wenn L deterministisch kontextfrei ist, dann ist der Schnitt das auch.

Beweis Sei $M = (Z, \Sigma, \Gamma, \delta, z_0, \#, E)$ ein PDA mit Endzuständen mit $L(M) = L$ und $M' = (Z', \Sigma, \delta', z'_0, E')$ ein DFA mit $L(M') = L'$. Konstruiere PDA mit Endzuständen: $M'' = (Z \times Z', \Sigma, \delta'', (z_0, z'_0), \#, E \times E')$ mit

- ▶ $((z_k, z'_k), B_1 \cdots B_m) \in \delta''((z_i, z'_i), a, A)$ falls $(z_k, B_1 \cdots B_m) \in \delta(z_i, a, A)$ und $\delta'(z'_i, a) = z'_k$
- ▶ $((z_k, z'_i), B_1 \cdots B_m) \in \delta''((z_i, z'_i), \varepsilon, A)$ falls $(z_k, B_1 \cdots B_m) \in \delta(z_i, \varepsilon, A)$.

Weitere Eigenschaften deterministischer kontextfreier Sprachen

Satz

Der Schnitt einer kontextfreien Sprachen L mit einer regulären Sprache L' ist kontextfrei. Wenn L deterministisch kontextfrei ist, dann ist der Schnitt das auch.

Beweis Sei $M = (Z, \Sigma, \Gamma, \delta, z_0, \#, E)$ ein PDA mit Endzuständen mit $L(M) = L$ und $M' = (Z', \Sigma, \delta', z'_0, E')$ ein DFA mit $L(M') = L'$. Konstruiere PDA mit Endzuständen: $M'' = (Z \times Z', \Sigma, \delta'', (z_0, z'_0), \#, E \times E')$ mit

- ▶ $((z_k, z'_k), B_1 \cdots B_m) \in \delta''((z_i, z'_i), a, A)$ falls $(z_k, B_1 \cdots B_m) \in \delta(z_i, a, A)$ und $\delta'(z'_i, a) = z'_k$
 - ▶ $((z_k, z'_i), B_1 \cdots B_m) \in \delta''((z_i, z'_i), \varepsilon, A)$ falls $(z_k, B_1 \cdots B_m) \in \delta(z_i, \varepsilon, A)$.
1. $L(M'') = L(M) \cap L(M')$, denn M'' simuliert M und M' gleichzeitig, und akzeptiert nur, wenn beide Automaten akzeptieren.

Weitere Eigenschaften deterministischer kontextfreier Sprachen

Satz

Der Schnitt einer kontextfreien Sprachen L mit einer regulären Sprache L' ist kontextfrei. Wenn L deterministisch kontextfrei ist, dann ist der Schnitt das auch.

Beweis Sei $M = (Z, \Sigma, \Gamma, \delta, z_0, \#, E)$ ein PDA mit Endzuständen mit $L(M) = L$ und $M' = (Z', \Sigma, \delta', z'_0, E')$ ein DFA mit $L(M') = L'$. Konstruiere PDA mit Endzuständen: $M'' = (Z \times Z', \Sigma, \delta'', (z_0, z'_0), \#, E \times E')$ mit

- ▶ $((z_k, z'_k), B_1 \cdots B_m) \in \delta''((z_i, z'_i), a, A)$ falls $(z_k, B_1 \cdots B_m) \in \delta(z_i, a, A)$ und $\delta'(z'_i, a) = z'_k$
 - ▶ $((z_k, z'_i), B_1 \cdots B_m) \in \delta''((z_i, z'_i), \varepsilon, A)$ falls $(z_k, B_1 \cdots B_m) \in \delta(z_i, \varepsilon, A)$.
1. $L(M'') = L(M) \cap L(M')$, denn M'' simuliert M und M' gleichzeitig, und akzeptiert nur, wenn beide Automaten akzeptieren.
 2. M'' ist deterministisch, wenn M deterministisch ist. □

Entscheidbarkeitsproblem deterministischer kontextfreier Sprachen

Satz

Das Problem, ob eine deterministisch kontextfreie Sprache äquivalent zu einer regulären Sprache ist, ist entscheidbar.

Entscheidbarkeitsproblem deterministischer kontextfreier Sprachen

Satz

Das Problem, ob eine deterministisch kontextfreie Sprache äquivalent zu einer regulären Sprache ist, ist entscheidbar.

Beweis Sei L_1 durch einen DPDA und L_2 durch einen DFA gegeben. Um zu zeigen, dass $L_1 = L_2$, reicht es zu zeigen, dass $L_1 \subseteq L_2$ und $L_2 \subseteq L_1$.

Entscheidbarkeitsproblem deterministischer kontextfreier Sprachen

Satz

Das Problem, ob eine deterministisch kontextfreie Sprache äquivalent zu einer regulären Sprache ist, ist entscheidbar.

Beweis Sei L_1 durch einen DPDA und L_2 durch einen DFA gegeben. Um zu zeigen, dass $L_1 = L_2$, reicht es zu zeigen, dass $L_1 \subseteq L_2$ und $L_2 \subseteq L_1$.

Dies entspricht $L_1 \cap \overline{L_2} = \emptyset$ und $\overline{L_1} \cap L_2 = \emptyset$.

Entscheidbarkeitsproblem deterministischer kontextfreier Sprachen

Satz

Das Problem, ob eine deterministisch kontextfreie Sprache äquivalent zu einer regulären Sprache ist, ist entscheidbar.

Beweis Sei L_1 durch einen DPDA und L_2 durch einen DFA gegeben. Um zu zeigen, dass $L_1 = L_2$, reicht es zu zeigen, dass $L_1 \subseteq L_2$ und $L_2 \subseteq L_1$.

Dies entspricht $L_1 \cap \overline{L_2} = \emptyset$ und $\overline{L_1} \cap L_2 = \emptyset$.

Beides ist entscheidbar, da

- ▶ DPDAs und DFAs abgeschlossen unter Komplementbildung sind
- ▶ die Schnittbildung zwischen einem DPDA und einem DFA durch einen DPDA konstruierbar ist
- ▶ das Leerheitsproblem für kontextfreie Grammatiken entscheidbar ist. □

7c

Turingmaschinen

Prof. Dr. Jasmin Blanchette

Lehr- und Forschungseinheit für
Theoretische Informatik und Theorembeweisen

Stand: 4. Juni 2024
Basierend auf Folien von PD Dr. David Sabel



Wiederholung: Typ 1- und Typ 0-Grammatiken

- ▶ Im Gegensatz zu Typ 2 ist die Linke Seite von Typ 0- und Typ 1-Produktionen eine Satzform $\ell \in (V \cup \Sigma)^+$.
- ▶ Mit Typ 1-Grammatiken gilt $|\ell| \leq |r|$ für alle Produktionen $\ell \rightarrow r$.
- ▶ Typ 1-Grammatiken nennen wir auch **kontextsensitiv**.

Hintergrund zu Turingmaschinen

Nun führen wir ein Maschinenmodell ein, das zu Typ 1 und Typ 0 passt:
die **Turingmaschinen** (für Typ 1: mit Einschränkungen).

Hintergrund zu Turingmaschinen

Nun führen wir ein Maschinenmodell ein, das zu Typ 1 und Typ 0 passt: die **Turingmaschinen** (für Typ 1: mit Einschränkungen).

Einschränkungen der Kellerautomaten:

- ▶ PDAs erkennen genau die **kontextfreien Sprachen**, daher müssen Automaten für Typ 1- und Typ 0-Sprachen „mehr können“.
- ▶ Wesentliche Beschränkung bei PDAs:
Der Zugriff auf den Speicher ist **nur von oben möglich**.

Hintergrund zu Turingmaschinen

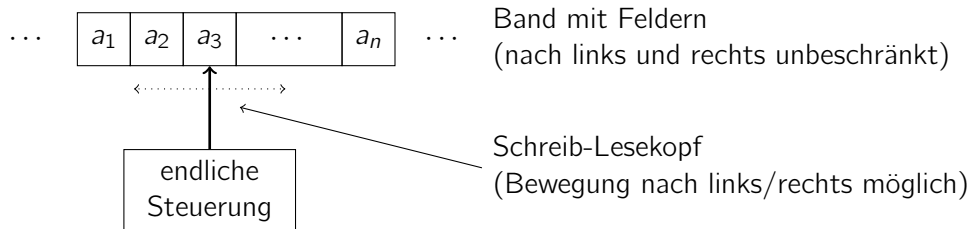
Nun führen wir ein Maschinenmodell ein, das zu Typ 1 und Typ 0 passt: die **Turingmaschinen** (für Typ 1: mit Einschränkungen).

Einschränkungen der Kellerautomaten:

- ▶ PDAs erkennen genau die **kontextfreien Sprachen**, daher müssen Automaten für Typ 1- und Typ 0-Sprachen „mehr können“.
- ▶ Wesentliche Beschränkung bei PDAs:
Der Zugriff auf den Speicher ist **nur von oben möglich**.
- ▶ Z.B. kann man $\{a^i b^i c^i \mid i \in \mathbb{N}_{>0}\}$ nicht mit einem PDA erkennen, da man die Anzahl i
 - ▶ beim Lesen der a 's im Keller speichert
 - ▶ beim Lesen der b 's vergleichen muss und das geht nur durch sukzessives Entnehmen aus dem Keller
 - ▶ beim Lesen der c 's nicht mehr hat.

Mit **beliebigem Lesen** des Speichers wäre es kein Problem, $a^i b^i c^i$ zu erkennen.

Illustration einer Turingmaschine



Definition einer Turingmaschine

Definition

Eine **Turingmaschine** (TM) ist ein 7-Tupel $M = (Z, \Sigma, \Gamma, \delta, z_0, \square, E)$, wobei:

- ▶ Z ist eine endliche Menge von **Zuständen**
- ▶ Σ ist das (endliche) **Eingabealphabet**
- ▶ $\Gamma \supset \Sigma$ ist das (endliche) **Bandalphabet**
- ▶ δ ist die **Überföhrungsfunktion**
 - ▶ für **deterministische TM** (DTM): $\delta : (Z \setminus E) \times \Gamma \rightarrow Z \times \Gamma \times \{L, R, N\}$
 - ▶ für **nichtdeterministische TM** (NTM): $\delta : (Z \setminus E) \times \Gamma \rightarrow \mathcal{P}(Z \times \Gamma \times \{L, R, N\})$
- ▶ $z_0 \in Z$ ist der **Startzustand**
- ▶ $\square \in \Gamma \setminus \Sigma$ ist das **Blank-Symbol**
- ▶ $E \subseteq Z$ ist die Menge der **Endzustände**.

Zustandsübergang

Für deterministische Turingmaschinen:

- ▶ Ein Eintrag $\delta(z, a) = (z', b, x)$ bedeutet: Falls die TM im Zustand z ist und das Zeichen a an der aktuellen Position des Schreib-Lesekopfs ist, dann
 1. Wechsle in Zustand z' .
 2. Ersetze a durch b auf dem Band.
 3. Falls $x = L$: Verschiebe den Schreib-Lesekopf eine Position nach links.
 4. Falls $x = R$: Verschiebe den Schreib-Lesekopf eine Position nach rechts.
 5. Falls $x = N$: Lasse Schreib-Lesekopf unverändert (Neutral).

Zustandsübergang

Für deterministische Turingmaschinen:

- ▶ Ein Eintrag $\delta(z, a) = (z', b, x)$ bedeutet: Falls die TM im Zustand z ist und das Zeichen a an der aktuellen Position des Schreib-Lesekopfs ist, dann
 1. Wechsle in Zustand z' .
 2. Ersetze a durch b auf dem Band.
 3. Falls $x = L$: Verschiebe den Schreib-Lesekopf eine Position nach links.
 4. Falls $x = R$: Verschiebe den Schreib-Lesekopf eine Position nach rechts.
 5. Falls $x = N$: Lasse Schreib-Lesekopf unverändert (Neutral).

Für nichtdeterministische Turingmaschinen:

- ▶ $\delta(z, a)$ ist eine Menge solcher möglichen Schritte und die NTM macht in einem Lauf irgendeinen davon (nichtdeterministisch).

Definition

Sei $M = (Z, \Sigma, \Gamma, \delta, z_0, \square, E)$ eine Turingmaschine.

Eine **Konfiguration** von M ist ein Wort $wzw' \in \Gamma^* Z \Gamma^*$, wobei:

- ▶ z ist der aktuelle Zustand von M
- ▶ $\dots \square \square ww' \square \square \dots$ steht auf dem Band
- ▶ der Schreib-Lesekopf steht auf dem ersten Symbol von w' .

Definition

Sei $M = (Z, \Sigma, \Gamma, \delta, z_0, \square, E)$ eine Turingmaschine.

Eine **Konfiguration** von M ist ein Wort $wzw' \in \Gamma^* Z \Gamma^*$, wobei:

- ▶ z ist der aktuelle Zustand von M
- ▶ $\dots \square \square ww' \square \square \dots$ steht auf dem Band
- ▶ der Schreib-Lesekopf steht auf dem ersten Symbol von w' .

Definition

Sei $M = (Z, \Sigma, \Gamma, \delta, z_0, \square, E)$ eine Turingmaschine.

Für ein Eingabewort w ist die **Startkonfiguration** $Start_M(w)$ von M das Wort $z_0 w$.

Im Spezialfall $w = \varepsilon$ ist die Startkonfiguration $z_0 \square$.

D.h. am Anfang steht der Schreib-Lesekopf auf dem ersten Symbol der Eingabe.

Übergangsrelation einer deterministischen Turingmaschine

Definition

Sei $M = (Z, \Sigma, \Gamma, \delta, z_0, \square, E)$ eine deterministische Turingmaschine.

Die Relation $\vdash_M \subseteq \Gamma^* Z \Gamma^* \times \Gamma^* Z \Gamma^*$ ist definiert durch

- ▶ $b_1 \cdots b_m z a_1 \cdots a_n \vdash_M b_1 \cdots b_m z' c a_2 \cdots a_n$,
wenn $\delta(z, a_1) = (z', c, N)$, $m \geq 0$, $n \geq 1$ und $z \notin E$
- ▶ $b_1 \cdots b_m z a_1 \cdots a_n \vdash_M b_1 \cdots b_{m-1} z' b_m c a_2 \cdots a_n$,
wenn $\delta(z, a_1) = (z', c, L)$, $m \geq 1$, $n \geq 1$ und $z \notin E$
- ▶ $b_1 \cdots b_m z a_1 \cdots a_n \vdash_M b_1 \cdots b_m c z' a_2 \cdots a_n$,
wenn $\delta(z, a_1) = (z', c, R)$, $m \geq 0$, $n \geq 2$ und $z \notin E$

Übergangsrelation einer deterministischen Turingmaschine

Definition

Sei $M = (Z, \Sigma, \Gamma, \delta, z_0, \square, E)$ eine deterministische Turingmaschine.

Die Relation $\vdash_M \subseteq \Gamma^* Z \Gamma^* \times \Gamma^* Z \Gamma^*$ ist definiert durch

- ▶ $b_1 \cdots b_m z a_1 \cdots a_n \vdash_M b_1 \cdots b_m z' c a_2 \cdots a_n$,
wenn $\delta(z, a_1) = (z', c, N)$, $m \geq 0$, $n \geq 1$ und $z \notin E$
- ▶ $b_1 \cdots b_m z a_1 \cdots a_n \vdash_M b_1 \cdots b_{m-1} z' b_m c a_2 \cdots a_n$,
wenn $\delta(z, a_1) = (z', c, L)$, $m \geq 1$, $n \geq 1$ und $z \notin E$
- ▶ $b_1 \cdots b_m z a_1 \cdots a_n \vdash_M b_1 \cdots b_m c z' a_2 \cdots a_n$,
wenn $\delta(z, a_1) = (z', c, R)$, $m \geq 0$, $n \geq 2$ und $z \notin E$
- ▶ $b_1 \cdots b_m z a_1 \vdash_M b_1 \cdots b_m c z' \square$, wenn $\delta(z, a_1) = (z', c, R)$, $m \geq 0$ und $z \notin E$
- ▶ $z a_1 \cdots a_n \vdash_M z' \square c a_2 \cdots a_n$, wenn $\delta(z, a_1) = (z', c, L)$, $n \geq 1$ und $z \notin E$.

Übergangsrelation einer nichtdeterministischen Turingmaschine

Definition

Sei $M = (Z, \Sigma, \Gamma, \delta, z_0, \square, E)$ eine nichtdeterministische Turingmaschine.

Die Relation $\vdash_M \subseteq \Gamma^* Z \Gamma^* \times \Gamma^* Z \Gamma^*$ ist definiert durch

- ▶ $b_1 \cdots b_m z a_1 \cdots a_n \vdash_M b_1 \cdots b_m z' c a_2 \cdots a_n$,
wenn $(z', c, N) \in \delta(z, a_1)$, $m \geq 0$, $n \geq 1$ und $z \notin E$
- ▶ $b_1 \cdots b_m z a_1 \cdots a_n \vdash_M b_1 \cdots b_{m-1} z' b_m c a_2 \cdots a_n$,
wenn $(z', c, L) \in \delta(z, a_1)$, $m \geq 1$, $n \geq 1$ und $z \notin E$
- ▶ $b_1 \cdots b_m z a_1 \cdots a_n \vdash_M b_1 \cdots b_m c z' a_2 \cdots a_n$,
wenn $(z', c, R) \in \delta(z, a_1)$, $m \geq 0$, $n \geq 2$ und $z \notin E$
- ▶ $b_1 \cdots b_m z a_1 \vdash_M b_1 \cdots b_m c z' \square$, wenn $(z', c, R) \in \delta(z, a_1)$, $m \geq 0$ und $z \notin E$
- ▶ $z a_1 \cdots a_n \vdash_M z' \square c a_2 \cdots a_n$, wenn $(z', c, L) \in \delta(z, a_1)$, $n \geq 1$ und $z \notin E$.

Übergangsrelation einer Turingmaschine

Weitere Notationen:

- ▶ $\vdash_M^*$ ist die reflexiv-transitive Hülle von $\vdash_M$.
- ▶ $\vdash_M^i$ ist die i -fache Anwendung von $\vdash_M$.
- ▶ Wenn M eindeutig ist, schreiben wir $\vdash$ statt $\vdash_M$.

Übergangsrelation einer Turingmaschine

Weitere Notationen:

- ▶ $\vdash_M^*$ ist die reflexiv-transitive Hülle von $\vdash_M$.
- ▶ $\vdash_M^i$ ist die i -fache Anwendung von $\vdash_M$.
- ▶ Wenn M eindeutig ist, schreiben wir $\vdash$ statt $\vdash_M$.

Bemerkung:

- ▶ Mit unserer Definition **hält** die Turingmaschine **an**, sobald sie einen Endzustand erreicht hat.
(Das Buch von Schöning erlaubt weiterrechnen.)

Akzeptierte Sprache einer Turingmaschine

Definition

Sei $M = (Z, \Sigma, \Gamma, \delta, z_0, \square, E)$ eine Turingmaschine.

Die von M akzeptierte Sprache $L(M)$ ist definiert als

$$L(M) := \{w \in \Sigma^* \mid \text{Start}_M(w) \vdash_M^* uzv \text{ für } u, v \in \Gamma^*, z \in E\}$$

Akzeptierte Sprache einer Turingmaschine

Definition

Sei $M = (Z, \Sigma, \Gamma, \delta, z_0, \square, E)$ eine Turingmaschine.

Die von M akzeptierte Sprache $L(M)$ ist definiert als

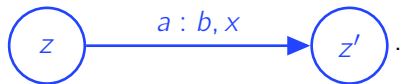
$$L(M) := \{w \in \Sigma^* \mid \text{Start}_M(w) \vdash_M^* uzv \text{ für } u, v \in \Gamma^*, z \in E\}$$

Weitere Begriffe:

- ▶ Die TM akzeptiert heißt, die TM erreicht einen Endzustand.
- ▶ Die TM verwirft heißt, die TM erreicht keinen Endzustand.

Notation als Zustandsgraph

- ▶ Die graphische Darstellung ist ähnlich der von DFAs, NFAs und PDAs.
- ▶ Für $\delta(z, a) = (z', b, x)$ bzw. $(z', b, x) \in \delta(z, a)$ zeichnen wir



- ▶ Beachte, dass das Blank-Symbol bekannt sein muss (üblicherweise $\square$).

Beispiel für eine deterministische Turingmaschine

DTM $M = (\{z_0, z_1, z_2, z_3\}, \{0, 1\}, \{0, 1, \square\}, \delta, z_0, \square, \{z_3\})$ mit

$\delta(z_0, 0) = (z_0, 0, R)$	$\delta(z_0, 1) = (z_0, 1, R)$	$\delta(z_0, \square) = (z_1, \square, L)$
$\delta(z_1, 0) = (z_2, 1, L)$	$\delta(z_1, 1) = (z_1, 0, L)$	$\delta(z_1, \square) = (z_3, 1, N)$
$\delta(z_2, 0) = (z_2, 0, L)$	$\delta(z_2, 1) = (z_2, 1, L)$	$\delta(z_2, \square) = (z_3, \square, R)$

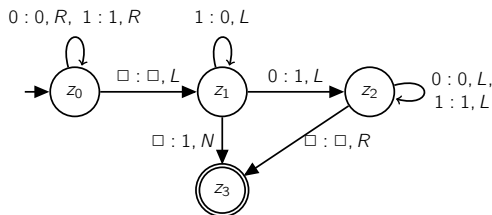
turingmachinesimulator.com/shared/istktezldr

Beispiel für eine deterministische Turingmaschine

DTM $M = (\{z_0, z_1, z_2, z_3\}, \{0, 1\}, \{0, 1, \square\}, \delta, z_0, \square, \{z_3\})$ mit

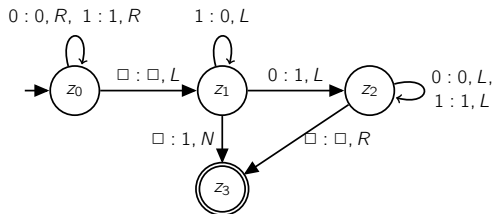
$$\begin{array}{lll} \delta(z_0, 0) = (z_0, 0, R) & \delta(z_0, 1) = (z_0, 1, R) & \delta(z_0, \square) = (z_1, \square, L) \\ \delta(z_1, 0) = (z_2, 1, L) & \delta(z_1, 1) = (z_1, 0, L) & \delta(z_1, \square) = (z_3, 1, N) \\ \delta(z_2, 0) = (z_2, 0, L) & \delta(z_2, 1) = (z_2, 1, L) & \delta(z_2, \square) = (z_3, \square, R) \end{array}$$

Zustandsgraph zu M :



turingmachinesimulator.com/shared/istktezldr

Beispiel für eine deterministische Turingmaschine



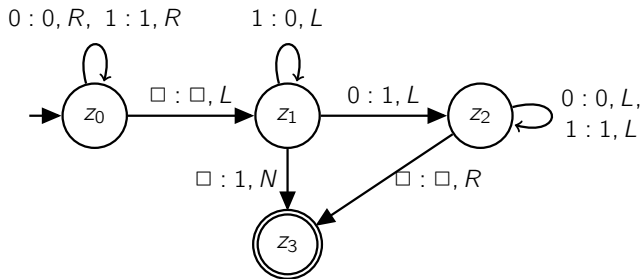
M interpretiert die Eingabe $w \in \{0, 1\}^*$ als Binärzahl und addiert 1:

- ▶ In z_0 wird das rechte Ende gesucht, dann in z_1 gewechselt.
- ▶ In z_1 wird versucht, 1 zur aktuellen Ziffer hinzu zu addieren:
Gelingt das ohne Übertrag, dann wird in z_2 gewechselt.
Bei Übertrag: Weitermachen in z_1 und +1 zur nächsten Ziffer links.
- ▶ In z_2 wird bis zum Anfang links gelaufen, dann in z_3 gewechselt.
- ▶ In z_3 wird akzeptiert.

Beispiel für einen akzeptierenden Lauf

DTM M :

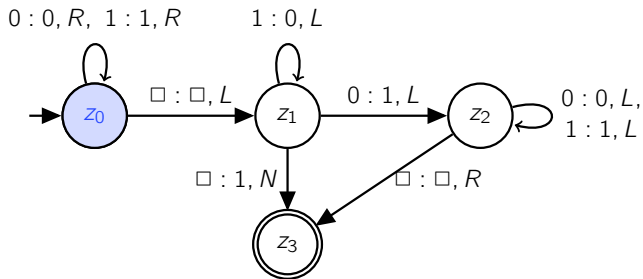
Eingabe: 0011



Beispiel für einen akzeptierenden Lauf

DTM M :

Eingabe: 0011

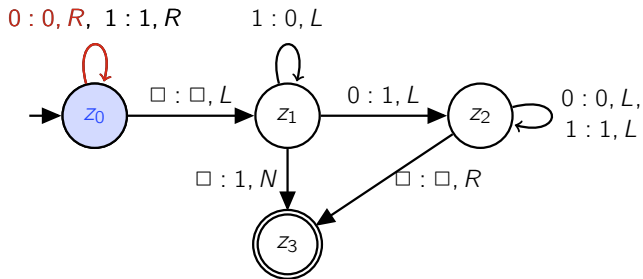


z_0 0011

Beispiel für einen akzeptierenden Lauf

DTM M :

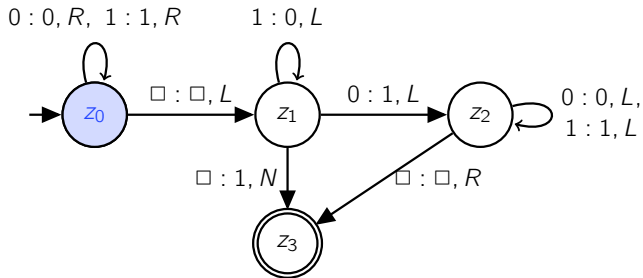
Eingabe: 0011



$z_0 0011$

Beispiel für einen akzeptierenden Lauf

DTM M :

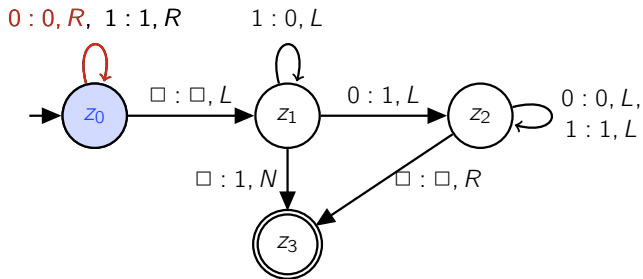


Eingabe: 0011

$z_0 0011$
 $\vdash 0 z_0 011$

Beispiel für einen akzeptierenden Lauf

DTM M :

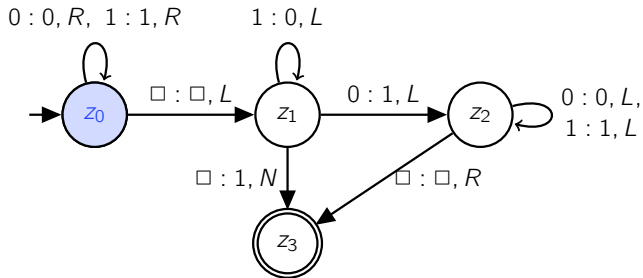


Eingabe: 0011

$z_0 0011$
 $\vdash 0z_0 011$

Beispiel für einen akzeptierenden Lauf

DTM M :

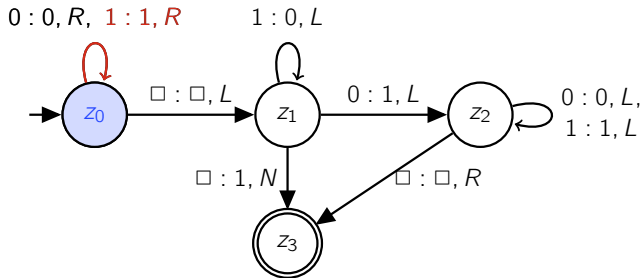


Eingabe: 0011

$z_0 0011$
 $\vdash 0z_0 011$
 $\vdash 00z_0 11$

Beispiel für einen akzeptierenden Lauf

DTM M :

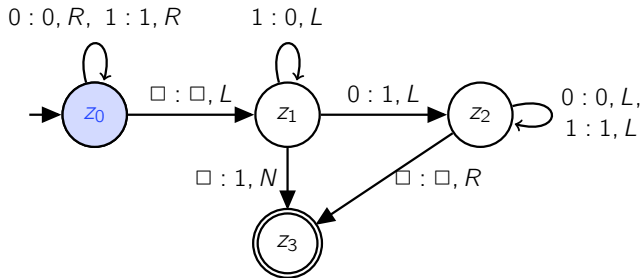


Eingabe: 0011

$z_0 0011$
 $\vdash 0z_0 011$
 $\vdash 00z_0 11$

Beispiel für einen akzeptierenden Lauf

DTM M :

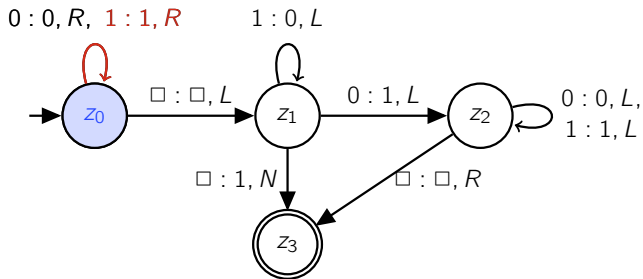


Eingabe: 0011

$z_0 0011$
 $\vdash 0z_0 011$
 $\vdash 00z_0 11$
 $\vdash 00\textcolor{blue}{z_0} 11$

Beispiel für einen akzeptierenden Lauf

DTM M :

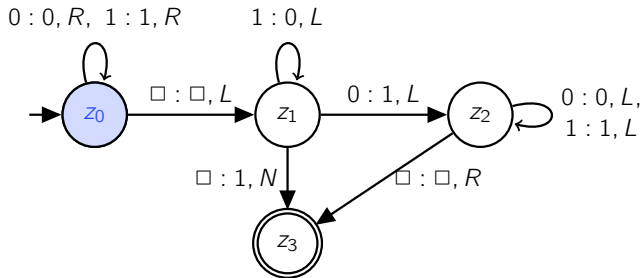


Eingabe: 0011

$z_0 0011$
 $\vdash 0z_0 011$
 $\vdash 00z_0 11$
 $\vdash 001z_0 1$

Beispiel für einen akzeptierenden Lauf

DTM M :

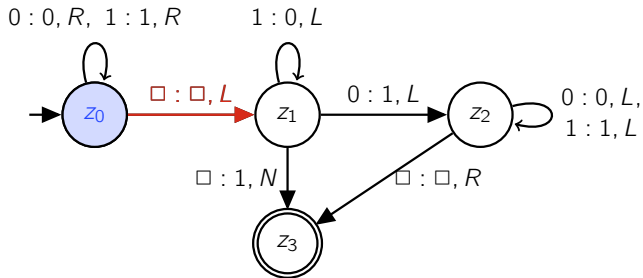


Eingabe: 0011

$z_0 0011$
 $\vdash 0z_0 011$
 $\vdash 00z_0 11$
 $\vdash 001z_0 1$
 $\vdash 0011z_0 \square$

Beispiel für einen akzeptierenden Lauf

DTM M :

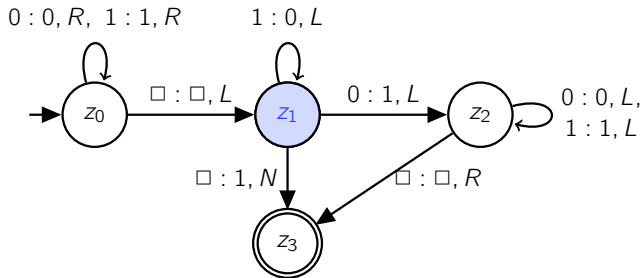


Eingabe: 0011

$z_0 0011$
 $\vdash 0z_0 011$
 $\vdash 00z_0 11$
 $\vdash 001z_0 1$
 $\vdash 0011z_0 \square$

Beispiel für einen akzeptierenden Lauf

DTM M :

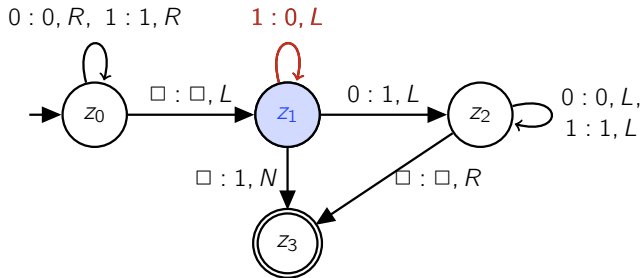


Eingabe: 0011

$z_0 0011$
 $\vdash 0z_0 011$
 $\vdash 00z_0 11$
 $\vdash 001z_0 1$
 $\vdash 0011z_0 \square$
 $\vdash 001z_1 1\square$

Beispiel für einen akzeptierenden Lauf

DTM M :

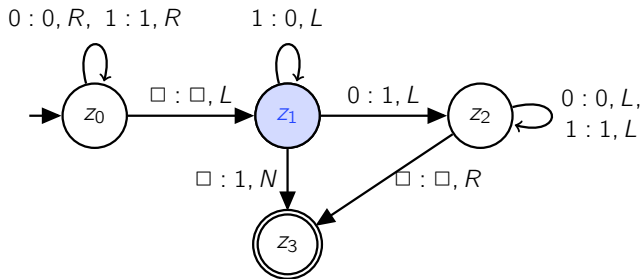


Eingabe: 0011

$z_0 0011$
 $\vdash 0z_0 011$
 $\vdash 00z_0 11$
 $\vdash 001z_0 1$
 $\vdash 0011z_0 \square$
 $\vdash 001z_1 1\square$

Beispiel für einen akzeptierenden Lauf

DTM M :

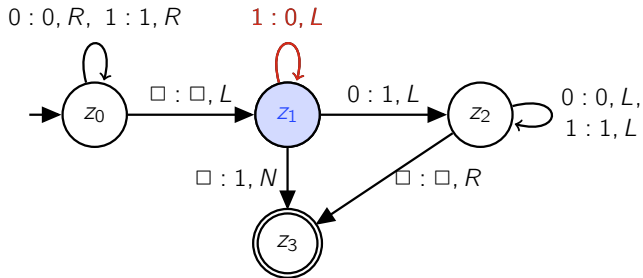


Eingabe: 0011

$z_0 0011$
 $\vdash 0z_0 011$
 $\vdash 00z_0 11$
 $\vdash 001z_0 1$
 $\vdash 0011z_0 \square$
 $\vdash 001z_1 1 \square$
 $\vdash 00z_1 10 \square$

Beispiel für einen akzeptierenden Lauf

DTM M :

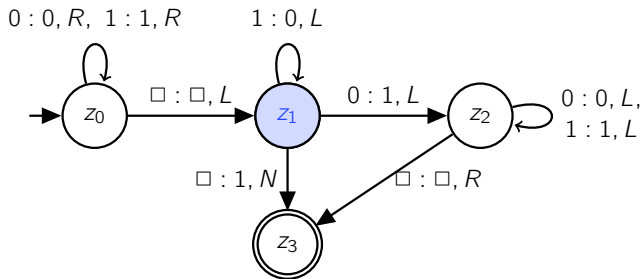


Eingabe: 0011

$z_0 0011$
 $\vdash 0z_0 011$
 $\vdash 00z_0 11$
 $\vdash 001z_0 1$
 $\vdash 0011z_0 \square$
 $\vdash 001z_1 1 \square$
 $\vdash 00z_1 10 \square$

Beispiel für einen akzeptierenden Lauf

DTM M :

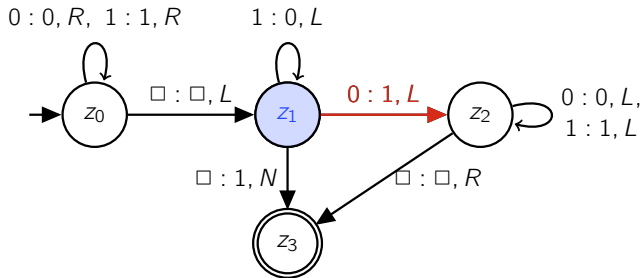


Eingabe: 0011

$z_0 0011$
 $\vdash 0z_0 011$
 $\vdash 00z_0 11$
 $\vdash 001z_0 1$
 $\vdash 0011z_0 \square$
 $\vdash 001z_1 1\square$
 $\vdash 00z_1 10\square$
 $\vdash 0z_1 000\square$

Beispiel für einen akzeptierenden Lauf

DTM M :

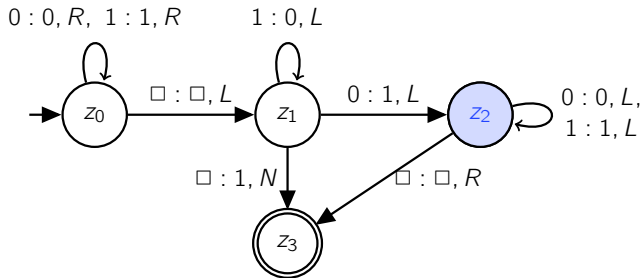


Eingabe: 0011

$z_0 0011$
 $\vdash 0z_0 011$
 $\vdash 00z_0 11$
 $\vdash 001z_0 1$
 $\vdash 0011z_0 \square$
 $\vdash 001z_1 1 \square$
 $\vdash 00z_1 10 \square$
 $\vdash 0z_1 000 \square$

Beispiel für einen akzeptierenden Lauf

DTM M :

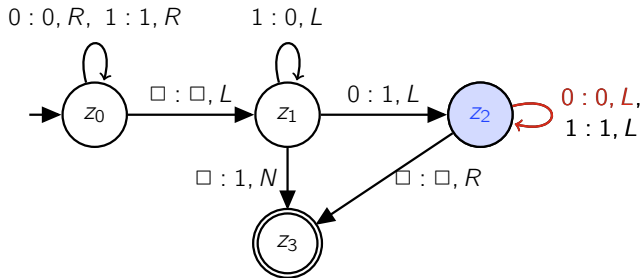


Eingabe: 0011

$z_0 0011$
 $\vdash 0z_0 011$
 $\vdash 00z_0 11$
 $\vdash 001z_0 1$
 $\vdash 0011z_0 \square$
 $\vdash 001z_1 1 \square$
 $\vdash 00z_1 10 \square$
 $\vdash 0z_1 000 \square$
 $\vdash z_2 0100 \square$

Beispiel für einen akzeptierenden Lauf

DTM M :

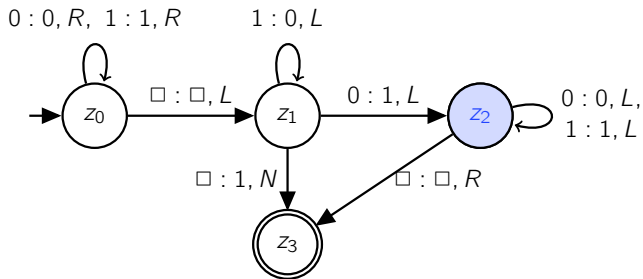


Eingabe: 0011

$z_0 0011$
 $\vdash 0z_0 011$
 $\vdash 00z_0 11$
 $\vdash 001z_0 1$
 $\vdash 0011z_0 \square$
 $\vdash 001z_1 1 \square$
 $\vdash 00z_1 10 \square$
 $\vdash 0z_1 000 \square$
 $\vdash z_2 0100 \square$

Beispiel für einen akzeptierenden Lauf

DTM M :

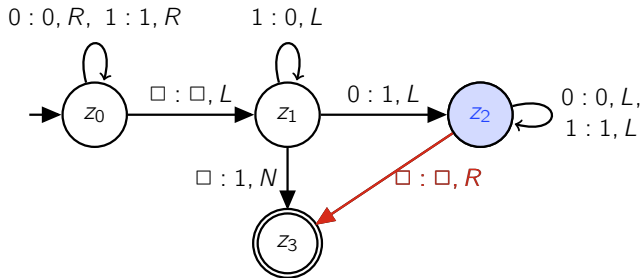


Eingabe: 0011

$z_0 0011$
 $\vdash 0z_0 011$
 $\vdash 00z_0 11$
 $\vdash 001z_0 1$
 $\vdash 0011z_0 \square$
 $\vdash 001z_1 1 \square$
 $\vdash 00z_1 10 \square$
 $\vdash 0z_1 000 \square$
 $\vdash z_2 0100 \square$
 $\vdash z_2 \square 0100 \square$

Beispiel für einen akzeptierenden Lauf

DTM M :

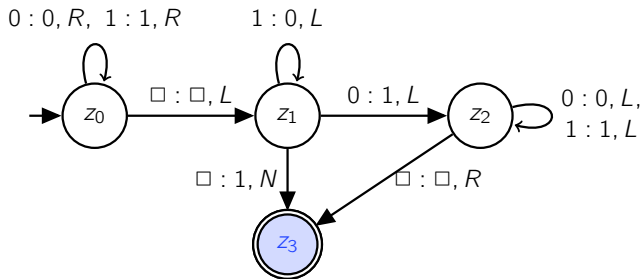


Eingabe: 0011

$z_0 0011$
 $\vdash 0z_0 011$
 $\vdash 00z_0 11$
 $\vdash 001z_0 1$
 $\vdash 0011z_0 \square$
 $\vdash 001z_1 1 \square$
 $\vdash 00z_1 10 \square$
 $\vdash 0z_1 000 \square$
 $\vdash z_2 0100 \square$
 $\vdash z_2 \square 0100 \square$

Beispiel für einen akzeptierenden Lauf

DTM M :

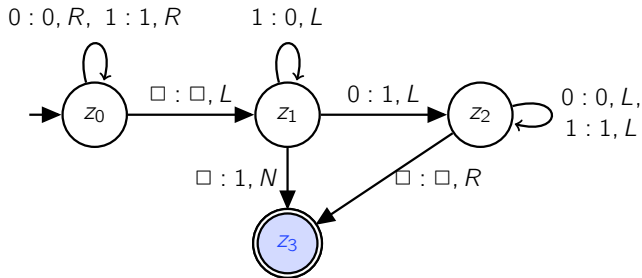


Eingabe: 0011

$z_0 0011$
 $\vdash 0z_0 011$
 $\vdash 00z_0 11$
 $\vdash 001z_0 1$
 $\vdash 0011z_0 \square$
 $\vdash 001z_1 1 \square$
 $\vdash 00z_1 10 \square$
 $\vdash 0z_1 000 \square$
 $\vdash z_2 0100 \square$
 $\vdash z_2 \square 0100 \square$
 $\vdash \square z_3 0100 \square$

Beispiel für einen akzeptierenden Lauf

DTM M :



Eingabe: 0011
 $\in L(M)$

$z_0 0011$
 $\vdash 0z_0 011$
 $\vdash 00z_0 11$
 $\vdash 001z_0 1$
 $\vdash 0011z_0 \square$
 $\vdash 001z_1 1 \square$
 $\vdash 00z_1 10 \square$
 $\vdash 0z_1 000 \square$
 $\vdash z_2 0100 \square$
 $\vdash z_2 \square 0100 \square$
 $\vdash \square z_3 0100 \square$

Linear beschränkte Turingmaschinen

Grundgedanke:

- ▶ **Linear beschränkte Turingmaschinen** (*linear bounded automata*, **LBAs**) sind spezielle Turingmaschinen.
- ▶ Der Schreib-Lesekopf darf den **Bereich der Eingabe** auf dem Band **nicht verlassen**.
- ▶ Zum Erkennen des Anfangs und des Endes wird die Eingabe in **spitzen Klammern** gesetzt: Statt w ist die Eingabe nun $\langle w \rangle$.

Definition eines LBA

Definition

Eine **linear beschränkte Turingmaschine** (*linear bounded automaton*, **LBA**) ist ein 8-Tupel $M = (Z, \Sigma, \Gamma, \delta, z_0, \langle, \rangle, E)$, wobei:

- ▶ $Z, \Sigma, \Gamma, \delta, z_0$ und E sind wie bei nichtdeterministischen Turingmaschinen
- ▶ $\langle, \rangle \in \Sigma$ sind **Start- bzw. Endmarker**
- ▶ δ überschreibt keinen der Marker
- ▶ bei $\langle$ gibt δ nie L aus und bei $\rangle$ gibt δ nie R aus.

Definition eines LBA

Definition

Eine **linear beschränkte Turingmaschine** (*linear bounded automaton*, **LBA**) ist ein 8-Tupel $M = (Z, \Sigma, \Gamma, \delta, z_0, \langle, \rangle, E)$, wobei:

- ▶ $Z, \Sigma, \Gamma, \delta, z_0$ und E sind wie bei nichtdeterministischen Turingmaschinen
- ▶ $\langle, \rangle \in \Sigma$ sind **Start-** bzw. **Endmarker**
- ▶ δ überschreibt keinen der Marker
- ▶ bei $\langle$ gibt δ nie L aus und bei $\rangle$ gibt δ nie R aus.

Definition

Sei $M = (Z, \Sigma, \Gamma, \delta, z_0, \langle, \rangle, E)$ ein LBA.

Die von M **akzeptierte Sprache** $L(M)$ ist definiert als

$$L(M) := \{w \in (\Sigma - \{\langle, \rangle\})^* \mid z_0 \langle w \rangle \vdash_M^* uzv \text{ für } u, v \in \Gamma^*, z \in E\}$$

Theorem (Satz von Kuroda)

Die LBAs akzeptieren genau die kontextsensitiven Sprachen.

Satz

Die nichtdeterministischen Turingmaschinen akzeptierten genau die Typ 0-Sprachen.

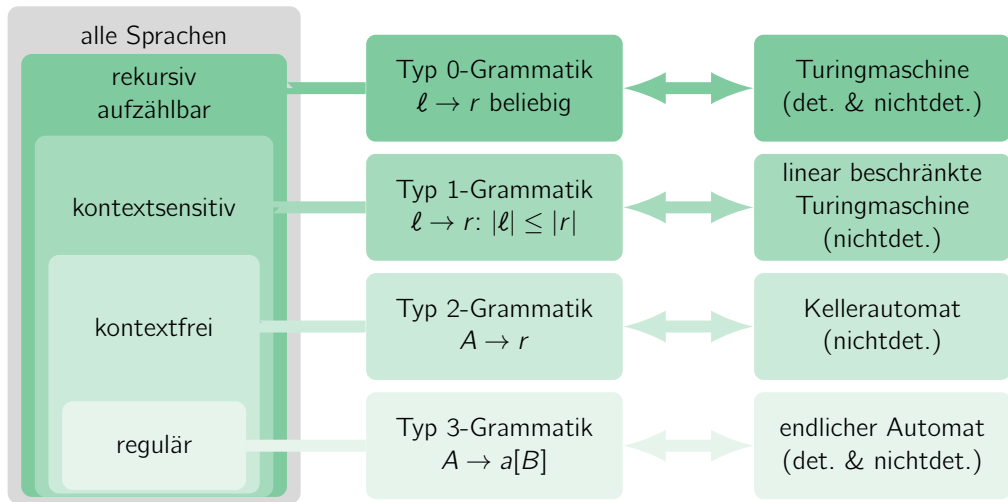
Beweis Nächste Vorlesung (nur FSK).



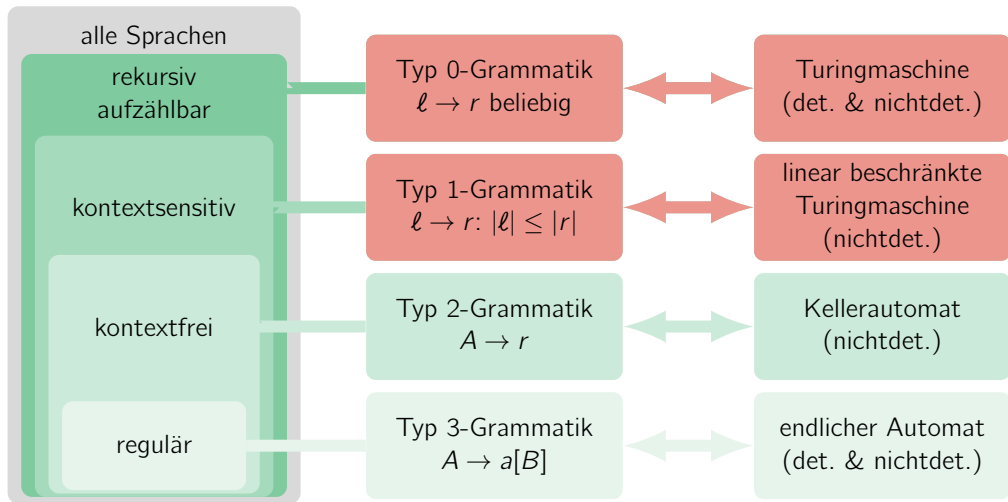
Deterministische vs. nichtdeterministische Turingmaschinen

- ▶ Nichtdeterministische Turingmaschinen können durch deterministische Turingmaschinen simuliert werden (mithilfe von Dovetailing).
- ▶ Daher gilt der letzte Satz auch für deterministische Turingmaschinen, d.h. die deterministischen Turingmaschinen akzeptierten genau die Typ 0-Sprachen.
- ▶ Der Unterschied zwischen NTMs und DTMs kommt erst zum Tragen, wenn wir das Laufzeitverhalten betrachten (im Kursteil zur Komplexitätstheorie).

Überblick über Grammatiken und Maschinenmodelle



Überblick über Grammatiken und Maschinenmodelle



Trennende Beispiele

- ▶ Die Sprache $\{a^n b^n \mid n \in \mathbb{N}\}$ ist vom Typ 2 aber nicht vom Typ 3.
- ▶ Die Sprache $\{a^n b^n c^n \mid n \in \mathbb{N}\}$ ist vom Typ 1 aber nicht vom Typ 2.
- ▶ Die Sprache

$$H = \{M\#w \mid \text{die durch } M \text{ beschriebene} \\ \text{Turingmaschine hält bei Eingabe } w\}$$

ist vom Typ 0 aber nicht vom Typ 1.

(Die Sprache H ist das Halteproblem, welches wir später noch genauer betrachten und erläutern.)

- ▶ Das Komplement von H ist nicht vom Typ 0.

Deterministisch vs. nichtdeterministisch

Deterministischer Automat	Nichtdeterministischer Automat	Äquivalent?
DFA	NFA	ja
DPDA	PDA	nein
DLBA	LBA	unbekannt
DTM	NTM	ja

Abschlusseigenschaften

Sprachklasse	Schnitt	Vereinigung	Komplement	Produkt	Kleenescher Abschluss
Typ 3	ja	ja	ja	ja	ja
Det. kontextfrei	nein	nein	ja	nein	nein
Typ 2	nein	ja	nein	ja	ja
Typ 1	ja	ja	ja	ja	ja
Typ 0	ja	ja	nein	ja	ja

Entscheidbarkeiten

Sprachklasse	Wortproblem	Leerheitsproblem	Äquivalenzproblem	Schnittproblem
Typ 3	ja	ja	ja	ja
Det. kontextfrei	ja	ja	ja	nein
Typ 2	ja	ja	nein	nein
Typ 1	ja	nein	nein	nein
Typ 0	nein	nein	nein	nein

Komplexität des Wortproblems

Sprachklasse	Komplexität
Typ 3, DFA gegeben	lineare Komplexität
deterministisch kontextfrei, DPDA gegeben	lineare Komplexität
Typ 2, Chomsky-Normalform gegeben	$O(n^3)$
Typ 1	exponentiell (nächste Vorlesung, FSK)
Typ 0	unlösbar

8a

Linear beschränkte Turingmaschinen (Fortsetzung)

Prof. Dr. Jasmin Blanchette

Lehr- und Forschungseinheit für
Theoretische Informatik und TheorembeweisenStand: 11. Juni 2024
Basierend auf Folien von PD Dr. David Sabel

Wiederholung: Kontextsensitive Grammatiken

- ▶ Die Linke Seite von Produktionen ist $\ell \in (V \cup \Sigma)^+$.
- ▶ Es gilt $|\ell| \leq |r|$ für alle Produktionen $\ell \rightarrow r$.

Definition

Eine **linear beschränkte Turingmaschine** (*linear bounded automaton*, **LBA**) ist ein 8-Tupel $M = (Z, \Sigma, \Gamma, \delta, z_0, \langle, \rangle, E)$, wobei:

- ▶ $Z, \Sigma, \Gamma, \delta, z_0$ und E sind wie bei nichtdeterministischen Turingmaschinen
- ▶ $\langle, \rangle \in \Sigma$ sind **Start- bzw. Endmarker**
- ▶ δ überschreibt keinen der Marker
- ▶ bei $\langle$ gibt δ nie L aus und bei $\rangle$ gibt δ nie R aus.

Der Satz von Kuroda

Unser Ziel ist es, den Satz von Kuroda zu beweisen:

Theorem (Satz von Kuroda)

Die LBAs akzeptieren genau die kontextsensitiven Sprachen.

Der Satz von Kuroda

Unser Ziel ist es, den Satz von Kuroda zu beweisen:

Theorem (Satz von Kuroda)

Die LBAs akzeptieren genau die kontextsensitiven Sprachen.

Der Beweis erfolgt in zwei Teilen:

- ▶ Jede kontextsensitive Sprache wird von einem LBA erkannt.
- ▶ Die akzeptierte Sprache von LBAs ist kontextsensitiv.

Kuroda-Normalform für Typ 1-Sprachen

Wie bei kontextfreien Sprachen gibt es Normalformen auch für kontextsensitive Sprachen: [die Kuroda-Normalform](#).

Sie ist nach dem Linguisten Sige-Yuki Kuroda benannt.

Kuroda-Normalform für Typ 1-Sprachen

Wie bei kontextfreien Sprachen gibt es Normalformen auch für kontextsensitive Sprachen: [die Kuroda-Normalform](#).

Sie ist nach dem Linguisten Sige-Yuki Kuroda benannt.

Definition

Eine Typ 1-Grammatik $G = (V, \Sigma, P, S)$ ist in [Kuroda-Normalform](#), falls alle Produktionen in P einer der folgenden vier Formen entsprechen:

$$A \rightarrow a \quad A \rightarrow B \quad A \rightarrow BC \quad AB \rightarrow CD$$

wobei $a \in \Sigma$ und $A, B, C, D \in V$.

Kuroda-Normalform für Typ 1-Sprachen

Wie bei kontextfreien Sprachen gibt es Normalformen auch für kontextsensitive Sprachen: [die Kuroda-Normalform](#).

Sie ist nach dem Linguisten Sige-Yuki Kuroda benannt.

Definition

Eine Typ 1-Grammatik $G = (V, \Sigma, P, S)$ ist in [Kuroda-Normalform](#), falls alle Produktionen in P einer der folgenden vier Formen entsprechen:

$$A \rightarrow a \quad A \rightarrow B \quad A \rightarrow BC \quad AB \rightarrow CD$$

wobei $a \in \Sigma$ und $A, B, C, D \in V$.

Die Kuroda-Normalform erweitert kontextfreie Grammatiken um Regeln von der Form $AB \rightarrow CD$.

Herstellen der Kuroda-Normalform

Satz

Sei L eine kontextsensitive Sprache mit $\varepsilon \notin L$.

Dann gibt es eine Grammatik in Kuroda-Normalform, die L erzeugt.

Herstellen der Kuroda-Normalform

Satz

Sei L eine kontextsensitive Sprache mit $\varepsilon \notin L$.

Dann gibt es eine Grammatik in Kuroda-Normalform, die L erzeugt.

Beweis Algorithmus 10 (siehe Skript) bewerkstelligt dies.



Kontextsensitive Sprachen werden von LBAs akzeptiert

Satz

Für jede kontextsensitive Sprache L gibt es einen LBA M mit $L(M) = L$.

Kontextsensitive Sprachen werden von LBAs akzeptiert

Satz

Für jede kontextsensitive Sprache L gibt es einen LBA M mit $L(M) = L$.

Beweis Sei $G = (V, \Sigma, P, S)$ mit $L(G) = L \setminus \{\varepsilon\}$ eine kontextsensitive Grammatik in Kuroda-Normalform.

Kontextsensitive Sprachen werden von LBAs akzeptiert

Satz

Für jede kontextsensitive Sprache L gibt es einen LBA M mit $L(M) = L$.

Beweis Sei $G = (V, \Sigma, P, S)$ mit $L(G) = L \setminus \{\varepsilon\}$ eine kontextsensitive Grammatik in Kuroda-Normalform.

Grundgedanke:

- ▶ Konstruiere den LBA M mit Bandalphabet $\Gamma := \Sigma \cup V \cup \{\langle, \rangle, x\}$.
- ▶ M versucht nichtdeterministisch für $w \in \Sigma^*$ das Startsymbol S der Grammatik rückwärts herzuleiten, durch rückwärts Anwenden der Produktionen $\ell \rightarrow r \in P$, wobei Vorkommen von r durch ℓ ersetzt werden.
- ▶ Akzeptiere, wenn $\langle S \rangle$ (bzw. $\langle \rangle$, falls $S \rightarrow \varepsilon \in P$) auf dem Band vorkommt.
- ▶ Nichtdeterminismus wird verwendet, um zu beschließen, welche Produktion wird rückwärts angewendet und für welches Vorkommen einer rechten Seite.

Beispiel für einen Lauf des konstruierten LBA

Grammatik $G = (\{S, A, B\}, \{a, b\}, P, S)$ in Kuroda-Normalform mit $P = \{S \rightarrow AB, S \rightarrow A, A \rightarrow a, B \rightarrow b\}$.

Eingabewort: ab

Beispiel für einen Lauf des konstruierten LBA

Grammatik $G = (\{S, A, B\}, \{a, b\}, P, S)$ in Kuroda-Normalform mit $P = \{S \rightarrow AB, S \rightarrow A, A \rightarrow a, B \rightarrow b\}$.

Eingabewort: ab

Akzeptierender Lauf von M :

1. Am Anfang steht $\langle ab \rangle$ auf dem Band.

Beispiel für einen Lauf des konstruierten LBA

Grammatik $G = (\{S, A, B\}, \{a, b\}, P, S)$ in Kuroda-Normalform mit $P = \{S \rightarrow AB, S \rightarrow A, A \rightarrow a, B \rightarrow b\}$.

Eingabewort: ab

Akzeptierender Lauf von M :

1. Am Anfang steht $\langle ab \rangle$ auf dem Band.
2. b wird durch B ersetzt. Auf dem Band steht nun $\langle aB \rangle$.

Beispiel für einen Lauf des konstruierten LBA

Grammatik $G = (\{S, A, B\}, \{a, b\}, P, S)$ in Kuroda-Normalform mit $P = \{S \rightarrow AB, S \rightarrow A, A \rightarrow a, B \rightarrow b\}$.

Eingabewort: ab

Akzeptierender Lauf von M :

1. Am Anfang steht $\langle ab \rangle$ auf dem Band.
2. b wird durch B ersetzt. Auf dem Band steht nun $\langle aB \rangle$.
3. a wird durch A ersetzt. Auf dem Band steht nun $\langle AB \rangle$.

Beispiel für einen Lauf des konstruierten LBA

Grammatik $G = (\{S, A, B\}, \{a, b\}, P, S)$ in Kuroda-Normalform mit $P = \{S \rightarrow AB, S \rightarrow A, A \rightarrow a, B \rightarrow b\}$.

Eingabewort: ab

Akzeptierender Lauf von M :

1. Am Anfang steht $\langle ab \rangle$ auf dem Band.
2. b wird durch B ersetzt. Auf dem Band steht nun $\langle aB \rangle$.
3. a wird durch A ersetzt. Auf dem Band steht nun $\langle AB \rangle$.
4. AB wird durch xS ersetzt. Auf dem Band steht nun $\langle xS \rangle$.

Beispiel für einen Lauf des konstruierten LBA

Grammatik $G = (\{S, A, B\}, \{a, b\}, P, S)$ in Kuroda-Normalform mit $P = \{S \rightarrow AB, S \rightarrow A, A \rightarrow a, B \rightarrow b\}$.

Eingabewort: ab

Akzeptierender Lauf von M :

1. Am Anfang steht $\langle ab \rangle$ auf dem Band.
2. b wird durch B ersetzt. Auf dem Band steht nun $\langle aB \rangle$.
3. a wird durch A ersetzt. Auf dem Band steht nun $\langle AB \rangle$.
4. AB wird durch xS ersetzt. Auf dem Band steht nun $\langle xS \rangle$.
5. Das x wird durch $\langle$ ersetzt. Auf dem Band steht $\langle \langle S \rangle$.

Beispiel für einen Lauf des konstruierten LBA

Grammatik $G = (\{S, A, B\}, \{a, b\}, P, S)$ in Kuroda-Normalform mit $P = \{S \rightarrow AB, S \rightarrow A, A \rightarrow a, B \rightarrow b\}$.

Eingabewort: ab

Akzeptierender Lauf von M :

1. Am Anfang steht $\langle ab \rangle$ auf dem Band.
2. b wird durch B ersetzt. Auf dem Band steht nun $\langle aB \rangle$.
3. a wird durch A ersetzt. Auf dem Band steht nun $\langle AB \rangle$.
4. AB wird durch xS ersetzt. Auf dem Band steht nun $\langle xS \rangle$.
5. Das x wird durch $\langle$ ersetzt. Auf dem Band steht $\langle \langle S \rangle$.
6. M akzeptiert.

Kontextsensitive Sprachen werden von LBAs akzeptiert

Beweis (Fortsetzung) Suche nach einer rechter Seite r :

1. Beginne links an der Eingabe und laufe diese durch.
2. Speichere das Symbol links vom Schreib-Lesekopf im aktuellen Zustand.
3. Stelle mit dem aktuellen Symbol fest, ob es passende Produktion gibt.
(Da rechte Seiten von Produktionen in Kuroda-Normalform aus maximal zwei Zeichen bestehen, genügt dies.)

Kontextsensitive Sprachen werden von LBAs akzeptiert

Beweis (Fortsetzung) Ersetzung von r durch ℓ :

1. Für $A \rightarrow a$ und $A \rightarrow B$ wird das aktuelle Symbol durch A ersetzt, anschließend wird der nächste Schritt gestartet.
2. Für $AB \rightarrow CD$, wird B geschrieben und der Kopf nach links wechseln, dann A geschrieben und der nächste Schritt gestartet.
3. Für $A \rightarrow BC$ schreibe A und wechsele nach links, schreibe x , fahre ganz nach links und starte die Prozedur zum Verschieben der Zeichen nach rechts, solange bis x überschrieben ist.

Kontextsensitive Sprachen werden von LBAs akzeptiert

Beweis (Fortsetzung) Verschieben nach rechts:

1. Speichere das linkeste Symbol im Zustand und schreibe $\langle$.
2. Laufe nach rechts. Dabei wird das aktuelle Symbol mit dem gespeicherten vertauscht.
3. Beende Vertauschen, nachdem x mit einem anderen Symbol vertauscht wird. $\square$

Bemerkungen und Typ 0-Grammatiken

- ▶ Der Grundgedanke der Konstruktion funktioniert auch für Typ 1-Grammatiken nicht in Kuroda-Normalform, ist aber komplizierter.

Bemerkungen und Typ 0-Grammatiken

- ▶ Der Grundgedanke der Konstruktion funktioniert auch für Typ 1-Grammatiken nicht in Kuroda-Normalform, ist aber komplizierter.
- ▶ Der Grundgedanke der Konstruktion funktioniert auch für Typ 0-Grammatiken: Der Platz ist allerdings dann unbeschränkt, und eine NTM wird benötigt.

Bemerkungen und Typ 0-Grammatiken

- ▶ Der Grundgedanke der Konstruktion funktioniert auch für Typ 1-Grammatiken nicht in Kuroda-Normalform, ist aber komplizierter.
- ▶ Der Grundgedanke der Konstruktion funktioniert auch für Typ 0-Grammatiken: Der Platz ist allerdings dann unbeschränkt, und eine NTM wird benötigt.

Satz

Für jede Typ 0-Sprache L gibt es eine nichtdeterministischen Turingmaschine M mit $L(M) = L$.

Beweis Der Beweis des letzten Satzes kann angepasst werden, damit er eine NTM konstruiert, die eine beliebige Typ 0-Grammatik simuliert. $\square$

Die akzeptierte Sprache von LBAs ist kontextsensitiv

Satz

Sei $M = (Z, \Sigma, \Gamma, \delta, z_0, \langle, \rangle, E)$ ein LBA. Dann ist $L(M)$ kontextsensitiv.

Die akzeptierte Sprache von LBAs ist kontextsensitiv

Satz

Sei $M = (Z, \Sigma, \Gamma, \delta, z_0, \langle, \rangle, E)$ ein LBA. Dann ist $L(M)$ kontextsensitiv.

Beweis Konstruiere eine Typ 1-Grammatik G mit $L(G) = L(M)$.

Die akzeptierte Sprache von LBAs ist kontextsensitiv

Satz

Sei $M = (Z, \Sigma, \Gamma, \delta, z_0, \langle, \rangle, E)$ ein LBA. Dann ist $L(M)$ kontextsensitiv.

Beweis Konstruiere eine Typ 1-Grammatik G mit $L(G) = L(M)$.

Grundgedanke:

1. Erzeuge ein beliebiges Wort w und die Startkonfiguration von M für w .
2. Simuliere M zum Prüfen, ob $w \in L(M)$.
3. Wenn M akzeptiert, erzeuge w endgültig.

Die akzeptierte Sprache von LBAs ist kontextsensitiv

Satz

Sei $M = (Z, \Sigma, \Gamma, \delta, z_0, \langle, \rangle, E)$ ein LBA. Dann ist $L(M)$ kontextsensitiv.

Beweis Konstruiere eine Typ 1-Grammatik G mit $L(G) = L(M)$.

Grundgedanke:

1. Erzeuge ein beliebiges Wort w und die Startkonfiguration von M für w .
2. Simuliere M zum Prüfen, ob $w \in L(M)$.
3. Wenn M akzeptiert, erzeuge w endgültig.

Die Variablen V von G sind S , A und Tupeln von der Form $\begin{bmatrix} u \\ v \end{bmatrix}$, wobei $u \in \Sigma$ und $v \in (\Gamma \cup Z)^+$.

Die oberen Komponenten einer Tupelfolge $\begin{bmatrix} u_1 \\ v_1 \end{bmatrix} \cdots \begin{bmatrix} u_n \\ v_n \end{bmatrix}$ ergeben ein Wort w .
Die unteren Komponenten ergeben eine LBA-Konfiguration.

Beispiel für eine Ableitung der konstruierten Grammatik

LBA $M = (Z, \Sigma, \Gamma, \delta, z_0, \langle, \rangle, E)$ mit

$Z = \{z_0, z_1, z_2\}$, $\Sigma = \{a, \langle, \rangle\}$, $\Gamma = \Sigma \cup \{x\}$, $E = \{z_2\}$ und

$$\begin{array}{lll} \delta(z_0, a) = (z_1, x, R) & \delta(z_0, \langle) = (z_0, \langle, R) & \delta(z_0, \rangle) = (z_0, \rangle, L) \\ \delta(z_1, a) = (z_2, x, R) & \delta(z_1, \langle) = (z_1, \langle, R) & \delta(z_1, \rangle) = (z_1, \rangle, L) \end{array}$$

Beispiel für eine Ableitung der konstruierten Grammatik

LBA $M = (Z, \Sigma, \Gamma, \delta, z_0, \langle, \rangle, E)$ mit

$Z = \{z_0, z_1, z_2\}$, $\Sigma = \{a, \langle, \rangle\}$, $\Gamma = \Sigma \cup \{x\}$, $E = \{z_2\}$ und

$$\begin{array}{lll} \delta(z_0, a) = (z_1, x, R) & \delta(z_0, \langle) = (z_0, \langle, R) & \delta(z_0, \rangle) = (z_0, \rangle, L) \\ \delta(z_1, a) = (z_2, x, R) & \delta(z_1, \langle) = (z_1, \langle, R) & \delta(z_1, \rangle) = (z_1, \rangle, L) \end{array}$$

Akzeptierender Lauf auf aa :

$z_0 \langle aa \rangle$

Beispiel für eine Ableitung der konstruierten Grammatik

LBA $M = (Z, \Sigma, \Gamma, \delta, z_0, \langle, \rangle, E)$ mit

$Z = \{z_0, z_1, z_2\}$, $\Sigma = \{a, \langle, \rangle\}$, $\Gamma = \Sigma \cup \{x\}$, $E = \{z_2\}$ und

$$\begin{array}{lll} \delta(z_0, a) = (z_1, x, R) & \delta(z_0, \langle) = (z_0, \langle, R) & \delta(z_0, \rangle) = (z_0, \rangle, L) \\ \delta(z_1, a) = (z_2, x, R) & \delta(z_1, \langle) = (z_1, \langle, R) & \delta(z_1, \rangle) = (z_1, \rangle, L) \end{array}$$

Akzeptierender Lauf auf aa :

$z_0 \langle aa \rangle$

Beispiel für eine Ableitung der konstruierten Grammatik

LBA $M = (Z, \Sigma, \Gamma, \delta, z_0, \langle, \rangle, E)$ mit

$Z = \{z_0, z_1, z_2\}$, $\Sigma = \{a, \langle, \rangle\}$, $\Gamma = \Sigma \cup \{x\}$, $E = \{z_2\}$ und

$$\begin{array}{lll} \delta(z_0, a) = (z_1, x, R) & \delta(z_0, \langle) = (z_0, \langle, R) & \delta(z_0, \rangle) = (z_0, \rangle, L) \\ \delta(z_1, a) = (z_2, x, R) & \delta(z_1, \langle) = (z_1, \langle, R) & \delta(z_1, \rangle) = (z_1, \rangle, L) \end{array}$$

Akzeptierender Lauf auf aa :

$$z_0 \langle aa \rangle \vdash \langle z_0 aa \rangle$$

Beispiel für eine Ableitung der konstruierten Grammatik

LBA $M = (Z, \Sigma, \Gamma, \delta, z_0, \langle, \rangle, E)$ mit

$Z = \{z_0, z_1, z_2\}$, $\Sigma = \{a, \langle, \rangle\}$, $\Gamma = \Sigma \cup \{x\}$, $E = \{z_2\}$ und

$$\begin{array}{lll} \delta(z_0, a) = (z_1, x, R) & \delta(z_0, \langle) = (z_0, \langle, R) & \delta(z_0, \rangle) = (z_0, \rangle, L) \\ \delta(z_1, a) = (z_2, x, R) & \delta(z_1, \langle) = (z_1, \langle, R) & \delta(z_1, \rangle) = (z_1, \rangle, L) \end{array}$$

Akzeptierender Lauf auf aa :

$$z_0 \langle aa \rangle \vdash \langle z_0 aa \rangle$$

Beispiel für eine Ableitung der konstruierten Grammatik

LBA $M = (Z, \Sigma, \Gamma, \delta, z_0, \langle, \rangle, E)$ mit

$Z = \{z_0, z_1, z_2\}$, $\Sigma = \{a, \langle, \rangle\}$, $\Gamma = \Sigma \cup \{x\}$, $E = \{z_2\}$ und

$$\begin{array}{lll} \delta(z_0, a) = (z_1, x, R) & \delta(z_0, \langle) = (z_0, \langle, R) & \delta(z_0, \rangle) = (z_0, \rangle, L) \\ \delta(z_1, a) = (z_2, x, R) & \delta(z_1, \langle) = (z_1, \langle, R) & \delta(z_1, \rangle) = (z_1, \rangle, L) \end{array}$$

Akzeptierender Lauf auf aa :

$$z_0 \langle aa \rangle \vdash \langle z_0 aa \rangle \vdash \langle x z_1 a \rangle$$

Beispiel für eine Ableitung der konstruierten Grammatik

LBA $M = (Z, \Sigma, \Gamma, \delta, z_0, \langle, \rangle, E)$ mit

$Z = \{z_0, z_1, z_2\}$, $\Sigma = \{a, \langle, \rangle\}$, $\Gamma = \Sigma \cup \{x\}$, $E = \{z_2\}$ und

$$\begin{array}{lll} \delta(z_0, a) = (z_1, x, R) & \delta(z_0, \langle) = (z_0, \langle, R) & \delta(z_0, \rangle) = (z_0, \rangle, L) \\ \delta(z_1, a) = (z_2, x, R) & \delta(z_1, \langle) = (z_1, \langle, R) & \delta(z_1, \rangle) = (z_1, \rangle, L) \end{array}$$

Akzeptierender Lauf auf aa :

$$z_0 \langle aa \rangle \vdash \langle z_0 aa \rangle \vdash \langle x z_1 a \rangle$$

Beispiel für eine Ableitung der konstruierten Grammatik

LBA $M = (Z, \Sigma, \Gamma, \delta, z_0, \langle, \rangle, E)$ mit

$Z = \{z_0, z_1, z_2\}$, $\Sigma = \{a, \langle, \rangle\}$, $\Gamma = \Sigma \cup \{x\}$, $E = \{z_2\}$ und

$$\begin{array}{lll} \delta(z_0, a) = (z_1, x, R) & \delta(z_0, \langle) = (z_0, \langle, R) & \delta(z_0, \rangle) = (z_0, \rangle, L) \\ \delta(z_1, a) = (z_2, x, R) & \delta(z_1, \langle) = (z_1, \langle, R) & \delta(z_1, \rangle) = (z_1, \rangle, L) \end{array}$$

Akzeptierender Lauf auf aa :

$z_0 \langle aa \rangle \vdash \langle z_0 aa \rangle \vdash \langle x z_1 a \rangle \vdash \langle x x z_2 \rangle$

Beispiel für eine Ableitung der konstruierten Grammatik

LBA $M = (Z, \Sigma, \Gamma, \delta, z_0, \langle, \rangle, E)$ mit

$Z = \{z_0, z_1, z_2\}$, $\Sigma = \{a, \langle, \rangle\}$, $\Gamma = \Sigma \cup \{x\}$, $E = \{z_2\}$ und

$$\begin{array}{lll} \delta(z_0, a) = (z_1, x, R) & \delta(z_0, \langle) = (z_0, \langle, R) & \delta(z_0, \rangle) = (z_0, \rangle, L) \\ \delta(z_1, a) = (z_2, x, R) & \delta(z_1, \langle) = (z_1, \langle, R) & \delta(z_1, \rangle) = (z_1, \rangle, L) \end{array}$$

Akzeptierender Lauf auf aa :

$$z_0 \langle aa \rangle \vdash \langle z_0 aa \rangle \vdash \langle xz_1 a \rangle \vdash \langle xxz_2 \rangle$$

Ableitung von aa :

S

Beispiel für eine Ableitung der konstruierten Grammatik

LBA $M = (Z, \Sigma, \Gamma, \delta, z_0, \langle, \rangle, E)$ mit

$Z = \{z_0, z_1, z_2\}$, $\Sigma = \{a, \langle, \rangle\}$, $\Gamma = \Sigma \cup \{x\}$, $E = \{z_2\}$ und

$$\begin{array}{lll} \delta(z_0, a) = (z_1, x, R) & \delta(z_0, \langle) = (z_0, \langle, R) & \delta(z_0, \rangle) = (z_0, \rangle, L) \\ \delta(z_1, a) = (z_2, x, R) & \delta(z_1, \langle) = (z_1, \langle, R) & \delta(z_1, \rangle) = (z_1, \rangle, L) \end{array}$$

Akzeptierender Lauf auf aa :

$$z_0 \langle aa \rangle \vdash \langle z_0 aa \rangle \vdash \langle x z_1 a \rangle \vdash \langle x x z_2 \rangle$$

Ableitung von aa :

$$S \Rightarrow \left[\begin{smallmatrix} a \\ z_0 \langle a \end{smallmatrix} \right] A$$

Beispiel für eine Ableitung der konstruierten Grammatik

LBA $M = (Z, \Sigma, \Gamma, \delta, z_0, \langle, \rangle, E)$ mit
 $Z = \{z_0, z_1, z_2\}$, $\Sigma = \{a, \langle, \rangle\}$, $\Gamma = \Sigma \cup \{x\}$, $E = \{z_2\}$ und

$$\begin{array}{lll} \delta(z_0, a) = (z_1, x, R) & \delta(z_0, \langle) = (z_0, \langle, R) & \delta(z_0, \rangle) = (z_0, \rangle, L) \\ \delta(z_1, a) = (z_2, x, R) & \delta(z_1, \langle) = (z_1, \langle, R) & \delta(z_1, \rangle) = (z_1, \rangle, L) \end{array}$$

Akzeptierender Lauf auf aa :

$$z_0 \langle aa \rangle \vdash \langle z_0 aa \rangle \vdash \langle x z_1 a \rangle \vdash \langle x x z_2 \rangle$$

Ableitung von aa :

$$S \Rightarrow \begin{bmatrix} a \\ z_0 \langle a \rangle \end{bmatrix} A \Rightarrow \begin{bmatrix} a \\ z_0 \langle a \rangle \end{bmatrix} \begin{bmatrix} a \\ a \rangle \end{bmatrix}$$

Beispiel für eine Ableitung der konstruierten Grammatik

LBA $M = (Z, \Sigma, \Gamma, \delta, z_0, \langle, \rangle, E)$ mit
 $Z = \{z_0, z_1, z_2\}$, $\Sigma = \{a, \langle, \rangle\}$, $\Gamma = \Sigma \cup \{x\}$, $E = \{z_2\}$ und

$$\begin{array}{lll} \delta(z_0, a) = (z_1, x, R) & \delta(z_0, \langle) = (z_0, \langle, R) & \delta(z_0, \rangle) = (z_0, \rangle, L) \\ \delta(z_1, a) = (z_2, x, R) & \delta(z_1, \langle) = (z_1, \langle, R) & \delta(z_1, \rangle) = (z_1, \rangle, L) \end{array}$$

Akzeptierender Lauf auf aa :

$$z_0 \langle aa \rangle \vdash \langle z_0 aa \rangle \vdash \langle x z_1 a \rangle \vdash \langle x x z_2 \rangle$$

Ableitung von aa :

$$S \Rightarrow \begin{bmatrix} a \\ z_0 \langle a \rangle \end{bmatrix} A \Rightarrow \begin{bmatrix} a \\ z_0 \langle a \rangle \end{bmatrix} \begin{bmatrix} a \\ a \rangle \end{bmatrix} \Rightarrow \begin{bmatrix} a \\ \langle z_0 a \rangle \end{bmatrix} \begin{bmatrix} a \\ a \rangle \end{bmatrix}$$

Beispiel für eine Ableitung der konstruierten Grammatik

LBA $M = (Z, \Sigma, \Gamma, \delta, z_0, \langle, \rangle, E)$ mit

$Z = \{z_0, z_1, z_2\}$, $\Sigma = \{a, \langle, \rangle\}$, $\Gamma = \Sigma \cup \{x\}$, $E = \{z_2\}$ und

$$\begin{array}{lll} \delta(z_0, a) = (z_1, x, R) & \delta(z_0, \langle) = (z_0, \langle, R) & \delta(z_0, \rangle) = (z_0, \rangle, L) \\ \delta(z_1, a) = (z_2, x, R) & \delta(z_1, \langle) = (z_1, \langle, R) & \delta(z_1, \rangle) = (z_1, \rangle, L) \end{array}$$

Akzeptierender Lauf auf aa :

$$z_0 \langle aa \rangle \vdash \langle z_0 aa \rangle \vdash \langle x z_1 a \rangle \vdash \langle x x z_2 \rangle$$

Ableitung von aa :

$$S \Rightarrow \begin{bmatrix} a \\ z_0 \langle a \rangle \end{bmatrix} A \Rightarrow \begin{bmatrix} a \\ z_0 \langle a \rangle \end{bmatrix} \begin{bmatrix} a \\ a \rangle \end{bmatrix} \Rightarrow \begin{bmatrix} a \\ \langle z_0 a \rangle \end{bmatrix} \begin{bmatrix} a \\ a \rangle \end{bmatrix} \Rightarrow \begin{bmatrix} a \\ \langle x \rangle \end{bmatrix} \begin{bmatrix} a \\ z_1 a \rangle \end{bmatrix}$$

Beispiel für eine Ableitung der konstruierten Grammatik

LBA $M = (Z, \Sigma, \Gamma, \delta, z_0, \langle, \rangle, E)$ mit

$Z = \{z_0, z_1, z_2\}$, $\Sigma = \{a, \langle, \rangle\}$, $\Gamma = \Sigma \cup \{x\}$, $E = \{z_2\}$ und

$$\begin{array}{lll} \delta(z_0, a) = (z_1, x, R) & \delta(z_0, \langle) = (z_0, \langle, R) & \delta(z_0, \rangle) = (z_0, \rangle, L) \\ \delta(z_1, a) = (z_2, x, R) & \delta(z_1, \langle) = (z_1, \langle, R) & \delta(z_1, \rangle) = (z_1, \rangle, L) \end{array}$$

Akzeptierender Lauf auf aa :

$$z_0 \langle aa \rangle \vdash \langle z_0 aa \rangle \vdash \langle x z_1 a \rangle \vdash \langle x x z_2 \rangle$$

Ableitung von aa :

$$S \Rightarrow \begin{bmatrix} a \\ z_0 \langle a \rangle \end{bmatrix} A \Rightarrow \begin{bmatrix} a \\ z_0 \langle a \rangle \end{bmatrix} \begin{bmatrix} a \\ a \rangle \end{bmatrix} \Rightarrow \begin{bmatrix} a \\ \langle z_0 a \rangle \end{bmatrix} \begin{bmatrix} a \\ a \rangle \end{bmatrix} \Rightarrow \begin{bmatrix} a \\ \langle x \rangle \end{bmatrix} \begin{bmatrix} a \\ z_1 a \rangle \end{bmatrix} \Rightarrow \begin{bmatrix} a \\ \langle x \rangle \end{bmatrix} \begin{bmatrix} a \\ x z_2 \rangle \end{bmatrix}$$

Beispiel für eine Ableitung der konstruierten Grammatik

LBA $M = (Z, \Sigma, \Gamma, \delta, z_0, \langle, \rangle, E)$ mit

$Z = \{z_0, z_1, z_2\}$, $\Sigma = \{a, \langle, \rangle\}$, $\Gamma = \Sigma \cup \{x\}$, $E = \{z_2\}$ und

$$\begin{array}{lll} \delta(z_0, a) = (z_1, x, R) & \delta(z_0, \langle) = (z_0, \langle, R) & \delta(z_0, \rangle) = (z_0, \rangle, L) \\ \delta(z_1, a) = (z_2, x, R) & \delta(z_1, \langle) = (z_1, \langle, R) & \delta(z_1, \rangle) = (z_1, \rangle, L) \end{array}$$

Akzeptierender Lauf auf aa :

$$z_0 \langle aa \rangle \vdash \langle z_0 aa \rangle \vdash \langle x z_1 a \rangle \vdash \langle x x z_2 \rangle$$

Ableitung von aa :

$$S \Rightarrow \begin{bmatrix} a \\ z_0 \langle a \rangle \end{bmatrix} A \Rightarrow \begin{bmatrix} a \\ z_0 \langle a \rangle \end{bmatrix} \begin{bmatrix} a \\ a \rangle \end{bmatrix} \Rightarrow \begin{bmatrix} a \\ \langle z_0 a \rangle \end{bmatrix} \begin{bmatrix} a \\ a \rangle \end{bmatrix} \Rightarrow \begin{bmatrix} a \\ \langle x \rangle \end{bmatrix} \begin{bmatrix} a \\ z_1 a \rangle \end{bmatrix} \Rightarrow \begin{bmatrix} a \\ \langle x \rangle \end{bmatrix} \begin{bmatrix} a \\ x z_2 \rangle \end{bmatrix} \Rightarrow \begin{bmatrix} a \\ \langle x \rangle \end{bmatrix} a$$

Beispiel für eine Ableitung der konstruierten Grammatik

LBA $M = (Z, \Sigma, \Gamma, \delta, z_0, \langle, \rangle, E)$ mit

$Z = \{z_0, z_1, z_2\}$, $\Sigma = \{a, \langle, \rangle\}$, $\Gamma = \Sigma \cup \{x\}$, $E = \{z_2\}$ und

$$\begin{array}{lll} \delta(z_0, a) = (z_1, x, R) & \delta(z_0, \langle) = (z_0, \langle, R) & \delta(z_0, \rangle) = (z_0, \rangle, L) \\ \delta(z_1, a) = (z_2, x, R) & \delta(z_1, \langle) = (z_1, \langle, R) & \delta(z_1, \rangle) = (z_1, \rangle, L) \end{array}$$

Akzeptierender Lauf auf aa :

$$z_0 \langle aa \rangle \vdash \langle z_0 aa \rangle \vdash \langle x z_1 a \rangle \vdash \langle x x z_2 \rangle$$

Ableitung von aa :

$$S \Rightarrow \begin{bmatrix} a \\ z_0 \langle a \rangle \end{bmatrix} A \Rightarrow \begin{bmatrix} a \\ z_0 \langle a \rangle \end{bmatrix} \begin{bmatrix} a \\ a \rangle \end{bmatrix} \Rightarrow \begin{bmatrix} a \\ \langle z_0 a \rangle \end{bmatrix} \begin{bmatrix} a \\ a \rangle \end{bmatrix} \Rightarrow \begin{bmatrix} a \\ \langle x \rangle \end{bmatrix} \begin{bmatrix} a \\ z_1 a \rangle \end{bmatrix} \Rightarrow \begin{bmatrix} a \\ \langle x \rangle \end{bmatrix} \begin{bmatrix} a \\ x z_2 \rangle \end{bmatrix} \Rightarrow \begin{bmatrix} a \\ \langle x \rangle \end{bmatrix} a \Rightarrow aa$$

Die akzeptierte Sprache von LBAs ist kontextsensitiv

Beweis (Fortsetzung) Konstruiere $G = (V, \Sigma - \{\langle, \rangle\}, P_0 \cup P_1 \cup P_2 \cup P_3, S)$.

1. Erzeuge ein beliebiges Wort $w \in \Sigma^*$ und die Startkonfiguration von M für w :

► Regeln zur Erzeugung von $w \in \Sigma^*$ und Startkonfiguration zw:

$$P_0 := \{S \rightarrow \begin{bmatrix} a \\ z_0 \langle a \rangle \end{bmatrix} A \mid a \in \Sigma\} \cup \{S \rightarrow \begin{bmatrix} a \\ z_0 \langle a \rangle \end{bmatrix} \mid a \in \Sigma\} \\ \cup \{A \rightarrow \begin{bmatrix} a \\ a \end{bmatrix} A \mid a \in \Sigma\} \cup \{A \rightarrow \begin{bmatrix} a \\ a \end{bmatrix} \mid a \in \Sigma\}$$

Die akzeptierte Sprache von LBAs ist kontextsensitiv

Beweis (Fortsetzung) Konstruiere $G = (V, \Sigma - \{\langle, \rangle\}, P_0 \cup P_1 \cup P_2 \cup P_3, S)$.

1. Erzeuge ein beliebiges Wort $w \in \Sigma^*$ und die Startkonfiguration von M für w :

► Regeln zur Erzeugung von $w \in \Sigma^*$ und Startkonfiguration zw:

$$P_0 := \{S \rightarrow \begin{bmatrix} a \\ z_0 \langle a \end{bmatrix} A \mid a \in \Sigma\} \cup \{S \rightarrow \begin{bmatrix} a \\ z_0 \langle a \end{bmatrix} \mid a \in \Sigma\} \\ \cup \{A \rightarrow \begin{bmatrix} a \\ a \end{bmatrix} A \mid a \in \Sigma\} \cup \{A \rightarrow \begin{bmatrix} a \\ a \end{bmatrix} \mid a \in \Sigma\}$$

► Für $a_1 \cdots a_n \in \Sigma^*$ mit $n \geq 1$ gilt

$$S \Rightarrow^* \begin{bmatrix} a_1 \\ z_0 \langle a_1 \end{bmatrix} \begin{bmatrix} a_2 \\ a_2 \end{bmatrix} \cdots \begin{bmatrix} a_n \\ a_n \end{bmatrix}$$

Die akzeptierte Sprache von LBAs ist kontextsensitiv

Beweis (Fortsetzung) Konstruiere $G = (V, \Sigma - \{\langle, \rangle\}, P_0 \cup P_1 \cup P_2 \cup P_3, S)$.

1. Erzeuge ein beliebiges Wort $w \in \Sigma^*$ und die Startkonfiguration von M für w :

► Regeln zur Erzeugung von $w \in \Sigma^*$ und Startkonfiguration zw:

$$P_0 := \{S \rightarrow \begin{bmatrix} a \\ z_0 \langle a \rangle \end{bmatrix} A \mid a \in \Sigma\} \cup \{S \rightarrow \begin{bmatrix} a \\ z_0 \langle a \rangle \end{bmatrix} \mid a \in \Sigma\} \\ \cup \{A \rightarrow \begin{bmatrix} a \\ a \end{bmatrix} A \mid a \in \Sigma\} \cup \{A \rightarrow \begin{bmatrix} a \\ a \end{bmatrix} \mid a \in \Sigma\}$$

► Für $a_1 \cdots a_n \in \Sigma^*$ mit $n \geq 1$ gilt

$$S \Rightarrow^* \begin{bmatrix} a_1 \\ z_0 \langle a_1 \rangle \end{bmatrix} \begin{bmatrix} a_2 \\ a_2 \end{bmatrix} \cdots \begin{bmatrix} a_n \\ a_n \end{bmatrix}$$

► ε kann dadurch nicht erzeugt werden, daher:

$$P_1 := \{S \rightarrow \varepsilon \mid \varepsilon \in L(M)\}$$

Die akzeptierte Sprache von LBAs ist kontextsensitiv

Beweis (Fortsetzung)

2. Simuliere M zum Prüfen, ob $w \in L(M)$.

- Regelmenge P_2 simuliert M auf den unteren Komponenten:

$$\begin{aligned} P_2 := & \left\{ \begin{bmatrix} d \\ z a \end{bmatrix} \rightarrow \begin{bmatrix} d \\ z' b \end{bmatrix} \mid (z', b, N) \in \delta(z, a), d \in \Sigma \right\} \\ & \cup \left\{ \begin{bmatrix} d \\ c \end{bmatrix} \begin{bmatrix} e \\ z a \end{bmatrix} \rightarrow \begin{bmatrix} d \\ z' c \end{bmatrix} \begin{bmatrix} e \\ b \end{bmatrix} \mid (z', b, L) \in \delta(z, a), c \in \Gamma, d, e \in \Sigma \right\} \\ & \cup \left\{ \begin{bmatrix} d \\ z a \end{bmatrix} \begin{bmatrix} e \\ c \end{bmatrix} \rightarrow \begin{bmatrix} d \\ b \end{bmatrix} \begin{bmatrix} e \\ z' c \end{bmatrix} \mid (z', b, R) \in \delta(z, a), c \in \Gamma, d, e \in \Sigma \right\} \\ & \cup \dots \end{aligned}$$

- Weitere Regeln sind notwendig, um mit $\langle$ und $\rangle$ richtig umzugehen.

Die akzeptierte Sprache von LBAs ist kontextsensitiv

Beweis (Fortsetzung)

3. Wenn M akzeptiert, erzeuge w endgültig.

- ▶ Nach Akzeptieren durch M erstellt Regelmenge P_3 aus Tupelfolgen das Wort $a_1 \cdots a_n$.

$$\begin{aligned} P_3 := & \{ \begin{bmatrix} b \\ z a \end{bmatrix} \rightarrow b \mid z \in E, a \in \Gamma, b \in \Sigma \} \\ & \cup \{ \begin{bmatrix} b \\ a \end{bmatrix} \rightarrow b \mid a \in \Gamma, b \in \Sigma \} \\ & \cup \dots \end{aligned}$$

- ▶ Weitere Regeln sind notwendig um mit $\langle$ und $\rangle$ richtig umzugehen.

Die akzeptierte Sprache von LBAs ist kontextsensitiv

Beweis (Fortsetzung)

3. Wenn M akzeptiert, erzeuge w endgültig.

- ▶ Nach Akzeptieren durch M erstellt Regelmenge P_3 aus Tupelfolgen das Wort $a_1 \cdots a_n$.

$$P_3 := \left\{ \begin{bmatrix} b \\ z a \end{bmatrix} \rightarrow b \mid z \in E, a \in \Gamma, b \in \Sigma \right\} \\ \cup \left\{ \begin{bmatrix} b \\ a \end{bmatrix} \rightarrow b \mid a \in \Gamma, b \in \Sigma \right\} \\ \cup \dots$$

- ▶ Weitere Regeln sind notwendig um mit $\langle$ und $\rangle$ richtig umzugehen.
- ▶ Wenn $z \in E$, gilt

$$\begin{bmatrix} a_1 \\ b_1 \end{bmatrix} \cdots \begin{bmatrix} a_m \\ b_m \end{bmatrix} \begin{bmatrix} a_{m+1} \\ z b_{m+1} \end{bmatrix} \begin{bmatrix} a_{m+2} \\ b_{m+2} \end{bmatrix} \cdots \begin{bmatrix} a_n \\ b_n \end{bmatrix} \Rightarrow^* a_1 \cdots a_n$$

Die akzeptierte Sprache von LBAs ist kontextsensitiv

Beweis (Fortsetzung)

3. Wenn M akzeptiert, erzeuge w endgültig.

- ▶ Nach Akzeptieren durch M erstellt Regelmenge P_3 aus Tupelfolgen das Wort $a_1 \cdots a_n$.

$$P_3 := \left\{ \begin{bmatrix} b \\ z a \end{bmatrix} \rightarrow b \mid z \in E, a \in \Gamma, b \in \Sigma \right\} \\ \cup \left\{ \begin{bmatrix} b \\ a \end{bmatrix} \rightarrow b \mid a \in \Gamma, b \in \Sigma \right\} \\ \cup \dots$$

- ▶ Weitere Regeln sind notwendig um mit $\langle$ und $\rangle$ richtig umzugehen.
- ▶ Wenn $z \in E$, gilt

$$\begin{bmatrix} a_1 \\ b_1 \end{bmatrix} \cdots \begin{bmatrix} a_m \\ b_m \end{bmatrix} \begin{bmatrix} a_{m+1} \\ z b_{m+1} \end{bmatrix} \begin{bmatrix} a_{m+2} \\ b_{m+2} \end{bmatrix} \cdots \begin{bmatrix} a_n \\ b_n \end{bmatrix} \Rightarrow^* a_1 \cdots a_n$$

Dann gilt für alle $w \in (\Sigma - \{\langle, \rangle\})^*$: $S \Rightarrow_G^* w$ g.d.w. $w \in L(M)$.

Die akzeptierte Sprache von LBAs ist kontextsensitiv

Beweis (Fortsetzung)

3. Wenn M akzeptiert, erzeuge w endgültig.

- ▶ Nach Akzeptieren durch M erstellt Regelmenge P_3 aus Tupelfolgen das Wort $a_1 \cdots a_n$.

$$P_3 := \{ \begin{bmatrix} b \\ z a \end{bmatrix} \rightarrow b \mid z \in E, a \in \Gamma, b \in \Sigma \} \\ \cup \{ \begin{bmatrix} b \\ a \end{bmatrix} \rightarrow b \mid a \in \Gamma, b \in \Sigma \} \\ \cup \dots$$

- ▶ Weitere Regeln sind notwendig um mit $\langle$ und $\rangle$ richtig umzugehen.
- ▶ Wenn $z \in E$, gilt

$$\begin{bmatrix} a_1 \\ b_1 \end{bmatrix} \cdots \begin{bmatrix} a_m \\ b_m \end{bmatrix} \begin{bmatrix} a_{m+1} \\ z b_{m+1} \end{bmatrix} \begin{bmatrix} a_{m+2} \\ b_{m+2} \end{bmatrix} \cdots \begin{bmatrix} a_n \\ b_n \end{bmatrix} \Rightarrow^* a_1 \cdots a_n$$

Dann gilt für alle $w \in (\Sigma - \{\langle, \rangle\})^*$: $S \Rightarrow_G^* w$ g.d.w. $w \in L(M)$.

Außerdem gilt, dass G kontextsensitiv ist, da es keine verkürzenden Regeln gibt (außer $S \rightarrow \varepsilon$, falls vorhanden).



Anpassung des Beweises für Typ 0-Sprachen

Die Konstruktion der Typ 1-Grammatik aus einem LBA kann für beliebige NTMs angepasst werden.

Grundgedanke:

- ▶ Zusätzliche Tupel $\begin{bmatrix} \$ \\ v \end{bmatrix}$ für \$ ein neues Symbol.
- ▶ Konfiguration, die länger als das Eingabewort sind, werden so dargestellt:

$$\begin{bmatrix} a_1 \\ c_1 \end{bmatrix} \cdots \begin{bmatrix} a_n \\ c_n \end{bmatrix} \begin{bmatrix} \$ \\ c_{n+1} \end{bmatrix} \cdots \begin{bmatrix} \$ \\ z_j c_m \end{bmatrix} \begin{bmatrix} \$ \\ c_r \end{bmatrix}$$

- ▶ Regelmenge P_3 enthält Regeln $\begin{bmatrix} \$ \\ c_j \end{bmatrix} \rightarrow \varepsilon$, die nicht vom Typ 1 sind.

Anpassung des Beweises für Typ 0-Sprachen

Die Konstruktion der Typ 1-Grammatik aus einem LBA kann für beliebige NTMs angepasst werden.

Grundgedanke:

- ▶ Zusätzliche Tupel $\begin{bmatrix} \$ \\ v \end{bmatrix}$ für \$ ein neues Symbol.
- ▶ Konfiguration, die länger als das Eingabewort sind, werden so dargestellt:

$$\begin{bmatrix} a_1 \\ c_1 \end{bmatrix} \cdots \begin{bmatrix} a_n \\ c_n \end{bmatrix} \begin{bmatrix} \$ \\ c_{n+1} \end{bmatrix} \cdots \begin{bmatrix} \$ \\ z_j c_m \end{bmatrix} \begin{bmatrix} \$ \\ c_r \end{bmatrix}$$

- ▶ Regelmenge P_3 enthält Regeln $\begin{bmatrix} \$ \\ c_j \end{bmatrix} \rightarrow \varepsilon$, die nicht vom Typ 1 sind.

Daher:

Satz

Die nichtdeterministischen Turingmaschinen akzeptierten genau die Typ 0-Sprachen.

1. LBA-Problem

Erkennen deterministische LBAs dieselben Sprachen wie nichtdeterministische LBAs?

Das Problem ist bis heute ungeklärt.

1. LBA-Problem

Erkennen deterministische LBAs dieselben Sprachen wie nichtdeterministische LBAs?

Das Problem ist bis heute ungeklärt.

2. LBA-Problem

Sind die kontextsensitiven Sprachen abgeschlossen unter Komplementbildung?

Das Problem wurde 1964 formuliert von Kuroda, 1987 gelöst von sowohl Neil Immerman als auch Róbert Szelepcsényi.

1. LBA-Problem

Erkennen deterministische LBAs dieselben Sprachen wie nichtdeterministische LBAs?

Das Problem ist bis heute ungeklärt.

2. LBA-Problem

Sind die kontextsensitiven Sprachen abgeschlossen unter Komplementbildung?

Das Problem wurde 1964 formuliert von Kuroda, 1987 gelöst von sowohl Neil Immerman als auch Róbert Szelepcsényi.

Überraschenderweise **positiv**:

Theorem (Satz von Immerman und Szelepcsényi)

Die kontextsensitiven Sprachen sind abgeschlossen unter Komplementbildung.

2. LBA-Problem

Theorem (Satz von Immerman und Szelepcsényi)

Die kontextsensitiven Sprachen sind abgeschlossen unter Komplementbildung.

Naiver Beweis:

- ▶ Sei $G = (V, \Sigma, P, S)$ eine Typ 1-Grammatik mit $L(G) = L$.
- ▶ Konstruiere den LBA M für $\bar{L} = \Sigma^* \setminus L$.
 1. Sei $w \in \Sigma^*$.
 2. Prüfe, ob $S \Rightarrow_G^* w$ gilt (vgl. Beweis des Satzes von Kuroda).
 3. Wenn ja, dann verwirf.
 4. Wenn nein, dann akzeptiere.

2. LBA-Problem

Theorem (Satz von Immerman und Szelepcsényi)

Die kontextsensitiven Sprachen sind abgeschlossen unter Komplementbildung.

Naiver Beweis:

- ▶ Sei $G = (V, \Sigma, P, S)$ eine Typ 1-Grammatik mit $L(G) = L$.
- ▶ Konstruiere den LBA M für $\bar{L} = \Sigma^* \setminus L$.
 1. Sei $w \in \Sigma^*$.
 2. Prüfe, ob $S \Rightarrow_G^* w$ gilt (vgl. Beweis des Satzes von Kuroda).
 3. Wenn ja, dann verwirf.
 4. Wenn nein, dann akzeptiere.

Problem: Die Berechnung von $S \Rightarrow_G^* w$ mit einem LBA ist **nichtdeterministisch**.
Der Test $S \Rightarrow_G^* w$ kann **fehlschlagen**, auch wenn $S \Rightarrow_G^* w$ gilt.

2. LBA-Problem

Theorem (Satz von Immerman und Szelepcsényi)

Die kontextsensitiven Sprachen sind abgeschlossen unter Komplementbildung.

Naiver Beweis:

- ▶ Sei $G = (V, \Sigma, P, S)$ eine Typ 1-Grammatik mit $L(G) = L$.
- ▶ Konstruiere den LBA M für $\bar{L} = \Sigma^* \setminus L$.
 1. Sei $w \in \Sigma^*$.
 2. Prüfe, ob $S \Rightarrow_G^* w$ gilt (vgl. Beweis des Satzes von Kuroda).
 3. Wenn ja, dann verwirf.
 4. Wenn nein, dann akzeptiere.

Problem: Die Berechnung von $S \Rightarrow_G^* w$ mit einem LBA ist **nichtdeterministisch**.
Der Test $S \Rightarrow_G^* w$ kann **fehlschlagen**, auch wenn $S \Rightarrow_G^* w$ gilt.

Lösung: Siehe Skript.

8b

Entscheiden des Wortproblems für Typ 1-Grammatiken

Prof. Dr. Jasmin Blanchette

Lehr- und Forschungseinheit für
Theoretische Informatik und TheorembeweisenStand: 11. Juni 2024
Basierend auf Folien von PD Dr. David Sabel

Definition

Eine Sprache L ist **entscheidbar**, wenn es einen Algorithmus gibt, der bei Eingabe eines Wortes w in endlicher Zeit feststellt, ob $w \in L$ gilt oder nicht.

Wortproblem für Typ 1-Grammatiken

Definition

Das **Wortproblem** für Typ i -Grammatiken ist die Frage, ob für eine gegebene Typ i -Grammatik $G = (V, \Sigma, P, S)$ und ein Wort $w \in \Sigma^*$ $w \in L(G)$ gilt oder nicht.

Wortproblem für Typ 1-Grammatiken

Definition

Das **Wortproblem** für Typ i -Grammatiken ist die Frage, ob für eine gegebene Typ i -Grammatik $G = (V, \Sigma, P, S)$ und ein Wort $w \in \Sigma^*$ $w \in L(G)$ gilt oder nicht.

Satz

Das Wortproblem für Typ 1-Grammatiken ist entscheidbar:
Es gibt einen Algorithmus, der bei Eingabe von Typ 1-Grammatik G und Wort w nach endlicher Zeit entscheidet, ob $w \in L(G)$ gilt oder nicht.

Wortproblem für Typ 1-Grammatiken

Definition

Das **Wortproblem** für Typ i -Grammatiken ist die Frage, ob für eine gegebene Typ i -Grammatik $G = (V, \Sigma, P, S)$ und ein Wort $w \in \Sigma^*$ $w \in L(G)$ gilt oder nicht.

Satz

Das Wortproblem für Typ 1-Grammatiken ist entscheidbar:
Es gibt einen Algorithmus, der bei Eingabe von Typ 1-Grammatik G und Wort w nach endlicher Zeit entscheidet, ob $w \in L(G)$ gilt oder nicht.

Beweis Algorithmus 2 auf späterer Folie bewerkstelligt dies.



Ein naiver Ansatz

Seien eine Grammatik $G = (V, \Sigma, P, S)$ und ein Wort $w \in \Sigma^*$.

Schritte:

1. Beginne mit der Menge $L_0 := \{S\}$.
2. Wiederhole für $i = 0, 1, 2, \dots$:
 - 2.1 Wende die Produktionen von P auf die Satzformen in L_i an.
Sei N das Ergebnis. Setze $L_{i+1} := L_i \cup N$.
 - 2.2 Falls $w \in L_{i+1}$, stoppe und gib ja aus.
 - 2.3 Falls $L_{i+1} = L_i$, stoppe und gib nein aus.

Ein naiver Ansatz

Seien eine Grammatik $G = (V, \Sigma, P, S)$ und ein Wort $w \in \Sigma^*$.

Schritte:

1. Beginne mit der Menge $L_0 := \{S\}$.
2. Wiederhole für $i = 0, 1, 2, \dots$:
 - 2.1 Wende die Produktionen von P auf die Satzformen in L_i an.
Sei N das Ergebnis. Setze $L_{i+1} := L_i \cup N$.
 - 2.2 Falls $w \in L_{i+1}$, stoppe und gib ja aus.
 - 2.3 Falls $L_{i+1} = L_i$, stoppe und gib nein aus.

Problem: Der Ansatz terminiert nicht zwingend.

Ein naiver Ansatz

Seien eine Grammatik $G = (V, \Sigma, P, S)$ und ein Wort $w \in \Sigma^*$.

Schritte:

1. Beginne mit der Menge $L_0 := \{S\}$.
2. Wiederhole für $i = 0, 1, 2, \dots$:
 - 2.1 Wende die Produktionen von P auf die Satzformen in L_i an.
Sei N das Ergebnis. Setze $L_{i+1} := L_i \cup N$.
 - 2.2 Falls $w \in L_{i+1}$, stoppe und gib ja aus.
 - 2.3 Falls $L_{i+1} = L_i$, stoppe und gib nein aus.

Problem: Der Ansatz terminiert nicht zwingend.

Bei Nichtterminierung ist $w \notin L$ (aber das erfahren wir nie).

Beispiel für den naiven Ansatz

Grammatik $G = (\{S, B\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBc, S \rightarrow abc, cB \rightarrow Bc, bB \rightarrow bb\}$ und
Wort $aabbcc$

Beispiel für den naiven Ansatz

Grammatik $G = (\{S, B\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBc, S \rightarrow abc, cB \rightarrow Bc, bB \rightarrow bb\}$ und
Wort $aabbcc$

Schritte:

0. $L_0 := \{S\}$

Beispiel für den naiven Ansatz

Grammatik $G = (\{S, B\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBc, S \rightarrow abc, cB \rightarrow Bc, bB \rightarrow bb\}$ und
Wort $aabbcc$

Schritte:

0. $L_0 := \{S\}$

1. $L_1 := L_0 \cup \{aSBc, abc\}$

Beispiel für den naiven Ansatz

Grammatik $G = (\{S, B\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBc, S \rightarrow abc, cB \rightarrow Bc, bB \rightarrow bb\}$ und
Wort $aabbcc$

Schritte:

0. $L_0 := \{S\}$
1. $L_1 := L_0 \cup \{aSBc, abc\}$

Beispiel für den naiven Ansatz

Grammatik $G = (\{S, B\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBc, S \rightarrow abc, cB \rightarrow Bc, bB \rightarrow bb\}$ und
Wort $aabbcc$

Schritte:

0. $L_0 := \{S\}$
1. $L_1 := L_0 \cup \{aSBc, abc\}$

Beispiel für den naiven Ansatz

Grammatik $G = (\{S, B\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBc, S \rightarrow abc, cB \rightarrow Bc, bB \rightarrow bb\}$ und
Wort $aabbcc$

Schritte:

0. $L_0 := \{S\}$
1. $L_1 := L_0 \cup \{aSBc, abc\}$
2. $L_2 := L_1 \cup \{aaSBcBc, aabcBc\}$

Beispiel für den naiven Ansatz

Grammatik $G = (\{S, B\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBc, S \rightarrow abc, cB \rightarrow Bc, bB \rightarrow bb\}$ und
Wort $aabbcc$

Schritte:

0. $L_0 := \{S\}$
1. $L_1 := L_0 \cup \{aSBc, abc\}$
2. $L_2 := L_1 \cup \{aaSBcBc, aabcBc\}$

Beispiel für den naiven Ansatz

Grammatik $G = (\{S, B\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBc, S \rightarrow abc, cB \rightarrow Bc, bB \rightarrow bb\}$ und
Wort $aabbcc$

Schritte:

0. $L_0 := \{S\}$
1. $L_1 := L_0 \cup \{aSBc, abc\}$
2. $L_2 := L_1 \cup \{aaSBcBc, aabcBc\}$

Beispiel für den naiven Ansatz

Grammatik $G = (\{S, B\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBc, S \rightarrow abc, cB \rightarrow Bc, bB \rightarrow bb\}$ und
Wort $aabbcc$

Schritte:

0. $L_0 := \{S\}$
1. $L_1 := L_0 \cup \{aSBc, abc\}$
2. $L_2 := L_1 \cup \{aaSBcBc, aabcBc\}$
3. $L_3 := L_2 \cup \{aaaSBcBcBc, aaabcBcBc, aaSBBcc, aabBcc\}$

Beispiel für den naiven Ansatz

Grammatik $G = (\{S, B\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBc, S \rightarrow abc, cB \rightarrow Bc, bB \rightarrow bb\}$ und
Wort $aabbcc$

Schritte:

0. $L_0 := \{S\}$
1. $L_1 := L_0 \cup \{aSBc, abc\}$
2. $L_2 := L_1 \cup \{aaSBcBc, aabcBc\}$
3. $L_3 := L_2 \cup \{aaaSBcBcBc, aaabcBcBc, aaSBBcc, aabBcc\}$

Beispiel für den naiven Ansatz

Grammatik $G = (\{S, B\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBc, \textcolor{red}{S} \rightarrow \textcolor{blue}{abc}, cB \rightarrow Bc, bB \rightarrow bb\}$ und
Wort $aabbcc$

Schritte:

0. $L_0 := \{S\}$
1. $L_1 := L_0 \cup \{aSBc, abc\}$
2. $L_2 := L_1 \cup \{aa\textcolor{red}{S}BcBc, aabcBc\}$
3. $L_3 := L_2 \cup \{aaaSBcBcBc, aa\textcolor{blue}{abc}BcBc, aaSBBcc, aabBcc\}$

Beispiel für den naiven Ansatz

Grammatik $G = (\{S, B\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBc, S \rightarrow abc, cB \rightarrow Bc, bB \rightarrow bb\}$ und
Wort $aabbcc$

Schritte:

0. $L_0 := \{S\}$
1. $L_1 := L_0 \cup \{aSBc, abc\}$
2. $L_2 := L_1 \cup \{aaSBcBc, aabcBc\}$
3. $L_3 := L_2 \cup \{aaaSBcBcBc, aaabcBcBc, aaSBcBc, aabBcc\}$

Beispiel für den naiven Ansatz

Grammatik $G = (\{S, B\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBc, S \rightarrow abc, cB \rightarrow Bc, bB \rightarrow bb\}$ und
Wort $aabbcc$

Schritte:

0. $L_0 := \{S\}$
1. $L_1 := L_0 \cup \{aSBc, abc\}$
2. $L_2 := L_1 \cup \{aaSBcBc, aabcBc\}$
3. $L_3 := L_2 \cup \{aaaSBcBcBc, aaabcBcBc, aaSBBcc, aabBcc\}$

Beispiel für den naiven Ansatz

Grammatik $G = (\{S, B\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBc, S \rightarrow abc, cB \rightarrow Bc, bB \rightarrow bb\}$ und
Wort $aabbcc$

Schritte:

0. $L_0 := \{S\}$
1. $L_1 := L_0 \cup \{aSBc, abc\}$
2. $L_2 := L_1 \cup \{aaSBcBc, aabcBc\}$
3. $L_3 := L_2 \cup \{aaaSBcBcBc, aaabcBcBc, aaSBBcc, aabBcc\}$
4. $L_4 := L_3 \cup \{aaaaSBcBcBcBc, aaaabcBcBcBc, \dots, aabbcc\}$

Beispiel für den naiven Ansatz

Grammatik $G = (\{S, B\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBc, S \rightarrow abc, cB \rightarrow Bc, bB \rightarrow bb\}$ und
Wort $aabbcc$

Schritte:

0. $L_0 := \{S\}$
1. $L_1 := L_0 \cup \{aSBc, abc\}$
2. $L_2 := L_1 \cup \{aaSBcBc, aabcBc\}$
3. $L_3 := L_2 \cup \{aaaSBcBcBc, aaabcBcBc, aaSBBcc, aabBcc\}$
4. $L_4 := L_3 \cup \{aaaaSBcBcBcBc, aaaabcBcBcBc, \dots, aabbcc\}$

Beispiel für den naiven Ansatz

Grammatik $G = (\{S, B\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBc, \textcolor{red}{S} \rightarrow \textcolor{blue}{abc}, cB \rightarrow Bc, bB \rightarrow bb\}$ und
Wort $aabbcc$

Schritte:

0. $L_0 := \{S\}$
1. $L_1 := L_0 \cup \{aSBc, abc\}$
2. $L_2 := L_1 \cup \{aaSBcBc, aabcBc\}$
3. $L_3 := L_2 \cup \{aaa\textcolor{red}{S}BcBcBc, aaabcBcBc, aaSBBcc, aabBcc\}$
4. $L_4 := L_3 \cup \{aaaaSBcBcBcBc, aaa\textcolor{blue}{abc}BcBcBc, \dots, aabbcc\}$

Beispiel für den naiven Ansatz

Grammatik $G = (\{S, B\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBc, S \rightarrow abc, cB \rightarrow Bc, \textcolor{red}{b}B \rightarrow \textcolor{blue}{b}b\}$ und
Wort $aabbcc$

Schritte:

0. $L_0 := \{S\}$
1. $L_1 := L_0 \cup \{aSBc, abc\}$
2. $L_2 := L_1 \cup \{aaSBcBc, aabcBc\}$
3. $L_3 := L_2 \cup \{aaaSBcBcBc, aaabcBcBc, aaSBBcc, aa\textcolor{red}{b}Bcc\}$
4. $L_4 := L_3 \cup \{aaaaSBcBcBcBc, aaaabcBcBcBc, \dots, aa\textcolor{blue}{b}bcc\}$

Beispiel für den naiven Ansatz

Grammatik $G = (\{S, B\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBc, S \rightarrow abc, cB \rightarrow Bc, bB \rightarrow bb\}$ und
Wort $aabbcc$

Schritte:

0. $L_0 := \{S\}$
1. $L_1 := L_0 \cup \{aSBc, abc\}$
2. $L_2 := L_1 \cup \{aaSBcBc, aabcBc\}$
3. $L_3 := L_2 \cup \{aaaSBcBcBc, aaabcBcBc, aaSBBcc, aabBcc\}$
4. $L_4 := L_3 \cup \{aaaaSBcBcBcBc, aaaabcBcBcBc, \dots, aabbcc\}$

Aus $aabbcc \in L_4$ folgt $aabbcc \in L(G)$.

Weiteres Beispiel für den naiven Ansatz

Grammatik $G = (\{S, B\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBc, S \rightarrow abc, cB \rightarrow Bc, bB \rightarrow bb\}$ und
Wort *cababa*

Weiteres Beispiel für den naiven Ansatz

Grammatik $G = (\{S, B\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBc, S \rightarrow abc, cB \rightarrow Bc, bB \rightarrow bb\}$ und
Wort $cababa$

Schritte:

0. $L_0 := \{S\}$
1. $L_1 := L_0 \cup \{aSBc, abc\}$
2. $L_2 := L_1 \cup \{aaSBcBc, aabcBc\}$
3. $L_3 := L_2 \cup \{aaaSBcBcBc, aaabcBcBc, aaSBBcc, aabBcc\}$
- $\vdots$

Weiteres Beispiel für den naiven Ansatz

Grammatik $G = (\{S, B\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBc, S \rightarrow abc, cB \rightarrow Bc, bB \rightarrow bb\}$ und
Wort $cababa$

Schritte:

0. $L_0 := \{S\}$
1. $L_1 := L_0 \cup \{aSBc, abc\}$
2. $L_2 := L_1 \cup \{aaSBcBc, aabcBc\}$
3. $L_3 := L_2 \cup \{aaaSBcBcBc, aaabcBcBc, aaSBBcc, aabBcc\}$
- $\vdots$

Der Ansatz **terminiert nicht**.

D.h. $cababa \notin L(G)$ (aber das erfahren wir nie).

Ein besserer Ansatz

Seien eine Grammatik $G = (V, \Sigma, P, S)$ und ein Wort $w \in \Sigma^*$ der Länge n .
Schritte:

1. Beginne mit der Menge $L_0^n := \{S\}$.
(Hier ist n ein Index und keine Potenz.)
2. Wiederhole für $i = 0, 1, 2, \dots$:
 - 2.1 Wende die Produktionen von P auf die Satzformen in L_i^n an.
Sei N das Ergebnis. Setze $L_{i+1}^n := L_i^n \cup N'$,
wobei N' aus den Satzformen der Länge $\leq n$ aus N besteht.
 - 2.2 Falls $w \in L_{i+1}^n$, stoppe und gib **ja** aus.
 - 2.3 Falls $L_{i+1}^n = L_i^n$, stoppe und gib **nein** aus.

Ein besserer Ansatz

Seien eine Grammatik $G = (V, \Sigma, P, S)$ und ein Wort $w \in \Sigma^*$ der Länge n .
Schritte:

1. Beginne mit der Menge $L_0^n := \{S\}$.
(Hier ist n ein Index und keine Potenz.)
2. Wiederhole für $i = 0, 1, 2, \dots$:
 - 2.1 Wende die Produktionen von P auf die Satzformen in L_i^n an.
Sei N das Ergebnis. Setze $L_{i+1}^n := L_i^n \cup N'$,
wobei N' aus den Satzformen der Länge $\leq n$ aus N besteht.
 - 2.2 Falls $w \in L_{i+1}^n$, stoppe und gib **ja** aus.
 - 2.3 Falls $L_{i+1}^n = L_i^n$, stoppe und gib **nein** aus.

Der Ansatz **terminiert immer** (Beweis später),
hat aber **exponentielle Laufzeitkomplexität**.

Ein besserer Ansatz

Seien eine Grammatik $G = (V, \Sigma, P, S)$ und ein Wort $w \in \Sigma^*$ der Länge n .
Schritte:

1. Beginne mit der Menge $L_0^n := \{S\}$.
(Hier ist n ein Index und keine Potenz.)
2. Wiederhole für $i = 0, 1, 2, \dots$:
 - 2.1 Wende die Produktionen von P auf die Satzformen in L_i^n an.
Sei N das Ergebnis. Setze $L_{i+1}^n := L_i^n \cup N'$,
wobei N' aus den Satzformen der Länge $\leq n$ aus N besteht.
 - 2.2 Falls $w \in L_{i+1}^n$, stoppe und gib **ja** aus.
 - 2.3 Falls $L_{i+1}^n = L_i^n$, stoppe und gib **nein** aus.

Der Ansatz **terminiert immer** (Beweis später),
hat aber **exponentielle Laufzeitkomplexität**.

Er ist auch **inkorrekt für Typ 0-Grammatiken**.

Deshalb **fokussieren wir uns auf Typ 1-Grammatiken** (mit 1. Sonderregel).

Beispiel für den besseren Ansatz

Grammatik $G = (\{S, B\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBc, S \rightarrow abc, cB \rightarrow Bc, bB \rightarrow bb\}$ und
Wort $aabbcc$

Schritte:

Beispiel für den besseren Ansatz

Grammatik $G = (\{S, B\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBc, S \rightarrow abc, cB \rightarrow Bc, bB \rightarrow bb\}$ und
Wort $aabbcc$

Schritte:

0. $L_0^6 := \{S\}$

Beispiel für den besseren Ansatz

Grammatik $G = (\{S, B\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBc, S \rightarrow abc, cB \rightarrow Bc, bB \rightarrow bb\}$ und
Wort $aabbcc$

Schritte:

0. $L_0^6 := \{S\}$
1. $L_1^6 := L_0^6 \cup \{aSBc, abc\}$

Beispiel für den besseren Ansatz

Grammatik $G = (\{S, B\}, \{a, b, c\}, P, S)$ mit
 $P = \{\textcolor{red}{S} \rightarrow \textcolor{blue}{aSBc}, S \rightarrow abc, cB \rightarrow Bc, bB \rightarrow bb\}$ und
Wort $aabbcc$

Schritte:

0. $L_0^6 := \{\textcolor{red}{S}\}$
1. $L_1^6 := L_0^6 \cup \{\textcolor{blue}{aSBc}, abc\}$

Beispiel für den besseren Ansatz

Grammatik $G = (\{S, B\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBc, S \rightarrow abc, cB \rightarrow Bc, bB \rightarrow bb\}$ und
Wort $aabbcc$

Schritte:

0. $L_0^6 := \{S\}$
1. $L_1^6 := L_0^6 \cup \{aSBc, abc\}$

Beispiel für den besseren Ansatz

Grammatik $G = (\{S, B\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBc, S \rightarrow abc, cB \rightarrow Bc, bB \rightarrow bb\}$ und
Wort $aabbcc$

Schritte:

0. $L_0^6 := \{S\}$
1. $L_1^6 := L_0^6 \cup \{aSBc, abc\}$
2. $L_2^6 := L_1^6 \cup \{aabcBc\}$

Beispiel für den besseren Ansatz

Grammatik $G = (\{S, B\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBc, S \rightarrow abc, cB \rightarrow Bc, bB \rightarrow bb\}$ und
Wort $aabbcc$

Schritte:

0. $L_0^6 := \{S\}$
1. $L_1^6 := L_0^6 \cup \{aS Bc, abc\}$
2. $L_2^6 := L_1^6 \cup \{aabc Bc\}$

Beispiel für den besseren Ansatz

Grammatik $G = (\{S, B\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBc, S \rightarrow abc, cB \rightarrow Bc, bB \rightarrow bb\}$ und
Wort $aabbcc$

Schritte:

0. $L_0^6 := \{S\}$
1. $L_1^6 := L_0^6 \cup \{aSBc, abc\}$
2. $L_2^6 := L_1^6 \cup \{aabcBc\}$ ~~$aSBc \Rightarrow aaSBcBc$~~

Beispiel für den besseren Ansatz

Grammatik $G = (\{S, B\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBc, S \rightarrow abc, cB \rightarrow Bc, bB \rightarrow bb\}$ und
Wort $aabbcc$

Schritte:

0. $L_0^6 := \{S\}$
1. $L_1^6 := L_0^6 \cup \{aSBc, abc\}$
2. $L_2^6 := L_1^6 \cup \{aabcBc\}$
3. $L_3^6 := L_2^6 \cup \{aabBcc\}$

Beispiel für den besseren Ansatz

Grammatik $G = (\{S, B\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBc, S \rightarrow abc, cB \rightarrow Bc, bB \rightarrow bb\}$ und
Wort $aabbcc$

Schritte:

0. $L_0^6 := \{S\}$
1. $L_1^6 := L_0^6 \cup \{aSBc, abc\}$
2. $L_2^6 := L_1^6 \cup \{aab\textcolor{red}{c}Bc\}$
3. $L_3^6 := L_2^6 \cup \{aab\textcolor{blue}{B}cc\}$

Beispiel für den besseren Ansatz

Grammatik $G = (\{S, B\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBc, S \rightarrow abc, cB \rightarrow Bc, \textcolor{red}{bB} \rightarrow \textcolor{blue}{bb}\}$ und
Wort $aabbcc$

Schritte:

0. $L_0^6 := \{S\}$
1. $L_1^6 := L_0^6 \cup \{aSBc, abc\}$
2. $L_2^6 := L_1^6 \cup \{aabcBc\}$
3. $L_3^6 := L_2^6 \cup \{aabBcc\}$
4. $L_4^6 := L_3^6 \cup \{aabbcc\}$

Beispiel für den besseren Ansatz

Grammatik $G = (\{S, B\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBc, S \rightarrow abc, cB \rightarrow Bc, bB \rightarrow bb\}$ und
Wort $aabbcc$

Schritte:

0. $L_0^6 := \{S\}$
1. $L_1^6 := L_0^6 \cup \{aSBc, abc\}$
2. $L_2^6 := L_1^6 \cup \{aabcBc\}$
3. $L_3^6 := L_2^6 \cup \{aa\textcolor{red}{b}Bcc\}$
4. $L_4^6 := L_3^6 \cup \{aa\textcolor{blue}{b}bcc\}$

Beispiel für den besseren Ansatz

Grammatik $G = (\{S, B\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBc, S \rightarrow abc, cB \rightarrow Bc, bB \rightarrow bb\}$ und
Wort $aabbcc$

Schritte:

0. $L_0^6 := \{S\}$
1. $L_1^6 := L_0^6 \cup \{aSBc, abc\}$
2. $L_2^6 := L_1^6 \cup \{aabcBc\}$
3. $L_3^6 := L_2^6 \cup \{aabBcc\}$
4. $L_4^6 := L_3^6 \cup \{aabbcc\}$
5. $L_5^6 := L_4^6$

Beispiel für den besseren Ansatz

Grammatik $G = (\{S, B\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBc, S \rightarrow abc, cB \rightarrow Bc, bB \rightarrow bb\}$ und
Wort $aabbcc$

Schritte:

0. $L_0^6 := \{S\}$
1. $L_1^6 := L_0^6 \cup \{aSBc, abc\}$
2. $L_2^6 := L_1^6 \cup \{aabcBc\}$
3. $L_3^6 := L_2^6 \cup \{aabBcc\}$
4. $L_4^6 := L_3^6 \cup \{aabbcc\}$
5. $L_5^6 := L_4^6$

Aus $aabbcc \in L_4^6$ folgt $aabbcc \in L(G)$.

Weiteres Beispiel für den besseren Ansatz

Grammatik $G = (\{S, B\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBc, S \rightarrow abc, cB \rightarrow Bc, bB \rightarrow bb\}$ und
Wort *cababa*

Weiteres Beispiel für den besseren Ansatz

Grammatik $G = (\{S, B\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBc, S \rightarrow abc, cB \rightarrow Bc, bB \rightarrow bb\}$ und
Wort *cababa*

Schritte:

0. $L_0^6 := \{S\}$
1. $L_1^6 := L_0^6 \cup \{aSBc, abc\}$
2. $L_2^6 := L_1^6 \cup \{aabcBc\}$
3. $L_3^6 := L_2^6 \cup \{aabBcc\}$
4. $L_4^6 := L_3^6 \cup \{aabbcc\}$
5. $L_5^6 := L_4^6$

Weiteres Beispiel für den besseren Ansatz

Grammatik $G = (\{S, B\}, \{a, b, c\}, P, S)$ mit
 $P = \{S \rightarrow aSBc, S \rightarrow abc, cB \rightarrow Bc, bB \rightarrow bb\}$ und
Wort $cababa$

Schritte:

0. $L_0^6 := \{S\}$
1. $L_1^6 := L_0^6 \cup \{aSBc, abc\}$
2. $L_2^6 := L_1^6 \cup \{aabcBc\}$
3. $L_3^6 := L_2^6 \cup \{aabBcc\}$
4. $L_4^6 := L_3^6 \cup \{aabbcc\}$
5. $L_5^6 := L_4^6$

Aus $cababa \notin L_4^6$ folgt $cababa \notin L(G)$.

Gegenbeispiel für eine Typ 0-Grammatik

Grammatik $G = (\{S\}, \{a, b\}, P, S)$ mit
 $P = \{S \rightarrow aaaa, aaaa \rightarrow bb\}$ und
Wort bb

Schritte:

Gegenbeispiel für eine Typ 0-Grammatik

Grammatik $G = (\{S\}, \{a, b\}, P, S)$ mit
 $P = \{S \rightarrow aaaa, aaaa \rightarrow bb\}$ und
Wort bb

Schritte:

0. $L_0^2 := \{S\}$

Gegenbeispiel für eine Typ 0-Grammatik

Grammatik $G = (\{S\}, \{a, b\}, P, S)$ mit
 $P = \{S \rightarrow aaaa, aaaa \rightarrow bb\}$ und
Wort bb

Schritte:

0. $L_0^2 := \{S\}$
1. $L_1^2 := L_0^2$

Gegenbeispiel für eine Typ 0-Grammatik

Grammatik $G = (\{S\}, \{a, b\}, P, S)$ mit
 $P = \{S \rightarrow aaaa, aaaa \rightarrow bb\}$ und
Wort bb

Schritte:

0. $L_0^2 := \{S\}$

1. $L_1^2 := L_0^2$

$bb \notin L_0^2$ und jedoch $bb \in L(G)$.

Formale Definition der Menge L_i^n

Definition

Sei $G = (V, \Sigma, P, S)$ eine Typ 1-Grammatik. Für $i, n \in \mathbb{N}$ sei

$$L_i^n := \{w \in (V \cup \Sigma)^* \mid |w| \leq n \text{ und } S \Rightarrow_G^k w \text{ mit } k \leq i\}$$

Formale Definition der Menge L_i^n

Definition

Sei $G = (V, \Sigma, P, S)$ eine Typ 1-Grammatik. Für $i, n \in \mathbb{N}$ sei

$$L_i^n := \{w \in (V \cup \Sigma)^* \mid |w| \leq n \text{ und } S \Rightarrow_G^k w \text{ mit } k \leq i\}$$

Informell:

L_i^n = Menge aller Satzformen der Länge höchstens n ,
die in höchstens i Schritten von S aus ableitbar sind

Formale Definition der Prozedur

Seien eine Typ 1-Grammatik $G = (V, \Sigma, P, S)$ (mit 1. Sonderregel) und ein Wort $w \in \Sigma^*$.

Sei

$$\text{next}(L, n) := L \cup \{v \mid u \in L, u \Rightarrow_G v \text{ und } |v| \leq n\}$$

Schritte:

1. Beginne mit $L_0^n := \{S\}$ wenn $n > 0$, sonst $L_0^n := \emptyset$.
2. Wiederhole für $i = 0, 1, 2, \dots$:
 - 2.1 Setze $L_{i+1}^n := \text{next}(L_i^n, n)$.
 - 2.2 Falls $w \in L_{i+1}^n$, stoppe und gib ja aus.
 - 2.3 Falls $L_{i+1}^n = L_i^n$, stoppe und gib nein aus.

Terminierung der Prozedur

Satz

Die Prozedur terminiert.

Terminierung der Prozedur

Satz

Die Prozedur terminiert.

Beweis Durch Widerspruch. Wir nehmen an, dass die Berechnung nicht stoppt.

Satz

Die Prozedur terminiert.

Beweis Durch Widerspruch. Wir nehmen an, dass die Berechnung nicht stoppt.

Da $L_i^n \subseteq L_{i+1}^n$ und $L_i^n \neq L_{i+1}^n$, muss für alle $i \in \mathbb{N}$ gelten: $L_i^n \subset L_{i+1}^n$.

Satz

Die Prozedur terminiert.

Beweis Durch Widerspruch. Wir nehmen an, dass die Berechnung nicht stoppt.

Da $L_i^n \subseteq L_{i+1}^n$ und $L_i^n \neq L_{i+1}^n$, muss für alle $i \in \mathbb{N}$ gelten: $L_i^n \subset L_{i+1}^n$.

Daher gilt $|L_0^n| < |L_1^n| < |L_2^n| < \dots$. Die Mengen werden also beliebig groß.

Satz

Die Prozedur terminiert.

Beweis Durch Widerspruch. Wir nehmen an, dass die Berechnung nicht stoppt.

Da $L_i^n \subseteq L_{i+1}^n$ und $L_i^n \neq L_{i+1}^n$, muss für alle $i \in \mathbb{N}$ gelten: $L_i^n \subset L_{i+1}^n$.

Daher gilt $|L_0^n| < |L_1^n| < |L_2^n| < \dots$. Die Mengen werden also beliebig groß.

Gleichzeitig gibt es eine obere Schranke für die Mächtigkeit der Mengen:

$|L_i^n| \leq (|V \cup \Sigma| + 1)^n$ für alle $i \in \mathbb{N}$. Widerspruch. □

Satz

Die Prozedur ist korrekt: Sei w ein Wort der Länge $\leq n$.
Wenn $L_{i+1}^n = L_i^n$, dann ist $w \in L_i^n$ g.d.w. $w \in L(G)$.

Satz

Die Prozedur ist korrekt: Sei w ein Wort der Länge $\leq n$.

Wenn $L_{i+1}^n = L_i^n$, dann ist $w \in L_i^n$ g.d.w. $w \in L(G)$.

Beweis Zur Erinnerung:

L_i^n = Menge aller Satzformen der Länge höchstens n ,
die in höchstens i Schritten von S aus ableitbar sind

Satz

Die Prozedur ist korrekt: Sei w ein Wort der Länge $\leq n$.

Wenn $L_{i+1}^n = L_i^n$, dann ist $w \in L_i^n$ g.d.w. $w \in L(G)$.

Beweis Zur Erinnerung:

L_i^n = Menge aller Satzformen der Länge höchstens n ,
die in höchstens i Schritten von S aus ableitbar sind

Wenn $L_{i+1}^n = L_i^n$, dann gilt $L_{i+k}^n = L_i^n$ für alle $k \in \mathbb{N}$ und daher enthält L_i^n alle aus S ableitbaren Wörter der Länge höchstens n und keine anderen. $\square$

Algorithmus 2: Entscheiden des Wortproblems für Typ 1-Grammatiken

Eingabe: Typ 1-Grammatik $G = (V, \Sigma, P, S)$ (mit 1. Sonderregel) und $w \in \Sigma^*$

Ausgabe: *Ja*, wenn $w \in L(G)$, und *Nein*, wenn $w \notin L(G)$

Beginn

$n := |w|;$

$L := \{S\};$

wiederhole

$L_{\text{old}} := L;$

$L := \text{next}(L_{\text{old}}, n);$

bis $w \in L$ oder $L_{\text{old}} = L;$

wenn $w \in L$ **dann**

return *Ja*;

sonst

return *Nein*;

Ende

Ende

- ▶ Das Wortproblem für Typ 0-Grammatiken ist **unentscheidbar**.

- ▶ Das Wortproblem für Typ 0-Grammatiken ist **unentscheidbar**.
- ▶ Der Algorithmus für Typ 1-Grammatiken hat **exponentielle Laufzeitkomplexität**.

- ▶ Das Wortproblem für Typ 0-Grammatiken ist **unentscheidbar**.
- ▶ Der Algorithmus für Typ 1-Grammatiken hat **exponentielle Laufzeitkomplexität**.
- ▶ Das Wortproblem für Typ 2- (und 3-)Grammatiken ist in **polynomieller Zeit** lösbar (durch den CYK-Algorithmus).

8c

Intuitive und Turing-Berechenbarkeit

Prof. Dr. Jasmin Blanchette

Lehr- und Forschungseinheit für
Theoretische Informatik und Theorembeweisen

Stand: 25. Juni 2024
Basierend auf Folien von PD Dr. David Sabel



Intuitive Berechenbarkeit

- ▶ Was **kann** mit einem Computerprogramm berechnet werden?
- ▶ Was **kann nicht** mit einem Computerprogramm berechnet werden?
- ▶ Aus der Programmiererfahrung hat man ein gewisses Gefühl dafür, was berechnet werden kann (und was nicht).
- ▶ Das ist der **intuitive Begriff der Berechenbarkeit**.
- ▶ Wie beweist man, dass etwas **nicht berechnet** werden kann?
Diese Frage ist auch von praktischem Nutzen:
Die Suche nach passendem Algorithmus ist sinnlos.

- ▶ Berühmte Mathematiker/Informatiker versuchten in den 1930er Jahren den Begriff der Berechenbarkeit zu formalisieren.
- ▶ Dafür entwarfen sie verschiedene Modelle.
- ▶ Insbesondere sind zu nennen:
Alan Turing (Turingmaschine) und
Alonzo Church (Lambda-Kalkül)

Berechenbare Funktion

Eine (partielle oder totale) Funktion $f : \mathbb{N}^k \rightarrow \mathbb{N}$ nennen wir **berechenbar**, wenn es einen Algorithmus einer modernen Programmiersprache gibt, der f berechnet.

- ▶ Bei Eingabe $(n_1, \dots, n_k)$ stoppt der Algorithmus nach endlich vielen Berechnungsschritten und gibt den Wert von $f(n_1, \dots, n_k)$ aus.
- ▶ Wenn $f(n_1, \dots, n_k)$ undefiniert ist (f ist also eine partielle Funktion), dann stoppt der Algorithmus nicht.

Beispiele für berechenbare Funktionen

Der Algorithmus

Eingabe: Zahl $n \in \mathbb{N}$

Beginn

```
| solange true tue  
|   | skip
```

berechnet $f_1 : \mathbb{N} \rightarrow \mathbb{N}$ mit $f_1(x) = \text{undefiniert}$ für alle x .

Beispiele für berechenbare Funktionen

Der Algorithmus

Eingabe: Zahl $n \in \mathbb{N}$

Beginn

```
| solange true tue  
|   | skip
```

berechnet $f_1 : \mathbb{N} \rightarrow \mathbb{N}$ mit $f_1(x) = \text{undefiniert}$ für alle x .

Der Algorithmus

Eingabe: Zahlen $n_1, n_2 \in \mathbb{N}$

Beginn

```
| helf :=  $n_1 + n_2$ ;  
| return helf
```

berechnet die Funktion $f_2 : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$ mit $f_2(x, y) = x + y$.

Beispiele für berechenbare Funktionen

Der Algorithmus

Eingabe: Zahl $n \in \mathbb{N}$

Beginn

```
| solange true tue  
|   | skip
```

berechnet $f_1 : \mathbb{N} \rightarrow \mathbb{N}$ mit $f_1(x) = \text{undefiniert}$ für alle x .

Der Algorithmus

Eingabe: Zahlen $n_1, n_2 \in \mathbb{N}$

Beginn

```
| helf :=  $n_1 + n_2$ ;  
| return helf
```

berechnet die Funktion $f_2 : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$ mit $f_2(x, y) = x + y$.

f_1 und f_2 sind daher berechenbar.

Beispiele für berechenbare Funktionen

$$f_3(n) = \begin{cases} 1 & \text{falls } n \text{ ein Präfix der Ziffern der Dezimalzahldarstellung von } \pi \text{ ist} \\ 0 & \text{sonst} \end{cases}$$

Z.B. gilt

- ▶ $f_3(31) = 1$ und $f_3(314) = 1$
- ▶ $f_3(2) = 0$ und $f_3(315) = 0$

Ist f_3 berechenbar?

Beispiele für berechenbare Funktionen

$$f_3(n) = \begin{cases} 1 & \text{falls } n \text{ ein Präfix der Ziffern der Dezimalzahldarstellung von } \pi \text{ ist} \\ 0 & \text{sonst} \end{cases}$$

Z.B. gilt

- ▶ $f_3(31) = 1$ und $f_3(314) = 1$
- ▶ $f_3(2) = 0$ und $f_3(315) = 0$

Ist f_3 berechenbar?

Ja. Sei n eine k -stellige Zahl. Es gibt Algorithmen, die die ersten k Stellen von π berechnen. Danach kann man vergleichen.

Beispiele für berechenbare Funktionen

$$f_4(n) = \begin{cases} 1 & \text{falls die Dezimalzahldarstellung von } \pi \text{ das Wort } 3^n \text{ als Teilwort enthält} \\ 0 & \text{sonst} \end{cases}$$

Ist f_4 berechenbar?

Beispiele für berechenbare Funktionen

$$f_4(n) = \begin{cases} 1 & \text{falls die Dezimalzahldarstellung von } \pi \text{ das Wort } 3^n \text{ als Teilwort enthält} \\ 0 & \text{sonst} \end{cases}$$

Ist f_4 berechenbar?

Ja.

- ▶ Entweder $f_4(n) = 1$ für alle n
- ▶ oder es gibt n_0 , sodass π 3^{n_0} als Teilwort hat, aber alle Teilwörter 3^n mit $n > n_0$ nicht mehr besitzt.

$$\text{Dann ist } f_4(n) = \begin{cases} 1 & \text{falls } n \leq n_0 \\ 0 & \text{falls } n > n_0 \end{cases}$$

- ▶ Für beide Fälle können wir Algorithmen angeben, die f_4 berechnen.

Beispiele für berechenbare Funktionen

$$f_4(n) = \begin{cases} 1 & \text{falls die Dezimalzahldarstellung von } \pi \text{ das Wort } 3^n \text{ als Teilwort enthält} \\ 0 & \text{sonst} \end{cases}$$

Ist f_4 berechenbar?

Ja.

- Entweder $f_4(n) = 1$ für alle n
- oder es gibt n_0 , sodass π 3^{n_0} als Teilwort hat, aber alle Teilwörter 3^n mit $n > n_0$ nicht mehr besitzt.

$$\text{Dann ist } f_4(n) = \begin{cases} 1 & \text{falls } n \leq n_0 \\ 0 & \text{falls } n > n_0 \end{cases}$$

- Für beide Fälle können wir Algorithmen angeben, die f_4 berechnen.

Beachte: Unsere Definition von Berechenbarkeit ist **nicht konstruktiv**.

Wir müssen keinen Algorithmus liefern, sondern nur einen Beweis, dass einer existiert.

Beispiele für berechenbare Funktionen

$$f_5(n) = \begin{cases} 1 & \text{falls deterministische LBAs genau die gleichen} \\ & \text{Sprachen erkennen wie nichtdeterministische LBAs} \\ 0 & \text{sonst} \end{cases}$$

Ist f_5 berechenbar?

Beispiele für berechenbare Funktionen

$$f_5(n) = \begin{cases} 1 & \text{falls deterministische LBAs genau die gleichen} \\ & \text{Sprachen erkennen wie nichtdeterministische LBAs} \\ 0 & \text{sonst} \end{cases}$$

Ist f_5 berechenbar?

Ja. Entweder ist $f_5(x) = 1$ oder ist $f_5(x) = 0$.

Beide Funktionen sind berechenbar

(unabhängig davon, ob das 1. LBA-Problem eine positive oder negative Lösung hat).

Beispiele für berechenbare Funktionen

$$f^r(n) = \begin{cases} 1 & \text{falls } n \text{ ein Präfix der Ziffern der Dezimalzahldarstellung von } r \text{ ist} \\ 0 & \text{sonst} \end{cases}$$

Ist f^r für jedes $r \in \mathbb{R}$ berechenbar?

Beispiele für berechenbare Funktionen

$$f^r(n) = \begin{cases} 1 & \text{falls } n \text{ ein Präfix der Ziffern der Dezimalzahldarstellung von } r \text{ ist} \\ 0 & \text{sonst} \end{cases}$$

Ist f^r für jedes $r \in \mathbb{R}$ berechenbar?

Nein. Wir bräuchten genauso viele verschiedene Algorithmen wie es reelle Zahlen gibt. Es gibt nur abzählbar viele Algorithmen einer Programmiersprache, aber überabzählbar viele reelle Zahlen.

Churchsche These

Im Folgenden untersuchen wir Modelle zur Berechenbarkeit:

- ▶ Turingmaschinen (in dieser Stunde)
- ▶ WHILE-Programme (nur FSK)
- ▶ GOTO-Programme (nur FSK)
- ▶ μ -rekursive Funktionen (nur FSK)

Churchsche These

Im Folgenden untersuchen wir Modelle zur Berechenbarkeit:

- ▶ Turingmaschinen (in dieser Stunde)
- ▶ WHILE-Programme (nur FSK)
- ▶ GOTO-Programme (nur FSK)
- ▶ μ -rekursive Funktionen (nur FSK)

Alle führen zum **selben Begriff** der Berechenbarkeit.

Churchsche These

Die Klasse der turingberechenbaren (äquivalent WHILE-berechenbaren, GOTO-berechenbaren, μ -rekursiven) Funktionen stimmt genau mit der Klasse der intuitiv berechenbaren Funktionen überein.

Churchsche These

Im Folgenden untersuchen wir Modelle zur Berechenbarkeit:

- ▶ Turingmaschinen (in dieser Stunde)
- ▶ WHILE-Programme (nur FSK)
- ▶ GOTO-Programme (nur FSK)
- ▶ μ -rekursive Funktionen (nur FSK)

Alle führen zum **selben Begriff** der Berechenbarkeit.

Churchsche These

Die Klasse der turingberechenbaren (äquivalent WHILE-berechenbaren, GOTO-berechenbaren, μ -rekursiven) Funktionen stimmt genau mit der Klasse der intuitiv berechenbaren Funktionen überein.

Die Churchsche These kann man nicht beweisen, da der Begriff „intuitiv berechenbar“ nicht formal gefasst werden kann.

Wiederholung: Turingmaschinen

- ▶ Eine Turingmaschine ist ein 7-Tupel $M = (Z, \Sigma, \Gamma, \delta, z_0, \square, E)$ mit Zuständen Z , Eingabealphabet Σ , Bandalphabet $\Gamma \supset \Sigma$, Blank-Symbol $\square$, Startzustand z_0 , Endzuständen $E \subseteq Z$ und Überföhrungsfunktion δ .
- ▶ DTM: $\delta : (Z \setminus E) \times \Gamma \rightarrow Z \times \Gamma \times \{L, R, N\}$
- ▶ NTM: $\delta : (Z \setminus E) \times \Gamma \rightarrow \mathcal{P}(Z \times \Gamma \times \{L, R, N\})$
- ▶ TM-Konfiguration $a_1 \cdots a_m \mathbf{z} a_{m+1} \cdots a_n$, wobei $a_1 \cdots a_n \in \Gamma^*$ der entdeckte Teil des Bands ist, TM in Zustand $\mathbf{z}$ ist und der Schreib-Lesekopf unter a_{m+1} ist.

Definition

Eine Funktion $f : \Sigma^* \rightarrow \Sigma^*$ heißt **turingberechenbar**, falls es eine deterministische Turingmaschine $M = (Z, \Sigma, \Gamma, \delta, z_0, \square, E)$ gibt, sodass für alle $u, v \in \Sigma^*$ gilt:

$$f(u) = v$$

g.d.w.

es gibt $z \in E$, sodass $Start_M(u) \vdash^* \square \dots \square z v \square \dots \square$

Wenn $f(u)$ undefiniert ist (f ist also eine partielle Funktion), dann kann die Maschine ewig laufen.

Definition

Eine Funktion $f : \mathbb{N}^k \rightarrow \mathbb{N}$ heißt **turingberechenbar**, falls es eine deterministische Turingmaschine $M = (Z, \Sigma, \Gamma, \delta, z_0, \square, E)$ gibt, sodass für alle $n_1, \dots, n_k, m \in \mathbb{N}$ gilt:

$$f(n_1, \dots, n_k) = m \\ \text{g.d.w.}$$

es gibt $z \in E$, sodass $z_0 \text{bin}(n_1) \# \dots \# \text{bin}(n_k) \vdash^* \square \dots \square z \text{bin}(m) \square \dots \square$

wobei $\text{bin}(n)$ die Binärzahldarstellung von $n \in \mathbb{N}$ ist.

Wenn $f(n_1, \dots, n_k)$ undefiniert ist (f ist also eine partielle Funktion), dann kann die Maschine ewig laufen.

Eine Konsequenz der Definition ist:

Falls $f(u)$ undefiniert ist, dann kann die Maschine ewig laufen.

Beispiele für turingberechenbare Funktionen

Nachfolgerfunktion

Die Funktion $f(x) = x + 1$ für alle $x \in \mathbb{N}$ ist turingberechenbar.
Wir haben ein Beispiel bereits gesehen.

Beispiele für turingberechenbare Funktionen

Nachfolgerfunktion

Die Funktion $f(x) = x + 1$ für alle $x \in \mathbb{N}$ ist turingberechenbar.
Wir haben ein Beispiel bereits gesehen.

Identitätsfunktion

Die Funktion $f(x) = x$ für alle $x \in \mathbb{N}$ ist turingberechenbar:
Für die Turingmaschine $M = (\{z_0\}, \{0, 1, \#\}, \{0, 1, \#, \square\}, \delta, z_0, \square, \{z_0\})$ gilt:
 $z_0 \text{bin}(n) \vdash^* z_0 \text{bin}(n)$ für alle $n \in \mathbb{N}$.

Beispiele für turingberechenbare Funktionen

Nachfolgerfunktion

Die Funktion $f(x) = x + 1$ für alle $x \in \mathbb{N}$ ist turingberechenbar.
Wir haben ein Beispiel bereits gesehen.

Identitätsfunktion

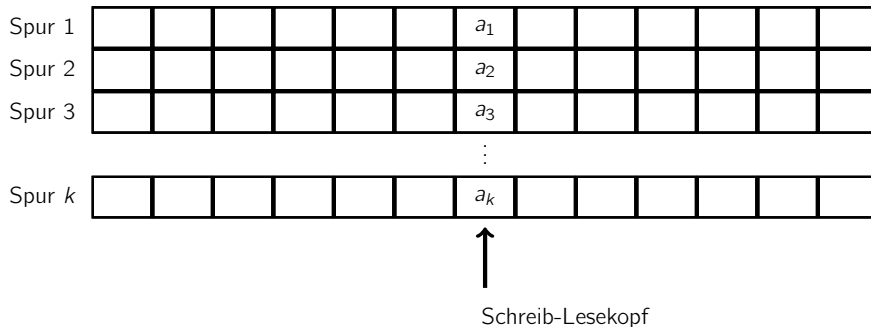
Die Funktion $f(x) = x$ für alle $x \in \mathbb{N}$ ist turingberechenbar:
Für die Turingmaschine $M = (\{z_0\}, \{0, 1, \#\}, \{0, 1, \#, \square\}, \delta, z_0, \square, \{z_0\})$ gilt:
 $z_0 \text{bin}(n) \vdash^* z_0 \text{bin}(n)$ für alle $n \in \mathbb{N}$.

Überall undefinierte Funktion

Die Funktion $f(x) = \text{undefiniert}$ für alle x ist turingberechenbar, da die Turingmaschine $M = (\{z_0\}, \{0, 1, \#\}, \{0, 1, \#, \square\}, \delta, z_0, \square, \emptyset)$ mit $\delta(z_0, a) = (z_0, a, N)$ für keine Eingabe akzeptiert (sondern stets in eine Endlosschleife geht).

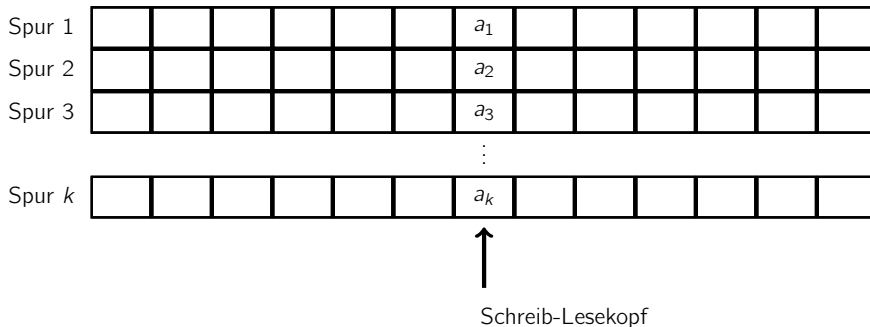
Mehrspuren-Turingmaschinen

Erweiterung: Das Band hat k Spuren:



Mehrspuren-Turingmaschinen

Erweiterung: Das Band hat k Spuren:



Mehrspuren-Turingmaschinen machen manche Konstruktionen einfacher.

Definition

Eine k -Spuren-Turingmaschine (für $k \in \mathbb{N}_{>0}$) ist ein 7-Tupel $M = (Z, \Sigma, \Gamma, \delta, z_0, \square, E)$, wobei:

- ▶ Z ist eine endliche Menge von Zuständen
- ▶ Σ ist das (endliche) Eingabealphabet
- ▶ $\Gamma \supset \Sigma$ ist das (endliche) Bandalphabet
- ▶ δ ist die Überföhrungsfunktion
 - ▶ für DTM: $\delta : (Z \setminus E) \times \Gamma^k \rightarrow Z \times \Gamma^k \times \{L, R, N\}$
 - ▶ für NTM: $\delta : (Z \setminus E) \times \Gamma^k \rightarrow \mathcal{P}(Z \times \Gamma^k \times \{L, R, N\})$
- ▶ $z_0 \in Z$ ist der Startzustand
- ▶ $\square \in \Gamma \setminus \Sigma$ ist das Blank-Symbol
- ▶ $E \subseteq Z$ ist die Menge der Endzustände.

Mehrspuren-Turingmaschinen

Für Berechenbarkeit erfolgt die Ein- und Ausgabe auf der ersten Spur.
Anfangs stehen alle anderen leer.

Satz

Jede Mehrspuren-Turingmaschine kann von einer 1-Spur-Turingmaschine simuliert werden.

Mehrspuren-Turingmaschinen

Für Berechenbarkeit erfolgt die Ein- und Ausgabe auf der ersten Spur.
Anfangs stehen alle anderen leer.

Satz

Jede Mehrspuren-Turingmaschine kann von einer 1-Spur-Turingmaschine simuliert werden.

Beweis Konstruktion einer 1-Spur-TM mit $\Gamma \cup \Gamma^k$ als Bandalphabet, Σ als Eingabealphabet und $\square$ als Blank-Symbol:

Mehrspuren-Turingmaschinen

Für Berechenbarkeit erfolgt die Ein- und Ausgabe auf der ersten Spur.
Anfangs stehen alle anderen leer.

Satz

Jede Mehrspuren-Turingmaschine kann von einer 1-Spur-Turingmaschine simuliert werden.

Beweis Konstruktion einer 1-Spur-TM mit $\Gamma \cup \Gamma^k$ als Bandalphabet, Σ als Eingabealphabet und $\square$ als Blank-Symbol:

1. Aus Eingabe $w \in \Sigma^*$ erzeuge die Mehrspurendarstellung:
Ersetze $a \in \Sigma$ durch das k -Tupel $(a, \square, \dots, \square)$.

Mehrspuren-Turingmaschinen

Für Berechenbarkeit erfolgt die Ein- und Ausgabe auf der ersten Spur.
Anfangs stehen alle anderen leer.

Satz

Jede Mehrspuren-Turingmaschine kann von einer 1-Spur-Turingmaschine simuliert werden.

Beweis Konstruktion einer 1-Spur-TM mit $\Gamma \cup \Gamma^k$ als Bandalphabet, Σ als Eingabealphabet und $\square$ als Blank-Symbol:

1. Aus Eingabe $w \in \Sigma^*$ erzeuge die Mehrspurendarstellung:
Ersetze $a \in \Sigma$ durch das k -Tupel $(a, \square, \dots, \square)$.
2. Simuliere anschließend die Mehrspurenmaschine.

Mehrspuren-Turingmaschinen

Für Berechenbarkeit erfolgt die Ein- und Ausgabe auf der ersten Spur.
Anfangs stehen alle anderen leer.

Satz

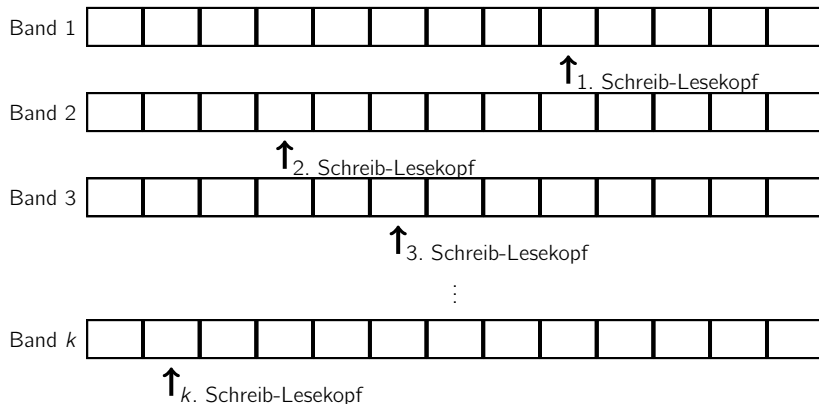
Jede Mehrspuren-Turingmaschine kann von einer 1-Spur-Turingmaschine simuliert werden.

Beweis Konstruktion einer 1-Spur-TM mit $\Gamma \cup \Gamma^k$ als Bandalphabet, Σ als Eingabealphabet und $\square$ als Blank-Symbol:

1. Aus Eingabe $w \in \Sigma^*$ erzeuge die Mehrspurendarstellung:
Ersetze $a \in \Sigma$ durch das k -Tupel $(a, \square, \dots, \square)$.
2. Simuliere anschließend die Mehrspurenmaschine.
3. Nach Akzeptanz der Mehrspurenmaschine:
Erzeuge 1-Spur-Darstellung, d.h. ersetze alle k -Tupel $(a_1, \dots, a_k)$ durch a_1 . $\square$

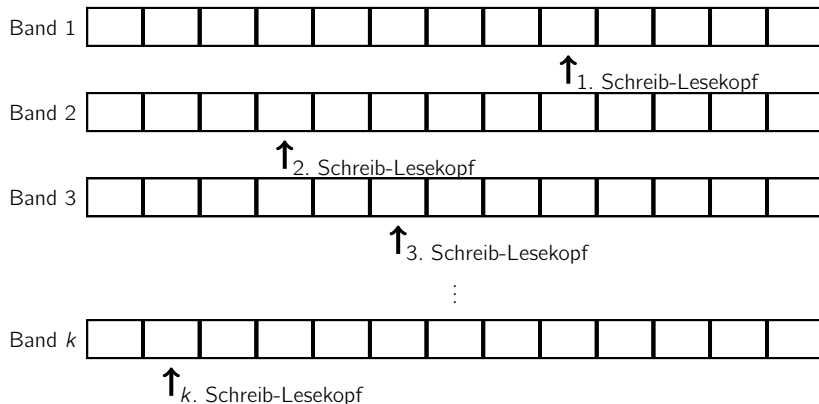
Mehrband-Turingmaschinen

Erweiterung: Die Schreib-Leseköpfe bewegen sich unabhängig.



Mehrband-Turingmaschinen

Erweiterung: Die Schreib-Leseköpfe bewegen sich unabhängig.



Mehrband-Turingmaschinen machen manche Konstruktionen noch einfacher.

Mehrband-Turingmaschinen

Definition

Eine k -Band-Turingmaschine (für $k \in \mathbb{N}_{>0}$) ist ein 7-Tupel $(Z, \Sigma, \Gamma, \delta, z_0, \square, E)$ mit

- ▶ Z ist eine endliche Menge von Zuständen
- ▶ Σ ist das (endliche) Eingabealphabet
- ▶ $\Gamma \supset \Sigma$ ist das (endliche) Bandalphabet
- ▶ δ ist die Überföhrungsfunktion
 - ▶ für DTM: $\delta : (Z \setminus E) \times \Gamma^k \rightarrow Z \times \Gamma^k \times \{L, R, N\}^k$
 - ▶ für NTM: $\delta : (Z \setminus E) \times \Gamma^k \rightarrow \mathcal{P}(Z \times \Gamma^k \times \{L, R, N\}^k)$
- ▶ $z_0 \in Z$ ist der Startzustand
- ▶ $\square \in \Gamma \setminus \Sigma$ ist das Blank-Symbol
- ▶ $E \subseteq Z$ ist die Menge der Endzustände.

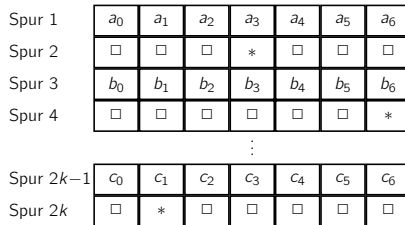
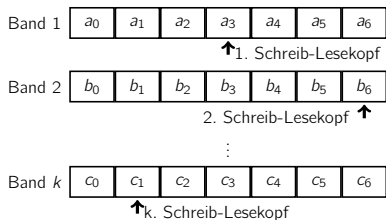
Mehrband-Turingmaschinen

Für Berechenbarkeit erfolgt die Ein- und Ausgabe auf der ersten Spur.
Anfangs stehen alle anderen leer.

Theorem

Jede Mehrband-Turingmaschine kann von einer 1-Band-Turingmaschine simuliert werden.

Beweis Sei M eine k -Band-TM. Wir simulieren M mit einer $2k$ -Spuren-TM M' , die sich wiederum mit einer 1-Spuren-TM simulieren lässt.



Mehrband-Turingmaschinen

Die Eingabe $w \in \Sigma^*$ liegt auf dem Eingabeband von M' .

Mehrband-Turingmaschinen

Die Eingabe $w \in \Sigma^*$ liegt auf dem Eingabeband von M' .

1. Erzeuge Darstellung in $2k$ Spuren.

Mehrband-Turingmaschinen

Die Eingabe $w \in \Sigma^*$ liegt auf dem Eingabeband von M' .

1. Erzeuge Darstellung in $2k$ Spuren.
2. Simuliere anschließend Berechnungsschritte von M . Für jeden Schritt:
 - 2.1 Lies den verwendeten Bandbereich von links nach rechts, um die Kopfpositionen zu finden.
 - 2.2 Speichere dabei alle k Bandinhalte an den Kopfpositionen durch Zustände.
 - 2.3 Führe den Schritt von M aus, durch Anpassen der Bandinhalte und der Kopfpositionen.

Mehrband-Turingmaschinen

Die Eingabe $w \in \Sigma^*$ liegt auf dem Eingabeband von M' .

1. Erzeuge Darstellung in $2k$ Spuren.
2. Simuliere anschließend Berechnungsschritte von M . Für jeden Schritt:
 - 2.1 Lies den verwendeten Bandbereich von links nach rechts, um die Kopfpositionen zu finden.
 - 2.2 Speichere dabei alle k Bandinhalte an den Kopfpositionen durch Zustände.
 - 2.3 Führe den Schritt von M aus, durch Anpassen der Bandinhalte und der Kopfpositionen.
3. Bei Akzeptanz durch M transformiere die Spurendarstellung in die Darstellung der Ausgabe. □

9a

Konstruktionen von Turingmaschinen und LOOP-Programme

Prof. Dr. Jasmin Blanchette

Lehr- und Forschungseinheit für
Theoretische Informatik und TheorembeweisenStand: 18. Juni 2024
Basierend auf Folien von PD Dr. David Sabel

Definition

Eine Funktion $f : \Sigma^* \rightarrow \Sigma^*$ heißt **turingberechenbar**, falls es eine deterministische Turingmaschine $M = (Z, \Sigma, \Gamma, \delta, z_0, \square, E)$ gibt, sodass für alle $u, v \in \Sigma^*$ gilt:

$$f(u) = v$$

g.d.w.

es gibt $z \in E$, sodass $Start_M(u) \vdash^* \square \dots \square z v \square \dots \square$

Wiederholung: Turingberechenbarkeit

Definition

Eine Funktion $f : \mathbb{N}^k \rightarrow \mathbb{N}$ heißt **turingberechenbar**, falls es eine deterministische Turingmaschine $M = (Z, \Sigma, \Gamma, \delta, z_0, \square, E)$ gibt, sodass für alle $n_1, \dots, n_k, m \in \mathbb{N}$ gilt:

$$f(n_1, \dots, n_k) = m$$

g.d.w.

es gibt $z \in E$, sodass $z_0 \text{bin}(n_1) \# \dots \# \text{bin}(n_k) \vdash^* \square \dots \square z \text{bin}(m) \square \dots \square$

wobei $\text{bin}(n)$ die Binärzahldarstellung von $n \in \mathbb{N}$ ist.

Wiederholung: Mehrband-Turingmaschinen

Definition

Eine k -Band-Turingmaschine (für $k \in \mathbb{N}_{>0}$) ist ein 7-Tupel $(Z, \Sigma, \Gamma, \delta, z_0, \square, E)$ mit

- ▶ Z ist eine endliche Menge von Zuständen
- ▶ Σ ist das (endliche) Eingabealphabet
- ▶ $\Gamma \supset \Sigma$ ist das (endliche) Bandalphabet
- ▶ δ ist die Überföhrungsfunktion
 - ▶ für DTM: $\delta : Z \times \Gamma^k \rightarrow Z \times \Gamma^k \times \{L, R, N\}^k$
 - ▶ für NTM: $\delta : Z \times \Gamma^k \rightarrow \mathcal{P}(Z \times \Gamma^k \times \{L, R, N\}^k)$
- ▶ $z_0 \in Z$ ist der Startzustand
- ▶ $\square \in \Gamma \setminus \Sigma$ ist das Blank-Symbol
- ▶ $E \subseteq Z$ ist die Menge der Endzustände.

Konstruktion von Turingmaschinen und Notationen

DTMs können Programme einer einfachen imperativen Programmiersprache mit Zuweisungen, Verzweigungen und Schleifen simulieren.

Notationen:

- ▶ Wenn M eine 1-Band-Turingmaschine ist, dann schreiben wir $M(i, k)$ für die k -Band-Turingmaschine (mit $i \leq k$), die die Operationen von M auf dem i -ten Band durchführt und alle anderen Bänder unverändert lässt.
- ▶ Wenn k nicht von Bedeutung, schreiben wir $M(i)$ statt $M(i, k)$.

Konstruktion von Turingmaschinen

Beispiel:

- ▶ Die TM, die 1 addiert nennen wir „ $\text{Band} := \text{Band} + 1$ “.
- ▶ Die k -Band-TM, die 1 auf dem i -ten Band addiert nennen wir „ $\text{Band} := \text{Band} + 1(i, k)$ “, „ $\text{Band} := \text{Band} + 1(i)$ “ oder sogar „ $\text{Band } i := \text{Band } i + 1$ “.

Konstruktion von Turingmaschinen

Beispiel:

- ▶ Die TM, die 1 addiert nennen wir „ $\text{Band} := \text{Band} + 1$ “.
- ▶ Die k -Band-TM, die 1 auf dem i -ten Band addiert nennen wir „ $\text{Band} := \text{Band} + 1(i, k)$ “, „ $\text{Band} := \text{Band} + 1(i)$ “ oder sogar „ $\text{Band } i := \text{Band } i + 1$ “.

Weitere Turingmaschinen folgen. Die Konstruktionen sind relativ einfach.

- ▶ „ $\text{Band } i := (\text{Band } i) - 1$ “:
 k -Band-TM ($k \geq i$), die eine angepasste Subtraktion von 1 auf Band i durchführt. Beispiel für die Anpassung: $0 - 1 = 0$.
- ▶ „ $\text{Band } i := 0$ “:
 k -Band-TM ($k \geq i$), die Band i mit 0 überschreibt.
- ▶ „ $\text{Band } i := \text{Band } j$ “:
 k -Band-TM ($k \geq i$ und $k \geq j$), welche die Zahl von Band j auf Band i kopiert.

Hintereinanderschaltung von Turingmaschinen

Seien $M_i = (Z_i, \Sigma, \Gamma_i, \delta_i, z_{0i}, \square, E_i)$ für $i \in \{1, 2\}$ k -Band-TMs.

O.B.d.A. $Z_1 \cap Z_2 = \emptyset$.

Hintereinanderschaltung von Turingmaschinen

Seien $M_i = (Z_i, \Sigma, \Gamma_i, \delta_i, z_{0i}, \square, E_i)$ für $i \in \{1, 2\}$ k -Band-TMs.

O.B.d.A. $Z_1 \cap Z_2 = \emptyset$.

Die TM $M_1; M_2$ führt M_1 und M_2 hintereinandergeschaltet aus:

$M_1; M_2 = (Z_1 \cup Z_2, \Sigma, \Gamma_1 \cup \Gamma_2, \delta, z_{01}, \square, E_2)$ mit

$$\delta(z, (a_1, \dots, a_k)) = \begin{cases} \delta_1(z, (a_1, \dots, a_k)) & \text{falls } z \in Z_1 \setminus E_1 \\ (z_{02}, (a_1, \dots, a_k), N^k) & \text{falls } z \in E_1 \\ \delta_2(z, (a_1, \dots, a_k)) & \text{falls } z \in Z_2 \setminus E_2 \end{cases}$$

Hintereinanderschaltung von Turingmaschinen

Seien $M_i = (Z_i, \Sigma, \Gamma_i, \delta_i, z_{0i}, \square, E_i)$ für $i \in \{1, 2\}$ k -Band-TMs.

O.B.d.A. $Z_1 \cap Z_2 = \emptyset$.

Die TM $M_1; M_2$ führt M_1 und M_2 hintereinandergeschaltet aus:

$M_1; M_2 = (Z_1 \cup Z_2, \Sigma, \Gamma_1 \cup \Gamma_2, \delta, z_{01}, \square, E_2)$ mit

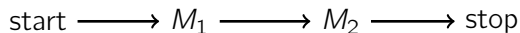
$$\delta(z, (a_1, \dots, a_k)) = \begin{cases} \delta_1(z, (a_1, \dots, a_k)) & \text{falls } z \in Z_1 \setminus E_1 \\ (z_{02}, (a_1, \dots, a_k), N^k) & \text{falls } z \in E_1 \\ \delta_2(z, (a_1, \dots, a_k)) & \text{falls } z \in Z_2 \setminus E_2 \end{cases}$$

Die TM $M_1; M_2$

- ▶ führt erst M_1 aus
- ▶ wechselt im Endzustand $z \in E_1$ in Startzustand z_{02} von M_2
- ▶ führt anschließend M_2 aus.

Hintereinanderschaltung von Turingmaschinen

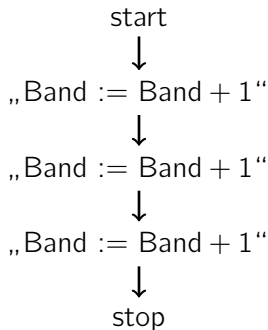
Flussdiagramm für $M_1; M_2$:



Hintereinanderschaltung von Turingmaschinen

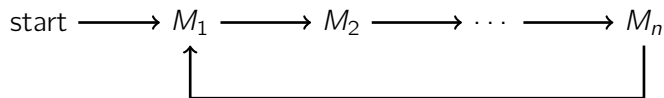
Beispiel: „ $\text{Band} := \text{Band} + 3$ “ wird konstruiert durch
„ $\text{Band} := \text{Band} + 1$ “; „ $\text{Band} := \text{Band} + 1$ “; „ $\text{Band} := \text{Band} + 1$ “

Flussdiagramm dazu:



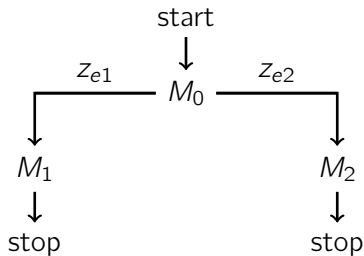
Hintereinanderschaltung von Turingmaschinen

Zyklische Verkettung von $M_1, \dots, M_n$:



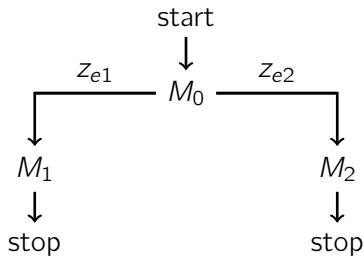
Verzweigende Fortsetzung

Seien M_0, M_1, M_2 TMs und seien z_{e1} und z_{e2} die Endzustände von M_0 .
Verzweigende Fortsetzung von M_0 :



Verzweigende Fortsetzung

Seien M_0, M_1, M_2 TMs und seien z_{e1} und z_{e2} die Endzustände von M_0 .
Verzweigende Fortsetzung von M_0 :



Die Konstruktion fügt Übergänge

$$\delta(z_{e1}, (a_1, \dots, a_k)) = (z_{01}, (a_1, \dots, a_k), N^k)$$

$$\delta(z_{e2}, (a_1, \dots, a_k)) = (z_{02}, (a_1, \dots, a_k), N^k)$$

ein, wobei z_{0i} der Startzustand von M_i ist (für $i \in \{1, 2\}$).

Beispiel für Test auf 0

Folgende TM M_0 prüft, ob das Band eine 0 enthält oder nicht.

M_0 hat die Zustände $\{z_0, z_1, ja, nein\}$ und

$$\delta(z_0, a) = (nein, a, N) \quad \text{für } a \neq 0$$

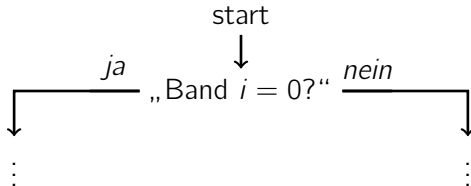
$$\delta(z_0, 0) = (z_1, 0, R)$$

$$\delta(z_1, a) = (nein, a, L) \quad \text{für } a \neq \square$$

$$\delta(z_1, \square) = (ja, \square, L)$$

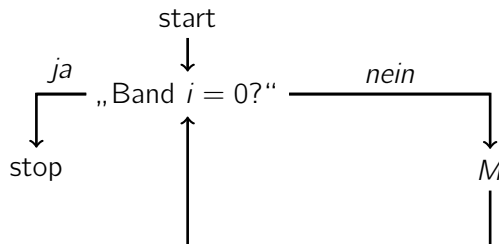
mit z_0 Startzustand und ja und $nein$ Endzustände.

Notationen: „Band = 0?“ und „Band $i = 0$?“.



Schleife

Mit Verzweigung, „Band $i = 0$?“, zyklischer Hintereinanderschaltung und einer TM M erstellen wir die Schleife



Die TM M wird solange wieder aufgerufen, bis das i -te Band die Zahl 0 enthält.

Die Maschine nennen wir „**WHILE** Band $i \neq 0$ **DO** M “.

LOOP-, WHILE-, GOTO-Programme

Ziel:

- ▶ Betrachte drei einfache imperative Programmiersprachen
 - ▶ LOOP-Programme
 - ▶ WHILE-Programme
 - ▶ GOTO-Programme
- und die dazugehörigen Berechenbarkeitsbegriffe.
- ▶ Welche Berechenbarkeitsbegriffe sind gleich bzw. verschieden (untereinander aber auch bezüglich Turingberechenbarkeit)?

Syntax von LOOP-Programmen

LOOP-Programme werden durch die kontextfreie Grammatik (V, Σ, P, Prg) erzeugt, wobei:

$$\begin{aligned} V &= \{Prg, Var, Id, Const\} \\ \Sigma &= \{\mathbf{LOOP}, \mathbf{DO}, \mathbf{END}, x, 0, \dots, 9, ,, :=, +, -\} \\ P &= \{Prg \rightarrow \mathbf{LOOP} \text{ } Var \mathbf{DO} \text{ } Prg \mathbf{END} \\ &\quad | Prg; Prg \\ &\quad | Var := Var + Const \\ &\quad | Var := Var - Const, \\ Var &\rightarrow x_{Id}, \\ Const &\rightarrow Id, \\ Id &\rightarrow 0 \mid 1 \mid \dots \mid 9 \mid 1Id \mid 2Id \mid \dots \mid 9Id\} \end{aligned}$$

Beachte:

- ▶ *Var* erzeugt Variablen $x_0, x_1, x_2, \dots$
- ▶ *Const* erzeugt alle natürlichen Zahlen.

Definition (Variablenbelegung)

Eine **Variablenbelegung** ρ ist eine endliche Abbildung mit Einträgen $x_i \mapsto n$ mit x_i ist Variable und $n \in \mathbb{N}$.

Wir definieren

$$\rho(x_i) := \begin{cases} n & \text{wenn } x_i \mapsto n \in \rho \\ 0 & \text{sonst} \end{cases}$$

Definition (Variablenbelegung)

Eine **Variablenbelegung** ρ ist eine endliche Abbildung mit Einträgen $x_i \mapsto n$ mit x_i ist Variable und $n \in \mathbb{N}$.

Wir definieren

$$\rho(x_i) := \begin{cases} n & \text{wenn } x_i \mapsto n \in \rho \\ 0 & \text{sonst} \end{cases}$$

Wir definieren auch

$$\rho\{x_i \mapsto m\}(x_j) := \begin{cases} m & \text{wenn } x_j = x_i \\ \rho(x_j) & \text{wenn } x_j \neq x_i \end{cases}$$

Definition (Variablenbelegung)

Eine **Variablenbelegung** ρ ist eine endliche Abbildung mit Einträgen $x_i \mapsto n$ mit x_i ist Variable und $n \in \mathbb{N}$.

Wir definieren

$$\rho(x_i) := \begin{cases} n & \text{wenn } x_i \mapsto n \in \rho \\ 0 & \text{sonst} \end{cases}$$

Wir definieren auch

$$\rho\{x_i \mapsto m\}(x_j) := \begin{cases} m & \text{wenn } x_j = x_i \\ \rho(x_j) & \text{wenn } x_j \neq x_i \end{cases}$$

Die Notation $\rho\{x_i \mapsto m\}$ steht also für die Variablenbelegung, die überall mit ρ übereinstimmt außer bei x_i , wofür m ausgegeben wird.

Definition

Die Berechnungsschritte $(\rho, P) \xrightarrow{\text{LOOP}} (\rho', P')$, wobei ρ, ρ' Variablenbelegungen und P, P' LOOP-Programme oder das leere Programm ε sind, sind durch folgende Regeln definiert:

- ▶ $(\rho, x_i := x_j + c) \xrightarrow{\text{LOOP}} (\rho', \varepsilon)$, wobei $\rho' = \rho\{x_i \mapsto \rho(x_j) + c\}$
- ▶ $(\rho, x_i := x_j - c) \xrightarrow{\text{LOOP}} (\rho', \varepsilon)$, wobei $\rho' = \rho\{x_i \mapsto \max(0, \rho(x_j) - c)\}$
- ▶ $(\rho, P_1; P_2) \xrightarrow{\text{LOOP}} (\rho', P_2)$ wenn $(\rho, P_1) \xrightarrow{\text{LOOP}} (\rho', \varepsilon)$
- ▶ $(\rho, P_1; P_2) \xrightarrow{\text{LOOP}} (\rho', P'_1; P_2)$ wenn $(\rho, P_1) \xrightarrow{\text{LOOP}} (\rho', P'_1)$ und $P'_1 \neq \varepsilon$
- ▶ $(\rho, \text{LOOP } x_i \text{ DO } P \text{ END}) \xrightarrow{\text{LOOP}} (\rho, \underbrace{P; \dots; P}_{\rho(x_i)\text{-mal}})$

Definition

Die Berechnungsschritte $(\rho, P) \xrightarrow{\text{LOOP}} (\rho', P')$, wobei ρ, ρ' Variablenbelegungen und P, P' LOOP-Programme oder das leere Programm ε sind, sind durch folgende Regeln definiert:

- ▶ $(\rho, x_i := x_j + c) \xrightarrow{\text{LOOP}} (\rho', \varepsilon)$, wobei $\rho' = \rho\{x_i \mapsto \rho(x_j) + c\}$
- ▶ $(\rho, x_i := x_j - c) \xrightarrow{\text{LOOP}} (\rho', \varepsilon)$, wobei $\rho' = \rho\{x_i \mapsto \max(0, \rho(x_j) - c)\}$
- ▶ $(\rho, P_1; P_2) \xrightarrow{\text{LOOP}} (\rho', P_2)$ wenn $(\rho, P_1) \xrightarrow{\text{LOOP}} (\rho', \varepsilon)$
- ▶ $(\rho, P_1; P_2) \xrightarrow{\text{LOOP}} (\rho', P'_1; P_2)$ wenn $(\rho, P_1) \xrightarrow{\text{LOOP}} (\rho', P'_1)$ und $P'_1 \neq \varepsilon$
- ▶ $(\rho, \text{LOOP } x_i \text{ DO } P \text{ END}) \xrightarrow{\text{LOOP}} (\rho, \underbrace{P; \dots; P}_{\rho(x_i)\text{-mal}})$

Wir schreiben $\xrightarrow{\text{LOOP}}^i$ für i Schritte und $\xrightarrow{\text{LOOP}}^*$ für 0 oder beliebig viele Schritte.

Beispiel für die Semantik von LOOP-Programmen

Programm: $x_2 := x_1 + 1;$
LOOP x_2 **DO** $x_3 := x_3 + 1$ **END**

Variablenbelegung: $\{x_1 \mapsto 2\}$

Beispiel für die Semantik von LOOP-Programmen

Programm: $x_2 := x_1 + 1;$

Variablenbelegung: $\{x_1 \mapsto 2\}$

LOOP x_2 **DO** $x_3 := x_3 + 1$ **END**

Ausführung:

$(\{x_1 \mapsto 2\}, x_2 := x_1 + 1; \text{LOOP } x_2 \text{ DO } x_3 := x_3 + 1 \text{ END})$

Beispiel für die Semantik von LOOP-Programmen

Programm: $x_2 := x_1 + 1;$

Variablenbelegung: $\{x_1 \mapsto 2\}$

LOOP x_2 **DO** $x_3 := x_3 + 1$ **END**

Ausführung:

$(\{x_1 \mapsto 2\}, x_2 := x_1 + 1; \text{LOOP } x_2 \text{ DO } x_3 := x_3 + 1 \text{ END})$
 $\xrightarrow{\text{LOOP}} (\{x_1 \mapsto 2, x_2 \mapsto 3\}, \text{LOOP } x_2 \text{ DO } x_3 := x_3 + 1 \text{ END})$
da $(\{x_1 \mapsto 2\}, x_2 := x_1 + 1) \xrightarrow{\text{LOOP}} (\{x_1 \mapsto 2, x_2 \mapsto 3\}, \epsilon)$

Beispiel für die Semantik von LOOP-Programmen

Programm: $x_2 := x_1 + 1;$

Variablenbelegung: $\{x_1 \mapsto 2\}$

LOOP x_2 **DO** $x_3 := x_3 + 1$ **END**

Ausführung:

$(\{x_1 \mapsto 2\}, x_2 := x_1 + 1; \text{LOOP } x_2 \text{ DO } x_3 := x_3 + 1 \text{ END})$
 $\xrightarrow{\text{LOOP}} (\{x_1 \mapsto 2, x_2 \mapsto 3\}, \text{LOOP } x_2 \text{ DO } x_3 := x_3 + 1 \text{ END})$
da $(\{x_1 \mapsto 2\}, x_2 := x_1 + 1) \xrightarrow{\text{LOOP}} (\{x_1 \mapsto 2, x_2 \mapsto 3\}, \epsilon)$
 $\xrightarrow{\text{LOOP}} (\{x_1 \mapsto 2, x_2 \mapsto 3\}, x_3 := x_3 + 1; x_3 := x_3 + 1; x_3 := x_3 + 1)$

Beispiel für die Semantik von LOOP-Programmen

Programm: $x_2 := x_1 + 1;$

Variablenbelegung: $\{x_1 \mapsto 2\}$

LOOP x_2 **DO** $x_3 := x_3 + 1$ **END**

Ausführung:

$(\{x_1 \mapsto 2\}, x_2 := x_1 + 1; \text{LOOP } x_2 \text{ DO } x_3 := x_3 + 1 \text{ END})$
 $\xrightarrow{\text{LOOP}} (\{x_1 \mapsto 2, x_2 \mapsto 3\}, \text{LOOP } x_2 \text{ DO } x_3 := x_3 + 1 \text{ END})$
da $(\{x_1 \mapsto 2\}, x_2 := x_1 + 1) \xrightarrow{\text{LOOP}} (\{x_1 \mapsto 2, x_2 \mapsto 3\}, \epsilon)$
 $\xrightarrow{\text{LOOP}} (\{x_1 \mapsto 2, x_2 \mapsto 3\}, x_3 := x_3 + 1; x_3 := x_3 + 1; x_3 := x_3 + 1)$
 $\xrightarrow{\text{LOOP}} (\{x_1 \mapsto 2, x_2 \mapsto 3, x_3 \mapsto 1\}, x_3 := x_3 + 1; x_3 := x_3 + 1)$
da $(\{x_1 \mapsto 2, x_2 \mapsto 3\}, x_3 := x_3 + 1) \xrightarrow{\text{LOOP}} (\{x_1 \mapsto 2, x_2 \mapsto 3, x_3 \mapsto 1\}, \epsilon)$

Beispiel für die Semantik von LOOP-Programmen

Programm: $x_2 := x_1 + 1;$

Variablenbelegung: $\{x_1 \mapsto 2\}$

LOOP x_2 **DO** $x_3 := x_3 + 1$ **END**

Ausführung:

$$\begin{aligned} & (\{x_1 \mapsto 2\}, x_2 := x_1 + 1; \text{LOOP } x_2 \text{ DO } x_3 := x_3 + 1 \text{ END}) \\ \xrightarrow{\text{LOOP}} & (\{x_1 \mapsto 2, x_2 \mapsto 3\}, \text{LOOP } x_2 \text{ DO } x_3 := x_3 + 1 \text{ END}) \\ & \text{da } (\{x_1 \mapsto 2\}, x_2 := x_1 + 1) \xrightarrow{\text{LOOP}} (\{x_1 \mapsto 2, x_2 \mapsto 3\}, \varepsilon) \\ \xrightarrow{\text{LOOP}} & (\{x_1 \mapsto 2, x_2 \mapsto 3\}, x_3 := x_3 + 1; x_3 := x_3 + 1; x_3 := x_3 + 1) \\ \xrightarrow{\text{LOOP}} & (\{x_1 \mapsto 2, x_2 \mapsto 3, x_3 \mapsto 1\}, x_3 := x_3 + 1; x_3 := x_3 + 1) \\ & \text{da } (\{x_1 \mapsto 2, x_2 \mapsto 3\}, x_3 := x_3 + 1) \xrightarrow{\text{LOOP}} (\{x_1 \mapsto 2, x_2 \mapsto 3, x_3 \mapsto 1\}, \varepsilon) \\ \xrightarrow{\text{LOOP}} & (\{x_1 \mapsto 2, x_2 \mapsto 3, x_3 \mapsto 2\}, x_3 := x_3 + 1) \\ & \text{da } (\{x_1 \mapsto 2, x_2 \mapsto 3, x_3 \mapsto 1\}, x_3 := x_3 + 1) \xrightarrow{\text{LOOP}} (\{x_1 \mapsto 2, x_2 \mapsto 3, x_3 \mapsto 2\}, \varepsilon) \end{aligned}$$

Beispiel für die Semantik von LOOP-Programmen

Programm: $x_2 := x_1 + 1;$

Variablenbelegung: $\{x_1 \mapsto 2\}$

LOOP x_2 **DO** $x_3 := x_3 + 1$ **END**

Ausführung:

$(\{x_1 \mapsto 2\}, x_2 := x_1 + 1; \text{LOOP } x_2 \text{ DO } x_3 := x_3 + 1 \text{ END})$
 $\xrightarrow{\text{LOOP}} (\{x_1 \mapsto 2, x_2 \mapsto 3\}, \text{LOOP } x_2 \text{ DO } x_3 := x_3 + 1 \text{ END})$
da $(\{x_1 \mapsto 2\}, x_2 := x_1 + 1) \xrightarrow{\text{LOOP}} (\{x_1 \mapsto 2, x_2 \mapsto 3\}, \epsilon)$
 $\xrightarrow{\text{LOOP}} (\{x_1 \mapsto 2, x_2 \mapsto 3\}, x_3 := x_3 + 1; x_3 := x_3 + 1; x_3 := x_3 + 1)$
 $\xrightarrow{\text{LOOP}} (\{x_1 \mapsto 2, x_2 \mapsto 3, x_3 \mapsto 1\}, x_3 := x_3 + 1; x_3 := x_3 + 1)$
da $(\{x_1 \mapsto 2, x_2 \mapsto 3\}, x_3 := x_3 + 1) \xrightarrow{\text{LOOP}} (\{x_1 \mapsto 2, x_2 \mapsto 3, x_3 \mapsto 1\}, \epsilon)$
 $\xrightarrow{\text{LOOP}} (\{x_1 \mapsto 2, x_2 \mapsto 3, x_3 \mapsto 2\}, x_3 := x_3 + 1)$
da $(\{x_1 \mapsto 2, x_2 \mapsto 3, x_3 \mapsto 1\}, x_3 := x_3 + 1) \xrightarrow{\text{LOOP}} (\{x_1 \mapsto 2, x_2 \mapsto 3, x_3 \mapsto 2\}, \epsilon)$
 $\xrightarrow{\text{LOOP}} (\{x_1 \mapsto 2, x_2 \mapsto 3, x_3 \mapsto 3\}, \epsilon)$

Definition (LOOP-berechenbare Funktion)

Eine Funktion $f : \mathbb{N}^k \rightarrow \mathbb{N}$ heißt **LOOP-berechenbar**, wenn es ein LOOP-Programm P gibt, sodass für alle $n_1, \dots, n_k \in \mathbb{N}$ gilt $(\rho, P) \xrightarrow[\text{LOOP}]{*} (\rho', \varepsilon)$, wobei $\rho = \{x_1 \mapsto n_1, \dots, x_k \mapsto n_k\}$ und $\rho'(x_0) = f(n_1, \dots, n_k)$.

Definition (LOOP-berechenbare Funktion)

Eine Funktion $f : \mathbb{N}^k \rightarrow \mathbb{N}$ heißt **LOOP-berechenbar**, wenn es ein LOOP-Programm P gibt, sodass für alle $n_1, \dots, n_k \in \mathbb{N}$ gilt $(\rho, P) \xrightarrow[\text{LOOP}]{*} (\rho', \varepsilon)$, wobei $\rho = \{x_1 \mapsto n_1, \dots, x_k \mapsto n_k\}$ und $\rho'(x_0) = f(n_1, \dots, n_k)$.

D.h. das LOOP-Programm

- ▶ empfängt die Eingaben über die Variablen $x_1, \dots, x_k$
- ▶ liefert sein Ergebnis in Variable x_0 .

Beispiel für die LOOP-Berechenbarkeit

Die Funktion $f(n_1) = n_1 + c$ ist LOOP-berechenbar.

Das Programm $x_0 := x_1 + c$ belegt dies, denn für alle $n_1 \in \mathbb{N}$:

$$(\{x_1 \mapsto n_1\}, x_0 := x_1 + c) \xrightarrow{\text{LOOP}} (\{x_0 \mapsto n_1 + c, x_1 \mapsto n_1\}, \varepsilon)$$

Terminierung von LOOP-Programmen

Satz

Alle LOOP-Programme terminieren. Daher sind alle LOOP-berechenbaren Funktionen total.

Terminierung von LOOP-Programmen

Satz

Alle LOOP-Programme terminieren. Daher sind alle LOOP-berechenbaren Funktionen total.

Beweis Zeige für alle (ρ, P) : Es gibt $j \in \mathbb{N}$ und ρ' , sodass $(\rho, P) \xrightarrow[\text{LOOP}]{j} (\rho', \varepsilon)$.
Durch Induktion über die Größe von P .

Terminierung von LOOP-Programmen

Satz

Alle LOOP-Programme terminieren. Daher sind alle LOOP-berechenbaren Funktionen total.

Beweis Zeige für alle (ρ, P) : Es gibt $j \in \mathbb{N}$ und ρ' , sodass $(\rho, P) \xrightarrow[\text{LOOP}]{j} (\rho', \varepsilon)$.
Durch Induktion über die Größe von P .

► Fall $(\rho, x_i := x_j \pm c)$: Es wird genau 1 Schritt benötigt.

Terminierung von LOOP-Programmen

Satz

Alle LOOP-Programme terminieren. Daher sind alle LOOP-berechenbaren Funktionen total.

Beweis Zeige für alle (ρ, P) : Es gibt $j \in \mathbb{N}$ und ρ' , sodass $(\rho, P) \xrightarrow[\text{LOOP}]{j} (\rho', \varepsilon)$.
Durch Induktion über die Größe von P .

- ▶ Fall $(\rho, x_i := x_j \pm c)$: Es wird genau 1 Schritt benötigt.
- ▶ Fall $P_1; P_2$: Die Induktionshypothese liefert j_1 und j_2 mit $(\rho, P_1; P_2) \xrightarrow[\text{LOOP}]{j_1} (\rho', P_2) \xrightarrow[\text{LOOP}]{j_2} (\rho'', \varepsilon)$.
Es werden genau $j_1 + j_2$ Schritte benötigt.

Terminierung von LOOP-Programmen

Satz

Alle LOOP-Programme terminieren. Daher sind alle LOOP-berechenbaren Funktionen total.

Beweis Zeige für alle (ρ, P) : Es gibt $j \in \mathbb{N}$ und ρ' , sodass $(\rho, P) \xrightarrow[\text{LOOP}]{j} (\rho', \varepsilon)$.
Durch Induktion über die Größe von P .

- ▶ Fall $(\rho, x_i := x_j \pm c)$: Es wird genau 1 Schritt benötigt.
- ▶ Fall $P_1; P_2$: Die Induktionshypothese liefert j_1 und j_2 mit $(\rho, P_1; P_2) \xrightarrow[\text{LOOP}]{j_1} (\rho', P_2) \xrightarrow[\text{LOOP}]{j_2} (\rho'', \varepsilon)$.
Es werden genau $j_1 + j_2$ Schritte benötigt.
- ▶ Fall **LOOP** x_i **DO** P **END**: Die Induktionshypothese j_i 's und ρ_i 's mit $(\rho_1, \text{LOOP } x_i \text{ DO } P \text{ END}) \xrightarrow[\text{LOOP}]{} (\rho_1, P; P; \dots; P) \xrightarrow[\text{LOOP}]{j_1} (\rho_2, P; \dots; P) \xrightarrow[\text{LOOP}]{j_2} \dots \xrightarrow[\text{LOOP}]{j_n} (\rho_{n+1}, \varepsilon)$ mit $n = \rho_1(x_i)$.
Es werden genau $1 + j_1 + j_2 + \dots + j_n$ Schritte benötigt. □

- ▶ Da es partielle turingberechenbare Funktionen gibt, gilt:
Es gibt **turingberechenbare Funktionen**, die **nicht LOOP-berechenbar** sind.
Ein Beispiel ist die überall undefinierte Funktion.

- ▶ Da es partielle turingberechenbare Funktionen gibt, gilt:
Es gibt **turingberechenbare Funktionen**, die **nicht LOOP-berechenbar** sind.
Ein Beispiel ist die überall undefinierte Funktion.
- ▶ Es gilt sogar:
Es gibt **intuitiv berechenbare Funktionen**, die **total** sind, aber trotzdem **nicht LOOP-berechenbar** sind.
Ein Beispiel ist die Ackermannfunktion (später heute).

Kodierung weiterer Befehle mit LOOP-Programmen

Befehl: $x_i := c$

Kodierung: $x_i := x_n + c$

wobei x_n keine der Eingabevariablen ist und
an keiner anderen Stelle im Programm verwendet wird
(und daher $\rho(x_n) = 0$)

Kodierung weiterer Befehle mit LOOP-Programmen

Befehl: $x_i := c$

Kodierung: $x_i := x_n + c$

wobei x_n keine der Eingabevariablen ist und
an keiner anderen Stelle im Programm verwendet wird
(und daher $\rho(x_n) = 0$)

Befehl: $x_i := x_j$

Kodierung: $x_i := x_j + 0$

Kodierung weiterer Befehle mit LOOP-Programmen

Befehl: $x_i := c$

Kodierung: $x_i := x_n + c$

wobei x_n keine der Eingabevariablen ist und
an keiner anderen Stelle im Programm verwendet wird
(und daher $\rho(x_n) = 0$)

Befehl: $x_i := x_j$

Kodierung: $x_i := x_j + 0$

Befehl: **IF** $x_i = 0$ **THEN** P **END**

Kodierung: $x_n := 1$;
LOOP x_i **DO** $x_n := 0$ **END**;
LOOP x_n **DO** P **END**

wobei x_n nicht in der Eingabe und nicht in P vorkommt

Kodierung weiterer Befehle mit LOOP-Programmen

Befehl: **IF** $x_i = 0$ **THEN** P_1 **ELSE** P_2 **END**

Kodierung: $x_m := 1$;
 $x_n := 1$;
LOOP x_i **DO** $x_m := 0$ **END**;
LOOP x_m **DO** $x_n := 0$; P_1 **END**;
LOOP x_n **DO** P_2 **END**

wobei x_m, x_n nicht in der Eingabe
und nicht sonst irgendwo im Programm vorkommen

Kodierung weiterer Befehle mit LOOP-Programmen

Befehl: **IF** $x_i = 0$ **THEN** P_1 **ELSE** P_2 **END**

Kodierung: $x_m := 1$;
 $x_n := 1$;
LOOP x_i **DO** $x_m := 0$ **END**;
LOOP x_m **DO** $x_n := 0$; P_1 **END**;
LOOP x_n **DO** P_2 **END**

wobei x_m, x_n nicht in der Eingabe
und nicht sonst irgendwo im Programm vorkommen

Kompliziertere if-Bedingungen gehen auch.

Kodierung weiterer Befehle mit LOOP-Programmen

Befehl: $x_i := x_j + x_k$

Kodierung: $x_\ell := x_j$;
LOOP x_k **DO** $x_\ell := x_\ell + 1$ **END**;
 $x_i := x_\ell$

wobei x_ℓ nicht in der Eingabe
und nicht sonst irgendwo im Programm vorkommt

Kodierung weiterer Befehle mit LOOP-Programmen

Befehl: $x_i := x_j + x_k$

Kodierung: $x_\ell := x_j$;
LOOP x_k **DO** $x_\ell := x_\ell + 1$ **END**;
 $x_i := x_\ell$

wobei x_ℓ nicht in der Eingabe
und nicht sonst irgendwo im Programm vorkommt

Dies zeigt auch, dass die Additionsfunktion $f(x_1, x_2) = x_1 + x_2$ LOOP-berechenbar ist.
Andere Rechenoperationen (wie $*$, mod , div) gehen analog.

9b

WHILE- und GOTO-Programme

Prof. Dr. Jasmin Blanchette

Lehr- und Forschungseinheit für
Theoretische Informatik und TheorembeweisenStand: 18. Juni 2024
Basierend auf Folien von PD Dr. David Sabel

Syntax von WHILE-Programmen

WHILE-Programme werden durch die kontextfreie Grammatik (V, Σ, P, Prg) erzeugt, wobei:

$$V = \{Prg, Var, Id, Const\}$$

$$\Sigma = \{\mathbf{WHILE}, \mathbf{LOOP}, \mathbf{DO}, \mathbf{END}, x, 0, \dots, 9, ,, :=, +, -\}$$

$$P = \{Prg \rightarrow \begin{array}{l} \mathbf{WHILE} \text{ } Var \neq 0 \mathbf{DO} \text{ } Prg \mathbf{END} \\ | \mathbf{LOOP} \text{ } Var \mathbf{DO} \text{ } Prg \mathbf{END} \\ | Prg; Prg \\ | Var := Var + Const \\ | Var := Var - Const, \end{array}\right.$$

$$Var \rightarrow xId,$$

$$Const \rightarrow Id,$$

$$Id \rightarrow 0 \mid 1 \mid \dots \mid 9 \mid 1Id \mid 2Id \mid \dots 9Id\}$$

Beachte: WHILE-Programme [erweitern](#) LOOP-Programme um das **WHILE**-Konstrukt.

Definition

Die Berechnungsschritte $(\rho, P) \xrightarrow{\text{WHILE}} (\rho', P')$, wobei ρ, ρ' Variablenbelegungen und P, P' WHILE-Programme oder das leere Programm ε sind, sind durch folgende Regeln definiert:

- ▶ $(\rho, x_i := x_j + c) \xrightarrow{\text{WHILE}} (\rho', \varepsilon)$, wobei $\rho' = \rho\{x_i \mapsto \rho(x_j) + c\}$
- ▶ $(\rho, x_i := x_j - c) \xrightarrow{\text{WHILE}} (\rho', \varepsilon)$, wobei $\rho' = \rho\{x_i \mapsto \max(0, \rho(x_j) - c)\}$
- ▶ $(\rho, P_1; P_2) \xrightarrow{\text{WHILE}} (\rho', P_2)$ wenn $(\rho, P_1) \xrightarrow{\text{WHILE}} (\rho', \varepsilon)$
- ▶ $(\rho, P_1; P_2) \xrightarrow{\text{WHILE}} (\rho', P'_1; P_2)$ wenn $(\rho, P_1) \xrightarrow{\text{WHILE}} (\rho', P'_1)$ und $P'_1 \neq \varepsilon$
- ▶ $(\rho, \mathbf{LOOP} \ x_i \ \mathbf{DO} \ P \ \mathbf{END}) \xrightarrow{\text{WHILE}} (\rho, \underbrace{P; \dots; P}_{\rho(x_i)\text{-mal}})$

Definition

- ▶ $(\rho, \mathbf{WHILE } x_i \neq 0 \mathbf{ DO } P \mathbf{ END}) \xrightarrow{\text{WHILE}} (\rho, \varepsilon)$ wenn $\rho(x_i) = 0$
- ▶ $(\rho, \mathbf{WHILE } x_i \neq 0 \mathbf{ DO } P \mathbf{ END}) \xrightarrow{\text{WHILE}} (\rho, P; \mathbf{WHILE } x_i \neq 0 \mathbf{ DO } P \mathbf{ END})$ wenn $\rho(x_i) \neq 0$

Semantik von WHILE-Programmen

Definition

- ▶ $(\rho, \mathbf{WHILE } x_i \neq 0 \mathbf{ DO } P \mathbf{ END}) \xrightarrow{\text{WHILE}} (\rho, \varepsilon)$ wenn $\rho(x_i) = 0$
- ▶ $(\rho, \mathbf{WHILE } x_i \neq 0 \mathbf{ DO } P \mathbf{ END}) \xrightarrow{\text{WHILE}} (\rho, P; \mathbf{WHILE } x_i \neq 0 \mathbf{ DO } P \mathbf{ END})$ wenn $\rho(x_i) \neq 0$

Wir schreiben $\xrightarrow{\text{WHILE}}^i$ für i Schritte und $\xrightarrow{\text{WHILE}}^*$ für 0 oder beliebig viele Schritte.

Definition

Eine Funktion $f : \mathbb{N}^k \rightarrow \mathbb{N}$ heißt **WHILE-berechenbar**, wenn es ein WHILE-Programm P gibt, das die folgenden Bedingungen erfüllt:

- ▶ Für alle $n_1, \dots, n_k \in \mathbb{N}$, sodass $f(n_1, \dots, n_k)$ definiert ist, gilt $(\rho, P) \xrightarrow[\text{WHILE}]^* (\rho', \varepsilon)$, wobei $\rho = \{x_1 \mapsto n_1, \dots, x_k \mapsto n_k\}$ und $\rho'(x_0) = f(n_1, \dots, n_k)$.
- ▶ Falls $f(n_1, \dots, n_k)$ nicht definiert ist, stoppt das Programm P nicht, d.h. für alle $i \in \mathbb{N}$ gibt es ρ' und P' , sodass $(\rho, P) \xrightarrow[\text{WHILE}]^i (\rho', P')$.

LOOP-berechenbar impliziert WHILE-berechenbar

Satz

Jede LOOP-berechenbare Funktion ist auch WHILE-berechenbar.

LOOP-berechenbar impliziert WHILE-berechenbar

Satz

Jede LOOP-berechenbare Funktion ist auch WHILE-berechenbar.

Beweis Das ist offensichtlich, da jedes LOOP-Programm auch ein WHILE-Programm mit derselben Semantik ist. □

LOOP-Schleife durch WHILE-Schleife ersetzen

Das **LOOP**-Konstrukt ist überflüssig in WHILE: Ersetze

LOOP x_i **DO** P **END**

durch

$x_m := x_i;$

WHILE $x_m \neq 0$ **DO** $x_m := x_m - 1; P$ **END**

wobei x_m nirgends sonst vorkommt.

WHILE-berechenbar impliziert turingberechenbar

Theorem

Jede WHILE-berechenbare Funktion ist auch turingberechenbar.

WHILE-berechenbar impliziert turingberechenbar

Theorem

Jede WHILE-berechenbare Funktion ist auch turingberechenbar.

Beweis Wir haben bereits gezeigt, dass Zuweisung, Hintereinanderschaltung und **WHILE** durch Turingmaschinen darstellbar sind.

WHILE-berechenbar impliziert turingberechenbar

Theorem

Jede WHILE-berechenbare Funktion ist auch turingberechenbar.

Beweis Wir haben bereits gezeigt, dass Zuweisung, Hintereinanderschaltung und **WHILE** durch Turingmaschinen darstellbar sind.

Dabei liegen die Variablen x_i jeweils auf Band i einer Mehrband-Turingmaschine.

WHILE-berechenbar impliziert turingberechenbar

Theorem

Jede WHILE-berechenbare Funktion ist auch turingberechenbar.

Beweis Wir haben bereits gezeigt, dass Zuweisung, Hintereinanderschaltung und **WHILE** durch Turingmaschinen darstellbar sind.

Dabei liegen die Variablen x_i jeweils auf Band i einer Mehrband-Turingmaschine.

LOOP wird vorher in **LOOP**-freies WHILE-Programm transformiert. □

Syntax von GOTO-Programmen

GOTO-Programme werden durch die kontextfreie Grammatik (V, Σ, P, Prg) erzeugt, wobei:

$$\begin{aligned} V &= \{Prg, Var, Id, Const\} \\ \Sigma &= \{\mathbf{GOTO}, \mathbf{HALT}, \mathbf{IF}, \mathbf{THEN}, x, 0, \dots, 9, ,, :=, +, -\} \\ P &= \{Prg \rightarrow M_{Id}: \mathbf{GOTO} M_{Id} \\ &\quad | M_{Id}: \mathbf{IF} x_i = 0 \mathbf{THEN GOTO} M_{Id} \\ &\quad | M_{Id}: \mathbf{HALT} \\ &\quad | M_{Id}: Var := Var + Const \\ &\quad | M_{Id}: Var := Var - Const \\ &\quad | Prg; Prg, \\ Var &\rightarrow x_{Id}, \\ Const &\rightarrow Id, \\ Id &\rightarrow 0 \mid 1 \mid \dots \mid 9 \mid 1Id \mid 2Id \mid \dots 9Id\} \end{aligned}$$

Befehle sind mit **verschiedenen** M_i : markiert. Wir lassen sie manchmal weg.

Semantik von GOTO-Programmen

Definition

Seien P_0 ein normalisiertes GOTO-Programm, P ein GOTO-Programm und ρ eine Variablenbelegung. Dann sind die Berechnungsschritte $\xrightarrow[\text{GOTO}]{P_0}$ durch folgende Regeln definiert:

- ▶ $(\rho, M_i: x_j := x_k + c; P) \xrightarrow[\text{GOTO}]{P_0} (\rho', P)$, wobei $\rho' = \rho\{x_j \mapsto \rho(x_k) + c\}$
- ▶ $(\rho, M_i: x_j := x_k + c) \xrightarrow[\text{GOTO}]{P_0} (\rho', \varepsilon)$, wobei $\rho' = \rho\{x_j \mapsto \rho(x_k) + c\}$
- ▶ $(\rho, M_i: x_j := x_k - c; P) \xrightarrow[\text{GOTO}]{P_0} (\rho', P)$, wobei $\rho' = \rho\{x_j \mapsto \max(0, \rho(x_k) - c)\}$
- ▶ $(\rho, M_i: x_j := x_k - c) \xrightarrow[\text{GOTO}]{P_0} (\rho', \varepsilon)$, wobei $\rho' = \rho\{x_j \mapsto \max(0, \rho(x_k) - c)\}$
- ▶ $(\rho, M_i: \mathbf{GOTO} M_k; P) \xrightarrow[\text{GOTO}]{M_1: A_1; \dots; M_n: A_n} (\rho, M_k: A_k; \dots; M_n: A_n)$ wenn $1 \leq k \leq n$
- ▶ $(\rho, M_i: \mathbf{GOTO} M_k) \xrightarrow[\text{GOTO}]{M_1: A_1; \dots; M_n: A_n} (\rho, M_k: A_k; \dots; M_n: A_n)$ wenn $1 \leq k \leq n$

Definition

- ▶ $(\rho, M_i: \text{IF } x_r = 0 \text{ THEN GOTO } M_k; P) \xrightarrow[\text{GOTO}]{M_1: A_1; \dots; M_n: A_n} (\rho, M_k: A_k; \dots; M_n: A_n)$
wenn $1 \leq k \leq n$ und $\rho(x_r) = 0$
- ▶ $(\rho, M_i: \text{IF } x_r = 0 \text{ THEN GOTO } M_k) \xrightarrow[\text{GOTO}]{M_1: A_1; \dots; M_n: A_n} (\rho, M_k: A_k; \dots; M_n: A_n)$
wenn $1 \leq k \leq n$ und $\rho(x_r) = 0$
- ▶ $(\rho, M_i: \text{IF } x_r = 0 \text{ THEN GOTO } M_k; P) \xrightarrow[\text{GOTO}]{P_0} (\rho, P)$ wenn $\rho(x_r) \neq 0$
- ▶ $(\rho, M_i: \text{IF } x_r = 0 \text{ THEN GOTO } M_k) \xrightarrow[\text{GOTO}]{P_0} (\rho, \varepsilon)$ wenn $\rho(x_r) \neq 0$
- ▶ $(\rho, M_i: \text{HALT}; P) \xrightarrow[\text{GOTO}]{P_0} (\rho, M_i: \text{HALT})$
- ▶ $(\rho, \varepsilon) \xrightarrow[\text{GOTO}]{P_0} (\rho, \varepsilon)$

Semantik von GOTO-Programmen

Definition

- ▶ $(\rho, M_i: \text{IF } x_r = 0 \text{ THEN GOTO } M_k; P) \xrightarrow[\text{GOTO}]{M_1: A_1; \dots; M_n: A_n} (\rho, M_k: A_k; \dots; M_n: A_n)$
wenn $1 \leq k \leq n$ und $\rho(x_r) = 0$
- ▶ $(\rho, M_i: \text{IF } x_r = 0 \text{ THEN GOTO } M_k) \xrightarrow[\text{GOTO}]{M_1: A_1; \dots; M_n: A_n} (\rho, M_k: A_k; \dots; M_n: A_n)$
wenn $1 \leq k \leq n$ und $\rho(x_r) = 0$
- ▶ $(\rho, M_i: \text{IF } x_r = 0 \text{ THEN GOTO } M_k; P) \xrightarrow[\text{GOTO}]{P_0} (\rho, P)$ wenn $\rho(x_r) \neq 0$
- ▶ $(\rho, M_i: \text{IF } x_r = 0 \text{ THEN GOTO } M_k) \xrightarrow[\text{GOTO}]{P_0} (\rho, \varepsilon)$ wenn $\rho(x_r) \neq 0$
- ▶ $(\rho, M_i: \text{HALT}; P) \xrightarrow[\text{GOTO}]{P_0} (\rho, M_i: \text{HALT})$
- ▶ $(\rho, \varepsilon) \xrightarrow[\text{GOTO}]{P_0} (\rho, \varepsilon)$

Wir schreiben $\xrightarrow[\text{GOTO}]{P_0}^i$ für i Schritte und $\xrightarrow[\text{GOTO}]{P_0}^*$ für 0 oder beliebig viele Schritte.

Definition

Ein GOTO-Programme ist **normalisiert**, wenn es von der Form $M_1 : A_1; \dots; M_n : A_n$ ist. Diese Form ist durch konsistentes Umbenennen der Markierungen herstellbar.

Definition

Ein GOTO-Programme ist **normalisiert**, wenn es von der Form $M_1 : A_1; \dots; M_n : A_n$ ist. Diese Form ist durch konsistentes Umbenennen der Markierungen herstellbar.

Eine Funktion $f : \mathbb{N}^k \rightarrow \mathbb{N}$ heißt **GOTO-berechenbar**, wenn es ein normalisiertes GOTO-Programm P gibt, das die folgenden Bedingungen erfüllt:

- ▶ Für alle $n_1, \dots, n_k \in \mathbb{N}$, sodass $f(n_1, \dots, n_k)$ definiert ist, gilt $(\rho, P) \xrightarrow[\text{GOTO}]{P}^* (\rho', \mathbf{HALT})$, wobei $\rho = \{x_1 \mapsto n_1, \dots, x_k \mapsto n_k\}$ und $\rho'(x_0) = f(n_1, \dots, n_k)$.
- ▶ Falls $f(n_1, \dots, n_k)$ nicht definiert ist, stoppt das Programm P nicht, d.h. für alle $i \in \mathbb{N}$ gibt es ρ' und P' , sodass $(\rho, P) \xrightarrow[\text{GOTO}]{P}^i (\rho', P')$.

WHILE-berechenbar impliziert GOTO-berechenbar

Satz

Jede WHILE-berechenbare Funktion ist auch GOTO-berechenbar.

WHILE-berechenbar impliziert GOTO-berechenbar

Satz

Jede WHILE-berechenbare Funktion ist auch GOTO-berechenbar.

Beweis Wir simulieren ein WHILE-Programm durch ein GOTO-Programm.

- ▶ $x_j := x_k \pm c$ und $P_1; P_2$ bleiben ungeändert.

WHILE-berechenbar impliziert GOTO-berechenbar

Satz

Jede WHILE-berechenbare Funktion ist auch GOTO-berechenbar.

Beweis Wir simulieren ein WHILE-Programm durch ein GOTO-Programm.

- ▶ $x_j := x_k \pm c$ und $P_1; P_2$ bleiben ungeändert.
- ▶ Ersetze **LOOP**-Schleifen durch **WHILE**-Schleifen.

WHILE-berechenbar impliziert GOTO-berechenbar

Satz

Jede WHILE-berechenbare Funktion ist auch GOTO-berechenbar.

Beweis Wir simulieren ein WHILE-Programm durch ein GOTO-Programm.

- ▶ $x_j := x_k \pm c$ und $P_1; P_2$ bleiben ungeändert.
- ▶ Ersetze **LOOP**-Schleifen durch **WHILE**-Schleifen.
- ▶ Ersetze **WHILE** $x_i \neq 0$ **DO** P **END** durch

```
 $M_j$ :   IF  $x_i = 0$  THEN GOTO  $M_k$ ;  
       $P'$ ;  
       $\vdots$   
 $M_{k-1}$ : GOTO  $M_j$ ;  
 $M_k$ :
```

wobei P' das GOTO-Programm zu P ist.

WHILE-berechenbar impliziert GOTO-berechenbar

Satz

Jede WHILE-berechenbare Funktion ist auch GOTO-berechenbar.

Beweis Wir simulieren ein WHILE-Programm durch ein GOTO-Programm.

- ▶ $x_j := x_k \pm c$ und $P_1; P_2$ bleiben ungeändert.
- ▶ Ersetze **LOOP**-Schleifen durch **WHILE**-Schleifen.
- ▶ Ersetze **WHILE** $x_i \neq 0$ **DO** P **END** durch

```
 $M_j:$    IF  $x_i = 0$  THEN GOTO  $M_k$ ;  
       $P'$ ;  
       $\vdots$   
 $M_{k-1}:$  GOTO  $M_j$ ;  
 $M_k:$ 
```

wobei P' das GOTO-Programm zu P ist.

- ▶ Füge **HALT** am Ende ein.

WHILE-berechenbar impliziert GOTO-berechenbar

Beweis (Fortsetzung) Sei P das WHILE-Programm und P' das GOTO-Programm.

Dann gilt:

- ▶ Falls $(\rho, P) \xrightarrow[\text{WHILE}]{*} (\rho', \varepsilon)$, dann $(\rho, P') \xrightarrow[\text{GOTO}]{P'}^* (\rho'', \mathbf{HALT})$
mit $\rho''(x_0) = \rho'(x_0)$.
- ▶ Falls $(\rho, P') \xrightarrow[\text{GOTO}]{P'}^* (\rho', \mathbf{HALT})$, dann $(\rho, P) \xrightarrow[\text{WHILE}]{*} (\rho'', \varepsilon)$
mit $\rho''(x_0) = \rho'(x_0)$.



GOTO-berechenbar impliziert WHILE-berechenbar

Satz

Jede GOTO-berechenbare Funktion ist auch WHILE-berechenbar.

Beweis Sei $M_1: A_1; \dots; M_n: A_n$ ein normalisiertes GOTO-Programm, sodass x_M nicht darin vorkommt. Definiere folgendes passendes WHILE-Programm:

```
 $x_M := 1;$   
WHILE  $x_M \neq 0$  DO  
  IF  $x_M = 1$  THEN  $P_1$  END;  
   $\vdots$   
  IF  $x_M = n$  THEN  $P_n$  END;  
END
```

wobei P_i auf der nächsten Folie definiert ist.

GOTO-berechenbar impliziert WHILE-berechenbar

Beweis (Fortsetzung) P_i ist definiert wie folgt:

- ▶ Falls $A_i = x_j := x_k \pm c$:
 $x_j := x_k \pm c; x_M := x_M + 1$
- ▶ Falls $A_i = \text{GOTO } M_j$:
 $x_M := j$
- ▶ Falls $A_i = \text{IF } x_k = 0 \text{ THEN GOTO } M_j$:
 $\text{IF } x_k = 0 \text{ THEN } x_M := j \text{ ELSE } x_M := x_M + 1 \text{ END}$
- ▶ Falls $A_i = \text{HALT}$:
 $x_M := 0$.

GOTO-berechenbar impliziert WHILE-berechenbar

Beweis (Fortsetzung) Sei P das GOTO-Programm und P' das WHILE-Programm.

Dann gilt:

- ▶ Falls $(\rho, P) \xrightarrow[\text{GOTO}]{P}^* (\rho', \mathbf{HALT})$, dann $(\rho, P') \xrightarrow[\text{WHILE}]{}^* (\rho'', \varepsilon)$
mit $\rho''(x_0) = \rho'(x_0)$.
- ▶ Falls $(\rho, P') \xrightarrow[\text{WHILE}]{}^* (\rho', \varepsilon)$, dann $(\rho, P) \xrightarrow[\text{GOTO}]{P}^* (\rho'', \mathbf{HALT})$
mit $\rho''(x_0) = \rho'(x_0)$.



GOTO-berechenbar ist äquivalent zu WHILE-berechenbar

Theorem

WHILE- und GOTO-Berechenbarkeit sind äquivalente Begriffe. D.h. jede WHILE-berechenbare Funktion ist auch GOTO-berechenbar und umgekehrt ist jede GOTO-berechenbare Funktion auch WHILE-berechenbar.

GOTO-berechenbar ist äquivalent zu WHILE-berechenbar

Theorem

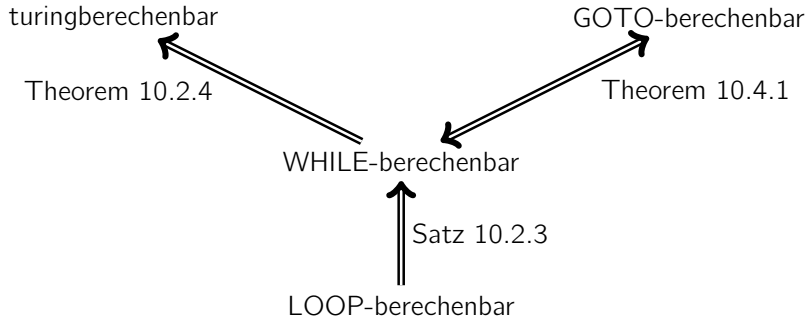
WHILE- und GOTO-Berechenbarkeit sind äquivalente Begriffe. D.h. jede WHILE-berechenbare Funktion ist auch GOTO-berechenbar und umgekehrt ist jede GOTO-berechenbare Funktion auch WHILE-berechenbar.

Die Übersetzung von GOTO-Programmen in WHILE-Programme zeigen auch:

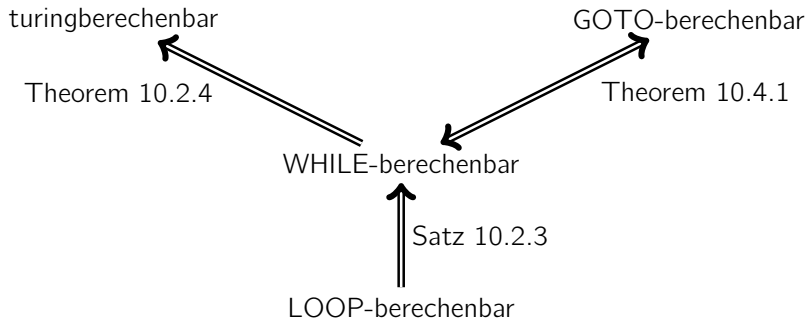
Satz

WHILE-Programme benötigen nur eine **WHILE**-Schleife.

Fazit bisher



Fazit bisher

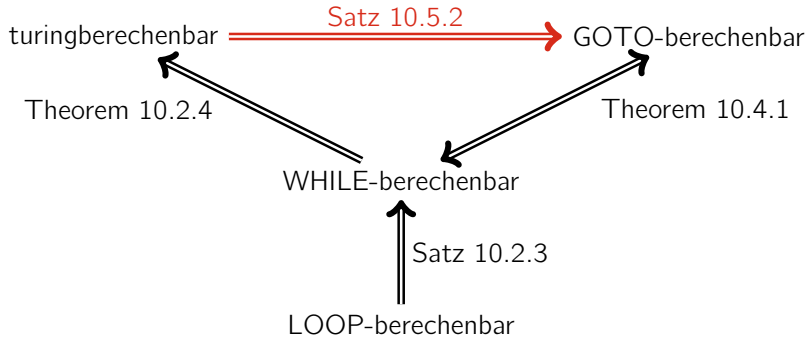


Unser Ziel:

Theorem

Turingberechenbarkeit, WHILE-Berechenbarkeit und GOTO-Berechenbarkeit sind äquivalente Begriffe.

Fazit bisher



Unser Ziel:

Theorem

Turingberechenbarkeit, WHILE-Berechenbarkeit und GOTO-Berechenbarkeit sind äquivalente Begriffe.

Turingberechenbar impliziert GOTO-berechenbar

Satz

Jede turingberechenbare Funktion ist auch GOTO-berechenbar.

Beweis Grundgedanke:

- ▶ Stelle TM-Konfiguration durch 3 Zahlen dar.
- ▶ Lege Zahlen in Programmvariablen ab.

Darstellung von Wörtern als natürliche Zahlen

- ▶ Sei $\Gamma = \{a_1, \dots, a_m\}$ das Bandalphabet.
- ▶ Zeichen a_i wird mit Index i identifiziert. (Beachte: $i \neq 0$.)
D.h. $a_{i_1} \cdots a_{i_n} \in \Gamma^*$ ist äquivalent zur Folge $(i_1 \dots i_n)$.
- ▶ Sei $b > m$.

Darstellung von Wörtern als natürliche Zahlen

- ▶ Sei $\Gamma = \{a_1, \dots, a_m\}$ das Bandalphabet.
- ▶ Zeichen a_i wird mit Index i identifiziert. (Beachte: $i \neq 0$.)
D.h. $a_{i_1} \cdots a_{i_n} \in \Gamma^*$ ist äquivalent zur Folge $(i_1 \dots i_n)$.
- ▶ Sei $b > m$.

Kodierung: Das Wort $a_{i_1} \dots a_{i_n}$ lässt sich als Zahl $(i_1 \cdots i_n)_b$ im Stellenwertsystem zur Basis b darstellen. Der Wert von $a_{i_1} \cdots a_{i_n}$ ist

$$(i_1 \cdots i_n)_b = \sum_{j=1}^n i_j \cdot b^{n-j}$$

Darstellung von Wörtern als natürliche Zahlen

- ▶ Sei $\Gamma = \{a_1, \dots, a_m\}$ das Bandalphabet.
- ▶ Zeichen a_i wird mit Index i identifiziert. (Beachte: $i \neq 0$.)
D.h. $a_{i_1} \cdots a_{i_n} \in \Gamma^*$ ist äquivalent zur Folge $(i_1 \dots i_n)$.
- ▶ Sei $b > m$.

Kodierung: Das Wort $a_{i_1} \dots a_{i_n}$ lässt sich als Zahl $(i_1 \cdots i_n)_b$ im Stellenwertsystem zur Basis b darstellen. Der Wert von $a_{i_1} \cdots a_{i_n}$ ist

$$(i_1 \cdots i_n)_b = \sum_{j=1}^n i_j \cdot b^{n-j}$$

Umkehrung: Berechne $(i_1 \cdots i_n)$ aus einer natürlichen Zahl.

- ▶ Teile wiederholt mit Rest durch Basis b bis kein Rest mehr entsteht.
- ▶ Liste der Reste: $r_0, \dots, r_m$.
- ▶ Ergibt $(r_m \cdots r_0)_b$ zur Basis b und repräsentiert daher das Wort $a_{r_m} \cdots a_{r_0}$.

Beispiel für die Darstellung von Wörtern als natürliche Zahlen

Sei $\Gamma = \{\square, 0, 1\}$, sodass $a_1 = \square, a_2 = 0, a_3 = 1$.

Beispiel für die Darstellung von Wörtern als natürliche Zahlen

Sei $\Gamma = \{\square, 0, 1\}$, sodass $a_1 = \square, a_2 = 0, a_3 = 1$.

Betrachte $\square 110\square$, d.h. $a_1 a_3 a_3 a_2 a_1$. Sei $b = 4$.

Beispiel für die Darstellung von Wörtern als natürliche Zahlen

Sei $\Gamma = \{\square, 0, 1\}$, sodass $a_1 = \square, a_2 = 0, a_3 = 1$.

Betrachte $\square 110\square$, d.h. $a_1 a_3 a_3 a_2 a_1$. Sei $b = 4$.

Identifiziere $\square 110\square$ mit (13321) und

$$(13321)_4 = 1*4^4 + 3*4^3 + 3*4^2 + 2*4^1 + 1*4^0 = 1*256 + 3*64 + 3*16 + 2*4 + 1*1 = 505.$$

Beispiel für die Darstellung von Wörtern als natürliche Zahlen

Sei $\Gamma = \{\square, 0, 1\}$, sodass $a_1 = \square, a_2 = 0, a_3 = 1$.

Betrachte $\square 110\square$, d.h. $a_1 a_3 a_3 a_2 a_1$. Sei $b = 4$.

Identifiziere $\square 110\square$ mit (13321) und

$$(13321)_4 = 1 \cdot 4^4 + 3 \cdot 4^3 + 3 \cdot 4^2 + 2 \cdot 4^1 + 1 \cdot 4^0 = 1 \cdot 256 + 3 \cdot 64 + 3 \cdot 16 + 2 \cdot 4 + 1 \cdot 1 = 505.$$

Umgekehrt ergibt

$$505/4 = 126 \quad \text{Rest } 1$$

$$126/4 = 31 \quad \text{Rest } 2$$

$$31/4 = 7 \quad \text{Rest } 3$$

$$7/4 = 1 \quad \text{Rest } 3$$

$$1/4 = 0 \quad \text{Rest } 1$$

$$0/4 = 0 \quad \text{Rest } 0$$

Dies ergibt die Zahl $(13321)_4$, welche das Wort $a_1 a_3 a_3 a_2 a_1 = \square 110\square$ darstellt.

Kodierung von Turingmaschinen-Konfiguration im GOTO-Programm

Eine beliebige Konfiguration einer Turingmaschine mit $\Gamma = \{a_1, \dots, a_m\}$ ist von der Form

$$a_{i_1} \cdots a_{i_n} Z_k a_{j_1} \cdots a_{j_m}$$

Kodierung von Turingmaschinen-Konfiguration im GOTO-Programm

Eine beliebige Konfiguration einer Turingmaschine mit $\Gamma = \{a_1, \dots, a_m\}$ ist von der Form

$$a_{i_1} \cdots a_{i_n} z_k a_{j_1} \cdots a_{j_m}$$

Sie wird im GOTO-Programm durch 3 Variablen x_z, x_p, x_s dargestellt:

$$x_z = k$$

$$x_p = (i_1 \cdots i_n)_b$$

$$x_s = (j_m \cdots j_1)_b$$

Beachte: x_s verwendet umgekehrte Reihenfolge der Ziffern.

GOTO-Programme für Übergänge

Für $\delta(z_k, a_{j_1}) = (z_\ell, a_{j'}, x)$ mit $x \in \{N, L, R\}$ sei $DELTA(k, j_1)$ das zugehörige GOTO-Programm. Wir gehen die verschiedenen Fälle durch.

GOTO-Programme für Übergänge

Für $\delta(z_k, a_{j_1}) = (z_\ell, a_{j'}, x)$ mit $x \in \{N, L, R\}$ sei $DELTA(k, j_1)$ das zugehörige GOTO-Programm. Wir gehen die verschiedenen Fälle durch.

- Fall $\delta(z_k, a_{j_1}) = (z_\ell, a_{j'}, N)$:

Der Übergang ist

$$a_{i_1} \cdots a_{i_n} \textcolor{blue}{z}_k a_{j_1} \cdots a_{j_m} \vdash a_{i_1} \cdots a_{i_n} \textcolor{blue}{z}_\ell a_{j'} a_{j_2} \cdots a_{j_m}$$

Daher:

- x_z muss auf den Wert ℓ aktualisiert werden.
- x_p bleibt unverändert.
- x_s muss von $(j_m \cdots j_1)_b$ zu $(j_m \cdots j_2 j')_b$ aktualisiert werden.

GOTO-Programme für Übergänge

Für $\delta(z_k, a_{j_1}) = (z_\ell, a_{j'}, x)$ mit $x \in \{N, L, R\}$ sei $DELTA(k, j_1)$ das zugehörige GOTO-Programm. Wir gehen die verschiedenen Fälle durch.

- Fall $\delta(z_k, a_{j_1}) = (z_\ell, a_{j'}, N)$:

Der Übergang ist

$$a_{i_1} \cdots a_{i_n} \textcolor{blue}{z}_k a_{j_1} \cdots a_{j_m} \vdash a_{i_1} \cdots a_{i_n} \textcolor{blue}{z}_\ell a_{j'} a_{j_2} \cdots a_{j_m}$$

Daher:

- x_z muss auf den Wert ℓ aktualisiert werden.
- x_p bleibt unverändert.
- x_s muss von $(j_m \cdots j_1)_b$ zu $(j_m \cdots j_2 j')_b$ aktualisiert werden.

Das Programm $DELTA(k, j_1)$ sei dann

$$x_z := \ell;$$

$$x_s := x_s \text{ div } b;$$

$$x_s := b * x_s + j'$$

GOTO-Programme für Übergänge

- Fall $\delta(z_k, a_{j_1}) = (z_\ell, a_{j'}, R)$:

Der Übergang ist

$$a_{i_1} \cdots a_{i_n} \textcolor{blue}{z_k} a_{j_1} \cdots a_{j_m} \vdash a_{i_1} \cdots a_{i_n} a_{j'} \textcolor{blue}{z_\ell} a_{j_2} \cdots a_{j_m}$$

falls $m > 1$. Daher:

- x_z muss auf den Wert ℓ aktualisiert werden.
- x_p muss von $(i_1 \cdots i_n)_b$ zu $(i_1 \cdots i_n j')_b$ aktualisiert werden.
- x_s muss von $(j_m \cdots j_1)_b$ zu $(j_m \cdots j_2)_b$ aktualisiert werden.

GOTO-Programme für Übergänge

- Fall $\delta(z_k, a_{j_1}) = (z_\ell, a_{j'}, R)$:

Der Übergang ist

$$a_{i_1} \cdots a_{i_n} \textcolor{blue}{z_k} a_{j_1} \cdots a_{j_m} \vdash a_{i_1} \cdots a_{i_n} a_{j'} \textcolor{blue}{z_\ell} a_{j_2} \cdots a_{j_m}$$

falls $m > 1$. Daher:

- x_z muss auf den Wert ℓ aktualisiert werden.
- x_p muss von $(i_1 \cdots i_n)_b$ zu $(i_1 \cdots i_n j')_b$ aktualisiert werden.
- x_s muss von $(j_m \cdots j_1)_b$ zu $(j_m \cdots j_2)_b$ aktualisiert werden.

Das Programm $DELTA(k, j_1)$ sei dann

$$x_z := \ell;$$
$$x_p := b * x_p + j';$$
$$x_s := x_s \text{ div } b$$

GOTO-Programme für Übergänge

Für den Fall $m = 1$ gilt

$$a_{i_1} \cdots a_{i_n} z_k a_{j_1} \vdash a_{i_1} \cdots a_{i_n} a_{j'} z_\ell \square$$

Das Programm $DELTA(k, j_1)$ sei dann

$$x_z := \ell;$$

$$x_p := b * x_p + j'; \quad \text{wobei } \square = a_r.$$

$$x_s := r$$

GOTO-Programme für Übergänge

- ▶ Fall $\delta(z_k, a_{j_1}) = (z_\ell, a_{j'}, L)$:

GOTO-Programme für Übergänge

- Fall $\delta(z_k, a_{j_1}) = (z_\ell, a_{j'}, L)$:

Der Übergang ist

$$a_{i_1} \cdots a_{i_n} \textcolor{blue}{z_k} a_{j_1} \cdots a_{j_m} \vdash a_{i_1} \cdots a_{i_{n-1}} \textcolor{blue}{z_\ell} a_{i_n} a_{j'} a_{j_2} \cdots a_{j_m}$$

falls $n > 0$. Daher:

- x_z muss auf den Wert ℓ aktualisiert werden.
- x_p muss von $(i_1 \cdots i_n)_b$ zu $(i_1 \cdots i_{n-1})_b$ aktualisiert werden.
- x_s muss von $(j_m \cdots j_1)_b$ zu $(j_m \cdots j_2 j' i_n)_b$ aktualisiert werden.

GOTO-Programme für Übergänge

- Fall $\delta(z_k, a_{j_1}) = (z_\ell, a_{j'}, L)$:

Der Übergang ist

$$a_{i_1} \cdots a_{i_n} \textcolor{blue}{z_k} a_{j_1} \cdots a_{j_m} \vdash a_{i_1} \cdots a_{i_{n-1}} \textcolor{blue}{z_\ell} a_{i_n} a_{j'} a_{j_2} \cdots a_{j_m}$$

falls $n > 0$. Daher:

- x_z muss auf den Wert ℓ aktualisiert werden.
- x_p muss von $(i_1 \cdots i_n)_b$ zu $(i_1 \cdots i_{n-1})_b$ aktualisiert werden.
- x_s muss von $(j_m \cdots j_1)_b$ zu $(j_m \cdots j_2 j' i_n)_b$ aktualisiert werden.

Das Programm $DELTA(k, j_1)$ dazu:

$x_z := \ell;$	// neuer Zustand z_ℓ
$x_s := x_s \text{ div } b;$	// Abschneiden der letzten Stelle: $(j_m \cdots j_1) \rightarrow (j_m \cdots j_2)$
$x_s := b * x_s + j';$	// j' als letzte Stelle hinzufügen: $(j_m \cdots j_2) \rightarrow (j_m \cdots j_2 j')$
$x_s := b * x_s + (x_p \text{ mod } b);$	// $i_n = x_p \text{ mod } b$ am Ende hinzufügen: // $(j_m \cdots j_2 j') \rightarrow (j_m \cdots j_2 j' i_n)$
$x_p := x_p \text{ div } b$	// Abschneiden der letzten Stelle: $(i_1 \cdots i_n) \rightarrow (i_1 \cdots i_{n-1})$

GOTO-Programme für Übergänge

Für den Fall $n = 0$ gilt

$$z_k a_{j_1} \cdots a_{j_m} \vdash z_\ell \square a_{j_1} \cdots a_{j_m}$$

Setze zuerst $x_p := r$, wobei $\square = a_r$ und führe dann das Programm für den allgemeinen Fall aus.

Turingberechenbar impliziert GOTO-berechenbar

Die Turingmaschine wird durch das folgende GOTO-Programm simuliert (mit Eingaben in $x_1, \dots, x_k$ und Ausgabe in x_0):

$M_1 : P_1;$

$M_2 : P_2;$

$M_3 : P_3$

- ▶ P_1 erzeugt für die Eingabe in $x_1, \dots, x_k$ die Binärzahldarstellung und dann die Darstellung der TM-Konfiguration in x_p, x_z, x_s .
- ▶ P_2 : Siehe nächste Folie.
- ▶ P_3 verwendet die Darstellung der Endkonfiguration, um die Ausgabe in der Ausgabevariablen x_0 zu erzeugen.

Turingberechenbar impliziert GOTO-berechenbar

Programmteil P_2 hat die Form:

```
 $M_2 :$     $x_a := x_s \bmod b;$   
         IF  $x_z = 1$  and  $x_a = 1$  THEN GOTO  $M_{1,1};$   
         IF  $x_z = 1$  and  $x_a = 2$  THEN GOTO  $M_{1,2};$   
          $\vdots$   
         IF  $x_z = q$  and  $x_a = m$  THEN GOTO  $M_{q,m};$   
 $M_{1,1} :$   $P_{1,1};$   
         GOTO  $M_2;$   
 $M_{1,2} :$   $P_{1,2};$   
         GOTO  $M_2;$   
          $\vdots$   
 $M_{q,m} :$   $P_{q,m}$   
         GOTO  $M_2$ 
```

Programm $P_{i,j}$ ist **GOTO** M_3 , wenn $z_i \in E$ und $DELTA(i,j)$ sonst.

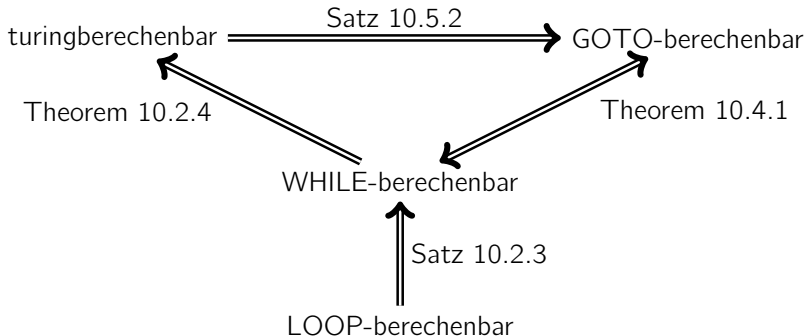


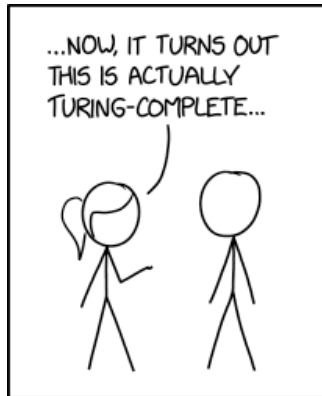
Theorem

Turingberechenbarkeit, WHILE-Berechenbarkeit und GOTO-Berechenbarkeit sind äquivalente Begriffe.

Theorem

Turingberechenbarkeit, WHILE-Berechenbarkeit und GOTO-Berechenbarkeit sind äquivalente Begriffe.





THIS PHRASE EITHER MEANS
SOMEONE SPENT SIX MONTHS
GETTING A DISHWASHER TO
PLAY MARIO OR YOU'RE UNDER
ATTACK BY A NATION-STATE.

xkcd.com/2556/

9c

Die Ackermannfunktion

Prof. Dr. Jasmin Blanchette

Lehr- und Forschungseinheit für
Theoretische Informatik und Theorembeweisen

Stand: 18. Juni 2024

Basierend auf Folien von PD Dr. David Sabel



Die Ackermannfunktion

Die Ackermannfunktion ist eine von Wilhelm Ackermann in den 1920er Jahre vorgeschlagene sehr schnell wachsende Funktion.

Wir betrachten eine Variante von Rózsa Péter.

Die Ackermannfunktion

Die Ackermannfunktion ist eine von Wilhelm Ackermann in den 1920er Jahre vorgeschlagene sehr schnell wachsende Funktion.

Wir betrachten eine Variante von Rózsa Péter.

Die Ackermannfunktion $a : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$ ist definiert durch

$$a(x, y) := \begin{cases} y + 1 & \text{falls } x = 0 \\ a(x - 1, 1) & \text{falls } x > 0 \text{ und } y = 0 \\ a(x - 1, a(x, y - 1)) & \text{falls } x > 0 \text{ und } y > 0 \end{cases}$$

Die Ackermannfunktion

Die Ackermannfunktion ist eine von Wilhelm Ackermann in den 1920er Jahre vorgeschlagene sehr schnell wachsende Funktion.

Wir betrachten eine Variante von Rózsa Péter.

Die Ackermannfunktion $a : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$ ist definiert durch

$$a(x, y) := \begin{cases} y + 1 & \text{falls } x = 0 \\ a(x - 1, 1) & \text{falls } x > 0 \text{ und } y = 0 \\ a(x - 1, a(x, y - 1)) & \text{falls } x > 0 \text{ und } y > 0 \end{cases}$$

Unser nächstes Ziel:

Theorem

Die Ackermannfunktion ist **WHILE-berechenbar**, aber sie ist **nicht LOOP-berechenbar** (obwohl sie total ist).

Einige Werte der Ackermannfunktion

Tabelle mit $a(x, y)$ -Einträgen:

		x				
		0	1	2	3	4
y	0	1	2	3	5	13
	1	2	3	5	13	65533
	2	3	4	7	29	$2^{65536} - 3$
	3	4	5	9	61	$a(3, 2^{65536} - 3)$
	4	5	6	11	125	$a(3, a(4, 3))$

Totalität der Ackermannfunktion

Lemma

Die Ackermannfunktion ist total.

Totalität der Ackermannfunktion

Lemma

Die Ackermannfunktion ist total.

Beweis Mehrmaliges Entfalten der dritten Zeile in

$$a(x, y) = \begin{cases} y + 1 & \text{falls } x = 0 \\ a(x - 1, 1) & \text{falls } x > 0 \text{ und } y = 0 \\ a(x - 1, a(x, y - 1)) & \text{falls } x > 0 \text{ und } y > 0 \end{cases}$$

zeigt

$$a(x, y) = \underbrace{a(x - 1, a(x - 1, \dots, a(x - 1, 1) \dots))}_{(y+1)\text{-mal}} \quad \text{falls } x > 0 \text{ und } y > 0$$

Alle rekursiven Aufrufe sind nun echt kleiner bezüglich des ersten Arguments.
Daher terminiert die Ackermannfunktion stets. □

Monotonie-Eigenschaften der Ackermannfunktion

Lemma

Die folgenden Monotonie-Eigenschaften gelten für die Ackermannfunktion a :

1. $y < a(x, y)$
2. $a(x, y) < a(x, y + 1)$
3. $a(x, y + 1) \leq a(x + 1, y)$
4. $a(x, y) < a(x + 1, y)$
5. falls $x \leq x'$ und $y \leq y'$, dann gilt auch $a(x, y) \leq a(x', y')$.

Monotonie-Eigenschaften der Ackermannfunktion

Lemma

Die folgenden Monotonie-Eigenschaften gelten für die Ackermannfunktion a :

1. $y < a(x, y)$
2. $a(x, y) < a(x, y + 1)$
3. $a(x, y + 1) \leq a(x + 1, y)$
4. $a(x, y) < a(x + 1, y)$
5. falls $x \leq x'$ und $y \leq y'$, dann gilt auch $a(x, y) \leq a(x', y')$.

Beweis Durch Induktion. Siehe Skript.



Berechenbarkeit der Ackermannfunktion

Es ist intuitiv klar, dass die Ackermannfunktion mit jeder modernen Programmiersprache implementierbar ist. Daher ist sie auch intuitiv berechenbar.

Berechenbarkeit der Ackermannfunktion

Es ist intuitiv klar, dass die Ackermannfunktion mit jeder modernen Programmiersprache implementierbar ist. Daher ist sie auch intuitiv berechenbar.

Zudem:

Satz

Die Ackermannfunktion ist WHILE-berechenbar.

Berechenbarkeit der Ackermannfunktion

Es ist intuitiv klar, dass die Ackermannfunktion mit jeder modernen Programmiersprache implementierbar ist. Daher ist sie auch intuitiv berechenbar.

Zudem:

Satz

Die Ackermannfunktion ist WHILE-berechenbar.

Beweis Erstelle ein WHILE-Programm, welches die rekursive Berechnung durchführt mit einem Keller.

Berechenbarkeit der Ackermannfunktion

Es ist intuitiv klar, dass die Ackermannfunktion mit jeder modernen Programmiersprache implementierbar ist. Daher ist sie auch intuitiv berechenbar.

Zudem:

Satz

Die Ackermannfunktion ist WHILE-berechenbar.

Beweis Erstelle ein WHILE-Programm, welches die rekursive Berechnung durchführt mit einem Keller.

Daher ist der erste Schritt im Beweis:

- ▶ Darstellung des Kellers
- ▶ Implementierung von Operationen auf dem Keller als WHILE-Programme.

Danach wird das Programm angegeben, das die Kelleroperationen durchführt.

Darstellung vom Keller

Darstellung vom Keller:

- ▶ Folge von Zahlen $\langle n_1, \dots, n_k \rangle$ sodass n_1 ganz oben liegt.
Leerer Keller: 0.

Darstellung vom Keller

Darstellung vom Keller:

- Folge von Zahlen $\langle n_1, \dots, n_k \rangle$ sodass n_1 ganz oben liegt.
Leerer Keller: 0.

Darstellung von Folgen mit fester Länge k als eine einzige Zahl:

- $\langle n_1, \dots, n_k \rangle = c(n_1, c(n_2, (\dots, c(n_k, 0) \dots)))$
wobei $c(x, y)$ eine Bijektion $\mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$ ist.

Wir nehmen an, dass *left* und *right* existieren mit $\text{left}(c(x, y)) = x$ und $\text{right}(c(x, y)) = y$. Genauere Details zu c , *left*, *right* werden später nochmal erörtert.

Darstellung vom Keller

Darstellung vom Keller:

- ▶ Folge von Zahlen $\langle n_1, \dots, n_k \rangle$ sodass n_1 ganz oben liegt.
Leerer Keller: 0.

Darstellung von Folgen mit fester Länge k als eine einzige Zahl:

- ▶ $\langle n_1, \dots, n_k \rangle = c(n_1, c(n_2, (\dots, c(n_k, 0) \dots)))$
wobei $c(x, y)$ eine Bijektion $\mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$ ist.

Wir nehmen an, dass *left* und *right* existieren mit $left(c(x, y)) = x$ und $right(c(x, y)) = y$. Genauere Details zu c , *left*, *right* werden später nochmal erörtert.

Darstellung vom Keller im WHILE-Programm:

- ▶ Variable *stack*, die $\langle n_1, \dots, n_k \rangle = c(n_1, c(n_2, (\dots, c(n_k, 0) \dots)))$ speichert
- ▶ Variable *stacksize*, die die Größe des Kellers speichert
- ▶ Invariante: $\langle n_1, \dots, n_k \rangle$ stellt $a(n_k, a(n_{k-1}, \dots a(n_2, n_1) \dots))$ dar.

- $push(x, stack)$ legt Zahl x oben auf den Keller.

WHILE-Programm dazu:

$$\begin{aligned} stack &:= c(x, stack); \\ stacksize &:= stacksize + 1 \end{aligned}$$

- $x := pop(stack)$ entfernt das oberste Element vom Keller und setzt x auf dessen Wert.

WHILE-Programm dazu:

$$\begin{aligned} x &:= left(stack); \\ stack &:= right(stack); \\ stacksize &:= stacksize - 1 \end{aligned}$$

WHILE-Programm zur Berechnung von $a(x, y)$

```
stack := 0;
stacksize := 0;
push(x, stack);
push(y, stack);
WHILE stacksize  $\neq$  1 DO
  y := pop(stack);
  x := pop(stack);
  IF x = 0 THEN push(y + 1, stack)
  ELSE IF y = 0 THEN push(x - 1, stack); push(1, stack)
  ELSE push(x - 1, stack); push(x, stack); push(y - 1, stack)
  END IF
END
END;
result := pop(stack)
```

$\langle n_1, \dots, n_k \rangle$ stellt $a(n_k, a(n_{k-1}, \dots a(n_2, n_1) \dots))$ dar

$$a(x, y) = \begin{cases} y + 1 & \text{falls } x = 0 \\ a(x - 1, 1) & \text{falls } x > 0 \text{ und } y = 0 \\ a(x - 1, a(x, y - 1)) & \text{falls } x > 0 \text{ und } y > 0 \end{cases}$$



Maximale LOOP-berechenbare Zahlen

Sei P ein LOOP-Programm.

Seien $x_0, \dots, x_k$ die in P vorkommenden Variablen.

Maximale LOOP-berechenbare Zahlen

Sei P ein LOOP-Programm.

Seien $x_0, \dots, x_k$ die in P vorkommenden Variablen.

Sei

$$f_P(n) = \max \left\{ \sum_{i=0}^k \rho'(x_i) \mid \sum_{i=0}^k \rho(x_i) \leq n \text{ und } (\rho, P) \xrightarrow[\text{LOOP}]{*} (\rho', \varepsilon) \right\}$$

Maximale LOOP-berechenbare Zahlen

Sei P ein LOOP-Programm.

Seien $x_0, \dots, x_k$ die in P vorkommenden Variablen.

Sei

$$f_P(n) = \max \left\{ \sum_{i=0}^k \rho'(x_i) \mid \sum_{i=0}^k \rho(x_i) \leq n \text{ und } (\rho, P) \xrightarrow[\text{LOOP}]{}^* (\rho', \varepsilon) \right\}$$

$f_P(n)$ ist also die maximale Zahl, die als **Summe aller Endbelegungen ρ' der Variablen $x_0, \dots, x_k$** zustande kommt, über alle initialen Variablenbelegungen ρ , die in der **Summe der Werte $\rho(x_0), \dots, \rho(x_k)$ den Wert n** nicht überschreiten.

Beispiel für eine maximale LOOP-berechenbare Zahl

Wir berechnen $f_P(5)$ für das LOOP-Programm P :

$$x_0 := x_1 + 100$$

Beispiel für eine maximale LOOP-berechenbare Zahl

Wir berechnen $f_P(5)$ für das LOOP-Programm P :

$$x_0 := x_1 + 100$$

Wir wählen $\rho(x_1) = 5$ und $\rho(x_i) = 0$ für alle anderen Variablen x_i .

Daher $\sum_{i=0}^k \rho(x_i) = 5 \leq 5$.

Beispiel für eine maximale LOOP-berechenbare Zahl

Wir berechnen $f_P(5)$ für das LOOP-Programm P :

$$x_0 := x_1 + 100$$

Wir wählen $\rho(x_1) = 5$ und $\rho(x_i) = 0$ für alle anderen Variablen x_i .

Daher $\sum_{i=0}^k \rho(x_i) = 5 \leq 5$.

Die Ausführung von P führt zu folgender Variablenbelegung ρ' :

$$\rho'(x_0) = 105$$

$$\rho'(x_1) = 5$$

$$\rho'(x_i) = 0 \quad \text{für alle anderen Variablen } x_i$$

Daher $\sum_{i=0}^k \rho'(x_i) = 110$.

Beispiel für eine maximale LOOP-berechenbare Zahl

Wir berechnen $f_P(5)$ für das LOOP-Programm P :

$$x_0 := x_1 + 100$$

Wir wählen $\rho(x_1) = 5$ und $\rho(x_i) = 0$ für alle anderen Variablen x_i .

Daher $\sum_{i=0}^k \rho(x_i) = 5 \leq 5$.

Die Ausführung von P führt zu folgender Variablenbelegung ρ' :

$$\rho'(x_0) = 105$$

$$\rho'(x_1) = 5$$

$$\rho'(x_i) = 0 \quad \text{für alle anderen Variablen } x_i$$

Daher $\sum_{i=0}^k \rho'(x_i) = 110$.

Da es keine Weise gibt, einen höheren Wert für $\sum_{i=0}^k \rho'(x_i)$ zu erreichen, ist $f_P(5) = 110$.

Maximale LOOP-berechenbare Zahlen

Satz

Für jedes LOOP-Programm P gibt es eine Konstante k , sodass $f_P(n) < a(k, n)$ für alle $n \in \mathbb{N}$.

Maximale LOOP-berechenbare Zahlen

Satz

Für jedes LOOP-Programm P gibt es eine Konstante k , sodass $f_P(n) < a(k, n)$ für alle $n \in \mathbb{N}$.

Beweis Normalisiere P zunächst:

- Ersetze Zuweisungen $x_i := x_j + c$ mit $c > 1$ durch
 $x_i := x_j + 1; \underbrace{x_i := x_j + 1; \dots x_i := x_j + 1}_{(c-1)\text{-mal}}$.

Maximale LOOP-berechenbare Zahlen

Satz

Für jedes LOOP-Programm P gibt es eine Konstante k , sodass $f_P(n) < a(k, n)$ für alle $n \in \mathbb{N}$.

Beweis Normalisiere P zunächst:

- ▶ Ersetze Zuweisungen $x_i := x_j + c$ mit $c > 1$ durch $x_i := x_j + 1; \underbrace{x_i := x_j + 1; \dots x_i := x_j + 1}_{(c-1)\text{-mal}}$.
- ▶ Für **LOOP** x_i **DO** Q **END** und x_i kommt in Q vor:
Ersetze **LOOP** x_i **DO** Q **END** durch $x'_i := x_i; \textbf{LOOP } x'_i \textbf{ DO } Q \textbf{ END}; x'_i := 0$,
wobei x'_i eine neue Variable ist.

Maximale LOOP-berechenbare Zahlen

Satz

Für jedes LOOP-Programm P gibt es eine Konstante k , sodass $f_P(n) < a(k, n)$ für alle $n \in \mathbb{N}$.

Beweis Normalisiere P zunächst:

- ▶ Ersetze Zuweisungen $x_i := x_j + c$ mit $c > 1$ durch
$$x_i := x_j + 1; \underbrace{x_i := x_i + 1; \dots x_i := x_i + 1}_{(c-1)\text{-mal}}$$
- ▶ Für **LOOP** x_i **DO** Q **END** und x_i kommt in Q vor:
Ersetze **LOOP** x_i **DO** Q **END** durch $x'_i := x_i; \text{ **LOOP** } x'_i \text{ **DO** } Q \text{ **END**}; x'_i := 0$,
wobei x'_i eine neue Variable ist.

Beides verändert f_P nicht.

Zeige Behauptung für normalisierte Programme.

Maximale LOOP-berechenbare Zahlen

Beweis (Fortsetzung) Zu zeigen: Für jedes normalisierte LOOP-Programm P gibt es eine Konstante k , sodass $f_P(n) < a(k, n)$ für alle $n \in \mathbb{N}$.

Maximale LOOP-berechenbare Zahlen

Beweis (Fortsetzung) Zu zeigen: Für jedes normalisierte LOOP-Programm P gibt es eine Konstante k , sodass $f_P(n) < a(k, n)$ für alle $n \in \mathbb{N}$.

Durch Induktion über die Größe des normalisierten Programms.

Maximale LOOP-berechenbare Zahlen

Beweis (Fortsetzung) Zu zeigen: Für jedes normalisierte LOOP-Programm P gibt es eine Konstante k , sodass $f_P(n) < a(k, n)$ für alle $n \in \mathbb{N}$.

Durch Induktion über die Größe des normalisierten Programms.

Wir müssen folgende Fälle betrachten:

- ▶ Zuweisung $x_i := x_j + c$ mit $c \in \mathbb{Z}$ und $c \leq 1$
- ▶ Sequenz $P_1; P_2$
- ▶ LOOP-Schleife **LOOP** x_i **DO** Q **END**, wobei x_i nicht in Q vorkommt.

Maximale LOOP-berechenbare Zahlen

Beweis (Fortsetzung)

- ▶ Fall $x_i := x_j + c$ mit $c \in \mathbb{Z}$ und $c \leq 1$: Dann gilt $f_P(n) \leq 2n + 1$, da im maximalen Fall:
 - ▶ $\rho(x_k) = 0$ für $k \neq j$
 - ▶ $\rho(x_j) = n$
 - ▶ $c = 1$
 - ▶ $\rho'(x_i) = n + 1$
 - ▶ $\rho'(x_j) = n$
 - ▶ $\rho'(x_k) = 0$ für $k \neq j$ und $k \neq i$

Maximale LOOP-berechenbare Zahlen

Beweis (Fortsetzung)

- ▶ Fall $x_i := x_j + c$ mit $c \in \mathbb{Z}$ und $c \leq 1$: Dann gilt $f_P(n) \leq 2n + 1$, da im maximalen Fall:
 - ▶ $\rho(x_k) = 0$ für $k \neq j$
 - ▶ $\rho(x_j) = n$
 - ▶ $c = 1$
 - ▶ $\rho'(x_i) = n + 1$
 - ▶ $\rho'(x_j) = n$
 - ▶ $\rho'(x_k) = 0$ für $k \neq j$ und $k \neq i$

Mit $a(2, y) = 2y + 3$ (siehe Skript) folgt

$$f_P(n) \leq 2n + 1 < 2n + 3 = a(2, n)$$

D.h. die Aussage $f_P(n) < a(k, n)$ gilt mit $k = 2$.

Maximale LOOP-berechenbare Zahlen

Beweis (Fortsetzung)

- Fall $P_1; P_2$: Die Induktionshypothese liefert $f_{P_i}(n) < a(k_i, n)$ für $i \in \{1, 2\}$.

Maximale LOOP-berechenbare Zahlen

Beweis (Fortsetzung)

- Fall $P_1; P_2$: Die Induktionshypothese liefert $f_{P_i}(n) < a(k_i, n)$ für $i \in \{1, 2\}$.

Es gilt

$$f_P(n) \leq f_{P_2}(f_{P_1}(n))$$

Maximale LOOP-berechenbare Zahlen

Beweis (Fortsetzung)

- Fall $P_1; P_2$: Die Induktionshypothese liefert $f_{P_i}(n) < a(k_i, n)$ für $i \in \{1, 2\}$.

Es gilt

$$\begin{aligned} f_P(n) &\leq f_{P_2}(f_{P_1}(n)) \\ &< a(k_2, f_{P_1}(n)) \end{aligned} \quad (\text{IH})$$

Maximale LOOP-berechenbare Zahlen

Beweis (Fortsetzung)

- Fall $P_1; P_2$: Die Induktionshypothese liefert $f_{P_i}(n) < a(k_i, n)$ für $i \in \{1, 2\}$.

Es gilt

$$\begin{aligned} f_P(n) &\leq f_{P_2}(f_{P_1}(n)) \\ &< a(k_2, f_{P_1}(n)) && \text{(IH)} \\ &\leq a(k_2, a(k_1, n)) && \text{(IH, 1)} \end{aligned}$$

(1) Monotonie: $a(x, y) \leq a(x', y')$ falls $x \leq x'$ und $y \leq y'$

Maximale LOOP-berechenbare Zahlen

Beweis (Fortsetzung)

- Fall $P_1; P_2$: Die Induktionshypothese liefert $f_{P_i}(n) < a(k_i, n)$ für $i \in \{1, 2\}$.

Es gilt

$$\begin{aligned} f_P(n) &\leq f_{P_2}(f_{P_1}(n)) \\ &< a(k_2, f_{P_1}(n)) && \text{(IH)} \\ &\leq a(k_2, a(k_1, n)) && \text{(IH, 1)} \\ &\leq a(\max\{k_1, k_2\}, a(\max\{k_1, k_2\} + 1, n)) && \text{(1)} \end{aligned}$$

(1) Monotonie: $a(x, y) \leq a(x', y')$ falls $x \leq x'$ und $y \leq y'$

Maximale LOOP-berechenbare Zahlen

Beweis (Fortsetzung)

- Fall $P_1; P_2$: Die Induktionshypothese liefert $f_{P_i}(n) < a(k_i, n)$ für $i \in \{1, 2\}$.

Es gilt

$$\begin{aligned} f_P(n) &\leq f_{P_2}(f_{P_1}(n)) \\ &< a(k_2, f_{P_1}(n)) && \text{(IH)} \\ &\leq a(k_2, a(k_1, n)) && \text{(IH, 1)} \\ &\leq a(\max\{k_1, k_2\}, a(\max\{k_1, k_2\} + 1, n)) && \text{(1)} \\ &= a(\max\{k_1, k_2\} + 1, n + 1) && \text{(2)} \end{aligned}$$

(1) Monotonie: $a(x, y) \leq a(x', y')$ falls $x \leq x'$ und $y \leq y'$

(2) Definition von a : $a(x, y) = a(x - 1, a(x, y - 1))$ falls $x > 0$ und $y > 0$

Maximale LOOP-berechenbare Zahlen

Beweis (Fortsetzung)

- Fall $P_1; P_2$: Die Induktionshypothese liefert $f_{P_i}(n) < a(k_i, n)$ für $i \in \{1, 2\}$.

Es gilt

$$\begin{aligned} f_P(n) &\leq f_{P_2}(f_{P_1}(n)) \\ &< a(k_2, f_{P_1}(n)) && \text{(IH)} \\ &\leq a(k_2, a(k_1, n)) && \text{(IH, 1)} \\ &\leq a(\max\{k_1, k_2\}, a(\max\{k_1, k_2\} + 1, n)) && \text{(1)} \\ &= a(\max\{k_1, k_2\} + 1, n + 1) && \text{(2)} \\ &\leq a(\max\{k_1, k_2\} + 2, n) && \text{(3)} \end{aligned}$$

- (1) Monotonie: $a(x, y) \leq a(x', y')$ falls $x \leq x'$ und $y \leq y'$
(2) Definition von a : $a(x, y) = a(x - 1, a(x, y - 1))$ falls $x > 0$ und $y > 0$
(3) Monotonie: $a(x, y + 1) \leq a(x + 1, y)$

Maximale LOOP-berechenbare Zahlen

Beweis (Fortsetzung)

- Fall $P_1; P_2$: Die Induktionshypothese liefert $f_{P_i}(n) < a(k_i, n)$ für $i \in \{1, 2\}$.

Es gilt

$$\begin{aligned} f_P(n) &\leq f_{P_2}(f_{P_1}(n)) \\ &< a(k_2, f_{P_1}(n)) && \text{(IH)} \\ &\leq a(k_2, a(k_1, n)) && \text{(IH, 1)} \\ &\leq a(\max\{k_1, k_2\}, a(\max\{k_1, k_2\} + 1, n)) && \text{(1)} \\ &= a(\max\{k_1, k_2\} + 1, n + 1) && \text{(2)} \\ &\leq a(\max\{k_1, k_2\} + 2, n) && \text{(3)} \end{aligned}$$

(1) Monotonie: $a(x, y) \leq a(x', y')$ falls $x \leq x'$ und $y \leq y'$

(2) Definition von a : $a(x, y) = a(x - 1, a(x, y - 1))$ falls $x > 0$ und $y > 0$

(3) Monotonie: $a(x, y + 1) \leq a(x + 1, y)$

Daher gilt $f_P(n) < a(k, n)$ für $k = \max\{k_1, k_2\} + 2$.

Maximale LOOP-berechenbare Zahlen

Beweis (Fortsetzung)

- ▶ Fall **LOOP** x_i **DO** Q **END** und x_i kommt nicht in Q vor:
Siehe Skript.



Die Ackermannfunktion ist nicht LOOP-berechenbar

Theorem

Die Ackermannfunktion ist nicht LOOP-berechenbar.

Die Ackermannfunktion ist nicht LOOP-berechenbar

Theorem

Die Ackermannfunktion ist nicht LOOP-berechenbar.

Beweis Durch Widerspruch.

Die Ackermannfunktion ist nicht LOOP-berechenbar

Theorem

Die Ackermannfunktion ist nicht LOOP-berechenbar.

Beweis Durch Widerspruch.

Nehme an, dass a LOOP-berechenbar ist. Dann ist auch $g(x_1) = a(x_1, x_1)$ LOOP-berechenbar.

Die Ackermannfunktion ist nicht LOOP-berechenbar

Theorem

Die Ackermannfunktion ist nicht LOOP-berechenbar.

Beweis Durch Widerspruch.

Nehme an, dass a LOOP-berechenbar ist. Dann ist auch $g(x_1) = a(x_1, x_1)$ LOOP-berechenbar.

Sei P ein LOOP-Programm, dass g berechnet.

Die Ackermannfunktion ist nicht LOOP-berechenbar

Theorem

Die Ackermannfunktion ist nicht LOOP-berechenbar.

Beweis Durch Widerspruch.

Nehme an, dass a LOOP-berechenbar ist. Dann ist auch $g(x_1) = a(x_1, x_1)$ LOOP-berechenbar.

Sei P ein LOOP-Programm, dass g berechnet.

Dann gilt $g(x_1) \leq f_P(x_1)$, da $\rho'(x_0) = g(x_1)$ nach Ausführung von P .

Die Ackermannfunktion ist nicht LOOP-berechenbar

Theorem

Die Ackermannfunktion ist nicht LOOP-berechenbar.

Beweis Durch Widerspruch.

Nehme an, dass a LOOP-berechenbar ist. Dann ist auch $g(x_1) = a(x_1, x_1)$ LOOP-berechenbar.

Sei P ein LOOP-Programm, dass g berechnet.

Dann gilt $g(x_1) \leq f_P(x_1)$, da $\rho'(x_0) = g(x_1)$ nach Ausführung von P .

Es gibt eine Konstante k , sodass $f_P(n) < a(k, n)$.

Die Ackermannfunktion ist nicht LOOP-berechenbar

Theorem

Die Ackermannfunktion ist nicht LOOP-berechenbar.

Beweis Durch Widerspruch.

Nehme an, dass a LOOP-berechenbar ist. Dann ist auch $g(x_1) = a(x_1, x_1)$ LOOP-berechenbar.

Sei P ein LOOP-Programm, dass g berechnet.

Dann gilt $g(x_1) \leq f_P(x_1)$, da $\rho'(x_0) = g(x_1)$ nach Ausführung von P .

Es gibt eine Konstante k , sodass $f_P(n) < a(k, n)$.

Starte P mit $\rho = \{x_1 \mapsto k\}$.

Die Ackermannfunktion ist nicht LOOP-berechenbar

Theorem

Die Ackermannfunktion ist nicht LOOP-berechenbar.

Beweis Durch Widerspruch.

Nehme an, dass a LOOP-berechenbar ist. Dann ist auch $g(x_1) = a(x_1, x_1)$ LOOP-berechenbar.

Sei P ein LOOP-Programm, dass g berechnet.

Dann gilt $g(x_1) \leq f_P(x_1)$, da $\rho'(x_0) = g(x_1)$ nach Ausführung von P .

Es gibt eine Konstante k , sodass $f_P(n) < a(k, n)$.

Starte P mit $\rho = \{x_1 \mapsto k\}$.

Dann gilt $g(k) \leq f_P(k) < a(k, k) = g(k)$. Widerspruch.



Theorem

Es gibt totale WHILE-berechenbare (bzw. GOTO-berechenbare, turingberechenbare) Funktionen, die nicht LOOP-berechenbar sind.

10a

Primitiv und μ -rekursive Funktionen

Prof. Dr. Jasmin Blanchette

Lehr- und Forschungseinheit für
Theoretische Informatik und Theorembeweisen

Stand: 25. Juni 2024

Basierend auf Folien von PD Dr. David Sabel



Weitere Formalismen zur [Definition der Berechenbarkeit](#):

- ▶ primitiv rekursive Funktionen
- ▶ μ -rekursive Funktionen.

Weitere Formalismen zur [Definition der Berechenbarkeit](#):

- ▶ primitiv rekursive Funktionen
- ▶ μ -rekursive Funktionen.

Wir werden schließlich sehen:

- ▶ Primitiv rekursive Funktionen entsprechen genau den LOOP-berechenbaren Funktionen.
- ▶ μ -rekursive Funktionen entsprechen genau den turingberechenbaren (bzw. WHILE-, GOTO-berechenbaren) Funktionen.

Definition

Eine Funktion $f : \mathbb{N}^k \rightarrow \mathbb{N}$ ist **primitiv rekursiv**, wenn sie der folgenden Definition genügt:

- ▶ Jede **konstante Funktion** $f(x_1, \dots, x_k) = c \in \mathbb{N}$ ist primitiv rekursiv.
- ▶ Die **Projektionsfunktionen** $\pi_i^k(x_1, \dots, x_k) = x_i$ sind primitiv rekursiv.
- ▶ Die **Nachfolgerfunktion** $\text{succ}(x) = x + 1$ ist primitiv rekursiv.
- ▶ **Komposition/Einsetzung**: Wenn $g : \mathbb{N}^m \rightarrow \mathbb{N}$ und für $i = 1, \dots, m$: $h_i : \mathbb{N}^k \rightarrow \mathbb{N}$ primitiv rekursiv sind, dann ist auch f mit
 $f(x_1, \dots, x_k) = g(h_1(x_1, \dots, x_k), \dots, h_m(x_1, \dots, x_k))$ primitiv rekursiv.

(Fortsetzung folgt.)

Definition

- **Rekursion:** Wenn $g : \mathbb{N}^{k-1} \rightarrow \mathbb{N}$ und $h : \mathbb{N}^{k+1} \rightarrow \mathbb{N}$ primitiv rekursiv sind, dann ist auch f mit

$$f(x_1, \dots, x_k) = \begin{cases} g(x_2, \dots, x_k) & \text{falls } x_1 = 0 \\ h(f(x_1 - 1, x_2, \dots, x_k), x_1 - 1, x_2, \dots, x_k) & \text{sonst} \end{cases}$$

primitiv rekursiv.

Additionsfunktion

$add(x_1, x_2) = x_1 + x_2$ ist primitiv rekursiv:

$$add(x_1, x_2) = \begin{cases} x_2 & \text{falls } x_1 = 0 \\ succ(add(x_1 - 1, x_2)) & \text{sonst} \end{cases}$$

Grundgedanke: x_1 -mal 1 zu x_2 addieren.

Additionsfunktion

$add(x_1, x_2) = x_1 + x_2$ ist primitiv rekursiv:

$$add(x_1, x_2) = \begin{cases} x_2 & \text{falls } x_1 = 0 \\ succ(add(x_1 - 1, x_2)) & \text{sonst} \end{cases}$$

Grundgedanke: x_1 -mal 1 zu x_2 addieren.

Die verwendeten Funktionen g und h aus der Definition der primitiv rekursiven Funktionen sind hier

- ▶ $g = \pi_1^1$
- ▶ $h(x_1, x_2, x_3) = succ(\pi_1^3(x_1, x_2, x_3))$

Komponenten eines Tupels entfernen/vertauschen/vervielfachen

Wenn $g : \mathbb{N}^4 \rightarrow \mathbb{N}$ primitiv rekursiv ist, dann ist auch z.B. $f : \mathbb{N}^3 \rightarrow \mathbb{N}$ mit

$$f(n_1, n_2, n_3) = g(n_2, n_3, n_3, n_2)$$

denn

$$f(n_1, n_2, n_3) = g(\pi_2^3(n_1, n_2, n_3), \pi_3^3(n_1, n_2, n_3), \pi_3^3(n_1, n_2, n_3), \pi_2^3(n_1, n_2, n_3))$$

Multiplikationsfunktion

$mult(x_1, x_2) = x_1 \cdot x_2$ ist primitiv rekursiv:

$$mult(x_1, x_2) = \begin{cases} 0 & \text{falls } x_1 = 0 \\ add(mult(x_1 - 1, x_2), x_2) & \text{sonst} \end{cases}$$

Grundgedanke: x_1 -mal x_2 zu 0 addieren.

Rekursion durch das i -te Argument

Für $1 \leq i \leq k$ kann man

$$f(x_1, \dots, x_k) = \begin{cases} g(x_1, \dots, x_{i-1}, x_{i+1}, x_k), & \text{falls } x_i = 0 \\ h(f(x_1, \dots, x_{i-1}, x_i - 1, x_{i+1}, \dots, x_k), x_1, \dots, x_{i-1}, x_i - 1, x_{i+1}, \dots, x_k), & \text{sonst} \end{cases}$$

durch $f(x_1, \dots, x_k) = f'(x_i, x_1 \dots, x_{i-1}, x_{i+1}, \dots, x_k)$ darstellen, wobei

$$\begin{aligned} f'(y_1, \dots, y_k) &= \begin{cases} g(y_2, \dots, y_k) & \text{falls } y_1 = 0 \\ h'(f'(y_1 - 1, y_2, \dots, y_k), y_1 - 1, y_2, \dots, y_k) & \text{sonst} \end{cases} \\ h'(y_0, \dots, y_k) &= h(y_0, y_2, y_3, \dots, y_i, y_1, y_{i+1}, y_{i+2}, \dots, y_k) \end{aligned}$$

Angepasste Differenz

Im Allgemeinen ist $x_1 - x_2$ nicht primitiv rekursiv, weil der undefinierte Fall $x_1 < x_2$ nicht darstellbar ist.

Angepasste Differenz

Im Allgemeinen ist $x_1 - x_2$ nicht primitiv rekursiv, weil der undefinierte Fall $x_1 < x_2$ nicht darstellbar ist.

Hingegen ist die angepasste Differenz, die 0 liefert falls $x_1 < x_2$, primitiv rekursiv:

$$\text{sub}(x_1, x_2) = \begin{cases} x_1 & \text{falls } x_2 = 0 \\ \text{pred}(\text{sub}(x_1, x_2 - 1)) & \text{sonst} \end{cases}$$

wobei

$$\text{pred}(x_1) = \begin{cases} 0 & \text{falls } x_1 = 0 \\ x_1 - 1 & \text{sonst} \end{cases}$$

Nächstes Ziel

Wir wollen zeigen:

Die primitiv rekursive Funktionen sind genau die LOOP-berechenbaren Funktionen.

Nächstes Ziel

Wir wollen zeigen:

Die primitiv rekursive Funktionen sind genau die LOOP-berechenbaren Funktionen.

Dafür benötigen wir:

- ▶ eine Darstellung einer Variablenbelegung ρ als **eine einzige Zahl**, um sie der primitiv rekursiven Funktion als Argument zu übergeben (d.h. eindeutige Darstellung eines Tupels natürlicher Zahlen als eine einzige Zahl)
- ▶ Operationen zum Konvertieren in beide Richtungen.

Eine solches Verfahren nennt man **Gödelisierung** (nach Kurt Gödel).

Gödelisierung

Wir wollen Tupel von natürlichen Zahlen $(x_0, \dots, x_k)$ bijektiv in die natürlichen Zahlen abbilden mit **primitiv rekursiven** Funktionen.

Gödelisierung

Wir wollen Tupel von natürlichen Zahlen $(x_0, \dots, x_k)$ bijektiv in die natürlichen Zahlen abbilden mit **primitiv rekursiven** Funktionen.

Lösung:

$$c(x, y) = \binom{x + y + 1}{2} + x$$

die mithilfe des Binomialkoeffizients definiert ist.

Gödelisierung

Wir wollen Tupel von natürlichen Zahlen $(x_0, \dots, x_k)$ bijektiv in die natürlichen Zahlen abbilden mit **primitiv rekursiven** Funktionen.

Lösung:

$$c(x, y) = \binom{x + y + 1}{2} + x$$

die mithilfe des Binomialkoeffizients definiert ist.

Werte von $c(x, y)$ für $x, y \in \{0, \dots, 4\}$:

		x				
		0	1	2	3	4
y	0	0	2	5	9	14
	1	1	4	8	13	19
	2	3	7	12	18	25
	3	6	11	17	24	32
	4	10	16	23	31	40

Die Funktion c ist primitiv rekursiv, da

$$\binom{0}{2} = 0 \text{ und } \binom{n+1}{2} = \binom{n}{2} + n$$

Für $(k+1)$ -Tupel definieren wir

$$\langle x_0, x_1, \dots, x_k \rangle = c(x_0, c(x_1, \dots, c(x_k, 0) \dots))$$

$\langle \rangle$ ist primitiv rekursiv, da c primitiv rekursiv und Komposition primitiv rekursiv ist.

Die Komponenten eines Tupels können zurückgewonnen werden.

Seien *left* und *right* Funktionen mit

- ▶ $\text{left}(c(x, y)) = x$ und
- ▶ $\text{right}(c(x, y)) = y$.

Im Skript wird gezeigt:

- ▶ *left* und *right* existieren.
- ▶ *left* und *right* sind primitiv rekursiv.

Zugriff auf beliebige Komponenten ist auch möglich.

Programmiere $d_i(\langle x_0, \dots, x_k \rangle) = x_i$ durch

$$\begin{aligned}d_0(x) &= \text{left}(x) \\d_1(x) &= \text{left}(\text{right}(x)) \\d_i(x) &= \text{left}(\underbrace{\text{right}(\text{right}(\dots \text{right}(x) \dots))}_{i\text{-mal}})\end{aligned}$$

Damit sind auch die d_i -Funktionen primitiv rekursiv.

Von LOOP-Programm berechnete Funktion

- ▶ Sei P ein LOOP-Programm.
- ▶ Seien $x_0, x_1, \dots, x_n$ alle vom Programm P verwendeten Variablen.
- ▶ Die von P berechnete Funktion ist

$$g_P(\langle x_0, \dots, x_n \rangle) = \langle x'_0, \dots, x'_n \rangle$$

LOOP-Programme berechnen primitiv rekursive Funktionen

Lemma

Für jedes LOOP-Programm P ist die zugehörige Funktion g_P primitiv rekursiv.

LOOP-Programme berechnen primitiv rekursive Funktionen

Lemma

Für jedes LOOP-Programm P ist die zugehörige Funktion g_P primitiv rekursiv.

Beweis Durch Induktion über die Größe von jedem Teilprogramm Q .

LOOP-Programme berechnen primitiv rekursive Funktionen

Lemma

Für jedes LOOP-Programm P ist die zugehörige Funktion g_P primitiv rekursiv.

Beweis Durch Induktion über die Größe von jedem Teilprogramm Q .

► Fall Q ist Zuweisung $x_j := x_j \pm c$: Für g_Q muss gelten:

$$g_Q(\langle m_0, \dots, m_n \rangle) = \langle m_0, \dots, m_{i-1}, m_j \pm c, m_i, \dots, m_n \rangle$$

LOOP-Programme berechnen primitiv rekursive Funktionen

Lemma

Für jedes LOOP-Programm P ist die zugehörige Funktion g_P primitiv rekursiv.

Beweis Durch Induktion über die Größe von jedem Teilprogramm Q .

► Fall Q ist Zuweisung $x_j := x_j \pm c$: Für g_Q muss gelten:

$$g_Q(\langle m_0, \dots, m_n \rangle) = \langle m_0, \dots, m_{i-1}, m_j \pm c, m_i, \dots, m_n \rangle$$

Primitiv rekursive Implementierung:

$$g_Q(x) = \langle d_0(x), \dots, d_{i-1}(x), d_j(x) \pm c, d_{i+1}(x), \dots, d_n(x) \rangle$$

LOOP-Programme berechnen primitiv rekursive Funktionen

Beweis (Fortsetzung)

- ▶ Fall Q ist eine Sequenz $Q_1; Q_2$: Die Induktionshypothese liefert primitiv rekursive Funktionen g_{Q_1}, g_{Q_2} .

LOOP-Programme berechnen primitiv rekursive Funktionen

Beweis (Fortsetzung)

- Fall Q ist eine Sequenz $Q_1; Q_2$: Die Induktionshypothese liefert primitiv rekursive Funktionen g_{Q_1}, g_{Q_2} .

Die Funktion $g_Q(x) = g_{Q_2}(g_{Q_1}(x))$ ist primitiv rekursiv.

LOOP-Programme berechnen primitiv rekursive Funktionen

Beweis (Fortsetzung)

- ▶ Fall Q ist eine Sequenz $Q_1; Q_2$: Die Induktionshypothese liefert primitiv rekursive Funktionen g_{Q_1}, g_{Q_2} .

Die Funktion $g_Q(x) = g_{Q_2}(g_{Q_1}(x))$ ist primitiv rekursiv.

- ▶ Fall Q ist **LOOP** x_i **DO** P **END**: Die Induktionshypothese liefert die primitiv rekursive Funktion g_P .

LOOP-Programme berechnen primitiv rekursive Funktionen

Beweis (Fortsetzung)

- Fall Q ist eine Sequenz $Q_1; Q_2$: Die Induktionshypothese liefert primitiv rekursive Funktionen g_{Q_1}, g_{Q_2} .

Die Funktion $g_Q(x) = g_{Q_2}(g_{Q_1}(x))$ ist primitiv rekursiv.

- Fall Q ist **LOOP** x_i **DO** P **END**: Die Induktionshypothese liefert die primitiv rekursive Funktion g_P .

Die Konstruktion von g_Q wendet g_P x_i -mal an.

$$\begin{aligned} g_Q(x) &= \text{run}(d_i(x), x) \\ \text{run}(n, x) &= \begin{cases} x, & \text{falls } n = 0 \\ g_P(\text{run}(n-1, x)), & \text{sonst} \end{cases} \end{aligned}$$

run ist primitiv rekursiv. Damit ist auch g_Q primitiv rekursiv. □

LOOP-berechenbare Funktionen sind primitiv rekursiv

Satz

Jede LOOP-berechenbare Funktion ist primitiv rekursiv.

LOOP-berechenbare Funktionen sind primitiv rekursiv

Satz

Jede LOOP-berechenbare Funktion ist primitiv rekursiv.

Beweis Sei $f : \mathbb{N}^k \rightarrow \mathbb{N}$ eine LOOP-berechenbare Funktion.

LOOP-berechenbare Funktionen sind primitiv rekursiv

Satz

Jede LOOP-berechenbare Funktion ist primitiv rekursiv.

Beweis Sei $f : \mathbb{N}^k \rightarrow \mathbb{N}$ eine LOOP-berechenbare Funktion.

Daher gibt es ein LOOP-Programm P mit $\rho = \{x_1 \mapsto n_1, \dots, x_k \mapsto n_k\}$,
 $(\rho, P) \xrightarrow[\text{LOOP}]^* (\rho', \varepsilon)$ und $\rho'(x_0) = f(n_1, \dots, n_k)$.

LOOP-berechenbare Funktionen sind primitiv rekursiv

Satz

Jede LOOP-berechenbare Funktion ist primitiv rekursiv.

Beweis Sei $f : \mathbb{N}^k \rightarrow \mathbb{N}$ eine LOOP-berechenbare Funktion.

Daher gibt es ein LOOP-Programm P mit $\rho = \{x_1 \mapsto n_1, \dots, x_k \mapsto n_k\}$,
 $(\rho, P) \xrightarrow[\text{LOOP}]{*} (\rho', \varepsilon)$ und $\rho'(x_0) = f(n_1, \dots, n_k)$.

Es gilt $f(n_1, \dots, n_k) = d_0(g_P(\langle 0, n_1, \dots, n_k, 0, \dots, 0 \rangle))$.

LOOP-berechenbare Funktionen sind primitiv rekursiv

Satz

Jede LOOP-berechenbare Funktion ist primitiv rekursiv.

Beweis Sei $f : \mathbb{N}^k \rightarrow \mathbb{N}$ eine LOOP-berechenbare Funktion.

Daher gibt es ein LOOP-Programm P mit $\rho = \{x_1 \mapsto n_1, \dots, x_k \mapsto n_k\}$,
 $(\rho, P) \xrightarrow[\text{LOOP}]{*} (\rho', \varepsilon)$ und $\rho'(x_0) = f(n_1, \dots, n_k)$.

Es gilt $f(n_1, \dots, n_k) = d_0(g_P(\langle 0, n_1, \dots, n_k, 0, \dots, 0 \rangle))$.

Da $\langle \rangle$, d_0 und g_P primitiv rekursiv sind, ist auch f primitiv rekursiv. □

Primitiv rekursive Funktionen sind LOOP-berechenbar

Satz

Jede primitiv rekursive Funktion ist LOOP-berechenbar.

Primitiv rekursive Funktionen sind LOOP-berechenbar

Satz

Jede primitiv rekursive Funktion ist LOOP-berechenbar.

Beweis Durch Induktion über die Größe der primitiv rekursiven Funktion.

Primitiv rekursive Funktionen sind LOOP-berechenbar

Satz

Jede primitiv rekursive Funktion ist LOOP-berechenbar.

Beweis Durch Induktion über die Größe der primitiv rekursiven Funktion.

- Fall $f(x) = c$, $f = succ$ oder $f = \pi_n^k$: Dann gibt es auch ein LOOP-Programm dazu.

Primitiv rekursive Funktionen sind LOOP-berechenbar

Beweis (Fortsetzung)

- ▶ Fall $f(x_1, \dots, x_k) = h(g_1(x_1, \dots, x_k), \dots, g_n(x_1, \dots, x_k))$:
Die Induktionshypothese liefert LOOP-Programme P_h und $P_{g_1}, \dots, P_{g_n}$, die $h, g_1, \dots, g_n$ berechnen.

Primitiv rekursive Funktionen sind LOOP-berechenbar

Beweis (Fortsetzung)

- Fall $f(x_1, \dots, x_k) = h(g_1(x_1, \dots, x_k), \dots, g_n(x_1, \dots, x_k))$:

Die Induktionshypothese liefert LOOP-Programme P_h und $P_{g_1}, \dots, P_{g_n}$, die $h, g_1, \dots, g_n$ berechnen.

Konstruiere Programm für f nach dem Schema

$$y_1 := g_1(x_1, \dots, x_k);$$
$$\vdots$$
$$y_n := g_n(x_1, \dots, x_k);$$
$$x_0 := h(y_1, \dots, y_n)$$

Primitiv rekursive Funktionen sind LOOP-berechenbar

Beweis (Fortsetzung)

- Fall $f(x_1, \dots, x_k) = h(g_1(x_1, \dots, x_k), \dots, g_n(x_1, \dots, x_k))$:

Die Induktionshypothese liefert LOOP-Programme P_h und $P_{g_1}, \dots, P_{g_n}$, die $h, g_1, \dots, g_n$ berechnen.

Konstruiere Programm für f nach dem Schema

$$y_1 := g_1(x_1, \dots, x_k);$$
$$\vdots$$
$$y_n := g_n(x_1, \dots, x_k);$$
$$x_0 := h(y_1, \dots, y_n)$$

Genauer: $P_{g_1}, \dots, P_{g_n}, P_h$ abändern, sodass sie auf disjunkten Variablenmengen arbeiten, entsprechende Variableninhalte für $x_1, \dots, x_k$ verdoppeln.

Primitiv rekursive Funktionen sind LOOP-berechenbar

Beweis (Fortsetzung)

► Fall $f(x_1, \dots, x_k) = \begin{cases} g(x_2, \dots, x_k) & \text{falls } x_1 = 0 \\ h(f(x_1 - 1, x_2, \dots, x_k), x_1 - 1, x_2, \dots, x_k) & \text{sonst} \end{cases}$

wobei $g : \mathbb{N}^{k-1} \rightarrow \mathbb{N}$ und $h : \mathbb{N}^{k+1} \rightarrow \mathbb{N}$ primitiv rekursiv sind:

Primitiv rekursive Funktionen sind LOOP-berechenbar

Beweis (Fortsetzung)

► Fall $f(x_1, \dots, x_k) = \begin{cases} g(x_2, \dots, x_k) & \text{falls } x_1 = 0 \\ h(f(x_1 - 1, x_2, \dots, x_k), x_1 - 1, x_2, \dots, x_k) & \text{sonst} \end{cases}$

wobei $g : \mathbb{N}^{k-1} \rightarrow \mathbb{N}$ und $h : \mathbb{N}^{k+1} \rightarrow \mathbb{N}$ primitiv rekursiv sind:

Die Induktionshypothese liefert LOOP-Programme, die g, h berechnen.

Konstruiere LOOP-Programm für f nach dem Schema

$y := 0;$

$x_0 := g(x_2, \dots, x_k);$

LOOP x_1 **DO**

$x_0 := h(x_0, y, x_2, \dots, x_k);$

$y := y + 1$

END



Äquivalenz von LOOP-Berechenbarkeit und von primitive Rekursion

Theorem

Die primitiv rekursiven Funktionen sind genau die LOOP-berechenbaren Funktionen.

Definition

Sei $h : \mathbb{N}^{k+1} \rightarrow \mathbb{N}$ eine (partielle oder totale) Funktion.

Dann ist $\mu h : \mathbb{N}^k \rightarrow \mathbb{N}$ definiert als

$$(\mu h)(x_1, \dots, x_k) = \begin{cases} n & \text{falls } h(n, x_1, \dots, x_k) = 0 \text{ und f\"ur} \\ & \text{alle } m < n: h(m, x_1, \dots, x_k) \text{ ist definiert} \\ & \text{und } h(m, x_1, \dots, x_k) > 0 \\ \text{undefiniert} & \text{sonst} \end{cases}$$

Definition

Sei $h : \mathbb{N}^{k+1} \rightarrow \mathbb{N}$ eine (partielle oder totale) Funktion.

Dann ist $\mu h : \mathbb{N}^k \rightarrow \mathbb{N}$ definiert als

$$(\mu h)(x_1, \dots, x_k) = \begin{cases} n & \text{falls } h(n, x_1, \dots, x_k) = 0 \text{ und f\"ur} \\ & \text{alle } m < n: h(m, x_1, \dots, x_k) \text{ ist definiert} \\ & \text{und } h(m, x_1, \dots, x_k) > 0 \\ \text{undefiniert} & \text{sonst} \end{cases}$$

Der μ -Operator „sucht“ nach der ersten Nullstelle von h .

Wenn diese nicht existiert (entweder da h keine Nullstelle hat, oder da h undefiniert ist f\"ur Werte, die kleiner als die Nullstelle sind), dann ist auch μh undefiniert.

Beispiel für den μ -Operator

Sei $bus(x_1, x_2) = sub(x_2, x_1)$, wobei sub die angepasste Differenz berechnet.

$$(\mu bus)(5) = ?$$

Beispiel für den μ -Operator

Sei $bus(x_1, x_2) = sub(x_2, x_1)$, wobei sub die angepasste Differenz berechnet.

$(\mu bus)(5) = 5$ weil

$$bus(0, 5) = 5$$

$$bus(1, 5) = 4$$

$$bus(2, 5) = 3$$

$$bus(3, 5) = 2$$

$$bus(4, 5) = 1$$

$$bus(5, 5) = 0.$$

Definition

Eine Funktion $f : \mathbb{N}^k \rightarrow \mathbb{N}$ ist μ -rekursiv, wenn sie der folgenden Definition genügt:

- ▶ Jede konstante Funktion $f(x_1, \dots, x_k) = c \in \mathbb{N}$ ist μ -rekursiv.
- ▶ Die Projektionsfunktionen $\pi_i^k(x_1, \dots, x_k) = x_i$ sind μ -rekursiv.
- ▶ Die Nachfolgerfunktion $\text{succ}(x) = x + 1$ ist μ -rekursiv.
- ▶ Komposition/Einsetzung: Wenn $g : \mathbb{N}^m \rightarrow \mathbb{N}$ und für $i = 1, \dots, m$: $h_i : \mathbb{N}^k \rightarrow \mathbb{N}$ μ -rekursiv sind, dann ist auch f mit
$$f(x_1, \dots, x_k) = g(h_1(x_1, \dots, x_k), \dots, h_m(x_1, \dots, x_k))$$
 μ -rekursiv.

(Fortsetzung folgt.)

Definition

- **Rekursion:** Wenn $g : \mathbb{N}^{k-1} \rightarrow \mathbb{N}$ und $h : \mathbb{N}^{k+1} \rightarrow \mathbb{N}$ μ -rekursiv sind, dann ist

$$f(x_1, \dots, x_k) = \begin{cases} g(x_2, \dots, x_k) & \text{falls } x_1 = 0 \\ h(f(x_1 - 1, x_2, \dots, x_k), x_1 - 1, x_2, \dots, x_k) & \text{sonst} \end{cases}$$

auch μ -rekursiv.

- **μ -Operator:** Wenn $h : \mathbb{N}^{k+1} \rightarrow \mathbb{N}$ μ -rekursiv ist, dann ist auch $f = \mu h$ μ -rekursiv.

WHILE-berechenbare Funktionen sind μ -rekursiv

Satz

Jede WHILE-berechenbare Funktion ist μ -rekursiv.

WHILE-berechenbare Funktionen sind μ -rekursiv

Satz

Jede WHILE-berechenbare Funktion ist μ -rekursiv.

Beweis Analog zum Beweis, dass die LOOP-berechenbaren Funktionen primitiv rekursiv sind.

WHILE-berechenbare Funktionen sind μ -rekursiv

Satz

Jede WHILE-berechenbare Funktion ist μ -rekursiv.

Beweis Analog zum Beweis, dass die LOOP-berechenbaren Funktionen primitiv rekursiv sind.

Neuer Fall: P ist **WHILE** $x_i \neq 0$ **DO** Q **END**.

WHILE-berechenbare Funktionen sind μ -rekursiv

Satz

Jede WHILE-berechenbare Funktion ist μ -rekursiv.

Beweis Analog zum Beweis, dass die LOOP-berechenbaren Funktionen primitiv rekursiv sind.

Neuer Fall: P ist **WHILE** $x_i \neq 0$ **DO** Q **END**.

Die Induktionshypothese liefert μ -rekursive Funktion g_Q für Q . Konstruiere:

$$\begin{aligned} g_P(x) &= \text{run}(\mu(\text{run}_i)(x), x) \\ \text{run}_i(n, x) &= d_i(\text{run}(n, x)) \\ \text{run}(n, x) &= \begin{cases} x & \text{falls } n = 0 \\ g_Q(\text{run}(n-1, x)) & \text{sonst} \end{cases} \end{aligned}$$

WHILE-berechenbare Funktionen sind μ -rekursiv

Beweis (Fortsetzung)

$$g_P(x) = \text{run}((\mu \text{run}_i)(x), x)$$

$$\text{run}_i(n, x) = d_i(\text{run}(n, x))$$

$$\text{run}(n, x) = \begin{cases} x & \text{falls } n = 0 \\ g_Q(\text{run}(n-1, x)) & \text{sonst} \end{cases}$$

- ▶ $\text{run}(n, x)$ führt g_Q n -mal aus.
- ▶ $(\mu \text{run}_i)(x)$ berechnet, wie oft die Schleife minimal durchlaufen werden muss, bis x_i den Wert 0 hat.
- ▶ Divergiert die Schleife, so ist μrun_i undefiniert und g_P undefiniert. □

μ -rekursive Funktionen sind WHILE-berechenbar

Satz

Jede μ -rekursive Funktion ist WHILE-berechenbar.

μ -rekursive Funktionen sind WHILE-berechenbar

Satz

Jede μ -rekursive Funktion ist WHILE-berechenbar.

Beweis Analog zum Beweis, dass die primitiv rekursiven Funktionen LOOP-berechenbar sind.

μ -rekursive Funktionen sind WHILE-berechenbar

Satz

Jede μ -rekursive Funktion ist WHILE-berechenbar.

Beweis Analog zum Beweis, dass die primitiv rekursiven Funktionen LOOP-berechenbar sind.

Neuer Fall: $f = \mu h$ für eine μ -rekursive Funktion h .

μ -rekursive Funktionen sind WHILE-berechenbar

Satz

Jede μ -rekursive Funktion ist WHILE-berechenbar.

Beweis Analog zum Beweis, dass die primitiv rekursiven Funktionen LOOP-berechenbar sind.

Neuer Fall: $f = \mu h$ für eine μ -rekursive Funktion h .

Die Induktionshypothese liefert ein WHILE-Programm P_h , das h berechnet.
Konstruiere P_f nach dem Schema

```
x0 := 0;  
y := h(0, x1, ..., xn);  
WHILE y ≠ 0 DO  
  x0 := x0 + 1;  
  y := h(x0, ..., xn)  
END
```

μ -rekursive Funktionen sind WHILE-berechenbar

Beweis (Fortsetzung)

```
 $x_0 := 0;$   
 $y := h(0, x_1, \dots, x_n);$   
WHILE  $y \neq 0$  DO  
   $x_0 := x_0 + 1;$   
   $y := h(x_0, \dots, x_n)$   
END
```

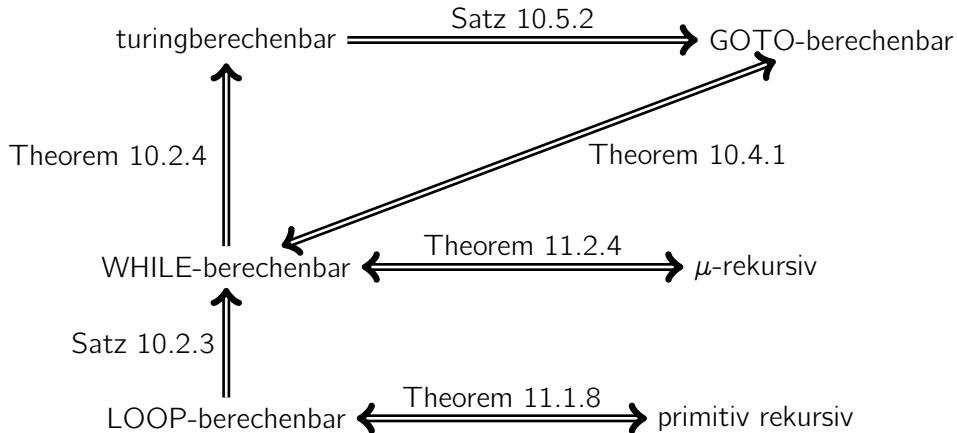
- ▶ Die **WHILE**-Schleife berechnet den minimalen x_0 , sodass $h(x_0, \dots, x_n) = 0$.
- ▶ Wenn dieser Wert nicht existiert, terminiert die Schleife nicht.
- ▶ Dies entspricht der Berechnung von μh . □

Äquivalenz von WHILE-Berechenbarkeit und von μ -Rekursion

Theorem

Die μ -rekursiven Funktionen entsprechen genau den WHILE-berechenbaren (und damit auch den GOTO- und turingberechenbaren) Funktionen.

Überblick über die Berechenbarkeitsformalismen



10b

Unentscheidbarkeit und das spezielle Halteproblem

Prof. Dr. Jasmin Blanchette

Lehr- und Forschungseinheit für
Theoretische Informatik und Theorembeweisen

Stand: 2. Juli 2024
Basierend auf Folien von PD Dr. David Sabel



Definition

Eine Sprache L ist **entscheidbar**, wenn es einen Algorithmus gibt, der bei Eingabe eines Wortes w in endlicher Zeit feststellt, ob $w \in L$ gilt oder nicht.

Definition

Eine Sprache L ist **entscheidbar**, wenn es einen Algorithmus gibt, der bei Eingabe eines Wortes w in endlicher Zeit feststellt, ob $w \in L$ gilt oder nicht.

Präziser:

Definition

Eine Sprache $L \subseteq \Sigma^*$ ist **entscheidbar**, wenn die **charakteristische Funktion** von L , $\chi_L : \Sigma^* \rightarrow \{0, 1\}$ mit

$$\chi_L(w) = \begin{cases} 1 & \text{falls } w \in L \\ 0 & \text{sonst} \end{cases}$$

berechenbar ist.

Der χ_L -berechnende Algorithmus terminiert in jedem Fall und liefert ein Ergebnis.

Definition

Eine Sprache L ist **semi-entscheidbar**, falls $\chi'_L : \Sigma^* \rightarrow \{0, 1\}$ mit

$$\chi'_L(w) = \begin{cases} 1 & \text{falls } w \in L \\ \text{undefiniert} & \text{sonst} \end{cases}$$

berechenbar ist.

Der χ'_L -berechnende Algorithmus terminiert nur, falls $w \in L$,
und läuft anderenfalls endlos.

Zusammenhang zwischen Entscheidbarkeit und Semi-Entscheidbarkeit

Satz

Eine Sprache L ist genau dann entscheidbar, wenn L und $\overline{L}$ jeweils semi-entscheidbar sind.

Zusammenhang zwischen Entscheidbarkeit und Semi-Entscheidbarkeit

Satz

Eine Sprache L ist genau dann entscheidbar, wenn L und $\bar{L}$ jeweils semi-entscheidbar sind.

Beweis

$\Rightarrow$ Konstruiere aus DTM, die χ_L berechnet, zwei DTMs, die χ'_L und $\chi'_{\bar{L}}$ berechnen.

Zusammenhang zwischen Entscheidbarkeit und Semi-Entscheidbarkeit

Satz

Eine Sprache L ist genau dann entscheidbar, wenn L und $\bar{L}$ jeweils semi-entscheidbar sind.

Beweis

⇒ Konstruiere aus DTM, die χ_L berechnet, zwei DTMs, die χ'_L und $\chi'_{\bar{L}}$ berechnen.

⇐ Gegeben seien DTMs M_L und $M_{\bar{L}}$, die χ'_L und $\chi'_{\bar{L}}$ berechnen.

Grundgedanke: Lasse M_L und $M_{\bar{L}}$ parallel laufen.

Konstruiere eine DTM, die χ_L berechnet:

1. Starte mit $i = 1$.
2. Simuliere i Schritte von M_L .
3. Wenn diese akzeptiert (mit Ausgabe 1), dann akzeptiere mit Ausgabe 1.
4. Ansonsten simuliere i Schritte von $M_{\bar{L}}$.
5. Wenn diese akzeptiert (mit Ausgabe 1), dann akzeptiere mit Ausgabe 0.
6. Ansonsten erhöhe i um 1 und starte von neuem.



Entscheidbarkeit von Komplement

Korollar

Wenn L entscheidbar ist, dann ist auch $\overline{L}$ entscheidbar.

Entscheidbarkeit von Komplement

Korollar

Wenn L entscheidbar ist, dann ist auch $\overline{L}$ entscheidbar.

Beweis Da L entscheidbar ist, sind L und $\overline{L}$ semi-entscheidbar.

Entscheidbarkeit von Komplement

Korollar

Wenn L entscheidbar ist, dann ist auch $\overline{L}$ entscheidbar.

Beweis Da L entscheidbar ist, sind L und $\overline{L}$ semi-entscheidbar.
Daher sind $\overline{\overline{L}} = L$ und $\overline{L}$ semi-entscheidbar.

Entscheidbarkeit von Komplement

Korollar

Wenn L entscheidbar ist, dann ist auch $\overline{L}$ entscheidbar.

Beweis Da L entscheidbar ist, sind L und $\overline{L}$ semi-entscheidbar.

Daher sind $\overline{\overline{L}} = L$ und $\overline{L}$ semi-entscheidbar.

Daher ist $\overline{L}$ entscheidbar.



Definition

Eine Sprache $L \subseteq \Sigma^*$ heißt **rekursiv aufzählbar**, falls

- ▶ $L = \emptyset$ oder
- ▶ es eine **totale berechenbare** Funktion $f : \mathbb{N} \rightarrow \Sigma^*$ gibt, sodass $L = \bigcup_{i \in \mathbb{N}} f(i)$.
Man sagt dann „ f zählt L auf“.

Beispiel für die rekursive Aufzählbarkeit

Lemma

Die Sprache Σ^* ist rekursiv aufzählbar.

Beispiel für die rekursive Aufzählbarkeit

Lemma

Die Sprache Σ^* ist rekursiv aufzählbar.

Beweis Wir zeigen nur den Fall $|\Sigma| = 1$. Sei $\Sigma = \{a\}$ und $n \in \mathbb{N}$.

Beispiel für die rekursive Aufzählbarkeit

Lemma

Die Sprache Σ^* ist rekursiv aufzählbar.

Beweis Wir zeigen nur den Fall $|\Sigma| = 1$. Sei $\Sigma = \{a\}$ und $n \in \mathbb{N}$.

Konstruiere eine 2-Band-Turingmaschine M , die n in Binärzahldarstellung auf das Eingabeband erhält.

Beispiel für die rekursive Aufzählbarkeit

Lemma

Die Sprache Σ^* ist rekursiv aufzählbar.

Beweis Wir zeigen nur den Fall $|\Sigma| = 1$. Sei $\Sigma = \{a\}$ und $n \in \mathbb{N}$.

Konstruiere eine 2-Band-Turingmaschine M , die n in Binärzahldarstellung auf das Eingabeband erhält.

M erzeugt auf dem anderen Band das leere Wort.

Beispiel für die rekursive Aufzählbarkeit

Lemma

Die Sprache Σ^* ist rekursiv aufzählbar.

Beweis Wir zeigen nur den Fall $|\Sigma| = 1$. Sei $\Sigma = \{a\}$ und $n \in \mathbb{N}$.

Konstruiere eine 2-Band-Turingmaschine M , die n in Binärzahldarstellung auf das Eingabeband erhält.

M erzeugt auf dem anderen Band das leere Wort.

Anschließend zählt M die Zahl auf dem Eingabeband um 1 herunter und fügt bei jedem Herunterzählen ein a hinzu.

Beispiel für die rekursive Aufzählbarkeit

Lemma

Die Sprache Σ^* ist rekursiv aufzählbar.

Beweis Wir zeigen nur den Fall $|\Sigma| = 1$. Sei $\Sigma = \{a\}$ und $n \in \mathbb{N}$.

Konstruiere eine 2-Band-Turingmaschine M , die n in Binärzahldarstellung auf das Eingabeband erhält.

M erzeugt auf dem anderen Band das leere Wort.

Anschließend zählt M die Zahl auf dem Eingabeband um 1 herunter und fügt bei jedem Herunterzählen ein a hinzu.

Dies wird wiederholt bis auf dem Eingabeband 0 steht.

Beispiel für die rekursive Aufzählbarkeit

Lemma

Die Sprache Σ^* ist rekursiv aufzählbar.

Beweis Wir zeigen nur den Fall $|\Sigma| = 1$. Sei $\Sigma = \{a\}$ und $n \in \mathbb{N}$.

Konstruiere eine 2-Band-Turingmaschine M , die n in Binärzahldarstellung auf das Eingabeband erhält.

M erzeugt auf dem anderen Band das leere Wort.

Anschließend zählt M die Zahl auf dem Eingabeband um 1 herunter und fügt bei jedem Herunterzählen ein a hinzu.

Dies wird wiederholt bis auf dem Eingabeband 0 steht.

Dann steht auf dem anderen Band $f(n) = \underbrace{a \cdots a}_n$.



Rekursiv aufzählbar = semi-entscheidbar

Satz

Eine Sprache ist genau dann rekursiv aufzählbar, wenn sie semi-entscheidbar ist.

Rekursiv aufzählbar = semi-entscheidbar

Satz

Eine Sprache ist genau dann rekursiv aufzählbar, wenn sie semi-entscheidbar ist.

Beweis

⇒ Sei f die totale berechenbare Funktion, die L aufzählt.

Dann berechnet der folgende Algorithmus $\chi'_L(w)$:

```
für  $i = 0, 1, 2, 3, \dots$  tue  
  wenn  $f(i) = w$  dann  
    stoppe und gib 1 aus  
Ende  
Ende
```

Rekursiv aufzählbar = semi-entscheidbar

Satz

Eine Sprache ist genau dann rekursiv aufzählbar, wenn sie semi-entscheidbar ist.

Beweis

⇒ Sei f die totale berechenbare Funktion, die L aufzählt.

Dann berechnet der folgende Algorithmus $\chi'_L(w)$:

```
für  $i = 0, 1, 2, 3, \dots$  tue  
  wenn  $f(i) = w$  dann  
    stoppe und gib 1 aus  
Ende  
Ende
```

⇐ Siehe Skript.

Zusammenfassung: Äquivalente Eigenschaften

Die folgenden Eigenschaften sind äquivalent:

Zusammenfassung: Äquivalente Eigenschaften

Die folgenden Eigenschaften sind äquivalent:

- ▶ L ist vom Typ 0.

Zusammenfassung: Äquivalente Eigenschaften

Die folgenden Eigenschaften sind äquivalent:

- ▶ L ist vom Typ 0.
- ▶ L ist semi-entscheidbar.

Zusammenfassung: Äquivalente Eigenschaften

Die folgenden Eigenschaften sind äquivalent:

- ▶ L ist vom Typ 0.
- ▶ L ist semi-entscheidbar.
- ▶ L ist rekursiv aufzählbar.

Zusammenfassung: Äquivalente Eigenschaften

Die folgenden Eigenschaften sind äquivalent:

- ▶ L ist vom Typ 0.
- ▶ L ist semi-entscheidbar.
- ▶ L ist rekursiv aufzählbar.
- ▶ Es gibt eine Turingmaschine M , die L akzeptiert (d.h. $L(M) = L$).

Zusammenfassung: Äquivalente Eigenschaften

Die folgenden Eigenschaften sind äquivalent:

- ▶ L ist vom Typ 0.
- ▶ L ist semi-entscheidbar.
- ▶ L ist rekursiv aufzählbar.
- ▶ Es gibt eine Turingmaschine M , die L akzeptiert (d.h. $L(M) = L$).
- ▶ χ'_L ist turing-, WHILE- bzw. GOTO-berechenbar.

Rekursiv aufzählbar $\neq$ abzählbar

Eine Sprache L ist **abzählbar**, wenn es eine **totale** Funktion $f : \mathbb{N} \rightarrow L$ gibt, sodass $\bigcup_{i \in \mathbb{N}} f(i) = L$.

Beachte: Abzählbarkeit fordert nicht, dass f **berechenbar** ist.

Gödelisierung von Turingmaschinen

Ziel: Stelle [Turingmaschinenbeschreibung](#) als natürliche Zahl in [Binärzahldarstellung](#) dar.

Andere Turingmaschinen können die Beschreibung als Eingabe erhalten, erzeugen usw.

Gödelisierung von Turingmaschinen

Sei $(Z, \Sigma, \Gamma, \delta, z_0, \square, E)$ eine DTM mit $\Sigma = \{0, 1\}$ und

- ▶ $\Gamma = \{a_0, \dots, a_k\}$, wobei $a_0 = \square$, $a_1 = \#$, $a_2 = 0$, $a_3 = 1$
- ▶ $Z = \{z_0, \dots, z_n\}$
- ▶ $E = \{z_n\}$

Gödelisierung von Turingmaschinen

Sei $(Z, \Sigma, \Gamma, \delta, z_0, \square, E)$ eine DTM mit $\Sigma = \{0, 1\}$ und

- ▶ $\Gamma = \{a_0, \dots, a_k\}$, wobei $a_0 = \square$, $a_1 = \#$, $a_2 = 0$, $a_3 = 1$
- ▶ $Z = \{z_0, \dots, z_n\}$
- ▶ $E = \{z_n\}$

Für $\delta(z_p, a_i) = (z_q, a_j, D)$ erzeuge Wort über Alphabet $\{0, 1, \#\}$:

$$\#\#bin(p)\#bin(i)\#bin(q)\#bin(j)\#bin(D_m)$$

mit $D_m = 0$, falls $D = L$, $D_m = 1$, falls $D = R$, $D_m = 2$, falls $D = N$

Gödelisierung von Turingmaschinen

Sei $(Z, \Sigma, \Gamma, \delta, z_0, \square, E)$ eine DTM mit $\Sigma = \{0, 1\}$ und

- ▶ $\Gamma = \{a_0, \dots, a_k\}$, wobei $a_0 = \square$, $a_1 = \#$, $a_2 = 0$, $a_3 = 1$
- ▶ $Z = \{z_0, \dots, z_n\}$
- ▶ $E = \{z_n\}$

Für $\delta(z_p, a_i) = (z_q, a_j, D)$ erzeuge Wort über Alphabet $\{0, 1, \#\}$:

$$\#\#bin(p)\#bin(i)\#bin(q)\#bin(j)\#bin(D_m)$$

mit $D_m = 0$, falls $D = L$, $D_m = 1$, falls $D = R$, $D_m = 2$, falls $D = N$

Für δ schreibe alle δ -Wörter hintereinander.

Gödelisierung von Turingmaschinen

Sei $(Z, \Sigma, \Gamma, \delta, z_0, \square, E)$ eine DTM mit $\Sigma = \{0, 1\}$ und

- ▶ $\Gamma = \{a_0, \dots, a_k\}$, wobei $a_0 = \square$, $a_1 = \#$, $a_2 = 0$, $a_3 = 1$
- ▶ $Z = \{z_0, \dots, z_n\}$
- ▶ $E = \{z_n\}$

Für $\delta(z_p, a_i) = (z_q, a_j, D)$ erzeuge Wort über Alphabet $\{0, 1, \#\}$:

$$\#\#bin(p)\#bin(i)\#bin(q)\#bin(j)\#bin(D_m)$$

mit $D_m = 0$, falls $D = L$, $D_m = 1$, falls $D = R$, $D_m = 2$, falls $D = N$

Für δ schreibe alle δ -Wörter hintereinander.

Schließlich: Kodiere Alphabet $\{0, 1, \#\}$ durch $\{0 \mapsto 00, 1 \mapsto 01, \# \mapsto 11\}$.

Wende dies auf die Kodierung von δ an.

Gödelisierung von Turingmaschinen

Sei $(Z, \Sigma, \Gamma, \delta, z_0, \square, E)$ eine DTM mit $\Sigma = \{0, 1\}$ und

- ▶ $\Gamma = \{a_0, \dots, a_k\}$, wobei $a_0 = \square$, $a_1 = \#$, $a_2 = 0$, $a_3 = 1$
- ▶ $Z = \{z_0, \dots, z_n\}$
- ▶ $E = \{z_n\}$

Für $\delta(z_p, a_i) = (z_q, a_j, D)$ erzeuge Wort über Alphabet $\{0, 1, \#\}$:

$$\#\#bin(p)\#bin(i)\#bin(q)\#bin(j)\#bin(D_m)$$

mit $D_m = 0$, falls $D = L$, $D_m = 1$, falls $D = R$, $D_m = 2$, falls $D = N$

Für δ schreibe alle δ -Wörter hintereinander.

Schließlich: Kodiere Alphabet $\{0, 1, \#\}$ durch $\{0 \mapsto 00, 1 \mapsto 01, \# \mapsto 11\}$.

Wende dies auf die Kodierung von δ an.

Wir bezeichnen mit w_M die so kodierte DTM M .

Gödelisierung von Turingmaschinen

- ▶ Nicht jedes Wort über $\{0, 1\}$ entspricht der Kodierung einer DTM.
- ▶ Sei $\hat{M}$ eine beliebige aber feste DTM.
- ▶ Definiere für jedes $w \in \{0, 1\}^*$ die zugehörige DTM M_w :

$$M_w := \begin{cases} M & \text{falls } w = w_M \\ \hat{M} & \text{sonst} \end{cases}$$

Definition

Das **spezielle Halteproblem** ist die Sprache

$$K := \{w \in \{0, 1\}^* \mid M_w \text{ hält für Eingabe } w\}$$

Unentscheidbarkeit des speziellen Halteproblems

Satz

Das spezielle Halteproblem ist nicht entscheidbar (und damit **unentscheidbar**).

Unentscheidbarkeit des speziellen Halteproblems

Satz

Das spezielle Halteproblem ist nicht entscheidbar (und damit **unentscheidbar**).

Beweis Durch Widerspruch. Wir nehmen an, dass K entscheidbar ist.

Unentscheidbarkeit des speziellen Halteproblems

Satz

Das spezielle Halteproblem ist nicht entscheidbar (und damit **unentscheidbar**).

Beweis Durch Widerspruch. Wir nehmen an, dass K entscheidbar ist.

Dann ist χ_K berechenbar, und es gibt eine DTM M , die χ_K berechnet.

Unentscheidbarkeit des speziellen Halteproblems

Satz

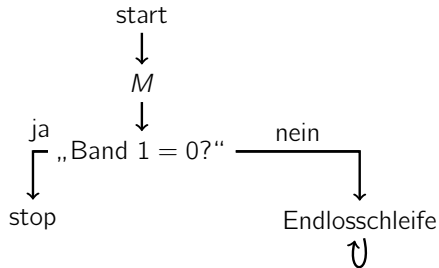
Das spezielle Halteproblem ist nicht entscheidbar (und damit **unentscheidbar**).

Beweis Durch Widerspruch. Wir nehmen an, dass K entscheidbar ist.

Dann ist χ_K berechenbar, und es gibt eine DTM M , die χ_K berechnet.

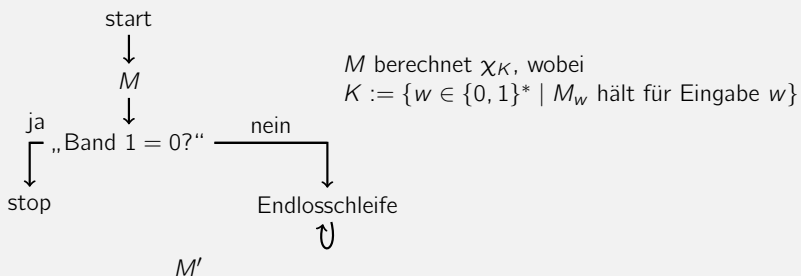
Konstruiere M' wie folgt:

1. M' lässt M ablaufen.
2. Wenn M mit 0 auf dem Band endet, dann akzeptiert M' .
3. Wenn M mit 1 auf dem Band endet, dann läuft M' in eine Endlosschleife.



Unentscheidbarkeit des speziellen Halteproblems

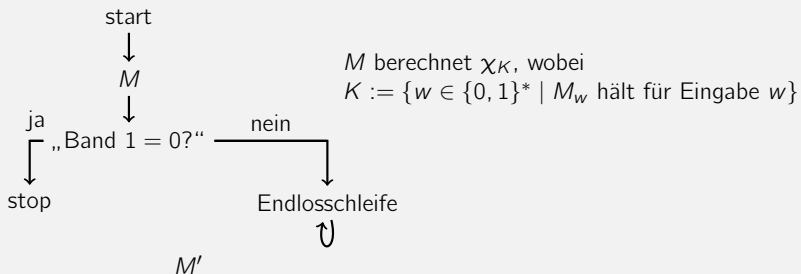
Zur Erinnerung:



Beweis (Fortsetzung) Betrachte nun M' auf der Eingabe $w_{M'}$:

Unentscheidbarkeit des speziellen Halteproblems

Zur Erinnerung:

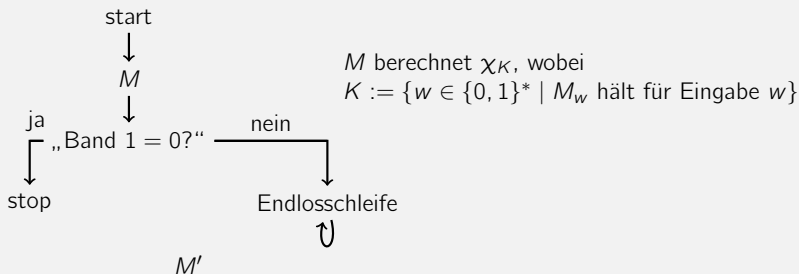


Beweis (Fortsetzung) Betrachte nun M' auf der Eingabe $w_{M'}$:

M' hält für Eingabe $w_{M'}$

Unentscheidbarkeit des speziellen Halteproblems

Zur Erinnerung:

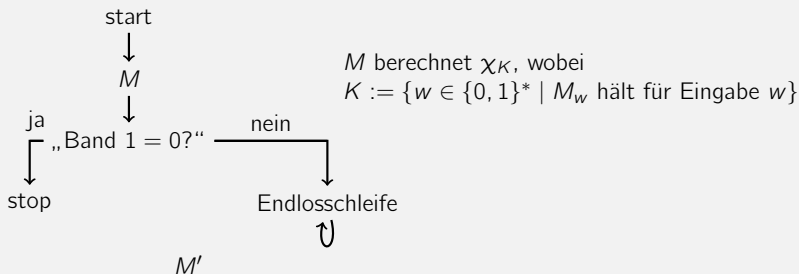


Beweis (Fortsetzung) Betrachte nun M' auf der Eingabe $w_{M'}$:

M' hält für Eingabe $w_{M'}$
g.d.w. M angesetzt auf $w_{M'}$ gibt 0 aus

Unentscheidbarkeit des speziellen Halteproblems

Zur Erinnerung:



Beweis (Fortsetzung) Betrachte nun M' auf der Eingabe $w_{M'}$:

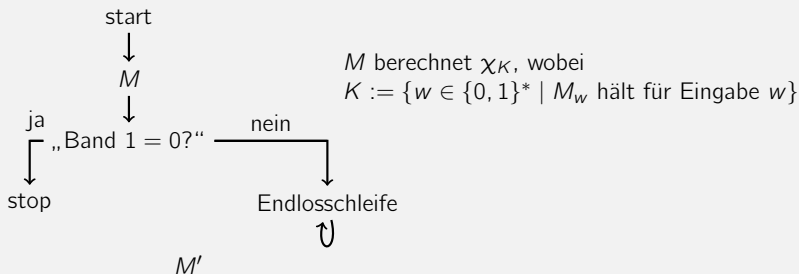
M' hält für Eingabe $w_{M'}$

g.d.w. M angesetzt auf $w_{M'}$ gibt 0 aus

g.d.w. $\chi_K(w_{M'}) = 0$

Unentscheidbarkeit des speziellen Halteproblems

Zur Erinnerung:



Beweis (Fortsetzung) Betrachte nun M' auf der Eingabe $w_{M'}$:

M' hält für Eingabe $w_{M'}$

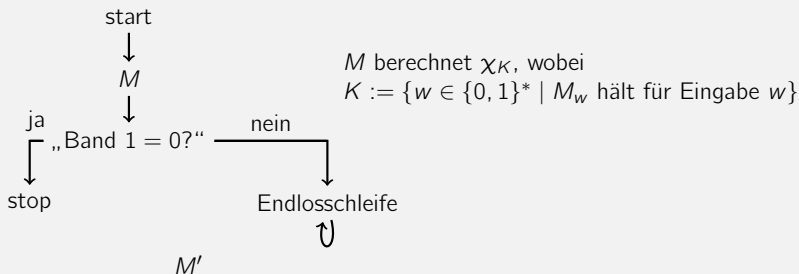
g.d.w. M angesetzt auf $w_{M'}$ gibt 0 aus

g.d.w. $\chi_K(w_{M'}) = 0$

g.d.w. $w_{M'} \notin K$

Unentscheidbarkeit des speziellen Halteproblems

Zur Erinnerung:

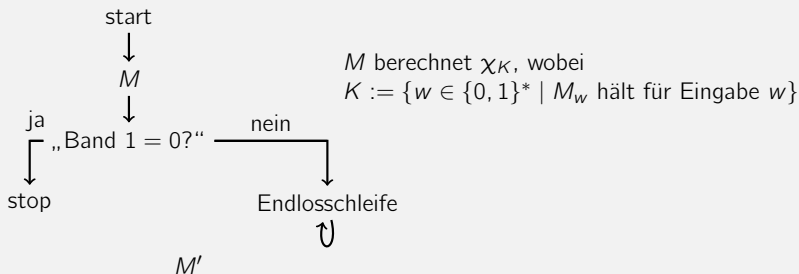


Beweis (Fortsetzung) Betrachte nun M' auf der Eingabe $w_{M'}$:

- M' hält für Eingabe $w_{M'}$
- g.d.w. M angesetzt auf $w_{M'}$ gibt 0 aus
- g.d.w. $\chi_K(w_{M'}) = 0$
- g.d.w. $w_{M'} \notin K$
- g.d.w. M' hält nicht für Eingabe $w_{M'}$

Unentscheidbarkeit des speziellen Halteproblems

Zur Erinnerung:



Beweis (Fortsetzung) Betrachte nun M' auf der Eingabe $w_{M'}$:

M' hält für Eingabe $w_{M'}$

g.d.w. M angesetzt auf $w_{M'}$ gibt 0 aus

g.d.w. $\chi_K(w_{M'}) = 0$

g.d.w. $w_{M'} \notin K$

g.d.w. M' hält nicht für Eingabe $w_{M'}$

Unentscheidbarkeit des speziellen Halteproblems

Beweis (Fortsetzung) Es gilt also:

M' hält für Eingabe $w_{M'}$ g.d.w. M' hält nicht für Eingabe $w_{M'}$.

Widerspruch. K ist daher nicht entscheidbar, sondern unentscheidbar. □

Unentscheidbarkeit des speziellen Halteproblems

Beweis (Fortsetzung) Es gilt also:

M' hält für Eingabe $w_{M'}$ g.d.w. M' hält nicht für Eingabe $w_{M'}$.

Widerspruch. K ist daher nicht entscheidbar, sondern unentscheidbar. □

Der Beweis, dass K unentscheidbar ist, verwendet ein [Diagonalisierungsargument](#), das wir gleich genauer erläutern.

Unentscheidbarkeit des speziellen Halteproblems

Alternativer Beweis

	w_1	w_2	w_3	...
M_{w_1}	ja	nein	ja	...
M_{w_2}	nein	nein	ja	...
M_{w_3}	ja	nein	ja	...
...	...	...	...	...

Unentscheidbarkeit des speziellen Halteproblems

Alternativer Beweis

	w_1	w_2	w_3	...
M_{w_1}	ja	nein	ja	...
M_{w_2}	nein	nein	ja	...
M_{w_3}	ja	nein	ja	...
...	...	...	...	...
M_{w_D}				

Eintrag in Zeile i und Spalte j : ja, wenn M_{w_i} bei Eingabe w_j akzeptiert, nein sonst.

Unentscheidbarkeit des speziellen Halteproblems

Alternativer Beweis

	w_1	w_2	w_3	...
M_{w_1}	ja	nein	ja	...
M_{w_2}	nein	nein	ja	...
M_{w_3}	ja	nein	ja	...
...	...	...	...	...
M_{w_D}	nein			

Eintrag in Zeile i und Spalte j : ja, wenn M_{w_i} bei Eingabe w_j akzeptiert, nein sonst.

Sei $D = \overline{K} = \{w_i \mid M_{w_i} \text{ hält nicht für Eingabe } w_i\}$.

Unentscheidbarkeit des speziellen Halteproblems

Alternativer Beweis

	w_1	w_2	w_3	$\dots$
M_{w_1}	ja	nein	ja	$\dots$
M_{w_2}	nein	nein	ja	$\dots$
M_{w_3}	ja	nein	ja	$\dots$
$\dots$	$\dots$	$\dots$	$\dots$	$\dots$
M_{w_D}	nein	ja		

Eintrag in Zeile i und Spalte j : ja, wenn M_{w_i} bei Eingabe w_j akzeptiert, nein sonst.

Sei $D = \overline{K} = \{w_i \mid M_{w_i} \text{ hält nicht für Eingabe } w_i\}$.

Wir nehmen an, dass w_D die Beschreibung von DTM M_{w_D} ist, die χ_D berechnet.

Unentscheidbarkeit des speziellen Halteproblems

Alternativer Beweis

	w_1	w_2	w_3	...
M_{w_1}	ja	nein	ja	...
M_{w_2}	nein	nein	ja	...
M_{w_3}	ja	nein	ja	...
...	...	...	...	...
M_{w_D}	nein	ja	nein	

Eintrag in Zeile i und Spalte j : ja, wenn M_{w_i} bei Eingabe w_j akzeptiert, nein sonst.

Sei $D = \overline{K} = \{w_i \mid M_{w_i} \text{ hält nicht für Eingabe } w_i\}$.

Wir nehmen an, dass w_D die Beschreibung von DTM M_{w_D} ist, die χ_D berechnet.

Der Eintrag in Zeile M_{w_D} und Spalte w_D ist ja g.d.w. der Eintrag nein ist.

Widerspruch. $D = \overline{K}$ und K sind daher nicht entscheidbar. □

Unentscheidbarkeit des speziellen Halteproblems

Alternativer Beweis

	w_1	w_2	w_3	$\dots$	w_D
M_{w_1}	ja	nein	ja	$\dots$	$\dots$
M_{w_2}	nein	nein	ja	$\dots$	$\dots$
M_{w_3}	ja	nein	ja	$\dots$	$\dots$
$\dots$	$\dots$	$\dots$	$\dots$	$\dots$	$\dots$
M_{w_D}	nein	ja	nein	$\dots$	

Eintrag in Zeile i und Spalte j : ja, wenn M_{w_i} bei Eingabe w_j akzeptiert, nein sonst.

Sei $D = \overline{K} = \{w_i \mid M_{w_i} \text{ hält nicht für Eingabe } w_i\}$.

Wir nehmen an, dass w_D die Beschreibung von DTM M_{w_D} ist, die χ_D berechnet.

Der Eintrag in Zeile M_{w_D} und Spalte w_D ist ja g.d.w. der Eintrag nein ist.

Widerspruch. $D = \overline{K}$ und K sind daher nicht entscheidbar. □

Unentscheidbarkeit des speziellen Halteproblems

Alternativer Beweis

	w_1	w_2	w_3	$\dots$	w_D
M_{w_1}	ja	nein	ja	$\dots$	$\dots$
M_{w_2}	nein	nein	ja	$\dots$	$\dots$
M_{w_3}	ja	nein	ja	$\dots$	$\dots$
$\dots$	$\dots$	$\dots$	$\dots$	$\dots$	$\dots$
M_{w_D}	nein	ja	nein	$\dots$	??

Eintrag in Zeile i und Spalte j : ja, wenn M_{w_i} bei Eingabe w_j akzeptiert, nein sonst.

Sei $D = \overline{K} = \{w_i \mid M_{w_i} \text{ hält nicht für Eingabe } w_i\}$.

Wir nehmen an, dass w_D die Beschreibung von DTM M_{w_D} ist, die χ_D berechnet.

Der Eintrag in Zeile M_{w_D} und Spalte w_D ist ja g.d.w. der Eintrag nein ist.

Widerspruch. $D = \overline{K}$ und K sind daher nicht entscheidbar. □

10c

Reduktion und der Satz von Rice

Prof. Dr. Jasmin Blanchette

Lehr- und Forschungseinheit für
Theoretische Informatik und Theorembeweisen

Stand: 2. Juli 2024
Basierend auf Folien von PD Dr. David Sabel



- ▶ **Reduktion** ist ein Hilfsmittel, um Unentscheidbarkeit nachzuweisen.
- ▶ Statt Unentscheidbarkeit von Sprache L von Grund auf neu zu beweisen, zeige:
Wenn man L entscheiden könnte, dann könnte man auch K (d.h. das spezielle Halteproblem) entscheiden.
- ▶ Da K bereits als unentscheidbar gezeigt wurde, folgt L ist unentscheidbar.
- ▶ Statt K können wir eine beliebige Sprache nehmen, die bereits als unentscheidbar bewiesen ist.

Definition von Reduktion

Definition

Seien $L_1 \subseteq \Sigma_1^*$ und $L_2 \subseteq \Sigma_2^*$ Sprachen.

Dann sagen wir L_1 ist auf L_2 **reduzierbar** (geschrieben $L_1 \leq L_2$), falls es eine **totale berechenbare** Funktion $f : \Sigma_1^* \rightarrow \Sigma_2^*$ gibt, sodass für alle $w \in \Sigma_1^*$ gilt: $w \in L_1$ g.d.w. $f(w) \in L_2$.

Die Funktion f nennt man **Reduktion**.

Definition von Reduktion

Definition

Seien $L_1 \subseteq \Sigma_1^*$ und $L_2 \subseteq \Sigma_2^*$ Sprachen.

Dann sagen wir L_1 ist auf L_2 **reduzierbar** (geschrieben $L_1 \leq L_2$), falls es eine **totale berechenbare** Funktion $f : \Sigma_1^* \rightarrow \Sigma_2^*$ gibt, sodass für alle $w \in \Sigma_1^*$ gilt: $w \in L_1$ g.d.w. $f(w) \in L_2$.

Die Funktion f nennt man **Reduktion**.

Eselsbrücke:

$$\underbrace{L_1}_{\text{„kleines“ Problem}} \leq \underbrace{L_2}_{\text{„großes“ Problem}}$$

Definition von Reduktion

Definition

Seien $L_1 \subseteq \Sigma_1^*$ und $L_2 \subseteq \Sigma_2^*$ Sprachen.

Dann sagen wir L_1 ist auf L_2 **reduzierbar** (geschrieben $L_1 \leq L_2$), falls es eine **totale berechenbare** Funktion $f : \Sigma_1^* \rightarrow \Sigma_2^*$ gibt, sodass für alle $w \in \Sigma_1^*$ gilt: $w \in L_1$ g.d.w. $f(w) \in L_2$.

Die Funktion f nennt man **Reduktion**.

Eselsbrücke:

$L_1 \leq L_2$
„kleines“ Problem „großes“ Problem

► $\leq$ sagt die Wahrheit.

► „Reduktion“ täuscht.

Man reduziert das „kleine“ Problem auf das „große“.

Nachweis der (Semi-)Entscheidbarkeit

Satz

Wenn $L_1 \leq L_2$ und L_2 entscheidbar ist, dann ist auch L_1 entscheidbar.

Nachweis der (Semi-)Entscheidbarkeit

Satz

Wenn $L_1 \leq L_2$ und L_2 entscheidbar ist, dann ist auch L_1 entscheidbar.

Beweis Sei f die $L_1 \leq L_2$ bezeugende Funktion. Da L_2 entscheidbar ist, ist χ_{L_2} berechenbar. Es gilt

$$\chi_{L_1}(w) = 1 \text{ g.d.w. } w \in L_1 \text{ g.d.w. } f(w) \in L_2 \text{ g.d.w. } \chi_{L_2}(f(w)) = 1$$

Damit ist $\chi_{L_1}(w) = \chi_{L_2}(f(w))$ berechenbar.



Nachweis der (Semi-)Entscheidbarkeit

Satz

Wenn $L_1 \leq L_2$ und L_2 entscheidbar ist, dann ist auch L_1 entscheidbar.

Beweis Sei f die $L_1 \leq L_2$ bezeugende Funktion. Da L_2 entscheidbar ist, ist χ_{L_2} berechenbar. Es gilt

$$\chi_{L_1}(w) = 1 \text{ g.d.w. } w \in L_1 \text{ g.d.w. } f(w) \in L_2 \text{ g.d.w. } \chi_{L_2}(f(w)) = 1$$

Damit ist $\chi_{L_1}(w) = \chi_{L_2}(f(w))$ berechenbar. □

Satz

Wenn $L_1 \leq L_2$ und L_2 semi-entscheidbar ist, dann ist auch L_1 semi-entscheidbar.

Nachweis der (Semi-)Entscheidbarkeit

Satz

Wenn $L_1 \leq L_2$ und L_2 entscheidbar ist, dann ist auch L_1 entscheidbar.

Beweis Sei f die $L_1 \leq L_2$ bezeugende Funktion. Da L_2 entscheidbar ist, ist χ_{L_2} berechenbar. Es gilt

$$\chi_{L_1}(w) = 1 \text{ g.d.w. } w \in L_1 \text{ g.d.w. } f(w) \in L_2 \text{ g.d.w. } \chi_{L_2}(f(w)) = 1$$

Damit ist $\chi_{L_1}(w) = \chi_{L_2}(f(w))$ berechenbar. □

Satz

Wenn $L_1 \leq L_2$ und L_2 semi-entscheidbar ist, dann ist auch L_1 semi-entscheidbar.

Beweis Analog. □

Nachweis der Unentscheidbarkeit

Mit Kontraposition folgt:

Lemma

Wenn $L_1 \leq L_2$ und L_1 unentscheidbar ist, dann ist auch L_2 unentscheidbar.

Das ist die Richtung, die wir meistens brauchen:

1. L_1 sei eine bekannt unentscheidbare Sprache (z.B. K).
2. Reduziere L_1 auf L_2 durch Angabe einer totalen berechenbaren Funktion f mit $w \in L_1$ g.d.w. $f(w) \in L_2$.
3. Damit folgt, dass L_2 unentscheidbar ist.

Definition

Das (allgemeine) Halteproblem ist die Sprache
 $H := \{w\#x \mid \text{TM } M_w \text{ hält für Eingabe } x\}.$

Unentscheidbarkeit des Halteproblems

Satz

Das allgemeine Halteproblem ist unentscheidbar.

Unentscheidbarkeit des Halteproblems

Satz

Das allgemeine Halteproblem ist unentscheidbar.

Beweis Wir reduzieren das **spezielle Halteproblem** auf das **allgemeine Halteproblem**, und zeigen daher $K \leq H$. Sei $f(w) = w\#w$.

Unentscheidbarkeit des Halteproblems

Satz

Das allgemeine Halteproblem ist unentscheidbar.

Beweis Wir reduzieren das spezielle Halteproblem auf das allgemeine Halteproblem, und zeigen daher $K \leq H$. Sei $f(w) = w\#w$.

Dann gilt

$$w \in K$$

Unentscheidbarkeit des Halteproblems

Satz

Das allgemeine Halteproblem ist unentscheidbar.

Beweis Wir reduzieren das spezielle Halteproblem auf das allgemeine Halteproblem, und zeigen daher $K \leq H$. Sei $f(w) = w\#w$.

Dann gilt

$w \in K$
g.d.w. M_w hält für Eingabe w

Unentscheidbarkeit des Halteproblems

Satz

Das allgemeine Halteproblem ist unentscheidbar.

Beweis Wir reduzieren das spezielle Halteproblem auf das allgemeine Halteproblem, und zeigen daher $K \leq H$. Sei $f(w) = w\#w$.

Dann gilt

$w \in K$
g.d.w. M_w hält für Eingabe w

$f(w) \in H$

Unentscheidbarkeit des Halteproblems

Satz

Das allgemeine Halteproblem ist unentscheidbar.

Beweis Wir reduzieren das **spezielle Halteproblem** auf das **allgemeine Halteproblem**, und zeigen daher $K \leq H$. Sei $f(w) = w\#w$.

Dann gilt

$$w \in K$$

g.d.w. M_w hält für Eingabe w

$$w\#w \in H$$

g.d.w. $f(w) \in H$

Unentscheidbarkeit des Halteproblems

Satz

Das allgemeine Halteproblem ist unentscheidbar.

Beweis Wir reduzieren das **spezielle Halteproblem** auf das **allgemeine Halteproblem**, und zeigen daher $K \leq H$. Sei $f(w) = w\#w$.

Dann gilt

$$w \in K$$

g.d.w. M_w hält für Eingabe w

g.d.w. $w\#w \in H$

g.d.w. $f(w) \in H$

Unentscheidbarkeit des Halteproblems

Satz

Das allgemeine Halteproblem ist unentscheidbar.

Beweis Wir reduzieren das **spezielle Halteproblem** auf das **allgemeine Halteproblem**, und zeigen daher $K \leq H$. Sei $f(w) = w\#w$.

Dann gilt

- $w \in K$
- g.d.w. M_w hält für Eingabe w
- g.d.w. $w\#w \in H$
- g.d.w. $f(w) \in H$

f kann durch eine TM berechnet werden. Daher gilt $K \leq H$. Da K unentscheidbar ist, ist H unentscheidbar. □

Halteproblem auf leerem Band

Definition

Das **Halteproblem auf leerem Band** ist die Sprache
 $H_0 := \{w \mid M_w \text{ hält für die leere Eingabe}\}.$

Unentscheidbarkeit des Halteproblems auf leerem Band

Satz

Das Halteproblem auf leerem Band ist unentscheidbar.

Unentscheidbarkeit des Halteproblems auf leerem Band

Satz

Das Halteproblem auf leerem Band ist unentscheidbar.

Beweis Wir reduzieren H auf H_0 : Sei $f(w_M \# x) = w_{M_x}$, wobei die DTM M_x erst x auf das Band schreibt, sich dann wie M verhält.

Unentscheidbarkeit des Halteproblems auf leerem Band

Satz

Das Halteproblem auf leerem Band ist unentscheidbar.

Beweis Wir reduzieren H auf H_0 : Sei $f(w_M \# x) = w_{M_x}$, wobei die DTM M_x erst x auf das Band schreibt, sich dann wie M verhält.

Dann gilt

$$w_M \# x \in H$$

Unentscheidbarkeit des Halteproblems auf leerem Band

Satz

Das Halteproblem auf leerem Band ist unentscheidbar.

Beweis Wir reduzieren H auf H_0 : Sei $f(w_M \# x) = w_{M_x}$, wobei die DTM M_x erst x auf das Band schreibt, sich dann wie M verhält.

Dann gilt

$w_M \# x \in H$
g.d.w. M hält für Eingabe x

Unentscheidbarkeit des Halteproblems auf leerem Band

Satz

Das Halteproblem auf leerem Band ist unentscheidbar.

Beweis Wir reduzieren H auf H_0 : Sei $f(w_M \# x) = w_{M_x}$, wobei die DTM M_x erst x auf das Band schreibt, sich dann wie M verhält.

Dann gilt

$w_M \# x \in H$
g.d.w. M hält für Eingabe x

$f(w_M \# x) \in H_0$

Unentscheidbarkeit des Halteproblems auf leerem Band

Satz

Das Halteproblem auf leerem Band ist unentscheidbar.

Beweis Wir reduzieren H auf H_0 : Sei $f(w_M \# x) = w_{M_x}$, wobei die DTM M_x erst x auf das Band schreibt, sich dann wie M verhält.

Dann gilt

$w_M \# x \in H$
g.d.w. M hält für Eingabe x

$w_{M_x} \in H_0$
g.d.w. $f(w_M \# x) \in H_0$

Unentscheidbarkeit des Halteproblems auf leerem Band

Satz

Das Halteproblem auf leerem Band ist unentscheidbar.

Beweis Wir reduzieren H auf H_0 : Sei $f(w_M \# x) = w_{M_x}$, wobei die DTM M_x erst x auf das Band schreibt, sich dann wie M verhält.

Dann gilt

$$w_M \# x \in H$$

g.d.w. M hält für Eingabe x

$$M_x \text{ hält für die leere Eingabe}$$

g.d.w. $w_{M_x} \in H_0$

g.d.w. $f(w_M \# x) \in H_0$

Unentscheidbarkeit des Halteproblems auf leerem Band

Satz

Das Halteproblem auf leerem Band ist unentscheidbar.

Beweis Wir reduzieren H auf H_0 : Sei $f(w_M \# x) = w_{M_x}$, wobei die DTM M_x erst x auf das Band schreibt, sich dann wie M verhält.

Dann gilt

$$w_M \# x \in H$$

g.d.w. M hält für Eingabe x

g.d.w. M_x hält für die leere Eingabe

g.d.w. $w_{M_x} \in H_0$

g.d.w. $f(w_M \# x) \in H_0$

Unentscheidbarkeit des Halteproblems auf leerem Band

Satz

Das Halteproblem auf leerem Band ist unentscheidbar.

Beweis Wir reduzieren H auf H_0 : Sei $f(w_M \# x) = w_{M_x}$, wobei die DTM M_x erst x auf das Band schreibt, sich dann wie M verhält.

Dann gilt

$$w_M \# x \in H$$

g.d.w. M hält für Eingabe x

g.d.w. M_x hält für die leere Eingabe

g.d.w. $w_{M_x} \in H_0$

g.d.w. $f(w_M \# x) \in H_0$

f kann durch eine Turingmaschine berechnet werden. Daher gilt $H \leq H_0$.

Da H unentscheidbar ist, ist H_0 unentscheidbar. □

Satz von Rice

Sei $\mathcal{R}$ die Klasse aller turingberechenbaren Funktionen. Sei $\mathcal{S}$ eine beliebige Teilmenge, sodass $\emptyset \subset \mathcal{S} \subset \mathcal{R}$. Dann ist folgende Sprache unentscheidbar:

$$C(\mathcal{S}) = \{w \mid \text{die von } M_w \text{ berechnete Funktion liegt in } \mathcal{S}\}$$

Satz von Rice

Sei $\mathcal{R}$ die Klasse aller turingberechenbaren Funktionen. Sei $\mathcal{S}$ eine beliebige Teilmenge, sodass $\emptyset \subset \mathcal{S} \subset \mathcal{R}$. Dann ist folgende Sprache unentscheidbar:

$$C(\mathcal{S}) = \{w \mid \text{die von } M_w \text{ berechnete Funktion liegt in } \mathcal{S}\}$$

Der Satz wurde von Henry Gordon Rice 1953 veröffentlicht. Er zeigt:

- ▶ Fast alle interessanten Eigenschaften von Turingmaschinen sind algorithmisch nicht entscheidbar.
- ▶ Z.B. folgt, dass die Sprache $L = \{w \mid M_w \text{ berechnet eine konstante Funktion}\}$ nicht entscheidbar ist.

Satz von Rice

Sei $\mathcal{R}$ die Klasse aller turingberechenbaren Funktionen. Sei $\mathcal{S}$ eine beliebige Teilmenge, sodass $\emptyset \subset \mathcal{S} \subset \mathcal{R}$. Dann ist folgende Sprache unentscheidbar:

$$C(\mathcal{S}) = \{w \mid \text{die von } M_w \text{ berechnete Funktion liegt in } \mathcal{S}\}$$

Beweis Sei $\Omega(x) = \text{undefiniert}$ für alle x .

Satz von Rice

Sei $\mathcal{R}$ die Klasse aller turingberechenbaren Funktionen. Sei $\mathcal{S}$ eine beliebige Teilmenge, sodass $\emptyset \subset \mathcal{S} \subset \mathcal{R}$. Dann ist folgende Sprache unentscheidbar:

$$C(\mathcal{S}) = \{w \mid \text{die von } M_w \text{ berechnete Funktion liegt in } \mathcal{S}\}$$

Beweis Sei $\Omega(x) = \text{undefiniert}$ für alle x .

Zeige:

1. $H_0 \leq C(\mathcal{S})$ falls $\Omega \notin \mathcal{S}$.
2. $H_0 \leq C(\mathcal{S})$ falls $\Omega \in \mathcal{S}$.

Satz von Rice

Sei $\mathcal{R}$ die Klasse aller turingberechenbaren Funktionen. Sei $\mathcal{S}$ eine beliebige Teilmenge, sodass $\emptyset \subset \mathcal{S} \subset \mathcal{R}$. Dann ist folgende Sprache unentscheidbar:

$$C(\mathcal{S}) = \{w \mid \text{die von } M_w \text{ berechnete Funktion liegt in } \mathcal{S}\}$$

Beweis Sei $\Omega(x) = \text{undefiniert}$ für alle x .

Zeige:

1. $H_0 \leq C(\mathcal{S})$ falls $\Omega \notin \mathcal{S}$.
2. $H_0 \leq C(\mathcal{S})$ falls $\Omega \in \mathcal{S}$.

Wir beweisen nur Punkt 1, da Punkt 2 analog geht (siehe Skript).

Der Satz von Rice

- Fall $\Omega \notin \mathcal{S}$: Da $\emptyset \in \mathcal{S}$, gibt es eine Funktion $q \in \mathcal{S}$, die von einer DTM Q berechnet wird.

Der Satz von Rice

- Fall $\Omega \notin \mathcal{S}$: Da $\emptyset \subset \mathcal{S}$, gibt es eine Funktion $q \in \mathcal{S}$, die von einer DTM Q berechnet wird.

Wir konstruieren eine DTM M^* . Für DTM M und Eingabe y :

1. M^* simuliert M auf leerer Eingabe.
2. Wenn M anhält, dann simuliert M^* die DTM Q mit Eingabe y .

Der Satz von Rice

- Fall $\Omega \notin \mathcal{S}$: Da $\emptyset \subset \mathcal{S}$, gibt es eine Funktion $q \in \mathcal{S}$, die von einer DTM Q berechnet wird.

Wir konstruieren eine DTM M^* . Für DTM M und Eingabe y :

1. M^* simuliert M auf leerer Eingabe.
2. Wenn M anhält, dann simuliert M^* die DTM Q mit Eingabe y .

Sei f die Funktion, die aus der Beschreibung w für DTM M_w die Beschreibung $f(w)$ von M_w^* erstellt. Diese Funktion ist total und berechenbar.

Der Satz von Rice

- Fall $\Omega \notin \mathcal{S}$: Da $\emptyset \subset \mathcal{S}$, gibt es eine Funktion $q \in \mathcal{S}$, die von einer DTM Q berechnet wird.

Wir konstruieren eine DTM M^* . Für DTM M und Eingabe y :

1. M^* simuliert M auf leerer Eingabe.
2. Wenn M anhält, dann simuliert M^* die DTM Q mit Eingabe y .

Sei f die Funktion, die aus der Beschreibung w für DTM M_w die Beschreibung $f(w)$ von M_w^* erstellt. Diese Funktion ist total und berechenbar.

Wir müssen noch zeigen, dass $w \in H_0$ g.d.w. $f(w) \in C(\mathcal{S})$.

Der Satz von Rice

$\Rightarrow$ Dann gilt

$$w \in H_0$$

Der Satz von Rice

$\Rightarrow$ Dann gilt

$$w \in H_0 \Rightarrow M_w \text{ hält auf leerer Eingabe}$$

Der Satz von Rice

$\Rightarrow$ Dann gilt

$$\begin{aligned} w \in H_0 &\Rightarrow M_w \text{ hält auf leerer Eingabe} \\ &\Rightarrow M_w^* \text{ berechnet } q \end{aligned}$$

Der Satz von Rice

$\Rightarrow$ Dann gilt

$w \in H_0 \Rightarrow M_w$ hält auf leerer Eingabe

$\Rightarrow M_w^*$ berechnet q

$\Rightarrow$ die von M_w^* berechnete Funktion liegt in $\mathcal{S}$

Der Satz von Rice

⇒ Dann gilt

$w \in H_0 \Rightarrow M_w$ hält auf leerer Eingabe

⇒ M_w^* berechnet q

⇒ die von M_w^* berechnete Funktion liegt in $\mathcal{S}$

⇒ $f(w) \in C(\mathcal{S})$

Der Satz von Rice

$\Rightarrow$ Dann gilt

$w \in H_0 \Rightarrow M_w$ hält auf leerer Eingabe

$\Rightarrow M_w^*$ berechnet q

$\Rightarrow$ die von M_w^* berechnete Funktion liegt in $\mathcal{S}$

$\Rightarrow f(w) \in C(\mathcal{S})$

$\Leftarrow$ Statt $f(w) \in C(\mathcal{S}) \Rightarrow w \in H_0$ beweisen wir die Kontraposition
 $w \notin H_0 \Rightarrow f(w) \notin C(\mathcal{S})$.

Der Satz von Rice

$\Rightarrow$ Dann gilt

$w \in H_0 \Rightarrow M_w$ hält auf leerer Eingabe

$\Rightarrow M_w^*$ berechnet q

$\Rightarrow$ die von M_w^* berechnete Funktion liegt in $\mathcal{S}$

$\Rightarrow f(w) \in C(\mathcal{S})$

$\Leftarrow$ Statt $f(w) \in C(\mathcal{S}) \Rightarrow w \in H_0$ beweisen wir die Kontraposition
 $w \notin H_0 \Rightarrow f(w) \notin C(\mathcal{S})$.

$w \notin H_0$

Der Satz von Rice

$\Rightarrow$ Dann gilt

$w \in H_0 \Rightarrow M_w$ hält auf leerer Eingabe

$\Rightarrow M_w^*$ berechnet q

$\Rightarrow$ die von M_w^* berechnete Funktion liegt in $\mathcal{S}$

$\Rightarrow f(w) \in C(\mathcal{S})$

$\Leftarrow$ Statt $f(w) \in C(\mathcal{S}) \Rightarrow w \in H_0$ beweisen wir die Kontraposition
 $w \notin H_0 \Rightarrow f(w) \notin C(\mathcal{S})$.

$w \notin H_0 \Rightarrow M_w$ hält nicht auf leerer Eingabe

Der Satz von Rice

$\Rightarrow$ Dann gilt

$w \in H_0 \Rightarrow M_w$ hält auf leerer Eingabe

$\Rightarrow M_w^*$ berechnet q

$\Rightarrow$ die von M_w^* berechnete Funktion liegt in $\mathcal{S}$

$\Rightarrow f(w) \in C(\mathcal{S})$

$\Leftarrow$ Statt $f(w) \in C(\mathcal{S}) \Rightarrow w \in H_0$ beweisen wir die Kontraposition
 $w \notin H_0 \Rightarrow f(w) \notin C(\mathcal{S})$.

$w \notin H_0 \Rightarrow M_w$ hält nicht auf leerer Eingabe

$\Rightarrow M_w^*$ berechnet Ω

Der Satz von Rice

$\Rightarrow$ Dann gilt

$w \in H_0 \Rightarrow M_w$ hält auf leerer Eingabe

$\Rightarrow M_w^*$ berechnet q

$\Rightarrow$ die von M_w^* berechnete Funktion liegt in $\mathcal{S}$

$\Rightarrow f(w) \in C(\mathcal{S})$

$\Leftarrow$ Statt $f(w) \in C(\mathcal{S}) \Rightarrow w \in H_0$ beweisen wir die Kontraposition
 $w \notin H_0 \Rightarrow f(w) \notin C(\mathcal{S})$.

$w \notin H_0 \Rightarrow M_w$ hält nicht auf leerer Eingabe

$\Rightarrow M_w^*$ berechnet Ω

$\Rightarrow$ die von M_w^* berechnete Funktion liegt nicht in $\mathcal{S}$

Der Satz von Rice

$\Rightarrow$ Dann gilt

$w \in H_0 \Rightarrow M_w$ hält auf leerer Eingabe

$\Rightarrow M_w^*$ berechnet q

$\Rightarrow$ die von M_w^* berechnete Funktion liegt in $\mathcal{S}$

$\Rightarrow f(w) \in C(\mathcal{S})$

$\Leftarrow$ Statt $f(w) \in C(\mathcal{S}) \Rightarrow w \in H_0$ beweisen wir die Kontraposition
 $w \notin H_0 \Rightarrow f(w) \notin C(\mathcal{S})$.

$w \notin H_0 \Rightarrow M_w$ hält nicht auf leerer Eingabe

$\Rightarrow M_w^*$ berechnet Ω

$\Rightarrow$ die von M_w^* berechnete Funktion liegt nicht in $\mathcal{S}$

$\Rightarrow f(w) \notin C(\mathcal{S})$

Der Satz von Rice

$\Rightarrow$ Dann gilt

$w \in H_0 \Rightarrow M_w$ hält auf leerer Eingabe

$\Rightarrow M_w^*$ berechnet q

$\Rightarrow$ die von M_w^* berechnete Funktion liegt in $\mathcal{S}$

$\Rightarrow f(w) \in C(\mathcal{S})$

$\Leftarrow$ Statt $f(w) \in C(\mathcal{S}) \Rightarrow w \in H_0$ beweisen wir die Kontraposition
 $w \notin H_0 \Rightarrow f(w) \notin C(\mathcal{S})$.

$w \notin H_0 \Rightarrow M_w$ hält nicht auf leerer Eingabe

$\Rightarrow M_w^*$ berechnet Ω

$\Rightarrow$ die von M_w^* berechnete Funktion liegt nicht in $\mathcal{S}$

$\Rightarrow f(w) \notin C(\mathcal{S})$

Daher $H_0 \leq C(\mathcal{S})$. Da H_0 unentscheidbar ist, ist damit auch $C(\mathcal{S})$ unentscheidbar. □

Anwendung des Satzes von Rice

Sei L eine Sprache, die als unentscheidbar zu beweisen ist.

Schritte:

1. Definiere Menge S von Funktionen.
2. Zeige Nichttrivialität von S .
3. Begründe, dass S richtig gewählt, d.h. $C(S) = L$.
4. Der Satz von Rice zeigt dann das Resultat.

Erstes Beispiel für die Anwendung des Satzes von Rice

Satz

Es ist unentscheidbar, ob eine Turingmaschine für jede Eingabe $i \in \mathbb{N}$ die Zahl $i + 1 \in \mathbb{N}$ berechnet.

Beweis Sei $\text{succ}(i) = i + 1$. Sei $\mathcal{S} := \{\text{succ}\}$.

Erstes Beispiel für die Anwendung des Satzes von Rice

Satz

Es ist unentscheidbar, ob eine Turingmaschine für jede Eingabe $i \in \mathbb{N}$ die Zahl $i + 1 \in \mathbb{N}$ berechnet.

Beweis Sei $\text{succ}(i) = i + 1$. Sei $\mathcal{S} := \{\text{succ}\}$.

$\mathcal{S}$ ist nicht trivial:

- ▶ $\emptyset \subset \mathcal{S}$: klar
- ▶ $\mathcal{S} \subset \mathcal{R}$: f mit $f(i) = i + 2$ ist berechenbar, aber $f \notin \mathcal{S}$.

Erstes Beispiel für die Anwendung des Satzes von Rice

Satz

Es ist unentscheidbar, ob eine Turingmaschine für jede Eingabe $i \in \mathbb{N}$ die Zahl $i + 1 \in \mathbb{N}$ berechnet.

Beweis Sei $\text{succ}(i) = i + 1$. Sei $\mathcal{S} := \{\text{succ}\}$.

$\mathcal{S}$ ist nicht trivial:

- ▶ $\emptyset \subset \mathcal{S}$: klar
- ▶ $\mathcal{S} \subset \mathcal{R}$: f mit $f(i) = i + 2$ ist berechenbar, aber $f \notin \mathcal{S}$.

Mit Satz von Rice:

$$C(\mathcal{S}) = \{w \mid \text{die von } M_w \text{ berechnete Funktion ist } \text{succ}\}$$

ist nicht entscheidbar.



Zweites Beispiel für die Anwendung des Satzes von Rice

Satz

Es ist unentscheidbar, ob für die Turingmaschine M gilt, dass $L(M) = \emptyset$.

Beweis Sei $\mathcal{S} := \{\Omega\}$.

Zweites Beispiel für die Anwendung des Satzes von Rice

Satz

Es ist unentscheidbar, ob für die Turingmaschine M gilt, dass $L(M) = \emptyset$.

Beweis Sei $\mathcal{S} := \{\Omega\}$.

$\mathcal{S}$ ist nicht trivial:

- ▶ $\emptyset \subset \mathcal{S}$: klar
- ▶ $\mathcal{S} \subset \mathcal{R}$: f mit $f(x) = x$ ist berechenbar, aber $f \notin \mathcal{S}$.

Zweites Beispiel für die Anwendung des Satzes von Rice

Satz

Es ist unentscheidbar, ob für die Turingmaschine M gilt, dass $L(M) = \emptyset$.

Beweis Sei $\mathcal{S} := \{\Omega\}$.

$\mathcal{S}$ ist nicht trivial:

- ▶ $\emptyset \subset \mathcal{S}$: klar
- ▶ $\mathcal{S} \subset \mathcal{R}$: f mit $f(x) = x$ ist berechenbar, aber $f \notin \mathcal{S}$.

Mit Satz von Rice:

$$\begin{aligned} C(\mathcal{S}) &= \{w \mid \text{die von } M_w \text{ berechnete Funktion liegt in } \mathcal{S}\} \\ &= \{w \mid M_w \text{ akzeptiert nie}\} \\ &= \{w \mid L(M_w) = \emptyset\} \end{aligned}$$

ist nicht entscheidbar.



Drittes Beispiel für die Anwendung des Satzes von Rice

Satz

Es ist unentscheidbar, ob für die Turingmaschine M gilt, dass M für alle Eingaben hält.

Drittes Beispiel für die Anwendung des Satzes von Rice

Satz

Es ist unentscheidbar, ob für die Turingmaschine M gilt, dass M für alle Eingaben hält.

Beweis Sei $\mathcal{S} := \{f \mid f(x) \text{ ist total und berechenbar}\}$.

Drittes Beispiel für die Anwendung des Satzes von Rice

Satz

Es ist unentscheidbar, ob für die Turingmaschine M gilt, dass M für alle Eingaben hält.

Beweis Sei $\mathcal{S} := \{f \mid f(x) \text{ ist total und berechenbar}\}$.

$\mathcal{S}$ ist nicht trivial:

- ▶ $\emptyset \subset \mathcal{S}$: Z.B. gilt $id \in \mathcal{S}$ mit $id(x) = x$ für alle $x \in \mathbb{N}$.
- ▶ $\mathcal{S} \subset \mathcal{R}$: $f(1) = \text{undefiniert}$ und $f(x) = 0$ für $x \neq 1$, ist berechenbar und $f \notin \mathcal{S}$.

Drittes Beispiel für die Anwendung des Satzes von Rice

Satz

Es ist unentscheidbar, ob für die Turingmaschine M gilt, dass M für alle Eingaben hält.

Beweis Sei $\mathcal{S} := \{f \mid f(x) \text{ ist total und berechenbar}\}$.

$\mathcal{S}$ ist nicht trivial:

- ▶ $\emptyset \subset \mathcal{S}$: Z.B. gilt $id \in \mathcal{S}$ mit $id(x) = x$ für alle $x \in \mathbb{N}$.
- ▶ $\mathcal{S} \subset \mathcal{R}$: $f(1) = \text{undefiniert}$ und $f(x) = 0$ für $x \neq 1$, ist berechenbar und $f \notin \mathcal{S}$.

Mit Satz von Rice:

$$\begin{aligned} C(\mathcal{S}) &= \{w \mid \text{die von } M_w \text{ berechnete Funktion liegt in } \mathcal{S}\} \\ &= \{w \mid M_w \text{ akzeptiert für jede Eingabe}\} \end{aligned}$$

ist nicht entscheidbar.



Bemerkungen

Der Satz von Rice lässt sich anwenden, auf Eigenschaften von $L(M)$ bzw. der von M berechneten Funktion, aber er macht **keine Aussage über Eigenschaften von M** .

Bemerkungen

Der Satz von Rice lässt sich anwenden, auf Eigenschaften von $L(M)$ bzw. der von M berechneten Funktion, aber er macht **keine Aussage über Eigenschaften von M** .

Beispiele:

- Ist es entscheidbar, ob M höchstens 100 Zustände hat?

☞ Der Satz von Rice ist hier **nicht** anwendbar.

(Das Problem ist sogar entscheidbar.)

Bemerkungen

Der Satz von Rice lässt sich anwenden, auf Eigenschaften von $L(M)$ bzw. der von M berechneten Funktion, aber er macht **keine Aussage über Eigenschaften von M** .

Beispiele:

- ▶ Ist es entscheidbar, ob M höchstens 100 Zustände hat?
☞ Der Satz von Rice ist hier **nicht** anwendbar.
(Das Problem ist sogar entscheidbar.)
- ▶ Ist es entscheidbar, ob M für jede Eingabe nach 1000 Schritten anhält?
☞ Der Satz von Rice ist hier **nicht** anwendbar.
(Das Problem ist sogar entscheidbar.)

Bemerkungen

Der Satz von Rice lässt sich anwenden, auf Eigenschaften von $L(M)$ bzw. der von M berechneten Funktion, aber er macht **keine Aussage über Eigenschaften von M** .

Beispiele:

- ▶ Ist es entscheidbar, ob M höchstens 100 Zustände hat?
 Der Satz von Rice ist hier **nicht** anwendbar.
(Das Problem ist sogar entscheidbar.)
- ▶ Ist es entscheidbar, ob M für jede Eingabe nach 1000 Schritten anhält?
 Der Satz von Rice ist hier **nicht** anwendbar.
(Das Problem ist sogar entscheidbar.)
- ▶ Ist es entscheidbar, ob M für höchstens 50 verschiedene Eingaben anhält?
 Der Satz von Rice ist anwendbar, da die Eigenschaft auch etwas über die berechnete Funktion aussagt (sie soll für höchstens 50 Eingaben definiert sein).
(Das Problem ist dann unentscheidbar.)

11a

Das Postsche Korrespondenzproblem

Prof. Dr. Jasmin Blanchette

Lehr- und Forschungseinheit für
Theoretische Informatik und Theorembeweisen

Stand: 3. Juli 2024
Basierend auf Folien von PD Dr. David Sabel



Das Postsche Korrespondenzproblem

- ▶ Das Problem wurde vorgeschlagen von Emil Post 1946.
- ▶ Es ist ein einfaches aber unentscheidbares Problem.
- ▶ Es wird häufig verwendet, um es auf andere Probleme zu reduzieren und deren Unentscheidbarkeit zu zeigen.
- ▶ Es hat nichts mit Turingmaschinen und deren Akzeptanzverhalten zu tun (im Gegensatz zu den Varianten des Halteproblems).

Definition des Postschen Korrespondenzproblems

Definition

Gegeben sei ein Alphabet Σ und eine Folge von Wortpaaren

$$K = ((x_1, y_1), \dots, (x_k, y_k))$$

mit $x_i, y_i \in \Sigma^+$. Das **Postsche Korrespondenzproblem (PCP)** ist die Frage, ob es für die gegebene Folge K eine nichtleere Folge von Indizes $i_1, \dots, i_m$ mit $i_j \in \{1, \dots, k\}$ gibt, sodass

$$x_{i_1} \cdots x_{i_m} = y_{i_1} \cdots y_{i_m}$$

Definition des Postschen Korrespondenzproblems

Definition

Gegeben sei ein Alphabet Σ und eine Folge von Wortpaaren

$$K = ((x_1, y_1), \dots, (x_k, y_k))$$

mit $x_i, y_i \in \Sigma^+$. Das **Postsche Korrespondenzproblem (PCP)** ist die Frage, ob es für die gegebene Folge K eine nichtleere Folge von Indizes $i_1, \dots, i_m$ mit $i_j \in \{1, \dots, k\}$ gibt, sodass

$$x_{i_1} \cdots x_{i_m} = y_{i_1} \cdots y_{i_m}$$

Als formale Sprache:

$\text{PCP} = \{\text{code}(K) \in \Sigma^* \mid \text{für } K = ((x_1, y_1), \dots, (x_k, y_k)) \text{ gibt es eine nichtleere Folge von Indizes } i_1, \dots, i_m \text{ mit } i_j \in \{1, \dots, k\}, \text{ sodass } x_{i_1} \cdots x_{i_m} = y_{i_1} \cdots y_{i_m}\}$

PCP ist wie ein Domino-Spiel

Spielsteinarten: $\left(\begin{bmatrix} x_1 \\ y_1 \end{bmatrix}, \dots, \begin{bmatrix} x_k \\ y_k \end{bmatrix} \right)$.

Gesucht ist eine Aneinanderreihung der Spielsteine, sodass oben wie unten dasselbe Wort abgelesen werden kann. Dabei dürfen beliebig (aber endlich) viele Spielsteine verwendet werden.

PCP ist wie ein Domino-Spiel

Spielsteinarten: $\left(\begin{bmatrix} x_1 \\ y_1 \end{bmatrix}, \dots, \begin{bmatrix} x_k \\ y_k \end{bmatrix} \right).$

Gesucht ist eine Aneinanderreihung der Spielsteine, sodass oben wie unten dasselbe Wort abgelesen werden kann. Dabei dürfen beliebig (aber endlich) viele Spielsteine verwendet werden.

Beispiel:

Sei $K = \left(\begin{bmatrix} a \\ aba \end{bmatrix}, \begin{bmatrix} baa \\ aa \end{bmatrix}, \begin{bmatrix} ab \\ bb \end{bmatrix} \right).$

PCP ist wie ein Domino-Spiel

Spielsteinarten: $\left(\begin{bmatrix} x_1 \\ y_1 \end{bmatrix}, \dots, \begin{bmatrix} x_k \\ y_k \end{bmatrix} \right).$

Gesucht ist eine Aneinanderreihung der Spielsteine, sodass oben wie unten dasselbe Wort abgelesen werden kann. Dabei dürfen beliebig (aber endlich) viele Spielsteine verwendet werden.

Beispiel:

Sei $K = \left(\begin{bmatrix} a \\ aba \end{bmatrix}, \begin{bmatrix} baa \\ aa \end{bmatrix}, \begin{bmatrix} ab \\ bb \end{bmatrix} \right).$

(1, 2, 3, 2) ist eine Lösung, da

$$\begin{bmatrix} a \\ aba \end{bmatrix} \begin{bmatrix} baa \\ aa \end{bmatrix} \begin{bmatrix} ab \\ bb \end{bmatrix} \begin{bmatrix} baa \\ aa \end{bmatrix} = abaaabbaa$$
$$= abaaabbaa$$

Beispiel für PCP

$$\text{Sei } K = \left(\begin{bmatrix} ab \\ bba \end{bmatrix}, \begin{bmatrix} ba \\ baa \end{bmatrix}, \begin{bmatrix} ba \\ aba \end{bmatrix}, \begin{bmatrix} bba \\ b \end{bmatrix} \right).$$

Beispiel für PCP

$$\text{Sei } K = \left(\begin{bmatrix} ab \\ bba \end{bmatrix}, \begin{bmatrix} ba \\ baa \end{bmatrix}, \begin{bmatrix} ba \\ aba \end{bmatrix}, \begin{bmatrix} bba \\ b \end{bmatrix} \right).$$

Die kürzeste Lösung benötigt 66 Wortpaare:

(2, 1, 3, 1, 1, 2, 4, 2, 1, 3, 1, 3, 1, 1, 3, 1, 1, 2, 4, 1, 1, 2, 4, 3, 1, 4, 4, 3, 1, 1, 1, 2, 4, 2, 4, 4, 4, 3, 1, 3, 1, 4, 2, 4, 1, 1, 2, 4, 1, 4, 4, 3, 1, 4, 4, 3, 4, 4, 3, 4, 2, 4, 1, 4, 4, 3).

Unentscheidbarkeit des PCP

Der Beweis von $H \leq \text{PCP}$ erfolgt in zwei Schritten:

1. $\text{MPCP} \leq \text{PCP}$
2. $H \leq \text{MPCP}$.

Unentscheidbarkeit des PCP

Der Beweis von $H \leq \text{PCP}$ erfolgt in zwei Schritten:

1. $\text{MPCP} \leq \text{PCP}$
2. $H \leq \text{MPCP}$.

MPCP ist das **Modifizierte Postsche Korrespondenzproblem**:
Zulässige Lösungen müssen mit dem Index 1 beginnen.

Unentscheidbarkeit des PCP

Der Beweis von $H \leq \text{PCP}$ erfolgt in zwei Schritten:

1. $\text{MPCP} \leq \text{PCP}$
2. $H \leq \text{MPCP}$.

MPCP ist das **Modifizierte Postsche Korrespondenzproblem**:

Zulässige Lösungen müssen mit dem Index 1 beginnen.

Damit folgt aus der Unentscheidbarkeit von H die Unentscheidbarkeit von MPCP und damit die Unentscheidbarkeit des PCP.

Definition

Gegeben sei ein Alphabet Σ und eine Folge von Wortpaaren

$$K = ((x_1, y_1), \dots, (x_k, y_k))$$

mit $x_i, y_i \in \Sigma^+$. Das **modifizierte Postsche Korrespondenzproblem (MPCP)** ist die Frage, ob es für die gegebene Folge K eine nichtleere Folge von Indizes $i_1 = 1, i_2, \dots, i_m$ mit $i_j \in \{1, \dots, k\}$ gibt, sodass

$$x_{i_1} \cdots x_{i_m} = y_{i_1} \cdots y_{i_m}$$

Reduktion von MPCP auf PCP

Lemma

$\text{MPCP} \leq \text{PCP}$.

Reduktion von MPCP auf PCP

Lemma

$\text{MPCP} \leq \text{PCP}$.

Beweis Gesucht ist ein totales und berechenbares f , sodass:
 K ist MPCP-lösbar g.d.w. $f(K)$ PCP-lösbar ist.

Reduktion von MPCP auf PCP

Lemma

$\text{MPCP} \leq \text{PCP}$.

Beweis Gesucht ist ein totales und berechenbares f , sodass:

K ist MPCP-lösbar g.d.w. $f(K)$ PCP-lösbar ist.

Für $w = a_1 \cdots a_n \in \Sigma^+$ seien

$$\bar{w} = \#a_1\#a_2\#\cdots\#a_n\# \quad \acute{w} = a_1\#a_2\#\cdots\#a_n\# \quad \grave{w} = \#a_1\#a_2\#\cdots\#a_n$$

Reduktion von MPCP auf PCP

Lemma

$\text{MPCP} \leq \text{PCP}$.

Beweis Gesucht ist ein totales und berechenbares f , sodass:

K ist MPCP-lösbar g.d.w. $f(K)$ PCP-lösbar ist.

Für $w = a_1 \cdots a_n \in \Sigma^+$ seien

$$\bar{w} = \#a_1\#a_2\#\cdots\#a_n\# \quad \acute{w} = a_1\#a_2\#\cdots\#a_n\# \quad \grave{w} = \#a_1\#a_2\#\cdots\#a_n$$

$$\text{Sei } f\left(\begin{bmatrix} x_1 \\ y_1 \end{bmatrix}, \dots, \begin{bmatrix} x_k \\ y_k \end{bmatrix}\right) = \left(\begin{bmatrix} \bar{x}_1 \\ \grave{y}_1 \end{bmatrix}, \begin{bmatrix} \acute{x}_1 \\ \grave{y}_1 \end{bmatrix}, \dots, \begin{bmatrix} \acute{x}_k \\ \grave{y}_k \end{bmatrix}, \begin{bmatrix} \$ \\ \#\$ \end{bmatrix}\right).$$

Beispiel für MPCP

$$\text{Sei } K = \left(\begin{bmatrix} a \\ aba \end{bmatrix}, \begin{bmatrix} baa \\ aa \end{bmatrix}, \begin{bmatrix} ab \\ bb \end{bmatrix} \right).$$

Beispiel für MPCP

Sei $K = \left(\begin{bmatrix} a \\ aba \end{bmatrix}, \begin{bmatrix} baa \\ aa \end{bmatrix}, \begin{bmatrix} ab \\ bb \end{bmatrix} \right)$.

$(1, 2, 3, 2)$ ist eine MPCP-Lösung:

$$\begin{bmatrix} a \\ aba \end{bmatrix} \begin{bmatrix} baa \\ aa \end{bmatrix} \begin{bmatrix} ab \\ bb \end{bmatrix} \begin{bmatrix} baa \\ aa \end{bmatrix}$$

Beispiel für MPCP

Sei $K = \left(\begin{bmatrix} a \\ aba \end{bmatrix}, \begin{bmatrix} baa \\ aa \end{bmatrix}, \begin{bmatrix} ab \\ bb \end{bmatrix} \right)$.

$(1, 2, 3, 2)$ ist eine MPCP-Lösung:

$$\begin{bmatrix} a \\ aba \end{bmatrix} \begin{bmatrix} baa \\ aa \end{bmatrix} \begin{bmatrix} ab \\ bb \end{bmatrix} \begin{bmatrix} baa \\ aa \end{bmatrix}$$

$$f(K) = \left(\begin{bmatrix} \#a\# \\ \#a\#b\#a \end{bmatrix}, \begin{bmatrix} a\# \\ \#a\#b\#a \end{bmatrix}, \begin{bmatrix} b\#a\#a\# \\ \#a\#a \end{bmatrix}, \begin{bmatrix} a\#b\# \\ \#b\#b \end{bmatrix}, \begin{bmatrix} \$ \\ \#\$ \end{bmatrix} \right).$$

Beispiel für MPCP

Sei $K = \left(\begin{bmatrix} a \\ aba \end{bmatrix}, \begin{bmatrix} baa \\ aa \end{bmatrix}, \begin{bmatrix} ab \\ bb \end{bmatrix} \right)$.

$(1, 2, 3, 2)$ ist eine MPCP-Lösung:

$$\begin{bmatrix} a \\ aba \end{bmatrix} \begin{bmatrix} baa \\ aa \end{bmatrix} \begin{bmatrix} ab \\ bb \end{bmatrix} \begin{bmatrix} baa \\ aa \end{bmatrix}$$

$$f(K) = \left(\begin{bmatrix} \#a\# \\ \#a\#b\#a \end{bmatrix}, \begin{bmatrix} a\# \\ \#a\#b\#a \end{bmatrix}, \begin{bmatrix} b\#a\#a\# \\ \#a\#a \end{bmatrix}, \begin{bmatrix} a\#b\# \\ \#b\#b \end{bmatrix}, \begin{bmatrix} \$ \\ \#\$ \end{bmatrix} \right).$$

$(1, 3, 4, 3, 5)$ ist eine PCP-Lösung:

$$\begin{bmatrix} \#a\# \\ \#a\#b\#a \end{bmatrix} \begin{bmatrix} b\#a\#a\# \\ \#a\#a \end{bmatrix} \begin{bmatrix} a\#b\# \\ \#b\#b \end{bmatrix} \begin{bmatrix} b\#a\#a\# \\ \#a\#a \end{bmatrix} \begin{bmatrix} \$ \\ \#\$ \end{bmatrix}$$

Beispiel für MPCP

Sei $K = \left(\begin{bmatrix} a \\ aba \end{bmatrix}, \begin{bmatrix} baa \\ aa \end{bmatrix}, \begin{bmatrix} ab \\ bb \end{bmatrix} \right)$.

$(1, 2, 3, 2)$ ist eine MPCP-Lösung:

$$\begin{bmatrix} a \\ aba \end{bmatrix} \begin{bmatrix} baa \\ aa \end{bmatrix} \begin{bmatrix} ab \\ bb \end{bmatrix} \begin{bmatrix} baa \\ aa \end{bmatrix}$$

$$f(K) = \left(\begin{bmatrix} \#a\# \\ \#a\#b\#a \end{bmatrix}, \begin{bmatrix} a\# \\ \#a\#b\#a \end{bmatrix}, \begin{bmatrix} b\#a\#a\# \\ \#a\#a \end{bmatrix}, \begin{bmatrix} a\#b\# \\ \#b\#b \end{bmatrix}, \begin{bmatrix} \$ \\ \#\$ \end{bmatrix} \right).$$

$(1, 3, 4, 3, 5)$ ist eine PCP-Lösung:

$$\begin{bmatrix} \#a\# \\ \#a\#b\#a \end{bmatrix} \begin{bmatrix} b\#a\#a\# \\ \#a\#a \end{bmatrix} \begin{bmatrix} a\#b\# \\ \#b\#b \end{bmatrix} \begin{bmatrix} b\#a\#a\# \\ \#a\#a \end{bmatrix} \begin{bmatrix} \$ \\ \#\$ \end{bmatrix}$$

So ist $(1, 3, 4, 3, 5, 1, 3, 4, 3, 5)$.

Reduktion von MPCP auf PCP

Beweis Gesucht ist ein totales und berechenbares f , sodass:

K ist MPCP-lösbar g.d.w. $f(K)$ PCP-lösbar ist.

Für $w = a_1 \cdots a_n \in \Sigma^+$ seien

$$\bar{w} = \#a_1\#a_2\#\cdots\#a_n\# \quad \acute{w} = a_1\#a_2\#\cdots\#a_n\# \quad \grave{w} = \#a_1\#a_2\#\cdots\#a_n$$

$$\text{Sei } f\left(\begin{bmatrix} x_1 \\ y_1 \end{bmatrix}, \dots, \begin{bmatrix} x_k \\ y_k \end{bmatrix}\right) = \left(\begin{bmatrix} \bar{x}_1 \\ \grave{y}_1 \end{bmatrix}, \begin{bmatrix} \acute{x}_1 \\ \grave{y}_1 \end{bmatrix}, \dots, \begin{bmatrix} \acute{x}_k \\ \grave{y}_k \end{bmatrix}, \begin{bmatrix} \$ \\ \#\$ \end{bmatrix}\right).$$

Reduktion von MPCP auf PCP

Beweis Gesucht ist ein totales und berechenbares f , sodass:

K ist MPCP-lösbar g.d.w. $f(K)$ PCP-lösbar ist.

Für $w = a_1 \cdots a_n \in \Sigma^+$ seien

$$\bar{w} = \#a_1\#a_2\#\cdots\#a_n\# \quad \acute{w} = a_1\#a_2\#\cdots\#a_n\# \quad \grave{w} = \#a_1\#a_2\#\cdots\#a_n$$

$$\text{Sei } f\left(\begin{bmatrix} x_1 \\ y_1 \end{bmatrix}, \dots, \begin{bmatrix} x_k \\ y_k \end{bmatrix}\right) = \left(\begin{bmatrix} \bar{x}_1 \\ \grave{y}_1 \end{bmatrix}, \begin{bmatrix} \acute{x}_1 \\ \grave{y}_1 \end{bmatrix}, \dots, \begin{bmatrix} \acute{x}_k \\ \grave{y}_k \end{bmatrix}, \begin{bmatrix} \$ \\ \#\$ \end{bmatrix}\right).$$

$\Rightarrow$ Wenn $(1, i_2, \dots, i_m)$ eine Lösung für K ist,
dann ist $(1, i_2+1, \dots, i_m+1, k+2)$ eine Lösung für $f(K)$.

Reduktion von MPCP auf PCP

Beweis Gesucht ist ein totales und berechenbares f , sodass:

K ist MPCP-lösbar g.d.w. $f(K)$ PCP-lösbar ist.

Für $w = a_1 \cdots a_n \in \Sigma^+$ seien

$$\bar{w} = \#a_1\#a_2\#\cdots\#a_n\# \quad \acute{w} = a_1\#a_2\#\cdots\#a_n\# \quad \grave{w} = \#a_1\#a_2\#\cdots\#a_n$$

$$\text{Sei } f\left(\begin{bmatrix} x_1 \\ y_1 \end{bmatrix}, \dots, \begin{bmatrix} x_k \\ y_k \end{bmatrix}\right) = \left(\begin{bmatrix} \bar{x}_1 \\ \grave{y}_1 \end{bmatrix}, \begin{bmatrix} \acute{x}_1 \\ \grave{y}_1 \end{bmatrix}, \dots, \begin{bmatrix} \acute{x}_k \\ \grave{y}_k \end{bmatrix}, \begin{bmatrix} \$ \\ \#\$ \end{bmatrix}\right).$$

$\Rightarrow$ Wenn $(1, i_2, \dots, i_m)$ eine Lösung für K ist,
dann ist $(1, i_2+1, \dots, i_m+1, k+2)$ eine Lösung für $f(K)$.

$\Leftarrow$ Wenn $(i_1, \dots, i_m)$ eine Lösung für $f(K)$ ist,
dann muss $i_1 = 1$ und $k+2$ muss in der Lösung vorkommen.
Sei ℓ der kleinste Index mit $i_\ell = k+2$.

Dann ist $(1, i_2 - 1, \dots, i_{\ell-1} - 1)$ eine Lösung für K . □

Reduktion von H auf MPCP

Lemma

$H \leq \text{MPCP}$.

Reduktion von H auf MPCP

Lemma

$H \leq \text{MPCP}$.

Beweis Seien w eine Turingmaschinenbeschreibung und x eine Eingabe.
Gesucht ist ein totales und berechenbares f , sodass:
die DTM M_w auf Eingabe x anhält g.d.w. $f(w\#x)$ MPCP-lösbar ist.

Reduktion von H auf MPCP

Lemma

$H \leq \text{MPCP}$.

Beweis Seien w eine Turingmaschinenbeschreibung und x eine Eingabe.
Gesucht ist ein totales und berechenbares f , sodass:
die DTM M_w auf Eingabe x anhält g.d.w. $f(w\#x)$ MPCP-lösbar ist.

Sei $M_w = (Z, \Sigma, \Gamma, \delta, z_0, \square, E)$.

Reduktion von H auf MPCP

Lemma

$H \leq \text{MPCP}$.

Beweis Seien w eine Turingmaschinenbeschreibung und x eine Eingabe.
Gesucht ist ein totales und berechenbares f , sodass:
die DTM M_w auf Eingabe x anhält g.d.w. $f(w\#x)$ MPCP-lösbar ist.

Sei $M_w = (Z, \Sigma, \Gamma, \delta, z_0, \square, E)$.

Als Alphabet für das MPCP nehmen wir $\Gamma \cup Z \cup \{\#\}$.

Reduktion von H auf MPCP

Lemma

$H \leq \text{MPCP}$.

Beweis Seien w eine Turingmaschinenbeschreibung und x eine Eingabe.
Gesucht ist ein totales und berechenbares f , sodass:
die DTM M_w auf Eingabe x anhält g.d.w. $f(w\#x)$ MPCP-lösbar ist.

Sei $M_w = (Z, \Sigma, \Gamma, \delta, z_0, \square, E)$.

Als Alphabet für das MPCP nehmen wir $\Gamma \cup Z \cup \{\#\}$.

Grundgedanke: Lösungen des MPCP simulieren Übergangsfolgen der DTM.

Reduktion von H auf MPCP

Lemma

$H \leq \text{MPCP}$.

Beweis Seien w eine Turingmaschinenbeschreibung und x eine Eingabe.
Gesucht ist ein totales und berechenbares f , sodass:
die DTM M_w auf Eingabe x anhält g.d.w. $f(w\#x)$ MPCP-lösbar ist.

Sei $M_w = (Z, \Sigma, \Gamma, \delta, z_0, \square, E)$.

Als Alphabet für das MPCP nehmen wir $\Gamma \cup Z \cup \{\#\}$.

Grundgedanke: Lösungen des MPCP simulieren Übergangsfolgen der DTM.

Das erste Wortpaar (mit dem jede Lösung anfangen muss) ist $\begin{bmatrix} x_1 \\ y_1 \end{bmatrix} = \begin{bmatrix} \# \\ \#z_0w\# \end{bmatrix}$.

Weitere Paare lassen sich in Gruppen von „Regeln“ aufteilen:
Kopierregeln, Übergangsregeln, Löschregeln, Abschlussregeln.

Die Kopierregeln

► $\begin{bmatrix} a \\ a \end{bmatrix}$ für alle $a \in \Gamma \cup \{\#\}$

Die Übergangsregeln

► $\begin{bmatrix} za \\ z'c \end{bmatrix}$ falls $\delta(z, a) = (z', c, N)$

Die Übergangsregeln

- ▶ $\begin{bmatrix} za \\ z'c \end{bmatrix}$ falls $\delta(z, a) = (z', c, N)$
- ▶ $\begin{bmatrix} za \\ cz' \end{bmatrix}$ falls $\delta(z, a) = (z', c, R)$

Die Übergangsregeln

- ▶ $\begin{bmatrix} za \\ z'c \end{bmatrix}$ falls $\delta(z, a) = (z', c, N)$
- ▶ $\begin{bmatrix} za \\ cz' \end{bmatrix}$ falls $\delta(z, a) = (z', c, R)$
- ▶ $\begin{bmatrix} bza \\ z'bc \end{bmatrix}$ falls $\delta(z, a) = (z', c, L)$ für alle $b \in \Gamma$

Die Übergangsregeln

- ▶ $\begin{bmatrix} za \\ z'c \end{bmatrix}$ falls $\delta(z, a) = (z', c, N)$
- ▶ $\begin{bmatrix} za \\ cz' \end{bmatrix}$ falls $\delta(z, a) = (z', c, R)$
- ▶ $\begin{bmatrix} bza \\ z'bc \end{bmatrix}$ falls $\delta(z, a) = (z', c, L)$ für alle $b \in \Gamma$
- ▶ $\begin{bmatrix} \#za \\ \#z'\square c \end{bmatrix}$ falls $\delta(z, a) = (z', c, L)$
- ▶ $\begin{bmatrix} z\# \\ z'c\# \end{bmatrix}$ falls $\delta(z, \square) = (z', c, N)$
- ▶ $\begin{bmatrix} z\# \\ cz'\# \end{bmatrix}$ falls $\delta(z, \square) = (z', c, R)$
- ▶ $\begin{bmatrix} bz\# \\ z'bc\# \end{bmatrix}$ falls $\delta(z, \square) = (z', c, L)$ für alle $b \in \Gamma$

Die Löschregeln

- ▶ $\begin{bmatrix} az_e \\ z_e \end{bmatrix}$ für alle $a \in \Gamma, z_e \in E$
- ▶ $\begin{bmatrix} z_e a \\ z_e \end{bmatrix}$ für alle $a \in \Gamma, z_e \in E$

Die Abschlussregeln

► $\begin{bmatrix} z_e \# \# \\ \# \end{bmatrix}$ für alle $z_e \in E$

Beispiel für die Reduktion von H auf MPCP

$$z_0abc \vdash dz_1bc \vdash dez_2c \vdash defz_3\Box \vdash defz_e\Box$$

Beispiel für die Reduktion von H auf MPCP

$$z_0abc \vdash dz_1bc \vdash dez_2c \vdash defz_3\Box \vdash defz_e\Box$$

Lösende Spielsteinfolge:

$$\begin{array}{cccccccccccccccccccc}
 \left[\begin{array}{c} \# \\ \#z_0abc\# \end{array} \right] & \left[\begin{array}{c} z_0a \\ dz_1 \end{array} \right] & \left[\begin{array}{c} b \\ b \end{array} \right] & \left[\begin{array}{c} c \\ c \end{array} \right] & \left[\begin{array}{c} \# \\ \# \end{array} \right] & \left[\begin{array}{c} d \\ d \end{array} \right] & \left[\begin{array}{c} z_1b \\ ez_2 \end{array} \right] & \left[\begin{array}{c} c \\ c \end{array} \right] & \left[\begin{array}{c} \# \\ \# \end{array} \right] & \left[\begin{array}{c} d \\ d \end{array} \right] & \left[\begin{array}{c} e \\ e \end{array} \right] & \left[\begin{array}{c} z_2c \\ fz_3 \end{array} \right] & \left[\begin{array}{c} \# \\ \# \end{array} \right] & \left[\begin{array}{c} d \\ d \end{array} \right] & \left[\begin{array}{c} e \\ e \end{array} \right] & \left[\begin{array}{c} f \\ f \end{array} \right] & \left[\begin{array}{c} z_3\# \\ z_e\Box\# \end{array} \right] \\
 \left[\begin{array}{c} d \\ d \end{array} \right] & \left[\begin{array}{c} e \\ e \end{array} \right] & \left[\begin{array}{c} f \\ f \end{array} \right] & \left[\begin{array}{c} z_e\Box \\ z_e \end{array} \right] & \left[\begin{array}{c} \# \\ \# \end{array} \right] & \left[\begin{array}{c} d \\ d \end{array} \right] & \left[\begin{array}{c} e \\ e \end{array} \right] & \left[\begin{array}{c} fz_e \\ z_e \end{array} \right] & \left[\begin{array}{c} \# \\ \# \end{array} \right] & \left[\begin{array}{c} d \\ d \end{array} \right] & \left[\begin{array}{c} ez_e \\ z_e \end{array} \right] & \left[\begin{array}{c} \# \\ \# \end{array} \right] & \left[\begin{array}{c} dz_e \\ z_e \end{array} \right] & \left[\begin{array}{c} \# \\ \# \end{array} \right] & \left[\begin{array}{c} z_e\#\# \\ \# \end{array} \right]
 \end{array}$$

Reduktion von H auf MPCP

Beweis (Fortsetzung)

Wir müssen zeigen:

die DTM M_w auf Eingabe x anhält g.d.w. $f(w\#x)$ genau dann MPCP-lösbar ist.

$\Rightarrow$ Wenn M_w einen akzeptierenden Lauf hat, dann gibt es eine Folge

$$K_0 \vdash K_1 \vdash \dots \vdash K_n$$

wobei $K_0 = z_0x$ und $K_n = uz_ev$ für ein $z_e \in E$.

Reduktion von H auf MPCP

Beweis (Fortsetzung)

Wir müssen zeigen:

die DTM M_w auf Eingabe x anhält g.d.w. $f(w\#x)$ genau dann MPCP-lösbar ist.

$\Rightarrow$ Wenn M_w einen akzeptierenden Lauf hat, dann gibt es eine Folge

$$K_0 \vdash K_1 \vdash \dots \vdash K_n$$

wobei $K_0 = z_0x$ und $K_n = uz_ev$ für ein $z_e \in E$.

Dann hat das MPCP eine Lösung, die oben und unten das Wort

$$\#K_0\#K_1\#\dots\#K_n\#K_{n+1}\#\dots\#K_m\#\#$$

erzeugt, wobei $K_m = z_e$ und jedes K_i mit $i \in \{n+1, \dots, m\}$ jeweils aus K_{i-1} entsteht durch Löschen eines der benachbarten Zeichen von z_e in $u'z_ev'$ entsteht.

Beweis (Fortsetzung)

Die obere Folge hinkt der unteren um eine Konfiguration hinterher:

$$\begin{array}{c} \#K_1\#K_2\#\cdots\#K_i\# \\ \#K_1\#K_2\#\cdots\#K_i\#K_{i+1}\# \end{array}$$

Beweis (Fortsetzung)

Die obere Folge hinkt der unteren um eine Konfiguration hinterher:

$$\begin{array}{c} \#K_1\#K_2\#\cdots\#K_i\# \\ \#K_1\#K_2\#\cdots\#K_i\#K_{i+1}\# \end{array}$$

Verlängerung bis K_n :

1. Wende **Kopierregeln** an bis in die Nähe des Zustands.
2. Wende **Übergangsregeln** an.
3. Wende **Kopierregeln** an zum Vervollständigen

Verlängerung ab K_n :

1. **Löschregeln** anwenden, um die Symbole auf dem Band zu löschen.
2. Wenn in unterer Folge $z_e\#$ steht, dann **Abschlussregel** anwenden.

Beweis (Fortsetzung)

- ⇐ Jede **Lösung für das MPCP** (welches ja mit dem ersten Spielstein beginnen muss) erzeugt einen **akzeptierende Lauf**, der bezeugt, dass die Turingmaschine bei Eingabe x hält.

Beweis (Fortsetzung)

← Jede **Lösung für das MPCP** (welches ja mit dem ersten Spielstein beginnen muss) erzeugt einen **akzeptierende Lauf**, der bezeugt, dass die Turingmaschine bei Eingabe x hält.

Wegen der Kopienregeln können MPCP-Lösungen Wiederholungen von der Form **$\#K\#K\#$** enthalten. Diese können vereinfacht werden, um einen Lauf zu erhalten.

Beweis (Fortsetzung)

← Jede **Lösung für das MPCP** (welches ja mit dem ersten Spielstein beginnen muss) erzeugt einen **akzeptierende Lauf**, der bezeugt, dass die Turingmaschine bei Eingabe x hält.

Wegen der Kopienregeln können MPCP-Lösungen Wiederholungen von der Form **$\#K\#K\#$** enthalten. Diese können vereinfacht werden, um einen Lauf zu erhalten.

Eine MPCP-Lösung kann auch mehreren Läufen hintereinander entsprechen. Dann betrachten wir nur den ersten Lauf.

Beweis (Fortsetzung)

← Jede **Lösung für das MPCP** (welches ja mit dem ersten Spielstein beginnen muss) erzeugt einen **akzeptierende Lauf**, der bezeugt, dass die Turingmaschine bei Eingabe x hält.

Wegen der Kopienregeln können MPCP-Lösungen Wiederholungen von der Form **$\#K\#K\#$** enthalten. Diese können vereinfacht werden, um einen Lauf zu erhalten.

Eine MPCP-Lösung kann auch mehreren Läufen hintereinander entsprechen. Dann betrachten wir nur den ersten Lauf.

Schließlich prüfen wir, dass f total und berechenbar ist.



Satz

Das Postsche Korrespondenzproblem (sowie das modifizierte Postsche Korrespondenzproblem) ist unentscheidbar.

Satz

Das Postsche Korrespondenzproblem (sowie das modifizierte Postsche Korrespondenzproblem) ist unentscheidbar.

Beweis Da H unentscheidbar ist und $H \leq \text{MPCP} \leq \text{PCP}$ gilt, folgt, dass MPCP und PCP unentscheidbar sind. $\square$

PCP für binäre Alphabete

Lemma

Das Postsche Korrespondenzproblem über dem Alphabet Σ mit $|\Sigma| = 2$ (01-PCP) ist unentscheidbar.

PCP für binäre Alphabete

Lemma

Das Postsche Korrespondenzproblem über dem Alphabet Σ mit $|\Sigma| = 2$ (01-PCP) ist unentscheidbar.

Beweis Wir reduzieren PCP auf 01-PCP.

PCP für binäre Alphabete

Lemma

Das Postsche Korrespondenzproblem über dem Alphabet Σ mit $|\Sigma| = 2$ (01-PCP) ist unentscheidbar.

Beweis Wir reduzieren PCP auf 01-PCP.

Sei $K = ((x_1, y_1), \dots, (x_k, y_k))$ eine Instanz des PCP über dem Alphabet $\{a_1, \dots, a_j\}$.

PCP für binäre Alphabete

Lemma

Das Postsche Korrespondenzproblem über dem Alphabet Σ mit $|\Sigma| = 2$ (01-PCP) ist unentscheidbar.

Beweis Wir reduzieren PCP auf 01-PCP.

Sei $K = ((x_1, y_1), \dots, (x_k, y_k))$ eine Instanz des PCP über dem Alphabet $\{a_1, \dots, a_j\}$.
O.B.d.A. sei $\Sigma = \{0, 1\}$.

PCP für binäre Alphabete

Lemma

Das Postsche Korrespondenzproblem über dem Alphabet Σ mit $|\Sigma| = 2$ (01-PCP) ist unentscheidbar.

Beweis Wir reduzieren PCP auf 01-PCP.

Sei $K = ((x_1, y_1), \dots, (x_k, y_k))$ eine Instanz des PCP über dem Alphabet $\{a_1, \dots, a_j\}$.

O.B.d.A. sei $\Sigma = \{0, 1\}$.

Sei $f(a_i) = 10^i$,

$f(\varepsilon) = \varepsilon$, $f(a_i w) = f(a_i) f(w)$ und

$f(K) = (f(x_1), f(y_1)), \dots, (f(x_k), f(y_k))$.

PCP für binäre Alphabete

Lemma

Das Postsche Korrespondenzproblem über dem Alphabet Σ mit $|\Sigma| = 2$ (01-PCP) ist unentscheidbar.

Beweis Wir reduzieren PCP auf 01-PCP.

Sei $K = ((x_1, y_1), \dots, (x_k, y_k))$ eine Instanz des PCP über dem Alphabet $\{a_1, \dots, a_j\}$.

O.B.d.A. sei $\Sigma = \{0, 1\}$.

Sei $f(a_i) = 10^i$,

$f(\varepsilon) = \varepsilon$, $f(a_i w) = f(a_i) f(w)$ und

$f(K) = (f(x_1), f(y_1)), \dots, (f(x_k), f(y_k))$.

Dann gilt: $i_1, \dots, i_n$ ist eine PCP-Lösung für K g.d.w. $i_1, \dots, i_n$ eine 01-PCP-Lösung für $f(K)$ ist.

PCP für binäre Alphabete

Lemma

Das Postsche Korrespondenzproblem über dem Alphabet Σ mit $|\Sigma| = 2$ (01-PCP) ist unentscheidbar.

Beweis Wir reduzieren PCP auf 01-PCP.

Sei $K = ((x_1, y_1), \dots, (x_k, y_k))$ eine Instanz des PCP über dem Alphabet $\{a_1, \dots, a_j\}$.

O.B.d.A. sei $\Sigma = \{0, 1\}$.

Sei $f(a_i) = 10^i$,

$f(\varepsilon) = \varepsilon$, $f(a_i w) = f(a_i) f(w)$ und

$f(K) = (f(x_1), f(y_1)), \dots, (f(x_k), f(y_k))$.

Dann gilt: $i_1, \dots, i_n$ ist eine PCP-Lösung für K g.d.w. $i_1, \dots, i_n$ eine 01-PCP-Lösung für $f(K)$ ist.

Schließlich ist f total und berechenbar. □

Beispiel für 01-PCP

$$\text{Sei } K = \left(\begin{bmatrix} a_2 a_1 a_1 \\ a_1 a_1 \end{bmatrix}, \begin{bmatrix} a_3 \\ a_3 a_2 a_1 \end{bmatrix}, \begin{bmatrix} a_1 a_2 \\ a_2 a_2 \end{bmatrix} \right)$$

Beispiel für 01-PCP

Sei $K = \left(\begin{bmatrix} a_2 a_1 a_1 \\ a_1 a_1 \end{bmatrix}, \begin{bmatrix} a_3 \\ a_3 a_2 a_1 \end{bmatrix}, \begin{bmatrix} a_1 a_2 \\ a_2 a_2 \end{bmatrix} \right)$

$(2, 1, 3, 1)$ ist eine PCP-Lösung:

$$\begin{bmatrix} a_3 \\ a_3 a_2 a_1 \end{bmatrix} \begin{bmatrix} a_2 a_1 a_1 \\ a_1 a_1 \end{bmatrix} \begin{bmatrix} a_1 a_2 \\ a_2 a_2 \end{bmatrix} \begin{bmatrix} a_2 a_1 a_1 \\ a_1 a_1 \end{bmatrix}$$

Beispiel für 01-PCP

$$\text{Sei } K = \left(\begin{bmatrix} a_2 a_1 a_1 \\ a_1 a_1 \end{bmatrix}, \begin{bmatrix} a_3 \\ a_3 a_2 a_1 \end{bmatrix}, \begin{bmatrix} a_1 a_2 \\ a_2 a_2 \end{bmatrix} \right)$$

$(2, 1, 3, 1)$ ist eine PCP-Lösung:

$$\begin{bmatrix} a_3 \\ a_3 a_2 a_1 \end{bmatrix} \begin{bmatrix} a_2 a_1 a_1 \\ a_1 a_1 \end{bmatrix} \begin{bmatrix} a_1 a_2 \\ a_2 a_2 \end{bmatrix} \begin{bmatrix} a_2 a_1 a_1 \\ a_1 a_1 \end{bmatrix}$$

$$f(K) = \left(\begin{bmatrix} 1001010 \\ 1010 \end{bmatrix}, \begin{bmatrix} 1000 \\ 100010010 \end{bmatrix}, \begin{bmatrix} 10100 \\ 100100 \end{bmatrix} \right)$$

Beispiel für 01-PCP

$$\text{Sei } K = \left(\begin{bmatrix} a_2 a_1 a_1 \\ a_1 a_1 \end{bmatrix}, \begin{bmatrix} a_3 \\ a_3 a_2 a_1 \end{bmatrix}, \begin{bmatrix} a_1 a_2 \\ a_2 a_2 \end{bmatrix} \right)$$

(2, 1, 3, 1) ist eine PCP-Lösung:

$$\begin{bmatrix} a_3 \\ a_3 a_2 a_1 \end{bmatrix} \begin{bmatrix} a_2 a_1 a_1 \\ a_1 a_1 \end{bmatrix} \begin{bmatrix} a_1 a_2 \\ a_2 a_2 \end{bmatrix} \begin{bmatrix} a_2 a_1 a_1 \\ a_1 a_1 \end{bmatrix}$$

$$f(K) = \left(\begin{bmatrix} 1001010 \\ 1010 \end{bmatrix}, \begin{bmatrix} 1000 \\ 100010010 \end{bmatrix}, \begin{bmatrix} 10100 \\ 100100 \end{bmatrix} \right)$$

(2, 1, 3, 1) ist eine 01-PCP-Lösung:

$$\begin{bmatrix} 1000 \\ 100010010 \end{bmatrix} \begin{bmatrix} 1001010 \\ 1010 \end{bmatrix} \begin{bmatrix} 10100 \\ 100100 \end{bmatrix} \begin{bmatrix} 1001010 \\ 1010 \end{bmatrix}$$

PCP für unäre Alphabete

Lemma

Das PCP für unäre Alphabete ist entscheidbar.

PCP für unäre Alphabete

Lemma

Das PCP für unäre Alphabete ist entscheidbar.

Beweis Alle Wortpaare sind von der Form $\begin{bmatrix} a^n \\ a^m \end{bmatrix}$.

PCP für unäre Alphabete

Lemma

Das PCP für unäre Alphabete ist entscheidbar.

Beweis Alle Wortpaare sind von der Form $\begin{bmatrix} a^n \\ a^m \end{bmatrix}$.

Wenn $|x_i| < |y_i|$ für alle (x_i, y_i) gilt, dann gibt es keine Lösung.

Lemma

Das PCP für unäre Alphabete ist entscheidbar.

Beweis Alle Wortpaare sind von der Form $\begin{bmatrix} a^n \\ a^m \end{bmatrix}$.

Wenn $|x_i| < |y_i|$ für alle (x_i, y_i) gilt, dann gibt es keine Lösung.

Wenn $|x_i| > |y_i|$ für alle (x_i, y_i) gilt, dann gibt es keine Lösung.

PCP für unäre Alphabete

Lemma

Das PCP für unäre Alphabete ist entscheidbar.

Beweis Alle Wortpaare sind von der Form $\begin{bmatrix} a^n \\ a^m \end{bmatrix}$.

Wenn $|x_i| < |y_i|$ für alle (x_i, y_i) gilt, dann gibt es keine Lösung.

Wenn $|x_i| > |y_i|$ für alle (x_i, y_i) gilt, dann gibt es keine Lösung.

Wenn $(x_i, y_i) = (a^n, a^{n+r})$ und $(x_j, y_j) = (a^{m+s}, a^m)$ mit $s, r \geq 0$, dann ist das PCP immer lösbar.

Die Lösung ist $\underbrace{i, \dots, i}_{s\text{-mal}}, \underbrace{j, \dots, j}_{r\text{-mal}}$, denn oben $a^{s \cdot n + r \cdot (m+s)}$ und unten $a^{s \cdot (n+r) + r \cdot m}$.

Daher oben wie unten $sn + rm + rs$ viele a 's.



Beispiel für PCP mit unärem Alphabet

$$\text{Sei } K = \left(\begin{bmatrix} a \\ aaaa \end{bmatrix}, \begin{bmatrix} aaa \\ a \end{bmatrix} \right)$$

Beispiel für PCP mit unärem Alphabet

$$\text{Sei } K = \left(\begin{bmatrix} a \\ aaaa \end{bmatrix}, \begin{bmatrix} aaa \\ a \end{bmatrix} \right)$$

(1, 1, 2, 2, 2) ist eine Lösung:

$$\begin{bmatrix} a \\ aaaa \end{bmatrix} \begin{bmatrix} a \\ aaaa \end{bmatrix} \begin{bmatrix} aaa \\ a \end{bmatrix} \begin{bmatrix} aaa \\ a \end{bmatrix} \begin{bmatrix} aaa \\ a \end{bmatrix}$$

Anzahl k der Spielsteinarten beschränken

PCP mit k vielen verschiedenen Spielsteinarten:

- ▶ $k = 1$ und $k = 2$: als entscheidbar gezeigt 1982
- ▶ $k \geq 5$: als unentscheidbar gezeigt 2015
- ▶ $k = 3$ und $k = 4$: unbekannt.

PCP ist semi-entscheidbar:

1. Probiere alle Folgen von i Wortpaaren aus.
2. Lasse i wachsen.

Diese Prozedur findet eine Lösung, wenn eine existiert, in endlich vielen Schritten, aber terminiert nicht, wenn keine Lösung existiert.

Da $H \leq \text{PCP}$ folgt auch, dass H semi-entscheidbar ist.

Da $H \leq \text{PCP}$ folgt auch, dass H semi-entscheidbar ist.

D.h. es gibt eine DTM, die sich bei Eingabe $w\#x$ so verhält wie M_w auf Eingabe x was das Halten betrifft.

Da $H \leq \text{PCP}$ folgt auch, dass H semi-entscheidbar ist.

D.h. es gibt eine DTM, die sich bei Eingabe $w\#x$ so verhält wie M_w auf Eingabe x was das Halten betrifft.

Ferner: Es gibt eine DTM U , die sich bei Eingabe $w\#x$ so verhält wie M_w auf Eingabe x .

Universelle Turingmaschine

Da $H \leq \text{PCP}$ folgt auch, dass H semi-entscheidbar ist.

D.h. es gibt eine DTM, die sich bei Eingabe $w\#x$ so verhält wie M_w auf Eingabe x was das Halten betrifft.

Ferner: Es gibt eine DTM U , die sich bei Eingabe $w\#x$ so verhält wie M_w auf Eingabe x .

U nennt man eine **universelle Turingmaschine**:

- ▶ U verhält sich wie ein Interpreter für Turingmaschinen.
- ▶ U wird durch die Eingabe w programmiert und x ist dann die eigentliche Eingabe für das Programm.

11b

Laufzeitkomplexität und die Klassen $\mathcal{P}$ und $\mathcal{NP}$

Prof. Dr. Jasmin Blanchette

Lehr- und Forschungseinheit für
Theoretische Informatik und Theorembeweisen

Stand: 3. Juli 2024
Basierend auf Folien von PD Dr. David Sabel



Die Komplexitätstheorie ist ein Teilgebiet der Theoretischen Informatik, das sich mit der Komplexität von **entscheidbaren Problemen** befasst.

Maße zum Messen des Ressourcenbedarfs von Algorithmen:

- ▶ Rechenzeit
- ▶ Platzbedarf
- ▶ usw.

Die Komplexität eines Problems ist die Komplexität des **besten** Algorithmus bezüglich des Maßes.

Das Ziel ist die Einordnung von Problemen in **Komplexitätsklassen**.

1. Teilbereich: Nachweis von **oberen Schranken**:

- ▶ Finde möglichst guten Algorithmus für konkretes Problem.
- ▶ Analysiere die Laufzeit- und Platzkomplexität.
- ▶ Gesucht sind möglichst genaue Schranken (bezüglich der O -Notation).

2. Teilbereich: Nachweis von **unteren Schranken**:

- ▶ Zeige, dass es keinen besseren Algorithmus gibt.
- ▶ Schwieriger, da man über **alle** Algorithmen argumentieren muss.
- ▶ Gesucht werden möglichst genaue Schranken (bezüglich der Ω -Notation).

3. Teilbereich: Auswirkung der **Maschinenmodelle auf den Ressourcenbedarf** insbesondere **Determinismus vs. Nichtdeterminismus**:

- ▶ Eine wichtige ungelöste Frage ist das **$\mathcal{P}$ -vs.- $\mathcal{NP}$ -Problem**.
- ▶ Komplexitätsklassen sind im Allgemeinen ungenauer (größer) als in den vorher genannten Teilbereichen.
- ▶ Wir beschäftigen uns im letzten Teil der Lehrveranstaltung hiermit.

Turingmaschinen:

- ▶ Wir betrachten nur **entscheidbare** Sprachen.
- ▶ Deswegen nehmen wir an:
Die Turingmaschinen, welche diese Sprachen entscheiden,
halten auf jeder Eingabe an.
- ▶ DTMs: Wir nehmen an, dass es „Verwirf“-Zustände gibt,
für die die Turingmaschine keine Nachfolgekonfiguration besitzen.
- ▶ NTMs haben solche Zustände von Haus aus.

Mehrband- vs. Einband-Turingmaschinen:

- ▶ Wir nehmen **Mehrband-TMs**, da sie flexibler sind.
- ▶ Unterschied: n Schritte auf Mehrband-TM können in $O(n^2)$ Schritten auf Einband-TM ausgeführt werden.
- ▶ Der Unterschied macht sich in den Komplexitätsklassen $\mathcal{P}$ und $\mathcal{NP}$ nicht bemerkbar.

Die Klasse $TIME(f(n))$

Definition

Sei M eine stets anhaltende Mehrband-DTM mit Startzustand z_0 .

Für Eingabe w definieren wir

$$time_M(w) := i$$

wenn für die Startkonfiguration für z_0 und w nach i Schritten ein Endzustand oder Verwirf-Zustand erreichbar ist.

Die Klasse $TIME(f(n))$

Definition

Sei M eine stets anhaltende Mehrband-DTM mit Startzustand z_0 .

Für Eingabe w definieren wir

$$time_M(w) := i$$

wenn für die Startkonfiguration für z_0 und w nach i Schritten ein Endzustand oder Verwirf-Zustand erreichbar ist.

Definition

Für eine Funktion $f : \mathbb{N} \rightarrow \mathbb{N}$ sei die Klasse $TIME(f(n))$ genau die Menge der Sprachen L , für die es eine stets anhaltende Mehrband-DTM M gibt mit $L(M) = L$ und $time_M(w) \leq f(|w|)$ für alle $w \in \Sigma^*$.

Die Klasse $TIME(f(n))$

Definition

Sei M eine stets anhaltende Mehrband-DTM mit Startzustand z_0 .

Für Eingabe w definieren wir

$$time_M(w) := i$$

wenn für die Startkonfiguration für z_0 und w nach i Schritten ein Endzustand oder Verwirf-Zustand erreichbar ist.

Definition

Für eine Funktion $f : \mathbb{N} \rightarrow \mathbb{N}$ sei die Klasse $TIME(f(n))$ genau die Menge der Sprachen L , für die es eine stets anhaltende Mehrband-DTM M gibt mit $L(M) = L$ und $time_M(w) \leq f(|w|)$ für alle $w \in \Sigma^*$.

Eine Sprache ist daher in $TIME(f(n))$, wenn sie von einer DTM für jede Eingabe der Länge n in $\leq f(n)$ Schritten entschieden wird.

Definition

Ein **Polynom** ist eine Funktion $p : \mathbb{N} \rightarrow \mathbb{N}$ von der Form

$$p(n) = \sum_{i=0}^k a_i \cdot n^i = a_k n^k + a_{k-1} n^{k-1} + \dots + a_1 n + a_0$$

mit $a_i \in \mathbb{N}$ und $k \in \mathbb{N}$.

Definition

Die Klasse $\mathcal{P}$ ist definiert als

$$\mathcal{P} := \bigcup_{p \text{ Polynom}} \text{TIME}(p(n))$$

Definition

Die Klasse $\mathcal{P}$ ist definiert als

$$\mathcal{P} := \bigcup_{p \text{ Polynom}} \text{TIME}(p(n))$$

Sprechweise: Problem ist **effizient lösbar** = Problem in $\mathcal{P}$.

Zugehörigkeit zu $\mathcal{P}$

Sprache L ist in Klasse $\mathcal{P}$ enthalten, wenn:

Es gibt eine stets anhaltende DTM M und ein Polynom p mit

- ▶ $L(M) = L$
- ▶ $\text{time}_M(w) \leq p(|w|)$ für jedes $w \in \Sigma^*$.

Nichtdeterminismus

Auch für NTMs nehmen wir an, dass sie auf allen Berechnungspfaden anhalten.

Beachte: Ein Wort wird **akzeptiert**, wenn es auf **einem** Berechnungspfad akzeptiert wird.

Auch für NTMs nehmen wir an, dass sie auf allen Berechnungspfaden anhalten.

Beachte: Ein Wort wird **akzeptiert**, wenn es auf **einem** Berechnungspfad akzeptiert wird.

Definition

Sei M eine stets anhaltende Mehrband-NTM, die für jede Eingabe anhält.
Dann definieren wir die Laufzeit von M als

$$ntime_M(w) := \max \{i \mid \text{für die Startkonfiguration für } z_0 \text{ und } w \text{ hält } M \text{ in } i \text{ Schritten}\}$$

Auch für NTMs nehmen wir an, dass sie auf allen Berechnungspfaden anhalten.

Beachte: Ein Wort wird **akzeptiert**, wenn es auf **einem** Berechnungspfad akzeptiert wird.

Definition

Sei M eine stets anhaltende Mehrband-NTM, die für jede Eingabe anhält.
Dann definieren wir die Laufzeit von M als

$$ntime_M(w) := \max \{i \mid \text{für die Startkonfiguration für } z_0 \text{ und } w \text{ hält } M \text{ in } i \text{ Schritten}\}$$

Beachte: Schöning verwendet eine andere Definition. Es macht für die Definition der Klasse $\mathcal{NP}$ aber keinen Unterschied.

Die Klassen $NTIME(f(n))$ und $\mathcal{NP}$

Definition

Für eine Funktion $f : \mathbb{N} \rightarrow \mathbb{N}$ sei die Klasse $NTIME(f(n))$ genau die Menge der Sprachen L , für die es eine stets anhaltende Mehrband-NTM M gibt mit $L(M) = L$ und $ntime_M(w) \leq f(|w|)$ für alle $w \in \Sigma^*$.

Die Klassen $NTIME(f(n))$ und $\mathcal{NP}$

Definition

Für eine Funktion $f : \mathbb{N} \rightarrow \mathbb{N}$ sei die Klasse $NTIME(f(n))$ genau die Menge der Sprachen L , für die es eine stets anhaltende Mehrband-NTM M gibt mit $L(M) = L$ und $ntime_M(w) \leq f(|w|)$ für alle $w \in \Sigma^*$.

Definition

Die Klasse $\mathcal{NP}$ ist definiert als

$$\mathcal{NP} := \bigcup_{p \text{ Polynom}} NTIME(p(n))$$

$$\mathcal{P} \subseteq \mathcal{NP}$$

Lemma

Es gilt $TIME(f(n)) \subseteq NTIME(f(n))$ und damit auch $\mathcal{P} \subseteq \mathcal{NP}$.

Lemma

Es gilt $TIME(f(n)) \subseteq NTIME(f(n))$ und damit auch $\mathcal{P} \subseteq \mathcal{NP}$.

Beweis Jede DTM kann leicht als NTM aufgefasst werden, wobei es nur einen möglichen Berechnungspfad gibt.

Lemma

Es gilt $TIME(f(n)) \subseteq NTIME(f(n))$ und damit auch $\mathcal{P} \subseteq \mathcal{NP}$.

Beweis Jede DTM kann leicht als NTM aufgefasst werden, wobei es nur einen möglichen Berechnungspfad gibt.

Dieser bestimmt $ntime_M(w)$ und es gilt für diese Turingmaschine $time_M(w) = ntime_M(w)$.

Lemma

Es gilt $TIME(f(n)) \subseteq NTIME(f(n))$ und damit auch $\mathcal{P} \subseteq \mathcal{NP}$.

Beweis Jede DTM kann leicht als NTM aufgefasst werden, wobei es nur einen möglichen Berechnungspfad gibt.

Dieser bestimmt $ntime_M(w)$ und es gilt für diese Turingmaschine $time_M(w) = ntime_M(w)$.

Damit folgt, dass aus $L \in TIME(f(n))$ auch $L \in NTIME(f(n))$ folgt.

Lemma

Es gilt $TIME(f(n)) \subseteq NTIME(f(n))$ und damit auch $\mathcal{P} \subseteq \mathcal{NP}$.

Beweis Jede DTM kann leicht als NTM aufgefasst werden, wobei es nur einen möglichen Berechnungspfad gibt.

Dieser bestimmt $ntime_M(w)$ und es gilt für diese Turingmaschine $time_M(w) = ntime_M(w)$.

Damit folgt, dass aus $L \in TIME(f(n))$ auch $L \in NTIME(f(n))$ folgt.

Schließlich zeigt dies auch $\mathcal{P} \subseteq \mathcal{NP}$. □

Die Frage „Gilt $\mathcal{P} = \mathcal{NP}$ oder $\mathcal{P} \neq \mathcal{NP}$?“ ist bis heute **ungelöst**.

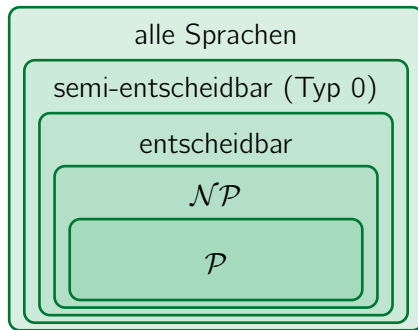
- ▶ Das $\mathcal{P}$ -vs.- $\mathcal{NP}$ -Problem ist eines der sieben sogenannten Millennium-Probleme, die vom Clay Mathematics Institute im Jahr 2000 als Liste ungelöster Probleme der Mathematik herausgegeben wurde.
- ▶ Für dessen Lösung wurde ein Preisgeld von einer Million US-Dollar ausgelobt.

Ein wesentlicher Grund, der für $\mathcal{P} \neq \mathcal{NP}$ spricht:

Man müsste für $\mathcal{P} = \mathcal{NP}$ einen deterministischen Polynomialzeitalgorithmus finden, für ein Problem, für das bisher nur deterministische Exponentialzeitalgorithmen bekannt sind.

Viele haben gesucht, keiner hat einen solchen Algorithmus gefunden.

Lage der Komplexitätsklasse



Dabei unklar, ob $\mathcal{NP} = \mathcal{P}$

11c

$\mathcal{NP}$ -Vollständigkeit

Prof. Dr. Jasmin Blanchette

Lehr- und Forschungseinheit für
Theoretische Informatik und Theorembeweisen

Stand: 3. Juli 2024
Basierend auf Folien von PD Dr. David Sabel



Definition

Für eine Funktion $f : \mathbb{N} \rightarrow \mathbb{N}$ sei die Klasse $TIME(f(n))$ genau die Menge der Sprachen L , für die es eine stets anhaltende Mehrband-DTM M gibt mit $L(M) = L$ und $time_M(w) \leq f(|w|)$ für alle $w \in \Sigma^*$.

Definition

Die Klasse $\mathcal{P}$ ist definiert als

$$\mathcal{P} := \bigcup_{p \text{ Polynom}} TIME(p(n))$$

Definition

Für eine Funktion $f : \mathbb{N} \rightarrow \mathbb{N}$ sei die Klasse $\text{NTIME}(f(n))$ genau die Menge der Sprachen L , für die es eine stets anhaltende Mehrband-NTM M gibt mit $L(M) = L$ und $\text{ntime}_M(w) \leq f(|w|)$ für alle $w \in \Sigma^*$.

Definition

Die Klasse $\mathcal{NP}$ ist definiert als

$$\mathcal{NP} := \bigcup_{p \text{ Polynom}} \text{NTIME}(p(n))$$

Wiederholung: $\mathcal{P}$ vs. $\mathcal{NP}$

Die Frage „Gilt $\mathcal{P} = \mathcal{NP}$ oder $\mathcal{P} \neq \mathcal{NP}$?“ ist bis heute **ungelöst**.

- ▶ $\mathcal{P} \subseteq \mathcal{NP}$ ist klar.
- ▶ Es gibt gute Gründe, $\mathcal{P} \neq \mathcal{NP}$ zu vermuten.

Bedeutung von $\mathcal{P}$ vs. $\mathcal{NP}$

Obwohl man die $\mathcal{P}$ -vs.- $\mathcal{NP}$ -Frage nicht geklärt hat,
will man wissen, wie schwer ein Problem ist:

Bedeutung von $\mathcal{P}$ vs. $\mathcal{NP}$

Obwohl man die $\mathcal{P}$ -vs.- $\mathcal{NP}$ -Frage nicht geklärt hat, will man wissen, wie schwer ein Problem ist:

- ▶ Wenn man weiß, dass das Problem in $\mathcal{P}$ liegt, dann existiert ein **effizienter Algorithmus**.

Bedeutung von $\mathcal{P}$ vs. $\mathcal{NP}$

Obwohl man die $\mathcal{P}$ -vs.- $\mathcal{NP}$ -Frage nicht geklärt hat, will man wissen, wie schwer ein Problem ist:

- ▶ Wenn man weiß, dass das Problem in $\mathcal{P}$ liegt, dann existiert ein **effizienter Algorithmus**.
- ▶ Wenn man nur weiß, dass das Problem in $\mathcal{NP}$ liegt, dann kennt man nur Algorithmen, die in **deterministischer Exponentialzeit laufen**.

Bedeutung von $\mathcal{P}$ vs. $\mathcal{NP}$

Obwohl man die $\mathcal{P}$ -vs.- $\mathcal{NP}$ -Frage nicht geklärt hat, will man wissen, wie schwer ein Problem ist:

- ▶ Wenn man weiß, dass das Problem in $\mathcal{P}$ liegt, dann existiert ein **effizienter Algorithmus**.
- ▶ Wenn man nur weiß, dass das Problem in $\mathcal{NP}$ liegt, dann kennt man nur Algorithmen, die in **deterministischer Exponentialzeit laufen**.

Heute: **$\mathcal{NP}$ -Vollständigkeit**:

Zeige, dass ein gegebenes Problem zu den **schwersten Problemen** in $\mathcal{NP}$ zählt.

Definition

Seien $L_1 \subseteq \Sigma_1^*$ und $L_2 \subseteq \Sigma_2^*$ Sprachen.

Dann sagen wir L_1 ist auf L_2 **polynomiell reduzierbar** (geschrieben $L_1 \leq_p L_2$), falls es eine **totale und in deterministischer Polynomialzeit berechenbare** Funktion $f : \Sigma_1^* \rightarrow \Sigma_2^*$ gibt, sodass für alle $w \in \Sigma_1^*$ gilt: $w \in L_1$ g.d.w. $f(w) \in L_2$. Die Funktion f nennt man **Polynomialzeit-Reduktion**.

Definition

Seien $L_1 \subseteq \Sigma_1^*$ und $L_2 \subseteq \Sigma_2^*$ Sprachen.

Dann sagen wir L_1 ist auf L_2 **polynomiell reduzierbar** (geschrieben $L_1 \leq_p L_2$), falls es eine **totale und in deterministischer Polynomialzeit berechenbare** Funktion $f : \Sigma_1^* \rightarrow \Sigma_2^*$ gibt, sodass für alle $w \in \Sigma_1^*$ gilt: $w \in L_1$ g.d.w. $f(w) \in L_2$. Die Funktion f nennt man **Polynomialzeit-Reduktion**.

Die Definition von $L_1 \leq_p L_2$ ist analog zu der von $L_1 \leq L_2$, mit dem Zusatz, dass f in deterministischer Polynomialzeit berechenbar sein muss.

Polynomialzeit-Reduktion

Definition

Seien $L_1 \subseteq \Sigma_1^*$ und $L_2 \subseteq \Sigma_2^*$ Sprachen.

Dann sagen wir L_1 ist auf L_2 **polynomiell reduzierbar** (geschrieben $L_1 \leq_p L_2$), falls es eine **totale und in deterministischer Polynomialzeit berechenbare** Funktion $f : \Sigma_1^* \rightarrow \Sigma_2^*$ gibt, sodass für alle $w \in \Sigma_1^*$ gilt: $w \in L_1$ g.d.w. $f(w) \in L_2$. Die Funktion f nennt man **Polynomialzeit-Reduktion**.

Die Definition von $L_1 \leq_p L_2$ ist analog zu der von $L_1 \leq L_2$, mit dem Zusatz, dass f in deterministischer Polynomialzeit berechenbar sein muss.

Analogie:

$$\begin{aligned} L_1 \leq L_2 \text{ und } L_2 \text{ (semi-)entscheidbar} \\ \implies L_1 \text{ (semi-)entscheidbar} \end{aligned}$$

$$\begin{aligned} L_1 \leq_p L_2 \text{ und } L_2 \in (\mathcal{N})\mathcal{P} \\ \implies L_1 \in (\mathcal{N})\mathcal{P} \end{aligned}$$

Nachweis der Zugehörigkeit zu $\mathcal{P}$

Lemma

Falls $L_1 \leq_p L_2$ und $L_2 \in \mathcal{P}$, dann gilt $L_1 \in \mathcal{P}$.

Nachweis der Zugehörigkeit zu $\mathcal{P}$

Lemma

Falls $L_1 \leq_p L_2$ und $L_2 \in \mathcal{P}$, dann gilt $L_1 \in \mathcal{P}$.

Beweis Seien $L_1 \leq_p L_2$ und f in Polynomialzeit berechenbar.

Nachweis der Zugehörigkeit zu $\mathcal{P}$

Lemma

Falls $L_1 \leq_p L_2$ und $L_2 \in \mathcal{P}$, dann gilt $L_1 \in \mathcal{P}$.

Beweis Seien $L_1 \leq_p L_2$ und f in Polynomialzeit berechenbar.

Sei M_f die DTM, die f in Polynomialzeit berechnet.

Nachweis der Zugehörigkeit zu $\mathcal{P}$

Lemma

Falls $L_1 \leq_p L_2$ und $L_2 \in \mathcal{P}$, dann gilt $L_1 \in \mathcal{P}$.

Beweis Seien $L_1 \leq_p L_2$ und f in Polynomialzeit berechenbar.

Sei M_f die DTM, die f in Polynomialzeit berechnet.

Seien $L_2 \in \mathcal{P}$ und M_2 eine DTM, sodass $L(M_2) = L_2$,
wobei M_2 stets in deterministischer Polynomialzeit anhält.

Nachweis der Zugehörigkeit zu $\mathcal{P}$

Lemma

Falls $L_1 \leq_p L_2$ und $L_2 \in \mathcal{P}$, dann gilt $L_1 \in \mathcal{P}$.

Beweis Seien $L_1 \leq_p L_2$ und f in Polynomialzeit berechenbar.

Sei M_f die DTM, die f in Polynomialzeit berechnet.

Seien $L_2 \in \mathcal{P}$ und M_2 eine DTM, sodass $L(M_2) = L_2$,
wobei M_2 stets in deterministischer Polynomialzeit anhält.

Sei $M_f; M_2$ die Hintereinanderausführung von M_f und M_2 . Dann gilt $L(M_f; M_2) = L_1$.

Nachweis der Zugehörigkeit zu $\mathcal{P}$

Lemma

Falls $L_1 \leq_p L_2$ und $L_2 \in \mathcal{P}$, dann gilt $L_1 \in \mathcal{P}$.

Beweis Seien $L_1 \leq_p L_2$ und f in Polynomialzeit berechenbar.

Sei M_f die DTM, die f in Polynomialzeit berechnet.

Seien $L_2 \in \mathcal{P}$ und M_2 eine DTM, sodass $L(M_2) = L_2$,
wobei M_2 stets in deterministischer Polynomialzeit anhält.

Sei $M_f; M_2$ die Hintereinanderausführung von M_f und M_2 . Dann gilt $L(M_f; M_2) = L_1$.

$M_f; M_2$ hält stets in deterministischer Polynomialzeit.

Nachweis der Zugehörigkeit zu $\mathcal{P}$

Lemma

Falls $L_1 \leq_p L_2$ und $L_2 \in \mathcal{P}$, dann gilt $L_1 \in \mathcal{P}$.

Beweis Seien $L_1 \leq_p L_2$ und f in Polynomialzeit berechenbar.

Sei M_f die DTM, die f in Polynomialzeit berechnet.

Seien $L_2 \in \mathcal{P}$ und M_2 eine DTM, sodass $L(M_2) = L_2$,
wobei M_2 stets in deterministischer Polynomialzeit anhält.

Sei $M_f; M_2$ die Hintereinanderausführung von M_f und M_2 . Dann gilt $L(M_f; M_2) = L_1$.

$M_f; M_2$ hält stets in deterministischer Polynomialzeit.

Daher gilt $L_1 \in \mathcal{P}$. □

Nachweis der Zugehörigkeit zu $\mathcal{NP}$

Lemma

Falls $L_1 \leq_p L_2$ und $L_2 \in \mathcal{NP}$, dann gilt $L_1 \in \mathcal{NP}$.

Nachweis der Zugehörigkeit zu $\mathcal{NP}$

Lemma

Falls $L_1 \leq_p L_2$ und $L_2 \in \mathcal{NP}$, dann gilt $L_1 \in \mathcal{NP}$.

Der Beweis ist analog:

Beweis Seien $L_1 \leq_p L_2$ und f in Polynomialzeit berechenbar.

Nachweis der Zugehörigkeit zu $\mathcal{NP}$

Lemma

Falls $L_1 \leq_p L_2$ und $L_2 \in \mathcal{NP}$, dann gilt $L_1 \in \mathcal{NP}$.

Der Beweis ist analog:

Beweis Seien $L_1 \leq_p L_2$ und f in Polynomialzeit berechenbar.

Sei M_f die DTM, die f in Polynomialzeit berechnet.

Nachweis der Zugehörigkeit zu $\mathcal{NP}$

Lemma

Falls $L_1 \leq_p L_2$ und $L_2 \in \mathcal{NP}$, dann gilt $L_1 \in \mathcal{NP}$.

Der Beweis ist analog:

Beweis Seien $L_1 \leq_p L_2$ und f in Polynomialzeit berechenbar.

Sei M_f die DTM, die f in Polynomialzeit berechnet.

Seien $L_2 \in \mathcal{NP}$ und M_2 eine NTM, sodass $L(M_2) = L_2$,
wobei M_2 stets in nichtdeterministischer Polynomialzeit anhält.

Nachweis der Zugehörigkeit zu $\mathcal{NP}$

Lemma

Falls $L_1 \leq_p L_2$ und $L_2 \in \mathcal{NP}$, dann gilt $L_1 \in \mathcal{NP}$.

Der Beweis ist analog:

Beweis Seien $L_1 \leq_p L_2$ und f in Polynomialzeit berechenbar.

Sei M_f die DTM, die f in Polynomialzeit berechnet.

Seien $L_2 \in \mathcal{NP}$ und M_2 eine NTM, sodass $L(M_2) = L_2$,
wobei M_2 stets in nichtdeterministischer Polynomialzeit anhält.

Sei $M_f; M_2$ die Hintereinanderausführung von M_f (deterministisch) und M_2 (nichtdeterministisch). Dann gilt: $L(M_f; M_2) = L_1$.

Nachweis der Zugehörigkeit zu $\mathcal{NP}$

Lemma

Falls $L_1 \leq_p L_2$ und $L_2 \in \mathcal{NP}$, dann gilt $L_1 \in \mathcal{NP}$.

Der Beweis ist analog:

Beweis Seien $L_1 \leq_p L_2$ und f in Polynomialzeit berechenbar.

Sei M_f die DTM, die f in Polynomialzeit berechnet.

Seien $L_2 \in \mathcal{NP}$ und M_2 eine NTM, sodass $L(M_2) = L_2$,
wobei M_2 stets in nichtdeterministischer Polynomialzeit anhält.

Sei $M_f; M_2$ die Hintereinanderausführung von M_f (deterministisch) und M_2 (nichtdeterministisch). Dann gilt: $L(M_f; M_2) = L_1$.

$M_f; M_2$ hält stets in nichtdeterministischer Polynomialzeit.

Nachweis der Zugehörigkeit zu $\mathcal{NP}$

Lemma

Falls $L_1 \leq_p L_2$ und $L_2 \in \mathcal{NP}$, dann gilt $L_1 \in \mathcal{NP}$.

Der Beweis ist analog:

Beweis Seien $L_1 \leq_p L_2$ und f in Polynomialzeit berechenbar.

Sei M_f die DTM, die f in Polynomialzeit berechnet.

Seien $L_2 \in \mathcal{NP}$ und M_2 eine NTM, sodass $L(M_2) = L_2$,
wobei M_2 stets in nichtdeterministischer Polynomialzeit anhält.

Sei $M_f; M_2$ die Hintereinanderausführung von M_f (deterministisch) und M_2 (nichtdeterministisch). Dann gilt: $L(M_f; M_2) = L_1$.

$M_f; M_2$ hält stets in nichtdeterministischer Polynomialzeit.

Daher gilt $L_1 \in \mathcal{NP}$. □

Transitivität der Polynomialzeit-Reduktion

Lemma

Die Relation $\leq_p$ ist transitiv, d.h. wenn $L_1 \leq_p L_2$ und $L_2 \leq_p L_3$, dann gilt auch $L_1 \leq_p L_3$.

Transitivität der Polynomialzeit-Reduktion

Lemma

Die Relation $\leq_p$ ist transitiv, d.h. wenn $L_1 \leq_p L_2$ und $L_2 \leq_p L_3$, dann gilt auch $L_1 \leq_p L_3$.

Beweis Die Komposition von zwei Polynomen bleibt ein Polynom. □

Definition

Eine Sprache L heißt $\mathcal{NP}$ -vollständig, wenn gilt

1. $L \in \mathcal{NP}$ und
2. L ist $\mathcal{NP}$ -schwer: für alle $L' \in \mathcal{NP}$ gilt $L' \leq_p L$.

Definition

Eine Sprache L heißt $\mathcal{NP}$ -vollständig, wenn gilt

1. $L \in \mathcal{NP}$ und
2. L ist $\mathcal{NP}$ -schwer: für alle $L' \in \mathcal{NP}$ gilt $L' \leq_p L$.

$\mathcal{NP}$ -vollständige Probleme sind die schwierigsten Probleme in $\mathcal{NP}$.

Definition

Eine Sprache L heißt $\mathcal{NP}$ -vollständig, wenn gilt

1. $L \in \mathcal{NP}$ und
2. L ist $\mathcal{NP}$ -schwer: für alle $L' \in \mathcal{NP}$ gilt $L' \leq_p L$.

$\mathcal{NP}$ -vollständige Probleme sind die schwierigsten Probleme in $\mathcal{NP}$.

$\mathcal{NP}$ -Schwere besagt, dass man mit dem $\mathcal{NP}$ -vollständigen Problem **alle anderen** Probleme aus $\mathcal{NP}$ lösen kann.

Definition

Eine Sprache L heißt $\mathcal{NP}$ -vollständig, wenn gilt

1. $L \in \mathcal{NP}$ und
2. L ist $\mathcal{NP}$ -schwer: für alle $L' \in \mathcal{NP}$ gilt $L' \leq_p L$.

$\mathcal{NP}$ -vollständige Probleme sind die schwierigsten Probleme in $\mathcal{NP}$.

$\mathcal{NP}$ -Schwere besagt, dass man mit dem $\mathcal{NP}$ -vollständigen Problem **alle anderen** Probleme aus $\mathcal{NP}$ lösen kann.

$\mathcal{NP}$ -schwer wird manchmal auch $\mathcal{NP}$ -hart genannt.

Nachweis der $\mathcal{NP}$ -Vollständigkeit

Nachweis der $\mathcal{NP}$ -Vollständigkeit einer Sprache L :

1. Zugehörigkeit zu $\mathcal{NP}$:

Gib eine Polynomialzeit-beschränkte NTM an, die L entscheidet.

(Alternativ: Gib eine Polynomialzeit-Reduktion von $L \leq_p L_1$ an mit $L_1 \in \mathcal{NP}$.)

Nachweis der $\mathcal{NP}$ -Vollständigkeit

Nachweis der $\mathcal{NP}$ -Vollständigkeit einer Sprache L :

1. Zugehörigkeit zu $\mathcal{NP}$:

Gib eine Polynomialzeit-beschränkte NTM an, die L entscheidet.

(Alternativ: Gib eine Polynomialzeit-Reduktion von $L \leq_p L_1$ an mit $L_1 \in \mathcal{NP}$.)

2. $\mathcal{NP}$ -Schwere:

Statt jedes mal neu zu beweisen, dass **alle** Probleme aus $\mathcal{NP}$ auf L polynomiell reduzierbar sind, wähle ein $\mathcal{NP}$ -schweres Problem L_0 und zeige $L_0 \leq_p L$.

Nachweis der $\mathcal{NP}$ -Vollständigkeit

Nachweis der $\mathcal{NP}$ -Vollständigkeit einer Sprache L :

1. Zugehörigkeit zu $\mathcal{NP}$:

Gib eine Polynomialzeit-beschränkte NTM an, die L entscheidet.

(Alternativ: Gib eine Polynomialzeit-Reduktion von $L \leq_p L_1$ an mit $L_1 \in \mathcal{NP}$.)

2. $\mathcal{NP}$ -Schwere:

Statt jedes mal neu zu beweisen, dass **alle** Probleme aus $\mathcal{NP}$ auf L polynomiell reduzierbar sind, wähle ein $\mathcal{NP}$ -schweres Problem L_0 und zeige $L_0 \leq_p L$.

Da L_0 $\mathcal{NP}$ -schwer, gilt $L' \leq_p L_0$ für alle $L' \in \mathcal{NP}$ und damit $L' \leq_p L_0 \leq_p L$ und mit Transitivität: $L' \leq_p L$ für alle $L' \in \mathcal{NP}$.

Nachweis der $\mathcal{NP}$ -Vollständigkeit

Nachweis der $\mathcal{NP}$ -Vollständigkeit einer Sprache L :

1. Zugehörigkeit zu $\mathcal{NP}$:

Gib eine Polynomialzeit-beschränkte NTM an, die L entscheidet.

(Alternativ: Gib eine Polynomialzeit-Reduktion von $L \leq_p L_1$ an mit $L_1 \in \mathcal{NP}$.)

2. $\mathcal{NP}$ -Schwere:

Statt jedes mal neu zu beweisen, dass **alle** Probleme aus $\mathcal{NP}$ auf L polynomiell reduzierbar sind, wähle ein $\mathcal{NP}$ -schweres Problem L_0 und zeige $L_0 \leq_p L$.

Da L_0 $\mathcal{NP}$ -schwer, gilt $L' \leq_p L_0$ für alle $L' \in \mathcal{NP}$ und damit $L' \leq_p L_0 \leq_p L$ und mit Transitivität: $L' \leq_p L$ für alle $L' \in \mathcal{NP}$.

Daher ist L $\mathcal{NP}$ -schwer.

Nachweis der $\mathcal{NP}$ -Schwere

Analog zum Vorgehen wie bei der Unentscheidbarkeit, wesentlicher Unterschied:
Polynomialzeit-Reduktion:

$L_1 \leq L_2$ und L_1 unentscheidbar
 $\implies L_2$ unentscheidbar

$L_1 \leq_p L_2$ und L_1 $\mathcal{NP}$ -schwer
 $\implies L_2$ $\mathcal{NP}$ -schwer

Bedingung für $\mathcal{P} = \mathcal{NP}$

Satz

Sei L ein $\mathcal{NP}$ -vollständiges Problem. Dann gilt $L \in \mathcal{P}$ g.d.w. $\mathcal{P} = \mathcal{NP}$.

Bedingung für $\mathcal{P} = \mathcal{NP}$

Satz

Sei L ein $\mathcal{NP}$ -vollständiges Problem. Dann gilt $L \in \mathcal{P}$ g.d.w. $\mathcal{P} = \mathcal{NP}$.

Beweis

$\Leftarrow$ Offensichtlich.

Bedingung für $\mathcal{P} = \mathcal{NP}$

Satz

Sei L ein $\mathcal{NP}$ -vollständiges Problem. Dann gilt $L \in \mathcal{P}$ g.d.w. $\mathcal{P} = \mathcal{NP}$.

Beweis

$\Leftarrow$ Offensichtlich.

$\Rightarrow$ Sei L $\mathcal{NP}$ -vollständig und $L \in \mathcal{P}$.

Bedingung für $\mathcal{P} = \mathcal{NP}$

Satz

Sei L ein $\mathcal{NP}$ -vollständiges Problem. Dann gilt $L \in \mathcal{P}$ g.d.w. $\mathcal{P} = \mathcal{NP}$.

Beweis

$\Leftarrow$ Offensichtlich.

$\Rightarrow$ Sei L $\mathcal{NP}$ -vollständig und $L \in \mathcal{P}$.

Aus $\mathcal{NP}$ -Schwere von L folgt:

Für alle $L' \in \mathcal{NP}$: $L' \leq_p L$ und damit $L' \in \mathcal{P}$.

Bedingung für $\mathcal{P} = \mathcal{NP}$

Satz

Sei L ein $\mathcal{NP}$ -vollständiges Problem. Dann gilt $L \in \mathcal{P}$ g.d.w. $\mathcal{P} = \mathcal{NP}$.

Beweis

$\Leftarrow$ Offensichtlich.

$\Rightarrow$ Sei L $\mathcal{NP}$ -vollständig und $L \in \mathcal{P}$.

Aus $\mathcal{NP}$ -Schwere von L folgt:

Für alle $L' \in \mathcal{NP}$: $L' \leq_p L$ und damit $L' \in \mathcal{P}$.

Da dies für alle $L' \in \mathcal{NP}$ gilt, folgt $\mathcal{P} = \mathcal{NP}$. □

Bedingung für $\mathcal{P} = \mathcal{NP}$

Satz

Sei L ein $\mathcal{NP}$ -vollständiges Problem. Dann gilt $L \in \mathcal{P}$ g.d.w. $\mathcal{P} = \mathcal{NP}$.

Beweis

$\Leftarrow$ Offensichtlich.

$\Rightarrow$ Sei L $\mathcal{NP}$ -vollständig und $L \in \mathcal{P}$.

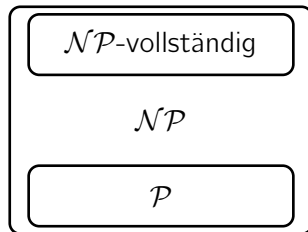
Aus $\mathcal{NP}$ -Schwere von L folgt:

Für alle $L' \in \mathcal{NP}$: $L' \leq_p L$ und damit $L' \in \mathcal{P}$.

Da dies für alle $L' \in \mathcal{NP}$ gilt, folgt $\mathcal{P} = \mathcal{NP}$. □

Also: Es reicht aus nachzuweisen, dass **ein** $\mathcal{NP}$ -vollständiges Problem in $\mathcal{P}$ bzw. nicht in $\mathcal{P}$ liegt, um die $\mathcal{P}$ -vs.- $\mathcal{NP}$ -Frage ein für allemal beantworten zu können.

Vermutete Lage der Probleme



Unter der Annahme $\mathcal{P} \neq \mathcal{NP}$ gibt es Probleme in $\mathcal{NP}$, die nicht in $\mathcal{P}$ liegen und nicht $\mathcal{NP}$ -vollständig sind (Ladner 1975).

Was fehlt noch? Ein erstes Problem L_0 , dass man direkt als $\mathcal{NP}$ -vollständig beweist. Ein solches L_0 und den $\mathcal{NP}$ -Vollständigkeitsbeweis sehen wir in der nächsten Vorlesung.

Was fehlt noch? Ein erstes Problem L_0 , dass man direkt als $\mathcal{NP}$ -vollständig beweist. Ein solches L_0 und den $\mathcal{NP}$ -Vollständigkeitsbeweis sehen wir in der nächsten Vorlesung.

Danach können wir $\mathcal{NP}$ -Vollständigkeit von L zeigen durch

1. $L \in \mathcal{NP}$
2. $L_0 \leq_p L$.

Was fehlt noch? Ein erstes Problem L_0 , dass man direkt als $\mathcal{NP}$ -vollständig beweist. Ein solches L_0 und den $\mathcal{NP}$ -Vollständigkeitsbeweis sehen wir in der nächsten Vorlesung.

Danach können wir $\mathcal{NP}$ -Vollständigkeit von L zeigen durch

1. $L \in \mathcal{NP}$
2. $L_0 \leq_p L$.

Danach lernen wir eine Auswahl an $\mathcal{NP}$ -vollständigen Problemen kennen.

12a

$\mathcal{NP}$ -Vollständigkeit von SAT

Prof. Dr. Jasmin Blanchette

Lehr- und Forschungseinheit für
Theoretische Informatik und Theorembeweisen

Stand: 9. Juli 2024

Basierend auf Folien von PD Dr. David Sabel und Dr. Jan Johannsen



Wiederholung: $\mathcal{NP}$ -Vollständigkeit

Definition

Eine Sprache L heißt $\mathcal{NP}$ -vollständig, wenn gilt

1. $L \in \mathcal{NP}$ und
2. L ist $\mathcal{NP}$ -schwer: für alle $L' \in \mathcal{NP}$ gilt $L' \leq_p L$.

Wiederholung: $\mathcal{NP}$ -Vollständigkeit

Definition

Eine Sprache L heißt $\mathcal{NP}$ -vollständig, wenn gilt

1. $L \in \mathcal{NP}$ und
2. L ist $\mathcal{NP}$ -schwer: für alle $L' \in \mathcal{NP}$ gilt $L' \leq_p L$.

Beweistechnik für $\mathcal{NP}$ -Vollständigkeit von L :

1. Zeige $L \in \mathcal{NP}$.
2. Zeige $L_0 \leq_p L$ für ein bekanntes $\mathcal{NP}$ -vollständiges Problem L_0 .
Dann folgt die $\mathcal{NP}$ -Schwere von L .

Wir brauchen ein erstes Problem, dessen $\mathcal{NP}$ -Vollständigkeit wir von Hand nachweisen müssen.

Dafür nehmen wir das [Erfüllbarkeitsproblem der Aussagenlogik](#) (kurz [SAT](#)).

Definition

Die **aussagenlogischen Formeln** über einer Menge X von **Variablen** sind durch folgende Regeln definiert:

- ▶ Jede Variable $x \in X$ ist eine Formel.
- ▶ Ist F eine Formel, dann auch $\neg F$.
- ▶ Sind F und G Formeln, dann auch $(F \wedge G)$, $(F \vee G)$, $(F \implies G)$ und $(F \iff G)$.

Definition

Sei $I : X \rightarrow \{0, 1\}$ eine Belegung der Variablen X mit den Wahrheitswerten 0 („falsch“) und 1 („wahr“).

Definition

Sei $I : X \rightarrow \{0, 1\}$ eine Belegung der Variablen X mit den Wahrheitswerten 0 („falsch“) und 1 („wahr“).

Der Wert $I(F)$ einer gegebenen Formel F wird durch folgende Regeln definiert:

- ▶ $I(x)$ ist durch I gegeben.
- ▶ $I(\neg F) := 1 - I(F)$.
- ▶ $I(F \wedge G) := \min \{I(F), I(G)\}$.
- ▶ $I(F \vee G) := \max \{I(F), I(G)\}$.
- ▶ $I(F \implies G) := I(\neg F \vee G)$.
- ▶ $I(F \iff G) := I((F \implies G) \wedge (G \implies F))$.

Definition

Sei $I : X \rightarrow \{0, 1\}$ eine Belegung der Variablen X mit den Wahrheitswerten 0 („falsch“) und 1 („wahr“).

Der Wert $I(F)$ einer gegebenen Formel F wird durch folgende Regeln definiert:

- ▶ $I(x)$ ist durch I gegeben.
- ▶ $I(\neg F) := 1 - I(F)$.
- ▶ $I(F \wedge G) := \min \{I(F), I(G)\}$.
- ▶ $I(F \vee G) := \max \{I(F), I(G)\}$.
- ▶ $I(F \implies G) := I(\neg F \vee G)$.
- ▶ $I(F \iff G) := I((F \implies G) \wedge (G \implies F))$.

I erfüllt F , wenn $I(F) = 1$.

F ist erfüllbar, wenn es eine Belegung I gibt mit $I(F) = 1$.

Definition

Das **Erfüllbarkeitsproblem der Aussagenlogik** (kurz **SAT**) lässt sich wie folgt formulieren.

gegeben: eine aussagenlogische Formel F

gefragt: Ist F erfüllbar, d.h. gibt es eine erfüllende Belegung der Variablen, sodass F den Wert 1 erhält?

Definition

Das **Erfüllbarkeitsproblem der Aussagenlogik** (kurz **SAT**) lässt sich wie folgt formulieren.

gegeben: eine aussagenlogische Formel F

gefragt: Ist F erfüllbar, d.h. gibt es eine erfüllende Belegung der Variablen, sodass F den Wert 1 erhält?

Als formale Sprache:

$\text{SAT} = \{\text{code}(F) \in \Sigma^* \mid F \text{ ist eine erfüllbare Formel der Aussagenlogik}\}$

Zugehörigkeit von SAT zu $\mathcal{NP}$

Lemma

$\text{SAT} \in \mathcal{NP}$.

Zugehörigkeit von SAT zu $\mathcal{NP}$

Lemma

$\text{SAT} \in \mathcal{NP}$.

Beweis Konstruiere eine NTM M mit $\text{code}(F)$ als Eingabe.

Zugehörigkeit von SAT zu $\mathcal{NP}$

Lemma

$\text{SAT} \in \mathcal{NP}$.

Beweis Konstruiere eine NTM M mit $\text{code}(F)$ als Eingabe.

M berechnet, welche Variablen in F vorkommen. Seien diese $X = \{x_1, \dots, x_n\}$.

Zugehörigkeit von SAT zu $\mathcal{NP}$

Lemma

$\text{SAT} \in \mathcal{NP}$.

Beweis Konstruiere eine NTM M mit $\text{code}(F)$ als Eingabe.

M berechnet, welche Variablen in F vorkommen. Seien diese $X = \{x_1, \dots, x_n\}$.

M verwendet Nichtdeterminismus, um eine Belegung $I : X \rightarrow \{0, 1\}$ zu „raten“.

Zugehörigkeit von SAT zu $\mathcal{NP}$

Lemma

$\text{SAT} \in \mathcal{NP}$.

Beweis Konstruiere eine NTM M mit $\text{code}(F)$ als Eingabe.

M berechnet, welche Variablen in F vorkommen. Seien diese $X = \{x_1, \dots, x_n\}$.

M verwendet Nichtdeterminismus, um eine Belegung $I : X \rightarrow \{0, 1\}$ zu „raten“.

Jede der 2^n nichtdeterministischen Berechnungen berechnet einen Wert von $I(F)$.

Zugehörigkeit von SAT zu $\mathcal{NP}$

Lemma

$\text{SAT} \in \mathcal{NP}$.

Beweis Konstruiere eine NTM M mit $\text{code}(F)$ als Eingabe.

M berechnet, welche Variablen in F vorkommen. Seien diese $X = \{x_1, \dots, x_n\}$.

M verwendet Nichtdeterminismus, um eine Belegung $I : X \rightarrow \{0, 1\}$ zu „raten“.

Jede der 2^n nichtdeterministischen Berechnungen berechnet einen Wert von $I(F)$.

Akzeptanz, wenn $I(F) = 1$, sonst verwerfen.

Zugehörigkeit von SAT zu $\mathcal{NP}$

Lemma

$\text{SAT} \in \mathcal{NP}$.

Beweis Konstruiere eine NTM M mit $\text{code}(F)$ als Eingabe.

M berechnet, welche Variablen in F vorkommen. Seien diese $X = \{x_1, \dots, x_n\}$.

M verwendet Nichtdeterminismus, um eine Belegung $I : X \rightarrow \{0, 1\}$ zu „raten“.

Jede der 2^n nichtdeterministischen Berechnungen berechnet einen Wert von $I(F)$.

Akzeptanz, wenn $I(F) = 1$, sonst verwerfen.

Da jede Belegung geprüft wird, gilt:

M akzeptiert eine Formel F g.d.w. $\text{code}(F) \in \text{SAT}$.

Zugehörigkeit von SAT zu $\mathcal{NP}$

Lemma

$\text{SAT} \in \mathcal{NP}$.

Beweis Konstruiere eine NTM M mit $\text{code}(F)$ als Eingabe.

M berechnet, welche Variablen in F vorkommen. Seien diese $X = \{x_1, \dots, x_n\}$.

M verwendet Nichtdeterminismus, um eine Belegung $I : X \rightarrow \{0, 1\}$ zu „raten“.

Jede der 2^n nichtdeterministischen Berechnungen berechnet einen Wert von $I(F)$.

Akzeptanz, wenn $I(F) = 1$, sonst verwerfen.

Da jede Belegung geprüft wird, gilt:

M akzeptiert eine Formel F g.d.w. $\text{code}(F) \in \text{SAT}$.

Jeder Berechnungspfad von M läuft in Polynomialzeit in $|\text{code}(F)|$,
da die Anzahl der Variablen durch die Eingabegröße beschränkt ist. □

Polynomielle Verifizierbarkeit als Beweistechnik

Der Nachweis, dass eine Sprache in $\mathcal{NP}$ liegt, geht oft so wie bei SAT:

1. Verwende Nichtdeterminismus, um potentielle Lösung zu raten.
2. Zeige, dass eine Lösung in Polynomialzeit verifiziert werden kann.

$\mathcal{NP}$ -Schwere von SAT

Wir müssen zeigen:

$$L \leq_p \text{SAT für alle } L \in \mathcal{NP}$$

$\mathcal{NP}$ -Schwere von SAT

Wir müssen zeigen:

$$L \leq_p \text{SAT für alle } L \in \mathcal{NP}$$

Da $L \in \mathcal{NP}$, gibt es eine polynomiell zeitbeschränkte NTM M , die L akzeptiert.

$\mathcal{NP}$ -Schwere von SAT

Wir müssen zeigen:

$$L \leq_p \text{SAT für alle } L \in \mathcal{NP}$$

Da $L \in \mathcal{NP}$, gibt es eine polynomiell zeitbeschränkte NTM M , die L akzeptiert.
Für Wort w erstelle Formel $f(w) = F$ (in deterministischer Polynomialzeit), sodass gilt:

F ist erfüllbar g.d.w. M akzeptiert w

$\mathcal{NP}$ -Schwere von SAT

Wir müssen zeigen:

$$L \leq_p \text{SAT für alle } L \in \mathcal{NP}$$

Da $L \in \mathcal{NP}$, gibt es eine polynomiell zeitbeschränkte NTM M , die L akzeptiert.
Für Wort w erstelle Formel $f(w) = F$ (in deterministischer Polynomialzeit), sodass gilt:

F ist erfüllbar g.d.w. M akzeptiert w

In den nächsten Folien werden wir F definieren und zeigen, dass sie die erwünschte Eigenschaft hat.

$\mathcal{NP}$ -Schwere von SAT

Wir müssen zeigen:

$$L \leq_p \text{SAT für alle } L \in \mathcal{NP}$$

Da $L \in \mathcal{NP}$, gibt es eine polynomiell zeitbeschränkte NTM M , die L akzeptiert. Für Wort w erstelle Formel $f(w) = F$ (in deterministischer Polynomialzeit), sodass gilt:

F ist erfüllbar g.d.w. M akzeptiert w

In den nächsten Folien werden wir F definieren und zeigen, dass sie die erwünschte Eigenschaft hat.

F wird das Verhalten von M auf w kodieren. Aus einer erfüllende Belegung I für F wird sich einen akzeptierenden Lauf von M ablesen lassen und umgekehrt.

Hilfssatz für den $\mathcal{NP}$ -Schwere-Beweis von SAT

Lemma

Für aussagenlogische Variablen $\{x_1, \dots, x_n\}$ gibt es eine aussagenlogische Formel $\text{exactlyOne}(x_1, \dots, x_n)$, sodass

$I(\text{exactlyOne}(x_1, \dots, x_n)) = 1$ g.d.w. I setzt genau eine der Variablen x_i auf 1
und alle anderen auf 0

Dabei ist die Größe der Formel $\text{exactlyOne}(x_1, \dots, x_n)$ in $O(n^2)$.

Hilfssatz für den $\mathcal{NP}$ -Schwere-Beweis von SAT

Lemma

Für aussagenlogische Variablen $\{x_1, \dots, x_n\}$ gibt es eine aussagenlogische Formel $exactlyOne(x_1, \dots, x_n)$, sodass

$I(exactlyOne(x_1, \dots, x_n)) = 1$ g.d.w. I setzt genau eine der Variablen x_i auf 1 und alle anderen auf 0

Dabei ist die Größe der Formel $exactlyOne(x_1, \dots, x_n)$ in $O(n^2)$.

Beweis Wir nehmen

$$exactlyOne(x_1, \dots, x_n) := (x_1 \vee \dots \vee x_n) \wedge \bigwedge_{1 \leq i < j \leq n} \neg(x_i \wedge x_j)$$

Hilfssatz für den $\mathcal{NP}$ -Schwere-Beweis von SAT

Lemma

Für aussagenlogische Variablen $\{x_1, \dots, x_n\}$ gibt es eine aussagenlogische Formel $\text{exactlyOne}(x_1, \dots, x_n)$, sodass

$I(\text{exactlyOne}(x_1, \dots, x_n)) = 1$ g.d.w. I setzt genau eine der Variablen x_i auf 1 und alle anderen auf 0

Dabei ist die Größe der Formel $\text{exactlyOne}(x_1, \dots, x_n)$ in $O(n^2)$.

Beweis Wir nehmen

$$\text{exactlyOne}(x_1, \dots, x_n) := (x_1 \vee \dots \vee x_n) \wedge \bigwedge_{1 \leq i < j \leq n} \neg(x_i \wedge x_j)$$

► $(x_1 \vee \dots \vee x_n)$ sichert „mindestens 1“ zu.

► $\bigwedge_{1 \leq i < j \leq n} \neg(x_i \wedge x_j)$ sichert „höchstens 1“ zu.

□

$\mathcal{NP}$ -Schwere von SAT

Lemma

SAT ist $\mathcal{NP}$ -schwer.

$\mathcal{NP}$ -Schwere von SAT

Lemma

SAT ist $\mathcal{NP}$ -schwer.

Beweis Sei $L \in \mathcal{NP}$ beliebig. Wir müssen zeigen: $L \leq_p \text{SAT}$.

$\mathcal{NP}$ -Schwere von SAT

Lemma

SAT ist $\mathcal{NP}$ -schwer.

Beweis Sei $L \in \mathcal{NP}$ beliebig. Wir müssen zeigen: $L \leq_p \text{SAT}$.

Sei M eine NTM mit $L(M) = L$, w eine Eingabe für M und $\text{ntime}_M(w) \leq p(|w|)$.

$\mathcal{NP}$ -Schwere von SAT

Lemma

SAT ist $\mathcal{NP}$ -schwer.

Beweis Sei $L \in \mathcal{NP}$ beliebig. Wir müssen zeigen: $L \leq_p \text{SAT}$.

Sei M eine NTM mit $L(M) = L$, w eine Eingabe für M und $\text{ntime}_M(w) \leq p(|w|)$.

Wir werden eine aussagenlogische Formel F konstruieren, sodass gilt

F ist erfüllbar g.d.w. M akzeptiert w

Dabei muss F in Polynomialzeit konstruierbar sein.

$\mathcal{NP}$ -Schwere von SAT

Lemma

SAT ist $\mathcal{NP}$ -schwer.

Beweis Sei $L \in \mathcal{NP}$ beliebig. Wir müssen zeigen: $L \leq_p \text{SAT}$.

Sei M eine NTM mit $L(M) = L$, w eine Eingabe für M und $\text{ntime}_M(w) \leq p(|w|)$.

Wir werden eine aussagenlogische Formel F konstruieren, sodass gilt

F ist erfüllbar g.d.w. M akzeptiert w

Dabei muss F in Polynomialzeit konstruierbar sein.

D.h. wir geben eine in Polynomialzeit berechenbare Funktion $f(w)$ an, sodass $f(w) \in \text{SAT}$ g.d.w. M akzeptiert w .

$\mathcal{NP}$ -Schwere von SAT

Notationen:

Eingabe: $w = a_1 \cdots a_n \in \Sigma^*$

Zustände: $Z = \{z_0, \dots, z_k\}$

Bandalphabet: $\Gamma = \{b_1, \dots, b_\ell\}$

Startzustand: z_0

$\mathcal{NP}$ -Schwere von SAT

Notationen:

Eingabe:	$w = a_1 \cdots a_n \in \Sigma^*$	Zustände:	$Z = \{z_0, \dots, z_k\}$
Bandalphabet:	$\Gamma = \{b_1, \dots, b_\ell\}$	Startzustand:	z_0

Aussagenlogische Variablen in der Formel F :

Variable	Index-Bereich	Bedeutung
$State_{t,z}$	$t = 0, 1, \dots, p(n)$ $z = z_0, \dots, z_k$	$State_{t,z} = 1$ g.d.w. nach t Schritten ist M im Zustand z .
$Pos_{t,i}$	$t = 0, 1, \dots, p(n)$ $i = -p(n), \dots, p(n)$	$Pos_{t,i} = 1$ g.d.w. nach t Schritten ist der Schreib-Lesekopf auf Position i
$Tape_{t,i,b}$	$t = 0, 1, \dots, p(n)$ $i = -p(n), \dots, p(n)$ $b = b_1, \dots, b_\ell$	$Tape_{t,i,b} = 1$ g.d.w. nach t Schritten steht in Position i das Zeichen b

$\mathcal{NP}$ -Schwere von SAT

Notationen:

Eingabe:	$w = a_1 \cdots a_n \in \Sigma^*$	Zustände:	$Z = \{z_0, \dots, z_k\}$
Bandalphabet:	$\Gamma = \{b_1, \dots, b_\ell\}$	Startzustand:	z_0

Aussagenlogische Variablen in der Formel F :

Variable	Index-Bereich	Bedeutung
$State_{t,z}$	$t = 0, 1, \dots, p(n)$ $z = z_0, \dots, z_k$	$State_{t,z} = 1$ g.d.w. nach t Schritten ist M im Zustand z .
$Pos_{t,i}$	$t = 0, 1, \dots, p(n)$ $i = -p(n), \dots, p(n)$	$Pos_{t,i} = 1$ g.d.w. nach t Schritten ist der Schreib-Lesekopf auf Position i
$Tape_{t,i,b}$	$t = 0, 1, \dots, p(n)$ $i = -p(n), \dots, p(n)$ $b = b_1, \dots, b_\ell$	$Tape_{t,i,b} = 1$ g.d.w. nach t Schritten steht in Position i das Zeichen b

Die Bereiche reichen aus, da die TM nicht mehr als $p(n)$ Schritte macht.

Aufbau der Formel F :

$$F = \textit{Rand} \wedge \textit{Anfang} \wedge \textit{Übergang} \wedge \textit{Ende}$$

- ▶ *Rand*: Randbedingungen
- ▶ *Anfang*: Anfangsbedingungen
- ▶ *Übergang*: Bedingungen für den Zustandsübergang
- ▶ *Ende*: Endbedingung

Randbedingungen

Zu jedem Zeitpunkt t

Randbedingungen

Zu jedem Zeitpunkt t

- ... ist M in genau einem Zustand z :

$$\bigwedge_{t \in \{0, \dots, p(n)\}} \text{exactlyOne}(\text{State}_{t,z_0}, \dots, \text{State}_{t,z_k})$$

Randbedingungen

Zu jedem Zeitpunkt t

- ... ist M in genau einem Zustand z :

$$\bigwedge_{t \in \{0, \dots, p(n)\}} \text{exactlyOne}(\text{State}_{t, z_0}, \dots, \text{State}_{t, z_k})$$

- ... ist der Schreib-Lesekopf von M in genau einer Position auf dem Band:

$$\bigwedge_{t \in \{0, \dots, p(n)\}} \text{exactlyOne}(\text{Pos}_{t, -p(n)}, \dots, \text{Pos}_{t, p(n)})$$

Randbedingungen

Zu jedem Zeitpunkt t

- ... ist M in genau einem Zustand z :

$$\bigwedge_{t \in \{0, \dots, p(n)\}} \text{exactlyOne}(\text{State}_{t,z_0}, \dots, \text{State}_{t,z_k})$$

- ... ist der Schreib-Lesekopf von M in genau einer Position auf dem Band:

$$\bigwedge_{t \in \{0, \dots, p(n)\}} \text{exactlyOne}(\text{Pos}_{t,-p(n)}, \dots, \text{Pos}_{t,p(n)})$$

- ... befindet sich in jeder Bandzelle genau ein Symbol aus Γ :

$$\bigwedge_{t \in \{0, \dots, p(n)\}} \bigwedge_{i \in \{-p(n), \dots, p(n)\}} \text{exactlyOne}(\text{Tape}_{t,i,b_1}, \dots, \text{Tape}_{t,i,b_\ell})$$

Daher:

$$\textit{Rand} := \bigwedge_{t \in \{0, \dots, p(n)\}} \left(\begin{array}{l} \textit{exactlyOne}(\textit{State}_{t,z_0}, \dots, \textit{State}_{t,z_k}) \\ \wedge \textit{exactlyOne}(\textit{Pos}_{t,-p(n)}, \dots, \textit{Pos}_{t,p(n)}) \\ \wedge \bigwedge_{i \in \{-p(n), \dots, p(n)\}} \textit{exactlyOne}(\textit{Tape}_{t,i,b_1}, \dots, \textit{Tape}_{t,i,b_\ell}) \end{array} \right)$$

Anfangsbedingungen

Die Bedingungen zum Zeitpunkt $t = 0$:

Anfangsbedingungen

Die Bedingungen zum Zeitpunkt $t = 0$:

- ▶ M ist im Startzustand: $State_{0,z_0}$

Anfangsbedingungen

Die Bedingungen zum Zeitpunkt $t = 0$:

- ▶ M ist im Startzustand: $State_{0,z_0}$
- ▶ Der Schreib-Lesekopf ist auf Position 0: $Pos_{0,0}$

Anfangsbedingungen

Die Bedingungen zum Zeitpunkt $t = 0$:

- ▶ M ist im Startzustand: $State_{0,z_0}$
- ▶ Der Schreib-Lesekopf ist auf Position 0: $Pos_{0,0}$
- ▶ Die Eingabe $w = a_1 \dots a_n$ steht auf dem Band und alle anderen Zellen enthalten das Blank-Symbol:

$$\left(\bigwedge_{i \in \{0, \dots, n-1\}} Tape_{0,i,a_{i+1}} \right) \wedge \left(\bigwedge_{i \in \{-p(n), \dots, -1\}} Tape_{0,i,\square} \right) \wedge \left(\bigwedge_{i \in \{n, \dots, p(n)\}} Tape_{0,i,\square} \right)$$

Daher:

$$\text{Anfang} := \text{State}_{0,z_0} \wedge \text{Pos}_{0,0} \wedge \\ \left(\bigwedge_{i \in \{0, \dots, n-1\}} \text{Tape}_{0,i,a_{i+1}} \right) \wedge \left(\bigwedge_{i \in \{-p(n), \dots, -1\}} \text{Tape}_{0,i,\square} \right) \wedge \left(\bigwedge_{i \in \{n, \dots, p(n)\}} \text{Tape}_{0,i,\square} \right)$$

Übergangsbedingungen

Für Übergang von t zu $t + 1$:

Übergangsbedingungen

Für Übergang von t zu $t + 1$:

- Zustand, Bandinhalt, Position ändern.

Sei $dir(N) = 0$, $dir(L) = -1$, $dir(R) = 1$:

$$\bigwedge_{\substack{t \in \{0, \dots, p(n)-1\}, z \in Z, \\ i \in \{-p(n)+1, \dots, p(n)-1\}, b \in \Gamma}} \left((State_{t,z} \wedge Pos_{t,i} \wedge Tape_{t,i,b}) \right. \\ \left. \implies \bigvee_{(z', b', y) \in \delta(z, b)} (State_{t+1, z'} \wedge Pos_{t+1, i+dir(y)} \wedge Tape_{t+1, i, b'}) \right)$$

Übergangsbedingungen

Für Übergang von t zu $t + 1$:

- Zustand, Bandinhalt, Position ändern.

Sei $dir(N) = 0$, $dir(L) = -1$, $dir(R) = 1$:

$$\bigwedge_{\substack{t \in \{0, \dots, p(n)-1\}, z \in Z, \\ i \in \{-p(n)+1, \dots, p(n)-1\}, b \in \Gamma}} \left((State_{t,z} \wedge Pos_{t,i} \wedge Tape_{t,i,b}) \right. \\ \left. \implies \bigvee_{(z',b',y) \in \delta(z,b)} (State_{t+1,z'} \wedge Pos_{t+1,i+dir(y)} \wedge Tape_{t+1,i,b'}) \right)$$

- Zellen auf denen der Schreib-Lesekopf nicht steht, bleiben unverändert:

$$\bigwedge_{\substack{t \in \{0, \dots, p(n)-1\}, \\ i \in \{-p(n), \dots, p(n)\}, \\ b \in \Gamma}} ((\neg Pos_{t,i} \wedge Tape_{t,i,b}) \implies Tape_{t+1,i,b})$$

Übergangsbedingungen

Daher:

Übergang :=

$$\bigwedge_{\substack{t \in \{0, \dots, p(n)-1\}, \\ z \in Z, \\ i \in \{-p(n)+1, \dots, p(n)-1\}, \\ b \in \Gamma}} \left(\begin{aligned} & (State_{t,z} \wedge Pos_{t,i} \wedge Tape_{t,i,b}) \\ & \implies \bigvee_{(z',b',y) \in \delta(z,b)} (State_{t+1,z'} \wedge Pos_{t+1,i+dir(y)} \wedge Tape_{t+1,i,b'}) \end{aligned} \right) \\ \wedge \left(\bigwedge_{\substack{t \in \{0, \dots, p(n)-1\}, \\ i \in \{-p(n), \dots, p(n)\}, \\ b \in \Gamma}} ((\neg Pos_{t,i} \wedge Tape_{t,i,b}) \implies Tape_{t+1,i,b}) \right)$$

Endbedingung

Ein Endzustand wird erreicht:

$$\textit{Ende} := \bigvee_{z \in E, t \in \{0, \dots, p(n)\}} \textit{State}_{t,z}$$

$\mathcal{NP}$ -Schwere von SAT

Wenn es eine **Belegung** I der Variablen von F gibt mit $I(F) = 1$, dann kann daraus ein **akzeptierender Lauf** für M auf Eingabe w konstruiert werden.

$\mathcal{NP}$ -Schwere von SAT

Wenn es eine **Belegung** I der Variablen von F gibt mit $I(F) = 1$, dann kann daraus ein **akzeptierender Lauf** für M auf Eingabe w konstruiert werden.

Umgekehrt: Wenn M die Eingabe w akzeptiert, dann $z_0 w \vdash_M^r uz_e v$ mit $z_e \in E$ und $r \leq p(n)$. Der Lauf liefert eine **Belegung** I mit $I(F) = 1$.

$\mathcal{NP}$ -Schwere von SAT

Wenn es eine **Belegung** I der Variablen von F gibt mit $I(F) = 1$, dann kann daraus ein **akzeptierender Lauf** für M auf Eingabe w konstruiert werden.

Umgekehrt: Wenn M die Eingabe w akzeptiert, dann $z_0 w \vdash_M^r uz_e v$ mit $z_e \in E$ und $r \leq p(n)$. Der Lauf liefert eine **Belegung** I mit $I(F) = 1$.

Damit gilt: **F ist erfüllbar** g.d.w. **M akzeptiert w** .

$\mathcal{NP}$ -Schwere von SAT

Wenn es eine **Belegung** I der Variablen von F gibt mit $I(F) = 1$, dann kann daraus ein **akzeptierender Lauf** für M auf Eingabe w konstruiert werden.

Umgekehrt: Wenn M die Eingabe w akzeptiert, dann $z_0 w \vdash_M^r uz_e v$ mit $z_e \in E$ und $r \leq p(n)$. Der Lauf liefert eine **Belegung** I mit $I(F) = 1$.

Damit gilt: **F ist erfüllbar** g.d.w. **M akzeptiert w** .

F kann in (deterministischer) Polynomialzeit berechnet werden:

Teilformel	Größe (Anzahl an Variablenvorkommen)
<i>Rand</i>	$O(p(n)^3)$
<i>Anfang</i>	$O(p(n))$
<i>Übergang</i>	$O(p(n)^2)$
<i>Ende</i>	$O(p(n))$
F	$O(p(n)^3)$

Daher: $L \leq_p \text{SAT}$ für alle $L \in \mathcal{NP}$. D.h. SAT ist $\mathcal{NP}$ -schwer.



Satz von Cook

Insgesamt haben wir gezeigt:

Satz (Satz von Cook)

Das Erfüllbarkeitsproblem der Aussagenlogik (SAT) ist $\mathcal{NP}$ -vollständig.

12b

$\mathcal{NP}$ -Vollständigkeit von 3-CNF-SAT

Prof. Dr. Jasmin Blanchette

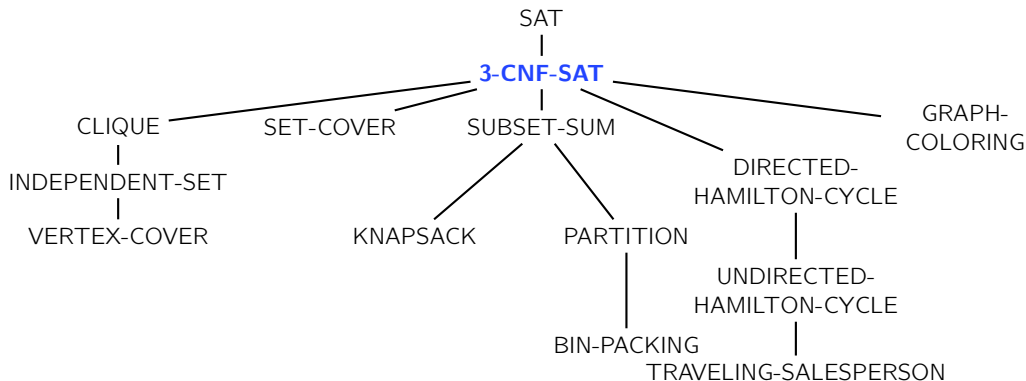
Lehr- und Forschungseinheit für
Theoretische Informatik und Theorembeweisen

Stand: 9. Juli 2024
Basierend auf Folien von PD Dr. David Sabel



Überblick über $\mathcal{NP}$ -Vollständigkeitsbeweise

Wir weisen $\mathcal{NP}$ -Schwere nach durch Polynomialzeitreduktion bekannter $\mathcal{NP}$ -vollständiger Probleme auf das neue Problem.



Wiederholung: $\mathcal{NP}$ -Vollständigkeit

Definition

Eine Sprache L heißt $\mathcal{NP}$ -vollständig, wenn gilt

1. $L \in \mathcal{NP}$ und
2. L ist $\mathcal{NP}$ -schwer: für alle $L' \in \mathcal{NP}$ gilt $L' \leq_p L$.

Wiederholung: $\mathcal{NP}$ -Vollständigkeit

Definition

Eine Sprache L heißt $\mathcal{NP}$ -vollständig, wenn gilt

1. $L \in \mathcal{NP}$ und
2. L ist $\mathcal{NP}$ -schwer: für alle $L' \in \mathcal{NP}$ gilt $L' \leq_p L$.

Beweistechnik für $\mathcal{NP}$ -Vollständigkeit von L :

1. Zeige $L \in \mathcal{NP}$.
2. Zeige $L_0 \leq_p L$ für ein bekanntes $\mathcal{NP}$ -vollständiges Problem L_0 (z.B. SAT).
Dann folgt die $\mathcal{NP}$ -Schwere von L .

Konjunktive Normalform

Die aussagenlogische Formel F ist in **konjunktiver Normalform (CNF)**, wenn sie von der folgenden Form ist:

$$\bigwedge_{i=1}^m \left(\bigvee_{j=1}^{n_i} L_{i,j} \right)$$

- ▶ Jedes **Literal** $L_{i,j}$ ist eine **aussagenlogische Variable** oder deren **Negation**.
- ▶ $(\bigvee_{j=1}^{n_i} L_{i,j})$ ist eine **Klausel**.

Konjunktive Normalform

Die aussagenlogische Formel F ist in **konjunktiver Normalform (CNF)**, wenn sie von der folgenden Form ist:

$$\bigwedge_{i=1}^m \left(\bigvee_{j=1}^{n_i} L_{i,j} \right)$$

- ▶ Jedes **Literal** $L_{i,j}$ ist eine **aussagenlogische Variable oder deren Negation**.
- ▶ $(\bigvee_{j=1}^{n_i} L_{i,j})$ ist eine **Klausel**.

Beispiel:

$$x_1 \wedge (x_2 \vee \neg x_4 \vee x_3) \wedge (x_2 \vee x_4)$$

Konjunktive Normalform

Die aussagenlogische Formel F ist in **konjunktiver Normalform (CNF)**, wenn sie von der folgenden Form ist:

$$\bigwedge_{i=1}^m \left(\bigvee_{j=1}^{n_i} L_{i,j} \right)$$

- ▶ Jedes **Literal** $L_{i,j}$ ist eine **aussagenlogische Variable oder deren Negation**.
- ▶ $(\bigvee_{j=1}^{n_i} L_{i,j})$ ist eine **Klausel**.

Beispiel:

$$x_1 \wedge (x_2 \vee \neg x_4 \vee x_3) \wedge (x_2 \vee x_4)$$

Eine erfüllende Belegung muss mindestens 1 Literal pro Klausel wahr machen.

Konjunktive Normalform

Die aussagenlogische Formel F ist in **konjunktiver Normalform (CNF)**, wenn sie von der folgenden Form ist:

$$\bigwedge_{i=1}^m \left(\bigvee_{j=1}^{n_i} L_{i,j} \right)$$

- ▶ Jedes **Literal** $L_{i,j}$ ist eine **aussagenlogische Variable oder deren Negation**.
- ▶ $(\bigvee_{j=1}^{n_i} L_{i,j})$ ist eine **Klausel**.

Beispiel:

$$x_1 \wedge (x_2 \vee \neg x_4 \vee x_3) \wedge (x_2 \vee x_4)$$

Eine erfüllende Belegung muss mindestens 1 Literal pro Klausel wahr machen.

Wir nehmen an, dass es keine Klauseln gibt, die x und $\neg x$ enthalten.

Diese Klauseln sind immer wahr und können gelöscht werden.

Definition

Das **3-CNF-SAT-Problem** lässt sich wie folgt formulieren.

gegeben: eine aussagenlogische Formel F in CNF,
sodass jede Klausel höchstens 3 Literale enthält
(d.h. eine aussagenlogische Formel F in **3-CNF**)

gefragt: Ist F erfüllbar? Genauer: Gibt es eine erfüllende Belegung der Variablen,
sodass F den Wert 1 erhält?

Zugehörigkeit von 3-CNF-SAT zu $\mathcal{NP}$

Satz

3-CNF-SAT ist $\mathcal{NP}$ -vollständig.

Zugehörigkeit von 3-CNF-SAT zu $\mathcal{NP}$

Satz

3-CNF-SAT ist $\mathcal{NP}$ -vollständig.

Beweis

1. 3-CNF-SAT ist in $\mathcal{NP}$:

Rate nichtdeterministisch eine Belegung der Variablen.

Zugehörigkeit von 3-CNF-SAT zu $\mathcal{NP}$

Satz

3-CNF-SAT ist $\mathcal{NP}$ -vollständig.

Beweis

1. 3-CNF-SAT ist in $\mathcal{NP}$:

Rate nichtdeterministisch eine Belegung der Variablen.

Prüfe deterministisch, ob die Belegung die 3-CNF wahr macht. Akzeptiere in diesem Fall, sonst verwirf.

Zugehörigkeit von 3-CNF-SAT zu $\mathcal{NP}$

Satz

3-CNF-SAT ist $\mathcal{NP}$ -vollständig.

Beweis

1. 3-CNF-SAT ist in $\mathcal{NP}$:

Rate nichtdeterministisch eine Belegung der Variablen.

Prüfe deterministisch, ob die Belegung die 3-CNF wahr macht. Akzeptiere in diesem Fall, sonst verwirf.

Dies geht in Polynomialzeit in der Größe der 3-CNF.

Zugehörigkeit von 3-CNF-SAT zu $\mathcal{NP}$

Satz

3-CNF-SAT ist $\mathcal{NP}$ -vollständig.

Beweis

1. 3-CNF-SAT ist in $\mathcal{NP}$:

Rate nichtdeterministisch eine Belegung der Variablen.

Prüfe deterministisch, ob die Belegung die 3-CNF wahr macht. Akzeptiere in diesem Fall, sonst verwirf.

Dies geht in Polynomialzeit in der Größe der 3-CNF.

Daher kann 3-CNF-SAT auf einer NTM in Polynomialzeit entschieden werden.

$\mathcal{NP}$ -Schwere von 3-CNF-SAT

Satz

3-CNF-SAT ist $\mathcal{NP}$ -vollständig.

Beweis (Fortsetzung)

2. 3-CNF-SAT ist $\mathcal{NP}$ -schwer:

Wir zeigen $\text{SAT} \leq_p \text{3-CNF-SAT}$.

Satz

3-CNF-SAT ist $\mathcal{NP}$ -vollständig.

Beweis (Fortsetzung)

2. 3-CNF-SAT ist $\mathcal{NP}$ -schwer:

Wir zeigen $\text{SAT} \leq_p \text{3-CNF-SAT}$.

Gesucht wird eine polynomiell berechenbare, totale Funktion f , sodass:
 F ist erfüllbar g.d.w. die 3-CNF $f(F)$ ist erfüllbar.

CNF-Transformation

Die **CNF-Transformation** ist ein Verfahren,
um F in eine **äquivalente** CNF $f(F)$ umzuformen.

CNF-Transformation

Die **CNF-Transformation** ist ein Verfahren, um F in eine **äquivalente** CNF $f(F)$ umzuformen.

Schritte:

1. Löse $\iff$ und $\implies$ auf.
2. Schiebe Negationen nach innen und anschließend multipliziere aus.
(Wende Distributivität, Kommutativität an, um CNF herzustellen.)

CNF-Transformation

Die **CNF-Transformation** ist ein Verfahren, um F in eine **äquivalente** CNF $f(F)$ umzuformen.

Schritte:

1. Löse $\iff$ und $\implies$ auf.
2. Schiebe Negationen nach innen und anschließend multipliziere aus.
(Wende Distributivität, Kommutativität an, um CNF herzustellen.)

Zum Beispiel wird $x_1 \iff x_2$ zu $(\neg x_1 \vee x_2) \wedge (x_1 \vee \neg x_2)$.

CNF-Transformation

Die **CNF-Transformation** ist ein Verfahren, um F in eine **äquivalente** CNF $f(F)$ umzuformen.

Schritte:

1. Löse $\iff$ und $\implies$ auf.
2. Schiebe Negationen nach innen und anschließend multipliziere aus.
(Wende Distributivität, Kommutativität an, um CNF herzustellen.)

Zum Beispiel wird $x_1 \iff x_2$ zu $(\neg x_1 \vee x_2) \wedge (x_1 \vee \neg x_2)$.

Aber der Algorithmus hat im schlechtesten Fall **exponentielle Laufzeit** und kann Klauseln erzeugen, die **mehr als drei Literale** enthalten.

CNF-Transformation

Die **CNF-Transformation** ist ein Verfahren, um F in eine **äquivalente** CNF $f(F)$ umzuformen.

Schritte:

1. Löse $\iff$ und $\implies$ auf.
2. Schiebe Negationen nach innen und anschließend multipliziere aus.
(Wende Distributivität, Kommutativität an, um CNF herzustellen.)

Zum Beispiel wird $x_1 \iff x_2$ zu $(\neg x_1 \vee x_2) \wedge (x_1 \vee \neg x_2)$.

Aber der Algorithmus hat im schlechtesten Fall **exponentielle Laufzeit** und kann Klauseln erzeugen, die **mehr als drei Literale** enthalten.

Er kann daher **nicht** direkt für eine Polynomialzeitreduktion von SAT auf 3-CNF-SAT verwendet werden.

$\mathcal{NP}$ -Schwere von 3-CNF-SAT

Satz

3-CNF-SAT ist $\mathcal{NP}$ -vollständig.

Beweis (Fortsetzung)

2. 3-CNF-SAT ist $\mathcal{NP}$ -schwer:

Wir zeigen $\text{SAT} \leq_p \text{3-CNF-SAT}$.

Gesucht wird eine polynomiell berechenbare, totale Funktion f , sodass:
 F ist erfüllbar g.d.w. die 3-CNF $f(F)$ ist erfüllbar.

$\mathcal{NP}$ -Schwere von 3-CNF-SAT

Satz

3-CNF-SAT ist $\mathcal{NP}$ -vollständig.

Beweis (Fortsetzung)

2. 3-CNF-SAT ist $\mathcal{NP}$ -schwer:

Wir zeigen $\text{SAT} \leq_p \text{3-CNF-SAT}$.

Gesucht wird eine polynomiell berechenbare, totale Funktion f , sodass:
 F ist erfüllbar g.d.w. die 3-CNF $f(F)$ ist erfüllbar.

Beachte: f muss die **Erfüllbarkeit erhalten**, aber nicht die **Äquivalenz**.

$\mathcal{NP}$ -Schwere von 3-CNF-SAT

Satz

3-CNF-SAT ist $\mathcal{NP}$ -vollständig.

Beweis (Fortsetzung)

2. 3-CNF-SAT ist $\mathcal{NP}$ -schwer:

Wir zeigen $\text{SAT} \leq_p \text{3-CNF-SAT}$.

Gesucht wird eine polynomiell berechenbare, totale Funktion f , sodass:
 F ist erfüllbar g.d.w. die 3-CNF $f(F)$ ist erfüllbar.

Beachte: f muss die **Erfüllbarkeit erhalten**, aber nicht die **Äquivalenz**.

Die **Tseitin-Transformation** ist ein Verfahren, um F in eine erfüllbarkeitsäquivalente 3-CNF $f(F)$ umzuformen, sodass $f(F)$ **polynomielle Größe** in $|F|$ hat.

Tseitin-Transformation

Schritte:

1. Schiebe alle Negationen nach innen vor die Literale, dabei werden die Regeln
 $\neg\neg F \rightsquigarrow F$, $\neg(F \wedge G) \rightsquigarrow \neg F \vee \neg G$, $\neg(F \vee G) \rightsquigarrow \neg F \wedge \neg G$,
 $\neg(F \iff G) \rightsquigarrow \neg F \iff G$ und $\neg(F \implies G) \rightsquigarrow F \wedge \neg G$ angewendet.

Tseitin-Transformation

Schritte:

1. Schiebe alle Negationen nach innen vor die Literale, dabei werden die Regeln $\neg\neg F \rightsquigarrow F$, $\neg(F \wedge G) \rightsquigarrow \neg F \vee \neg G$, $\neg(F \vee G) \rightsquigarrow \neg F \wedge \neg G$, $\neg(F \iff G) \rightsquigarrow \neg F \iff G$ und $\neg(F \implies G) \rightsquigarrow F \wedge \neg G$ angewendet.
2. Bilde den Syntaxbaum der Formel, mit Literalen auf den Blättern.

Tseitin-Transformation

Schritte:

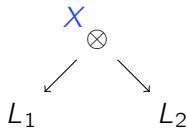
1. Schiebe alle Negationen nach innen vor die Literale, dabei werden die Regeln $\neg\neg F \rightsquigarrow F$, $\neg(F \wedge G) \rightsquigarrow \neg F \vee \neg G$, $\neg(F \vee G) \rightsquigarrow \neg F \wedge \neg G$, $\neg(F \iff G) \rightsquigarrow \neg F \iff G$ und $\neg(F \implies G) \rightsquigarrow F \wedge \neg G$ angewendet.
2. Bilde den Syntaxbaum der Formel, mit Literalen auf den Blättern.
3. Führe für jeden Nichtblatt-Knoten neue aussagenlogische Variable X ein, die als **Synonym für eine Teilformel** steht.

Tseitin-Transformation

Schritte:

1. Schiebe alle Negationen nach innen vor die Literale, dabei werden die Regeln $\neg\neg F \rightsquigarrow F$, $\neg(F \wedge G) \rightsquigarrow \neg F \vee \neg G$, $\neg(F \vee G) \rightsquigarrow \neg F \wedge \neg G$, $\neg(F \iff G) \rightsquigarrow \neg F \iff G$ und $\neg(F \implies G) \rightsquigarrow F \wedge \neg G$ angewendet.
2. Bilde den Syntaxbaum der Formel, mit Literalen auf den Blättern.
3. Führe für jeden Nichtblatt-Knoten neue aussagenlogische Variable X ein, die als **Synonym für eine Teilformel** steht.

4. Pro Gabelung



erzeuge die Formel $X \iff (L_1 \otimes L_2)$, wobei

$\otimes \in \{\wedge, \vee, \implies, \iff\}$ und L_1 bzw. L_2 entweder die neue Variable oder das Literal am Blatt ist.

5. Konjugiere die erzeugten Formeln zu F' und schließlich erzeuge $W \wedge F'$, wobei W die Variable für die Wurzel ist.
Die erzeugte Formel ist von der Form

$$W \wedge \bigwedge_{i,j,k} (X_i \iff (L_j \otimes_i L_k))$$

5. Konjugiere die erzeugten Formeln zu F' und schließlich erzeuge $W \wedge F'$, wobei W die Variable für die Wurzel ist.

Die erzeugte Formel ist von der Form

$$W \wedge \bigwedge_{i,j,k} (X_i \iff (L_j \otimes_i L_k))$$

6. Berechne für jede Teilformel $X_i \iff (L_j \otimes_i L_k)$ die CNF mit der üblichen CNF-Transformation.

5. Konjugiere die erzeugten Formeln zu F' und schließlich erzeuge $W \wedge F'$, wobei W die Variable für die Wurzel ist.

Die erzeugte Formel ist von der Form

$$W \wedge \bigwedge_{i,j,k} (X_i \iff (L_j \otimes_i L_k))$$

6. Berechne für jede Teilformel $X_i \iff (L_j \otimes_i L_k)$ die CNF mit der üblichen CNF-Transformation.
7. Lösche doppelte Vorkommen von Literalen.

5. Konjugiere die erzeugten Formeln zu F' und schließlich erzeuge $W \wedge F'$, wobei W die Variable für die Wurzel ist.

Die erzeugte Formel ist von der Form

$$W \wedge \bigwedge_{i,j,k} (X_i \iff (L_j \otimes_i L_k))$$

6. Berechne für jede Teilformel $X_i \iff (L_j \otimes_i L_k)$ die CNF mit der üblichen CNF-Transformation.
7. Lösche doppelte Vorkommen von Literalen.

Diese Prozedur ergibt eine Formel in 3-CNF, da pro Klausel nur drei verschiedene Variablen vorkommen können.

Beispiel für die Tseitin-Transformation

Sei $F = \neg(\neg(x_3 \implies \neg x_1) \vee x_2)$.

Beispiel für die Tseitin-Transformation

Sei $F = \neg(\neg(x_3 \implies \neg x_1) \vee x_2)$.

Negationen nach innen schieben:

$$\neg(\neg(x_3 \implies \neg x_1) \vee x_2)$$

$$\rightsquigarrow \neg\neg(x_3 \implies \neg x_1) \wedge \neg x_2$$

$$\rightsquigarrow (x_3 \implies \neg x_1) \wedge \neg x_2$$

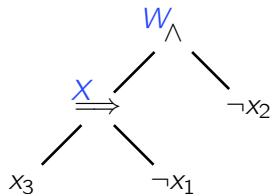
Beispiel für die Tseitin-Transformation

Sei $F = \neg(\neg(x_3 \implies \neg x_1) \vee x_2)$.

Negationen nach innen schieben:

$$\begin{aligned} & \neg(\neg(x_3 \implies \neg x_1) \vee x_2) \\ \rightsquigarrow & \neg\neg(x_3 \implies \neg x_1) \wedge \neg x_2 \\ \rightsquigarrow & (x_3 \implies \neg x_1) \wedge \neg x_2 \end{aligned}$$

Syntaxbaum dazu:



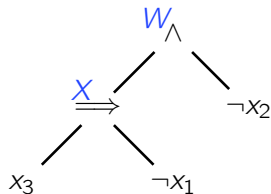
Beispiel für die Tseitin-Transformation

Sei $F = \neg(\neg(x_3 \implies \neg x_1) \vee x_2)$.

Negationen nach innen schieben:

$$\begin{aligned} & \neg(\neg(x_3 \implies \neg x_1) \vee x_2) \\ \rightsquigarrow & \neg\neg(x_3 \implies \neg x_1) \wedge \neg x_2 \\ \rightsquigarrow & (x_3 \implies \neg x_1) \wedge \neg x_2 \end{aligned}$$

Syntaxbaum dazu:



Formel: $W \wedge (W \iff (X \wedge \neg x_2)) \wedge (X \iff (x_3 \implies \neg x_1))$

Beispiel für die Tseitin-Transformation

Berechnung der CNFs der kleinen Teilformeln:

$$W \wedge (W \iff (X \wedge \neg x_2)) \wedge (X \iff (x_3 \implies \neg x_1))$$

Beispiel für die Tseitin-Transformation

Berechnung der CNFs der kleinen Teilformeln:

$$\begin{aligned} & W \wedge (W \iff (X \wedge \neg x_2)) \wedge (X \iff (x_3 \implies \neg x_1)) \\ \leadsto & W \wedge (W \vee \neg(X \wedge \neg x_2)) \wedge (\neg W \vee (X \wedge \neg x_2)) \wedge (X \iff (x_3 \implies \neg x_1)) \end{aligned}$$

Beispiel für die Tseitin-Transformation

Berechnung der CNFs der kleinen Teilformeln:

$$W \wedge (W \iff (X \wedge \neg x_2)) \wedge (X \iff (x_3 \implies \neg x_1))$$

$$\leadsto W \wedge (W \vee \neg(X \wedge \neg x_2)) \wedge (\neg W \vee (X \wedge \neg x_2)) \wedge (X \iff (x_3 \implies \neg x_1))$$

$$\leadsto W \wedge (W \vee \neg X \vee x_2) \wedge (\neg W \vee (X \wedge \neg x_2)) \wedge (X \iff (x_3 \implies \neg x_1))$$

Beispiel für die Tseitin-Transformation

Berechnung der CNFs der kleinen Teilformeln:

$$\begin{aligned} & W \wedge (W \iff (X \wedge \neg x_2)) \wedge (X \iff (x_3 \implies \neg x_1)) \\ \leadsto & W \wedge (W \vee \neg(X \wedge \neg x_2)) \wedge (\neg W \vee (X \wedge \neg x_2)) \wedge (X \iff (x_3 \implies \neg x_1)) \\ \leadsto & W \wedge (W \vee \neg X \vee x_2) \wedge (\neg W \vee (X \wedge \neg x_2)) \wedge (X \iff (x_3 \implies \neg x_1)) \\ \leadsto & W \wedge (W \vee \neg X \vee x_2) \wedge (\neg W \vee X) \wedge (\neg W \vee \neg x_2) \wedge (X \iff (x_3 \implies \neg x_1)) \end{aligned}$$

Beispiel für die Tseitin-Transformation

Berechnung der CNFs der kleinen Teilformeln:

$$\begin{aligned} & W \wedge (W \iff (X \wedge \neg x_2)) \wedge (X \iff (x_3 \implies \neg x_1)) \\ \leadsto & W \wedge (W \vee \neg(X \wedge \neg x_2)) \wedge (\neg W \vee (X \wedge \neg x_2)) \wedge (X \iff (x_3 \implies \neg x_1)) \\ \leadsto & W \wedge (W \vee \neg X \vee x_2) \wedge (\neg W \vee (X \wedge \neg x_2)) \wedge (X \iff (x_3 \implies \neg x_1)) \\ \leadsto & W \wedge (W \vee \neg X \vee x_2) \wedge (\neg W \vee X) \wedge (\neg W \vee \neg x_2) \wedge (X \iff (x_3 \implies \neg x_1)) \\ \leadsto & W \wedge (W \vee \neg X \vee x_2) \wedge (\neg W \vee X) \wedge (\neg W \vee \neg x_2) \wedge (\neg X \vee (x_3 \implies \neg x_1)) \wedge (X \vee \neg(x_3 \implies \neg x_1)) \end{aligned}$$

Beispiel für die Tseitin-Transformation

Berechnung der CNFs der kleinen Teilformeln:

$$\begin{aligned} & W \wedge (W \iff (X \wedge \neg x_2)) \wedge (X \iff (x_3 \implies \neg x_1)) \\ \rightsquigarrow & W \wedge (W \vee \neg(X \wedge \neg x_2)) \wedge (\neg W \vee (X \wedge \neg x_2)) \wedge (X \iff (x_3 \implies \neg x_1)) \\ \rightsquigarrow & W \wedge (W \vee \neg X \vee x_2) \wedge (\neg W \vee (X \wedge \neg x_2)) \wedge (X \iff (x_3 \implies \neg x_1)) \\ \rightsquigarrow & W \wedge (W \vee \neg X \vee x_2) \wedge (\neg W \vee X) \wedge (\neg W \vee \neg x_2) \wedge (X \iff (x_3 \implies \neg x_1)) \\ \rightsquigarrow & W \wedge (W \vee \neg X \vee x_2) \wedge (\neg W \vee X) \wedge (\neg W \vee \neg x_2) \wedge (\neg X \vee (x_3 \implies \neg x_1)) \wedge (X \vee \neg(x_3 \implies \neg x_1)) \\ \rightsquigarrow & W \wedge (W \vee \neg X \vee x_2) \wedge (\neg W \vee X) \wedge (\neg W \vee \neg x_2) \wedge (\neg X \vee \neg x_3 \vee \neg x_1) \wedge (X \vee \neg(\neg x_3 \vee \neg x_1)) \end{aligned}$$

Beispiel für die Tseitin-Transformation

Berechnung der CNFs der kleinen Teilformeln:

$$\begin{aligned} & W \wedge (W \iff (X \wedge \neg x_2)) \wedge (X \iff (x_3 \implies \neg x_1)) \\ \leadsto & W \wedge (W \vee \neg(X \wedge \neg x_2)) \wedge (\neg W \vee (X \wedge \neg x_2)) \wedge (X \iff (x_3 \implies \neg x_1)) \\ \leadsto & W \wedge (W \vee \neg X \vee x_2) \wedge (\neg W \vee (X \wedge \neg x_2)) \wedge (X \iff (x_3 \implies \neg x_1)) \\ \leadsto & W \wedge (W \vee \neg X \vee x_2) \wedge (\neg W \vee X) \wedge (\neg W \vee \neg x_2) \wedge (X \iff (x_3 \implies \neg x_1)) \\ \leadsto & W \wedge (W \vee \neg X \vee x_2) \wedge (\neg W \vee X) \wedge (\neg W \vee \neg x_2) \wedge (\neg X \vee (x_3 \implies \neg x_1)) \wedge (X \vee \neg(x_3 \implies \neg x_1)) \\ \leadsto & W \wedge (W \vee \neg X \vee x_2) \wedge (\neg W \vee X) \wedge (\neg W \vee \neg x_2) \wedge (\neg X \vee \neg x_3 \vee \neg x_1) \wedge (\textcolor{red}{X} \vee \neg(\neg x_3 \vee \neg x_1)) \\ \leadsto & W \wedge (W \vee \neg X \vee x_2) \wedge (\neg W \vee X) \wedge (\neg W \vee \neg x_2) \wedge (\neg X \vee \neg x_3 \vee \neg x_1) \wedge (\textcolor{blue}{X} \vee (x_3 \wedge x_1)) \end{aligned}$$

Beispiel für die Tseitin-Transformation

Berechnung der CNFs der kleinen Teilformeln:

$$\begin{aligned} & W \wedge (W \iff (X \wedge \neg x_2)) \wedge (X \iff (x_3 \implies \neg x_1)) \\ \rightsquigarrow & W \wedge (W \vee \neg(X \wedge \neg x_2)) \wedge (\neg W \vee (X \wedge \neg x_2)) \wedge (X \iff (x_3 \implies \neg x_1)) \\ \rightsquigarrow & W \wedge (W \vee \neg X \vee x_2) \wedge (\neg W \vee (X \wedge \neg x_2)) \wedge (X \iff (x_3 \implies \neg x_1)) \\ \rightsquigarrow & W \wedge (W \vee \neg X \vee x_2) \wedge (\neg W \vee X) \wedge (\neg W \vee \neg x_2) \wedge (X \iff (x_3 \implies \neg x_1)) \\ \rightsquigarrow & W \wedge (W \vee \neg X \vee x_2) \wedge (\neg W \vee X) \wedge (\neg W \vee \neg x_2) \wedge (\neg X \vee (x_3 \implies \neg x_1)) \wedge (X \vee \neg(x_3 \implies \neg x_1)) \\ \rightsquigarrow & W \wedge (W \vee \neg X \vee x_2) \wedge (\neg W \vee X) \wedge (\neg W \vee \neg x_2) \wedge (\neg X \vee \neg x_3 \vee \neg x_1) \wedge (X \vee \neg(\neg x_3 \vee \neg x_1)) \\ \rightsquigarrow & W \wedge (W \vee \neg X \vee x_2) \wedge (\neg W \vee X) \wedge (\neg W \vee \neg x_2) \wedge (\neg X \vee \neg x_3 \vee \neg x_1) \wedge (\neg X \vee (x_3 \wedge x_1)) \\ \rightsquigarrow & W \wedge (W \vee \neg X \vee x_2) \wedge (\neg W \vee X) \wedge (\neg W \vee \neg x_2) \wedge (\neg X \vee \neg x_3 \vee \neg x_1) \wedge (X \vee x_3) \wedge (X \vee x_1) \end{aligned}$$

Beispiel für die Tseitin-Transformation

Berechnung der CNFs der kleinen Teilformeln:

$$\begin{aligned} & W \wedge (W \iff (X \wedge \neg x_2)) \wedge (X \iff (x_3 \implies \neg x_1)) \\ \rightsquigarrow & W \wedge (W \vee \neg(X \wedge \neg x_2)) \wedge (\neg W \vee (X \wedge \neg x_2)) \wedge (X \iff (x_3 \implies \neg x_1)) \\ \rightsquigarrow & W \wedge (W \vee \neg X \vee x_2) \wedge (\neg W \vee (X \wedge \neg x_2)) \wedge (X \iff (x_3 \implies \neg x_1)) \\ \rightsquigarrow & W \wedge (W \vee \neg X \vee x_2) \wedge (\neg W \vee X) \wedge (\neg W \vee \neg x_2) \wedge (X \iff (x_3 \implies \neg x_1)) \\ \rightsquigarrow & W \wedge (W \vee \neg X \vee x_2) \wedge (\neg W \vee X) \wedge (\neg W \vee \neg x_2) \wedge (\neg X \vee (x_3 \implies \neg x_1)) \wedge (X \vee \neg(x_3 \implies \neg x_1)) \\ \rightsquigarrow & W \wedge (W \vee \neg X \vee x_2) \wedge (\neg W \vee X) \wedge (\neg W \vee \neg x_2) \wedge (\neg X \vee \neg x_3 \vee \neg x_1) \wedge (X \vee \neg(\neg x_3 \vee \neg x_1)) \\ \rightsquigarrow & W \wedge (W \vee \neg X \vee x_2) \wedge (\neg W \vee X) \wedge (\neg W \vee \neg x_2) \wedge (\neg X \vee \neg x_3 \vee \neg x_1) \wedge (X \vee (x_3 \wedge x_1)) \\ \rightsquigarrow & W \wedge (W \vee \neg X \vee x_2) \wedge (\neg W \vee X) \wedge (\neg W \vee \neg x_2) \wedge (\neg X \vee \neg x_3 \vee \neg x_1) \wedge (X \vee x_3) \wedge (X \vee x_1) \end{aligned}$$

Die erzeugte Formel $f(F)$ ist erfüllbarkeitsäquivalent zu $F = \neg(\neg(x_3 \implies \neg x_1) \vee x_2)$.

Zum Beispiel:

Beispiel für die Tseitin-Transformation

Berechnung der CNFs der kleinen Teilformeln:

$$\begin{aligned} & W \wedge (W \iff (X \wedge \neg x_2)) \wedge (X \iff (x_3 \implies \neg x_1)) \\ \rightsquigarrow & W \wedge (W \vee \neg(X \wedge \neg x_2)) \wedge (\neg W \vee (X \wedge \neg x_2)) \wedge (X \iff (x_3 \implies \neg x_1)) \\ \rightsquigarrow & W \wedge (W \vee \neg X \vee x_2) \wedge (\neg W \vee (X \wedge \neg x_2)) \wedge (X \iff (x_3 \implies \neg x_1)) \\ \rightsquigarrow & W \wedge (W \vee \neg X \vee x_2) \wedge (\neg W \vee X) \wedge (\neg W \vee \neg x_2) \wedge (X \iff (x_3 \implies \neg x_1)) \\ \rightsquigarrow & W \wedge (W \vee \neg X \vee x_2) \wedge (\neg W \vee X) \wedge (\neg W \vee \neg x_2) \wedge (\neg X \vee (x_3 \implies \neg x_1)) \wedge (X \vee \neg(x_3 \implies \neg x_1)) \\ \rightsquigarrow & W \wedge (W \vee \neg X \vee x_2) \wedge (\neg W \vee X) \wedge (\neg W \vee \neg x_2) \wedge (\neg X \vee \neg x_3 \vee \neg x_1) \wedge (X \vee \neg(\neg x_3 \vee \neg x_1)) \\ \rightsquigarrow & W \wedge (W \vee \neg X \vee x_2) \wedge (\neg W \vee X) \wedge (\neg W \vee \neg x_2) \wedge (\neg X \vee \neg x_3 \vee \neg x_1) \wedge (X \vee (x_3 \wedge x_1)) \\ \rightsquigarrow & W \wedge (W \vee \neg X \vee x_2) \wedge (\neg W \vee X) \wedge (\neg W \vee \neg x_2) \wedge (\neg X \vee \neg x_3 \vee \neg x_1) \wedge (X \vee x_3) \wedge (X \vee x_1) \end{aligned}$$

Die erzeugte Formel $f(F)$ ist erfüllbarkeitsäquivalent zu $F = \neg(\neg(x_3 \implies \neg x_1) \vee x_2)$.

Zum Beispiel:

Erfüllende Belegung I für $f(F)$:

$$I(x_1) = 0, I(x_2) = 0, I(x_3) = 1, I(\textcolor{blue}{W}) = 1, I(\textcolor{blue}{X}) = 1$$

Beispiel für die Tseitin-Transformation

Berechnung der CNFs der kleinen Teilformeln:

$$\begin{aligned} & W \wedge (W \iff (X \wedge \neg x_2)) \wedge (X \iff (x_3 \implies \neg x_1)) \\ \rightsquigarrow & W \wedge (W \vee \neg(X \wedge \neg x_2)) \wedge (\neg W \vee (X \wedge \neg x_2)) \wedge (X \iff (x_3 \implies \neg x_1)) \\ \rightsquigarrow & W \wedge (W \vee \neg X \vee x_2) \wedge (\neg W \vee (X \wedge \neg x_2)) \wedge (X \iff (x_3 \implies \neg x_1)) \\ \rightsquigarrow & W \wedge (W \vee \neg X \vee x_2) \wedge (\neg W \vee X) \wedge (\neg W \vee \neg x_2) \wedge (X \iff (x_3 \implies \neg x_1)) \\ \rightsquigarrow & W \wedge (W \vee \neg X \vee x_2) \wedge (\neg W \vee X) \wedge (\neg W \vee \neg x_2) \wedge (\neg X \vee (x_3 \implies \neg x_1)) \wedge (X \vee \neg(x_3 \implies \neg x_1)) \\ \rightsquigarrow & W \wedge (W \vee \neg X \vee x_2) \wedge (\neg W \vee X) \wedge (\neg W \vee \neg x_2) \wedge (\neg X \vee \neg x_3 \vee \neg x_1) \wedge (X \vee \neg(\neg x_3 \vee \neg x_1)) \\ \rightsquigarrow & W \wedge (W \vee \neg X \vee x_2) \wedge (\neg W \vee X) \wedge (\neg W \vee \neg x_2) \wedge (\neg X \vee \neg x_3 \vee \neg x_1) \wedge (X \vee (x_3 \wedge x_1)) \\ \rightsquigarrow & W \wedge (W \vee \neg X \vee x_2) \wedge (\neg W \vee X) \wedge (\neg W \vee \neg x_2) \wedge (\neg X \vee \neg x_3 \vee \neg x_1) \wedge (X \vee x_3) \wedge (X \vee x_1) \end{aligned}$$

Die erzeugte Formel $f(F)$ ist erfüllbarkeitsäquivalent zu $F = \neg(\neg(x_3 \implies \neg x_1) \vee x_2)$.

Zum Beispiel:

Erfüllende Belegung I für $f(F)$:

$$I(x_1) = 0, I(x_2) = 0, I(x_3) = 1, I(W) = 1, I(X) = 1$$

Erfüllende Belegung I' für F :

$$I'(x_1) = 0, I'(x_2) = 0, I'(x_3) = 1$$

Erfüllbarkeitsäquivalenz der Tseitin-Transformation

Beweis (Fortsetzung)

Wir zeigen: F ist erfüllbar g.d.w. $f(F)$ ist erfüllbar.

Erfüllbarkeitsäquivalenz der Tseitin-Transformation

Beweis (Fortsetzung)

Wir zeigen: F ist erfüllbar g.d.w. $f(F)$ ist erfüllbar.

$\Leftarrow$ Sei I eine Belegung mit $I(f(F)) = 1$.

Für die Restriktion I' von I auf die Variablen von F gilt $I'(F) = 1$.

Erfüllbarkeitsäquivalenz der Tseitin-Transformation

Beweis (Fortsetzung)

Wir zeigen: F ist erfüllbar g.d.w. $f(F)$ ist erfüllbar.

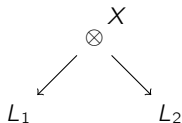
$\Leftarrow$ Sei I eine Belegung mit $I(f(F)) = 1$.

Für die Restriktion I' von I auf die Variablen von F gilt $I'(F) = 1$.

$\Rightarrow$ Sei I eine Belegung mit $I(F) = 1$.

Sei I' die Belegung mit

- ▶ $I'(x) = I(x)$ für alle Variablen x , die in F vorkommen
- ▶ $I'(X) = I'(L_1 \otimes L_2)$ für



Dann gilt $I'(f(F)) = 1$.

Komplexität der Tseitin-Transformation

Beweis (Fortsetzung)

Es bleibt zu zeigen, dass f polynomiell berechenbar ist.

Komplexität der Tseitin-Transformation

Beweis (Fortsetzung)

Es bleibt zu zeigen, dass f polynomiell berechenbar ist.

Die Größe der Teilformeln, die in CNF gebracht werden, ist konstant.

Die Anzahl der Teilformeln, die erzeugt werden, ist proportional zur Größe des Syntaxbaums.

Die Größe der 3-CNF ist also polynomiell in der ursprünglichen Formel F .

Die Berechnung in **Polynomialzeit** geht.

Komplexität der Tseitin-Transformation

Beweis (Fortsetzung)

Es bleibt zu zeigen, dass f polynomiell berechenbar ist.

Die Größe der Teilformeln, die in CNF gebracht werden, ist konstant.

Die Anzahl der Teilformeln, die erzeugt werden, ist proportional zur Größe des Syntaxbaums.

Die Größe der 3-CNF ist also polynomiell in der ursprünglichen Formel F .

Die Berechnung in **Polynomialzeit** geht.

Daher: $\text{SAT} \leq_p \text{3-CNF-SAT}$.

Da SAT $\mathcal{NP}$ -schwer ist, folgt dass 3-CNF-SAT $\mathcal{NP}$ -schwer ist. □

12c

$\mathcal{NP}$ -Vollständigkeit von CLIQUE, INDEPENDENT-SET und VERTEX-COVER

Prof. Dr. Jasmin Blanchette

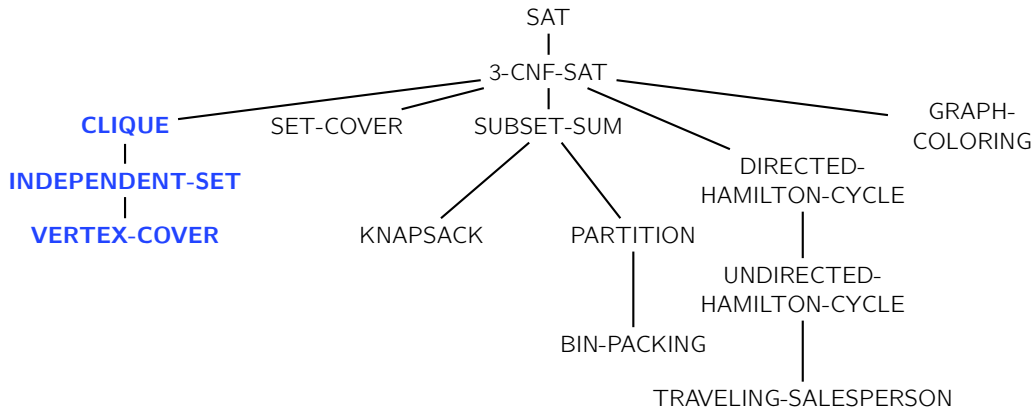
Lehr- und Forschungseinheit für
Theoretische Informatik und Theorembeweisen

Stand: 9. Juli 2024

Basierend auf Folien von PD Dr. David Sabel



Überblick über $\mathcal{NP}$ -Vollständigkeitsbeweise



Ungerichtete Graphen

Ein **ungerichteter Graph** $G = (V, E)$ besteht aus

- ▶ einer endlichen Menge V von Knoten (vertices)
- ▶ einer endlichen Menge E von Kanten (edges), wobei Kanten aus zwei Knoten bestehen, und für alle $\{u, v\} \in E$ gilt $u \neq v$.

Ungerichtete Graphen

Ein **ungerichteter Graph** $G = (V, E)$ besteht aus

- ▶ einer endlichen Menge V von Knoten (vertices)
- ▶ einer endlichen Menge E von Kanten (edges), wobei Kanten aus zwei Knoten bestehen, und für alle $\{u, v\} \in E$ gilt $u \neq v$.

Beispiel: $G = (V, E)$ mit

$V = \{1, 2, 3, 4, 5, 6, 7, 8, 9\}$ und

$E = \{\{1, 2\}, \{1, 3\}, \{2, 4\}, \{3, 4\}, \{3, 5\}, \{4, 5\},$
 $\{2, 6\}, \{4, 7\}, \{6, 7\}, \{6, 8\}, \{8, 9\}\}$

Ungerichtete Graphen

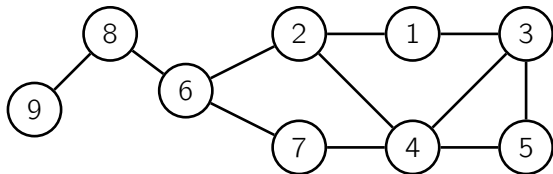
Ein **ungerichteter Graph** $G = (V, E)$ besteht aus

- ▶ einer endlichen Menge V von Knoten (vertices)
- ▶ einer endlichen Menge E von Kanten (edges), wobei Kanten aus zwei Knoten bestehen, und für alle $\{u, v\} \in E$ gilt $u \neq v$.

Beispiel: $G = (V, E)$ mit

$V = \{1, 2, 3, 4, 5, 6, 7, 8, 9\}$ und

$E = \{\{1, 2\}, \{1, 3\}, \{2, 4\}, \{3, 4\}, \{3, 5\}, \{4, 5\},$
 $\{2, 6\}, \{4, 7\}, \{6, 7\}, \{6, 8\}, \{8, 9\}\}$



Ungerichtete Graphen

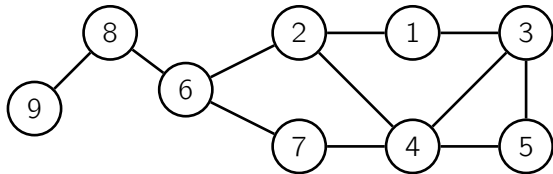
Ein **ungerichteter Graph** $G = (V, E)$ besteht aus

- ▶ einer endlichen Menge V von Knoten (vertices)
- ▶ einer endlichen Menge E von Kanten (edges), wobei Kanten aus zwei Knoten bestehen, und für alle $\{u, v\} \in E$ gilt $u \neq v$.

Beispiel: $G = (V, E)$ mit

$V = \{1, 2, 3, 4, 5, 6, 7, 8, 9\}$ und

$E = \{\{1, 2\}, \{1, 3\}, \{2, 4\}, \{3, 4\}, \{3, 5\}, \{4, 5\},$
 $\{2, 6\}, \{4, 7\}, \{6, 7\}, \{6, 8\}, \{8, 9\}\}$



Beachte: Wir **verbieten**

- ▶ Schlingen $\{u, u\} \in E$
- ▶ Mehrfachkanten (= mehrere Kanten zwischen u und v)
- ▶ Hypergraphen (Kanten mit nicht genau 2 Knoten).

Gerichtete Graphen

Bei **gerichteten Graphen** sind die Kanten gerichtet: $(u, v) \in E$ statt $\{u, v\} \in E$.
Daher sind Kanten (u, v) und (v, u) verschieden.

Gerichtete Graphen

Bei **gerichteten Graphen** sind die Kanten gerichtet: $(u, v) \in E$ statt $\{u, v\} \in E$.
Daher sind Kanten (u, v) und (v, u) verschieden.

Beispiel: $G = (V, E)$ mit

$V = \{1, 2, 3, 4, 5, 6, 7, 8, 9\}$ und

$E = \{(1, 2), (1, 3), (2, 4), (3, 4), (3, 5), (4, 5),$
 $(2, 6), (4, 7), (6, 7), (6, 8), (8, 9)\}$

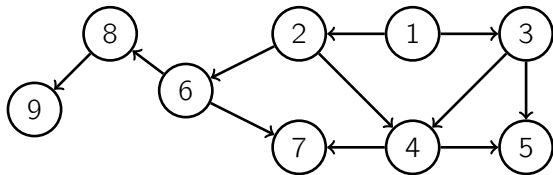
Gerichtete Graphen

Bei **gerichteten Graphen** sind die Kanten gerichtet: $(u, v) \in E$ statt $\{u, v\} \in E$.
Daher sind Kanten (u, v) und (v, u) verschieden.

Beispiel: $G = (V, E)$ mit

$V = \{1, 2, 3, 4, 5, 6, 7, 8, 9\}$ und

$E = \{(1, 2), (1, 3), (2, 4), (3, 4), (3, 5), (4, 5),$
 $(2, 6), (4, 7), (6, 7), (6, 8), (8, 9)\}$



Cliquen in Graphen

Definition

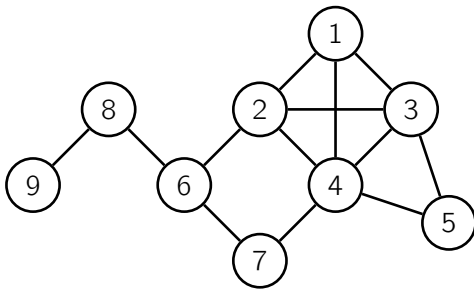
Für einen ungerichteten Graph $G = (V, E)$ ist eine **Clique der Größe k** eine Menge $V' \subseteq V$, sodass $|V'| = k$ und für alle $u, v \in V'$ mit $u \neq v$ gilt: $\{u, v\} \in E$.

Cliquen in Graphen

Definition

Für einen ungerichteten Graph $G = (V, E)$ ist eine **Clique der Größe k** eine Menge $V' \subseteq V$, sodass $|V'| = k$ und für alle $u, v \in V'$ mit $u \neq v$ gilt: $\{u, v\} \in E$.

Beispiel:

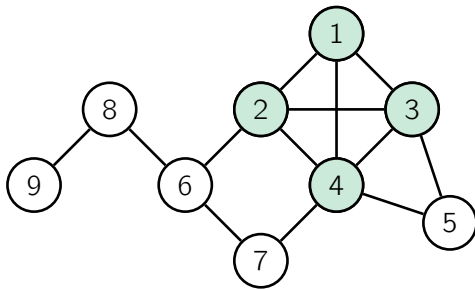


Cliquen in Graphen

Definition

Für einen ungerichteten Graph $G = (V, E)$ ist eine **Clique der Größe k** eine Menge $V' \subseteq V$, sodass $|V'| = k$ und für alle $u, v \in V'$ mit $u \neq v$ gilt: $\{u, v\} \in E$.

Beispiel (Größe 4):



Definition

Das **CLIQUE-Problem** lässt sich wie folgt formulieren.

gegeben: ein ungerichteter Graph $G = (V, E)$ und eine Zahl $k \in \mathbb{N}$

gefragt: Besitzt G eine Clique der Größe k ?

$\mathcal{NP}$ -Vollständigkeit von CLIQUE

Satz

CLIQUE ist $\mathcal{NP}$ -vollständig.

Satz

CLIQUE ist $\mathcal{NP}$ -vollständig.

Beweis

1. CLIQUE $\in \mathcal{NP}$:

Rate nichtdeterministisch eine Menge $V' \subseteq V$ von k Knoten.

Prüfe deterministisch (in Polynomialzeit), ob für alle $u, v \in V' : \{u, v\} \in E$ gilt.

Falls ja, akzeptiere, sonst verwirf.

Daher kann eine NTM konstruiert werden, die CLIQUE in Polynomialzeit entscheidet.

$\mathcal{NP}$ -Vollständigkeit von CLIQUE

Beweis (Fortsetzung)

2. CLIQUE ist $\mathcal{NP}$ -schwer:

Wir zeigen $3\text{-CNF-SAT} \leq_p \text{CLIQUE}$.

$\mathcal{NP}$ -Vollständigkeit von CLIQUE

Beweis (Fortsetzung)

2. CLIQUE ist $\mathcal{NP}$ -schwer:

Wir zeigen $3\text{-CNF-SAT} \leq_p \text{CLIQUE}$.

Sei $F = K_1 \wedge \dots \wedge K_m$ eine 3-CNF, wobei

$$K_i = (L_{i,1} \vee L_{i,2} \vee L_{i,3})$$

für $i \in \{1, \dots, m\}$. Falls K_i weniger als drei Literale hat, verdopple Literale.

$\mathcal{NP}$ -Vollständigkeit von CLIQUE

Beweis (Fortsetzung)

2. CLIQUE ist $\mathcal{NP}$ -schwer:

Wir zeigen $3\text{-CNF-SAT} \leq_p \text{CLIQUE}$.

Sei $F = K_1 \wedge \dots \wedge K_m$ eine 3-CNF, wobei

$$K_i = (L_{i,1} \vee L_{i,2} \vee L_{i,3})$$

für $i \in \{1, \dots, m\}$. Falls K_i weniger als drei Literale hat, verdopple Literale.

Notation: $\bar{L}$ ist negiertes Literal zu L : $\bar{x} = \neg x$ und $\overline{\bar{x}} = x$.

$\mathcal{NP}$ -Vollständigkeit von CLIQUE

Beweis (Fortsetzung)

2. CLIQUE ist $\mathcal{NP}$ -schwer:

Wir zeigen $3\text{-CNF-SAT} \leq_p \text{CLIQUE}$.

Sei $F = K_1 \wedge \dots \wedge K_m$ eine 3-CNF, wobei

$$K_i = (L_{i,1} \vee L_{i,2} \vee L_{i,3})$$

für $i \in \{1, \dots, m\}$. Falls K_i weniger als drei Literale hat, verdopple Literale.

Notation: $\bar{L}$ ist negiertes Literal zu L : $\bar{x} = \neg x$ und $\overline{\neg x} = x$.

Wir erzeugen einen Graphen $f(G)$.

Für jedes Literal $L_{i,j}$ erzeuge Knoten (i,j) im Graph, d.h.

$$V := \{(i,j) \mid i \in \{1, \dots, m\} \text{ und } j \in \{1, 2, 3\}\}$$

Beweis (Fortsetzung)

Für die Kanten:

- ▶ Erzeuge **keine Kanten innerhalb** der drei Knoten $\{(i, 1), (i, 2), (i, 3)\}$ **einer Klausel** für jedes i .

Beweis (Fortsetzung)

Für die Kanten:

- ▶ Erzeuge **keine Kanten innerhalb** der drei Knoten $\{(i, 1), (i, 2), (i, 3)\}$ **einer Klausel** für jedes i .
- ▶ Erzeuge Kanten zwischen Literalen verschiedenen Klauseln, aber **ohne dass sich zwei verbundene Literale $L_{i,j}$ und $L_{i',j'}$ widersprechen** (d.h. $L_{i,j} \neq \overline{L_{i',j'}}$).

Beweis (Fortsetzung)

Für die Kanten:

- ▶ Erzeuge **keine Kanten innerhalb** der drei Knoten $\{(i, 1), (i, 2), (i, 3)\}$ **einer Klausel** für jedes i .
- ▶ Erzeuge Kanten zwischen Literalen verschiedenen Klauseln, aber **ohne dass sich zwei verbundene Literale $L_{i,j}$ und $L_{i',j'}$ widersprechen** (d.h. $L_{i,j} \neq \overline{L_{i',j'}}$).

Daher:

$$E := \left\{ \{(i,j), (i',j')\} \mid \begin{array}{l} i \neq i' \text{ und } i, i' \in \{1, \dots, m\} \text{ und } j, j' \in \{1, 2, 3\} \\ \text{und } L_{i,j} \neq \overline{L_{i',j'}} \end{array} \right\}$$

Beweis (Fortsetzung)

Für die Kanten:

- ▶ Erzeuge **keine Kanten innerhalb** der drei Knoten $\{(i, 1), (i, 2), (i, 3)\}$ **einer Klausel** für jedes i .
- ▶ Erzeuge Kanten zwischen Literalen verschiedenen Klauseln, aber **ohne dass sich zwei verbundene Literale $L_{i,j}$ und $L_{i',j'}$ widersprechen** (d.h. $L_{i,j} \neq \overline{L_{i',j'}}$).

Daher:

$$E := \left\{ \{(i,j), (i',j')\} \mid \begin{array}{l} i \neq i' \text{ und } i, i' \in \{1, \dots, m\} \text{ und } j, j' \in \{1, 2, 3\} \\ \text{und } L_{i,j} \neq \overline{L_{i',j'}} \end{array} \right\}$$

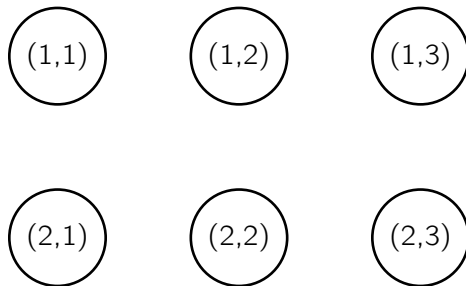
$f(F) = ((V, E), m)$ ist total und in Polynomialzeit berechenbar.

Erstes Beispiel für die Reduktion

Sei $F = (x_1 \vee x_1 \vee x_1) \wedge (\neg x_1 \vee \neg x_1 \vee \neg x_1)$.

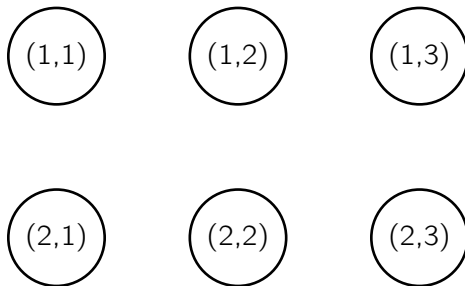
Erstes Beispiel für die Reduktion

Sei $F = (x_1 \vee x_1 \vee x_1) \wedge (\neg x_1 \vee \neg x_1 \vee \neg x_1)$. Graph:



Erstes Beispiel für die Reduktion

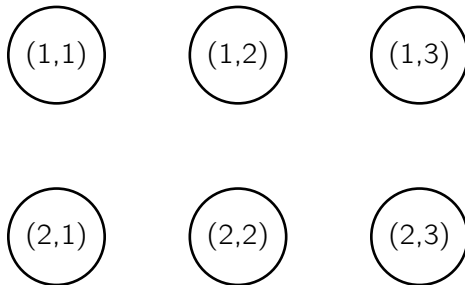
Sei $F = (x_1 \vee x_1 \vee x_1) \wedge (\neg x_1 \vee \neg x_1 \vee \neg x_1)$. Graph:



Es gibt keine Kanten, da sich $(1, i)$ und $(2, j)$ stets widersprechen.

Erstes Beispiel für die Reduktion

Sei $F = (x_1 \vee x_1 \vee x_1) \wedge (\neg x_1 \vee \neg x_1 \vee \neg x_1)$. Graph:



Es gibt keine Kanten, da sich $(1, i)$ und $(2, j)$ stets widersprechen.

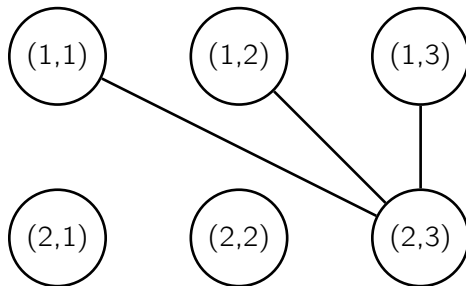
Es gibt keine Clique der Größe 2 aber auch keine erfüllende Belegung der Variablen von F .

Zweites Beispiel für die Reduktion

Sei $F = (x_1 \vee x_1 \vee x_1) \wedge (\neg x_1 \vee \neg x_1 \vee \neg x_2)$.

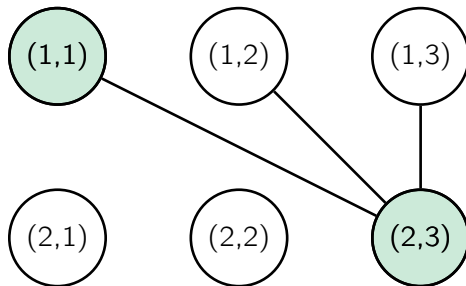
Zweites Beispiel für die Reduktion

Sei $F = (x_1 \vee x_1 \vee x_1) \wedge (\neg x_1 \vee \neg x_1 \vee \neg x_2)$. Graph:



Zweites Beispiel für die Reduktion

Sei $F = (x_1 \vee x_1 \vee x_1) \wedge (\neg x_1 \vee \neg x_1 \vee \neg x_2)$. Graph:



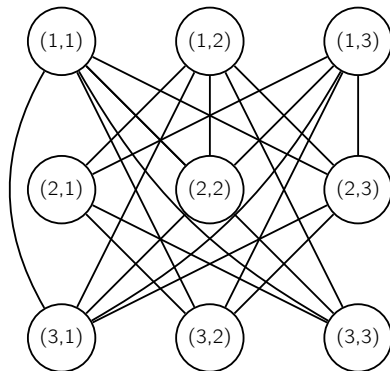
Z.B. ist $\{(1, 1), (2, 3)\}$ eine Clique der Größe 2.
Die zugehörige Belegung ist $I(x_1) = 1, I(x_2) = 0$.

Drittes Beispiel für die Reduktion

Sei $F = (x_1 \vee x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \neg x_2 \vee \neg x_3)$.

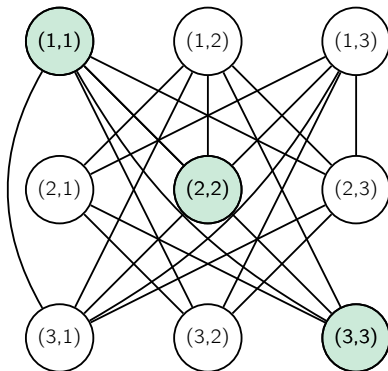
Drittes Beispiel für die Reduktion

Sei $F = (x_1 \vee x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \neg x_2 \vee \neg x_3)$. Graph:



Drittes Beispiel für die Reduktion

Sei $F = (x_1 \vee x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \neg x_2 \vee \neg x_3)$. Graph:



Z.B. ist $\{(1, 1), (2, 2), (3, 3)\}$ eine Clique der Größe 3.

Die zugehörige Belegung ist $I(x_1) = 1, I(x_2) = 1, I(x_3) = 0$.

Beweis (Fortsetzung)

Zu zeigen: F ist erfüllbar g.d.w. (V, E) eine Clique der Größe m hat.

$\implies$ Wenn F erfüllbar ist, gibt es eine Belegung I , sodass in jeder Klausel K_i mindestens ein Literal wahr ist.

Beweis (Fortsetzung)

Zu zeigen: F ist erfüllbar g.d.w. (V, E) eine Clique der Größe m hat.

$\implies$ Wenn F erfüllbar ist, gibt es eine Belegung I , sodass in jeder Klausel K_i mindestens ein Literal wahr ist.

D.h. es gibt Literale $L_{1,j_1}, \dots, L_{m,j_m}$ mit $I(L_{1,j_1}) = 1, \dots, I(L_{m,j_m}) = 1$.

Beweis (Fortsetzung)

Zu zeigen: F ist erfüllbar g.d.w. (V, E) eine Clique der Größe m hat.

$\implies$ Wenn F erfüllbar ist, gibt es eine Belegung I , sodass in jeder Klausel K_i mindestens ein Literal wahr ist.

D.h. es gibt Literale $L_{1,j_1}, \dots, L_{m,j_m}$ mit $I(L_{1,j_1}) = 1, \dots, I(L_{m,j_m}) = 1$.

Diese Literale können sich paarweise nicht widersprechen.

D.h. sie sind im Graph paarweise miteinander verbunden.

Beweis (Fortsetzung)

Zu zeigen: F ist erfüllbar g.d.w. (V, E) eine Clique der Größe m hat.

⇒ Wenn F erfüllbar ist, gibt es eine Belegung I , sodass in jeder Klausel K_i mindestens ein Literal wahr ist.

D.h. es gibt Literale $L_{1,j_1}, \dots, L_{m,j_m}$ mit $I(L_{1,j_1}) = 1, \dots, I(L_{m,j_m}) = 1$.

Diese Literale können sich paarweise nicht widersprechen.

D.h. sie sind im Graph paarweise miteinander verbunden.

Daher formt $\{(1, j_1), \dots, (m, j_m)\}$ eine Clique der Größe m .

$\mathcal{NP}$ -Vollständigkeit von CLIQUE

Beweis (Fortsetzung)

- ⇐ Wir nehmen an, dass (V, E) eine Clique der Größe m hat und beweisen, dass F erfüllbar ist.

$\mathcal{NP}$ -Vollständigkeit von CLIQUE

Beweis (Fortsetzung)

⇐ Wir nehmen an, dass (V, E) eine Clique der Größe m hat und beweisen, dass F erfüllbar ist.

Sei $V' = \{(i_1, j_1), \dots, (i_m, j_m)\}$ eine Clique der Größe m .

$\mathcal{NP}$ -Vollständigkeit von CLIQUE

Beweis (Fortsetzung)

⇐ Wir nehmen an, dass (V, E) eine Clique der Größe m hat und beweisen, dass F erfüllbar ist.

Sei $V' = \{(i_1, j_1), \dots, (i_m, j_m)\}$ eine Clique der Größe m .

Da (i, x) und (i, y) nie miteinander verbunden sind, müssen alle $i_1, \dots, i_m$ paarweise verschieden sein, also $\{i_1, \dots, i_m\} = \{1, \dots, m\}$. Mit anderen Worten gehören die Literale zu verschiedenen Klauseln.

$\mathcal{NP}$ -Vollständigkeit von CLIQUE

Beweis (Fortsetzung)

⇐ Wir nehmen an, dass (V, E) eine Clique der Größe m hat und beweisen, dass F erfüllbar ist.

Sei $V' = \{(i_1, j_1), \dots, (i_m, j_m)\}$ eine Clique der Größe m .

Da (i, x) und (i, y) nie miteinander verbunden sind, müssen alle $i_1, \dots, i_m$ paarweise verschieden sein, also $\{i_1, \dots, i_m\} = \{1, \dots, m\}$. Mit anderen Worten gehören die Literale zu verschiedenen Klauseln.

Da sie verbunden sind, widersprechen sich die Literale $L_{i_1, j_1}, \dots, L_{i_m, j_m}$ paarweise nicht.

$\mathcal{NP}$ -Vollständigkeit von CLIQUE

Beweis (Fortsetzung)

⇐ Wir nehmen an, dass (V, E) eine Clique der Größe m hat und beweisen, dass F erfüllbar ist.

Sei $V' = \{(i_1, j_1), \dots, (i_m, j_m)\}$ eine Clique der Größe m .

Da (i, x) und (i, y) nie miteinander verbunden sind, müssen alle $i_1, \dots, i_m$ paarweise verschieden sein, also $\{i_1, \dots, i_m\} = \{1, \dots, m\}$. Mit anderen Worten gehören die Literale zu verschiedenen Klauseln.

Da sie verbunden sind, widersprechen sich die Literale $L_{i_1, j_1}, \dots, L_{i_m, j_m}$ paarweise nicht.

Definiere die Belegung I mit $I(x) = 1$ wenn $L_{i_k, j_k} = x$ und $I(x) = 0$ wenn $L_{i_k, j_k} = \neg x$ und $I(y) = 1$ für alle anderen Variablen.

$\mathcal{NP}$ -Vollständigkeit von CLIQUE

Beweis (Fortsetzung)

⇐ Wir nehmen an, dass (V, E) eine Clique der Größe m hat und beweisen, dass F erfüllbar ist.

Sei $V' = \{(i_1, j_1), \dots, (i_m, j_m)\}$ eine Clique der Größe m .

Da (i, x) und (i, y) nie miteinander verbunden sind, müssen alle $i_1, \dots, i_m$ paarweise verschieden sein, also $\{i_1, \dots, i_m\} = \{1, \dots, m\}$. Mit anderen Worten gehören die Literale zu verschiedenen Klauseln.

Da sie verbunden sind, widersprechen sich die Literale $L_{i_1, j_1}, \dots, L_{i_m, j_m}$ paarweise nicht.

Definiere die Belegung I mit $I(x) = 1$ wenn $L_{i_k, j_k} = x$ und $I(x) = 0$ wenn $L_{i_k, j_k} = \neg x$ und $I(y) = 1$ für alle anderen Variablen.

I macht die Literale $L_{i_1, j_1}, \dots, L_{i_m, j_m}$ wahr und da mindestens ein Literal pro Klausel wahr ist, ist F wahr.

$\mathcal{NP}$ -Vollständigkeit von CLIQUE

Beweis (Fortsetzung)

$\Leftarrow$ Wir nehmen an, dass (V, E) eine Clique der Größe m hat und beweisen, dass F erfüllbar ist.

Sei $V' = \{(i_1, j_1), \dots, (i_m, j_m)\}$ eine Clique der Größe m .

Da (i, x) und (i, y) nie miteinander verbunden sind, müssen alle $i_1, \dots, i_m$ paarweise verschieden sein, also $\{i_1, \dots, i_m\} = \{1, \dots, m\}$. Mit anderen Worten gehören die Literale zu verschiedenen Klauseln.

Da sie verbunden sind, widersprechen sich die Literale $L_{i_1, j_1}, \dots, L_{i_m, j_m}$ paarweise nicht.

Definiere die Belegung I mit $I(x) = 1$ wenn $L_{i_k, j_k} = x$ und $I(x) = 0$ wenn $L_{i_k, j_k} = \neg x$ und $I(y) = 1$ für alle anderen Variablen.

I macht die Literale $L_{i_1, j_1}, \dots, L_{i_m, j_m}$ wahr und da mindestens ein Literal pro Klausel wahr ist, ist F wahr.

Daher: $3\text{-CNF-SAT} \leq_p \text{CLIQUE}$.

Da 3-CNF-SAT $\mathcal{NP}$ -schwer ist, folgt dass CLIQUE $\mathcal{NP}$ -schwer ist. □

Unabhängige Knotenmenge

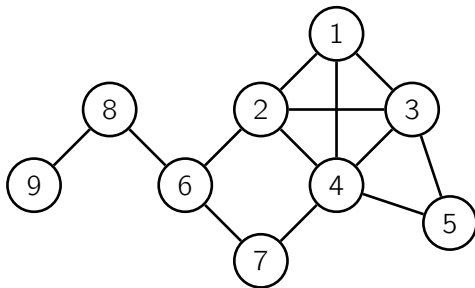
Definition

Für einen Graphen $G = (V, E)$ ist $V' \subseteq V$ eine **unabhängige Knotenmenge** (*independent set*), wenn keine zwei Knoten aus V' über eine Kante verbunden sind, d.h. $u, v \in V'$ impliziert $\{u, v\} \notin E$.

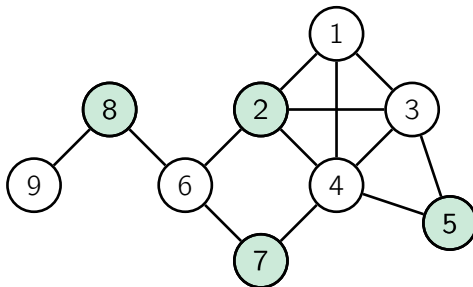
Beachte:

- ▶ $\emptyset$ ist immer eine unabhängige Knotenmenge.
- ▶ Man möchte typischerweise eine große Menge V' finden.

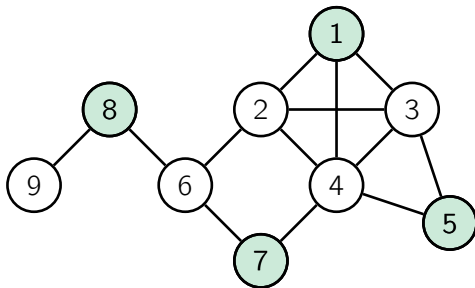
Beispiele für unabhängige Knotenmengen



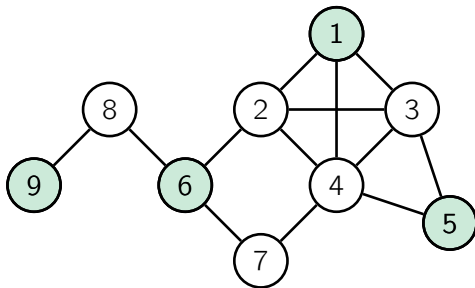
Beispiele für unabhängige Knotenmengen



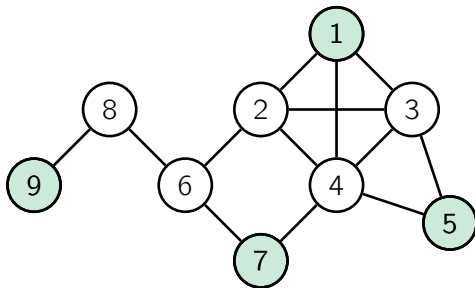
Beispiele für unabhängige Knotenmengen



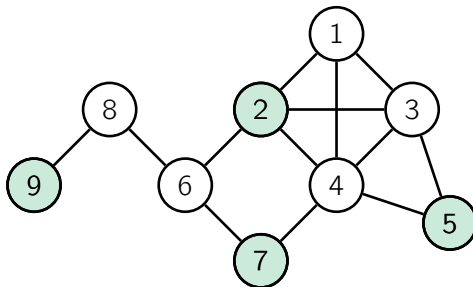
Beispiele für unabhängige Knotenmengen



Beispiele für unabhängige Knotenmengen



Beispiele für unabhängige Knotenmengen

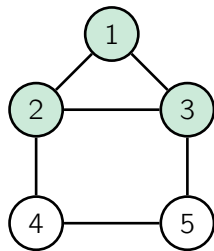


Komplementgraph

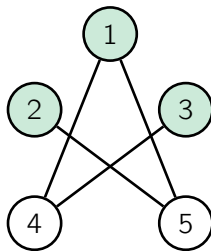
Für einen Graphen $G = (V, E)$ ist der **Komplementgraph zu G** der Graph $\overline{G} = (V, \overline{E})$ mit

$$\overline{E} := \{\{u, v\} \mid u, v \in V, u \neq v, \{u, v\} \notin E\}$$

Beispiel für den Komplementgraphen

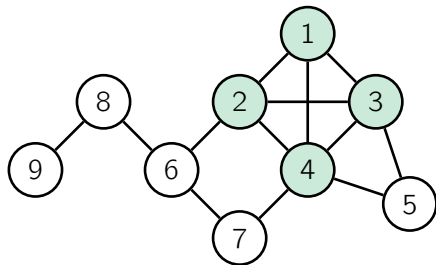


Graph

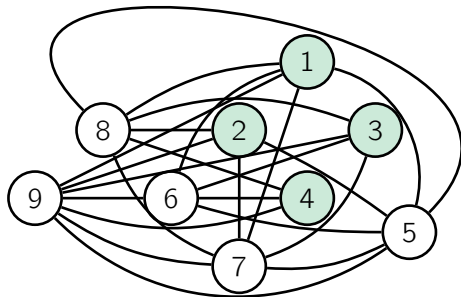


Komplementgraph

Weiteres Beispiel für den Komplementgraphen



Graph



Komplementgraph

Unabhängige Knotenmengen vs. Cliques

Lemma

Für jeden ungerichteten Graph G gilt: G hat eine unabhängige Knotenmenge der Größe k g.d.w. $\overline{G}$ eine Clique der Größe k hat.

Unabhängige Knotenmengen vs. Cliques

Lemma

Für jeden ungerichteten Graph G gilt: G hat eine unabhängige Knotenmenge der Größe k g.d.w. $\overline{G}$ eine Clique der Größe k hat.

Beweis Sei $V' \subseteq V$.

Unabhängige Knotenmengen vs. Cliques

Lemma

Für jeden ungerichteten Graph G gilt: G hat eine unabhängige Knotenmenge der Größe k g.d.w. $\overline{G}$ eine Clique der Größe k hat.

Beweis Sei $V' \subseteq V$.

V' ist eine unabhängige Knotenmenge der Größe k

Unabhängige Knotenmengen vs. Cliques

Lemma

Für jeden ungerichteten Graph G gilt: G hat eine unabhängige Knotenmenge der Größe k g.d.w. $\overline{G}$ eine Clique der Größe k hat.

Beweis Sei $V' \subseteq V$.

V' ist eine unabhängige Knotenmenge der Größe k
g.d.w. $\{u, v\} \notin E$ für $u, v \in V'$ und $|V'| = k$

Unabhängige Knotenmengen vs. Cliques

Lemma

Für jeden ungerichteten Graph G gilt: G hat eine unabhängige Knotenmenge der Größe k g.d.w. $\overline{G}$ eine Clique der Größe k hat.

Beweis Sei $V' \subseteq V$.

V' ist eine unabhängige Knotenmenge der Größe k
g.d.w. $\{u, v\} \notin E$ für $u, v \in V'$ und $|V'| = k$

V' ist eine Clique der Größe k in $\overline{G}$

Unabhängige Knotenmengen vs. Cliques

Lemma

Für jeden ungerichteten Graph G gilt: G hat eine unabhängige Knotenmenge der Größe k g.d.w. $\overline{G}$ eine Clique der Größe k hat.

Beweis Sei $V' \subseteq V$.

V' ist eine unabhängige Knotenmenge der Größe k
g.d.w. $\{u, v\} \notin E$ für $u, v \in V'$ und $|V'| = k$

$\{u, v\} \in \overline{E}$ für $u, v \in V'$, $u \neq v$ und $|V'| = k$
g.d.w. V' ist eine Clique der Größe k in $\overline{G}$



Unabhängige Knotenmengen vs. Cliques

Lemma

Für jeden ungerichteten Graph G gilt: G hat eine unabhängige Knotenmenge der Größe k g.d.w. $\overline{G}$ eine Clique der Größe k hat.

Beweis Sei $V' \subseteq V$.

V' ist eine unabhängige Knotenmenge der Größe k
g.d.w. $\{u, v\} \notin E$ für $u, v \in V'$ und $|V'| = k$

g.d.w. $\{u, v\} \in \overline{E}$ für $u, v \in V'$, $u \neq v$ und $|V'| = k$

g.d.w. V' ist eine Clique der Größe k in $\overline{G}$



Das INDEPENDENT-SET-Problem

Definition

Das **INDEPENDENT-SET-Problem** lässt sich wie folgt formulieren.

gegeben: ein ungerichteter Graph $G = (V, E)$ und eine Zahl $k \in \mathbb{N}$

gefragt: Besitzt G eine unabhängige Knotenmenge der Größe k ?

$\mathcal{NP}$ -Vollständigkeit von INDEPENDENT-SET

Satz

INDEPENDENT-SET ist $\mathcal{NP}$ -vollständig.

$\mathcal{NP}$ -Vollständigkeit von INDEPENDENT-SET

Satz

INDEPENDENT-SET ist $\mathcal{NP}$ -vollständig.

Beweis

1. INDEPENDENT-SET $\in \mathcal{NP}$:

Rate nichtdeterministisch eine Menge $V' \subseteq V$ von k Knoten.

Verifiziere deterministisch, ob für jedes Paar $\{u, v\} \in V'$ gilt: $\{u, v\} \notin E$.

Dies geht in Polynomialzeit.

Daher kann INDEPENDENT-SET in Polynomialzeit auf einer NTM entschieden werden.

$\mathcal{NP}$ -Vollständigkeit von INDEPENDENT-SET

Beweis

2. INDEPENDENT-SET ist $\mathcal{NP}$ -schwer:

Wir zeigen $\text{CLIQUE} \leq_p \text{INDEPENDENT-SET}$.

$\mathcal{NP}$ -Vollständigkeit von INDEPENDENT-SET

Beweis

2. INDEPENDENT-SET ist $\mathcal{NP}$ -schwer:

Wir zeigen $\text{CLIQUE} \leq_p \text{INDEPENDENT-SET}$.

Sei $f((V, E), m) = ((V, \bar{E}), m)$ mit $\bar{E} = \{\{u, v\} \mid u, v \in V, \{u, v\} \notin E\}$.

$\mathcal{NP}$ -Vollständigkeit von INDEPENDENT-SET

Beweis

2. INDEPENDENT-SET ist $\mathcal{NP}$ -schwer:

Wir zeigen $\text{CLIQUE} \leq_p \text{INDEPENDENT-SET}$.

Sei $f((V, E), m) = ((V, \bar{E}), m)$ mit $\bar{E} = \{\{u, v\} \mid u, v \in V, \{u, v\} \notin E\}$.

Die Funktion f ist total und kann in Polynomialzeit berechnet werden.

$\mathcal{NP}$ -Vollständigkeit von INDEPENDENT-SET

Beweis

2. INDEPENDENT-SET ist $\mathcal{NP}$ -schwer:

Wir zeigen $\text{CLIQUE} \leq_p \text{INDEPENDENT-SET}$.

Sei $f((V, E), m) = ((V, \bar{E}), m)$ mit $\bar{E} = \{\{u, v\} \mid u, v \in V, \{u, v\} \notin E\}$.

Die Funktion f ist total und kann in Polynomialzeit berechnet werden.

Dann gilt: (V, E) hat eine Clique der Größe m g.d.w.

$(V, \bar{E})$ eine unabhängige Knotenmenge der Größe m hat.

$\mathcal{NP}$ -Vollständigkeit von INDEPENDENT-SET

Beweis

2. INDEPENDENT-SET ist $\mathcal{NP}$ -schwer:

Wir zeigen $\text{CLIQUE} \leq_p \text{INDEPENDENT-SET}$.

Sei $f((V, E), m) = ((V, \bar{E}), m)$ mit $\bar{E} = \{\{u, v\} \mid u, v \in V, \{u, v\} \notin E\}$.

Die Funktion f ist total und kann in Polynomialzeit berechnet werden.

Dann gilt: (V, E) hat eine Clique der Größe m g.d.w.

$(V, \bar{E})$ eine unabhängige Knotenmenge der Größe m hat.

Daher: $\text{CLIQUE} \leq_p \text{INDEPENDENT-SET}$.

Da CLIQUE $\mathcal{NP}$ -schwer ist, folgt dass INDEPENDENT-SET $\mathcal{NP}$ -schwer ist. □

Überdeckende Knotenmenge

Definition

Für einen Graphen $G = (V, E)$ ist $V' \subseteq V$ eine **überdeckende Knotenmenge** (*vertex cover*), wenn jede Kante aus E mindestens einen Knoten in V' hat, d.h. für alle Knoten $u, v \in V$: $\{u, v\} \in E$ impliziert $u \in V' \vee v \in V'$.

Überdeckende Knotenmenge

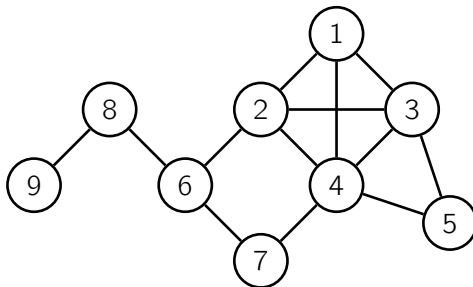
Definition

Für einen Graphen $G = (V, E)$ ist $V' \subseteq V$ eine **überdeckende Knotenmenge** (*vertex cover*), wenn jede Kante aus E mindestens einen Knoten in V' hat, d.h. für alle Knoten $u, v \in V$: $\{u, v\} \in E$ impliziert $u \in V' \vee v \in V'$.

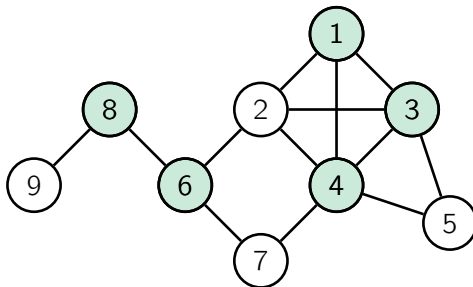
Beachte:

- ▶ V ist immer eine überdeckende Knotenmenge.
- ▶ Man möchte typischerweise eine kleine Menge V' finden.

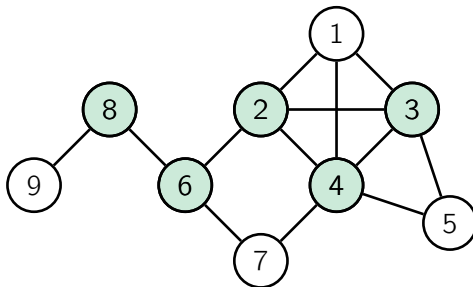
Beispiele für überdeckende Knotenmengen



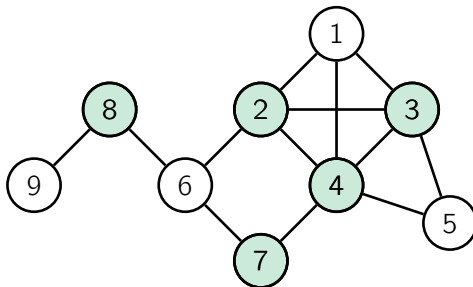
Beispiele für überdeckende Knotenmengen



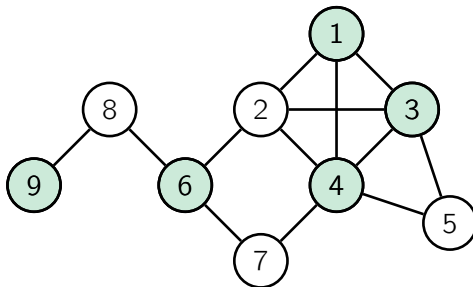
Beispiele für überdeckende Knotenmengen



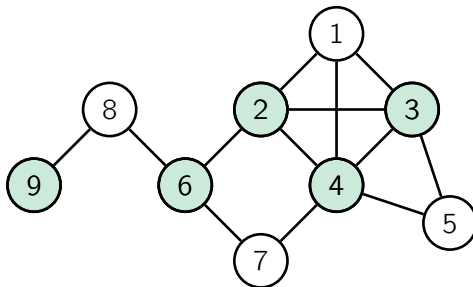
Beispiele für überdeckende Knotenmengen



Beispiele für überdeckende Knotenmengen



Beispiele für überdeckende Knotenmengen



Überdeckende Knotenmengen vs. unabhängige Knotenmengen

Lemma

$G = (V, E)$ hat eine unabhängige Knotenmenge der Größe k g.d.w.
 G hat eine überdeckende Knotenmenge der Größe $|V| - k$.

Überdeckende Knotenmengen vs. unabhängige Knotenmengen

Lemma

$G = (V, E)$ hat eine unabhängige Knotenmenge der Größe k g.d.w.
 G hat eine überdeckende Knotenmenge der Größe $|V| - k$.

Beweis Sei $V' \subseteq V$.

Überdeckende Knotenmengen vs. unabhängige Knotenmengen

Lemma

$G = (V, E)$ hat eine unabhängige Knotenmenge der Größe k g.d.w.
 G hat eine überdeckende Knotenmenge der Größe $|V| - k$.

Beweis Sei $V' \subseteq V$.

V' ist eine unabhängige Knotenmenge

Überdeckende Knotenmengen vs. unabhängige Knotenmengen

Lemma

$G = (V, E)$ hat eine unabhängige Knotenmenge der Größe k g.d.w.
 G hat eine überdeckende Knotenmenge der Größe $|V| - k$.

Beweis Sei $V' \subseteq V$.

V' ist eine unabhängige Knotenmenge
g.d.w. $\{u, v\} \notin E$ für alle $u, v \in V'$

Überdeckende Knotenmengen vs. unabhängige Knotenmengen

Lemma

$G = (V, E)$ hat eine unabhängige Knotenmenge der Größe k g.d.w.
 G hat eine überdeckende Knotenmenge der Größe $|V| - k$.

Beweis Sei $V' \subseteq V$.

V' ist eine unabhängige Knotenmenge

g.d.w. $\{u, v\} \notin E$ für alle $u, v \in V'$

g.d.w. $u \notin V' \vee v \notin V'$ für alle $\{u, v\} \in E$

Überdeckende Knotenmengen vs. unabhängige Knotenmengen

Lemma

$G = (V, E)$ hat eine unabhängige Knotenmenge der Größe k g.d.w.
 G hat eine überdeckende Knotenmenge der Größe $|V| - k$.

Beweis Sei $V' \subseteq V$.

V' ist eine unabhängige Knotenmenge

g.d.w. $\{u, v\} \notin E$ für alle $u, v \in V'$

g.d.w. $u \notin V' \vee v \notin V'$ für alle $\{u, v\} \in E$

$V \setminus V'$ ist eine überdeckende Knotenmenge

Überdeckende Knotenmengen vs. unabhängige Knotenmengen

Lemma

$G = (V, E)$ hat eine unabhängige Knotenmenge der Größe k g.d.w.
 G hat eine überdeckende Knotenmenge der Größe $|V| - k$.

Beweis Sei $V' \subseteq V$.

V' ist eine unabhängige Knotenmenge

g.d.w. $\{u, v\} \notin E$ für alle $u, v \in V'$

g.d.w. $u \notin V' \vee v \notin V'$ für alle $\{u, v\} \in E$

$(u \in V \setminus V') \vee (v \in V \setminus V')$ für alle $\{u, v\} \in E$

g.d.w. $V \setminus V'$ ist eine überdeckende Knotenmenge

Überdeckende Knotenmengen vs. unabhängige Knotenmengen

Lemma

$G = (V, E)$ hat eine unabhängige Knotenmenge der Größe k g.d.w.
 G hat eine überdeckende Knotenmenge der Größe $|V| - k$.

Beweis Sei $V' \subseteq V$.

V' ist eine unabhängige Knotenmenge

g.d.w. $\{u, v\} \notin E$ für alle $u, v \in V'$

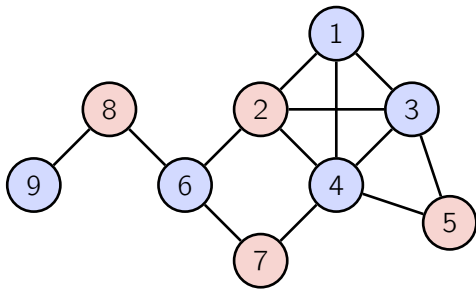
g.d.w. $u \notin V' \vee v \notin V'$ für alle $\{u, v\} \in E$

g.d.w. $(u \in V \setminus V') \vee (v \in V \setminus V')$ für alle $\{u, v\} \in E$

g.d.w. $V \setminus V'$ ist eine überdeckende Knotenmenge



Beispiel



unabhängige Knotenmenge



überdeckende Knotenmenge

Das VERTEX-COVER-Problem

Definition

Das VERTEX-COVER-Problem lässt sich wie folgt formulieren.

gegeben: ein ungerichteter Graph $G = (V, E)$ und eine Zahl $k \in \mathbb{N}$

gefragt: Besitzt G eine überdeckende Knotenmenge der Größe höchstens k ?

$\mathcal{NP}$ -Vollständigkeit von VERTEX-COVER

Satz

VERTEX-COVER ist $\mathcal{NP}$ -vollständig.

$\mathcal{NP}$ -Vollständigkeit von VERTEX-COVER

Satz

VERTEX-COVER ist $\mathcal{NP}$ -vollständig.

Beweis

1. VERTEX-COVER $\in \mathcal{NP}$:

Sei $G = (V, E)$.

Rate nichtdeterministisch eine Menge $V' \subseteq V$ von k Knoten.

Prüfe deterministisch (in Polynomialzeit), ob für alle $\{u, v\} \in E$: $u \in V' \vee v \in V'$.

D.h. VERTEX-COVER wird in Polynomialzeit von einer NTM entschieden.

$\mathcal{NP}$ -Vollständigkeit von VERTEX-COVER

Beweis (Fortsetzung)

2. VERTEX-COVER ist $\mathcal{NP}$ -schwer:

Wir zeigen $\text{INDEPENDENT-SET} \leq_p \text{VERTEX-COVER}$.

$\mathcal{NP}$ -Vollständigkeit von VERTEX-COVER

Beweis (Fortsetzung)

2. VERTEX-COVER ist $\mathcal{NP}$ -schwer:

Wir zeigen $\text{INDEPENDENT-SET} \leq_p \text{VERTEX-COVER}$.

Sei $f((V, E, m)) = (V, E, |V| - m)$.

$\mathcal{NP}$ -Vollständigkeit von VERTEX-COVER

Beweis (Fortsetzung)

2. VERTEX-COVER ist $\mathcal{NP}$ -schwer:

Wir zeigen $\text{INDEPENDENT-SET} \leq_p \text{VERTEX-COVER}$.

Sei $f((V, E, m)) = (V, E, |V| - m)$.

Die Funktion f ist total und kann in Polynomialzeit berechnet werden.

$\mathcal{NP}$ -Vollständigkeit von VERTEX-COVER

Beweis (Fortsetzung)

2. VERTEX-COVER ist $\mathcal{NP}$ -schwer:

Wir zeigen $\text{INDEPENDENT-SET} \leq_p \text{VERTEX-COVER}$.

Sei $f((V, E, m)) = (V, E, |V| - m)$.

Die Funktion f ist total und kann in Polynomialzeit berechnet werden.

Zudem gilt:

(V, E) hat eine unabhängige Knotenmenge der Größe m g.d.w.

(V, E) hat eine überdeckende Knotenmenge der Größe $|V| - m$.

$\mathcal{NP}$ -Vollständigkeit von VERTEX-COVER

Beweis (Fortsetzung)

2. VERTEX-COVER ist $\mathcal{NP}$ -schwer:

Wir zeigen $\text{INDEPENDENT-SET} \leq_p \text{VERTEX-COVER}$.

Sei $f((V, E, m)) = (V, E, |V| - m)$.

Die Funktion f ist total und kann in Polynomialzeit berechnet werden.

Zudem gilt:

(V, E) hat eine unabhängige Knotenmenge der Größe m g.d.w.

(V, E) hat eine überdeckende Knotenmenge der Größe $|V| - m$.

Daher: $\text{INDEPENDENT-SET} \leq_p \text{VERTEX-COVER}$.

Da INDEPENDENT-SET $\mathcal{NP}$ -schwer ist, folgt dass VERTEX-COVER $\mathcal{NP}$ -schwer ist. □

13a

 **$\mathcal{NP}$ -Vollständigkeit von SET-COVER, SUBSET-SUM,
KNAPSACK, PARTITION und BIN-PACKING**

Prof. Dr. Jasmin Blanchette

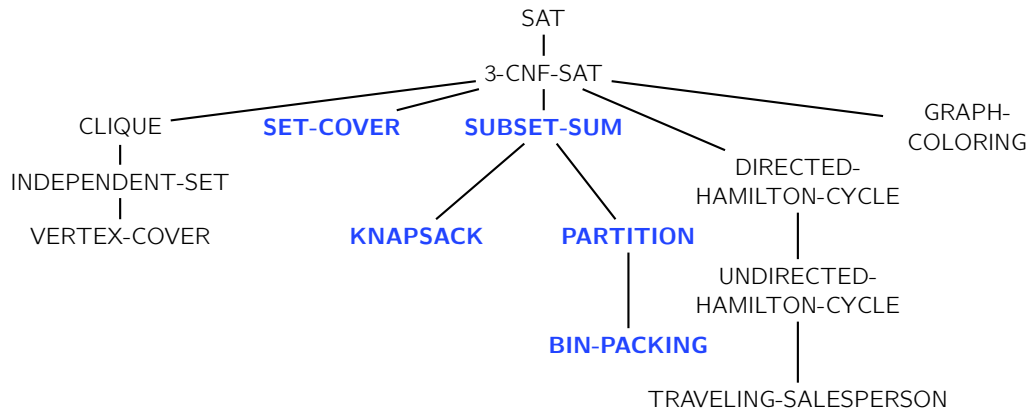
Lehr- und Forschungseinheit für
Theoretische Informatik und Theorembeweisen

Stand: 16. Juli 2024

Basierend auf Folien von PD Dr. David Sabel



Überblick über $\mathcal{NP}$ -Vollständigkeitsbeweise



Das SET-COVER-Problem

Definition

Das **SET-COVER-Problem** lässt sich wie folgt formulieren.

gegeben: Mengen $T_1, \dots, T_k$ mit $T_1, \dots, T_k \subseteq M$,
wobei M eine endliche Grundmenge ist, und
eine Zahl $n \leq k$

gefragt: Gibt es eine Auswahl von n Mengen $T_{i_1}, \dots, T_{i_n}$ ($i_j \in \{1, \dots, k\}$) mit
 $T_{i_1} \cup \dots \cup T_{i_n} = M$?

Das SET-COVER-Problem

Definition

Das **SET-COVER-Problem** lässt sich wie folgt formulieren.

gegeben: Mengen $T_1, \dots, T_k$ mit $T_1, \dots, T_k \subseteq M$,
wobei M eine endliche Grundmenge ist, und
eine Zahl $n \leq k$

gefragt: Gibt es eine Auswahl von n Mengen $T_{i_1}, \dots, T_{i_n}$ ($i_j \in \{1, \dots, k\}$) mit
 $T_{i_1} \cup \dots \cup T_{i_n} = M$?

Beispiel:

Gegeben seien $T_1 = \{1, 2, 3, 5\}$, $T_2 = \{1, 2\}$, $T_3 = \{3, 4\}$, $T_4 = \{3\}$ mit
 $M = \{1, 2, 3, 4, 5\}$ und $n = 2$.

Das SET-COVER-Problem

Definition

Das **SET-COVER-Problem** lässt sich wie folgt formulieren.

gegeben: Mengen $T_1, \dots, T_k$ mit $T_1, \dots, T_k \subseteq M$,
wobei M eine endliche Grundmenge ist, und
eine Zahl $n \leq k$

gefragt: Gibt es eine Auswahl von n Mengen $T_{i_1}, \dots, T_{i_n}$ ($i_j \in \{1, \dots, k\}$) mit
 $T_{i_1} \cup \dots \cup T_{i_n} = M$?

Beispiel:

Gegeben seien $T_1 = \{1, 2, 3, 5\}$, $T_2 = \{1, 2\}$, $T_3 = \{3, 4\}$, $T_4 = \{3\}$ mit
 $M = \{1, 2, 3, 4, 5\}$ und $n = 2$.

Lösung: T_1, T_3 , da $T_1 \cup T_3 = M$.

$\mathcal{NP}$ -Vollständigkeit von SET-COVER

Satz

SET-COVER ist $\mathcal{NP}$ -vollständig.

$\mathcal{NP}$ -Vollständigkeit von SET-COVER

Satz

SET-COVER ist $\mathcal{NP}$ -vollständig.

Beweis

1. SET-COVER $\in \mathcal{NP}$:

Rate nichtdeterministisch die n Mengen $T_{i_1}, \dots, T_{i_n}$.

Verifiziere deterministisch, ob $T_{i_1} \cup \dots \cup T_{i_n} = M$ gilt.

Daher kann SET-COVER in Polynomialzeit auf einer NTM entschieden werden.

$\mathcal{NP}$ -Vollständigkeit von SET-COVER

Beweis (Fortsetzung)

2. SET-COVER ist $\mathcal{NP}$ -schwer, erster Versuch:

Wir zeigen $3\text{-CNF-SAT} \leq_p \text{SET-COVER}$.

$\mathcal{NP}$ -Vollständigkeit von SET-COVER

Beweis (Fortsetzung)

2. SET-COVER ist $\mathcal{NP}$ -schwer, erster Versuch:

Wir zeigen $3\text{-CNF-SAT} \leq_p \text{SET-COVER}$.

Sei $F = K_1 \wedge \dots \wedge K_m$ eine 3-CNF.

$\mathcal{NP}$ -Vollständigkeit von SET-COVER

Beweis (Fortsetzung)

2. SET-COVER ist $\mathcal{NP}$ -schwer, erster Versuch:

Wir zeigen $3\text{-CNF-SAT} \leq_p \text{SET-COVER}$.

Sei $F = K_1 \wedge \dots \wedge K_m$ eine 3-CNF.

Seien $x_1, \dots, x_n$ die in F vorkommenden aussagenlogischen Variablen.

$\mathcal{NP}$ -Vollständigkeit von SET-COVER

Beweis (Fortsetzung)

2. SET-COVER ist $\mathcal{NP}$ -schwer, erster Versuch:

Wir zeigen $3\text{-CNF-SAT} \leq_p \text{SET-COVER}$.

Sei $F = K_1 \wedge \dots \wedge K_m$ eine 3-CNF.

Seien $x_1, \dots, x_n$ die in F vorkommenden aussagenlogischen Variablen.

Setze $M := \{1, \dots, m\}$.

$\mathcal{NP}$ -Vollständigkeit von SET-COVER

Beweis (Fortsetzung)

2. SET-COVER ist $\mathcal{NP}$ -schwer, erster Versuch:

Wir zeigen $3\text{-CNF-SAT} \leq_p \text{SET-COVER}$.

Sei $F = K_1 \wedge \dots \wedge K_m$ eine 3-CNF.

Seien $x_1, \dots, x_n$ die in F vorkommenden aussagenlogischen Variablen.

Setze $M := \{1, \dots, m\}$.

Für $i \in \{1, \dots, n\}$ sei

$$P_i := \{j \mid \text{Literal } x_i \text{ kommt in Klausel } K_j \text{ vor}\}$$

$$N_i := \{j \mid \text{Literal } \neg x_i \text{ kommt in Klausel } K_j \text{ vor}\}$$

$\mathcal{NP}$ -Vollständigkeit von SET-COVER

Beweis (Fortsetzung)

2. SET-COVER ist $\mathcal{NP}$ -schwer, erster Versuch:

Wir zeigen $3\text{-CNF-SAT} \leq_p \text{SET-COVER}$.

Sei $F = K_1 \wedge \dots \wedge K_m$ eine 3-CNF.

Seien $x_1, \dots, x_n$ die in F vorkommenden aussagenlogischen Variablen.

Setze $M := \{1, \dots, m\}$.

Für $i \in \{1, \dots, n\}$ sei

$$P_i := \{j \mid \text{Literal } x_i \text{ kommt in Klausel } K_j \text{ vor}\}$$

$$N_i := \{j \mid \text{Literal } \neg x_i \text{ kommt in Klausel } K_j \text{ vor}\}$$

Das Mengensystem sei $P_1, \dots, P_n, N_1, \dots, N_n \subseteq M$.

Gesucht wird eine Vereinigung von n Mengen.

$\mathcal{NP}$ -Vollständigkeit von SET-COVER

Beweis (Fortsetzung)

2. SET-COVER ist $\mathcal{NP}$ -schwer, erster Versuch:

Wir zeigen $3\text{-CNF-SAT} \leq_p \text{SET-COVER}$.

Sei $F = K_1 \wedge \dots \wedge K_m$ eine 3-CNF.

Seien $x_1, \dots, x_n$ die in F vorkommenden aussagenlogischen Variablen.

Setze $M := \{1, \dots, m\}$.

Für $i \in \{1, \dots, n\}$ sei

$$P_i := \{j \mid \text{Literal } x_i \text{ kommt in Klausel } K_j \text{ vor}\}$$

$$N_i := \{j \mid \text{Literal } \neg x_i \text{ kommt in Klausel } K_j \text{ vor}\}$$

Das Mengensystem sei $P_1, \dots, P_n, N_1, \dots, N_n \subseteq M$.

Gesucht wird eine Vereinigung von n Mengen.

Grundgedanke: Wenn P_i bzw. N_i gewählt wird, dann werden die Klauseln, die das Literal enthalten, erfüllt. Das Ziel ist, alle m Klauseln zu erfüllen.

Beispiel für den ersten Versuch

3-CNF:

$$(x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \neg x_2) \wedge (\neg x_1 \vee \neg x_2 \vee x_3)$$

Beispiel für den ersten Versuch

3-CNF:

$$(x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \neg x_2) \wedge (\neg x_1 \vee \neg x_2 \vee x_3)$$

SET-COVER-Instanz dazu:

$$P_1 = \{j \mid x_1 \text{ kommt in Klausel } K_j \text{ vor}\} = \{1, 2\}$$

$$N_1 = \{j \mid \neg x_1 \text{ kommt in Klausel } K_j \text{ vor}\} = \{3\}$$

$$P_2 = \{j \mid x_2 \text{ kommt in Klausel } K_j \text{ vor}\} = \{1\}$$

$$N_2 = \{j \mid \neg x_2 \text{ kommt in Klausel } K_j \text{ vor}\} = \{2, 3\}$$

$$P_3 = \{j \mid x_3 \text{ kommt in Klausel } K_j \text{ vor}\} = \{1, 3\}$$

$$N_3 = \{j \mid \neg x_3 \text{ kommt in Klausel } K_j \text{ vor}\} = \emptyset$$

Gesucht wird eine Vereinigung von $n = 3$ Mengen U_1, U_2, U_3 mit $U_1 \cup U_2 \cup U_3 = \{1, 2, 3\}$.

Beispiel für den ersten Versuch

3-CNF:

$$(x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \neg x_2) \wedge (\neg x_1 \vee \neg x_2 \vee x_3)$$

SET-COVER-Instanz dazu:

$$P_1 = \{j \mid x_1 \text{ kommt in Klausel } K_j \text{ vor}\} = \{1, 2\}$$

$$N_1 = \{j \mid \neg x_1 \text{ kommt in Klausel } K_j \text{ vor}\} = \{3\}$$

$$P_2 = \{j \mid x_2 \text{ kommt in Klausel } K_j \text{ vor}\} = \{1\}$$

$$N_2 = \{j \mid \neg x_2 \text{ kommt in Klausel } K_j \text{ vor}\} = \{2, 3\}$$

$$P_3 = \{j \mid x_3 \text{ kommt in Klausel } K_j \text{ vor}\} = \{1, 3\}$$

$$N_3 = \{j \mid \neg x_3 \text{ kommt in Klausel } K_j \text{ vor}\} = \emptyset$$

Gesucht wird eine Vereinigung von $n = 3$ Mengen U_1, U_2, U_3 mit $U_1 \cup U_2 \cup U_3 = \{1, 2, 3\}$.

Lösung: z.B. P_1, N_2, N_3 .

Beispiel für den ersten Versuch

3-CNF:

$$(x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \neg x_2) \wedge (\neg x_1 \vee \neg x_2 \vee x_3)$$

SET-COVER-Instanz dazu:

$$P_1 = \{j \mid x_1 \text{ kommt in Klausel } K_j \text{ vor}\} = \{1, 2\}$$

$$N_1 = \{j \mid \neg x_1 \text{ kommt in Klausel } K_j \text{ vor}\} = \{3\}$$

$$P_2 = \{j \mid x_2 \text{ kommt in Klausel } K_j \text{ vor}\} = \{1\}$$

$$N_2 = \{j \mid \neg x_2 \text{ kommt in Klausel } K_j \text{ vor}\} = \{2, 3\}$$

$$P_3 = \{j \mid x_3 \text{ kommt in Klausel } K_j \text{ vor}\} = \{1, 3\}$$

$$N_3 = \{j \mid \neg x_3 \text{ kommt in Klausel } K_j \text{ vor}\} = \emptyset$$

Gesucht wird eine Vereinigung von $n = 3$ Mengen U_1, U_2, U_3 mit $U_1 \cup U_2 \cup U_3 = \{1, 2, 3\}$.

Lösung: z.B. P_1, N_2, N_3 .

Belegung dazu: $I(x_1) = 1, I(x_2) = 0, I(x_3) = 0$.

Weiteres Beispiel für den ersten Versuch

3-CNF:

$$(x_1 \vee x_2) \wedge (\neg x_1 \vee \neg x_2) \wedge (x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_2)$$

Weiteres Beispiel für den ersten Versuch

3-CNF:

$$(x_1 \vee x_2) \wedge (\neg x_1 \vee \neg x_2) \wedge (x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_2)$$

SET-COVER-Instanz dazu:

$$P_1 = \{j \mid x_1 \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m+1\} = \{1, 3\}$$

$$N_1 = \{j \mid \neg x_1 \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m+1\} = \{2, 4\}$$

$$P_2 = \{j \mid x_2 \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m+2\} = \{1, 4\}$$

$$N_2 = \{j \mid \neg x_2 \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m+2\} = \{2, 3\}$$

Gesucht wird eine Vereinigung von $n = 2$ Mengen U_1, U_2 mit $U_1 \cup U_2 = \{1, 2, 3, 4\}$.

Lösung: z.B. P_1, N_1 .

Weiteres Beispiel für den ersten Versuch

3-CNF:

$$(x_1 \vee x_2) \wedge (\neg x_1 \vee \neg x_2) \wedge (x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_2)$$

SET-COVER-Instanz dazu:

$$P_1 = \{j \mid x_1 \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m+1\} = \{1, 3\}$$

$$N_1 = \{j \mid \neg x_1 \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m+1\} = \{2, 4\}$$

$$P_2 = \{j \mid x_2 \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m+2\} = \{1, 4\}$$

$$N_2 = \{j \mid \neg x_2 \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m+2\} = \{2, 3\}$$

Gesucht wird eine Vereinigung von $n = 2$ Mengen U_1, U_2 mit $U_1 \cup U_2 = \{1, 2, 3, 4\}$.

Lösung: z.B. P_1, N_1 .

Aber **diese Lösung** entspricht **keiner gültigen Belegung**.

$\mathcal{NP}$ -Vollständigkeit von SET-COVER

Beweis (Fortsetzung)

2. SET-COVER ist $\mathcal{NP}$ -schwer, zweiter Versuch:

Wir zeigen $3\text{-CNF-SAT} \leq_p \text{SET-COVER}$.

Sei $F = K_1 \wedge \dots \wedge K_m$ eine 3-CNF.

Seien $x_1, \dots, x_n$ die in F vorkommenden aussagenlogischen Variablen.

Setze $M := \{1, \dots, m + n\}$.

$\mathcal{NP}$ -Vollständigkeit von SET-COVER

Beweis (Fortsetzung)

2. SET-COVER ist $\mathcal{NP}$ -schwer, zweiter Versuch:

Wir zeigen $3\text{-CNF-SAT} \leq_p \text{SET-COVER}$.

Sei $F = K_1 \wedge \dots \wedge K_m$ eine 3-CNF.

Seien $x_1, \dots, x_n$ die in F vorkommenden aussagenlogischen Variablen.

Setze $M := \{1, \dots, m + n\}$.

Für $i \in \{1, \dots, n\}$ sei

$$P_i := \{j \mid \text{Literal } x_i \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m + i\}$$

$$N_i := \{j \mid \text{Literal } \neg x_i \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m + i\}$$

Das Mengensystem sei $P_1, \dots, P_n, N_1, \dots, N_n \subseteq M$.

Gesucht wird eine Vereinigung von n Mengen.

$\mathcal{NP}$ -Vollständigkeit von SET-COVER

Beweis (Fortsetzung)

2. SET-COVER ist $\mathcal{NP}$ -schwer, zweiter Versuch:

Wir zeigen $3\text{-CNF-SAT} \leq_p \text{SET-COVER}$.

Sei $F = K_1 \wedge \dots \wedge K_m$ eine 3-CNF.

Seien $x_1, \dots, x_n$ die in F vorkommenden aussagenlogischen Variablen.

Setze $M := \{1, \dots, m + n\}$.

Für $i \in \{1, \dots, n\}$ sei

$$P_i := \{j \mid \text{Literal } x_i \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m + i\}$$

$$N_i := \{j \mid \text{Literal } \neg x_i \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m + i\}$$

Das Mengensystem sei $P_1, \dots, P_n, N_1, \dots, N_n \subseteq M$.

Gesucht wird eine Vereinigung von n Mengen.

Grundgedanke: Für jedes i muss mindestens (und daher höchstens) eins von P_i, N_i gewählt werden.

Beispiel für den zweiten Versuch

3-CNF:

$$(x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \neg x_2) \wedge (\neg x_1 \vee \neg x_2 \vee x_3)$$

Beispiel für den zweiten Versuch

3-CNF:

$$(x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \neg x_2) \wedge (\neg x_1 \vee \neg x_2 \vee x_3)$$

SET-COVER-Instanz dazu:

$$P_1 = \{j \mid x_1 \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m+1\} = \{1, 2, 4\}$$

$$N_1 = \{j \mid \neg x_1 \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m+1\} = \{3, 4\}$$

$$P_2 = \{j \mid x_2 \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m+2\} = \{1, 5\}$$

$$N_2 = \{j \mid \neg x_2 \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m+2\} = \{2, 3, 5\}$$

$$P_3 = \{j \mid x_3 \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m+3\} = \{1, 3, 6\}$$

$$N_3 = \{j \mid \neg x_3 \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m+3\} = \{6\}$$

Beispiel für den zweiten Versuch

3-CNF:

$$(x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \neg x_2) \wedge (\neg x_1 \vee \neg x_2 \vee x_3)$$

SET-COVER-Instanz dazu:

$$P_1 = \{j \mid x_1 \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m+1\} = \{1, 2, 4\}$$

$$N_1 = \{j \mid \neg x_1 \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m+1\} = \{3, 4\}$$

$$P_2 = \{j \mid x_2 \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m+2\} = \{1, 5\}$$

$$N_2 = \{j \mid \neg x_2 \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m+2\} = \{2, 3, 5\}$$

$$P_3 = \{j \mid x_3 \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m+3\} = \{1, 3, 6\}$$

$$N_3 = \{j \mid \neg x_3 \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m+3\} = \{6\}$$

Gesucht wird eine Vereinigung von $n = 3$ Mengen U_1, U_2, U_3 mit $U_1 \cup U_2 \cup U_3 = \{1, 2, 3, 4, 5, 6\}$.

Beispiel für den zweiten Versuch

3-CNF:

$$(x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \neg x_2) \wedge (\neg x_1 \vee \neg x_2 \vee x_3)$$

SET-COVER-Instanz dazu:

$$P_1 = \{j \mid x_1 \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m+1\} = \{1, 2, 4\}$$

$$N_1 = \{j \mid \neg x_1 \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m+1\} = \{3, 4\}$$

$$P_2 = \{j \mid x_2 \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m+2\} = \{1, 5\}$$

$$N_2 = \{j \mid \neg x_2 \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m+2\} = \{2, 3, 5\}$$

$$P_3 = \{j \mid x_3 \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m+3\} = \{1, 3, 6\}$$

$$N_3 = \{j \mid \neg x_3 \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m+3\} = \{6\}$$

Gesucht wird eine Vereinigung von $n = 3$ Mengen U_1, U_2, U_3 mit $U_1 \cup U_2 \cup U_3 = \{1, 2, 3, 4, 5, 6\}$.

Lösung: z.B. P_1, N_2, N_3 .

Beispiel für den zweiten Versuch

3-CNF:

$$(x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \neg x_2) \wedge (\neg x_1 \vee \neg x_2 \vee x_3)$$

SET-COVER-Instanz dazu:

$$P_1 = \{j \mid x_1 \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m+1\} = \{1, 2, 4\}$$

$$N_1 = \{j \mid \neg x_1 \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m+1\} = \{3, 4\}$$

$$P_2 = \{j \mid x_2 \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m+2\} = \{1, 5\}$$

$$N_2 = \{j \mid \neg x_2 \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m+2\} = \{2, 3, 5\}$$

$$P_3 = \{j \mid x_3 \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m+3\} = \{1, 3, 6\}$$

$$N_3 = \{j \mid \neg x_3 \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m+3\} = \{6\}$$

Gesucht wird eine Vereinigung von $n = 3$ Mengen U_1, U_2, U_3 mit $U_1 \cup U_2 \cup U_3 = \{1, 2, 3, 4, 5, 6\}$.

Lösung: z.B. P_1, N_2, N_3 .

Belegung dazu: $I(x_1) = 1, I(x_2) = 0, I(x_3) = 0$.

Weiteres Beispiel für den zweiten Versuch

3-CNF:

$$(x_1 \vee x_2) \wedge (\neg x_1 \vee \neg x_2) \wedge (x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_2)$$

Weiteres Beispiel für den zweiten Versuch

3-CNF:

$$(x_1 \vee x_2) \wedge (\neg x_1 \vee \neg x_2) \wedge (x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_2)$$

SET-COVER-Instanz dazu:

$$P_1 = \{j \mid x_1 \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m+1\} = \{1, 3, 5\}$$

$$N_1 = \{j \mid \neg x_1 \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m+1\} = \{2, 4, 5\}$$

$$P_2 = \{j \mid x_2 \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m+2\} = \{1, 4, 6\}$$

$$N_2 = \{j \mid \neg x_2 \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m+2\} = \{2, 3, 6\}$$

Weiteres Beispiel für den zweiten Versuch

3-CNF:

$$(x_1 \vee x_2) \wedge (\neg x_1 \vee \neg x_2) \wedge (x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_2)$$

SET-COVER-Instanz dazu:

$$P_1 = \{j \mid x_1 \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m+1\} = \{1, 3, 5\}$$

$$N_1 = \{j \mid \neg x_1 \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m+1\} = \{2, 4, 5\}$$

$$P_2 = \{j \mid x_2 \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m+2\} = \{1, 4, 6\}$$

$$N_2 = \{j \mid \neg x_2 \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m+2\} = \{2, 3, 6\}$$

Gesucht wird eine Vereinigung von $n = 2$ Mengen U_1, U_2 mit $U_1 \cup U_2 = \{1, 2, 3, 4, 5, 6\}$.

Weiteres Beispiel für den zweiten Versuch

3-CNF:

$$(x_1 \vee x_2) \wedge (\neg x_1 \vee \neg x_2) \wedge (x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_2)$$

SET-COVER-Instanz dazu:

$$P_1 = \{j \mid x_1 \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m+1\} = \{1, 3, 5\}$$

$$N_1 = \{j \mid \neg x_1 \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m+1\} = \{2, 4, 5\}$$

$$P_2 = \{j \mid x_2 \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m+2\} = \{1, 4, 6\}$$

$$N_2 = \{j \mid \neg x_2 \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m+2\} = \{2, 3, 6\}$$

Gesucht wird eine Vereinigung von $n = 2$ Mengen U_1, U_2 mit $U_1 \cup U_2 = \{1, 2, 3, 4, 5, 6\}$.

Es gibt **keine Lösung** aber auch **keine Belegung** für die 3-CNF.

$\mathcal{NP}$ -Vollständigkeit von SET-COVER

Beweis (Fortsetzung)

Zur Erinnerung:

$$P_i := \{j \mid \text{Literal } x_i \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m + i\}$$

$$N_i := \{j \mid \text{Literal } \neg x_i \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m + i\}$$

$\mathcal{NP}$ -Vollständigkeit von SET-COVER

Beweis (Fortsetzung)

Zur Erinnerung:

$$P_i := \{j \mid \text{Literal } x_i \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m + i\}$$

$$N_i := \{j \mid \text{Literal } \neg x_i \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m + i\}$$

Die Reduktionsfunktion ist total und kann in Polynomialzeit berechnet werden.

$\mathcal{NP}$ -Vollständigkeit von SET-COVER

Beweis (Fortsetzung)

Zur Erinnerung:

$$P_i := \{j \mid \text{Literal } x_i \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m + i\}$$

$$N_i := \{j \mid \text{Literal } \neg x_i \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m + i\}$$

Die Reduktionsfunktion ist total und kann in Polynomialzeit berechnet werden.

Zu zeigen: F ist erfüllbar g.d.w. es eine Vereinigung von n Mengen

$U_1, \dots, U_n \in \{P_1, N_1, \dots, P_n, N_n\}$ gibt, sodass $U_1 \cup \dots \cup U_n = M$.

$\mathcal{NP}$ -Vollständigkeit von SET-COVER

Beweis (Fortsetzung)

Zur Erinnerung:

$$P_i := \{j \mid \text{Literal } x_i \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m+i\}$$

$$N_i := \{j \mid \text{Literal } \neg x_i \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m+i\}$$

Die Reduktionsfunktion ist total und kann in Polynomialzeit berechnet werden.

Zu zeigen: F ist erfüllbar g.d.w. es eine Vereinigung von n Mengen

$U_1, \dots, U_n \in \{P_1, N_1, \dots, P_n, N_n\}$ gibt, sodass $U_1 \cup \dots \cup U_n = M$.

$\implies$ Wenn F erfüllbar ist, gibt es eine Belegung I mit $I(F) = 1$.

$\mathcal{NP}$ -Vollständigkeit von SET-COVER

Beweis (Fortsetzung)

Zur Erinnerung:

$$P_i := \{j \mid \text{Literal } x_i \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m + i\}$$

$$N_i := \{j \mid \text{Literal } \neg x_i \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m + i\}$$

Die Reduktionsfunktion ist total und kann in Polynomialzeit berechnet werden.

Zu zeigen: F ist erfüllbar g.d.w. es eine Vereinigung von n Mengen

$U_1, \dots, U_n \in \{P_1, N_1, \dots, P_n, N_n\}$ gibt, sodass $U_1 \cup \dots \cup U_n = M$.

$\implies$ Wenn F erfüllbar ist, gibt es eine Belegung I mit $I(F) = 1$.

Wenn $I(x_i) = 1$, dann wähle P_i , sonst wähle N_i . Dies ergibt n gewählte Mengen.

$\mathcal{NP}$ -Vollständigkeit von SET-COVER

Beweis (Fortsetzung)

Zur Erinnerung:

$$P_i := \{j \mid \text{Literal } x_i \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m + i\}$$

$$N_i := \{j \mid \text{Literal } \neg x_i \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m + i\}$$

Die Reduktionsfunktion ist total und kann in Polynomialzeit berechnet werden.

Zu zeigen: F ist erfüllbar g.d.w. es eine Vereinigung von n Mengen

$U_1, \dots, U_n \in \{P_1, N_1, \dots, P_n, N_n\}$ gibt, sodass $U_1 \cup \dots \cup U_n = M$.

$\implies$ Wenn F erfüllbar ist, gibt es eine Belegung I mit $I(F) = 1$.

Wenn $I(x_i) = 1$, dann wähle P_i , sonst wähle N_i . Dies ergibt n gewählte Mengen.

Jede Zahl aus M kommt vor:

- ▶ Da jede Klausel durch I erfüllt ist, kommen alle $1, \dots, m$ vor.
- ▶ Da jede Variable x_i mit 0 oder 1 belegt wird, kommen alle $m + 1, \dots, m + n$ vor.

Die Vereinigung der n Mengen ergibt daher M .

$\mathcal{NP}$ -Vollständigkeit von SET-COVER

Beweis (Fortsetzung)

Zur Erinnerung:

$$P_i := \{j \mid \text{Literal } x_i \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m + i\}$$

$$N_i := \{j \mid \text{Literal } \neg x_i \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m + i\}$$

$\mathcal{NP}$ -Vollständigkeit von SET-COVER

Beweis (Fortsetzung)

Zur Erinnerung:

$$P_i := \{j \mid \text{Literal } x_i \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m+i\}$$

$$N_i := \{j \mid \text{Literal } \neg x_i \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m+i\}$$

Zu zeigen: F ist erfüllbar g.d.w. es eine Vereinigung von n Mengen $U_1, \dots, U_n \in \{P_1, N_1, \dots, P_n, N_n\}$ gibt, sodass $U_1 \cup \dots \cup U_n = M$.

$\Leftarrow$ Seien $U_1, \dots, U_n \in \{P_1, N_1, \dots, P_n, N_n\}$ mit $U_1 \cup \dots \cup U_n = M$.

$\mathcal{NP}$ -Vollständigkeit von SET-COVER

Beweis (Fortsetzung)

Zur Erinnerung:

$$P_i := \{j \mid \text{Literal } x_i \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m + i\}$$

$$N_i := \{j \mid \text{Literal } \neg x_i \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m + i\}$$

Zu zeigen: F ist erfüllbar g.d.w. es eine Vereinigung von n Mengen $U_1, \dots, U_n \in \{P_1, N_1, \dots, P_n, N_n\}$ gibt, sodass $U_1 \cup \dots \cup U_n = M$.

$\Leftarrow$ Seien $U_1, \dots, U_n \in \{P_1, N_1, \dots, P_n, N_n\}$ mit $U_1 \cup \dots \cup U_n = M$.

Da $m + 1, \dots, m + n$ in der Vereinigung sind, muss für jedes $i \in \{1, \dots, n\}$ genau entweder P_i oder N_i in der Vereinigung sein. O.B.d.A. $U_i \in \{P_i, N_i\}$.

$\mathcal{NP}$ -Vollständigkeit von SET-COVER

Beweis (Fortsetzung)

Zur Erinnerung:

$$P_i := \{j \mid \text{Literal } x_i \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m + i\}$$

$$N_i := \{j \mid \text{Literal } \neg x_i \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m + i\}$$

Zu zeigen: F ist erfüllbar g.d.w. es eine Vereinigung von n Mengen $U_1, \dots, U_n \in \{P_1, N_1, \dots, P_n, N_n\}$ gibt, sodass $U_1 \cup \dots \cup U_n = M$.

$\Leftarrow$ Seien $U_1, \dots, U_n \in \{P_1, N_1, \dots, P_n, N_n\}$ mit $U_1 \cup \dots \cup U_n = M$.

Da $m + 1, \dots, m + n$ in der Vereinigung sind, muss für jedes $i \in \{1, \dots, n\}$ genau entweder P_i oder N_i in der Vereinigung sein. O.B.d.A. $U_i \in \{P_i, N_i\}$.

Setze $I(x_i) = 1$ wenn $U_i = P_i$ und $I(x_i) = 0$ wenn $U_i = N_i$.

$\mathcal{NP}$ -Vollständigkeit von SET-COVER

Beweis (Fortsetzung)

Zur Erinnerung:

$$P_i := \{j \mid \text{Literal } x_i \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m + i\}$$

$$N_i := \{j \mid \text{Literal } \neg x_i \text{ kommt in Klausel } K_j \text{ vor}\} \cup \{m + i\}$$

Zu zeigen: F ist erfüllbar g.d.w. es eine Vereinigung von n Mengen $U_1, \dots, U_n \in \{P_1, N_1, \dots, P_n, N_n\}$ gibt, sodass $U_1 \cup \dots \cup U_n = M$.

$\Leftarrow$ Seien $U_1, \dots, U_n \in \{P_1, N_1, \dots, P_n, N_n\}$ mit $U_1 \cup \dots \cup U_n = M$.

Da $m + 1, \dots, m + n$ in der Vereinigung sind, muss für jedes $i \in \{1, \dots, n\}$ genau entweder P_i oder N_i in der Vereinigung sein. O.B.d.A. $U_i \in \{P_i, N_i\}$.

Setze $I(x_i) = 1$ wenn $U_i = P_i$ und $I(x_i) = 0$ wenn $U_i = N_i$.

Da $1, \dots, m$ in der Vereinigung sind, wird in jeder Klausel ein Literal durch I wahr gemacht. Daher ist F erfüllbar. $\square$

Das SUBSET-SUM-Problem

Definition

Das **SUBSET-SUM-Problem** lässt sich wie folgt formulieren.

gegeben: natürliche Zahlen $a_1, \dots, a_k \in \mathbb{N}$ und $s \in \mathbb{N}$

gefragt: Gibt es eine Teilmenge $I \subseteq \{1, \dots, k\}$, sodass $\sum_{i \in I} a_i = s$?

Das SUBSET-SUM-Problem

Definition

Das **SUBSET-SUM-Problem** lässt sich wie folgt formulieren.

gegeben: natürliche Zahlen $a_1, \dots, a_k \in \mathbb{N}$ und $s \in \mathbb{N}$

gefragt: Gibt es eine Teilmenge $I \subseteq \{1, \dots, k\}$, sodass $\sum_{i \in I} a_i = s$?

Beispiel:

Gegeben seien $a_1, \dots, a_6 = 1, 21, 5, 16, 12, 19$ und $s = 49$.

Das SUBSET-SUM-Problem

Definition

Das **SUBSET-SUM-Problem** lässt sich wie folgt formulieren.

gegeben: natürliche Zahlen $a_1, \dots, a_k \in \mathbb{N}$ und $s \in \mathbb{N}$

gefragt: Gibt es eine Teilmenge $I \subseteq \{1, \dots, k\}$, sodass $\sum_{i \in I} a_i = s$?

Beispiel:

Gegeben seien $a_1, \dots, a_6 = 1, 21, 5, 16, 12, 19$ und $s = 49$.

Lösung: 2, 4, 5, da $21 + 16 + 12 = 49$.

Das SUBSET-SUM-Problem

Definition

Das **SUBSET-SUM-Problem** lässt sich wie folgt formulieren.

gegeben: natürliche Zahlen $a_1, \dots, a_k \in \mathbb{N}$ und $s \in \mathbb{N}$

gefragt: Gibt es eine Teilmenge $I \subseteq \{1, \dots, k\}$, sodass $\sum_{i \in I} a_i = s$?

Beispiel:

Gegeben seien $a_1, \dots, a_6 = 1, 21, 5, 16, 12, 19$ und $s = 49$.

Lösung: 2, 4, 5, da $21 + 16 + 12 = 49$.

Satz

Das SUBSET-SUM-Problem ist $\mathcal{NP}$ -vollständig.

Das SUBSET-SUM-Problem

Definition

Das **SUBSET-SUM-Problem** lässt sich wie folgt formulieren.

gegeben: natürliche Zahlen $a_1, \dots, a_k \in \mathbb{N}$ und $s \in \mathbb{N}$

gefragt: Gibt es eine Teilmenge $I \subseteq \{1, \dots, k\}$, sodass $\sum_{i \in I} a_i = s$?

Beispiel:

Gegeben seien $a_1, \dots, a_6 = 1, 21, 5, 16, 12, 19$ und $s = 49$.

Lösung: 2, 4, 5, da $21 + 16 + 12 = 49$.

Satz

Das SUBSET-SUM-Problem ist $\mathcal{NP}$ -vollständig.

Beweis Siehe Skript.



Definition

Das **KNAPSACK-Problem** lässt sich wie folgt formulieren.

gegeben: k Gegenstände mit Gewichten $w_1, \dots, w_k \in \mathbb{N}$ und
Nutzenwerten $n_1, \dots, n_k \in \mathbb{N}$,
sowie zwei Schwellenwerte $s_w, s_n \in \mathbb{N}$

gefragt: Gibt es eine Teilmenge $I \subseteq \{1, \dots, k\}$, sodass $\sum_{i \in I} w_i \leq s_w$ und $\sum_{i \in I} n_i \geq s_n$?

Definition

Das **KNAPSACK-Problem** lässt sich wie folgt formulieren.

gegeben: k Gegenstände mit Gewichten $w_1, \dots, w_k \in \mathbb{N}$ und
Nutzenwerten $n_1, \dots, n_k \in \mathbb{N}$,
sowie zwei Schwellenwerte $s_w, s_n \in \mathbb{N}$

gefragt: Gibt es eine Teilmenge $I \subseteq \{1, \dots, k\}$, sodass $\sum_{i \in I} w_i \leq s_w$ und $\sum_{i \in I} n_i \geq s_n$?

Beachte: Für $w_i = n_i$ und $s_n = s_w$ ergibt sich genau das SUBSET-SUM-Problem.

$\mathcal{NP}$ -Vollständigkeit von KNAPSACK

Satz

KNAPSACK ist $\mathcal{NP}$ -vollständig.

$\mathcal{NP}$ -Vollständigkeit von KNAPSACK

Satz

KNAPSACK ist $\mathcal{NP}$ -vollständig.

Beweis

1. $\text{KNAPSACK} \in \mathcal{NP}$:

Rate eine Teilmenge $I \subseteq \{1, \dots, k\}$ nichtdeterministisch.

Prüfe deterministisch, ob $\sum_{i \in I} w_i \leq s_w$ und $\sum_{i \in I} n_i \geq s_n$.

Daher kann KNAPSACK in Polynomialzeit auf einer NTM entschieden werden.

$\mathcal{NP}$ -Vollständigkeit von KNAPSACK

Beweis (Fortsetzung)

2. KNAPSACK ist $\mathcal{NP}$ -schwer:

Wir zeigen $\text{SUBSET-SUM} \leq_p \text{KNAPSACK}$.

Beweis (Fortsetzung)

2. KNAPSACK ist $\mathcal{NP}$ -schwer:

Wir zeigen $\text{SUBSET-SUM} \leq_p \text{KNAPSACK}$.

Sei $((a_1, \dots, a_k), s)$ eine SUBSET-SUM-Instanz.

Beweis (Fortsetzung)

2. KNAPSACK ist $\mathcal{NP}$ -schwer:

Wir zeigen $\text{SUBSET-SUM} \leq_p \text{KNAPSACK}$.

Sei $((a_1, \dots, a_k), s)$ eine SUBSET-SUM-Instanz.

Sei $f((a_1, \dots, a_k), s) = ((w_1, \dots, w_k), (n_1, \dots, n_k), s_w, s_m)$ mit $w_i = a_i$, $n_i = a_i$ für $i \in \{1, \dots, k\}$ und $s_w = s$ und $s_m = s$.

Beweis (Fortsetzung)

2. KNAPSACK ist $\mathcal{NP}$ -schwer:

Wir zeigen $\text{SUBSET-SUM} \leq_p \text{KNAPSACK}$.

Sei $((a_1, \dots, a_k), s)$ eine SUBSET-SUM-Instanz.

Sei $f((a_1, \dots, a_k), s) = ((w_1, \dots, w_k), (n_1, \dots, n_k), s_w, s_m)$ mit $w_i = a_i$, $n_i = a_i$ für $i \in \{1, \dots, k\}$ und $s_w = s$ und $s_m = s$.

f ist in polynomieller Zeit von einer DTM berechenbar.

Beweis (Fortsetzung)

2. KNAPSACK ist $\mathcal{NP}$ -schwer:

Wir zeigen $\text{SUBSET-SUM} \leq_p \text{KNAPSACK}$.

Sei $((a_1, \dots, a_k), s)$ eine SUBSET-SUM-Instanz.

Sei $f((a_1, \dots, a_k), s) = ((w_1, \dots, w_k), (n_1, \dots, n_k), s_w, s_m)$ mit $w_i = a_i$, $n_i = a_i$ für $i \in \{1, \dots, k\}$ und $s_w = s$ und $s_m = s$.

f ist in polynomieller Zeit von einer DTM berechenbar.

$((a_1, \dots, a_k), s)$ ist SUBSET-SUM-lösbar g.d.w.

$f((a_1, \dots, a_k), s)$ KNAPSACK-lösbar ist. □

Das PARTITION-Problem

Definition

Das **PARTITION-Problem** lässt sich wie folgt formulieren.

gegeben: natürliche Zahlen $a_1, \dots, a_k \in \mathbb{N}$

gefragt: Gibt es eine Teilmenge $I \subseteq \{1, \dots, k\}$, sodass $\sum_{i \in I} a_i = \sum_{i \in \{1, \dots, k\} \setminus I} a_i$?

Das PARTITION-Problem

Definition

Das **PARTITION-Problem** lässt sich wie folgt formulieren.

gegeben: natürliche Zahlen $a_1, \dots, a_k \in \mathbb{N}$

gefragt: Gibt es eine Teilmenge $I \subseteq \{1, \dots, k\}$, sodass $\sum_{i \in I} a_i = \sum_{i \in \{1, \dots, k\} \setminus I} a_i$?

Beispiel:

Gegeben seien $k = 4$ und $a_1, \dots, a_4 = 7, 2, 8, 13$.

Das PARTITION-Problem

Definition

Das **PARTITION-Problem** lässt sich wie folgt formulieren.

gegeben: natürliche Zahlen $a_1, \dots, a_k \in \mathbb{N}$

gefragt: Gibt es eine Teilmenge $I \subseteq \{1, \dots, k\}$, sodass $\sum_{i \in I} a_i = \sum_{i \in \{1, \dots, k\} \setminus I} a_i$?

Beispiel:

Gegeben seien $k = 4$ und $a_1, \dots, a_4 = 7, 2, 8, 13$.

Lösung: z.B. $I = \{1, 3\}$, da $\sum_{i \in \{1,3\}} a_i = 15 = \sum_{i \in \{2,4\}} a_i$.

$\mathcal{NP}$ -Vollständigkeit von PARTITION

Satz

PARTITION ist $\mathcal{NP}$ -vollständig.

$\mathcal{NP}$ -Vollständigkeit von PARTITION

Satz

PARTITION ist $\mathcal{NP}$ -vollständig.

Beweis

1. PARTITION $\in \mathcal{NP}$:

Rate nichtdeterministisch $I \subseteq \{1, \dots, k\}$.

Prüfe deterministisch, ob $\sum_{i \in I} a_i = \sum_{i \in \{1, \dots, k\} \setminus I} a_i$.

Daher kann PARTITION in Polynomialzeit auf einer NTM entschieden werden.

$\mathcal{NP}$ -Vollständigkeit von PARTITION

Beweis (Fortsetzung)

2. PARTITION ist $\mathcal{NP}$ -schwer:

Wir zeigen $\text{SUBSET-SUM} \leq_p \text{PARTITION}$.

Beweis (Fortsetzung)

2. PARTITION ist $\mathcal{NP}$ -schwer:

Wir zeigen $\text{SUBSET-SUM} \leq_p \text{PARTITION}$.

Sei $f((a_1, \dots, a_k), s) = (a_1, \dots, a_k, a_{k+1}, a_{k+2})$ mit $a_{k+1} = A + s$ und $a_{k+2} = 2A - s$, wobei $A = \sum_{i=1}^k a_i$.

Beweis (Fortsetzung)

2. PARTITION ist $\mathcal{NP}$ -schwer:

Wir zeigen $\text{SUBSET-SUM} \leq_p \text{PARTITION}$.

Sei $f((a_1, \dots, a_k), s) = (a_1, \dots, a_k, a_{k+1}, a_{k+2})$ mit $a_{k+1} = A + s$ und $a_{k+2} = 2A - s$, wobei $A = \sum_{i=1}^k a_i$.

Beachte: $\sum_{i=1}^{k+2} a_i = 4A$.

Beweis (Fortsetzung)

2. PARTITION ist $\mathcal{NP}$ -schwer:

Wir zeigen $\text{SUBSET-SUM} \leq_p \text{PARTITION}$.

Sei $f((a_1, \dots, a_k), s) = (a_1, \dots, a_k, a_{k+1}, a_{k+2})$ mit $a_{k+1} = A + s$ und $a_{k+2} = 2A - s$, wobei $A = \sum_{i=1}^k a_i$.

Beachte: $\sum_{i=1}^{k+2} a_i = 4A$.

Die Funktion f ist total und kann in polynomieller Zeit berechnet werden.

Beispiel für die SUBSET-SUM-Instanzkonstruktion

SUBSET-SUM-Instanz:

$((1, 2, 3, 4, 5, 6), 14)$

Beispiel für die SUBSET-SUM-Instanzkonstruktion

SUBSET-SUM-Instanz:

$$((1, 2, 3, 4, 5, 6), 14)$$

Wir haben $s = 14$, $A = 21$, $A + s = 35$, $2A = 42$ und $2A - s = 28$.

Beispiel für die SUBSET-SUM-Instanzkonstruktion

SUBSET-SUM-Instanz:

$$((1, 2, 3, 4, 5, 6), 14)$$

Wir haben $s = 14$, $A = 21$, $A + s = 35$, $2A = 42$ und $2A - s = 28$.

PARTITION-Instanz zur SUBSET-SUM-Instanz:

$$(1, 2, 3, 4, 5, 6, 35, 28)$$

Beispiel für die SUBSET-SUM-Instanzkonstruktion

SUBSET-SUM-Instanz:

$$((1, 2, 3, 4, 5, 6), 14)$$

Wir haben $s = 14$, $A = 21$, $A + s = 35$, $2A = 42$ und $2A - s = 28$.

PARTITION-Instanz zur SUBSET-SUM-Instanz:

$$(1, 2, 3, 4, 5, 6, 35, 28)$$

Lösung der PARTITION-Instanz: z.B. $I = \{1, 3, 4, 6, 8\}$,
da $1 + 3 + 4 + 6 + 28 = 42 = 2 + 5 + 35$.

Beispiel für die SUBSET-SUM-Instanzkonstruktion

SUBSET-SUM-Instanz:

$$((1, 2, 3, 4, 5, 6), 14)$$

Wir haben $s = 14$, $A = 21$, $A + s = 35$, $2A = 42$ und $2A - s = 28$.

PARTITION-Instanz zur SUBSET-SUM-Instanz:

$$(1, 2, 3, 4, 5, 6, 35, 28)$$

Lösung der PARTITION-Instanz: z.B. $I = \{1, 3, 4, 6, 8\}$,
da $1 + 3 + 4 + 6 + 28 = 42 = 2 + 5 + 35$.

Lösung der SUBSET-SUM-Instanz: $\{1, 3, 4, 6\}$.
Tatsächlich $1 + 3 + 4 + 6 = 14$.

Beweis (Fortsetzung)

Wir zeigen: $((a_1, \dots, a_k), s)$ ist SUBSET-SUM-lösbar g.d.w.
 $(a_1, \dots, a_k, a_{k+1}, a_{k+2})$ PARTITION-lösbar ist.

Beweis (Fortsetzung)

Wir zeigen: $((a_1, \dots, a_k), s)$ ist SUBSET-SUM-lösbar g.d.w.

$(a_1, \dots, a_k, a_{k+1}, a_{k+2})$ PARTITION-lösbar ist.

$\implies$ Wenn $I \subseteq \{1, \dots, k\}$ eine Lösung für $((a_1, \dots, a_k), s)$ ist, dann ist $I \cup \{k+2\}$ eine Lösung für $(a_1, \dots, a_k, a_{k+1}, a_{k+2})$, weil $\sum_{I \cup \{k+2\}} a_i = s + (2A - s) = 2A$, was die Hälfte von $\sum_{i=1}^{k+2} a_i = 4A$ ausmacht.

$\mathcal{NP}$ -Vollständigkeit von PARTITION

Beweis (Fortsetzung)

Wir zeigen: $((a_1, \dots, a_k), s)$ ist SUBSET-SUM-lösbar g.d.w.

$(a_1, \dots, a_k, a_{k+1}, a_{k+2})$ PARTITION-lösbar ist.

$\Rightarrow$ Wenn $I \subseteq \{1, \dots, k\}$ eine Lösung für $((a_1, \dots, a_k), s)$ ist, dann ist $I \cup \{k+2\}$ eine Lösung für $(a_1, \dots, a_k, a_{k+1}, a_{k+2})$, weil $\sum_{I \cup \{k+2\}} a_i = s + (2A - s) = 2A$, was die Hälfte von $\sum_{i=1}^{k+2} a_i = 4A$ ausmacht.

$\Leftarrow$ Wenn $(a_1, \dots, a_k, a_{k+1}, a_{k+2})$ eine Lösung $I \subseteq \{1, \dots, k+2\}$ hat, dann $\sum_{i \in I} a_i = 2A = \sum_{i \in \{1, \dots, k+2\} \setminus I} a_i$.

$\mathcal{NP}$ -Vollständigkeit von PARTITION

Beweis (Fortsetzung)

Wir zeigen: $((a_1, \dots, a_k), s)$ ist SUBSET-SUM-lösbar g.d.w.

$(a_1, \dots, a_k, a_{k+1}, a_{k+2})$ PARTITION-lösbar ist.

$\Rightarrow$ Wenn $I \subseteq \{1, \dots, k\}$ eine Lösung für $((a_1, \dots, a_k), s)$ ist, dann ist $I \cup \{k+2\}$ eine Lösung für $(a_1, \dots, a_k, a_{k+1}, a_{k+2})$, weil $\sum_{I \cup \{k+2\}} a_i = s + (2A - s) = 2A$, was die Hälfte von $\sum_{i=1}^{k+2} a_i = 4A$ ausmacht.

$\Leftarrow$ Wenn $(a_1, \dots, a_k, a_{k+1}, a_{k+2})$ eine Lösung $I \subseteq \{1, \dots, k+2\}$ hat, dann $\sum_{i \in I} a_i = 2A = \sum_{i \in \{1, \dots, k+2\} \setminus I} a_i$.

$k+1$ und $k+2$ können nicht gleichzeitig in I sein, da sonst die Summe zu groß ist.

$\mathcal{NP}$ -Vollständigkeit von PARTITION

Beweis (Fortsetzung)

Wir zeigen: $((a_1, \dots, a_k), s)$ ist SUBSET-SUM-lösbar g.d.w.

$(a_1, \dots, a_k, a_{k+1}, a_{k+2})$ PARTITION-lösbar ist.

$\Rightarrow$ Wenn $I \subseteq \{1, \dots, k\}$ eine Lösung für $((a_1, \dots, a_k), s)$ ist, dann ist $I \cup \{k+2\}$ eine Lösung für $(a_1, \dots, a_k, a_{k+1}, a_{k+2})$, weil $\sum_{I \cup \{k+2\}} a_i = s + (2A - s) = 2A$, was die Hälfte von $\sum_{i=1}^{k+2} a_i = 4A$ ausmacht.

$\Leftarrow$ Wenn $(a_1, \dots, a_k, a_{k+1}, a_{k+2})$ eine Lösung $I \subseteq \{1, \dots, k+2\}$ hat, dann $\sum_{i \in I} a_i = 2A = \sum_{i \in \{1, \dots, k+2\} \setminus I} a_i$.

$k+1$ und $k+2$ können nicht gleichzeitig in I sein, da sonst die Summe zu groß ist.

- ▶ Wenn $k+2 \in I$, dann $I' = I \setminus \{k+2\}$.
- ▶ Wenn $k+1 \in I$, dann $I' = (\{1, \dots, k+2\} \setminus I) \setminus \{k+2\}$.

$\mathcal{NP}$ -Vollständigkeit von PARTITION

Beweis (Fortsetzung)

Wir zeigen: $((a_1, \dots, a_k), s)$ ist SUBSET-SUM-lösbar g.d.w.

$(a_1, \dots, a_k, a_{k+1}, a_{k+2})$ PARTITION-lösbar ist.

$\Rightarrow$ Wenn $I \subseteq \{1, \dots, k\}$ eine Lösung für $((a_1, \dots, a_k), s)$ ist, dann ist $I \cup \{k+2\}$ eine Lösung für $(a_1, \dots, a_k, a_{k+1}, a_{k+2})$, weil $\sum_{i \in I \cup \{k+2\}} a_i = s + (2A - s) = 2A$, was die Hälfte von $\sum_{i=1}^{k+2} a_i = 4A$ ausmacht.

$\Leftarrow$ Wenn $(a_1, \dots, a_k, a_{k+1}, a_{k+2})$ eine Lösung $I \subseteq \{1, \dots, k+2\}$ hat, dann $\sum_{i \in I} a_i = 2A = \sum_{i \in \{1, \dots, k+2\} \setminus I} a_i$.

$k+1$ und $k+2$ können nicht gleichzeitig in I sein, da sonst die Summe zu groß ist.

► Wenn $k+2 \in I$, dann $I' = I \setminus \{k+2\}$.

► Wenn $k+1 \in I$, dann $I' = (\{1, \dots, k+2\} \setminus I) \setminus \{k+2\}$.

In beiden Fällen ist $\sum_{i \in I'} a_i = 2A - (2A - s) = s$ und

die Indexmenge I' ist daher eine Lösung von $((a_1, \dots, a_k), s)$. □

Das BIN-PACKING-Problem

Definition

Das **BIN-PACKING-Problem** lässt sich wie folgt formulieren.

gegeben: natürliche Zahlen $a_1, \dots, a_k \in \mathbb{N}$,
die Behältergröße $b \in \mathbb{N}$ und
die Anzahl der Behälter m

gefragt: Kann man alle gegebenen Zahlen so auf die Behälter aufteilen, sodass keiner der Behälter überläuft?

Formal: Gibt es eine totale Funktion $assign : \{1, \dots, k\} \rightarrow \{1, \dots, m\}$,
sodass für alle Behälter $j \in \{1, \dots, m\}$ gilt: $(\sum_{i: assign(i)=j} a_i) \leq b$?

$\mathcal{NP}$ -Vollständigkeit von BIN-PACKING

Satz

BIN-PACKING ist $\mathcal{NP}$ -vollständig.

Satz

BIN-PACKING ist $\mathcal{NP}$ -vollständig.

Beweis

1. BIN-PACKING $\in \mathcal{NP}$:

Rate nichtdeterministisch für jede Zahl a_i in welchen Behälter sie gehört.

Prüfe deterministisch, dass die geratene Zuordnung keinen Behälter überlaufen lässt.

BIN-PACKING kann daher in Polynomialzeit auf einer NTM entschieden werden.

Beweis (Fortsetzung)

2. BIN-PACKING ist $\mathcal{NP}$ -schwer:

Wir zeigen $\text{PARTITION} \leq_p \text{BIN-PACKING}$.

Beweis (Fortsetzung)

2. BIN-PACKING ist $\mathcal{NP}$ -schwer:

Wir zeigen $\text{PARTITION} \leq_p \text{BIN-PACKING}$.

Sei $(a_1, \dots, a_k)$ eine PARTITION-Instanz. Sei $f(a_1, \dots, a_k)$ die BIN-PACKING-Instanz mit Zahlen $a_1, \dots, a_k$,

Behältergröße $b = \left\lfloor \frac{\sum_{i=1}^k a_i}{2} \right\rfloor$ und $m = 2$ Behältern.

Beweis (Fortsetzung)

2. BIN-PACKING ist $\mathcal{NP}$ -schwer:

Wir zeigen $\text{PARTITION} \leq_p \text{BIN-PACKING}$.

Sei $(a_1, \dots, a_k)$ eine PARTITION-Instanz. Sei $f(a_1, \dots, a_k)$ die BIN-PACKING-Instanz mit Zahlen $a_1, \dots, a_k$,

Behältergröße $b = \left\lfloor \frac{\sum_{i=1}^k a_i}{2} \right\rfloor$ und $m = 2$ Behältern.

Die Funktion f ist total und kann in polynomieller Zeit berechnet werden.

Beweis (Fortsetzung)

Wir zeigen: $(a_1, \dots, a_k)$ ist PARTITION-lösbar g.d.w.
 $((a_1, \dots, a_k), b, 2)$ BIN-PACKING-lösbar ist.

Beweis (Fortsetzung)

Wir zeigen: $(a_1, \dots, a_k)$ ist PARTITION-lösbar g.d.w.

$((a_1, \dots, a_k), b, 2)$ BIN-PACKING-lösbar ist.

Wenn $\sum_{i=1}^k a_i$ ungerade ist, dann ist die PARTITION-Instanz unlösbar und die BIN-PACKING-Instanz ebenso. Sonst:

$\mathcal{NP}$ -Vollständigkeit von BIN-PACKING

Beweis (Fortsetzung)

Wir zeigen: $(a_1, \dots, a_k)$ ist PARTITION-lösbar g.d.w.

$((a_1, \dots, a_k), b, 2)$ BIN-PACKING-lösbar ist.

Wenn $\sum_{i=1}^k a_i$ ungerade ist, dann ist die PARTITION-Instanz unlösbar und die BIN-PACKING-Instanz ebenso. Sonst:

$\Leftarrow$ Wenn $I \subseteq \{1, \dots, k\}$ eine Lösung für PARTITION ist, dann ist

$$\text{assign}(i) = \begin{cases} 1 & \text{wenn } i \in I \\ 2 & \text{wenn } i \notin I \end{cases}$$

eine Lösung für BIN-PACKING.

$\mathcal{NP}$ -Vollständigkeit von BIN-PACKING

Beweis (Fortsetzung)

Wir zeigen: $(a_1, \dots, a_k)$ ist PARTITION-lösbar g.d.w.

$((a_1, \dots, a_k), b, 2)$ BIN-PACKING-lösbar ist.

Wenn $\sum_{i=1}^k a_i$ ungerade ist, dann ist die PARTITION-Instanz unlösbar und die BIN-PACKING-Instanz ebenso. Sonst:

$\Leftarrow$ Wenn $I \subseteq \{1, \dots, k\}$ eine Lösung für PARTITION ist, dann ist

$$\text{assign}(i) = \begin{cases} 1 & \text{wenn } i \in I \\ 2 & \text{wenn } i \notin I \end{cases}$$

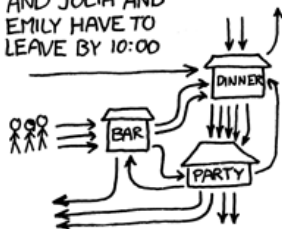
eine Lösung für BIN-PACKING.

$\Rightarrow$ Sei assign eine Lösung für BIN-PACKING.

Mit $I = \{i \mid 1 \leq i \leq k, \text{assign}(i) = 1\}$ kann eine Lösung für PARTITION erstellt werden. □



SOMEONE HAS TO GET PAUL, AND JULIA AND EMILY HAVE TO LEAVE BY 10:00



THE LOGISTICS OF WHO CAN GET DRUNK ARE NONTRIVIAL.



xkcd.com/589/

13b

**$\mathcal{NP}$ -Vollständigkeit von
GRAPH-COLORING, (UN)DIRECTED-HAMILTON-CYCLE und
TRAVELING-SALESPERSON**

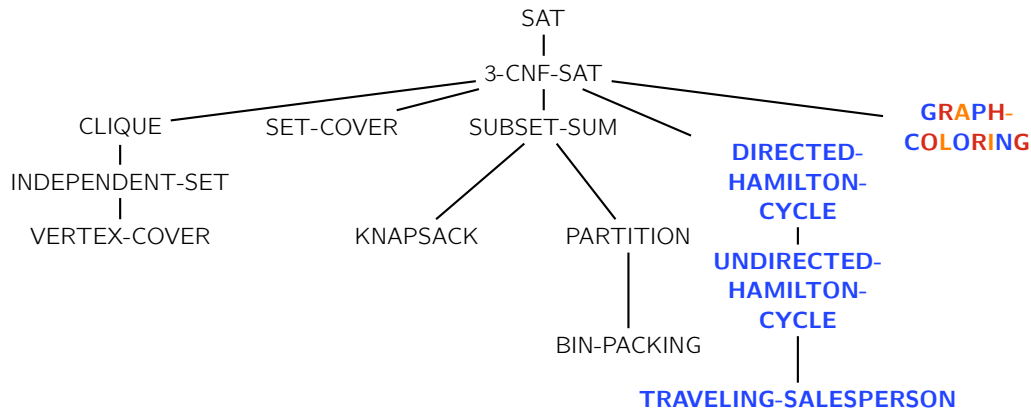
Prof. Dr. Jasmin Blanchette

Lehr- und Forschungseinheit für
Theoretische Informatik und Theorembeweisen

Stand: 16. Juli 2024
Basierend auf Folien von PD Dr. David Sabel



Überblick über $\mathcal{NP}$ -Vollständigkeitsbeweise



Definition

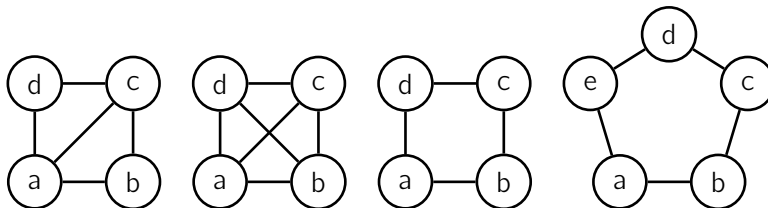
Das **GRAPH-COLORING-Problem** lässt sich wie folgt formulieren.

gegeben: ein ungerichteter Graph $G = (V, E)$ und eine Zahl $k \in \mathbb{N}$

gefragt: Gibt es eine Färbung der Knoten in V mit höchstens k Farben (eine sogenannte k -Färbung), sodass keine zwei benachbarten Knoten in G die gleiche Farbe erhalten?

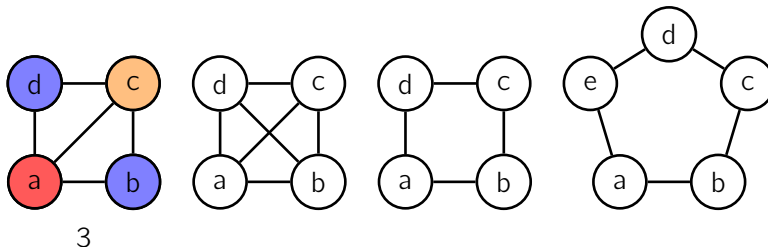
Beispiele für Färbungen

Wie viele Farben sind jeweils minimal notwendig?



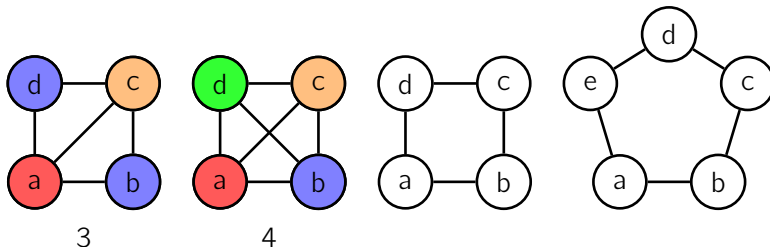
Beispiele für Färbungen

Wie viele Farben sind jeweils minimal notwendig?



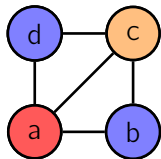
Beispiele für Färbungen

Wie viele Farben sind jeweils minimal notwendig?

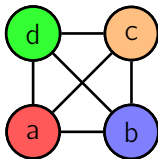


Beispiele für Färbungen

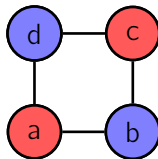
Wie viele Farben sind jeweils minimal notwendig?



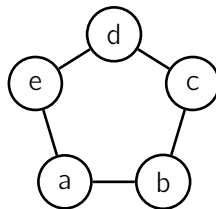
3



4

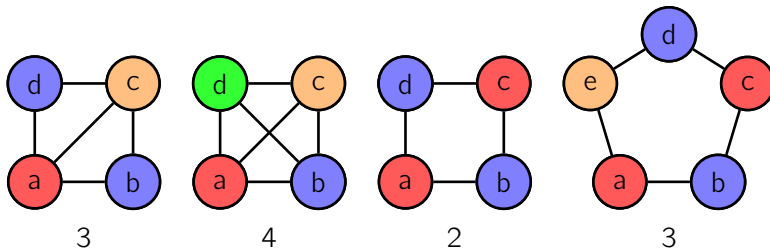


2

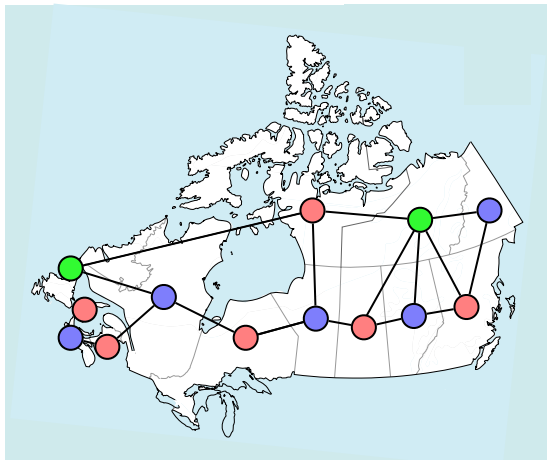


Beispiele für Färbungen

Wie viele Farben sind jeweils minimal notwendig?



Weiteres Beispiel für eine Färbung



Adanak

Satz

GRAPH-COLORING ist $\mathcal{NP}$ -vollständig.

$\mathcal{NP}$ -Vollständigkeit von GRAPH-COLORING

Satz

GRAPH-COLORING ist $\mathcal{NP}$ -vollständig.

Beweis

1. GRAPH-COLORING $\in \mathcal{NP}$:

Rate nichtdeterministisch die Farbe aus $\{1, \dots, k\}$ für jeden Knoten $v \in V$.

Prüfe, dass die Farben von u und v stets verschieden sind, für alle $\{u, v\} \in E$.

Das geht in Polynomialzeit.

Daher kann GRAPH-COLORING auf einer NTM in polynomieller Zeit entschieden werden.

Beweis (Fortsetzung)

2. GRAPH-COLORING ist $\mathcal{NP}$ -schwer:

Wir zeigen $3\text{-CNF-SAT} \leq_p \text{GRAPH-COLORING}$.

Beweis (Fortsetzung)

2. GRAPH-COLORING ist $\mathcal{NP}$ -schwer:

Wir zeigen $3\text{-CNF-SAT} \leq_p \text{GRAPH-COLORING}$.

Sei $H = K_1 \wedge \dots \wedge K_m$ eine 3-CNF, sodass jede Klausel K_i genau drei Literale enthält.

Beweis (Fortsetzung)

2. GRAPH-COLORING ist $\mathcal{NP}$ -schwer:

Wir zeigen $3\text{-CNF-SAT} \leq_p \text{GRAPH-COLORING}$.

Sei $H = K_1 \wedge \dots \wedge K_m$ eine 3-CNF, sodass jede Klausel K_i genau drei Literale enthält.

Seien $x_1, \dots, x_n$ die in H vorkommenden aussagenlogischen Variablen.

Beweis (Fortsetzung)

2. GRAPH-COLORING ist $\mathcal{NP}$ -schwer:

Wir zeigen $3\text{-CNF-SAT} \leq_p \text{GRAPH-COLORING}$.

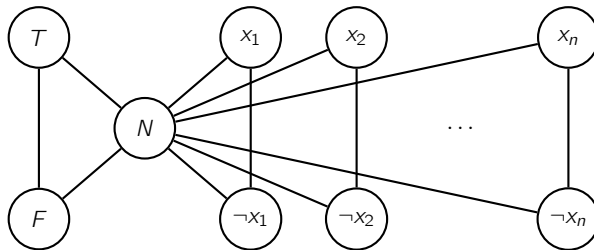
Sei $H = K_1 \wedge \dots \wedge K_m$ eine 3-CNF, sodass jede Klausel K_i genau drei Literale enthält.

Seien $x_1, \dots, x_n$ die in H vorkommenden aussagenlogischen Variablen.

Wir erzeugen ein GRAPH-COLORING-Problem $f(H) = (G, k)$ mit $k = 3$.

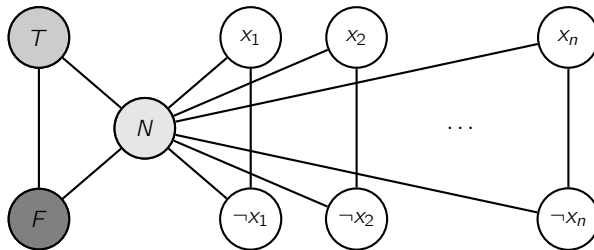
Konstruktion der GRAPH-COLORING-Instanz

Konstruiere zunächst den folgenden Teilgraphen (die „Palette“):



Konstruktion der GRAPH-COLORING-Instanz

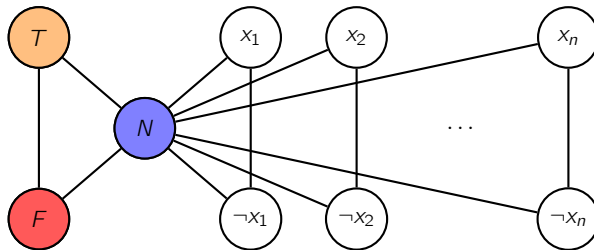
Konstruiere zunächst den folgenden Teilgraphen (die „Palette“):



Wenn der Graph 3-färbbar ist, dann müssen T, N, F verschiedene Farben haben.

Konstruktion der GRAPH-COLORING-Instanz

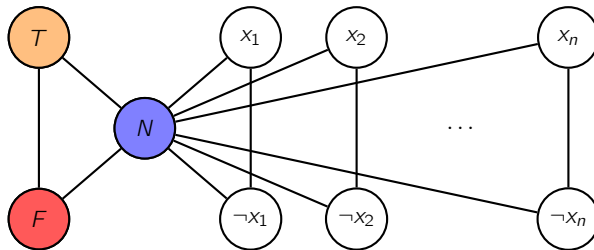
Konstruiere zunächst den folgenden Teilgraphen (die „Palette“):



Wenn der Graph 3-färbbar ist, dann müssen T, N, F verschiedene Farben haben. O.B.d.A. seien dies genau die Farben **Orange**, **Rot** und **Blau**.

Konstruktion der GRAPH-COLORING-Instanz

Konstruiere zunächst den folgenden Teilgraphen (die „Palette“):

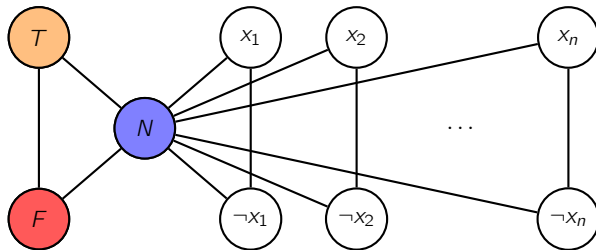


Wenn der Graph 3-färbbar ist, dann müssen T, N, F verschiedene Farben haben. O.B.d.A. seien dies genau die Farben **Orange**, **Rot** und **Blau**.

Für 3-Färbung muss $(x_i, \neg x_i)$ mit $(\text{Orange}, \text{Rot})$ oder $(\text{Rot}, \text{Orange})$ gefärbt werden.

Konstruktion der GRAPH-COLORING-Instanz

Konstruiere zunächst den folgenden Teilgraphen (die „Palette“):



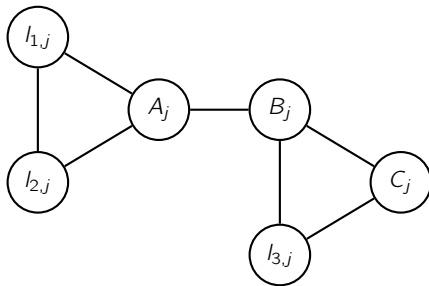
Wenn der Graph 3-färbbar ist, dann müssen T , N , F verschiedene Farben haben. O.B.d.A. seien dies genau die Farben **Orange**, **Rot** und **Blau**.

Für 3-Färbung muss $(x_i, \neg x_i)$ mit $(\text{Orange}, \text{Rot})$ oder $(\text{Rot}, \text{Orange})$ gefärbt werden.

Die Palette gibt eine Belegung an: Die 3-Färbung erzeugt $I(x_i) = 1$, wenn x_i mit **Orange** gefärbt ist, und $I(x_i) = 0$, wenn x_i mit **Rot** gefärbt ist.

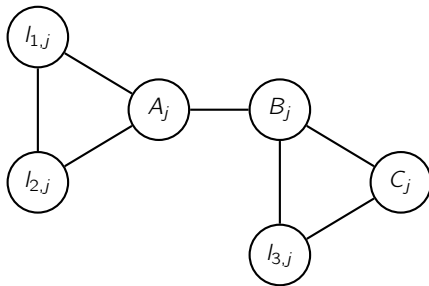
Konstruktion der GRAPH-COLORING-Instanz

Erzeuge für jede Klausel $K_j = L_1 \vee L_2 \vee L_3$ den folgenden Teilgraphen O_j (ein „Oder-Gatter“):



Konstruktion der GRAPH-COLORING-Instanz

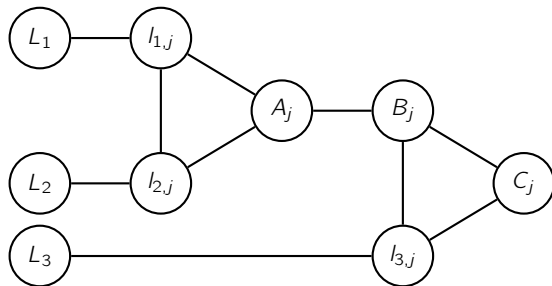
Erzeuge für jede Klausel $K_j = L_1 \vee L_2 \vee L_3$ den folgenden Teilgraphen O_j (ein „Oder-Gatter“):



Grundgedanke: O_j soll das logische Oder der drei Literale in K_j „berechnen“.

Konstruktion der GRAPH-COLORING-Instanz

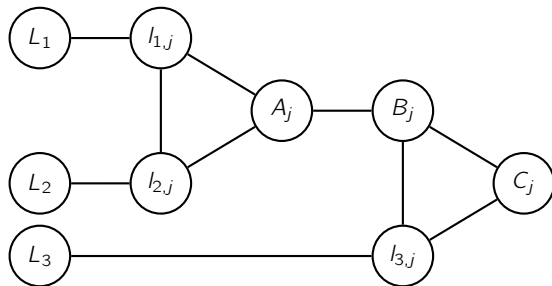
Erzeuge für jede Klausel $K_j = L_1 \vee L_2 \vee L_3$ den folgenden Teilgraphen O_j (ein „Oder-Gatter“):



Grundgedanke: O_j soll das logische Oder der drei Literale in K_j „berechnen“. Verbinde die Eingänge mit L_1 , L_2 und L_3 aus der Palette. Die Farben der L_i sind daher F oder T .

Konstruktion der GRAPH-COLORING-Instanz

Erzeuge für jede Klausel $K_j = L_1 \vee L_2 \vee L_3$ den folgenden Teilgraphen O_j (ein „Oder-Gatter“):



Grundgedanke: O_j soll das logische Oder der drei Literale in K_j „berechnen“.

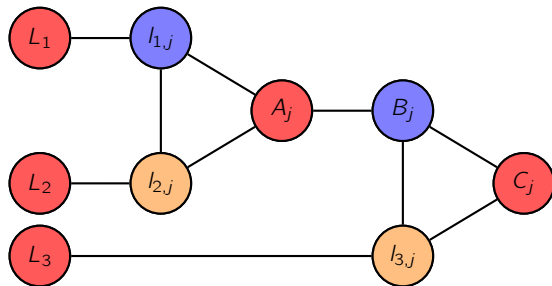
Verbinde die Eingänge mit L_1 , L_2 und L_3 aus der Palette.

Die Farben der L_i sind daher F oder T .

C_j kann mit T gefärbt werden g.d.w. ein L_i mit T gefärbt ist.

Konstruktion der GRAPH-COLORING-Instanz

Erzeuge für jede Klausel $K_j = L_1 \vee L_2 \vee L_3$ den folgenden Teilgraphen O_j (ein „Oder-Gatter“):



Grundgedanke: O_j soll das logische Oder der drei Literale in K_j „berechnen“.

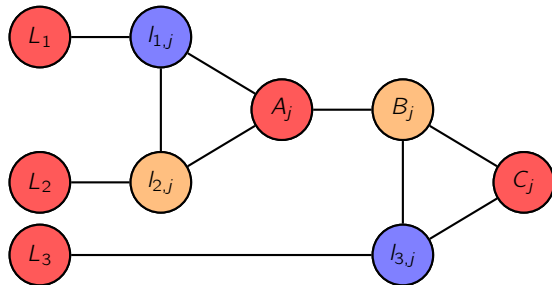
Verbinde die Eingänge mit L_1 , L_2 und L_3 aus der Palette.

Die Farben der L_i sind daher F oder T .

C_j kann mit T gefärbt werden g.d.w. ein L_i mit T gefärbt ist.

Konstruktion der GRAPH-COLORING-Instanz

Erzeuge für jede Klausel $K_j = L_1 \vee L_2 \vee L_3$ den folgenden Teilgraphen O_j (ein „Oder-Gatter“):



Grundgedanke: O_j soll das logische Oder der drei Literale in K_j „berechnen“.

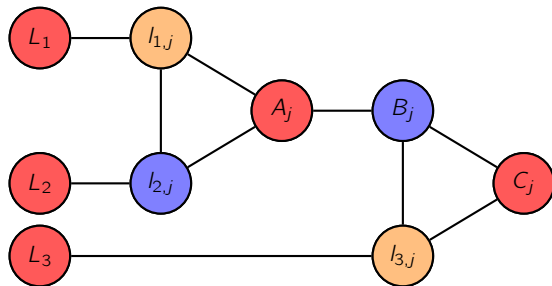
Verbinde die Eingänge mit L_1 , L_2 und L_3 aus der Palette.

Die Farben der L_i sind daher F oder T .

C_j kann mit T gefärbt werden g.d.w. ein L_i mit T gefärbt ist.

Konstruktion der GRAPH-COLORING-Instanz

Erzeuge für jede Klausel $K_j = L_1 \vee L_2 \vee L_3$ den folgenden Teilgraphen O_j (ein „Oder-Gatter“):



Grundgedanke: O_j soll das logische Oder der drei Literale in K_j „berechnen“.

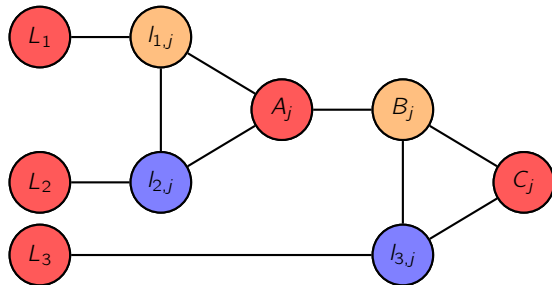
Verbinde die Eingänge mit L_1 , L_2 und L_3 aus der Palette.

Die Farben der L_i sind daher F oder T .

C_j kann mit T gefärbt werden g.d.w. ein L_i mit T gefärbt ist.

Konstruktion der GRAPH-COLORING-Instanz

Erzeuge für jede Klausel $K_j = L_1 \vee L_2 \vee L_3$ den folgenden Teilgraphen O_j (ein „Oder-Gatter“):



Grundgedanke: O_j soll das logische Oder der drei Literale in K_j „berechnen“.

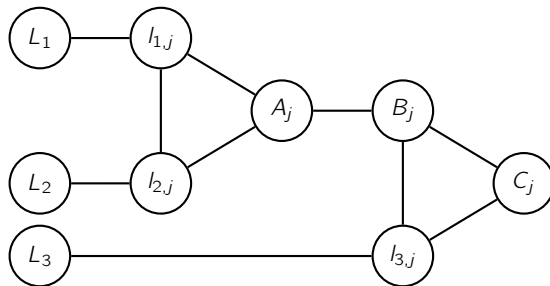
Verbinde die Eingänge mit L_1 , L_2 und L_3 aus der Palette.

Die Farben der L_i sind daher F oder T .

C_j kann mit T gefärbt werden g.d.w. ein L_i mit T gefärbt ist.

Konstruktion der GRAPH-COLORING-Instanz

Erzeuge für jede Klausel $K_j = L_1 \vee L_2 \vee L_3$ den folgenden Teilgraphen O_j (ein „Oder-Gatter“):



Grundgedanke: O_j soll das logische Oder der drei Literale in K_j „berechnen“.

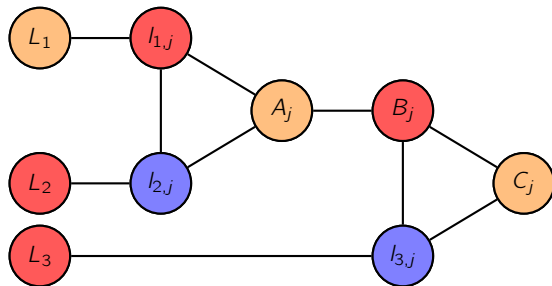
Verbinde die Eingänge mit L_1 , L_2 und L_3 aus der Palette.

Die Farben der L_i sind daher F oder T .

C_j kann mit T gefärbt werden g.d.w. ein L_i mit T gefärbt ist.

Konstruktion der GRAPH-COLORING-Instanz

Erzeuge für jede Klausel $K_j = L_1 \vee L_2 \vee L_3$ den folgenden Teilgraphen O_j (ein „Oder-Gatter“):



Grundgedanke: O_j soll das logische Oder der drei Literale in K_j „berechnen“.

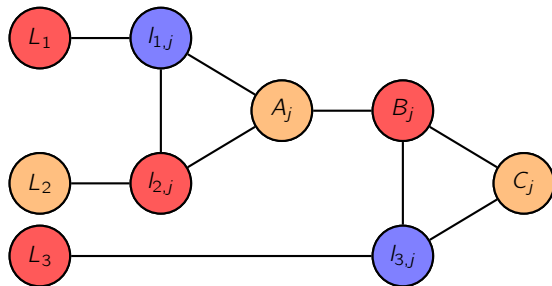
Verbinde die Eingänge mit L_1 , L_2 und L_3 aus der Palette.

Die Farben der L_i sind daher F oder T .

C_j kann mit T gefärbt werden g.d.w. ein L_i mit T gefärbt ist.

Konstruktion der GRAPH-COLORING-Instanz

Erzeuge für jede Klausel $K_j = L_1 \vee L_2 \vee L_3$ den folgenden Teilgraphen O_j (ein „Oder-Gatter“):



Grundgedanke: O_j soll das logische Oder der drei Literale in K_j „berechnen“.

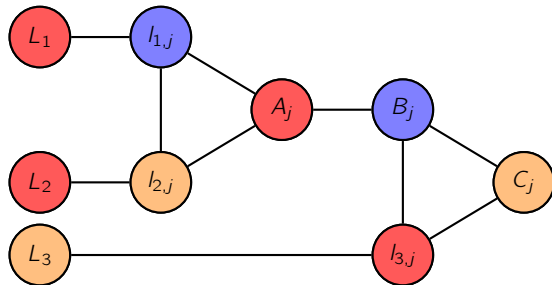
Verbinde die Eingänge mit L_1 , L_2 und L_3 aus der Palette.

Die Farben der L_i sind daher F oder T .

C_j kann mit T gefärbt werden g.d.w. ein L_i mit T gefärbt ist.

Konstruktion der GRAPH-COLORING-Instanz

Erzeuge für jede Klausel $K_j = L_1 \vee L_2 \vee L_3$ den folgenden Teilgraphen O_j (ein „Oder-Gatter“):



Grundgedanke: O_j soll das logische Oder der drei Literale in K_j „berechnen“.

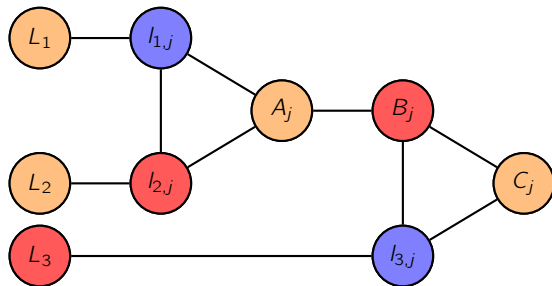
Verbinde die Eingänge mit L_1 , L_2 und L_3 aus der Palette.

Die Farben der L_i sind daher F oder T .

C_j kann mit T gefärbt werden g.d.w. ein L_i mit T gefärbt ist.

Konstruktion der GRAPH-COLORING-Instanz

Erzeuge für jede Klausel $K_j = L_1 \vee L_2 \vee L_3$ den folgenden Teilgraphen O_j (ein „Oder-Gatter“):



Grundgedanke: O_j soll das logische Oder der drei Literale in K_j „berechnen“.

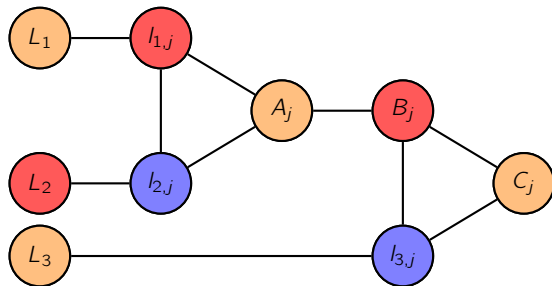
Verbinde die Eingänge mit L_1 , L_2 und L_3 aus der Palette.

Die Farben der L_i sind daher F oder T .

C_j kann mit T gefärbt werden g.d.w. ein L_i mit T gefärbt ist.

Konstruktion der GRAPH-COLORING-Instanz

Erzeuge für jede Klausel $K_j = L_1 \vee L_2 \vee L_3$ den folgenden Teilgraphen O_j (ein „Oder-Gatter“):



Grundgedanke: O_j soll das logische Oder der drei Literale in K_j „berechnen“.

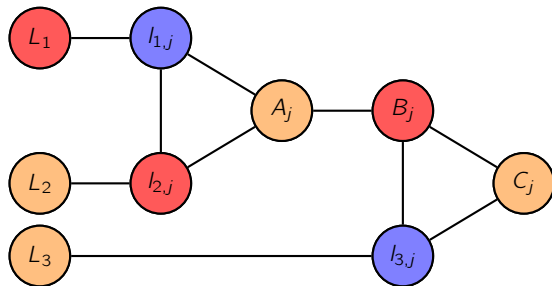
Verbinde die Eingänge mit L_1 , L_2 und L_3 aus der Palette.

Die Farben der L_i sind daher F oder T .

C_j kann mit T gefärbt werden g.d.w. ein L_i mit T gefärbt ist.

Konstruktion der GRAPH-COLORING-Instanz

Erzeuge für jede Klausel $K_j = L_1 \vee L_2 \vee L_3$ den folgenden Teilgraphen O_j (ein „Oder-Gatter“):



Grundgedanke: O_j soll das logische Oder der drei Literale in K_j „berechnen“.

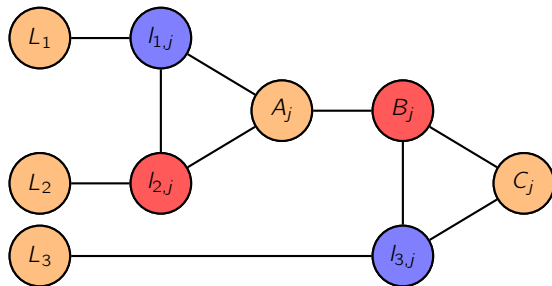
Verbinde die Eingänge mit L_1 , L_2 und L_3 aus der Palette.

Die Farben der L_i sind daher F oder T .

C_j kann mit T gefärbt werden g.d.w. ein L_i mit T gefärbt ist.

Konstruktion der GRAPH-COLORING-Instanz

Erzeuge für jede Klausel $K_j = L_1 \vee L_2 \vee L_3$ den folgenden Teilgraphen O_j (ein „Oder-Gatter“):



Grundgedanke: O_j soll das logische Oder der drei Literale in K_j „berechnen“.

Verbinde die Eingänge mit L_1 , L_2 und L_3 aus der Palette.

Die Farben der L_i sind daher F oder T .

C_j kann mit T gefärbt werden g.d.w. ein L_i mit T gefärbt ist.

Konstruktion der GRAPH-COLORING-Instanz

Erzeuge die Oder-Gatter $O_1, \dots, O_m$ für jeweils $K_1, \dots, K_m$.

Konstruktion der GRAPH-COLORING-Instanz

Erzeuge die Oder-Gatter $O_1, \dots, O_m$ für jeweils $K_1, \dots, K_m$.

Für jedes $j \in \{1, \dots, m\}$, sei $K_j = L_1 \vee L_2 \vee L_3$.

Verbinde jedes $l_{i,j}$ von O_j mit L_i aus der „Palette“.

Konstruktion der GRAPH-COLORING-Instanz

Erzeuge die Oder-Gatter $O_1, \dots, O_m$ für jeweils $K_1, \dots, K_m$.

Für jedes $j \in \{1, \dots, m\}$, sei $K_j = L_1 \vee L_2 \vee L_3$.

Verbinde jedes $l_{i,j}$ von O_j mit L_i aus der „Palette“.

Verbinde jeweils C_j mit N und C_j mit F .

Eine zulässige Färbung muss daher C_j auf T setzen.

Konstruktion der GRAPH-COLORING-Instanz

Erzeuge die Oder-Gatter $O_1, \dots, O_m$ für jeweils $K_1, \dots, K_m$.

Für jedes $j \in \{1, \dots, m\}$, sei $K_j = L_1 \vee L_2 \vee L_3$.

Verbinde jedes $l_{i,j}$ von O_j mit L_i aus der „Palette“.

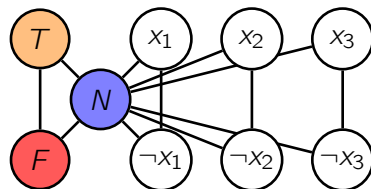
Verbinde jeweils C_j mit N und C_j mit F .

Eine zulässige Färbung muss daher C_j auf T setzen.

Damit haben wir den Graphen G konstruiert.

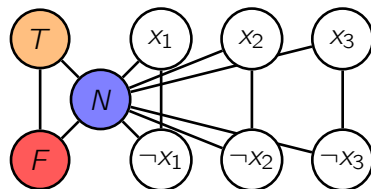
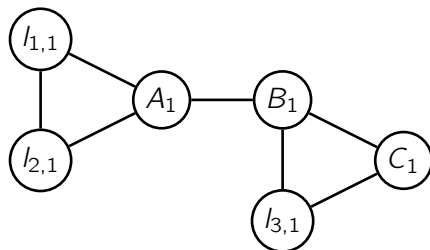
Konstruktion der GRAPH-COLORING-Instanz

Beispiel: Sei $H = K_1$, wobei $K_1 = x_1 \vee \neg x_3 \vee \neg x_2$.



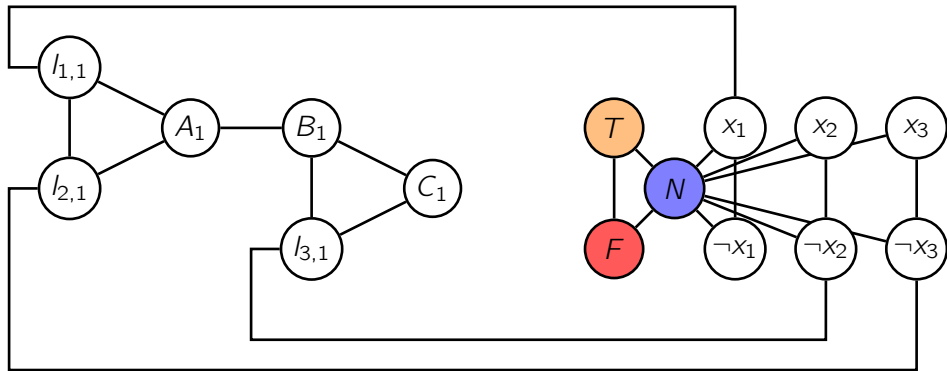
Konstruktion der GRAPH-COLORING-Instanz

Beispiel: Sei $H = K_1$, wobei $K_1 = x_1 \vee \neg x_3 \vee \neg x_2$.



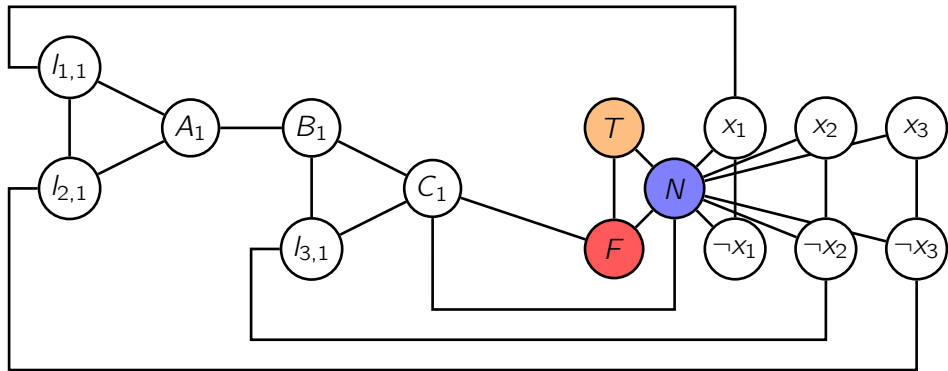
Konstruktion der GRAPH-COLORING-Instanz

Beispiel: Sei $H = K_1$, wobei $K_1 = x_1 \vee \neg x_3 \vee \neg x_2$.



Konstruktion der GRAPH-COLORING-Instanz

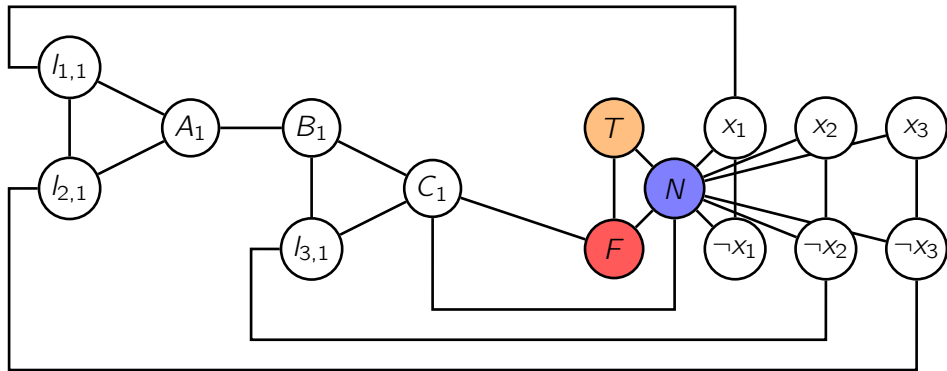
Beispiel: Sei $H = K_1$, wobei $K_1 = x_1 \vee \neg x_3 \vee \neg x_2$.



Konstruktion der GRAPH-COLORING-Instanz

Beispiel: Sei $H = K_1$, wobei $K_1 = x_1 \vee \neg x_3 \vee \neg x_2$.

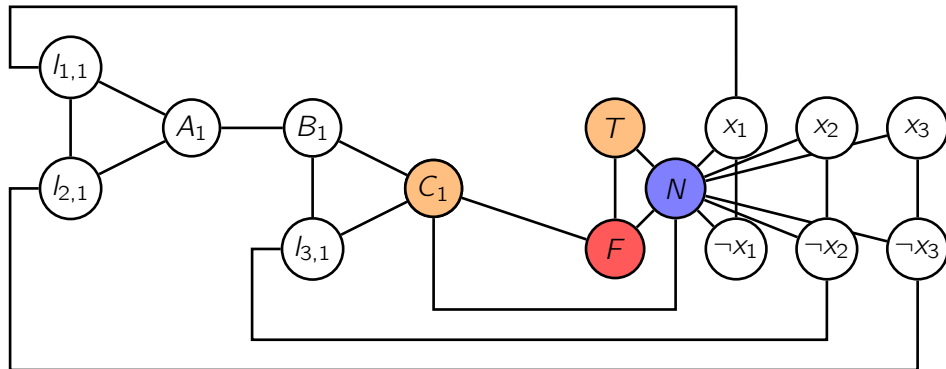
H ist erfüllbar g.d.w. folgender Graph G 3-färbbar ist.



Konstruktion der GRAPH-COLORING-Instanz

Beispiel: Sei $H = K_1$, wobei $K_1 = x_1 \vee \neg x_3 \vee \neg x_2$.

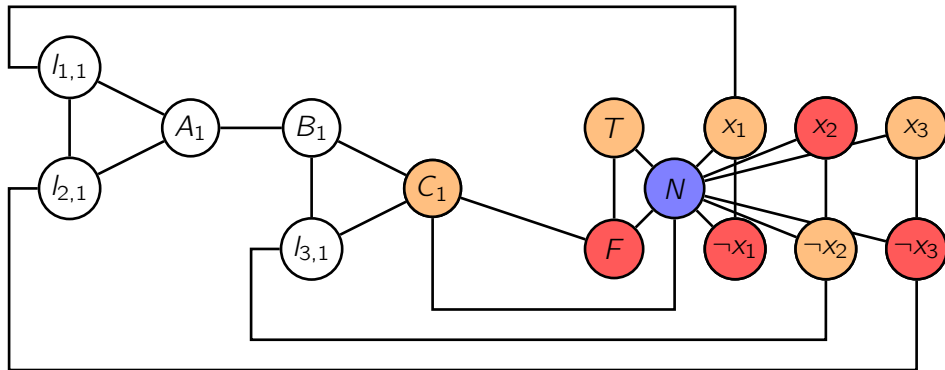
H ist erfüllbar g.d.w. folgender Graph G 3-färbbar ist.



Konstruktion der GRAPH-COLORING-Instanz

Beispiel: Sei $H = K_1$, wobei $K_1 = x_1 \vee \neg x_3 \vee \neg x_2$.

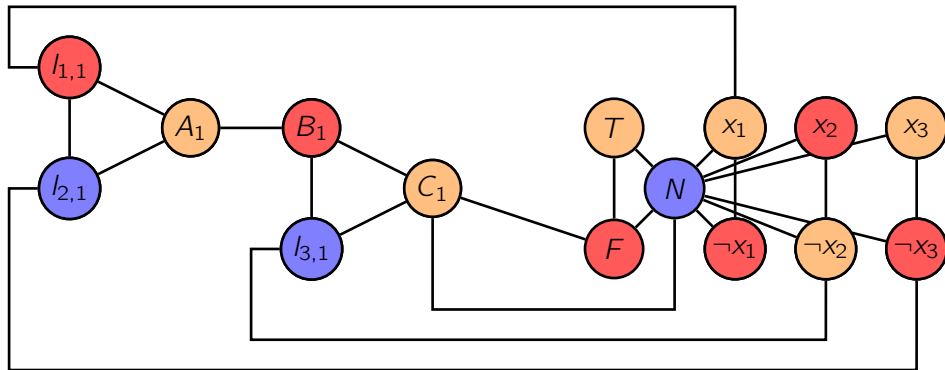
H ist erfüllbar g.d.w. folgender Graph G 3-färbbar ist.



Konstruktion der GRAPH-COLORING-Instanz

Beispiel: Sei $H = K_1$, wobei $K_1 = x_1 \vee \neg x_3 \vee \neg x_2$.

H ist erfüllbar g.d.w. folgender Graph G 3-färbbar ist.



$\mathcal{NP}$ -Vollständigkeit von GRAPH-COLORING

Die Funktion f ist total und kann in Polynomialzeit berechnet werden.

$\mathcal{NP}$ -Vollständigkeit von GRAPH-COLORING

Die Funktion f ist total und kann in Polynomialzeit berechnet werden.

Wir zeigen: H ist erfüllbar g.d.w. der Graph G 3-färbbar ist.

$\mathcal{NP}$ -Vollständigkeit von GRAPH-COLORING

Die Funktion f ist total und kann in Polynomialzeit berechnet werden.

Wir zeigen: H ist erfüllbar g.d.w. der Graph G 3-färbbar ist.

$\Leftarrow$ Sei G 3-färbbar.

$\mathcal{NP}$ -Vollständigkeit von GRAPH-COLORING

Die Funktion f ist total und kann in Polynomialzeit berechnet werden.

Wir zeigen: H ist erfüllbar g.d.w. der Graph G 3-färbbar ist.

$\Leftarrow$ Sei G 3-färbbar.

Dann sind alle C_j mit T gefärbt.

$\mathcal{NP}$ -Vollständigkeit von GRAPH-COLORING

Die Funktion f ist total und kann in Polynomialzeit berechnet werden.

Wir zeigen: H ist erfüllbar g.d.w. der Graph G 3-färbbar ist.

$\Leftarrow$ Sei G 3-färbbar.

Dann sind alle C_j mit T gefärbt.

x_i und $\neg x_i$ sind mit T und F oder umgekehrt gefärbt.

$\mathcal{NP}$ -Vollständigkeit von GRAPH-COLORING

Die Funktion f ist total und kann in Polynomialzeit berechnet werden.

Wir zeigen: H ist erfüllbar g.d.w. der Graph G 3-färbbar ist.

$\Leftarrow$ Sei G 3-färbbar.

Dann sind alle C_j mit T gefärbt.

x_i und $\neg x_i$ sind mit T und F oder umgekehrt gefärbt.

Wir definieren die Belegung I mit $I(x_i) = 1$ wenn x_i mit T und $I(x_i) = 0$ sonst.

$\mathcal{NP}$ -Vollständigkeit von GRAPH-COLORING

Die Funktion f ist total und kann in Polynomialzeit berechnet werden.

Wir zeigen: H ist erfüllbar g.d.w. der Graph G 3-färbbar ist.

$\Leftarrow$ Sei G 3-färbbar.

Dann sind alle C_j mit T gefärbt.

x_i und $\neg x_i$ sind mit T und F oder umgekehrt gefärbt.

Wir definieren die Belegung I mit $I(x_i) = 1$ wenn x_i mit T und $I(x_i) = 0$ sonst.

Die Belegung I macht die Formel H wahr, da in jeder Klausel ein Literal durch I wahr gemacht wird.

$\mathcal{NP}$ -Vollständigkeit von GRAPH-COLORING

Die Funktion f ist total und kann in Polynomialzeit berechnet werden.

Wir zeigen: H ist erfüllbar g.d.w. der Graph G 3-färbbar ist.

$\Leftarrow$ Sei G 3-färbbar.

Dann sind alle C_j mit T gefärbt.

x_i und $\neg x_i$ sind mit T und F oder umgekehrt gefärbt.

Wir definieren die Belegung I mit $I(x_i) = 1$ wenn x_i mit T und $I(x_i) = 0$ sonst.

Die Belegung I macht die Formel H wahr, da in jeder Klausel ein Literal durch I wahr gemacht wird.

$\Rightarrow$ Analog kann aus einer erfüllenden Belegung I eine 3-Färbung erzeugt werden. □

Definition

In einem gerichteten Graphen ist ein **Hamilton-Kreis** ein Kreis, der genau alle Knoten einmal besucht.

Formal: Für $G = (V, E)$ mit $V = \{v_1, \dots, v_n\}$ ist ein Hamilton-Kreis eine Permutation $\pi : \{1, \dots, n\} \rightarrow \{1, \dots, n\}$, sodass $(v_{\pi(i)}, v_{\pi(i+1)}) \in E$ und $(v_{\pi(n)}, v_{\pi(1)}) \in E$.

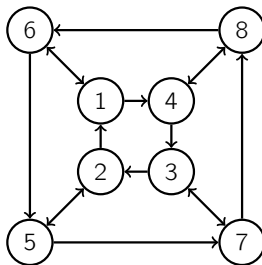
Gerichtete Hamilton-Kreise

Definition

In einem gerichteten Graphen ist ein **Hamilton-Kreis** ein Kreis, der genau alle Knoten einmal besucht.

Formal: Für $G = (V, E)$ mit $V = \{v_1, \dots, v_n\}$ ist ein Hamilton-Kreis eine Permutation $\pi : \{1, \dots, n\} \rightarrow \{1, \dots, n\}$, sodass $(v_{\pi(i)}, v_{\pi(i+1)}) \in E$ und $(v_{\pi(n)}, v_{\pi(1)}) \in E$.

Beispiel:



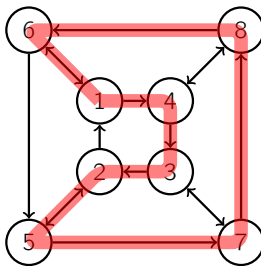
Gerichtete Hamilton-Kreise

Definition

In einem gerichteten Graphen ist ein **Hamilton-Kreis** ein Kreis, der genau alle Knoten einmal besucht.

Formal: Für $G = (V, E)$ mit $V = \{v_1, \dots, v_n\}$ ist ein Hamilton-Kreis eine Permutation $\pi : \{1, \dots, n\} \rightarrow \{1, \dots, n\}$, sodass $(v_{\pi(i)}, v_{\pi(i+1)}) \in E$ und $(v_{\pi(n)}, v_{\pi(1)}) \in E$.

Beispiel:



Das DIRECTED-HAMILTON-CYCLE-Problem

Definition

Das **DIRECTED-HAMILTON-CYCLE-Problem** lässt sich wie folgt formulieren.

gegeben: ein gerichteter Graph $G = (V, E)$ mit $V = \{v_1, \dots, v_n\}$

gefragt: Gibt es einen Hamilton-Kreis in G ?

Das DIRECTED-HAMILTON-CYCLE-Problem

Definition

Das **DIRECTED-HAMILTON-CYCLE-Problem** lässt sich wie folgt formulieren.

gegeben: ein gerichteter Graph $G = (V, E)$ mit $V = \{v_1, \dots, v_n\}$

gefragt: Gibt es einen Hamilton-Kreis in G ?

Satz

DIRECTED-HAMILTON-CYCLE ist $\mathcal{NP}$ -vollständig.

Das DIRECTED-HAMILTON-CYCLE-Problem

Definition

Das **DIRECTED-HAMILTON-CYCLE-Problem** lässt sich wie folgt formulieren.

gegeben: ein gerichteter Graph $G = (V, E)$ mit $V = \{v_1, \dots, v_n\}$

gefragt: Gibt es einen Hamilton-Kreis in G ?

Satz

DIRECTED-HAMILTON-CYCLE ist $\mathcal{NP}$ -vollständig.

Beweis Siehe Skript.



Ungerichtete Hamilton-Kreise

Definition

In einem ungerichteten Graphen ist ein **Hamilton-Kreis** ein Kreis, der genau alle Knoten einmal besucht.

Formal: Für $G = (V, E)$ mit $V = \{v_1, \dots, v_n\}$ ist ein Hamilton-Kreis eine Permutation $\pi : \{1, \dots, n\} \rightarrow \{1, \dots, n\}$, sodass $\{v_{\pi(i)}, v_{\pi(i+1)}\} \in E$ und $\{v_{\pi(n)}, v_{\pi(1)}\} \in E$.

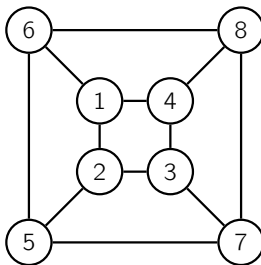
Ungerichtete Hamilton-Kreise

Definition

In einem ungerichteten Graphen ist ein **Hamilton-Kreis** ein Kreis, der genau alle Knoten einmal besucht.

Formal: Für $G = (V, E)$ mit $V = \{v_1, \dots, v_n\}$ ist ein Hamilton-Kreis eine Permutation $\pi : \{1, \dots, n\} \rightarrow \{1, \dots, n\}$, sodass $\{v_{\pi(i)}, v_{\pi(i+1)}\} \in E$ und $\{v_{\pi(n)}, v_{\pi(1)}\} \in E$.

Beispiel:



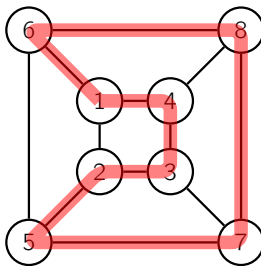
Ungerichtete Hamilton-Kreise

Definition

In einem ungerichteten Graphen ist ein **Hamilton-Kreis** ein Kreis, der genau alle Knoten einmal besucht.

Formal: Für $G = (V, E)$ mit $V = \{v_1, \dots, v_n\}$ ist ein Hamilton-Kreis eine Permutation $\pi : \{1, \dots, n\} \rightarrow \{1, \dots, n\}$, sodass $\{v_{\pi(i)}, v_{\pi(i+1)}\} \in E$ und $\{v_{\pi(n)}, v_{\pi(1)}\} \in E$.

Beispiel:



Das UNDIRECTED-HAMILTON-CYCLE-Problem

Definition

Das **UNDIRECTED-HAMILTON-CYCLE-Problem** lässt sich wie folgt formulieren.

gegeben: ein ungerichteter Graph $G = (V, E)$

gefragt: Gibt es einen Hamilton-Kreis in G ?

$\mathcal{NP}$ -Vollständigkeit von UNDIRECTED-HAMILTON-CYCLE

Satz

UNDIRECTED-HAMILTON-CYCLE ist $\mathcal{NP}$ -vollständig.

Beweis

1. UNDIRECTED-HAMILTON-CYCLE $\in \mathcal{NP}$:

Rate die Permutation π nichtdeterministisch.

Prüfe, ob $\{v_{\pi(i)}, v_{\pi(i+1)}\} \in E$ für jedes i und $\{v_{\pi(n)}, v_{\pi(1)}\} \in E$ gilt.

Dies kann auf einer NTM in Polynomialzeit durchgeführt werden.

Beweis (Fortsetzung)

2. UNDIRECTED-HAMILTON-CYCLE ist $\mathcal{NP}$ -schwer:

Wir zeigen $\text{DIRECTED-HAMILTON-CYCLE} \leq_p$
UNDIRECTED-HAMILTON-CYCLE.

$\mathcal{NP}$ -Vollständigkeit von UNDIRECTED-HAMILTON-CYCLE

Beweis (Fortsetzung)

2. UNDIRECTED-HAMILTON-CYCLE ist $\mathcal{NP}$ -schwer:

Wir zeigen $\text{DIRECTED-HAMILTON-CYCLE} \leq_p \text{UNDIRECTED-HAMILTON-CYCLE}$.

Sei f die Funktion, die aus einem gerichteten Graphen (V, E) einen ungerichteten Graphen macht, sodass

$$f(V) := \bigcup \{ \{v_{in}, v, v_{out}\} \mid v \in V \}$$

$$f(E) := \{ \{u_{out}, v_{in}\} \mid (u, v) \in E \} \cup \bigcup \{ \{ \{v_{in}, v\}, \{v, v_{out}\} \} \mid v \in V \}$$

$\mathcal{NP}$ -Vollständigkeit von UNDIRECTED-HAMILTON-CYCLE

Beweis (Fortsetzung)

2. UNDIRECTED-HAMILTON-CYCLE ist $\mathcal{NP}$ -schwer:

Wir zeigen $\text{DIRECTED-HAMILTON-CYCLE} \leq_p \text{UNDIRECTED-HAMILTON-CYCLE}$.

Sei f die Funktion, die aus einem gerichteten Graphen (V, E) einen ungerichteten Graphen macht, sodass

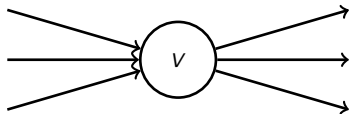
$$f(V) := \bigcup \{\{v_{in}, v, v_{out}\} \mid v \in V\}$$

$$f(E) := \{\{u_{out}, v_{in}\} \mid (u, v) \in E\} \cup \bigcup \{\{\{v_{in}, v\}, \{v, v_{out}\}\} \mid v \in V\}$$

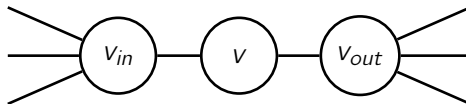
Die Funktion f ist total und kann in Polynomialzeit berechnet werden.

$\mathcal{NP}$ -Vollständigkeit von UNDIRECTED-HAMILTON-CYCLE

D.h. jeder Knoten v mit Ein- und Ausgängen



wird ersetzt durch



$\mathcal{NP}$ -Vollständigkeit von UNDIRECTED-HAMILTON-CYCLE

Wir zeigen: G hat einen gerichteten Hamilton-Kreis g.d.w.
 $f(G)$ einen ungerichteten Hamilton-Kreis hat.

$\mathcal{NP}$ -Vollständigkeit von UNDIRECTED-HAMILTON-CYCLE

Wir zeigen: G hat einen gerichteten Hamilton-Kreis g.d.w.

$f(G)$ einen ungerichteten Hamilton-Kreis hat.

$\implies$ Falls $v_{\pi(n)}, \dots, v_{\pi(1)}, v_{\pi(n)}$ ein gerichteter Hamilton-Kreis ist,
dann ist $v_{in,\pi(1)}, v_{\pi(1)}, v_{out,\pi(1)}, \dots, v_{in,\pi(n)}, v_{\pi(n)}, v_{out,\pi(n)}, v_{in,\pi(1)}$
ein ungerichteter Hamilton-Kreis.

$\mathcal{NP}$ -Vollständigkeit von UNDIRECTED-HAMILTON-CYCLE

Wir zeigen: G hat einen gerichteten Hamilton-Kreis g.d.w.

$f(G)$ einen ungerichteten Hamilton-Kreis hat.

- $\Rightarrow$ Falls $v_{\pi(n)}, \dots, v_{\pi(1)}, v_{\pi(n)}$ ein gerichteter Hamilton-Kreis ist, dann ist $v_{in,\pi(1)}, v_{\pi(1)}, v_{out,\pi(1)}, \dots, v_{in,\pi(n)}, v_{\pi(n)}, v_{out,\pi(n)}, v_{in,\pi(1)}$ ein ungerichteter Hamilton-Kreis.
- $\Leftarrow$ In $f(G)$ kann der Knoten v durch einen Hamilton-Kreis nur in einer Richtung durchlaufen werden: von v_{in} durch v durch v_{out} oder umgekehrt.

$\mathcal{NP}$ -Vollständigkeit von UNDIRECTED-HAMILTON-CYCLE

Wir zeigen: G hat einen gerichteten Hamilton-Kreis g.d.w.

$f(G)$ einen ungerichteten Hamilton-Kreis hat.

- ⇒ Falls $v_{\pi(n)}, \dots, v_{\pi(1)}, v_{\pi(n)}$ ein gerichteter Hamilton-Kreis ist,
dann ist $v_{in,\pi(1)}, v_{\pi(1)}, v_{out,\pi(1)}, \dots, v_{in,\pi(n)}, v_{\pi(n)}, v_{out,\pi(n)}, v_{in,\pi(1)}$
ein ungerichteter Hamilton-Kreis.
- ⇐ In $f(G)$ kann der Knoten v durch einen Hamilton-Kreis nur in einer Richtung
durchlaufen werden: von v_{in} durch v durch v_{out} oder umgekehrt.
 - ▶ Falls $v_{in,\pi(1)}, v_{\pi(1)}, v_{out,\pi(1)}, \dots, v_{in,\pi(n)}, v_{\pi(n)}, v_{out,\pi(n)}, v_{in,\pi(1)}$
ein ungerichteter Hamilton-Kreis ist,
dann ist $v_{\pi(1)}, \dots, v_{\pi(n)}, v_{\pi(1)}$ ein gerichteter Hamilton-Kreis.

$\mathcal{NP}$ -Vollständigkeit von UNDIRECTED-HAMILTON-CYCLE

Wir zeigen: G hat einen gerichteten Hamilton-Kreis g.d.w.

$f(G)$ einen ungerichteten Hamilton-Kreis hat.

- ⇒ Falls $v_{\pi(n)}, \dots, v_{\pi(1)}, v_{\pi(n)}$ ein gerichteter Hamilton-Kreis ist,
dann ist $v_{in,\pi(1)}, v_{\pi(1)}, v_{out,\pi(1)}, \dots, v_{in,\pi(n)}, v_{\pi(n)}, v_{out,\pi(n)}, v_{in,\pi(1)}$
ein ungerichteter Hamilton-Kreis.
- ⇐ In $f(G)$ kann der Knoten v durch einen Hamilton-Kreis nur in einer Richtung
durchlaufen werden: von v_{in} durch v durch v_{out} oder umgekehrt.
 - ▶ Falls $v_{in,\pi(1)}, v_{\pi(1)}, v_{out,\pi(1)}, \dots, v_{in,\pi(n)}, v_{\pi(n)}, v_{out,\pi(n)}, v_{in,\pi(1)}$
ein ungerichteter Hamilton-Kreis ist,
dann ist $v_{\pi(1)}, \dots, v_{\pi(n)}, v_{\pi(1)}$ ein gerichteter Hamilton-Kreis.
 - ▶ Falls $v_{out,\pi(1)}, v_{\pi(1)}, v_{in,\pi(1)}, \dots, v_{out,\pi(n)}, v_{\pi(n)}, v_{in,\pi(n)}, v_{out,\pi(1)}$
ein ungerichteter Hamilton-Kreis ist,
dann ist $v_{\pi(n)}, \dots, v_{\pi(1)}, v_{\pi(n)}$ ein gerichteter Hamilton-Kreis.

□

Das TRAVELING-SALESPERSON-Problem

Definition

Das **TRAVELING-SALESPERSON-Problem** lässt sich wie folgt formulieren.

gegeben: eine Menge von Knoten (Städten) $V = \{v_1, \dots, v_n\}$,
eine $(n \times n)$ -Matrix $(M_{i,j})$ mit Entfernungen $M_{i,j} \in \mathbb{N}$ zwischen den
Städten v_i und v_j , sowie
eine Zahl $k \in \mathbb{N}$

gefragt: Gibt es eine Rundreise, die alle Städte besucht, der Startort gleich dem
Zielort ist, und nicht länger als k ist?

Formal: Gibt es eine Permutation $\pi : \{1, \dots, n\} \rightarrow \{1, \dots, n\}$, sodass
 $(\sum_{i=1}^{n-1} M_{\pi(i), \pi(i+1)}) + M_{\pi(n), \pi(1)} \leq k$?

$\mathcal{NP}$ -Vollständigkeit von TRAVELING-SALESPERSON

Satz

Das TRAVELING-SALESPERSON-Problem ist $\mathcal{NP}$ -vollständig.

$\mathcal{NP}$ -Vollständigkeit von TRAVELING-SALESPERSON

Satz

Das TRAVELING-SALESPERSON-Problem ist $\mathcal{NP}$ -vollständig.

Beweis

► TRAVELING-SALESPERSON $\in \mathcal{NP}$:

Rate die Permutation π nichtdeterministisch.

Prüfe, ob $(\sum_{i=1}^{n-1} M_{\pi(i),\pi(i+1)}) + M_{\pi(n),\pi(1)} \leq k$ gilt.

Dies kann auf einer NTM in Polynomialzeit durchgeführt werden.

$\mathcal{NP}$ -Vollständigkeit von TRAVELING-SALESPERSON

Beweis (Fortsetzung)

- ▶ TRAVELING-SALESPERSON ist $\mathcal{NP}$ -schwer:

Wir zeigen $\text{UNDIRECTED-HAMILTON-CYCLE} \leq_p$
TRAVELING-SALESPERSON.

$\mathcal{NP}$ -Vollständigkeit von TRAVELING-SALESPERSON

Beweis (Fortsetzung)

► TRAVELING-SALESPERSON ist $\mathcal{NP}$ -schwer:

Wir zeigen $\text{UNDIRECTED-HAMILTON-CYCLE} \leq_p$
TRAVELING-SALESPERSON.

Sei $G = (V, E)$ mit $V = \{1, \dots, n\}$. Dann sei $f(G) = (V, (M_{i,j}, n))$ mit

$$M_{i,j} = \begin{cases} 1 & \text{falls } \{i,j\} \in E \\ 2 & \text{sonst} \end{cases}$$

$\mathcal{NP}$ -Vollständigkeit von TRAVELING-SALESPERSON

Beweis (Fortsetzung)

► TRAVELING-SALESPERSON ist $\mathcal{NP}$ -schwer:

Wir zeigen $\text{UNDIRECTED-HAMILTON-CYCLE} \leq_p$
TRAVELING-SALESPERSON.

Sei $G = (V, E)$ mit $V = \{1, \dots, n\}$. Dann sei $f(G) = (V, (M_{i,j}, n))$ mit

$$M_{i,j} = \begin{cases} 1 & \text{falls } \{i,j\} \in E \\ 2 & \text{sonst} \end{cases}$$

Die Funktion f ist total und kann in Polynomialzeit berechnet werden.

$\mathcal{NP}$ -Vollständigkeit von TRAVELING-SALESPERSON

Beweis (Fortsetzung)

► TRAVELING-SALESPERSON ist $\mathcal{NP}$ -schwer:

Wir zeigen $\text{UNDIRECTED-HAMILTON-CYCLE} \leq_p \text{TRAVELING-SALESPERSON}$.

Sei $G = (V, E)$ mit $V = \{1, \dots, n\}$. Dann sei $f(G) = (V, (M_{i,j}, n))$ mit

$$M_{i,j} = \begin{cases} 1 & \text{falls } \{i, j\} \in E \\ 2 & \text{sonst} \end{cases}$$

Die Funktion f ist total und kann in Polynomialzeit berechnet werden.

Wir zeigen: G hat einen Hamilton-Kreis g.d.w.

$f(G)$ eine Rundreise der Länge n hat.

$\mathcal{NP}$ -Vollständigkeit von TRAVELING-SALESPERSON

Beweis (Fortsetzung)

► TRAVELING-SALESPERSON ist $\mathcal{NP}$ -schwer:

Wir zeigen $\text{UNDIRECTED-HAMILTON-CYCLE} \leq_p \text{TRAVELING-SALESPERSON}$.

Sei $G = (V, E)$ mit $V = \{1, \dots, n\}$. Dann sei $f(G) = (V, (M_{i,j}, n))$ mit

$$M_{i,j} = \begin{cases} 1 & \text{falls } \{i, j\} \in E \\ 2 & \text{sonst} \end{cases}$$

Die Funktion f ist total und kann in Polynomialzeit berechnet werden.

Wir zeigen: G hat einen Hamilton-Kreis g.d.w.

$f(G)$ eine Rundreise der Länge n hat.

⇒ Wenn G einen Hamilton-Kreis hat, dann ist das eine Rundreise der Länge n .

$\mathcal{NP}$ -Vollständigkeit von TRAVELING-SALESPERSON

Beweis (Fortsetzung)

► TRAVELING-SALESPERSON ist $\mathcal{NP}$ -schwer:

Wir zeigen $\text{UNDIRECTED-HAMILTON-CYCLE} \leq_p \text{TRAVELING-SALESPERSON}$.

Sei $G = (V, E)$ mit $V = \{1, \dots, n\}$. Dann sei $f(G) = (V, (M_{i,j}, n))$ mit

$$M_{i,j} = \begin{cases} 1 & \text{falls } \{i,j\} \in E \\ 2 & \text{sonst} \end{cases}$$

Die Funktion f ist total und kann in Polynomialzeit berechnet werden.

Wir zeigen: G hat einen Hamilton-Kreis g.d.w.

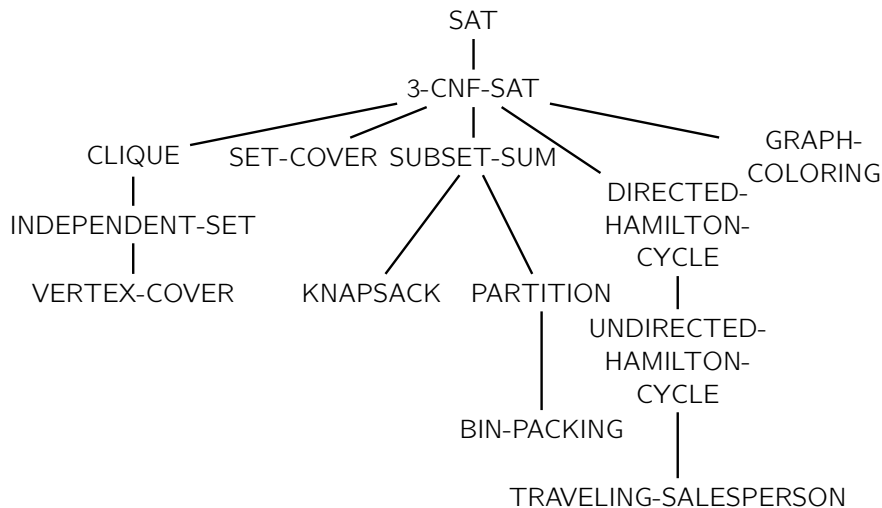
$f(G)$ eine Rundreise der Länge n hat.

⇒ Wenn G einen Hamilton-Kreis hat, dann ist das eine Rundreise der Länge n .

⇐ Wenn $f(G)$ eine Rundreise der Länge $\leq n$ hat, dann muss die Länge genau n sein, und es können nur Kanten mit Entfernung 1 verwendet werden.

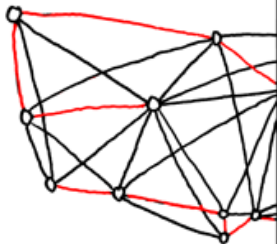
Daher hat G einen Hamilton-Kreis. □

Zusammenfassung der $\mathcal{NP}$ -Vollständigkeitsbeweise



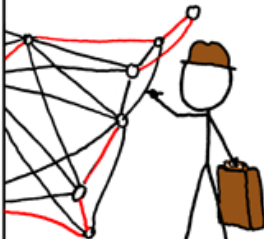
BRUTE-FORCE
SOLUTION:

$$O(n!)$$



DYNAMIC
PROGRAMMING
ALGORITHMS:

$$O(n^2 2^n)$$



SELLING ON EBAY:
 $O(1)$

STILL WORKING
ON YOUR ROUTE?

SHUT THE
HELL UP.



xkcd.com/399/

13c

Wiederholung und Fragestunde

Prof. Dr. Jasmin Blanchette

Lehr- und Forschungseinheit für
Theoretische Informatik und Theorembeweisen

Stand: 16. Juli 2024
Basierend auf Folien von PD Dr. David Sabel



Inhaltsübersicht Teil I: Formale Sprachen und Automatentheorie

- ▶ Chomsky-Grammatiken und die Chomsky-Hierarchie
- ▶ Reguläre Sprachen: DFAs, Minimierung von DFAs, NFAs (ohne und mit ϵ -Übergängen), reguläre Ausdrücke, Abschlusseigenschaften, Entscheidbarkeitsresultate, Pumping-Lemma, Satz von Myhill und Nerode
- ▶ Kontextfreie Sprachen: Chomsky-Normalform, Pumping-Lemma, Greibach-Normalform, Abschlusseigenschaften, CYK-Algorithmus, Kellerautomaten (PDAs und DPDAs), Entscheidbarkeitsresultate
- ▶ Kontextsensitive und Typ 0-Sprachen: Turingmaschinen (DTMs und NTMs), LBAs, Entscheiden des Wortproblems für Typ 1-Sprachen

Blau: Nur FSK

Inhaltsübersicht Teil II: Berechenbarkeitstheorie

- ▶ Berechenbarkeit
- ▶ Turingmaschinen und Turingberechenbarkeit
- ▶ LOOP-, WHILE-, GOTO-Programme und λ -Berechenbarkeit
- ▶ Primitiv rekursive und μ -rekursive Funktionen
- ▶ Unentscheidbarkeit: Halteproblem
- ▶ Reduktionen, PCP, Satz von Rice

Blau: Nur FSK

Inhaltsübersicht Teil III: Komplexitätstheorie

- ▶ $\mathcal{P}$ und $\mathcal{NP}$
- ▶ $\mathcal{NP}$ -Schwere und $\mathcal{NP}$ -Vollständigkeit
- ▶ Polynomialzeitreduktionen
- ▶ Satz von Cook
- ▶ $\mathcal{NP}$ -vollständige Probleme

Klassiker: „X angeben“

- ▶ Automaten angeben
- ▶ Grammatik angeben
- ▶ Regulären Ausdruck angeben

Klassiker: „Rechenaufgaben“

- ▶ NFA in DFA mit Potenzmengenkonstruktion transformieren
- ▶ DFA minimieren
- ▶ Chomsky-Normalform berechnen
- ▶ CYK-Algorithmus ausführen
- ▶ Wortproblem für Typ 1-Grammatiken entscheiden

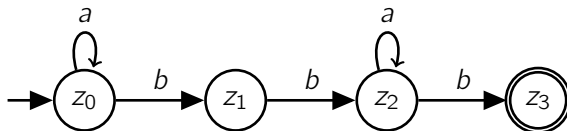
Blau: Nur FSK

Beispielaufgabe zu „Automaten angeben“

Geben Sie einen NFA über $\Sigma = \{a, b\}$ an, der $L = \{a^i b b a^j b \mid i, j \in \mathbb{N}\}$ erkennt.

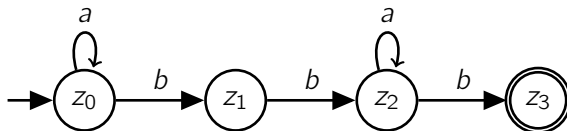
Beispielaufgabe zu „Automaten angeben“

Geben Sie einen NFA über $\Sigma = \{a, b\}$ an, der $L = \{a^i b b a^j b \mid i, j \in \mathbb{N}\}$ erkennt.



Beispielaufgabe zu „Automaten angeben“

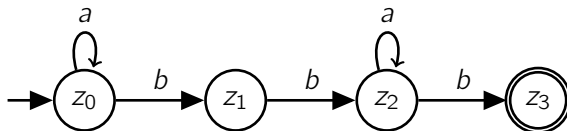
Geben Sie einen NFA über $\Sigma = \{a, b\}$ an, der $L = \{a^i b b a^j b \mid i, j \in \mathbb{N}\}$ erkennt.



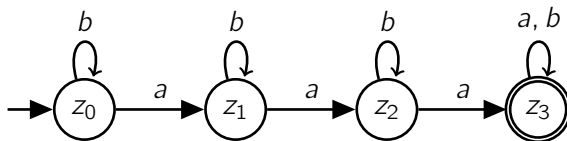
Geben Sie einen DFA über $\Sigma = \{a, b\}$ an, der $L = \{w \mid \#_a(w) > 2\}$ erkennt.

Beispielaufgabe zu „Automaten angeben“

Geben Sie einen NFA über $\Sigma = \{a, b\}$ an, der $L = \{a^i b b a^j b \mid i, j \in \mathbb{N}\}$ erkennt.

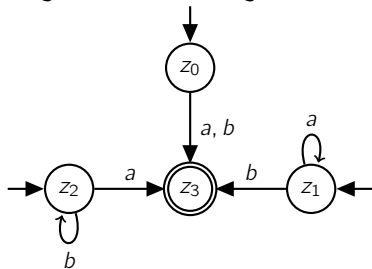


Geben Sie einen DFA über $\Sigma = \{a, b\}$ an, der $L = \{w \mid \#_a(w) > 2\}$ erkennt.



Beispielaufgabe zu endlichen Automaten

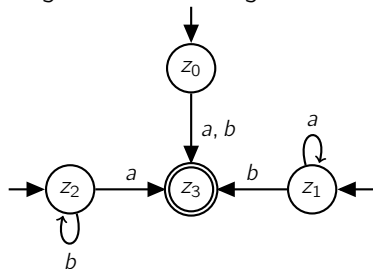
Gegeben sei der folgende NFA über $\Sigma = \{a, b\}$.



- a) Welche Sprache erkennt der gezeigte NFA?
- b) Geben Sie die Sprache durch einen regulären Ausdruck an.
- c) Erzeugen Sie einen äquivalenten DFA durch die Potenzmengenkonstruktion (erreichbare Zustände reichen aus).
- d) Minimieren Sie den DFA (Partitionstabelle erforderlich).

Beispielaufgabe zu endlichen Automaten

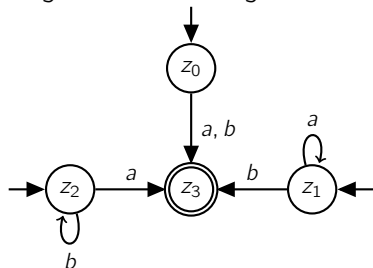
Gegeben sei der folgende NFA über $\Sigma = \{a, b\}$.



a) Welche Sprache erkennt der gezeigte NFA?

Beispielaufgabe zu endlichen Automaten

Gegeben sei der folgende NFA über $\Sigma = \{a, b\}$.

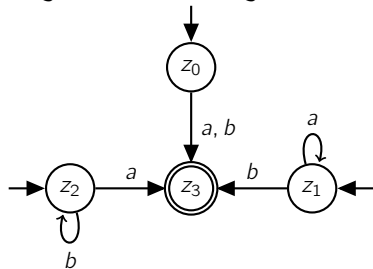


a) Welche Sprache erkennt der gezeigte NFA?

$$\begin{aligned} L &= \{a, b\} \cup \{b^i a \mid i \geq 0\} \cup \{a^i b \mid i \geq 0\} \\ &= \{b^i a \mid i \geq 0\} \cup \{a^i b \mid i \geq 0\} \end{aligned}$$

Beispielaufgabe zu endlichen Automaten

Gegeben sei der folgende NFA über $\Sigma = \{a, b\}$.



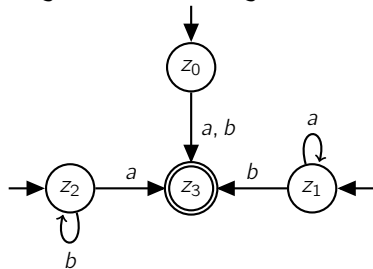
a) Welche Sprache erkennt der gezeigte NFA?

$$\begin{aligned} L &= \{a, b\} \cup \{b^i a \mid i \geq 0\} \cup \{a^i b \mid i \geq 0\} \\ &= \{b^i a \mid i \geq 0\} \cup \{a^i b \mid i \geq 0\} \end{aligned}$$

b) Geben Sie die Sprache durch einen regulären Ausdruck an.

Beispielaufgabe zu endlichen Automaten

Gegeben sei der folgende NFA über $\Sigma = \{a, b\}$.



- a) Welche Sprache erkennt der gezeigte NFA?

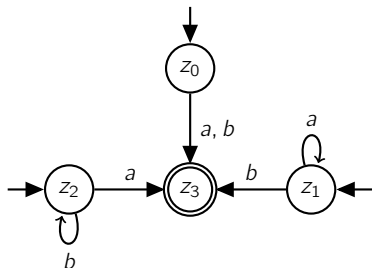
$$\begin{aligned} L &= \{a, b\} \cup \{b^i a \mid i \geq 0\} \cup \{a^i b \mid i \geq 0\} \\ &= \{b^i a \mid i \geq 0\} \cup \{a^i b \mid i \geq 0\} \end{aligned}$$

- b) Geben Sie die Sprache durch einen regulären Ausdruck an.

$$L = L(\alpha) \text{ mit } \alpha = (a^* b | b^* a)$$

Beispielaufgabe zu endlichen Automaten

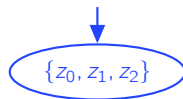
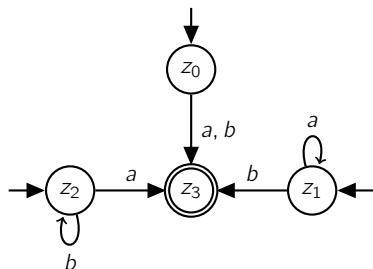
Gegeben sei der folgende NFA über $\Sigma = \{a, b\}$.



- c) Erzeugen Sie einen äquivalenten DFA durch die Potenzmengenkonstruktion (erreichbare Zustände reichen aus).

Beispielaufgabe zu endlichen Automaten

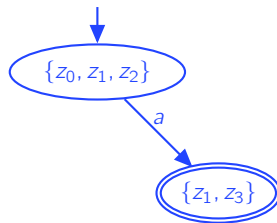
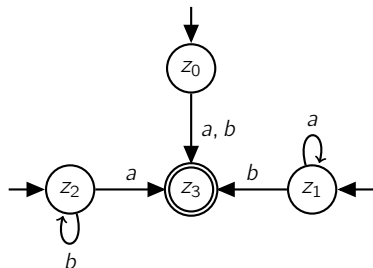
Gegeben sei der folgende NFA über $\Sigma = \{a, b\}$.



- c) Erzeugen Sie einen äquivalenten DFA durch die Potenzmengenkonstruktion (erreichbare Zustände reichen aus).

Beispielaufgabe zu endlichen Automaten

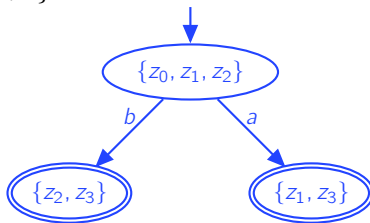
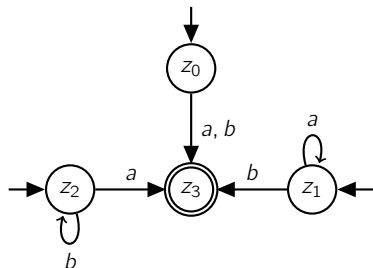
Gegeben sei der folgende NFA über $\Sigma = \{a, b\}$.



- c) Erzeugen Sie einen äquivalenten DFA durch die Potenzmengenkonstruktion (erreichbare Zustände reichen aus).

Beispielaufgabe zu endlichen Automaten

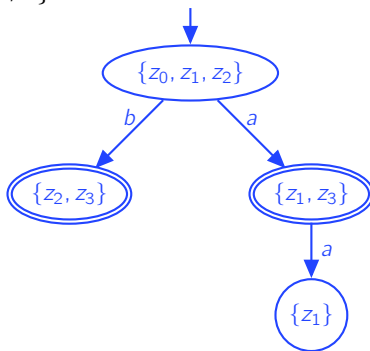
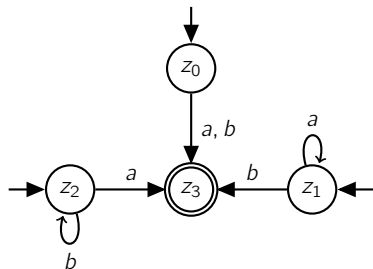
Gegeben sei der folgende NFA über $\Sigma = \{a, b\}$.



- c) Erzeugen Sie einen äquivalenten DFA durch die Potenzmengenkonstruktion (erreichbare Zustände reichen aus).

Beispielaufgabe zu endlichen Automaten

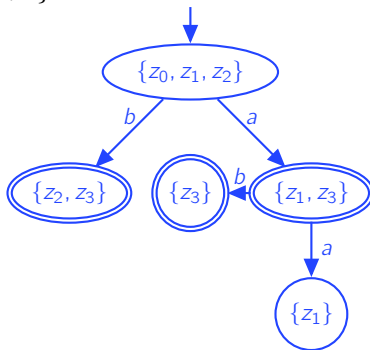
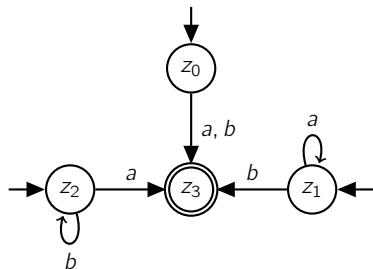
Gegeben sei der folgende NFA über $\Sigma = \{a, b\}$.



- c) Erzeugen Sie einen äquivalenten DFA durch die Potenzmengenkonstruktion (erreichbare Zustände reichen aus).

Beispielaufgabe zu endlichen Automaten

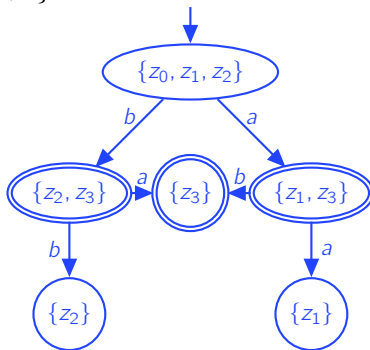
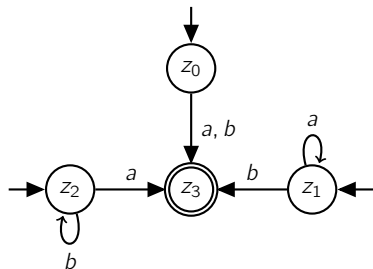
Gegeben sei der folgende NFA über $\Sigma = \{a, b\}$.



- c) Erzeugen Sie einen äquivalenten DFA durch die Potenzmengenkonstruktion (erreichbare Zustände reichen aus).

Beispielaufgabe zu endlichen Automaten

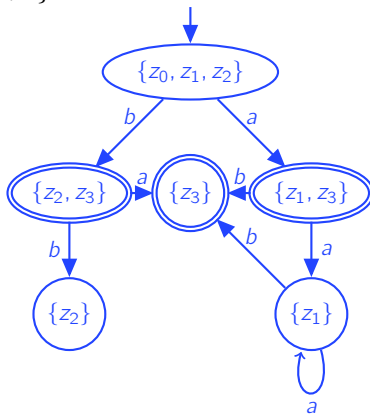
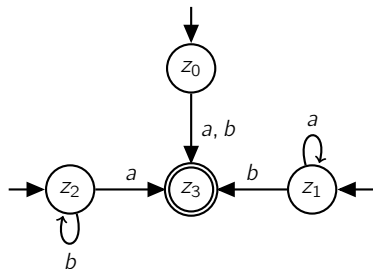
Gegeben sei der folgende NFA über $\Sigma = \{a, b\}$.



- c) Erzeugen Sie einen äquivalenten DFA durch die Potenzmengenkonstruktion (erreichbare Zustände reichen aus).

Beispielaufgabe zu endlichen Automaten

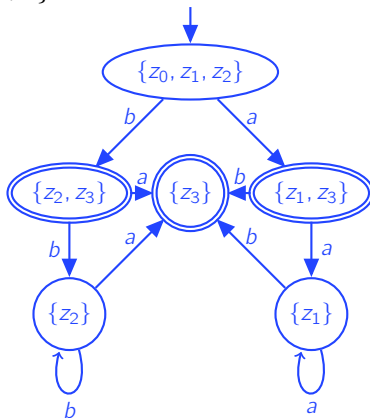
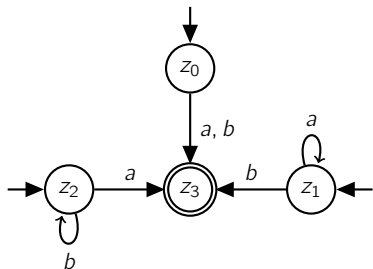
Gegeben sei der folgende NFA über $\Sigma = \{a, b\}$.



- c) Erzeugen Sie einen äquivalenten DFA durch die Potenzmengenkonstruktion (erreichbare Zustände reichen aus).

Beispielaufgabe zu endlichen Automaten

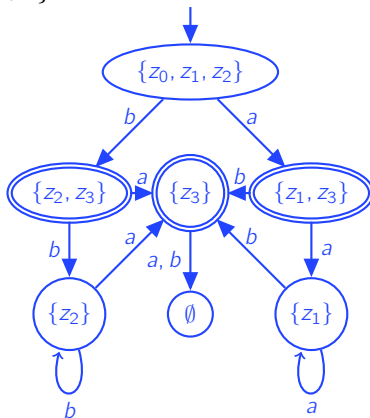
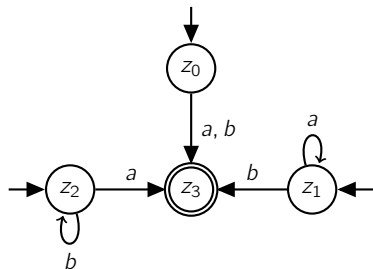
Gegeben sei der folgende NFA über $\Sigma = \{a, b\}$.



- c) Erzeugen Sie einen äquivalenten DFA durch die Potenzmengenkonstruktion (erreichbare Zustände reichen aus).

Beispielaufgabe zu endlichen Automaten

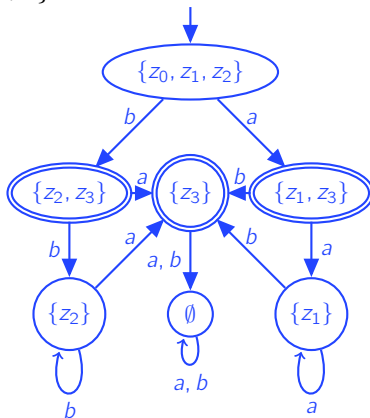
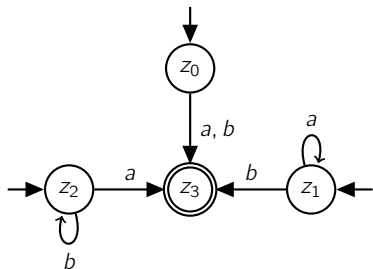
Gegeben sei der folgende NFA über $\Sigma = \{a, b\}$.



- c) Erzeugen Sie einen äquivalenten DFA durch die Potenzmengenkonstruktion (erreichbare Zustände reichen aus).

Beispielaufgabe zu endlichen Automaten

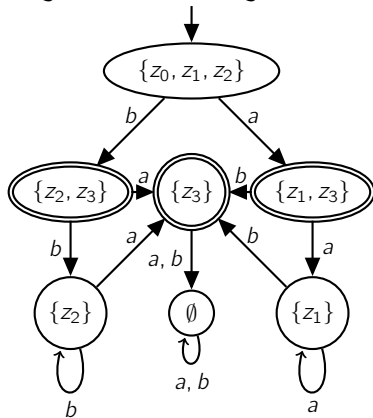
Gegeben sei der folgende NFA über $\Sigma = \{a, b\}$.



- c) Erzeugen Sie einen äquivalenten DFA durch die Potenzmengenkonstruktion (erreichbare Zustände reichen aus).

Beispielaufgabe zu endlichen Automaten

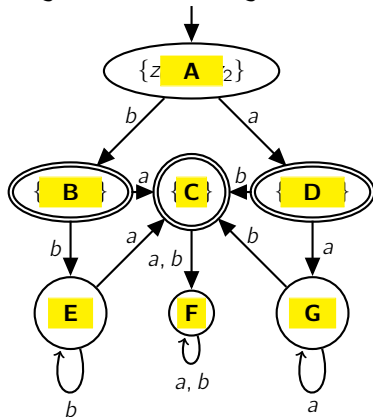
Gegeben sei der folgende DFA über $\Sigma = \{a, b\}$.



d) Minimieren Sie den DFA (Partitionstabelle erforderlich).

Beispielaufgabe zu endlichen Automaten

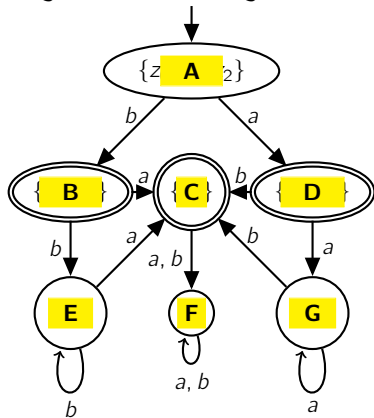
Gegeben sei der folgende DFA über $\Sigma = \{a, b\}$.



d) Minimieren Sie den DFA (Partitionstabelle erforderlich).

Beispielaufgabe zu endlichen Automaten

Gegeben sei der folgende DFA über $\Sigma = \{a, b\}$.



Partitionstabelle:

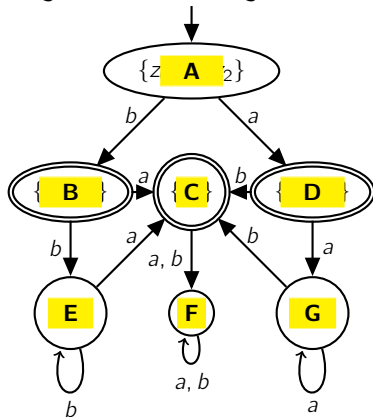
0.

A	E	F	G		B	C	D
---	---	---	---	--	---	---	---

d) Minimieren Sie den DFA (Partitionstabelle erforderlich).

Beispielaufgabe zu endlichen Automaten

Gegeben sei der folgende DFA über $\Sigma = \{a, b\}$.



Partitionstabelle:

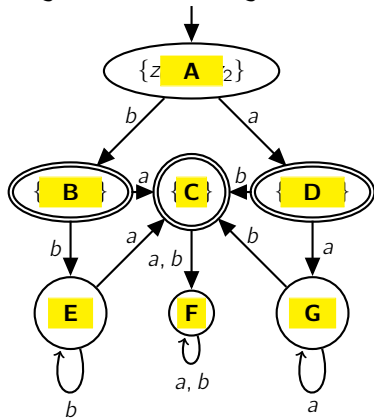
0.	A	E	F	G	B	C	D
1.	A	E	F	G	B	C	D

 mit a

d) Minimieren Sie den DFA (Partitionstabelle erforderlich).

Beispielaufgabe zu endlichen Automaten

Gegeben sei der folgende DFA über $\Sigma = \{a, b\}$.



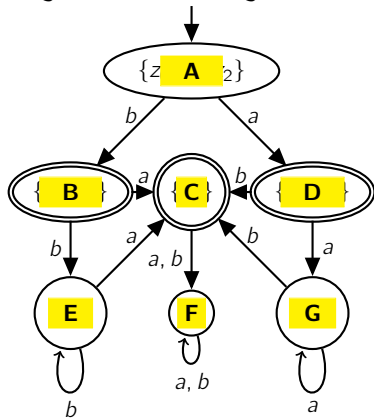
Partitionstabelle:

0.	A	E	F	G	B	C	D	
1.	A	E	F	G	B	C	D	mit a
2.	A	E	F	G	B	C	D	mit b

d) Minimieren Sie den DFA (Partitionstabelle erforderlich).

Beispielaufgabe zu endlichen Automaten

Gegeben sei der folgende DFA über $\Sigma = \{a, b\}$.



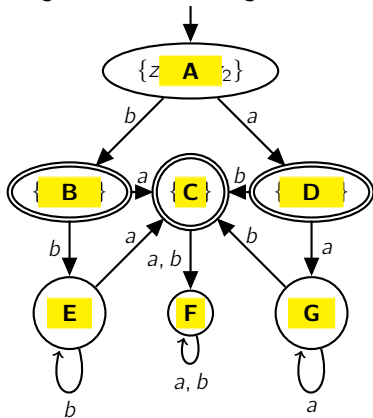
Partitionstabelle:

0.	A	E	F	G	B	C	D	
1.	A	E	F	G	B	C	D	mit a
2.	A	E	F	G	B	C	D	mit b
3.	A	E	F	G	B	C	D	mit a

d) Minimieren Sie den DFA (Partitionstabelle erforderlich).

Beispielaufgabe zu endlichen Automaten

Gegeben sei der folgende DFA über $\Sigma = \{a, b\}$.



Partitionstabelle:

0.	A	E	F	G	B	C	D	
1.	A	E	F	G	B	C	D	mit a
2.	A	E	F	G	B	C	D	mit b
3.	A	E	F	G	B	C	D	mit a

Der Automat war schon minimal.

d) Minimieren Sie den DFA (Partitionstabelle erforderlich).

Beispielaufgabe zum CYK-Algorithmus

Sei $G = (V, \Sigma, P, S)$ mit $V = \{S, A, B, C\}$, $\Sigma = \{a, b\}$ und $P = \{S \rightarrow CS \mid b, A \rightarrow a, B \rightarrow b, C \rightarrow AC \mid b\}$.

Führen Sie den CYK-Algorithmus für $aaaaab$ aus. Liegt das Wort in $L(G)$?

Beispielaufgabe zum CYK-Algorithmus

Sei $G = (V, \Sigma, P, S)$ mit $V = \{S, A, B, C\}$, $\Sigma = \{a, b\}$ und $P = \{S \rightarrow CS \mid b, A \rightarrow a, B \rightarrow b, C \rightarrow AC \mid b\}$.

Führen Sie den CYK-Algorithmus für *aaaaab* aus. Liegt das Wort in $L(G)$?

	a 1	a 2	a 3	a 4	a 5	b 6
1	A	A	A	A	A	B, C, S
2					C	
3				C		
4			C			
5		C				
6	C					

Beispielaufgabe zum CYK-Algorithmus

Sei $G = (V, \Sigma, P, S)$ mit $V = \{S, A, B, C\}$, $\Sigma = \{a, b\}$ und $P = \{S \rightarrow CS \mid b, A \rightarrow a, B \rightarrow b, C \rightarrow AC \mid b\}$.

Führen Sie den CYK-Algorithmus für *aaaaab* aus. Liegt das Wort in $L(G)$?

	a	a	a	a	a	b
	1	2	3	4	5	6
1	A	A	A	A	A	B, C, S
2					C	
3				C		
4			C			
5		C				
6	C					

Da unten links nicht das Startsymbol S in der Tabelle steht, liegt das Wort nicht in $L(G)$.

Klassiker: „Beweisaufgaben“

- ▶ Nichtregulärheit einer Sprache zeigen mit dem Pumping-Lemma für reguläre Sprachen
- ▶ Nichtregulärheit einer Sprache zeigen mit dem Satz von Myhill und Nerode
- ▶ Nicht-Kontextfreiheit einer Sprache zeigen mit dem Pumping-Lemma für kontextfreie Sprachen
- ▶ Unentscheidbarkeit zeigen mit einer Reduktion
- ▶ Unentscheidbarkeit zeigen mit dem Satz von Rice
- ▶ $\mathcal{NP}$ -Schwere zeigen mit einer Polynomialzeitreduktion

Blau: nur FSK

Beispielaufgabe zu „Nichtregulärheit beweisen“

Zeigen Sie, dass $L = \{a^j b^j \mid j \in \mathbb{N}\}$ nicht regulär ist.

Beispielaufgabe zu „Nichtregulärheit beweisen“

Zeigen Sie, dass $L = \{a^j b^j \mid j \in \mathbb{N}\}$ nicht regulär ist.

Mit dem Pumping-Lemma.

Sei $n > 0$ beliebig.

Wir wählen $z \in L$ als $z = a^n b^n$. Wir haben $|z| \geq n$.

Sei $z = uvw$ eine beliebige Zerlegung von z ,
sodass $|uv| \leq n$, $|v| \geq 1$ und $uv^i w \in L$ für jedes $i \in \mathbb{N}$. Dann $u = a^i$, $v = a^j$,
 $w = a^k b^n$ mit $i + j + k = n$ und $j \geq 1$.

Wir wählen $i = 0$. Dann gilt $uv^0 w = a^{n-j} b^n \notin L$. Widerspruch.



Beispielaufgabe zu „Nichtregulärheit beweisen“

Zeigen Sie, dass $L = \{a^j b^j \mid j \in \mathbb{N}\}$ nicht regulär ist durch Verwendung des Satzes von Myhill und Nerode.

Beispielaufgabe zu „Nichtregulärheit beweisen“

Zeigen Sie, dass $L = \{a^j b^j \mid j \in \mathbb{N}\}$ nicht regulär ist durch Verwendung des Satzes von Myhill und Nerode.

Die Nerode-Relation $\sim_L$ eine Sprache L ist so definiert: $u \sim_L v$ g.d.w.

$$\forall w \in \Sigma^*, uw \in L \text{ g.d.w. } vw \in L$$

Der Satz von Myhill und Nerode sagt: $\text{Index}(\sim_L)$ ist endlich g.d.w. L regulär ist.

Beispielaufgabe zu „Nichtregulärheit beweisen“

Zeigen Sie, dass $L = \{a^j b^j \mid j \in \mathbb{N}\}$ nicht regulär ist durch Verwendung des Satzes von Myhill und Nerode.

Die Nerode-Relation $\sim_L$ eine Sprache L ist so definiert: $u \sim_L v$ g.d.w.

$$\forall w \in \Sigma^*, uw \in L \text{ g.d.w. } vw \in L$$

Der Satz von Myhill und Nerode sagt: $\text{Index}(\sim_L)$ ist endlich g.d.w. L regulär ist.

Für die Aufgabe müssen wir unendlich viele verschiedene Äquivalenzklassen finden.

Beispielaufgabe zu „Nichtregulärheit beweisen“

Zeigen Sie, dass $L = \{a^j\$b^j \mid j \in \mathbb{N}\}$ nicht regulär ist durch Verwendung des Satzes von Myhill und Nerode.

Die Nerode-Relation $\sim_L$ eine Sprache L ist so definiert: $u \sim_L v$ g.d.w.

$$\forall w \in \Sigma^*, uw \in L \text{ g.d.w. } vw \in L$$

Der Satz von Myhill und Nerode sagt: $\text{Index}(\sim_L)$ ist endlich g.d.w. L regulär ist.

Für die Aufgabe müssen wir unendlich viele verschiedene Äquivalenzklassen finden.

Für $u_i = a^i\$$ und $w_i = b^i$ gilt $u_i\$w_i \in L$, aber $u_j\$w_i \notin L$.

Damit $u_i \not\sim_L u_j$ für $i \neq j$.

Beispielaufgabe zu „Nichtregulärheit beweisen“

Zeigen Sie, dass $L = \{a^j b^j \mid j \in \mathbb{N}\}$ nicht regulär ist durch Verwendung des Satzes von Myhill und Nerode.

Die Nerode-Relation $\sim_L$ eine Sprache L ist so definiert: $u \sim_L v$ g.d.w.

$$\forall w \in \Sigma^*, uw \in L \text{ g.d.w. } vw \in L$$

Der Satz von Myhill und Nerode sagt: $\text{Index}(\sim_L)$ ist endlich g.d.w. L regulär ist.

Für die Aufgabe müssen wir unendlich viele verschiedene Äquivalenzklassen finden.

Für $u_i = a^i$ und $w_i = b^i$ gilt $u_i w_i \in L$, aber $u_j w_i \notin L$.

Damit $u_i \not\sim_L u_j$ für $i \neq j$.

Es gibt also unendlich viele disjunkte Äquivalenzklassen: $[u_1]_{\sim_L}, [u_2]_{\sim_L}, \dots$

Daher: $\text{Index}(\sim_L) = \infty$.

Mit dem Satz von Myhill und Nerode folgt, dass L nicht regulär ist. □

Beispielaufgabe zu „Unentscheidbarkeit beweisen“

Sei $X = \{w \mid M_w \text{ hält genau bei Eingabe } 0\}$.

Zeigen Sie die Unentscheidbarkeit mithilfe einer Reduktion.

Beispielaufgabe zu „Unentscheidbarkeit beweisen“

Sei $X = \{w \mid M_w \text{ hält genau bei Eingabe } 0\}$.

Zeigen Sie die Unentscheidbarkeit mithilfe einer Reduktion.

Sei $H_0 = \{w \mid M_w \text{ hält bei leerer Eingabe}\}$. Aus der Vorlesung: H_0 ist unentscheidbar.

Beispielaufgabe zu „Unentscheidbarkeit beweisen“

Sei $X = \{w \mid M_w \text{ hält genau bei Eingabe } 0\}$.

Zeigen Sie die Unentscheidbarkeit mithilfe einer Reduktion.

Sei $H_0 = \{w \mid M_w \text{ hält bei leerer Eingabe}\}$. Aus der Vorlesung: H_0 ist unentscheidbar.

Wir zeigen $H_0 \leq X$:

Beispielaufgabe zu „Unentscheidbarkeit beweisen“

Sei $X = \{w \mid M_w \text{ hält genau bei Eingabe } 0\}$.

Zeigen Sie die Unentscheidbarkeit mithilfe einer Reduktion.

Sei $H_0 = \{w \mid M_w \text{ hält bei leerer Eingabe}\}$. Aus der Vorlesung: H_0 ist unentscheidbar.

Wir zeigen $H_0 \leq X$:

Die Reduktionsfunktion f nimmt eine Turingmaschinenbeschreibung und erstellt daraus eine neue Turingmaschinenbeschreibung.

Beispielaufgabe zu „Unentscheidbarkeit beweisen“

Sei $X = \{w \mid M_w \text{ hält genau bei Eingabe } 0\}$.

Zeigen Sie die Unentscheidbarkeit mithilfe einer Reduktion.

Sei $H_0 = \{w \mid M_w \text{ hält bei leerer Eingabe}\}$. Aus der Vorlesung: H_0 ist unentscheidbar.

Wir zeigen $H_0 \leq X$:

Die Reduktionsfunktion f nimmt eine Turingmaschinenbeschreibung und erstellt daraus eine neue Turingmaschinenbeschreibung.

Sei w ein Wort. Sei M_w die Turingmaschine zu w .

Beispielaufgabe zu „Unentscheidbarkeit beweisen“

Sei $X = \{w \mid M_w \text{ hält genau bei Eingabe } 0\}$.

Zeigen Sie die Unentscheidbarkeit mithilfe einer Reduktion.

Sei $H_0 = \{w \mid M_w \text{ hält bei leerer Eingabe}\}$. Aus der Vorlesung: H_0 ist unentscheidbar.

Wir zeigen $H_0 \leq X$:

Die Reduktionsfunktion f nimmt eine Turingmaschinenbeschreibung und erstellt daraus eine neue Turingmaschinenbeschreibung.

Sei w ein Wort. Sei M_w die Turingmaschine zu w .

f erstellt daraus eine Turingmaschine N , sodass

- ▶ N prüft, ob die Eingabe 0 ist. Falls nicht, geht N in eine Endlosschleife.
- ▶ N löscht das Eingabeband.
- ▶ N simuliert M_w bei leerer Eingabe.
- ▶ Wenn M_w akzeptiert, dann akzeptiert N , ansonsten läuft N endlos.

Beispielaufgabe zu „Unentscheidbarkeit beweisen“

Es gilt:

$$w \in H_0$$

Beispielaufgabe zu „Unentscheidbarkeit beweisen“

Es gilt:

$$w \in H_0$$

g.d.w. M_w hält bei leerer Eingabe

Beispielaufgabe zu „Unentscheidbarkeit beweisen“

Es gilt:

$$w \in H_0$$

g.d.w. M_w hält bei leerer Eingabe

$$f(w) \in X$$

Beispielaufgabe zu „Unentscheidbarkeit beweisen“

Es gilt:

$$w \in H_0$$

g.d.w. M_w hält bei leerer Eingabe

$$N = M_{f(w)} \text{ hält genau bei Eingabe } 0$$

g.d.w. $f(w) \in X$

Beispielaufgabe zu „Unentscheidbarkeit beweisen“

Es gilt:

$$w \in H_0$$

g.d.w. M_w hält bei leerer Eingabe

g.d.w. $N = M_{f(w)}$ hält genau bei Eingabe 0

g.d.w. $f(w) \in X$

Beispielaufgabe zu „Unentscheidbarkeit beweisen“

Es gilt:

$$w \in H_0$$

g.d.w. M_w hält bei leerer Eingabe

g.d.w. $N = M_{f(w)}$ hält genau bei Eingabe 0

g.d.w. $f(w) \in X$

Da f total und berechenbar ist, gilt $H_0 \leq X$.



Weitere typische Aufgaben

- ▶ Formalismen ineinander überführen (z.B. regulärer Ausdruck in DFA)
- ▶ Programme schreiben als Turingmaschine, WHILE-, LOOP-, GOTO-Programm, primitiv rekursive Funktion, μ -rekursive Funktion
- ▶ Sprachen in der Chomsky-Hierarchie einordnen

Blau: Nur FSK

SORRY GUYS NO COMIC TODAY, I'VE GOTTA GO
TO THE DOCTOR TO GET MY THIGHS ROTATED.
BUT HERE'S SOME NEW CHARACTER ART I'M WORKING ON!

